

```

1  ; *****
2  ; IBMDOS7.S (PCDOS 7.1 Kernel) - RETRO DOS v5.0 by ERDOGAN TAN - 01/01/2024
3  ; -----
4  ; Last Update: 29/01/2026 - Retro DOS v5.0 (Modified PCDOS 7.1)
5  ; -----
6  ; Beginning: 22/04/2019 (Retro DOS 4.0), 03/11/2022 (Retro DOS 4.2)
7  ; -----
8  ; Assembler: NASM version 2.15
9  ; -----
10 ; ((nasm ibmdos7.s -l ibmdos7.txt -o IBMDOS.COM -Z error.txt))
11 ; -o IBMDOS7.BIN
12 ; *****
13 ; main file: 'retrodos5.s'
14 ; incbin 'IBMDOS7.BIN'
15 ; =====
16 ; Modified from 'msdos6.s' (modified MSDOS 6.21 kernel src as Retro DOS v4.2)
17 ; 29/09/2023 /// Retro DOS v4.2 (2023) -> Modified MSDOS 6.22 IO.SYS+MSDOS.SYS
18 ; =====
19 ;
20 ; 30/12/2022 - Retro DOS v4.2 Kernel ('msdos6.s')
21 ; Modified from 'msdos5.s' (29/12/2022, Retro DOS v4.1 Kernel) file
22 ; as below:
23 ; 1) MS-DOS version has been changed to 6.22 (It was 5.0)
24 ; 2) Retro DOS version has been changed to 4.2 (It was 4.1)
25 ; (The content has not been changed except kernel version because the kernel
26 ; code is already compatible with MSDOS 6.x and it is optimized before.)
27 ; (But IO.SYS part of the kernel is not same with Retro DOS v4.1 code.)
28 ;
29 ; -----
30 ;
31 ; 03/11/2022 - Erdogan Tan (Istanbul)
32 ;
33 ; Note: This code is a part of Retro DOS 4.0 kernel source code
34 ; (as included binary, 'MSDOS5.BIN')
35 ; Equivalent of MSDOS 5.0 MSDOS.SYS kernel file
36 ;
37 ; ((MSDOS 6.0 kernel source code has been modified by using disassembled
38 ; MSDOS 5.0 MSDOS.SYS)) -- Disassembler: HEX-RAYS IDA Pro --
39 ; ((Disassembly -Reverse engineering- reference: MSDOS 6.0 kernel src))
40 ;
41 ; ----- Retro DOS v2 (v3) boot sector loads RETRODOS.SYS (MSDOS.SYS)
42 ; at 1000h:0000h and loader (initialization) part of RETRODOS kernel
43 ; moves IO.SYS (DOSBIOSCODE & DOSBIOSDATA, 'IOSYS5.BIN') to 70h:0000h.
44 ; Then SYSINIT code to the next segment (4D6h for current version)..
45 ; SYSINIT code relocates itself and DOSBIOSCODE and MSDOS.SYS
46 ; (MSDOS5.BIN) according to request/setting in 'config.sys' file.
47 ;
48 ; -----
49 ;
50 ; 01/01/2024 - Retro DOS v5.0 Kernel ('ibmdos7.s')
51 ; Modified from 'msdos6.s' (29/09/2023, Retro DOS v4.2 kernel: MSDOS.SYS) file
52 ; as below:
53 ;
54 ; 1) Retro DOS v5.0 IBMDOS.COM is based on disassembled source code
55 ; of PCDOS 7.1 IBMDOS.COM (2003) and it is derived using Retro DOS v4.2
56 ; MSDOS.SYS source code. Retro DOS v5.0 (IBMDOS.COM) kernel source code
57 ; is modified (and optimized) and so, it is not same with the original
58 ; PCDOS 7.1 IBMDOS.COM.
59 ;
60 ; 2) Retro DOS v4.2 MSDOS.SYS is based on disassembled source code
61 ; of MSDOS 6.21 MSDOS.SYS, derived using MSDOS 6.0 source code.
62 ; (And then it has been verified and updated by comparing it with
63 ; the disassembled source code of MSDOS 6.22 kernel file MSDOS.SYS.)
64 ;
65 ; 3) Labels, names, comments, explanations and structure definitions
66 ; about procedures and code details are almost entirely taken from
67 ; the original MSDOS 6.0 source code, except for the details that
68 ; Erdogan Tan personally experienced. Some of them are incompatible
69 ; with PCDOS 7.1 code. But they have not been deleted to preserve
70 ; the originality of the descriptions.)
71 ;
72 ; ('msdos6.s' has been converted to 'ibmdos7.s' and 'retrodos42.s' has been
73 ; converted to 'retrodos5.s'. 'ibmdos7.s' is IBMDOS.COM source code file
74 ; while 'retrodos5.s' is source code of Retro DOS v5 kernel file 'PCDOS.SYS'.
75 ; 'retrodos5.s' includes 'ibmdos7.bin' or IBMDOS.COM as binary file.)
76 ;
77 ; -----
78 ;
79 ; =====
80 ; Most of comments in this file are from the original MSDOS 6.0 source code
81 ; -----
82 ;
83 ; MSDOS 6.0 kernel source files:
84 ; MSDATA.ASM,
85 ; (MSHEAD.ASM, MSCONST.ASM, CONST2.ASM, MS_DATA.ASM,
86 ; DOSTAB.ASM, LMSTUB.ASM, WPATCH.INC, MPATCH.ASM)
87 ; Mstable.ASM, MScode.ASM, MSDOSME.ASM (DOSMES.INC), TIME.ASM,
88 ; GETSET.ASM, PARSE.ASM, MISC.ASM, MISC2.ASM, CRIT.ASM, CPMIO.ASM,
89 ; CPMIO2.ASM, FCBIO.ASM, FCBIO2.ASM, SEARCH.ASM, PATH.ASM, IOCTL.ASM,
90 ; DELETE.ASM, RENAME.ASM, INFO.ASM, DUP.ASM, CREATE.ASM, OPEN.ASM,
91 ; DINFO.ASM, ISEARCH.ASM, BUF.ASM, ABORT.ASM, CLOSE.ASM, DIRCALL.ASM,
92 ; DISK.ASM, DISK2.ASM, DISK3.ASM, DIR.ASM, DIR2.ASM, DEV.ASM,
93 ; MKNODE.ASM, ROM.ASM, FCB.ASM, MCTRLC.ASM, FAT.ASM, MSPROC.ASM
94 ; ALLOC.ASM, SRVCALL.ASM, UTIL.ASM, MACRO.ASM, MACRO2.ASM, HANDLE.ASM
95 ; FILE.ASM, LOCK.ASM, ROMFIND.ASM, SHARE.ASM, MSINIT.ASM, ORIGIN.ASM
96 ;
97 ; MSDOS 2.0 kernel source files:
98 ; MSDOS.ASM (STDSW.ASM + MSHEAD.ASM + MSDATA.ASM)
99 ; MScode.ASM
100 ; DOSMES.ASM ... STDIO.ASM, TIME.ASM, XENIX.ASM, XENIX2.ASM
101 ;
102 ; =====
103 ; DOSLINK
104 ; =====
105 ; msdos mscode dosmes misc getset dircall alloc dev dir +
106 ; disk fat rom stdbuf stdcall stdctrlc stdfcb stdproc +
107 ; stdio time xenix xenix2
108 ;
109 ; =====
110 ; This MSDOS source code is verified & modified by using IDA Pro Disassembler
111 ; output in TASM syntax (July 2018 -> NASM syntax) [ IBMDOS.COM, 17/03/1987 ]
112 ; =====
113 ; #####
114 ; # This file is generated by The Interactive Disassembler (IDA) #
115 ; # Copyright (c) 2010 by Hex-Rays SA, <support@hex-rays.com> #
116 ; # Licensed to: Freeware version #
117 ; #####
118 ;
119 ;
120 ; Input MD5 : 75959BC417C19135B982F7959EE9C92A
121 ;
122 ; -----
123 ; File Name : C:\Documents and Settings\Erdogan Tan\Desktop\MSDOS621.BIN
124 ; Format : Binary file

```

```

125 ;=====
126 ; MSDOS621.BIN = MSDOS.SYS, 13/02/1994, 38138 bytes (MSDOS 6.21 kernel) 2019
127 ;-----
128 ; MSDOS5.BIN = MSDOS.SYS, 11/11/1991, 37394 bytes (MSDOS 5.0 kernel) 2022
129 ;-----
130 ;
131 ; MSDOS.ASM
132 ;=====
133 ;
134 ;TITLE    Standard MSDOS
135 ;NAME     MSDOS_2
136 ;
137 ; Number of disk I/O buffers
138 ;
139 ; INCLUDE STDSW.ASM
140 ; INCLUDE MSHEAD.ASM
141 ; INCLUDE MSDATA.ASM
142 ;
143 ; END
144 ;=====
145 ; STDSW.ASM
146 ;=====
147 ;
148 TRUE EQU 0FFFFH
149 FALSE EQU ~TRUE ; NOT TRUE
150 ;
151 ; Use the switches below to produce the standard Microsoft version or the IBM
152 ; version of the operating system
153 ;MSVER EQU false
154 ;IBM EQU true
155 ;WANG EQU FALSE
156 ;ALTVECT EQU FALSE
157 ;
158 ; Set this switch to cause DOS to move itself to the end of memory
159 ;HIGHMEM EQU FALSE
160 ;
161 ; IF IBM
162 ESCCH EQU 0 ;character to begin escape seq.
163 CANCEL EQU 27 ;Cancel with escape
164 TOGLINS EQU TRUE ;One key toggles insert mode
165 TOGLPRN EQU TRUE ;One key toggles printer echo
166 ZEROEXT EQU TRUE
167 ; ELSE
168 ; IF WANG
169 ;ESCCH EQU 1FH ;Are we assembling for WANG?
170 ; ;Yes. Use 1FH for escape character
171 ; ELSE
172 ;ESCCH EQU 1BH
173 ; ENDIF
174 ;CANCEL EQU "X"-@" ;Cancel with Ctrl-X
175 ;TOGLINS EQU WANG ;Separate keys for insert mode on
176 ; ;and off if not WANG
177 ;TOGLPRN EQU FALSE ;Separate keys for printer echo on
178 ; ;and off
179 ;ZEROEXT EQU TRUE
180 ; ENDIF
181 ;
182 ;=====
183 ; MSHEAD.ASM
184 ;=====
185 ;
186 ;-----
187 ; TITLE MSHEAD.ASM -- MS-DOS DEFINITIONS
188 ;-----
189 ;
190 ; MS-DOS High-performance operating system for the 8086 version 1.28
191 ; by Microsoft MSDOS development group:
192 ; Tim Paterson (Ret.)
193 ; Aaron Reynolds
194 ; Nancy Panners (Parenting)
195 ; Mark Zbikowski
196 ; Chris Peters (BIOS) (ret.)
197 ;
198 ; ***** Revision History *****
199 ; >> EVERY change must noted below!! <<
200 ;
201 ; 0.34 12/29/80 General release, updating all past customers
202 ; 0.42 02/25/81 32-byte directory entries added
203 ; 0.56 03/23/81 Variable record and sector sizes
204 ; 0.60 03/27/81 Ctrl-C exit changes, including register save on user stack
205 ; 0.74 04/15/81 Recognize I/O devices with file names
206 ; 0.75 04/17/81 Improve and correct buffer handling
207 ; 0.76 04/23/81 Correct directory size when not 2^N entries
208 ; 0.80 04/27/81 Add console input without echo, Functions 7 & 8
209 ; 1.00 04/28/81 Renumbr for general release
210 ; 1.01 05/12/81 Fix bug in 'STORE'
211 ; 1.10 07/21/81 Fatal error trapping, NUL device, hidden files, date & time,
212 ; RENAME fix, general cleanup
213 ; 1.11 09/03/81 Don't set CURRENT BLOCK to 0 on open; fix SET FILE SIZE
214 ; 1.12 10/09/81 Zero high half of CURRENT BLOCK after all (CP/M programs don't)
215 ; 1.13 10/29/81 Fix classic "no write-through" error in buffer handling
216 ; 1.20 12/31/81 Add time to FCB; separate FAT from DPT; Kill SMALLDIR; Add
217 ; FLUSH and MAPDEV calls; allow disk mapping in DSKCHG; Lots
218 ; of smaller improvements
219 ; 1.21 01/06/82 HIGHMEM switch to run DOS in high memory
220 ; 1.22 01/12/82 Add VERIFY system call to enable/disable verify after write
221 ; 1.23 02/11/82 Add defaulting to parser; use variable escape character Don't
222 ; zero extent field in IBM version (back to 1.01!)
223 ; 1.24 03/01/82 Restore fcn. 27 to 1.0 level; add fcn. 28
224 ; 1.25 03/03/82 Put marker (00) at end of directory to speed searches
225 ; 1.26 03/03/82 Directory buffers searched as a circular queue, current buffer
226 ; is searched first when possible to minimize I/O
227 ; 03/03/82 STORE routine optimized to tack on partial sector tail as
228 ; full sector write when file is growing
229 ; 03/09/82 Multiple I/O buffers
230 ; 03/29/82 Two bugs: Delete all case resets search to start at beginning
231 ; of directory (infinite loop possible otherwise), DSKRESET
232 ; must invalidate all buffers (disk and directory).
233 ; 1.27 03/31/82 Installable device drivers
234 ; Function call 47 - Get pointer to device table list
235 ; Function call 48 - Assign CON AUX LIST
236 ; 04/01/82 Spooler interrupt (INT 28) added.
237 ; 1.28 04/15/82 DOS restructured to use ASSUMES and PROC labels around system
238 ; call entries. Most CS relative references changed to SS
239 ; relative with an eye toward putting a portion of the DOS in
240 ; ROM. DOS source also broken into header, data and code pieces
241 ; 04/15/82 GETDMA and GETVECT calls added as 24 and 32. These calls
242 ; return the current values.
243 ; 04/15/82 INDOS flag implemented for interrupt processing along with
244 ; call to return flag location (call 29)
245 ; 04/15/82 Volume ID attribute added
246 ; 04/17/82 Changed ABORT return to user to a long ret from a long jump to
247 ; avoid a CS relative reference.
248 ; 04/17/82 Put call to STATCHK in dispatcher to catch ^C more often

```

```

249 ; 04/20/82 Added INT int_upooler into loop ^S wait
250 ; 04/22/82 Dynamic disk I/O buffer allocation and call to manage them
251 ; call 49.
252 ; 04/23/82 Added GETDSKPTDL as call 50, similar to GETFATPT(DL), returns
253 ; address of DPB
254 ; 04/29/82 Mod to WRTDEV to look for ^C or ^S at console input when
255 ; writting to console device via file I/O. Added a console
256 ; output attribute to devices.
257 ; 04/30/82 Call to en/dis able ^C check in dispatcher call 51
258 ; 04/30/82 Code to allow assignment of func 1-12 to disk files as well
259 ; as devices.... pipes, redirection now possible
260 ; 04/30/82 Expanded GETLIST call to 2.0 standard
261 ; 05/04/82 Change to INT int_fatal_abort callout int HARDERR. DOS SS
262 ; (data segment) stashed in ES, INT int_fatal_abort routines must
263 ; preserve ES. This mod so HARDERR can be ROMed.
264 ; 1.29 06/01/82 Installable block and character devices as per 2.0 spec
265 ; 06/04/82 Fixed Bug in CLOSE regarding call to CHKFWRT. It got left
266 ; out back about 1.27 or so (oops). ARR
267 ; 1.30 06/07/82 Directory sector buffering added to main DOS buffer queue
268 ; 1.40 06/15/82 Tree structured directories. XENIX Path Parser MKDIR CHDIR
269 ; RMDIR Xenix calls
270 ; 1.41 06/13/82 Made GETBUFFR call PLACEBUF
271 ; 1.50 06/17/82 FATS cached in buffer pool, get FAT pointer calls disappear
272 ; Frees up lots of memory.
273 ; 1.51 06/24/82 BREAKDOWN modified to do EXACT one sector read/write through
274 ; system buffers
275 ; 1.52 06/30/82 OPEN, CLOSE, READ, WRITE, DUP, DUP2, LSEEK implemented
276 ; 1.53 07/01/82 OPEN CLOSE mod for Xenix calls, saves and gets remote dir
277 ; 1.54 07/11/82 Function calls 1-12 make use of new 2.0 PDB. Init code
278 ; changed to set file handle environment.
279 ; 2.00 08/01/82 Number for IBM release
280 ; 01/19/83 No environ bug in EXEC
281 ; 01/19/83 MS-DOS OEM INT 21 extensions (SET_OEM_HANDLER)
282 ; 01/19/83 Performance bug fix in cooked write to NUL
283 ; 01/27/83 Growcnt fixed for 32-bits
284 ; 01/27/83 Find-first problem after create
285 ; 2.01 02/17/83 International DOS
286 ; 2.11 08/12/83 Dos split into several more modules for assembly on
287 ; an IBM PC
288 ; 08/07/2018 - Retro DOS v3.0 by Erdogan Tan
289 ; (MSHEAD.ASM, MSDOS 6.0, 1991) - mshead.asm 1.1 85/04/10 -
290 ; 2.10 03/09/83 Start of NETWORK support
291 ; New Buffer structure
292 ; New Sytem file table structure
293 ; FCB moved to internal representation
294 ; DOS re-organized
295 ; 2.11 04/21/83 Continuation of 2.10, preliminary Network
296 ; device interface.
297 ; 2.11 08/12/83 Dos split into several more modules for assembly on
298 ; an IBM PC
299 ; 2.50 09/12/83 More network stuff
300 ;
301 ; *****
302 ;
303 ; -----
304 ; EQUATES
305 ;
306 ; Interrupt Entry Points:
307 ;
308 ; INTBASE: ABORT
309 ; INTBASE+4: COMMAND
310 ; INTBASE+8: BASE EXIT ADDRESS
311 ; INTBASE+C: CONTROL-C ABORT
312 ; INTBASE+10H: FATAL ERROR ABORT
313 ; INTBASE+14H: BIOS DISK READ
314 ; INTBASE+18H: BIOS DISK WRITE
315 ; INTBASE+1CH: END BUT STAY RESIDENT (NOT SET BY DOS)
316 ; INTBASE+20H: SPOOLER INTERRUPT
317 ; INTBASE+40H: Long jump to CALL entry point
318 ;
319 ENTRYPOINTSEG EQU 0Ch
320 MAXDIF EQU 0FFFh
321 SAVEXIT EQU 10
322 ; 06/05/2019
323 WRAPOFFSET EQU 0FEF0h ; (MISC.ASM, MSDOS 6.0, 1991)
324 ;
325 ; INCLUDE DOSSYM.ASM
326 ; INCLUDE DEVSYM.ASM
327 ;
328 ; SUBTTL ^C, terminate/abort/exit and Hard error actions
329 ; PAGE
330 ; There are three kinds of context resets that can occur during normal DOS
331 ; functioning: ^C trap, terminate/abort/exit, and Hard-disk error. These must
332 ; be handles in a clean fashion that allows nested executions along with the
333 ; ability to trap one's own errors.
334 ;
335 ; ^C trap - A process may elect to catch his own ^Cs. This is achieved by
336 ; using the $GET_INTERRUPT_VECTOR and $SET_INTERRUPT_VECTOR as
337 ; follows:
338 ;
339 ; $GET_INTERRUPT_VECTOR for INT int_ctrl_c
340 ; Save it in static memory.
341 ; $SET_INTERRUPT_VECTOR for INT int_ctrl_c
342 ;
343 ; The interrupt service routine must preserve all registers and
344 ; return carry set iff the operation is to be aborted (via abort
345 ; system call), otherwise, carry is reset and the operation is
346 ; restarted. ANY DEVIATION FROM THIS WILL LEAD TO UNRELIABLE
347 ; RESULTS.
348 ;
349 ; To restore original ^C processing (done on terminate/abort/exit),
350 ; restore INT int_ctrl_c from the saved vector.
351 ;
352 ; Hard-disk error -- The interrupt service routine for INT int_fatal_abort must
353 ; also preserve registers and return one of three values in AL: 0 and
354 ; 1 imply retry and ignore (???) and 2 indicates an abort. The user
355 ; himself is not to issue the abort, rather, the dos will do it for
356 ; him by simulating a normal abort/exit system call. ANY DEVIATION
357 ; FROM THIS WILL LEAD TO UNRELIABLE RESULTS.
358 ;
359 ; terminate/abort/exit -- The user may not, under any circumstances trap an
360 ; abort call. This is reserved for knowledgeable system programs.
361 ; ANY DEVIATION FROM THIS WILL LEAD TO UNRELIABLE RESULTS.
362 ;
363 ;SUBTTL SEGMENT DECLARATIONS
364 ;
365 ; The following are all of the segments used. They are declared in the order
366 ; that they should be placed in the executable
367 ;
368 ;
369 ; segment ordering for MSDOS
370 ;
371 ;
372 ;START SEGMENT BYTE PUBLIC 'START'

```

```

374 ;START ENDS
375 ;CONSTANTS SEGMENT BYTE PUBLIC 'CONST'
376 ;CONSTANTS ENDS
377
378 ;DATA SEGMENT WORD PUBLIC 'DATA'
379 ;DATA ENDS
380
381 ;CODE SEGMENT BYTE PUBLIC 'CODE'
382 ;CODE ENDS
383
384 ;LAST SEGMENT BYTE PUBLIC 'LAST'
385 ;LAST ENDS
386
387 ;DOSGROUP GROUP CODE,CONSTANTS,DATA,LAST
388
389 ; The following segment is defined such that the data/const classes appear
390 ; before the code class for ROMification
391
392 ;START SEGMENT BYTE PUBLIC 'START'
393 ; ASSUME CS:DOSGROUP,DS:NOHING,ES:NOHING,SS:NOHING
394 ; JMP DOSINIT
395 ;START ENDS
396
397 ;=====
398 ; BPB.INC, MSDOS 6.0, 1991
399 ;=====
400 ; 09/07/2018 - Retro DOS v3.0
401
402 ;-----+-----+-----+-----+-----+-----+-----+-----+-----+-----;
403 ; C A V E A T P R O G R A M M E R ;
404 ; ;
405
406 ** BIOS PARAMETER BLOCK DEFINITION
407 ;
408 ; The BPB contains information about the disk structure. It dates
409 ; back to the earliest FAT systems and so FAT information is
410 ; intermingled with physical driver information.
411 ;
412 ; A boot sector contains a BPB for its device; for other disks
413 ; the driver creates a BPB. DOS keeps copies of some of this
414 ; information in the DPB.
415 ;
416 ; The BDS structure contains a BPB within it.
417
418 ; 01/01/2024
419 %if 0
420
421 struc A_BPB
422 .BPB_BYTESPERSECTOR: resw 1
423 .BPB_SECTORS PERCLUSTER: resb 1
424 .BPB_RESERVEDSECTORS: resw 1
425 .BPB_NUMBEROFFATS: resb 1
426 .BPB_ROOTENTRIES: resw 1
427 .BPB_TOTALSECTORS: resw 1
428 .BPB_MEDIADESCRIPTOR: resb 1
429 .BPB_SECTORS PERFAT: resw 1
430 .BPB_SECTORS PERTRACK: resw 1
431 .BPB_HEADS: resw 1
432 .BPB_HIDDENSECTORS: resw 1
433 resw 1
434 .BPB_BIGTOTALSECTORS: resw 1
435 resw 1
436 resb 6 ; NOTE: many times these
437 ; ; 6 bytes are omitted
438 ; ; when BPB manipulations
439 ; ; are performed!
440 .size:
441 endstruc
442
443 %else
444
445 ; 14/04/2024
446 ; 01/01/2024 - Retro DOS v5.0
447
448 struc A_BPB
449 00000000 ???? .BYTESPERSECTOR: resw 1
450 00000002 ?? .SECTORS PERCLUSTER: resb 1
451 00000003 ???? .RESERVEDSECTORS: resw 1
452 00000005 ?? .NUMBEROFFATS: resb 1
453 00000006 ???? .ROOTENTRIES: resw 1
454 00000008 ???? .TOTALSECTORS: resw 1
455 0000000A ?? .MEDIADESCRIPTOR: resb 1
456 0000000B ???? .SECTORS PERFAT: resw 1
457 0000000D ???? .SECTORS PERTRACK: resw 1
458 0000000F ???? .HEADS: resw 1
459 00000011 ?????????? .HIDDENSECTORS: resd 1
460 00000015 ?????????? .BIGTOTALSECTORS: resd 1
461 ;..... FAT32 ..... + 28
462 00000019 ?????????? .FATSIZE32: resd 1
463 0000001D ???? .EXTFLAGS: resw 1
464 0000001F ???? .FSVER: resw 1
465 00000021 ?????????? .ROOTDIRCLUSTER: resd 1
466 00000025 ???? .FSINFOSECTOR: resw 1 ; (offset from FAT32 bs)
467 00000027 ???? .BACKUPBOOTSECTOR: resw 1 ; (offset from FAT32 bs)
468 00000029 <res ch> .RESERVEDBYTES: resb 12 ; (zero bytes)
469 ; 14/04/2024
470 00000035 ?????????????? resb 6 ; A_BPB.size must be 59
471 .size:
472 endstruc
473
474 %endif
475
476 ;
477 ; C A V E A T P R O G R A M M E R ;
478 ;-----+-----+-----+-----+-----+-----+-----+-----+-----+-----;
479
480 ;=====
481 ; BUFFER.INC, MSDOS 6.0, 1991
482 ;=====
483 ; 04/05/2019 - Retro DOS v4.0
484 ; 03/01/2024 - Retro DOS v5.0
485
486 ; <Disk I/O Buffer Header>
487
488 ;-----+-----+-----+-----+-----+-----+-----+-----+-----+-----;
489 ; C A V E A T P R O G R A M M E R ;
490 ; ;
491
492 ; Field definition for I/O buffer information
493
494 ; 03/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
495
496 struc BUFFINFO

```

```

497 00000000 ????.buf_next: resw 1 ; Pointer to next buffer in list
498 00000002 ????.buf_prev: resw 1 ; Pointer to prev buffer in list
499 00000004 ?? .buf_ID: resb 1 ; Drive of buffer (bit 7 = 0)
500 ; SFT table index (bit 7 = 1)
501 ; = FFH if buffer free
502 00000005 ?? .buf_flags: resb 1 ; Bit 7 = 1 if Remote file buffer
503 ; = 0 if Local device buffer
504 ; Bit 6 = 1 if buffer dirty
505 ; Bit 5 = Reserved
506 ; Bit 4 = Search bit (bit 7 = 1)
507 ; Bit 3 = 1 if buffer is DATA
508 ; Bit 2 = 1 if buffer is DIR
509 ; Bit 1 = 1 if buffer is FAT
510 ; Bit 0 = Reserved
511 00000006 ???????? .buf_sector: resd 1 ; Sector number of buffer (flags bit 7 = 0)
512 ; The next two items are often refed as a word (flags bit 7 = 0)
513 0000000A ?? .buf_wrtcnt: resb 1 ; For FAT sectors, # times sector written out
514 0000000B ???? .buf_wrtctinc: resw 1 ; " , # sectors between each write
515 0000000D ???? resw 1 ; * ; 03/01/2024 ; PC DOS 7.1
516 ; hw of sectors per FAT
517 0000000F ???????? .buf_DPB: resd 1 ; Pointer to drive parameters
518 00000013 ???? .buf_fill: resw 1 ; How full buffer is (flags bit 7 = 1)
519 00000015 ?? .buf_reserved: resb 1 ; make DWORD boundary for 386
520 00000016 ???? resw 1 ; * ; 03/01/2024 ; PC DOS 7.1
521 ; reserved word for dword boundary
522 .size: ; 20 bytes ; MSDOS 5.0 to 6.22
523 ; 24 bytes ; PC DOS 7.1 ; 03/01/2024
524 endstruc
525
526 %define buf_offset BUFFINFO.buf_sector ; 22/07/2019
527 ;For buf_flags bit 7 = 1, this is the byte
528 ;offset of the start of the buffer in
529 ;the file pointed to by buf_ID. Thus
530 ;the buffer starts at location
531 ;buf_offset in the file and contains
532 ;buf_fill bytes.
533
534 BUFINSIZ EQU BUFFINFO.size
535
536 buf_Free EQU OFFh ; buf_id of free buffer
537
538 ;Flag byte masks
539 buf_isnet EQU 10000000B
540 buf_dirty EQU 01000000B
541 ***
542 buf_visit EQU 00100000B
543 ***
544 buf_snbuf EQU 00010000B
545
546 buf_isDATA EQU 00001000B
547 buf_isDIR EQU 00000100B
548 buf_isFAT EQU 00000010B
549 buf_type_0 EQU 11110001B ; AND sets type to "none"
550
551 buf_NetID EQU BUFINSIZ
552
553 ;
554 ; C A V E A T P R O G R A M M E R
555 ;-----+-----+-----+-----+-----+-----+-----+-----+-----+-----;
556
557 =====
558 ; DOSSSYM.INC, MSDOS 6.0, 1991
559 =====
560 ; 04/05/2019 - Retro DOS v4.0
561
562 ; <Control character definitions>
563
564 C_DEL EQU 7Fh ; ASCII rubout or delete previous char
565 C_BS EQU 08h ; ^H ASCII backspace
566 C_CR EQU 0Dh ; ^M ASCII carriage return
567 C_LF EQU 0Ah ; ^J ASCII linefeed
568 C_ETB EQU 17h ; ^W ASCII end of transmission
569 C_NAK EQU 15h ; ^U ASCII negative acknowledge
570 C_ETX EQU 03h ; ^C ASCII end of text
571 C_HT EQU 09h ; ^I ASCII tab
572
573 ; <User stack inside of system call>
574 ; Location of user registers relative user stack pointer
575
576 struc user_env ; user_environ
577 00000000 ???? .user_AX: resw 1
578 00000002 ???? .user_BX: resw 1
579 00000004 ???? .user_CX: resw 1
580 00000006 ???? .user_DX: resw 1
581 00000008 ???? .user_SI: resw 1
582 0000000A ???? .user_DI: resw 1
583 0000000C ???? .user_BP: resw 1
584 0000000E ???? .user_DS: resw 1
585 00000010 ???? .user_ES: resw 1
586 00000012 ???? .user_IP: resw 1
587 00000014 ???? .user_CS: resw 1
588 00000016 ???? .user_F: resw 1
589 .size:
590 endstruc
591
592 ; ---- <Disk map> ----
593
594 ; MSDOS partitions the disk into 4 sections:
595 ;
596 phys sector 0: +-----+
597 | boot/reserved |
598 +-----+
599 | File allocation|
600 | table(s) |
601 | (multiple copies |
602 | are kept) |
603 +-----+
604 | Directory |
605 +-----+
606 | File space |
607 +-----+
608 | Unaddressable |
609 | (to end of disk)|
610 +-----+
611
612 ; All partition boundaries are sector boundaries. The size of the FAT is
613 ; adjusted to maximize the file space addressable.
614
615 ; <File allocation Table information>
616
617 ; The File Allocation Table uses a 12-bit entry for each allocation unit on
618 ; the disk. These entries are packed, two for every three bytes. The contents
619 ; of entry number N is found by 1) multiplying N by 1.5; 2) adding the result
620 ; to the base address of the Allocation Table; 3) fetching the 16-bit word

```

[illegible]



```
745 addr_int_abort EQU 4 * int_abort
746 addr_int_command EQU 4 * int_command
747 addr_int_terminate EQU 4 * int_terminate
748 addr_int_ctrl_c EQU 4 * int_ctrl_c
749 addr_int_fatal_abort EQU 4 * int_fatal_abort
750 addr_int_disk_read EQU 4 * int_disk_read
751 addr_int_disk_write EQU 4 * int_disk_write
752 addr_int_keep_process EQU 4 * int_keep_process
753 ;-----+-----;
754 ; C A V E A T P R O G R A M M E R ;
755 ;-----+-----;
756 ;
757 addr_int_spooler EQU 4 * int_spooler
758 addr_int_fastcon EQU 4 * int_fastcon
759 addr_int_ibm EQU 4 * int_IBM
760 ;
761 ; C A V E A T P R O G R A M M E R ;
762 ;-----+-----;
763 ;=====
764 ; DIRENT.INC, MSDOS 6.0, 1991
765 ;=====
766 ; 04/05/2019 - Retro DOS v4.0
767 ;=====
768 ; BREAK <Directory entry>
769 ;
770 ;
771 ; +-----+-----+
772 ; | (12 BYTE) filename/ext | 0 0
773 ; +-----+-----+
774 ; | (BYTE) attributes | 11 B
775 ; +-----+-----+
776 ; | (10 BYTE) reserved | 12 C
777 ; +-----+-----+
778 ; | (WORD) time of last write | 22 16
779 ; +-----+-----+
780 ; | (WORD) date of last write | 24 18
781 ; +-----+-----+
782 ; | (WORD) First cluster | 26 1A
783 ; +-----+-----+
784 ; | (DWORD) file size | 28 1C
785 ; +-----+-----+
786 ;
787 ; First byte of filename = E5 -> free directory entry
788 ; = 00 -> end of allocated directory
789 ; Time: Bits 0-4=seconds/2, bits 5-10=minute, 11-15=hour
790 ; Date: Bits 0-4=day, bits 5-8=month, bits 9-15=year-1980
791 ;
792 ; 01/01/2024
793 %if 0
794 struct dir_entry
795 .dir_name: resb 11 ; file name
796 .dir_attr: resb 1 ; attribute bits
797 .dir_codepg: resw 1 ; code page DOS 4.00
798 .dir_extcluster: resw 1 ; extended attribute starting cluster
799 .dir_attr2: resb 1 ; reserved
800 .dir_pad: resb 5 ; reserved for expansion
801 .dir_time: resw 1 ; time of last write
802 .dir_date: resw 1 ; date of last write
803 .dir_first: resw 1 ; first allocation unit of file
804 .dir_size_l: resw 1 ; low 16 bits of file size
805 .dir_size_h: resw 1 ; high 16 bits of file size
806 .size:
807 endstruc
808 %else
809 ; 01/01/2024 - Retro DOS v5.0 (*)
810 struct dir_entry
811 .dir_name: resb 11 ; file name (short file name)
812 .dir_attr: resb 1 ; file attributes (*)
813 .dir_nt_res: resb 1 ; reserved for use by windows NT. 0
814 .dir_pad: ;resb 7 ; (creation time, last access date
815 ; for use by windows)
816 .dir_crtime_tenth:
817 resb 1
818 .dir_crtime: resw 1
819 .dir_crdate: resw 1
820 .dir_lstaccdate:
821 resw 1
822 .dir_fclus_hi: resw 1 ; FAT32 fs ; high word of first cluster number
823 .dir_time: resw 1 ; time of last write
824 .dir_date: resw 1 ; date of last write
825 .dir_fclus:
826 .dir_first: resw 1 ; first cluster (alloc. unit) of file
827 .dir_file_size:
828 .dir_size_l: resw 1 ; low 16 bits of file size
829 .dir_size_h: resw 1 ; high 16 bits of file size
830 .size:
831 endstruc
832 %endif
833 attr_read_only EQU 1h
834 attr_hidden EQU 2h
835 attr_system EQU 4h
836 attr_volume_id EQU 8h
837 attr_directory EQU 10h
838 attr_archive EQU 20h
839 attr_device EQU 40h ; This is a VERY special bit.
840 ; NO directory entry on a disk EVER
841 ; has this bit set. It is set non-zero
842 ; when a device is found by GETPATH
843 attr_all EQU attr_hidden+attr_system+attr_directory
844 ; OR of hard attributes for FINDENTRY
845 attr_ignore EQU attr_read_only+attr_archive
846 ; ignore this(ese) attribute(s)
847 ; during search first/next
848 attr_changeable EQU attr_read_only+attr_hidden+attr_system+attr_archive
849 ; changeable via CHMOD
850 DIRFREE equ 0E5h ; stored in dir_name[0] to indicate free slot
851 ; -----
852 ; 01/01/2024 - Retro DOS v5.0
853 ; ref: FAT32 File System Specification v1.03 (Microsoft, December 6, 2000) (*)
854 attr_long_name equ attr_read_only|attr_hidden|attr_system|attr_volume_id
855 attr_longname_mask equ attr_long_name|attr_directory|attr_archive
```

[illegible]



```

993 ;
994 ; The system file table entries are allocated in contiguous groups.
995 ; There may be more than one such groups; the SF "superstructure"
996 ; tracks the groups.
997 ; -----
998
999 struct SFT
1000 .SFLink: resd 1
1001 .SFCount: resw 1 ; number of entries
1002 .SFTable: resw 1 ; beginning of array of the following
1003 .size:
1004 endstruc
1005
1006 ; -----
1007 ; ** System file table entry
1008 ;
1009 ; These are the structures which are at SFTABLE in the SF structure.
1010 ; -----
1011
1012 ; 01/01/2024 - Retro DOS v5.0
1013 ; 25/04/2019 - Retro DOS v4.0
1014
1015 struct SF_ENTRY
1016 .sf_ref_count: resw 1 ; 0 ; number of processes sharing entry
1017 ; if FCB then ref count
1018 .sf_mode: resw 1 ; 2 ; mode of access or high bit on if FCB
1019 .sf_attr: resb 1 ; 4 ; attribute of file
1020 .sf_flags: resw 1 ; 5 ; Bits 8-15
1021 ; Bit 15 = 1 if remote file
1022 ; = 0 if local file or device
1023 ; Bit 14 = 1 if date/time is not to be
1024 ; set from clock at CLOSE. Set by
1025 ; FILETIMES and FCB_CLOSE. Reset by
1026 ; other reseters of the dirty bit
1027 ; (WRITE)
1028 ; Bit 13 = Pipe bit (reserved)
1029 ;
1030 ; Bits 0-7 (old FCB_devid bits)
1031 ; If remote file or local file, bit
1032 ; 6=0 if dirty Device ID number, bits
1033 ; 0-5 if local file.
1034 ; bit 7=0 for local file, bit 7
1035 ; =1 for local I/O device
1036 ; If local I/O device, bit 6=0 if EOF (input)
1037 ; Bit 5=1 if Raw mode
1038 ; Bit 0=1 if console input device
1039 ; Bit 1=1 if console output device
1040 ; Bit 2=1 if null device
1041 ; Bit 3=1 if clock device
1042 .sf_devptr: resd 1 ; 7 ; Points to DPB if local file, points
1043 ; to device header if local device,
1044 ; points to net device header if
1045 ; remote
1046 .sf_firclus: resw 1 ; 11 ; First cluster of file (bit 15 = 0)
1047 .sf_time: resw 1 ; 13 ; Time associated with file
1048 .sf_date: resw 1 ; 15 ; Date associated with file
1049 .sf_size: resd 1 ; 17 ; Size associated with file
1050 .sf_position: resd 1 ; 21 ; Read/Write pointer or LRU count for FCBS
1051
1052 ; Starting here, the next 7 bytes may be used by the file system to store
1053 ; an ID
1054
1055 ; 09/07/2018 - Retro DOS v3.0
1056
1057 ; MSDOS 3.3 SF.INC, 1987
1058 .sf_cluspos: resw 1 ; Position of last cluster accessed
1059 .sf_lstclus: resw 1 ; Last cluster accessed
1060 .sf_dirsec: resw 1 ; Sector number of directory sector
1061 ; for this file
1062 .sf_dirpos: resb 1 ; Offset of this entry in the above
1063
1064 ; MSDOS 6.0, SF.INC, 1991
1065 .sf_cluspos: resw 1 ; 25 ; Position of last cluster accessed
1066 .sf_dirsec: resd 1 ; 27 ; Sector number of directory sector
1067 ; for this file
1068 .sf_dirpos: resb 1 ; 31 ; Offset of this entry in the above
1069
1070 ; End of 7 bytes of file-system specific info.
1071
1072 .sf_name: resb 11 ; 32 ; 11 character name that is in the
1073 ; directory entry. This is used by
1074 ; close to detect file deleted and
1075 ; disk changed errors.
1076 .sf_fclus32: ; 22/03/2024
1077 ; SHARING INFO
1078 .sf_chain: resd 1 ; 43 ; link to next SF
1079 .sf_UID: resw 1 ; 47
1080 .sf_PID: resw 1 ; 49 ; owner process identifier (PSP segment)
1081 .sf_MFT: resw 1 ; 51
1082
1083 ; MSDOS 6.0, SF.INC, 1991
1084 .sf_lstclus: resw 1 ; 53 ; AN009; Last cluster accessed
1085 .sf_IFS_HDR: resd 1 ; 55 ; pointer to IFS drive or 0:0 for files
1086
1087 .size:
1088 endstruc
1089
1090 ; 20/07/2018
1091 ; MSDOS 3.3, SF.INC, 1987
1092 %define sf_netid SF_ENTRY.sf_cluspos ; byte
1093 %define sf_OpenAge SF_ENTRY.sf_position+2 ; word
1094 %define sf_LRU SF_ENTRY.sf_position ; word
1095 ; MSDOS 6.0, SF.INC, 1991
1096 %define sf_fsda SF_ENTRY.sf_cluspos ; byte ;DOS 4.00
1097 %define sf_serial_ID SF_ENTRY.sf_firclus ; word ;DOS 4.00
1098
1099 ; 19/07/2018
1100 ; MSDOS 3.3, SF.INC, 1987
1101
1102 sf_default_number EQU 5
1103
1104 ; Note that we need to mark an SFT as being busy for OPEN/CREATE. This is
1105 ; because an INT 24 may prevent us from 'freeing' it. We mark this as such
1106 ; by placing a -1 in the ref_count field.
1107
1108 sf_busy EQU -1
1109
1110 ; mode mask for FCB detection
1111 sf_isFCB EQU 1000000000000000B
1112
1113 ; Flag word masks
1114 sf_isnet EQU 1000000000000000B
1115 sf_close_nodate EQU 0100000000000000B
1116 sf_pipe EQU 0010000000000000B

```

```

1117 sf_no_inherit EQU 0001000000000000B
1118 sf_net_spool EQU 0000100000000000B
1119
1120 ; 25/04/2019
1121 sf_entry_size equ SF_ENTRY.size ; 59 (MSDOS 6.0)
1122
1123 ; -----
1124 ; Local file/device flag masks
1125 ; -----
1126
1127 devid_file_clean EQU 40h ; true if file and not written
1128 devid_file_mask_drive EQU 3Fh ; mask for drive number
1129
1130 devid_device EQU 80h ; true if a device
1131 devid_device_EOF EQU 40h ; true if end of file reached
1132 devid_device_raw EQU 20h ; true if in raw mode
1133 devid_device_special EQU 10h ; true if special device
1134 devid_device_clock EQU 08h ; true if clock device
1135 devid_device_null EQU 04h ; true if null device
1136 devid_device_con_out EQU 02h ; true if console output
1137 devid_device_con_in EQU 01h ; true if console input
1138
1139 ; -----
1140 ; structure of devid field as returned by IOCTL is:
1141 ;
1142 ; BIT 7 6 5 4 3 2 1 0
1143 ; |---|---|---|---|---|---|---|
1144 ; | I | E | R | S | I | I | I | I |
1145 ; | S | O | A | P | S | S | S | S |
1146 ; | D | F | W | E | C | N | C | C |
1147 ; | E | | | C | L | U | O | I |
1148 ; | V | | | L | K | L | T | N |
1149 ; |---|---|---|---|---|---|---|
1150 ; ISDEV = 1 if this channel is a device
1151 ; = 0 if this channel is a disk file
1152 ;
1153 ; If ISDEV = 1
1154 ;
1155 ; EOF = 0 if End Of File on input
1156 ; RAW = 1 if this device is in Raw mode
1157 ; = 0 if this device is cooked
1158 ; ISCLK = 1 if this device is the clock device
1159 ; ISNUL = 1 if this device is the null device
1160 ; ISCOT = 1 if this device is the console output
1161 ; ISCIN = 1 if this device is the console input
1162 ;
1163 ; If ISDEV = 0
1164 ; EOF = 0 if channel has been written
1165 ; Bits 0-5 are the block device number for
1166 ; the channel (0 = A, 1 = B, ...)
1167 ; -----
1168
1169 devid_ISDEV EQU 80h
1170 devid_EOF EQU 40h
1171 devid_RAW EQU 20h
1172 devid_SPECIAL EQU 10h
1173 devid_ISCLK EQU 08h
1174 devid_ISNUL EQU 04h
1175 devid_ISCOT EQU 02h
1176 devid_ISCIN EQU 01h
1177
1178 devid_block_dev EQU 1Fh ; mask for block device number
1179
1180 ; =====
1181 ; PDB.INC, MSDOS 6.0, 1991
1182 ; =====
1183 ; 04/05/2019 - Retro DOS v4.0
1184 ; 08/07/2018 - Retro DOS v3.0
1185
1186 ; -----
1187 ; BREAK <Process data block>
1188 ; -----
1189 ; ** Process data block (otherwise known as program header)
1190 ;
1191 ;
1192 ; These offset are documented in the MSDOS Encyclopedia, so nothing
1193 ; can be rearranged here, ever. Reserved areas are probably safe
1194 ; for use.
1195 ; -----
1196
1197 FILPERPROC EQU 20
1198
1199 struc PDB ; Process_data_block
1200 .EXIT_CALL: resw 1 ; INT int_abort system terminate
1201 .BLOCK_LEN: resw 1 ; size of execution block
1202 .CPM_CALL: resb 1 ;
1203 .EXIT: resd 1 ; pointer to exit routine
1204 .CTRL_C: resd 1 ; pointer to ^C routine
1205 .FATAL_ABORT: resd 1 ; pointer to fatal error
1206 .PARENT_PID: resw 1 ; PID of parent (terminate PID)
1207 .JFN_TABLE: resb FILPERPROC ; indices into system table
1208 .ENVIRON: resw 1 ; segment address of environment
1209 .USER_STACK: resd 1 ; stack of self during system calls
1210 .JFN_Length: resw 1 ; number of handles allowed
1211 .JFN_Pointer: resd 1 ; pointer to JFN table
1212 .Next_PDB: resd 1 ; pointer to nested PDB's
1213 .InterCon: resb 1 ; MSDOS 6.0 ; *** jh-3/28/90 ***
1214 .Append: resb 1 ; MSDOS 6.0 ; *** Not sure if still used ***
1215 .Novell_Used: resb 2 ; MSDOS 6.0 ; Novell shell (redir) uses these
1216 .Version: resw 1 ; MSDOS 6.0 ; DOS version reported to this app
1217 .PAD1: resb 14 ; 0Eh
1218 .CALL_SYSTEM: resb 5 ; portable method of system call
1219 .PAD2: resb 7 ; reserved so FCB 1 can be used as
1220 ; an extended FCB
1221 ;endstruc ; MSDOS 3.3
1222 ; MSDOS 6.0
1223
1224 .FCB1: resb 16 ; 10h ; default FCB 1
1225 .FCB2: resb 16 ; 10h ; default FCB 2
1226 .PAD3: resb 4 ; not sure if this is used by PDB_FCB2
1227 .TAIL: resb 128 ; command tail and default DTA
1228 endstruc
1229
1230 ; =====
1231 ; EXE.INC, MSDOS 6.0, 1991
1232 ; =====
1233 ; 04/05/2019 - Retro DOS v4.0
1234
1235 ; ** EXE.INC - Definitions for the EXEC command and EXE files
1236 ; -----
1237 ; The following get used as arguments to the EXEC system call. They indicate
1238 ; whether or not the program is executed or whether or not a program header
1239 ; gets created.
1240

```

```

1241      exec_func_no_execute EQU 1; no execute bit
1242      exec_func_overlay    EQU 2; overlay bit
1243
1244      struc EXEC0
1245      .ENVIRON: resw 1      ; seg addr of environment
1246      .COM_LINE: resd 1     ; pointer to asciz command line
1247      .5C_FCB:  resd 1     ; default fcb at 5C
1248      .6C_FCB:  resd 1     ; default fcb at 6C
1249      .size:
1250      endstruc
1251
1252      struc EXEC1
1253      .ENVIRON: resw 1      ; seg addr of environment
1254      .COM_LINE: resd 1     ; pointer to asciz command line
1255      .5C_FCB:  resd 1     ; default fcb at 5C
1256      .6C_FCB:  resd 1     ; default fcb at 6C
1257      .SP:      resw 1      ; stack pointer of program
1258      .SS:      resw 1      ; stack seg register of program
1259      .IP:      resw 1      ; entry point IP
1260      .CS:      resw 1      ; entry point CS
1261      .size:
1262      endstruc
1263
1264      struc EXEC3
1265      .load_addr: resw 1     ; seg address of load point
1266      .reloc_fac: resw 1     ; relocation factor
1267      endstruc
1268
1269      ;** Exit codes (in upper byte) for terminating programs
1270
1271      EXIT_TERMINATE      EQU      0
1272      EXIT_ABORT          EQU      0
1273      EXIT_CTRL_C         EQU      1
1274      EXIT_HARD_ERROR     EQU      2
1275      EXIT_KEEP_PROCESS   EQU      3
1276
1277      ;** EXE File Header Description
1278
1279      struc EXE
1280      .signature: resw 1     ; must contain 4D5A (yay zibo!)
1281      .len_mod_512: resw 1   ; low 9 bits of length
1282      .pages:      resw 1    ; number of 512b pages in file
1283      .rle_count:  resw 1    ; count of reloc entries
1284      .par_dir:    resw 1    ; number of paragraphs before image
1285      .min_BSS:    resw 1    ; minimum number of para of BSS
1286      .max_BSS:    resw 1    ; max number of para of BSS
1287      .SS:         resw 1    ; stack of image
1288      .SP:         resw 1    ; SP of image
1289      .chksum:     resw 1    ; checksum of file (ignored)
1290      .IP:         resw 1    ; IP of entry
1291      .CS:         resw 1    ; CS of entry
1292      .rle_table:  resw 1    ; byte offset of reloc table
1293      .iov:        resw 1    ; overlay number (0 for root)
1294      .sym_tab:    resd 1    ; offset of symbol table in file
1295      .size:
1296      endstruc
1297
1298      exe_valid_signature EQU 5A4Dh
1299      exe_valid_old_signature EQU 4D5Ah
1300
1301      ;** EXE file symbol info definitions
1302
1303      struc symbol_entry
1304      .value:      resd 1
1305      .type:       resw 1
1306      .len:        resb 1
1307      .name:       resb 255
1308      endstruc
1309
1310      ;** Data structure passed for ExecReady call
1311
1312      struc ERStruc
1313      .ER_Reserved: resw 1    ; reserved, should be zero
1314      .ER_Flags:    resw 1
1315      .ER_ProgName: resd 1    ; ptr to ASCIIZ str of prog name
1316      .ER_PSP:      resw 1    ; PSP of the program
1317      .ER_StartAddr: resd 1   ; Start CS:IP of the program
1318      .ER_ProgSize: resd 1    ; Program size including PSP
1319      .size:
1320      endstruc
1321
1322      ;** bit fields in ER_Flags
1323
1324      ER_EXE      equ      0001h
1325      ER_OVERLAY  equ      0002h
1326
1327
1328      ;=====
1329      ; ARENA.INC, MSDOS 6.0, 1991
1330      ;=====
1331      ; 24/04/2019 - Retro DOS v4.0
1332      ; 04/08/2018 - Retro DOS v3.0
1333
1334      ;BREAK <Memory arena structure>
1335
1336      ;** Arena Header
1337
1338      struc ARENA
1339      .SIGNATURE: resb 1      ; 4D for valid item, 5A for last item
1340      .OWNER:      resw 1     ; owner of arena item
1341      .SIZE:       resw 1     ; size in paragraphs of item
1342      .RESERVED:   resb 3     ; reserved
1343      .NAME:       resb 8     ; owner file name
1344      .headersize:
1345      endstruc
1346
1347      ; 20/05/2019 - Retro DOS v4.0
1348      ARENAHEADERSIZE equ ARENA.headersize
1349
1350      ; CAUTION: The routines in ALLOC.ASM rely on the fact that arena_signature
1351      ; and arena_owner_system are all equal to zero and are contained in DI.
1352      ; change them and change ALLOC.ASM.
1353
1354      arena_owner_system EQU 0      ; free block indication
1355
1356      arena_signature_normal EQU 4Dh ; valid signature, not end of arena
1357      arena_signature_end    EQU 5Ah ; valid signature, last block in arena
1358
1359      FIRST_FIT EQU 00000000B
1360      BEST_FIT  EQU 00000001B
1361      LAST_FIT  EQU 00000010B
1362
1363      ; MSDOS 6.0
1364      LOW_FIRST EQU 00000000B      ; M001

```

```

1365 HIGH_FIRST EQU 10000000B ; M001
1366 HIGH_ONLY EQU 01000000B ; M001
1367
1368 LINKSTATE EQU 00000001B ; M002
1369
1370 HF_MASK EQU ~HIGH_FIRST ; M001
1371 HO_MASK EQU ~HIGH_ONLY ; M001
1372
1373 STRAT_MASK EQU HF_MASK & HO_MASK ; M001;
1374 ; M026: used to mask of bits
1375 ; M026: 6 & 7 of AllocMethod
1376
1377 ;=====
1378 ; MI.INC, MSDOS 6.0, 1991
1379 ;=====
1380 ; 07/07/2018 - Retro DOS v3.0
1381
1382 ;BREAK <Machine instruction, flag definitions and character types>
1383
1384 mi_INT EQU 0CDh
1385 mi_long_jump EQU 0EAh
1386 mi_Long_CALL EQU 09Ah
1387 mi_Long_RET EQU 0CBh
1388 mi_Near_RET EQU 0C3h
1389
1390 ;
1391 f_Overflow EQU xxxxoditszxaxpxc
1392 f_Direction EQU 0000100000000000B
1393 f_Interrupt EQU 0000001000000000B
1394 f_Trace EQU 0000000100000000B
1395 f_Sign EQU 0000000010000000B
1396 f_Zero EQU 0000000001000000B
1397 f_Aux EQU 0000000000010000B
1398 f_Parity EQU 0000000000000100B
1399 f_Carry EQU 0000000000000001B
1400
1401 ;=====
1402 ; FILEMODE.INC, MSDOS 6.0, 1991
1403 ;=====
1404 ; 13/07/2018 - Retro DOS v3.0
1405 ; 29/04/2019 - Retro DOS v4.0
1406
1407 ;** Standard I/O file handles
1408
1409 stdin EQU 0
1410 stdout EQU 1
1411 stderr EQU 2
1412 stdaux EQU 3
1413 stdprn EQU 4
1414
1415 ;** File Modes
1416 ; <Xenix subfunction assignments> ; MSDOS 3.3 FILEMODE.INC
1417
1418 open_for_read EQU 0
1419 open_for_write EQU 1
1420 open_for_both EQU 2
1421
1422 ; MSDOS 6.0
1423 OPEN_FOR_BOTH equ 2
1424 EXEC_OPEN equ 3 ; access code of 3 indicates that open was
1425 ; made from exec
1426
1427 access_mask EQU 0Fh ; 09/08/2018
1428
1429 SHARING_MASK equ 0F0h
1430 SHARING_COMPAT equ 000h
1431 SHARING_DENY_BOTH equ 010h
1432 SHARING_DENY_WRITE equ 020h
1433 SHARING_DENY_READ equ 030h
1434 SHARING_DENY_NONE equ 040h
1435 SHARING_NET_FCB equ 070h
1436 SHARING_NO_INHERIT equ 080h
1437
1438 ; 29/04/2019
1439
1440 ;** Extended Open Definitions
1441
1442 RESERVED_BITS_MASK equ 0FE00h ; reserved bits for extended open flags
1443 EXISTS_MASK equ 0Fh ; "file exists" action field
1444 NOT_EXISTS_MASK equ 0F0h
1445
1446 ;* SF_MODE values
1447
1448 AUTO_COMMIT_WRITE equ 4000h
1449 INT_24_ERROR equ 2000h
1450
1451 ;* Flags in EXTOPEN_ON
1452
1453 EXT_OPEN_ON equ 01h
1454 EXT_FILE_NOT_EXISTS equ 04h
1455 EXT_OPEN_I24_OFF equ 02h
1456
1457 ;* Flags in EXTOPEN_FLAG
1458
1459 ACTION_OPENED equ 01h
1460 ACTION_CREATED_OPENED equ 02h
1461 ACTION_REPLACED_OPENED equ 03h
1462 EXT_EXISTS_OPEN equ 01h
1463 EXT_EXISTS_FAIL equ 00h
1464 EXT_NEXISTS_CREATE equ 10h
1465
1466 ;** Extended Open Structure
1467
1468 struc EXT_OPEN_PARM
1469 .SET_LIST: resd 1
1470 .NUM_OF_PARM: resw 1
1471 endstruc
1472
1473 ;=====
1474 ; SYSCALL.INC, MSDOS 6.0, 1991
1475 ;=====
1476 ; 29/04/2019 - Retro DOS v4.0
1477 ; 09/07/2018 - Retro DOS v3.0 (SYSCALL.INC, MSDOS 3.3, 1987)
1478
1479 ; <system call definitions>
1480
1481 ABORT EQU 0 ; 0 0
1482 STD_CON_INPUT EQU 1 ; 1 1
1483 STD_CON_OUTPUT EQU 2 ; 2 2
1484 STD_AUX_INPUT EQU 3 ; 3 3
1485 STD_AUX_OUTPUT EQU 4 ; 4 4
1486 STD_PRINTER_OUTPUT EQU 5 ; 5 5
1487 RAW_CON_IO EQU 6 ; 6 6
1488 RAW_CON_INPUT EQU 7 ; 7 7

```

|      |                                |        |      |    |  |
|------|--------------------------------|--------|------|----|--|
| 1489 | STD_CON_INPUT_NO_ECHO          | EQU 8  | ; 8  | 8  |  |
| 1490 | STD_CON_STRING_OUTPUT          | EQU 9  | ; 9  | 9  |  |
| 1491 | STD_CON_STRING_INPUT           | EQU 10 | ; 10 | A  |  |
| 1492 | STD_CON_INPUT_STATUS           | EQU 11 | ; 11 | B  |  |
| 1493 | STD_CON_INPUT_FLUSH            | EQU 12 | ; 12 | C  |  |
| 1494 | DISK_RESET                     | EQU 13 | ; 13 | D  |  |
| 1495 | SET_DEFAULT_DRIVE              | EQU 14 | ; 14 | E  |  |
| 1496 | FCB_OPEN                       | EQU 15 | ; 15 | F  |  |
| 1497 | FCB_CLOSE                      | EQU 16 | ; 16 | 10 |  |
| 1498 | DIR_SEARCH_FIRST               | EQU 17 | ; 17 | 11 |  |
| 1499 | DIR_SEARCH_NEXT                | EQU 18 | ; 18 | 12 |  |
| 1500 | FCB_DELETE                     | EQU 19 | ; 19 | 13 |  |
| 1501 | FCB_SEQ_READ                   | EQU 20 | ; 20 | 14 |  |
| 1502 | FCB_SEQ_WRITE                  | EQU 21 | ; 21 | 15 |  |
| 1503 | FCB_CREATE                     | EQU 22 | ; 22 | 16 |  |
| 1504 | FCB_RENAME                     | EQU 23 | ; 23 | 17 |  |
| 1505 | GET_DEFAULT_DRIVE              | EQU 25 | ; 25 | 19 |  |
| 1506 | SET_DMA                        | EQU 26 | ; 26 | 1A |  |
| 1507 |                                |        |      |    |  |
| 1508 |                                |        |      |    |  |
| 1509 |                                |        |      |    |  |
| 1510 | GET_DEFAULT_DPB                | EQU 31 | ; 31 | 1F |  |
| 1511 |                                |        |      |    |  |
| 1512 |                                |        |      |    |  |
| 1513 |                                |        |      |    |  |
| 1514 | FCB_RANDOM_READ                | EQU 33 | ; 33 | 21 |  |
| 1515 | FCB_RANDOM_WRITE               | EQU 34 | ; 34 | 22 |  |
| 1516 | GET_FCB_FILE_LENGTH            | EQU 35 | ; 35 | 23 |  |
| 1517 | GET_FCB_POSITION               | EQU 36 | ; 36 | 24 |  |
| 1518 | SET_INTERRUPT_VECTOR           | EQU 37 | ; 37 | 25 |  |
| 1519 | CREATE_PROCESS_DATA_BLOCK      | EQU 38 | ; 38 | 26 |  |
| 1520 | FCB_RANDOM_READ_BLOCK          | EQU 39 | ; 39 | 27 |  |
| 1521 | FCB_RANDOM_WRITE_BLOCK         | EQU 40 | ; 40 | 28 |  |
| 1522 | PARSE_FILE_DESCRIPTOR          | EQU 41 | ; 41 | 29 |  |
| 1523 | GET_DATE                       | EQU 42 | ; 42 | 2A |  |
| 1524 | SET_DATE                       | EQU 43 | ; 43 | 2B |  |
| 1525 | GET_TIME                       | EQU 44 | ; 44 | 2C |  |
| 1526 | SET_TIME                       | EQU 45 | ; 45 | 2D |  |
| 1527 | SET_VERIFY_ON_WRITE            | EQU 46 | ; 46 | 2E |  |
| 1528 | ; Extended functionality group |        |      |    |  |
| 1529 | GET_DMA                        | EQU 47 | ; 47 | 2F |  |
| 1530 | GET_VERSION                    | EQU 48 | ; 48 | 30 |  |
| 1531 | KEEP_PROCESS                   | EQU 49 | ; 49 | 31 |  |
| 1532 |                                |        |      |    |  |
| 1533 |                                |        |      |    |  |
| 1534 |                                |        |      |    |  |
| 1535 | GET_DPB                        | EQU 50 | ; 50 | 32 |  |
| 1536 |                                |        |      |    |  |
| 1537 |                                |        |      |    |  |
| 1538 |                                |        |      |    |  |
| 1539 | SET_CTRL_C_TRAPPING            | EQU 51 | ; 51 | 33 |  |
| 1540 | GET_INDOS_FLAG                 | EQU 52 | ; 52 | 34 |  |
| 1541 | GET_INTERRUPT_VECTOR           | EQU 53 | ; 53 | 35 |  |
| 1542 | GET_DRIVE_FREESPACE            | EQU 54 | ; 54 | 36 |  |
| 1543 | CHAR_OPER                      | EQU 55 | ; 55 | 37 |  |
| 1544 | INTERNATIONAL                  | EQU 56 | ; 56 | 38 |  |
| 1545 | ; XENIX CALLS                  |        |      |    |  |
| 1546 | ; Directory Group              |        |      |    |  |
| 1547 | MKDIR                          | EQU 57 | ; 57 | 39 |  |
| 1548 | RMDIR                          | EQU 58 | ; 58 | 3A |  |
| 1549 | CHDIR                          | EQU 59 | ; 59 | 3B |  |
| 1550 | ; File Group                   |        |      |    |  |
| 1551 | CREAT                          | EQU 60 | ; 60 | 3C |  |
| 1552 | OPEN                           | EQU 61 | ; 61 | 3D |  |
| 1553 | CLOSE                          | EQU 62 | ; 62 | 3E |  |
| 1554 | READ                           | EQU 63 | ; 63 | 3F |  |
| 1555 | WRITE                          | EQU 64 | ; 64 | 40 |  |
| 1556 | UNLINK                         | EQU 65 | ; 65 | 41 |  |
| 1557 | LSEEK                          | EQU 66 | ; 66 | 42 |  |
| 1558 | CHMOD                          | EQU 67 | ; 67 | 43 |  |
| 1559 | IOCTL                          | EQU 68 | ; 68 | 44 |  |
| 1560 | XDUP                           | EQU 69 | ; 69 | 45 |  |
| 1561 | XDUP2                          | EQU 70 | ; 70 | 46 |  |
| 1562 | CURRENT_DIR                    | EQU 71 | ; 71 | 47 |  |
| 1563 | ; Memory Group                 |        |      |    |  |
| 1564 | ALLOC                          | EQU 72 | ; 72 | 48 |  |
| 1565 | DEALLOC                        | EQU 73 | ; 73 | 49 |  |
| 1566 | SETBLOCK                       | EQU 74 | ; 74 | 4A |  |
| 1567 | ; Process Group                |        |      |    |  |
| 1568 | EXEC                           | EQU 75 | ; 75 | 4B |  |
| 1569 | EXIT                           | EQU 76 | ; 76 | 4C |  |
| 1570 | _WAIT                          | EQU 77 | ; 77 | 4D |  |
| 1571 | FIND_FIRST                     | EQU 78 | ; 78 | 4E |  |
| 1572 | ; Special Group                |        |      |    |  |
| 1573 | FIND_NEXT                      | EQU 79 | ; 79 | 4F |  |
| 1574 | ; SPECIAL SYSTEM GROUP         |        |      |    |  |
| 1575 |                                |        |      |    |  |
| 1576 |                                |        |      |    |  |
| 1577 |                                |        |      |    |  |
| 1578 | SET_CURRENT_PDB                | EQU 80 | ; 80 | 50 |  |
| 1579 | GET_CURRENT_PDB                | EQU 81 | ; 81 | 51 |  |
|      |                                |        |      |    |  |

```

1613 XNAMETRANS EQU 96 ; 96 60
1614 PATHPARSE EQU 97 ; 97 61
1615 GETCURRENTPSP EQU 98 ; 98 62
1616 HONGEUL EQU 99 ; 99 63
1617 ;-----+-----+-----+-----+-----+-----+-----+-----+-----;
1618 ; C A V E A T P R O G R A M M E R ;
1619 ; ;
1620 SET_PRINTER_FLAG EQU 100 ; 100 64
1621 ; ;
1622 ; C A V E A T P R O G R A M M E R ;
1623 ;-----+-----+-----+-----+-----+-----+-----+-----+-----;
1624 GETEXTCNTRY EQU 101 ; 101 65
1625 GETSETCDPG EQU 102 ; 102 66
1626 EXTHANDLE EQU 103 ; 103 67
1627 COMMIT EQU 104 ; 104 68
1628
1629 ; 29/04/2019 - Retro DOS v4.0
1630 ; (MSDOS 6.0, SYSCALL.INC, 1987)
1631
1632 GetSetMediaID EQU 105 ; 105 69
1633 IFS_IOCTL EQU 107 ; 107 6B
1634 ExtOpen EQU 108 ; 108 6C
1635
1636 ;-----+-----+-----+-----+-----+-----+-----+-----+-----;
1637 ; C A V E A T P R O G R A M M E R ;
1638 ; ;
1639 ;ifdef ROMEXEC
1640 ;ROM_FIND_FIRST EQU 109 ; 109 6D
1641 ;ROM_FIND_NEXT EQU 110 ; 110 6E
1642 ;ROM_EXCLUDE EQU 111 ; 111 6F ; M035
1643 ;endif
1644 ; ;
1645 ; C A V E A T P R O G R A M M E R ;
1646 ;-----+-----+-----+-----+-----+-----+-----+-----+-----;
1647
1648 SET_OEM_HANDLER EQU 248 ; 248 F8
1649 ;OEM_C1 EQU 249 ; 249 F9
1650 ;OEM_C2 EQU 250 ; 250 FA
1651 ;OEM_C3 EQU 251 ; 251 FB
1652 ;OEM_C4 EQU 252 ; 252 FC
1653 ;OEM_C5 EQU 253 ; 253 FD
1654 ;OEM_C6 EQU 254 ; 254 FE
1655 ;OEM_C7 EQU 255 ; 255 FF
1656
1657 ; 01/01/2024 - Retro DOS v5.0 - PCDOS 7.1 IBMDOS.COM
1658 ; (PCDOS 7.1 extension to MSDOS 6.22 system calls)
1659
1660 ExtCountryInfo equ 112 ; 70h
1661 LONGNAME equ 113 ; 71h
1662 LFNFINDCLOSE equ 114 ; 72h
1663 FAT32EXT equ 115 ; 73h
1664
1665 ;=====
1666 ; VERSIONA.INC (MSDOS 6.0, 1991)
1667 ;=====
1668 ; 24/04/2019 - Retro DOS 4.0
1669
1670 ;MAJOR_VERSION EQU 6
1671 ;;MINOR_VERSION EQU 00
1672 ;MINOR_VERSION EQU 21 ; MSDOS 6.21
1673
1674 ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
1675 ;MAJOR_VERSION EQU 5
1676 ;MINOR_VERSION EQU 0
1677
1678 ; 30/12/2022 - Retro DOS v4.2
1679 ;MAJOR_VERSION EQU 6
1680 ;MINOR_VERSION EQU 22
1681
1682 ; 01/01/2024 - Retro DOS v5.0
1683 MAJOR_VERSION EQU 7
1684 MINOR_VERSION EQU 10 ; PCDOS 7.1 (7.10)
1685
1686 ;=====
1687 ; INTNAT.INC, MSDOS 3.3, 1987
1688 ;=====
1689 ; 09/07/2018 - Retro DOS 3.0
1690
1691 ; Current structure of the data returned by the international call
1692
1693 struc INTERNAT_BLOCK ; (-*-) Same with MSDOS 2.11 & MSDOS 6.0
1694 .Date_tim_format:
1695     RESW 1 ; 0-USA, 1-EUR, 2-JAP
1696 .Currency_sym:
1697     RESB 5 ; Currency Symbol 5 bytes
1698 .Thous_sep:
1699     RESB 2 ; Thousands separator 2 bytes
1700 .Decimal_sep:
1701     RESB 2 ; Decimal separator 2 bytes
1702 .Date_sep:
1703     RESB 2 ; Date separator 2 bytes
1704 .Time_sep:
1705     RESB 2 ; Time separator 2 bytes
1706 .Bit_field:
1707     RESB 1 ; Bit values
1708         ; Bit 0 = 0 if currency symbol first
1709         ; Bit 1 = 1 if currency symbol last
1710         ; Bit 1 = 0 if No space after currency symbol
1711         ; Bit 1 = 1 if space after currency symbol
1712 .Currency_cents:
1713     RESB 1 ; Number of places after currency dec point
1714 .Time_24:
1715     RESB 1 ; 1 if 24 hour time, 0 if 12 hour time
1716 .Map_call:
1717     RESW 1 ; Address of case mapping call (DWORD)
1718     RESW 1 ; THIS IS TWO WORDS SO IT CAN BE INITIALIZED
1719             ; in pieces.
1720 .Data_sep:
1721     RESB 2 ; Data list separator character
1722 .size:
1723 endstruc
1724
1725 ; Max size of the block returned by the INTERNATIONAL call
1726 internat_block_max EQU 32
1727
1728 ;=====
1729 ; SYSVAR.INC (MSDOS 6.0, 1991)
1730 ;=====
1731 ; 08/07/2018 - Retro DOS v3.0
1732
1733 ; 01/01/2024 - Retro DOS v5.0
1734
1735 ;SysInitVars STRUC
1736
```



```

1737 struct SYSI
1738 .DPB:      resd 1      ; 0      ; DPB chain
1739 .SFT:      resd 1      ; 4      ; SFT chain
1740 .CLOCK:    resd 1      ; 8      ; CLOCK device
1741 .CON:      resd 1      ; 12     ; CON device
1742 .MAXSEC:   resw 1 ; 16 ; maximum sector size
1743 .BUF:      resd 1      ; 18     ; points to Hashinitvar
1744 .CDS:      resd 1      ; 22     ; CDS list
1745 .FCB:      resd 1      ; 26     ; FCB chain
1746 .Keep:     resw 1      ; 30     ; keep count
1747 .NUMIO:    resb 1      ; 32     ; Number of block devices
1748 .NCDS:     resb 1      ; 33     ; number of CDS's
1749 .DEV:      resd 1      ; 34     ; device list
1750 ; 09/07/2018
1751 ; Above parameters are described in MSDOS 3.3 SYSVAR.INC (85/04/10)
1752 ; Following parameters are used with MSDOS 6.0 (Retro DOS v4.0)
1753 .ATTR:     resw 1      ; 38     ; null device attribute word
1754 .STRAT:     resw 1      ; 40     ; null device strategy entry point
1755 .INTER:     resw 1      ; 42     ; null device interrupt entry point
1756 .NAME:      resb 8      ; 44     ; null device name
1757 .SPLICE:    resb 1 ; 52 ; TRUE -> splicees being done
1758 .IBMDOS_SIZE: resw 1 ; 53 ; DOS size in paragraphs
1759 .IFS_DOSCALL@: resd 1 ; 55 ; IFS DOS service routine entry
1760 .IFS:       resd 1      ; 59     ; IFS header chain
1761 .BUFFERS:   resw 2 ; 63 ; BUFFERS= values (m,n)
1762 .BOOT_DRIVE: resb 1      ; 67     ; boot drive A=1 B=2,..
1763 .DWMOVE:    resb 1 ; 68 ; 1 if 386 machine
1764 .EXT_MEM:   resw 1 ; 69 ; Extended memory size in KB.
1765 endstruc
1766 ;SysInitVars ENDS
1767
1768 ;This is added for more information exchange between DOS, BIOS.
1769 ;DOS will give the pointer to SysInitTable in ES:DI. - J.K. 5/29/86
1770
1771 ;SysInitVars_Ext struc
1772 struc SYSI_EXT
1773 .SysInitVars: resd 1      ; Points to the above structure.
1774 .Country_Tab: resd 1      ; DOS_Country_cdpq_info
1775 endstruc
1776 ;SysInitVars_Ext ends
1777
1778 ;=====
1779 ; IOCTL.INC - MSDOS 6.0 - 1991
1780 ;=====
1781 ; 09/07/2018 - Retro DOS v3.0
1782
1783 ;*** J.K.
1784 ;General Guide -
1785 ;Category Code:
1786 ; 0... .... DOS Defined
1787 ; 1... .... User defined
1788 ; .xxx xxxx Code
1789
1790 ;Function Code:
1791 ; 0... .... Return error if unsupported
1792 ; 1... .... Ignore if unsupported
1793 ; .0.. .... Intercepted by DOS
1794 ; .1.. .... Passed to driver
1795 ; ..0. .... Sends data/commands to device
1796 ; ..1. .... Queries data/info from device
1797 ; ...x .... Subfunction
1798 ;
1799 ; Note that "Sends/queries" data bit is intended only to regularize the
1800 ; function set. It plays no critical role; some functions may contain both
1801 ; command and query elements. The convention is that such commands are
1802 ; defined as "sends data".
1803
1804 ;*****,*
1805 ; BLOCK DRIVERS ;*
1806 ;*****,*
1807
1808 ; IOCTL SUB-FUNCTIONS
1809 ; (MSDOS 3.3 + MSDOS 6.0)
1810 IOCTL_GET_DEVICE_INFO EQU 0
1811 IOCTL_SET_DEVICE_INFO EQU 1
1812 IOCTL_READ_HANDLE EQU 2
1813 IOCTL_WRITE_HANDLE EQU 3
1814 IOCTL_READ_DRIVE EQU 4
1815 IOCTL_WRITE_DRIVE EQU 5
1816 IOCTL_GET_INPUT_STATUS EQU 6
1817 IOCTL_GET_OUTPUT_STATUS EQU 7
1818 IOCTL_CHANGEABLE? EQU 8
1819 IOCTL_DeviceLocOrRem? EQU 9
1820 IOCTL_HandleLocOrRem? EQU 0Ah ;10
1821 IOCTL_SHARING_RETRY EQU 0Bh ;11
1822 GENERIC_IOCTL_HANDLE EQU 0Ch ;12
1823 GENERIC_IOCTL EQU 0Dh ;13
1824 ; (MSDOS 6.0 + MSDOS 3.3)
1825 IOCTL_GET_DRIVE_MAP EQU 0Eh ;14
1826 IOCTL_SET_DRIVE_MAP EQU 0Fh ;15
1827 ; (MSDOS 6.0)
1828 IOCTL_QUERY_HANDLE EQU 10h ;16
1829 IOCTL_QUERY_BLOCK EQU 11h ;17
1830
1831 ; GENERIC IOCTL CATEGORY CODES
1832 IOC_OTHER EQU 0 ; other device control J.K. 4/29/86
1833 IOC_SE EQU 1 ; SERIAL DEVICE CONTROL
1834 IOC_TC EQU 2 ; TERMINAL CONTROL
1835 IOC_SC EQU 3 ; SCREEN CONTROL
1836 IOC_KC EQU 4 ; KEYBOARD CONTROL
1837 IOC_PC EQU 5 ; PRINTER CONTROL
1838 IOC_DC EQU 8 ; DISK CONTROL (SAME AS RAWIO)
1839
1840 ; GENERIC IOCTL SUB-FUNCTIONS
1841 RAWIO EQU 8
1842
1843 ; RAWIO SUB-FUNCTIONS
1844 ; (MSDOS 3.3 + MSDOS 6.0)
1845 GET_DEVICE_PARAMETERS EQU 60H
1846 SET_DEVICE_PARAMETERS EQU 40H
1847 READ_TRACK EQU 61H
1848 WRITE_TRACK EQU 41H
1849 VERIFY_TRACK EQU 62H
1850 FORMAT_TRACK EQU 42H
1851 ; (MSDOS 6.0)
1852 GET_MEDIA_ID EQU 66h ;AN000;AN003;changed from 63h
1853 SET_MEDIA_ID EQU 46h ;AN000;AN003;changed from 43h
1854 GET_ACCESS_FLAG EQU 67h ;AN002;AN003;Unpublished function.Changed from 64h
1855 SET_ACCESS_FLAG EQU 47h ;AN002;AN003;Unpublished function.Changed from 44h
1856 SENSE_MEDIA_TYPE EQU 68H ;Added for 5.00
1857
1858 ; SPECIAL FUNCTION FOR GET DEVICE PARAMETERS
1859 BUILD_DEVICE_BPB EQU 00000001B
1860

```

```

1861 ; SPECIAL FUNCTIONS FOR SET DEVICE PARAMETERS
1862 INSTALL_FAKE_BPB EQU 000000001B
1863 ONLY_SET_TRACKLAYOUT EQU 000000010B
1864 TRACKLAYOUT_IS_GOOD EQU 000000100B
1865
1866 ; SPECIAL FUNCTION FOR FORMAT TRACK
1867 ; (MSDOS 3.3 + MSDOS 6.0)
1868 STATUS_FOR_FORMAT EQU 000000001B
1869 ; (MSDOS 6.0)
1870 DO_FAST_FORMAT EQU 000000010B ;AN001;
1871
1872 ; CODES RETURNED FROM FORMAT STATUS CALL
1873 FORMAT_NO_ROM_SUPPORT EQU 000000001B
1874 FORMAT_COMB_NOT_SUPPORTED EQU 000000010B
1875
1876 ; DEVICETYPE VALUES
1877 ; (MSDOS 3.3 + MSDOS 6.0)
1878 MAX_SECTORS_IN_TRACK EQU 63 ; MAXIMUM SECTORS ON A DISK.(was 40 in DOS 3.2)
1879 DEV_5INCH EQU 0
1880 DEV_5INCH96TPI EQU 1
1881 DEV_3INCH720KB EQU 2
1882 DEV_8INCHSS EQU 3
1883 DEV_8INCHDS EQU 4
1884 DEV_HARDDISK EQU 5
1885 DEV_OTHER EQU 7
1886 ; (MSDOS 6.0)
1887 ;DEV_3INCH1440KB EQU 7
1888 DEV_3INCH2880KB EQU 9
1889 ; Retro DOS v2.0 - 26/03/2018
1890 ;;DEV_TAPE EQU 6
1891 ;;DEV_ERIMO EQU 8
1892 ;DEV_3INCH2880KB EQU 9
1893 DEV_3INCH1440KB EQU 10
1894
1895 ; (MSDOS 3.3)
1896 ;MAX_DEV_TYPE EQU 7
1897
1898 ; (MSDOS 6.0)
1899 MAX_DEV_TYPE EQU 10 ; MAXIMUM DEVICE TYPE THAT WE
1900 ; CURRENTLY SUPPORT.
1901
1902 00000000 ???? struc A_SECTORTABLE
1903 00000002 ???? .ST_SECTORNUMBER: resw 1
1904 .ST_SECTORSIZE: resw 1
1905 .size:
1906 endstruc
1907
1908 ;=====
1909 ; DEVSYS.INC
1910 ;=====
1911 ; 07/07/2018 - Retro DOS v3.0
1912 ; 30/04/2019 - Retro DOS v4.0 (DEVSYS.INC, MSDOS 6.0, 1991)
1913 ; 21/02/2024 - Retro DOS v5.0
1914
1915 ;** DevSym.inc - Device Symbols
1916
1917 ; The device table list has the form:
1918 struc SYSDEV
1919 00000000 ???????? .NEXT: resd 1 ;Pointer to next device header
1920 00000004 ???? .ATT: resw 1 ;Attributes of the device
1921 00000006 ???? .STRAT: resw 1 ;Strategy entry point
1922 00000008 ???? .INT: resw 1 ;Interrupt entry point
1923 0000000A ???????????????? .NAME: resb 8 ;Name of device (only first byte used for block)
1924 .size:
1925 endstruc
1926
1927 ;
1928 ; ATTRIBUTE BIT MASKS
1929 ;
1930 ; CHARACTER DEVICES:
1931 ;
1932 ; BIT 15 -> MUST BE 1
1933 ; 14 -> 1 IF THE DEVICE UNDERSTANDS IOCTL CONTROL STRINGS
1934 ; 13 -> 1 IF THE DEVICE SUPPORTS OUTPUT-UNTIL-BUSY
1935 ; 12 -> UNUSED
1936 ; 11 -> 1 IF THE DEVICE UNDERSTANDS OPEN/CLOSE
1937 ; 10 -> MUST BE 0
1938 ; 9 -> MUST BE 0
1939 ; 8 -> UNUSED
1940 ; 7 -> UNUSED
1941 ; 6 -> UNUSED
1942 ; 5 -> UNUSED
1943 ; 4 -> 1 IF DEVICE IS RECIPIENT OF INT 29H
1944 ; 3 -> 1 IF DEVICE IS CLOCK DEVICE
1945 ; 2 -> 1 IF DEVICE IS NULL DEVICE
1946 ; 1 -> 1 IF DEVICE IS CONSOLE OUTPUT
1947 ; 0 -> 1 IF DEVICE IS CONSOLE INPUT
1948 ;
1949 ; BLOCK DEVICES:
1950 ;
1951 ; BIT 15 -> MUST BE 0
1952 ; 14 -> 1 IF THE DEVICE UNDERSTANDS IOCTL CONTROL STRINGS
1953 ; 13 -> 1 IF THE DEVICE DETERMINES MEDIA BY EXAMINING THE FAT ID BYTE.
1954 ; THIS REQUIRES THE FIRST SECTOR OF THE FAT TO *ALWAYS* RESIDE IN
1955 ; THE SAME PLACE.
1956 ; 12 -> UNUSED
1957 ; 11 -> 1 IF THE DEVICE UNDERSTANDS OPEN/CLOSE/REMOVABLE MEDIA
1958 ; 10 -> MUST BE 0
1959 ; 9 -> MUST BE 0
1960 ; 8 -> UNUSED
1961 ; 7 -> UNUSED
1962 ; 6 -> IF DEVICE HAS SUPPORT FOR GETMAP/SETMAP OF LOGICAL DRIVES.
1963 ; IF THE DEVICE UNDERSTANDS GENERIC IOCTL FUNCTION CALLS.
1964 ; 5 -> UNUSED
1965 ; 4 -> UNUSED
1966 ; 3 -> UNUSED
1967 ; 2 -> UNUSED
1968 ; 1 -> UNUSED
1969 ; 0 -> UNUSED
1970 ;
1971 ;Attribute bit masks
1972 DEVTYP EQU 8000H ;Bit 15 - 1 if char, 0 if block
1973 DEVIOCTL EQU 4000H ;Bit 14 - CONTROL mode bit
1974 ISFATBYDEV EQU 2000H ;Bit 13 - Device uses FAT ID bytes, comp media.
1975
1976 ; 09/07/2018 - Retro DOS (DEVSYS.INC, MSDOS 3.3, 1987)
1977
1978 OUTTILBUSY EQU 2000H ; OUTPUT UNTIL BUSY IS ENABLED
1979 ISNET EQU 1000H ; BIT 12 - 1 IF A NET DEVICE, 0 IF
1980 ; NOT. CURRENTLY BLOCK ONLY.
1981 DEVOPCL EQU 0800H ; BIT 11 - 1 IF THIS DEVICE HAS
1982 ; OPEN,CLOSE AND REMOVABLE MEDIA
1983 ; ENTRY POINTS, 0 IF NOT
1984

```



```

1985 EXTENTBIT EQU 0400H ; BIT 10 - CURRENTLY 0 ON ALL DEVS
1986 ; THIS BIT IS RESERVED FOR FUTURE USE
1987 ; TO EXTEND THE DEVICE HEADER BEYOND
1988 ; ITS CURRENT FORM.
1989
1990 ; NOTE BIT 9 IS CURRENTLY USED ON IBM SYSTEMS TO INDICATE "DRIVE IS SHARED".
1991 ; SEE IOCTL FUNCTION 9. THIS USE IS NOT DOCUMENTED, IT IS USED BY SOME
1992 ; OF THE UTILITIES WHICH ARE SUPPOSED TO FAIL ON SHARED DRIVES ON SERVER
1993 ; MACHINES (FORMAT,CHKDSK,RECOVER,...).
1994
1995 IOQUERY EQU 0080H ;Bit 7 - Supports generic IOCTL query
1996
1997 DEV320 EQU 0040H ;BIT 6 - FOR BLOCK DEVICES, THIS
1998 ;DEVICE SUPPORTS SET/GET MAP OF
1999 ;LOGICAL DRIVES, AND SUPPORTS
2000 ;GENERIC IOCTL CALLS.
2001 ;FOR CHARACTER DEVICES, THIS
2002 ;DEVICE SUPPORTS GENERIC IOCTL.
2003 ;THIS IS A DOS 3.2 DEVICE DRIVER.
2004
2005 ISSPEC EQU 0010H ;Bit 4 - This device is special ; 15/03/2018
2006 ;ISIBM EQU 0010H ;Bit 4 - This device is special
2007 ISCLOCK EQU 0008H ;Bit 3 - This device is the clock device.
2008 ISNULL EQU 0004H ;Bit 2 - This device is the null device.
2009 ISCOU EQU 0002H ;Bit 1 - This device is the console output.
2010 ISCI EQU 0001H ;Bit 0 - This device is the console input.
2011
2012 EXTDRV EQU 0002h ;BIT 1 - BLOCK DEVICE EXTENDED DRIVER
2013 ; (MSDOS 6.0, DEVSYM.INC, 1991) ; 30/04/2019
2014
2015 ;Static Request Header
2016 struc SRHEAD
2017 .REQLEN: resb 1 ;Length in bytes of request block
2018 .REQUNIT: resb 1 ;Device unit number
2019 .REQFUNC: resb 1 ;Type of request
2020 .REQSTAT: resw 1 ;Status word
2021 .RESERVED: resb 8 ;Reserved for queue links
2022 .size:
2023 endstruc
2024
2025 ;Status word masks
2026 STERR EQU 8000H ;Bit 15 - Error
2027 STBUI EQU 0200H ;Bit 9 - Busy
2028 STDON EQU 0100H ;Bit 8 - Done
2029 STECODE EQU 00FFH ;Error code
2030 WRECODE EQU 0
2031
2032 ;Function codes
2033 ;DINITHL EQU 26 ;Size of init header
2034 ; 11/04/2024 - Retro DOS 5.0
2035 DINITHL EQU 25 ; PCDOS 7.1 ;Size of init header
2036 DMEDHL EQU 15 ;Size of media check header
2037 DBPBHL EQU 22 ;Size of Get BPB header
2038 DRDWRHL EQU 22 ;Size of RD/WR header
2039 DRDNDHL EQU 14 ;Size of non destructive read header
2040 DSTATHL EQU 13 ;Size of status header
2041 ; 21/02/2024
2042 ;DFLSHL EQU 15 ;Size of flush header
2043 DFLSHL EQU 13 ; PCDOS 7.1 IBMDOS.COM ; 21/02/2024
2044
2045 DEVINIT EQU 0 ;Initialization
2046 DEVMDCH EQU 1 ;Media check
2047 DEVBPB EQU 2 ;Get BPB
2048 DEVRDIOCTL EQU 3 ;IOCTL read
2049 DEVRD EQU 4 ;Read
2050 DEVRDND EQU 5 ;Non destructive read no wait (character devs)
2051 DEVIST EQU 6 ;Input status
2052 DEVIFL EQU 7 ;Input flush
2053 DEVRT EQU 8 ;write
2054 DEVRTV EQU 9 ;write with verify
2055 DEVOST EQU 10 ;Output status
2056 DEVOFL EQU 11 ;Output flush
2057 DEVRIOCTL EQU 12 ;IOCTL write
2058
2059 ; 09/07/2018 - Retro DOS v3.0 (DEVSYM.INC, MSDOS 3.3, 1987)
2060 DEVOPN EQU 13 ;DEVICE OPEN
2061 DEVCLS EQU 14 ;DEVICE CLOSE
2062 DOPCLHL EQU 13 ;SIZE OF OPEN/CLOSE HEADER
2063 DEVRMD EQU 15 ;REMOVABLE MEDIA
2064 ; 07/08/2018 - Retro DOS v3.0
2065 REMHL EQU 13 ;SIZE OF REMOVABLE MEDIA HEADER
2066 GENIOCTL EQU 19
2067
2068 ; THE NEXT THREE ARE USED IN DOS 4.0
2069 ; 20
2070 ; 21
2071 ; 22
2072
2073 DEVGETOWN EQU 23 ;GET DEVICE OWNER
2074 DEVSETOWN EQU 24 ;SET DEVICE OWNER
2075 ; 18/05/2019 - Retro DOS v4.0
2076 IOCTL_QUERY EQU 25 ;Query generic ioctl support
2077
2078 OWNHL EQU 13 ;SIZE OF DEVICE OWNER HEADER
2079
2080 DEVOUT EQU 16 ; OUTPUT UNTIL BUSY.
2081 DEVOUTL EQU DEVRT ; LENGTH OF OUTPUT UNTIL BUSY
2082
2083 ; ADDED FOR DOS 5.00
2084
2085 ; GENERIC IOCTL REQUEST STRUCTURE
2086 ; SEE THE DOS 4.0 DEVICE DRIVER SPEC FOR FURTHER ELABORATION.
2087
2088 struc IOCTL_REQ
2089 .SRHEAD: resb SRHEAD.size ; GENERIC IOCTL ADDITION.
2090 .MAJORFUNCTION: resb 1 ;FUNCTION CODE
2091 .MINORFUNCTION: resb 1 ;FUNCTION CATEGORY
2092 .REG_SI: resw 1
2093 .REG_DI: resw 1
2094 .GENERICIOCTL_PACKET: resd 1 ; POINTER TO DATA BUFFER
2095 .size: ; 07/08/2018
2096 endstruc
2097
2098 ; DEFINITIONS FOR IOCTL_REQ.MINORFUNCTION
2099 GEN_IOCTL_WRT_TRK EQU 40H
2100 GEN_IOCTL_RD_TRK EQU 60H
2101 GEN_IOCTL_FN_TST EQU 20H ; USED TO DIFF. BET READS AND WRTS
2102
2103 ;; 32-bit absolute read/write input list structure
2104
2105 struc ABS_32RW
2106 .SECTOR_RBA: resd 1 ; relative block address
2107 .ABS_RW_COUNT: resw 1 ; number of sectors to be transferred
2108

```

```

2109 00000006 ???????? .BUFFER_ADDR:      resd 1      ; data address
2110 .size:
2111 endstruc
2112
2113 ;; media ID info
2114
2115 struc MEDIA_ID_INFO
2116 00000000000000000000 .MEDIA_level:      resw 1      ; info level
2117 000000002 ???????? .MEDIA_Serial:      resd 1      ; serial #
2118 000000006 <res Bh> .MEDIA_Label:      resb 11     ; volume label
2119 000000011 ???????? .MEDIA_System:      resb 8      ; system type
2120 .size:
2121 endstruc
2122
2123 ; equates for DOS34_FLAG
2124 ; (BUGBUG: why are bits 0,1,3 and 4 not defined.)
2125
2126 FROM_DISK_RESET      EQU 000000000100b ;from disk reset
2127 Force_I24_Fail      EQU 000000100000b ;form IFS CALL BACK
2128 Disable_EOF_I24     EQU 000001000000b ;disable EOF int24 for input status
2129 DBCS_VOLID          EQU 000010000000b ;indicate from volume id
2130 DBCS_VOLID2         EQU 000100000000b ;indicate 8th char is DBCS
2131 CTRL_BREAK_FLAG     EQU 001000000000b ;indicate control break is input
2132 SEARCH_FASTOPEN     EQU 010000000000b ;set fastopen flag for search
2133 EXEC_AWARE_REDIREX  EQU 100000000000b ;M018: this bit is set by a redir
2134 ;M018: that knows how to handle
2135 ;M018: open for exec
2136
2137 NO_FROM_DISK_RESET   EQU ~FROM_DISK_RESET ;not from disk reset
2138 NO_Force_I24_Fail   EQU ~Force_I24_Fail ;not form IFS CALL BACK
2139 NO_Disable_EOF_I24  EQU ~Disable_EOF_I24
2140
2141 ;=====
2142 ; ERROR.INC (MSDOS 6.0, 1991)
2143 ;=====
2144 ; 16/07/2018 - Retro DOS v3.0
2145
2146 ** ERROR.INC - DOS Error Codes
2147
2148 ; The newer (DOS 2.0 and above) "XENIX-style" calls
2149 ; return error codes through AX. If an error occurred then
2150 ; the carry bit will be set and the error code is in AX. If no error
2151 ; occurred then the carry bit is reset and AX contains returned info.
2152
2153 ; Since the set of error codes is being extended as we extend the operating
2154 ; system, we have provided a means for applications to ask the system for a
2155 ; recommended course of action when they receive an error.
2156
2157 ; The GetExtendedError system call returns a universal error, an error
2158 ; location and a recommended course of action. The universal error code is
2159 ; a symptom of the error REGARDLESS of the context in which GetExtendedError
2160 ; is issued.
2161
2162 ; 2.0 error codes
2163
2164 error_invalid_function      EQU 1
2165 error_file_not_found       EQU 2
2166 error_path_not_found       EQU 3
2167 error_too_many_open_files  EQU 4
2168 error_access_denied        EQU 5
2169 error_invalid_handle       EQU 6
2170 error_arena_trashed        EQU 7
2171 error_not_enough_memory    EQU 8
2172 error_invalid_block        EQU 9
2173 error_bad_environment      EQU 10
2174 error_bad_format           EQU 11
2175 error_invalid_access       EQU 12
2176 error_invalid_data         EQU 13
2177 ;**** reserved            EQU 14 ; ****
2178 error_invalid_drive        EQU 15
2179 error_current_directory    EQU 16
2180 error_not_same_device      EQU 17
2181 error_no_more_files        EQU 18
2182
2183 ; These are the universal int 24 mappings for the old INT 24 set of errors
2184
2185 error_write_protect        EQU 19
2186 error_bad_unit            EQU 20
2187 error_not_ready           EQU 21
2188 error_bad_command         EQU 22
2189 error_CRC                 EQU 23
2190 error_bad_length          EQU 24
2191 error_seek                EQU 25
2192 error_not_DOS_disk        EQU 26
2193 error_sector_not_found    EQU 27
2194 error_out_of_paper        EQU 28
2195 error_write_fault         EQU 29
2196 error_read_fault          EQU 30
2197 error_gen_failure         EQU 31
2198
2199 ; the new 3.0 error codes reported through INT 24
2200
2201 error_sharing_violation    EQU 32
2202 error_lock_violation      EQU 33
2203 error_wrong_disk          EQU 34
2204 error_FCB_unavailable     EQU 35
2205 error_sharing_buffer_exceeded EQU 36
2206 error_Code_Page_Mismatched EQU 37 ; DOS 4.00 ;AN000;
2207 error_handle_EOF          EQU 38 ; DOS 4.00 ;AN000;
2208 error_handle_Disk_Full    EQU 39 ; DOS 4.00 ;AN000;
2209
2210 ; New OEM network-related errors are 50-79
2211
2212 error_not_supported        EQU 50
2213
2214 error_net_access_denied    EQU 65 ;M028
2215
2216 ; End of INT 24 reportable errors
2217
2218 error_file_exists          EQU 80
2219 error_DUP_FCB              EQU 81 ; ****
2220 error_cannot_make          EQU 82
2221 error_FAIL_I24             EQU 83
2222
2223 ; New 3.0 network related error codes
2224
2225 error_out_of_structures    EQU 84
2226 error_already_assigned     EQU 85
2227 error_invalid_password     EQU 86
2228 error_invalid_parameter    EQU 87
2229 error_NET_write_fault       EQU 88
2230 error_sys_comp_not_loaded  EQU 90 ; DOS 4.00 ;AN000;
2231
2232 ; BREAK <Interrupt 24 error codes>

```

```

2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356

; ** Int24 Error Codes
error_I24_write_protect EQU 0
error_I24_bad_unit EQU 1
error_I24_not_ready EQU 2
error_I24_bad_command EQU 3
error_I24_CRC EQU 4
error_I24_bad_length EQU 5
error_I24_Seek EQU 6
error_I24_not_DOS_disk EQU 7
error_I24_sector_not_found EQU 8
error_I24_out_of_paper EQU 9
error_I24_write_fault EQU 0Ah
error_I24_read_fault EQU 0Bh
error_I24_gen_failure EQU 0Ch
; NOTE: Code 0DH is used by MT-DOS.
error_I24_wrong_disk EQU 0Fh

; THE FOLLOWING ARE MASKS FOR THE AH REGISTER ON Int 24
;
; NOTE: ABORT is ALWAYS allowed
Allowed_FAIL EQU 00001000B
Allowed_RETRY EQU 00010000B
Allowed_IGNORE EQU 00100000B

I24_operation EQU 00000001B ;Z if READ,NZ if Write
I24_area EQU 00000110B ; 00 if DOS
; 01 if FAT
; 10 if root DIR
; 11 if DATA
I24_class EQU 10000000B ;Z if DISK, NZ if FAT or char

; BREAK <GetExtendedError CLASSEs ACTIONs LOCUSs>

; ** The GetExtendedError call takes an error code and returns CLASS,
; ACTION and LOCUS codes to help programs determine the proper action
; to take for error codes that they don't explicitly understand.

; Values for error CLASS
errCLASS_OutRes EQU 1 ; Out of Resource
errCLASS_TempSit EQU 2 ; Temporary Situation
errCLASS_Auth EQU 3 ; Permission problem
errCLASS_Intrn EQU 4 ; Internal System Error
errCLASS_HrdFail EQU 5 ; Hardware Failure
errCLASS_SysFail EQU 6 ; System Failure
errCLASS_Apperr EQU 7 ; Application Error
errCLASS_NotFnd EQU 8 ; Not Found
errCLASS_BadFmt EQU 9 ; Bad Format
errCLASS_Locked EQU 10 ; Locked
errCLASS_Media EQU 11 ; Media Failure
errCLASS_Already EQU 12 ; Collision with Existing Item
errCLASS_Unk EQU 13 ; Unknown/other

; Values for error ACTION
errACT_Retry EQU 1 ; Retry
errACT_DlyRet EQU 2 ; Delay Retry, retry after pause
errACT_User EQU 3 ; Ask user to regive info
errACT_Abort EQU 4 ; abort with clean up
errACT_Panic EQU 5 ; abort immediately
errACT_Ignore EQU 6 ; ignore
errACT_IntRet EQU 7 ; Retry after User Intervention

; Values for error LOCUS
errLOC_Unk EQU 1 ; No appropriate value
errLOC_Disk EQU 2 ; Random Access Mass Storage
errLOC_Net EQU 3 ; Network
errLOC_SerDev EQU 4 ; Serial Device
errLOC_Mem EQU 5 ; Memory

;=====
; INT2A.INC (MSDOS 6.0, 1991)
;=====
; 04/05/2019 - Retro DOS v4.0

; ** Int 2A functions
; -----
; Int 2A is an interface to the network code; it's also overloaded
; as a critical section handler since critical sections
; were originally created to support the net.
; -----

; -----
; ** This table was created by examining the source and may not be
; complete or completely accurate - JGL
;
; M010 MD 8/31/90 - Added definition for AH = 5
;
; (ah) = 0 installation check
; (returns ah !=0 if installed)
; (ah) = 1 cooked net bios call
; (ah) = 3 query drive shared
; (ds:si) = "n:" asciz string
; (ah) = 4 net bios
; (al) = 0 cooked net bios call
; (al) = 1 raw net bios call
; (al) = 2 ???
;
; (ah) = 5 Get Net Adaptor Resources. CX returns the number of
; NCBS available/outstanding. DX returns the number of
; sessions. Supposedly, this is documented in an old
; IBM PC-LAN reference. Lotus Notes uses it. DOS LAN
; Manager 2.0 Enhanced responds to it. But it should
; not be used, as it is a hack, only to get Lotus
; Notes running.
;
; (ah) = 80h enter critical section
; (ah) = 81h leave critical section
; (ah) = 82h free all critical sections (Leave-all)
; (ah) = 84h entering idle loop (don't understand how this works)
; -----

; ** Critical section definitions
; -----
; Although DOS is not designed to be reentrant there are some hacks
; which various programs use to make it so, in a limited fashion.
; Both WIN386 and some servers block copy a section of the DOS data
; area so that DOS can be reentered on behalf of another thread/program.
; DOS's global data structures, such as the memory arena, are not
; in this area, so critical section indicators are used to protect

```

```

2357 ; those areas. DOS flags a critical section by issuing an INT_IBM
2358 ; (int 2Ah) at each critical section entry and exit. Some clients
2359 ; (such as WIN386) just don't "context switch" the DOS when one
2360 ; of these is in effect, others, such as the IBM server, go ahead
2361 ; and reenter the DOS and if they get an int 2A to reenter the same
2362 ; critical section they then switch away from that second thread and
2363 ; let the first one finish and exit the section.
2364 ; -----
2365 ;
2366 ; These below are subject to leave-all sections
2367 critDisk EQU 1 ; Disk I/O critical section
2368 critShare EQU 1 ; Sharer I/O critical section
2369 critMem EQU 1 ; memory maintenance critical section
2370 critSFT EQU 1 ; sft table allocation
2371 critDevice EQU 2 ; Device I/O critical section
2372 critNet EQU 5 ; network critical section
2373 critIFS EQU 6 ; ifsfunc critical section
2374 ; These below are not subject to leave-all sections
2375 critASSIGN EQU 8 ; Assign has munged a system call
2376 ;
2377 ;=====
2378 ; MULT.INC (MSDOS 6.0, 1991)
2379 ;=====
2380 ; 04/05/2019 - Retro DOS v4.0
2381 ;
2382 ;Break <Multiplex channels>
2383 ;
2384 ; -----
2385 ; The current set of defined multiplex channels is (* means documented):
2386 ;
2387 ; Channel(h) Issuer Receiver Function
2388 ; 00 server PSPRINT print job control
2389 ; *01 print/apps PRINT Queueing of files
2390 ; 02 BIOS REDIR signal open/close of printers
2391 ;
2392 ; 05 command REDIR obtain text of net int 24 message
2393 ; *06 server/assign ASSIGN Install check
2394 ;
2395 ; 08 external driver IBMBIO interface to internal routines
2396 ;
2397 ; 10 sharer/server Sharer install check
2398 ; 11 DOS/server Redir install check/redirection funcs
2399 ; 12 sharer/redir DOS dos functions and structure maint
2400 ; 13 MSNET MSNET movement of NCBS
2401 ; 13 external driver IBMBIO Reset_Int_13, allows installation
2402 ; of alternative INT_13 drivers after
2403 ; boot_up
2404 ; 14 (IBM) DOS NLSFUNC down load NLS country info,DOS 3.3
2405 ; 14 (MS) APPS POPUP MSDOS 4 popup screen functions
2406 ; 15 APPS MSCDEX CD-ROM extensions interface
2407 ; 16 WIN386 WIN386 windows communications
2408 ; 17 Clipboard WINDOWS Clipboard interface
2409 ; *18 Applications MS-Manger Toggle interface to manager
2410 ; 19 Shell
2411 ; 1A Ansi.sys
2412 ; 1B Fastopen,Vdisk IBMBIO EMS INT 67H stub handler
2413 ;
2414 ; 40h OS/2
2415 ; 41h Lanman
2416 ; 42h Lanman
2417 ; 43h Himem
2418 ; AL = 20h reserved for Mach 20 Himem support
2419 ; AL = 30h reserved for Himem external A20 code
2420 ;
2421 ; 44h Dosextdender
2422 ; 45h Windows profiler
2423 ; 46h windows/286 DOS extender
2424 ; 47h Basic Compiler Vn. 7.0
2425 ; 48h Doskey
2426 ; 49h DOS 5.x install
2427 ; 4Ah Multi Purpose
2428 ; multMULTSWPDSK 0 - Swap Disk in drive A (BIOS)
2429 ; multMULTGETHMAPTR 1 - Get available HMA & ptr
2430 ; multMULTALLOCHMA 2 - Allocate HMA (bx == no of bytes)
2431 ; multMULTTASKSHELL 5 - Shell/switcher API
2432 ; multMULTRPLTOM 6 - Top Of Memory for RPL support
2433 ;
2434 ; multSmartdrv 10h
2435 ; multMagicdrv 11h
2436 ; 4Bh Task Switcher API
2437 ;
2438 ; 4Ch APPS APM Advanced power management
2439 ; 4Dh Kana Kanji Converter, MSKK
2440 ;
2441 ; 51h ODI real mode support driver (for Chicago)
2442 ;
2443 ; 53h POWER.EXE - used for broadcasting APM events ; M036
2444 ; 54h POWER.EXE - used for POWER API ; M036
2445 ;
2446 ; 55h COMMAND.COM
2447 ; multCOMFIRST 0 - API to determine whether 1st
2448 ; instance of command.com
2449 ; multCOMFIRSTROM 1 - API to determine whether 1st
2450 ; instance of ROM COMMAND
2451 ;
2452 ; 56h Sewell Development
2453 ; INTERLNK
2454 ;
2455 ; 57h Iomega Corp.
2456 ;
2457 ; ABh Unspecified IBM use
2458 ; ACh Graphics
2459 ; ADh NLS (toronto)
2460 ; AEh
2461 ; AFh Mode
2462 ; B0h GRAFTABL GRAFTABL
2463 ;
2464 ; D7h Banyan VINES
2465 ; -----
2466 ;MUX 00-3F reserved for IBM
2467 ;MUX 80-BF reserved for IBM
2468 ;
2469 ;MUX 40-7F reserved for Microsoft
2470 ;
2471 ;MUX C0-FF users
2472 multSHARE EQU 10h ; sharer
2473 ; 1 MFT_enter
2474 ; 2 MFTclose
2475 ; 3 MFTclU
2476 ; 4 MFTcloseP
2477 ; 5 MFTclon
2478 ; 6 set_block
2479 ; 7 clr_block
2480 ; 8 chk_block

```

```

2481 ; 9 MFT_get
2482 ; 10 ShSave
2483 ; 11 ShChk
2484 ; 12 ShCol
2485 ; 13 ShCloseFile
2486
2487 MultNET EQU 11h ; Network support
2488 MultIFS EQU 11h ; Network support
2489 ; 1 IFS_RMDIR
2490 ; 2 IFS_SEQ_RMDIR
2491 ; 3 IFS_MKDIR
2492 ; 4 IFS_SEQ_MKDIR
2493 ; 5 IFS_CHDIR
2494 ; 6 IFS_CLOSE
2495 ; 7 IFS_COMMIT
2496 ; 8 IFS_READ
2497 ; 9 IFS_WRITE
2498 ; 10 IFS_LOCK
2499 ; 11 IFS_UNLOCK
2500 ; 12 IFS_DISK_INFO
2501 ; 13 IFS_SET_FILE_ATTRIBUTE
2502 ; 14 IFS_SEQ_SET_FILE_ATTRIBUTE
2503 ; 15 IFS_GET_FILE_INFO
2504 ; 16 IFS_SEQ_GET_FILE_INFO
2505 ; 17 IFS_RENAME
2506 ; 18 IFS_SEQ_RENAME
2507 ; 19 IFS_DELETE
2508 ; 20 IFS_SEQ_DELETE
2509 ; 21 IFS_OPEN
2510 ; 22 IFS_SEQ_OPEN
2511 ; 23 IFS_CREATE
2512 ; 24 IFS_SEQ_CREATE
2513 ; 25 IFS_SEQ_SEARCH_FIRST
2514 ; 26 IFS_SEQ_SEARCH_NEXT
2515 ; 27 IFS_SEARCH_FIRST
2516 ; 28 IFS_SEARCH_NEXT
2517 ; 29 IFS_ABORT
2518 ; 30 IFS_ASSOPER
2519 ; 31 Printer_SET_STRING
2520 ; 32 IFSFlushBuf
2521 ; 33 IFSBufWrite
2522 ; 34 IFSResetEnvironment
2523 ; 35 IFSspoolCheck
2524 ; 36 IFSspoolClose
2525 ; 37 IFSDeviceOper
2526 ; 38 IFSspoolEchoCheck
2527 ; 39 - - - Unused - - -
2528 ; 40 - - - Unused - - -
2529 ; 41 - - - Unused - - -
2530 ; 42 SERVER_DOSCALL_CLOSEFILES_FOR_UID
2531 ; 43 DEVICE_IOCTL
2532 ; 44 IFS_UPDATE_CB
2533 ; 45 IFS_FILE_XATTRIBUTES
2534 ; 46 IFS_XOPEN
2535 ; 47 IFS_DEPENDENT_IOCTL
2536
2537 MultDOS EQU 12h ; DOS call back
2538 ; 1 DOS_CLOSE
2539 ; 2 RECSET
2540 ; 3 Get DOSGROUP
2541 ; 4 PATHCHRCMP
2542 ; 5 OUT
2543 ; 6 NET_I24_ENTRY
2544 ; 7 PLACEBUF
2545 ; 8 FREE_SFT
2546 ; 9 BUFWRITE
2547 ; 10 SHARE_VIOLATION
2548 ; 11 SHARE_ERROR
2549 ; 12 SET_SFT_MODE
2550 ; 13 DATE16
2551 ; 14 SETVISIT
2552 ; 15 SCANPLACE
2553 ; 16 SKIPVISIT
2554 ; 17 StrCpy
2555 ; 18 StrLen
2556 ; 19 UCase
2557 ; 20 POINTCOMP
2558 ; 21 CHECKFLUSH
2559 ; 22 SFFromSFN
2560 ; 23 GetCDSFromDrv
2561 ; 24 Get_User_Stack
2562 ; 25 GetThisDrv
2563 ; 26 DriveFromText
2564 ; 27 SETYEAR
2565 ; 28 DSUM
2566 ; 29 DSLIDE
2567 ; 30 StrCmp
2568 ; 31 initcds
2569 ; 32 pjfnfromhandle
2570 ; 33 $NameTrans
2571 ; 34 CAL_LK
2572 ; 35 DEVNAME
2573 ; 36 Idle
2574 ; 37 DStrLen
2575 ; 38 NLS_OPEN DOS 3.3
2576 ; 39 $CLOSE DOS 3.3
2577 ; 40 NLS_LSEEK DOS 3.3
2578 ; 41 $READ DOS 3.3
2579 ; 42 FastInit DOS 4.0
2580 ; 43 NLS_IOCTL DOS 3.3
2581 ; 44 GetDevList DOS 3.3
2582 ; 45 NLS_GETEXT DOS 3.3
2583 ; 46 MSG_RETRIEVAL DOS 4.0
2584 ; 47 FAKE_VERSION DOS 4.0
2585
2586 NLSFUNC EQU 14h ; NLSFUNC CALL , DOS 3.3
2587 ; 0 NLSInstall
2588 ; 1 ChgCodePage
2589 ; 2 GetExtInfo
2590 ; 3 SetCodePage
2591 ; 4 GetCntry
2592
2593 multANSI EQU 1Ah ; ANSI multiplex number
2594 ; 0 INSTALL_CHECK ; install check for ANSI
2595 ; 1 IOCTL_2F ; 2F interface to IOCTL
2596 ; 2 DA_INFO_2F ; J.K. Information passing to ANSI.
2597
2598 multMULT EQU 4Ah
2599 multMAGIC EQU 256*multMULT + 11h
2600 multMULTRPLTOM EQU 06h
2601
2602 ; 0 swap disk function for single floppy drive m/cs
2603 ; BIOS broadcasts with cx==0, and apps who handle
2604 ; swap disk messaging set cx == -1. BIOS sets dl == requested

```

```

2605 ; drive
2606 ;
2607 ; 1 Get available HMA & pointer to it. Returns in BX & ES:DI
2608 ; 2 Allocate HMA. BX == number of bytes in HMA to be allocated
2609 ; returns pointer in ES:DI
2610 ;
2611 ; 3-4 currently used by nobody
2612 ; 5 Switcher API
2613 ; 6 Top of Memory for RPL.
2614 ; BIOS issues INT 2f AX=4a06 & DX = Top of Mem and any RPL
2615 ; code present in TOM should respond with a new TOM in DX
2616 ; to protect itself from MSLOAD & SYSINIT tromping over it.
2617 ; SYSINIT builds an arena with owner type 8 & name 'RPL' to
2618 ; protect the RPL code from COMMAND.COM transient protion.
2619 ; It is the responsibility of RPL program to release the mem.
2620 ; 7 Reserved for PROTMAN support.
2621 ; 10 smartdrv 4.0
2622 ; 11 dblspace api
2623 ; 12 MRCI api
2624 ; 13 dblspace/mrci stealth packet api
2625
2626 MultAPM EQU 4ch ; Obsolete ???
2627 ; 00h APM_VER_CHK
2628 ; 01h APM_SUS_SYS_REQ
2629 ; FFh APM_SUS_RES_BATT_NOTIFY
2630
2631 MultPWR_BRDCST EQU 53h ; Used by POWER.EXE to broadcast ; M036
2632 ; APM events ; M036
2633 MultPWR_API EQU 54h ; Used for accessing POWER.EXE's API ; M036
2634
2635 ;FASTOPEN is not chained through INT 2F ; DOS 3.3 F.C.
2636 ; it calls Multdos 42 to set up an entry routine address
2637 ; 0 Install status (reserved)
2638 ; 1 Lookup
2639 ; 2 Insert
2640 ; 3 Delete
2641 ; 4 Purge (reserved)
2642
2643 ;=====
2644 ; FIND.INC (MSDOS 6.0, 1991)
2645 ;=====
2646 ; 17/05/2019 - Retro DOS v4.0
2647 ; 09/07/2018 - Retro DOS v3.0 (MSDOS 3.3, 1987)
2648
2649 ;Break <find first/next buffer>
2650
2651 struc find_buf
2652 .drive: resb 1 ; drive of search
2653 .name: resb 11 ; formatted name
2654 .sattr: resb 1 ; attribute of search
2655 .LastEnt: resw 1 ; LastEnt
2656 .DirStart: resw 1 ; DirStart
2657 .NETID: resb 4 ; MSDOS 6.0 ; Reserved for NET
2658 .attr: resb 1 ; attribute found
2659 .time: resw 1 ; time
2660 .date: resw 1 ; date
2661 .size_l: resw 1 ; low(size)
2662 .size_h: resw 1 ; high(size)
2663 .pname: resb 13 ; packed name
2664 .size:
2665 endstruc
2666
2667 ;=====
2668 ; DOSCNTRY.INC (MSDOS 6.0, 1991)
2669 ;=====
2670 ; 29/04/2019 - Retro DOS v4.0
2671 ; 09/07/2018 - Retro DOS v3.0 (MSDOS 3.3, 1987)
2672
2673 ;Equates for COUNTRY INFORMATION.
2674 SetCountryInfo EQU 1 ;country info
2675 SetUcase EQU 2 ;uppercase table
2676 SetLcase EQU 3 ;lowercase table (Reserved)
2677 SetUcaseFile EQU 4 ;uppercase file spec table
2678 SetFileList EQU 5 ;valid file character list
2679 SetCollate EQU 6 ;collating sequence
2680 SetDBCS EQU 7 ;double byte character set
2681 SetALL EQU -1 ;all the entries
2682
2683 ;DOS country and code page information table structure.
2684 ;Internally, IBMDOS gives a pointer to this table.
2685 ;IBMBIO, MODE and NLSFUNC modules communicate with IBMDOS through
2686 ;this structure.
2687
2688 struc DOS_CCDPG ; DOS_country_cdpd_info
2689 .ccInfo_reserved: resb 8 ;reserved for internal use
2690 .ccPath_CountrySys: resb 64 ;path and filename for country info
2691 .ccSysCodePage: resw 1 ;system code page id
2692 .ccNumber_of_entries: resw 1 ; (default value = 6)
2693 .ccSetUcase: resb 1 ; (default value = SetUcase)
2694 .ccUcase_ptr: resd 1 ;pointer to Ucase table
2695
2696 .ccSetUcaseFile: resb 1 ; (default value = SetUcaseFile)
2697 .ccFileUcase_ptr: resd 1 ;pointer to File Ucase table
2698
2699 .ccSetFileList: resb 1 ; (default value = SetFileList)
2700 .ccFileChar_ptr: resd 1 ;pointer to File char list table
2701
2702 .ccSetCollate: resb 1 ; (default value = SetCollate)
2703 .ccCollate_ptr: resd 1 ;pointer to collate table
2704
2705 ; MSDOS 6.0
2706 .ccSetDBCS: resb 1 ; (default value = SetDBCS)
2707 .ccDBCS_ptr: resd 1 ; pointer to DBCS table
2708
2709 .ccSetCountryInfo: resb 1 ; (default value = SetCountryInfo)
2710 .ccCountryInfoLen: resw 1 ;length of country info
2711 .ccDosCountry: resw 1 ;system country code id
2712 .ccDosCodePage: resw 1 ;system code page id
2713 .ccDFormat: resw 1 ;date format
2714 .ccCurSymbol: resb 5 ;5 byte of (currency symbol+0)
2715 .cc1000Sep: resb 2 ;2 byte of (1000 sep. + 0)
2716 .ccDecSep: resb 2 ;2 byte of (Decimal sep. + 0)
2717 .ccDateSep: resb 2 ;2 byte of (date sep. + 0)
2718 .ccTimeSep: resb 2 ;2 byte of (time sep. + 0)
2719 .ccCFormat: resb 1 ;currency format flags
2720 .ccSigDigits: resb 1 ;# of digits in currency
2721 .ccTFormat: resb 1 ;time format
2722 .ccMono_ptr: resd 1 ;monocase routine entry point
2723 .ccListSep: resb 2 ;data list separator
2724 .ccReserved_area: resw 5 ;reserved
2725 .size:
2726 endstruc
2727
2728 ;Ucase table

```



[illegible]

```

2853 ; | Current Extent(word) |
2854 ; +-----+
2855 ; | Record size (word) |
2856 ; +-----+
2857 ; | File Size (2 words) |
2858 ; +-----+
2859 ; | Date of write |
2860 ; +-----+
2861 ; | Time of write |
2862 ; +-----+
2863 ; +-----+
2864 ; C A V E A T P R O G R A M M E R ;
2865 ; +-----+
2866 ; | 8 bytes reserved |
2867 ; +-----+
2868 ;
2869 ; C A V E A T P R O G R A M M E R ;
2870 ; +-----+
2871 ; | next record number |
2872 ; +-----+
2873 ; | random record number |
2874 ; +-----+
2875 ;
2876 ;
2877 ;
2878 struct SYS_FCB
2879 .drive: resb 1
2880 .name: resb 8
2881 .ext: resb 3
2882 .EXTENT: resw 1
2883 .RECSIZ: resw 1 ; Size of record (user settable)
2884 .FILSIZ: resw 1 ; Size of file in bytes; used with the
2885 ; following word
2886 .DRVBP: resw 1 ; BP for SEARCH FIRST and SEARCH NEXT
2887 .FDATE: resw 1 ; Date of last writing
2888 .FTIME: resw 1 ; Time of last writing
2889 ; +-----+
2890 ; C A V E A T P R O G R A M M E R ;
2891 ; +-----+
2892 .reserved: resb 8 ; RESERVED
2893 ;
2894 ; C A V E A T P R O G R A M M E R ;
2895 ; +-----+
2896 .NR: resb 1 ; Next record
2897 .RR: resb 4 ; Random record
2898 .size:
2899 endstruc
2900
2901 FILDIRENT EQU SYS_FCB.FILSIZ ; Used only by SEARCH FIRST and SEARCH
2902 ; NEXT
2903 ; 20/07/2018
2904 %define fcb_sfn SYS_FCB.reserved ; byte
2905
2906 ; Note that fcb_net_handle, fcb_nsl_drive, fcb_nsl_drive and fcb_l_drive
2907 ; all must point to the same byte. Otherwise, the FCBRegen will fail.
2908 ; NOTE about this byte (fcb_nsl_drive)
2909 ; The high two bits of this byte are used as follows to indicate the FCB type
2910 ; 00 means a local file or device with sharing loaded
2911 ; 10 means a remote (network) file
2912 ; 01 means a local file with no sharing loaded
2913 ; 11 means a local device with no sharing loaded
2914
2915 ; 20/07/2018
2916
2917 ;
2918 ; Network FCB
2919 ;
2920
2921 %define fcb_net_drive SYS_FCB.reserved+1 ; byte
2922 %define fcb_net_handle SYS_FCB.reserved+2 ; word
2923 %define fcb_netID SYS_FCB.reserved+4 ; dword
2924
2925 ;
2926 ; No sharing local file FCB
2927 ;
2928
2929 %define fcb_nsl_drive SYS_FCB.reserved+1 ; byte
2930 %define fcb_nsl_bits SYS_FCB.reserved+2 ; byte
2931 %define fcb_nsl_firclus SYS_FCB.reserved+3 ; word
2932 %define fcb_nsl_dirsec SYS_FCB.reserved+5 ; word
2933 %define fcb_nsl_dirpos SYS_FCB.reserved+7 ; byte
2934
2935 ;
2936 ; No sharing local device FCB
2937 ;
2938
2939 %define fcb_nsl_drive SYS_FCB.reserved+1 ; byte
2940 %define fcb_nsl_drvptr SYS_FCB.reserved+2 ; dword
2941
2942 ;
2943 ; Sharing local FCB
2944 ;
2945
2946 %define fcb_l_drive SYS_FCB.reserved+1 ; byte
2947 %define fcb_l_firclus SYS_FCB.reserved+2 ; word
2948 %define fcb_l_mfs SYS_FCB.reserved+4 ; word
2949 %define fcb_l_attr SYS_FCB.reserved+6 ; byte
2950
2951 ;
2952 ; Bogusness: the four cases are:
2953 ;
2954 ; local file 00
2955 ; local device 40
2956 ; local sharing C0
2957 ; network 80
2958 ;
2959 ; Since sharing and network collide, we cannot use a test instruction for
2960 ; deciding whether a network or a share check is involved
2961 ;
2962 FCBDEVICE EQU 040h
2963 FCBNETWORK EQU 080h
2964 FCBSHARE EQU 0C0h
2965
2966 ; FCBSPCIAL must be able to mask off both net and share
2967 FCBSPCIAL EQU 080h
2968 FCBMASK EQU 0C0h
2969
2970 ;=====
2971 ; FASTOPEN.INC, MSDOS 6.0, 1991
2972 ;=====
2973 ; 11/07/2018 - Retro DOS v3.0
2974 ; 25/04/2019 - Retro DOS v4.0
2975 ; 21/02/2024 - Retro DOS v5.0
2976

```



```

2977      struc FEI      ; FASTOPEN_EXTENDED_INFO
2978      .dirpos:       resb 1
2979      .dirsec:       resd 1 ; MSDOS 6.0
2980      ;.dirsec:       resw 1 ; MSDOS 3.3
2981      .clusnum:      resw 1
2982      .resw 1 ; PCDOS 7.1 ; 21/02/2024
2983      .lastent:      resw 1 ; for search first ; MSDOS 6.0
2984      .firststart:   resw 1 ; for search first ; MSDOS 6.0
2985      .size:
2986      endstruc
2987
2988      ; 23/07/2018
2989      ;FASTOPEN NAME CACHING Subfunctions
2990      FONC_Look_up     equ 1
2991      FONC_insert equ 2
2992      FONC_delete equ 3
2993      FONC_update equ 4
2994      FONC_purge equ 5 ;reserved for the future use.
2995      FONC_Rename equ 6 ;AN001
2996
2997      ; 27/07/2018
2998      ;FastOpen Data Structure
2999      struc fastopen_entry ;Fastopen Entry pointer in DOS
3000      .entry_size:       resw 1 ; = 4 ; size of the following
3001      .name_caching:     resd 1
3002      ; MSDOS 6.0
3003      ;.fatchain_caching: resd 1;reserved for future use
3004      .size:
3005      endstruc
3006
3007      ; 27/07/2018
3008      ;Equates used in DOS.
3009      FastOpen_Set      equ 00000001b
3010      FastOpen_Reset    equ 11111110b
3011      Lookup_Success    equ 00000010b
3012      Lookup_Reset      equ 11111101b
3013      Special_Fill_Set   equ 00000100b
3014      Special_Fill_Reset equ 11111011b
3015      No_Lookup         equ 00001000b
3016      Set_For_Search    equ 00010000b ;DCR 167
3017
3018      ; 09/08/2018
3019      ; (FASTXXX.INC, MSDOS 6.0, 1991)
3020      ; Fastxxx equates
3021      FastOpen_ID      equ 1
3022      FastSeek_ID      equ 2
3023      Fast_yes         equ 10000000b ; 80h ; fastxxx flag
3024
3025      ;Structure definitions
3026      ;
3027      struc Fasttable_Entry ; Fastxxx Entry pointer in DOS
3028      .Fast_Entry_Num: resw 1 ; number of entries
3029      .FastOpen_Seek:   resd 1 ; fastopen & fastseek entry address
3030      endstruc
3031
3032      ;=====
3033      ; LOCK.INC, MSDOS 6.0, 1991
3034      ;=====
3035      ; 14/07/2018 - Retro DOS v3.0
3036
3037      ;** LOCK.INC - Definitions for Record Locking
3038
3039      ;** LOCK functions
3040
3041      LOCK_ALL          equ 0
3042      UNLOCK_ALL        equ 1
3043      LOCK_MUL_RANGE    equ 2
3044      UNLOCK_MUL_RANGE  equ 3
3045      LOCK_READ          equ 4
3046      WRITE_UNLOCK      equ 5
3047      LOCK_ADD          equ 6
3048
3049      ;** Structure for Lock buffer
3050
3051      struc LockBuf
3052      .Lock_position:   resd 1 ; file position for LOCK
3053      .Lock_length:     resd 1 ; number of bytes to LOCK
3054      endstruc
3055
3056      ;=====
3057      ; DPL.ASM, MSDOS 6.0, 1991
3058      ;=====
3059      ; 04/08/2018 - Retro DOS v3.0
3060
3061      ; (SRVCALL.ASM)
3062
3063      struc DPL
3064      .AX: resw 1 ; AX register
3065      .BX: resw 1 ; BX register
3066      .CX: resw 1 ; CX register
3067      .DX: resw 1 ; DX register
3068      .SI: resw 1 ; SI register
3069      .DI: resw 1 ; DI register
3070      .DS: resw 1 ; DS register
3071      .ES: resw 1 ; ES register
3072      .rsrvd: resw 1 ; Reserved
3073      .UID: resw 1 ; User (Machine) ID (0 = local machine)
3074      .PID: resw 1 ; Process ID (0 = local user PID)
3075      .size:
3076      endstruc
3077
3078      ;-----
3079      ; DOSDATA
3080      ;-----
3081      ;=====
3082      ; 24/04/2019 - Retro DOS v4.0
3083
3084      DosDataSg equ 3 ; DOS Data Segment address (dw in 'retrodos4.s')
3085      ; ((just after resident IO.SYS code&data))
3086
3087      ;=====
3088      ; WIN386.INC, MSDOS 6.0, 1991
3089      ;=====
3090      ; 24/04/2019 - Retro DOS 4.0
3091
3092      ;
3093      ; Symbols and structures relating to WIN386 support.
3094      ;
3095      ; Used by files in both the DOS and the BIOS.
3096      ;
3097      ; Created: 7-13-89 by MRW
3098      ;
3099      ;
3100      ; WIN386 broadcast int 2fh multiplex number and subfunction numbers

```

```

3101      Multwin386      equ      16h      ; Int 2f multiplex number
3102
3103      win386_Init      equ      05h      ; win386 initialization
3104      win386_Exit      equ      06h      ; win386 exit
3105      win386_Devcall   equ      07h      ; win386 device call out
3106      win386_InitDone  equ      08h      ; win386 initialization is complete
3107
3108      ; When win386_Devcall is broadcast, BX is the Device ID. DOS must
3109      ; answer call outs from the DOSMGR
3110
3111      win386_DOSMGR     equ      15H
3112
3113      ; The following structures are used to communicate instance data to
3114      ; win386 from the DOS and the BIOS. See win386 API documentation
3115      ; (chapter 3, "Call Out Interfaces") for further description.
3116
3117      struc win386_SIS   ; Startup Info Structure
3118      .Version:         resb      2      ; db 3, 0
3119      .Next_Dev_Ptr:     resd      1      ; pointer to next SIS in list
3120      .Virt_Dev_File_Ptr: resd      1
3121      .Reference_Data:   resd      1
3122      .Instance_Data_Ptr: resd      1      ; pointer to instance data array
3123      endstruc
3124
3125      size_of_win386_SIS equ 18 ; 24/04/2019 - Retro DOS v4.0
3126
3127      struc win386_IIS   ; Instance Item Structure
3128      .Ptr:              resd      1      ; pointer to an instance item
3129      .Size:             resw      1      ; size of an instance item
3130      endstruc
3131
3132      size_of_win386_IIS equ 6 ; 24/04/2019 - Retro DOS v4.0
3133
3134      ;win386 DOSMGR function return values to indicate operation done
3135
3136      WIN_OP_DONE      equ      0B97Ch ;
3137      DOSMGR_OP_DONE   equ      0A2ABh ;
3138
3139      ;M021
3140      ; WINoldap callout multiplex number
3141
3142      WINOLDAP         equ      46h      ;
3143
3144      ;=====
3145      ;-----
3146      ; DOSCODE
3147      ;-----
3148      ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
3149
3150      ;=====
3151      ; MSHEAD.ASM (MSDOS 6.0, 1991)
3152      ;=====
3153      ; 16/07/2018 - Retro DOS 3.0
3154      ;-----
3155      ; 24/04/2019 - Retro DOS 4.0
3156
3157      ; MSDOS 6.0
3158      ;-----
3159      ; FILE : ORIGIN.INC
3160      ;-----
3161      ; This is included in origin.asm and mshead.asm. Contains the equate that
3162      ; is used for ORGing the DOS code.
3163      ;
3164      ; Brief Description of the necessacity of this ORG:
3165      ; -----
3166      ;
3167      ; A special problem exists when running out of the HMA. The HMA starts at
3168      ; address FFFF:10. There is no place in the HMA with an offset of zero.
3169      ; This means programs running out off the HMA must use non-zero offset base
3170      ; addresses. It also means that if we're running multiple programs from the
3171      ; HMA, the base offset of each segment must atleast be as big as all of the
3172      ; HMA segments that precede it.
3173      ;
3174      ; One solution to this problem to ORG each module at 64K minus its size.
3175      ; For instance a code segment 1234h bytes in length would org'd at edcbh.
3176      ; This gives max. flexibility regarding it's location in the HMA. By
3177      ; selecting segment values between f124h and ffffh it could be located
3178      ; anywhere in the HMA. The problem with this is that programs with such
3179      ; high ORGs would not be able to run in low RAM.
3180      ;
3181      ; A compromise solution is to set the ORG address somewhere between 0010h
3182      ; and ffffh - their size. In the particular case of the BIOS and the DOS
3183      ; the following solution has been implemented:
3184      ;
3185      ; The Bios Code segment will have a very small offset and run at the very
3186      ; front of the HMA, after the VDISK header. THE Dos Code segment will have
3187      ; a base offset of (700+<min. size off RAM based BIOS>+<min. size of the DOS
3188      ; DATA segment when DOS is running low>). This will reflect the lowest
3189      ; possible physical address at which DOS code will run, while still providing
3190      ; max. possible flexibility in HMA positioning. This offset MUST NOT be
3191      ; smaller then that 20+size of Bios Code segment when running high. This is
3192      ; mostly true.
3193      ;
3194      ; Also this ORG'd value must be communicated to the BIOS. This is done by
3195      ; putting this value after the first jmp instruction in the DOS code in
3196      ; mshead.asm.
3197      ;
3198      ; In order for the stripz utility to know how many zeroes to be stripped
3199      ; out, this value is placed at the beginning of the binary in origin.asm.
3200      ;
3201      ; Revision History:
3202      ;
3203      ; Currently this is being done manually. Therefore any change in the DOS DATA
3204      ; Size or the BIOS size should be reflected here. --- Feb 90
3205      ;
3206      ; BDSIZE.INC contains the equates for BIODATASIZE, BIOCODESIZ and DOSDATASIZ.
3207      ; A utility called getsize will obtain the corresponding values from msdos
3208      ; and msbio.map and update the values in BDSIZ.INC if they are different.
3209      ; DOS should now be built using the batch file makedos.bat which invokes this
3210      ; utility. The FORMAT of BDSIZE.INC should not be changed as getsize is
3211      ; dependant on that. --- Apr 3 '90
3212      ;
3213      ; For ROMDOS, however, there is no need to org the doscode to any location
3214      ; other than zero. Therefore the stripz utility will not need to be used,
3215      ; so the offset will not need to be included at the beginning of the code
3216      ; segment. Also, the BIOS can just assume that the resident code begins
3217      ; at offset zero within the segment.
3218      ;
3219      ;
3220      ;-----
3221      ;
3222      BIODATASTART      EQU      00700h
3223      ;include bdsiz.inc ; this sets the values:
3224

```

```

3225 ; BIODATASIZ
3226 ; BIOCODESIZ
3227 ; DOSDATASIZ
3228
3229 ; 05/12/2022
3230 ;BIODATASIZ EQU 00910H ; 0900h for MSDOS 6.21 IO.SYS
3231 ; ; 0900h for MSDOS 5.0 IO.SYS
3232 ;BIOCODESIZ EQU 01A70H ; 1A70h for MSDOS 6.21 IO.SYS
3233 ; ; 1A60h for MSDOS 5.0 IO.SYS
3234 ;DOSDATASIZ EQU 01370H ; 1370h for MSDOS 6.21 IO.SYS
3235 ; ; 1370h for MSDOS 5.0 IO.SYS
3236
3237 ;ifndef ROMDOS
3238 ;BYTSTART EQU BIODATASTART+BIODATASIZ+BIOCODESIZ+DOSDATASIZ
3239 ;PARASTART EQU (BYTSTART + 0FH) AND (NOT 0FH)
3240 ;
3241 ;else
3242 ;
3243 ;BYTSTART EQU 0
3244 ;PARASTART EQU 0
3245 ;
3246 ;endif ; ROMDOS
3247
3248 ; 24/04/2019 - Retro DOS v4.0 - Modification
3249 ; -----
3250 ;MSDAT001E equ 136Ah ; 4970 ; for MSDOS 6.21
3251 ;MSDAT001E equ 1370h ; 4976 ; for Retro DOS v4.0 modif. 25/05/2019
3252 ;DOSDATASIZE equ MSDAT001E
3253 ; 05/12/2022
3254 ;DOSDATASIZE equ $ ; 29/04/2019 ; -only- for RETRO DOS v4.0 :
3255 ;_PARASTART_ equ DOSDATASIZE ; segment value will point to start of
3256 ; ; of DOSDATA (in low memory) while
3257 ; ; dos/kernel code starts just after
3258 ; ; this data block ((org = DOSDATASIZE))
3259 ; ; (in low memory or in HMA)
3260 ; -----
3261
3262 ; 04/11/2022
3263 ; -----
3264 ; NOTE:
3265 ; Microsoft dos programmers were calling 'IO.SYS' as dos 'BIOS'
3266 ; (Also, they were calling 'ROMBIOS' as 'ROM' only!)
3267 ; -----
3268
3269 ; -----
3270 ; 06/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
3271 ; -----
3272 ; -----
3273 ; 01/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
3274 ; -----
3275 ; -----
3276 ; -----
3277 ; Start of (PCDOS 7.1) IBMDOS.COM
3278 ; -----
3279
3280 ;segment .code vstart=3DD0h ; 06/12/2022
3281 ; 29/09/2023
3282 ;segment .code vstart=3DE0h ; 19/09/2023 - Retro DOS v4.2 (Modified MSDOS 6.22)
3283 ; -----
3284 ; 01/01/2024
3285 segment .code vstart=3F10h ; 01/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1)
3286
3287 ; =====
3288
3289 ;[ORG 3DE0h]
3290
3291 ;[ORG _PARASTART_] ; [org 136Ah]
3292
3293 ;[ORG 1370h] ; 25/05/2019 - Retro DOS v4.0
3294
3295 ; 01/01/2024 - Retro DOS v5.0
3296 ;[ORG 3F10h]
3297
3298 ; 05/12/2022 - RetroDOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
3299 ;PARASTART equ 3DD0h ; BIOSDATASTART+BIOSDATASIZE
3300 ; ; +BIOSCODESIZE+DOSDATASIZE (rounded up)
3301
3302 ; 29/09/2023
3303 ; 19/09/2023 - Retro DOS v4.2 (Modified MSDOS 6.22 MSDOS.SYS)
3304 ;PARASTART equ 3DE0h ; (MSDOS 6.22 MSDOS.SYS)
3305
3306 ; 01/01/2024 - Retro DOS v4.2 (Modified PCDOS 7.1 IBMDOS.COM)
3307 PARASTART equ 3F10h ; (PCDOS 7.1 IBMDOS.COM)
3308
3309 [ORG PARASTART]
3310
3311 _$STARTCODE:
3312
3313 ;PARASTART:
3314 JMP DOSINIT
3315
3316 ;dw PARASTART ; PARASTART = 3DE0h for MSDOS 6.0, 6.22
3317 ; 04/11/2022 ; PARASTART = 3DD0h for MSDOS 5.0
3318 dw _$STARTCODE ; PARASTART = 3F10h for PCDOS 7.1 ; 01/01/2024
3319
3320 BioDataSeg:
3321 dw 0070h ; Bios data segment fixed at 70h
3322
3323 ; DosDSeg is a data word in the DOSCODE segment that is loaded with
3324 ; the segment address of DOSDATA. This is purely an optimization, that
3325 ; allows getting the DOS data segment without going through the
3326 ; BIOS data segment. It is used by the "getdseg" macro.
3327
3328 DosDSeg:
3329 dw 0
3330
3331 ; =====
3332 ; Mstable.ASM (MSDOS 6.0, 1991)
3333 ; =====
3334 ; 16/07/2018 - Retro DOS 3.0
3335 ; 29/04/2019 - Retro DOS 4.0
3336
3337 ; (MSDOS version)
3338 ; DOSCODE:3DE9h (MSDOS 6.21, MSDOS.SYS)
3339 ;db 6
3340 ;db 20
3341 ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
3342 ; DOSCODE:3DD9h (MSDOS 5.0, MSDOS.SYS)
3343 ;db 5
3344 ;db 0
3345
3346 ; Offset 0c78h in IBMDOS.COM (MSDOS 3.3, 1987)
3347 MSVERS: ; MS-DOS version in hex for $GET_VERSION
3348 DB MAJOR_VERSION ; DOS_MAJOR_VERSION

```

```

3349 0000000A 0A      MS Minor: DB Minor_Version ; DOS_Minor_Version
3350
3351 ;;hkn YRTAB & MONTAB moved to DOSDATA in ms_data.asm
3352 ;   I_am   YRTAB,8,<200,166,200,165,200,165,200,165> ; [SYSTEM]
3353 ;   I_am   MONTAB,12,<31,28,31,30,31,30,31,31,30,31,30,31> ; [SYSTEM]
3354
3355 ; DOSTAB.ASM (MSDOS 6.0, 1991)
3356 ; YRTAB & MONTAB moved from TABLE segment in ms_table.asm
3357
3358 ;   I_am   YRTAB,8,<200,166,200,165,200,165,200,165>
3359 ;   I_am   MONTAB,12,<31,28,31,30,31,30,31,31,30,31,30,31>
3360
3361 ; This is the error code mapping table for INT 21 errors. This table defines
3362 ; those error codes which are "allowed" for each system call. If the error
3363 ; code ABOUT to be returned is not "allowed" for the call, the correct action
3364 ; is to return the "real" error via Extended error, and one of the allowed
3365 ; errors on the actual call.
3366
3367 ; The table is organized as follows:
3368
3369 ;     Each entry in the table is of variable size, but the first
3370 ;     two bytes are always:
3371
3372 ;         call#,Cnt of bytes following this byte
3373
3374 ; EXAMPLE:
3375 ;     Call 61 (OPEN)
3376
3377 ;         DB      61,5,12,3,2,4,5
3378
3379 ;         61 is the AH INT 21 call value for OPEN.
3380 ;         5 indicates that there are 5 bytes after this byte (12,3,2,4,5).
3381 ;         Next five bytes are those error codes which are "allowed" on OPEN.
3382 ;         The order of these values is not important EXCEPT FOR THE LAST ONE (in
3383 ;         this case 5). The last value will be the one returned on the call if
3384 ;         the "real" error is not one of the allowed ones.
3385
3386 ; There are a number of calls (for instance all of the FCB calls) for which
3387 ; there is NO entry. This means that NO error codes are returned on this
3388 ; call, so set up an Extended error and leave the current error code alone.
3389
3390 ; The table is terminated by a call value of 0FFh
3391
3392 ;PUBLIC I21_MAP_E_TAB
3393 ;     ; 10/08/2018
3394
3395 ;     29/04/2019
3396 ;     DOSCODE:3DE9h (MSDOS 6.21, MSDOS.SYS)
3397 ;     04/11/2022
3398 ;     DOSCODE:3DDBh (MSDOS 5.0 MSDOS.SYS)
3399 ;     01/01/2024
3400 ;     DOSCODE:3F1Bh (PCDOS 7.1 IBMDOS.COM)
3401
3402 I21_MAP_E_TAB:      ; LABEL BYTE
3403 0000000B 38020102  DB  INTERNATIONAL,2,error_invalid_function,error_file_not_found
3404 0000000F 3903030205 DB  MKDIR,3,error_path_not_found,error_file_not_found,error_access_denied
3405 00000014 3A041003  DB  RMDIR,4,error_current_directory,error_path_not_found
3406 00000018 0205      DB  error_file_not_found,error_access_denied
3407 0000001A 38020203  DB  CHDIR,2,error_file_not_found,error_path_not_found
3408 0000001E 3C040302  DB  CREAT,4,error_path_not_found,error_file_not_found
3409 00000022 04      DB  error_too_many_open_files
3410 00000023 05      DB  error_access_denied
3411
3412 00000024 3D0603020C DB  ; MSDOS 6.0
3413 00000029 04      DB  OPEN,6,error_path_not_found,error_file_not_found,error_invalid_access
3414 0000002A 1A05      DB  error_too_many_open_files
3415      DB  error_not_DOS_disk,error_access_denied
3416      ; MSDOS 3.3
3417      DB  OPEN,5,error_path_not_found,error_file_not_found,error_invalid_access
3418 0000002C 3E0106  DB  error_too_many_open_files,error_access_denied
3419 0000002F 3F020605  DB  CLOSE,1,error_invalid_handle
3420 00000033 40020605  DB  READ,2,error_invalid_handle,error_access_denied
3421 00000037 4103030205 DB  WRITE,2,error_invalid_handle,error_access_denied
3422 0000003C 42020601  DB  UNLINK,3,error_path_not_found,error_file_not_found,error_access_denied
3423 00000040 4304030201 DB  LSEEK,2,error_invalid_handle,error_invalid_function
3424 00000045 05      DB  CHMOD,4,error_path_not_found,error_file_not_found,error_invalid_function
3425 00000046 44050F0D01 DB  error_access_denied
3426 0000004B 0605      DB  IOCTL,5,error_invalid_drive,error_invalid_data,error_invalid_function
3427 0000004D 45020604  DB  error_invalid_handle,error_access_denied
3428 00000051 46020604  DB  XDUP,2,error_invalid_handle,error_too_many_open_files
3429      DB  XDUP2,2,error_invalid_handle,error_too_many_open_files
3430 00000055 47021A0F  DB  ; MSDOS 6.0
3431      DB  CURRENT_DIR,2,error_not_DOS_disk,error_invalid_drive
3432      ; MSDOS 3.3
3433 00000059 48020708 DB  ;DB  CURRENT_DIR,1,error_invalid_drive
3434 0000005D 49020709 DB  ALLOC,2,error_arena_trashed,error_not_enough_memory
3435 00000061 4A03070908 DB  DEALLOC,2,error_arena_trashed,error_invalid_block
3436 00000066 4B08030102 DB  SETBLOCK,3,error_arena_trashed,error_invalid_block,error_not_enough_memory
3437 0000006B 040B0A  DB  EXEC,8,error_path_not_found,error_invalid_function,error_file_not_found
3438 0000006E 0805      DB  error_too_many_open_files,error_bad_format,error_bad_environment
3439 00000070 4E03030212 DB  error_not_enough_memory,error_access_denied
3440 00000075 4F0112  DB  FIND_FIRST,3,error_path_not_found,error_file_not_found,error_no_more_files
3441      DB  FIND_NEXT,1,error_no_more_files
3442 00000078 5605110302 DB  ; MSDOS 6.0
3443 0000007D 1005      DB  RENAME,5,error_not_same_device,error_path_not_found,error_file_not_found
3444      DB  error_current_directory,error_access_denied
3445      ; MSDOS 3.3
3446 0000007F 57040608 DB  ;DB  RENAME,4,error_not_same_device,error_path_not_found,error_file_not_found
3447      DB  error_access_denied
3448 00000083 0B01      ; MSDOS 6.0
3449 00000085 580101  DB  ;DB  FILE_TIMES,2,error_invalid_handle,error_invalid_function
3450 00000088 5A040302 DB  ALLOCOPER,1,error_invalid_function
3451 0000008C 0405      DB  CREATETEMPFILE,4,error_path_not_found,error_file_not_found
3452 0000008E 5B055003 DB  error_too_many_open_files,error_access_denied
3453 00000092 020405  DB  CREATENEF,5,error_file_exists,error_path_not_found
3454 00000095 5C040601 DB  error_file_not_found,error_too_many_open_files,error_access_denied
3455 00000099 2421      DB  LOCKOPER,4,error_invalid_handle,error_invalid_function
3456 0000009B 65020102 DB  error_sharing_buffer_exceeded,error_lock_violation
3457 0000009F 66020102 DB  GETTEXTCNTRY,2,error_invalid_function,error_file_not_found ;DOS 3.3
3458 000000A3 680106  DB  GETSETCDPG,2,error_invalid_function,error_file_not_found ;DOS 3.3
3459 000000A6 67030408 DB  COMMIT,1,error_invalid_handle ;DOS 3.3
3460 000000AA 01      DB  EXTHANDLE,3,error_too_many_open_files,error_not_enough_memory
3461      DB  error_invalid_function
3462 000000AB 6C0A  DB  ; MSDOS 6.0
3463 000000AD 03020C  DB  ExtOpen,10
3464 000000B0 045008  DB  error_path_not_found,error_file_not_found,error_invalid_access
3465 000000B3 1A0D      DB  error_too_many_open_files,error_file_exists,error_not_enough_memory
3466 000000B5 0105      DB  error_not_DOS_disk,error_invalid_data
3467 000000B7 69040F0D DB  error_invalid_function,error_access_denied
3468 000000BB 0105      DB  GetSetMediaID,4,error_invalid_drive,error_invalid_data
3469      DB  error_invalid_function,error_access_denied
3470      ; 01/01/2024
3471
3472

```

[illegible]

|      |                 |  |                                                   |
|------|-----------------|--|---------------------------------------------------|
|      |                 |  | ; short_addr _\$CREATE_PROCESS_DATA_BLOCK ; 38 26 |
| 3595 | 00000138 [4F16] |  |                                                   |
| 3596 |                 |  |                                                   |
| 3597 |                 |  | C A V E A T P R O G R A M M E R                   |
| 3598 |                 |  |                                                   |
| 3599 | 0000013A [6822] |  | short_addr _\$FCB_RANDOM_READ_BLOCK ; 39 27       |
| 3600 | 0000013C [6422] |  | short_addr _\$FCB_RANDOM_WRITE_BLOCK ; 40 28      |
| 3601 | 0000013E [D212] |  | short_addr _\$PARSE_FILE_DESCRIPTOR ; 41 29       |
| 3602 | 00000140 [D80A] |  | short_addr _\$GET_DATE ; 42 2A                    |
| 3603 | 00000142 [F50A] |  | short_addr _\$SET_DATE ; 43 2B                    |
| 3604 | 00000144 [140B] |  | short_addr _\$GET_TIME ; 44 2C                    |
| 3605 | 00000146 [250B] |  | short_addr _\$SET_TIME ; 45 2D                    |
| 3606 | 00000148 [C30C] |  | short_addr _\$SET_VERIFY_ON_WRITE ; 46 2E         |
| 3607 |                 |  |                                                   |
| 3608 |                 |  | ; Extended functionality group                    |
| 3609 | 0000014A [340F] |  | short_addr _\$GET_DMA ; 47 2F                     |
| 3610 | 0000014C [9C0C] |  | short_addr _\$GET_VERSION ; 48 30                 |
| 3611 | 0000014E [1572] |  | short_addr _\$KEEP_PROCESS ; 49 31                |
| 3612 |                 |  | C A V E A T P R O G R A M M E R                   |
| 3613 |                 |  |                                                   |
| 3614 |                 |  |                                                   |
| 3615 | 00000150 [4413] |  | short_addr _\$GET_DPB ; 50 32                     |
| 3616 |                 |  |                                                   |
| 3617 |                 |  | C A V E A T P R O G R A M M E R                   |
| 3618 |                 |  |                                                   |
| 3619 | 00000152 [3D02] |  | short_addr _\$SET_CTRL_C_TRAPPING ; 51 33         |
| 3620 | 00000154 [2C13] |  | short_addr _\$GET_INDOS_FLAG ; 52 34              |
| 3621 | 00000156 [690F] |  | short_addr _\$GET_INTERRUPT_VECTOR ; 53 35        |
| 3622 | 00000158 [030F] |  | short_addr _\$GET_DRIVE_FREESPACE ; 54 36         |
| 3623 | 0000015A [A50F] |  | short_addr _\$CHAR_OPER ; 55 37                   |
| 3624 | 0000015C [CA0C] |  | short_addr _\$INTERNATIONAL ; 56 38               |
| 3625 |                 |  |                                                   |
| 3626 |                 |  | ; XENIX CALLS                                     |
| 3627 | 0000015E [1C28] |  | ; Directory Group                                 |
| 3628 | 00000160 [3D27] |  | short_addr _\$MKDIR ; 57 39                       |
| 3629 | 00000162 [8C27] |  | short_addr _\$RMDIR ; 58 3A                       |
| 3630 |                 |  | short_addr _\$CHDIR ; 59 3B                       |
| 3631 | 00000164 [A180] |  | ; File Group                                      |
| 3632 | 00000166 [C97F] |  | short_addr _\$CREAT ; 60 3C                       |
| 3633 | 00000168 [6D77] |  | short_addr _\$OPEN ; 61 3D                        |
| 3634 | 0000016A [7478] |  | short_addr _\$CLOSE ; 62 3E                       |
| 3635 | 0000016C [D178] |  | short_addr _\$READ ; 63 3F                        |
| 3636 | 0000016E [1381] |  | short_addr _\$WRITE ; 64 40                       |
| 3637 | 00000170 [D678] |  | short_addr _\$UNLINK ; 65 41                      |
| 3638 | 00000172 [AE80] |  | short_addr _\$LSEEK ; 66 42                       |
| 3639 | 00000174 [8428] |  | short_addr _\$CHMOD ; 67 43                       |
| 3640 | 00000176 [3A79] |  | short_addr _\$IOCTL ; 68 44                       |
| 3641 | 00000178 [5879] |  | short_addr _\$DUP ; 69 45                         |
| 3642 | 0000017A [D926] |  | short_addr _\$DUP2 ; 70 46                        |
| 3643 |                 |  | short_addr _\$CURRENT_DIR ; 71 47                 |
| 3644 | 0000017C [0E73] |  | ; Memory Group                                    |
| 3645 | 0000017E [8374] |  | short_addr _\$ALLOC ; 72 48                       |
| 3646 | 00000180 [5F74] |  | short_addr _\$DEALLOC ; 73 49                     |
| 3647 |                 |  | short_addr _\$SETBLOCK ; 74 4A                    |
| 3648 | 00000182 [E86B] |  | ; Process Group                                   |
| 3649 | 00000184 [4D72] |  | short_addr _\$EXEC ; 75 4B                        |
| 3650 | 00000186 [DE6B] |  | short_addr _\$EXIT ; 76 4C                        |
| 3651 | 00000188 [2326] |  | short_addr _\$WAIT ; 77 4D                        |
| 3652 |                 |  | short_addr _\$FIND_FIRST ; 78 4E                  |
| 3653 | 0000018A [7726] |  | ; Special Group                                   |
| 3654 |                 |  | short_addr _\$FIND_NEXT ; 79 4F                   |
| 3655 |                 |  | ; SPECIAL SYSTEM GROUP                            |
| 3656 |                 |  | C A V E A T P R O G R A M M E R                   |
| 3657 |                 |  |                                                   |
| 3658 | 0000018C [A202] |  | short_addr _\$SET_CURRENT_PDB ; 80 50             |
| 3659 | 0000018E [AE02] |  | short_addr _\$GET_CURRENT_PDB ; 81 51             |
| 3660 | 00000190 [3813] |  | short_addr _\$GET_IN_VARS ; 82 52                 |
| 3661 | 00000192 [6514] |  | short_addr _\$SETDPB ; 83 53                      |
| 3662 |                 |  |                                                   |
| 3663 |                 |  | C A V E A T P R O G R A M M E R                   |
| 3664 |                 |  |                                                   |
| 3665 | 00000194 [BE0C] |  | short_addr _\$GET_VERIFY_ON_WRITE ; 84 54         |
| 3666 |                 |  | C A V E A T P R O G R A M M E R                   |
| 3667 |                 |  |                                                   |
| 3668 |                 |  |                                                   |
| 3669 | 00000196 [3E16] |  | short_addr _\$DUP_PDB ; 85 55                     |
| 3670 |                 |  |                                                   |
| 3671 |                 |  | C A V E A T P R O G R A M M E R                   |
| 3672 |                 |  |                                                   |
| 3673 | 00000198 [3981] |  | short_addr _\$RENAME ; 86 56                      |
| 3674 | 0000019A [6679] |  | short_addr _\$FILE_TIMES ; 87 57                  |
| 3675 | 0000019C [B874] |  | short_addr _\$ALLOCOPER ; 88 58                   |
| 3676 |                 |  |                                                   |
| 3677 |                 |  | ; 08/07/2018 - Retro DOS v3.0                     |
| 3678 |                 |  |                                                   |
| 3679 |                 |  | ; MSDOS 6.0 - MSTABLE.ASM, 1991                   |
| 3680 |                 |  |                                                   |
| 3681 |                 |  | ; Network extention system calls                  |
| 3682 | 0000019E [B90F] |  | short_addr _\$GetExtendedError ; 89 59            |
| 3683 | 000001A0 [C581] |  | short_addr _\$CreateTempFile ; 90 5A              |
| 3684 | 000001A2 [AD81] |  | short_addr _\$CreateNewFile ; 91 5B               |
| 3685 |                 |  |                                                   |



[illegible]

```

3842 ; -----
3843 ; BREAK <Copyright notice and version>
3844 ; -----
3845
3846 ;CODSTRT EQU $
3847
3848 ; 08/07/2018 - Retro DOS v3.0 by Erdogan Tan
3849 ; (MSTABLE.ASM, MSDOS 6.0, 1991)
3850
3851 ; NOTE WARNING: This declaration of HEADER must be THE LAST thing in this
3852 ; module. The reason is so that the data alignments are the same in
3853 ; IBM-DOS and MS-DOS up through header.
3854
3855 ;PUBLIC HEADER
3856
3857 HEADER: ; LABEL BYTE
3858 ;IF DEBUG
3859 ;DB 13,10,"Debugging DOS version "
3860 ;DB MAJOR_VERSION + "0"
3861 ;DB " "
3862 ;DB (MINOR_VERSION / 10) + "0"
3863 ;DB (MINOR_VERSION MOD 10) + "0"
3864 ;ENDIF
3865
3866 ; 05/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
3867 ; (MSDOS 5.0 MSDOS.SYS compatibility)
3868 %if 0
3869 ;IF NOT IBM
3870 DB 13,10,"MS-DOS version "
3871 DB MAJOR_VERSION + "0"
3872 DB " "
3873 DB (MINOR_VERSION / 10) + "0"
3874 DB (MINOR_VERSION MOD 10) + "0"
3875 DB (MINOR_VERSION % 10) + "0"
3876
3877 ;IF HIGHMEM
3878 DB "H"
3879 ;ENDIF
3880
3881 ;DB 13,10,"Copyright 1981,82,83,84,88 Microsoft Corp.",13,10,"$"
3882 ; 30/04/2019 - Retro DOS v4.0
3883 DB 13,10,"Copyright 1981-1993 Microsoft Corp.",13,10,"$"
3884
3885 ;ENDIF
3886
3887 %endif
3888
3889 ;IF DEBUG
3890 ; DB 13,10,"$"
3891 ;ENDIF
3892
3893 ;include copyrigh.inc
3894
3895 ; DOSCODE:3FDDh (MSDOS 6.21, MSDOS.SYS)
3896
3897 ;DB "MS DOS Version 6 (C)Copyright 1981-1993 Microsoft Corp "
3898 ;DB "Licensed Material - Property of Microsoft "
3899 ;DB "All rights reserved "
3900
3901 ; 05/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
3902 ; DOSCODE:3FCDh (MSDOS 5.0, MSDOS.SYS)
3903
3904 ; 28/12/2022 - Retro DOS v4.1
3905 %if 0
3906 ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
3907 ms_copyright:
3908 db 'MS DOS Version 5.00 (C)Copyright 1981-1991 Microsoft Corp '
3909 db 'Licensed Material - Property of Microsoft '
3910 db 'All rights reserved '
3911
3912 %endif
3913
3914 ;; 28/12/2022 - Retro DOS v4.1
3915 ;ms_copyright:
3916 ;db 13,10,"MS DOS Version 5.0"
3917 ;db 13,10,"Copyright 1981-1991 Microsoft Corp.",13,10,"$",0
3918
3919 ; ; 21/09/2023 - Retro DOS v4.2 MSDOS.SYS
3920 ; ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:3FDDh (File offset: 509))
3921 ;ms_copyright:
3922 ; db 'MS DOS Version 6 (C)Copyright 1981-1994 Microsoft Corp '
3923 ; db 'Licensed Material - Property of Microsoft All rights reserved '
3924
3925 ; 01/01/2024 - Retro DOS v5.0
3926
3927 ; 20/09/2023 - Retro DOS v4.2
3928 ;ms_copyright:
3929 ;db 13,10,"MS DOS Version 6.22"
3930 ;db 13,10,"Copyright 1981-1994 Microsoft Corp.",13,10,"$",0
3931
3932 ;=====
3933 ; MSCODE.ASM
3934 ;=====
3935
3936 ; Retro DOS v2.0 (NASM 2.11) source code modifications by Erdogan Tan
3937 ; 03/03/2018
3938
3939 ;
3940 ; MSCODE.ASM -- MSDOS code
3941 ;
3942
3943 ;INCLUDE DOSSEG.ASM
3944 ;INCLUDE STDSW.ASM
3945
3946 ;CODE SEGMENT BYTE PUBLIC 'CODE'
3947 ;ASSUME CS:DOSGROUP,DS:NOTHING,ES:NOTHING,SS:NOTHING
3948
3949 ;.xcref
3950 ;INCLUDE DOSSYM.ASM
3951 ;INCLUDE DEVSYM.ASM
3952 ;.cref
3953 ;.list
3954
3955 ;IFNDEF KANJI
3956 ;KANJI EQU 0 ; FALSE
3957 ;ENDIF
3958
3959 ;IFNDEF IBM
3960 ;IBM EQU 0
3961 ;ENDIF
3962
3963 ;IFNDEF HIGHMEM
3964 ;HIGHMEM EQU 0
3965 ;ENDIF

```



```

3966
3967 ;i_need USER_SP,WORD
3968 ;i_need USER_SS,WORD
3969 ;i_need SAVEDS,WORD
3970 ;i_need SAVEBX,WORD
3971 ;i_need INDOS, BYTE
3972 ;i_need NSP,WORD
3973 ;i_need NSS,WORD
3974 ;i_need CURRENTPDB,WORD
3975 ;i_need AUXSTACK, BYTE
3976 ;i_need CONSWAP, BYTE
3977 ;i_need IDLEINT, BYTE
3978 ;i_need NOSETDIR, BYTE
3979 ;i_need ERRORMODE, BYTE
3980 ;i_need IOSTACK, BYTE
3981 ;i_need WPERR, BYTE
3982 ;i_need DSKSTACK, BYTE
3983 ;i_need CNTCFLAG, BYTE
3984 ;i_need LEAVEADDR,WORD
3985 ;i_need NULLDEVPT,DWORD
3986
3987 ;IF NOT IBM
3988 ;i_need OEM_HANDLER,DWORD
3989 ;ENDIF
3990
3991 ;EXTRN DSKSTATCHK:NEAR,GETBP:NEAR,DSKREAD:NEAR,DSKWRITE:NEAR
3992
3993 ;=====
3994 ; MSDISP.ASM, MSDOS 6.0, 1991
3995 ;=====
3996 ; 11/07/2018 - Retro DOS v3.0
3997 ; 01/05/2019 - Retro DOS v4.0
3998
3999 ; DosCode SEGMENT
4000
4001 ; =====
4002
4003 ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
4004 ; DOSCODE:4045h (MSDOS 5.0, MSDOS.SYS)
4005
4006 ; 01/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
4007 ; DOSCODE:4123h (PCDOS 7.1, IBMDOS.COM)
4008
4009 _$SET_CTRL_C_TRAPPING:
4010 ; 01/05/2019 - Retro DOS v4.0
4011
4012 0000023D 3C07 cmp al,7 ; 01/01/2024 - Retro DOS v5.0
4013 ;cmp AL,6 ; Is this a valid subfunction?
4014 0000023F 7603 jbe short scct_1 ; If yes continue processing
4015
4016 00000241 B0FF mov AL,0FFh ; Else set AL to -1 and
4017 00000243 CF iret
4018
4019 00000244 1E scct_1: push DS
4020
4021 ;getdseg <DS> ; DS -> DosData, ASSUME DS:DosSeg
4022 00000245 2E8E1E[0700] mov ds,[cs:DosDSeg]
4023
4024 0000024A 50 push AX ; DL only register that can change
4025 0000024B 56 push SI
4026
4027 0000024C BE[3703] mov SI,CNTCFLAG ; DS:SI --> Ctrl C Status byte
4028 0000024F 30E4 xor AH,AH ; Clear high byte of AX
4029 00000251 09C0 or AX,AX ; Check for subfunction 0
4030 00000253 7504 jnz short scct_2 ; If not 0 jmp to next check
4031
4032 00000255 8A14 mov DL,[SI] ; Else move current ctrl C status
4033 00000257 EB32 jmp SHORT scct_9s ; into DL and jmp to exit
4034
4035 00000259 48 scct_2: dec AX ; Now dec AX and see if it was 1
4036 0000025A 7507 jnz short scct_3 ; If not 0 it wasn't 1 so do next chk
4037
4038 0000025C 80E201 and DL,1 ; Else mask off bit 0 of DL and
4039 0000025F 8814 mov [SI],DL ; save it as new Ctrl C status
4040 00000261 EB28 jmp SHORT scct_9s ; Jmp to exit
4041
4042 00000263 48 scct_3: dec AX ; Dec AX again to see if it was 2
4043 00000264 7507 jnz short scct_4 ; If not 0 wasn't 2 so go to next chk
4044
4045 00000266 80E201 and DL,1 ; Else mask off bit 0 of DL and
4046 00000269 8614 xchg [SI],DL ; Exchange DL with old status byte
4047 0000026B EB1E jmp SHORT scct_9s ; Jump to exit (returning old status)
4048
4049 0000026D 3C03 scct_4: cmp al,3 ; 01/01/2024
4050 ;cmp AX,3 ; Test for 5 after it was dec twice
4051 0000026F 7506 jne short scct_5 ; If not equal then not get boot drv
4052 00000271 8A16[6900] mov DL,[BOOTDRIVE] ; Else return boot drive in DL
4053 00000275 EB14 jmp SHORT scct_9s ; Jump to exit (returning boot drive)
4054
4055 ; 01/01/2024 - Retro DOS v5.0
4056 00000277 7212 jb short scct_9s ; PC DOS 7.1
4057 ;
4058 00000279 3C04 cmp al,4 ; 01/01/2024
4059 ;cmp AX,4 ; Test for 6 after it was dec twice
4060 ;jne short scct_9s ; If not equal then not get version
4061 0000027B 7512 jne short scct_6 ; 01/01/2024 ; PC DOS 7.1
4062
4063 ;mov BX,(Minor_Version SHL 8) + Major_Version
4064
4065 ;;mov bx,1406h ; 6.20 ; DOSCODE:4092h (MSDOS 6.21, MSDOS.SYS)
4066 ;;mov bx,1606h ; 6.22 ; DOSCODE:4092h (MSDOS 6.22, MSDOS.SYS)
4067
4068 ;mov bx,0A07h ; 7.10 ; DOSCODE:4163h (PCDOS 7.1, IMB DOS.COM)
4069 0000027D BB070A mov bx,(MINOR_VERSION<<8)+MAJOR_VERSION ; true dos version
4070
4071 ;mov dl,0 ; revision 0
4072 ;mov DL,DOSREVN ; 0
4073
4074 ;xor dh,dh ; assume vanilla DOS
4075 ; 01/01/2024
4076 00000280 BA0000 mov dx,0
4077 00000283 3836[660D] cmp byte [DosHashMA],dh ; 0
4078 ;cmp byte [DosHashMA],0 ; is DOS in HMA? (M021)
4079 ;;je short @F
4080 ;je short scct_6
4081 00000287 7402 je short scct_9s ; 01/01/2024
4082 ; 01/01/2024
4083 00000289 B610 mov dh,10h ; version flags bit 4
4084 ;or DH,DOSINHMA ; 10h ; 'DOS in HMA' status
4085 ;@@:
4086 ;scct_6:
4087 ;ifdef ROMDOS
4088 ; or DH,DOSINROM ; 08h
4089 ;endif ; ROMDOS

```

```

4090
4091 ; 01/01/2024 (PCDOS 7.1)
4092 ; jmp short short scct_9s
4093
4094 ; 01/01/2024
4095 ; scct_6:
4096 ; ....
4097
4098 scct_9s:
4099 pop SI
4100 pop AX
4101 pop DS
4102 scct_9f:
4103 iret
4104
4105 ; 01/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
4106 scct_6:
4107 and byte [DOS_FLAG],0DFh ; clear bit 5 of DOS flag
4108 cmp dl,1
4109 jne short scct_9s
4110 or byte [DOS_FLAG],20h ; set bit 5 of DOS flag
4111 jmp short scct_9s
4112
4113 SetCtrlShortEntry: ; This allows a conditional entry
4114 ; from main dispatch code
4115 jmp SHORT _$SET_CTRL_C_TRAPPING
4116
4117 ; =====
4118 ;
4119 ; The following two routines are dispatched to directly with ints disabled
4120 ; immediately after the int 21h entry. no DIS state is set.
4121 ;
4122 ; $Set_current_PDB takes BX and sets it to be the current process
4123 ; *** THIS FUNCTION CALL IS SUBJECT TO CHANGE!!! ***
4124 ;
4125 ; =====
4126
4127 _$SET_CURRENT_PDB:
4128 push DS
4129 ; getdseg <DS> ; DS -> DosData, ASSUME DS:DosSeg
4130 mov ds,[cs:DosDSeg]
4131 mov [CurrentPDB],BX ; Set new PSP segment from caller's BX
4132 pop DS
4133 iret
4134
4135 ; =====
4136 ;
4137 ; $get_current_PDB returns in BX the current process
4138 ; *** THIS FUNCTION CALL IS SUBJECT TO CHANGE!!! ***
4139 ;
4140 ; =====
4141
4142 _$GET_CURRENT_PDB:
4143 push DS
4144 ; getdseg <DS> ; DS -> DosData, ASSUME DS:DosSeg
4145 mov ds,[cs:DosDSeg]
4146 mov BX,[CurrentPDB] ; Return current PSP segment in BX
4147 pop DS
4148 iret
4149
4150 ; =====
4151 ;
4152 ; Sets the Printer Flag to whatever is in AL.
4153 ; NOTE: THIS PROCEDURE IS SUBJECT TO CHANGE!!!
4154 ;
4155 ; =====
4156
4157 _$SET_PRINTER_FLAG:
4158 push ds
4159 ; getdseg <DS> ; DS -> DosData, ASSUME DS:DosSeg
4160 mov ds,[cs:DosDSeg]
4161 mov [PRINTER_FLAG],AL ; set printer flag from caller's AL
4162 pop ds
4163 iret
4164
4165 ; 01/05/2019 - Retro DOS v4.0
4166 ; 08/07/2018 - Retro DOS v3.0
4167 ; (MSDISP.ASM, MSDOS 6.0, 1991)
4168
4169 ; -----
4170 ; BREAK <System call entry points and dispatcher>
4171 ; -----
4172
4173 ; DOSCODE:40CCh (MSDOS 6.21, MSDOS.SYS)
4174
4175 ; =====
4176 ;
4177 ; The Quit entry point is where all INT 20h's come from. These are old- style
4178 ; exit system calls. The CS of the caller indicates which Process is dying.
4179 ; The error code is presumed to be 0. We simulate an ABORT system call.
4180 ;
4181 ; =====
4182
4183 SYSTEM_CALL: ; PROC NEAR
4184
4185 ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
4186 ; DOSCODE:40BFh (MSDOS 5.0, MSDOS.SYS)
4187
4188 ; entry QUIT
4189 QUIT: ; INT 20H entry point
4190 ; MOV AH,0
4191 xor ah,ah ; 08/07/2018
4192 jmp SHORT SAVREGS
4193
4194 ; -----
4195 ;
4196 ; The system call in AH is out of the range that we know how
4197 ; to handle. We arbitrarily set the contents of AL to 0 and
4198 ; IRET. Note that we CANNOT set the carry flag to indicate an
4199 ; error as this may break some programs compatability.
4200
4201 BADCALL:
4202 ; MOV AL,0
4203 xor al,al ; 08/07/2018
4204 IRET: ; 06/05/2019
4205 _IRET:
4206 iret
4207
4208 ; -----
4209 ;
4210 ; 01/05/2019 - Retro DOS v4.0
4211 ; DOSCODE:40D3h (MSDOS 6.21 MSDOS.SYS)
4212 ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
4213 ; DOSCODE:40C6h (MSDOS 5.0 MSDOS.SYS)

```

```

4214
4215 ; An alternative method of entering the system is to perform a
4216 ; CALL 5 in the program segment prefix with the contents of CL
4217 ; indicating what system call the user would like. A subset of
4218 ; the possible system calls is allowed here only the
4219 ; CPM-compatible calls may get dispatched.
4220
4221 ; System call entry point and dispatcher
4222 CALL_ENTRY:
4223 000002CC 1E      push    DS
4224 ;getdseg <DS>      ; DS -> DosData, ASSUME DS:DosSeg
4225 000002CD 2E8E1E[0700] mov     ds,[cs:DosDSeg]
4226 000002D2 8F06[EC05] pop     word [SAVEDS]      ; Save original DS
4227
4228 000002D6 58      POP     AX      ; IP from the long call at 5
4229 000002D7 58      POP     AX      ; Segment from the long call at 5
4230 000002D8 8F06[8405] POP     WORD [USER_SP]    ; IP from the CALL 5
4231
4232 ; Re-order the stack to simulate an interrupt 21.
4233
4234 000002DC 9C      PUSHF      ; Start re-ordering the stack
4235 000002DD FA      CLI
4236 000002DE 50      PUSH     AX      ; Save segment
4237 000002DF FF36[8405] PUSH     WORD [USER_SP]    ; Stack now ordered as if INT had been used
4238 ; 04/11/2022
4239 ; DOSCODE:40EAh (MSDOS 6.21 MSDOS.SYS)
4240 ; DOSCODE:40DDh (MSDOS 5.0 MSDOS.SYS)
4241 000002E3 FF36[EC05] push    word [SAVEDS]
4242 000002E7 1F      pop     ds
4243 ;
4244 ; cmp     cl,36
4245 000002E8 80F924   CMP     CL,MAXCALL      ; This entry point doesn't get as many calls
4246 000002EB 77DC     JA      SHORT BADCALL
4247 000002ED 88CC     MOV     AH,CL
4248 ; 08/07/2018
4249 000002EF EB0E     jmp     short SAVREGS
4250
4251 ; -----
4252
4253 ; 01/05/2019 - Retro DOS v4.0
4254 ; 01/01/2024 - Retro DOS v5.0
4255
4256 ; This is the normal INT 21 entry point. We first perform a
4257 ; quick test to see if we need to perform expensive DOS-entry
4258 ; functions. Certain system calls are done without interrupts
4259 ; being enabled.
4260
4261 ;entry COMMAND      ; Interrupt call entry point (int 21h)
4262
4263 ; DOSCODE:40F8h (MSDOS 6.21, MSDOS.SYS)
4264 ; 04/11/2022
4265 ; DOSCODE:40EBh (MSDOS 5.0, MSDOS.SYS)
4266 ; 01/01/2024
4267 ; DOSCODE:41D7h (PCDOS 7.1, IBMDOS.COM)
4268
4269 COMMAND:
4270 ; 22/12/2022
4271 000002F1 FA      cli
4272
4273 ; 01/05/2019 - Retro DOS v4.0
4274 ; 08/07/2018 - Retro DOS v3.0
4275
4276 ; 22/12/2022
4277 ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
4278 ; 01/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
4279
4280 ; IF NOT IBM
4281 000002F2 80FCF8   CMP     AH,SET_OEM_HANDLER
4282 000002F5 7203     JB      SHORT NOTOEM
4283 000002F7 E98701   JMP     _$SET_OEM_HANDLER
4284
4285 NOTOEM:
4286 ;ENDIF
4287
4288 ; DOSCODE:40F8h (MSDOS 6.21, MSDOS.SYS)
4289 ; DOSCODE:40EBh (MSDOS 5.0, MSDOS.SYS)
4290 ; 01/01/2024 - Retro DOS v5.0
4291 ; DOSCODE:41D7h (PCDOS 7.1, IBMDOS.COM)
4292
4293 ; 22/12/2022
4294 ; cli      ; 08/07/2018
4295
4296 ; 01/01/2024
4297 _COMMAND: ; MSDOS 3.3 (IBMDOS)
4298 ; cmp     ah,6Ch ; MSDOS 6.21
4299 ; cmp     ah,73h ; PCDOS 7.1 ; Max int 21h function call number
4300 ; 04/11/2022
4301 000002FA 80FC73   CMP     AH,MAXCOM ; 6Ch for MSDOS 6.0 (6.21,6.22) & MSDOS 5.0
4302 ; 73h for PCDOS 7.1
4303 ; JBE     SHORT SAVREGS
4304 000002FD 77CA     JA      SHORT BADCALL ; 08/07/2018
4305
4306 ; 31/05/2019
4307
4308 ; The following set of calls are issued by the server at
4309 ; *arbitrary* times and, therefore, must be executed on
4310 ; the user's entry stack and executed with interrupts off.
4311
4312 SAVREGS:
4313 ; 01/05/2019 - Retro DOS v4.0
4314 ; 10/08/2018
4315 ; 08/07/2018 - Retro DOS v3.0
4316 000002FF 80FC33   cmp     ah,33h      ; Check Minimum special case #
4317 ; je      _$SET_CTRL_C_TRAPPING
4318 ; je      short SetCtrlShortEntry ; If equal jmp directly to function
4319 00000302 7218     jb      short SaveAllRegs ; Not special case so continue
4320 ; 04/11/2022
4321 je      short SetCtrlShortEntry ; If equal jmp directly to function
4322 cmp     ah,64h      ; Check Max case number
4323 ja      short SaveAllRegs ; Not special case so continue
4324 0000030B 74AD     je      short _$SET_PRINTER_FLAG ; If equal jmp directly to function
4325 0000030D 80FC51   cmp     ah,51h      ; Is this a Get PSP call (51h)?
4326 00000310 749C     je      short _$GET_CURRENT_PDB ; Yes, jmp directly to function
4327 ; 25/06/2024
4328 cmp     ah,50h      ; Is this a Set PSP call (50h) ?
4329 00000315 748B     je      short _$SET_CURRENT_PDB ; Yes, jmp directly to function
4330 00000317 80FC62   cmp     ah,62h      ; Is this a Get PSP call (62h)?
4331 0000031A 7492     je      short _$GET_CURRENT_PDB ; Yes, jmp directly to function
4332
4333 SaveAllRegs:
4334 ; 01/05/2019 - Retro DOS v4.0
4335 ; 01/01/2024 - Retro DOS v5.0
4336
4337 0000031C 06      push     ES

```

```

4338 0000031D 1E      push    DS
4339 0000031E 55      push    BP
4340 0000031F 57      push    DI
4341 00000320 56      push    SI
4342 00000321 52      push    DX
4343 00000322 51      push    CX
4344 00000323 53      push    BX
4345 00000324 50      push    AX
4346
4347 00000325 8CD8      mov     AX,DS
4348      ;getdseg <DS>          ; DS -> DosData, ASSUME DS:DosSeg
4349 00000327 2E8E1E[0700]  mov     ds,[cs:DosDSeg]
4350 0000032C A3[EC05]      mov     [SAVEDS],AX      ; save caller's DS
4351 0000032F 891E[EA05]      mov     [SAVEBX],BX
4352
4353      ;INC     BYTE [INDOS]          ; Flag that we're in the DOS
4354
4355      ; 08/07/2018 - Retro DOS v3.0
4356      ;xor     ax,ax
4357      ;mov     [USER_ID],ax
4358      ;mov     ax,[CurrentPDB]
4359      ;mov     [PROC_ID],ax
4360
4361      ; 01/05/2019
4362
4363      ; Note: Nsp and Nss have to be unconditionally initialized here
4364      ; even if INDOS is zero. Programs like CROSSTALK 3.7 depend on
4365      ; this!!!
4366
4367 00000333 A1[8405]      MOV     AX,[USER_SP]
4368 00000336 A3[F205]      MOV     [NSP],AX
4369 00000339 A1[8605]      MOV     AX,[USER_SS]
4370 0000033C A3[F005]      MOV     [NSS],AX
4371
4372 0000033F 31C0      xor     AX,AX ; 0
4373 00000341 A2[7205]      mov     [FSHARING],AL      ; allow redirection
4374
4375 00000344 F606[5B0F]01  test    byte [IsWin386],1    ; WIN386 patch. Do not update USER_ID
4376 00000349 7503      jnz     short set_indos_flag ; if win386 present
4377 0000034B A3[3E03]      mov     [USER_ID],AX
4378      set_indos_flag:
4379 0000034E FE06[2103]  INC     BYTE [INDOS]          ; Flag that we're in the DOS
4380
4381      ; 01/01/2024 (PCDOS 7.1 IBMDOS.COM)
4382 00000352 FE06[B812]  inc     byte [INDOS_FLAG]      ; duplicated INDOS flag (what for ?)
4383
4384 00000356 8926[8405]      MOV     [USER_SP],SP
4385 0000035A 8C16[8605]      MOV     [USER_SS],SS
4386
4387 0000035E A1[3003]      mov     AX,[CurrentPDB]
4388 00000361 A3[3C03]      mov     [PROC_ID],AX
4389 00000364 8ED8      mov     DS,AX
4390 00000366 58      pop     AX
4391 00000367 50      push    AX
4392
4393      ; save user stack in his area for later returns (possibly from EXEC)
4394
4395 00000368 89262E00      MOV     [PDB.USER_STACK],SP      ; mov [2Eh], sp
4396 0000036C 8C163000      MOV     [PDB.USER_STACK+2],SS    ; mov [30h], ss
4397
4398      ; 18/07/2018
4399      ;mov     byte [CS:FSHARING], 0
4400
4401      ;MOV     BX,CS          ; no holes here.
4402      ;MOV     SS,BX
4403
4404      ;getdseg <ss>          ; ss -> dosdat, already flag is CLI
4405 00000370 2E8E16[0700]  mov     ss,[cs:DosDSeg]
4406      ;entry REDISP
4407
4408 00000375 BC[A007]      REDISP: MOV     SP,AUXSTACK      ; Enough stack for interrupts
4409 00000378 FB      STI          ; stack is in our space now...
4410
4411 00000379 8CD3      mov     bx,ss
4412 0000037B 8EDB      mov     ds,bx
4413
4414 0000037D 93      xchg     ax,bx
4415
4416 0000037E 31C0      xor     ax,ax ; 0
4417
4418      ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
4419      ; MSDOS 5.0 MSDOS.SYS - DOSCODE:416Eh (from org 3DD0h)
4420      ; MSDOS 6.21 MSDOS.SYS - DOSCODE:417Bh (from org 3DE0h)
4421
4422      ; (Note: ss: segment prefix was not needed here! ds=ss ! -04/11/2022-)
4423
4424      ;mov     [ss:EXTOPEN_ON],al ; 0; Clear extended open flag
4425      ;and     word [ss:DOS34_FLAG],EXEC_AWARE_REDIR
4426      ;and     word [ss:DOS34_FLAG],800h ; clear all bits except bit 11
4427      ;mov     [ss:CONSWAP],al ; 0 ; random clean up of possibly mis-set flags
4428      ;mov     [ss:NoSetDir],al ; 0 ; set directories on search
4429      ;mov     [ss:FAILERR],al ; 0 ; FAIL not in progress
4430      ;inc     ax
4431      ;inc     AL          ; AL = 1
4432      ;mov     [ss:IDLEINT],al ; presume that we can issue INT 28h
4433
4434      ; 15/12/2022
4435 00000380 A2[F605]      mov     [EXTOPEN_ON],al ; 0 ; Clear extended open flag
4436      ;and     word [DOS34_FLAG],EXEC_AWARE_REDIR
4437 00000383 8126[1106]0008  and     word [DOS34_FLAG],800h ; clear all bits except bit 11
4438 00000389 A2[5703]      mov     [CONSWAP],al ; 0 ; random clean up of possibly mis-set flags
4439      ;mov     byte [IDLEINT],1
4440 0000038C A2[4C03]      mov     [NoSetDir],al ; 0 ; set directories on search
4441 0000038F A2[4A03]      mov     [FAILERR],al ; 0 ; FAIL not in progress
4442 00000392 40      inc     ax
4443      ;inc     al          ; AL = 1
4444 00000393 A2[5803]      mov     [IDLEINT],al ; presume that we can issue INT 28h
4445
4446 00000396 93      XCHG     AX,BX          ; Restore AX and BX = 1
4447
4448 00000397 88E3      MOV     BL,AH
4449 00000399 D1E3      SHL     BX,1          ; 2 bytes per call in table
4450
4451 0000039B FC      CLD
4452      ; Since the DOS maintains mucho state information across system
4453      ; calls, we must be very careful about which stack we use.
4454      ; First, all abort operations must be on the disk stack. This
4455      ; is due to the fact that we may be hitting the disk (close
4456      ; operations, flushing) and may need to report an INT 24.
4457
4458 0000039C 08E4      OR      AH,AH
4459 0000039E 7416      JZ      SHORT DSKROUT      ; ABORT
4460
4461      ;CMP     AH,12

```

```

4462             ;JBE      SHORT IOROUT          ; Character I/O
4463             ;CMP      AH,GET_CURRENT_PDB    ; INT 24h needs GET,SET PDB
4464             ;JZ SHORT IOROUT
4465             ;CMP      AH,SET_CURRENT_PDB
4466             ;JNZ      SHORT DSKROUT
4467
4468             ; Second, PRINT and PSPRINT and the server issue
4469             ; GetExtendedError calls at INT 28 and INT 24 time.
4470             ; This call MUST, therefore, use the AUXSTACK.
4471
4472             ; 10/08/2018
4473 000003A0 80FC59 cmp     ah,GETEXTENDEDERROR ; 59h
4474 000003A3 7439 je      short DISPCALL
4475
4476             ; 01/05/2019
4477
4478             ; Old 1-12 system calls may be either on the IOSTACK (normal
4479             ; operation) or on the AUXSTACK (at INT 24 time).
4480
4481 000003A5 80FC0C cmp     ah,12 ; STD_CON_INPUT_FLUSH ; 0Ch
4482 000003A8 770C ja      short DSKROUT
4483
4484 IOROUT:
4485             ; 04/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
4486             ; (ss: prefix was not needed here! ds=ss)
4487             ; cmp     byte [ss:ERRORMODE],0 ; Are we in an INT 24h?
4488             ; 15/12/2022
4489 000003AA 803E[2003]00 cmp     BYTE [ERRORMODE],0 ; Are we in an INT 24h?
4490 000003AF 752D jnz     SHORT DISPCALL ; Stay on AUXSTACK if INT 24h
4491 000003B1 BC[A00A] mov     SP,IOSTACK
4492 000003B4 EB28 jmp     SHORT DISPCALL
4493
4494             ; We are on a system call that is classified as "the rest".
4495             ; We place ourselves onto the DSKSTACK and away we go.
4496             ; We know at this point:
4497             ; * An INT 24 cannot be in progress. Therefore we reset
4498             ;   ErrorMode and WpErr
4499             ; * That there can be no critical sections in effect.
4500             ;   We signal the server to remove all the resources.
4501
4502 DSKROUT:
4503             ; 01/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
4504             ; 15/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
4505             ; 08/07/2018 - Retro DOS v3.0
4506 000003B6 A3[3A03] mov     [USER_IN_AX],ax ; Remember what user is doing
4507             ; 01/01/2024
4508             ; mov     byte [EXTERR_LOCUS],1 ; errLOC_Unk (Default)
4509             ; MOV     BYTE [WPERR],-1 ; error mode, so good place to
4510             ; ; make sure flags are reset
4511 000003B9 C706[2203]FF01 mov     word [WPERR],1FFh
4512
4513 000003BF C606[2003]00 MOV     BYTE [ERRORMODE],0 ; Cannot make non 1-12 calls in
4514
4515             ; 04/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
4516             ; (ss: prefix was not needed here! ds=ss)
4517
4518             ; mov     [ss:USER_IN_AX],ax ; Remember what user is doing
4519             ; mov     byte [ss:EXTERR_LOCUS],1 ; errLOC_Unk (Default)
4520             ; mov     byte [ss:ERRORMODE],0 ; Cannot make non 1-12 calls in
4521             ; mov     byte [ss:WPERR],-1 ; error mode, so good place to
4522             ; ; make sure flags are reset
4523
4524 000003C4 50 push     ax
4525 000003C5 B482 mov     ah,82h ; Release all resource information
4526 000003C7 CD2A int     2Ah ; Microsoft Networks
4527             ; END DOS CRITICAL SECTIONS 0 THROUGH 7
4528 000003C9 58 pop     ax
4529
4530             ; Since we are going to be running on the DSKStack and since
4531             ; INT 28 people will use the DSKStack, we must turn OFF the
4532             ; generation of INT 28's.
4533
4534             ; 15/12/2022
4535             ; mov     byte [ss:IDLEINT],0
4536             ;
4537             ; mov     sp,DSKSTACK
4538             ; test    byte [ss:CNTCFLAG],-1 ; 0FFh
4539             ; jz short DISPCALL
4540
4541 000003CA C606[5803]00 mov     byte [IDLEINT],0
4542
4543 000003CF BC[2009] MOV     SP,DSKSTACK
4544 000003D2 F606[3703]FF TEST    BYTE [CNTCFLAG],-1
4545 000003D7 7405 JZ      SHORT DISPCALL
4546
4547 000003D9 50 PUSH     AX
4548             ; invoke DSKSTATCHK
4549 000003DA E8265A CALL    DSKSTATCHK
4550 000003DD 58 POP     AX
4551
4552 DISPCALL:
4553             ; 01/05/2019 - Retro DOS v4.0
4554             ; mov     bx,[CS:BX+DISPATCH]
4555             ; 15/12/2022
4556 000003E3 871E[EA05] xchg     bx,[SAVEBX]
4557 000003E7 8E1E[EC05] MOV     DS,[SAVEDS]
4558
4559             ; 04/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
4560             ; (ss: prefix was not needed here! ds=ss)
4561             ; xchg     bx,[ss:SAVEBX]
4562             ; mov     ds,[ss:SAVEDS]
4563
4564 000003EB 36FF16[EA05] call    word [ss:SAVEBX] ; near call
4565
4566             ; The EXEXA200FF bit of DOS_FLAG will now be unconditionally cleared
4567             ; here. Please see under M003, M009 and M068 tags in dossym.inc
4568             ; for explanation. Also NOTE that a call to ExecReady (ax=4b05) will
4569             ; return to LeaveDOS and hence will not clear this bit. This is
4570             ; because this bit is used to indicate to the next int 21 call that
4571             ; the previous int 21 was an exec.
4572             ;
4573             ; So do not add any code between the call above and the label
4574             ; LeaveDOS if it needs to be executed even for ax=4b05
4575
4576             ; and byte [ss:DOS_FLAG],~EXECA200FF
4577             ; and byte [ss:DOS_FLAG],0FBh ; clear bit 2
4578
4579             ; 01/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
4580 000003F0 368026[8600]DB and     byte [ss:DOS_FLAG],0DBh ; clear bit 2 and bit 5
4581
4582             ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
4583             ; DOSCODE:41F7h
4584
4585             ; 01/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)

```

```

4586 ; DOSCODE:42D4h
4587
4588 ;entry LEAVE
4589 ;;;_LEAVE: ; Exit from a system call
4590 LeaveDOS: ; 18/07/2018
4591 ;ASSUME SS:NOTHING ; User routines may misbehave
4592 CLI
4593
4594 ; 01/05/2019
4595 ;getdseg <DS> ; DS -> DosData, ASSUME DS:DosSeg
4596 mov ds,[cs:DosDSeg]
4597 cmp byte [A20OFF_COUNT],0 ; M068: Q: is count 0
4598 jne short disa20 ; M068: N: dec count and turn a20 off
4599
4600 LeaveA20on:
4601 DEC BYTE [INDOS]
4602
4603 ; 01/01/2024 (PCDOS 7.1 IBMDOS.COM)
4604 dec byte [INDOS_FLAG] ; duplicated INDOS flag (what for ?)
4605
4606 ; 04/11/2022
4607 mov ss,[USER_SS]
4608 MOV SP,[USER_SP]
4609 ;MOV SS,[USER_SS]
4610 MOV BP,SP
4611 ;MOV [BP.user_AX],AL
4612 ; 04/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
4613 ;mov [bp+0],al ; MSDOS 5.0 MSDOS.SYS - DOSCODE:4212h
4614 ;MOV [BP+user_env.user_AX],AL ; user_env.user_AX = 0
4615
4616 ; 15/12/2022
4617 MOV [BP],AL ; mov [bp+0],al
4618
4619 ;MOV AX,[NSP]
4620 ;MOV [USER_SP],AX
4621 ;MOV AX,[NSS]
4622 ;MOV [USER_SS],AX
4623
4624 ; 01/01/2024
4625 les ax,[NSS]
4626 mov [USER_SS],ax
4627 mov [USER_SP],es
4628 pop AX
4629 pop BX
4630 pop CX
4631 pop DX
4632 pop SI
4633 pop DI
4634 pop BP
4635 pop DS
4636 pop ES
4637
4638 IRET
4639
4640 disa20: ; M068 - Start
4641 mov bx,[A20OFF_PSP] ; bx = PSP for which a20 to be off'd
4642 cmp bx,[CurrentPDB] ; Q: do the PSP's match
4643 jne short LeaveA20on ; N: don't clear bit and don't turn
4644 ; a20 off
4645 ; Y: turn a20 off and dec a20off_count
4646 dec byte [A20OFF_COUNT] ; M068 - End
4647 ; Start - M004
4648 push ds ; segment of stub
4649 mov bx,disa20_iret ; offset in stub
4650 push bx
4651 retf ; go to stub
4652 ; End - M004
4653
4654 ;SYSTEM_CALL ENDP
4655
4656 ; DOSCODE:424Ch (MSDOS 6.21, MSDOS.SYS)
4657 ; 04/11/2022
4658 ; DOSCODE:423Fh (MSDOS 5.0, MSDOS.SYS)
4659
4660 ; =====
4661 ; Restore_world restores all registers ('cept SS:SP, CS:IP, flags) from
4662 ; the stack prior to giving the user control
4663 ; =====
4664
4665 ; 01/05/2019 - Retro DOS v4.0
4666
4667 ;procedure restore_world,NEAR
4668 restore_world:
4669 ;getdseg <es> ; es -> dosdata
4670 mov es,[cs:DosDSeg]
4671
4672 POP WORD [ES:RESTORE_TMP]
4673
4674 POP AX
4675 POP BX
4676 POP CX
4677 POP DX
4678 POP SI
4679 POP DI
4680 POP BP
4681 POP DS
4682
4683 jmp word [ES:RESTORE_TMP]
4684
4685 ;restore_world ENDP
4686
4687 ; 01/05/2019 - Retro DOS v4.0 (MSDOS 6.0, MSDISP.ASM, 1991)
4688
4689 ; DOSCODE:4263h (MSDOS 6.21, MSDOS.SYS)
4690 ; 04/11/2022
4691 ; DOSCODE:4256h (MSDOS 5.0, MSDOS.SYS)
4692
4693 ; =====
4694 ; Save_world saves complete registers on the stack
4695 ; =====
4696
4697 ;procedure save_world,NEAR
4698 save_world:
4699 ;getdseg <es> ; es -> dosdata
4700 mov es,[cs:DosDSeg]
4701
4702 POP WORD [ES:RESTORE_TMP]
4703
4704 ; 12/05/2019
4705 PUSH DS
4706
4707
4708
4709

```



```

4710 00000463 55          PUSH    BP
4711 00000464 57          PUSH    DI
4712 00000465 56          PUSH    SI
4713 00000466 52          PUSH    DX
4714 00000467 51          PUSH    CX
4715 00000468 53          PUSH    BX
4716 00000469 50          PUSH    AX
4717
4718 0000046A 26FF36[EE05]        push    word [ES:RESTORE_TMP]
4719
4720 0000046F 55          push    BP
4721 00000470 89E5        mov     BP,SP
4722 00000472 8E4614      mov     ES,[BP+20]    ; es was pushed before call
4723 00000475 5D          pop     BP
4724
4725 00000476 C3          retn
4726
4727 ;save_world ENDP
4728
4729 ; 01/05/2019
4730
4731 ; DOSCODE:4282h (MSDOS 6.21, MSDOS.SYS)
4732 ; 04/11/2022
4733 ; DOSCODE:4275h (MSDOS 5.0, MSDOS.SYS)
4734
4735 ; =====
4736 ;
4737 ; Get_User_Stack returns the user's stack (and hence registers) in DS:SI
4738 ;
4739 ; =====
4740
4741 ;procedure get_user_stack,NEAR
4742 Get_User_Stack:
4743 ;getdseg <DS>          ; DS -> DosData, ASSUME DS:DosSeg
4744 mov     ds,[cs:DosDSeg]
4745 lds     si,[USER_SP]
4746 retn
4747
4748 ;get_user_stack ENDP
4749
4750 ; 22/12/2022
4751 ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0, MSDOS.SYS)
4752 ; 01/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1, IBMDOS.COM)
4753 ;%if 0
4754
4755 ; -----
4756 ;
4757 ; Set_OEM_Handler -- Set OEM sys call address and handle OEM calls
4758 ; Inputs:
4759 ;   User registers, User Stack, INTS disabled
4760 ;   If CALL F8, DS:DX is new handler address
4761 ; Function:
4762 ;   Process OEM INT 21 extensions
4763 ; Outputs:
4764 ;   Jumps to OEM_HANDLER if appropriate
4765 ;
4766 ; -----
4767
4768 ;IF NOT IBM
4769
4770 _$SET_OEM_HANDLER:
4771 ; 01/05/2019 - Retro DOS v4.0
4772
4773 ;(cmp     ah,SET OEM HANDLER ; 0F8h)
4774 ;(jb     short NOTOOEM)
4775
4776 00000481 06          push     es ; *
4777 ;getdseg <es>          ; es -> dosdata
4778 00000482 2E8E06[0700]  mov     es,[cs:DosDSeg]
4779
4780 00000487 750C        jne     short check_trueversion_request ; check Retro DOS true version
4781 ; (message) request
4782 ; AH = 0F8h = SET OEM HANDLER
4783
4784 00000489 268916[1400]  MOV     [es:OEM_HANDLER],DX ; Set Handler
4785 0000048E 268C1E[1600]  MOV     [es:OEM_HANDLER+2],DS
4786
4787 00000493 07          pop     es ; *
4788
4789 00000494 CF          IRET          ; Quick return, Have altered no registers
4790
4791 check_trueversion_request:
4792 ; 18/07/2019 - Retro DOS v3.0
4793
4794 ; Retro DOS v2.0 - 20/04/2018
4795 00000495 83F8FF      CMP     AX,0FFFFh
4796 ; 18/07/2018
4797 00000498 7520        jne     short DO_OEM_FUNC ; 01/05/2019
4798
4799 ; 01/05/2019
4800 0000049A 07          pop     es ; *
4801
4802 0000049B B40E        mov     ah,0Eh
4803
4804 ; Retro DOS v4.0 feature only!
4805 0000049D 81FBA101    cmp     bx,417 ; Signature to bypass
4806 ; Retro DOS true version message
4807 000004A1 7414        je     short true_version_iret
4808
4809 000004A3 56          push     si
4810 000004A4 53          push     bx
4811
4812 000004A5 BE[C200]  mov     si,RETRODOSMSG
4813 wrdosmsg:
4814 ;movb     ah,0Eh
4815 000004A8 BB0700  mov     bx,7
4816 wrdosmsg_nxt:
4817 000004AB 2EAC        cs lodsb
4818 000004AD 3C24        cmp     al,'$'
4819 000004AF 7404        je     short wrdosmsg_ok
4820 000004B1 CD10        int     10h
4821 000004B3 EBF6        jmp     short wrdosmsg_nxt
4822
4823 wrdosmsg_ok:
4824 000004B5 5B          pop     bx
4825 000004B6 5E          pop     si
4826
4827 true_version_iret:
4828 ; ah = 0Eh
4829 ;mov     al,40h ; Retro DOS v4.0
4830 ;
4831 ;mov     al,41h ; Retro DOS v4.1
4832 ; 30/12/2022
4833 ;mov     al,42h ; Retro DOS v4.2

```

```

4834             ; 01/01/2024
4835 000004B7 B050      mov     al,50h ; Retro DOS v5.0
4836 000004B9 CF       ired
4837
4838             ; If above F8 try to jump to handler
4839
4840 DO_OEM_FUNC:
4841             ; 01/05/2019
4842 000004BA 26833E[1400]FF      cmp     word [es:OEM_HANDLER],-1
4843 000004C0 7504             jne     short OEM_JMP
4844 000004C2 07             pop     es ; *
4845 000004C3 E903FE             jmp     BADCALL ; Handler not initialized
4846
4847 000004C6 06             OEM_JMP:
4848 000004C7 1F             push    es
4849 000004C8 07             pop     ds ; DOSDATA segment !
4850             pop     es ; *
4851
4852             ; 22/12/2022
4853             sti         ; (enable interrupts before jumping to private handler)
4854 000004CA FF2E[1400]      jmp     far [OEM_HANDLER]
4855
4856             ;
4857             ;
4858             ; -----
4859             ;
4860             ;%endif
4861
4862             ;=====
4863             ; MCODE.ASM, MSDOS 6.0, 1991
4864             ;=====
4865             ; 17/07/2018 - Retro DOS v3.0
4866
4867             ; TITLE MISC DOS ROUTINES - Int 25 and 26 handlers and other
4868             ; NAME IBMCODE
4869
4870             ;BREAK <NullDev -- Driver for null device>
4871
4872             ; ROMDOS note:
4873             ; NUL device driver used to be here, but it was removed and placed in
4874             ; DOSDATA, because the entry points have to be in the segment as the
4875             ; header, which is also in DOSDATA.
4876
4877             ;BREAK <AbsDRD, AbsDWRT -- INT int_disk_read, int_disk_write handlers>
4878
4879             ;
4880             ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0, MSDOS.SYS)
4881             ;
4882             ; DOSCODE:428Ch (MSDOS 6.21 MSDOS.SYS)
4883             ; DOSCODE:427Fh (MSDOS 5.0 MSDOS.SYS)
4884
4885             ; 01/01/2024 - Retro DOS v5.0
4886             ; DOSCODE:435Fh (PCDOS 7.1 IBMDOS.COM)
4887
4888             ;Public MSC001S,MSC001E
4889             ;MSC001S label byte
4890             ;IF IBM
4891             ; Codes returned by BIOS
4892             ERRIN:
4893 000004CE 02             DB      2 ; NO RESPONSE
4894 000004CF 06             DB      6 ; SEEK FAILURE
4895 000004D0 0C             DB     12 ; GENERAL ERROR
4896 000004D1 04             DB      4 ; BAD CRC
4897 000004D2 08             DB      8 ; SECTOR NOT FOUND
4898 000004D3 00             DB      0 ; WRITE ATTEMPT ON WRITE-PROTECT DISK
4899
4900             ERROUT:
4901             ; DISK ERRORS RETURNED FROM INT 25 and 26
4902 000004D4 80             DB     80H ; NO RESPONSE
4903 000004D5 40             DB     40H ; Seek failure
4904 000004D6 02             DB      2 ; Address Mark not found
4905 000004D7 10             DB     10H ; BAD CRC
4906 000004D8 04             DB      4 ; SECTOR NOT FOUND
4907 000004D9 03             DB      3 ; WRITE ATTEMPT TO WRITE-PROTECT DISK
4908
4909             NUMERR EQU     $-ERROUT
4910             ;ENDIF
4911             ;MSC001E label byte
4912             ;
4913             ;=====
4914             ; MCODE.ASM - MSDOS 6.0 - 1991
4915             ;=====
4916             ; 18/07/2018 - Retro DOS v3.0
4917             ; 15/05/2019 - Retro DOS v4.0
4918
4919             ; 02/01/2024 - Retro DOS v5.0
4920             ; PCDOS 7.1 IBMDOS.COM - DOSCODE:436Bh
4921
4922             ;BREAK <AbsDRD, AbsDWRT -- INT int_disk_read, int_disk_write handlers>
4923
4924             ; AbsSetup - setup for abs disk functions
4925             ;
4926             ; -----
4927             AbsSetup:
4928             ; 02/01/2024 (PCDOS 7.1, Retro DOS 5.0)
4929             ;;;
4930             ;
4931 000004DA 1E             push    ds ; *
4932 000004DB 16             push    ss
4933 000004DC 1F             pop     ds
4934             ;
4935 000004DD 8826[DB0A]      mov     [absdrw_extd],ah
4936             ;mov     [ss:absdrw_extd],ah ; Extended ABS Disk Read/Write flag
4937             ; (AH=1 for INT 21h ax=7305h function)
4938 000004E1 08E4             or      ah,ah
4939 000004E3 7508             jnz     short AbsSetup1 ; INT 21h AX=7305h
4940
4941             ; INT 25h
4942 000004E5 FE06[B812]      inc     byte [INDOS_FLAG]
4943             ;inc     byte [ss:INDOS_FLAG] ; windows DOSBOX's INDOS flag ?
4944             ;;;
4945 000004E9 FE06[2103]      inc     byte [INDOS]
4946             ;inc     byte [ss:INDOS] ; SS override
4947             AbsSetup1:
4948             STI
4949             CLD
4950             ; 02/01/2024
4951             ;PUSH DS
4952             ;push    ss
4953             ;pop     ds
4954 000004EF E83C01         CALL    GETBP
4955             ; 02/01/2024
4956 000004F2 1F             pop     ds ; *
4957 000004F3 7229             JC      short errdriv ; PM. error drive ;AN000;

```



```

4958
4959
4960 ; 02/01/2024
4961 ;mov word [es:bp+1Fh]
4962 ;MOV WORD [ES:BP+DPB.FREE_CNT],-1 ; do not trust user at all.
4963 ;errdriv:
4964 ;POP DS
4965 ;jnc short AbsSetup2
4966 ;AbsSetup_retn:
4967 ;retn
4968
4969 AbsSetup2:
4970 ; 15/05/2019 - Retro DOS v4.0
4971 ; MSDOS 6.0
4972 ; SS override
4973 MOV word [SS:HIGH_SECTOR],0 ;>32mb from API ;AN000;
4974 CALL RW32_CONVERT ;>32mb convert 32bit format to 16bit ;AN000;
4975 ;jc short AbsSetup_retn
4976 ;call SET_RQ_SC_PARAMS ;LB. set up SC parms ;AN000;
4977 ; 02/01/2024 - Retro DOS v5.0
4978 ;jc short errdriv
4979 ;call null_sub ; retn ; PCDOS 7.1 IBMDOS.COM
4980
4981 ; MSDOS 3.3 (& MSDOS 6.0)
4982 PUSH DS
4983 PUSH SI
4984 PUSH AX
4985
4986 push ss
4987 pop ds
4988 MOV SI,OPENBUF
4989 MOV [SI],AL
4990 ADD BYTE [SI],"A"
4991 MOV WORD [SI+1],003AH ; ":" ,0
4992 MOV AX,0300H
4993 CLC
4994 INT int_IBM ; int 2Ah ; will set carry if shared
4995
4996 ; 04/11/2022
4997 ; (INT 2Ah - AX = 0300h)
4998 ; Microsoft Networks - CHECK DIRECT I/O
4999 ; DS:SI -> ASCIIIZ disk device name (may be full path or
5000 ; only drive specifier--must include the colon)
5001 ; Return: CF clear if absolute disk access allowed
5002
5003 POP AX
5004 POP SI
5005 POP DS
5006 jnc short AbsSetup_retn
5007
5008 ; 02/01/2024
5009 errdriv:
5010 ;mov word [ss:EXTERR],32h
5011 MOV word [ss:EXTERR],error_not_supported
5012 ; 02/01/2024 - Retro DOS v5.0
5013 mov word [ss:AbsDskErr],207h ; PCDOS 7.1 IBMDOS.COM
5014
5015 ; 02/01/2024
5016 AbsSetup_retn:
5017 retn
5018
5019 ;-----
5020 ;
5021 ; Procedure Name : ABSDRD
5022 ;
5023 ; Interrupt 25 handler. Performs absolute disk read.
5024 ; Inputs: AL - 0-based drive number
5025 ; DS:BX point to destination buffer
5026 ; CX number of logical sectors to read
5027 ; DX starting logical sector number (0-based)
5028 ; Outputs: Original flags still on stack
5029 ; Carry set
5030 ; AH error from BIOS
5031 ; AL same as low byte of DI from INT 24
5032 ;-----
5033 ;
5034 ;procedure ABSDRD,FAR
5035 ABSDRD:
5036 ; 15/05/2019 - Retro DOS v4.0
5037 ; MSDOS 6.21 (DOSCODE:42E5h)
5038 ; 04/11/2022
5039 ; MSDOS 5.0 (DOSCODE:42D8h)
5040
5041 ; 02/01/2024 - Retro DOS v5.0
5042 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:43C4h
5043
5044 ;;;
5045 xor ah,ah ; ah=0 ; Interrupt 25h handler (ah=0)
5046 xor si,si ; si=0 ; clear read/write mode flags
5047 ; (used with INT 21h ax=7305h)
5048 FAT32_ABSDRD: ; ah=1 si=0 cf=0 (jump from '_$FAT32EXT')
5049 ; 25/06/2024
5050 ;cli ; *
5051 ;clc ; not necessary
5052 ; INT 21h
5053 ; AX = 7305h
5054 ; FAT32 - EXTENDED ABSOLUTE DISK READ/WRITE
5055 ; CX = FFFFh
5056 ; DL = drive number (01h=A:, etc.)
5057 ; SI = read/write mode flags
5058 ; DS:BX -> disk I/O packet
5059
5060 ; Extended Absolute Disk Read/write mode flags:
5061 ;
5062 ; Bit(s) Description
5063 ; 0 direction (0=read, 1=write)
5064 ; 12-1 reserved (0)
5065 ; 14-13 write type (should be 00 on reads).
5066 ; 00 unknown data.
5067 ; 01 FAT data.
5068 ; 10 directory data.
5069 ; 11 file data
5070 ; 15 reserved (0)
5071
5072 ; Format of disk read/write packet:
5073 ;
5074 ; Offset Size Description
5075 ; 00h DWORD sector number
5076 ; 04h WORD number of sectors to r/w
5077 ; 06h DWORD transfer address
5078
5079 ; ref: Ralf Brown's Interrupt List
5080
5081 ;;;
absdrd_1:

```

```

5082             ; MSDOS 6.0
5083             ; 25/06/2024 - Retro DOS v5.0
5084 00000531 FA  CLI ; *
5085
5086             ; set up ds to point to DOSDATA
5087
5088 00000532 50    push    ax                ; preserve AX value
5089 00000533 8CD8  mov     ax,ds                ; store DS value in AX
5090             ;getdseg <ds>
5091 00000535 2E8E1E[0700] mov     ds,[cs:DosDSeg]
5092 0000053A A3[630D]  mov     [TEMPSEG],ax            ; store DS value in TEMPSEG
5093 0000053D 58      pop     ax                ; restore AX value
5094
5095             ; M072:
5096             ; We shall save es on the user stack here. We need to use ES in
5097             ; order to access the DOSDATA variables AbsRdwr_SS/SP at exit
5098             ; time in order to restore the user stack.
5099
5100 0000053E 06    push    es ; ****                ; M072
5101
5102             ; 25/06/2024
5103             ; 02/01/2024 - RetroDOS v5.0 (PCDOS 7.1 IBMDOS.COM)
5104             ;;;
5105 0000053F 7305   jnc     short absdrd_2            ; (not jumped from ABSDRWT) absolute disk read
5106
5107             ;(jumped from ABSDRWT)
5108 00000541 08E4   or      ah,ah
5109 00000543 F9     stc                      ; absolute disk write
5110 00000544 EB02   jmp     short absdrd_3
5111 absdrd_2:      or      ah,ah
5112 00000546 08E4   absdrd_3:
5113             jnz     short absdrd_4            ; EXTENDED ABSOLUTE DISK READ/WRITE
5114 00000548 7510   ;;;
5115
5116             MOV     [AbsRdwr_SS],SS            ; M013
5117 0000054A 8C16[1B06] MOV     [AbsRdwr_SP],SP            ; M013
5118 0000054E 8926[1D06]
5119
5120             ; set up ss to point to DOSDATA
5121             ;
5122             ; NOTE! Due to an obscure bug in the 80286, you cannot use the ROMDOS
5123             ; version of the getdseg macro with the SS register! An interrupt will
5124             ; sneak through.
5125
5126             ;ifndef ROMDOS
5127             ;getdseg <ss>                ; cli in entry of routine
5128
5129 00000552 2E8E16[0700] mov     ss,[cs:DosDSeg]
5130
5131             ;else
5132             ; mov     ds, cs:[BioDataSeg]
5133             ; assume  ds:bdata
5134             ;
5135             ; mov     ss, ds:[DosDataSg]
5136             ; assume  ss:DOSDATA
5137             ;
5138             ;endif ; ROMDOS
5139
5140 00000557 BC[2009] MOV     SP,DSKSTACK            ; 02/01/2024
5141                                     ;"@#IBM:12.01.2003.build_1.32#@ IBMDOS.COM(USA)"
5142                                     ; (PCDOS 7.1 IBMDOS.COM)
5143 0000055A 8E1E[630D] absdrd_4:
5144             mov     ds,[TEMPSEG]            ; restore DS value
5145 0000055E 06      push    es ; *** (MSDOS 6.21)
5146 0000055F E8F6FE  call    save_world            ; save all regs
5147
5148 00000562 06      PUSH    ES ; **
5149
5150             ; 02/01/2024 - RetroDOS v5.0
5151             ;;;
5152 00000563 7303   jnc     short absdrd_5            ; absolute disk read
5153 00000565 E9A400  jmp     absdrwt_3            ; (jumping back to) absolute disk write
5154 absdrd_5:      ;;;
5155
5156             CALL     AbsSetup
5157 00000568 E86FFF  JC      short ILEAVE
5158 0000056B 723D
5159
5160             ; Here is a gross temporary fix to get around a serious design flaw in
5161             ; the secondary cache. The secondary cache does not check for media
5162             ; changed (it should). Hence, you can change disks, do an absolute
5163             ; read, and get data from the previous disk. To get around this,
5164             ; we just won't use the secondary cache for absolute disk reads.
5165             ; -mw 8/5/88
5166
5167             ;EnterCrit critDisk
5168 0000056D E87213  call    ECritDisk
5169 00000570 36C606[7511]FF MOV     byte [ss:CurSC_DRIVE],-1 ; invalidate SC ;AN000;
5170             ;LeaveCrit critDisk
5171 00000576 E89613  call    LCritDisk
5172
5173             ;invoke DSKREAD
5174 00000579 E8C73A  CALL    DSKREAD
5175 0000057C 7513   jnz short ERR_LEAVE            ;Jump if read unsuccessful.
5176
5177 0000057E 89F9   mov     cx,di
5178 00000580 368C1E[0E06] mov     [ss:TEMP_VAR2],ds
5179 00000585 36891E[0C06] mov     [ss:TEMP_VAR],bx
5180
5181             ; CX = # of contiguous sectors read. (These constitute a block of
5182             ; sectors, also termed an "Extent".)
5183             ; [HIGH_SECTOR]:DX = physical sector # of first sector in extent.
5184             ; [TEMP_VAR2]:[TEMP_VAR] = Transfer address (destination data address).
5185             ; ES:BP -> Drive Parameter Block (DPB).
5186
5187             ; The Buffer Queue must now be scanned: the contents of any dirty
5188             ; buffers must be "read" into the transfer memory block, so that the
5189             ; transfer memory reflects the most recent data.
5190
5191             ;invokeDskRdBufScan            ;This trashes DS, but don't care.
5192 0000058A E8483D  call    DskRdBufScan
5193 0000058D EB1B   jmp     short ILEAVE
5194
5195             TLEAVE:
5196 0000058F 7419   JZ      short ILEAVE
5197
5198             ERR_LEAVE:
5199             ; 15/07/2018 - Retro DOS v3.0
5200             ;IF IBM
5201             ; Translate the error code to ancient 1.1 codes
5202 00000591 06      PUSH    ES ; *
5203 00000592 0E      PUSH    CS
5204 00000593 07      POP     ES
5205 00000594 30E4   XOR     AH,AH                ; Null error code

```

```

5206             ;mov     cx,6
5207 00000596 B90600      MOV CX,NUMERR          ; Number of possible error conditions
5208 00000599 BF[CE04]    MOV DI,ERRIN          ; Point to error conditions
5209 0000059C F2AE        REPNE     SCASB
5210 0000059E 7504        JNZ SHORT LEAVECODE      ; Not found
5211             ;mov     ah,[ES:DI+5]
5212 000005A0 268A6505     MOV AH,[ES:DI+NUMERR-1]  ; Get translation
5213 LEAVECODE:
5214 000005A4 07           POP ES ; *
5215             ; 15/05/2019 - Retro DOS v4.0
5216 000005A5 36A3[Bf0D]   mov     [ss:AbsDskErr],ax
5217             ;ENDIF
5218
5219 000005A9 F9           STC
5220 ILEAVE:
5221             ; 15/05/2019
5222 000005AA 07           POP ES ; **
5223 000005AB E893FE       call    restore_world
5224 000005AE 07           pop     es ; *** (MSDOS 6.21)
5225
5226             ; 02/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
5227             ;;;
5228 000005AF 9C           pushf
5229 000005B0 36803E[DB0A]00 cmp     byte [ss:absdrw_extd],0
5230             ; FAT32- EXTENDED ABSOLUTE DISK READ/WRITE flag
5231 000005B6 751F        jnz     short ILEAVE_EXTD      ; INT 21h AX=7305h
5232             ; INT 25h
5233 000005B8 9D           popf
5234             ;;;
5235
5236 000005B9 FA           CLI
5237 000005BA 36A1[Bf0D]   mov     ax,[ss:AbsDskErr]      ; restore error
5238 000005BE 36FE0E[2103] DEC     BYTE [SS:INDOS]
5239             ;
5240             ; 02/01/2024 (PCDOS 7.1 IBMDOS.COM)
5241 000005C3 36FE0E[B812] dec     byte [ss:INDOS_FLAG] ; Windows DOSBOX's INDOS flag ?
5242             ;
5243             ; push     ss
5244 000005C9 07           pop     es ; es - dosdata
5245 000005CA 268E16[1B06] mov     ss,[es:AbsRdwr_SS] ; M013
5246 000005CF 268B26[1D06] mov     sp,[es:AbsRdwr_SP] ; M013
5247 000005D4 07           pop     es ; *****
5248             ; Note es was saved on user
5249             ; stack at entry
5250             ; M072 - End
5250 000005D5 FB           STI
5251 000005D6 CB           RETF ; ! FAR return !
5252
5253             ; 02/01/2024 - Retro DOS v5.0
5254             ; (PCDOS 7.1 IBMDOS.COM)
5255             ;;;
5256 ILEAVE_EXTD:          ; return from INT 21h AX=7305h
5257 000005D7 9D           popf
5258 000005D8 36A1[Bf0D]   mov     ax,[ss:AbsDskErr]      ; restore error
5259 000005DC 07           pop     es ; *****
5260 000005DD FB           sti
5261 000005DE C3           retn
5262             ;;;
5263
5264 ;ABSDRD      ENDP
5265
5266 -----
5267 ;
5268 ; Procedure Name : ABSDWRT
5269 ;
5270 ; Interrupt 26 handler. Performs absolute disk write.
5271 ; Inputs:  AL - 0-based drive number
5272 ;          DS:BX point to source buffer
5273 ;          CX number of logical sectors to write
5274 ;          DX starting logical sector number (0-based)
5275 ; Outputs: Original flags still on stack
5276 ;          Carry set
5277 ;          AH error from BIOS
5278 ;          AL same as low byte of DI from INT 24
5279 ;
5280 -----
5281 ;procedure   ABSDWRT,FAR
5282 ABSDWRT:
5283             ; 15/05/2019 - Retro DOS v4.0
5284             ; MSDOS 6.21 (DOSCODE:436Ch)
5285             ; 04/11/2022
5286             ; MSDOS 5.0 (DOSCODE:435Fh)
5287
5288             ; 03/01/2024 - Retro DOS v5.0
5289             ; PCDOS 7.1 IBMDOS.COM - DOSCODE:4477h
5290
5291             ;;;
5292 000005DF 30E4        xor     ah,ah ; ah=0          ; Interrupt 26h handler (ah=0)
5293 000005E1 BE0100      mov     si,1          ; set direction flag/bit to write
5294             ; (used with INT 21h ax=7305h)
5295 FAT32_ABSDWRT:      ; ah=1 si=1 cf=0 (jump from '_$FAT32EXT')
5296
5297             ; INT 21h
5298             ; AX = 7305h
5299             ; FAT32 - EXTENDED ABSOLUTE DISK READ/WRITE
5300             ; CX = FFFFh
5301             ; DL = drive number (01h=A:, etc.)
5302             ; SI = read/write mode flags
5303             ; DS:BX -> disk I/O packet
5304
5305             ; Extended Absolute Disk Read/Write mode flags:
5306             ;
5307             ; Bit(s) Description
5308             ; 0 direction (0=read, 1=write)
5309             ; 12-1 reserved (0)
5310             ; 14-13 write type (should be 00 on reads).
5311             ; 00 unknown data.
5312             ; 01 FAT data.
5313             ; 10 directory data.
5314             ; 11 file data
5315             ; 15 reserved (0)
5316
5317             ; Format of disk read/write packet:
5318             ;
5319             ; Offset Size Description
5320             ; 00h DWORD sector number
5321             ; 04h WORD number of sectors to r/w
5322             ; 06h DWORD transfer address
5323
5324             ; ref: Ralf Brown's Interrupt List
5325 000005E4 3C02        cmp     al,2
5326 000005E6 7220        jb     short absdrwt_2      ; floppy disk
5327             ; hard disk
5328 000005E8 53           push     bx
5329 000005E9 1E           push     ds

```

```

5330 000005EA 2E8E1E[0700]      mov     ds,[cs:DosDSeg]
5331 000005EF 30FF              xor     bh,bh
5332 000005F1 88C3              mov     bl,al
5333                                ; NOTE: PC DOS 7.1 kernel does not set
5334                                ; DOS_FLAG bit 6 or drive_flags bit 7
5335                                ; (It appears that these bits are set
5336                                ; by windows or a system utility or
5337                                ; driver that knows the addresses of
5338                                ; these FLAGS in the DOSDATA segment.)
5339                                ; Erdogan Tan - 03/01/2024
5340
5341 000005F3 F687[0813]80      test    byte [drive_flags+bx],80h
5342                                ; test bit 7 (locked bit) ; 29/01/2024
5343 000005F8 7505              jnz     short absdwr1_1
5344                                ; locked (logical drive) -allowed to abs write-
5345 000005FA F606[8600]40      test    byte [DOS_FLAG],40h
5346                                ; NOTE: lock/unlock are MSDOS/PCDOS 7 extd functions
5347                                ; test bit 6 (large disk support -windows- bit?)
5348                                ; (windows OS running bit ?) ; 23/02/2024
5349                                ; NOTE: Retro DOS v5 kernel must set this bit.
5350
5351 000005FF 1F              absdwr1_1:
5352 00000600 5B              pop     ds
5353 00000601 7505              pop     bx
5354 00000603 F9              jnz     short absdwr1_2
5355 00000604 E817FF          stc
5356 00000607 CB              call    errdriv
5357                                ; error
5358                                retf
5359
5360                                ;absdwr1_2:
5361                                ;;;
5362                                ; 03/01/2024
5363                                %if 0
5364                                CLI
5365                                ;
5366                                ; set up ds to point to DOSDATA
5367
5368                                push    ax
5369                                mov     ax,ds
5370                                ;getdseg <ds>
5371                                mov     ds,[cs:DosDSeg]
5372                                mov     [TEMPSEG],ax
5373                                pop     ax
5374
5375                                ; M072:
5376                                ; we shall save es on the user stack here. We need to use ES in
5377                                ; order to access the DOSDATA variables AbsRdWr_SS/SP at exit
5378                                ; time in order to restore the user stack.
5379
5380                                push    es ; ****
5381                                ; M072
5382
5383                                MOV     [AbsRdWr_SS],SS
5384                                ; M013
5385                                MOV     [AbsRdWr_SP],SP
5386                                ; M013
5387
5388                                ; set up ss to point to DOSDATA
5389                                ;
5390                                ; NOTE! Due to an obscure bug in the 80286, you cannot use the
5391                                ; ROMDOS version of the getdseg macro with the SS register!
5392                                ; An interrupt will sneak through.
5393
5394                                ;ifndef ROMDOS
5395                                ;getdseg <ss>
5396                                ; cli in entry of routine
5397                                mov     ss,[cs:DosDSeg]
5398                                ;else
5399                                ; mov     ds, cs:[BioDataSeg]
5400                                ; assume ds:bdata
5401                                ;
5402                                ; mov     ss, ds:[DosDataSg]
5403                                ; assume ss:DOSDATA
5404                                ;
5405                                ;endif ; ROMDOS
5406
5407                                MOV     SP,DSKSTACK
5408                                ; we are now switched to DOS's disk stack
5409
5410                                mov     ds,[TEMPSEG]
5411                                ; restore user's ds
5412
5413                                push    es ; *** (MSDOS 6.21)
5414
5415                                call    save_world
5416                                ; save all regs
5417
5418                                PUSH    ES ; **
5419                                %endif
5420                                ; 03/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
5421                                ;;;
5422                                absdwr1_2:
5423                                ; 25/06/2024
5424                                ;cli
5425                                stc
5426                                ; writable disk
5427                                ; ('jumped from ABSDWR1' sign for common r/w code)
5428                                jmp     absdwr1_1
5429                                ; jump to ABSDRD (common r/w) code
5430                                ;;;
5431
5432                                absdwr1_3:
5433                                CALL    AbsSetup
5434                                JC      short ILEAVE
5435
5436                                ; 03/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
5437                                call    chk_set_first_access
5438
5439                                ;EnterCrit critDisk
5440                                call    ECritDisk
5441                                MOV     byte [ss:CurSC_DRIVE],-1 ; invalidate SC ;AN000;
5442                                CALL    Fastxxx_Purge
5443                                ; purge fatopen ;AN000;
5444                                ;LeaveCrit critDisk
5445                                call    LCritDisk
5446
5447                                ;M039
5448                                ;
5449                                ; DS:BX = transfer address (source data address).
5450                                ; CX = # of contiguous sectors to write. (These constitute a block of
5451                                ; sectors, also termed an "Extent".)
5452                                ; [HIGH_SECTOR]:DX = physical sector # of first sector in extent.
5453                                ; ES:BP -> Drive Parameter Block (DPB).
5454                                ; [CURSC_DRIVE] = -1 (invalid drive).
5455                                ;
5456                                ; Free any buffered sectors which are in Extent; they are being over-
5457                                ; written. Note that all the above registers are preserved for
5458                                ; DSKWRITE.
5459                                ;
5460                                push    ds
5461                                ;invoke DskwrtBufPurge
5462                                ; This trashes DS.
5463                                call    DskwrtBufPurge
5464                                pop     ds
5465                                ;M039
5466                                ;invoke DSKWRITE
5467                                call    DSKWRITE

```

```

5454 0000062B E961FF      JMP     TLEAVE
5455
5456 ;ABSDWRT ENDP
5457
5458 ;-----
5459 ;
5460 ; Procedure Name : GETBP
5461 ;
5462 ; Inputs:
5463 ;   AL = Logical unit number (A = 0)
5464 ; Function:
5465 ;   Find Drive Parameter Block
5466 ; Outputs:
5467 ;   ES:BP points to DPB
5468 ;   [THISDPB] = ES:BP
5469 ;   Carry set if unit number bad or unit is a NET device.
5470 ;   Later case sets extended error error_I24_not_supported
5471 ; No other registers altered
5472 ;-----
5473 ;
5474 ;
5475 ; 07/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
5476 ; 04/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
5477 ; PCDOS 76.1 IBMDOS.COM - DOSCODE:44C7h
5478
5479 GETBP:
5480 ; 15/05/2019 - Retro DOS v4.0
5481 ; 11/07/2018 - Retro DOS v3.0
5482 PUSH    AX
5483 ADD     AL,1      ; No increment; need carry flag
5484 JC      SHORT SKIPGET
5485 CALL    GETTHISDRV
5486 ; MSDOS 6.0
5487 JNC     SHORT SKIPGET      ;PM. good drive      ;AN000;
5488
5489 ; 23/03/2024 - Retro DOS v5.0
5490 ;XOR    AH,AH      ;DCR. ax= error code ;AN000;
5491 ;CMP    AX,error_not_DOS_disk ;DCR. is unknown media ? ;AN000;
5492 ;JZ     SHORT SKIPGET      ;DCR. yes, let it go ;AN000;
5493 ;STC
5494 ;DCR.      ;AN000;
5495 mov     ah,0
5496 MOV     [EXTERR],AX      ;PM. invalid drive or Non DOS drive ;AN000;
5497 MOV     WORD [AbsDskErr],201h
5498 SKIPGET:
5499 POP     AX
5500 JC      SHORT GETBP_RET_N ; 15/12/2022
5501 ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
5502 ;jnc    short getbp_t
5503 ;retn
5504 getbp_t:
5505 LES     BP,[THISCDS]
5506 ; 15/12/2022
5507 test    byte [es:bp+curdir.flags+1],curdir_isnet>>8
5508 ; 07/12/2022
5509 ;TEST   WORD [ES:BP+43H],8000H
5510 ;TEST   WORD [ES:BP+curdir.flags],curdir_isnet ; clears carry
5511 JZ      SHORT GETBP_CDS
5512 GETBP_err: ; 04/01/2024
5513 MOV     WORD [EXTERR],error_not_supported ; 32h
5514 STC
5515 GETBP_RET_N:
5516 RETN
5517
5518 GETBP_CDS:
5519 ;LES    BP,[ES:BP+45H]
5520 LES     BP,[ES:BP+curdir.devptr]
5521 ; 04/01/2024 (PCDOS 7.1 IBMDOS.COM)
5522 ;;;
5523 push    ax
5524 ; 25/06/2024
5525 mov     ax,es
5526 or      ax,bp
5527 pop     ax
5528 jz      short GETBP_err ; zero address, error
5529 ;;;
5530 ;-----
5531 GOTDPB:
5532 ; Load THISDPB from ES:BP
5533 MOV     [THISDPB],BP
5534 MOV     [THISDPB+2],ES
5535 RETN
5536
5537 ;BREAK <SYS_RET_OK SYS_RET_ERR CAL_LK ETAB_LK set system call returns>
5538
5539 ;-----
5540 ;
5541 ; Procedure Name : SYS_RETURN
5542 ;
5543 ; These are the general system call exit mechanisms. All internal system
5544 ; calls will transfer (jump) to one of these at the end. Their sole purpose
5545 ; is to set the user's flags and set his AX register for return.
5546 ;-----
5547 ;
5548 ;procedure    SYS_RETURN,NEAR
5549 SYS_RETURN:
5550 ;entry        SYS_RET_OK
5551 SYS_RET_OK:
5552 call    Get_User_Stack
5553 ; turn off user's carry flag
5554 SYS_RET_OK_c1c: ; 25/06/2019
5555 ;;and        word [SI+16h],0FFFEh
5556 ;and        word [SI+user_env.user_F],~f_Carry
5557 ; 25/06/2019
5558 and      byte [SI+user_env.user_F],~f_Carry ; 0FEh
5559 ; 04/01/2024
5560 ;JMP     SHORT DO_RET
5561 DO_RET:
5562 ;MOV     [SI+user_env.user_AX],AX ; Really only sets AH
5563 MOV     [SI],AX
5564 RETN
5565
5566 ;entry        SYS_RET_ERR
5567 SYS_RET_ERR:
5568 XOR     AH,AH      ; hack to allow for smaller error rets
5569 call    ETAB_LK      ; Make sure code is OK, EXTERR gets set
5570 CALL    ErrorMap
5571
5572 ;entry        From_GetSet
5573 From_GetSet:
5574 call    Get_User_Stack
5575 ; signal carry to user
5576 ;;or        word [SI+16h],1
5577 ;OR       word [SI+user_env.user_F],f_Carry

```

```

5578             ; 25/06/2019
5579 00000683 804C1601 or byte [SI+user_env.user_F],f_Carry
5580 00000687 F9 STC ; also, signal internal error
5581             ; 04/01/2024
5582 00000688 EBEB jmp short DO_RET
5583 ;DO_RET:
5584             ;;MOV [SI+user_env.user_AX],AX ; Really only sets AH
5585             ;MOV [SI],AX
5586             ;RETN
5587
5588             ;entry FCB_RET_OK
5589 FCB_RET_OK:
5590             ;entry NO_OP ; obsolete system calls dispatch to here
5591 NO_OP:
5592 0000068A 30C0 XOR AL,AL
5593 0000068C C3 retn
5594
5595             ;entry FCB_RET_ERR
5596 FCB_RET_ERR:
5597 0000068D 30E4 XOR AH,AH
5598 0000068F 36A3[2403] mov [ss:EXTERR],AX
5599 00000693 E80300 CALL ErrorMap
5600 00000696 B0FF MOV AL,-1
5601 00000698 C3 retn
5602
5603             ;entry ErrorMap
5604 ErrorMap:
5605 00000699 56 PUSH SI
5606             ; ERR_TABLE_21 is now in DOSDATA
5607 0000069A BE[DB0D] MOV SI,ERR_TABLE_21
5608             ; SS override for FAILERR and EXTERR
5609 0000069D 36803E[4A03]00 CMP byte [ss:FAILERR],0 ; Check for SPECIAL case.
5610 000006A3 7407 JZ short EXTENDED_NORMAL ; All is OK.
5611             ; Oops, this is the REAL reason
5612             ;mov word [ss:EXTERR],53h
5613 000006A5 36C706[2403]5300 MOV word [ss:EXTERR],error_FAIL_I24
5614 EXTENDED_NORMAL:
5615 000006AC E80200 call CAL_LK ; Set CLASS,ACTION,LOCUS for EXTERR
5616 000006AF 5E POP SI
5617 000006B0 C3 retn
5618
5619             ;EndProc SYS_RETURN
5620
5621 ;-----
5622 ;
5623 ; Procedure Name : CAL_LK
5624 ;
5625 ; Inputs:
5626 ; SI is OFFSET in DOSDATA of CLASS,ACTION,LOCUS Table to use
5627 ; (DS NEED not be DOSDATA)
5628 ; [EXTERR] is set with error
5629 ; Function:
5630 ; Look up and set CLASS ACTION and LOCUS values for GetExtendedError
5631 ; Outputs:
5632 ; [EXTERR_CLASS] set
5633 ; [EXTERR_ACTION] set
5634 ; [EXTERR_LOCUS] set (EXCEPT on certain errors as determined by table)
5635 ; Destroys SI, FLAGS
5636 ;-----
5637 ;
5638 ;
5639 ;procedure CAL_LK,NEAR
5640 CAL_LK:
5641 000006B1 1E PUSH DS
5642 000006B2 50 PUSH AX
5643 000006B3 53 PUSH BX
5644
5645 ;M048 Context DS ; DS:SI -> Table
5646 ;
5647 ; Since this function can be called thru int 2f we shall not assume that SS
5648 ; is DOSDATA
5649
5650 ;getdseg <ds> ; M048: DS:SI -> Table
5651 ; 15/05/2019 - Retro DOS v4.0
5652 000006B4 2E8E1E[0700] mov ds,[cs:DosDSeg]
5653
5654 ; 18/07/2018
5655 ;push ss
5656 ;pop ds
5657
5658 000006B9 8B1E[2403] MOV BX,[EXTERR] ; Get error in BL
5659 TABLK1:
5660 000006BD AC LODSB
5661
5662 000006BE 3CFF CMP AL,0FFH
5663 000006C0 7409 JZ short GOT_VALS ; End of table
5664 000006C2 38D8 CMP AL,BL
5665 000006C4 7405 JZ short GOT_VALS ; Got entry
5666 000006C6 83C603 ADD SI,3 ; Next table entry
5667 ; 15/08/2018
5668 000006C9 EBF2 JMP short TABLK1
5669
5670 GOT_VALS:
5671 000006CB AD LODSW ; AL is CLASS, AH is ACTION
5672
5673 000006CC 80FCFF CMP AH,0FFH
5674 000006CF 7404 JZ short NO_SET_ACT
5675 000006D1 8826[2603] MOV [EXTERR_ACTION],AH ; Set ACTION
5676 NO_SET_ACT:
5677 000006D5 3CFF CMP AL,0FFH
5678 000006D7 7403 JZ short NO_SET_CLS
5679 000006D9 A2[2703] MOV [EXTERR_CLASS],AL ; Set CLASS
5680 NO_SET_CLS:
5681 000006DC AC LODSB ; Get LOCUS
5682
5683 000006DD 3CFF CMP AL,0FFH
5684 000006DF 7403 JZ short NO_SET_LOC
5685 000006E1 A2[2303] MOV [EXTERR_LOCUS],AL
5686 NO_SET_LOC:
5687 000006E4 5B POP BX
5688 000006E5 58 POP AX
5689 000006E6 1F POP DS
5690 000006E7 C3 retn
5691
5692 ;EndProc CAL_LK
5693
5694 ;-----
5695 ;
5696 ; Procedure Name : ETAB_LK
5697 ;
5698 ; Inputs:
5699 ; AX is error code
5700 ; [USER_IN_AX] has AH value of system call involved
5701 ; Function:

```



```

5702 ; Make sure error code is appropriate to this call.
5703 ; Outputs:
5704 ; AX MAY be mapped error code
5705 ; [EXTERR] = Input AX
5706 ; Destroys ONLY AX and FLAGS
5707 ;
5708 ;-----
5709
5710 ;procedure ETAB_LK,NEAR
5711
5712 ETAB_LK: ; 10/08/2018 - Retro DOS v3.0
5713 000006E8 1E    PUSH    DS
5714 000006E9 56    PUSH    SI
5715 000006EA 51    PUSH    CX
5716 000006EB 53    PUSH    BX
5717
5718 ;Context DS                      ; SS is DOSDATA
5719
5720 000006EC 16    push    ss
5721 000006ED 1F    pop     ds
5722
5723 000006EE A3[2403] MOV     [EXTERR],AX      ; Set EXTERR with "real" error
5724
5725 ; I21_MAP_E_TAB is now in DOSCODE
5726 000006F1 BE[0B00] MOV     SI,I21_MAP_E_TAB
5727 000006F4 88C7 MOV     BH,AL           ; Real code to BH
5728 000006F6 8A1E[3B03] MOV     BL,[USER_IN_AX+1] ; Sys call to BL
5729
5730 TABLK2:
5731 ; 15/05/2019 - Retro DOS v4.0
5732 000006FA 2E    CS
5733 000006FB AD    lodsw    ; MSDOS 6.0 (MSDOS 6.21 - MSDOS.SYS, DOSCODE:447Dh)
5734
5735 ; 18/07/2018 - Retro DOS v3.0
5736 ;lodsw    ; IBMDOS.COM (MSDOS 3.3) - Offset 16F7h
5737
5737 000006FC 3CFF    CMP     AL,0FFH        ; End of table?
5738 000006FE 740C    JZ      short NOT_IN_TABLE ; Yes
5739 00000700 38D8    CMP     AL,BL          ; Found call?
5740 00000702 740C    JZ      short GOT_CALL    ; Yes
5741 00000704 86E0    XCHG    AH,AL          ; Count to AL
5742 00000706 30E4    XOR     AH,AH          ; Make word for add
5743 00000708 01C6    ADD     SI,AX          ; Next table entry
5744 0000070A EBEE    JMP     short TABLK2
5745
5746 NOT_IN_TABLE:
5747 0000070C 88F8    MOV     AL,BH          ; Restore original code
5748 0000070E EB0C    JMP     SHORT NO_MAP
5749
5750 GOT_CALL:
5751 00000710 88E1    MOV     CL,AH
5752 00000712 30ED    XOR     CH,CH          ; Count of valid err codes to CX
5753
5754 CHECK_CODE:
5755 ; 15/05/2019 - Retro DOS v4.0
5756 00000714 2E    CS
5757 00000715 AC    lodsb    ; MSDOS 6.0 (MSDOS 6.21 - MSDOS.SYS, DOSCODE:4497h)
5758
5759 ; 18/07/2018
5760 ;lodsb    ; IBMDOS.COM (MSDOS 3.3) - Offset 1710h
5761
5761 00000716 38F8    CMP     AL,BH          ; Code OK?
5762 00000718 7402    JZ      short NO_MAP    ; Yes
5763 0000071A E2F8    LOOP    CHECK_CODE
5764
5765 NO_MAP:
5766 0000071C 30E4    XOR     AH,AH          ; AX is now valid code
5767 0000071E 5B      POP     BX
5768 0000071F 59      POP     CX
5769 00000720 5E      POP     SI
5770 00000721 1F      POP     DS
5771 00000722 C3      retn
5772
5773 ;EndProc ETAB_LK
5774
5775 ; 18/07/2018 - Retro DOS v3.0
5776 ;-----
5777 ; BREAK <DOS 2F Handler and default NET 2F handler>
5778
5779 ;IF installed ; (*)
5780
5781 ;-----
5782 ;
5783 ; Procedure Name : SetBad
5784 ;
5785 ; SetBad sets up info for bad functions
5786 ;
5787 ;-----
5788
5789 SetBad:
5790 ;mov     ax,1
5791 00000723 B80100 MOV     AX,error_invalid_function ; ALL NET REQUESTS get inv func
5792
5793 ; MSDOS 3.3
5794 ;;mov     byte [cs:EXTERR_LOCUS],1
5795 ;MOV     byte [CS:EXTERR_LOCUS],errLOC_Unk
5796
5797 ; set up ds to point to DOSDATA
5798
5799 ; 15/05/2019 - Retro DOS v4.0
5800 00000726 1E    push    ds
5801
5802 ;getdseg <ds>
5803 00000727 2E8E1E[0700] mov     ds,[cs:DosDSeg]
5804
5805 0000072C C606[2303]01 MOV     byte [EXTERR_LOCUS],errLOC_Unk ; 1
5806
5807 00000731 1F    pop     ds      ;hkn; restore ds
5808
5809 STC
5810 00000733 C3      retn
5811
5812 ;-----
5813 ;
5814 ; Procedure Name : BadCall
5815 ;
5816 ; BadCall is the initial routine for bad function calls
5817 ;
5818 ;-----
5819
5820 BadCall:
5821 00000734 E8ECFF call    SetBad
5822 00000737 CB      retf
5823
5824 ;-----
5825 ;

```

```

5826 ; OKCall always sets carry to off.
5827 ;
5828 ;-----
5829
5830 OKCall:
5831 00000738 F8 CLC
5832 00000739 CB retf
5833
5834 ;-----
5835 ;
5836 ; Procedure Name : INT2F
5837 ;
5838 ; INT 2F handler works as follows:
5839 ; PUSH AX
5840 ; MOV AX,multiplex:function
5841 ; INT 2F
5842 ; POP ...
5843 ; The handler itself needs to make the AX available for the various routines.
5844 ;-----
5845 ;
5846 ;
5847 ; 15/05/2019 - Retro DOS v4.0
5848
5849 ;KERNEL_SEGMENT equ 70h
5850 ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
5851 DOSBIODATASEG equ 70h
5852
5853 ; retrodos4.s - offset in BIOSDATA
5854 bios_i2f equ 5
5855
5856 ;PUBLIC Int2F
5857 ;INT2F PROC FAR
5858
5859 ; 15/05/2019
5860 ; DOSCODE:44BDh (MSDOS 6.21, MSDOS.SYS)
5861
5862 ; 04/11/2022
5863 ; DOSCODE:44B0h (MSDOS 5.0, MSDOS.SYS)
5864
5865 ; 15/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
5866 ; 18/07/2018 - Retro DOS v3.0
5867 ; 05/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
5868 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:45DAh
5869
5870 INT2F:
5871 ; Offset 172Fh in IBMDOS.COM (MSDOS 3.3), 1987
5872 INT2FNT:
5873 ; ASSUME CS:DOSCODE,DS:NOTHING,ES:NOTHING,SS:NOTHING
5874 STI
5875 ; cmp ah,11h
5876 0000073B 80FC11 CMP AH,MultNET
5877 0000073E 750A JNZ short INT2FSHR
5878
5879 TestInstall:
5880 OR AL,AL
5881 JZ short Leave2F
5882
5883 BadFunc:
5884 CALL SetBad
5885
5886 ;entry Leave2F
5887 Leave2F:
5888 RETF 2 ; long return + clear flags off stack
5889
5890 INT2FSHR:
5891 ; cmp ah,10h
5892 CMP AH,MultSHARE ; is this a share request
5893 JZ short TestInstall ; yes, check for installation
5894
5895 INT2FNLS:
5896 ; cmp ah,14h
5897 CMP AH,NLSFUNC ; is this a DOS 3.3 NLSFUNC request
5898 JZ short TestInstall ; yes check for installation
5899
5900 INT2FDOS:
5901 ; ASSUME CS:DOSCODE,DS:NOTHING,ES:NOTHING,SS:NOTHING
5902
5903 ; 18/07/2018
5904 ; MSDOS 3.3
5905 ; cmp ah,12h
5906 CMP AH,MultDOS
5907 JZ short DispatchDOS
5908 ;ret
5909
5910 ; 15/05/2019
5911 ; MSDOS 6.0
5912 ; cmp ah,12h ; 07/12/2022
5913 CMP AH,MultDOS
5914 JNZ short check_win ;check if win386 broadcast
5915 jmp DispatchDOS
5916
5917 ; .... win386 ....
5918
5919 check_win:
5920 ; cmp ah,16h
5921 cmp ah,Multwin386 ; Is this a broadcast from win386?
5922 je short Win386_Msg
5923
5924 ; M044
5925 ; Check if the callout is from winoldap indicating swapping out or in
5926 ; of windows. If so, do special action of going and saving last para
5927 ; of the windows memory arena which winoldap does not save due to a
5928 ; bug
5929
5930 cmp ah,WINOLDAP ; 46h ; from winoldap?
5931 jne short next_i2f ; no, chain on
5932 ; 15/12/2022
5933 jmp winold_swap ; yes, do desired action
5934 je short winold_swap
5935 jmp next_i2f
5936
5937 ; 15/12/2022
5938 ;next_i2f:
5939 ; jmp bios_i2f
5940 ; jmp far ptr 70h:5 ; MSDOS 6.21 (MSDOS.SYS, DOSCODE:44F1h)
5941 ; jmp KERNEL_SEGMENT:bios_i2f
5942 ; 04/11/2022
5943 ; jmp DOSBIODATASEG:bios_i2f
5944
5945 ; IRET ; This assume that we are at the head
5946 ; of the list
5947
5948 ;INT2F ENDP
5949
5950 ; 15/05/2019 - Retro DOS v4.0
5951
5952 ; We have received a message from win386. There are three possible
5953 ; messages we could get from win386:
5954 ;
5955 ;
5956 ; Init - for this, we set the Iswin386 flag and return a pointer

```



```

5950 ; to the win386 startup info structure.
5951 ; Exit - for this, we clear the Iswin386 flag.
5952 ; DOSMGR query - for this, we need to indicate that instance data
5953 ; has already been handled. this is indicated by setting
5954 ; CX to a non-zero value.
5955
5956 win386_Msg:
5957 00000769 1E push ds
5958
5959 ;getdseg <DS> ; ds is DOSDATA
5960 0000076A 2E8E1E[0700] mov ds,[cs:DosDSeg]
5961
5962 ; For WIN386 2.xx instance data
5963
5964 0000076F 3C03 cmp al,3 ; win386 2.xx instance data call?
5965 00000771 7503 jne short win386_Msg_exit
5966 00000773 E94801 jmp Oldwin386Init ; yes, return instance data
5967
5968 win386_Msg_exit:
5969 00000776 3C06 cmp al,win386_Exit ; 6 ; is it an exit call?
5970 00000778 7503 jne short win386_Msg_devcall
5971 0000077A E94A01 jmp win386_Leaving
5972
5973 win386_Msg_devcall:
5974 0000077D 3C07 cmp al,win386_Devcall ; 7 ; is it call from DOSMGR?
5975 0000077F 7503 jne short win386_Msg_init
5976 00000781 E97E01 jmp win386_Query
5977
5978 win386_Msg_init:
5979 00000784 3C05 cmp al,win386_Init ; 5 ; is it an init call?
5980 00000786 7403 je short win386_Starting
5981 00000788 E90001 jmp win_nexti2f ; no, return
5982
5983 win386_Starting:
5984 ; 05/01/2024 - Retro DOS v5.0
5985 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:4630h
5986 ;;;
5987 0000078B 50 push ax
5988 0000078C 51 push cx
5989 0000078D 57 push di
5990 0000078E 06 push es
5991 0000078F 1E push ds
5992 00000790 07 pop es
5993 00000791 BF[FB01] mov di,INBUF
5994 00000794 B90500 mov cx,5
5995 00000797 FC cld
5996
5997 win386_s_floop:
5998 00000798 B8434F mov ax,'CO'; 4F43h
5999 0000079B AB stosw
6000 0000079C B84E20 mov ax,'N '; 204Eh
6001 0000079F AB stosw
6002 000007A0 83C737 add di,55 ; (write 5 times 'CON ' and skip 55 bytes between them)
6003 ; what for ?
6004 loop win386_s_floop
6005 pop es
6006 pop di
6007 pop cx
6008 pop ax
6009 ;;;
6010 ; 17/12/2022
6011 test dl,1
6012 ;test dx,1 ; is this really win386?
6013 jz short win386_vchk ; YES! go and handle it
6014 jmp win_nexti2f ; NO! It's win 286 dos extender! M002
6015
6016 win386_vchk:
6017 ; M018 -- start of block changes
6018 ; The VxD needs to be loaded only for win 3.0. If version is greater
6019 ; than 030Ah, we skip the VxD presence check
6020
6021 ;M067 -- Begin changes
6022 ; If win 3.0 is run, the VxD ptr has been initialized. If win 3.1 is now
6023 ; run, it tries to unnecessarily load the VxD even though it is not needed.
6024 ; So, we null out the VxD ptr before the check.
6025
6026 ;mov word [win386_Info+6],0
6027 mov word [win386_Info+win386_SIS.Virt_Dev_File_Ptr],0
6028 ;mov word [win386_Info+8],0
6029 mov word [win386_Info+win386_SIS.Virt_Dev_File_Ptr+2],0
6030
6031 ;M067 -- End changes
6032
6033 ;ifdef JAPAN
6034 ; cmp di,0300h ; version >= 300 i.e 3.10 ;M037
6035 ;else
6036 ; cmp di,030Ah ; version >= 30a i.e 3.10 ;M037
6037 ; 05/01/2024 - PC DOS 7.1 - Retro DOS v5.0
6038 cmp di,0400h ; version >= 400 ; 05/01/2024
6039 ;endif
6040 ;jae novxd31 ; yes, VxD not needed ;M037
6041 jnb short win386_vxd
6042 jmp novxd31
6043
6044 ; 15/12/2022
6045 winold_swap:
6046 push ds
6047 push es
6048 push si
6049 push di
6050 push cx
6051
6052 ;getdseg <ds> ;ds = DOSDATA
6053 mov ds,[cs:DosDSeg]
6054
6055 cmp al,1 ;swap windows out call
6056 jne short swapin ;no, check if Swap in call
6057 call getwinlast
6058 push ds
6059 pop es
6060 mov ds,si ;ds = memory arena of windows
6061 xor si,si
6062 ;mov di,6 ; 05/01/2024
6063 mov di,winoldPatch1 ; 6
6064 mov cx,8
6065 cld
6066 ;push cx
6067 rep movsb ;save first 8 bytes
6068 ;pop cx
6069 ; 25/06/2024
6070 mov cl,8
6071 ;mov di,1176h ; 05/01/2024
6072 mov di,winoldPatch2 ; 1176h
6073 rep movsb ;save next 8 bytes
6074 jmp short winold_done
6075
6076 swapin:
6077 cmp al,2 ;swap windows in call?
6078 jne short winold_done ;no, something else, pass it on

```

```

6074 000007F3 E86901      call    getwinlast
6075 000007F6 8EC6        mov     es,si
6076 000007F8 31FF        xor     di,di
6077 000007FA BE[0600]     mov     si,winoldPatch1
6078 000007FD B90800     mov     cx,8
6079 00000800 FC          cld
6080                      ;push    cx
6081 00000801 F3A4        rep     movsb          ;restore first 8 bytes
6082                      ;pop     cx
6083                      ; 25/06/2024
6084 00000803 B108        mov     cl,8
6085 00000805 BE[7611]     mov     si,winoldPatch2
6086 00000808 F3A4        rep     movsb          ;restore next 8 bytes
6087 winold_done:
6088 0000080A 59          pop     cx
6089 0000080B 5F          pop     di
6090 0000080C 5E          pop     si
6091 0000080D 07          pop     es
6092 0000080E 1F          pop     ds
6093 0000080F EB7B        jmp     short next_i2f    ;chain on
6094                      ; 15/12/2022
6095                      ;jmp     next_i2f
6096
6097 win386_vxd:
6098 00000811 50          push    ax
6099 00000812 53          push    bx
6100 00000813 51          push    cx
6101 00000814 52          push    dx
6102 00000815 56          push    si
6103 00000816 57          push    di          ; save regs !!dont change order!!
6104
6105 00000817 8B1E[8C00]   mov     bx,[UMB_HEAD]    ; M062 - Start
6106 0000081B 83FBFF   cmp     bx,0FFFFh        ; Q: have umbs been initialized
6107 0000081E 741F        je      short Vxd31       ; N: continue
6108                      ; Y: save arena associated with
6109                      ;     umb_head
6110
6111 00000820 C606[DA0D]01  mov     byte [UmbSaveFlag],1 ; indicate that we're saving
6112                      ; umb_arena
6113 00000825 1E          push    ds
6114 00000826 06          push    es
6115
6116                      ;mov     ax,ds
6117                      ;mov     es,ax          ; es -> dosdata
6118                      ; 05/01/2024
6119 00000827 1E          push    ds
6120 00000828 07          pop     es
6121
6122 00000829 8EDB        mov     ds,bx
6123 0000082B 31F6        xor     si,si          ; ds:si -> umb_head
6124
6125                      ; 05/01/2024 PC DOS 7.1
6126                      ;;;
6127                      ;cld    ; not necessary (XOR already clears CF)
6128
6129 restore_umbhead:      ; !! PC DOS 7.1 bug !!
6130                      ; jump from 'win386_Leaving' here was/is wrong
6131                      ; (DI and SI would be reversed for 'win386_Leaving')
6132                      ; Erdogan Tan - 05/01/2024
6133                      ;;;
6134
6135 0000082D FC          cld
6136
6137 0000082E BF[2F11]   mov     di,UmbSave1
6138 00000831 B90B00     mov     cx,11
6139 00000834 F3A4        rep     movsb
6140
6141 00000836 BF[D50D]   mov     di,UmbSave2
6142                      ;mov     cx,5
6143                      ; 18/12/2022
6144 00000839 B105        mov     cl,5
6145 0000083B F3A4        rep     movsb
6146
6147                      ; 05/01/2024 PC DOS 7.1
6148                      ;;;
6149                      ;jnb     short restore_umbhead_c ; (not jumped from 'win386_Leaving')
6150                      ;jmp     restore_umbhead_ok ; (jumped from 'win386_Leaving' just after 'stc')
6151 ;restore_umbhead_c:
6152                      ;;;
6153
6154 0000083D 07          pop     es
6155 0000083E 1F          pop     ds          ; M062 - End
6156
6157 vxd31:
6158                      ;test    byte [DOS_FLAG],2
6159 0000083F F606[8600]02  test    byte [DOS_FLAG],SUPPRESS_WINA20 ; M066
6160 00000844 7408        jz      short Dont_Supress ; M066
6161 00000846 5F          pop     di          ; M066
6162 00000847 5E          pop     si          ; M066
6163 00000848 5A          pop     dx          ; M066
6164 00000849 59          pop     cx          ; M066
6165 0000084A 5B          pop     bx          ; M066
6166 0000084B 58          pop     ax          ; M066
6167 0000084C EB55        jmp     short noVxd31    ; M066
6168
6169                      ; We check here if the vxd is available in the root of the boot drive.
6170                      ; We do an extended open to suppress any error messages
6171
6172 Dont_Supress:
6173 0000084E A0[6900]   mov     al,[BOOTDRIVE]
6174 00000851 0440        add     al,'A' - 1      ; get drive letter
6175 00000853 A2[F812]   mov     [VxDpath],al    ; path is root of bootdrive
6176                      ;mov     ah,ExtOpen ;6ch ; extended open
6177                      ;mov     al,0 ; no extended attributes
6178                      ; 18/12/2022
6179 00000856 B8006C   mov     ax,ExtOpen<<8 ; 6C00h
6180 00000859 BB8020   mov     bx,2080h        ; read access, compatibility mode
6181                      ; no inherit, suppress crit err
6182 0000085C B90700   mov     cx,7            ; hidden,system,read-only attr
6183                      ;05/01/2024
6184                      ;inc     dx ; dx bit 0 = 1 ; fail if file does not exist
6185 0000085F BA0100   mov     dx,1            ; fail if file does not exist
6186
6187 00000862 BE[F812]   mov     si,VxDpath ; "c:\\wina20.386"
6188                      ; path of vxd file
6189 00000865 BFFFFF   mov     di,0FFFFh       ; no extended attributes
6190
6191 00000868 CD21        int     21h            ; do extended open
6192
6193 0000086A 5F          pop     di
6194 0000086B 5E          pop     si
6195 0000086C 5A          pop     dx
6196 0000086D 59          pop     cx
6197

```

```

6198 0000086E 7321      jnc     short VxDthere      ; we found the VxD, go ahead
6199
6200      ; We could not find the VxD. Cannot let windows load. Return cx != 0
6201      ; to indicate error to windows after displaying message to user that
6202      ; VxD needs to be present to run windows in enhanced mode.
6203
6204 00000870 52      push    dx
6205 00000871 1E      push    ds
6206 00000872 56      push    si
6207 00000873 BE[300A]    mov     si,NovxDErrMsg
6208 00000876 0E      push    cs
6209 00000877 1F      pop     ds
6210 00000878 B96300    mov     cx,VxDMesLen ; 99      ;
6211 0000087B B402      mov     ah,2          ; write char to console
6212 0000087D FC      cld
6213
VxDlp:
6214 0000087E AC      lodsb
6215 0000087F 86D0      xchg    dl,al          ; get char in dl
6216 00000881 CD21      int     21h
6217 00000883 E2F9      loop   VxDlp
6218
6219 00000885 5E      pop     si
6220 00000886 1F      pop     ds
6221 00000887 5A      pop     dx
6222 00000888 5B      pop     bx
6223 00000889 58      pop     ax            ;all registers restored
6224 0000088A 41      inc     cx            ;cx != 0 to indicate error
6225      ; 15/12/2022
6226      ; jmp     win_nexti2f      ;chain on
6227      ; jmp     short win_nexti2f
6228
6229      ; 15/12/2022
6230
win_nexti2f:
6231 0000088B 1F      pop     ds
6232      ; jmp     short next_i2f      ; go to BIOS i2f handler
6233      ; 15/12/2022
6234
next_i2f:
6235      ; jmp     bios_i2f
6236      ; jmp     far ptr 70h:5 ; MSDOS 6.21 (MSDOS.SYS, DOSCODE:44F1h)
6237      ; jmp     KERNEL_SEGMENT:bios_i2f
6238      ; 04/11/2022
6239 0000088C EA05007000    jmp     DOSBIODATASEG:bios_i2f
6240
6241
VxDthere:
6242 00000891 89C3      mov     bx,ax
6243 00000893 B43E      mov     ah,CLOSE ; 3Eh
6244 00000895 CD21      int     21h          ;close the file
6245
6246      ; Update the VxD ptr in the instance data structure with path to VxD
6247
6248      ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
6249      ; mov     bx,win386_Info
6250      ; mov     word [bx+win386_SIS.Virt_Dev_File_Ptr],VxDpath
6251      ; mov     word [bx+win386_SIS.Virt_Dev_File_Ptr+2],ds
6252      ; 15/12/2022
6253 00000897 C706[E70E][F812]    mov     word [win386_Info+win386_SIS.Virt_Dev_File_Ptr],VxDpath
6254 0000089D 8C1E[E90E]    mov     word [win386_Info+win386_SIS.Virt_Dev_File_Ptr+2],ds
6255
6256 000008A1 5B      pop     bx
6257 000008A2 58      pop     ax
6258
novxD31:
6259      ; M018; End of block changes
6260
6261 000008A3 800E[5B0F]01    or      byte [IsWin386],1      ; Indicate WIN386 present
6262 000008A8 800E[650D]01    or      byte [redir_patch],1    ; Enable critical sections; M002
6263
6264      ; M002;
6265      ; Save the previous es:bx (instance data ptr) into our instance table
6266
6267 000008AD 52      push    dx            ; M002
6268 000008AE 89DA      mov     dx,bx          ; M002
6269      ; point ES:BX to win386_Info ; M002
6270 000008B0 BB[E10E]    mov     bx,win386_Info
6271 000008B3 895702    mov     [bx+2],dx      ; M002
6272 000008B6 8C4704    mov     [bx+4],es      ; M002
6273 000008B9 5A      pop     dx            ; M002
6274 000008BA 1E      push    ds            ; M002
6275 000008BB 07      pop     es            ; M002
6276      ; jmp     win_nexti2f      ; M002
6277      ; 15/12/2022
6278 000008BC EBCD      jmp     short win_nexti2f
6279
6280      ; 15/12/2022
6281      ; Code to return win386 2.xx instance table
6282
oldwin386Init:
6283 000008BE 58      pop     ax            ; discard ds pushed on stack
6284 000008BF BE[FC10]    mov     si,OldInstanceJunk
6285      ; ds:si = instance table
6286 000008C2 B84852    mov     ax,5248h ; 'HR'      ; indicate instance data present
6287      ; jmp     next_i2f
6288      ; 15/12/2022
6289 000008C5 EBC5      jmp     short next_i2f
6290
6291
win386_Leaving:
6292      ; 15/12/2022
6293 000008C7 F6C201    test    dl,1
6294      ; test    dx,1          ; is this really win386?
6295      ; jz      short win386_Leaving_c
6296      ; jmp     win_nexti2f      ; NO! It's win 286 dos extender! M002
6297      ; 15/12/2022
6298 000008CA 75BF      jnz     short win_nexti2f
6299
6300
win386_Leaving_c:
6301      ; M062 - Start
6302 000008CC 803E[DA0D]01    cmp     byte [UmbSaveFlag],1    ; Q: was umb_arena saved at win start
6303      ; up.
6304 000008D1 7523      jne     short noub     ; N: not saved
6305 000008D3 C606[DA0D]00    mov     byte [UmbSaveFlag],0    ; Y: clear UmbSaveFlag and restore
6306      ; previously saved umb_head
6307
6308      ; 05/01/2024 - PC DOS 7.1 (has BUG here!)
6309      ; PC DOS 7.1 IBMDOS.COM - DOSCODE:472Bh
6310      ;;;
6311      ; push    ax ; (not necessary)
6312      ; push    es
6313      ; push    cx
6314      ; push    si
6315      ; push    di
6316      ; mov     es,[UMB_HEAD]
6317      ; xor     di,di
6318      ; stc
6319      ; jmp     restore_umbhead      ; !! PC DOS 7.1 bug !! IBMDOS.COM - DOSCODE:4737h
6320      ; (jumped code does not restore umbhead,
6321      ; MSDOS 6.22 "win386_Leaving" code was/is correct,

```

```

6322 ; ; modified code -in PC DOS 7.1 IBMDOS.COM- is wrong)
6323 ; ; ;
6324 ; ; ;
6325 ; ; ;
6326 ; ; ;
6327 ; 05/01/2024 - Retro DOS v5.0 (Modified PC DOS 7.1)
6328 ;push ax ; (not necessary)
6329 000008D8 06 push es
6330 000008D9 51 push cx
6331 000008DA 56 push si
6332 000008DB 57 push di
6333 ;
6334 ;mov ax,[UMB_HEAD]
6335 ;mov es,ax
6336 ; 05/01/2024
6337 000008DC 8E06[8C00] mov es,[UMB_HEAD]
6338 000008E0 31FF xor di,di ; es:di -> umb_head
6339 ;
6340 000008E2 FC cld
6341 ;
6342 000008E3 BE[2F11] mov si,UmbSave1
6343 000008E6 B90B00 mov cx,11
6344 000008E9 F3A4 rep movsb
6345 000008EB BE[D50D] mov si,UmbSave2
6346 ;mov cx,5
6347 ; 18/12/2022
6348 000008EE B105 mov cl,5
6349 000008F0 F3A4 rep movsb
6350 ;
6351 restore_umbhead_ok: ; 05/01/2024 - PC DOS 7.1 IBMDOS.COM - DOSCODE:473Ah
6352 000008F2 5F pop di
6353 000008F3 5E pop si
6354 000008F4 59 pop cx
6355 000008F5 07 pop es
6356 ; 05/01/2024
6357 ;pop ax
6358 ;
6359 ; and byte [IsWin386],0 ; win386 is gone
6360 ; 26/06/2024
6361 000008F6 8026[5B0F]FE and byte [IsWin386],0FEh ; ~1
6362 000008FB 8026[650D]00 and byte [redir_patch],0 ; Disable critical sections ; M002
6363 00000900 EB89 jmp short win_nexti2f
6364 ;
6365 ; ; 15/12/2022
6366 ; ; Code to return Win386 2.xx instance table
6367 ; OldWin386Init:
6368 ; pop ax ; discard ds pushed on stack
6369 ; mov si,OldInstanceJunk
6370 ; ;
6371 ; mov ax,5248h ; 'RH' ; ds:si = instance table
6372 ; jmp next_i2f ; indicate instance data present
6373 ; ; 15/12/2022
6374 ; jmp short _next_i2f
6375 ;
6376 win386_Query:
6377 00000902 83FB15 cmp bx,win386_DOSMGR ; 15h; is this from DOSMGR?
6378 00000905 7584 jne short win_nexti2f ; no, ignore it & chain to next
6379 00000907 09C9 or cx,cx ; is it an instance query?
6380 00000909 7508 jnz short dosmgr_func ; no, some DOSMGR query
6381 0000090B 41 inc cx ; indicate that data is instanced
6382 ;
6383 ; M001; we were previously returning a null ptr in es:bx. This will not work.
6384 ; M001; WIN386 needs a ptr to a table in es:bx with the following offsets:
6385 ; M001;
6386 ; M001; OFFSETS STRUC
6387 ; M001; Major_version db ?
6388 ; M001; Minor_version db ?
6389 ; M001; SavedS dw ?
6390 ; M001; SaveBX dw ?
6391 ; M001; Indos dw ?
6392 ; M001; User_id dw ?
6393 ; M001; CritPatch dw ?
6394 ; M001; OFFSETS ENDS
6395 ; M001;
6396 ; M001; User_Id is the only variable really important for proper functioning
6397 ; M001; of win386. The other variables are used at init time to patch stuff
6398 ; M001; out. In DOS 5.0, we do the patching ourselves. But we still need to
6399 ; M001; pass this table because win386 depends on this table to get the
6400 ; M001; User_Id offset.
6401 ; M001;
6402 0000090C BB[4D0F] mov bx,win386_DOSVars ; M001
6403 0000090F 1E push ds ; M001
6404 00000910 07 pop es ; es:bx points at offset table ; M001
6405 00000911 EB40 jmp short PopIret ; M001
6406 ;
6407 ; 15/12/2022
6408 ; ; Code to return win386 2.xx instance table
6409 ; OldWin386Init:
6410 ; pop ax ; discard ds pushed on stack
6411 ; mov si,OldInstanceJunk
6412 ; ;
6413 ; mov ax,5248h ; 'RH' ; ds:si = instance table
6414 ; jmp next_i2f ; indicate instance data present
6415 ; ; 15/12/2022
6416 ; jmp short _next_i2f
6417 ;
6418 dosmgr_func:
6419 00000913 49 dec cx
6420 00000914 7435 jz short win386_patch ; call to patch DOS
6421 00000916 49 dec cx
6422 00000917 743A jz short PopIret ; remove DOS patches, ignore
6423 00000919 49 dec cx
6424 0000091A 7439 jz short win386_size ; get size of DOS data structures
6425 0000091C 49 dec cx
6426 0000091D 7428 jz short win386_inst ; instance more data
6427 ;dec cx
6428 ;jnz short PopIret ; no functions above this
6429 ; 05/01/2024 (PC DOS 7.1 IBMDOS.COM DOSCODE:4771h)
6430 0000091F E232 loop PopIret
6431 ;
6432 ; Get DOS device driver size -- es:di points at device driver header
6433 ; In DOS 4.x, the para before the device header contains an arena
6434 ; header for the driver.
6435 ;
6436 00000921 8CC0 mov ax,es ; ax = device header segment
6437 ;
6438 ; We check to see if we have a memory arena for this device driver.
6439 ; The way to do this would be to look at the previous para to see if
6440 ; it has a 'D' marking it as an arena and also see if the owner-field
6441 ; in the arena is the same as the device header segment. These two
6442 ; checks together should take care of all cases
6443 ;
6444 00000923 48 dec ax ; get arena header
6445 00000924 06 push es

```

```

6446 00000925 8EC0      mov     es,ax          ; arena header for device driver
6447
6448 00000927 26803D44    cmp     byte [es:di],'D'    ; is it a device arena?
6449 0000092B 7517      jnz     short cantsize     ; no, cant size this driver
6450 0000092D 40        inc     ax                ; get back device header segment
6451 0000092E 26394501    cmp     [es:di+1],ax        ; owner field pointing at driver?
6452 00000932 7510      jnz     short cantsize     ; no, not a proper arena
6453
6454 00000934 268B4503    mov     ax,[es:di+3]        ; get arena size in paras
6455 00000938 07        pop     es
6456
6457      ; We have to multiply by 16 to get the number of bytes in (bx:cx)
6458      ; Speed is not critical and so we choose the shortest method
6459      ; -- use "mul"
6460
6461 00000939 BB1000      mov     bx,16
6462 0000093C F7E3      mul     bx
6463 0000093E 89C1      mov     cx,ax
6464 00000940 89D3      mov     bx,dx
6465 00000942 EB09      jmp     short win386_done   ; return with device driver size
6466
cantsize:
6467      pop     es
6468 00000945 31C0      xor     ax,ax
6469
win386_inst:
6470      xor     dx,dx          ; 05/01/2024
6471 00000949 EB08      jmp     short PopIret      ; ask DOSMGR to use its methods
6472      ; return
6473
win386_patch:
6474      ; dx contains bits marking the patches to be applied. We return
6475      ; the field with all bits set to indicate that all patches have been
6476      ; done
6477
6478 0000094B 89D3      mov     bx,dx              ; move patch bitfield to bx
6479      jmp     short win386_done ; done, return
6480      ; 15/12/2022
6481      ; 15/12/2022
6482
win386_done:
6483      mov     ax,WIN_OP_DONE    ; 0B97Ch
6484 00000950 BAAB2      mov     dx,DOSMGR_OP_DONE    ; 0A2ABh
6485
PopIret:
6486      pop     ds
6487 00000954 CF        iret
6488
6489
win386_size:
6490      ; Return the size of DOS data structures -- currently only CDS size
6491
6492      ; 17/12/2022
6493 00000955 F6C201    test    dl,1
6494      ;test    dx,1          ; check for CDS size bit
6495 00000958 74F9      jz      short PopIret      ; no, unknown structure -- return
6496
6497      mov     cx,curdirLen      ; 88
6498 0000095D EBEE      jmp     short win386_done   ; return with the size
6499
6500      ; 05/01/2024
6501      %if 0
6502
win386_inst:
6503      ; WIN386 check to see if DOS has identified the CDS,SFT and device
6504      ; chain as instance data. Currently, we let the WIN386 DOSMGR handle
6505      ; this by returning a status of not previously instanced. The basic
6506      ; structure of these things have not changed and so the current
6507      ; DOSMGR code should be able to work it out
6508
6509      xor     dx,dx            ; make sure dx has a not done value
6510      jmp     short PopIret    ; skip done indication
6511      %endif
6512
6513      ; 15/12/2022
6514
win386_done:
6515      mov     ax,WIN_OP_DONE    ; 0B97Ch
6516      mov     dx,DOSMGR_OP_DONE ; 0A2ABh
6517
PopIret:
6518      pop     ds
6519      iret                    ; return back up the chain
6520
6521      ; 15/12/2022
6522
win_nexti2f:
6523      pop     ds
6524      jmp     next_i2f         ; go to BIOS i2f handler
6525
6526
;End WIN386 support
6527
6528      ; 15/05/2019
6529
6530      ;M044; Start of changes
6531      ; winoldap has a bug in that its calculations for the windows memory image
6532      ; to save is off by 1 para. This para can happen to be a windows arena if the
6533      ; DOS top of memory happens to be at an odd boundary (as is the case when
6534      ; UMBS are present). This is because Windows builds its arenas only at even
6535      ; para boundaries. This arena now gets trashed when windows is swapped back
6536      ; in leading to a crash. winoldap issues callouts when it swaps windows out
6537      ; and back in. we sit on these callouts. On the windows swapout, we save the
6538      ; last para of the windows memory block and then restore this para on the
6539      ; windows swapin callout.
6540
6541
getwinlast:
6542      ; 07/12/2022
6543 0000095F 8B36[3003]    mov     si,[CurrentPDB]
6544 00000963 4E        dec     si
6545 00000964 8EC6      mov     es,si
6546 00000966 2603360300    add     si,[es:3]
6547 0000096B C3        retn
6548
6549      ; 15/12/2022
6550      %if 0
6551
winold_swap:
6552      push    ds
6553      push    es
6554      push    si
6555      push    di
6556      push    cx
6557
6558      ;getdseg <ds>          ;ds = DOSDATA
6559      mov     ds,[cs:DosDseg]
6560
6561      cmp     al,1            ;swap windows out call
6562      jne     short swapin    ;no, check if Swap in call
6563      call    getwinlast
6564      push    ds
6565      pop     es
6566      mov     ds,si            ;ds = memory arena of Windows
6567      xor     si,si
6568      mov     di,winoldPatch1
6569      mov     cx,8

```

```

6570      cld
6571      push    cx
6572      rep     movsb          ;save first 8 bytes
6573      pop     cx
6574      mov     di,winoldPatch2
6575      rep     movsb          ;save next 8 bytes
6576      jmp     short winold_done
6577  swapin:
6578      cmp     al,2           ;swap windows in call?
6579      jne     short winold_done ;no, something else, pass it on
6580      call    getwinlast
6581      mov     es,si
6582      xor     di,di
6583      mov     si,winoldPatch1
6584      mov     cx,8
6585      cld
6586      push    cx
6587      rep     movsb          ;restore first 8 bytes
6588      pop     cx
6589      mov     si,winoldPatch2
6590      rep     movsb          ;restore next 8 bytes
6591  winold_done:
6592      pop     cx
6593      pop     di
6594      pop     si
6595      pop     es
6596      pop     ds
6597      jmp     next_i2f       ;chain on
6598
6599  %endif
6600
6601  ;M044; End of changes
6602
6603  ; -----
6604  ; 06/01/2024 - Retro DOS v5.0 (Modified PCDS 7.1 IBMDOS.COM/MSDOS.SYS)
6605  ; PCDS 7.1 IBMDOS.COM - DOSCODE:4811h
6606
6607      ; INT 2Fh AX=1231h
6608      ; Windows95 - SET/CLEAR "REPORT WINDOWS TO DOS PROGRAMS" FLAG
6609      ; (Ref: Ralf Brown's Interrupt List)
6610  int_2Fh_1231h:
6611      0000096C 1E      push    ds
6612      0000096D 2E8E1E[0700]  mov     ds,[cs:DosDSeg]
6613      00000972 31C0      xor     ax,ax ; 0
6614      00000974 08D2      or      dl,dl
6615      00000976 7507      jnz     short not_1231_d1_0
6616      00000978 C606[5C0F]01  mov     byte [IsWin386+1],1 ; set byte after "ISWIN386" to 01h
6617      0000097D EB1A      jmp     short int_2f_1231h_retn
6618
6619      ;nop
6620
6621  not_1231_d1_0:
6622      0000097F 80FA01      cmp     dl,1
6623      00000982 7507      jne     short not_1231_d1_1 ; clear "ISWIN386" bit 1
6624      00000984 800E[5B0F]02  or      byte [IsWin386],2 ; set "ISWIN386" bit 1
6625      00000989 EB0E      jmp     short int_2f_1231h_retn
6626
6627      ;nop
6628
6629  not_1231_d1_1:
6630      0000098B 80FA02      cmp     dl,2
6631      0000098E 7507      jne     short not_1231_d1_2
6632      00000990 8026[5B0F]FD  and     byte [IsWin386],0FDh ; clear bit 1
6633      00000995 EB02      jmp     short int_2f_1231h_retn
6634  not_1231_d1_2:
6635      00000997 40          inc     ax ; return error, ax = 1
6636      00000998 F9          stc
6637  int_2f_1231h_retn:
6638      00000999 1F          pop     ds
6639      0000099A C3          retn
6640
6641  ; -----
6642
6643  ; 15/05/2019
6644
6645  ; 06/01/2024 - Retro DOS v5.0
6646  ; PCDS 7.1 IBMDOS.COM - DOSCODE:4842h
6647
6648  DispatchDOS:
6649      0000099B 2EFF36[D401]  PUSH    word [CS:F00] ; push return address
6650      000009A0 2EFF36[D601]  PUSH    word [CS:DTab] ; push table address
6651      000009A5 50          PUSH    AX ; push index
6652      000009A6 55          PUSH    BP
6653      000009A7 89E5      MOV     BP,SP
6654      ; stack looks like:
6655      ; 0 BP
6656      ; 2 DISPATCH
6657      ; 4 TABLE
6658      ; 6 RETURN
6659      ; 8 LONG-RETURN
6660      ; C FLAGS
6661      ; E AX
6662
6663      MOV     AX,[BP+0Eh] ; get AX value
6664      POP     BP
6665      call    TableDispatch
6666      JMP     BadFunc ; return indicates invalid function
6667
6668  INT2F_etcetera:
6669      ;entry DosGetGroup
6670  DosGetGroup:
6671      ; MSDOS 3.3
6672      ;push cs
6673      ;pop ds
6674      ;retn
6675
6676      ; MSDOS 6.0
6677  ;SR; Cannot use CS now
6678      ;
6679      ; PUSH CS
6680      ; POP DS
6681
6682      ; 04/11/2022
6683      ; (MSDOS 5.0 MSDOS.SYS - DOSCODE:46FBh)
6684
6685      ;getdseg <ds>
6686      000009B3 2E8E1E[0700]  mov     ds,[cs:DosDSeg]
6687      000009B8 C3          retn
6688
6689      ;entry DOSInstall
6690  DOSInstall:
6691      000009B9 B0FF      MOV     AL,0FFh
6692      000009BB C3          retn
6693

```



```

6694 ;ENDIF ; (*)
6695
6696
6697 ; 15/05/2019 - Retro DOS v4.0
6698 ; 06/01/2024 - Retro DOS v5.0
6699
6700 ;-----
6701 ;
6702 ; Procedure Name : RW32_CONVERT
6703 ;
6704 ; Input: same as ABSDRD and ABSDWRT
6705 ; ES:BP -> DPB
6706 ; Functions: convert 32bit absolute RW input parms to 16bit input parms
6707 ; Output: carry set when CX=-1 and drive is less then 32mb
6708 ; carry clear, parms ok
6709 ;
6710 ;-----
6711
6712 ; 06/01/2024
6713 RW32_CONVERT:
6714 000009BC 83F9FF CMP CX,-1 ;>32mb new format ? ;AN000;
6715 ;inc CX ; * ; 01 -> 0
6716 000009BF 7423 JZ short new32format ;>32mb yes ;AN000;
6717 ;dec CX
6718
6719 ;cmp word [es:bp+0Fh],0
6720 000009C1 26837E0F00 cmp word [es:bp+DPB.FAT_SIZE],0 ; PCDOS 7.1
6721 000009C6 741A JZ short rw32_conv_err ; FAT32 fs
6722
6723 000009C8 50 PUSH AX ;>32mb save ax ;AN000;
6724 000009C9 52 PUSH DX ;>32mb save dx ;AN000;
6725 ;mov ax,[es:bp+0Dh]
6726 000009CA 268B460D MOV AX,[ES:BP+DPB.MAX_CLUSTER] ;>32mb get max cluster # ;AN000;
6727 ;mov dl,[es:bp+4]
6728 000009CE 268A5604 MOV DL,[ES:BP+DPB.CLUSTER_MASK] ;>32mb ;AN000;
6729 000009D2 80FAFE CMP DL,0FEh ; 254 ;>32mb removable ? ;AN000;
6730 000009D5 7407 JZ short letold ;>32mb yes ;AN000;
6731 ;INC DL ;>32mb ;AN000;
6732 ; 17/12/2022
6733 000009D7 42 inc dx
6734 000009D8 30F6 XOR DH,DH ;>32mb dx = sector/cluster ;AN000;
6735 000009DA F7E2 MUL DX ;>32mb dx:ax= max sector # ;AN000;
6736 000009DC 09D2 OR DX,DX ; (clears CF) ;>32mb > 32mb ? ;AN000;
6737 letold:
6738 000009DE 5A POP DX ;>32mb restore dx ;AN000;
6739 000009DF 58 POP AX ;>32mb restore ax ;AN000;
6740 000009E0 7418 JZ short old_style ; cf=0 ;>32mb no ;AN000;
6741
6742 ; 06/01/2024
6743 ;push ds
6744 ;getdseg <ds>
6745 ;mov ds,[cs:DosDSeg]
6746 ;mov word [AbsDskErr],207h ;>32mb bad address mark
6747 ;pop ds
6748 rw32_conv_err:
6749 000009E2 F9 STC ;>32mb ;AN000;
6750 000009E3 C3 retn ;>32mb ;AN000;
6751
6752 new32format:
6753 ;mov dx,[bx+2]
6754 000009E4 8B5702 MOV DX,[BX+ABS_32RW.SECTOR_RBA+2] ;>32mb ;AN000;
6755
6756 000009E7 1E push ds ; set up ds to DOSDATA
6757 ;getdseg <ds>
6758 000009E8 2E8E1E[0700] mov ds,[cs:DosDSeg]
6759 000009ED 8916[0706] MOV [HIGH_SECTOR],DX ;>32mb ;AN000;
6760 000009F1 1F pop ds
6761
6762 000009F2 8B17 mov dx,[bx]
6763 ;MOV DX,[BX+ABS_32RW.SECTOR_RBA] ;>32mb ;AN000;
6764 ;mov cx,[bx+4]
6765 000009F4 8B4F04 MOV CX,[BX+ABS_32RW.ABS_RW_COUNT] ;>32mb ;AN000;
6766 ;lds bx,[bx+6]
6767 000009F7 C55F06 LDS BX,[BX+ABS_32RW.BUFFER_ADDR] ;>32mb ;AN000;
6768 old_style:
6769 ; 06/01/2024
6770 ; cf=0
6771 ;CLC
6772 000009FA C3 retn ;>32mb ;AN000;
6773
6774 ;-----
6775 ;
6776 ; Procedure Name : Fastxxx_Purge
6777 ;
6778 ; Input: None
6779 ; Functions: Purge Fastopen/ Cache Buffers
6780 ; Output: None
6781 ;
6782 ;-----
6783
6784 ; 05/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
6785
6786 Fastxxx_Purge:
6787 000009FB 50 PUSH AX ; save regs. ;AN000;
6788 000009FC 56 PUSH SI ;AN000;
6789 000009FD 52 PUSH DX ;AN000;
6790
6791 000009FE 1E push ds ; set up ds to DOSDATA
6792 ;getdseg <ds>
6793 000009FF 2E8E1E[0700] mov ds,[cs:DosDSeg]
6794
6795 00000A04 F606[4611]80 TEST byte [FastOpenFlg],Fast_yes ; 80h
6796 ; fastopen installed ? ;AN000;
6797 00000A09 1F pop ds
6798 00000A0A 740B JZ short nofast ; no ;AN000;
6799 00000A0C B401 MOV AH,FastOpen_ID ; 1 ;AN000;
6800
6801 00000A0E B005 dofast:
6802 MOV AL,FONC_purge ; 5 ; purge ;AN000;
6803 ;mov dl,[es:bp+0]
6804 ; 05/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
6805 ;MOV DL,[ES:BP+DPB.DRIVE] ; set up drive number ;AN000;
6806 ; 15/12/2022
6807 00000A10 268A5600 mov dl,[es:bp]
6808 00000A14 E85923 ;invoke Fast_Dispatch ; call fastopen/seek ;AN000;
6809 call Fast_Dispatch
6810 00000A17 5A POP DX ;AN000;
6811 00000A18 5E POP SI ; restore regs ;AN000;
6812 00000A19 58 POP AX ;AN000;
6813 00000A1A C3 retn ; exit
6814
6815 ;=====
6816 ; DOSMES.INC (MSDOS 6.0, 1991)
6817 ;=====

```

```

6818 ; 29/04/2019 - Retro DOS v4.0
6819
6820 ;include dossym.inc
6821 ;include dosmac.inc
6822 ;include doscntry.inc
6823
6824 ; DOSCODE Segment
6825
6826 ; 17/07/2018 - Retro DOS v3.0 [ DOSMES.INC (MSDOS 3.3, 1987) ]
6827 ; -----
6828 ;include divmes.inc
6829
6830 ; DOSCODE:48C3h (PCDOS 7.1, IBMDOS.COM) - 06/04/2024 -
6831 ; -----
6832 ; DOSCODE:4778h (MSDOS 6.21, MSDOS.SYS)
6833 ; -----
6834 ; DOSCODE:476Bh (MSDOS 5.0, MSDOS.SYS) - 05/11/2022 -
6835
6836 ; THIS IS THE ONLY DOS "MESSAGE". IT DOES NOT NEED A TERMINATOR.
6837 ;PUBLIC DIVMES
6838
6839 00000A1B 0D0A44697669646520- DIVMES: DB 13,10,"Divide overflow",13,10
6839 00000A24 6F766572666C6F770D-
6839 00000A2D 0A
6840
6841 ;PUBLIC DivMesLen
6842 DivMesLen:
6843 00000A2E 1300 DW $-DIVMES ; 19 ; Length of the above message in bytes
6844
6845 ; DOSCODE:478Dh (MSDOS 6.21, MSDOS.SYS)
6846 ; -----
6847 ; DOSCODE:4780h (MSDOS 5.0, MSDOS.SYS) - 05/11/2022 -
6848
6849 ; (MSDOS 6.0)
6850 ; VxD not found error message
6851
6852 NovVxDErrMsg:
6853 00000A30 596F75206D75737420- db 'You must have the file WINA20.386 in the root of your boot drive'
6853 00000A39 686176652074686520-
6853 00000A42 66696C652057494E41-
6853 00000A4B 32302E33383620696E-
6853 00000A54 2074686520726F6F74-
6853 00000A5D 206F6620796F757220-
6853 00000A66 626F6F742064726976-
6853 00000A6F 65
6854 00000A70 0D0A746F2072756E20- db 0Dh,0Ah,'to run windows in Enhanced Mode',0Dh,0Ah
6854 00000A79 57696E646F77732069-
6854 00000A82 6E20456E68616E6365-
6854 00000A8B 64204D6F64650D0A
6855
6856 VxDMesLen equ $ - NovVxDErrMsg ; 99
6857
6858 ; 13/05/2019 - Retro DOS v4.0
6859 ; 05/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
6860
6861 ;include yesno.asm (MNSDOS 6.0)
6862 ; -----
6863 ; DOSCODE:47F0h (MSDOS 6.21, MSDOS.SYS)
6864 ; DOSCODE:47E3h (MSDOS 5.0, MSDOS.SYS) - 05/11/2022 -
6865
6866 ; This is for country Yes and No
6867
6868 ; 06/01/2024 (Retro DOS 5.0 - PCDOS 7.1 IBMDOS.COM)
6869 ;NLS_YES: db 'Y'
6870 ;NLS_NO: db 'N'
6871 ;NLS_yes2: db 'y'
6872 ;NLS_no2: db 'n'
6873
6874 ; -----
6875
6876 ; DOSCODE:493Bh (PCDOS 7.1, IBMDOS.COM) - 06/04/2024 -
6877 ; -----
6878 ; DOSCODE:47F4h (MSDOS 6.21, MSDOS.SYS)
6879 ; DOSCODE:47E7h (MSDOS 5.0, MSDOS.SYS) - 05/11/2022 -
6880
6881 ;SUBTTL EDIT FUNCTION ASSIGNMENTS AND HEADERS
6882
6883 ; The following two tables implement the current buffered input editing
6884 ; routines. The tables are pairwise associated in reverse order for ease
6885 ; in indexing. That is; The first entry in ESCTAB corresponds to the last
6886 ; entry in ESCFUNC, and the last entry in ESCTAB to the first entry in ESCFUNC.
6887
6888 ;PUBLIC CANCHAR
6889 CANCHAR:
6890 00000A93 1B DB CANCEL ; 1Bh ;Cancel line character
6891
6892 ;PUBLIC ESCCHAR
6893 ESCCHAR:
6894 00000A94 00 DB ESCCH ; 0 ;Lead-in character for escape sequences
6895
6896 ;IF NOT Rainbow
6897
6898 ESCTAB: ; LABEL BYTE
6899
6900 ;IF IBM
6901 00000A95 40 DB 64 ; Ctrl-Z - F6
6902 00000A96 4D DB 77 ; Copy one char - -->
6903 00000A97 3B DB 59 ; Copy one char - F1
6904 00000A98 53 DB 83 ; Skip one char - DEL
6905 00000A99 3C DB 60 ; Copy to char - F2
6906 00000A9A 3E DB 62 ; Skip to char - F4
6907 00000A9B 3D DB 61 ; Copy line - F3
6908 00000A9C 3D DB 61 ; Kill line (no change to template) - Not used
6909 00000A9D 3F DB 63 ; Reedit line (new template) - F5
6910 00000A9E 4B DB 75 ; Backspace - <--
6911 00000A9F 52 DB 82 ; Enter insert mode - INS (toggle)
6912 00000AA0 52 DB 82 ; Exit insert mode - INS (toggle)
6913 00000AA1 41 DB 65 ; Escape character - F7
6914 00000AA2 41 DB 65 ; End of table
6915 ;ENDIF
6916
6917 ESCEND: ; LABEL BYTE
6918
6919 ESCTABLEN EQU ESCEND-ESCTAB
6920
6921 ESCFUNC: ; LABEL WORD
6922
6923 00000AA3 [DF19] short_addr GETCH ; Ignore the escape sequence
6924 00000AA5 [5C1A] short_addr TWOESC
6925 00000AA7 [511B] short_addr EXITINS
6926 00000AA9 [511B] short_addr ENTERINS
6927 00000AAB [571A] short_addr BACKSP
6928 00000AAD [3D1B] short_addr REEDIT
6929 00000AAF [441A] short_addr KILNEW

```



```

6930 00000AB1 [D31A]      short_addr COPYLIN
6931 00000AB3 [051B]      short_addr SKIPSTR
6932 00000AB5 [D91A]      short_addr COPYSTR
6933 00000AB7 [FC1A]      short_addr SKIPONE
6934 00000AB9 [DE1A]      short_addr COPYONE
6935 00000ABB [DE1A]      short_addr COPYONE
6936 00000ABD [581B]      short_addr CTRLZ
6937
6938      ;ENDIF
6939
6940      ; DOSMES.INC (MSDOS 6.0, 1991)
6941      ;
6942      ; DOSMES.ASM (MSDOS 2.11, 1983)
6943
6944      ; OEMFunction key is expected to process a single function
6945      ; key input from a device and dispatch to the proper
6946      ; routines leaving all registers UNTOUCHED.
6947
6948      ; Inputs:  CS, SS are DOSGROUP
6949      ; Outputs: None. This function is expected to JMP to onw of
6950      ; the following labels:
6951      ;
6952      ;          GetCh      - ignore the sequence
6953      ;          TwoEsc     - insert an ESCChar in the buffer
6954      ;          ExitIns    - toggle insert mode
6955      ;          EnterIns   - toggle insert mode
6956      ;          BackSp     - move backwards one space
6957      ;          ReEdit     - reedit the line with a new template
6958      ;          Ki1New     - discard the current line and start from scratch
6959      ;          CopyLin    - copy the rest of the template into the line
6960      ;          SkipStr    - read the next character and skip to it in the template
6961      ;          CopyStr    - read next char and copy from template to line until char
6962      ;          SkipOne    - advance position in template one character
6963      ;          CopyOne    - copy next character in template into line
6964      ;          CtrlZ      - place a ^Z into the template
6965      ; Registers that are allowed to be modified by this function are:
6966      ;          AX, CX, BP
6967
6968      ; 13/05/2019 - Retro DOS v4.0
6969      ;
6970      ; DOSCODE:4820h (MSDOS 6.21, MSDOS.SYS)
6971
6972      ; 05/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
6973      ;
6974      ; DOSCODE:4813h (MSDOS 5.0, MSDOS.SYS)
6975
6976      ; 06/01/2024 - Retro DOS v5.0
6977      ;
6978      ; DOSCODE:4967h (PCDOS 7.1, IBMDOS.COM)
6979
6980      OEMFunctionKey:
6981      00000ABF E8800E      CALL    _$STD_CON_INPUT_NO_ECHO    ; Get the second byte of the sequence
6982      00000AC2 B10E      MOV     CL,ESCTABLEN ; 14      ; length of table for scan
6983      00000AC4 57        PUSH   DI      ; save DI (cannot change it!)
6984      00000AC5 BF950A]    MOV     DI,ESCTAB      ; offset of second byte table
6985      00000AC8 06        push    es
6986      00000AC9 0E        push    cs
6987      00000ACA 07        pop     es
6988      00000ACB F2AE      REPNE   SCASB      ; Look it up in the table
6989      00000ACD 07        pop     es
6990      00000ACE 5F        POP     DI      ; restore DI
6991      00000ACF D1E1      SHL     CX,1      ; convert byte offset to word
6992      00000AD1 89CD      MOV     BP,CX      ; move to indexable register
6993      ; JMP     word [BP+ESCFUNC] ; Go to the right routine
6994      00000AD3 2EFAA6[A30A] JMP     word [CS:BP+ESCFUNC]
6995
6996      ;DOSCODE ENDS
6997
6998      ;=====
6999      ; TIME.ASM (MSDOS 6.0, 1991)
7000      ;=====
7001      ; Retro DOS v3.0 - 18/07/2018
7002
7003      ; SYSCALL.ASM (MSDOS 2.11, 1983)
7004      ;
7005      ; Retro DOS v2.0 - 13/03/2018
7006
7007      ;** TIME.ASM - System Calls and low level routines for DATE and TIME
7008
7009      ;BREAK <DATE AND TIME - SYSTEM CALLS 42,43,44,45>
7010
7011      ;** $GET_DATE - Get Current Date
7012      ;-----
7013      ; ENTRY none
7014      ; EXIT (cx:dx) = current date
7015      ; USES all
7016
7017      ; 05/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
7018      ; 06/01/2024 - Retro DOS v5.0 (Modified MSDOS 7.1 IBMDOS.COM)
7019
7020      _$GET_DATE: ;System call 42
7021
7022      00000AD8 16        PUSH   SS
7023      00000AD9 1F        POP     DS
7024      00000ADA E8AD00      CALL    READTIME      ;check for rollover to next day
7025      00000ADD A15203]    MOV     AX,[YEAR]
7026
7027      ; WARNING!!!! DAY and MONTH must be adjacently allocated!
7028
7029      00000AE0 8B1E[5003]    MOV     BX,[DAY]      ; fetch both day and month
7030      00000AE4 E890F9      CALL    Get_user_stack ;Get pointer to user registers
7031      ;MOV     [SI+6],BX      ;DH=month, DL=day
7032      00000AE7 895C06      MOV     [SI+user_env.user_DX],BX
7033      00000AEA 05BC07      ADD     AX,1980      ;Put bias back
7034      ;MOV     [SI+4],AX      ;CX=year
7035      00000AED 894404      MOV     [SI+user_env.user_CX],AX
7036      00000AF0 36A0[5603]    MOV     AL,[SS:WEEKDAY] ;hkn; SS override
7037      RET20:      ; 05/11/2022
7038      RET24:      ; 18/12/2022
7039      00000AF4 C3        RETN
7040
7041      ;** $SET_DATE - Set Current Date
7042      ;-----
7043      ; ENTRY (cx:dx) = current date
7044      ; EXIT (al) = -1 iff bad date
7045      ; (al) = 0 if ok
7046      ; USES all
7047
7048      _$SET_DATE: ;System call 43
7049
7050      00000AF5 B0FF      MOV     AL,-1      ;Be ready to flag error
7051      00000AF7 81E9BC07    SUB     CX,1980      ;Fix bias in year
7052      ;JC     SHORT RET24      ;Error if not big enough
7053      ; 05/11/2022

```

```

7054 00000AFB 72F7      jc      short RET20
7055 00000AFD 83F977    cmp     cx,119      ;Year must be less than 2100
7056 00000B00 77F2      ja      short RET24
7057 00000B02 08F6      or      dh,dh
7058                      ;JZ      short RET24
7059                      ; 05/11/2022
7060 00000B04 74EE      jz      short RET20
7061 00000B06 08D2      or      dl,dl
7062                      ;JZ      short RET24      ;Error if either month or day is 0
7063                      ; 05/11/2022
7064 00000B08 74EA      jz      short RET20
7065 00000B0A 80FE0C    cmp     dh,12      ;Check against max. month
7066 00000B0D 77E5      ja      short RET24
7067 00000B0F 16          push    ss
7068 00000B10 1F          pop     ds
7069                      ;CALL   DODATE
7070                      ; 18/12/2022
7071 00000B11 E90501    jmp     DODATE
7072      ;RET24:
7073      ;RETN
7074
7075      ;** $GET_TIME - Get Current Time
7076      ;-----
7077      ; ENTRY   none
7078      ; EXIT    (cx:dx) = current time
7079      ; USES    all
7080
7081      _$GET_TIME:      ;System call 44
7082
7083      push    ss
7084      pop     ds
7085      call    READTIME
7086      call    Get_User_Stack ;Get pointer to user registers
7087      ;MOV     [SI+6],DX
7088      mov     [SI+user_env.user_DX],DX
7089      ;MOV     [SI+4],CX
7090      mov     [SI+user_env.user_CX],CX
7091      set_time_ok:      ; 06/01/2024
7092      xor     al,al
7093      RET26:
7094      RETN
7095
7096      ;** $SET_TIME - Set Current Time
7097      ;-----
7098      ; ENTRY   (cx:dx) = time
7099      ; EXIT    (al) = 0 if Ok
7100      ;         (al) = -1 if invalid
7101      ; USES    ALL
7102
7103      _$SET_TIME:      ;System call 45
7104
7105      mov     al,-1      ;Flag in case of error
7106      cmp     ch,24      ;Check hours
7107      jae     short RET26
7108      cmp     cl,60      ;Check minutes
7109      jae     short RET26
7110      cmp     dh,60      ;Check seconds
7111      jae     short RET26
7112      cmp     dl,100     ;Check 1/100's
7113      jae     short RET26
7114      push    cx
7115      push    dx
7116      push    ss
7117      pop     ds
7118
7119      ; 07/02/2024
7120      %if 0
7121      mov     bx,TIMEBUF
7122      mov     cx,6
7123      ; 06/02/2024 ; *
7124      ;XOR     dx,dx
7125      ;MOV     ax,dx
7126      ;xor     ax,ax
7127      ;cld     ; 06/01/2024
7128      push    bx
7129      ;CALL    SETREAD
7130      ; 06/02/2024 ; *
7131      call    SETREAD_X
7132      %else
7133      call    SETREAD_XT
7134      %endif
7135
7136      push    ds
7137      lds     si,[BCLOCK]
7138      call    DEVIOCALL2      ;Get correct day count
7139      pop     ds
7140      pop     bx
7141      call    SETWRITE
7142      pop     word [TIMEBUF+4]
7143      pop     word [TIMEBUF+2]
7144      lds     si,[BCLOCK]
7145      call    DEVIOCALL2      ;Set the time
7146      ; 06/01/2024
7147      ;XOR     al,al
7148      ;RETN
7149      jmp     short set_time_ok
7150
7151      ; 11/07/2018 - Retro DOS v3.0
7152      ; Retro DOS v2.0 - 14/03/2018
7153
7154      FOURYEARS EQU 3*365 + 366 ; = 1461
7155
7156      ;SUBTTL DATE16, READTIME, DODATE -- GUTS OF TIME AND DATE
7157      ;-----
7158      ; Date16 returns the current date in AX, current time in DX
7159      ; AX - YYYYYYMMMMDDDDD years months days
7160      ; DX - HHHHHMMMMSSSSSS hours minutes seconds/2
7161
7162      DATE16:
7163
7164      ;M048      Context DS
7165      ;
7166      ; Since this function can be called thru int 2f we shall not assume that SS
7167      ; is DOSDATA
7168
7169      ;push    ss
7170      ;pop     ds
7171
7172      ;getdseg <ds>      ; M048
7173
7174      ; 13/05/2019 - Retro DOS v4.0
7175      mov     ds,[cs:DosDSEG]
7176
7177      push    cx

```

```

7178 00000B66 06          PUSH    ES
7179 00000B67 E82000      CALL    READTIME
7180 00000B6A 07          POP     ES
7181 00000B6B D0E1      SHL     CL,1          ;Minutes to left part of byte
7182 00000B6D D0E1      SHL     CL,1
7183 00000B6F D1E1      SHL     CX,1          ;Push hours and minutes to left end
7184 00000B71 D1E1      SHL     CX,1
7185 00000B73 D1E1      SHL     CX,1
7186 00000B75 D0EE      SHR     DH,1          ;Count every two seconds
7187 00000B77 08F1      OR      CL,DH          ;Combine seconds with hours and minutes
7188 00000B79 89CA      MOV     DX,CX
7189
7190          ; WARNING! MONTH and YEAR must be adjacently allocated
7191
7192 00000B7B A1[5103]      MOV     AX,[MONTH]    ;Fetch month and year
7193 00000B7E B104      MOV     CL,4
7194 00000B80 D2E0      SHL     AL,CL          ;Push month to left to make room for day
7195 00000B82 D1E0      SHL     AX,1
7196 00000B84 59          POP     CX
7197 00000B85 0A06[5003]  OR      AL,[DAY]
7198 RET21:
7199          RETN
7200
7201          ;-----
7202
7203 READTIME:
7204
7205          ;Gets time in CX:DX. Figures new date if it has changed.
7206          ;Uses AX, CX, DX.
7207
7208 00000B8A C706[BD0D]0000 MOV     word [DATE_FLAG],0 ; reset date flag for CPMIO
7209 00000B90 56          PUSH    SI
7210 00000B91 53          PUSH    BX
7211
7212 00000B92 BB[B603]      MOV     BX,TIMEBUF
7213          ; 07/02/2024
7214          %if 0
7215          MOV     CX,6
7216          ; 06/02/2024
7217          ;;XOR    DX,DX
7218          ;;MOV    AX,DX
7219          ; 06/01/2024
7220          ;xor     ax,ax
7221          ;cld
7222          ;CALL    SETREAD
7223          ; 06/02/2024
7224          call    SETREAD_X
7225          %else
7226          call    SETREAD_XTC
7227          %endif
7228 00000B98 1E          PUSH    DS
7229 00000B99 C536[2E00]      LDS     SI,[BCLOCK]
7230 00000B9D E8F546      CALL    DEVIOCALL2    ;Get correct date and time
7231 00000BA0 1F          POP     DS
7232 00000BA1 5B          POP     BX
7233 00000BA2 5E          POP     SI
7234 00000BA3 A1[B603]      MOV     AX,[TIMEBUF]
7235 00000BA6 8B0E[B803]      MOV     CX,[TIMEBUF+2]
7236 00000BAA 8B16[BA03]      MOV     DX,[TIMEBUF+4]
7237 00000BAE 3B06[5403]      CMP     AX,[DAYCNT]    ;See if day count is the same
7238          ;JZ      SHORT RET22
7239 00000BB2 74D5      JZ      SHORT RET21 ; 18/07/2018
7240          ;cmp     ax,43830
7241 00000BB4 3D36AB      CMP     AX,FOURYEARS*30 ;Number of days in 120 years
7242 00000BB7 733D      JAE     SHORT RET22    ;Ignore if too large
7243 00000BB9 A3[5403]      MOV     [DAYCNT],AX
7244 00000BBC 56          PUSH    SI
7245 00000BBD 51          PUSH    CX
7246 00000BBE 52          PUSH    DX          ;Save time
7247 00000BBF 31D2      XOR     DX,DX
7248          ;mov     CX,1461
7249 00000BC1 B9B505      MOV     CX,FOURYEARS    ;Number of days in 4 years
7250 00000BC4 F7F1      DIV     CX          ;Compute number of 4-year units
7251 00000BC6 D1E0      SHL     AX,1
7252 00000BC8 D1E0      SHL     AX,1
7253 00000BCA D1E0      SHL     AX,1          ;Multiply by 8 (no. of half-years)
7254 00000BCC 89C1      MOV     CX,AX          ;<240 implies AH=0
7255
7256 00000BCE BE[140D]      MOV     SI,YRTAB        ;Table of days in each year
7257
7258          ;CALL    DSLIDE          ;Find out which of four years we're in
7259          ; 26/06/2024
7260 00000BD1 E82500      call    DSLIDE1 ; ah = 0
7261 00000BD4 D1E9      SHR     CX,1          ;Convert half-years to whole years
7262 00000BD6 7304      JNC     SHORT SK        ;Extra half-year?
7263 00000BD8 81C2C800  ADD     DX,200
7264
7265 00000BDC E82400      SK:    CALL    SETYEAR
7266 00000BDF B101      MOV     CL,1          ;At least at first month in year
7267
7268 00000BE1 BE[1C0D]      MOV     SI,MONTAB      ;Table of days in each month
7269
7270          ;CALL    DSLIDE          ;Find out which month we're in
7271          ; 26/06/2024
7272 00000BE4 E81200      call    DSLIDE1 ; ah = 0
7273 00000BE7 880E[5103]      MOV     [MONTH],CL
7274 00000BEB 42          INC     DX
7275 00000BEC 8816[5003]      MOV     [DAY],DL
7276 00000BF0 E88C00      CALL    WKDAY          ;Set day of week
7277 00000BF3 5A          POP     DX
7278 00000BF4 59          POP     CX
7279 00000BF5 5E          POP     SI
7280 RET22:
7281 00000BF6 C3          RETN
7282
7283          ;-----
7284
7285 DSLIDE:
7286          ; 26/06/2024
7287 00000BF7 B400      MOV     AH,0
7288          ; 06/01/2024
7289          ; (AH=0)
7290 DSLIDE1:
7291 00000BF9 AC          LODSB          ;Get count of days
7292 00000BFA 39C2      CMP     DX,AX          ;See if it will fit
7293          ;JB      SHORT RET23      ;If not, done
7294 00000BFC 72F8      jnb     short RET22 ; 13/05/2019 - Retro DOS v4.0
7295 00000BFE 29C2      SUB     DX,AX
7296 00000C00 41          INC     CX          ;Count one more month/year
7297 00000C01 EBF6      JMP     SHORT DSLIDE1
7298
7299          ;-----
7300
7301 SETYEAR:

```

```

7302
7303 ;Set year with value in CX. Adjust length of February for this year.
7304
7305 ; NOTE: This can also be called thru int 2f. If this is called then it will
7306 ; set DS to DOSDATA. Since the only guy calling this should be the DOS
7307 ; redir, DS will be DOSDATA anyway. It is going to be in-efficient to
7308 ; preserve DS as CHKYR is also called as a routine.
7309
7310 ; MSDOS 6.0 (18/07/2018) ; *
7311
7312 ;GETDSEG DS
7313
7314 ;PUSH CS ; *
7315 ;POP DS ; *
7316
7317 ; 13/05/2019 - Retro DOS v4.0
7318 00000C03 2E8E1E[0700] mov ds,[cs:DosDseg]
7319
7320 ; Offset 18CEh in IBMDOS.COM (MSDOS 3.3), 1987
7321 ; 05/11/2022
7322 ; DOSCODE:4970h in MSDOS.SYS (MSDOS 5.0), 1991
7323
7324 00000C08 880E[5203] MOV [YEAR],CL
7325 CHKYR:
7326 00000C0C F6C103 TEST CL,3 ;Check for leap year
7327 00000C0F B01C MOV AL,28
7328 00000C11 7502 JNZ SHORT SAVFEB ;28 days if no leap year
7329 00000C13 FEC0 INC AL ;Add leap day
7330 SAVFEB:
7331 00000C15 A2[1D0D] mov [february],al
7332 ;MOV [MONTAB+1],AL ;Store for February
7333 RET23:
7334 00000C18 C3 RETN
7335
7336 ;-----
7337
7338 DODATE:
7339 00000C19 E8F0FF CALL CHKYR ;Set Feb. up for new year
7340 00000C1C 88F0 MOV AL,DH
7341
7342 00000C1E BB[1B0D] MOV BX,MONTAB-1 ;DOSDATA:0D1Bh for MSDOS 6.21
7343 ; 06/01/2024
7344 ;DOSDATA:0D1Bh for PCDOS 7.1
7345
7346 00000C21 D7 XLAT ;Look up days in month
7347 00000C22 38D0 CMP AL,DL
7348 00000C24 B0FF MOV AL,-1 ;Restore error flag, just in case
7349 ;JB SHORT RET25 ;Error if too many days
7350 00000C26 72F0 jb short RET23 ; 18/07/2018
7351 00000C28 E8D8FF CALL SETYEAR
7352
7353 ;
7354 ; WARNING! DAY and MONTH must be adjacently allocated
7355 ;
7356 00000C2B 8916[5003] MOV [DAY],DX ;Set both day and month
7357 00000C2F D1E9 SHR CX,1
7358 00000C31 D1E9 SHR CX,1
7359 00000C33 88B505 ;mov ax,1461
7360 00000C36 89D3 MOV AX,FOURYEARS
7361 00000C38 F7E1 MOV BX,DX
7362 00000C3A 8A0E[5203] MUL CX
7363 00000C3E 80E103 MOV CL,[YEAR]
7364 AND CL,3
7365 00000C41 BE[140D] MOV SI,YRTAB
7366
7367 00000C44 89C2 MOV DX,AX
7368 00000C46 D1E1 SHL CX,1 ;Two entries per year, so double count
7369 00000C48 E84700 CALL DSUM ;Add up the days in each year
7370 00000C4B 88F9 MOV CL,BH ;Month of year
7371
7372 00000C4D BE[1C0D] MOV SI,MONTAB
7373
7374 00000C50 49 DEC CX ;Account for months starting with one
7375 00000C51 E83E00 CALL DSUM ;Add up days in each month
7376 00000C54 88D9 MOV CL,BL ;Day of month
7377 00000C56 49 DEC CX ;Account for days starting with one
7378 00000C57 01CA ADD DX,CX ;Add in to day total
7379 00000C59 92 XCHG AX,DX ;Get day count in AX
7380 00000C5A A3[5403] MOV [DAYCNT],AX
7381 00000C5D 56 PUSH SI
7382 00000C5E 53 PUSH BX
7383 00000C5F 50 PUSH AX
7384
7385 ; 07/02/2024
7386 %if 0
7387 MOV BX,TIMEBUF
7388 MOV CX,6
7389 ; 06/02/2024 ; *
7390 ;;XOR DX,DX
7391 ;;MOV AX,DX
7392 ;; 06/01/2024
7393 ;xor ax,ax
7394 ;cwd
7395 PUSH BX
7396 ;CALL SETREAD
7397 ; 06/02/2024 ; *
7398 call SETREAD_X
7399 %else
7400 00000C60 E8A946 call SETREAD_XT
7401 %endif
7402
7403 00000C63 1E PUSH DS
7404 00000C64 C536[2E00] LDS SI,[BCLOCK]
7405 00000C68 E82A46 CALL DEVIOCALL2 ;Get correct date and time
7406 00000C6B 1F POP DS
7407 00000C6C 5B POP BX
7408 00000C6D E8D946 CALL SETWRITE
7409 00000C70 8F06[B603] POP WORD [TIMEBUF]
7410 00000C74 1E PUSH DS
7411 00000C75 C536[2E00] LDS SI,[BCLOCK]
7412 00000C79 E81946 CALL DEVIOCALL2 ;Set the date
7413 00000C7C 1F POP DS
7414 00000C7D 5B POP BX
7415 00000C7E 5E POP SI
7416
7417 00000C7F A1[5403] WKDAY: MOV AX,[DAYCNT]
7418 00000C82 31D2 XOR DX,DX
7419 00000C84 B90700 MOV CX,7
7420 00000C87 40 INC AX
7421 00000C88 40 INC AX ;First day was Tuesday
7422 00000C89 F7F1 DIV CX ;Compute day of week
7423 00000C8B 8816[5603] MOV [WEEKDAY],DL
7424 00000C8F 30C0 XOR AL,AL ;Flag OK
7425 RET25:

```

```

7426 00000C91 C3      RETN
7427
7428
7429
7430
7431
7432
7433
7434
7435
7436
7437
7438
7439 00000C92 B400
7440 00000C94 E305
7441
7442
7443 00000C96 AC
7444 00000C97 01C2
7445 00000C99 E2FB
7446
7447 00000C9B C3
7448
7449
7450
7451
7452
7453
7454
7455
7456
7457
7458
7459
7460
7461
7462
7463
7464
7465
7466
7467
7468
7469
7470
7471
7472
7473
7474
7475
7476
7477
7478
7479
7480
7481
7482
7483
7484
7485
7486
7487
7488
7489
7490
7491
7492
7493
7494
7495
7496
7497
7498
7499
7500
7501
7502
7503
7504
7505
7506
7507
7508
7509
7510
7511
7512
7513
7514
7515
7516
7517 00000C9C 36C50E[B203]
7518 00000CA1 8CDB
7519
7520
7521
7522
7523
7524 00000CA3 16
7525 00000CA4 1F
7526
7527
7528
7529
7530
7531
7532
7533
7534
7535
7536 00000CA5 3C01
7537 00000CA7 7502
7538
7539
7540
7541
7542
7543 00000CA9 30FF
7544
7545
7546
7547
7548
7549

;-----
; ** DSUM - Compute the sum of a string of bytes
;
; ENTRY (cx) = byte count
;        (ds:si) = byte address
;        (dx) = sum register, initialized by caller
; EXIT (dx) updated
; USES ax, cx, dx, si, flags
;
DSUM:
    MOV     AH,0
    JCXZ    DSUM9 ; 13/05/2019 - Retro DOS v4.0
    ;JCXZ    RET25 ; 18/07/2018
DSUM1:
    LODSB
    ADD     DX,AX
    LOOP    DSUM1
DSUM9:
    RETN

;=====
; GETSET.ASM (MSDOS 6.0, 1991)
;=====
; 29/04/2019 - Retro DOS v4.0
; 18/07/2018 - Retro DOS v3.0 (GETSET.ASM, MSDOS 6.0, 1991)
;
; 12/03/2018 - Retro DOS v2.0
;
;TITLE     GETSET - GETting and SETting MS-DOS system calls
;NAME      GETSET
;
;CODE      SEGMENT BYTE PUBLIC 'CODE'
;          ASSUME SS:DOSGROUP,CS:DOSGROUP
;
;USERNUM:
;    DW     0 ; 24 bit user number
;    DB     0
;    IF     IBM
;OEMNUM: DB 0 ; 8 bit OEM number
;    ELSE
;OEMNUM: DB 0FFH ; 8 bit OEM number
;    ENDIF
;
;MSVERS:   ; MS-DOS version in hex for $GET_VERSION
; 08/07/2018 - Retro DOS v3.0
;MSMAJOR: DB MAJOR_VERSION ; DOS_MAJOR_VERSION
;MSMINOR: DB MINOR_VERSION ; DOS_MINOR_VERSION
;
;BREAK <$Get_Version -- Return MSDOS version number>
;-----
; 05/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
; DOSCODE:4A0Fh (MSDOS 5.0 MSDOS.SYS)
;
; 06/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
; DOSCODE:4B5Fh (PCDOS 7.1 IBMDOS.COM)
;
_$GET_VERSION:
; Inputs:
; None
; Function:
; Return MS-DOS version number
; Outputs:
; OEM number in BH
; User number in BL:CX (24 bits)
; Version number as AL:AH in binary
; NOTE: On pre 1.28 DOSs AL will be zero
;
; MSDOS 6.0
;
; Fake_Count is used to lie about the version numbers to support
; old binarys. See ms_table.asm for more info.
;
; if input al = 00
; (bh) = OEM number
; else if input al = 01
; (bh) = version flags
;
; bits 0-2 = DOS internal revision
; bits 3-7 = DOS type flags
; bit 3 = DOS is in ROM
; bit 4 = DOS in in HMA
; bits 5-7 = reserved
; M007 change - only bit 3 is now valid. other bits
; are 0 when AL = 1
;
; 06/01/2024 (PCDOS 7.1 IBMDOS.COM)
    lds     cx, [ss:USERNUM]
    mov     bx, ds
;
; MSDOS 3.3 (IBMDOS.COM, offset 196Dh)
;-----
; MSDOS 6.21 (MSDOS.SYS, DOSCODE:4A1Ch)
;
    PUSH    SS
    POP     DS
;
; 06/01/2024
;MOV     BX,[USERNUM+2]
;MOV     CX,[USERNUM]
;
; 13/05/2019 - Retro DOS v4.0
;
; If AL == 1, ROMDOS will return BH = dos internal version # &
; DOS flags
    cmp     AL,1
    jne     short Norm_Vers
;
;ifdef ROMDOS
;    mov     BH,DOSINROM ; Just set the bit for ROM version
;                    (DOSINROM = 8)
;else
;    xor     bh,bh ; otherwise return 0
;endif
;M007 end
;endif
;
Norm_Vers:
    MOV     AX,[MSVERS] ; MSDOS 3.3
;
; MSDOS 6.0 ; MSVERS is a label in TABLE segment

```

```

7550             ; 26/06/2024 - Retro DOS v5.0
7551             ; (PCDOS 7.1 IBMDOS.COM)
7552             ; 13/05/2019 - Retro DOS v4.0
7553             ;push ds             ; Get the version number from the
7554 00000CAB 8E1E[3003] mov ds,[CurrentPDB] ; current app's PSP segment
7555             ;mov ax,[40h]
7556 00000CAF A14000 mov ax,[PDB.Version] ; AX = DOS version number
7557             ; 07/12/2022
7558             ;pop ds
7559 00000CB2 E8C2F7 call Get_User_Stack
7560             ; Put values for return registers
7561             ; in the proper place on the user's
7562             ; stack addressed by DS:SI
7563             ; 06/01/2024 (PCDOS 7.1 IBMDOS.COM)
7564 gdrvfspc_ret:
7565             ;MOV [SI+user_env.user_AX],AX
7566 00000CB5 8904 MOV [SI],AX
7567             ;MOV [SI+4],CX
7568 00000CB7 894C04 mov [SI+user_env.user_CX],CX
7569 set_user_bx:
7570             ;MOV [SI+2],BX
7571 00000CBA 895C02 mov [SI+user_env.user_BX],BX
7572             RETN
7573 00000CBD C3
7574
7575             ; 18/07/2018 - Retro DOS v3.0
7576
7577 ;BREAK <$Get/Set_Verify_on_Write - return/set verify-after-write flag>
7578 ;-----
7579
7580 ;** $Get_Verify_On_Write - Get Status of Verify on write flag
7581 ;
7582 ; ENTRY none
7583 ; EXIT (al) = value of VERIFY flag
7584 ; USES all
7585
7586 _$GET_VERIFY_ON_WRITE:
7587
7588 ;hkn; SS override
7589 00000CBE 36A0[FF02] MOV AL,[SS:VERFLG] ; Retro DOS v2.0 - 12/03/2018
7590 00000CC2 C3 retn
7591
7592 ;** $Set_Verify_On_Write - Set Status of Verify on write flag
7593 ;
7594 ; ENTRY (al) = value of VERIFY flag
7595 ; EXIT none
7596 ; USES all
7597
7598 _$SET_VERIFY_ON_WRITE:
7599
7600 AND AL,1
7601 00000CC3 2401 ;hkn; SS override
7602 00000CC5 36A2[FF02] MOV [SS:VERFLG],AL ; Retro DOS v2.0 - 12/03/2018
7603 RET27: ; 18/07/2018
7604 retn
7605 00000CC9 C3
7606
7607             ; 19/07/2018 - Retro DOS v3.0
7608
7609 ;BREAK <$International - return country-dependent information>
7610 ;-----
7611 ;
7612 ; Procedure Name : $INTERNATIONAL
7613 ;
7614 ; Inputs:
7615 ; MOV AH,International
7616 ; MOV AL,country (al = 0 => current country)
7617 ; [MOV BX,country]
7618 ; LDS DX,block
7619 ; INT 21
7620 ; Function:
7621 ; give users an idea of what country the application is running
7622 ; Outputs:
7623 ; IF DX != -1 on input (get country)
7624 ; AL = 0 means return current country table.
7625 ; 0<AL<0FFH means return country table for country AL
7626 ; AL = 0FF means return country table for country BX
7627 ; No Carry:
7628 ; Register BX will contain the 16-bit country code.
7629 ; Register AL will contain the low 8 bits of the country code.
7630 ; The block pointed to by DS:DX is filled in with the information
7631 ; for the particular country.
7632 ; BYTE Size of this table excluding this byte and the next
7633 ; BYTE Country code represented by this table
7634 ; A sequence of n bytes, where n is the number specified
7635 ; by the first byte above and is not > internat_block_max,
7636 ; in the correct order for being returned by the
7637 ; INTERNATIONAL call as follows:
7638 ; WORD Date format 0=mdy, 1=dmy, 2=ymd
7639 ; 5 BYTE Currency symbol null terminated
7640 ; 2 BYTE thousands separator null terminated
7641 ; 2 BYTE Decimal point null terminated
7642 ; 2 BYTE Date separator null terminated
7643 ; 2 BYTE Time separator null terminated
7644 ; 1 BYTE Bit field. Currency format.
7645 ; Bit 0. =0 $ before # =1 $ after #
7646 ; Bit 1. no. of spaces between # and $ (0 or 1)
7647 ; 1 BYTE No. of significant decimal digits in currency
7648 ; 1 BYTE Bit field. Time format.
7649 ; Bit 0. =0 12 hour clock =1 24 hour
7650 ; DWORD Call address of case conversion routine
7651 ; 2 BYTE Data list separator null terminated.
7652 ; Carry:
7653 ; Register AX has the error code.
7654 ; IF DX = -1 on input (set current country)
7655 ; AL = 0 is an error
7656 ; 0<AL<0FFH means set current country to country AL
7657 ; AL = 0FF means set current country to country BX
7658 ; No Carry:
7659 ; Current country SET
7660 ; Register AL will contain the low 8 bits of the country code.
7661 ; Carry:
7662 ; Register AX has the error code.
7663 ;-----
7664
7665 ;procedure $INTERNATIONAL,NEAR ; DOS 3.3
7666
7667 ; 13/05/2019 - Retro DOS v4.0
7668 ; DOSCODE:4A4Dh (MSDOS 6.21, MSDOS.SYS)
7669
7670 ; 05/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
7671 ; DOSCODE:4A40h (MSDOS 5.0, MSDOS.SYS)
7672
7673 ; 06/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 PCDOS.COM)

```



```

7674 ; DOSCODE:4B8Dh (PCDOS 7.1, IBMDOS.COM)
7675
7676 _$INTERNATIONAL: ; IBMDOS.COM (MSDOS 3.3), offset 1992h
7677
7678 00000CCA 3CFF      CMP     AL,0FFh
7679 00000CCC 7404      JZ      short BX_HAS_CODE ; -1 means country code is in BX
7680 00000CCE 88C3      MOV     BL,AL ; Put AL country code in BX
7681 00000CD0 30FF      XOR     BH,BH
7682 BX_HAS_CODE:
7683 00000CD2 1E      PUSH    DS
7684 00000CD3 07      POP     ES
7685 00000CD4 52      PUSH    DX
7686 00000CD5 5F      POP     DI ; User buffer to ES:DI
7687
7688 ;hkn; SS is DOSDATA
7689 ; context DS
7690
7691 00000CD6 16      push    ss
7692 00000CD7 1F      pop     ds
7693
7694 00000CD8 83FFFF      CMP     DI,-1
7695 00000CDB 745D      JZ      short international_set
7696 00000CDD 09DB      OR      BX,BX
7697 00000CDF 7505      JNZ     short international_find
7698
7699 ;hkn; country_cdpq is in DOSDATA segment.
7700 00000CE1 BE[2A12] MOV     SI,COUNTRY_CDPG
7701
7702 00000CE4 EB39      JMP     SHORT international_copy
7703
7704 international_find:
7705 ;MOV     BP,0 ; flag it for GetCntry only
7706 ; 06/01/2024
7707 00000CE6 31ED      xor     bp,bp ; 0
7708 00000CE8 E80A00 CALL    international_get
7709 00000CEB 7255      JC      short errtn
7710 ;CMP     BX,0 ; nlsfunc finished it ?
7711 ; 06/01/2024
7712 00000CED 09DB      or      bx,bx
7713 00000CEF 752E      JNZ     SHORT international_copy ; no, copy by myself
7714 00000CF1 89D3      MOV     BX,DX ; put country back
7715 00000CF3 EB3A      JMP     SHORT international_ok3
7716
7717 international_get:
7718 00000CF5 BE[2A12] MOV     SI,COUNTRY_CDPG
7719
7720 ;hkn; country_cdpq is in DOSDATA segment.
7721 ;hkn; use ss override to access COUNTRY_CDPG fields
7722
7723 ; MSDOS 3.3
7724 ;;cmp     bx,[SI+63h]
7725 ;CMP     BX,[SI+DOS_CCDPG.ccDosCountry]
7726 ;jz      short RET27
7727
7728 ; 13/05/2019 - Retro DOS v4.0
7729
7730 ; MSDOS 6.0
7731 ;cmp     bx,[ss:si+68h]
7732 00000CF8 363B5C68 CMP     BX,[ss:SI+DOS_CCDPG.ccDosCountry] ; = current country id
7733 00000CFC 74CB      jz      short RET27 ; return if equal
7734
7735 00000CFE 89DA      MOV     DX,BX
7736 00000D00 31DB      XOR     BX,BX ; bx = 0, default code page
7737 ;CallInstall NLSInstall,NLSFUNC,0 ; check if NLSFUNC in memory
7738 00000D02 B80014 mov     ax,1400h
7739 00000D05 CD2F int     2Fh ; - Multiplex - NLSFUNC.COM - INSTALLATION CHECK
7740 ; Return: AL = 00h not installed, OK to install
7741 ; 01h not installed, not OK
7742 ; FFh installed
7743 00000D07 3CFF      CMP     AL,0FFh
7744 00000D09 7510      JNZ     short interr ; not in memory
7745
7746 ; 06/01/2024
7747 00000D0B B80314 mov     ax,1403h ; set country info
7748
7749 ;cmp     bp,0
7750 00000D0E 09ED      or      bp,bp ; GetCntry ?
7751 00000D10 7501      JNZ     short stcdpg
7752
7753 ;CallInstall GetCntry,NLSFUNC,4 ; get country info
7754 ;mov     ax,1404h
7755 00000D12 40      inc     ax ; AX = 1404h ; get country info
7756
7757 ; 06/01/2024
7758 ;int     2Fh ; - Multiplex - NLSFUNC.COM - GET COUNTRY INFO
7759 ; ; BX = code page, DX = country code,
7760 ; ; DS:SI -> internal code page structure
7761 ; ; ES:DI -> user buffer
7762 ; ; Return: AL = status
7763 ;
7764 ;JMP     short chkok
7765
7766 ;nop
7767
7768 stcdpg:
7769 ;CallInstall SetCodePage,NLSFUNC,3 ; set country info
7770 ; 06/01/2024
7771 ;mov     ax,1403h
7772 gscdpg:
7773 00000D13 CD2F int     2Fh ; - Multiplex - NLSFUNC.COM - SET COUNTRY INFO
7774 ; ; DS:SI -> internal code page structure
7775 ; ; BX = code page, DX = country code
7776 ; ; Return: AL = status
7777
7778 00000D15 08C0      or      al,al ; success ?
7779 ;retz ; yes
7780 00000D17 74B0      jz      short RET27
7781
7782 setcarry:
7783 00000D19 F9      STC ; set carry
7784 00000D1A C3      retn
7785
7786 00000D1B B0FF      MOV     AL,0FFh ; flag nlsfunc error
7787 00000D1D EBFA      JMP     short setcarry
7788
7789 international_copy:
7790
7791 ;hkn; country_cdpq is in DOSDATA segment.
7792 ;hkn; use ss override to access COUNTRY_CDPG fields
7793
7794 ; MSDOS 3.3
7795 ;;mov     bx,[SI+63h]
7796 ;mov     BX,[SI+DOS_CCDPG.ccDosCountry]
7797 ;mov     SI,COUNTRY_CDPG+DOS_CCDPG.ccDFormat ; 08/09/2018

```

```

7798
7799 ; 13/05/2019 - Retro DOS v4.0
7800
7801 ; MSDOS 6.0
7802 ;mov bx,[ss:si+68h]
7803 00000D1F 368B5C68 MOV BX,[ss:SI+DOS_CCDPG.ccDosCountry] ; = current country id
7804 00000D23 BE[9612] MOV SI,COUNTRY_CDPG+DOS_CCDPG.ccDFormat ; COUNTRY_CDPG + 108
7805
7806 ;mov cx,24
7807 00000D26 B91800 MOV CX,OLD_COUNTRY_SIZE
7808
7809 ; MSDOS 6.0
7810
7811 ;hkn; must set up DS to SS so that international info can be copied
7812
7813 00000D29 1E push ds
7814
7815 00000D2A 16 push ss ; cs -> ss
7816 00000D2B 1F pop ds
7817
7818 00000D2C F3A4 REP MOVSB ; copy country info
7819
7820 ; MSDOS 6.0
7821
7822 00000D2E 1F pop ds ;hkn; restore ds
7823
7824 international_ok3:
7825 00000D2F E845F7 call Get_User_Stack
7826 ;ASSUME DS:NOTHING
7827 ;;MOV [SI+2],BX
7828 ;MOV [SI+user_env.user_BX],BX
7829 ; 26/06/2024 (PCDOS 7.1 IBMDOS.COM)
7830 00000D32 E885FF call set_user_bx
7831
7832 00000D35 89D8 MOV AX,BX ; Return country code in AX too.
7833 ;SYS_RET_OK_jmp:
7834 ; 05/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
7835 ; 09/01/2024
7836 ;nono: ; 15/12/2022
7837 SYS_RET_OK_jmp:
7838 00000D37 E934F9 jmp SYS_RET_OK
7839
7840 international_set:
7841
7842 ;hkn; ASSUME DS:DOSGROUP
7843 ;ASSUME DS:DOSDATA
7844
7845 00000D3A BD0100 MOV BP,1 ; flag it for SetCodePage only
7846 00000D3D E8B5FF CALL international_get
7847 00000D40 73F3 JNC short international_ok
7848
7849 00000D42 3CFF errtn: CMP AL,0FFH
7850 00000D44 7403 JZ short errtn2
7851
7852 00000D46 E92FF9 errtn1: jmp SYS_RET_ERR ; return what we got from NLSFUNC
7853
7854 errtn2: ;error error_invalid_function; NLSFUNC not existent
7855
7856 ;mov al,1
7857 00000D49 B001 mov al,error_invalid_function
7858 00000D4B EBF9 jmp short errtn1 ; 13/05/2019 - Retro DOS v4.0
7859
7860 ;errtn3:
7861 ; jmp SYS_RET_ERR
7862
7863 ;EndProc $INTERNATIONAL
7864
7865 ; -----
7866
7867 ; 06/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 PCDOS.COM)
7868 ; DOSCODE:4C10h (PCDOS 7.1, IBMDOS.COM)
7869
7870 _$ExtCountryInfo:
7871 ; INT 21h, AH = 70h
7872 ; GET/SET INTERNATIONALIZATION INFORMATION
7873 ; ****
7874 ; AL = subfunction
7875 ; 00h SET general internationalization info
7876 ; CX = buffer size (up to 38 bytes)
7877 ; DS:SI -> buffer containing internationalization info
7878 ; first three bytes are skipped, the rest is copied to
7879 ; somewhere in the DOS data segment
7880 ; 01h SET extended internationalization info
7881 ; CX = number of bytes to set (up to 58 bytes)
7882 ; DS:SI -> buffer containing internationalization info
7883 ; 02h GET extended internationalization info
7884 ; CX = buffer size in bytes (up to 58 bytes used)
7885 ; ES:DI -> buffer
7886 ; ****
7887 ; (Ref: Ralf Brown's Interrupt List) - had some mistakes -
7888 00000D4D 3C02 cmp al,2
7889 00000D4F 77F8 ja short errtn2
7890 00000D51 06 push es
7891 00000D52 16 push ss
7892 00000D53 750F jnz short ext_cntry_inf_1
7893 00000D55 07 pop es ; AX = GET 35 bytes info (from offset 3 to 37)
7894 ; (38 bytes buffer is used)
7895 00000D56 BF[9212] mov di,_COUNTRY_ID
7896 00000D59 268B45FE mov ax,[es:di-2] ; NEW_COUNTRY_SIZE = 38
7897 00000D5D BB0300 mov bx,3 ; skip the 1st 3 bytes of the buffer
7898 00000D60 01DE add si,bx
7899 00000D62 EB13 jmp short ext_cntry_inf_4
7900
7901 ext_cntry_inf_1:
7902 00000D64 FEC8 dec al
7903 00000D66 7506 jnz short ext_cntry_inf_2 ; AX = 2
7904 ; AX = 1 (set)
7905 00000D68 07 pop es
7906 00000D69 BF[BA12] mov di,_ENU ; "ENU"
7907 00000D6C EB04 jmp short ext_cntry_inf_3
7908
7909 ext_cntry_inf_2:
7910 00000D6E 1F pop ds ; AX = 2 (get)
7911 00000D6F BE[BA12] mov si,_ENU ; "ENU"
7912
7913 ext_cntry_inf_3: ; CODE XREF: DOSCODE:4C2FAj
7914 00000D72 31DB xor bx,bx ; 0
7915 00000D74 B83A00 mov ax,58 ; 3Ah
7916
7917 ext_cntry_inf_4:
7918 00000D77 39C1 cmp cx,ax ; > 38 ? (58)
7919 ;jb short ext_cntry_inf_5 ; no
7920 00000D79 7602 jna short ext_cntry_inf_5 ; 06/01/2024
7921 00000D7B 89C1 mov cx,ax ; yes, decrease size to 38 (58)

```



```

7922
7923
7924 00000D7D 89C8
7925 00000D7F 29D9
7926 00000D81 F3A4
7927 00000D83 07
7928 00000D84 E91D03
7929
7930
7931
7932
7933
7934
7935
7936
7937
7938
7939
7940
7941
7942
7943
7944
7945
7946
7947
7948
7949
7950
7951
7952
7953
7954
7955
7956
7957
7958
7959
7960
7961
7962
7963
7964
7965
7966
7967
7968
7969
7970
7971
7972
7973
7974
7975
7976
7977
7978 00000D87 3C20
7979 00000D89 726A
7980
7981 00000D8B A880
7982 00000D8D 7505
7983
7984
7985 00000D8F BB[FC0A]
7986 00000D92 EB05
7987
7988
7989
7990
7991
7992 00000D94 247F
7993
7994
7995 00000D96 BB[7E0B]
7996
7997 00000D99 3C20
7998 00000D9B 750D
7999 00000D9D 88D0
8000 00000D9F E81A50
8001 00000DA2 E8D2F6
8002
8003 00000DA5 884406
8004 00000DA8 EB24
8005
8006 00000DAA 3C23
8007 00000DAC 7523
8008
8009 00000DAE 31C0
8010
8011
8012
8013
8014
8015
8016
8017
8018
8019 00000DB0 363A16[3613]
8020 00000DB5 7416
8021
8022 00000DB7 363A16[3813]
8023 00000DBC 740F
8024
8025 00000DBE 363A16[3713]
8026 00000DC3 7409
8027
8028 00000DC5 363A16[3913]
8029 00000DCA 7402
8030
8031 00000DCC 40
8032
8033 00000DCD 40
8034
8035
8036
8037
8038
8039
8040 00000DCE E99DF8
8041
8042
8043 00000DD1 89D6
8044 00000DD3 3C21
8045 00000DD5 750D

ext_cntry_inf_5:
    mov     ax,cx          ; buffer (filled) size
    sub     cx,bx          ; copy byte count
    rep movsb
    pop     es
    jmp     ret_ax_to_user_cx ; ax -> user's cx

; -----
; 19/07/2018
;BREAK <$GetExtCntry - return extended country-dependent information>
; -----
;
; Procedure Name : $GetExtCntry
;
; Inputs:
;   if AL >= 20H
;   AL= 20H   capitalize single char, DL= char
;   21H   capitalize string, CX= string length
;   22H   capitalize ASCII string
;   23H   YES/NO check, DL=1st char DH= 2nd char (DBCS)
;   80H bit 0 = use normal upper case table
;   1 = use file upper case table
;   DS:DX points to string
;
; else
;
;   MOV     AH,$GetExtCntry ; DOS 3.3
;   MOV     AL,INFO_ID      ( info type,-1 selects all )
;   MOV     BX,CODE_PAGE    ( -1 = active code page )
;   MOV     DX,COUNTRY_ID   ( -1 = active country )
;   MOV     CX,SIZE         ( amount of data to return )
;   LES     DI,COUNTRY_INFO ( buffer for returned data )
;   INT     21
;
; Function:
;   give users extended country dependent information
;   or capitalize chars
;
; Outputs:
;   No Carry:
;     extended country info is succesfully returned
;
;   Carry:
;     Register AX has the error code.
;     AX=0, NO   for YES/NO CHECK
;     1, YES
; -----
;
;procedure   $GetExtCntry,NEAR    ; DOS 3.3
;
;   ; 06/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
;   ; 06/01/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
;
;   ; MSDOS 6.0
;_GetExtCntry:
;   CMP     AL,CAP_ONE_CHAR      ; < 20H ?
;   JB      short notcap
;
;capcap:
;   TEST    AL,UPPER_TABLE ; 80h ; which upper case table
;   JNZ     short fileupper      ; file upper case
;
;hkn; UCASE_TAB in DOSDATA
;   MOV     BX,UCASE_TAB+2      ; get normal upper case
;   JMP     SHORT capit
;
;fileupper:
;   ; 06/01/2024 (PCDOS 7.1 IBMDOS.COM - DOSCODE:4C57h)
;   ; ((Note: This must be a bugfix, because bit 7 of AX is 1 here!))
;   ; AL >= 80h
;   and     al,7Fh
;
;hkn; FILE_UCASE_TAB in DOSDATA
;   MOV     BX,FILE_UCASE_TAB+2 ; get file upper case
;
;capit:
;   CMP     AL,CAP_ONE_CHAR ; 20h ; cap one char ?
;   JNZ     short chkyes      ; no
;   MOV     AL,DL              ; set up AL
;   call    GETLET3            ; upper case it
;   call    Get_user_stack     ; get user stack
;   mov     [si+6],al
;   MOV     [SI+user_env.user_DX],AL ; user's DL=AL
;   JMP     SHORT nono         ; done
;
;chkyes:
;   CMP     AL,CHECK_YES_NO    ; 23h ; check YES or NO ?
;   JNZ     short capstring    ; no
;
;   XOR     AX,AX              ; presume NO
;
;hkn; NLS_YES, NLS_NO, NLS_yes2, NLS_no2 is defined in msdos.cl3 which is
;hkn; included in yesno.asm in the DOSCODE segment.
;
;   ; 29/01/2026 - Retro DOS v5.0 BugFix
;   ; cs: -> ss:
;   ; 06/08/2018 - Retro DOS v3.0
;   ; 13/05/2019 - Retro DOS v4.0
;   ;cmp     dl,'y'
;   CMP     DL,[ss:NLS_YES]      ; is 'Y' ?
;   JZ      short yesyes        ; yes
;   ;cmp     dl,'y'
;   CMP     DL,[ss:NLS_yes2]     ; is 'y' ?
;   JZ      short yesyes        ; yes
;   ;cmp     dl,'n'
;   CMP     DL,[ss:NLS_NO]       ; is 'N'?
;   JZ      short nono          ; no
;   ;cmp     dl,'n'
;   CMP     DL,[ss:NLS_no2]      ; is 'n' ?
;   JZ      short nono          ; no
;
;dbcs_char:
;   INC     AX                  ; not YES or NO
;
;yesyes:
;   INC     AX                  ; return 1
;   ; 15/12/2022
;   ; 09/01/2024 - Retro DOS v5.0
;
;nono:
;   jmp     short SYS_RET_OK_jmp ;
;   ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
;   ; 09/01/2024
;   jmp     SYS_RET_OK          ; done
;
;capstring:
;   MOV     SI,DX                ; si=dx
;   CMP     AL,CAP_STRING ; 21h ; cap string ?
;   JNZ     short capasci      ; no

```

```

8046      ;OR      CX,CX      ; check count 0
8047      ;JZ      short nono      ; yes finished
8048      ; 06/01/2024
8049 0000DD7 E3F5      jcxz      nono
8050
8051      concap:      ;
8052      LODSB      ; get char
8053      call      GETLET3      ; upper case it
8054      MOV      byte [SI-1],AL      ; store back
8055      ;next99:      ;
8056      LOOP      concap      ; continue
8057      JMP      short nono      ; done
8058      capascii:      ;
8059      CMP      AL,CAP_ASCIIIZ ; 22h      ; cap ASCIIIZ string ?
8060      JNZ      short capinval      ; no
8061      concap2:      ;
8062      LODSB      ; get char
8063      or      al,al      ; end of string ?
8064      JZ      short nono      ; yes
8065      call      GETLET3      ; upper case it
8066      MOV      [SI-1],AL      ; store back
8067      JMP      short concap2      ; continue
8068
8069      ; MSDOS 3.3 (& MSDOS 6.0)
8070
8071      ; Offset 1A19h in IBMDOS.COM (MSDOS 3.3), 1987
8072      ; _$GetExtCntry:
8073
8074      notcap:      ;
8075      CMP      CX,5      ; minimum size is 5
8076      jnb      short sizeerror
8077      ; 09/01/2024
8078      jnb      short capinval      ; (size error)
8079
8080      GEC_CONT:
8081      ;hkn; SS is DOSDATA
8082      ;context DS
8083
8084      push      ss
8085      ;pop      es ; ! (Retro DOS v3.0 BUG) !
8086      pop      ds ; 13/05/2019 - Retro DOS v4.0
8087
8088      ;hkn; COUNTRY_CDPG is in DOSDATA
8089      MOV      SI,COUNTRY_CDPG
8090
8091      ; -----
8092      ; 06/01/2024 - Retro DOS v5.0
8093      ; PC DOS 7.1 IBMDOS.COM - DOSCODE:4CC6h
8094      ;;;
8095      or      al,al
8096      jnz      short GETCNTRY
8097      ; AL = 0 (INT 21h, AX=6500h)
8098      ; Set extended country-dependent information
8099      ; (SET GENERAL INTERNATIONALIZATION INFO)
8100
8101      sub      cx,7      ; minimum 8 bytes
8102      jbe      short capinval      ; error_invalid_function
8103      mov      bx,[si+48h]      ; [SI+DOS_CCDPG.ccSysCodePage]
8104      lea      si,[si+66h]      ; SI+DOS_CCDPG.ccCountryInfoLen
8105      mov      ax,[si]
8106      sub      ax,4
8107      cmp      cx,ax
8108      jbe      short set_intern_inf
8109      mov      cx,ax
8110      set_intern_inf:
8111      mov      ax,cx
8112      add      ax,4
8113      mov      [es:di+1], ax      ; info length/size (will be written)
8114      add      si,6      ; DOS_CCDPG.ccDFormat
8115      add      di,7      ; points to date format
8116      call     XCHGP      ; ds:si = user's buffer + 6
8117      ; es:di = country info buffer + 7
8118      jmp      short OK_RETN
8119      ;;;
8120      ; -----
8121
8122      ; 06/01/2024
8123      GETCNTRY:
8124      CMP      DX,-1 ; COUNTRY_ID      ; active country ?
8125      JNZ      short GETCDPG      ; no
8126
8127      ;hkn; use DS override to accesss country_cdpd fields
8128      ;mov      dx,[si+63h] ; MSDOS 3.3
8129      ;mov      dx,[si+68h] ; MSDOS 6.0
8130      MOV      DX,[SI+DOS_CCDPG.ccDosCountry]
8131      ; get active country id;smr;use DS
8132
8133      GETCDPG:
8134      CMP      BX,-1 ; CODE_PAGE      ; active code page?
8135      JNZ      short CHKAGAIN      ; no, check again
8136
8137      ;hkn; use DS override to accesss country_cdpd fields
8138      ;mov      bx,[si+65h] ; MSDOS 3.3
8139      ;mov      bx,[si+6Ah] ; MSDOS 6.0
8140      MOV      BX,[SI+DOS_CCDPG.ccDosCodePage]
8141      ; get active code page id;smr;Use DS
8142
8143      CHKAGAIN:
8144      cmp      dx,[si+68h] ; MSDOS 6.0
8145      CMP      DX,[SI+DOS_CCDPG.ccDosCountry]
8146      ; same as active country id?;smr;use DS
8147      JNZ      short CHKMLS      ; no
8148      ;mov      bx,[si+6Ah] ; MSDOS 6.0
8149      CMP      BX,[SI+DOS_CCDPG.ccDosCodePage]
8150      ; same as active code pg id?;smr;use DS
8151      JNZ      short CHKMLS      ; no
8152
8153      CHKTYPE:
8154      ;mov      bx,[si+48h]
8155      MOV      BX,[SI+DOS_CCDPG.ccSysCodePage]
8156      ; bx = sys code page id;smr;use DS
8157      ; save CX
8158      PUSH     CX
8159      ;mov      cx,[si+4Ah]
8160      MOV      CX,[SI+DOS_CCDPG.ccNumber_of_entries] ;smr;use DS
8161      ;mov      si,COUNTRY_CDPG+76
8162      MOV      SI,COUNTRY_CDPG+DOS_CCDPG.ccSetUcase ;smr;CDPG in DOSDATA
8163      NXTENTRY:
8164      CMP      AL,[SI]      ; compare info type;smr;use DS
8165      JZ      short FOUNDIT
8166      ADD      SI,5      ; next entry
8167      LOOP     NXTENTRY
8168      POP      CX
8169      capinval:
8170      ;error      error_invalid_function; info type not found
8171      ;mov      al,1
8172      mov      al,error_invalid_function
8173      ;SYS_RET_ERR_jmp:
8174      jmp      SYS_RET_ERR

```

```

8170             ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
8171 SYS_RET_ERR_jmp:
8172 00000E5D E918F8 jmp     SYS_RET_ERR
8173
8174 FOUNDIT:
8175 MOVSB             ; move info id byte
8176 POP CX            ; restore char count
8177 ;cmp al,1
8178 CMP AL,SetCountryInfo ; select country info type ?
8179 JZ short setsize
8180 MOV CX,4           ; 4 bytes will be moved
8181 MOV AX,5           ; 5 bytes will be returned in CX
8182
8183 OK_RET:
8184 REP MOVSB          ; copy info
8185 MOV CX,AX           ; CX = actual length returned
8186 MOV AX,BX           ; return sys code page in ax
8187
8188 GETDONE:
8189 call Get_User_Stack ; return actual length to user's CX
8190 ;mov [si+4],cx
8191 MOV [SI+user_env.user_CX],CX
8192 ;jmp SYS_RET_OK
8193 ; 15/12/2022
8194 ; 25/06/2019
8195 jmp SYS_RET_OK_c1c
8196 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
8197 ; 15/12/2022
8198 ;nono_jmp:
8199 ;jmp short nono
8200 setsize:
8201 SUB CX,3           ; size after length field
8202 CMP [SI],CX        ; less than table size ;smr;use ds
8203 JAE short setsize2 ; no
8204 MOV CX,[SI]         ; truncate to table size ;smr;use ds
8205
8206 setsize2:
8207 MOV [ES:DI],CX      ; copy actual length to user's buffer
8208 ;ADD DI,2           ; update index
8209 ;ADD SI,2
8210 ; 06/01/2024
8211 inc di
8212 inc di
8213 inc si
8214 inc si
8215 MOV AX,CX
8216 ADD AX,3           ; AX has the actual length
8217 JMP short OK_RET    ; go move it
8218
8219 CHKNLS:
8220 XOR AH,AH
8221 ;PUSH AX            ; save info type
8222 ;POP BP             ; bp = info type
8223 ; 06/01/2024
8224 mov bp,ax
8225
8226 ;CallInstall NLSInstall,NLSFUNC,0 ; check if NLSFUNC in memory
8227 mov ax,1400h
8228 int 2Fh           ; - Multiplex - NLSFUNC.COM - INSTALLATION CHECK
8229 ; Return: AL = 00h not installed, OK to install
8230 ; 01h not installed, not OK
8231 ; FFh installed
8232 CMP AL,0FFh
8233 JZ short NLSNXT    ; in memory
8234
8235 sizeerror:
8236 ; 09/01/2024 - Retro DOS v5.0
8237 ;
8238 ;error error_invalid_function
8239 ;mov al,1
8240 ;mov al,error_invalid_function
8241 ;jmp SYS_RET_ERR
8242 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
8243 ;sys_ret_err_jmp2:
8244 ; jmp short SYS_RET_ERR_jmp
8245 ; 09/01/2024
8246 jmp short capinval
8247
8248 NLSNXT:
8249 ;CallInstall GetExtInfo,NLSFUNC,2 ;get extended info
8250 mov ax,1402h
8251 int 2Fh           ; - Multiplex - NLSFUNC.COM - GET COUNTRY INFO
8252 ; BP = subfunction, BX = code page
8253 ; DX = country code, DS:SI -> internal code page structure
8254 ; ES:DI -> user buffer, CX = size of user buffer
8255 ; Return: AL = status
8256 ; 00h successful
8257 ; else DOS error code
8258
8259 CMP AL,0           ; success ?
8260 JNZ short NLSERROR
8261 ;mov ax,[si+48h] ; 13/05/2019
8262 MOV AX,[SI+DOS_CCDPG.ccSysCodePage]
8263 ; ax = sys code page id;smr;use ds;
8264 ;BUGBUG;check whether DS is OK after the above calls
8265 JMP short GETDONE
8266
8267 seterr:
8268 ; 15/12/2022
8269 NLSERROR:
8270 ;jmp SYS_RET_ERR ; return what is got from NLSFUNC
8271 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
8272 ;jmp short sys_ret_err_jmp2
8273 ; 15/12/2022
8274 jmp short SYS_RET_ERR_jmp
8275
8276 ;EndProc $GetExtCntry
8277
8278 ; 13/05/2019 - Retro DOS v4.0
8279 ; DOSCODE:4BD6h (MSDOS 6.21, MSDOS.SYS)
8280
8281 ;BREAK <$GetSetCdPg - get or set global code page>
8282 ;-----
8283 ;** $GetSetCdPg - Get or Set Global Code Page
8284 ;
8285 ; System call format:
8286 ;
8287 ; MOV AH,GetSetCdPg ; DOS 3.3
8288 ; MOV AL,n ; n = 1 : get code page, n = 2 : set code page
8289 ; MOV BX,CODE_PAGE (set code page only)
8290 ; INT 21
8291 ;
8292 ; ENTRY (al) = n
8293 ; (bx) = code page
8294 ; EXIT 'c' clear
8295 ; global code page is set (set global code page)
8296 ; (BX) = active code page id (get global code page)
8297 ; (DX) = system code page id (get global code page)
8298 ; 'c' set
8299 ;

```

```

8294 ; (AX) = error code
8295 ;
8296 ;procedure $GetSetCdPg,NEAR ; DOS 3.3
8297 ;
8298 ; 06/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
8299 ; DOSCODE:4BC9h
8300 ;
8301 ; 06/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
8302 ; DOSCODE:4D73h
8303
8304 _$GetSetCdPg:
8305 ;hkn; SS is DOSDATA
8306 ;context DS
8307
8308 push ss
8309 00000EB1 16 pop ds
8310 00000EB2 1F
8311
8312 ;hkn; COUNTRY_CDPG is in DOSDATA
8313 00000EB3 BE[2A12] MOV SI,COUNTRY_CDPG ; (DOSDATA:122Ah for MSDOS 6.21)
8314
8315 00000EB6 3C01 CMP AL,1 ; get global code page
8316 00000EB8 7512 JNZ short setglpg ; set global code page
8317
8318 ;;mov bx,[si+65h] ; MSDOS 3.3
8319 ;;mov bx,[si+6Ah] ; MSDOS 6.0
8320 00000EBA 8B5C6A MOV BX,[SI+DOS_CCDPG.ccdosCodePage] ; get active code page id;smr;use ds
8321
8322 ;mov dx,[si+48h]
8323 00000EBD 8B5448 MOV DX,[SI+DOS_CCDPG.ccSysCodePage] ; get sys code page id;smr;use ds
8324
8325 00000EC0 E8B4F5 call Get_User_Stack
8326 ;ASSUME DS:NOTHING
8327 ;;mov [si+2],bx
8328 ;MOV [SI+user_env.user_BX],BX ; update returned bx
8329 ; 06/01/2024 (PCDOS 7.1 IBMDOS.COM)
8330 00000EC3 E8F4FD call set_user_bx ; MOV [SI+user_env.user_BX],BX
8331 ;mov [si+6],dx
8332 00000EC6 895406 MOV [SI+user_env.user_DX],DX ; update returned dx
8333
8334 OK_RETURN:
8335 ; 15/12/2022
8336 00000EC9 E9A2F7 ;transfer SYS_RET_OK
8337 jmp SYS_RET_OK
8338 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
8339 ;jmp short nono_jmp
8340
8341 ;hkn; ASSUME DS:DOSGROUP
8342 ;ASSUME DS:DOSDATA
8343
8344 00000ECC 3C02 setglpg:
8345 00000ECE 752F CMP AL,2
8346 JNZ short nomem
8347
8348 ;;mov dx,[si+63h] ; MSDOS 3.3
8349 00000ED0 8B5468 ;mov dx,[si+68h] ; MSDOS 6.0
8350 MOV DX,[SI+DOS_CCDPG.ccdosCountry] ;smr;use ds
8351
8352 ;CallInstall NLSInstall,NLSFUNC,0 ; check if NLSFUNC in memory
8353 00000ED3 B80014 mov ax,1400h
8354 00000ED6 CD2F int 2Fh ; - Multiplex - NLSFUNC.COM - INSTALLATION CHECK
8355 ; Return: AL = 00h not installed, OK to install
8356 ; 01h not installed, not OK
8357 ; FFh installed
8358 00000ED8 3CFF CMP AL,0FFh
8359 00000EDA 7523 JNZ short nomem ; not in memory
8360
8361 ;CallInstall SetCodePage,NLSFUNC,1 ;set the code page
8362 00000EDC B80114 mov ax,1401h
8363 00000EDF CD2F int 2Fh ; - Multiplex - NLSFUNC.COM - CHANGE CODE PAGE
8364 ; DS:SI -> internal code page structure
8365 ; BX = new code page, DX = country code???
8366 ; Return: AL = status
8367 ; 00h successful
8368 ; else DOS error code
8369 00000EE1 08C0 ;cmp al,0
8370 00000EE3 74E4 or al,al ; success ?
8371 JZ short OK_RETURN ; yes
8372
8373 00000EE5 3C41 CMP AL,65 ; set device code page failed
8374 00000EE7 75C6 JNZ short seterr
8375 ;MOV AX,65
8376 ; 06/01/2024
8377 00000EE9 98 cbw
8378 00000EEA A3[2403] MOV [EXTERR],AX
8379 ;mov byte [EXTERR_ACTION],6
8380 ;mov byte [EXTERR_CLASS],5
8381 ;mov byte [EXTERR_LOCUS],4
8382 00000EED C606[2603]06 MOV byte [EXTERR_ACTION],errACT_Ignore
8383 00000EF2 C606[2703]05 MOV byte [EXTERR_CLASS],errCLASS_HrdFail
8384 00000EF7 C606[2303]04 MOV byte [EXTERR_LOCUS],errLOC_SerDev
8385 ;transfer From_GetSet
8386 jmp From_GetSet
8387
8388 ; 15/12/2022
8389 ;seterr:
8390 ;;;transfer SYS_RET_ERR
8391 ;;;jmp SYS_RET_ERR
8392 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
8393 ;jmp short NLSERROR
8394
8395 nomem:
8396 ;error error_invalid_function; function not defined
8397 ;mov al,1
8398 00000EFF B001 mov al,error_invalid_function
8399 00000F01 EBAC jmp short seterr
8400
8401 ;EndProc $GetSetCdPg
8402
8403 ; 13/05/2019 - Retro DOS v4.0
8404 ; DOSCODE:4C2Bh (MSDOS 6.21, MSDOS.SYS)
8405
8406 ;BREAK <$Get_Drive_Freespace -- Return bytes of free disk space on a drive>
8407 ;-----
8408 ;** $Get_Drive_Freespace - Return amount of drive free space
8409 ;
8410 ; $Get_Drive_Freespace returns the # of free allocation units on a
8411 ; drive.
8412 ;
8413 ; This call returns the same info in the same registers (except for the
8414 ; FAT pointer) as the old FAT pointer calls
8415 ;
8416 ; ENTRY DL = Drive number
8417 ; EXIT AX = Sectors per allocation unit
8418 ; = -1 if bad drive specified

```

```

8418 ; On User Stack
8419 ; BX = Number of free allocation units
8420 ; DX = Total Number of allocation units on disk
8421 ; CX = Sector size
8422
8423 ;procedure $GET_DRIVE_FREESPACE,NEAR
8424
8425 ; 09/01/2024
8426 ; 06/01/2024 - Retro DOS v5.0
8427 ; (PCDOS 7.1 IBMDOS.COM - DOSCODE:4DC4h)
8428
8429 _$GET_DRIVE_FREESPACE:
8430
8431 ;hkn; SS is DOSDATA
8432 ;context DS
8433 0000F03 16 push ss
8434 0000F04 1F pop ds
8435
8436 0000F05 88D0 MOV AL,DL
8437 ;invoke GetThisDrv ; Get drive
8438 0000F07 E8836C call GETTHISDRV
8439 SET_AX_RET:
8440 0000F0A 7220 JC short BADFDRV
8441
8442 ;invoke DISK_INFO
8443 0000F0C E85424 call DISK_INFO
8444 ; 09/01/2024
8445 ;XCHG DX,BX
8446 ;JC short SET_AX_RET ; User FAILED to I 24
8447 ; 26/06/2024
8448 ; 06/01/2024
8449 ;JC short BADFDRV
8450 ; 26/06/2024
8451 0000F0F 7304 jnc short gdrvfspc_1
8452 0000F11 87D3 xchg dx,bx
8453 0000F13 EB17 jmp short BADFDRV
8454
8455 ; 09/01/2024 - Retro DOS v5.0 (PCDOS 7.1)
8456 gdrvfspc_1:
8457 0000F15 30E4 XOR AH,AH ; Chuck Fat ID byte
8458 ;;;
8459 0000F17 57 push di
8460 0000F18 E80E09 call TestNet
8461 0000F1B 5F pop di
8462 0000F1C 7203 JC short gdrvfspc_2
8463 0000F1E E84625 call modify_cluster_count
8464 ; if hw of total clusters (di) > 0
8465 ; sectors per cluster and cluster counts
8466 ; will be modified (shifted)
8467 ; (but sectors per clust * clust count will be same)
8468 ; /// disk size -calculation- limit = 2 GB ///
8469
8470 0000F21 87D3 gdrvfspc_2:
8471 xchg dx,bx ; bx = free clusters (after xchg)
8472 ;;;
8473 0000F23 E851F5 DoSt:
8474 call Get_User_Stack
8475 ;ASSUME DS:NOTHING
8476 ;mov [si+6],dx
8477 ;mov [si+4],cx
8478 ;mov [si+2],bx
8479 ; 09/01/2024
8480 MOV [SI+user_env.user_DX],DX ; total clusters
8481 ;MOV [SI+user_env.user_CX],CX
8482 ;MOV [SI+user_env.user_BX],BX
8483 ;MOV [SI+user_env.user_AX],AX
8484 ;mov [si],ax
8485 ;return
8486 ;retn
8487 0000F29 E989FD ; 09/01/2024
8488 jmp gdrvfspc_ret ; ax = sectors per cluster (modified)
8489
8490 BADFDRV:
8491 ; MSDOS 3.3
8492 ;mov al,0Fh
8493 ;mov al,error_invalid_drive; Assume error
8494
8495 ; 13/05/2019 - Retro DOS v4.0
8496
8497 ; MSDOS 6.0 & MSDOS 3.3
8498 0000F2C E85EF7 ;invoke FCB_RET_ERR
8499 call FCB_RET_ERR
8500
8501 0000F32 EBEF MOV AX,-1
8502 JMP short DoSt
8503
8504 ;EndProc $GET_DRIVE_FREESPACE
8505
8506 ; BREAK <$Get_DMA, $Set_DMA -- Get/Set current DMA address>
8507 ;-----
8508 ** $Get_DMA - Get Disk Transfer Address
8509 ;
8510 ; ENTRY none
8511 ; EXIT ES:BX is current transfer address
8512 ; USES all
8513
8514 ; 09/01/2024
8515 _$GET_DMA:
8516 0000F34 368B1E[2C03] MOV BX,[SS:DMAADD]
8517 0000F39 368B0E[2E03] MOV CX,[SS:DMAADD+2]
8518 0000F3E E836F5 call Get_User_Stack
8519 ;mov [si+2],bx
8520 ;mov [si+10h],cx
8521 ; 09/01/2024
8522 0000F41 894C10 MOV [SI+user_env.user_BX],BX
8523 MOV [SI+user_env.user_ES],CX
8524 ;retn
8525 ; 09/01/2024
8526 0000F44 E973FD jmp set_user_bx
8527
8528 ** $Set_DMA - Set Disk Transfer Address
8529 ;-----
8530 ; ENTRY DS:DX is current transfer address
8531 ; EXIT none
8532 ; USES all
8533
8534 _$SET_DMA:
8535 0000F47 368916[2C03] MOV [SS:DMAADD],DX
8536 0000F4C 368C1E[2E03] MOV [SS:DMAADD+2],DS
8537 0000F51 C3 retn
8538
8539 ; BREAK <$Get_Default_Drive, $Set_Default_Drive -- Set/Get default drive>
8540 ;-----
8541 ** $Get_Default_Drive - Get Current Default Drive

```

```

8542 ;-----
8543 ; ENTRY none
8544 ; EXIT (AL) = drive number
8545 ; USES all
8546
8547 _$GET_DEFAULT_DRIVE:
8548 00000F52 36A0[3603] MOV AL,[SS:CURDRV]
8549 00000F56 C3 retn
8550
8551 ;** $Set_Default_Drive - Specify new Default Drive
8552 ;-----
8553 ; ENTRY (DL) = Drive number for new default drive
8554 ; EXIT (AL) = Number of drives, NO ERROR RETURN IF DRIVE NUMBER BAD
8555
8556 _$SET_DEFAULT_DRIVE:
8557 00000F57 88D0 MOV AL,DL
8558 00000F59 FEC0 INC AL ; A=1, B=2...
8559 00000F5B E8136C call GetVisDrv ; see if visible drive
8560 00000F5E 7204 JC short SETRET ; errors do not set
8561 00000F60 36A2[3603] MOV [SS:CURDRV],AL ; no, set
8562
8563 SETRET:
8564 00000F64 36A0[4700] MOV AL,[SS:CDSCOUNT] ; let user see what the count really is
8565 00000F68 C3 retn
8566
8567 ;BREAK <$Get/Set_Interrupt_Vector - Get/Set interrupt vectors>
8568 ;-----
8569
8570 ;** $Get_Interrupt_Vector - Get Interrupt Vector
8571 ;-----
8572 ; $Get_Interrupt_Vector is the official way for user pgms to get the
8573 ; contents of an interrupt vector.
8574 ;
8575 ; ENTRY (AL) = interrupt number
8576 ; EXIT (ES:BX) = current interrupt vector
8577
8578 _$GET_INTERRUPT_VECTOR:
8579 00000F69 E82E00 CALL RECSET
8580 00000F6C 26C41F LES BX,[ES:BX]
8581 00000F6F E805F5 call Get_User_Stack
8582
8583 set_user_es_bx:
8584 ; 09/01/2024 (PCDOS 7.1 IBMDOS.COM)
8585 ; mov [si+2],bx
8586 ; mov [si+10h],es
8587 00000F72 8C4410 MOV [SI+user_env.user_BX],BX
8588 ; mov [SI+user_env.user_ES],ES
8589 00000F75 E942FD ; retn
8590 jmp set_user_bx
8591
8592 ;** $Set_Interrupt_Vector - Set Interrupt Vector
8593 ;-----
8594 ; $Set_Interrupt_Vector is the official way for user pgms to set the
8595 ; contents of an interrupt vector.
8596 ;
8597 ; M004, M068: Also set A20OFF_COUNT to 1 if EXECA200FF bit has been set
8598 ; and if A200FF_COUNT is non-zero. See under tag M003 in inc\dosym.inc
8599 ; for explanation.
8600 ;
8601 ; ENTRY (AL) = interrupt number
8602 ; (ds:dx) = desired new vector value
8603 ; EXIT none
8604 ; USES all
8605
8606 ; 07/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
8607 ; 13/05/2019 - Retro DOS v4.0
8608
8609 _$SET_INTERRUPT_VECTOR:
8610 00000F78 E81F00 CALL RECSET
8611 00000F7B FA CLI ; Watch out!!!! Folks sometimes use
8612 00000F7C 268917 MOV [ES:BX],DX ; this for hardware ints (like timer).
8613 00000F7F 268C5F02 MOV [ES:BX+2],DS
8614 00000F83 FB STI
8615 ; M004, M068 - Start
8616 00000F84 36F606[8600]04 test byte [ss:DOS_FLAG],EXECA200FF ; 4
8617 ; Q: was the previous call an int 21h
8618 ; exec call
8619 ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
8620 ; jnz short siv_1 ; Y: go set count
8621 ; retn ; N: return
8622 ; 15/12/2022
8623 00000F8A 740D jz short siv_2
8624
8625 siv_1:
8626 00000F8C 36803E[8500]00 cmp byte [ss:A200FF_COUNT],0 ; Q: is count 0
8627 00000F92 7505 jnz short siv_2 ; N: done
8628 ; 20/09/2023
8629 00000F94 36FE06[8500] inc byte [ss:A200FF_COUNT]
8630 ; mov byte [ss:A200FF_COUNT],1 ; Y: set it to 1 to indicate to dos
8631 ; dispatcher to turn A20 Off before
8632 ; returning to user.
8633
8634 siv_2:
8635 ; 07/12/2022
8636 00000F99 C3 retn ; M004, M068 - End
8637
8638 RECSET:
8639 00000F9A 31DB XOR BX,BX
8640 00000F9C 8EC3 MOV ES,BX
8641 00000F9E 88C3 MOV BL,AL
8642 00000FA0 D1E3 SHL BX,1
8643 00000FA2 D1E3 SHL BX,1
8644 00000FA4 C3 retn
8645
8646 ; BREAK <$Char_Oper - hack on paths, switches so that xenix can look like PCDOS>
8647 ;-----
8648
8649 ;** $Char_Oper - Manipulate Switch Character
8650 ;
8651 ; This function was put in to facilitate XENIX path/switch compatibility
8652 ;
8653 ; ENTRY AL = function:
8654 ; 0 - read switch char
8655 ; 1 - set switch char (char in DL)
8656 ; 2 - read device availability
8657 ; Always returns available
8658 ; 3 - set device availability
8659 ; No longer supported (NOP)
8660 ; EXIT (al) = 0xff iff error
8661 ; (al) != 0xff if ok
8662 ; (dl) = character/flag, if "read switch char" subfunction
8663 ; USES AL, DL
8664 ;
8665 ; NOTE This already obsolete function has been deactivated in DOS 5.0
; The character / is always returned for subfunction 0,
; subfunction 2 always returns -1, all other subfunctions are ignored.

```



```

8666
8667 ; 13/05/2019 - Retro DOS v4.0
8668 ; DOSCODE:4CC9h (MSDOS 6.21, MSDOS.SYS)
8669
8670 ; 06/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
8671 ; DOSCODE:4CBCh (MSDOS 5.0, MSDOS.SYS)
8672
8673 _$CHAR_OPER:
8674 ; MSDOS 6.0
8675 00000FA5 08C0 or al,al ; get switch?
8676 00000FA7 B22F mov dl, '/' ; assume yes
8677 00000FA9 7407 jz short chop_1 ; jump if yes
8678 00000FAB 3C02 cmp al,2 ; check device availability?
8679 00000FAD B2FF mov dl,-1 ; assume yes
8680 00000FAF 7401 jz short chop_1 ; jump if yes
8681 00000FB1 C3 retn ; otherwise just quit
8682
8683 ; subfunctions requiring return of value to user come here. DL holds
8684 ; value to return
8685
8686 chop_1:
8687 00000FB2 E8C2F4 call Get_User_Stack
8688 00000FB5 895406 mov [SI+user_env.user_DX],dx ; store value for user
8689 00000FB8 C3 retn
8690
8691 ; MSDOS 3.3
8692 ; Offset 1B87h in IBMDOS.COM (MSDOS 3.3), 1987
8693 ; push ss
8694 ; pop ds
8695 ; cmp al,1
8696 ; jb short chop_1
8697 ; jz short chop_2
8698 ; cmp al,3
8699 ; jb short chop_3
8700 ; jz short chop_5
8701 ; mov al,0FFh
8702 ; retn
8703 chop_1:
8704 ; mov dl,[chSwitch]
8705 ; jmp short chop_4
8706 chop_2:
8707 ; mov [chSwitch],dl
8708 ; retn
8709 chop_3:
8710 ; mov dl, FFh
8711 chop_4:
8712 ; call Get_User_Stack
8713 ; mov [si+6],dx
8714 chop_5:
8715 ; retn
8716
8717 ;** $GetExtendedError - Return Extended error code
8718 -----
8719 ; This function reads up the extended error info from the static
8720 ; variables where it was stored.
8721 ;
8722 ; ENTRY none
8723 ; EXIT AX = Extended error code (0 means no extended error)
8724 ; BL = recommended action
8725 ; BH = class of error
8726 ; CH = locus of error
8727 ; ES:DI = may be pointer
8728 ; USES ALL
8729
8730 ; 06/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
8731
8732 _$GetExtendedError:
8733 00000FB9 16 push ss
8734 00000FBA 1F pop ds
8735 00000FBB A1[2403] MOV AX,[EXTErr]
8736 00000FBE C43E[2803] LES DI,[EXTErrPT]
8737 00000FC2 8B1E[2603] MOV BX,[EXTErr_ACTION] ; BL = Action, BH = Class
8738 00000FC6 8A2E[2303] MOV CH,[EXTErr_LOCUS]
8739 00000FCA E8AAF4 call Get_User_Stack
8740 ; mov [si+0Ah],di
8741 00000FCD 897C0A MOV [SI+user_env.user_DI],DI
8742
8743 ; 09/01/2024 (PCDOS 7.1 IBMDOS.COM)
8744 ; mov [si+10h],es
8745 ; MOV [SI+user_env.user_ES],ES
8746 ; mov [si+2],bx
8747 ; MOV [SI+user_env.user_BX],BX
8748 00000FD0 E89FFF call set_user_es_bx
8749
8750 ; mov [si+4],cx
8751 00000FD3 894C04 MOV [SI+user_env.user_CX],CX
8752 jmp_SYS_RET_OK:
8753 ; 15/12/2022
8754 ; jmp SYS_RET_OK
8755 ; 25/06/2019
8756 00000FD6 E998F6 jmp SYS_RET_OK_c1c ; 15/12/2022
8757 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
8758 ; jmp_SYS_RET_OK:
8759 ; jmp SYS_RET_OK
8760
8761 ; -----
8762 ; 09/01/2024
8763 %if 0
8764 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
8765 ; DOSCODE:4CF3h
8766 ; patch_or_unknown:
8767 ; get_code_page:
8768 push si
8769 mov si, COUNTRY_CDPG
8770 mov ax, [si+DOS_CDPG.ccDosCodePage]
8771 mov ax, [ss:si+6Ah]
8772 pop si
8773 retn
8774 %endif
8775 ; -----
8776
8777 ; 29/04/2019 - Retro DOS v4.0
8778
8779 ;BREAK <ECS_call - Extended Code System support function>
8780 -----
8781 ; Inputs:
8782 ; AL = 0 get lead byte table
8783 ; on return DS:SI has the table location
8784 ;
8785 ; AL = 1 set / reset interim console flag
8786 ; DL = flag (00H or 01H)
8787 ; no return
8788 ;
8789 ; AL = 2 get interim console flag

```

```

8790 ; on return DL = current flag value
8791 ;
8792 ; AL = OTHER then error, and returns with:
8793 ; AX = error_invalid_function
8794 ;
8795 ; NOTE: THIS CALL DOES GUARANTEE THAT REGISTER OTHER THAN
8796 ; SS:SP WILL BE PRESERVED!
8797 ;-----
8798
8799 _$ECS_Call:
8800 0000FD9 08C0 or al,al ; AL = 0 (get table)?
8801 ;jnz short _okok
8802 ; 15/12/2022
8803 0000FDB 7403 jz short get_lbt
8804 ;_okok:
8805 0000FDD E98EF6 jmp SYS_RET_OK
8806 get_lbt:
8807 0000FE0 E894F4 call Get_User_Stack ; *
8808
8809 ;hkn; dbcs_table moved low to dosdata
8810 ;mov word [si+8],DBCS_TAB+2
8811 0000FE3 C74408[020D] mov word [SI+user_env.user_SI],DBCS_TAB+2
8812
8813 0000FE8 06 push es
8814 ;getdseg <es> ; es = DOSDATA
8815 0000FE9 2E8E06[0700] mov es,[cs:DosDSeg]
8816 ;mov [si+14],es
8817 0000FEE 8C440E mov [SI+user_env.user_DS],es
8818 0000FF1 07 pop es
8819
8820 ; 15/12/2022
8821 0000FF2 EBE2 jmp short jmp_SYS_RET_OK ; jmp SYS_RET_OK_clic ; *
8822 ;_okok:
8823 ; 15/12/2022
8824 ;;transfer SYS_RET_OK
8825 ;jmp short jmp_SYS_RET_OK
8826 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
8827 ;jmp SYS_RET_OK
8828 ;jmp short jmp_SYS_RET_OK
8829
8830 ;-----
8831 ; 10/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM - DOSCODE:4EB4h)
8832 ;-----
8833
8834 ; INT 21h, AH=71h - LONG FILENAME FUNCTIONS
8835 ; INT 21h, AH=72h - LFN FindClose
8836 _$LONGNAME:
8837 0000FF4 30C0 xor al,al ; longname functions are not supported
8838 lfn_error:
8839 0000FF6 E87EF4 call Get_User_Stack
8840 0000FF9 834C1601 or word [si+16h],1 ; [SI+user_env.user_F],f_Carry
8841 0000FFD F9 stc
8842 0000FFE 8904 mov [si],ax ; [SI+user_env.user_ax]
8843 0001000 C3 retn
8844 ;-----
8845
8846 ; FAT32 - EXTENDED FUNCTIONS
8847 ; INT 21h, AH=73h
8848 _$FAT32EXT:
8849 00001001 3C05 cmp al,5 ; INT 21h AX = 7305h
8850 00001003 7609 jbe short valid_fat32_ext_function
8851 00001005 B001 mov al,1 ; error_invalid_function
8852 fat32_ext_func_err:
8853 00001007 E96EF6 jmp SYS_RET_ERR
8854
8855 function_5_invalid_cx:
8856 0000100A B057 mov al,57h ; error_invalid_parameter
8857 fat32_ext_func_err_j:
8858 0000100C EBF9 jmp short fat32_ext_func_err
8859
8860 valid_fat32_ext_function:
8861 0000100E 7524 jnz short not_function_5
8862 00001010 83F9FF cmp cx,0FFFFh ; Function 5 - FAT32 - EXTENDED ABSOLUTE DISK READ/WRITE
8863 00001013 75F5 jne short function_5_invalid_cx
8864 00001015 F7C6FE9F test si,9FEEh ; read/write mode flags
8865 00001019 75EF jnz short function_5_invalid_cx
8866 0000101B 88D0 mov al,dl ; drive number, 1 = A
8867 0000101D FEC8 dec al
8868 0000101F B401 mov ah,1
8869 00001021 F7C60100 test si,1
8870 00001025 7405 jz short function_5_read
8871 00001027 E8BAF5 call FAT32_ABSDWRT ; INT 21h AX = 7305h (SI bit 0 = 1)
8872 0000102A EB03 jmp short fat32_absdrw_ret
8873
8874 function_5_read:
8875 0000102C E802F5 call FAT32_ABSDRD ; INT 21h AX = 7305h (SI bit 0 = 0)
8876 fat32_absdrw_ret:
8877 0000102F 72C5 jc short lfn_error
8878 00001031 E93AF6 jmp SYS_RET_OK
8879
8880 not_function_5:
8881 00001034 3C03 cmp al,3 ; Function 3 - FAT32 - GET EXTENDED FREE SPACE ON DRIVE
8882 00001036 7475 je short function_73_3
8883 00001038 3C02 cmp al,2
8884 0000103A 7203 jb short chk_drive_lock_flush
8885 0000103C E90503 jmp _$GET_DPB ; Function 2 - FAT32 - "Get_ExtDPB" - GET EXTENDED DPB
8886 ; Function 4 - FAT32 - Set DPB TO USE FOR FORMATTING
8887
8888 chk_drive_lock_flush:
8889 0000103F 80FA1A cmp dl,26 ; MSDOS 7 - DRIVE LOCKING AND FLUSHING
8890 00001042 7604 jbe short drv_lock_flush_1
8891 00001044 B00F mov al,0Fh ; invalid drive number
8892 drv_lock_flush_err:
8893 ;jmp short fat32_ext_func_err_j ; ax = error code
8894 ; 10/01/2024
8895 00001046 EBBF jmp short fat32_ext_func_err
8896
8897 drv_lock_flush_1:
8898 00001048 FECA dec dl
8899 0000104A 7905 jns short drv_lock_flush_2
8900 0000104C 368A16[3603] mov dl,[ss:CURDRV] ; 0 = default/current drive)
8901 drv_lock_flush_2:
8902 00001051 B600 mov dh,0
8903 00001053 89D3 mov bx,dx
8904 00001055 80F901 cmp cl,1 ; which flag to get or set
8905 00001058 7604 jbe short drv_lock_flush_3
8906 0000105A B001 mov al,1 ; error_invalid_function
8907 ;jmp short drv_lock_flush_err
8908 ; 10/01/2024
8909 0000105C EBA9 jmp short fat32_ext_func_err
8910
8911 drv_lock_flush_3:
8912 0000105E 368AA7[0813] mov ah,[ss:drive_flags+bx]
8913 00001063 08C9 or cl,cl

```



```

8914 00001065 7422      jz      short get_set_indctd_flag ; use bit 1 and bit 2
8915 00001067 08C0      or       al,al          ; get drive's dirty-buffers flag
8916 00001069 7419      jz      short get_dirty_buf_flag ; use bit 3
8917 0000106B 80E4F7    and     ah,0F7h        ; clear bit 3
8918 0000106E 80E508    and     ch,8          ; isolate bit 3 of the new flag value
8919 00001071 08EC      or       ah,ch        ; set AH bit 3 according to CH bit 3
8920 00001073 3688A7[0813] mov     [ss:drive_flags+bx],ah ; set or reset dirty buffer flag
8921 00001078 F6C408    test    ah,8          ;
8922 0000107B 7505      jnz     short set_dirty_flag_ok      ; bit 3 is set/1
8923 0000107D B0FF      mov     al,0FFh
8924 0000107F E8165A    call    FLUSHBUF
8925                          set_dirty_flag_ok:
8926 00001082 EB26      jmp     short jmp_to_SYS_RET_OK
8927
8928                          get_dirty_buf_flag:
8929 00001084 80E408    and     ah,8          ; isolate dirty buffers flag
8930 00001087 EB19      jmp     short mov_flag_cl_to_al      ; AH = new flag and 08h (bit 3 used)
8931
8932                          get_set_indctd_flag:
8933 00001089 08C0      or       al,al
8934 0000108B 7412      jz      short get_indicated_flag
8935 0000108D 80E4F9    and     ah,0F9h        ; clear bit 1 and bit 2
8936 00001090 F6C502    test    ch,2          ; new value for indicated flag
8937 00001093 7403      jz      short reset_indctd_flags ; bit 1 is zero
8938 00001095 80CC06    or       ah,6          ; set bit 1 and bit 2
8939                          reset_indctd_flags:
8940 00001098 3688A7[0813] mov     [ss:drive_flags+bx],ah
8941 0000109D EB0B      jmp     short jmp_to_SYS_RET_OK
8942
8943                          get_indicated_flag:
8944 0000109F 80E406    and     ah,6          ; AH = new flag and 06h (bits 1 and 2 used)
8945                          mov_flag_cl_to_al:
8946 000010A2 88C8      mov     al,cl          ; CODE XREF: DOSCODE:4F47^j
8947                          ; value of CL on entry
8948 000010A4 E8D0F3    call    Get_User_Stack
8949 000010A7 894404    mov     [si+4],ax      ; [SI+user_env.user_cx] ; requested flag
8950                          jmp_to_SYS_RET_OK:
8951 000010AA E9C1F5    jmp     SYS_RET_OK
8952
8953                          function_73_3:
8954 000010AD 89D6      mov     si,dx          ; FAT32 - GET EXTENDED FREE SPACE ON DRIVE
8955                          ; AX = 7303h
8956                          ; DS:DX -> ASCIZ string for drive ("C:\" or "\\SERVER\Share")
8957                          ; ES:DI -> buffer for extended free space structure
8958                          ; CX = length of buffer for extended free space
8959 000010AF E8D76E    call    DriveFromText
8960 000010B2 92        xchg    ax,dx
8961 000010B3 AD        lodsw
8962 000010B4 FECA    dec     dl
8963 000010B6 80FA1A    cmp     dl,26
8964 000010B9 7323      jnb     short func_73_3_err2
8965 000010BB 08E4      or      ah,ah
8966 000010BD 751F      jnz     short func_73_3_err2
8967 000010BF E8284D    call    PATHCHRCMP
8968 000010C2 751A      jnz     short func_73_3_err2
8969 000010C4 FEC2      inc     dl
8970 000010C6 83F92C    cmp     cx,44          ; buffer (Structure) size must be 44
8971 000010C9 721C      jb      short func_73_3_err4
8972 000010CB 26837D0200 cmp     word [es:di+2],0 ; buffer structure version (must be 0)
8973 000010D0 7511      jnz     short func_73_3_err3
8974 000010D2 16        push    ss
8975 000010D3 1F        pop     ds
8976 000010D4 88D0      mov     al,dl
8977 000010D6 E8B46A    call    GETTHISDRV
8978 000010D9 7310      jnc     short fill_efs_struct_b
8979                          func_73_3_err1:
8980 000010DB E8AFF5    call    FCB_RET_ERR
8981                          func_73_3_err2:
8982 000010DE B00F      mov     al,0Fh        ; error_invalid_drive
8983                          jmp_to_SYS_RET_ERR:
8984 000010E0 E995F5    jmp     SYS_RET_ERR
8985
8986                          func_73_3_err3:
8987 000010E3 B057      mov     al,57h        ; error_invalid_parameter
8988                          jmp_to_jmp_SYS_RET_ERR:
8989 000010E5 EBF9      jmp     short jmp_to_SYS_RET_ERR
8990
8991                          func_73_3_err4:
8992 000010E7 B018      mov     al,18h        ; error_bad_length
8993 000010E9 EBFA      jmp     short jmp_to_jmp_SYS_RET_ERR
8994
8995                          fill_efs_struct_b:
8996 000010EB E87522    call    DISK_INFO
8997 000010EE 72EB      jc      short func_73_3_err1
8998 000010F0 30E4      xor     ah,ah
8999 000010F2 56        push    si             ; si:dx = free cluster count
9000 000010F3 57        push    di             ; di:bx = number of clusters
9001 000010F4 E880F3    call    Get_User_Stack
9002 000010F7 8E4410    mov     es,[si+10h]    ; user's buffer segment (in ES)
9003 000010FA 8B7C0A    mov     di,[si+0Ah]    ; user's buffer offset/address (in DI)
9004                          ; 10/01/2024
9005 000010FD 06        push    es
9006 000010FE 1F        pop     ds ; (*)
9007                          ;
9008                          ;mov     [es:di+10h],bx ; total number of clusters on the drive
9009                          ;mov     [es:di+20h],bx ; total allocation units, without adjustment for compression
9010 000010FF 895D10    mov     [di+10h],bx
9011 00001102 895D20    mov     [di+20h],bx
9012 00001105 5B        pop     bx
9013                          ;mov     [es:di+12h],bx ; total number of clusters on the drive, hw
9014                          ;mov     [es:di+22h],bx ; total allocation units, hw
9015                          ;mov     [es:di+0Ch],dx ; number of available clusters
9016                          ;mov     [es:di+1Ch],dx ; number of available allocation units, without adjustment
9017 00001106 895D12    mov     [di+12h],bx
9018 00001109 895D22    mov     [di+22h],bx
9019 0000110C 89550C    mov     [di+0Ch],dx
9020 0000110F 89551C    mov     [di+1Ch],dx
9021 00001112 5A        pop     dx
9022                          ;mov     [es:di+0Eh],dx ; number of available clusters, hw
9023                          ;mov     [es:di+1Eh],dx ; number of available allocation units, hw
9024                          ;mov     [es:di+8],cx ; bytes per sector
9025                          ;mov     [es:di+4],ax ; sectors per cluster (with adjustment for compression)
9026 00001113 89550E    mov     [di+0Eh],dx
9027 00001116 89551E    mov     [di+1Eh],dx
9028 00001119 894D08    mov     [di+8],cx
9029 0000111C 894504    mov     [di+4],ax
9030 0000111F 89C1      mov     cx,ax
9031 00001121 F7E3      mul     bx             ; 32 bit multiplication
9032 00001123 720B      jc      short dsk_cap_calc_overf ; disk capacity calculation overflow error
9033 00001125 89C3      mov     bx,ax
9034                          ;mov     ax,[es:di+20h] ; total allocation units, 1w
9035 00001127 8B4520    mov     ax,[di+20h]
9036 0000112A F7E1      mul     cx
9037 0000112C 01DA      add     dx,bx

```

```

9038 0000112E 7305      jnb     short dsk_cap_calc_ok
9039                      dsk_cap_calc_overf:
9040 00001130 B8FFFF      mov     ax,0FFFFh      ; set to 0FFFFFFFh
9041 00001133 89C2      mov     dx,ax
9042                      dsk_cap_calc_ok:
9043                      ; 10/01/2024
9044                      ; ds = es (*)
9045                      ;mov     [es:di+1Ah],dx
9046                      ;mov     [es:di+18h],ax      ; total number of physical sectors on the drive,
9047                      ; without adjustment for compression
9048 00001135 89551A      mov     [di+1Ah],dx
9049 00001138 894518      mov     [di+18h],ax
9050 0000113B 89C8      mov     ax,cx      ; 32 bit multiplication
9051                      ;mul     word [es:di+0Eh]      ; number of available clusters, hw
9052 0000113D F7650E      mul     word [di+0Eh]
9053 00001140 720B      jc      short dsk_free_calc_overf
9054 00001142 89C3      mov     bx,ax
9055                      ;mov     ax,[es:di+0Ch]      ; number of available clusters, lw
9056 00001144 8B450C      mov     ax,[di+0Ch]
9057 00001147 F7E1      mul     cx
9058 00001149 01DA      add     dx,bx
9059 0000114B 7305      jnc     short dsk_free_calc_ok
9060                      dsk_free_calc_overf:
9061 0000114D B8FFFF      mov     ax,0FFFFh
9062 00001150 89C2      mov     dx,ax
9063                      dsk_free_calc_ok:
9064                      ; 10/01/2024
9065                      ;mov     [es:di+16h],dx      ; hw
9066                      ;mov     [es:di+14h],ax      ; number of physical sectors available on the drive,
9067                      ; without adjustment for compression
9068 00001152 895516      mov     [di+16h],dx
9069 00001155 894514      mov     [di+14h],ax
9070 00001158 31C0      xor     ax,ax      ; 0
9071                      ;mov     [es:di+0Ah],ax      ; number of bytes per sector, high word = 0
9072                      ;mov     [es:di+6],ax      ; number of sectors per cluster, high word = 0
9073 0000115A 89450A      mov     [di+0Ah],ax
9074 0000115D 894506      mov     [di+6],ax
9075                      ;
9076                      ;mov     [es:di+24h],ax      ; reserved, 8 bytes zero
9077                      ;mov     [es:di+26h],ax
9078                      ;mov     [es:di+28h],ax
9079                      ;mov     [es:di+2Ah],ax
9080 00001160 83C724      add     di,24h ; 36 ; 10/01/2024
9081 00001163 AB          stosw
9082 00001164 AB          stosw
9083 00001165 AB          stosw
9084 00001166 AB          stosw
9085 00001167 B82C00      mov     ax,44
9086 0000116A 29C7      sub     di,ax ; 10/01/2024
9087                      ;mov     [es:di],ax      ; size of returned structure = 44
9088                      ; 10/01/2024
9089                      ;mov     [di],ax
9090 0000116C AB          stosw
9091 0000116D E9FEF4      jmp     SYS_RET_OK
9092
9093                      ; -----
9094
9095                      ; 12/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM/MSDOS.SYS)
9096                      ; PCDOS 7.1 IBMDOS.COM - DOSCODE:504Ah
9097
9098                      Set_DPBforFormat: ; INT 21h, AX = 7304h (continues from _$GET_DPB)
9099                      ; 12/01/2024      ; (Ref: Ralf Brown's Interrupt List)
9100 00001170 B81800      mov     ax,18h
9101 00001173 394404      cmp     [si+4],ax
9102                      ;cmp     word ptr [si+4],24      ; [SI+user_env.user_CX]
9103                      ; size of buffer (must be at least 18h)
9104                      ;jnb     short setdpbf_2
9105                      ;mov     al,18h      ; error_bad_length
9106                      ;setdpbf_1:
9107                      ;jmp     SYS_RET_ERR
9108 00001176 7223      jb      short setdpbf_1 ; al = 18h
9109                      setdpbf_2:
9110                      push     es      ; ES:BP = Drive parameter block
9111                      push     bp
9112 0000117A 8E4410      mov     es,[si+10h]      ; [SI+user_env.user_ES]
9113 0000117D 8B7C0A      mov     di,[si+0Ah]      ; [SI+user_env.user_DI]
9114 00001180 5E          pop     si
9115 00001181 1F          pop     ds
9116
9117 00001182 268B4504      mov     ax,[es:di+4]      ; (call) function number
9118 00001186 26837D0200      cmp     word [es:di+2],0      ; structure version (must be 0)
9119 0000118B 750C      jnz     short setdpbf_3
9120 0000118D 26837D0600      cmp     word [es:di+6],0      ; (must be 0)
9121 00001192 7505      jnz     short setdpbf_3
9122 00001194 83F804      cmp     ax,4      ; (max) 5 functions (0 to 4)
9123 00001197 7605      jbe     short setdpbf_4
9124                      setdpbf_3:
9125 00001199 B057      mov     al,57h      ; error_invalid_parameter
9126                      ;jmp     short setdpbf_1
9127                      setdpbf_1:
9128 0000119B E9DAF4      jmp     SYS_RET_ERR
9129
9130                      setdpbf_4:
9131 0000119E 26C7051800      mov     word [es:di],18h      ; (call) size
9132 000011A3 08C0      or     al,al
9133 000011A5 7403      jz      short setdpbf_5      ; invalidate DPB counts
9134 000011A7 E98200      jmp     setdpbf_18
9135
9136                      setdpbf_5:
9137 000011AA 31D2      xor     dx,dx      ; 0
9138 000011AC 8B5C0D      mov     bx,[si+0Dh]      ; DPB.MAX_CLUSTER
9139 000011AF 39540F      cmp     [si+0Fh],dx ; 0
9140                      ;cmp     word [si+0Fh],0      ; DPB.FAT_SIZE
9141 000011B2 7506      jnz     short setdpbf_6      ; not FAT32
9142 000011B4 8B542F      mov     dx,[si+2Fh]
9143 000011B7 8B5C2D      mov     bx,[si+2Dh]      ; DPB.LAST_CLUSTER
9144                      setdpbf_6:
9145 000011BA 268B450A      mov     ax,[es:di+0Ah]
9146 000011BE 268B4D08      mov     cx,[es:di+8]      ; new DPB free count
9147                      ; (00000000h=no change, FFFFFFFFh=unknown)
9148 000011C2 09C0      or     ax,ax
9149 000011C4 7502      jnz     short setdpbf_7
9150 000011C6 E322      jcxz    setdpbf_11
9151                      setdpbf_7:
9152 000011C8 83F8FF      cmp     ax,0FFFFh
9153 000011CB 7505      jne     short setdpbf_8
9154 000011CD 83F9FF      cmp     cx,0FFFFh
9155 000011D0 7408      je      short setdpbf_10      ; (set as UNKNOWN/INITIAL)
9156                      setdpbf_8:
9157 000011D2 39D0      cmp     ax,dx      ; must be < DPB.LAST_CLUSTER
9158 000011D4 7502      jne     short setdpbf_9
9159 000011D6 39D9      cmp     cx,bx
9160                      setdpbf_9:
9161 000011D8 73BF      jnb     short setdpbf_3

```

```

9162
9163 000011DA 804C1801
9164 000011DE 894C1F
9165 000011E1 837C0F00
9166 000011E5 7503
9167 000011E7 894421
9168
9169 000011EA 268B450E
9170 000011EE 268B4D0C
9171
9172 000011F2 09C0
9173 000011F4 7502
9174 000011F6 E32E
9175
9176 000011F8 83F8FF
9177 000011FB 7505
9178 000011FD 83F9FF
9179 00001200 7411
9180
9181 00001202 21C0
9182
9183 00001204 7505
9184 00001206 83F902
9185
9186 00001209 728E
9187
9188 0000120B 39D0
9189 0000120D 7502
9190 0000120F 39D9
9191
9192 00001211 7786
9193
9194 00001213 804C1801
9195 00001217 894C1D
9196 0000121A 837C0F00
9197 0000121E 7506
9198 00001220 89443B
9199 00001223 894C39
9200
9201 00001226 B80473
9202 00001229 E942F4
9203
9204
9205
9206 0000122C 48
9207 0000122D 7514
9208 0000122F 1E
9209 00001230 56
9210 00001231 26C57508
9211 00001235 5D
9212 00001236 07
9213 00001237 B95845
9214 0000123A BA5241
9215 0000123D E82502
9216 00001240 E92BF4
9217
9218
9219
9220 00001243 48
9221 00001244 7507
9222
9223
9224 00001246 804C1880
9225
9226 0000124A E921F4
9227
9228
9229 0000124D 837C0F00
9230 00001251 7405
9231 00001253 B00F
9232
9233
9234 00001255 E920F4
9235
9236
9237
9238 00001258 48
9239 00001259 7454
9240 0000125B 8B4435
9241
9242 0000125E 2689450C
9243 00001262 8B4437
9244 00001265 2689450E
9245 00001269 268B4D0A
9246 0000126D 268B4508
9247 00001271 83F8FF
9248 00001274 7505
9249 00001276 83F9FF
9250 00001279 74CF
9251
9252 0000127B 21C9
9253
9254
9255 0000127D 7509
9256 0000127F 83F802
9257
9258 00001282 7304
9259
9260
9261
9262 00001284 B057
9263 00001286 EB0D
9264
9265
9266 00001288 3B4C2F
9267 0000128B 7503
9268 0000128D 3B442D
9269
9270 00001290 77F2
9271 00001292 894C37
9272 00001295 894435
9273 00001298 804C1802
9274 0000129C 1E
9275 0000129D 56
9276 0000129E 5D
9277 0000129F 07
9278 000012A0 E83F06
9279 000012A3 E8C621
9280 000012A6 E86606
9281 000012A9 739F
9282 000012AB B01F
9283
9284 000012AD EBA6
9285

setdpbf_10:
    or     byte [si+18h],1          ; DPB.FIRST_ACCESS (bit 0 = 1)
    mov     [si+1Fh],cx             ; DPB.FREE_CNT
    cmp     word [si+0Fh],0         ; DP.FAT_SIZE
    jnz     short setdpbf_11        ; FAT12 or FAT16
    mov     [si+21h],ax             ; DPB.FREE_CNT_HW
setdpbf_11:
    mov     ax,[es:di+0Eh]
    mov     cx,[es:di+0Ch]          ; new DPB next-free
                                        ; (00000000h=no change, FFFFFFFFh=unknown)
    or     ax,ax
    jnz     short setdpbf_12
    jcxz    setdpbf_17
setdpbf_12:
    cmp     ax,0FFFFh
    jne     short setdpbf_13
    cmp     cx,0FFFFh
    je      short setdpbf_16        ; (set as UNKNOWN/INITIAL)
setdpbf_13:
    and     ax,ax
    ;cmp    ax,0                    ; must be >= 2
    jnz     short setdpbf_14
    cmp     cx,2
;setdpbf_14:
    jb      short setdpbf_3
setdpbf_14:
    cmp     ax,dx
    jne     short setdpbf_15        ; must be < DPB.LAST_CLUSTER
    cmp     cx,bx
setdpbf_15:
    ja      short setdpbf_3
setdpbf_16:
    or     byte [si+18h],1          ; DPB.FIRST_ACCESS (bit 0 = 1)
    mov     [si+1Dh],cx             ; DPB.NEXT_FREE
    cmp     word [si+0Fh],0         ; DPB.FAT_SIZE
    jnz     short setdpbf_17        ; FAT16 or FAT12
    mov     [si+3Bh],ax
    mov     [si+39h],cx             ; DPB.FAT32_NXTFREE
setdpbf_17:
    mov     ax,7304h                ; done (successful)
    jmp     SYS_RET_OK
setdpbf_18:
    ;dec    al
    dec     ax
    jnz     short setdpbf_19
    push    ds                       ; rebuild DPB from BPB
    push    si
    lds     si,[es:di+8]             ; BIOS Parameter Block
    pop     bp
    pop     es
    mov     cx,4558h                ; 'XE' (NASM syntax)
    mov     dx,4152h                ; 'RA' (NASM syntax)
    call    _$_SETDPB
    jmp     SYS_RET_OK
setdpbf_19:
    ;dec    al
    dec     ax
    jnz     short setdpbf_20
    ; force media change
    ; (next access to drive rebuild DPB)
    or     byte [si+18h],80h        ; DPB.FIRST_ACCESS (bit 7 = 1)
; 12/01/2024
setdpbf_29:
    jmp     SYS_RET_OK
setdpbf_20:
    cmp     word [si+0Fh],0         ; DPB.FAT_SIZE
    jz      short setdpbf_22        ; FAT32
    mov     al,0Fh                  ; error_invalid_drive
    ; (function 3 or 4 are only for drives with FAT32 fs)
setdpbf_21:
    jmp     SYS_RET_ERR
setdpbf_22:
    ;dec    al
    dec     ax
    jz      short setdpbf_30        ; get/set active FAT number and mirroring
    mov     ax,[si+35h]             ; DPB.ROOT_CLUSTER
    ; get/set root directory cluster number
    mov     [es:di+0Ch],ax          ; (ret) previous root directory cluster number
    mov     ax,[si+37h]
    mov     [es:di+0Eh],ax
    mov     cx,[es:di+0Ah]
    mov     ax,[es:di+8]            ; (call) new root directory cluster number
    cmp     ax,0FFFFh              ; -1 --> return only previous root dir cluster number
    jne     short setdpbf_23
    cmp     cx,0FFFFh
    je      short setdpbf_29
setdpbf_23:
    and     cx,cx
    ;cmp    cx,0                    ; cluster number must be >= 2
    ;jnz    short setdpbf_24
    jnz     short setdpbf_26
    cmp     ax,2
setdpbf_24:
    jnb     short setdpbf_26
setdpbf_25:
    ;jmp     setdpbf_3              ; error (invalid parameter)
    ; 12/01/2024
    mov     al,57h                  ; error_invalid_parameter
    jmp     short setdpbf_21
setdpbf_26:
    cmp     cx,[si+2Fh]              ; must be <= DPB.LAST_CLUSTER
    jne     short setdpbf_27
    cmp     ax,[si+2Dh]
setdpbf_27:
    ja      short setdpbf_25        ; error
    mov     [si+37h],cx             ; DPB.ROOT_CLUSTER
    mov     [si+35h],ax
    or     byte [si+18h],2          ; DPB.FIRST_ACCESS (bit 1 = 1)
    push    ds
    push    si
    pop     bp
    pop     es
    call    ECritDisk
    call    update_fat32_fsinfo
    call    LCritDisk
    jnc     short setdpbf_29
    mov     al,1Fh                  ; error_gen_failure
setdpbf_28:
    jmp     short setdpbf_21

```

```

9286         ; 12/01/2024
9287 ;setdpbf_29:
9288         ; jmp     SYS_RET_OK
9289
9290 setdpbf_30:
9291         ; 12/01/2024
9292         ; ax = 0
9293 000012AF 8164238F00 and     word [si+23h],8Fh    ; DPB.EXT_FLAGS (clear bit 4-6)
9294 000012B4 8B4C23     mov     cx,[si+23h]
9295 000012B7 26894D0C     mov     [es:di+0Ch],cx    ; (ret) previous active FAT/mirroring state
9296 000012BB 2689450E     mov     [es:di+0Eh],ax ; 0
9297         ; mov     word [es:di+0Eh],0    ; put zero to Set_DPBforFormat struct offset 14
9298 000012BF 268B5508     mov     dx,[es:di+8]    ; (call) new active FAT/mirroring state,
9299         ; or FFFFFFFFh to get
9300 000012C3 83FAFF     cmp     dx,0FFFFFFh
9301         ; jnz     short setdpbf_31
9302         ; jmp     SYS_RET_OK
9303         ; 12/01/2024
9304 000012C6 7482     je      short setdpbf_29
9305
9306 setdpbf_31:
9307 000012C8 F7C270FF     test    dx,0FF70h
9308         ; jz      short setdpbf_33    ; bit 4-6 of DPB.EXT_FLAGS must be 0
9309         ; mov     al,57h              ; error_invalid_parameter
9310         ; 12/01/2024
9311 000012CC 75B6     jnz     short setdpbf_25
9312 setdpbf_32:
9313         ; jmp     short setdpbf_28
9314         ; 12/01/2024
9315         ; jmp     short setdpbf_21
9316
9317 setdpbf_33:
9318 000012CE B001     mov     al,1    ; error_invalid_function
9319         ;                               ; (modification is not allowed)
9320         ; jmp     short setdpbf_32
9321         ; 12/01/2024
9322 000012D0 EB83     jmp     short setdpbf_21
9323
9324 ; -----
9325 ; =====
9326 ; PARSE.ASM, MSDOS 6.0, 1991
9327 ; =====
9328 ; 19/07/2018 - Retro DOS v3.0
9329 ; 15/05/2019 - Retro DOS v4.0
9330
9331 ; System calls for parsing command lines
9332 ;
9333 ; $PARSE_FILE_DESCRIPTOR
9334 ;
9335 ; Modification history:
9336 ;
9337 ; Created: ARR 30 March 1983
9338 ; EE PathParse 10 Sept 1983
9339 ;
9340 ;
9341 ;BREAK <$Parse_File_Descriptor -- Parse an arbitrary string into an FCB>
9342 ; -----
9343 ; Inputs:
9344 ; DS:SI Points to a command line
9345 ; ES:DI Points to an empty FCB
9346 ; Bit 0 of AL = 1 At most one leading separator scanned off
9347 ;               = 0 Parse stops if separator encountered
9348 ; Bit 1 of AL = 1 If drive field blank in command line - leave FCB
9349 ;               = 0 " " - put 0 in FCB
9350 ; Bit 2 of AL = 1 If filename field blank - leave FCB
9351 ;               = 0 " " - put blanks in FCB
9352 ; Bit 3 of AL = 1 If extension field blank - leave FCB
9353 ;               = 0 " " - put blanks in FCB
9354 ; Function:
9355 ; Parse command line into FCB
9356 ; Returns:
9357 ; AL = 1 if '*' or '?' in filename or extension, 0 otherwise
9358 ; DS:SI points to first character after filename
9359 ; -----
9360
9361 ; 13/01/2024
9362 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:51BFh
9363 ; MSDOS 6.22 MSDOS.SYS - DOSCODE:4D22h
9364
9365 _$PARSE_FILE_DESCRIPTOR:
9366     call    MAKEFCB
9367 000012D2 E88F49     push    SI
9368 000012D5 56         push    SI
9369 000012D6 E89EF1     call    Get_User_Stack
9370         ; pop     word [si+8]
9371 000012D9 8F4408     pop     word [SI+user_env.user_SI]
9372 000012DC C3         retn
9373
9374 ; -----
9375 ; 13/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM - DOSCODE:51CAh)
9376 ; -----
9377
9378 set_exerr_locus_unk:
9379     push    ax
9380 000012DE B001     mov     al,1    ; errLOC_Unk
9381
9382 set_exerr_locus:
9383     mov     [ss:EXERR_LOCUS],al
9384     pop     ax
9385     retn
9386
9387 set_exerr_locus_disk:
9388     push    ax
9389 000012E6 50         mov     al,2    ; errLOC_Disk
9390     jmp     short set_exerr_locus
9391
9392 set_exerr_locus_ser:
9393     push    ax
9394 000012EC B004     mov     al,4    ; errLOC_SerDev
9395     jmp     short set_exerr_locus
9396
9397 set_exerr_locus_mem:
9398     push    ax
9399 000012F0 50         mov     al,5    ; errLOC_Mem
9400     jmp     short set_exerr_locus
9401
9402 ; -----
9403 ; =====
9404 ; MISC.ASM, MSDOS 6.0, 1991
9405 ; =====
9406 ; 19/07/2018 - Retro DOS v3.0
9407 ; 29/04/2019 - Retro DOS v4.0

```

```

9410 ;ENTRYPOINTSEG EQU 0CH
9411 ;MAXDIF EQU 0FFFh
9412 ;SAVEXIT EQU 10
9413 ;WRAPOFFSET EQU 0FEF0h
9414
9415 ;
9416 ;-----
9417 ;
9418 ;** $SLEAZEFUNC - Get a Pointer to the Media Byte
9419 ;
9420 ; Return Stuff sort of like old get fat call
9421 ;
9422 ; ENTRY none
9423 ; EXIT DS:BX = Points to FAT ID byte (IBM only)
9424 ; GOD help anyone who tries to do ANYTHING except
9425 ; READ this ONE byte.
9426 ; DX = Total Number of allocation units on disk
9427 ; CX = Sector size
9428 ; AL = Sectors per allocation unit
9429 ; = -1 if bad drive specified
9430 ;
9431 ; USES all
9432 ;
9433 ;** $SLEAZEFUNC DL - Get a Pointer to the Media Byte
9434 ;
9435 ; Identical to $SLEAZEFUNC except (dl) = drive
9436 ;
9437 ; ENTRY (dl) = drive (0=default, 1=A, 2=B, etc.)
9438 ; EXIT DS:BX = Points to FAT ID byte (IBM only)
9439 ; GOD help anyone who tries to do ANYTHING except
9440 ; READ this ONE byte.
9441 ; DX = Total Number of allocation units on disk
9442 ; CX = Sector size
9443 ; AL = Sectors per allocation unit
9444 ; = -1 if bad drive specified
9445 ;
9446 ; USES all
9447 ;
9448 ;-----
9449 ; 10/01/2024 - Retro DOS v5.0
9450 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:51E2h
9451
9452 _$SLEAZEFUNC:
9453 ; 15/05/2019 - Retro DOS v4.0
9454 MOV DL,0
9455 _$SLEAZEFUNC DL:
9456 push ss
9457 pop ds
9458 MOV AL,DL
9459 call GETTHISDRV ; Get CDS structure
9460 SET_AL_RET:
9461 ; MSDOS 3.3
9462 ; ;mov al, 0Fh
9463 ; MOV AL,error_invalid_drive; Assume error ;AC000;
9464
9465 ; MSDOS 6.0 & MSDOS 3.3
9466 JC short BADSLDRIVE
9467
9468 call DISK_INFO
9469 ; JC short SET_AL_RET ; User FAILED to I 24
9470 JC short BADSLDRIVE
9471 MOV [FATBYTE],AH ; FAT (MEDIA) ID byte
9472
9473 ; 10/01/2024 - Retro DOS v5.0 (PC DOS 7.1)
9474 ; ;
9475 xor ah, ah ; AH = 0
9476 ; AL = sectors per cluster
9477 ; di:bx = number of clusters
9478 push di
9479 call TestNet
9480 pop di
9481 JC short sleazefunc1
9482 push cx ; bytes per sector
9483 call modify_cluster_count
9484 pop cx ; ax = sectors per cluster (modified)
9485 ; dx = number of clusters (modified)
9486 ; if bytes per cluster > 16384
9487 ; and hw of cluster count > 0
9488 ; bx = 0FFFEh (invalidated parm sign)
9489
9490 sleazefunc1:
9491 ; ;
9492 ; NOTE THAT A FIXED MEMORY CELL IS USED --> THIS CALL IS NOT
9493 ; RE-ENTRANT. USERS BETTER GET THE ID BYTE BEFORE THEY MAKE THE
9494 ; CALL AGAIN
9495
9496 ; MOV DI,FATBYTE
9497 ; 10/01/2024
9498 ; XOR AH,AH ; AL has sectors/cluster
9499 call Get_User_Stack
9500 ; mov [si+4],cx
9501 ; mov [si+6],bx
9502 ; mov [si+2],di
9503 MOV [SI+user_env.user_CX],CX
9504 MOV [SI+user_env.user_DX],BX
9505 MOV [SI+user_env.user_BX],DI
9506 ; 10/01/2024
9507 MOV word [SI+user_env.user_BX],FATBYTE
9508
9509 ; mov [si+0Eh],ss
9510 MOV [SI+user_env.user_DS],SS ; stash correct pointer
9511
9512 retn
9513
9514 BADSLDRIVE:
9515 jmp FCB_RET_ERR
9516
9517 ;-----
9518 ;
9519 ;** $Get_INDOS_Flag - Return location of DOS Critical Section Flag
9520 ;
9521 ; Returns location of DOS status for interrupt routines
9522 ;
9523 ; ENTRY none
9524 ; EXIT (es:bx) = flag location
9525 ; USES all
9526 ;
9527 ;-----
9528 ;
9529 _$GET_INDOS_FLAG:
9530 CALL Get_User_Stack
9531 ; MOV WORD [SI+2],INDOS
9532 MOV word [SI+user_env.user_BX],INDOS
9533

```

```

9534 getin_segm: ; 13/01/2024
9535 ;MOV [SI+10H],SS
9536 00001334 8C5410 MOV [SI+user_env.user_ES],SS
9537 00001337 C3 RETN
9538 ;
9539 ;
9540 ;-----
9541 ;
9542 ;** $Get_IN_Vars - Return Pointer to DOS Variables
9543 ;
9544 ; Return a pointer to interesting DOS variables This call is version
9545 ; dependent and is subject to change without notice in future versions.
9546 ; Use at risk.
9547 ;
9548 ; ENTRY none
9549 ; EXIT (es:bx) = address of SYSINITVAR
9550 ; uses ALL
9551 ;
9552 ;-----
9553 ;
9554 ;
9555 ; 13/01/2024 - Retro DOS v5.0
9556 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:5226h
9557 ; MSDOS 6.22 MSDOS.SYS - DOSCODE:4D65h
9558 ; MSDOS 5.0 MSDOS.SYS - DOSCODE:4D58h
9559 ;
9560 _$GET_IN_VARS:
9561 00001338 E83CF1 CALL Get_User_Stack
9562 ;MOV WORD [SI+2],SYSINITVAR
9563 ;MOV word [SI+user_env.user_BX],SYSINITVAR
9564 0000133B C74402[2600] MOV word [SI+user_env.user_BX],SYSINITVAR
9565 ; 13/01/2024
9566 ;MOV [SI+10H],SS
9567 ;MOV [SI+user_env.user_ES],SS
9568 ;RETN
9569 00001340 EBF2 jmp short getin_segm
9570 ;
9571 ;-----
9572 ;
9573 ;** $Get_Default_DPB - Return a pointer to the Default DPB
9574 ;
9575 ; Return pointer to drive parameter table for default drive
9576 ;
9577 ; ENTRY none
9578 ; EXIT (ds:bx) = DPB address
9579 ; USES all
9580 ;
9581 ;** $Get_DPB - Return a pointer to a specified DPB
9582 ;
9583 ; Return pointer to a specified drive parameter table
9584 ;
9585 ; ENTRY (dl) = drive # (0 = default, 1=A, 2=B, etc.)
9586 ; EXIT (al) = 0 iff ok
9587 ; (ds:bx) = DPB address
9588 ; (al) = -1 if bad drive
9589 ; USES all
9590 ;
9591 ;-----
9592 ;
9593 ; 10/01/2024
9594 %if 0
9595 ;
9596 ; 15/05/2019 - Retro DOS v4.0
9597 ;
9598 _$GET_DEFAULT_DPB:
9599 MOV DL,0
9600 _$GET_DPB:
9601 push ss
9602 pop ds
9603
9604 MOV AL,DL
9605 call GETTHISDRV ; Get CDS structure
9606 JC short ISNODRV ; no valid drive
9607 LES DI,[THISCDS] ; check for net CDS
9608 ;;test word [es:di+43h],8000h
9609 ;TEST word [ES:DI+curdir.flags],curdir_isnet
9610 ;test byte [es:di+44h],80h
9611 test byte [ES:DI+curdir.flags+1],(curdir_isnet>>8)
9612 JNZ short ISNODRV ; No DPB to point at on NET stuff
9613 call ECritDisk
9614 call FATREAD_CDS ; Force Media Check and return DPB
9615 call LCritDisk
9616 JC short ISNODRV ; User FAILED to I 24, only error we
9617 ; have.
9618 call Get_User_Stack
9619 ;mov [si+2],bp
9620 MOV [SI+user_env.user_BX],BP
9621 ;mov [si+0Eh],es
9622 MOV [SI+user_env.user_DS],ES
9623 XOR AL,AL
9624 retn
9625 ISNODRV:
9626 MOV AL,-1
9627 retn
9628
9629 %else
9630 ;
9631 ; 13/01/2024
9632 ; 10/01/2024 - Retro DOS v5.0
9633 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:5230h
9634 ;
9635 _$GET_DEFAULT_DPB:
9636 MOV DL,0
9637 00001342 B200 _$GET_DPB:
9638 push ss
9639 00001344 16 pop ds
9640 00001345 1F
9641 MOV AL,DL
9642 call GETTHISDRV ; Get CDS structure
9643 00001348 E84268 mov al,0Fh ; error_invalid_drive
9644 0000134B B00F jnc short getdpb_3
9645 0000134D 7315
9646 ; no valid drive
9647 getdpb_1:
9648 ; 13/01/2024
9649 call Get_User_Stack
9650 0000134F E825F1 cmp word [si],7302h ; INT 21h, AX = 7302h ? Get_ExtDPB
9651 00001352 813C0273 ; GET EXTENDED DPB
9652 je short getdpb_2
9653 00001356 7406 cmp word [si],7304h ; INT 21h, AX = 7304h ? Set_DPBforFormat
9654 00001358 813C0473 ; Set DPB TO USE FOR FORMATTING
9655 jne short ISNODRV
9656 0000135C 7503 getdpb_2:
9657

```



```

9658 0000135E E917F3      jmp     SYS_RET_ERR
9659
9660
9661 00001361 B0FF      ISNODRV: mov     al,0FFh ; -1          ; invalid (or network) drive
9662 00001363 C3          retn
9663
9664
9665 00001364 C43E[A205]   getdpb_3: les     di,[THISCDS] ; check for net CDS
9666                ; ;test word [es:di+43h],8000h
9667                ; TEST word [ES:DI+curdir.flags],curdir_isnet
9668                ; ;test byte [es:di+44h],80h
9669 00001368 26F6454480 test    byte [ES:DI+curdir.flags+1],(curdir_isnet>>8)
9670                ; JNZ short ISNODRV ; No DPB to point at on NET stuff
9671                ; 10/01/2024
9672 0000136D 75E0      jnz     short getdpb_1
9673 0000136F E87005     call    ECritDisk
9674 00001372 E8E651     call    FATREAD_CDS ; Force Media Check and return DPB
9675 00001375 E89705     call    LCritDisk
9676                ; JC short ISNODRV ; User FAILED to I 24,
9677 00001378 B053      mov     al,53h ; error_FAIL_I24 ; only error we have.
9678 0000137A 72D3      jc      short getdpb_1
9679 0000137C E8F8F0     call    Get_User_Stack
9680                ; 13/01/2024
9681 0000137F 813C0473     cmp     word [si],7304h ; INT 21h, AX = 7304h ?
9682 00001383 7503      jnz     short getdpb_4
9683 00001385 E9E8FD     jmp     Set_DPBforFormat
9684
9685                ; 13/01/2024
9686
9687 00001388 813C0273     getdpb_4: cmp     word [si],7302h ; INT 21h, AX = 7302h ?
9688 0000138C 7410      je      short getdpb_5
9689 0000138E 26837E0F00     cmp     word [es:bp+0Fh],0
9690 00001393 74BA      jz      short getdpb_1
9691 00001395 896C02     mov     [si+2],bp
9692 00001398 8C440E     mov     [si+0Eh],es
9693 0000139B 30C0      xor     al,al ; status = 0 = successful
9694 0000139D C3          retn
9695
9696
9697 0000139E B018      getdpb_5: mov     al,18h ; error_bad_length
9698 000013A0 837C043F     cmp     word [si+4],63 ; [SI+user_env.user_CX]
9699                ; length of buffer (must be 63 bytes)
9700                ; error
9701                ;
9702 000013A6 06          push    es ; ES:BP = Drive parameter block
9703 000013A7 55          push    bp
9704 000013A8 8E4410     mov     es,[si+16] ; [SI+user_env.user_ES]
9705 000013AB 8B7C0A     mov     di,[si+10] ; [SI+user_env.user_DI]
9706 000013AE 8B5C08     mov     bx,[si+8] ; [SI+user_env.user_SI] (!)
9707 000013B1 5E          pop     si
9708 000013B2 1F          pop     ds
9709
9710 000013B3 B83D00     mov     ax,61 ; length of following data (003Dh)
9711 000013B6 AB          stosw
9712 000013B7 89C1      mov     cx,ax
9713 000013B9 57          push    di
9714 000013BA F3A4      rep movsb
9715 000013BC 5F          pop     di
9716
9717                ; 13/01/2024
9718 000013BD 06          push    es
9719 000013BE 1F          pop     ds
9720 000013BF 31C0      xor     ax,ax ; 0
9721
9722 000013C1 81FBA6F1     cmp     bx,0F1A6h ; (!) signature (undocumented),
9723                ; must be 0F1A6h to get device driver
9724                ; address and next-DPB pointer
9725                ; (Ref: Ralf Brown's Interrupt List)
9726 000013C5 740E      je      short getdpb_6
9727
9728                ; xor ax,ax ; 0
9729 000013C7 48          dec     ax ; -1
9730                ; mov [es:di+19h],ax ; pointer to next DPB (invalidated)
9731                ; mov [es:di+1Bh],ax
9732                ; mov [es:di+13h],ax ; pointer to driver address (invalidated)
9733                ; mov [es:di+15h],ax
9734                ; 13/01/2024
9735 000013C8 894519     mov     [di+19h],ax ; -1 ; pointer to next DPB (invalidated)
9736 000013CB 89451B     mov     [di+1Bh],ax
9737 000013CE 894513     mov     [di+13h],ax ; pointer to driver address (invalidated)
9738 000013D1 894515     mov     [di+15h],ax
9739
9740                ; 13/01/2024
9741 000013D4 40          inc     ax ; 0
9742
9743 getdpb_6:
9744                ; 13/01/2024
9745                ; ax = 0
9746 000013D5 39450F     cmp     [di+0Fh],ax ; 0
9747                ; cmp [es:di+0Fh],ax ; 0 ; FAT (16 bit) size ?
9748 000013D8 7441      ; ;cmp word [es:di+0Fh],0
9749                ; jz short getdpb_8 ; FAT32
9750                ; ; FAT16 or FAT12
9751                ; mov ax,[es:di+0Dh] ; DPB.MAX_CLUSTER
9752                ; mov [es:di+2Dh],ax ; DPB.LAST_CLUSTER
9753                ; mov ax,[es:di+0Fh] ; DPB.FAT_SIZE
9754                ; mov [es:di+31h],ax ; DPB.FAT32_SIZE
9755                ; mov ax,[es:di+1Dh] ; DPB.NEXT_FREE
9756                ; mov [es:di+39h],ax ; DPB.FAT32_NXTFREE
9757                ; mov ax,[es:di+0Bh] ; DPB.FIRST_SECTOR
9758                ; mov [es:di+29h],ax ; DPB.FCLUS_FSECTOR
9759                ; 13/01/2024
9760 000013DA 8B450D     mov     ax,[di+0Dh] ; DPB.MAX_CLUSTER
9761 000013DD 89452D     mov     [di+2Dh],ax ; DPB.LAST_CLUSTER
9762 000013E0 8B450F     mov     ax,[di+0Fh] ; DPB.FAT_SIZE
9763 000013E3 894531     mov     [di+31h],ax ; DPB.FAT32_SIZE
9764 000013E6 8B451D     mov     ax,[di+1Dh] ; DPB.NEXT_FREE
9765 000013E9 894539     mov     [di+39h],ax ; DPB.FAT32_NXTFREE
9766 000013EC 8B450B     mov     ax,[di+0Bh] ; DPB.FIRST_SECTOR
9767 000013EF 894529     mov     [di+29h],ax ; DPB.FCLUS_FSECTOR
9768
9769 000013F2 31C0      xor     ax,ax ; 0
9770                ; mov [es:di+2Fh],ax ; DPB.LAST_CLUSTER high word
9771                ; mov [es:di+33h],ax ; DPB.FAT32_SIZE high word
9772                ; mov [es:di+3Bh],ax ; DPB.FAT32_NXTFREE high word
9773                ; mov [es:di+2Bh],ax ; DPB.FCLUS_FSECTOR high word
9774                ; mov [es:di+35h],ax ; DPB.ROOT_CLUSTER
9775                ; mov [es:di+37h],ax ; DPB.ROOT_CLUSTER high word
9776                ; mov [es:di+23h],ax ; DPB.EXT_FLAGS
9777                ; 13/01/2024
9778 000013F4 89452F     mov     [di+2Fh],ax ; 0 ; DPB.LAST_CLUSTER high word
9779 000013F7 894533     mov     [di+33h],ax ; DPB.FAT32_SIZE high word
9780 000013FA 89453B     mov     [di+3Bh],ax ; DPB.FAT32_NXTFREE high word
9781 000013FD 89452B     mov     [di+2Bh],ax ; DPB.FCLUS_FSECTOR high word
9782 00001400 894535     mov     [di+35h],ax ; DPB.ROOT_CLUSTER

```

```

9782 00001403 894537      mov     [di+37h],ax      ; DPB.ROOT_CLUSTER high word
9783 00001406 894523      mov     [di+23h],ax      ; DPB.EXT_FLAGS
9784 00001409 48          dec     ax                ; -1
9785                      ;mov     [es:di+25h],ax      ; DPB.FSINFO_SECTOR (invalidated)
9786                      ;mov     [es:di+27h],ax      ; DPB.BKBOOT_SECTOR (invalidated)
9787                      ; 13/01/2024
9788 0000140A 894525      mov     [di+25h],ax      ; DPB.FSINFO_SECTOR (invalidated)
9789 0000140D 894527      mov     [di+27h],ax      ; DPB.BKBOOT_SECTOR (invalidated)
9790 00001410 3B451F      cmp     ax,[di+1Fh]
9791                      ;cmp     ax,[es:di+1Fh]      ; DPB.FREE_COUNT (= -1 ?)
9792 00001413 7401      je      short getdpb_7
9793 00001415 40          inc     ax                ; -1 -> 0
9794
9795 00001416 894521      mov     [di+21h],ax
9796                      ;mov     [es:di+21h],ax      ; DPB.PB.FREE_COUNT high word
9797
9798 ;getdpb_8:
9799                      ; 26/06/2024
9799 00001419 31C0      xor     ax,ax            ; status = 0 = successful
9800
9801 0000141B E950F2      getdpb_8:  ; 03/07/2024
9802                      jmp     SYS_RET_OK
9803
9804 %endif
9805
9806 ;-----
9807 ;
9808 ;** $Disk_Reset - Flush out Dirty Buffers
9809 ;
9810 ; $DiskReset flushes and invalidates all buffers. BUGBUG - do
9811 ; we really invalidate? Should we? This screws non-removable
9812 ; caching. Maybe CHKDSK relies upon it, though....
9813 ;
9814 ; ENTRY none
9815 ; EXIT none
9816 ; USES all
9817 ;-----
9818 ;
9819 ;
9820 ; 06/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
9821 ; DOSCODE:4D94h
9822 ; 13/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
9823 ; DOSCODE:5322h
9824
9825 _$DISK_RESET:
9826 0000141E B0FF      mov     al,0FFh ; -1
9827 00001420 16          push    ss
9828 00001421 1F          pop     ds
9829                      ; 06/11/2022
9830                      ;MOV     AL,-1
9831 00001422 E8BD04      call    ECritDisk
9832                      ; MSDOS 6.0
9833                      ;;or     word [DOS34_FLAG],4
9834                      ;or     word [DOS34_FLAG],FROM_DISK_RESET ;AN000;
9835 00001425 800E[1106]04 or     byte [DOS34_FLAG],FROM_DISK_RESET ; 4 ; 15/05/2019
9836 0000142A E86B56      call    FLUSHBUF
9837                      ; MSDOS 6.0
9838                      ;and     word [DOS34_FLAG],0FFFBh
9839                      ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
9840                      ;and     word [DOS34_FLAG],NO_FROM_DISK_RESET ;AN000;
9841                      ; 15/12/2022
9842 0000142D 8026[1106]FB and     byte [DOS34_FLAG],NO_FROM_DISK_RESET ; 0FBh ; 15/05/2019
9843
9844                      ; 13/01/2024 - Retro DOS v5.0
9845                      ; (PCDOS 7.1 IBMDOS.COM)
9846                      ;;;
9847 00001432 C42E[2600]    les     bp,[DPBHEAD]
9848
9849 00001436 83FDFF      drst_1:  cmp     bp,0FFFFh      ; -1 ?
9850 00001439 7409      je      short drst_2    ; yes, it is the last DPB
9851 0000143B E82E20      call    update_fat32_fsinfo ; update FSINFO (sector) parameters
9852 0000143E 26C46E19    les     bp,[es:bp+19h] ; DPB.NEXT_DPB
9853 00001442 EBF2      jmp     short drst_1
9854
9855 drst_2:
9856      ;;;
9857 00001444 C706[8310]0000    mov     word [SC_STATUS],0 ; Throw out secondary cache ; M041
9858
9859 ;
9860 ; we will "ignore" any errors on the flush, and go ahead and invalidate. This
9861 ; call doesn't return any errors and it is supposed to FORCE a known state, so
9862 ; let's do it.
9863 ;
9864 ; Invalidate 'last-buffer' used
9865 ;
9866 0000144A BBFFFF      MOV     BX,-1 ; 0FFFFh
9867 0000144D 891E[2000]    MOV     [LastBuffer+2],BX
9868 00001451 891E[1E00]    MOV     [LastBuffer],BX
9869
9870 ; MSDOS 3.3
9871 ; IBMDOS.COM, Offset 1C66h
9872 ;;;
9873 ;lds     si,[BUFFHEAD]
9874 ;mov     ax,20FFh      ; .buf_ID, AL = FFh (Free buffer)
9875                      ; .buf_flags, AH = 0, reset/clear
9876
9877 ;DRST_1:
9878 ;;mov     [si+4],ax
9879 ;mov     [si+BUFFINFO.buf_ID],ax
9880 ;lds     si,[SI]
9881 ;cmp     si,bx ; -1
9882 ;je      short DRST_2
9883 ;;mov     [si+4],ax
9884 ;mov     [si+BUFFINFO.buf_ID],ax
9885 ;lds     si,[SI]
9886 ;cmp     si,bx
9887 ;jne     short DRST_1
9888 ;;;
9889
9890 ;DRST_2:
9891 call    LCritDisk
9892 MOV     AX,-1
9893 ; 07/12/2022
9894 ;mov     ax,0FFFFh
9895 ;CallInstal NetFlushBuf,MultNET,32,AX,AX
9896 push    ax ; * MSDOS 6.0 ; 15/05/2019
9897 mov     ax,1120h
9898 int     2Fh ; Multiplex - NETWORK REDIRECTOR - FLUSH ALL DISK BUFFERS
9899                      ; DS = DOS CS
9900                      ; Return: CF clear (successful)
9901 pop     ax ; * MSDOS 6.0 ; 15/05/2019
9902
9903 retn
9904
9905 ; 19/07/2018 - Retro DOS v3.0
9906 ;
9907 ; BREAK <$SetDPB - Create a valid DPB from a user-specified BPB>

```



```

9906 ;
9907 ;-----
9908 ;
9909 ;** $SetDPB - Create a DPB
9910 ;
9911 ; SetDPB Creates a valid DPB from a user-specified BPB
9912 ;
9913 ; ENTRY ES:BP Points to DPB
9914 ; DS:SI Points to BPB
9915 ; EXIT DPB setup
9916 ; USES ALL but BP, DS, ES
9917 ;-----
9918 ;
9919 ;
9920 ; 10/05/2019 - Retro DOS v4.0
9921 ;
9922 ; DOSCODE:4DD6h (MSDOS 6.21, MSDOS.SYS)
9923 ;
9924 ; MSDOS 6.0
9925 00001463 0300 word3: dw 3 ; M008 -- word value for divides
9926 ;
9927 ; 06/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0)
9928 ; DOSCODE:4DC9h (MSDOS 5.0, MSDOS.SYS)
9929 ;
9930 ; 13/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1)
9931 ; DOSCODE:5369h (PCDOS 7.1, IBMDOS.COM)
9932 ;
9933 ; 13/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1)
9934 ; Windows ME IO.SYS (Extracted) - BIOSCODE:4EF8h
9935 ;
9936 ;procedure $SETDPB,NEAR
9937 ;
9938 ; 12/04/2024
9939 _$SETDPB:
9940 MOV DI,BP
9941 00001465 89EF ;ADD DI,2 ; skip over dpb_drive and dpb_UNIT
9942 ; 13/01/2024
9943 inc di
9944 00001467 47 inc di
9945 00001468 47 inc di
9946 00001469 AD LODSW
9947 0000146A AB STOSW ; dpb_sector_size
9948 ;
9949 ; 13/01/2024
9950 ; Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
9951 ;;;
9952 0000146B 81F95845 cmp cx,4558h ; 'XE' (NASM syntax)
9953 ; CX = signature 4558h ('EX') for FAT32 extended BPB/DPB
9954 0000146F 7506 jne short not_fat32_extension
9955 00001471 81FA5241 cmp dx,4152h ; 'RA' (NASM syntax)
9956 ; DX = signature 4152h ('AR') for FAT32 extended BPB/DPB
9957 00001475 7402 je short chk_fat32_conditions
9958 not_fat32_extension:
9959 00001477 31C9 xor cx,cx ; 0 ; (Do not use FAT32 extensions -32 bit parameters-)
9960 chk_fat32_conditions:
9961 00001479 51 push cx ; (*)
9962 ;;;
9963 ;
9964 ; 13/01/2024
9965 ; MSDOS 6.0
9966 ;cmp byte [si+3],0
9967 ;CMP BYTE [SI+A_BPB.BPB_NUMBEROFFATS-2],0
9968 0000147A 807C0300 cmp byte [SI+A_BPB.NUMBEROFFATS-2],0 ; FAT file system drive ;AN000;
9969 ;JNZ short yesfat ; yes ;AN000;
9970 0000147E 740C jz short nofat ; 13/01/2024 (PCDOS 7.1)
9971 ;
9972 ; 13/01/2024 - Retro DOS v5.0
9973 ;;;
9974 ;cmp word [si+9],0 ; .BPB_SECTORSPEFAT ; BPB_FATsz16
9975 00001480 837C0900 cmp word [si+A_BPB.SECTORSPEFAT-2],0
9976 00001484 7516 jnz short yesfat
9977 ;cmp word [si+29],0 ; .BPB_FAT32VERSION ; BPB_FSVer
9978 00001486 837C1D00 cmp word [si+A_BPB.FSVER-2],0
9979 0000148A 7410 jz short yesfat
9980 nofat:
9981 ;;;
9982 ;
9983 ; 13/01/2024
9984 ;;mov byte [es:di+4],0
9985 ;MOV BYTE [ES:DI+DPB.FAT_COUNT-4],0
9986 ;JMP short setend ; NO ;AN000;
9987 ;
9988 ; 13/01/2024 (WINME IO.SYS)
9989 ;mov byte [es:di+4],0 ; DPB.FAT_COUNT
9990 ;xor eax,eax
9991 ;jmp setend
9992 ;
9993 ; 13/01/2024 (PCDOS7.1 IBMDOS.COM)
9994 0000148C 31C0 xor ax,ax ; 0
9995 0000148E 26884504 mov [es:di+DPB.FAT_COUNT-4],al ; DPB.FAT_COUNT = 0
9996 ;
9997 ; 13/01/2024 (not necessary)
9998 ;add di,15 ; DPB.DRIVER_ADDR
9999 ;
10000 00001492 83C60B add si,11 ; .BPB_SECTORSPEFAT
10001 ;
10002 ;mov [es:bp+15],ax ; DPB.FAT_SIZE = 0
10003 00001495 2689460F mov [es:bp+DPB.FAT_SIZE],ax
10004 ;
10005 00001499 E9B500 jmp setend
10006 ;
10007 yesfat: ; 10/08/2018
10008 0000149C 89C2 MOV DX,AX
10009 0000149E AC LODSB
10010 ;;;
10011 ; 13/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
10012 0000149F 08C0 or al,al
10013 000014A1 74E9 jz short nofat
10014 ;;;
10015 ;DEC AL
10016 ; 17/12/2022
10017 000014A3 48 dec ax
10018 000014A4 AA STOSB ; dpb_cluster_mask
10019 ;INC AL
10020 000014A5 40 inc ax
10021 000014A6 30E4 XOR AH,AH
10022 LOG2LOOP:
10023 000014A8 A801 test AL,1
10024 000014AA 7506 JNZ short SAVLOG
10025 000014AC FEC4 INC AH
10026 000014AE D0E8 SHR AL,1
10027 000014B0 EBF6 JMP SHORT LOG2LOOP
10028 SAVLOG:
10029 000014B2 88E0 MOV AL,AH

```

```

10030 000014B4 AA      STOSB                ; dpb_cluster_shift
10031 000014B5 88C3    MOV     BL,AL
10032 000014B7 A5      MOVSW                ; dpb_first_FAT Start of FAT (# of reserved sectors)
10033 000014B8 AC      LODSB
10034 000014B9 AA      STOSB                ; dpb_FAT_count Number of FATs
10035                ; OR     AL,AL                ; NONFAT ?                ;AN000;
10036                ; JZ     short setend        ; yes, don't do anything        ;AN000;
10037 000014BA 88C7    MOV     BH,AL
10038 000014BC AD      LODSW
10039 000014BD AB      STOSW                ; dpb_root_entries Number of directory entries
10040 000014BE B105    MOV     CL,5
10041 000014C0 D3EA    SHR     DX,CL                ; Directory entries per sector
10042 000014C2 48      DEC     AX
10043 000014C3 01D0    ADD     AX,DX                ; Cause Round Up
10044 000014C5 89D1    MOV     CX,DX
10045 000014C7 31D2    XOR     DX,DX
10046 000014C9 F7F1    DIV     CX
10047 000014CB 89C1    MOV     CX,AX                ; Number of (root) directory sectors
10048 000014CD 47      INC     DI
10049 000014CE 47      INC     DI                ; Skip dpb_first_sector
10050 000014CF A5      MOVSW                ; Total number of sectors in DSKSIZ (temp as dpb_max_cluster)
10051 000014D0 AC      LODSB
10052                ;mov     [es:bp+17h],al
10053 000014D1 26884617 MOV     [ES:BP+DPB.MEDIA],AL ; Media byte
10054 000014D5 AD      LODSW                ; Number of sectors in a FAT
10055
10056                ;;;
10057                ;MSDOS 3.3
10058                ;
10059                ;STOSB                ; DPB.FAT_SIZE
10060                ;MUL     BH
10061
10062                ;MSDOS 6.0
10063                ;
10064 000014D6 AB      STOSW                ; DPB.FAT_SIZE ;AC000;;>32mb dpb_FAT_size
10065
10066                ; 13/01/2024
10067                %if 0
10068                MOV     DL,BH                ;AN000;;>32mb
10069                XOR     DH,DH                ;AN000;;>32mb
10070                MUL     DX                ;AC000;;>32mb Space occupied by all FATs
10071                ;;;
10072
10073                ;add     ax,[es:bp+6]
10074                ADD     AX,[ES:BP+DPB.FIRST_FAT]
10075                STOSW                ; dpb_dir_sector
10076                ADD     AX,CX                ; Add number of (root) directory sectors
10077                ;mov     [es:bp+0Bh],ax
10078                MOV     [ES:BP+DPB.FIRST_SECTOR],AX
10079
10080                ; MSDOS 6.0
10081                MOV     CL,BL                ;F.C. >32mb                ;AN000;
10082                ;cmp     word [es:bp+0Bh],0
10083                ;CMP     WORD [ES:BP+DSKSIZ],0 ;F.C. >32mb                ;AN000;
10084                ;JNZ     short normal_dpb    ;F.C. >32mb                ;AN000;
10085                ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
10086                ; 15/12/2022
10087                ; 28/07/2019
10088                mov     bx,[ES:BP+DSKSIZ]
10089                or      bx,bx
10090                JNZ     short normal_dpb    ;F.C. >32mb                ;AN000;
10091                ;CMP     WORD [ES:BP+DSKSIZ],0 ;F.C. >32mb                ;AN000;
10092                ;JNZ     short normal_dpb    ;F.C. >32mb                ;AN000;
10093
10094                XOR     CH,CH                ;F.C. >32mb                ;AN000;
10095                ;mov     bx,[si+8]
10096                mov     bx,[SI+A_BPB.BIGTOTALSECTORS-A_BPB.SECTORSPERTRACK] ; 01/01/2024 (temporary)
10097                ;MOV     BX,[SI+A_BPB.BPB_BIGTOTALSECTORS-A_BPB.BPB_SECTORSPERTRACK] ;AN000;
10098                ;mov     dx,[si+10]
10099                mov     dx,[SI+A_BPB.BIGTOTALSECTORS-A_BPB.SECTORSPERTRACK+2] ; 01/01/2024 (temporary)
10100                ;MOV     DX,[SI+A_BPB.BPB_BIGTOTALSECTORS-A_BPB.BPB_SECTORSPERTRACK+2] ;AN000;
10101                SUB     BX,AX                ;AN000;;F.C. >32mb
10102                SBB     DX,0                ;AN000;;F.C. >32mb
10103                OR      CX,CX                ;AN000;;F.C. >32mb
10104                JZ      short norot         ;AN000;;F.C. >32mb
10105                rott:                ;AN000;;F.C. >32mb
10106                CLC                ;AN000;;F.C. >32mb
10107                RCR     DX,1                ;AN000;;F.C. >32mb
10108                RCR     BX,1                ;AN000;;F.C. >32mb
10109                LOOP    rott                ;AN000;;F.C. >32mb
10110                norot:                ;AN000;
10111                ; 15/12/2022
10112                ;MOV     AX,BX                ;AN000;;F.C. >32mb
10113                JMP     short setend        ;AN000;;F.C. >32mb
10114
10115                normal_dpb:
10116                ;sub     ax,[es:bp+0Bh]
10117                ;SUB     AX,[ES:BP+DSKSIZ]
10118                ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
10119                ; 15/12/2022
10120                ; bx = [es:bp+DSKSIZ]
10121                ;sub     ax,bx ; 28/07/2019
10122                ;SUB     AX,[ES:BP+DSKSIZ]
10123                ; 15/12/2022
10124                sub     bx,ax
10125                ;NEG     AX                ; Sectors in data area
10126                ;MOV     CL,BL                ; dpb_cluster_shift
10127                ; 15/12/2022
10128                ; CL = cluster shift
10129                ; BX = number of data sectors
10130                ;SHR     AX,CL                ; Div by sectors/cluster
10131                shr     bx,cl
10132                setend:
10133                ; M008 - CAS
10134                ;
10135                ; 15/12/2022
10136                inc     bx
10137                ;INC     AX                ; +2 (reserved), -1 (count -> max)
10138                ;
10139                ; There has been a bug in our fatsize calculation for so long
10140                ; that we can't correct it now without causing some user to
10141                ; experience data loss. There are even cases where allowing
10142                ; the number of clusters to exceed the fats is the optimal
10143                ; case -- where adding 2 more fat sectors would make the
10144                ; data field smaller so that there's nothing to use the extra
10145                ; fat sectors for.
10146                ;
10147                ; Note that this bug had very minor known symptoms. CHKDSK would
10148                ; still report that there was a cluster left when the disk was
10149                ; actually full. Very graceful failure for a corrupt system
10150                ; configuration. There may be worse cases that were never
10151                ; properly traced back to this bug. The problem cases only
10152                ; occurred when partition sizes were very near FAT sector
10153                ; rounding boundaries, which were rare cases.

```

```

10154 ; Also, it's possible that some third-party partition program might
10155 ; create a partition that had a less-than-perfect FAT calculation
10156 ; scheme. In this hypothetical case, the number of allocation
10157 ; clusters which don't actually have FAT entries to represent
10158 ; them might be larger and might create a more catastrophic
10159 ; failure. So we'll provide the safeguard of limiting the
10160 ; max_cluster to the amount that will fit in the FATs.
10161 ;
10162 ; ax = maximum legal cluster, ES:BP -> dpb
10163 ;
10164 ; make sure the number of fat sectors is actually enough to
10165 ; hold that many clusters. otherwise, back the number of
10166 ; clusters down
10167 ;
10168 ; 15/12/2022
10169 ; bx = number of clusters
10170 ;
10171 ; 19/07/2018 - Retro DOS v3.0
10172 ; MSDOS 6.0
10173 ; 15/12/2022
10174 ;mov bx,ax ; remember calculated # clusters
10175 ;
10176 ; 01/08/2018 (MSDOS 3.3)
10177 ;mov al,[ES:BP+DPB.FAT_SIZE]
10178 ;xor ah,ah
10179 ;
10180 ; 10/05/2019 - Retro DOS v4.0
10181 ;mov ax,[ES:BP+0Fh]
10182 mov ax,[ES:BP+DPB.FAT_SIZE]
10183 ;
10184 ;mul word [es:bp+2]
10185 mul word [ES:BP+DPB.SECTOR_SIZE] ; how big is the FAT?
10186 cmp bx,4096-10 ; 0FF6h ; test for 12 vs. 16 bit fat
10187 jb short setend_fat12
10188 shr dx,1
10189 ;
10190 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
10191 ; 15/12/2022
10192 ;cs3 7/2/92
10193 jnz short setend_faterr ; some bonehead gave us more fatspace
10194 ; than enough for the maximum FAT,
10195 ; so go ahead and use the calculated
10196 ; number of clusters.
10197 ;cs3 7/2/92
10198 ;
10199 rcr ax,1 ; find number of entries
10200 cmp ax,4096-10+1 ; would this truncation move us
10201 ; into 12-bit fatland?
10202 ; jb short setend_faterr ; then go ahead and let the
10203 ; inconsistency pass through
10204 ; ; rather than lose data by
10205 ; ; correcting the fat type
10206 jmp short setend_fat16
10207 ;
10208 setend_fat12:
10209 add ax,ax ; (fatsiz*2)/3 = # of fat entries
10210 adc dx,dx
10211 ;
10212 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
10213 ;cs3 7/2/92
10214 ; 15/12/2022
10215 cmp dx,3 ; if our fatspace is WAY more than
10216 jnb short setend_faterr ; we need, we may get an overflow
10217 ; here. check for it and use
10218 ; the calculated size in this case.
10219 ;cs3 7/2/92
10220 ;
10221 div word [cs:word3]
10222 ;
10223 setend_fat16:
10224 dec ax ; limit at 1
10225 cmp ax,bx ; is fat big enough?
10226 jbe short setend_fat ; use max value that'll fit
10227 ;
10228 setend_faterr:
10229 mov ax,bx ; use calculated value
10230 ;
10231 setend_fat:
10232 ;
10233 ; now ax = maximum legal cluster
10234 ;
10235 ; end M008
10236 ;
10237 ;mov [es:bp+0Dh], ax
10238 MOV [ES:BP+DPB.MAX_CLUSTER],AX
10239 ;
10240 ;;mov word [es:bp+1Ch],0 ; MSDOS 3.3
10241 ;mov word [es:bp+1Dh],0 ; MSDOS 6.0
10242 MOV word [ES:BP+DPB.NEXT_FREE],0
10243 ; Init so first ALLOC starts at
10244 ; beginning of FAT
10245 ;;mov word [es:bp+1Eh],-1 ; MSDOS 3.3
10246 ;mov word [es:bp+1Fh],-1 ; MSDOS 6.0
10247 MOV word [ES:BP+DPB.FREE_CNT],-1 ; current count is invalid.
10248 ;
10249 retn
10250 ;
10251 %else
10252 ; 13/01/2024 - Retro DOS v5.0
10253 ;;;
10254 xor dx,dx ; 0
10255 or ax,ax
10256 jnz short savlog1 ; 16 bit FAT size
10257 ; (*) FAT32 extensions
10258 pop dx ; (use 32 bit FAT and Root Dir size if >0)
10259 ; (*)
10260 push dx
10261 or dx,dx
10262 jz short savlog1 ; Do not use FAT32 extensions
10263 ; (do not use 32 bit FAT size field)
10264 mov ax,[si+12] ; .BPB_SECTORS PER FAT32 ; BPB_FATSz32
10265 mov dx,[si+14] ; .BPB_SECTORS PER FAT32+2
10266 savlog1:
10267 push cx ; (**) Root directory sectors
10268 mov cx,ax ; 32 bit multiply
10269 mov al,bh ; FAT count
10270 xor ah,ah
10271 mul dx
10272 xchg ax,cx
10273 mov dl,bh ; FAT count
10274 xor dh,dh
10275 mul dx
10276 add dx,cx
10277 pop cx ; (**)

```

```

10278 000014FC 39D0      cmp     ax,dx
10279 000014FE 7504      jne     short savlog2
10280
10281      ; 13/01/2024
10282      ;or     ax,ax
10283      ;jnz     short savlog2
10284
10285      ; 13/01/2024 (not necessary)
10286      ;inc     di
10287      ;inc     di
10288      ; di = DPB.DRIVER_ADDR
10289
10290      ;jmp     short setend
10291
10292      ; 13/01/2024
10293 00001500 09C0      or      ax,ax
10294 00001502 744D      jz      short setend
10295
10296 savlog2:
10297 00001504 26034606    add     ax,[es:bp+DPB.FIRST_FAT]
10298      ;add     ax,[es:bp+6] ; dx:ax = (total) FAT sectors
10299      add     dx,0
10300 0000150B AB      stosw      ; DPB.DIR_SECTOR
10301 0000150C 01C8      add     ax,cx ; + root directory size
10302 0000150E 2689460B    mov     [es:bp+DPB.FIRST_SECTOR],ax
10303      ;mov     [es:bp+11],ax ; DPB.FIRST_SECTOR ; First data sector
10304 00001512 83D200      adc     dx,0
10305 00001515 59      pop     cx ; (*)
10306 00001516 51      push    cx
10307 00001517 E308      jcxz    savlog3
10308 00001519 26894629    mov     [es:bp+DPB.FCLUS_FSECTOR],ax
10309      ;mov     [es:bp+41],ax ; DPB.BIG_FIRST_SECTOR ; FAT32 first sector field
10310 0000151D 2689562B    mov     [es:bp+DPB.FCLUS_FSECTOR+2],dx
10311      ;mov     [es:bp+43],dx
10312
10313 00001521 88D9      mov     cl,bl ; cluster shift
10314 00001523 26837E0D00    cmp     word [es:bp+DPB.MAX_CLUSTER],0
10315      ;cmp     word [es:bp+13],0 ; DPB.MAX_CLUSTER
10316      ; (contains 16 bit .BPB_TOTALSECTORS as temporary)
10317 00001528 751D      jnz     short normal_dpb
10318 0000152A 30ED      xor     ch,ch
10319 0000152C 52      push    dx ; (**)
10320 0000152D 8B5C08      mov     bx,[si+8] ; SI points to .BPB_SECTORS PER TRACK and SI+8 is
10321      ; .BPB_BIGTOTALSECTORS (32 bit total sectors)
10322 00001530 8B540A      mov     dx,[si+10]
10323 00001533 29C3      sub     bx,ax
10324 00001535 58      pop     ax ; (**)
10325 00001536 19C2      sbb     dx,ax ; dx:bx = data sectors (for cluster count calc)
10326 00001538 09C9      or      cx,cx
10327 0000153A 7407      jz      short norot
10328
10329 0000153C F8      rott:
10330 0000153D D1DA      rcr     dx,1
10331 0000153F D1DB      rcr     bx,1
10332 00001541 E2F9      loop    rott
10333
10334 00001543 89D8      norot:
10335 00001545 EB0A      mov     ax,bx ; dx:ax = cluster count
10336      jmp     short setend
10337
10338 00001547 262B460D    normal_dpb:
10339      sub     ax,[es:bp+DPB.MAX_CLUSTER]
10340      ;sub     ax,[es:bp+13] ; first sector - total sectors
10341      xor     dx,dx
10342      neg     ax ; data sectors = total sectors - first sector
10343      shr     ax,cl ; cluster count
10344
10345      ; 13/01/2024 (PCDOS 7.1 IBMDOS.COM)
10346 00001551 59      pop     cx ; (*) 0 = not 32 bit fat sectors
10347 00001552 51      push    cx ; (*)
10348
10349      ; si = BIOS Parameter Block + 13
10350
10351      ; 12/04/2024
10352      ; 13/01/2024 (PCDOS 7.1 IBMDOS.COM)
10353 00001553 83C001      add     ax,1
10354 00001556 83D200      adc     dx,0 ; calculated # clusters HW
10355 00001559 89C3      mov     bx,ax ; calculated # clusters LW
10356 0000155B 268B460F    mov     ax,[es:bp+DPB.FAT_SIZE]
10357      ;mov     ax,[es:bp+15] ; FAT size (16 bit)
10358 0000155F E30C      jcxz    setend1 ; Do not use 32 bit FAT sectors field
10359 00001561 31C9      xor     cx,cx
10360 00001563 09C0      or      ax,ax
10361 00001565 7506      jnz     short setend1
10362 00001567 8B440C      mov     ax,[si+12] ; .BPB_SECTORS PER FAT32 ; 32 bit FAT size field.
10363 0000156A 8B4C0E      mov     cx,[si+14] ; .BPB_SECTORS PER FAT32+2
10364
10365 0000156D 52      setend1:
10366      push    dx ; (**) ; cx:ax = FAT size (in sectors)
10367      ; dx:bx = calculated number of clusters
10368 0000156E 91      xchg     ax,cx
10369 0000156F 26F76602    mul     word [es:bp+DPB.SECTOR_SIZE]
10370      ;mul     word [es:bp+2] ; DPB.SECTOR_SIZE
10371 00001573 91      xchg     ax,cx
10372 00001574 26F76602    mul     word [es:bp+DPB.SECTOR_SIZE]
10373      ;mul     word [es:bp+2]
10374 00001578 01CA      add     dx,cx ; dx:ax = FAT size in bytes
10375 0000157A 59      pop     cx ; (**) ; calculated # clusters HW
10376 0000157B 09C9      or      cx,cx
10377 0000157D 750B      jnz     short setend2 ; FAT32
10378 0000157F 81FBF60F    jnz     short setend3 ; 12/04/2024 (BugFix)
10379 00001583 721E      cmp     bx,0FF6h
10380      jb      short setend_fat12 ; FAT12
10381
10382      ; (PCDOS 7.1 IBMDOS.COM)
10383      ;or      cx,cx ; HW of calculated cluster count
10384 00001585 83FBF6      jnz     short setend3
10385 00001588 720C      cmp     bx,0FFF6h
10386      jb      short setend4 ; FAT16
10387
10388      setend3:
10389      shr     dx,1 ; FAT32 ; 4 byte (32 bit) cluster number
10390      ; fatsiz/4 = # of fat entries
10391      rcr     ax,1
10392      shr     dx,1
10393      jz      short setend5 ; dx = 0
10394      rcr     ax,1
10395      jmp     short setend_fat16
10396
10397      setend4:
10398      shr     dx,1 ; FAT16 ; 2 byte (16 bit) cluster number
10399      ; fatsiz/2 = # of fat entries
10400      jnz     short setend_faterr ; dx > 0
10401
10402      setend5:
10403      rcr     ax,1 ; FAT16 ; 2 byte (16 bit) cluster number
10404      ; fatsiz/2 = # of fat entries

```

```

10402 0000159C 3DF70F      cmp     ax,0FF7h          ; 4096-10+1
10403 0000159F 721E      jnb     short setend_faterr
10404 000015A1 EB0E      jmp     short setend_fat16
10405
10406
10407 000015A3 01C0      setend_fat12:
10408                add     ax,ax          ; FAT12 ; 1.5 byte (12 bit) cluster number
10409                ; (fatsiz*2)/3 = # of fat entries
10410                adc     dx,dx
10411                cmp     dx,3          ; if our fatspace is more than we need
10412                ; use calculated size
10413 000015AA 7313      jnb     short setend_faterr
10414 000015AC 2EF736[6314]  div     word [cs:word3]
10415
10416 000015B1 83E801      setend_fat16:
10417 000015B4 83DA00      sub     ax,1
10418 000015B7 39CA      sbb     dx,0
10419 000015B9 7704      cmp     dx,cx          ; is fat big enough?
10420 000015BB 39D8      ja      short setend_faterr
10421 000015BD 7604      cmp     ax,bx
10422                jbe     short setend_fat32 ; use max value that'll fit
10423                setend_faterr:
10424                mov     ax,bx          ; use calculated value
10425                mov     dx,cx
10426
10427 000015C3 26837E0F00  setend_fat32:
10428                cmp     word [es:bp+DPB.FAT_SIZE],0
10429                ;cmp word [es:bp+15],0 ; DPB.FAT_SIZE ; 16 bit FAT size
10430                jnz     short setend6
10431                mov     word [es:bp+DPB.DIR_SECTOR],-1
10432                ;mov word [es:bp+17],0FFFFh; DPB.DIR_SECTOR
10433                ; (16 bit directory sector field)
10434                mov     word [es:bp+DPB.MAX_CLUSTER],0
10435                ;mov word [es:bp+13],0 ; DPB.MAX_CLUSTER (16 bit)
10436                jmp     short setend7
10437
10438 000015D8 2689460D  setend6:
10439                mov     [es:bp+DPB.MAX_CLUSTER],ax
10440                ;mov [es:bp+13],ax ; DPB.MAX_CLUSTER = calculated last cluster number
10441
10442 000015DC 59      setend7:
10443                pop     cx          ; (*)
10444                ; 1 = use FAT32 extensions
10445                ; 0 = don't use FAT32 extensions (32 bit fields)
10446                jcxz    setend_fat ; do not use FAT32 extensions
10447                ;mov [es:bp+45],ax ; DPB.MAX_CLUSTER32 ; dx:ax = last cluster number
10448                ;mov [es:bp+47],dx
10449                mov     [es:bp+DPB.LAST_CLUSTER],ax
10450                mov     [es:bp+DPB.LAST_CLUSTER+2],dx
10451                mov     ax,0FFFFh ; -1
10452                lea     di,[bp+DPB.FREE_CNT_HW]
10453                ;lea di,[bp+33] ; DPB.FAT32_EXT ; FAT32 extensions
10454                ; -1 = ready
10455
10456 000015ED AB      stosw
10457 000015EE 8D7410  lea     si,[si+16] ; FAT32 flags
10458 000015F1 A5      movsw ; DPB FAT32 flags ; [bp+23h]
10459 000015F2 83C606  add     si,6
10460 000015F5 AD      lodsw
10461 000015F6 8B54DC  mov     dx,[si-24h] ; FSINFO structure sector number
10462 000015F9 09C0      or     ax,ax ; .BPB_RESERVEDSECTORS ; Number of reserved sectors.
10463 000015FB 7404      jz     short setend8
10464 000015FD 39D0      cmp     ax,dx
10465 000015FF 7203      jb     short setend9
10466
10467 00001601 B8FFFF  setend8:
10468                mov     ax,0FFFFh ; -1 ; invalid
10469
10470 00001604 AB      setend9:
10471                stosw
10472                ; DPB FSINFO structure sector number
10473                ; [bp+25h]
10474                ; Sector number of the backup boot sector
10475                lodsw
10476                or     ax,ax
10477                jz     short setend10
10478                cmp     ax,dx
10479                jb     short setend11
10480
10481 0000160E B8FFFF  setend10:
10482                mov     ax,0FFFFh ; -1 ; invalid
10483
10484 00001611 AB      setend11:
10485                stosw
10486                ; DPB backup boot sector address
10487                ; [bp+27h]
10488                ; [bp+31h]
10489                add     di,8
10490                xor     dx,dx
10491                mov     ax,[es:di-34] ; [bp+0Fh] ; DPB.MAX_CLUSTER
10492                cmp     ax,dx
10493                jnz     short setend12 ; > 0 (not FAT32)
10494                mov     ax,[si-16] ; FAT32 Sectors per FAT ; .BPB_SECTORS PER FAT32
10495                mov     dx,[si-14]
10496
10497 00001625 AB      setend12:
10498                stosw ; DPB FAT32 FAT size in sectors ; [bp+31h]
10499                mov     ax,dx
10500                stosw
10501                sub     si,8 ; Root directory cluster number
10502                movsw ; DPB Root Dir Cluster ; [bp+35h]
10503                movsw
10504                xor     ax,ax ; DPB reserved ; [bp+39h]
10505                stosw
10506                stosw
10507                %endif
10508                ;;;
10509
10510 00001632 31C0      setend_fat:
10511                ; 13/01/2024 - Retro DOS v5.0
10512                xor     ax,ax ; 0
10513
10514                ;mov word [es:bp+1Dh],ax ; 0
10515                mov     word [es:bp+DPB.NEXT_FREE],ax ; 0
10516                ; Init so first ALLOC starts at
10517                ; begining of FAT
10518                dec     ax ; -1
10519                ;mov word [es:bp+1Fh],ax ; -1
10520                mov     word [es:bp+DPB.FREE_CNT],ax ; -1 ; current count is invalid.
10521
10522 0000163D C3      retn
10523
10524 ;EndProc $SETDPB
10525
10526 ;BREAK <$Create_Process_Data_Block,SetMem -- Set up process data block>
10527
10528 ;
10529 ;-----
10530 ;
10531 ;** $Dup_PDB
10532 ;
10533 ; Inputs: DX is new segment address of process
10534 ; SI is end of new allocation block
10535 ;
10536 ;-----
10537 ;
10538 ; 14/01/2024 - Retro DOS 5.0
10539 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:554Fh
10540

```

```

10526 _$DUP_PDB:
10527
10528 ;hkn; CreatePDB would have a CS override. This is not valid.
10529 ;hkn; Must set up ds in order to access CreatePDB. Also SS is
10530 ;hkn; has been assumed to be NOTHING. It may not have DOSDATA.
10531
10532 ; MSDOS 3.3
10533 ;MOV byte [CS:CreatePDB],0FFh ; indicate a new process
10534 ;MOV DS,[CS:CurrentPDB]
10535
10536 ; 15/05/2019 - Retro DOS v4.0
10537 ; MSDOS 6.0
10538 0000163E 2E8E1E[0700] mov ds,[cs:DosDSeg]
10539 00001643 C606[A803]FF MOV byte [CreatePDB],0FFh
10540 00001648 8E1E[3003] MOV DS,[CurrentPDB]
10541
10542 0000164C 56 PUSH SI
10543 0000164D EB0A JMP SHORT CreateCopy
10544
10545 ;
10546 ;-----
10547 ;
10548 ; Inputs:
10549 ; DX = Segment number of new base
10550 ; Function:
10551 ; Set up program base and copy term and ^C from int area
10552 ; Returns:
10553 ; None
10554 ; Called at DOS init
10555 ;
10556 ;-----
10557 ;
10558 ;
10559 ; 15/05/2019 - Retro DOS v4.0
10560 ; DOSCODE:4EB6h (MSDOS 6.21, MSDOS.SYS)
10561
10562 ; 06/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
10563 ; DOSCODE:4EA2h (MSDOS 5.0, MSDOS.SYS)
10564
10565 _$CREATE_PROCESS_DATA_BLOCK:
10566 ; Offset 1D02h in IBMDOS.COM (MSDOS 3.3), 1987
10567 0000164F E825EE CALL Get_User_Stack
10568 ;mov ds,[si+14h]
10569 00001652 8E5C14 MOV DS,[SI+user_env.user_CS]
10570 ;push word [2]
10571 00001655 FF360200 PUSH word [PDB.BLOCK_LEN] ;*
10572 CreateCopy:
10573 00001659 8EC2 MOV ES,DX
10574
10575 0000165B 31F6 XOR SI,SI ; copy entire PDB
10576 0000165D 89F7 MOV DI,SI
10577 0000165F B98000 MOV CX,128
10578 00001662 F3A5 REP MOVSW
10579
10580 ; DOS 3.3 7/9/86
10581 ;mov cx,20
10582 ;MOV CX,FILPERPROC ; copy handles in case of
10583 ; 15/12/2022
10584 00001664 B114 mov cl,FILPERPROC ; 06/07/2019
10585 ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
10586 ;mov cx,FILPERPROC
10587
10588 ;mov di,18h
10589 00001666 BF1800 MOV DI,PDB.JFN_TABLE ; Set Handle Count has been issued
10590 ;;PUSH DS ; * 15/05/2019
10591 ;;lds si,[34h]
10592 ;LDS SI,[PDB.JFN_Pointer]
10593 ;REP MOVSB
10594 ;;POP DS ; * 15/05/2019
10595 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
10596 ; 05/12/2022
10597 ; (push ds then pop ds is not needed here!)
10598 ;push ds
10599 ;lds si,[34h]
10600 00001669 C5363400 lds si,[PDB.JFN_Pointer]
10601 0000166D F3A4 rep movsb
10602 ;pop ds
10603
10604 ; DOS 3.3 7/9/86
10605 ;hkn ;CreatePDB would have a CS override. This is not valid.
10606 ;hkn ;Must set up ds in order to access CreatePDB. Also SS is
10607 ;hkn ;has been assumed to be NOTHING. It may not have DOSDATA.
10608
10609 0000166F 2E8E1E[0700] mov ds,[cs:DosDSeg] ; 15/05/2019
10610
10611 ;;test byte [cs:CreatePDB],0FFh
10612 ;cmp byte [CS:CreatePDB],0 ; Shall we create a process?
10613 ; 17/12/2022
10614 00001674 380E[A803] cmp [CreatePDB],cl ; 0
10615 ;cmp byte [CreatePDB],0 ; 15/05/2019
10616 00001678 744A JZ short Create_PDB_cont ; nope, old style call
10617
10618 ; Here we set up for a new process...
10619
10620 ;PUSH CS ; Called at DOSINIT time, NO SS
10621 ;POP DS
10622
10623 ; MSDOS 6.0
10624 ;;getdseg <ds> ; ds -> dosdata
10625 ;mov ds,[cs:DosDSeg] ; 15/05/2019
10626 ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
10627 ; (nonsense! but i put this for addr compatibility as temporary)
10628 ; 15/12/2022
10629 ;mov ds,[cs:DosDSeg] ; 15/05/2019
10630
10631 0000167A 31DB XOR BX,BX ; dup all jfns
10632 ;mov cx,20
10633 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
10634 ;MOV CX,FILPERPROC ; only 20 of them
10635 ; 15/12/2022
10636 0000167C B114 mov cl,FILPERPROC ; 06/07/2019
10637
10638 Create_dup_jfn:
10639 0000167E 06 PUSH ES ;** ; save new PDB
10640 0000167F E85160 call SFFromHandle ; get sf pointer
10641 00001682 B0FF MOV AL,-1 ; unassigned JFN
10642 00001684 7224 JC short CreateStash ; file was not really open
10643 ;;test word [es:di+5],1000h
10644 ;TEST word [ES:DI+SF_ENTRY.sf_flags],sf_no_inherit
10645 ; 15/05/2019
10646 ;test byte [es:di+6],10h
10647 00001686 26F6450610 test byte [ES:DI+SF_ENTRY.sf_flags+1],(sf_no_inherit>>8)
10648 0000168B 751D JNZ short CreateStash ; if no-inherit bit is set, skip dup.
10649

```



```

10650 ; we do not inherit network file handles.
10651
10652 ;mov ah,[es:di+2]
10653 0000168D 268A6502 MOV AH,[ES:DI+SF_ENTRY.sf_mode]
10654 ;and ah,0F0h
10655 00001691 80E4F0 AND AH,SHARING_MASK
10656 ;cmp ah,70h
10657 00001694 80FC70 CMP AH,SHARING_NET_FCB
10658 00001697 7411 JZ short CreateStash
10659
10660 ; The handle we have found is duplicatable (and inheritable). Perform
10661 ; duplication operation.
10662
10663 00001699 893E[9E05] MOV [THISSFT],DI
10664 0000169D 8C06[A005] MOV [THISSFT+2],ES
10665 000016A1 E8101A CALL DOS_DUP ; signal duplication
10666
10667 ; get the old sfh for copy
10668
10669 000016A4 E80F60 CALL PJFNFromHandle ; ES:DI is jfn
10670 000016A7 268A05 MOV AL,[ES:DI] ; get sfh
10671
10672 ; Take AL (old sfh or -1) and stash it into the new position
10673
10674 CreateStash:
10675 000016AA 07 POP ES ;**
10676 ;mov [es:bx+18h],al
10677 000016AB 26884718 MOV [ES:BX+PDB.JFN_TABLE],AL ; copy into new place!
10678 000016AF 43 INC BX ; next jfn...
10679 000016B0 E2CC LOOP Create_dup_jfn
10680
10681 000016B2 8B1E[3003] MOV BX,[CurrentPDB] ; get current process
10682 ; 06/11/2022
10683 ;mov [es:16h],bx
10684 000016B6 26891E1600 MOV [ES:PDB.PARENT_PID],BX; stash in child
10685 000016BB 8C06[3003] MOV [CurrentPDB],ES
10686 ;MOV DS,BX ; 28/07/2019
10687 ; 07/12/2022
10688 ;mov ds,[cs:DosDSeg]
10689 ; 15/12/2022
10690 ; ds = [cs:DosDSeg]
10691 000016BF C606[A803]00 MOV byte [CreatePDB],0 ; reset flag
10692 ;mov ds,bx
10693 ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
10694 ; 15/12/2022
10695 ;mov ds,bx
10696
10697 ; end of new process create
10698
10699 Create_PDB_cont:
10700 ;MOV BYTE [CS:CreatePDB],0 ; reset flag
10701
10702 ;hkn; It comes to this point from 2 places. So, change to DOSDATA temporarily
10703
10704 ;, 28/07/2019
10705 ;;push ds
10706 ;;mov ds,[cs:DosDSeg]
10707 ;mov byte [CreatePDB],0
10708 ;;pop ds
10709
10710 ; 05/12/2022
10711 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
10712 ; ; (push-pop ds is nonsense here!
10713 ; ; but i am using same code with original MSDOS.SYS
10714 ; ; for address compatibility.)
10715 ; push ds
10716 ; ds = [cs:DosDSeg] !
10717 ; mov ds,[cs:DosDSeg] ; again !
10718 ; mov byte [CreatePDB],0
10719 ; pop ds
10720
10721 000016C4 58 POP AX ;*
10722
10723 ;entry SETMEM
10724
10725 ; 17/12/2022
10726 ; cx = 0
10727
10728 ;-----
10729 ; Inputs:
10730 ; AX = Size of memory in paragraphs
10731 ; DX = Segment
10732 ; Function:
10733 ; Completely prepares a program base at the
10734 ; specified segment.
10735 ; Called at DOS init
10736 ; Outputs:
10737 ; DS = DX
10738 ; ES = DX
10739 ; [0] has INT int_abort
10740 ; [2] = First unavailable segment
10741 ; [5] to [9] form a long call to the entry point
10742 ; [10] to [13] have exit address (from int_terminate)
10743 ; [14] to [17] have ctrl-C exit address (from int_ctrl_c)
10744 ; [18] to [21] have fatal error address (from int_fatal_abort)
10745 ; DX,BP unchanged. All other registers destroyed.
10746 ;-----
10747
10748 SETMEM:
10749 ;XOR CX,CX
10750 ; 17/12/2022
10751 ; cx = 0
10752 000016C5 8ED9 MOV DS,CX
10753 000016C7 8EC2 MOV ES,DX
10754 ;mov si,88h
10755 000016C9 BE8800 MOV SI,addr_int_terminate
10756 ;mov di,10 ; 0Ah
10757 000016CC BF0A00 MOV DI,SAVEEXIT
10758 ;MOV CX,6
10759 ; 15/12/2022
10760 000016CF B106 MOV cl,6
10761 000016D1 F3A5 REP MOVSW
10762 000016D3 26A30200 MOV [ES:2],AX
10763 000016D7 29D0 SUB AX,DX
10764 000016D9 3DFF0F CMP AX,MAXDIF ; 0FFFh
10765 000016DC 7603 JBE short HAVDIF
10766 000016DE B8FF0F MOV AX,MAXDIF
10767
10768 000016E1 83E810 SUB AX,10h ; Allow for 100h byte "stack"
10769 000016E4 BB0C00 MOV BX,ENTRYPOINTSEG ; 0Ch; in .COM files
10770 000016E7 29C3 SUB BX,AX
10771 000016E9 B104 MOV CL,4
10772 000016EB D3E0 SHL AX,CL
10773 000016ED 8EDA MOV DS,DX

```

```

10774 ; (MSDOS 6.0 note)
10775 ;
10776 ; The address in BX:AX will be F01D:FEF0 if there is 64K or more
10777 ; memory in the system. This is equivalent to 0:c0 if A20 is OFF.
10778 ; If DOS is in HMA this equivalence is no longer valid as A20 is ON.
10779 ; But the BIOS which now resides in FFFF:30 has 5 bytes in FFFF:D0
10780 ; (F01D:FEF0) which is the same as the ones in 0:C0, thereby
10781 ; making this equivalence valid for this particular case. If however
10782 ; there is less than 64K remaining the address in BX:AX will not
10783 ; be the same as above. We will then stuff 0:c0, the call 5 address
10784 ; into the PSP.
10785 ;
10786 ;
10787 ; Therefore for the case where there is less than 64k remaining in
10788 ; the system old CPM Apps that look at PSP:6 to determine memory
10789 ; requirements will not work. Call 5, however will continue to work
10790 ; for all cases.
10791 ;
10792 ;
10793 ;mov [6],ax
10794 ;mov [8],bx
10795
10796 000016EF A30600 MOV [PDB.CPM_CALL+1],AX
10797 000016F2 891E0800 MOV [PDB.CPM_CALL+3],BX
10798
10799 ; 06/05/2019 - Retro DOS v4.0
10800 000016F6 3DF0FE cmp ax,WRAPOFFSET ; 0FEF0h ; Q: does the system have >= 64k of
10801 ; memory left
10802 000016F9 740C je short addr_ok ; Y: the above calculated address is
10803 ; OK
10804 ; N:
10805
10806 000016FB C7060600C000 MOV WORD [PDB.CPM_CALL+1],0C0h
10807 00001701 C70608000000 MOV WORD [PDB.CPM_CALL+3],0
10808
10809 addr_ok:
10810 00001707 C7060000CD20 ;mov word [0],20CDh
10811 MOV word [PDB.EXIT_CALL],(int_abort*256) + mi_INT
10812 0000170D C60605009A ;mov byte [5],9Ah
10813 MOV BYTE [PDB.CPM_CALL],mi_Long_CALL
10814 00001712 C7065000CD21 ;mov word [50h],21CDh
10815 MOV WORD [PDB.CALL_SYSTEM],(int_command*256) + mi_INT
10816 00001718 C6065200CB ;mov byte [52h],0CBh
10817 MOV BYTE [PDB.CALL_SYSTEM+2],mi_Long_RET
10818 0000171D C70634001800 ;mov word [34h],18h
10819 MOV WORD [PDB.JFN_Pointer],PDB.JFN_TABLE
10820 00001723 8C1E3600 ;mov word [36h],ds
10821 MOV WORD [PDB.JFN_Pointer+2],DS
10822 00001727 C70632001400 ;mov word [32h],20
10823 MOV WORD [PDB.JFN_Length],FILPERPROC
10824 ;
10825 ; The server runs several PDB's without creating them VIA EXEC. We need to
10826 ; enumerate all PDB's at CPS time in order to find all references to a
10827 ; particular SFT. We perform this by requiring that the server link together
10828 ; for us all sub-PDB's that he creates. The requirement for us, now, is to
10829 ; initialize this pointer.
10830 ;
10831 0000172D C7063800FFFF ;mov word [38h],-1
10832 MOV word [PDB.Next_PDB],-1
10833 00001733 C7063A00FFFF ;mov word [3Ah],-1
10834 MOV word [PDB.Next_PDB+2],-1
10835
10836 ; 06/05/2019
10837 ; Set the real version number in the PSP - 5.00
10838
10839 ;mov word [es:PDB.Version],1406h ; MSDOS 6.21 (DOSCODE:4FB6h)
10840 00001739 26C7064000070A ; 07/12/2022
10841 mov word [ES:PDB.Version],(MINOR_VERSION*256)+MAJOR_VERSION
10842 00001740 C3
10843
10844 ; 29/04/2019 - Retro DOS v4.0
10845
10846 ;BREAK <$SetMediaID -- get set media ID>
10847
10848 ;-----
10849 ; Inputs:
10850 ; BL= drive number as defined in IOCTL
10851 ; AL= 0 get media ID
10852 ; 1 set media ID
10853 ; DS:DX= buffer containing information
10854 ; DW 0 info level (set on input)
10855 ; DD ? serial #
10856 ; DB 11 dup(?) volume id
10857 ; DB 8 dup(?) file system type
10858 ; Function:
10859 ; Get or set media ID
10860 ; Returns:
10861 ; carry clear, DS:DX is filled
10862 ; carry set, error
10863 ;-----
10864
10865 ; 06/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
10866
10867 ; 14/01/2024
10868 %if 0
10869
10870 _$GSetMediaID:
10871 ; RAWIO - GET_MEDIA_ID
10872 mov cx,0866h ;AN000;MS.; assume get for IOCTL
10873 cmp al,0 ;AN001;MS.; get ?
10874 je short doioc1 ;AN000;MS.; yes
10875 ;cmp al,1 ;AN000;MS.; set ?
10876 ;jne short errorfunc ;AN000;MS.; no
10877 ; 15/12/2022
10878 dec al
10879 jnz short errorfunc ; al > 1
10880 ; RAWIO - SET_MEDIA_ID
10881 ;mov cx,0846h ;AN001;MS.;
10882 ; 15/12/2022
10883 mov cl,46h ; cx = 0846h
10884
10885 doioc1: ;AN000;
10886 mov al,0dh ;AN000;MS.; generic IOCTL
10887 ;invoke $IOCTL ;AN000;MS.; let IOCTL take care of it
10888 ;call _$IOCTL
10889 ;retn ;AN000;MS.;
10890 ; 15/12/2022
10891 jmp _$IOCTL
10892 errorfunc: ;AN000;
10893 ;error error_invalid_function;AN000;MS. ; invalid function
10894 ;mov al,1
10895 mov al,error_invalid_function
10896 jmp SYS_RET_ERR
10897
10898 %else

```



```

10898 ; 14/01/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
10899 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:5667h
10900
10901 _GSetMediaID:
10902 00001741 1E      push    ds
10903 00001742 56      push    si
10904 00001743 36C536[A205] lds     si,[ss:THISCDS]
10905 00001748 C57445      lds     si,[si+45h]
10906 ; [si+curdir.devptr]
10907 00001748 88740F      mov     si,[si+0Fh]
10908 0000174E B96608      mov     cx,0866h
10909 00001751 3C01      cmp     al,1
10910 00001753 7208      jnb     short doioctl1
10911 00001755 7733      ja      short errorfunc
10912 00001757 B146      mov     cl,46h ; cx = 0846h
10913 ; or si,si
10914 00001759 21F6      and     si,si ; 14/01/2024
10915 0000175B 7424      jz      short doioctl2
10916 doioctl1:
10917 0000175D 09F6      or      si,si
10918 0000175F 7522      jnz     short doioctl1
10919 00001761 1F      pop     ds
10920 00001762 1E      push    ds
10921 00001763 89D6      mov     si,dx ; disk info
10922 ; .....
10923 ; 00h WORD 0000h (info level)
10924 ; 02h DWORD disk serial number (binary)
10925 ; 06h 11 BYTES volume label or "NO NAME" if none present
10926 ; 11h 8 BYTES (AL=00h only) filesystem type
10927 ; "FAT12"
10928 ; "FAT16"
10929 ; "FAT32" ; PC DOS 7.1
10930 ; "CDROM"
10931 ; "CD001"
10932 ; "CDAUDIO"
10933 ; .....
10934 ; (ref: Ralf Brown's Interrupt List)
10935
10936
10937 00001765 817C114641      cmp     word [si+11h],4146h ; 'FA'
10938 0000176A 7517      jne     short doioctl1
10939 0000176C 817C135433      cmp     word [si+13h],3354h ; 'T3'
10940 00001771 7510      jne     short doioctl1
10941 00001773 817C153220      cmp     word [si+15h],2032h ; '2 '
10942 00001778 7509      jne     short doioctl1
10943 0000177A 817C172020      cmp     word [si+17h],2020h ; ' '
10944 0000177F 7502      jne     short doioctl1
10945 doioctl2:
10946 00001781 B548      mov     ch,48h ; cx = 4846h
10947 doioctl:
10948 00001783 5E      pop     si
10949 00001784 1F      pop     ds
10950 00001785 B00D      mov     al,0Dh ; generic IOCTL
10951 ; call _$IOCTL
10952 ; retn
10953 00001787 E9FA10      jmp     _$IOCTL ; let IOCTL take care of it
10954 errorfunc:
10955 0000178A B001      mov     al,1 ; error_invalid_function
10956 0000178C E9E9EE      jmp     SYS_RET_ERR
10957
10958 %endif
10959
10960 ; 16/05/2019 - Retro DOS v4.0
10961
10962 ; =====
10963 ; MISC2.ASM, MSDOS 6.0, 1991
10964 ; =====
10965 ; 20/07/2018 - Retro DOS v3.0
10966 ; 29/04/2019 - Retro DOS v4.0
10967
10968 ; Break <STRCMP - compare two ASCIZ strings DS:SI to ES:DI>
10969 ; -----
10970 ;
10971 ; Strcmp - compare ASCIZ DS:SI to ES:DI. Case INSENSITIVE. '/' = '\'
10972 ; Strings of different lengths don't match.
10973 ; Inputs: DS:SI - pointer to source string ES:DI - pointer to dest string
10974 ; Outputs: Z if strings same, NZ if different
10975 ; Registers modified: NONE
10976 ; -----
10977
10978 ; 07/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
10979
10980 ; 14/01/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
10981 ; (PC DOS 7.1 IBMDOS.COM DOSCODE:56B6h)
10982 ; (MSDOS 6.22 MSDOS.SYS DOSCODE:4FD7h)
10983 StrCmp:
10984 0000178F 56      push    si
10985 00001790 57      push    di
10986 00001791 50      push    ax
10987 Cmp1p:
10988 LODSB
10989 00001793 E80146      call    UCase ; convert to upper case
10990 00001796 E85146      call    PATHCHRCMP ; convert '/' to '\' ; 07/12/2022 ('\')
10991 00001799 88C4      MOV     AH,AL
10992 0000179B 268A05      MOV     AL,[ES:DI]
10993 0000179E 47      INC     DI
10994 0000179F E8F545      call    UCase ; convert to upper case
10995 000017A2 E84546      call    PATHCHRCMP ; convert '/' to '\' ; 07/12/2022 ('\')
10996 000017A5 38C4      CMP     AH,AL
10997 000017A7 7504      JNZ     short PopRet ; strings dif
10998
10999 OR      AL,AL
11000 JNZ     short Cmp1p ; More string
11001 PopRet:
11002 000017AD 58      pop     ax
11003 000017AE 5F      pop     di
11004 000017AF 5E      pop     si
11005 000017B0 C3      retn
11006
11007 ;Break <STRCPY - copy ASCIZ string from DS:SI to ES:DI>
11008 ; -----
11009 ;
11010 ; Strcpy - copy an ASCIZ string from DS:SI to ES:DI and make uppercase
11011 ; FStrcpy - copy an ASCIZ string from DS:SI to ES:DI. no modification of
11012 ; characters.
11013 ;
11014 ; Inputs: DS:SI - pointer to source string
11015 ; ES:DI - pointer to destination string
11016 ; Outputs: ES:DI point byte after nul byte at end of dest string
11017 ; DS:SI point byte after nul byte at end of source string
11018 ; Registers modified: SI,DI
11019 ; -----
11020 StrCpy:
11021

```

```

11022 000017B1 50      push    ax
11023 CPYLoop:
11024 000017B2 AC      LODSB
11025 000017B3 E8E145   call    UCASE          ; convert to upper case
11026 000017B6 E83146   call    PATHCHRCMP     ; convert / to \ ;
11027 000017B9 AA      STOSB
11028
11029 000017BA 08C0      OR      AL,AL
11030 000017BC 75F4      JNZ     short CPYLoop
11031 000017BE 58      pop     ax
11032 000017BF C3      retn
11033
11034 ;-----
11035 ; Procedure Name : FStrCpy
11036 ;-----
11037
11038 FStrCpy:
11039 000017C0 50      push    ax
11040 FCPYLoop:
11041 000017C1 AC      LODSB
11042 000017C2 AA      STOSB
11043 000017C3 08C0      OR      AL,AL
11044 000017C5 75FA      JNZ     short FCPYLoop
11045 000017C7 58      pop     ax
11046 000017C8 C3      retn
11047
11048 ; 20/07/2018 - Retro DOS v3.0
11049
11050 ; UCASE, IBMDOS.COM (MSDOS 3.3), 1987 - Offset 1E2Fh
11051 ;-----
11052
11053 ; UCASE:
11054 ; call    _UCASE    ; Offset 5518h (GetLet, Offset 5517h)
11055 ; retn
11056
11057 ; Break <StrLen - compute length of string ES:DI>
11058 ;-----
11059 ;** StrLen - Compute Length of String
11060 ;
11061 ; StrLen computes the length of a string, including the trailing 00
11062 ;
11063 ; ENTRY   (es:di) = address of string
11064 ; EXIT    (cx) = size of string
11065 ; USES    cx, flags
11066 ;-----
11067
11068 StrLen:
11069 000017C9 57      push    di
11070 000017CA 50      push    ax
11071 ; MOV     CX,-1
11072 000017CB B9FFFF   mov     cx,65535
11073 000017CE 30C0      XOR     AL,AL
11074 000017D0 F2AE      REPNE  SCASB
11075 000017D2 F7D1      NOT     CX
11076 000017D4 58      pop     ax
11077 000017D5 5F      pop     di
11078 000017D6 C3      retn
11079
11080 ;-----
11081 ;** DStrLen - Compute Length of String
11082 ;
11083 ; ENTRY   (ds:si) = address of string
11084 ; EXIT    (cx) = size of string, including trailing NUL
11085 ; USES    cx, flags
11086 ;-----
11087
11088 DStrLen: ; BUGBUG - this guy is a pig, who uses him?
11089 000017D7 E80300   CALL    XCHGP
11090 000017DA E8ECFF   CALL    StrLen
11091 ; CALL    XCHGP
11092 ; retn
11093 ; 18/12/2022
11094 ; jmp     short XCHGP
11095
11096 ;-----
11097 ;** XCHGP - Exchange Source and Destination Pointers
11098 ;
11099 ; XCHGP exchanges (DS:SI) and (ES:DI)
11100 ;
11101 ; ENTRY   none
11102 ; EXIT    pairs exchanged
11103 ; USES    SI, DI, DS, ES
11104 ;-----
11105
11106 XCHGP:
11107 000017DD 1E      push    ds
11108 000017DE 06      push    es
11109 000017DF 1F      pop     ds
11110 000017E0 07      pop     es
11111 000017E1 87F7     XCHG    SI,DI
11112
11113 000017E3 C3      xchgp_retn: retn
11114
11115 ; Break <Idle - wait for a specified amount of time>
11116 ;-----
11117 ;
11118 ; Idle - when retrying an operation due to a lock/sharing violation,
11119 ; we spin until RetryLoop is exhausted.
11120 ;
11121 ; Inputs: RetryLoop is the number of times we spin
11122 ; Outputs: wait
11123 ; Registers modified: none
11124 ;-----
11125
11126 Idle:
11127 ; test    byte [SS:FSHARING],0FFh
11128 000017E4 36803E[7205]00 cmp     byte [SS:FSHARING],0 ;hkn; SS override
11129 ; retnz
11130 000017EA 75F7     jnz     short xchgp_retn
11131 ; SAVE    <CX>
11132 000017EC 51      push    cx
11133 000017ED 368B0E[1C00] mov     cx,[ss:RetryLoop] ;hkn; SS override
11134 000017F2 E308     JCXZ    Idle3
11135
11136 000017F4 51      PUSH    CX
11137 000017F5 31C9     XOR     CX,CX
11138
11139 000017F7 E2FE     LOOP   Idle2
11140 000017F9 59      POP     CX
11141 000017FA E2F8     LOOP   Idle1
11142
11143 Idle3:
11144 000017FC 59      ; RESTORE <CX>
11145 000017FD C3      pop     cx
retn

```

```

11146
11147 ;Break      <TableDispatch - dispatch to a table>
11148 ;-----
11149 ;
11150 ; TableDispatch - given a table and an index, jmp to the appropriate
11151 ; routine. Preserve all input registers to the routine.
11152 ;
11153 ; Inputs: Push    return address
11154 ;         Push    Table address
11155 ;         Push    index (byte)
11156 ; Outputs:      appropriate routine gets jumped to.
11157 ;         return indicates invalid index
11158 ; Registers modified: none.
11159 ;-----
11160
11161 struct TFrame      ; TableFrame
11162 .OldBP:    resw 1 ; 0
11163 .OldRet:   resw 1 ; 2
11164 .Index:    resb 1 ; 4
11165 .Pad:      resb 1 ; 5
11166 .Tab:      resw 1 ; 6
11167 .NewRet:   resw 1 ; 8
11168 endstruct
11169
11170 TableDispatch:
11171     PUSH    BP
11172     MOV     BP,SP
11173     PUSH    BX
11174     ;mov     bx,[bp+6]
11175     MOV     BX,[BP+TFrame.Tab] ; get pointer to table
11176     MOV     BL,[CS:BX]         ; maximum index
11177     ;cmp     [bp+4],b1
11178     CMP     [BP+TFrame.Index],BL ; table error?
11179     JAE     short TableError   ; yes
11180     ;mov     b1,[bp+4]
11181     MOV     BL,[BP+TFrame.Index] ; get desired table index
11182     XOR     BH,BH               ; convert to word
11183     SHL     BX,1               ; convert to word pointer
11184     INC     BX                 ; point past first length byte
11185     ; 17/08/2018
11186     ;add     bx,[bp+6]
11187     ADD     BX,[BP+TFrame.Tab] ; get real offset
11188     MOV     BX,[CS:BX]         ; get contents of table entry
11189     ;mov     [bp+6],bx
11190     MOV     [BP+TFrame.Tab],BX ; put table entry into return address
11191     POP     BX                 ; restore BX
11192     POP     BP                 ; restore BP
11193     ADD     SP,4               ; clean off Index and our return addr
11194     retn                     ; do operation
11195
11196 TableError:
11197     POP     BX                 ; restore BX
11198     POP     BP                 ; restore BP
11199     RETN     6                 ; clean off Index, Table and RetAddr
1200
1201 ;Break      <TestNet - determine if a CDS is for the network>
1202 ;-----
1203 ;
1204 ; TestNet - examine CDS pointed to by ThisCDS and see if it indicates a
1205 ; network CDS. This will handle NULL cds also.
1206 ;
1207 ; Inputs: ThisCDS points to CDS or NULL
1208 ; Outputs:      ES:DI = ThisCDS
1209 ;             carry Set => network
1210 ;             carry Clear => local
1211 ; Registers modified: none.
1212 ;-----
1213
1214 TestNet:
1215     ;LES     DI,[CS:THISCDS]
1216     ; 16/05/2019 - Retro DOS v4.0
1217     mov     es,[cs:DosDseg]
1218     LES     DI,[ES:THISCDS]
1219     CMP     DI,-1
1220     JZ      short CMCRet      ; UNC? carry is clear
1221     ;test    word [es:di+43h],8000h
1222     ;TEST    word [ES:DI+curdir.flags],curdir_isnet
1223     ;test    byte [es:di+44h],80h
1224     TEST    byte [ES:DI+curdir.flags+1],(curdir_isnet>>8)
1225     JNZ     short CMCRet      ; jump has carry clear
1226     retn
1227 CMCRet:
1228     CMC
1229     retn
1230
1231 ;Break      <IsSFTNet - see if an sft is for the network>
1232 ;-----
1233 ;
1234 ; IsSFTNet - examine SF pointed to by ES:DI and see if it indicates a
1235 ; network file.
1236 ;
1237 ; Inputs: ES:DI point to SFT
1238 ; Outputs:      Zero set if not network sft
1239 ;             zero reset otherwise
1240 ;             Carry CLEAR!!!
1241 ; Registers modified: none.
1242 ;-----
1243
1244 IsSFTNet:
1245     ;test    word [es:di+5],8000h
1246     ;TEST    word [ES:DI+SF_ENTRY.sf_flags],sf_isnet
1247     ; 16/05/2019
1248     ;test    byte [es:di+6],80h
1249     TEST    byte [ES:DI+SF_ENTRY.sf_flags+1],(sf_isnet>>8)
1250     retn
1251
1252 ;Break      <FastInit - Initialize FastTable entries >
1253 ;-----
1254 ;
1255 ; DOS 4.00 2/9/87
1256 ; FastInit - initialize the FASTXXX routine entry
1257 ;             in the FastTable
1258 ;
1259 ; Inputs: BX = FASTXXX ID ( 1=fastopen )
1260 ;         DS:SI = address of FASTXXX routine entry
1261 ;         SI = -1 for query only
1262 ; Outputs:      Carry flag clear, if success
1263 ;             Carry flag set, if failure
1264 ;
1265 ;-----
1266
1267 ;Procedure FastInit,NEAR
1268 ; ASSUME CS:DOSCODE,SS:NOTHING
1269

```

```

11270 ; ; MSDOS 3.3
11271 ; ; IBMDOS.COM (1987) - Offset 1EB3h
11272 ;FastInit:
11273 ; mov di,FastTable ; FastOpenTable
11274 ; mov ax,[cs:di+4] ; Entry segment
11275 ; mov bx,cs ; get DOS segment
11276 ; cmp ax,bx ; first time installed ?
11277 ; je short ok_install ; yes
11278 ; stc ; set carry
11279 ; retn ; (cf=1 means) already installed !
11280
11281 ;ok_install:
11282 ; mov bx,FastTable ; FastOpenTable
11283 ; mov cx,ds
11284 ; ; set address of FASTXXX (FASTOPEN) routine entry
11285 ; mov [cs:bx+4],cx
11286 ; mov [cs:bx+2],si
11287 ; retn
11288
11289 ; 16/05/2019 - Retro DOS v4.0
11290
11291 ; 14/01/2024 - Retro DOS v5.0
11292 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:5773h
11293
11294 FastInit:
11295 ; MSDOS 6.0
11296 ;hkn; set up es to dosdataseg.
11297 00001848 06 push es
11298 ;getdseg <es> ; es -> dosdata
11299 00001849 2E8E06[0700] mov es,[cs:DosDseg]
11300
11301 ;hkn; FastTable is in DOSDATA
11302 0000184E BF[3E11] MOV DI,FastTable+2 ;AN000;FO. points to fastxxx entry
11303 00001851 4B DEC BX ;AN000;FO.;; decrement index
11304 00001852 89DA MOV DX,BX ;AN000;FO.;; save bx
11305 00001854 D1E3 SHL BX,1 ;AN000;FO.;; times 4, each entry is DWORD
11306 00001856 D1E3 SHL BX,1 ;AN000;FO.
11307 00001858 01DF ADD DI,BX ;AN000;FO. index to the entry
11308 0000185A 268B4502 MOV AX,[ES:DI+2] ;AN000;FO. get entry segment
11309 fcheck: ;AN000;
11310 MOV CX,CS ;AN000;FO.;; get DOS segment
11311 00001860 39C8 CMP AX,CX ;AN000;FO.;; first time installed ?
11312 00001862 7405 JZ short ok_install ;AN000;FO.;; yes
11313 00001864 09C0 OR AX,AX ;AN000;FO.;;
11314 ;JZ short ok_install ;AN000;FO.;;
11315 ;STC ;AN000;FO.;; already installed !
11316 ;JMP SHORT FSret ;AN000;FO. set carry
11317 ; 14/01/2024
11318 00001866 F9 stc
11319 00001867 7517 jnz short FSret
11320 ok_install: ;AN000;
11321 00001869 83FEFF CMP SI,-1 ;AN000;FO.;; Query only ?
11322 0000186C 7412 JZ short FSret ;AN000;FO.;; yes
11323 0000186E 8CD9 MOV CX,DS ;AN000;FO.;; get FASTXXX entry segment
11324 00001870 26894D02 MOV [ES:DI+2],CX ;AN000;FO.;; initialize routine entry
11325 00001874 268935 MOV [ES:DI],SI ;AN000;FO.;; initialize routine offset
11326
11327 ;hkn; FastFlg moved to DOSDATA
11328 00001877 BF[4611] MOV DI,FastFlg ;AN000;FO.;; get addr of FASTXXX flags
11329 0000187A 01D7 ADD DI,DX ;AN000;FO.;; index to a FASTXXX flag
11330 ;or byte [es:di],80h
11331 0000187C 26800D80 OR byte [ES:DI],Fast_yes ;AN000;FO.;; indicate installed
11332 FSret: ;AN000;
11333 00001880 07 pop es
11334 00001881 C3 retn ;AN000;FO.
11335
11336 ;EndProc FastInit
11337
11338 ;Break <FastRet - initial routine in FastOpenTable >
11339 -----
11340 ; DOS 3.3 6/10/86
11341 ; FastRet - indicate FASTXXXX not in memory
11342 ;
11343 ; Inputs: None
11344 ; Outputs: AX = -1 and carry flag set
11345 ;
11346 ; Registers modified: none.
11347 -----
11348
11349 FastRet:
11350 ;mov ax,-1
11351 ;stc
11352 ;retf
11353 00001882 F9 STC
11354 00001883 19C0 sbb ax,ax ; (ax) = -1, 'C' set
11355 00001885 CB RETF
11356
11357 ;Break <NLS_OPEN - do $open for NLSFUNC>
11358 -----
11359 ; DOS 3.3 6/10/86
11360 ; NLS_OPEN - call $OPEN for NLSFUNC
11361 ;
11362 ; Inputs: Same input as $OPEN except CL = mode
11363 ; Outputs: same output as $OPEN
11364 ;
11365 -----
11366
11367 ;hkn; NOTE! SS MUST HAVE BEEN SET UP TO DOSDATA BY THE TIME THESE
11368 ;hkn; NLS FUNCTIONS ARE CALLED!!! THERE FORE WE WILL USE SS OVERRIDES
11369 ;hkn; IN ORDER TO ACCESS DOS DATA VARIABLES!
11370
11371 NLS_OPEN:
11372 ; MOV BL,[CPSWFLAG] ; disable code page matching logic
11373 ; MOV BYTE [CPSWFLAG],0
11374 ; PUSH BX ; save current state
11375
11376 00001886 88C8 MOV AL,CL ; set up correct interface for $OPEN
11377 00001888 E83E67 call _$OPEN
11378
11379 ; POP BX ; restore current state
11380 ; MOV [CPSWFLAG],BL
11381
11382 0000188B C3 RETN
11383
11384 ;Break <NLS_LSEEK - do $LSEEK for NLSFUNC>
11385 -----
11386 ; DOS 3.3 6/10/86
11387 ; NLS_LSEEK - call $LSEEK for NLSFUNC
11388 ;
11389 ; Inputs: BP = open mode
11390 ; Outputs: same output as $LSEEK
11391 ;
11392 -----
11393

```

```

11394 ; 16/05/2019 - Retro DOS v4.0
11395
11396 NLS_LSEEK:
11397     PUSH    word [SS:USER_SP] ; save user stack
11398     PUSH    word [SS:USER_SS]
11399     CALL    Fake_User_Stack
11400     MOV     AX,BP ; set up correct interface for $LSEEK
11401     CALL    _$LSEEK
11402 NLS_SEEK_RET:
11403     ; 26/06/2024
11404     POP     word [SS:USER_SS] ; restore user stack
11405     POP     word [SS:USER_SP]
11406     RETN
11407
11408 ;Break    <Fake_User_Stack - save user stack>
11409 ;-----
11410 ;   DOS 3.3   6/10/86
11411 ;   Fake_User_Stack - save user stack pointer
11412 ;-----
11413
11414 Fake_User_Stack:
11415     MOV     AX,[SS:USER_SP_2F] ; replace with INT 2Fh stack
11416     MOV     [SS:USER_SP],AX
11417     MOV     AX,SS
11418     MOV     [SS:USER_SS],AX
11419     RETN
11420
11421 ;Break    <GetDevList - get device header list pointer>
11422 ;-----
11423 ;   DOS 3.3   7/25/86
11424 ;   GetDevList - get device header list pointer
11425 ;
11426 ;   Output: AX:BX points to the device header list
11427 ;-----
11428
11429 GetDevList:
11430     ; 16/05/2019 - Retro DOS v4.0
11431     MOV     SI,SysInitTable
11432     mov     ds,[cs:DosDSeg]
11433     LDS     SI,[SI]
11434     ;mov     ax,[si+34] ; SSYSINITVARS offset 34 = [SI+SYSI.DEV]
11435     MOV     AX,[SI+SYSI.DEV]
11436     ;mov     bx,[si+36] ; SSYSINITVARS offset 36 = [SI+SYSI.DEV+2]
11437     MOV     BX,[SI+SYSI.DEV+2]
11438     RETN
11439
11440 ;Break    <NLS_IOCTL - do $IOCTL for NLSFUNC>
11441 ;-----
11442 ;   DOS 3.3   7/25/86
11443 ;   NLS_IOCTL - call $IOCTL for NLSFUNC
11444 ;
11445 ;   Inputs: BP = function code 0CH
11446 ;   Outputs:      same output as generic $IOCTL
11447 ;-----
11448
11449 NLS_IOCTL:
11450     ; 16/05/2019 - Retro DOS v4.0
11451     PUSH    word [SS:USER_SP] ; save user stack
11452     PUSH    word [SS:USER_SS]
11453     CALL    Fake_User_Stack
11454     MOV     AX,BP ; set up correct interface for $IOCTL
11455     CALL    _$IOCTL
11456     ;POP     word [SS:USER_SS] ; restore user stack
11457     ;POP     word [SS:USER_SP]
11458     ;RETN
11459     ; 26/06/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
11460     jmp     short NLS_SEEK_RET
11461
11462 ;Break    <NLS_GETEXT- get extended error for NLSFUNC>
11463 ;-----
11464 ;   DOS 3.3   7/25/86
11465 ;   NLS_GETEXT -
11466 ;
11467 ;   Inputs: none
11468 ;   Outputs:      AX = extended error
11469 ;-----
11470
11471 NLS_GETEXT:
11472     ; 16/05/2019 - Retro DOS v4.0
11473     MOV     AX,[SS:EXTERR] ; return extended error
11474     ; 23/09/2023
11475 MSG_RETRIEVAL:
11476     RETN
11477
11478 ; 29/04/2019 - Retro DOS v4.0
11479
11480 ;Break    <MSG_RETRIEVAL- get beginning addr of system and parser messages>
11481 ;-----
11482 ;   DOS 4.00
11483 ;
11484 ;   Inputs: DL=0 get extended error message addr
11485 ;           =1 set extended error message addr
11486 ;           =2 get parser error message addr
11487 ;           =3 set parser error message addr
11488 ;           =4 get critical error message addr
11489 ;           =5 set critical error message addr
11490 ;           =6 get file system error message addr
11491 ;           =7 set file system error message addr
11492 ;           =8 get address for code reduction
11493 ;           =9 set address for code reduction
11494 ;
11495 ;   Function: get/set message address
11496 ;   Outputs:      ES:DI points to addr when get
11497 ;-----
11498
11499 ;Procedure MSG_RETRIEVAL,NEAR
11500 ;   ASSUME CS:DOSCODE,SS:NOTHING
11501
11502 ; 23/09/2023
11503 ;MSG_RETRIEVAL:
11504
11505 ;; NOTE: This function lives in command.com resident code now.
11506 ;; If the int 2F ever gets this far, we'll return registers
11507 ;; unchanged, which produces the same result as before, if
11508 ;; command.com wasn't present (and therefore no messages available).
11509 ;;
11510 ;;
11511 ;; I didn't point the entry in the 2F table to No_Op because
11512 ;; No_Op zeroes AL.
11513 ;;
11514 ;;
11515 ;;hkn; set up ds to point to DOSDATA
11516 ;; push    ds
11517 ;; getdseg <ds> ; ds -> dosdata

```

```

11518 ;
11519 ;
11520 ; PUSH AX ;AN000;;MS. save regs
11521 ; PUSH SI ;AN000;;MS. save regs
11522 ; MOV AX,DX ;AN000;;MS.
11523 ; MOV SI,OFFSET DOSDATA:MSG_EXTERROR ;AN000;;MS.
11524 ; test AL,1 ;AN000;;MS. get ?
11525 ; JZ toget ;AN000;;MS. yes
11526 ; DEC AL ;AN000;;MS.
11527 ; toget: ;AN000;;
11528 ; SHL AL,1 ;AN000;;MS. times 2
11529 ; XOR AH,AH ;AN000;;MS.
11530 ; ADD SI,AX ;AN000;;MS. position to the entry
11531 ; test DL,1 ;AN000;;MS. get ?
11532 ; JZ getget ;AN000;;MS. yes
11533 ; MOV WORD PTR DS:[SI],DI ;AN000;;MS. set MSG
11534 ; MOV WORD PTR DS:[SI+2],ES ;AN000;;MS. address to ES:DI
11535 ; JMP SHORT MSGret ;AN000;;MS. exit
11536 ; getget: ;AN000;;
11537 ; LES DI,DWORD PTR DS:[SI] ;AN000;;MS. get msg addr
11538 ; MSGret: ;AN000;;
11539 ; POP SI ;AN000;;MS.
11540 ; POP AX ;AN000;;MS.
11541 ;
11542 ; pop ds
11543 ;
11544 ; return ;AN000;;MS. exit
11545 ; 23/09/2023
11546 ; retn ; 29/04/2019
11547 ;
11548 ;=====
11549 ; ECritDisk, LCritDisk, ECritDevice, LCritDevice
11550 ; IBMDOS.COM (MSDOS 3.3), 1987 - Offset 1F36h
11551 ;=====
11552 ; 20/07/2018 - Retro DOS v3.0
11553 ;
11554 ; ; MSDOS 3.3
11555 ; ; 08/08/2018 - Retro DOS v3.0
11556 ; ECritMEM:
11557 ; ECritSFT:
11558 ;
11559 ; ECritDisk:
11560 ; retn
11561 ; ;push ax
11562 ;
11563 ; mov ax,8001h
11564 ; int 2Ah ; Microsoft Networks - BEGIN DOS CRITICAL SECTION
11565 ; ; AL = critical section number (00h-0Fh)
11566 ;
11567 ; pop ax
11568 ; retn
11569 ;
11570 ; ; MSDOS 3.3
11571 ; ; 08/08/2018 - Retro DOS v3.0
11572 ; LCritMEM:
11573 ; LCritSFT:
11574 ;
11575 ; LCritDisk:
11576 ; retn
11577 ; ;push ax
11578 ;
11579 ; mov ax,8101h
11580 ; int 2Ah ; Microsoft Networks - END DOS CRITICAL SECTION
11581 ; ; AL = critical section number (00h-0Fh)
11582 ;
11583 ; pop ax
11584 ; retn
11585 ; ECritDevice:
11586 ; retn
11587 ; ;push ax
11588 ;
11589 ; mov ax,8002h
11590 ; int 2Ah ; Microsoft Networks - BEGIN DOS CRITICAL SECTION
11591 ; ; AL = critical section number (00h-0Fh)
11592 ;
11593 ; pop ax
11594 ; retn
11595 ; LCritDevice:
11596 ; retn
11597 ; ;push ax
11598 ;
11599 ; mov ax,8102h
11600 ; int 2Ah ; Microsoft Networks - END DOS CRITICAL SECTION
11601 ; ; AL = critical section number (00h-0Fh)
11602 ;
11603 ; pop ax
11604 ; retn
11605 ;=====
11606 ; CRIT.ASM, MSDOS 6.0, 1991
11607 ;=====
11608 ; 12/05/2019 - Retro DOS v4.0
11609 ;
11610 ; Critical Section Routines
11611 ;
11612 ; MSDOS 6.21 - MSDOS.SYS - DOSCODE:513Ah
11613 ;
11614 ; 06/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
11615 ; DOSCODE:5126h (MSDOS 5.0 MSDOS.SYS)
11616 ;
11617 ; -----
11618 ; Each handler must leave everything untouched; including flags!
11619 ;
11620 ; Sleaze for time savings: first instruction is a return. This is patched
11621 ; by the sharer to be a PUSH AX to complete the correct routines.
11622 ; -----
11623 ; (DOSMAC.INC, MSDOS 6.0, 1991)
11624 ; -----
11625 ; Some old versions of the 80286 have a bug in the chip. The popf instruction
11626 ; will enable interrupts. Therefore in a section of code with interrupts
11627 ; disabled and you need a popf instruction use the 'popff' macro instead.
11628 ; -----
11629 ;
11630 ;%macro POPFF 0
11631 ; jmp $+3
11632 ; iret
11633 ; push cs
11634 ; call $-2
11635 ;%endmacro
11636 ;
11637 ; -----
11638 ;
11639 ; 14/01/2024 - Retro DOS v5.0
11640 ;%if 0
11641 ;

```



```

11642 ;Procedure ECritDisk,NEAR
11643 ;public ECritMEM
11644 ;public ECritSFT
11645 ECritMEM:
11646 ECritSFT:
11647 ;
11648 ECritDisk:
11649
11650 ;SR; Check if critical section is to be entered
11651
11652     pushf
11653     cmp     byte [ss:redir_patch],0
11654     jz      short ECritDisk_2
11655
11656 ; 06/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
11657 ; ;popff ; * (macro)
11658 ; jmp      short ECritDisk_1 ; *
11659 ;
11660 ;ECritDisk_iret: ; *
11661 ; iret ; *
11662
11663 ; 16/12/2022
11664 ; 13/11/2022
11665 ; jmp      short ECritDisk_1
11666 ; 06/11/2022
11667 ;ECritDisk_iret:
11668 ; iret
11669
11670 ; 06/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
11671 ECritDisk_1:
11672     push    cs ; *
11673     call    ECritDisk_iret ; *
11674
11675 ECritDisk_0:
11676     PUSH    AX
11677     ;MOV     AX,8000h+critDisk
11678     ;INT     int_IBM
11679     mov     ax,8001h
11680     int     2Ah ; Microsoft Networks - BEGIN DOS CRITICAL SECTION
11681             ; AL = critical section number (00h-0Fh)
11682     POP     AX
11683     retn
11684
11685 ; 16/12/2022
11686 ; 13/11/2022
11687 ECritDisk_iret: ; 12/05/2019 - Retro DOS v4.0
11688 LCritDisk_iret:
11689     iret
11690
11691 ECritDisk_2:
11692     ;popff ; *
11693     ;retn
11694 ; jmp      short ECritDisk_3 ; *
11695 ;ECritDisk_iret2: ; *
11696 ; iret
11697
11698 ; 16/12/2022
11699 ; 13/11/2022
11700 ; jmp      short ECritDisk_3
11701 ;ECritDisk_iret2:
11702 ; iret
11703
11704 ECritDisk_3:
11705     push    cs ; *
11706 ; 13/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
11707 ; call     ECritDisk_iret2 ; *
11708 ; retn
11709 ; 16/12/2022
11710 call     ECritDisk_iret
11711     retn
11712
11713 ;EndProc ECritDisk
11714
11715 ; -----
11716
11717 ;Procedure LCritDisk,NEAR
11718 ;public LCritMEM
11719 ;public LCritSFT
11720 LCritMEM:
11721 LCritSFT:
11722 ;
11723 LCritDisk:
11724
11725 ;SR; Check if critical section is to be entered
11726
11727     pushf
11728     cmp     byte [ss:redir_patch],0
11729     jz      short LCritDisk_2
11730 ;popff ; * (macro)
11731 ; jmp      short LCritDisk_1 ; *
11732 ;
11733 ;LCritDisk_iret: ; *
11734 ; iret ; *
11735
11736 ; 16/12/2022
11737 ; 13/11/2022
11738 ; jmp      short LCritDisk_1
11739 ;LCritDisk_iret:
11740 ; iret
11741
11742 LCritDisk_1:
11743     push    cs ; *
11744     call    LCritDisk_iret ; *
11745
11746 LCritDisk_0:
11747     PUSH    AX
11748     ;MOV     AX,8100h+critDisk
11749     ;INT     int_IBM
11750     mov     ax,8101h
11751     int     2Ah ; Microsoft Networks - END DOS CRITICAL SECTION
11752             ; AL = critical section number (00h-0Fh)
11753     POP     AX
11754     retn
11755
11756 ;LCritDisk_iret: ; 12/05/2019 - Retro DOS v4.0
11757 ; iret
11758
11759 LCritDisk_2:
11760     ;popff ; *
11761     ;retn
11762 ; jmp      short LCritDisk_3 ; *
11763 ;LCritDisk_iret2: ; *
11764 ; iret
11765

```

```

11766         ; 16/12/2022
11767         ; 13/11/2022
11768         ; jmp short LCritDisk_3
11769 ;LCritDisk_iret2:
11770         ; iret
11771
11772 LCritDisk_3:
11773     push    cs ; *
11774     ; 13/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
11775     ; call LCritDisk_iret2 ; *
11776     ; retn
11777     ; 16/12/2022
11778     call    LCritDisk_iret
11779     retn
11780
11781 ;EndProc LCritDisk
11782
11783 ; -----
11784
11785 ;Procedure ECritDevice,NEAR
11786
11787 ECritDevice:
11788
11789 ;SR; Check if critical section is to be entered
11790
11791     pushf
11792     cmp     byte [ss:redir_patch],0
11793     jz      short ECritDevice_2
11794     ; popff ; * (macro)
11795     ; jmp    short ECritDevice_1 ; *
11796
11797 ;ECritDevice_iret: ; *
11798 ; iret ; *
11799
11800     ; 16/12/2022
11801     ; 13/11/2022
11802     ; jmp    short ECritDevice_1
11803 ;ECritDevice_iret:
11804 ; iret
11805
11806 ECritDevice_1:
11807     push    cs ; *
11808     call    ECritDevice_iret ; *
11809
11810 ECritDevice_0:
11811     PUSH    AX
11812     ; MOV    AX,8000h+critDevice
11813     ; INT    int_IBM
11814     mov     ax,8002h
11815     int     2Ah ; Microsoft Networks - BEGIN DOS CRITICAL SECTION
11816             ; AL = critical section number (00h-0Fh)
11817     POP     AX
11818     retn
11819
11820     ; 16/12/2022
11821     ; 06/12/2022
11822 ECritDevice_iret: ; 12/05/2019 - Retro DOS v4.0
11823 LCritDevice_iret:
11824     iret
11825
11826 ECritDevice_2:
11827     ; popff ; *
11828     ; retn
11829     ; jmp    short ECritDevice_3 ; *
11830 ;ECritDevice_iret2: ; *
11831 ; iret
11832
11833     ; 16/12/2022
11834     ; 13/11/2022
11835     ; jmp    short ECritDevice_3
11836 ;ECritDevice_iret2:
11837 ; iret
11838
11839 ECritDevice_3:
11840     push    cs ; *
11841     ; 13/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
11842     ; call    ECritDevice_iret2 ; *
11843     ; retn
11844     ; 16/12/2022
11845     call    ECritDevice_iret
11846     retn
11847
11848 ;EndProc ECritDevice
11849
11850 ; -----
11851
11852 ;Procedure LCritDevice,NEAR
11853
11854 LCritDevice:
11855
11856 ;SR; Check if critical section is to be entered
11857
11858     pushf
11859     cmp     byte [ss:redir_patch],0
11860     jz      short LCritDevice_2
11861     ; popff ; * (macro)
11862     ; jmp    short LCritDevice_1 ; *
11863
11864 ;LCritDevice_iret: ; *
11865 ; iret ; *
11866
11867     ; 16/12/2022
11868     ; 13/11/2022
11869     ; jmp    short LCritDevice_1
11870 ;LCritDevice_iret:
11871 ; iret
11872
11873 LCritDevice_1:
11874     push    cs ; *
11875     call    LCritDevice_iret ; *
11876
11877 LCritDevice_0:
11878     PUSH    AX
11879     ; MOV    AX,8100h+critDevice
11880     ; INT    int_IBM
11881     mov     ax,8102h
11882     int     2Ah ; Microsoft Networks - END DOS CRITICAL SECTION
11883             ; AL = critical section number (00h-0Fh)
11884     POP     AX
11885     retn
11886
11887 ;LCritDevice_iret: ; 12/05/2019 - Retro DOS v4.0
11888 ; iret
11889

```



```

11890      LCRitDevice_2:
11891          ;;popff ; *
11892          ;;retn
11893      ; jmp short LCRitDevice_3 ; *
11894      ;LCritDevice_iret2: ; *
11895      ; iret
11896
11897          ; 16/12/2022
11898          ; 13/11/2022
11899          ; jmp short LCRitDevice_3
11900      ;LCritDevice_iret2:
11901      ;iret
11902
11903      LCRitDevice_3:
11904          push cs ; *
11905          ; 13/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
11906          ;call LCRitDevice_iret2 ; *
11907          ;retn
11908          ; 16/12/2022
11909          call LCRitDevice_iret
11910          retn
11911
11912      ;EndProc LCRitDevice
11913
11914      %endif
11915
11916          ; 15/01/2024 - Retro DOS v5.0 (Modified PC DOS 7.1)
11917          ; PC DOS 7.1 IBMDOS.COM - DOSCODE:580Dh
11918
11919      ; 15/01/2024 - Retro DOS v5.0
11920      %if 1
11921          ;;
11922          ; -----
11923      ECritMEM:
11924      ECritSFT:
11925      ; -----
11926          ; PC DOS 7.1 IBMDOS.COM
11927      ECritDisk:
11928          push cx
11929          mov ch,0
11930          mov cl,[ss:redir_patch]
11931          jcxz ECritDisk_3
11932          pop cx
11933          push ax
11934          mov ax,8001h ; BEGIN DOS CRITICAL SECTION
11935                      ; AL = critical section number (01h)
11936      ECritDisk_1:
11937      ; -----
11938      LCRitDisk_1:
11939      ECritDevice_1:
11940      LCRitDevice_1:
11941          push cx
11942          mov ch,0
11943          mov cl,[ss:Iswin386]
11944          jcxz ECritDisk_2
11945          pop cx
11946          push ax
11947          int 2Ah ; Microsoft Networks - BEGIN DOS CRITICAL SECTION
11948                      ; AL = critical section number (00h-0Fh)
11949          pop ax
11950          retn
11951
11952      ECritDisk_2:
11953          push es
11954          ;mov cx,0
11955          ; 15/01/2024
11956          ; cx = 0
11957          mov es,cx
11958          pushf ; simulate INT 2Ah
11959          call far [es:00A8h] ; call far (INT 2Ah vector)
11960          pop es
11961          pop cx
11962          pop ax
11963          retn
11964
11965      ECritDisk_3:
11966      ; -----
11967      LCRitDisk_3:
11968      ECritDevice_3:
11969      LCRitDevice_3:
11970          pop cx
11971          retn
11972
11973      ; -----
11974      LCRitMEM:
11975      LCRitSFT:
11976      ; -----
11977          ; PC DOS 7.1 IBMDOS.COM
11978      LCRitDisk:
11979          push cx
11980          mov ch,0
11981          mov cl,[ss:redir_patch]
11982          jcxz LCRitDisk_3
11983          pop cx
11984          push ax
11985          mov ax,8101h ; END DOS CRITICAL SECTION
11986                      ; AL = critical section number (01h)
11987          jmp short LCRitDisk_1
11988
11989      ; -----
11990      ECritDevice:
11991          push cx
11992          mov ch,0
11993          mov cl,[ss:redir_patch]
11994          jcxz ECritDevice_3
11995          pop cx
11996          push ax
11997          mov ax,8002h ; BEGIN DOS CRITICAL SECTION
11998                      ; AL = critical section number (02h)
11999          jmp short ECritDevice_1
12000
12001      ; -----
12002      LCRitDevice:
12003          push cx
12004          mov ch,0
12005          mov cl,[ss:redir_patch]
12006          jcxz LCRitDevice_3
12007          pop cx
12008          push ax
12009          mov ax,8102h ; END DOS CRITICAL SECTION
12010                      ; AL = critical section number (02h)
12011          jmp short LCRitDevice_1
12012
12013

```

```

12014 ; -----
12015 ;;;
12016 %endif
12017
12018 ;=====
12019 ; CPMIO.ASM, MSDOS 6.0, 1991
12020 ;=====
12021 ; 20/07/2018 - Retro DOS v3.0
12022 ;=====
12023 ;
12024 ; STDIO.ASM - (MSDOS 2.0)
12025 ;=====
12026 ;
12027 ;
12028 ; Standard device IO for MSDOS (first 12 function calls)
12029 ;
12030 ;
12031 ;.xlist
12032 ;.xcref
12033 ;INCLUDE STDSW.ASM
12034 ;INCLUDE DOSSEG.ASM
12035 ;.cref
12036 ;.list
12037
12038 ;TITLE   STDIO - device IO for MSDOS
12039 ;NAME     STDIO
12040
12041 ;INCLUDE IO.ASM
12042
12043 ; -----
12044 ;
12045 ; NOTE for Retro DOS v2.0 : (ERDOGAN TAN - 13/03/2018)
12046 ; IO.ASM is missing in MSDOS 2.0 kernel source code files !!!
12047 ; INSTEAD of IO.ASM, I have disassembled IBMDOS.COM (MSDOS 2.0)
12048 ; and I have used CPMIO.ASM (MSDOS 6.0 source code)
12049 ; to restore MSDOS 2.0 device IO source code
12050 ;
12051 ; (STRIN.ASM has '$STD_CON_STRING_INPUT' code.)
12052 ;
12053 ;=====
12054 ; STDIO.ASM - (MSDOS 2.0)
12055 ;=====
12056 ;
12057 ;
12058 ; Standard device IO for MSDOS (first 12 function calls)
12059 ;
12060 ;
12061 ;.xlist
12062 ;.xcref
12063 ;INCLUDE STDSW.ASM
12064 ;INCLUDE DOSSEG.ASM
12065 ;.cref
12066 ;.list
12067
12068 ;TITLE   STDIO - device IO for MSDOS
12069 ;NAME     STDIO
12070
12071 ;INCLUDE IO.ASM
12072
12073 ; -----
12074 ;
12075 ; NOTE for Retro DOS v2.0 : (ERDOGAN TAN - 13/03/2018)
12076 ; IO.ASM is missing in MSDOS 2.0 kernel source code files !!!
12077 ; INSTEAD of IO.ASM, I have disassembled IBMDOS.COM (MSDOS 2.0)
12078 ; and I have used CPMIO.ASM (MSDOS 6.0 source code)
12079 ; to restore MSDOS 2.0 device IO source code
12080 ;
12081 ; (STRIN.ASM has '$STD_CON_STRING_INPUT' code.)
12082 ;
12083 ;=====
12084 ; IO.ASM (MSDOS 2.0) (IBMDOS.COM 2.0) - STRIN.ASM (MSDOS 2.0, 19/08/1983)
12085 ;=====
12086 ; Retro DOS v2.0 by Erdogan Tan, 13/03/2018 - 14/03/2018
12087
12088 ; (Disassembled code of IBMDOS.COM, 08/03/1983) - Dissassembler: IDA Pro Free
12089 ; (Comments are from CPMIO.ASM - 1991, MSDOS 6.0)
12090
12091 ;=====
12092 ; CPMIO.ASM (MSDOS 6.0, 1991)
12093 ;=====
12094 ; Retro DOS v4.0 by Erdogan Tan, 04/05/2019
12095
12096 ; 08/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
12097
12098 ;** Standard device IO for MSDOS (first 12 function calls)
12099 ;
12100 ; TITLE   IBMCPMIO - device IO for MSDOS
12101 ; NAME     IBMCPMIO
12102
12103 ; Old style CP/M 1-12 system calls to talk to reserved devices
12104 ;
12105 ; $Std_Con_Input_No_Echo
12106 ; $Std_Con_String_Output
12107 ; $Std_Con_String_Input
12108 ; $RawConIO
12109 ; $RawConInput
12110 ; RAWOUT
12111 ; RAWOUT2
12112 ;
12113 ;
12114 ; The following routines form the console I/O group (funcs 1,2,6,7,8,9,10,11).
12115 ; They assume ES and DS NOTHING, while not strictly correct, this forces data
12116 ; references to be SS or CS relative which is desired.
12117 ;
12118 ; -----
12119 ;
12120 ; TITLE   CPMIO2 - device IO for MSDOS
12121 ; NAME     CPMIO2
12122 ;
12123 ;
12124 ; Microsoft Confidential
12125 ; Copyright (C) Microsoft Corporation 1991
12126 ; All Rights Reserved.
12127 ;
12128 ;
12129 ;** Old style CP/M 1-12 system calls to talk to reserved devices
12130 ;
12131 ; $Std_Con_Input
12132 ; $Std_Con_Output
12133 ; OUTT
12134 ; TAB
12135 ; BUFOUT
12136 ; $Std_Aux_Input
12137 ; $Std_Aux_Output

```

```

12138 ; $Std_Printer_Output
12139 ; $Std_Con_Input_Status
12140 ; $Std_Con_Input_Flush
12141 ;
12142 ; Revision History:
12143 ;
12144 ; AN000 version 4.00 - Jan. 1988
12145 ;
12146 ; The following routines form the console I/O group (funcs 1,2,6,7,8,9,10,11).
12147 ; They assume ES and DS NOTHING, while not strictly correct, this forces data
12148 ; references to be SS or CS relative which is desired.
12149 ;
12150 ;DOSCODE SEGMENT
12151 ; ASSUME SS:DOSDATA,CS:DOSCODE
12152 ;
12153 ;
12154 ;hkn; All the variables use SS override or DS. Therefore there is
12155 ;hkn; no need to specifically set up any seg regs unless SS assumption is
12156 ;hkn; not valid.
12157 ;
12158 ; DOSCODE:51BAh (MSDOS 6.21, MSDOS.SYS)
12159 ; 08/11/2022
12160 ; DOSCODE:51A6h (MSDOS 5.0, MSDOS.SYS)
12161 ;
12162 ;
12163 ;-----
12164 ; Procedure : $Std_Con_Input_No_Echo
12165 ;
12166 ;-----
12167 ;
12168 ;
12169 ;
12170 _$STD_CON_INPUT_NO_ECHO: ;system call 8
12171 ;
12172 ; Inputs:
12173 ; None
12174 ; Function:
12175 ; Input character from console, no echo
12176 ; Returns:
12177 ; AL = character
12178 ;
12179 00001942 1E push ds
12180 00001943 56 push si
12181 ;
12182 00001944 E86B45 INTEST: call STATCHK
12183 00001947 753A jnz short GET ; 08/09/2018
12184 ;*****
12185 ;hkn; SS override
12186 00001949 36803E[A00A]00 cmp byte [SS:PRINTER_FLAG],0 ; is printer idle?
12187 0000194F 7505 jnz short no_sys_wait
12188 00001951 B405 mov ah,5 ; get input status with system wait
12189 00001953 E87237 call IOFUNC
12190 no_sys_wait:
12191 ;*****
12192 00001956 B484 MOV AH,84h ; (Microsoft Networks - KEYBOARD BUSY LOOP)
12193 00001958 CD2A INT int_IBM ; int 2Ah
12194 ;
12195 ;;; 7/15/86 update the date in the idle loop
12196 ;;; Dec 19, 1986 D.C.L. changed following CMP to Byte Ptr from Word Ptr
12197 ;;; to shorten loop in consideration of the PC Convertible
12198 ;
12199 ;hkn; SS override
12200 0000195A 36803E[BD0D]FF CMP byte [SS:DATE_FLAG],-1; date is updated may be every
12201 00001960 751A JNZ short NoUpdate ; 65535 x ? ms if no one calls
12202 ;
12203 00001962 50 PUSH AX
12204 00001963 53 PUSH BX ; following is tricky,
12205 00001964 51 PUSH CX ; it may be called by critical handler
12206 00001965 52 PUSH DX ; at that time, DEVCALL is used by
12207 ; other's READ or WRITE
12208 00001966 1E PUSH DS ; save DS = SFT's segment
12209 ;
12210 ;hkn; READTIME must use ds = DOSDATA
12211 ;hkn; PUSH CS ; READTIME must use DS=CS
12212 ;
12213 00001967 16 PUSH SS ; 04/05/2019
12214 00001968 1F POP DS
12215 ;
12216 ;MOV AX,0 ; therefore, we save DEVCALL
12217 ; 26/06/2024
12218 00001969 31C0 xor ax,ax
12219 0000196B E89A02 CALL Save_Restore_Packet ; save DEVCALL packet
12220 ;invoke READTIME ; readtime
12221 0000196E E819F2 call READTIME
12222 00001971 B80100 MOV AX,1
12223 00001974 E89102 CALL Save_Restore_Packet ; restore DEVCALL packet
12224 ;
12225 ; ; MSDOS 3.3 (IBMDOS.COM, Offset 1F8Ch)
12226 ; ; (MSDOS 6.0 code does not contain IBM DOS FETCHI_TAG check)
12227 ; push bx
12228 ; mov bx,DATE_FLAG
12229 ; add bx,2 ; mov bx,FETCHI_FLAG
12230 ; cmp word [cs:bx],5872h
12231 ; jz short FETCHI_TAG_chk_ok
12232 ; call DOSINIT
12233 ;FETCHI_TAG_chk_ok:
12234 ; pop bx
12235 ;
12236 00001977 1F POP DS ; restore DS
12237 00001978 5A POP DX
12238 00001979 59 POP CX
12239 0000197A 58 POP BX
12240 0000197B 58 POP AX
12241 NoUpdate:
12242 ;
12243 ;hkn; SS override
12244 0000197C 36FF06[BD0D] INC word [SS:DATE_FLAG]
12245 ;
12246 ;;; 7/15/86 ;;;
12247 00001981 EBC1 JMP short INTEST
12248 GET:
12249 00001983 30E4 XOR AH,AH
12250 00001985 E84037 call IOFUNC
12251 00001988 5E POP SI
12252 00001989 1F POP DS
12253 ;;; 7/15/86
12254 ;hkn; SS override
12255 ; MSDOS 6.0
12256 0000198A 36C606[BC0D]00 MOV BYTE [SS:SCAN_FLAG],0
12257 ;
12258 ;
12259 00001990 3C00 CMP AL,0 ; extended code ( AL )
12260 00001992 7505 JNZ short noscan
12261 ;

```

```

12262 ;hkn; SS override
12263 ;MOV BYTE [SS:SCAN_FLAG],1 ; set this flag for ALT_Q key
12264 ; 20/06/2023
12265 00001994 36FE06[BC0D] ; inc byte [SS:SCAN_FLAG]
12266 noscan:
12267 00001999 C3 retn
12268 ;
12269 ;-----
12270 ;
12271 ;** $STD_CON_STRING_OUTPUT - Console String Output
12272 ;
12273 ;
12274 ; ENTRY (DS:DX) Point to output string '$' terminated
12275 ; EXIT none
12276 ; USES ALL
12277 ;
12278 ;-----
12279 ;
12280
12281 _$STD_CON_STRING_OUTPUT: ;System call 9
12282 0000199A 89D6 mov si,dx
12283 STRING_OUT1:
12284 0000199C AC lodsb
12285 0000199D 3C24 cmp al,'$'
12286 0000199F 74F8 je short noscan
12287 NEXT_STR1:
12288 000019A1 E88D02 call OUTT
12289 000019A4 EBF6 jmp short STRING_OUT1
12290 ;
12291 ;-----
12292 ;
12293 ;** $STD_CON_STRING_INPUT - Input Line from Console
12294 ;
12295 ; $STD_CON_STRING_INPUT Fills a buffer from console input until CR
12296 ;
12297 ; ENTRY (ds:dx) = input buffer
12298 ; EXIT none
12299 ; USES ALL
12300 ;
12301 ;-----
12302 ;
12303 ; 15/01/2024 - Retro DOS v5.0
12304 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:58D4h
12305
12306 _$STD_CON_STRING_INPUT: ;System call 10
12307 ;
12308 ; 15/01/2024
12309 ;mov ax,ss
12310 ;mov es,ax
12311 000019A6 16 push ss
12312 000019A7 07 pop es
12313 ;
12314 000019A8 89D6 mov si,dx
12315 000019AA 30ED xor ch,ch
12316 000019AC AD lodsw
12317 ;
12318 ; (AL) = the buffer length
12319 ; (AH) = the template length
12320 ;
12321 000019AD 08C0 or al,al
12322 000019AF 74E8 jz short noscan ;Buffer is 0 length!!?
12323 000019B1 88E3 mov bl,ah ;Init template counter
12324 000019B3 88EF mov bh,ch ;Init template counter
12325 ;
12326 ; (BL) = the number of bytes in the template
12327 ;
12328 000019B5 38D8 cmp al,bl
12329 000019B7 7605 jbe short NOEDIT ;If length of buffer inconsistent with contents
12330 000019B9 80380D cmp byte [bx+si],c_CR ; 0dh
12331 000019BC 7402 jz short EDITON ;If CR correctly placed EDIT is OK
12332 ;
12333 ; The number of chars in the template is >= the number of chars in buffer or
12334 ; there is no CR at the end of the template. This is an inconsistent state
12335 ; of affairs. Pretend that the template was empty:
12336 ;
12337 ;
12338 NOEDIT:
12339 000019BE 88EB mov bl,ch ;Reset buffer
12340 EDITON:
12341 000019C0 88C2 mov dl,al
12342 000019C2 4A dec dx ;DL is # of bytes we can put in the buffer
12343 ;
12344 ; Top level. We begin to read a line in.
12345 ;
12346 NEWLIN:
12347 000019C3 36A0[F901] mov al,[SS:CARPOS]
12348 000019C7 36A2[FA01] mov [SS:STARTPOS],al ;Remember position in raw buffer
12349 ;
12350 000019CB 56 push si
12351 000019CC BF[FB01] mov di,INBUF ;Build the new line here
12352 000019CF 36882E[7905] mov byte [SS:INSMODE],ch ;Insert mode off
12353 000019D4 88EF mov bh,ch ;No chars from template yet
12354 000019D6 88EE mov dh,ch ;No chars to new line yet
12355 000019D8 E867FF call _$STD_CON_INPUT_NO_ECHO ;Get first char
12356 000019DB 3C0A cmp al,c_LF ; 0Ah ;Linefeed
12357 000019DD 7503 jnz short GOTCH
12358 ;
12359 ; This is the main loop of reading in a character and processing it.
12360 ;
12361 ; (BH) = the index of the next byte in the template
12362 ; (BL) = the length of the template
12363 ; (DH) = the number of bytes in the buffer
12364 ; (DL) = the length of the buffer
12365 ;
12366 GETCH:
12367 000019DF E860FF call _$STD_CON_INPUT_NO_ECHO
12368 GOTCH:
12369 ;
12370 ; Brain-damaged Tim Patterson ignored ^F in case his BIOS did not flush the
12371 ; input queue.
12372 ;
12373 000019E2 3C06 cmp al,"F"-"@" ; CMP AL, 6 ; Ignore ^F
12374 000019E4 74F9 jz short GETCH
12375 ;
12376 ; If the leading char is the function-key lead byte
12377 ;
12378 ;cmp al,[SS:ESCCHAR]
12379 ;
12380 ; 04/05/2019 - Retro DOS v4.0
12381 ;
12382 ;hkn; ESCCHAR is in TABLE seg (DOSCODE)
12383 ;
12384 000019E6 2E3A06[940A] CMP AL,[cs:ESCCHAR]
12385 000019EB 7439 jz short ESCAPE ;change reserved keyword DBM 5-7-87

```

```

12386
12387
12388 ; Rubout and ^H are both destructive backspaces.
12389 000019ED 3C7F      cmp al,c_DEL ; 7FH
12390                  ;jz short BACKSPJ
12391                  ; 15/01/2024
12392 000019EF 7466      je      short BACKSP
12393 000019F1 3C08      cmp     al,c_BS ; 8
12394                  ;jz short BACKSPJ
12395                  ; 15/01/2024
12396 000019F3 7462      je      short BACKSP
12397
12398 ; 04/05/2019 - MSDOS 6.0, also MSDOS 6.21 has bug (bullshit) here.
12399 ; Two NOPs -instead of a JMP short, as two bytes-
12400 ; after CMP and a CMP again!
12401 ;
12402 ;
12403 ; -It would be better if they use a 'JMP short' to
12404 ; DOSCODE:5279h from DOSCODE:5271h and leave NOPs
12405 ; between them. Then, they would be able use a patch
12406 ; between 5271h and 5279h when if it will be required.
12407 ; I think Tim Patterson would not do this CMP mistake!-
12408 ;
12409 ; (MSDOS.SYS, from DOSCODE:5271h to DOSCODE:5279h)
12410 ;
12411 ; 08/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
12412 ;
12413 ; (Note: nops below might be used for patching code for windows 3.1)
12414 ;DOSCODE:526D      cmp     al, 8
12415 ;DOSCODE:526F      jz      short BACKSPJ
12416 ;DOSCODE:5271      cmp     al, 17h
12417 ;DOSCODE:5273      nop
12418 ;DOSCODE:5274      nop
12419 ;DOSCODE:5275      cmp     al, 15h
12420 ;DOSCODE:5277      nop
12421 ;DOSCODE:5278      nop
12422 ;DOSCODE:5279      cmp     al, 0Dh
12423 ;DOSCODE:527B      jz      short ENDLIN
12424 ;DOSCODE:527D      cmp     al, 0Ah
12425 ;DOSCODE:527F      jz      short PHYCRLF
12426
12427 ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
12428 ; DOSCODE:525Dh
12429
12430 ; 16/12/2022
12431 %if 0
12432 ; MSDOS 6.0
12433 ; ^W deletes backward once and then backs up until a letter is before the
12434 ; cursor
12435
12436 CMP     AL,"w"-"@" ; 17h
12437
12438 ; The removal of the comment characters before the jump statement will
12439 ; cause ^W to backup a word.
12440
12441 ;***JZ      short wordDel
12442 NOP
12443 NOP
12444
12445 CMP     AL,"u"-"@" ; 15h
12446
12447 ; The removal of the comment characters before the jump statement will
12448 ; cause ^U to clear a line.
12449
12450 ;***JZ      short LineDel
12451 NOP
12452 NOP
12453
12454 %endif
12455
12456 ; CR terminates the line.
12457
12458 000019F5 3C0D      cmp al,c_CR ; 0Dh
12459 000019F7 7430      jz      short ENDLIN
12460
12461 ; LF goes to a new line and keeps on reading.
12462
12463 000019F9 3C0A      cmp al,c_LF ; 0Ah
12464 000019FB 7442      jz      short PHYCRLF
12465
12466 ; ^X (or ESC) deletes the line and starts over
12467
12468 ; MSDOS 3.3
12469 ;cmp     al,[ss:CANCHAR] ; 1Bh
12470 ;jz      short KILNEW
12471
12472 ; MSDOS 6.0 (& MSDOS 6.21)
12473
12474 ;hkn;      CANCHAR is in TABLE seg (DOSCODE), so CS override
12475
12476 000019FD 2E3A06[930A] cmp al,[cs:CANCHAR] ; 1Bh
12477 00001A02 7440      jz      short KILNEW
12478
12479 ;cmp     al,CANCEL ; 1Bh ; Retro DOS v3.0
12480 ;jz      short KILNEW
12481
12482 ; Otherwise, we save the input character.
12483
12484 SAVCH:
12485 00001A04 38D6      cmp     dh,dI
12486 00001A06 7317      jnb     short BUFFUL ; buffer is full.
12487 00001A08 AA      stosb
12488 00001A09 FEC6      inc     dh ; increment count in buffer.
12489 00001A0B E8B702      call    BUFOUT ; Print control chars nicely
12490
12491 00001A0E 36803E[7905]00 cmp byte [SS:INSMODE], 0
12492 00001A14 75C9      jnz     short GETCH ; insertmode => don't advance template
12493 00001A16 38DF      cmp     bh,bI
12494 00001A18 73C5      jnb     short GETCH ; no more characters in template
12495 00001A1A 46      inc     si ; skip to next char in template
12496 00001A1B FEC7      inc     bh ; remember position in template
12497 00001A1D EBC0      jmp     short GETCH
12498
12499 ; 15/01/2024
12500 ;BACKSPJ:
12501 ;jmp     short BACKSP
12502
12503 BUFFUL:
12504 00001A1F B007      mov     al, 7 ; Bell to signal full buffer
12505 00001A21 E80D02      call    OUTT
12506 00001A24 EBB9      jmp     short GETCH
12507
12508 ESCAPE:
12509 ;transfer OEMFunctionKey

```

```

12510 00001A26 E996F0      JMP     OEMFunctionKey      ; let the OEM's handle the key dispatch
12511
12512      ENDLIN:
12513 00001A29 AA          stosb                      ; Put the CR in the buffer
12514 00001A2A E80402      call    OUTT                      ; Echo it
12515 00001A2D 5F          pop di                      ; Get start of user buffer
12516 00001A2E 8875FF      mov [di-1], dh          ; Tell user how many bytes
12517 00001A31 FEC6          inc dh                      ; DH is length including CR
12518
12519      COPYNEW:
12520          ; (IBMDOS.COM, MSDOS 2.0, STRIN.ASM)
12521          ;mov bp, es
12522          ;mov bx, ds
12523          ;mov es, bx
12524          ;mov ds, bp
12525          ;mov si, INBUF
12526          ;mov cl, dh
12527          ;rep movsb
12528          ;retn
12529
12530          ; CPMIO.ASM (MSDOS 6.0)
12531          ; (IBMDOS.COM, MSDOS 3.3, Offset 2061h)
12532          ;SAVE <DS,ES>
12533 00001A33 1E          PUSH    DS
12534 00001A34 06          PUSH    ES
12535          ;RESTORE <DS,ES>          ; XCHG ES,DS
12536 00001A35 1F          POP     DS
12537 00001A36 07          POP     ES
12538
12539      ;;hkn; INBUF is in DOSDATA
12540 00001A37 BE[FB01]      MOV     SI,INBUF
12541 00001A3A 88F1          MOV     CL,DH          ; set up count
12542 00001A3C F3A4          REP     MOVSB          ; Copy final line to user buffer
12543
12544 00001A3E C3          OLDBAK_RETN: RETN
12545
12546      ; Output a CRLF to the user screen and do NOT store it into the buffer
12547
12548      PHYCRLF:
12549          CALL    CRLF
12550 00001A42 EB9B          JMP short GETCH
12551
12552          ; MSDOS 6.0 (& MSDOS 3.3, IBMDOS.COM, 1987)
12553
12554      ; DOSCODE:52CAh (MSDOS 621, MSDOS.SYS)
12555
12556          ; Note: Following routines were not used in IBMDOS.COM
12557          ; -CTRL+W, CTRL+U is not activated-
12558          ; but they were in the kernel code!?)
12559
12560          ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
12561          ; DOSCODE:52B6h
12562
12563      ;;;;;;
12564
12565      ; 16/12/2022
12566      %if 0
12567      ;
12568      ; Delete the previous line
12569      ;
12570      LineDel:
12571          OR      DH,DH
12572          JZ      short GETCH          ; 06/12/2022
12573          Call    BackSpace
12574          JMP     short LineDel
12575
12576      %endif
12577
12578      ;
12579      ; delete the previous word.
12580      ;
12581      WordDel:
12582      wordLoop:
12583          ; Call    BackSpace          ; backspace the one spot
12584          ; OR      DH,DH
12585          ; JZ      short GetChj
12586          ; MOV     AL,[ES:DI-1]
12587          ; cmp     al,'0'
12588          ; jb      short GetChj
12589          ; cmp     al,'9'
12590          ; jbe     short wordLoop
12591          ; OR      AL,20h
12592          ; CMP     AL,'a'
12593          ; JB      short GetChj
12594          ; CMP     AL,'z'
12595          ; JBE     short wordLoop
12596      ;GetChj:
12597          ; JMP     GETCH
12598
12599      ; 16/12/2022
12600      %if 0
12601          ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
12602          ; (worddel is not called or jumped from anywhere!)
12603      wordDel:
12604      wordLoop:
12605          Call    BackSpace          ; backspace the one spot
12606          ; OR      DH,DH
12607          ; JZ      short GetChj
12608          ; MOV     AL,[ES:DI-1]
12609          ; cmp     al,'0'
12610          ; jb      short GetChj
12611          ; cmp     al,'9'
12612          ; jbe     short wordLoop
12613          ; OR      AL,20h
12614          ; CMP     AL,'a'
12615          ; JB      short GetChj
12616          ; CMP     AL,'z'
12617          ; JBE     short wordLoop
12618      GetChj:
12619          JMP     GETCH
12620
12621      %endif
12622
12623      ;;;;;;
12624
12625      ; DOSCODE:52F3h (MSDOS 621, MSDOS.SYS)
12626
12627      ; The user wants to throw away what he's typed in and wants to start over.
12628      ; We print the backslash and then go to the next line and tab to the correct
12629      ; spot to begin the buffered input.
12630
12631      KILNEW:
12632          mov al,'\'
12633          call    OUTT          ;Print the CANCEL indicator

```



```

12634 00001A49 5E                pop si                ;Remember start of edit buffer
12635
12636 00001A4A E81001          PUTNEW:
12637 00001A4D 36A0[FA01]      call    CRLF          ;Go to next line on screen
12638 00001A51 E85102          mov     al,[SS:STARTPOS]
12639 00001A54 E96CFF          call    TAB           ;Tab over
12640                                JMP      NEWLIN        ;Start over again
12641
12642                                ; Destructively back up one character position
12643
12644                                BACKSP:
12645 00001A57 E80800          ; 09/09/2018
12646 00001A5A EB83          call    BackSpace
12647                                JMP      short GETCH   ; 15/01/2024
12648
12649                                ; 15/01/2024
12650                                ;User really wants an ESC character in his line
12651 00001A5C 2EA0[940A]      TWOESC:
12652 00001A60 EBA2          mov     al,[cs:ESCCHAR] ; 10/06/2019
12653                                jmp      short SAVCH
12654
12655 00001A62 08F6          BackSpace:
12656 00001A64 7419          or      dh,dh
12657 00001A66 E85800          jz      short OLDBAK   ;No chars in line, do nothing to line
12658 00001A69 268A05        call    BACKUP          ;Do the backup
12659 00001A6C 3C20          mov     al,[es:di]     ;Get the deleted char
12660 00001A6E 730F          cmp     al,20h ; ' '
12661 00001A70 3C09          jnb     short OLDBAK   ;was a normal char
12662 00001A72 741B          cmp     al,c_HT ; 9
12663                                jz      short BAKTAB     ;was a tab, fix up users display
12664 00001A74 3C15          ;; 9/27/86 fix for ctrl-U backspace
12665 00001A76 7407          CMP     AL,"U"-"@" ; 15h ; ctrl-U is a section symbol not ^U
12666 00001A78 3C14          JZ      short OLDBAK   ;
12667 00001A7A 7403          CMP     AL,"T"-"@" ; 14h ; ctrl-T is a paragraphs symbol not ^T
12668                                JZ      short OLDBAK
12669 00001A7C E84500          ;; 9/27/86 fix for ctrl-U backspace
12670                                call    BACKMES        ;was a control char, zap the '^'
12671 00001A7F 36803E[7905]00  OLDBAK:
12672 00001A85 75B7          cmp     byte [SS:INSMODE], 0
12673 00001A87 08FF          jnz     short OLDBAK_RETN ;In insert mode, done
12674 00001A89 74B3          or      bh,bh
12675                                jz      short OLDBAK_RETN
12676 00001A8B FECF          dec     bh             ;Not advanced in template, stay where we are
12677 00001A8D 4E          dec     si             ;Go back in template
12678 00001A8E C3          retn
12679
12680 00001A8F 57          BAKTAB:
12681 00001A90 4F          push     di
12682 00001A91 FD          dec     di             ;Back up one char
12683 00001A92 88F1          std     di             ;Go backward
12684 00001A94 B020          mov     cl,dh          ;Number of chars currently in line
12685 00001A96 53          mov     al,20h ; ' '
12686 00001A97 B307          push     bx
12687 00001A99 E30E          mov     bl,7           ;Max
12688                                jcxz     FIGTAB         ;At start, do nothing
12689 00001A9B AE          FNDPOS:
12690 00001A9C 7609          scasb                    ;Look back
12691 00001A9E 26807D0109      jbe     short CHKCNT    ;
12692 00001AA3 7409          cmp     byte [es:di+1],9
12693 00001AA5 FECB          jz      short HAVTAB    ;Found a tab
12694                                dec     bl             ;Back one char if non tab control char
12695 00001AA7 E2F2          CHKCNT:
12696                                loop     FNDPOS
12697 00001AA9 362A1E[FA01]      FIGTAB:
12698                                sub     bl,[SS:STARTPOS]
12699 00001AAE 28F3          HAVTAB:
12700 00001AB0 00D9          sub     bl,dh
12701 00001AB2 80E107        add     cl,bl
12702 00001AB5 FC          and     cl,7           ;CX has correct number to erase
12703 00001AB6 5B          cld                    ;Back to normal
12704 00001AB7 5F          pop     bx
12705 00001AB8 74C5          pop     di
12706                                jz      short OLDBAK    ;Nothing to erase
12707 00001ABA E80700          TABBAK:
12708 00001ABD E2FB          call    BACKMES
12709 00001ABF EBBE          loop   TABBAK         ;Erase correct number of chars
12710                                jmp     short OLDBAK
12711
12712 00001AC1 FECE          BACKUP:
12713 00001AC3 4F          dec     dh             ;Back up in line
12714                                dec     di
12715 00001AC4 B008          BACKMES:
12716 00001AC6 E86801        mov     al,c_BS ; 8     ;Backspace
12717 00001AC9 B020          call    OUTT
12718 00001ACB E86301        mov     al,20h ; ' '    ;Erase
12719 00001ACE B008          call    OUTT
12720 00001AD0 E95E01        mov     al,c_BS ; 8     ;Backspace
12721                                jmp     OUTT            ;Done
12722                                ; 15/01/2024
12723                                ;User really wants an ESC character in his line
12724                                ;TWOESC:
12725                                ; mov     al,[cs:ESCCHAR] ; 10/06/2019
12726                                ; jmp     SAVCH
12727
12728                                ;Copy the rest of the template
12729                                COPYLIN:
12730 00001AD3 88D9          mov     cl,bl          ;Total size of template
12731 00001AD5 28F9          sub     cl,bh          ;Minus position in template, is number to move
12732 00001AD7 EB07          jmp     short COPYEACH
12733
12734                                COPYSTR:
12735 00001AD9 E83200          call    FINDOLD         ;Find the char
12736 00001ADC EB02          jmp     short COPYEACH  ;Copy up to it
12737
12738                                ;Copy one char from template to line
12739                                COPYONE:
12740 00001ADE B101          mov     cl,1
12741                                ;Copy CX chars from template to line
12742                                COPYEACH:
12743 00001AE0 36C606[7905]00  mov     byte [SS:INSMODE],0 ;All copies turn off insert mode
12744 00001AE6 38D6          cmp     dh,dh
12745 00001AE8 740F          jz      short GETCH2    ;At end of line, can't do anything
12746 00001AEA 38DF          cmp     bh,bl
12747 00001AEC 740B          jz      short GETCH2    ;At end of template, can't do anything
12748 00001AEE AC          lodsb
12749 00001AEF AA          stosb
12750 00001AF0 E8D201        call    BUFOUT
12751 00001AF3 FEC7          inc     bh             ;Ahead in template
12752 00001AF5 FEC6          inc     dh             ;Ahead in line
12753 00001AF7 E2E7          loop   COPYEACH
12754
12755 00001AF9 E9E3FE          GETCH2:
12756                                jmp     GETCH
12757                                ;Skip one char in template

```

```

12758
12759 00001AFC 38DF
12760 00001AFE 74F9
12761 00001B00 FEC7
12762 00001B02 46
12763
12764
12765 00001B03 EBF4
12766
12767
12768 00001B05 E80600
12769 00001B08 01CE
12770 00001B0A 00CF
12771
12772
12773 00001B0C EBEB
12774
12775
12776
12777
12778
12779
12780
12781
12782
12783 00001B0E E831FE
12784
12785
12786
12787
12788
12789
12790
12791
12792
12793
12794 00001B11 2E3A06[940A]
12795 00001B16 7505
12796
12797 00001B18 E827FE
12798 00001B1B EB1D
12799
12800 00001B1D 88D9
12801 00001B1F 28F9
12802 00001B21 7417
12803 00001B23 49
12804 00001B24 7414
12805 00001B26 06
12806 00001B27 1E
12807 00001B28 07
12808 00001B29 57
12809 00001B2A 89F7
12810 00001B2C 47
12811 00001B2D F2AE
12812 00001B2F 5F
12813 00001B30 07
12814 00001B31 7507
12815 00001B33 F6D1
12816 00001B35 00D9
12817 00001B37 28F9
12818
12819 00001B39 C3
12820
12821
12822 00001B3A 5D
12823
12824
12825
12826 00001B3B EBBC
12827
12828
12829 00001B3D B040
12830 00001B3F E8EF00
12831 00001B42 5F
12832 00001B43 57
12833 00001B44 06
12834 00001B45 1E
12835 00001B46 E8EAFE
12836 00001B49 1F
12837 00001B4A 07
12838 00001B4B 5E
12839 00001B4C 88F3
12840 00001B4E E9F9FE
12841
12842
12843
12844 00001B51 36F616[7905]
12845
12846
12847 00001B56 EBE3
12848
12849
12850
12851 00001B58 B01A
12852 00001B5A E9A7FE
12853
12854
12855
12856 00001B5D B00D
12857 00001B5F E8CF00
12858 00001B62 B00A
12859 00001B64 E9CA00
12860
12861
12862
12863
12864
12865
12866
12867
12868
12869
12870
12871
12872
12873
12874
12875
12876
12877
12878
12879
12880
12881

```

```

SKIPONE:
    cmp     bh,b1
    jz      short GETCH2          ;At end of template
    inc     bh                    ;Ahead in template
    inc     si
    ; jmp    GETCH
    ; 15/01/2024
    jmp     short GETCH2

SKIPSTR:
    call    FINDOLD              ;Find out how far to go
    add     si,cx                 ;Go there
    add     bh,c1
    ; jmp    GETCH
    ; 15/01/2024
    jmp     short GETCH2

;Get the next user char, and look ahead in template for a match
;CX indicates how many chars to skip to get there on output
;NOTE: WARNING: If the operation cannot be done, the return
; address is popped off and a jump to GETCH is taken.
; Make sure nothing extra on stack when this routine
; is called!!! (no PUSHes before calling it).

FINDOLD:
    call    _$STD_CON_INPUT_NO_ECHO
    ; STRIN.ASM (MSDOS 2.11, 19/07/2018)

    ;CMP     AL,[SS:ESCCHAR]
    ;JNZ     SHORT FINDSETUP

    ; CPMIO.ASM (MSDOS 6.0, 04/05/2019 - Retro DOS v4.0)

;hkn; ESCCHAR is in TABLE seg (DOSCODE), so CS override
    CMP     AL,[CS:ESCCHAR]      ; did he type a function key?
    JNZ     SHORT FINDSETUP      ; no, set up for scan

    CALL     _$STD_CON_INPUT_NO_ECHO ; eat next char
    JMP     SHORT NOTFND         ; go try again

FINDSETUP:
    mov     cl,b1
    sub     cl,bh                ;CX is number of chars to end of template
    jz      short NOTFND        ;At end of template
    dec     cx                   ;Cannot point past end, limit search
    jz      short NOTFND        ;If only one char in template, forget it

    push    es
    push    ds
    pop     es
    push    di
    mov     di,si                ;Template to ES:DI
    inc     di
    repne   scasb                ;Look
    pop     di
    pop     es
    jnz     short NOTFND        ;Didn't find the char
    not     cl                    ;Turn how far to go into how far we went
    add     cl,b1                ;Add size of template
    sub     cl,bh                ;Subtract current pos, result distance to skip

FINDOLD_RETN:
    retn

NOTFND:
    pop     bp                    ;Chuck return address
    ; jmp    GETCH
    ; 15/01/2024
    GETCH2_j:
    jmp     short GETCH2

REEDIT:
    mov     al,'@'                ;Output re-edit character
    call    OUTT
    pop     di
    push    di
    push    es
    push    ds
    call    COPYNEW                ;Copy current line into template
    pop     ds
    pop     es
    pop     si
    mov     bl,dh                ;Size of line is new size template
    jmp     PUTNEW                ;Start over again

EXITINS:
ENTERINS:
    not     byte [SS:INSMODE]
    ; jmp    GETCH
    ; 15/01/2024
    jmp     short GETCH2_j

;Put a real live ^Z in the buffer (embedded)
CTRLZ:
    mov     al,"z"-"@" ; 1Ah
    jmp     SAVCH

;Output a CRLF
CRLF:
    mov     al,c_CR ; 0Dh
    call    OUTT
    mov     al,c_LF ; 0Ah
    jmp     OUTT

;
;-----
; ** $RAW_CON_IO - Do Raw Console I/O
;
; Input or output raw character from console, no echo
;
; ENTRY DL = -1 if input
;         = output character if output
; EXIT (AL) = input character if input
; USES all
;
;-----
; 20/07/2018 - Retro DOS v3.0

; 04/05/2019 - Retro DOS v4.0
; DOSCODE:541Ch (MSDOS 6.21, MSDOS.SYS)

; 08/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
; DOSCODE:5408h (MSDOS 5.0, MSDOS.SYS)

```



```

12882 _$RAW_CON_IO: ; System call 6
12883
12884 MOV AL,DL
12885 CMP AL,-1
12886 JNZ SHORT RAWOUT ; 16/12/2022
12887 ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
12888 ;jz short rc1
12889 ;jmp short RAWOUT
12890 ; 16/12/2022
12891 ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
12892 ;nop
12893
12894 rc1: ; Get pointer to register save area
12895 LES DI,[SS:USER_SP] ; 12/03/2018
12896 XOR BX,BX
12897 ;CALL GET_IO_FCB ; MSDOS 2.11 (Retro DOS v2.0)
12898 CALL GET_IO_SFT ; MSDOS 3.3 & MSDOS 6.0
12899 ;JC SHORT RET17
12900 jc short FINDOLD_RETN
12901 MOV AH,1
12902 CALL IOFUNC
12903 JNZ SHORT RESFLG
12904 CALL SPOOLINT
12905 ;OR BYTE [ES:DI+16H],40H
12906 OR BYTE [ES:DI+user_env.user_F],40H ; Set user's zero flag
12907 XOR AL,AL
12908 RET17:
12909 RETN
12910
12911 RESFLG:
12912 ;AND BYTE [ES:DI+16H],0FFH-40H ; 0BFh
12913 AND BYTE [ES:DI+user_env.user_F],0FFH-40H
12914 ; Reset user's zero flag
12915 ;RILP:
12916 rc10: CALL SPOOLINT
12917
12918 ;
12919 ;-----
12920 ;
12921 ;** $Raw_CON_INPUT - Raw Console Input
12922 ;
12923 ; Input raw character from console, no echo
12924 ;
12925 ; ENTRY none
12926 ; EXIT (al) = character
12927 ; USES al
12928 ;
12929 ;-----
12930 ;
12931 ;rci0: invoke SPOOLINT
12932
12933 ;entry $RAW_CON_INPUT
12934
12935 ; 04/05/2019 - Retro DOS v4.0
12936
12937 ; DOSCODE:544Bh (MSDOS 6.21, MSDOS.SYS)
12938
12939 ; 15/01/2024 - Retro DOS v5.0
12940
12941 ; DOSCODE:5ACBh (PCDOS 7.1, IBMDOS.COM)
12942
12943 _$RAW_CON_INPUT: ; System call 7
12944
12945 push bx
12946 XOR BX,BX
12947 ;CALL GET_IO_FCB ; MSDOS 2.11 (Retro DOS v2.0)
12948 CALL GET_IO_SFT ; MSDOS 3.3 & MSDOS 6.0
12949 pop bx
12950 JC SHORT RET17
12951 MOV AH,1
12952 CALL IOFUNC
12953 ;JZ SHORT RILP ; MSDOS 2.11
12954 ;XOR AH,AH
12955 ;CALL IOFUNC
12956 ;RETN
12957 jnz short rci5 ; MSDOS 3.3 & MSDOS 6.0
12958 MOV AH,84h
12959 INT int_IBM ; int 2Ah
12960 JMP short rci0
12961
12962 rci5:
12963 XOR AH,AH
12964 ;CALL IOFUNC
12965 ;RETN
12966 ; 18/12/2022
12967 jmp IOFUNC
12968
12969 ; Output the character in AL to stdout
12970 ;
12971 ;entry RAWOUT
12972 RAWOUT:
12973 PUSH BX
12974 MOV BX,1
12975
12976 ;CALL GET_IO_FCB ; MSDOS 2.11 (Retro DOS v2.0)
12977 CALL GET_IO_SFT ; MSDOS 3.3 & MSDOS 6.0
12978 JC SHORT RAWRET1
12979
12980 ;
12981 ; MSDOS 2.11
12982 ;TEST BYTE [SI+18H],080H ; output to file?
12983 ;JZ SHORT RAWNORM ; if so, do normally
12984 ;PUSH DS
12985 ;PUSH SI
12986 ;LDS SI,[SI+19H] ; output to special?
12987 ;TEST BYTE [SI+4],ISSPEC
12988 ;POP SI
12989 ;
12990 ; MSDOS 3.3 & MSDOS 6.0
12991 ;mov bx,[si+5]
12992 MOV BX,[SI+SF_ENTRY.sf_flags] ;hkn; DS set up by get_io_sft
12993 ;
12994 ; If we are a network handle OR if we are not a local device then go do the
12995 ; output the hard way.
12996 ;
12997 ;and bx,8080h
12998 AND BX,sf_isnet+devid_device
12999 ;cmp bx,80h
13000 CMP BX,devid_device
13001 jnz short RAWNORM
13002 push ds
13003 ;lds bx,[si+7]
13004 LDS BX,[SI+SF_ENTRY.sf_devptr] ; output to special?
13005 ;test byte [bx+4],10h

```

```

13006 00001BC8 F6470410      TEST    BYTE [BX+SYSDEV.ATT],ISSPEC
13007                          ;
13008
13009 00001BCC 1F             POP     DS
13010 00001BCD 740E          JZ      SHORT RAWNORM      ; if not, do normally
13011
13012                          ; 15/01/2024
13013                          ;INT    int_fastcon ; int 29h; quickly output the char
13014
13015                          ; 26/06/2024
13016 00001BCF 1E           push    ds ; *
13017
13018                          ; 12/04/2024 - Retro DOS v5.0
13019                          ; (PCDOS 7.1 IBMBOS.COM)
13020 00001BD0 31DB          xor     bx,bx
13021 00001BD2 8EDB          mov     ds,bx ; 0
13022
13023                          ; 15/01/2024
13024 00001BD4 9C           pushf
13025 00001BD5 FF1EA400      call    far [29h*4] ; simulate INT 29h
13026                          ; call far [00A4h]
13027
13028 00001BD9 1F           pop     ds ; *
13029
13030                          ;JMP     SHORT RAWRET
13031 ;RAWNORM:
13032 ;      CALL    RAWOUT3
13033 RAWRET:
13034 00001BDA F8            CLC
13035 RAWRET1:
13036 00001BDB 5B           POP     BX
13037 RAWRET2:
13038 00001BDC C3           RETN
13039 RAWNORM:
13040 00001BDD E80700        CALL    RAWOUT3
13041 00001BE0 EBF8          jmp     short RAWRET
13042
13043 ;      Output the character in AL to handle in BX
13044 ;
13045 ;      entry    RAWOUT2
13046
13047 RAWOUT2:
13048 ;CALL    GET_IO_FCB      ; MSDOS 2.11 (Retro DOS v2.0)
13049 ;JC      SHORT RET18
13050 00001BE2 E8E422        CALL    GET_IO_SFT      ; MSDOS 3.3 & MSDOS 6.0
13051 00001BE5 72F5          JC      SHORT RAWRET2
13052
13053 00001BE7 50            PUSH    AX
13054 00001BE8 EB0C          JMP     SHORT RAWOSTRT
13055
13056 00001BEA E89742        ROLP:
13057                          CALL    SPOOLINT
13058
13059                          ; 01/05/2019 - Retro DOS v4.0
13060
13061                          ; MSDOS 6.0
13062 ;OR      word [ss:DOS34_FLAG],CTRL_BREAK_FLAG ; 001000000000b
13063 00001BED 36800E[1206]02 or     byte [ss:DOS34_FLAG+1],(CTRL_BREAK_FLAG>>8) ; 02h
13064 ;or     word [ss:DOS34_FLAG],200h
13065 ;AN002; set control break
13066 ;invoke DSKSTATCHK
13067 00001BF3 E80D42        call    DSKSTATCHK      ;AN002; check control break
13068
13069 00001BF6 B403          RAWOSTRT:
13070 00001BF8 E8CD34        MOV     AH,3
13071 00001BF8 74ED          CALL    IOFUNC
13072                          JZ      SHORT ROLP
13073
13074                          ; MSDOS 6.0
13075 ;SR;
13076 ; IOFUNC now returns ax = 0ffffh if there was an I24 on a status call and
13077 ; the user failed. We do not send a char if this happens. We however return
13078 ; to the caller with carry clear because this DOS call does not return any
13079 ; status.
13080 00001BFD 40            ;
13081 00001BFE 58            inc     ax      ;fail on I24 if ax = -1
13082 00001BFF 7405          POP     AX
13083 00001C01 B402          jz      short nosend    ;yes, do not send char
13084 00001C03 E8C234        MOV     AH,2
13085                          call    IOFUNC
13086 00001C06 F8            nosend:
13087 00001C07 C3            CLC
13088                          ; Clear carry indicating successful
13089                          retn
13090
13091                          ; MSDOS 3.3 & MSDOS 2.11
13092 ;POP     AX
13093 ;MOV     AH,2
13094 ;CALL    IOFUNC
13095 ;CLC
13096 ; Clear carry indicating successful
13097 ;RET18:
13098 ;RETN
13099
13100 ;;10/08/2018
13101 ; 20/07/2018 - Retro DOS v3.0
13102 ; -----
13103 ; Retro DOS v2.0 (MSDOS 2.11) - OUTMES
13104 ; -----
13105 ; This routine is called at DOS init
13106
13107 ;; ;procedure OUTMES,NEAR ; String output for internal messages
13108 ;; OUTMES:
13109 ;; ;LODS    CS:BYTE PTR [SI]
13110 ;; CS      LODSB
13111 ;; CMP     AL,"$" ; 24h
13112 ;; JZ      SHORT RET18
13113 ;; CALL    OUTT
13114 ;; JMP     SHORT OUTMES
13115 ; -----
13116 ; 20/07/2018 - Retro DOS v3.0
13117
13118 ; IBMDOS.COM (MSDOS 3.3 kernel) - Offset 2252h
13119
13120 ; -----
13121 ;
13122 ;
13123 ; Inputs:
13124 ; AX=0 save the DEVCALL request packet
13125 ; =1 restore the DEVCALL request packet
13126 ; Function:
13127 ; save or restore the DEVCALL packet
13128 ; Returns:
13129 ; none

```

```

13130 ;
13131 ;-----
13132 ;
13133 ;
13134 ; 04/05/2019 - Retro DOS v4.0
13135 ; DOSCODE:54B9h (MSDOS 6.21, MSDOS.SYS)
13136 ;
13137 ; 08/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
13138 ; DOSCODE:54A5h (MSDOS 5.0, MSDOS.SYS)
13139 ;
13140 ; 12/05/2019
13141 ;
13142 ; 15/01/2024 - Retro DOS v5.0
13143 ; DOSCODE:5B42h (PCDOS 7.1, IBMDOS.COM)
13144 ;
13145 Save_Restore_Packet:
13146     PUSH    DS
13147     PUSH    ES
13148     PUSH    SI
13149     PUSH    DI
13150 ;
13151 ; 16/12/2022
13152 ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
13153 ; 09/09/2018
13154     mov     di,FAKE_STACK_2F
13155     mov     si,DEVCALL
13156 ;
13157 ; 21/09/2023
13158     or      ax,ax
13159     ;CMP     AX,0 ; save packet
13160     JZ      short save_packet ; 16/12/2022
13161     ;je      short set_seg
13162 ;
13163 ; MSDOS 6.0
13164 restore_packet:
13165     ; MOV     SI,OFFSET DOSDATA:Packet_Temp ;source
13166     ; MOV     DI,OFFSET DOSDATA:DEVCALL ;destination
13167     ; MSDOS 3.3
13168     ;mov     si,FAKE_STACK_2F ; DOS_TEMP ; Packed_Temp
13169     ;mov     di,DEVCALL ; 09/09/2018
13170 ;
13171     ;JMP     short set_seg
13172 ;
13173 ; 16/12/2022
13174 ; 09/09/2018
13175     xchg    si,di ; DI = offset DEVCALL, SI = offset FAKE_STACK_2F
13176 ;
13177 ; 16/12/2022
13178 %if 0
13179     ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
13180     cmp     ax,0 ; save packet
13181     jz      short save_packet
13182     mov     si,FAKE_STACK_2F ; 07/12/2022
13183     mov     di,DEVCALL
13184     jmp     short set_seg
13185 ;
13186 ; MSDOS 6.0
13187 save_packet:
13188     ; MOV     DI,OFFSET DOSDATA:Packet_Temp ;destination
13189     ; MOV     SI,OFFSET DOSDATA:DEVCALL ;source
13190     ; 09/09/2018
13191     ; MSDOS 3.3
13192     ;mov     di,FAKE_STACK_2F ; DOS_TEMP ; Packed_Temp
13193     ;mov     si,DEVCALL ; 09/09/2018
13194 ;
13195     ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
13196     mov     di,FAKE_STACK_2F ; DOS_TEMP ; Packed_Temp
13197     mov     si,DEVCALL
13198 %endif
13199 ;
13200 ; 16/12/2022
13201 save_packet:
13202 ;set_seg:
13203     ; MSDOS 3.3
13204     ;mov     ax,cs
13205 ;
13206 ; MSDOS 6.0
13207     ;MOV     AX,SS ; set DS,ES to DOSDATA
13208     ;MOV     DS,AX
13209     ;MOV     ES,AX
13210     ; 15/01/2024
13211     push    ss
13212     pop     ds
13213     push    ds
13214     pop     es
13215 ;
13216     MOV     CX,11 ; 11 words to move
13217     REP     MOVSW
13218 ;
13219     POP     DI
13220     POP     SI
13221     POP     ES
13222     POP     DS
13223     retn
13224 ;
13225 ;=====
13226 ; CPMIO2.ASM, MSDOS 6.0, 1991
13227 ;=====
13228 ; 20/07/2018 - Retro DOS v3.0
13229 ; 01/05/2019 - Retro DOS v4.0
13230 ;
13231 ;hkn; All the variables use SS override or DS. Therefore there is
13232 ;hkn; no need to specifically set up any seg regs unless SS assumption is
13233 ;hkn; not valid.
13234 ;
13235 ;
13236 ;-----
13237 ;
13238 ;** $STD_CON_INPUT - System Call 1
13239 ;
13240 ; Input character from console, echo
13241 ;
13242 ; ENTRY none
13243 ; EXIT (al) = character
13244 ; USES ALL
13245 ;
13246 ;-----
13247 ;
13248 ;
13249 _$STD_CON_INPUT: ;system call 1
13250 ;
13251     CALL    _$STD_CON_INPUT_NO_ECHO
13252     PUSH    AX
13253     CALL    OUTT

```

```

13254 00001C2D 58      POP      AX
13255 CON_INPUT_RETN:
13256 00001C2E C3      RETN
13257
13258
13259
13260
13261
13262
13263
13264
13265
13266
13267
13268
13269
13270
13271
13272
13273
13274
13275
13276
13277
13278
13279
13280
13281
13282 00001C2F 88D0
13283
13284 00001C31 3C20
13285 00001C33 725C
13286 00001C35 3C7F
13287 00001C37 7405
13288
13289
13290 00001C39 36FE06[F901]
13291
13292 00001C3E 1E
13293 00001C3F 56
13294
13295
13296 00001C40 36FE06[0003]
13297
13298
13299 00001C45 368026[0003]3F
13300 00001C4B 7505
13301
13302 00001C4D 50
13303 00001C4E E86142
13304 00001C51 58
13305
13306 00001C52 E859FF
13307
13308 00001C55 5E
13309 00001C56 1F
13310
13311
13312
13313 00001C57 36F606[FE02]FF
13314 00001C5D 74CF
13315
13316 00001C5F 53
13317 00001C60 1E
13318 00001C61 56
13319 00001C62 B80100
13320
13321
13322
13323 00001C65 E86122
13324
13325 00001C68 7224
13326
13327
13328
13329
13330 00001C6A 8B5C05
13331
13332
13333 00001C6D F6C780
13334 00001C70 751C
13335
13336
13337 00001C72 F6C380
13338 00001C75 7417
13339
13340
13341
13342
13343
13344
13345 00001C77 B80400
13346 00001C7A E84C22
13347 00001C7D 720F
13348
13349
13350
13351 00001C7F F6440608
13352
13353
13354
13355
13356 00001C83 7503
13357 00001C85 E98700
13358
13359
13360
13361
13362 00001C88 36C606[FE02]00
13363
13364
13365
13366
13367
13368
13369 00001C8E E98100
13370
13371
13372
13373
13374
13375
13376
13377 00001C91 3C0D

;-----
; ** $STD_CON_OUTPUT - System Call 2
;
; Output character to console
;
; ENTRY (dl) = character
; EXIT  none
; USES  all
;-----

; DOSCODE:54E9h (MSDOS 6.21, MSDOS.SYS)
; 08/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
; DOSCODE:54D5h (MSDOS 5.0, MSDOS.SYS)
; 15/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
; DOSCODE:5B70h (PCDOS 7.1, IBMDOS.COM)

_$STD_CON_OUTPUT: ;system call 2
    MOV     AL,DL
OUTT:
    CMP     AL,20h ; " "
    JB      SHORT CTRLOUT
    CMP     AL,c_DEL ; 7Fh
    JZ      SHORT OUTCH
OUTCHA:
    ;INC     BYTE PTR [CARPOS]
    INC     BYTE [SS:CARPOS]
OUTCH:
    PUSH    DS
    PUSH    SI
    ;INC     BYTE PTR [CHARCO] ;invoke statchk...
    ;AND     BYTE PTR [CHARCO],00111111B ;AN000; every 64th char
    INC     BYTE [SS:CHARCO]
    ;AND     BYTE [SS:CHARCO],00111111B
    ; 01/05/2019 - Retro DOS v4.0
    and     byte [SS:CHARCO],3Fh
    JNZ     SHORT OUTSKIP
    PUSH    AX
    CALL    STATCHK
    POP     AX
OUTSKIP:
    CALL    RAWOUT ;output the character
    POP     SI
    POP     DS
    ;TEST    BYTE PTR [PFLAG],-1
    ;retz
    TEST    BYTE [SS:PFLAG],0FFh
    JZ      SHORT CON_INPUT_RETN
    PUSH    BX
    PUSH    DS
    PUSH    SI
    MOV     BX,1
    ; 20/07/2018 - Retro DOS v3.0
    ; MSDOS 3.3
    ; MSDOS 6.0 (CPMIO2.ASM)
    CALL    GET_IO_SFT ;hkn; GET_IO_SFT will set up DS:SI
    ;hkn; to sft entry
    JC      SHORT TRIPOPJ
    ; 01/05/2019 - Retro DOS v4.0
    ;mov     bx,[si+5]
    MOV     BX,[SI+SF_ENTRY.sf_flags]
    ;test    bx,8000h
    ;TEST    BX,sf_isnet ; 8000h ; output to NET?
    test    bh,(sf_isnet>>8) ; 80h
    JNZ     short TRIPOPJ ; if so, no echo
    ;;test    bx,80h
    ;TEST    BX,devid_device ; output to file?
    test    bl,devid_device ; 80h
    JZ      SHORT TRIPOPJ ; if so, no echo
    ; 14/03/2018
    ;call    GET_IO_FCB ; IBMDOS.COM, MSDOS 2.11
    ;jc      short TRIPOPJ
    ; MSDOS 2.11
    ;test    byte [SI+18H], 80h
    ;jz      short TRIPOPJ
    MOV     BX,4
    CALL    GET_IO_SFT
    JC      SHORT TRIPOPJ
    ;;test    word [si+5], 800h
    ;TEST    word [SI+SF_ENTRY.sf_flags],sf_net_spool ; 800H
    ;test    byte [si+6],8 ; 08/11/2022
    test    byte [SI+SF_ENTRY.sf_flags+1],(sf_net_spool>>8) ; 8
    ; StdPrn redirected?
    ;;JZ      SHORT LISSTRT2J ; No, OK to echo
    ;jz      LISSTRT2 ; 10/08/2018
    ; 16/12/2022
    jnz     short outch1
    jmp     LISSTRT2
    ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
    ;jz      short LISSTRT2J
outch1:
    ;MOV     BYTE [PFLAG],0
    MOV     BYTE [SS:PFLAG],0 ; If a spool, NEVER echo
    ; MSDOS 2.11
    ;mov     bx,4
    ;jmp     short LISSTRT2
TRIPOPJ:
    ; 20/07/2018
    JMP     TRIPOP
    ; 16/12/2022
    ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
;LISSTRT2J:
; JMP     LISSTRT2
CTRLOUT:
    CMP     AL,c_CR ; 0Dh

```

```

13378 00001C93 7420      JZ      SHORT ZERPOS
13379 00001C95 3C08      CMP     AL,C_BS ; 8
13380 00001C97 7424      JZ      SHORT BACKPOS
13381 00001C99 3C09      CMP     AL,C_HT ; 9
13382 00001C9B 75A1      JNZ     SHORT OUTCH
13383                      ;MOV     AL,[CARPOS]
13384 00001C9D 36A0[F901]    MOV     AL,[SS:CARPOS]
13385 00001CA1 0CF8      OR      AL,0F8H
13386 00001CA3 F6D8      NEG     AL
13387
13388 00001CA5 51      TAB:    PUSH    CX
13389 00001CA6 88C1      MOV     CL,AL
13390 00001CA8 B500      MOV     CH,0
13391 00001CAA E307      JCXZ    POPTAB
13392
13393 00001CAC B020      TABLP:  MOV     AL," "
13394 00001CAE E880FF    CALL    OUTT
13395 00001CB1 E2F9      LOOP    TABLP
13396
13397 00001CB3 59      POPTAB: POP     CX
13398
13399 00001CB4 C3      RETN
13400
13401
13402 ZERPOS:  ;MOV     BYTE PTR [CARPOS],0
13403 00001CB5 36C606[F901]00  MOV     BYTE [SS:CARPOS],0
13404                      ; 10/08/2018
13405 00001CBB EB81      JMP     short OUTCH ; 04/05/2019
13406
13407                      ; 18/12/2022
13408
13409 ;OUTJ:
13410 ;JMP     OUTT
13411
13412 BACKPOS:
13413 00001CBD 36FE0E[F901]  ;DEC     BYTE PTR [CARPOS]
13414 00001CC2 E979FF    DEC     BYTE [SS:CARPOS]
13415                      JMP     OUTCH
13416
13417 00001CC5 3C20      BUFOUT:  CMP     AL," "
13418 00001CC7 7315      JAE     SHORT OUTJ           ;Normal char
13419 00001CC9 3C09      CMP     AL,9
13420 00001CCB 7411      JZ      SHORT OUTJ           ;OUT knows how to expand tabs
13421                      ;DOS 3.3 7/14/86
13422 00001CCD 3C15      CMP     AL,"U"-"@" ; 15h      ; turn ^U to section symbol
13423 00001CCF 740D      JZ      short CTRLU
13424 00001CD1 3C14      CMP     AL,"T"-"@" ; 14h      ; turn ^T to paragraph symbol
13425 00001CD3 7409      JZ      short CTRLU
13426
13427 NOT_CTRLU:
13428 00001CD5 50      ;DOS 3.3 7/14/86
13429 00001CD6 B05E      PUSH    AX
13430 00001CD8 E856FF    MOV     AL,"^"
13431 00001CDB 58      CALL    OUTT           ;Print '^' before control chars
13432 00001CDC 0C40      POP     AX
13433                      OR      AL,40H           ;Turn it into Upper case mate
13434
13435 CTRLU:
13436 ;CALL    OUTT
13437 ; 18/12/2022
13438 OUTJ:
13439 jmp     OUTT
13440 ;BUFOUT_RETN:
13441 ;RETN
13442
13443 ;
13444 ;-----
13445 ;** $STD_AUX_INPUT - System Call 3
13446 ;
13447 ; $STD_AUX_INPUT returns a character from Aux Input
13448 ;
13449 ; ENTRY   none
13450 ; EXIT    (al) = character
13451 ; USES    all
13452 ;-----
13453 ;
13454 ; 08/11/2022 Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
13455
13456 _$STD_AUX_INPUT:  ;system call 3
13457
13458 CALL     STATCHK
13459 MOV      BX,3
13460 CALL     GET_IO_SFT      ; 20/07/2018 - MSDOS 3.3 (MSDOS 6.0)
13461 CALL     GET_IO_FCB      ; 14/03/2018 - MSDOS 2.11
13462 ;CALL
13463 ;retc
13464 ; 16/12/2022
13465 ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
13466 ;JC      SHORT BUFOUT_RETN
13467 ;JMP     SHORT TAISTRRT
13468 ; 07/12/2022
13469 00001CEA 7304      jnc     SHORT TAISTRRT
13470 00001CEC C3      retn
13471
13472 AUXILP:
13473 00001CED E89441    CALL    SPOOLINT
13474
13475 00001CF0 B401      TAISTRRT: MOV     AH,1
13476 00001CF2 E8D333    CALL    IOFUNC
13477 00001CF5 74F6      JZ      SHORT AUXILP
13478 00001CF7 30E4      XOR     AH,AH
13479                      ; 16/12/2022
13480                      ;CALL    IOFUNC
13481                      ;RETN
13482                      ; 07/12/2022
13483 00001CF9 E9CC33    jmp     IOFUNC
13484
13485 ;
13486 ;-----
13487 ;** $STD_AUX_OUTPUT - Output character to AUX
13488 ;
13489 ; ENTRY   (dl) = character
13490 ; EXIT    none
13491 ; USES    all
13492 ;-----
13493 ;
13494 _$STD_AUX_OUTPUT:  ;system call 4
13495
13496 PUSH     BX
13497 MOV      BX,3
13498 JMP      SHORT SENDOUT
13499 00001CFC 53
13500 00001CFD B80300
13501 00001D00 EB04

```

```

13502
13503
13504
13505
13506
13507
13508
13509
13510
13511
13512
13513
13514
13515
13516
13517 00001D02 53
13518 00001D03 BB0400
13519
13520 00001D06 88D0
13521 00001D08 50
13522 00001D09 E8A641
13523 00001D0C 58
13524 00001D0D 1E
13525 00001D0E 56
13526
13527 00001D0F E8D0FE
13528
13529 00001D12 5E
13530 00001D13 1F
13531 00001D14 5B
13532
13533 00001D15 C3
13534
13535
13536
13537
13538
13539
13540
13541
13542
13543
13544
13545
13546
13547
13548
13549
13550 00001D16 E89941
13551 00001D19 B000
13552
13553 00001D1B 74F8
13554
13555
13556 00001D1D 48
13557
13558 00001D1E C3
13559
13560
13561
13562
13563
13564
13565
13566
13567
13568
13569
13570
13571
13572
13573
13574
13575
13576 00001D1F 50
13577 00001D20 52
13578 00001D21 31DB
13579 00001D23 E8A321
13580
13581 00001D26 7205
13582 00001D28 B404
13583 00001D2A E89B33
13584
13585
13586 00001D2D 5A
13587 00001D2E 58
13588 00001D2F 88C4
13589 00001D31 3C01
13590 00001D33 7413
13591 00001D35 3C06
13592 00001D37 740F
13593 00001D39 3C07
13594 00001D3B 740B
13595 00001D3D 3C08
13596 00001D3F 7407
13597 00001D41 3C0A
13598 00001D43 7403
13599 00001D45 B000
13600 00001D47 C3
13601
13602
13603 00001D48 FA
13604
13605 00001D49 E929E6
13606
13607
13608
13609
13610
13611
13612
13613
13614
13615
13616
13617
13618
13619
13620
13621
13622
13623
13624
13625

```

```

;
;-----
;
; ** $STD_PRINTER_OUTPUT - Output character to printer
;
; ENTRY (dl) = character
; EXIT none
; USES al
;
;-----
;
;_STD_PRINTER_OUTPUT: ;System call 5
;
; PUSH BX
; MOV BX,4
SENDOUT:
; MOV AL,DL
; PUSH AX
; CALL STATCHK
; POP AX
; PUSH DS
; PUSH SI
LISSTR2:
; CALL RAWOUT2
TRIPOP:
; POP SI
; POP DS
; POP BX
SCIS_RETN: ; 20/07/2018
; RETN
;
;-----
;
; ** $STD_CON_INPUT_STATUS - System Call 11
;
; Check console input status
;
; ENTRY none
; EXIT AL = -1 character available, = 0 no character
; USES al
;
;-----
;
;_STD_CON_INPUT_STATUS: ;System call 11
;
; CALL STATCHK
; MOV AL,0 ; no xor!!
; retz
; JZ SHORT SCIS_RETN ; 15/04/2018
; OR AL,-1
; ; 15/01/2024 (PCDOS 7.1 IBMDOS.COM)
; dec ax ; al = -1
; SCIS_RETN:
; RETN
;
;-----
;
; ** $STD_CON_INPUT_FLUSH - System Call 12
;
; Flush console input buffer and perform call in AL
;
; ENTRY (AL) = DOS function to be called after flush (1,6,7,8,10)
; EXIT (al) = 0 iff (al) was not one of the supported fcns
; return arguments for the fcn supplied in (AL)
; USES al
;
;-----
;
;_STD_CON_INPUT_FLUSH: ;system call 12
;
; PUSH AX
; PUSH DX
; XOR BX,BX
; CALL GET_IO_SFT ; 20/07/2018 - MSDOS 3.3 (MSDOS 6.0)
; CALL GET_IO_FCB ; 14/03/2018 - MSDOS 2.11
; JC SHORT BADJFNCON
; MOV AH,4
; CALL IOFUNC
BADJFNCON:
; POP DX
; POP AX
; MOV AH,AL
; CMP AL,1
; JZ SHORT REDISPJ
; CMP AL,6
; JZ SHORT REDISPJ
; CMP AL,7
; JZ SHORT REDISPJ
; CMP AL,8
; JZ SHORT REDISPJ
; CMP AL,10
; JZ SHORT REDISPJ
; MOV AL,0
; RETN
REDISPJ:
; CLI
; ;transfer REDISP
; JMP REDISP
;
;=====
; FCBIO.ASM, MSDOS 6.0, 1991
;=====
; 20/07/2018 - Retro DOS v3.0
; 17/05/2019 - Retro DOS v4.0
;
; ** FCBIO.ASM - Ancient 1.0 1.1 FCB system calls
;
; $GET_FCB_POSITION
; $FCB_DELETE
; $GET_FCB_FILE_LENGTH
; $FCB_CLOSE
; $FCB_RENAME
; SaveFCBInfo
; ResetLRU
; SetOpenAge
; LRUFCB
; FCBRegen
; BlastSFT

```



```

13626 ; CheckFCB
13627 ; SFTFromFCB
13628 ; FCBHardErr
13629 ;
13630 ; Revision history:
13631 ;
13632 ;         Created: ARR 4 April 1983"
13633 ;         MZ 6 June 1983 completion of functions
13634 ;         MZ 15 Dec 1983 Brain damaged programs close FCBs multiple
13635 ;         times. Change so successive closes work by
13636 ;         always returning OK.Also, detect I/O to
13637 ;         already closed FCB and return EOF.
13638 ;         MZ 16 Jan 1984 More braindamage. Need to separate info
13639 ;         out of sft into FCB for reconnection
13640 ;
13641 ;         A000 version 4.00 Jan. 1988
13642 ;
13643 ;Break <$Get_FCB_Position - set random record fields to current pos>
13644 ;
13645 ;-----
13646 ; $Get_FCB_Position - look at an FCB, retrieve the current position from the
13647 ; extent and next record field and set the random record field to point
13648 ; to that record
13649 ;
13650 ; Inputs: DS:DX point to a possible extended FCB
13651 ; Outputs: The random record field of the FCB is set to the current record
13652 ; Registers modified: all
13653 ;
13654 ;-----
13655 ;
13656 ;
13657 ;_$GET_FCB_POSITION:
13658 ; call GetExtended ; point to FCB
13659 ; call GetExtent ; DX:AX is current record
13660 ; mov [si+21h],ax
13661 ; mov [SI+SYS_FCB.RR],AX ; drop in low order piece
13662 ; mov [si+23h],dl
13663 ; mov [SI+SYS_FCB.RR+2],DL ; drop in high order piece
13664 ; cmp word [si+0Eh],64
13665 ; cmp word [SI+SYS_FCB.RECSIZ],64
13666 ; jae short GetFCBBye
13667 ; mov [si+24h],dh
13668 ; mov [SI+SYS_FCB.RR+2+1],DH; Set 4th byte only if record size < 64
13669 ; GoodPath: ; 16/12/2022
13670 ; GetFCBBye:
13671 ; jmp FCB_RET_OK
13672 ;
13673 ;Break <$FCB_Delete - remove several files that match the input FCB>
13674 ;
13675 ;-----
13676 ;** $FCB_Delete - Delete from FCB Template
13677 ;
13678 ; given an FCB, remove all directory entries in the current
13679 ; directory that have names that match the FCB's ? marks.
13680 ;
13681 ; ENTRY (DS:DX) = address of FCB
13682 ; EXIT entries matching the FCB are deleted
13683 ; (al) = ff iff no entries were deleted
13684 ; USES all
13685 ;
13686 ;-----
13687 ;
13688 ; ; 08/11/2022 Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
13689 ;
13690 ;_$FCB_DELETE: ; System call 19
13691 ; ; OpenBuf is in DOSDATA
13692 ; MOV DI,OPENBUF ; appropriate place
13693 ;
13694 ; call TransFCB ; convert FCB to path
13695 ; JC short BadPath ; signal no deletions
13696 ;
13697 ; push SS
13698 ; pop DS ; SS is DOSDATA
13699 ;
13700 ; call DOS_DELETE ; wham
13701 ; JC short BadPath
13702 ; ; 16/12/2022
13703 ; jnc short GoodPath
13704 ; GoodPath:
13705 ; ; jmp FCB_RET_OK ; do a good return
13706 ; ; ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
13707 ; jmp short GetFCBBye
13708 ;
13709 ; BadPath:
13710 ; ; Error code is in AX
13711 ;
13712 ; jmp FCB_RET_ERR ; let someone else signal the error
13713 ;
13714 ;Break <$Get_FCB_File_Length - return the length of a file>
13715 ;
13716 ;-----
13717 ; $Get_FCB_File_Length - set the random record field to the length of the
13718 ; file in records (rounded up if partial).
13719 ;
13720 ; Inputs: DS:DX - point to a possible extended FCB
13721 ; Outputs: Random record field updated to reflect the number of records
13722 ; Registers modified: all
13723 ;
13724 ;-----
13725 ;
13726 ; ; 15/01/2024 - Retrodos v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
13727 ;
13728 ;_$GET_FCB_FILE_LENGTH:
13729 ;
13730 ; call GetExtended ; get real FCB pointer
13731 ; ; DX points to Input FCB
13732 ;
13733 ; MOV DI,OPENBUF ; OpenBuf is in DOSDATA
13734 ; ; appropriate buffer
13735 ;
13736 ; push ds ; save pointer to true FCB
13737 ; push si
13738 ; call TransFCB ; Trans name DS:DX, sets ATTRIB
13739 ; pop si
13740 ; pop ds
13741 ; JC short BadPath
13742 ; push ds ; save pointer
13743 ; push si
13744 ; push ss
13745 ; pop ds
13746 ; call GET_FILE_INFO ; grab the info
13747 ; pop si ; get pointer back
13748 ; pop ds
13749 ; JC short BadPath ; invalid something

```

```

13750             ; 15/01/2024
13751             ;MOV DX,BX (*)           ; get high order size
13752             ;MOV AX,DI (**)          ; get low order size
13753 00001D90 89D8      mov ax,bx ; hw of file size
13754             ;
13755             ;mov bx,[si+0Eh]
13756 00001D92 8B5C0E     MOV BX,[SI+SYS_FCB.RECSIZ]; get his record size
13757 00001D95 09DB       OR BX,BX           ; empty record => 0 size for file
13758 00001D97 7502      JNZ short GetSize ; not empty
13759             ;MOV BX,128
13760 00001D99 B380      mov b1,128 ; 15/01/2024
13761
GetSize:
13762             ; 15/01/2024
13763             ;MOV DI,AX                ; save low order word
13764             ;MOV AX,DX                ; move high order for divide
13765             ;xchg ax,dx ; (*)
13766             ; ax = hw of file size
13767
13768 00001D9B 31D2      XOR DX,DX           ; clear out high
13769 00001D9D F7F3      DIV BX              ; wham
13770 00001D9F 50        PUSH AX             ; save dividend
13771 00001DA0 89F8      MOV AX,DI ; (**)      ; get low order piece
13772 00001DA2 F7F3      DIV BX              ; wham
13773 00001DA4 89D1      MOV CX,DX           ; save remainder
13774 00001DA6 5A        POP DX              ; get high order dividend
13775 00001DA7 E306      JCXZ LengthStore    ; no roundup
13776 00001DA9 83C001    ADD AX,1
13777 00001DAC 83D200    ADC DX,0           ; 32-bit increment
13778
LengthStore:
13779             ;mov [si+21h],ax
13780 00001DAF 894421     MOV [SI+SYS_FCB.RR],AX ; store low order
13781             ;mov [si+23h],d1
13782 00001DB2 885423     MOV [SI+SYS_FCB.RR+2],DL ; store high order
13783 00001DB5 08F6      OR DH,DH
13784 00001DB7 74A8      JZ short GoodPath ; not storing insignificant zero
13785             ;mov [si+24h],dh
13786 00001DB9 887424     MOV [SI+SYS_FCB.RR+3],DH ; save that high piece
13787             ; 16/12/2022
13788
GoodRet:
13789             ;jmp FCB_RET_OK
13790 00001DBC EBA3      jmp short GoodPath
13791
;Break <$FCB_Close - close a file>
;-----
;
; $FCB_Close - given an FCB, look up the SFN and close it. Do not free it
; as the FCB may be used for further I/O
;
; Inputs: DS:DX point to FCB
; Outputs: AL = FF if file was not found on disk
; Registers modified: all
;-----
;
; 16/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
;
_$FCB_CLOSE: ; System call 16
;
XOR AL,AL ; default search attributes
call GetExtended ; DS:SI point to real FCB
JZ short NoAttr ; not extended
MOV AL,[SI-1] ; get attributes
NoAttr:
; SS override
MOV [SS:ATTRIB],AL ; stash away found attributes
call SFTFromFCB
JC short GoodRet ; MZ 16 Jan Assume death
; If the sharer is present, then the SFT is not regenable. Thus,
; there is no need to set the SFT's attribute.
;
; 9/8/86 F.C. save SFT attribute and restore it back when close is
; done
;
;mov al,[es:di+4]
MOV AL,[ES:DI+SF_ENTRY.sf_attr]
XOR AH,AH
PUSH AX
;
; 9/8/86 F.C. save SFT attribute and restore it back when close is
; done
;
call CheckShare
JNZ short NoStash
MOV AL,[SS:ATTRIB]
;mov [es:di+4],al
MOV [ES:DI+SF_ENTRY.sf_attr],AL ; attempted attribute for close
NoStash:
; 16/01/2024
;if 0
;mov ax,[si+14h]
MOV AX,[SI+SYS_FCB.FDATE] ; move in the time and date
;mov [es:di+0Fh],ax
MOV [ES:DI+SF_ENTRY.sf_date],AX
;mov ax,[si+16h]
MOV AX,[SI+SYS_FCB.FTIME]
;mov [es:di+0Dh],ax
MOV [ES:DI+SF_ENTRY.sf_time],AX
;mov ax,[si+10h]
MOV AX,[SI+SYS_FCB.FILSIZ]
;mov [es:di+11h],ax
MOV [ES:DI+SF_ENTRY.sf_size],AX
;mov ax,[si+12h]
MOV AX,[SI+SYS_FCB.FILSIZ+2]
;mov [es:di+13h],ax
MOV [ES:DI+SF_ENTRY.sf_size+2],AX
;or word [es:di+5],4000h
; 17/12/2022
or byte [ES:DI+SF_ENTRY.sf_flags+1],(sf_close_nodate>>8) ; 40h
;OR word [ES:DI+SF_ENTRY.sf_flags],sf_close_nodate
;else
; 16/01/2024 (PCDOS 7.1 IBMDOS.COM)
push ds
;lds ax,[si+14h]
lds ax,[SI+SYS_FCB.FDATE] ; move in the time and date
;mov [es:di+0Fh],ax
mov [es:di+SF_ENTRY.sf_date],ax
;mov [es:di+0Dh],ds
mov [es:di+SF_ENTRY.sf_time],ds
pop ds
;lds ax,[si+10h]
lds ax,[SI+SYS_FCB.FILSIZ]
;mov [es:di+11h],ax

```



```

13874 00001DF5 26894511      mov     [es:di+SF_ENTRY.sf_size],ax
13875                        ;mov     [es:di+13h],ds
13876 00001DF9 268C5D13      mov     [es:di+SF_ENTRY.sf_size+2],ds
13877                        ; 16/01/2024
13878                        ;;or     word [es:di+5],4000h
13879                        ;or     word [es:di+SF_ENTRY.sf_flags],sf_close_nodate
13880 00001DFD 26804D0640      or      byte [es:di+SF_ENTRY.sf_flags+1],(sf_close_nodate>>8) ; 40h
13881                        %endif
13882
13883 00001E02 16                push    ss
13884 00001E03 1F                pop     ds
13885 00001E04 E82119          call   DOS_CLOSE      ; wham
13886 00001E07 C43E[9E05]      LES     DI,[THISSFT]
13887
13888                        ;;; 9/8/86 F.C. restore SFT attribute
13889 00001E0B 59                POP     CX
13890                        ;mov     [es:di+4],cl
13891 00001E0C 26884D04      MOV     [ES:DI+SF_ENTRY.sf_attr],CL
13892                        ;;; 9/8/86 F.C. restore SFT attribute
13893
13894 00001E10 9C                PUSHF
13895                        ;test    word [es:di],0FFFFh
13896                        ;cmp     word [ES:DI+SF_ENTRY.sf_ref_count],0
13897                        ; zero ref count gets blasted
13898 00001E11 26833D00      cmp     word [ES:DI],0
13899 00001E15 7507                jnz     short CloseOK
13900 00001E17 50                PUSH    AX
13901 00001E18 B04D          MOV     AL,'M' ; 4Dh
13902 00001E1A E8FB02          call   BlastSFT
13903 00001E1D 58                POP     AX
13904
13905 00001E1E 9D                CloseOK: POPF
13906 00001E1F 739B          JNC     short GoodRet
13907                        ;cmp     al,6
13908 00001E21 3C06          CMP     AL,error_invalid_handle
13909 00001E23 7497          JZ      short GoodRet
13910                        ;mov     al,2
13911 00001E25 B002          MOV     AL,error_file_not_found
13912
13913                        fren90:
13914                        ; 16/12/2022
13915 00001E27 E963E8      fcb_close_err: jmp     FCB_RET_ERR
13916
13917                        ;
13918                        ;
13919                        ;-----
13920                        ;** $FCB_Rename - Rename a File
13921                        ;
13922                        ; $FCB_Rename - rename a file in place within a directory. Renames
13923                        ; multiple files copying from the meta characters.
13924                        ;
13925                        ; ENTRY DS:DX point to an FCB. The normal name field is the source
13926                        ; name of the files to be renamed. Starting at offset 11h
13927                        ; in the FCB is the destination name.
13928                        ; EXIT AL = 0 -> no error occurred and all files were renamed
13929                        ; AL = FF -> some files may have been renamed but:
13930                        ; rename to existing file or source file not found
13931                        ; USES ALL
13932                        ;
13933                        ;-----
13934                        ;
13935                        ; 08/11/2022 Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
13936                        ; 16/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
13937
13938                        _$FCB_RENAME: ; System call 23
13939
13940 00001E2A E81D04      call   GetExtended      ; get pointer to real FCB
13941 00001E2D 52                push    dx
13942 00001E2E 8A04          MOV     AL,[SI]         ; get drive byte
13943 00001E30 83C610      ADD     SI,10h          ; point to destination
13944
13945                        ; RenBuf is in DOSDATA
13946 00001E33 BF[3E04]      MOV     DI,RENBUFF      ; point to destination buffer
13947 00001E36 FF34          push    word [SI]
13948 00001E38 1E                push    ds
13949                        ;push    di
13950                        ; 16/01/2024 (Retro DOS v4 BugFix!)
13951 00001E39 56                push    si
13952 00001E3A 8804          MOV     [SI],AL         ; drop in real drive
13953 00001E3C 89F2          MOV     DX,SI           ; let TransFCB know where the FCB is
13954 00001E3E E8BB5D      call   TransFCB         ; munch this pathname
13955 00001E41 5E                pop     si
13956 00001E42 1F                pop     ds
13957 00001E43 8F04          pop     WORD [SI]       ; get path back
13958 00001E45 5A                pop     dx               ; Original FCB pointer
13959 00001E46 72DF          JC      short fren90    ; bad path -> error
13960
13961                        ; SS override for WFP_Start & Ren_WFP
13962 00001E48 368B36[B205]  MOV     SI,[ss:WFP_START] ; get pointer
13963 00001E4D 368936[B405]  MOV     [ss:REN_WFP],SI   ; stash it
13964
13965                        ; OpenBuf is in DOSDATA
13966 00001E52 BF[BE03]      MOV     DI,OPENBUF      ; appropriate spot
13967 00001E55 E8A45D      call   TransFCB         ; wham
13968                        ; NOTE that this call is pointing
13969                        ; back to the ORIGINAL FCB so
13970                        ; ATTRIB gets set correctly
13971 00001E58 72CD          JC      short fren90    ; error
13972
13973                        ;;;
13974                        ; 16/01/2024 - Retro DOS 5.0
13975                        ; (PCDOS 7.1 IBMDOS.COM)
13976 00001E5A 36C606[5411]43  mov     byte [ss:PATHNAMELEN], 67 ; DIRSTRLEN = 67
13977                        ;mov     word [ss:PATHNAMELEN], 67 ; set pathname length to 67
13978 00001E60 E8140F          call   DOS_RENAME
13979 00001E63 72C2          JC      short fren90
13980                        ; 16/12/2022
13981 00001E65 E922E8      jmp     FCB_RET_OK
13982
13983                        ; Error -
13984                        ;
13985                        ; (al) = error code
13986
13987                        ; 16/12/2022
13988                        ;fren90:
13989                        ; jmp     FCB_RET_ERR
13990                        ; ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
13991                        ; jmp     short fcb_close_err
13992
13993                        ;Break <Misbehavior fixers>
13994                        ;
13995                        ; FCBS suffer from several problems. First, they are maintained in the
13996                        ; user's space so he may move them at will. Second, they have a small
13997                        ; reserved area that may be used for system information. Third, there was

```

```

13998 ; never any "rules for behavior" for FCBs; there was no protocol for their
13999 ; usage.
14000 ;
14001 ; This results in the following misbehavior:
14002 ;
14003 ; infinite opens of the same file:
14004 ;
14005 ; while (TRUE) { while (TRUE) {
14006 ;     FCBOpen (FCB);     FCBOpen (FCB);
14007 ;     Read (FCB);        write (FCB);
14008 ; } }
14009 ;
14010 ; infinite opens of different files:
14011 ;
14012 ; while (TRUE) { while (TRUE) {
14013 ;     FCBOpen (FCB[i++]); FCBOpen (FCB[i++]);
14014 ;     Read (FCB);        write (FCB);
14015 ; } }
14016 ;
14017 ; multiple closes of the same file:
14018 ;
14019 ; FCBOpen (FCB);
14020 ; while (TRUE)
14021 ;     FCBClose (FCB);
14022 ;
14023 ; I/O after closing file:
14024 ;
14025 ; FCBOpen (FCB);
14026 ; while (TRUE) {
14027 ;     FCBWrite (FCB);
14028 ;     FCBClose (FCB);
14029 ; }
14030 ;
14031 ; The following is an implementation of a methodology for emulating the
14032 ; above with the exception of I/O after close. We are NOT attempting to
14033 ; resolve that particular misbehavior. We will enforce correct behaviour in
14034 ; FCBs when they refer to a network file or when there is file sharing on
14035 ; the local machine.
14036 ;
14037 ; The reserved fields of the FCB (10 bytes worth) is divided up into various
14038 ; structures depending on the file itself and the state of operations of the
14039 ; OS. The information contained in this reserved field is enough to
14040 ; regenerate the SFT for the local non-shared file. It is assumed that this
14041 ; regeneration procedure may be expensive. The SFT for the FCB is
14042 ; maintained in a LRU cache as the ONLY performance improvement.
14043 ;
14044 ; No regeneration of SFTs is attempted for network FCBs.
14045 ;
14046 ; To regenerate the SFT for a local FCB, it is necessary to determine if the
14047 ; file sharer is working. If the file sharer is present then the SFT is not
14048 ; regenerated.
14049 ;
14050 ; Finally, if there is no local sharing, the full name of the file is no
14051 ; longer available. We can make up for this by using the following
14052 ; information:
14053 ;
14054 ; The Drive number (from the DPB).
14055 ; The physical sector of the directory that contains the entry.
14056 ; The relative position of the entry in the sector.
14057 ; The first cluster field.
14058 ; The last used SFT.
14059 ; OR In the case of a device FCB
14060 ; The low 6 bits of sf_flags (indicating device type)
14061 ; The pointer to the device header
14062 ;
14063 ; We read in the particular directory sector and examine the indicated
14064 ; directory entry. If it matches, then we are kosher; otherwise, we fail.
14065 ;
14066 ; Some key items need to be remembered:
14067 ;
14068 ; Even though we are caching SFTs, they may contain useful sharing
14069 ; information. We enforce good behavior on the FCBs.
14070 ;
14071 ; Network support must not treat FCBs as impacting the ref counts on
14072 ; open VCs. The VCs may be closed only at process termination.
14073 ;
14074 ; If this is not an installed version of the DOS, file sharing will
14075 ; always be present.
14076 ;
14077 ; We MUST always initialize lstclus to = firclus when regenerating a
14078 ; file. Otherwise we start allocating clusters up the wazoo.
14079 ;
14080 ; Always initialize, during regeneration, the mode field to both isFCB
14081 ; and open_for_both. This is so the FCB code in the sharer can find the
14082 ; proper OI record.
14083 ;
14084 ; The test bits are:
14085 ;
14086 ; 00 -> local file
14087 ; 40 -> sharing local
14088 ; 80 -> network
14089 ; C0 -> local device
14090 ;
14091 ;Break <SaveFCBInfo - store pertinent information from an SFT into the FCB>
14092 ;-----
14093 ;
14094 ; SaveFCBInfo - given an FCB and its associated SFT, copy the relevant
14095 ; pieces of information into the FCB to allow for subsequent
14096 ; regeneration. Poke LRU also.
14097 ;
14098 ; Inputs: ThisSFT points to a complete SFT.
14099 ;         DS:SI point to the FCB (not an extended one)
14100 ; Outputs: The relevant reserved fields in the FCB are filled in.
14101 ;          DS:SI preserved
14102 ;          ES:DI point to sft
14103 ; Registers modified: All
14104 ;
14105 ;-----
14106 ;
14107 ;
14108 ; 08/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
14109 ; 20/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
14110 ;
14111 SaveFCBInfo:
14112 ;
14113 LES     DI,[SS:THISSFT]          ; SS override
14114 call    ISSFTNet
14115 JZ      short SaveLocal         ; if not network then save local info
14116 ;
14117 ;----- In net support -----
14118 ;
14119 ; 17/05/2019 - Retro DOS v4.0
14120 ;
14121 ; MSDOS 3.3

```

```

14122      ;mov ax,[es:di+1Dh]
14123      ;mov ax,[es:di+SF_ENTRY.sf_dirsec]
14124      ;mov [si+1Ah],ax
14125      ;mov [si+fcbl_net_handle],ax
14126      ;push es
14127      ;push di
14128      ;les di,[es:di+19h]
14129      ;LES DI,[ES:DI+sf_netid]
14130      ;mov [si+1Ch],di
14131      ;MOV [SI+fcbl_netID],DI      ; save net ID
14132      ;mov [si+1Eh],es
14133      ;MOV [SI+fcbl_netID+2],ES
14134      ;pop di
14135      ;pop es
14136
14137      ; MSDOS 6.0
14138      ;mov ax,[es:di+0Bh]
14139 00001E72 268B450B      MOV AX,[ES:DI+sf_serial_ID] ;AN000;;IFS. save IFS ID
14140      ;mov [si+1Ch],ax
14141 00001E76 89441C      MOV [SI+fcbl_netID],ax      ;AN000;;IFS.
14142
14143      ;mov bl,80h
14144 00001E79 B380      MOV BL,FCBNETWORK
14145
14146      ;----- END In net support -----
14147      ;
14148 00001E7B EB63      jmp SHORT SaveSFN
14149
14150 SaveLocal:
14151      ;IF Installed
14152 00001E7D E8BA65      call CheckShare
14153      ;JZ short SaveNoShare ; no sharer
14154      ;JMP short SaveShare ; sharer present
14155      ; 16/12/2022
14156      ; 28/07/2019
14157 00001E80 7559      jnz short SaveShare
14158      ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
14159      ;JZ short SaveNoShare ; no sharer
14160      ;JMP short SaveShare ; sharer present
14161
14162 SaveNoShare:
14163      ;test word [es:di+5],80h
14164      ;TEST word [ES:DI+SF_ENTRY.sf_flags],devid_device
14165 00001E82 26F6450580 test byte [ES:DI+SF_ENTRY.sf_flags],devid_device ; 80h
14166 00001E87 7542      JNZ short SaveNoShareDev ; Device
14167
14168      ; Save no sharing local file information
14169
14170      ;mov ax,[es:di+1Dh] ; MSDOS 3.3
14171      ;mov ax,[es:di+1Bh] ; MSDOS 6.0
14172 00001E89 268B451B      MOV AX,[ES:DI+SF_ENTRY.sf_dirsec] ; get directory sector F.C.
14173      ;mov [si+1Dh],ax
14174 00001E8D 89441D      MOV [SI+fcbl_nsl_dirsec],AX
14175
14176      ; MSDOS 6.0
14177
14178      ;SR; Store high byte of directory sector
14179      ;mov ax,[es:di+1Dh]
14180 00001E90 268B451D      mov ax,[es:di+SF_ENTRY.sf_dirsec+2] ; get high word
14181
14182      ; SR;
14183      ; We have to store the read-only and archive attributes of the file.
14184      ; We extract it from the SFT and store it in the top two bits of the
14185      ; sector number ( sector number == 22 bits only )
14186
14187      ;mov bl,[es:di+4]
14188 00001E94 268A5D04      mov bl,[es:di+SF_ENTRY.sf_attr]
14189 00001E98 88DF      mov bh,bl
14190 00001E9A D0CB      ror bl,1
14191 00001E9C D0E7      shl bh,1
14192 00001E9E 08FB      or bl,bh
14193 00001EA0 80E3C0      and bl,0C0h
14194 00001EA3 08D8      or al,bl
14195      ;mov [si+18h],al ; 08/11/2022
14196 00001EA5 884418      mov [si+fcbl_sfn],al ; sector number = 22 bits
14197
14198      ; MSDOS 6.0 (& MSDOS 3.3)
14199      ;mov al,[es:di+1Fh]
14200 00001EA8 268A451F      MOV AL,[ES:DI+SF_ENTRY.sf_dirpos] ; location in sector
14201      ;mov [si+1Fh],al
14202 00001EAC 88441F      MOV [SI+fcbl_nsl_dirpos],AL
14203
14204      ; 20/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
14205      ;;;
14206      ;mov ax,[es:di+0Bh] ; .sf_firclus:
14207      ;MOV AX,[ES:DI+SF_ENTRY.sf_firclus] ; first cluster
14208      ; 20/01/2024
14209      ; (PCDOS 7.1 IBMDOS.COM - DOSCODE:5DF5h)
14210      ; (Windows ME IO.SYS - BIOSCODE:5D60h)
14211 00001EAF 268B452B      mov ax,[es:di+2Bh] ; .sf_chain !!! (MSDOS 6.22)
14212      ;mov ax,[es:di+SF_ENTRY.sf_chain] ; first cluster (32 bit) !?
14213      ;;;
14214      ;mov [si+1Bh],ax
14215 00001EB3 89441B      MOV [SI+fcbl_nsl_firclus],AX
14216 00001EB6 B300      MOV BL,0
14217
14218      ; Create the bits field from the dirty/device bits of the flags word
14219      ; and the mode byte
14220
14221 SetFCBBits:
14222      ;mov ax,[es:di+5]
14223 00001EB8 268B4505      MOV AX,[ES:DI+SF_ENTRY.sf_flags]
14224 00001EBC 24C0      AND AL,0C0h ; mask off drive bits
14225      ;or al,[es:di+2]
14226 00001EBE 260A4502      OR AL,[ES:DI+SF_ENTRY.sf_mode] ; stick in open mode
14227      ;mov [si+1Ah], al
14228 00001EC2 88441A      MOV [SI+fcbl_nsl_bits],AL ; save dirty info
14229
14230      ; MSDOS 6.0
14231
14232      ; SR;
14233      ; Check if we came here for local file or device. If for local file,
14234      ; skip setting of SFT index
14235
14236 00001EC5 08DB      or bl,bl
14237 00001EC7 7428      jz short SaveNoSFN ; do not save SFN if local file
14238
14239 00001EC9 EB15      JMP short SaveSFN ; go and save SFN
14240
14241      ; Save no sharing local device information
14242
14243 SaveNoShareDev:
14244      ; 20/01/2024
14245      ;mov ax,[es:di+7]

```

```

14246             ;MOV     AX,[ES:DI+SF_ENTRY.sf_devptr]
14247             ;;mov     [si+1Ah],ax
14248             ;MOV     [SI+fcblnsld_drvptr],AX
14249             ;;mov     ax,[es:di+9]
14250             ;MOV     AX,[ES:DI+SF_ENTRY.sf_devptr+2]
14251             ;MOV     [SI+fcblnsld_drvptr+2],AX
14252             ; 20/01/2024 (PCDOS 7.1 IBMDOS.COM)
14253 00001ECB 06     push     es
14254 00001ECC 26C44507 les     ax,[es:di+SF_ENTRY.sf_devptr]
14255 00001ED0 89441A   mov     [si+fcblnsld_drvptr],ax
14256 00001ED3 8C441C   mov     [si+fcblnsld_drvptr+2],es
14257 00001ED6 07     pop     es
14258
14259             ;mov     bl,40h
14260 00001ED7 B340     MOV     BL,FCBDEVICE
14261             ; 28/12/2022
14262 00001ED9 EBDD     JMP     short SetFCBBits      ; go and save SFN
14263
14264 SaveShare:
14265             ;ENDIF
14266
14267 ;----- In share support -----
14268
14269             ;call     far [ss:ShSave]
14270 00001EDB 36FF1E[B800] Call     far [ss:JShare+(10*4)] ; 10 = ShSave ; SS override
14271
14272 ;----- end in share support -----
14273
14274             ; 17/05/2019
14275
14276 SaveSFN:
14277             ;lea     ax,[di-6]
14278 00001EE0 8D45FA   LEA     AX,[DI-SFT.SFTable]
14279
14280             ; Adjust for offset to table.
14281
14282 00001EE3 362B06[4000] SUB     AX,[SS:SFTFCB]      ; SS override for SftFCB
14283
14284 00001EE8 53     push     bx                ;bx = FCB type (net/Share or local)
14285             ;;mov     bl,53 ; MSDOS 3.3
14286             ;mov     bl,59 ; MSDOS 6.0
14287 00001EE9 833B     MOV     BL,SF_ENTRY.size
14288 00001EEB F6F3     DIV     BL
14289             ;mov     [si+18h],al
14290 00001EED 884418   MOV     [SI+fcblnsfn],AL      ; last used SFN
14291 00001EF0 5B     pop     bx                ;restore bx
14292
14293 SaveNoSFN:
14294             ;mov     ax,[es:di+5]
14295 00001EF1 268B4505 MOV     AX,[ES:DI+SF_ENTRY.sf_flags]
14296 00001EF5 243F     AND     AL,3Fh              ; get real drive
14297 00001EF7 08D8     OR      AL,BL
14298             ;mov     [si+19h],al
14299 00001EF9 884419   MOV     [SI+fcblnsdrive],AL
14300
14301 00001EFC 36A1[1000] MOV     AX,[SS:FCBLRU]      ; get lru count
14302 00001F00 40     INC     AX
14303             ;mov     [es:di+15h],ax
14304 00001F01 26894515 MOV     [ES:DI+sf_LRU],AX
14305 00001F05 7506     JNZ     short SimpleStuff
14306
14307             ; lru flag overflowed. Run through all FCB sfts and adjust:
14308             ; LRU < 8000H get set to 0. Others -= 8000h. This LRU = 8000h
14309
14310             ;mov     bx,15h
14311 00001F07 8B1500   MOV     BX,SF_ENTRY.sf_position
14312 00001F0A E80500   call     ResetLRU
14313
14314             ; Set new LRU to AX
14315 SimpleStuff:
14316 00001F0D 36A3[1000] MOV     [SS:FCBLRU],AX
14317 00001F11 C3     retn
14318
14319 ;Break      <ResetLRU - reset overflowed lru counts>
14320 ;
14321 ;-----
14322 ; ResetLRU - during lru updates, we may wrap at 64K. we must walk the
14323 ; entire set of SFTs and subtract 8000h from their lru counts and truncate
14324 ; at 0.
14325 ;
14326 ; Inputs: BX is offset into SFT field where lru firld is kept
14327 ;         ES:DI point to SFT currently being updated
14328 ; Outputs: All FCB SFTs have their lru fields truncated
14329 ;          AX has 8000h
14330 ; Registers modified: none
14331 ;
14332 ;-----
14333 ;
14334 ; 17/05/2019 - Retro DOS v4.0
14335
14336 ResetLRU:
14337             ; ResetLRU is only called from fcbio.asm. so SS can be assumed to be
14338             ; DOSDATA
14339
14340             MOV     AX,8000h
14341             push     es
14342             push     di
14343             ;LES     DI,[CS:SFTFCB]      ; get pointer to head
14344 00001F17 36C43E[4000] LES     DI,[SS:SFTFCB] ; MSDOS 6.0
14345             ;mov     cx,[es:di+4]
14346 00001F1C 268B4D04 MOV     CX,[ES:DI+SFT.SFCount]
14347             ;lea     di,[di+6]
14348 00001F20 8D7D06   LEA     DI,[DI+SFT.SFTable] ; point at table
14349
14350 ovScan:
14351             SUB     [ES:DI+BX],AX      ; decrement lru count
14352             JA      short ovLoop
14353             MOV     [ES:DI+BX],AX      ; truncate at 0
14354
14355 ovLoop:
14356             ;;add     di,53 ; MSDOS 3.3
14357             ;add     di,59 ; MSDOS 6.0
14358 00001F2B 83C73B   ADD     DI,SF_ENTRY.size      ; advance to next
14359 00001F2E E2F3     LOOP    ovScan
14360             pop     di
14361             pop     es
14362             MOV     [ES:DI+BX],AX
14363             retn
14364
14365 ;IF 0 ; we dont need this routine any more.
14366 ;
14367 ;Break      <SetOpenAge - update the open age of a SFT>
14368 ;-----
14369 ; SetOpenAge - In order to maintain the first N open files in the FCB cache,
14370 ; we keep the 'open age' or an LRU count based on opens. we update the

```

```

14370 ; count here and fill in the appropriate field.
14371 ;
14372 ; Inputs: ES:DI point to SFT
14373 ; Outputs: ES:DI has the open age field filled in.
14374 ; If open age has wraparound, we will have subtracted 8000h
14375 ; from all open ages.
14376 ; Registers modified: AX
14377 ;
14378 ;-----
14379 ;
14380 ;SetOpenAge:
14381 ; 20/07/2018 - Retro DOS v3.0
14382 ; MSDOS 3.3 - IBMDOS.COM, Offset 2597h
14383 ; (& MSDOS 6.0, FCBI0.ASM)
14384 ;
14385 ; SetOpenAge is called from fcbio2.asm. SS can be assumed to be valid.
14386 ;
14387 ; MOV AX,[CS:OpenLRU] ; SS override
14388 ; INC AX
14389 ; mov [es:di+17h],ax
14390 ; MOV [ES:DI+sf_OpenAge],AX
14391 ; JNZ short SetDone
14392 ; mov bx,17h
14393 ; MOV BX,SF_ENTRY.sf_position+2 ; mov bx,sf_OpenAge
14394 ; call ResetLRU
14395 ;SetDone:
14396 ; MOV [CS:OpenLRU],AX
14397 ; retn
14398 ;
14399 ;ENDIF ; SetOpenAge no longer needed
14400 ;
14401 ; 21/07/2018 - Retro DOS v3.0
14402 ; LRUFCB for MSDOS 6.0 !
14403 ;
14404 ;Break <LRUFCB - perform LRU on FCB sfts>
14405 ;-----
14406 ;
14407 ; LRUFCB - find LRU fcb in cache. Set ThisSFT and return it. We preserve
14408 ; the first keepcount sfts if they are network sfts or if sharing is
14409 ; loaded. If carry is set then NO BLASTING is NECESSARY.
14410 ;
14411 ; Inputs: none
14412 ; Outputs: ES:DI point to SFT
14413 ; ThisSFT points to SFT
14414 ; SFT is zeroed
14415 ; Carry set of closes failed
14416 ; Registers modified: none
14417 ;
14418 ;-----
14419 ;
14420 ; MSDOS 6.0
14421 ; IF 0 ; rewritten this routine
14422 ;
14423 ; LRUFCB: ; MSDOS 3.3 - IBMDOS.COM (1987) - Offset 25ADh
14424 ; call save_world
14425 ;
14426 ; Find nth oldest NET/SHARE FCB. We want to find its age for the second scan
14427 ; to find the lease recently used one that is younger than the open age. We
14428 ; operate by scanning the list n times finding the least age that is greater
14429 ; or equal to the previous minimum age.
14430 ;
14431 ; BP is the count of times we need to go through this loop.
14432 ; AX is the current acceptable minimum age to consider
14433 ;
14434 ; mov bp,[CS:KEEPCOUNT] ; k = keepcount;
14435 ; XOR AX,AX ; low = 0;
14436 ;
14437 ; If we've scanned the table n times, then we are done.
14438 ;
14439 ; lru1:
14440 ; CMP bp,0 ; while (k--) {
14441 ; JZ short lru75
14442 ; DEC bp
14443 ;
14444 ; Set up for scan.
14445 ;
14446 ; AX is the minimum age for consideration
14447 ; BX is the minimum age found during the scan
14448 ; SI is the position of the entry that corresponds to BX
14449 ;
14450 ; MOV BX,-1 ; min = 0xffff;
14451 ; MOV si,BX ; pos = 0xffff;
14452 ; LES DI,[CS:SFTFCB] ; for (CX=FCBcount; CX>0; CX--)
14453 ; mov cx,[es:di+4]
14454 ; MOV CX,[ES:DI+SFT.SFCcount]
14455 ; lea di,[di+6]
14456 ; LEA DI,[DI+SFT.SFTable]
14457 ;
14458 ; Innermost loop. If the current entry is free, then we are done. Or, if the
14459 ; current entry is busy (indicating a previous aborted allocation), then we
14460 ; are done. In both cases, we use the found entry.
14461 ;
14462 ; lru2:
14463 ; cmp word [es:di],0
14464 ; ;cmp word [es:di+SF_ENTRY.sf_ref_count],0
14465 ; jz short lru25
14466 ; ;cmp word [es:di],-1
14467 ; ;cmp word [es:di+SF_ENTRY.sf_ref_count],sf_busy
14468 ; cmp word [es:di],sf_busy
14469 ; jnz short lru3
14470 ;
14471 ; The entry is usable without further scan. Go and use it.
14472 ;
14473 ; lru25:
14474 ; MOV si,DI ; pos = i;
14475 ; JMP short lru11 ; goto got;
14476 ;
14477 ; See if the entry is for the network or for the sharer.
14478 ;
14479 ; If for the sharer or network then
14480 ; if the age < current minimum AND >= allowed minimum then
14481 ; this entry becomes current minimum
14482 ;
14483 ; lru3:
14484 ; test word [es:di+5],8000h
14485 ; TEST word [ES:DI+SF_ENTRY.sf_flags],sf_isnet
14486 ; ; if (!net[i]
14487 ; JNZ short lru35
14488 ; if installed
14489 ; call CheckShare ; && !sharing)
14490 ; JZ short lru5 ; else
14491 ; ENDIF
14492 ;
14493 ; This SFT is for the net or is for the sharer. See if it less than the

```

```

14494 ; current minimum.
14495 ;
14496 ;lru35:
14497 ;mov dx,[es:di+17h]
14498 ;MOV DX,[ES:DI+sf_OpenAge]
14499 ;CMP DX,AX ; if (age[i] >= low &&
14500 ;JB short lru5
14501 ;CMP DX,BX
14502 ;JAE short lru5 ; age[i] < min) {
14503 ;
14504 ; entry is new minimum. Remember his age.
14505 ;
14506 ; mov bx,DX ; min = age[i];
14507 ; mov si,di ; pos = i;
14508 ;
14509 ; End of loop. gp back for more
14510 ;
14511 ;lru5:
14512 ;add di,53
14513 ;add di,SF_ENTRY.size
14514 ;loop lru2 ; }
14515 ;
14516 ; The scan is complete. If we have successfully found a new minimum (pos != -1)
14517 ; set then threshold value to this new minimum + 1. Otherwise, the scan is
14518 ; complete. Go find LRU.
14519 ;
14520 ;lru6:
14521 ;cmp si,-1 ; position not -1?
14522 ;jz short lru75 ; no, done with everything
14523 ;lea ax,[bx+1] ; set new threshold age
14524 ;jmp short lru1 ; go and loop for more
14525 ;lru65:
14526 ;stc
14527 ;jmp short LRUDead ; return -1;
14528 ;
14529 ; Main loop is done. We have AX being the age+1 of the nth oldest sharer or
14530 ; network entry. We now make a second pass through to find the LRU entry
14531 ; that is local-no-share or has age >= AX
14532 ;
14533 ;lru75:
14534 ;mov bx,-1 ; min = 0xffff;
14535 ;mov si,bx ; pos = 0xffff;
14536 ;LES DI,[CS:SFTFCB] ; for (CX=FCBCount; CX>0; CX--)
14537 ;;mov cx,[es:di+4]
14538 ;MOV CX,[ES:DI+SFT.SFCount]
14539 ;;lea di,[di+6]
14540 ;LEA DI,[DI+SFT.SFTable]
14541 ;
14542 ; If this is is local-no-share then go check for LRU else if age >= threshold
14543 ; then check for lru.
14544 ;
14545 ;lru8:
14546 ;test word [es:di+5],8000h
14547 ;TEST word [ES:DI+SF_ENTRY.sf_flags],sf_isnet
14548 ;jnz short lru85 ; is for network, go check age
14549 ;call CheckShare ; sharer here?
14550 ;jz short lru86 ; no, go check lru
14551 ;
14552 ; Network or sharer. Check age
14553 ;
14554 ;lru85:
14555 ;cmp [es:di+17h],ax
14556 ;cmp [es:di+sf_OpenAge],ax
14557 ;jb short lru9 ; age is before threshold, skip it
14558 ;
14559 ; Check LRU
14560 ;
14561 ;lru86:
14562 ;cmp [es:di+15h],bx
14563 ;cmp [es:di+sf_LRU],bx ; is LRU less than current LRU?
14564 ;jae short lru9 ; no, skip this
14565 ;mov si,di ; remember position
14566 ;;mov bx,[es:di+15h]
14567 ;mov bx,[es:di+sf_LRU] ; remember new minimum LRU
14568 ;
14569 ; Done with this entry, go back for more.
14570 ;
14571 ;lru9:
14572 ;add di, 53
14573 ;add di,SF_ENTRY.size
14574 ;loop lru8
14575 ;
14576 ; Scan is complete. If we found NOTHING that satisfied us then we bomb
14577 ; out. The conditions here are:
14578 ;
14579 ; No local-no-shares AND all net/share entries are older than threshold
14580 ;
14581 ;lru10:
14582 ;cmp si,-1 ; if no one f
14583 ;jz short lru65 ; return -1;
14584 ;lru11:
14585 ;mov di,si
14586 ;MOV [CS:THISSFT],DI ; set thissft
14587 ;MOV [CS:THISSFT+2],ES
14588 ;
14589 ; If we have sharing or thissft is a net sft, then close it until ref count
14590 ; is 0.
14591 ;
14592 ;;test word [es:di+5],8000h
14593 ;TEST word [ES:DI+SF_ENTRY.sf_flags],sf_isnet
14594 ;JNZ short LRUClose
14595 ;IF INSTALLED
14596 ;call CheckShare
14597 ;JZ short LRUDone
14598 ;ENDIF
14599 ;
14600 ; Repeat close until ref count is 0
14601 ;
14602 ;LRUClose:
14603 ;push ss
14604 ;pop ds
14605 ;LES DI,[THISSFT]
14606 ;word [es:di],0
14607 ;CMP word [ES:DI+SFT.sf_ref_count],0 ; is ref count still <> 0?
14608 ;JZ short LRUDone ; nope, all done
14609 ;call DOS_CLOSE
14610 ;jnc short LRUClose ; no error => clean up
14611 ;;cmp al,6
14612 ;cmp al,error_invalid_handle
14613 ;jz short LRUClose
14614 ;stc
14615 ;JMP short LRUDead
14616 ;LRUDone:
14617 ;XOR AL,AL

```



```

14618 ; call BlastSFT ; fill SFT with 0 (AL), 'c' cleared
14619 ;
14620 ;LRUDead:
14621 ; call restore_world
14622 ; LES DI,[CS:THISST]
14623 ; jnc short LRUFCB_retn
14624 ;LRUFCB_err:
14625 ; ; mov al, 23h
14626 ; MOV AL,error_FCB_unavailable
14627 ;LRUFCB_retn:
14628 ; ret:
14629 ;
14630 ;ENDIF ; LRUFCB has been rewritten below.
14631 ;
14632 ; 17/05/2019 - Retro DOS v4.0
14633 ; LRUFCB for MSDOS 6.0 !
14634 -----
14635 ;
14636 ; LruFCB -- allocate the LRU SFT from the SFT Table. The LRU scheme
14637 ; maintains separate counts for net/Share and local SFTs. We allocate a
14638 ; net/Share SFT only if we do not find a local SFT. This helps keep
14639 ; net/Share SFTs which cannot be regenerated for as long as possible. We
14640 ; optimize regeneration operations by keeping track of the current local
14641 ; SFT. This avoids scanning of the SFTs as long as we have at least one
14642 ; local SFT in the SFT Block.
14643 ;
14644 ; Inputs: al = 0 => Regenerate SFT operation
14645 ; = 1 => Allocate new SFT for Open/Create
14646 ;
14647 ; Outputs: Carry clear
14648 ; es:di = Address of allocated SFT
14649 ; ThisSFT = Address of allocated SFT
14650 ;
14651 ; carry set if closes of net/Share files failed
14652 ; al = error_FCB_unavailable
14653 ;
14654 ; Registers affected: None
14655 ;
14656 -----
14657 ;LruFCB PROC NEAR
14658 LRUFCB:
14659 ; 17/05/2019 - Retro DOS v4.0
14660 ; DOSCODE:5805h (MSDOS 6.21, MSDOS.SYS)
14661 ;
14662 ; 08/11/2022 Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
14663 ; DOSCODE:57F1h (MSDOS 5.0, MSDOS.SYS)
14664 ;
14665 ; 20/01/2024 Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
14666 ; DOSCODE:5E7Ch (PCDOS 7.1, IBMDOS.COM)
14667 ;
14668 00001F36 06 push es ; * (MSDOS 6.21)
14669 00001F37 E81EE5 call save_world
14670 ;
14671 ;getdseg <ds> ;ds = DOSDATA
14672 mov ds,[cs:DosDSeg]
14673 00001F3A 2E8E1E[0700] or al,al ;Check if regenerate allocation
14674 00001F41 7516 jnz short lru1 ;Try to find SFT to use
14675 ;
14676 ; This is a regen call. If LocalSFT contains the address of a valid
14677 ; local SFT, just return that SFT to reuse
14678 ;
14679 ; 20/01/2024
14680 ;mov di,[LocalSFT]
14681 ;or di,[LocalSFT+2] ;is address == 0?
14682 ;jz short lru1 ;invalid local SFT, find one
14683 ;
14684 ; we have found a valid local SFT. Recycle this SFT
14685 ;
14686 00001F43 C43E[760F] les di,[LocalSFT]
14687 ;
14688 ; 20/01/2024 (PCDOS 7.1 IBMDOS.COM)
14689 mov cx,es
14690 or cx,di ; is address == 0?
14691 jz short lru1 ; invalid local SFT, find one
14692 ;
14693 gotLocalSFT:
14694 mov [THISST],di
14695 mov [THISST+2],es
14696 cld
14697 jmp LRUDone ;clear up SFT and return
14698 ;
14699 lru1:
14700 les di,[SFTFCB] ;es:di = SF Table for FCBs
14701 ;mov cx,[es:di+4]
14702 mov cx,[es:di+SFT.SFCount];cx = number of SFTs
14703 ;lea di,[di+6]
14704 lea di,[di+SFT.SFTable] ;es:di = first SFT
14705 ;
14706 ; We scan through all the SFTs scanning for a free one. It also
14707 ; remembers the LRU SFT for net/Share SFTs and local SFTs separately.
14708 ; bx = min. LRU for local SFTs
14709 ; si = pos. of local SFT with min. LRU
14710 ; dx = min. LRU for net/Share SFTs
14711 ; bp = pos. of net/Share SFT with min. LRU
14712 ;
14713 mov bx,-1 ; init. to 0xffff ( max. LRU value )
14714 mov si,bx
14715 mov dx,bx
14716 mov bp,bx
14717 ;
14718 findsFT:
14719 ;See if this SFT is a free one. If so, return it
14720 or word [es:di],0
14721 ;or word [es:di+SF_ENTRY.sf_ref_count],0 ;reference count = 0 ?
14722 jz short gotSFT ;yes, SFT is free
14723 ;cmp word [es:di],-1
14724 ;cmp word [es:di+SF_ENTRY.sf_ref_count],sf_busy ;Is it busy?
14725 cmp word [es:di],sf_busy ; -1
14726 jz short gotSFT ;no, can use it
14727 ;
14728 ; Check if this SFT is local and store its address in LocalSFT. Can be
14729 ; used for a later regen.
14730 ;
14731 ; 16/12/2022
14732 ; 08/11/2022
14733 ;test byte [es:di+6],80h
14734 test byte [es:di+SF_ENTRY.sf_flags+1],(sf_isnet>>8) ; 80h
14735 ; 08/11/2022 Retro DOS v4.0 (MSDOS 5.0 MSDOS.SYS compatibility)
14736 ;test word [es:di+5],8000h
14737 ;test word [ES:DI+SF_ENTRY.sf_flags],sf_isnet ; network SFT?
14738 jnz short lru5 ;yes, get net/Share LRU
14739 ;
14740 ;
14741 00001F7E 7531

```

```

14742
14743
14744 00001F80 E8B764 ;IF installed
14745 ;call CheckShare ;Share present?
14746 00001F83 752C ;ENDIF
14747 jnz short lru5 ;yes, get net/Share LRU
14748 ;Local SFT, register its address
14749
14750 ; !!HACK!!!
14751 ; There is a slightly dirty hack out here in a desperate bid to save
14752 ; code space. There is similar code duplicated at label 'gotSFT'. We
14753 ; enter from there if al = 0, update the LocalSFT variable, and since
14754 ; al = 0, we jump out of the loop to the exit point. I have commented
14755 ; out the code that previously existed at label 'gotSFT'
14756
14757
14758 00001F85 893E[760F] hackpoint:
14759 00001F89 8C06[780F] mov [LocalSFT],di
14760 mov [LocalSFT+2],es ;store local SFT address
14761 00001F8D 08C0 or al,al ;Is operation = REGEN?
14762 00001F8F 74BC jz short gotlocalSFT ;yes, return this SFT for reuse
14763
14764 ;Get LRU for local files
14765
14766 ;cmp [es:di+15h],bx
14767 00001F91 26395D15 cmp [es:di+sf_LRU],bx ;SFT.LRU < min?
14768 00001F95 7306 jae short lru4 ;no, skip
14769
14770 ;mov bx,[es:di+15h]
14771 00001F97 268B5D15 mov bx,[es:di+sf_LRU] ;yes, store new minimum
14772 00001F9B 89FE mov si,di ;store SFT position
14773
14774 lru4:
14775 00001F9D 83C73B add di,59
14776 00001FA0 E2CB add di,SF_ENTRY.size ;go to next SFT
14777 loop findSFT
14778
14779 ; 20/01/2024
14780 dec cx ; -1
14781
14782 ; Check whether we got a net/Share or local SFT. If local SFT
14783 ; available, we will reuse it instead of net/Share LRU
14784 00001FA3 89F7 mov di,si
14785 ;cmp si,-1 ;local SFT available?
14786 00001FA5 39CE cmp si,cx ; 20/01/2024
14787 00001FA7 7516 jnz short gotSFT ;yes, return it
14788
14789 ;No local SFT, see if we got a net/Share SFT
14790
14791 00001FA9 89EF mov di,bp
14792
14793 00001FAB 39CD cmp bp,cx ; -1 ; 20/01/2024
14794 ;cmp bp,-1 ;net/Share SFT available?
14795 00001FAD 752D jnz short gotnetSFT ;yes, return it
14796
14797 noSFT:
14798 ; NB: This error should never occur. We always must have an LRU SFT.
14799 ; This error can occur only if the SFT has been corrupted or the LRU
14800 ; count is not maintained properly.
14801 00001FAF EB4E jmp short errorbadSFT ;error, no FCB available.
14802
14803 ; Handle the LRU for net/Share SFTs
14804
14805 lru5:
14806 00001FB1 26395515 ;cmp [es:di+15h],dx
14807 00001FB5 73E6 cmp [es:di+sf_LRU],dx ;SFT.LRU < min?
14808 jae short lru4 ;no, skip
14809
14810 ;mov dx,[es:di+15h]
14811 00001FB7 268B5515 mov dx,[es:di+sf_LRU] ;yes, store new minimum
14812
14813 mov bp,di ;store SFT position
14814 00001FBD EBDE jmp short lru4 ;continue with next SFT
14815
14816 gotSFT:
14817 00001FBF 08C0 or al,al
14818 00001FC1 74C2 jz short hackpoint ;save es:di in LocalSFT
14819
14820 ; HACK!!!
14821 ; The code here differs from the code at 'hackpoint' only in the
14822 ; order of the check for al. If al = 0, we can jump to 'hackpoint'
14823 ; and then from there jump out to 'gotlocalSFT'. The original code
14824 ; has been commented out below and replaced by the code just above.
14825
14826 ;If regen, then this SFT can be registered as a local one ( even if free ).
14827
14828 ; or al,al ;Regen?
14829 jnz short notlocaluse ;yes, register it and return
14830
14831 ;Register this SFT as a local one
14832
14833 ; mov [LocalSFT],di
14834 ; mov [LocalSFT+2],es
14835 ; jmp gotlocalSFT ;return to caller
14836
14837 notlocaluse:
14838
14839 ; The caller is probably going to use this SFT for a net/Share file.
14840 ; We will come here only on a Open/Create when the caller($FCB_OPEN)
14841 ; does not really know whether it is a local file or not. We
14842 ; invalidate LocalSFT if the SFT we are going to use was previously
14843 ; registered as a local SFT that can be recycled.
14844 00001FC3 8CC0 mov ax,es
14845 00001FC5 393E[760F] cmp [LocalSFT],di ;Offset same?
14846 00001FC9 750E jne short notinvalid
14847 00001FCB 3906[780F] cmp [LocalSFT+2],ax ;Segments same?
14848 ;je short zeroLocalSFT ;no, no need to invalidate
14849 ; 20/01/2024 (PCDOS 7.1 IBMDOS.COM)
14850 00001FCF 7508 jne short notinvalid
14851
14852 zeroLocalSFT:
14853 00001FD1 31C0 xor ax,ax ; 0
14854 00001FD3 A3[760F] mov [LocalSFT],ax
14855 00001FD6 A3[780F] mov [LocalSFT+2],ax
14856
14857 notinvalid:
14858 00001FD9 E971FF jmp gotlocalSFT
14859
14860 ; The SFT we are going to use was registered in the LocalSFT variable.
14861 ; Invalidate this variable i.e LocalSFT = NULL
14862
14863 zeroLocalSFT:
14864 ;xor ax,ax ; 0
14865 ;mov [LocalSFT],ax
14866 ;mov [LocalSFT+2],ax

```



```

14866 ;
14867 ;jmp gotlocalSFT
14868
14869 gotnetSFT:
14870 ; We have an SFT that is currently net/Share. If it is going to be
14871 ; used for a regen, we know it has to be a local SFT. Update the
14872 ; LocalSFT variable
14873
14874 00001FDC 08C0 or al,al
14875 00001FDE 7508 jnz short closenet
14876
14877 00001FE0 893E[760F] mov [LocalSFT],di
14878 00001FE4 8C06[780F] mov [LocalSFT+2],es ;store local SFT address
14879
14880 00001FE8 893E[9E05] closenet: mov [THISSFT],di ; set thisstft
14881 00001FEC 8C06[A005] mov [THISSFT+2],es
14882
14883 ; If we have sharing or thisSFT is a net sft, then close it until ref
14884 ; count is 0.
14885 ; NB: we come here only if it is a net/Share SFT that is going to be
14886 ; recycled -- no need to check for this.
14887
14888 LRUclose:
14889 00001FF0 26833D00 cmp word [es:di],0
14890 ;cmp word [es:di+SF_ENTRY.sf_ref_count],0 ; is ref count still <> 0?
14891 00001FF4 740C jz short LRUDone ; nope, all done
14892
14893 00001FF6 E82F17 call DOS_CLOSE
14894 00001FF9 73F5 jnc short LRUclose ; no error => clean up
14895
14896 ; Bugbug: I dont know why we are trying to close after we get an
14897 ; error closing. Seems like we could have a potential infinite loop
14898 ; here. This has to be verified.
14899
14900 00001FFB 3C06 cmp al,error_invalid_handle ; 6
14901 00001FFD 74F1 je short LRUclose
14902
14903 00001FFF F9 errorbadSFT: stc
14904 00002000 EB05 JMP short LRUDead
14905
14906 00002002 30C0 LRUDone: XOR AL,AL
14907 00002004 E81101 call BlastSFT ; fill SFT with 0 (AL), 'c' cleared
14908
14909 LRUDead:
14910 00002007 E837E4 call restore_world ; use macro
14911
14912 0000200A 07 pop es ; * (MSDOS 6.21)
14913
14914 ;getdseg <es>
14915 0000200B 2E8E06[0700] mov es,[cs:DosDSeg]
14916 00002010 26C43E[9E05] les di,[es:THISSFT] ;es:di points at allocated SFT
14917
14918 ;;retnc
14919 ;jc short LruFCB_err
14920 ;retn
14921
14922 ; 16/12/2022
14923 ; 08/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
14924 00002015 7302 jnc short LruFCB_retn
14925 ;jc short LruFCB_err
14926 ;retn
14927
14928 LruFCB_err:
14929 00002017 B023 MOV AL,error_FCB_unavailable ; 23h
14930
14931 00002019 C3 LruFCB_retn: retn
14932
14933 ;LruFCB ENDP
14934
14935 ; 17/05/2019 - Retro DOS v4.0
14936
14937 ; DOSCODE:58F3h (MSDOS 6.21, MSDOS.SYS)
14938
14939 ; 26/06/2024
14940 %if 0
14941
14942 ; -----
14943 ;**** RegenCopyName -- This function copies the filename from the FCB to
14944 ; SFT and also to DOS local buffers. There was duplicate code in FCBRegen
14945 ; to copy the name to different destinations
14946 ;
14947 ; Inputs: ds:si = source string
14948 ; es:di = destination string
14949 ; cx = length of string
14950 ;
14951 ; Outputs: String copied to destination
14952 ;
14953 ; Registers affected: cx,di,si
14954 ; -----
14955
14956 RegenCopyName:
14957 CopyName:
14958 lodsb ;load character
14959 call UCase ; convert char to upper case
14960
14961 StuffChar2:
14962 STOSB ;store converted character
14963 LOOP CopyName ;
14964 DoneName:
14965 retn
14966
14967 %endif
14968
14969 ; -----
14970 ; 09/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
14971 ; 21/01/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
14972
14973 FCBRegen:
14974 ; called from SFTFromFCB. SS already DOSDATA
14975
14976 ; General data filling. Mode is sf_isFCB + open_for_both, date/time
14977 ; we do not fill, size we do no fill, position we do not fill,
14978 ; bit 14 of flags = TRUE, other bits = FALSE
14979
14980 ;mov al,[si+19h]
14981 0000201A 8A4419 MOV AL,[SI+fcbl_drive]
14982
14983 ; we discriminate based on the first two bits in the reserved field.
14984
14985 ;test al,80h
14986 0000201D A880 test AL,FCBSPECIAL ; check for no sharing test
14987 0000201F 741C JZ short RegenNoSharing ; yes, go regen from no sharing
14988
14989 ; The FCB is for a network or a sharing based system. At this point

```

```

14990 ; we have already closed the SFT for this guy and reconnection is
14991 ; impossible.
14992 ;
14993 ; Remember that he may have given us a FCB with bogus information in
14994 ; it. Check to see if sharing is present or if the redir is present.
14995 ; If either is around, presume that we have cycled out the FCB and
14996 ; give the hard error. Otherwise, just return with carry set.
14997
14998 00002021 E81664 call CheckShare ; test for sharer
14999 00002024 7509 JNZ short RegenFail ; yep, fail this.
15000
15001 ;mov ax,1100h
15002 00002026 B80011 MOV AX,MultNET<<8 ; install check on multnet
15003 00002029 CD2F int 2Fh ; Multiplex - NETWORK REDIRECTOR - INSTALLATION CHECK
15004 ; Return: AL = 00h not installed, OK to install
15005 ; 01h not installed, not OK to install
15006 ; FFh installed
15007 0000202B 08C0 OR AL,AL ; is it there?
15008 0000202D 740C JZ short RegenDead ; no, just fail the operation
15009 RegenFail:
15010 ; 17/05/2019 - Retro DOS v4.0
15011 ;MOV AX,[CS:USER_IN_AX] ; SS override
15012 0000202F 36A1[3A03] mov ax,[SS:USER_IN_AX] ; MSDOS 6.0
15013
15014 ;cmp ah,10h
15015 00002033 80FC10 cmp AH,FCB_CLOSE
15016 00002036 7403 jz short RegenDead
15017 00002038 E89801 call FCBHardErr ; massive hard error.
15018 RegenDead:
15019 0000203B F9 STC ; carry set
15020 FCBRegen_retn:
15021 0000203C C3 retn
15022
15023 ; Local FCB without sharing. Check to see if sharing is loaded. If
15024 ; so fail the operation.
15025
15026 RegenNoSharing:
15027 0000203D E8FA63 call CheckShare ; Sharing around?
15028 00002040 75ED JNZ short RegenFail
15029
15030 ; Find an SFT for this guy.
15031
15032 ; 17/05/2019 - Retro DOS v4.0
15033
15034 ; MSDOS 3.3
15035 ;call LRUFCB
15036 ;jc short FCBRegen_retn
15037
15038 ; MSDOS 6.0
15039 00002042 50 push ax
15040 00002043 B000 mov al,0 ;indicate it is a regen operation
15041 00002045 E8EEFE call LRUFCB
15042 00002048 58 pop ax
15043 00002049 72F1 jc short FCBRegen_retn
15044
15045 ;mov word [es:di+2],8002h
15046 0000204B 26C745020280 word [ES:DI+SF_ENTRY.sf_mode],sf_isFCB+open_for_both+SHARING_COMPAT
15047 00002051 243F AND AL,3Fh ; get drive number for flags
15048 00002053 98 CBW
15049 ;or ax,4000h
15050 00002054 0D0040 OR AX,sf_close_nodate ; normal FCB operation
15051
15052 ; The bits field consists of the upper two bits (dirty and device)
15053 ; from the SFT and the low 4 bits from the open mode.
15054
15055 ;mov cl,[si+1Ah]
15056 00002057 8A4C1A MOV CL,[SI+fcbsl_bits] ; stick in dirty bits.
15057 0000205A 88CD MOV CH,CL
15058 0000205C 80E5C0 AND CH,0C0h ; mask off the dirty/device bits
15059 0000205F 08E8 OR AL,CH
15060 ;and cl,0Fh
15061 00002061 80E10F AND CL,access_mask ; get the mode bits
15062 ;mov [es:di+2],cl
15063 00002064 26884D02 MOV [ES:DI+SF_ENTRY.sf_mode],CL
15064 ;mov [es:di+5],ax
15065 00002068 26894505 MOV [ES:DI+SF_ENTRY.sf_flags],AX ; initial flags
15066 ;MOV AX,[CS:PROC_ID] ; SS override
15067 0000206C 36A1[3C03] mov ax,[ss:PROC_ID] ; MSDOS 6.0
15068 ;mov [es:di+31h],ax
15069 00002070 26894531 MOV [ES:DI+SF_ENTRY.sf_PID],AX
15070 00002074 1E push ds
15071 00002075 56 push si
15072 00002076 06 push es
15073 00002077 57 push di
15074 00002078 16 push ss
15075 00002079 07 pop es
15076 0000207A BF[4B05] MOV DI,NAME1 ; NAME1 is in DOSDATA
15077
15078 0000207D B90800 MOV CX,8
15079 00002080 46 INC SI ; Skip past drive byte to name in FCB
15080
15081 ; MSDOS 3.3
15082 ;RegenCopyName:
15083 ;lodsb
15084 ;call UCASE
15085 ;stosb
15086 ;loop RegenCopyName
15087
15088 ; MSDOS 6.0
15089 00002081 E88C00 call RegenCopyName ;copy the name to NAME1
15090
15091 00002084 16 push ss ; SS is DOSDATA
15092 00002085 1F pop ds
15093
15094 ;mov byte [ATTRIB],16h
15095 00002086 C606[6B05]16 MOV byte [ATTRIB],attr_hidden+attr_system+attr_directory
15096 ; Must set this to something interesting
15097 ; to call DEVNAME.
15098 0000208B E8462D call DEVNAME ; check for device
15099 0000208E 5E pop si
15100 0000208F 07 pop es
15101 00002090 5E pop si
15102 00002091 1F pop ds
15103 00002092 7219 JC short RegenFileNoSharing ; not found on device list => file
15104
15105 ; Device found. We can ignore disk-specific info
15106
15107 ;mov [es:di+5],bh
15108 00002094 26887D05 MOV [ES:DI+SF_ENTRY.sf_flags],BH ; device parms
15109 ;mov byte [es:di+4],0
15110 00002098 26C6450400 MOV byte [ES:DI+SF_ENTRY.sf_attr],0 ; attribute
15111 ; SS override
15112 ; get device driver
15113 0000209D 36C536[9A05] LDS SI,[ss:DEVPT] ; MSDOS 6.0

```

```

15114 regen_save_dpb: ; 26/06/2024
15115 ;mov [es:di+7],si
15116 000020A2 26897507 MOV [ES:DI+SF_ENTRY.sf_devptr],SI
15117 ;mov [es:di+9],ds
15118 000020A6 268C5D09 MOV [ES:DI+SF_ENTRY.sf_devptr+2],DS
15119 000020AA C3 retn ; carry is clear
15120
15121 RegenDeadJ:
15122 000020AB EB8E JMP short RegenDead
15123
15124 ; File found. Just copy in the remaining pieces.
15125
15126 RegenFileNoSharing:
15127 ;mov ax,[es:di+5]
15128 000020AD 268B4505 MOV AX,[ES:DI+SF_ENTRY.sf_flags]
15129 000020B1 83E03F AND AX,03Fh
15130 000020B4 1E push ds
15131 000020B5 56 push si
15132 000020B6 E8075A call FIND_DPB
15133 ;;mov [es:di+7],si
15134 ;MOV [ES:DI+SF_ENTRY.sf_devptr],SI
15135 ;;mov [es:di+9],ds
15136 ;MOV [ES:DI+SF_ENTRY.sf_devptr+2],DS
15137 ; 26/06/2024 (PCDOS 7.1 IBMDOS.COM)
15138 000020B9 E8E6FF call regen_save_dpb
15139 000020BC 5E pop si
15140 000020BD 1F pop ds
15141 000020BE 72EB jc short RegenDeadJ ; if find DPB fails, then drive
15142 ; indicator was bogus
15143 ;mov ax,[si+1Dh]
15144 000020C0 8B441D MOV AX,[SI+fcbl_dirsec]
15145 ;;mov [es:di+1Dh],ax ; MSDOS 3.3
15146 ;mov [es:di+18h],ax ; MSDOS 6.0
15147 000020C3 2689451B MOV [ES:DI+SF_ENTRY.sf_dirsec],AX
15148
15149 ; MSDOS 6.0
15150
15151 ; SR;
15152 ; Extract the read-only and archive bits from the top 2 bits of the sector
15153 ; number
15154
15155 ;mov al,[si+18h]
15156 000020C7 8A4418 mov al,[si+fcbl_sfn]
15157 000020CA 24C0 and al,0C0h ;get the 2 attribute bits
15158 000020CC 88C4 mov ah,al
15159 000020CE D0C4 rol ah,1
15160 000020D0 D0E8 shr al,1
15161 000020D2 08E0 or al,ah
15162 000020D4 243F and al,03Fh ;mask off unused bits
15163 ;mov [es:di+4],al
15164 000020D6 26884504 mov [es:di+SF_ENTRY.sf_attr],al
15165
15166 ; SR;
15167 ; Update the higher word of the directory sector from the FCB
15168
15169 ;;mov al,[si+18h]
15170 000020DA 8A4418 mov al,[si+fcbl_sfn]
15171 000020DD 243F and al,03Fh ;mask off top 2 bits -- attr bits
15172 000020DF 28E4 sub ah,ah
15173 ;mov [es:di+1Dh],ax
15174 000020E1 2689451D mov [es:di+SF_ENTRY.sf_dirsec+2],ax ;update high word
15175
15176 ; 21/01/2024
15177 ; MSDOS 6.0 (& MSDOS 3.3)
15178
15179 000020E5 8B4418 mov ax,[si+18h]
15180 MOV AX,[SI+fcbl_dirsec]
15181 ;;mov [es:di+0Bh],ax
15182 ;MOV [ES:DI+SF_ENTRY.sf_dirsec],AX
15183 ;;
15184 000020E8 2689452B mov [es:di+2Bh],ax ; .sf_chain !!! (MSDOS 6.22)
15185 ;mov [es:di+SF_ENTRY.sf_chain],ax ; first cluster (32 bit) !?
15186 ;;
15187
15188 ;;mov [es:di+1Bh],ax ; MSDOS 3.3
15189 ;mov [es:di+35h],ax ; MSDOS 6.0
15190 000020EC 26894535 MOV [ES:DI+SF_ENTRY.sf_lstclus],AX
15191
15192 ;;
15193 ; 21/01/2024 (PCDOS 7.1 IBMDOS.COM)
15194 000020F0 31C0 xor ax,ax ; 0
15195 000020F2 2689452D mov [es:di+2Dh],ax ; 0
15196 ;mov [es:di+SF_ENTRY.sf_chain+2],ax
15197 ; ; .sf_chain ! (MSDOS 6.22)
15198 ; ; high word of first cluster (32 bit) !?
15199 000020F6 26894537 mov [es:di+37h],ax ; 0
15200 ;mov [es:di+SF_ENTRY.sf_lstclus+2],ax ; hw of last cluster
15201 ;;
15202
15203 ;mov al,[si+1Fh]
15204 000020FA 8A441F MOV AL,[SI+fcbl_dirpos]
15205 ;mov [es:di+1Fh],al
15206 000020FD 2688451F MOV [ES:DI+SF_ENTRY.sf_dirpos],AL
15207 ;INC word [ES:DI+SF_ENTRY.sf_ref_count]
15208 00002101 26FF05 inc word [ES:DI] ; Increment reference count.
15209 ; Existing FCB entries would be
15210 ; flushed unnecessarily because of
15211 ; check in CheckFCB of the ref_count.
15212 ; July 22/85 - BAS
15213 ;;
15214 ; 21/01/2024 (PCDOS 7.1 IBMDOS.COM)
15215 00002104 E81129 call set_sftfcb_entry ; put SFT entry number in the SFTFCB table
15216 ; as FCB index number
15217 ;;
15218
15219 ;lea si,[si+1]
15220 00002107 8D7401 LEA SI,[SI+SYS_FCB.name]
15221 ;lea di,[di+20h]
15222 0000210A 8D7D20 LEA DI,[DI+SF_ENTRY.sf_name]
15223 ;mov cx,11
15224 0000210D B90B00 MOV CX,SYS_FCB.EXTENT-SYS_FCB.name ; 12-1
15225
15226 ; 26/06/2024
15227 ; MSDOS 6.0
15228 ;call RegenCopyName ;copy name to SFT
15229 ; 26/06/2024
15230 ; cf = 0 (at the result of the 'test' instruction)
15231
15232 ; MSDOS 3.3
15233 ;RegenCopyName2:
15234 ;lodsb
15235 ;call UCASE
15236 ;stosb
15237 ;loop RegenCopyName2

```

```

15238
15239 ; 26/06/2024
15240 ; cf = 0
15241 ; clc
15242 ; retn
15243
15244 ; 26/06/2024
15245 %if 1
15246
15247 ; -----
15248 ; *** RegenCopyName -- This function copies the filename from the FCB to
15249 ; SFT and also to DOS local buffers. There was duplicate code in FCBRegen
15250 ; to copy the name to different destinations
15251 ;
15252 ; Inputs: ds:si = source string
15253 ; es:di = destination string
15254 ; cx = length of string
15255 ;
15256 ; Outputs: String copied to destination
15257 ;
15258 ; Registers affected: cx,di,si
15259 ; -----
15260
15261 RegenCopyName:
15262 CopyName:
15263     lodsb ; load character
15264     call UCase ; * ; convert char to upper case
15265 StuffChar2:
15266     STOSB ; store converted character
15267     LOOP CopyName ;
15268     ; 26/06/2024
15269     ; cf= 0 ; *
15270 DoneName:
15271     retn
15272
15273 %endif
15274
15275 ; 17/05/2019 - Retro DOS v4.0
15276 ; 21/01/2024 - Retro DOS v5.0
15277
15278 ; ** BlastSFT - Fill SFT with Garbage
15279 ; -----
15280 ; BlastSFT is used when an SFT is no longer needed; it's called with
15281 ; various garbage values to put into the SFT. I don't know why,
15282 ; presumably to help with debugging (jgl). We clear the few fields
15283 ; necessary to show that the SFT is free after filling it.
15284 ;
15285 ; ENTRY (es:di) = address of SFT
15286 ; (al) = fill character
15287 ; EXIT (ax) = -1
15288 ; 'C' clear
15289 ; USES AX, CX, Flags
15290
15291 BlastSFT:
15292 ; 21/01/2024 (PCDOS 7.1 IBMDOS.COM)
15293 ;;;
15294 call SFT_FREE
15295 ;;;
15296 push di
15297 ;mov cx,53 ; MSDOS 3.3
15298 ;mov cx,59 ; MSDOS 6.0
15299 mov cx,SF_ENTRY.size
15300 rep stosb
15301 pop di
15302 sub ax,ax ; 0 ; clear 'C'-----;
15303 mov [es:di],ax
15304 ;mov [es:di+SF_ENTRY.sf_ref_count],ax ; set ref count ;
15305 ;mov [es:di+15h],ax
15306 mov [es:di+sf_LRU],ax ; set lru ;
15307 dec ax ; -1 ;
15308 ;mov [es:di+17h],ax ; 0FFFFh ; -1
15309 mov [es:di+sf_OpenAge],ax ; set open age to -1 ;
15310 BlastSFT_retn:
15311 retn ; return with 'C' clear ;
15312
15313 ;Break <CheckFCB - see if the SFT pointed to by the FCB is still OK>
15314 ; -----
15315 ;
15316 ; CheckFCB - examine an FCB and its contents to see if it needs to be
15317 ; regenerated.
15318 ;
15319 ; Inputs: DS:SI point to FCB (not extended)
15320 ; AL is SFT index
15321 ; Outputs: Carry Set - FCB needs to be regened
15322 ; Carry clear - FCB is OK. ES:DI point to SFT
15323 ; Registers modified: AX and BX
15324 ; -----
15325 ;
15326 ; 09/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
15327 ; DOSCODE:59F0h (MSDOS 5.0, MSDOS.SYS)
15328 ;
15329 ; 21/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
15330 ; DOSCODE:607Eh (PCDOS 7.1 IBMDOS.COM)
15331 CheckFCB:
15332 ; called from $fcb_open and sftfromfcb. SS already set up to DOSDATA
15333 ;
15334 ; MSDOS 3.3
15335 ;
15336 ; LES DI,[CS:SFTFCB]
15337 ;
15338 ; MSDOS 6.0
15339 ;
15340 ; SR;
15341 ; We check if the given FCB is for a local file. If so, we return a
15342 ; bad SFT status forcing the caller to regenerate the SFT.
15343 ;
15344 ;test byte [si+19h],0C0h
15345 test byte [si+fcb_l_drive],FCBNETWORK|FCBSHARE|FCBDEVICE
15346 jz short BadSFT ; Local file, return bad SFT
15347 LES DI,[SS:SFTFCB] ; SS override
15348
15349 ; MSDOS 6.0 (& MSDOS 3.3)
15350 ;cmp [es:di+4],al
15351 CMP [ES:DI+SFT.SFCount],AL
15352 JC short BadSFT
15353 ;mov bl,53 ; MSDOS 3.3
15354 ;mov bl,59 ; MSDOS 6.0
15355 MOV BL,SF_ENTRY.size
15356 MUL BL
15357 ;lea di,[di+6]
15358 LEA DI,[DI+SFT.SFTable]
15359 ADD DI,AX
15360 ;MOV AX,[CS:PROC_ID] ; MSDOS 3.3
15361

```

```

15362 0000214B 36A1[3C03]      mov     ax,[SS:PROC_ID] ; MSDOS 6.0 ; SS override
15363                          ;cmp     [es:di+31h],ax
15364 0000214F 26394531      cmp     [ES:DI+SF_ENTRY.sf_PID],AX
15365 00002153 7546          jnz     short BadSFT ; must match process
15366 00002155 26833D00      cmp     word [es:di],0
15367                          ;CMP     word [ES:DI+SF_ENTRY.sf_ref_count],0
15368 00002159 7440          jz      short BadSFT ; must also be in use
15369                          ;mov     al,[si+19h]
15370 0000215B 8A4419      mov     AL,[SI+fcb_l_drive]
15371                          ;test    al,80h
15372 0000215E A880          test    AL,FCBSPECIAL ; a special FCB?
15373 00002160 7427          jz      short CheckNoShare ; No. try local or device
15374
15375                          ; Since we are a special FCB, try NOT to use a bogus test instruction.
15376                          ; FCBSHARE is a superset of FCBNETWORK.
15377
15378 00002162 50          PUSH     AX
15379                          ;and     al,0C0h
15380 00002163 24C0      AND      AL,FCBMASK
15381                          ;cmp     al,0C0h
15382 00002165 3CC0      CMP      AL,FCBSHARE ; net FCB?
15383 00002167 58          POP      AX
15384 00002168 7515      JNZ      short CheckNet ; no
15385                          ; yes
15386
15387 ;----- In share support -----
15388 ;
15389 ; call far [cs:JShare+(11*4)]
15390 0000216A 36FF1E[BC00] call     far [ss:JShare+(11*4)] ; 11 = Shchk ; SS override
15391 0000216F 722A      JC      short BadSFT
15392
15393 ; 21/01/2024
15394 %if 0
15395     JMP     SHORT CheckD
15396
15397 ;----- End in share support -----
15398 ;
15399 ; 09/11/2022
15400 ; (There is not any procedure/sub
15401 ; which calls or jumps to CheckFirClus here)
15402 ;;;
15403 CheckFirClus:
15404 ;cmp     bx,[es:di+0Bh]
15405 ; 07/12/2022
15406 CMP      BX,[ES:DI+SF_ENTRY.sf_firclus]
15407 JNZ      short BadSFT
15408 ;;;
15409 %endif
15410
15411 CheckD:
15412 AND      AL,3Fh
15413 ;mov     ah,[es:di+5]
15414 00002173 268A6505 MOV     AH,[ES:DI+SF_ENTRY.sf_flags]
15415 00002177 80E43F      AND      AH,3Fh
15416 0000217A 38C4      CMP      AH,AL
15417 ; 26/06/2024
15418 ; 16/12/2022
15419 ;jz      short BlastSFT_retn ; carry is clear
15420 ; 09/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
15421 0000217C 751D      jnz     short BadSFT
15422
15423 CheckD_retn:
15424     retn
15425
15426 ; 26/06/2024
15427 ;BadSFT:
15428 ; STC
15429 ; retn
15430
15431 CheckNet:
15432 ; 17/05/2019 - Retro DOS v4.0
15433
15434 ;----- In net support -----
15435 ; MSDOS 3.3
15436 ;;mov     ax,[si+1Ah]
15437 ;;mov     ax,[si+fcb_net_handle]
15438 ;;cmp     ax,[es:di+1Dh]
15439 ;;cmp     ax,[ES:DI+SF_ENTRY.sf_dirsec]
15440 ;;jnz     short BadSFT
15441 ;;cmp     ax,[es:di+19h]
15442 ;;cmp     ax,[ES:DI+sf_netid]
15443 ;;jnz     short BadSFT
15444 ;;mov     ax,[si+1Eh]
15445 ;;mov     ax,[si+fcb_l_attr]
15446 ;;cmp     ax,[es:di+1Bh]
15447 ;;cmp     ax,[es:di+SF_ENTRY.sf_lstclus]
15448 ;;jnz     short BadSFT
15449
15450 ; MSDOS 6.0
15451 ;mov     ax,[si+1Ch]
15452 0000217F 8B441C      MOV     AX,[SI+fcb_netID] ;AN000;IFS.DOS 4.00
15453 ; 09/11/2022
15454 ;cmp     ax,[es:di+0Bh]
15455 00002182 263B450B CMP     AX,[ES:DI+sf_serial_ID] ;AN000;IFS.DOS 4.00
15456 00002186 7513      JNZ     short BadSFT
15457
15458 ;----- END In net support -----
15459
15460 CheckNet_retn:
15461     retn
15462
15463 CheckNoShare:
15464
15465 ; 16/12/2022
15466 ; 09/11/2022 (following test instruction is nonsense!)
15467 ; (I am leaving it here for MSDOS 5.0 MSDOS.SYS compatibility)
15468 ; test al,40h
15469 ; test AL,FCBDEVICE ; Device?
15470 ; jnz short $+2 ; 09/11/2022
15471 ; JNZ short CheckNoShareDev ; Yes
15472
15473 ; MSDOS 3.3 - IBMDOS.COM - Offset 27EFh
15474 ;;mov     bx,[si+1Dh]
15475 ;;MOV     BX,[SI+fcb_ns_dirsec]
15476 ;;cmp     bx,[es:di+1Dh]
15477 ;;cmp     bx,[ES:DI+SF_ENTRY.sf_dirsec]
15478 ;;jnz     short BadSFT
15479 ;;mov     bl,[si+1Fh]
15480 ;;MOV     bl,[SI+fcb_ns_dirpos]
15481 ;;cmp     bl,[es:di+1Fh]
15482 ;;cmp     bl,[ES:DI+SF_ENTRY.sf_dirpos]
15483 ;;jnz     short BadSFT
15484 ;;mov     bl,[si+1Ah]
15485 ;;MOV     bl,[SI+fcb_ns_bits]

```

```

15486      ;mov bh,[es:di+5]
15487      ;MOV bh,[ES:DI+SF_ENTRY.sf_flags]
15488      ;xor bh,b1
15489      ;and bh,0C0h
15490      ;jnz short BadSFT
15491      ;xor b1,[es:di+2]
15492      ;xor b1,[ES:DI+SF_ENTRY.sf_mode]
15493      ;and b1,0Fh
15494      ;jnz short BadSFT
15495      ;push di
15496      ;push si
15497      ;lea di,[di+20h] ; MSDOS 3.3
15498      ;LEA DI,[DI+SF_ENTRY.sf_name]
15499      ;lea si,[si+1]
15500      ;LEA SI,[SI+SYS_FCB.name]
15501      ;mov cx,11
15502      ;MOV CX,SYS_FCB.EXTENT-SYS_FCB.name ; 12-1
15503      ;repe cmpsb
15504      ;pop si
15505      ;pop di
15506      ;jnz short BadSFT
15507      ;mov bx,[si+1Bh]
15508      ;MOV bx,[SI+fcf_nsl_firclus]
15509      ;jmp short CheckFirClus
15510
15511      ; MSDOS 6.0
15512
15513      ; SR;
15514      ; The code below to match a local FCB with its SFT can no longer be
15515      ; used. We just return a no-match status. This check is done right
15516      ; at the top.
15517
15518      CheckNoShareDev:
15519      ;mov bx,[si+1Ah]
15520      MOV BX,[SI+fcf_nsl_drvptr]
15521      ;cmp bx,[es:di+7]
15522      CMP BX,[ES:DI+SF_ENTRY.sf_devptr]
15523      JNZ short BadSFT
15524      ;mov bx,[si+1Ch]
15525      MOV BX,[SI+fcf_nsl_drvptr+2]
15526      ;cmp bx,[es:di+9]
15527      CMP BX,[ES:DI+SF_ENTRY.sf_devptr+2]
15528      ;JNZ short BadSFT
15529      ;JMP short CheckD
15530      ; 26/06/2024
15531      jz short CheckD
15532
15533      ; 26/06/2024
15534      BadSFT:
15535      STC
15536      retn
15537
15538      ;Break <SFTFromFCB - take a FCB and obtain a SFT from it>
15539      ;-----
15540      ;
15541      ; SFTFromFCB - the workhorse of this compatability crap. Check to see if
15542      ; the SFT for the FCB is Good. If so, make ThisSFT point to it. If not
15543      ; good, get one from the cache and regenerate it. Overlay the LRU field
15544      ; with PID
15545      ;
15546      ; Inputs: DS:SI point to FCB
15547      ; Outputs: ThisSFT point to appropriate SFT
15548      ; Carry clear -> OK ES:DI -> SFT
15549      ; Carry set -> error in ax
15550      ; Registers modified: ES,DI, AX
15551      ;
15552      ;-----
15553
15554      SFTFromFCB:
15555      ; called from fcbio and $fcb_close. SS already set up to DOSDATA
15556
15557      ; 17/05/2019 - Retro DOS v4.0
15558
15559      0000219D 50      push ax
15560      0000219E 53      push bx
15561      ;mov al,[si+18h]
15562      0000219F 8A4418  MOV AL,[SI+fcf_sfn] ; set SFN for check
15563      000021A2 E88CFF  call CheckFCB
15564      000021A5 5B      pop bx
15565      000021A6 58      pop ax
15566      ;MOV [CS:THISSFT],DI ; SS override
15567      ;MOV [CS:THISSFT+2],ES ; SS override
15568      000021A7 36893E[9E05] MOV [SS:THISSFT],DI ; SS override
15569      000021AC 368C06[A005] MOV [SS:THISSFT+2],ES ; SS override
15570      000021B1 7311      JNC short Set_SFT ; no problems, just set thissft
15571
15572      ; 09/11/2022 (MSDOS 5.0)
15573      ; 31/05/2019
15574      000021B3 06      push es ; * (MSDOS 6.21) & (MSDOS 5.0)
15575      000021B4 E8A1E2  call save_world
15576      000021B7 E860FE  call FCBRegen
15577      000021BA E884E2  call restore_world ; use macro restore world
15578      000021BD 07      pop es ; * (MSDOS 6.21) ; 31/05/2019 ; 09/11/2022 (MSDOS 5.0)
15579
15580      ;MOV AX,[CS:EXTERR] ; SS override
15581      000021BE 36A1[2403] MOV AX,[SS:EXTERR] ; SS override
15582      000021C2 72C4      jc short CheckNet_retn
15583
15584      Set_SFT:
15585      ;LES DI,[CS:THISSFT] ; SS override for THISSFT & PROC_ID
15586      000021C4 36C43E[9E05] les di,[ss:THISSFT]
15587      ;PUSH word [CS:PROC_ID] ; set process id
15588      000021C9 36FF36[3C03] push word [cs:PROC_ID]
15589      ;pop word [es:di+31h]
15590      000021CE 268F4531 POP word [ES:DI+SF_ENTRY.sf_PID]
15591      000021D2 C3      retn ; carry is clear
15592
15593      ;Break <FCBHardErr - generate INT 24 for hard errors on FCBS>
15594      ;-----
15595      ;
15596      ; FCBHardErr - signal to a user app that he is trying to use an
15597      ; unavailable FCB.
15598      ;
15599      ; Inputs: none.
15600      ; Outputs: none.
15601      ; Registers modified: all
15602      ;
15603      ;-----
15604
15605      ; 21/01/2024 - Retro DOS v5.0
15606      FCBHardErr:
15607      ; 17/05/2019 - Retro DOS v4.0
15608      000021D3 2E8E06[0700] mov es,[cs:DosDSeg]
15609

```



```

15610             ;mov     ax,23h
15611 000021D8 B82300 MOV     AX,error_FCB_unavailable
15612             ;;mov    byte [cs:ALLOWED],8
15613             ;MOV     byte [CS:ALLOWED],Allowed_FAIL
15614 000021DB 26C606[4B03]08 mov     byte [es:ALLOWED],Allowed_FAIL
15615
15616             ;LES     BP,[CS:THISDPB]
15617 000021E1 26C42E[8A05] les     bp,[es:THISDPB]
15618
15619 000021E6 BF0100 MOV     DI,1             ; Fake some registers
15620 000021E9 89F9 MOV     CX,DI
15621             ;;
15622             ; 21/01/2024 (PCDOS 7.1 IBMDOS.COM)
15623 000021EB 31D2 xor     dx,dx ; 0
15624             ;cmp     [es:bp+0Fh],dx
15625 000021ED 2639560F cmp     [es:bp+DPB.FAT_SIZE],dx ; 0
15626 000021F1 740B jz      short fcbharderr_fat32 ; FAT32
15627 000021F3 268916[0706] mov     [es:HIGH_SECTOR],dx ; 0
15628             ;mov     dx,[es:bp+0Bh]
15629 000021F8 268B560B mov     dx,[es:bp+DPB.FIRST_SECTOR]
15630 000021FC EB0D jmp     short fcbharderr_fat
15631 fcbharderr_fat32:
15632             ;mov     dx,[es:bp+2Bh]
15633 000021FE 268B562B mov     dx,[es:bp+DPB.FCLUS_FSECTOR+2]
15634 00002202 268916[0706] mov     [es:HIGH_SECTOR],dx
15635             ;mov     dx,[es:bp+29h]
15636 00002207 268B5629 mov     dx,[es:bp+DPB.FCLUS_FSECTOR]
15637 fcbharderr_fat:
15638             ;;
15639             ; 21/01/2024
15640             ;;mov     dx,[es:bp+0Bh]
15641             ;MOV     DX,[ES:BP+DPB.FIRST_SECTOR]
15642
15643 0000220B E8463E call    HARDERR
15644 0000220E F9 stc
15645 0000220F C3 retn
15646
15647 ;=====
15648 ; FCBI02.ASM, MSDOS 6.0, 1991
15649 ;=====
15650 ; 21/07/2018 - Retro DOS v3.0
15651 ; 17/05/2019 - Retro DOS v4.0
15652
15653 ;** FCBI02.ASM - Ancient 1.0 1.1 FCB system calls
15654 ;
15655 ; GetRR
15656 ; GetExtent
15657 ; SetExtent
15658 ; GetExtended
15659 ; GetRecSize
15660 ; FCBIO
15661 ; $FCB_OPEN
15662 ; $FCB_CREATE
15663 ; $FCB_RANDOM_WRITE_BLOCK
15664 ; $FCB_RANDOM_READ_BLOCK
15665 ; $FCB_SEQ_READ
15666 ; $FCB_SEQ_WRITE
15667 ; $FCB_RANDOM_READ
15668 ; $FCB_RANDOM_WRITE
15669 ;
15670 ; Revision history:
15671 ;
15672 ; Created: ARR 4 April 1983
15673 ; MZ 6 June 1983 completion of functions
15674 ; MZ 15 Dec 1983 Brain damaged programs close FCBs multiple
15675 ; times. Change so successive closes work by
15676 ; always returning OK.Also, detect I/O to
15677 ; already closed FCB and return EOF.
15678 ; MZ 16 Jan 1984 More braindamage. Need to separate info
15679 ; out of sft into FCB for reconnection
15680 ;
15681 ; A000 version 4.00 Jan. 1988
15682 ;
15683 ; Defintions for FCBop flags
15684 ;
15685 RANDOM equ 2 ; random operation
15686 FCBREAD equ 4 ; doing a read
15687 BLOCK equ 8 ; doing a block I/O
15688
15689 ;Break <GetRR - return the random record field in DX:AX>
15690 ;-----
15691 ;
15692 ; GetRR - correctly load DX:AX with the random record field (3 or 4 bytes)
15693 ; from the FCB pointed to by DS:SI
15694 ;
15695 ; Inputs: DS:SI point to an FCB
15696 ; BX has record size
15697 ; Outputs: DX:AX contain the contents of the random record field
15698 ; Registers modified: none
15699 ;-----
15700 ;
15701 GetRR:
15702 ;mov     ax,[si+21h]
15703 00002210 8B4421 MOV     AX,[SI+SYS_FCB.RR] ; get low order part
15704             ;mov     dx,[si+23h]
15705 00002213 8B5423 MOV     DX,[SI+SYS_FCB.RR+2] ; get high order part
15706 00002216 83FB40 CMP     BX,64 ; ignore MSB of RR if recsiz > 64
15707 00002219 7202 JB      short GetRRBye
15708 GetExtent_bye: ; 21/01/2024
15709 0000221B 30F6 XOR     DH,DH
15710 GetRRBye:
15711 0000221D C3 retn
15712
15713 ;Break <GetExtent - retrieve next location for sequential IO>
15714 ;-----
15715 ;
15716 ; GetExtent - Construct the next record to perform I/O from the EXTENT and
15717 ; NR fields in the FCB.
15718 ;
15719 ; Inputs: DS:SI - point to FCB
15720 ; Outputs: DX:AX contain the contents of the random record field
15721 ; Registers modified: none
15722 ;-----
15723 ;
15724 GetExtent:
15725 ;mov     al,[si+20h]
15726 0000221E 8A4420 MOV     AL,[SI+SYS_FCB.NR] ; get low order piece
15727             ;mov     dx,[si+0Ch]
15728 00002221 8B540C MOV     DX,[SI+SYS_FCB.EXTENT]; get high order piece
15729 00002224 D0E0 SHL     AL,1
15730 00002226 D1EA SHR     DX,1
15731 00002228 D0D8 RCR     AL,1 ; move low order bit of DL to high order of AH
15732 0000222A 88D4 MOV     AH,DL
15733 0000222C 88F2 MOV     DL,DH

```

```

15734      ; 21/01/2024 (PCDOS 7.1 IBMDOS.COM)
15735      ;XOR    DH,DH
15736      ;retn
15737 0000222E EBEB      jmp     short GetExtent_bye
15738
15739 ;Break <SetExtent - update the extent/NR field>
15740 ;-----
15741 ;
15742 ;   SetExtent - change the position of an FCB by filling in the extent/NR
15743 ;   fields
15744 ;
15745 ;   Inputs: DS:SI point to FCB
15746 ;           DX:AX is a record location in file
15747 ;   Outputs:      Extent/NR fields are filled in
15748 ;   Registers modified: CX
15749 ;-----
15750
15751 SetExtent:
15752      push    ax
15753      push    dx
15754      MOV     CX,AX
15755      AND     AL,7FH          ; next rec field
15756      ;mov     [si+20h],al
15757      MOV     [SI+SYS_FCB.NR],AL
15758      AND     CL,80H          ; save upper bit
15759      SHL     CX,1
15760      RCL     DX,1            ; move high bit of CX to low bit of DX
15761      MOV     AL,CH
15762      MOV     AH,DL
15763      ;mov     [si+0Ch], ax
15764      MOV     [SI+SYS_FCB.EXTENT],AX; all done
15765      pop     dx
15766      pop     ax
15767      retn
15768
15769 ;Break <GetExtended - find FCB in potential extended fcb>
15770 ;-----
15771 ;
15772 ;   GetExtended - Make DS:SI point to FCB from DS:DX
15773 ;
15774 ;   Inputs: DS:DX point to a possible extended FCB
15775 ;   Outputs:      DS:SI point to the FCB part
15776 ;               zeroflag set if not extended fcb
15777 ;   Registers modified: SI
15778 ;-----
15779
15780 GetExtended:
15781      MOV     SI,DX           ; point to Something
15782      CMP     BYTE [SI],-1    ; look for extension
15783      JNZ     short GetBye    ; not there
15784      ADD     SI,7            ; point to FCB
15785
15786 GetBye:
15787      CMP     SI,DX           ; set condition codes
15788      getextd_retn:
15789      retn
15790
15791 ;Break <GetRecSize - return in BX the FCB record size>
15792 ;-----
15793 ;
15794 ;   GetRecSize - return in BX the record size from the FCB at DS:SI
15795 ;
15796 ;   Inputs: DS:SI point to a non-extended FCB
15797 ;   Outputs:      BX contains the record size
15798 ;   Registers modified: None
15799 ;-----
15800
15801      ; 22/01/2024
15802      ; 07/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
15803 GetRecSize:
15804      ;mov     bx,[si+0Eh]
15805      MOV     BX,[SI+SYS_FCB.RECSIZ]; get his record size
15806      OR      BX,BX           ; is it nul?
15807      jz      short getextd_retn
15808      ; 22/01/2024 (BugFix)
15809      jnz     short getextd_retn
15810      ;MOV     BX,128          ; use default size
15811      mov     bl,128 ; (PCDOS 7.1 IBMDOS.COM)
15812      ;mov     [si+0Eh],bx
15813      MOV     [SI+SYS_FCB.RECSIZ],BX; stuff it back
15814      retn
15815
15816 ; 23/01/2024 - Retro DOS v5.0
15817 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:61B3h
15818
15819 ; 22/07/2018 - Retro DOS v3.0
15820
15821 ;BREAK <$FCB_Random_write_Block - write a block of records to a file >
15822 ;-----
15823 ;
15824 ;   $FCB_Random_write_Block - retrieve a location from the FCB, seek to it
15825 ;   and write a number of blocks from it.
15826 ;
15827 ;   Inputs: DS:DX point to an FCB
15828 ;   Outputs:      AL = 0 write was successful and the FCB position is updated
15829 ;               AL <> 0 Not enough room on disk for the output
15830 ;-----
15831
15832 _$FCB_RANDOM_WRITE_BLOCK:
15833      ;mov     AL,0Ah
15834      MOV     AL,RANDOM+BLOCK
15835      JMP     short FCBIO      ; 23/01/2024
15836
15837 ;BREAK <$FCB_Random_Read_Block - read a block of records to a file >
15838 ;-----
15839 ;
15840 ;   $FCB_Random_Read_Block - retrieve a location from the FCB, seek to it
15841 ;   and read a number of blocks from it.
15842 ;
15843 ;   Inputs: DS:DX point to an FCB
15844 ;   Outputs:      AL = error codes defined above
15845 ;-----
15846
15847 _$FCB_RANDOM_READ_BLOCK:
15848      ;mov     AL,0Eh
15849      MOV     AL,RANDOM+FCBREAD+BLOCK
15850      JMP     short FCBIO      ; 23/01/2024
15851
15852 ;BREAK <$FCB_Seq_Read - read the next record from a file >
15853 ;-----
15854 ;
15855 ;   $FCB_Seq_Read - retrieve the next record from an FCB and read it into
15856 ;   memory
15857

```



```

15858 ;
15859 ; Inputs: DS:DX point to an FCB
15860 ; Outputs: AL = error codes defined above
15861 ;
15862 ;-----
15863
15864 _$FCB_SEQ_READ:
15865 ;mov AL,4
15866 0000226C B004 MOV AL,FCBREAD
15867 0000226E EB0A JMP short FCBIO ; 23/01/2024
15868
15869 ;BREAK <$FCB_Seq_Write - write the next record to a file >
15870 ;-----
15871 ;
15872 ; $FCB_Seq_Write - retrieve the next record from an FCB and write it to the
15873 ; file
15874 ;
15875 ; Inputs: DS:DX point to an FCB
15876 ; Outputs: AL = error codes defined above
15877 ;
15878 ;-----
15879
15880 _$FCB_SEQ_WRITE:
15881 00002270 B000 MOV AL,0
15882 00002272 EB06 JMP short FCBIO ; 23/01/2024
15883
15884 ;BREAK <$FCB_Random_Read - Read a single record from a file >
15885 ;-----
15886 ;
15887 ; $FCB_Random_Read - retrieve a location from the FCB, seek to it and read a
15888 ; record from it.
15889 ;
15890 ; Inputs: DS:DX point to an FCB
15891 ; Outputs: AL = error codes defined above
15892 ;
15893 ;-----
15894
15895 _$FCB_RANDOM_READ:
15896 ;mov AL,6
15897 00002274 B006 MOV AL,RANDOM+FCBREAD
15898 ; 23/01/2024
15899 ;jmp FCBIO ; single block
15900 00002276 EB02 jmp short FCBIO
15901
15902 ;BREAK <$FCB_Random_Write - write a single record to a file >
15903 ;-----
15904 ;
15905 ; $FCB_Random_Write - retrieve a location from the FCB, seek to it and write
15906 ; a record to it.
15907 ;
15908 ; Inputs: DS:DX point to an FCB
15909 ; Outputs: AL = error codes defined above
15910 ;
15911 ;-----
15912
15913 _$FCB_RANDOM_WRITE:
15914 ;mov AL,2
15915 00002278 B002 MOV AL,RANDOM
15916 ; 23/01/2024
15917 ;;jmp FCBIO
15918 ;jmp short FCBIO
15919
15920 ;BREAK <FCBIO - do internal FCB I/O>
15921 ;-----
15922 ;
15923 ; FCBIO - look at FCBOP and merge all FCB operations into a single routine.
15924 ;
15925 ; Inputs: FCBOP flags which operations need to be performed
15926 ; DS:DX point to FCB
15927 ; CX may have count of number of records to xfer
15928 ; Outputs: AL has error code
15929 ; Registers modified: all
15930 ;-----
15931
15932 ; 09/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
15933 ; DOSCODE:5B17h (MSDOS 5.0 MSDOS.SYS)
15934
15935 ; 23/01/2024
15936 ; DOSCODE:5B2Bh (MSDOS 6.22 MSDOS.SYS)
15937
15938 ; 23/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
15939 ; DOSCODE:61C9h (PCDOS 7.1 IBMDOS.COM)
15940
15941 FCBIO:
15942
15943 FE0FEQU 1
15944 FTRIM EQU 2
15945
15946 %define FCBErr byte [bp-1] ; byte
15947 %define cRec word [bp-3] ; word
15948 ;%define RecPos word [bp-7] ; dword
15949 %define RecPosL word [bp-7] ; word
15950 %define RecPosH word [bp-5] ; word
15951 %define RecSize word [bp-9] ; word
15952 ;%define bPos word [bp-13] ; dword
15953 %define bPosL word [bp-13] ; word
15954 %define bPosH word [bp-11] ; word
15955 %define cByte word [bp-15] ; word
15956 %define cResult word [bp-17] ; word
15957 %define cRecRes word [bp-19] ; word
15958 %define FCBOp byte [bp-20] ; byte
15959 ; 23/01/2024
15960 %define bPos bp-13
15961
15962 ;Enter
15963
15964 0000227A 55 push bp
15965 0000227B 89E5 mov bp,sp
15966 0000227D 83EC14 sub sp,20
15967 ;mov [bp-20],al
15968 00002280 8846EC MOV FCBOp,AL
15969 ;mov byte [bp-1],0
15970 00002283 C646FF00 MOV FCBErr,0 ; FCBErr = 0;
15971 00002287 E8C0FF call GetExtended ; FCB = GetExtended ();
15972 ;test byte [bp-20],8
15973 0000228A F646EC08 TEST FCBOp,BLOCK ; if ((OP&BLOCK) == 0)
15974 0000228E 7503 JNZ short GetPos
15975 00002290 B90100 MOV CX,1 ; cRec = 1;
15976
15977 GetPos:
15978 ;mov [bp-3],cx
15979 00002293 894EFD MOV cRec,CX ;*Tail coalesce
15980 00002296 E885FF call GetExtent ; RecPos = GetExtent ();
15981 00002299 E8BBFF call GetRecSize ; RecSize = GetRecSize ();
;mov [bp-9],bx

```

```

15982 0000229C 895EF7      MOV     RecSize,BX
15983                ;test  byte [bp-20],2
15984 0000229F F646EC02    TEST    FCBOp,RANDOM      ; if ((OP&RANDOM) <> 0)
15985 000022A3 7403        JZ      short GetRec
15986 000022A5 E868FF      call   GetRR              ; RecPos = GetRR ();
15987 GetRec:
15988                ;mov    [bp-7],ax
15989 000022A8 8946F9      MOV     RecPosL,AX        ;*Tail coalesce
15990                ;mov    [bp-5],dx
15991 000022AB 8956FB      MOV     RecPosH,DX
15992 000022AE E87FFF      call   SetExtent         ; SetExtent (RecPos);
15993                ;mov    ax,[bp-5]
15994 000022B1 8B46FB      MOV     AX,RecPosH        ; bPos = RecPos * RecSize;
15995 000022B4 F7E3        MUL     BX
15996 000022B6 89C7        MOV     DI,AX
15997                ;mov    ax,[bp-7]
15998 000022B8 8B46F9      MOV     AX,RecPosL
15999 000022BB F7E3        MUL     BX
16000 000022BD 01FA      ADD     DX,DI
16001                ;mov    [bp-13],ax
16002 000022BF 8946F3      MOV     bPosL,AX
16003                ;mov    [bp-11],dx
16004 000022C2 8956F5      MOV     bPosH,DX
16005                ;mov    ax,[bp-3]
16006 000022C5 8B46FD      MOV     AX,cRec           ; cByte = cRec * RecSize;
16007 000022C8 F7E3        MUL     BX
16008                ;mov    [bp-15],ax
16009 000022CA 8946F1      MOV     cByte,AX
16010
16011                ;hkn;    SS override
16012 000022CD 360306[2C03]  ADD     AX,[SS:DMAADD]    ; if (cByte+DMA > 64K) {
16013 000022D2 83D200      ADC     DX,0
16014 000022D5 7419        JZ      short DoOper
16015                ;mov    byte [bp-1],2
16016 000022D7 C646FF02    MOV     FCBErr,FTRIM     ; FCBErr = FTRIM;
16017
16018                ;hkn;    SS override
16019 000022DB 36A1[2C03]  MOV     AX,[SS:DMAADD]    ; cRec = (64K-DMA)/RecSize;
16020 000022DF F7D8        NEG     AX
16021 000022E1 7501        JNZ     short DoDiv
16022 000022E3 48          DEC     AX
16023 DoDiv:
16024 000022E4 31D2      XOR     DX,DX
16025 000022E6 F7F3      DIV     BX
16026                ;mov    [bp-3],ax
16027 000022E8 8946FD      MOV     cRec,AX
16028 000022EB F7E3      MUL     BX                ; cByte = cRec * RecSize;
16029                ;mov    [bp-15],ax
16030 000022ED 8946F1      MOV     cByte,AX         ; }
16031
16032 000022F0 31DB      XOR     BX,BX
16033                ;mov    [bp-17],bx
16034 000022F2 895EEF      MOV     cResult,BX       ; cResult = 0;
16035                ;cmp    [bp-15],bx
16036 000022F5 395EF1      CMP     cByte,BX         ; if (cByte <> 0 ||
16037 000022F8 7506        JNZ     short DoGetExt
16038                ;test  byte [bp-1],2
16039 000022FA F646FF02    TEST    FCBErr,FTRIM     ; (FCBErr&FTRIM) == 0) {
16040                ;JZ      short DoGetExt
16041                ;JMP     short SkipOp
16042                ; 16/12/2022
16043                jnz     short SkipOp
16044                ; 09/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
16045                ;JZ      short DoGetExt
16046                ;JMP     short SkipOp
16047 DoGetExt:
16048 00002300 E89AFE      call   SFTFromFCB        ; if (!SFTFromFCB (SFT,FCB))
16049 00002303 730F      JNC     short ContinueOp
16050 FCBDeath:
16051 00002305 E885E3      call   FCB_RET_ERR       ; signal error, map for extended
16052                ;mov    word [bp-19],0
16053 00002308 C746ED0000  MOV     cRecRes,0        ; no bytes transferred
16054                ;mov    byte [bp-1],1
16055 0000230D C646FF01    MOV     FCBErr,FE0F      ; return FTRIM;
16056 00002311 E9E700      JMP     FCBSave          ; bam!
16057 ContinueOp:
16058                ; 23/01/2024
16059                ; (PCDOS 7.1 IBMDOS.COM)
16060                ;
16061                ;;mov    ax,[si+10h]
16062                ;MOV     AX,[SI+SYS_FCB.FILSI2]
16063                ;;mov    [es:di+11h],ax
16064                ;MOV     [ES:DI+SF_ENTRY.sf_size],AX
16065                ;;mov    ax,[si+12h]
16066                ;MOV     AX,[SI+SYS_FCB.FILSI2+2]
16067                ;;mov    [es:di+13h],ax
16068                ;MOV     [ES:DI+SF_ENTRY.sf_size+2],AX
16069                ;;;
16070                push    ds
16071 00002314 1E          lds     ax,[si+SYS_FCB.FILSI2]
16072 00002318 26894511  mov     [es:di+SF_ENTRY.sf_size],ax
16073 0000231C 268C5D13  mov     [es:di+SF_ENTRY.sf_size+2],ds
16074 00002320 C546F3      lds     ax,[bPos] ; lds ax,[bp-13]
16075 00002323 8CDA      mov     dx,ds
16076 00002325 1F          pop     ds
16077                ;;;
16078                ;;mov    ax,[bp-13]
16079                ;MOV     AX,bPosL
16080                ;mov    dx,[bp-11]
16081                ;MOV     DX,bPosH
16082
16083                ;mov    [es:di+15h],ax
16084 00002326 26894515  MOV     [ES:DI+SF_ENTRY.sf_position],AX
16085                ;xchg    dx,[es:di+17h]
16086 0000232A 26875517  XCHG    [ES:DI+SF_ENTRY.sf_position+2],DX
16087 0000232E 52          PUSH    DX                ; save away Open age.
16088                ;mov    cx,[bp-15]
16089 0000232F 8B4EF1      MOV     CX,cByte         ; cResult =
16090
16091                ;hkn; DOS_Read is in DOSCODE
16092 00002332 BF[773B]  MOV     DI,DOS_READ      ; *(OP&FCBRead ? DOS_Read
16093                ;test  byte [bp-20],4
16094 00002335 F646EC04    TEST    FCBOp,FCBREAD    ; : DOS_write)(cRec);
16095 00002339 7503        JNZ     short DoContext
16096
16097                ;hkn; DOS_Write is in DOSCODE
16098 0000233B BF[783D]  MOV     DI,DOS_WRITE
16099 DoContext:
16100                push    bp
16101 0000233F 1E          push    ds
16102 00002340 56          push    si
16103
16104                ;hkn; SS is DOSDATA
16105 00002341 16          push    ss

```

```

16106 00002342 1F          pop     ds
16107
16108 ;; Fix for disk full
16109 00002343 FFD7          CALL    DI      ; DOS_READ or DOS_WRITE
16110
16111 00002345 5E          pop     si
16112 00002346 1F          pop     ds
16113 00002347 5D          pop     bp
16114 00002348 72BB          JC      short FCBDeath
16115
16116 0000234A 36803E[0B06]00        CMP     BYTE [SS:DISK_FULL],0 ; treat disk full as error
16117 00002350 7406          JZ      short NODSKFULL
16118 00002352 36C606[0B06]00        MOV     BYTE [SS:DISK_FULL],0 ; clear the flag
16119
16120 ; 23/01/2024
16121 ; (PCDOS 7.1 IBMDOS.COM)
16122 ;;mov     byte [bp-1],1
16123 ;MOV     FCBErr,FE0F          ; set disk full flag
16124
16125 NODSKFULL:
16126 ;; Fix for disk full
16127 ;mov     [bp-17],cx
16128 00002358 894EEF          MOV     cResult,cx
16129 0000235B E80AFB          call   SaveFCBInfo          ; SaveFCBInfo (FCB);
16130 ;pop     word [es:di+17h]
16131 0000235E 268F4517        POP     WORD [ES:DI+SF_ENTRY.sf_position+2] ; restore open age
16132 ; (sf_OpenAge = SF_ENTRY.sf_position+2)
16133
16134 ; 23/01/2024
16135 ; (PCDOS 7.1 IBMDOS.COM)
16136 ;
16137 ;;mov     ax,[es:di+11h]
16138 ;MOV     AX,[ES:DI+SF_ENTRY.sf_size]
16139 ;;mov     [si+10h],ax
16140 ;MOV     [SI+SYS_FCB.FILSIZ],AX
16141 ;;mov     ax,[es:di+13h]
16142 ;MOV     AX,[ES:DI+SF_ENTRY.sf_size+2]
16143 ;;mov     [si+12h],ax
16144 ;MOV     [SI+SYS_FCB.FILSIZ+2],AX
16145 ;;
16146 00002362 06          push    es
16147 00002363 26C44511        les     ax,[es:di+SF_ENTRY.sf_size]
16148 00002367 894410        mov     [si+SYS_FCB.FILSIZ],ax
16149 0000236A 8C4412        mov     [si+SYS_FCB.FILSIZ+2],es
16150 0000236D 07          pop     es
16151 ;;;
16152 ;          }
16153 skipOp:
16154 ;mov     ax,[bp-17]
16155 0000236E 8B46EF          MOV     AX,cResult          ; cRecRes = cResult / RecSize;
16156 00002371 31D2          XOR     DX,DX
16157 ;div     word [bp-9]
16158 00002373 F776F7          DIV     RecSize
16159 ;mov     [bp-19],ax
16160 00002376 8946ED          MOV     cRecRes,AX
16161 ;add     [bp-7],ax
16162 00002379 0146F9          ADD     RecPosL,AX          ; RecPos += cRecResult;
16163 ;adc     word [bp-5],0
16164 0000237C 8356FB00        ADC     RecPosH,0
16165
16166 ; If we have not gotten the expected number of records, we signal an EOF
16167 ; condition. On input, this is EOF. On output this is usually disk full.
16168 ; BUT... Under 2.0 and before, all device output IGNORED this condition. So
16169 ; do we.
16170
16171 ;cmp     ax,[bp-3]
16172 00002380 3B46FD          CMP     AX,cRec          ; if (cRecRes <> cRec)
16173 00002383 7411          JZ      short TryBlank
16174 ;test    byte [bp-20],4
16175 00002385 F646EC04        TEST    FCBop,FCBREAD          ; if (OP&FCBRead || !DEVICE)
16176 00002389 7507          JNZ     short SetEOF
16177 ; 07/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
16178 ; MSDOS 3.3
16179 ;;test    word [es:di+5],80h
16180 ;TEST    word [ES:DI+SF_ENTRY.sf_flags],devid_device
16181 ;JNZ     short TryBlank
16182 ; MSDOS 5.0 & MSDOS 6.0
16183 ;test    byte [es:di+5],80h
16184 0000238B 26F6450580      test    byte [ES:DI+SF_ENTRY.sf_flags],devid_device
16185 00002390 7504          jnz     short TryBlank
16186
16187 SetEOF:
16188 ;mov     byte [bp-1],1
16189 00002392 C646FF01        MOV     FCBErr,FE0F          ; FCBErr = FE0F;
16190
16191 TryBlank:
16192 ;OR      DX,DX          ; if (cResult%RecSize <> 0) {
16193 ;JZ      short SetExt
16194 ;add     word [bp-7],1
16195 ;ADD     RecPosL,1          ; RecPos++;
16196 ;adc     word [bp-5],0
16197 ;ADC     RecPosH,0
16198 ;test    byte [bp-20],4
16199 000023A2 F646EC04        TEST    FCBop,FCBREAD          ; if(OP&FCBRead) <> 0) {
16200 000023A6 7418          JZ      short SetExt
16201 ;inc     word [bp-19]
16202 ;INC     cRecRes          ; cRecRes++;
16203 ;mov     byte [bp-1],3
16204 ;MOV     FCBErr,FTRIM+FE0F ; FCBErr = FTRIM | FE0F;
16205 000023AF 8B4EF7          MOV     CX,RecSize          ; Blank (RecSize-cResult%RecSize,
16206 000023B2 29D1          SUB     CX,DX          ; DMA+cResult);
16207 000023B4 30C0          XOR     AL,AL
16208 ;hkn;
16209 000023B6 36C43E[2C03]    les     di,[ss:DMAADD]
16210 ;add     di,[bp-17]
16211 000023BB 037EEF          ADD     DI,cResult
16212 000023BE F3AA          REP     STOSB          ; } }
16213
16214 SetExt:
16215 ;mov     dx,[bp-5]
16216 ;MOV     DX,RecPosH
16217 ;mov     ax,[bp-7]
16218 ;MOV     AX,RecPosL
16219 ;test    byte [bp-20],2
16220 000023C6 F646EC02        TEST    FCBop,RANDOM          ; if ((OP&Random) == 0 ||
16221 000023CA 7406          JZ      short DoSetExt
16222 ;test    byte [bp-20],8
16223 000023CC F646EC08        TEST    FCBop,BLOCK          ; (OP&BLOCK) <> 0)
16224 000023D0 7403          JZ      short TrySetRR
16225 DoSetExt:
16226 000023D2 E85BFE          call   SetExtent          ; SetExtent (RecPos, FCB);
16227 TrySetRR:
16228 ;test    byte [bp-20],8
16229 000023D5 F646EC08        TEST    FCBop,BLOCK          ; if ((op&BLOCK) <> 0)
16230 000023D9 740F          JZ      short TryReturn

```

```

16230          ;mov     [si+21h],ax
16231 000023DB 894421      MOV     [SI+SYS_FCB.RR],AX      ; FCB->RR = RecPos;
16232          ;mov     [si+23h],dl
16233 000023DE 885423      MOV     [SI+SYS_FCB.RR+2],DL
16234          ;cmp     word [si+0Eh],64
16235 000023E1 837C0E40     CMP     word [SI+SYS_FCB.RECSIZ],64
16236 000023E5 7303        JAE     short TryReturn
16237          ;mov     [si+24h],dh
16238 000023E7 887424      MOV     [SI+SYS_FCB.RR+2+1],DH; Set 4th byte only if record size < 64
16239
16240 TryReturn:
16241          ;test    byte [bp-20],4
16242 000023EA F646EC04     TEST    FCBop,FCBREAD      ; if (!(FCBOP & FCBREAD)) {
16243 000023EE 750B        JNZ     short FCBSave
16244          push     ds
16245 000023F0 1E          ; FCB->FDate = date;
16246 000023F1 E86CE7      call    DATE16             ; FCB->FTime = time;
16247          pop      ds
16248          ;mov     [si+14h],ax
16249 000023F5 894414      MOV     [SI+SYS_FCB.FDATE],AX
16250          ;mov     [si+16h],dx
16251 000023F8 895416      MOV     [SI+SYS_FCB.FTIME],DX ; }
16252
16253 FCBSave:
16254          ;test    byte [bp-20],8
16255 000023FB F646EC08     TEST    FCBop,BLOCK      ; if ((op&BLOCK) <> 0)
16256 000023FF 7409        JZ      short DoReturn
16257          ;mov     cx,[bp-19]
16258 00002401 8B4EED      MOV     CX,CRecRes      ; user_CX = cRecRes;
16259          call     Get_User_Stack
16260          ;mov     [si+4],cx
16261 00002404 E870E0      MOV     [SI+user_env.user_CX],CX
16262
16263 DoReturn:
16264          ;mov     al,[bp-1]
16265 0000240A 8A46FF      MOV     AL,FCBErr      ; return (FCBERR);
16266          ;Leave
16267          mov      sp,bp
16268          pop      bp
16269          retn
16270
16271 ; 22/07/2018 - Retro DOS v3.0
16272
16273 ;Break <$FCB_Open - open an old-style FCB>
16274 -----
16275 ;
16276 ; $FCB_Open - CPM compatability file open. The user has formatted an FCB
16277 ; for us and asked to have the rest filled in.
16278 ;
16279 ; Inputs: DS:DX point to an unopened FCB
16280 ; Outputs: AL indicates status 0 is ok FF is error
16281 ; FCB has the following fields filled in:
16282 ; Time/Date Extent/NR Size
16283 -----
16284
16285 ; 23/01/2024 - Retro DOS v5.0
16286 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:6362h
16287
16288 _$FCB_OPEN:      ; System call 15
16289
16290          ;mov     ax,2
16291 00002411 B80200      MOV     AX,SHARING_COMPAT+open_for_both
16292
16293 ;hkn; DOS_Open is in DOSCODE
16294          MOV     CX,DOS_OPEN
16295
16296 ; The following is common code for Creation and openning of FCBs. AX is
16297 ; either attributes (for create) or open mode (for open)... DS:DX points to
16298 ; the FCB
16299
16300 DoAccess:
16301          push     ds
16302          push     dx
16303          push     cx
16304          push     ax
16305          ; save FCB pointer away
16306
16307 ;hkn; OpenBuf is in DOSDATA
16308          MOV     DI,OPENBUF
16309          call    TransFCB      ; crunch the fcb
16310          pop      ax
16311          pop      cx
16312          pop      dx
16313          pop      ds
16314          JNC     short FindFCB      ; get fcb
16315          ; everything seems ok
16316
16317 FCBOpenErr:
16318          ; AL has error code
16319          jmp     FCB_RET_ERR
16320
16321 FindFCB:
16322          call    GetExtended      ; DS:SI will point to FCB
16323
16324 ; 17/05/2019 - Retro DOS v4.0
16325
16326 ; MSDOS 3.3
16327 ;call    LRUFEB
16328 ;jc      short HardMessage
16329
16330 ; MSDOS 6.0
16331          push     ax
16332          mov      al,1           ;indicate Open/Create operation
16333          call    LRUFEB         ; get a sft entry (no error)
16334          pop      ax
16335          jc       short HardMessage
16336
16337          ;mov     word [es:di+2],8000h
16338          mov      word [es:di+SF_ENTRY.sf_mode],sf_isFCB
16339          push     ds
16340          push     si
16341          push     bx
16342          push     SI,CX
16343          ; save fcb pointer
16344
16345 ;hkn; SS is DOSDATA
16346          push     ss
16347          pop      ds           ; let DOS_Open see variables
16348          CALL    SI ; DOS_OPEN or DOS_CREATE ; go open the file
16349          pop      bx
16350          pop      si
16351          pop      ds           ; get fcb
16352
16353 ;hkn; SS override
16354          LES     DI,[SS:THISSFT]      ; get sf pointer
16355          JNC     short FCBOk         ; operation succeeded
16356
16357 failopen:
16358          PUSH    AX
16359          MOV     AL,"R" ; 52h      ; clear out field (free sft)
16360          call    BlastSFT
16361          POP     AX
16362          ;cmp     ax,4
16363          CMP     AX,error_too_many_open_files

```

```

16354 00002459 7405          JZ      short HardMessage
16355          ;cmp      ax,24h
16356 0000245B 83F824      CMP     AX,error_sharing_buffer_exceeded
16357 0000245E 7505          jnz     short DeadFCB
16358          HardMessage:
16359 00002460 50          PUSH    AX
16360 00002461 E86FFD      call    FCBHardErr
16361 00002464 58          POP     AX
16362          DeadFCB:
16363          ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
16364          ;jmp      FCB_RET_ERR
16365 00002465 EBC0          jmp     short FCBOpenErr
16366          FCBOK:
16367          ; MSDOS 6.0
16368 00002467 E8D8F3      call    IsSFTNet          ;AN007;F.C. >32mb Non Fat file?
16369 0000246A 7531          JNZ     short FCBOK2      ;AN007;F.C. >32mb yes
16370 0000246C E8CB5F      call    CheckShare        ;AN000;F.C. >32mb share around?
16371          ;JNZ     short FCBOK2      ;AN000;F.C. >32mb yes
16372          ; 23/01/2024 - Retro DOS v5.0
16373          ; (PCDOS 7.1 IBMDOS.COM)
16374 0000246F 750A          jnz     short FCBOK1
16375
16376          ;SR;
16377          ; If we reach here, we know we have got a local SFT. Let's update the
16378          ; LocalSFT variable to reflect this.
16379
16380 00002471 36893E[760F]  mov     [ss:LocalSFT],di
16381 00002476 368C06[780F]  mov     [ss:LocalSFT+2],es; Store the SFT address
16382          ;;SR;
16383          ;; The check below is not valid anymore since we regenerate for media > 32M.
16384          ;;
16385          ;; CMP     WORD [ES:DI+SF_ENTRY.sf_dirsec+2],0
16386          ;;          ;AN000;F.C. >32mb if dirsec >32mb
16387          ;; JZ      short FCBOK2      ;AN000;F.C. >32mb then error
16388          ;; MOV     AX,error_sys_comp_not_loaded ;AN000;F.C. >32mb
16389          ;; JMP     short failopen    ;AN000;F.C. >32mb
16390
16391          ; 23/01/2024 - Retro DOS v5.0
16392          ; (PCDOS 7.1 IBMDOS.COM)
16393          ;;
16394          FCBOK1:
16395          ;test     byte [es:di+5],80h
16396 0000247B 26F6450580    test     byte [es:di+SF_ENTRY.sf_flags],devid_device
16397 00002480 751B          jnz     short FCBOK2      ; local device
16398          ;test     byte [es:di+4],8
16399 00002482 26F6450408    test     byte [es:di+SF_ENTRY.sf_attr],attr_volume_id
16400 00002487 7514          jnz     short FCBOK2
16401          ; local file
16402          push     es
16403 0000248A 57          push     di
16404          ;les     di,[es:di+07h]
16405 0000248B 26C47D07      les     di,[es:di+SF_ENTRY.sf_devptr] ; local file's DPB
16406          ;cmp     word [es:di+0Fh],0
16407 0000248F 26837D0F00    cmp     word [es:di+DPB.FAT_SIZE],0 ; (16 bit FAT size)
16408 00002494 5F          pop      di
16409 00002495 07          pop      es
16410 00002496 7505          jnz     short FCBOK2      ; not FAT32 (ok)
16411          ;mov     ax,0Fh          ; error_invalid_drive (for FAT32)
16412 00002498 B80F00      mov     ax,error_invalid_drive
16413 0000249B EBB2          jmp     short failopen
16414          ;;
16415
16416          FCBOK2:
16417          ; MSDOS 6.0 (& MSDOS 3.3)
16418 0000249D 26FF05      inc     word [es:di]
16419          ;INC     word [ES:DI+SF_ENTRY.sf_ref_count] ; increment reference count
16420
16421          ; 23/01/2024 - Retro DOS v5.0
16422          ; (PCDOS 7.1 IBMDOS.COM)
16423          ;;
16424 000024A0 E87525      call    set_sftfcb_entry
16425          ;;
16426
16427 000024A3 E8C2F9      call    SaveFCBInfo
16428
16429          ; MSDOS 3.3
16430          ;call    SetOpenAge
16431          ; MSDOS 6.0 (& MSDOS 3.3)
16432          ;test     word [es:di+5],80h
16433          ;TEST     word [ES:DI+SF_ENTRY.sf_flags],devid_device
16434 000024A6 26F6450580    test     byte [ES:DI+SF_ENTRY.sf_flags],devid_device ; 28/07/2019
16435 000024AB 7508          JNZ     short FCBNoDrive    ; do not munge drive on devices
16436
16437 000024AD 8A04          MOV     AL,[SI]          ; get drive byte
16438 000024AF E8DB56      call    GETTHISDRV        ; convert
16439          ;INC     AL
16440          ; 17/12/2022
16441 000024B2 40          inc     ax
16442 000024B3 8804          MOV     [SI],AL          ; stash in good drive letter
16443
16444          FCBNoDrive:
16445          ;mov     word [si+0Eh],128
16446 000024B5 C7440E8000    MOV     word [SI+SYS_FCB.RECSIZ],80h ; stuff in default record size
16447
16448          ; 23/01/2024 - Retro DOS v5.0
16449          ; (PCDOS 7.1 IBMDOS.COM)
16450          ;;
16451          ;;mov     ax,[es:di+0Dh]
16452          ;MOV     AX,[ES:DI+SF_ENTRY.sf_time] ; set time
16453          ;;mov     [si+16h],ax
16454          ;MOV     [SI+SYS_FCB.FTIME],AX
16455          ;;mov     ax,[es:di+0Fh]
16456          ;MOV     AX,[ES:DI+SF_ENTRY.sf_date] ; set date
16457          ;;mov     [si+14h],ax
16458          ;MOV     [SI+SYS_FCB.FDATE],AX
16459          ;;mov     ax,[es:di+11h]
16460          ;MOV     AX,[ES:DI+SF_ENTRY.sf_size] ; set sizes
16461          ;;mov     [si+10h],ax
16462          ;MOV     [SI+SYS_FCB.FILSIZ],AX
16463          ;;mov     ax,[es:di+13h]
16464          ;MOV     AX,[ES:DI+SF_ENTRY.sf_size+2]
16465          ;;mov     [si+12h],ax
16466          ;MOV     [SI+SYS_FCB.FILSIZ+2],AX
16467          ;
16468 000024BA 06          push     es
16469          ;les     ax,[es:di+0Dh]
16470 000024BB 26C4450D      les     ax,[es:di+SF_ENTRY.sf_time]
16471          ;mov     [si+16h],ax
16472 000024BF 894416      mov     [si+SYS_FCB.FTIME],ax ; set time
16473          ;mov     [si+14h],es
16474 000024C2 8C4414      mov     [si+SYS_FCB.FDATE],es ; set date
16475 000024C5 07          pop      es
16476 000024C6 06          push     es
16477          ;les     ax,[es:di+11h]

```

```

16478 000024C7 26C44511      les     ax,[es:di+SF_ENTRY.sf_size] ; set size
16479                        ;mov     [si+10h],ax
16480 000024CB 894410      mov     [si+SYS_FCB.FILSIZ],ax
16481                        ;mov     [si+12h],ax
16482 000024CE 8C4412      mov     [si+SYS_FCB.FILSIZ+2],es
16483 000024D1 07                pop     es
16484                        ;;;
16485
16486 000024D2 31C0                XOR     AX,AX                ; convenient zero
16487                        ;mov     [si+0Ch],ax
16488 000024D4 89440C      MOV     [SI+SYS_FCB.EXTENT],AX; point to beginning of file
16489
16490                        ; We must scan the set of FCB SFTs for one that appears to match the current
16491                        ; one. We cheat and use CheckFCB to match the FCBs.
16492
16493                        ;hkn;      SS override
16494 000024D7 36C43E[4000]  LES     DI,[SS:SFTFCB]      ; get the pointer to head of the list
16495                        ;mov     ah,[es:di+4]
16496 000024DC 268A6504  MOV     AH,[ES:DI+SFT.SFCount]; get number of SFTs to scan
16497
16498      OpenScan:
16499                        ;cmp     al,[si+18h]
16500 000024E0 3A4418      CMP     AL,[SI+fcf_sfn]      ; don't compare ourselves
16501 000024E3 7407                JZ      short SkipCheck
16502 000024E5 50                push    ax                  ; preserve count
16503 000024E6 E848FC      call   CheckFCB            ; do they match
16504 000024E9 58                pop     ax                  ; get count back
16505 000024EA 7309                JNC     short OpenFound     ; found a match!
16506
16507      SkipCheck:
16508                        INC     AL                ; advance to next FCB
16509 000024EC FEC0                CMP     AL,AH              ; table full?
16510 000024EE 38E0                JNZ     short OpenScan     ; no, go for more
16511
16512      OpenDone:
16513 000024F2 30C0                xor     al,al              ; return success
16514 000024F4 C3                retn
16515
16516                        ; The SFT at ES:DI is the one that is already in use for this FCB. We set the
16517                        ; FCB to use this one. We increment its ref count. We do NOT close it at all.
16518                        ; Consider:
16519                        ;
16520                        ; open (foo) delete (foo) open (bar)
16521                        ;
16522                        ; This causes us to recycle (potentially) bar through the same local SFT as
16523                        ; foo even though foo is no longer needed; this is due to the server closing
16524                        ; foo for us when we delete it. Unfortunately, we cannot see this closure.
16525                        ; If we were to CLOSE bar, the server would then close the only reference to
16526                        ; bar and subsequent I/O would be lost to the redirector.
16527                        ;
16528                        ; This gets solved by NOT closing the sft, but zeroing the ref count
16529                        ; (effectively freeing the SFT) and informing the sharer (if relevant) that
16530                        ; the SFT is no longer in use. Note that the SHARER MUST keep its ref counts
16531                        ; around. This will allow us to access the same file through multiple network
16532                        ; connections and NOT prematurely terminate when the ref count on one
16533                        ; connection goes to zero.
16534
16535      OpenFound:
16536                        ;mov     [si+18h],al
16537 000024F5 884418      MOV     [SI+fcf_sfn],AL      ; assign with this
16538 000024F8 26FF05      inc     word [es:di]
16539                        ;INC     word [ES:DI+SF_ENTRY.sf_ref_count]
16540                        ;;;
16541                        ; 23/01/2024 - Retro DOS v5.0
16542                        ; (PCDOS 7.1 IBMDOS.COM)
16543                        ;;;
16544 000024FB E81A25      call   set_sftfcb_entry
16545                        ;;;
16546                        ; 24/01/2024
16547 000024FE 16                push    ss
16548 000024FF 1F                pop     ds
16549
16550                        ;MOV     AX,[SS:FCBLRU]      ; update LRU counts
16551 00002500 A1[1000]      mov     ax,[FCBLRU] ; 24/01/2024
16552                        ;mov     [es:di+15h],ax
16553 00002503 26894515  MOV     [ES:DI+sf_LRU],AX
16554
16555                        ;
16556                        ; We have an FCB sft that is now of no use. We release sharing info and then
16557                        ; blast it to prevent other reuse.
16558                        ;
16559                        ;push     ss
16560                        ;pop      ds
16561
16562      LES     DI,[THISSFT]
16563 00002507 C43E[9E05]  dec     word [es:di]
16564 0000250B 26FF0D      dec     word [ES:DI+SF_ENTRY.sf_ref_count]
16565                        ; free the newly allocated SFT
16566 0000250E E8735F      call   ShareEnd
16567 00002511 B043      MOV     AL,'C' ; 43h
16568 00002513 E802FC      call   BlastsFT
16569 00002516 EBDA      JMP     short OpenDone
16570
16571      ;BREAK <$FCB_Create - create a new directory entry>
16572
16573      ;-----
16574      ; $FCB_Create - CPM compatability file create. The user has formatted an
16575      ; FCB for us and asked to have the rest filled in.
16576      ;
16577      ; Inputs: DS:DX point to an unopened FCB
16578      ; Outputs: AL indicates status 0 is ok FF is error
16579      ; FCB has the following fields filled in:
16580      ; Time/Date Extent/NR Size
16581      ;-----
16582
16583      _$FCB_CREATE:                ; System call 22
16584
16585      ;hkn; DOS_Create is in DOSCODE
16586 00002518 B9[CA30]      MOV     CX,DOS_CREATE      ; routine to call
16587 0000251B 31C0      XOR     AX,AX              ; attributes to create
16588 0000251D E82AFD      call   GetExtended        ; get extended FCB
16589 00002520 7403      JZ      short DoAccessJ    ; not an extended FCB
16590 00002522 8A44FF      MOV     AL,[SI-1]         ; get attributes
16591
16592      DoAccessJ:
16593 00002525 E9EFFE      JMP     DoAccess           ; do dirty work
16594
16595      ;=====
16596      ; SEARCH.ASM, MSDOS 6.0, 1991
16597      ;=====
16598      ; 22/07/2018 - Retro DOS v3.0
16599      ; 17/05/2019 - Retro DOS v4.0
16600
16601      ; DOSCODE:5DDFh (MSDOS 6.21, MSDOS.SYS)
16602
16603      ; 09/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
16604      ; DOSCODE:5DCBh (MSDOS 5.0, MSDOS.SYS)

```



```

16602 ;** Search.asm
16603 ;-----
16604 ; Directory search system calls.
16605 ; These will be passed direct text of the pathname from the user.
16606 ; They will need to be passed through the macro expander prior to
16607 ; being sent through the low-level stuff.
16608 ; I/O specs are defined in DISPATCH. The system calls are:
16609 ;
16610 ; $Dir_Search_First      written
16611 ; $Dir_Search_Next      written
16612 ; $Find_First           written
16613 ; $Find_Next            written
16614 ; PackName              written
16615 ;
16616 ; Modification history:
16617 ;
16618 ; Created: ARR 4 April 1983
16619 ;
16620 ;-----
16621 ; Procedure Name : $DIR_SEARCH_FIRST
16622 ;
16623 ; Inputs:
16624 ; DS:DX Points to unopened FCB
16625 ; Function:
16626 ; Directory is searched for first matching entry and the directory
16627 ; entry is loaded at the disk transfer address
16628 ; Returns:
16629 ; AL = -1 if no entries matched, otherwise 0
16630 ;-----
16631 ;
16632 ; IBMDOS.COM (MSDOS 3.3) - Offset 2B88h
16633 ;
16634 ; 24/01/2024
16635 ; MSDOS 5.0 MSDOS.SYS - DOSCODE:5DCBh
16636 ; MSDOS 6.22 MSDOS.SYS - DOSCODE:5DDFh
16637 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:647Bh
16638 ; Windows ME IO.SYS - BIOSCODE:61E5h
16639 ;
16640 _$DIR_SEARCH_FIRST:
16641 00002528 368916[A605] MOV [SS:THISFCB],DX
16642 0000252D 368C1E[A805] MOV [SS:THISFCB+2],DS
16643 00002532 89D6 MOV SI,DX
16644 00002534 803CFF CMP BYTE [SI],0FFh
16645 00002537 7503 JNZ short NORMFCB4
16646 00002539 83C607 ADD SI,7 ; Point to drive select byte
16647 NORMFCB4:
16648 0000253C FF34 push word [SI] ; Save original drive byte for later
16649
16650 0000253E 16 push ss
16651 0000253F 07 pop es ; get es to address DOSGroup
16652
16653 00002540 BF[BE03] MOV DI,OPENBUF ; appropriate buffer
16654 00002543 E8B656 call TransFCB ; convert the FCB, set SATTRIB EXTFCB
16655 00002546 7304 JNC short SearchIt ; no error, go and look
16656 00002548 5B pop bx ; Clean stack
16657
16658 ; Error code is in AX
16659
16660 ; 09/11/2022
16661 dcf_errj:
16662 00002549 E941E1 jmp FCB_RET_ERR ; error
16663
16664 SearchIt:
16665 0000254C 16 push ss
16666 0000254D 1F pop ds ; get ready for search
16667 ; push word [DMAADD]
16668 ; push word [DMAADD+2]
16669 ; 24/01/2024
16670 0000254E C43E[2C03] les di,[DMAADD]
16671 00002552 57 push di
16672 00002553 06 push es
16673 00002554 C706[2C03][BE04] MOV WORD [DMAADD],SEARCHBUF
16674 0000255A 8C1E[2E03] MOV WORD [DMAADD+2],DS
16675 ; MSDOS 3.3
16676 ; call DOS_SEARCH_FIRST
16677 ; MSDOS 6.0
16678 0000255E E8BD0F call GET_FAST_SEARCH ; search
16679 00002561 8F06[2E03] pop word [DMAADD+2]
16680 00002565 8F06[2C03] pop word [DMAADD]
16681 00002569 735A JNC short SearchSet ; no error, transfer info
16682 0000256B 5B pop bx ; Clean stack
16683
16684 ; Error code is in AX
16685
16686 ; 09/11/2022
16687 ; jmp FCB_RET_ERR
16688 0000256C EBDB jmp short dcf_errj
16689
16690 ;-----
16691 ;
16692 ; Procedure Name : $DIR_SEARCH_NEXT
16693 ;
16694 ; Inputs:
16695 ; DS:DX points to unopened FCB returned by $DIR_SEARCH_FIRST
16696 ; Function:
16697 ; Directory is searched for the next matching entry and the directory
16698 ; entry is loaded at the disk transfer address
16699 ; Returns:
16700 ; AL = -1 if no entries matched, otherwise 0
16701 ;-----
16702 ;
16703 ; 24/01/2024
16704 ; MSDOS 5.0 MSDOS.SYS - DOSCODE:5E5Fh
16705 ; MSDOS 6.22 MSDOS.SYS - DOSCODE:5E73h
16706 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:6517h
16707 ; Windows ME IO.SYS - BIOSCODE:6273h
16708 ;
16709 _$DIR_SEARCH_NEXT:
16710 0000256E 368916[A605] MOV [SS:THISFCB],DX
16711 00002573 368C1E[A805] MOV [SS:THISFCB+2],DS
16712 ; 24/01/2024 (PCDOS 7.1)
16713 ; MOV byte [SS:SATTRIB],0
16714 ; MOV byte [SS:EXTFCB],0
16715 00002578 B000 mov al,0
16716 0000257A 36A2[6D05] mov [SS:SATTRIB],al ; 0
16717 0000257E 36A2[6D05] mov [SS:SATTRIB],al ; 0
16718
16719 00002582 16 push ss
16720 00002583 07 pop es
16721
16722 00002584 BF[BE04] MOV DI,SEARCHBUF
16723
16724 00002587 89D6 MOV SI,DX
16725 00002589 803CFF CMP BYTE [SI],0FFh

```

```

16726 0000258C 750D      JNZ     short NORMFCB6
16727 0000258E 83C606    ADD     SI,6
16728 00002591 AC        LODSB
16729
16730 00002592 36A2[6D05]  MOV     [SS:SATTRIB],AL
16731 00002596 36FE0E[6C05] DEC     byte [SS:EXTFCB]
16732
NORMFCB6:
16733 0000259B AC        LODSB                ; Get original user drive byte
16734 0000259C 50        push     ax                ; Put it on stack
16735 0000259D 8A4414    MOV     AL,[SI+20]            ; Get correct search contin drive byte
16736 000025A0 AA        STOSB                ; Put in correct place
16737 000025A1 B90A00    MOV     CX,20/2
16738 000025A4 F3A5      REP     MOVSW                ; Transfer in rest of search contin info
16739
16740 000025A6 16        push     ss
16741 000025A7 1F        pop      ds
16742
16743        ;push word [DMAADD]
16744        ;push word [DMAADD+2]
16745        ; 24/01/2024
16746 000025A8 C43E[2C03] les     di,[DMAADD]
16747 000025AC 57        push     di
16748 000025AD 06        push     es
16749 000025AE C706[2C03][BE04] MOV     WORD [DMAADD],SEARCHBUF
16750 000025B4 8C1E[2E03] MOV     WORD [DMAADD+2],DS
16751 000025B8 E86210    call    DOS_SEARCH_NEXT        ; Find it
16752 000025BB 8F06[2E03] pop     word [DMAADD+2]
16753 000025BF 8F06[2C03] pop     word [DMAADD]
16754 000025C3 7249      JC      short SearchNoMore
16755        ; 24/01/2024
16756        ;JMP SearchSet                ; Ok set return
16757
;;; ; 24/01/2024 - Retro DOS v5.0
16758
16759
16760        ; The search was successful (or the search-next). We store the information
16761        ; into the user's FCB for continuation.
16762
SearchSet:
16763
16764 000025C5 BE[BE04]    MOV     SI,SEARCHBUF
16765 000025C8 C43E[A605]    LES     DI,[THISFCB]                ; point to the FCB
16766 000025CC F606[6C05]FF  TEST    byte [EXTFCB],0FFh
16767 000025D1 7403      JZ      short NORMFCB1
16768 000025D3 83C707    ADD     DI,7                ; Point past the extension
16769
NORMFCB1:
16770 000025D6 5B        pop      bx                ; Get original drive byte
16771 000025D7 08DB      OR      BL,BL
16772 000025D9 7506      JNZ     short SearchDrv
16773 000025DB 8A1E[3603] MOV     BL,[CURDRV]
16774 000025DF FEC3      INC     BL
16775
SearchDrv:
16776 000025E1 AC        LODSB                ; Get correct search contin drive byte
16777 000025E2 86C3      XCHG    AL,BL                ; Search byte to BL, user byte to AL
16778 000025E4 47        INC     DI
16779        ;STOSB                ; Store the correct "user" drive byte
16780        ; at the start of the search info
16781 000025E5 B90A00    MOV     CX,20/2
16782 000025E8 F3A5      REP     MOVSW                ; Rest of search cont info, SI -> entry
16783 000025EA 86C3      XCHG    AL,BL                ; User drive byte back to BL, search
16784        ; byte to AL
16785 000025EC AA        STOSB                ; search contin drive byte at end of
16786        ; contin info
16787 000025ED C43E[2C03] LES     DI,[DMAADD]
16788 000025F1 F606[6C05]FF  TEST    byte [EXTFCB],0FFh
16789 000025F6 740C      JZ      short NORMFCB2
16790 000025F8 B0FF      MOV     AL,0FFh
16791 000025FA AA        STOSB
16792 000025FB FEC0      INC     AL
16793        ;MOV CX,5
16794        ; 17/12/2022
16795        ;mov cl,5
16796        ;REP STOSB
16797        ; 03/07/2024
16798 000025FD AB        stosw
16799 000025FE AB        stosw
16800 000025FF AA        stosb
16801 00002600 A0[6D05]    MOV     AL,[SATTRIB]
16802 00002603 AA        STOSB
16803
NORMFCB2:
16804 00002604 88D8      MOV     AL,BL                ; User Drive byte
16805 00002606 AA        STOSB
16806        ;MOV CX,16
16807        ; 17/12/2022
16808 00002607 B110      mov     cl,16
16809 00002609 F3A5      REP     MOVSW
16810 0000260B E97CE0    jmp     FCB_RET_OK
16811
;;;
16812
SearchNoMore:
16813
16814 0000260E C43E[A605]    LES     DI,[THISFCB]
16815 00002612 F606[6C05]FF  TEST    byte [EXTFCB],0FFh
16816 00002617 7403      JZ      short NORMFCB8
16817 00002619 83C707    ADD     DI,7                ; Point past the extension
16818
NORMFCB8:
16819 0000261C 5B        pop      bx                ; Get original drive byte
16820 0000261D 26881D    MOV     [ES:DI],BL          ; Store the correct "user" drive byte
16821        ; at the right spot
16822
; error code is in AX
16823
16824 00002620 E96AE0    jmp     FCB_RET_ERR
16825
; 17/05/2019 - Retro DOS v4.0
16826
16827
16828        ; DOSCODE:5EE6h (MSDOS 6.21, MSDOS.SYS)
16829
;-----
16830
16831
16832        Procedure Name : $FIND_FIRST
16833
16834        Assembler usage:
16835        MOV AH, FindFirst
16836        LDS DX, name
16837        MOV CX, attr
16838        INT 21h
16839        ; DMA address has datablock
16840
16841        Error Returns:
16842        AX = error_path_not_found
16843        = error_no_more_files
16844
;-----
16845
16846        ; 09/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
16847        ; DOSCODE:5ED2h (MSDOS 5.0, MSDOS.SYS)
16848
16849        ; 24/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)

```



```

16850          ; DOSCODE:6594h (PCDOS 7.1, IBMDOS.COM)
16851
16852 _$FIND_FIRST:
16853     MOV     SI,DX          ; get name in appropriate place
16854     MOV     [SS:SATTRIB],CL      ; search attribute to correct loc
16855
16856     MOV     DI,OPENBUF      ; appropriate buffer
16857
16858     call    TransPathSet     ; convert the path
16859     JNC     short Find_it    ; no error, go and look
16860
16861 FindError:
16862     ;mov     al,3
16863     mov     al,error_path_not_found ; error and map into one.
16864     ; 09/11/2022
16865 FF_errj:
16866     jmp     SYS_RET_ERR
16867 Find_it:
16868     push    ss
16869     pop     ds
16870
16871     ;push    word [DMAADD]
16872     ;push    word [DMAADD+2]
16873     ; 24/01/2024 (PCDOS 7.1 IBMDOS.COM)
16874     les     di,[DMAADD]
16875     push    di
16876     push    es
16877     MOV     WORD [DMAADD],SEARCHBUF
16878     MOV     WORD [DMAADD+2],DS
16879     ; MSDOS 3.3
16880     ;call    DOS_SEARCH_FIRST
16881     ; MSDOS 6.0
16882     call    GET_FAST_SEARCH    ; search
16883     pop     word [DMAADD+2]
16884     pop     word [DMAADD]
16885
16886     ; 16/12/2022
16887     JNC     short FindSet     ; no error, transfer info
16888     jc      short FF_errj    ; jmp SYS_RET_ERR
16889     ;
16890     ;jmp     SYS_RET_ERR
16891     ; 09/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
16892 ;FFF_errj:
16893     ;jmp     short FF_errj    ; jmp SYS_RET_ERR
16894
16895 FindSet:
16896     MOV     SI,SEARCHBUF
16897     LES     DI,[DMAADD]
16898     MOV     CX,21
16899     REP     MOVSB
16900     PUSH    SI                ; Save pointer to start of entry
16901     ;mov     al,[si+0Bh]
16902     MOV     AL,[SI+dir_entry.dir_attr]
16903     STOSB
16904     ;add     si,16h ; 22
16905     ADD     SI,dir_entry.dir_time
16906     MOVSW   ; dir_time
16907     MOVSW   ; dir_date
16908     INC     SI
16909     INC     SI                ; skip dir_first
16910     MOVSW   ; dir_size (2 words)
16911     MOVSW
16912     POP     SI                ; Point back to dir_name
16913     CALL    PackName
16914     jmp     SYS_RET_OK        ; bye with no errors
16915
16916 ;-----
16917 ;
16918 ; Procedure Name : $FIND_NEXT
16919 ;
16920 ; Assembler usage:
16921 ; ; dma points at area returned by find_first
16922 ; MOV AH, findnext
16923 ; INT 21h
16924 ; ; next entry is at dma
16925 ;
16926 ; Error Returns:
16927 ; AX = error_no_more_files
16928 ;-----
16929
16930     ; 09/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
16931
16932     ; 24/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
16933     ; DOSCODE:65ECh (PCDOS 7.1, IBMDOS.COM)
16934
16935 _$FIND_NEXT:
16936     push    ss
16937     pop     es
16938
16939     MOV     DI,SEARCHBUF
16940
16941     LDS     SI,[SS:DMAADD]
16942
16943     MOV     CX,21
16944     REP     MOVSB                ; Put the search continuation info
16945     ; in the right place
16946     push    ss
16947     pop     ds                ; get ready for search
16948
16949     ; 24/01/2024 (Retro DOS v5-v4)
16950     ;push    word [DMAADD]
16951     ;push    word [DMAADD+2]
16952     les     di,[DMAADD]
16953     push    di
16954     push    es
16955     MOV     WORD [DMAADD],SEARCHBUF
16956     MOV     WORD [DMAADD+2],DS
16957     call    DOS_SEARCH_NEXT    ; Find it
16958     pop     word [DMAADD+2]
16959     pop     word [DMAADD]
16960     JNC     short FindSet     ; No error, set info
16961     ;jmp     SYS_RET_ERR
16962     ; 16/12/2022
16963     jmp     short FF_errj    ; jmp SYS_RET_ERR
16964     ; 09/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
16965     jmp     short FFF_errj    ; jmp SYS_RET_ERR
16966
16967 ;-----
16968 ;** PackName - Convert file names from FCB to ASCIZ format.
16969 ;
16970 ; PackName transfers a file name from DS:SI to ES:DI and converts it to
16971 ; the ASCIZ format.
16972 ;
16973 ; ENTRY (DS:SI) = 11 character FCB or dir entry name

```

```

16974 ; (ES:DI) = destination area (13 bytes)
16975 ; EXIT (ds:SI) and (es:DI) advanced
16976 ; USES al, CX, SI, DI, Flags (BUGBUG - not verified - jgl)
16977 ;-----
16978 ;
16979 ; 25/01/2024 - Retro DOS v5.0
16980 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:6627h
16981 ;
16982 PackName:
16983 ; Move over 8 characters to cover the name component, then trim it's
16984 ; trailing blanks.
16985 ;
16986 ;MOV CX,8 ; Pack the name
16987 ;REP MOVSB ; Move all of it
16988 ; 25/01/2024
16989 mov cx,4
16990 rep movsw
16991 main_kill_tail:
16992 cmp byte [es:di-1], " "
16993 jnz short find_check_dot
16994 dec di ; Back up over trailing space
16995 inc cx
16996 cmp cx,8
16997 jb short main_kill_tail
16998 find_check_dot:
16999 ;cmp word [si], (" " << 8) | " "
17000 cmp word [si], 2020h
17001 jnz short got_ext ; Some chars in extension
17002 cmp byte [si+2], " "
17003 jz short find_done ; No extension
17004 got_ext:
17005 mov al, "." ; 2Eh
17006 stosb
17007 ;mov cx,3
17008 ;; 18/12/2022
17009 ;;mov cl,3
17010 ;;REP MOVSB
17011 ;movsb
17012 ;movsb
17013 ;movsb
17014 ; 25/01/2024
17015 movsw
17016 movsb
17017 ext_kill_tail:
17018 cmp byte [es:di-1], " "
17019 jnz short find_done
17020 dec di ; Back up over trailing space
17021 jmp short ext_kill_tail
17022 find_done:
17023 xor ax, ax
17024 stosb ; NUL terminate
17025 retn
17026 ;-----
17027 ;
17028 ; 24/01/2024
17029 %if 0
17030 ; 17/05/2019 - Retro DOS v4.0
17031 GET_FAST_SEARCH:
17032 ; 22/07/2018
17033 ; MSDOS 6.0
17034 ; 17/12/2022
17035 or byte [ss:dos34_flag+1], (SEARCH_FASTOPEN>>8) ; 04h
17036 ;or word [ss:dos34_flag], SEARCH_FASTOPEN ; 400h
17037 ;FO.trigger fastopen ;AN000;
17038 ;call DOS_SEARCH_FIRST
17039 ;retn
17040 ; 17/12/2022
17041 jmp DOS_SEARCH_FIRST
17042 %endif
17043 ;-----
17044 ;=====
17045 ; PATH.ASM, MSDOS 6.0, 1991
17046 ;=====
17047 ; 06/08/2018 - Retro DOS v3.0
17048 ; 17/05/2019 - Retro DOS v4.0
17049 ;
17050 ; DOSCODE:5FB0h (MSDOS 6.21, MSDOS.SYS)
17051 ;
17052 ;** Directory related system calls. These will be passed direct text of the
17053 ; pathname from the user. They will need to be passed through the macro
17054 ; expander prior to being sent through the low-level stuff. I/O specs are
17055 ; defined in DISPATCH. The system calls are:
17056 ;
17057 ; $CURRENT_DIR Written
17058 ; $RMDIR Written
17059 ; $CHDIR Written
17060 ; $MKDIR Written
17061 ;
17062 ;
17063 ; Modification history:
17064 ;
17065 ; Created: ARR 4 April 1983
17066 ; MZ 10 May 1983 CurrentDir implemented
17067 ; MZ 11 May 1983 Rmdir, ChDir, Mkdir implemented
17068 ; EE 19 Oct 1983 Rmdir no longer allows you to delete a
17069 ; current directory.
17070 ; MZ 19 Jan 1983 Brain damaged applications rely on success
17071 ;
17072 ; I_Need ThisCDS,DWORD ; pointer to Current CDS
17073 ; I_Need WFP_Start,WORD ; pointer to beginning of directory text
17074 ; I_Need Curr_Dir_End,WORD ; offset to end of directory part
17075 ; I_Need OpenBuf,128 ; temp spot for translated name
17076 ; I_need fSplice,BYTE ; TRUE => do splice
17077 ; I_Need NoSetDir,BYTE ; TRUE => no exact match on splice
17078 ; I_Need cMeta,BYTE
17079 ; I_Need DrvErr,BYTE ;AN000;
17080 ;
17081 ;BREAK <$CURRENT_DIR - dump the current directory into user space>
17082 ;-----
17083 ;
17084 ; Procedure Name : $CURRENT_DIR
17085 ;
17086 ; Assembler usage:
17087 ; LDS SI,area
17088 ; MOV DL,drive
17089 ; INT 21h
17090 ;
17091 ; DS:SI is a pointer to 64 byte area that contains drive
17092 ; current directory.
17093 ; Error returns:
17094 ; AX = error_invalid_drive
17095 ;
17096 ;-----
17097

```

```

17098             ; 06/08/2018 - Retro DOS v3.0
17099             ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 2D4Eh
17100
17101             ; 25/01/2024 - Retro DOS v5.0
17102             ; MSDOS 5.0 MSDOS.SYS - DOSCODE:5F9Ch ; Retro DOS v4.1 (& v4.0)
17103             ; MSDOS 6.22 MSDOS.SYS - DOSCODE:5FB0h ; Retro DOS v4.2
17104             ; Windows ME IO.SYS - BIOSCODE:6393h
17105             ; PCDOS 7.1 IBMDOS.COM - DOSCODE:6664h ; Retro DOS v5.0
17106
17107     _$CURRENT_DIR:
17108     call ECritDisk
17109     mov AL,DL ; get drive number (0=def, 1=A)
17110     call GetVisDrv ; grab it
17111     jnc short CurrentValidate ; no error -> go and validate dir
17112 CurdirErr:
17113     call LCritDisk
17114
17115     ; MSDOS 3.3
17116     ;mov al,0Fh
17117
17118     ; MSDOS 6.0
17119     push ds
17120     mov ds,[cs:DosDSeg]
17121     mov al,[DrvErr] ;IFS. ;AN000;
17122     pop ds
17123
17124 Curdir_errj:
17125     jmp SYS_RET_ERR ;IFS. make noise ;AN000;
17126
17127 CurrentValidate:
17128     push ds ; save destination
17129     push si
17130
17131     ;LDS SI,[CS:THISCDS] ; MSDOS 3.3
17132
17133     ; MSDOS 6.0
17134     mov ds,[cs:DosDSeg]
17135     ; 25/01/2024 (PCDOS 7.1 IBMDOS.COM)
17136     mov byte [NoSetDir],0 ; *
17137
17138     ; 25/01/2024
17139     ;lds si,[THISCDS]
17140
17141     ; 16/12/2022
17142     %if 0
17143     ; 09/11/2022 (following test instruction is nonsense!)
17144     ; (I am leaving it here for MSDOS 5.0 MSDOS.SYS compatibility)
17145
17146     ;test word [si+43h],8000h
17147     TEST word [SI+curdir.flags],curdir_isnet
17148     ;jnz short $+2 ; 09/11/2022
17149     jnz short DoCheck
17150 %endif
17151
17152     ; Random optimization nuked due to some utilities using GetCurrentDir to do
17153     ; media check.
17154     ; CMP word [SI+curdir.ID],0
17155     ; JZ short GetDst
17156 DoCheck:
17157     ;MOV byte [cs:NoSetDir],0 ; interested only in contents
17158
17159     ; 25/01/2024
17160     ; MSDOS 6.0
17161     ;push ds
17162     ;mov ds,[cs:DosDSeg]
17163     ;mov byte [NoSetDir],0 ; *
17164     ;pop ds
17165
17166     MOV DI,OPENBUF
17167     call ValidateCDS ; output is ES:DI -> CDS
17168
17169     push es ; swap source and destination
17170     push di
17171     pop si
17172     pop ds
17173 GetDst:
17174     pop di
17175     pop es ; get real destination
17176     JC short CurdirErr
17177     ;ADD SI,curdir.text ; add si,0 ; 09/08/2018
17178     ;
17179     ; 09/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
17180     ; DOSCODE:5FE2h (MSDOS 5.0, MSDOS.SYS)
17181     ; 16/12/2022
17182     ;add si,0 ; add si,curdir.text
17183     ;
17184     ;add si,[si+4Fh] ; 17/05/2019
17185     ADD SI,[SI+curdir.end]
17186     CMP BYTE [SI],'\ ' ; 5ch ; root or subdirs present?
17187     JNZ short CurrentCopy
17188     INC SI
17189 CurrentCopy:
17190     ; call FStrCpy
17191     ;; 10/29/86 E5 char
17192     PUSH AX
17193     LODSB ; get char
17194     OR AL,AL
17195     JZ short FOK
17196     CMP AL,05H
17197     JZ short FCHANGE
17198     JMP short FFF
17199 FCPYNEXT:
17200     LODSB ; get char
17201 FFF:
17202     CMP AL,'\' ; beginning of directory
17203     JNZ short FOK ; no
17204     STOSB ; put into user's buffer
17205     LODSB ; 1st char of dir is 05?
17206     CMP AL,05H
17207     JNZ short FOK ; no
17208 FCHANGE:
17209     MOV AL,0E5H ; make it E5
17210 FOK:
17211     STOSB ; put into user's buffer
17212     OR AL,AL ; final char
17213     JNZ short FCPYNEXT ; no
17214     POP AX
17215
17216     ;; 10/29/86 E5 char
17217     xor AL,AL ; MZ 19 Jan 84
17218     call LCritDisk
17219     jmp SYS_RET_OK ; no more, bye!
17220
17221     ; 17/05/2019 - Retro DOS v4.0

```

```

17222 ; DOSCODE:6029h (MSDOS 6.21, MSDOS.SYS)
17223 ;BREAK <$Rmdir -- Remove a directory>
17224 -----
17225 ;
17226 ; Procedure Name : $Rmdir
17227 ;
17228 ; Inputs:
17229 ; DS:DX Points to asciz name
17230 ; Function:
17231 ; Delete directory if empty
17232 ; Returns:
17233 ; STD XENIX Return
17234 ; AX = error_path_not_found If path bad
17235 ; AX = error_access_denied If
17236 ; Directory not empty
17237 ; Path not directory
17238 ; Root directory specified
17239 ; Directory malformed (. and .. not first two entries)
17240 ; User tries to delete a current directory
17241 ; AX = error_current_directory
17242 -----
17243 ;
17244 ; 10/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
17245 ; DOSCODE:6015h (MSDOS 5.0, MSDOS.SYS)
17246 ;
17247 ; 25/01/2025 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
17248 ; DOSCODE:66C8h (PCDOS 7.1, IBMDOS.COM)
17249
17250 _$RMDIR:
17251     push    dx                ; Save ptr to name
17252     push    ds
17253     mov     si,dx             ; Load ptr into si
17254     mov     di,OPENBUF        ; di = ptr to buf for trans name
17255     push    di
17256     call    TransPathNoSet     ; Translate the name
17257     pop     di                ; di = ptr to buf for trans name
17258     jnc     short rmlset       ; If transpath succeeded, continue
17259     pop     ds
17260     pop     dx                ; Restore the name
17261     mov     al,3
17262     mov     al,error_path_not_found ; Otherwise, return an error
17263     ; 16/12/2022
17264 rmdir_errj: ; 10/08/2018
17265 chdir_errj:
17266     jmp     short curdir_errj
17267     jmp     SYS_RET_ERR
17268
17269 rmlset:
17270     cmp     byte [ss:CMETA],-1 ; if (cMeta >= 0)
17271     jnz     short rmerr        ; return (-1);
17272     push    ss
17273     pop     es
17274     xor     al,al              ; al = 0 , ie drive a:
17275
17276 rmloop:
17277     call    GetCDSFromDrv      ; Get curdir for drive in al
17278     jc      short rmcont       ; If error, exit loop & cont normally
17279
17280 ; 25/01/2024 - Retro DOS v5.0
17281 ; (PCDOS 7.1 IBMDOS.COM)
17282 ;;;
17283 ;;;test word [si+43h],4000h
17284 ;;;test word [si+curdir.flags],curdir_inuse
17285 test byte [si+curdir.flags+1],(curdir_inuse>>8)
17286 jz short rmdir_nxt
17287 ;;;
17288
17289 call StrCmp ; Are the 2 paths the same?
17290 jz short rmerr ; Yes, report error.
17291
17292 rmdir_nxt: ; 25/01/2024
17293 inc al ; No, inc al to next drive number
17294 jmp short rmloop ; Go check next drive.
17295
17296 rmerr:
17297 pop ds
17298 pop dx ; Restore ptr the name
17299 mov al,10h
17300 mov al,error_current_directory ; error
17301 ; 16/12/2022
17302 ; 10/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
17303 ;chdir_errj:
17304 jmp short rmdir_errj
17305
17306 rmcont:
17307 pop ds
17308 pop dx ; Restore ptr the name
17309 MOV SI,DOS_RMDIR
17310
17311 ; 25/01/2024 - Retro DOS v5.0
17312 ; (PCDOS 7.1 IBMDOS.COM)
17313 ;;;
17314 call TestNet
17315 mov di,67 ; DIRSTRLEN
17316 ; 26/06/2024
17317 mov al,67
17318 jnc short rmcont2 ; local directory
17319 mov di,128
17320 mov al,128
17321
17322 rmcont2:
17323 ; 26/06/2024
17324 mov [ss:PATHNAMELEN],di
17325 mov [ss:PATHNAMELEN],al
17326 ;;;
17327 JMP DoDirCall
17328
17329 ; 17/05/2019 - Retro DOS v4.0
17330 ; DOSCODE:6065h (MSDOS 6.21, MSDOS.SYS)
17331 ;BREAK <$ChDir -- Change current directory on a drive>
17332 -----
17333 ;
17334 ; $ChDir - Top-level change directory system call. This call is responsible
17335 ; for setting up the CDS for the specified drive appropriately. There are
17336 ; several cases to consider:
17337 ;
17338 ; o Local, simple CDS. In this case, we take the input path and convert
17339 ; it into a WFP. We verify the existence of this directory and then
17340 ; copy the WFP into the CDS and set up the ID field to point to the
17341 ; directory cluster.
17342 ; o Net CDS. We form the path from the root (including network prefix)
17343 ; and verify its existence (via DOS_Chdir). If successful, we copy the
17344 ; WFP back into the CDS.
17345 ; o SUBST'ed CDS. This is no different than the local, simple CDS.
17346 ; o JOIN'ed CDS. This is trouble as there are two CDS's at work. If we

```

```

17346 ; call TransPath, we will get the PHYSICAL CDS that the path refers to
17347 ; and the PHYSICAL WFP that the input path refers to. This is perfectly
17348 ; good for the validation but not for currency. We call TransPathNoSet
17349 ; to process the path but to return the logical CDS and the logical
17350 ; path. We then copy the logical path into the logical CDS.
17351 ;
17352 ;
17353 ; Inputs:
17354 ; DS:DX Points to asciz name
17355 ; Returns:
17356 ; STD XENIX Return
17357 ; AX = chdir_path_not_found if error
17358 ;
17359 ;-----
17360 ; 25/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
17361
17362 _$CHDIR:
17363 ; 25/01/2024
17364 ;;;
17365 0000278C 36C606[5411]43 mov byte [ss:PATHNAMELEN],67 ; Retro DOS v5.0
17366 ;mov word [ss:PATHNAMELEN],67 ; DIRSTRLEN = 67
17367 ;;;
17368 00002792 BF[BE03] MOV DI,OPENBUF ; spot for translated name
17369 00002795 89D6 MOV SI,DX ; get source
17370 00002797 E8C454 call TransPath ; go munge the path and get real CDS
17371 0000279A 7304 JNC short ChDirCrack ; no errors, try path
17372 ChDirErrP:
17373 ;mov al,3
17374 0000279C B003 MOV AL,error_path_not_found
17375 ChDirErr:
17376 ;jmp SYS_RET_ERR ; oops!
17377 ; 10/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
17378 0000279E EBAF jmp short chdir_errj
17379
17380 ChDirCrack:
17381 000027A0 803E[7A05]FF CMP byte [CMETA],-1 ; No meta chars allowed.
17382 000027A5 75F5 JNZ short ChDirErrP
17383
17384 ; We cannot do a ChDir (yet) on a raw CDS. This is treated as a path not
17385 ; found.
17386
17387 000027A7 C43E[A205] LES DI,[THISCDS]
17388 000027AB 83FFFF CMP DI,-1 ; if (ThisCDS == NULL)
17389 000027AE 74EC JZ short ChDirErrP ; error ();
17390
17391 ; Find out if the directory exists.
17392
17393 000027B0 E87F12 call DOS_CHDIR
17394 ;jc short ChDirErr
17395 ; 16/12/2022
17396 000027B3 729A jc short chdir_errj
17397
17398 ; Get back CDS to see if a join as seen. Set the currency pointer (only if
17399 ; not network). If one was seen, all we need to do is copy in the text
17400 ;
17401 ; 25/01/2024 - Retro DOS v5.0
17402 000027B5 FF36[DC0A] push word [DIRSTART_HW] ; *
17403
17404 000027B9 C43E[A205] LES DI,[THISCDS]
17405 ;test word [es:di+43h],2000h
17406 ; 17/12/2022
17407 000027BD 26F6454420 test byte [ES:DI+curdir.flags+1],curdir_splice>>8
17408 ;TEST word [ES:DI+curdir.flags],curdir_splice
17409
17410 ; 25/01/2024 (PCDOS 7.1)
17411 ;mov dx,[DIRSTART_HW]
17412
17413 000027C2 742B JZ short GotCDS
17414
17415 ; The CDS was joined. Let's go back and grab the logical CDS.
17416
17417 000027C4 06 push es
17418 000027C5 57 push di
17419 ;push dx ; 25/01/2024 (PCDOS 7.1)
17420 000027C6 51 push cx ; word [DIRSTART] ; save CDS and cluster...
17421 000027C7 E8ADDC call Get_User_Stack ; get original text
17422
17423 ;mov di,[si+6]
17424 000027CA 8B7C06 MOV DI,[SI+user_env.user_DX]
17425 ;mov ds,[si+0Eh]
17426 000027CD 8E5C0E MOV DS,[SI+user_env.user_DS]
17427
17428 000027D0 BE[BE03] MOV SI,OPENBUF ; spot for translated name
17429 000027D3 87F7 XCHG SI,DI
17430 000027D5 30C0 XOR AL,AL ; do no splicing
17431 000027D7 57 push di
17432 000027D8 E88F54 call TransPathNoSet ; Munge path
17433 000027DB 5E pop si
17434
17435 ; There should NEVER be an error here.
17436
17437 ;IF FALSE
17438 ; JNC SkipErr
17439 ; fmt <>,<>,<"$p: Internal CHDIR error\n">
17440 ;SkipErr:
17441 ;ENDIF
17442 000027DC C43E[A205] LES DI,[THISCDS] ; get new CDS
17443 ;mov word [es:di+49h],-1
17444 000027E0 26C74549FFFF MOV word [ES:DI+curdir.ID],-1
17445 ; no valid cluster here...
17446 ;;; 25/01/2024 (PCDOS 7.1)
17447 ;mov word [es:di+4Bh],-1
17448 000027E6 26C7454BFFFF mov word [es:di+curdir.ID+2],-1
17449 ;;;
17450 000027EC 59 pop cx ; word [DIRSTART]
17451 ;pop dx ; 25/01/2024 (PCDOS 7.1)
17452 000027ED 5F pop di
17453 000027EE 07 pop es
17454
17455 ; ES:DI point to the physical CDS, CX is the ID (local only)
17456
17457 GotCDS:
17458 ; 25/01/2024 - Retro DOS v5.0
17459 000027EF 5A pop dx ; word [DIRSTART_HW] ; *
17460
17461 ; wfp_start points to the text. See if it is long enough
17462
17463 ; MSDOS 3.3
17464 ;push ss
17465 ;pop ds
17466 ;mov si,[WFP_START]
17467 ;push cx
17468 ;call DStrLen
17469 ;cmp cx,67 ; cmp cx,DIRSTRLEN

```

```

17470             ;pop     cx
17471             ;ja      short ChDirErrP
17472
17473             ; MSDOS 6.0
17474 000027F0 E85C00 CALL     Check_PathLen          ;PTM.          ;AN000;
17475 000027F3 77A7   JA      short ChDirErrP
17476             ; MSDOS 3.3 & MSDOS 6.0
17477             ;TEST word [ES:DI+curdir.flags],curdir_isnet ; 8000h
17478             ; 17/12/2022
17479 000027F5 26F6454480 test    byte [ES:DI+curdir.flags+1],curdir_isnet>>8
17480 000027FA 7518   JNZ     short SkipRecency
17481             ; MSDOS 6.0
17482             ;test word [es:di+43h],2000h
17483             ; 17/12/2022
17484 000027FC 26F6454420 test    byte [ES:DI+curdir.flags+1],curdir_splice>>8
17485             ;TEST word [ES:DI+curdir.flags],curdir_splice
17486             ;PTM. for Join and Subst ;AN000;
17487 00002801 7405   JZ      short setdirclus          ;PTM.          ;AN000;
17488 00002803 B9FFFF   MOV     CX,-1                      ;PTM.          ;AN000;
17489             ;;; 25/01/2024 (PCDOS 7.1 IBMDOS.COM)
17490 00002806 89CA   mov     dx,cx
17491 setdirclus:
17492             ;mov     [es:di+49h],cx
17493 00002808 26894D49   MOV     [ES:DI+curdir.ID],CX
17494             ;;; 25/01/2024 (PCDOS 7.1 IBMDOS.COM)
17495 0000280C 2689554B   mov     [es:di+curdir.ID+2],dx
17496             ;;;
17497 00002810 C43E[A205]   LES     DI,[THISCDS]              ; get logical CDS
17498 skipRecency:
17499             call    FStrCpy
17500 00002817 30C0   XOR     AL,AL
17501 mkdir_ok:
17502 00002819 E952DE   jmp     SYS_RET_OK
17503
17504             ; 17/05/2019 - Retro DOS v4.0
17505
17506             ; DOSCODE:60E1h (MSDOS 6.21, MSDOS.SYS)
17507
17508             ;BREAK <$MkDir - Make a directory entry>
17509             ;-----
17510             ;
17511             ; Procedure Name : $MkDir
17512             ; Inputs:
17513             ; DS:DX Points to asciz name
17514             ; Function:
17515             ; Make a new directory
17516             ; Returns:
17517             ; STD XENIX Return
17518             ; AX = mkdir_path_not_found if path bad
17519             ; AX = mkdir_access_denied If
17520             ; Directory cannot be created
17521             ; Node already exists
17522             ; Device name given
17523             ; Disk or directory(root) full
17524             ;-----
17525
17526             ; 10/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
17527
17528             ; 25/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
17529             ; PCDOS 7.1 IBMDOS.COM - DOSCODE:67B0h
17530
17531 _$MKDIR:
17532             ;MOV     SI,DOS_MKDIR
17533             ;;;
17534             ; 25/01/2024 (PCDOS 7.1 IBMDOS.COM)
17535 0000281C 36C606[5411]43 mov     byte [ss:PATHNAMELEN],67 ; Retro DOS v5.0
17536             ;mov     word [ss:PATHNAMELEN],67 ; DIRSTRLEN (standard length = 67)
17537 mkdir_x:
17538 00002822 BE[1D39]   mov     si,DOS_MKDIR
17539             ;;;
17540 doDirCall:
17541 00002825 BF[BE03]   MOV     DI,OPENBUF              ; spot for translated name
17542
17543             push    si
17544 00002829 89D6   MOV     SI,DX                  ; get source
17545 0000282B E83054   call    TransPath              ; go munge the path
17546 0000282E 5E     pop     si
17547 0000282F 7305   JNC     short MkDirCrack       ; no errors, try path
17548 MkErrP:
17549 00002831 B003   MOV     AL,error_path_not_found ; oops!
17550 MkErr:
17551 00002833 E942DE   jmp     SYS_RET_ERR
17552 MkDirCrack:
17553 00002836 36803E[7A05]FF CMP     byte [SS:CMETA],-1
17554 0000283C 75F3   JNZ     short MkErrP
17555
17556             ; MSDOS 3.3
17557             ;push    ss
17558             ;pop     ds
17559             ;call    si
17560             ;jb      short MkErr
17561             ;;;jmp    short mkdir_ok
17562             ;jmp     SYS_RET_OK
17563
17564             ; MSDOS 6.0
17565 0000283E 56     PUSH    SI                      ;PTM.          ;AN000;
17566             ; 25/01/2024 (PCDOS 7.1 IBMDOS.COM)
17567             ; check path len > [PATNAMELEN] ; 67 or 128
17568 0000283F E80D00   CALL    Check_PathLen          ;PTM. check path len > 67 ? ;AN000;
17569 00002842 5E     POP     SI                      ;PTM.          ;AN000;
17570 00002843 7604   JBE     short pathok          ;PTM.          ;AN000;
17571             ;mov     al,5
17572 00002845 B005   MOV     AL,error_access_denied;PTM. ops!
17573             ;jmp     SYS_RET_ERR          ;PTM.
17574 00002847 EBEA   jmp     short MkErr
17575 pathok:
17576 00002849 FFD6   CALL    SI                      ; go get file
17577 0000284B 72E6   JC      short MkErr           ; error
17578             ; 16/12/2022
17579             ; 10/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
17580 0000284D EBCA   jmp     short mkdir_ok        ; ok
17581             ;jmp     SYS_RET_OK
17582
17583             ;-----
17584             ;
17585             ; Procedure Name : Check_PathLen
17586             ;
17587             ; Inputs:
17588             ; nothing
17589             ; Function:
17590             ; check if final path length greater than 67
17591             ; Returns:
17592             ; Above flag set if > 67
17593             ;

```



```

17594 ;-----
17595 ;
17596 ; 25/01/2024 - Retro DOS v5.0
17597 ; (PCDOS 7.1 IBMDOS.COM)
17598
17599 Check_PathLen:
17600 ; 09/09/2018
17601 ;mov SI,[WFP_START]
17602 0000284F 368B36[B205] MOV SI,[SS:WFP_START] ; MSDOS 6.0
17603
17604 Check_PathLen2:
17605 ; 25/01/2024 - Retro DOS v5.0
17606 ; Note: dx is not changed in 'Check_PathLen';
17607 ; no need to push/pop (*)
17608 ;
17609 00002854 16 push ss
17610 00002855 1F pop ds
17611 00002856 51 ;mov SI,[WFP_START] ; MSDOS 3.3
17612 push CX
17613 ; (PCDOS 7.1 IBMDOS.COM)
17614 00002857 E87DEF ;push dx ; 25/01/2024 (*)
17615 CALL DStrLen
17616 0000285A 3B0E[5411] ; (PCDOS 7.1 IBMDOS.COM)
17617 cmp CX,[PATHNAMELEN] ; 67 or 128
17618 ;CMP CX,DIRSTRLEN ; 67
17619 0000285E 59 ;pop dx
17620 0000285F C3 POP CX
17621 retn
17622
17623 ;=====
17624 ; IOCTL.ASM, MSDOS 6.0, 1991
17625 ;=====
17626 ; 07/08/2018 - Retro DOS v3.0
17627 ; 17/05/2019 - Retro DOS v4.0
17628
17629 ** IOCTL system call.
17630 ;-----
17631 $IOCTL
17632 ;
17633 Revision history:
17634 ;
17635 Created: ARR 4 April 1983
17636 ;
17637 GenericIOCTL added: KGS 22 April 1985
17638 ;
17639 A000 version 4.00 Jan. 1988
17640 ;
17641 Used jump table to dispatch IOCTL functions. HKN 3/12/90
17642 ;
17643 ;BREAK <IOCTL - munge on a handle to do device specific stuff>
17644 ;-----
17645 ;
17646 Assembler usage:
17647 MOV BX, Handle
17648 MOV DX, Data
17649 ;
17650 (or LDS DX,BUF
17651 MOV CX,COUNT)
17652 ;
17653 MOV AH, Ioctl
17654 MOV AL, Request
17655 INT 21h
17656 ;
17657 AH = 0 Return a combination of low byte of sf_flags and device driver
17658 attribute word in DX, handle in BX:
17659 DH = high word of device driver attributes
17660 DL = low byte of sf_flags
17661 1 Set the bits contained in DX to sf_flags. DH MUST be 0. Handle
17662 in BX.
17663 2 Read CX bytes from the device control channel for handle in BX
17664 into DS:DX. Return number read in AX.
17665 3 Write CX bytes to the device control channel for handle in BX from
17666 DS:DX. Return bytes written in AX.
17667 4 Read CX bytes from the device control channel for drive in BX
17668 into DS:DX. Return number read in AX.
17669 5 Write CX bytes to the device control channel for drive in BX from
17670 DS:DX. Return bytes written in AX.
17671 6 Return input status of handle in BX. If a read will go to the
17672 device, AL = 0FFh, otherwise 0.
17673 7 Return output status of handle in BX. If a write will go to the
17674 device, AL = 0FFh, otherwise 0.
17675 8 Given a drive in BX, return 1 if the device contains non-
17676 removable media, 0 otherwise.
17677 9 Return the contents of the device attribute word in DX for the
17678 drive in BX. 0200h is the bit for shared. 1000h is the bit for
17679 network. 8000h is the bit for local use.
17680 A Return 8000h if the handle in BX is for the network or not.
17681 B Change the retry delay and the retry count for the system. BX is
17682 the count and CX is the delay.
17683
17684 Error returns:
17685 AX = error_invalid_handle
17686 = error_invalid_function
17687 = error_invalid_data
17688 ;-----
17689 ;
17690 This is the documentation copied from DOS 4.0 it is much better
17691 than the above
17692 ;
17693 There are several basic forms of IOCTL calls:
17694 ;
17695 ** Get/Set device information: **
17696 ;
17697 ENTRY (AL) = function code
17698 0 - Get device information
17699 1 - Set device information
17700 (BX) = file handle
17701 (DX) = info for "Set Device Information"
17702 EXIT 'C' set if error
17703 (AX) = error code
17704 'C' clear if OK
17705 (DX) = info for "Get Device Information"
17706
17707 USES ALL
17708 ;
17709 ** Read/Write Control Data From/To Handle **
17710 ;
17711 ENTRY (AL) = function code
17712 2 - Read device control info
17713 3 - Write device control info
17714 (BX) = file handle
17715 (CX) = transfer count
17716 ;
17717 ;

```

```

17718 ; (DS:DX) = address for data
17719 ; EXIT 'C' set if error
17720 ; (AX) = error code
17721 ; 'C' clear if OK
17722 ; (AX) = count of bytes transfered
17723 ; USES ALL
17724 ;
17725 ;
17726 ; ** Read/Write Control Data From/To Block Device **
17727 ;
17728 ; ENTRY (AL) = function code
17729 ; 4 - Read device control info
17730 ; 5 - Write device control info
17731 ; (BL) = Drive number (0=default, 1='A', 2='B', etc)
17732 ; (CX) = transfer count
17733 ; (DS:DX) = address for data
17734 ; EXIT 'C' set if error
17735 ; (AX) = error code
17736 ; 'C' clear if OK
17737 ; (AX) = count of bytes transfered
17738 ; USES ALL
17739 ;
17740 ;
17741 ; ** Get Input/Output Status **
17742 ;
17743 ; ENTRY (AL) = function code
17744 ; 6 - Get Input status
17745 ; 7 - Get Output Status
17746 ; (BX) = file handle
17747 ; EXIT 'C' set if error
17748 ; (AX) = error code
17749 ; 'C' clear if OK
17750 ; (AL) = 00 if not ready
17751 ; (AL) = FF if ready
17752 ; USES ALL
17753 ;
17754 ;
17755 ; ** Get Drive Information **
17756 ;
17757 ; ENTRY (AL) = function code
17758 ; 8 - check for removable media
17759 ; 9 - Get device attributes
17760 ; (BL) = Drive number (0=default, 1='A', 2='B', etc)
17761 ; EXIT 'C' set if error
17762 ; (AX) = error code
17763 ; 'C' clear if OK
17764 ; (AX) = 0/1 media is removable/fixed (func. 8)
17765 ; (DX) = device attribute word (func. 9)
17766 ; USES ALL
17767 ;
17768 ;
17769 ; ** Get Redirected bit**
17770 ;
17771 ; ENTRY (AL) = function code
17772 ; 0Ah - Network stuff
17773 ; (BX) = file handle
17774 ; EXIT 'C' set if error
17775 ; (AX) = error code
17776 ; 'C' clear if OK
17777 ; (DX) = SFT flags word, 8000h set if network file
17778 ; USES ALL
17779 ;
17780 ;
17781 ; ** Change sharer retry parameters **
17782 ;
17783 ; ENTRY (AL) = function code
17784 ; 0Bh - Set retry parameters
17785 ; (CX) = retry loop count
17786 ; (DX) = number of retries
17787 ; EXIT 'C' set if error
17788 ; (AX) = error code
17789 ; 'C' clear if OK
17790 ; USES ALL
17791 ;
17792 ;
17793 ;
17794 ;
17795 ; ** New Standard Control **
17796 ;
17797 ; ALL NEW IOCTL FACILITIES SHOULD USE THIS FORM. THE OTHER
17798 ; FORMS ARE OBSOLETE.
17799 ;
17800 ;
17801 ;
17802 ; ENTRY (AL) = function code
17803 ; 0Ch - Control Function subcode
17804 ; (BX) = File Handle
17805 ; (CH) = Category Indicator
17806 ; (CL) = Function within category
17807 ; (DS:DX) = address for data, if any
17808 ; (SI) = Passed to device as argument, use depends upon function
17809 ; (DI) = Passed to device as argument, use depends upon function
17810 ; EXIT 'C' set if error
17811 ; (AX) = error code
17812 ; 'C' clear if OK
17813 ; (SI) = Return value, meaning is function dependent
17814 ; (DI) = Return value, meaning is function dependent
17815 ; (DS:DX) = Return address, use is function dependent
17816 ; USES ALL
17817 ;
17818 ; ===== Generic IOCTL Definitions for DOS 3.2 =====
17819 ; (See inc\ioctl.inc for more info)
17820 ;
17821 ; ENTRY (AL) = function code
17822 ; 0Dh - Control Function subcode
17823 ; (BL) = Drive Number (0 = Default, 1= 'A')
17824 ; (CH) = Category Indicator
17825 ; (CL) = Function within category
17826 ; (DS:DX) = address for data, if any
17827 ; (SI) = Passed to device as argument, use depends upon function
17828 ; (DI) = Passed to device as argument, use depends upon function
17829 ;
17830 ; EXIT 'C' set if error
17831 ; (AX) = error code
17832 ; 'C' clear if OK
17833 ; (DS:DX) = Return address, use is function dependent
17834 ; USES ALL
17835 ;
17836 ; -----
17837 ;
17838 ; 17/05/2019 - Retro DOS v4.0
17839 ; DOSCODE:611Eh (MSDOS 6.21, MSDOS.SYS)
17840 ;
17841 ; 11/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)

```



```

17842 ; DOSCODE:610Ah (MSDOS 5.0, MSDOS.SYS)
17843
17844 ; 14/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
17845 ; DOSCODE:67F7h (PCDOS 7.1, IBMDOS.COM)
17846
17847 IOCTLJMPTABLE: ;label word
17848 ; MSDOS 3.3 (& MSDOS 6.0)
17849 00002860 [9C28] dw ioctl_getset_data ; 0
17850 00002862 [9C28] dw ioctl_getset_data ; 1
17851 00002864 [E628] dw ioctl_control_string ; 2
17852 00002866 [E628] dw ioctl_control_string ; 3
17853 00002868 [7E2A] dw ioctl_get_dev ; 4
17854 0000286A [7E2A] dw ioctl_get_dev ; 5
17855 0000286C [FE28] dw ioctl_status ; 6
17856 0000286E [FE28] dw ioctl_status ; 7
17857 00002870 [D829] dw ioctl_rem_media ; 8
17858 00002872 [212A] dw ioctl_drive_attr ; 9
17859 00002874 [702A] dw ioctl_handle_redir ; A
17860 00002876 [132A] dw Set_Retry_Parameters ; B
17861 00002878 [1A29] dw GENERICIOCTLHANDLE ; C
17862 0000287A [3429] dw GENERICIOCTL ; D
17863 ; MSDOS 6.0 (& MSDOS 3.3)
17864 0000287C [1F2B] dw ioctl_drive_owner ; E
17865 0000287E [1F2B] dw ioctl_drive_owner ; F
17866 ; MSDOS 6.0
17867 00002880 [1A29] dw query_handle_support ; 10h
17868 00002882 [3429] dw query_device_support ; 11h
17869
17870 ; 11/11/2022
17871 _$IOCTL:
17872 00002884 8CDE MOV SI,DS ; Stash DS for calls 2,3,4 and 5
17873 00002886 16 push ss
17874 00002887 1F pop ds ;hkn; SS is DOSDATA
17875
17876 ; MSDOS 3.3
17877 ;cmp al,0Fh
17878 ; MSDOS 6.0
17879 00002888 3C11 cmp al,11h ; al must be between 0 & 11h
17880 0000288A 770D ja short ioctl_bad_funj2 ; if not bad function #
17881
17882 ; 14/01/2024
17883 ; 28/05/2019
17884 ;push AX ; 14/01/2024 ; Need to save AL for generic IOCTL
17885 0000288C 89C7 mov di,ax ; di NOT a PARM
17886 0000288E 81E7FF00 and di,0FFh ; di = al
17887 00002892 D1E7 shl di,1 ; di = index into jmp table
17888 ;pop AX ; Restore AL for generic IOCTL
17889
17890 00002894 2EFA5[6028] jmp word [CS:DI+IOCTLJMPTABLE]
17891
17892 ioctl_bad_funj2:
17893 00002899 E90401 JMP ioctl_bad_fun ; 10/08/2018
17894
17895 ;-----
17896 ;
17897 ; IOCTL: AL = 0,1
17898 ;
17899 ; ENTRY: DS = DOSDATA
17900 ;-----
17901
17902 ; 29/01/2024 - Retro DOS v5.0
17903 ioctl_getset_data:
17904 ; MSDOS 6.0
17905 0000289C E8344E call SFFromHandle ; ES:DI -> SFT
17906 0000289F 7305 JNC short ioctl_check_permissions ; have valid handle
17907
17908 ioctl_bad_handle:
17909 ;mov al,6
17910 000028A1 B006 mov al,error_invalid_handle
17911
17912 000028A3 E9D2DD ioctl_error:
17913 jmp SYS_RET_ERR
17914
17915 ioctl_check_permissions:
17916 CMP AL,0
17917 ;mov al,[es:di+5]
17918 MOV AL,[ES:DI+SF_ENTRY.sf_flags]; Get low byte of flags
17919 JZ short ioctl_read ; read the byte
17920
17921 or dh,dh
17922 JZ short ioctl_check_device ; can I set with this data?
17923 ;mov al,0Dh
17924 mov al,error_invalid_data ; no DH <> 0
17925 ;jmp SYS_RET_ERR
17926 jmp short ioctl_error
17927
17928 ioctl_check_device:
17929 test AL,devid_device ; 80h; can I set this handle?
17930 jz short ioctl_bad_funj2
17931 OR DL,devid_device ; Make sure user doesn't turn off the
17932 ; device bit!! He can muck with the
17933 ; others at will.
17934 ;MOV byte [EXTERR_LOCUS],errLOC_SerDev ; 4
17935 ; 29/01/2024
17936 ;;;
17937 call set_exerr_locus_ser ; (PCDOS 7.1 IBMDOS.COM)
17938 ;;;
17939 MOV BYTE [ES:DI+SF_ENTRY.sf_flags],DL ; AC000;MS.; Set flags
17940
17941 ioctl_ok:
17942 jmp SYS_RET_OK
17943
17944 ioctl_read:
17945 ;MOV byte [EXTERR_LOCUS],errLOC_Disk ; 2
17946 ; 29/01/2024
17947 ;;;
17948 call set_exerr_locus_disk ; (PCDOS 7.1 IBMDOS.COM)
17949 ;;;
17950 XOR AH,AH
17951 test AL,devid_device ; Should I set high byte
17952 JZ short ioctl_no_high ; no
17953 ;MOV byte [EXTERR_LOCUS],errLOC_SerDev ; 4
17954 ; 29/01/2024
17955 ;;;
17956 call set_exerr_locus_ser ; (PCDOS 7.1 IBMDOS.COM)
17957 ;;;
17958 ;les di,[es:di+7]
17959 LES DI,[ES:DI+SF_ENTRY.sf_devptr] ; Get device pointer
17960 ;mov ah,[es:di+5]
17961 MOV AH,[ES:DI+SYSDEV.ATT+1] ; Get high byte
17962
17963 ioctl_no_high:
17964 MOV DX,AX
17965
17966 ioctl_set_dx:
17967 ; 16/12/2022
17968 call Get_User_Stack
17969 ;mov [si+6],dx
17970 MOV [SI+user_env.user_DX],DX

```

```

17966             ;;jmp SYS_RET_OK
17967             ; 11/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
17968 ioctl_ok_j:
17969             ; 16/12/2022
17970 000028E3 E98BDD jmp SYS_RET_OK_c1c ; (after 'Get_User_Stack')
17971             ; jmp short ioctl_ok
17972             ; 26/07/2019
17973             ; jmp SYS_RET_OK_c1c
17974
17975
17976 -----
17977 ; IOCTL: AL = 2,3
17978
17979 ; ENTRY: DS = DOSDATA
17980 ; SI = user's DS
17981
17982 -----
17983
17984             ; 07/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
17985 ioctl_control_string:
17986 000028E6 E8EA4D call SFFromHandle ; ES:DI -> SFT
17987 000028E9 72B6 jc short ioctl_bad_handle; invalid handle
17988             ; 07/12/2022
17989 ;TEST word [ES:DI+SF_ENTRY.sf_flags],devid_device ; can I?
17990 ;jz short ioctl_bad_funj2 ; No it is a file
17991             ; MSDOS 5.0 & MSDOS 6.0
17992 000028EB 26F6450580 test byte [ES:DI+SF_ENTRY.sf_flags],devid_device ; can I?
17993 000028F0 74A7 jz short ioctl_bad_funj2 ; No it is a file
17994             ;MOV byte [EXERR_LOCUS],errLOC_SerDev ; 4
17995             ; 29/01/2024
17996             ;;;
17997 000028F2 E8F6E9 call set_exerr_locus_ser ; (PCDOS 7.1 IBMDOS.COM)
17998             ;;;
17999 000028F5 26C47D07 les di,[ES:DI+SF_ENTRY.sf_devptr] ; Get device pointer
18000 000028F9 30DB xor bl,bl ; Unit number of char dev = 0
18001 000028FB E98801 jmp ioctl_do_string
18002
18003 -----
18004 ; IOCTL: AL = 6,7
18005
18006 ; ENTRY: DS = DOSDATA
18007
18008 -----
18009
18010 ioctl_status:
18011 MOV AH,1
18012 000028FE B401 SUB AL,6 ; 6=0,7=1
18013 00002900 2C06 JZ short ioctl_get_status
18014 00002902 7402 MOV AH,3
18015 00002904 B403
18016 ioctl_get_status:
18017 00002906 50 PUSH AX
18018 00002907 E8BF15 call GET_IO_SFT
18019 0000290A 58 POP AX
18020             ;JNC short DO_IOFUNC
18021             ;JMP short ioctl_bad_handle; invalid SFT
18022             ; 16/12/2022
18023 0000290B 7294 jc short ioctl_bad_handle
18024 DO_IOFUNC:
18025 0000290D E8B827 call IOFUNC
18026 00002910 88C4 MOV AH,AL
18027 00002912 B0FF MOV AL,0FFh
18028             ;JNZ short ioctl_status_ret
18029             ; 29/01/2024
18030 00002914 75AE jnz short ioctl_ok
18031 00002916 FEC0 INC AL
18032 ioctl_status_ret:
18033             ; jmp SYS_RET_OK
18034             ; 11/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
18035             ; jmp short ioctl_ok_j
18036             ; 16/12/2022
18037 00002918 EBAA jmp short ioctl_ok
18038
18039 -----
18040 ; Generic IOCTL entry point. AL = C, D, 10h, 11h
18041
18042 ; here we invoke the Generic IOCTL using the IOCTL_Req structure.
18043 ; SI:DX -> Users Device Parameter Table
18044 ; IOCALL -> IOCTL_Req structure
18045
18046 ; If on entry AL >= IOCTL_QUERY_HANDLE the function is a
18047 ; QueryIOctlsupport call ELSE it's a standard generic IOct1
18048 ; call.
18049
18050 ; BUGBUG: Don't push anything on the stack between GENERIOCTL: and
18051 ; the call to Check_If_Net because Check_If_Net gets our
18052 ; return address off the stack if the drive is invalid.
18053
18054 -----
18055
18056             ; 29/01/2024 - Retro DOS v5.0
18057
18058 query_handle_support: ; Entry point for handles
18059 GENERICIOCTLHANDLE:
18060             call SFFromHandle ; Get SFT for device.
18061 0000291A E8B64D jc short ioctl_bad_handlej
18062 0000291D 727E
18063
18064 ;test word [es:di+5],8000h
18065 ;TEST word [ES:DI+SF_ENTRY.sf_flags],sf_isnet ; M031;
18066 ;test byte [es:di+6],80h
18067 0000291F 26F6450680 test byte [ES:DI+SF_ENTRY.sf_flags+1],(sf_isnet>>8)
18068 00002924 757A jnz short ioctl_bad_fun ; Cannot do this over net.
18069
18070 ;mov byte [EXERR_LOCUS],errLOC_SerDev ; 4
18071             ; 29/01/2024
18072             ;;;
18073 00002926 E8C2E9 call set_exerr_locus_ser ; (PCDOS 7.1 IBMDOS.COM)
18074             ;;;
18075
18076 ;les di,[es:di+7]
18077 00002929 26C47D07 les di,[es:di+SF_ENTRY.sf_devptr] ; Get pointer to device.
18078             ;;;
18079             ; 29/01/2024 - PC DOS 7.1 IBMDOS.COM
18080 0000292D C606[B103]FF mov byte [IOCTL_drvnum],0FFh ; invalidate drive number
18081             ; ; (for extended -lock/unlock- functions)
18082             ;;;
18083 00002932 EB16 jmp short Do_GenIOCTL
18084
18085             ; 29/01/2024 - Retro DOS v5.0
18086             ; PC DOS 7.1 IBMDOS.COM - DOSCODE:68DAh
18087             ; (WINME IO.SYS - BIOSCODE:6612h)
18088
18089 query_device_support: ; Entry point for devices:

```

```

18090      GENERICIOCTL:
18091      ;mov     byte [EXERR_LOCUS],errLOC_Disk ; 2
18092      ; 29/01/2024
18093      ;;;
18094      00002934 E8AFE9      call     set_exerr_locus_disk
18095      00002937 80FD48      cmp      ch,48h          ; category (extended, disk lock/unlock)
18096      0000293A 7405      je       short GenIOCTL_chk_net      ; extended (MSDOS/PCDOS 7)
18097      ;;;
18098
18099      0000293C 80FD08      cmp      ch,IOC_DC ; 8          ; Only disk devices are allowed to use
18100      0000293F 755F      jne      short ioctl_bad_fun      ; no handles with Generic IOCTL.
18101
18102      ; 29/01/2024
18103      ;;;
18104      GenIOCTL_chk_net:
18105      00002941 881E[B103]      mov      [IOCTL_drvnum],bl      ; drive number
18106      ;;;
18107
18108      00002945 E8C801      CALL     Check_If_Net          ; ES:DI := Get_hdr_block of device in BL
18109      00002948 7556      JNZ      short ioctl_bad_fun      ; There are no "net devices", and they
18110
18111      Do_GenIOCTL:
18112      ; 29/01/2024 - Retro DOS v5.0
18113      ;;;
18114      0000294A 80FD48      cmp      ch,48h          ; category code 48h for FAT32
18115      0000294D 7456      je       short GenIOCTL_extended ; MSDOS/PCDOS 7 functions (lock/unlock)
18116      ;cmp      ch,8
18117      0000294F 80FD08      cmp      ch,IOC_DC ; 8          ; disk control
18118      00002952 7451      je       short GenIOCTL_extended
18119      GenIOCTL_normal:
18120      ;;;
18121
18122      ;TEST     word [ES:DI+SYSDEV.ATT],DEV320
18123      ; Can device handle Generic IOCTL funcs
18124
18125      ; 09/09/2018
18126      00002954 26F6450440      test     byte [es:di+4],40h
18127      00002959 7445      TEST     byte [ES:DI+SYSDEV.ATT],DEV320 ; 0040h
18128      jz       short ioctl_bad_fun
18129
18130      ; 17/05/2019 - Retro DOS v4.0
18131
18132      ; MSDOS 6.0
18133      0000295B C606[7E03]13      mov      byte [IOCALL_REQFUNC],19 ; 13h
18134      mov      byte [IOCALL_REQFUNC],GENIOCTL ; Assume real Request
18135      ;cmp      al,10h
18136      00002962 7C0C      cmp      AL,IOCTL_QUERY_HANDLE ; See if this is just a query
18137      jl       short SetIOctlBlock
18138
18139      ;TEST     word [ES:DI+SYSDEV.ATT],IOQUERY ; See if device supports a query
18140      00002964 26F6450480      test     byte [es:di+4],80h
18141      00002969 7435      TEST     byte [ES:DI+SYSDEV.ATT],IOQUERY ; See if device supports a query
18142      jz       short ioctl_bad_fun      ; No support for query
18143
18144      ;mov      byte [IOCALL_REQFUNC],19h
18145      mov      byte [IOCALL_REQFUNC],IOCTL_QUERY ; Just a query (5.00)
18146
18147      00002970 06      SetIOctlBlock:
18148      00002971 57      PUSH     ES          ; DEVIOCALL2 expects Device header block
18149      PUSH     DI          ; in DS:SI
18150      ; Setup Generic IOCTL Request Block
18151      00002972 C606[7C03]17      ;mov      byte [IOCALL_REQLEN],23
18152      mov      byte [IOCALL_REQLEN],IOCTL_REQ.size
18153      ; 07/09/2018 (MSDOS 3.3)
18154      ;mov      byte [IOCALL_REQFUNC],19
18155      ;mov      byte [IOCALL_REQFUNC],GENIOCTL ; 07/09/2018
18156
18157      00002977 881E[7D03]      MOV      [IOCALL_REQUNIT],BL
18158      0000297B 882E[8903]      MOV      [IOCALL+IOCTL_REQ.MAJORFUNCTION],CH
18159      0000297F 880E[8A03]      MOV      [IOCALL+IOCTL_REQ.MINORFUNCTION],CL
18160      00002983 8936[8B03]      MOV      [IOCALL+IOCTL_REQ.REG_SI],SI
18161      00002987 893E[8D03]      MOV      [IOCALL+IOCTL_REQ.REG_DI],DI
18162      0000298B 8916[8F03]      MOV      [IOCALL+IOCTL_REQ.GENERICIOCTL_PACKET],DX
18163      0000298F 8936[9103]      MOV      [IOCALL+IOCTL_REQ.GENERICIOCTL_PACKET+2],SI
18164
18165      00002993 BB[7C03]      ;hkn; IOCALL is in DOSDATA
18166      MOV      BX,IOCALL
18167
18168      00002996 16      PUSH     SS
18169      00002997 07      POP      ES
18170
18171      00002998 5E      POP      SI          ; DS:SI -> Device header.
18172      00002999 1F      POP      DS
18173      0000299A E92201      ; 10/08/2018
18174      jmp      ioctl_do_IO          ; Perform Call to device driver
18175
18176      0000299D E901FF      ioctl_bad_handlej:
18177      jmp      ioctl_bad_handle
18178
18179      ; 29/01/2024
18180      000029A0 B001      ioctl_bad_fun:
18181      000029A2 E9D3DC      mov      al, error_invalid_function ; 1
18182      jmp      SYS_RET_ERR
18183
18184      ; 29/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
18185      000029A5 80F96A      GenIOCTL_extended:
18186      cmp      cl,6Ah          ; UNLOCK LOGICAL VOLUME
18187      000029A8 740E      ;je       short GenIOCTL_chk_lock
18188      000029AA 80F94A      je       short GenIOCTL_lock_unlock
18189      000029AD 75A5      cmp      cl,4Ah          ; LOCK LOGICAL VOLUME
18190      jne      short GenIOCTL_normal
18191      ;GenIOCTL_chk_lock:
18192      ;cmp      cl,4Ah          ; LOCK LOGICAL VOLUME
18193      000029AF 80FF04      ;jne      short GenIOCTL_lock_unlock
18194      000029B2 7404      cmp      bh,4          ; lock level (0-4)
18195      000029B4 08FF      je       short GenIOCTL_lock_unlock
18196      000029B6 75E8      or       bh,bh
18197      jnz      short ioctl_bad_fun
18198      000029B8 8A1E[B103]      GenIOCTL_lock_unlock:
18199      000029BC 30FF      mov      bl,[IOCTL_drvnum]      ; drive number (1=A:, 2=B: ..)
18200      000029BE 4B      xor      bh,bh
18201      000029BF 80FB1A      dec      bx
18202      000029C2 73DC      cmp      bl,26          ; logical disk number limit
18203      000029C4 80F96A      jnb      short ioctl_bad_fun
18204      000029C7 7507      cmp      cl,6Ah          ; UNLOCK LOGICAL VOLUME
18205      000029C9 80A7[0813]7F      jne      short GenIOCTL_lock
18206      000029CE EB05      and      byte [bx+drive_flags],7Fh ; UNLOCK
18207      jmp      short GenIOCTL_OK
18208      000029D0 808F[0813]80      GenIOCTL_lock:
18209      or       byte [bx+drive_flags],80h ; LOCK
18210      000029D5 E996DC      GenIOCTL_OK:
18211      jmp      SYS_RET_OK
18212
18213      ;-----
18214      ;

```

```

18214 ; IOCTL: AL = 8
18215 ;
18216 ; ENTRY: DS = DOSDATA
18217 ;
18218 ; BUGBUG: Don't push anything on the stack between ioctl_rem_media: and
18219 ; the call to Check_If_Net because Check_If_Net gets our
18220 ; return address off the stack if the drive is invalid.
18221 ;
18222 ;-----
18223 ;
18224 ; 30/01/2024
18225 ;
18226 ; ioctl_rem_media:
18227 ; MSDOS 3.3 (& MSDOS 6.0)
18228 000029D8 E83501 CALL Check_If_Net
18229 000029DB 75C3 JNZ short ioctl_bad_fun ; There are no "net devices", and they
18230 ; ; certainly don't know how to do this
18231 ; ; call.
18232 ;test word [es:di+4],800h
18233 ;TEST word [ES:DI+SYSDEV.ATT],DEVOPCL ; See if device can
18234 ;test byte [es:di+5],8
18235 000029DD 26F6450508 TEST byte [es:di+SYSDEV.ATT+1],(DEVOPCL>>8)
18236 000029E2 74BC JZ short ioctl_bad_fun ; NO
18237 ;
18238 ;hkn; SS override for IOCALL
18239 ; 30/01/2024
18240 ; ds = ss = DOSDATA segment ('Get_Driver_BL' in 'Check_If_Net')
18241 000029E4 C606[7E03]0F ;MOV byte [SS:IOCALL_REQFUNC],DEVRMD ; 15
18242 000029E9 B00D mov byte [IOCALL_REQFUNC],DEVRMD ; 15
18243 000029EB 88DC MOV AL,REMH ; 13
18244 ; MOV AH,BL ; Unit number
18245 000029ED A3[7C03] ;MOV [SS:IOCALL_REQLEN],AX
18246 000029F0 31C0 mov [IOCALL_REQLEN],ax
18247 ; XOR AX,AX
18248 000029F2 A3[7F03] ;MOV [SS:IOCALL_REQSTAT],AX
18249 mov [IOCALL_REQSTAT],ax ; 0
18250 000029F5 06 PUSH ES
18251 000029F6 1F POP DS
18252 000029F7 89FE MOV SI,DI ; DS:SI -> driver
18253 000029F9 16 PUSH SS
18254 000029FA 07 POP ES
18255 ;
18256 ;hkn; IOCALL is in DOSDATA (msconst.asm)
18257 ; 30/01/2024
18258 ; (ds <> ss, ss = DOSDATA segment)
18259 000029FB 8B[7C03] MOV BX,IOCALL ; ES:BX -> Call header
18260 000029FE 1E push ds
18261 000029FF 56 push si
18262 00002A00 E89228 call DEVIOCALL2
18263 00002A03 5E pop si
18264 00002A04 1F pop ds
18265 ;
18266 ;hkn; SS override
18267 00002A05 36A1[7F03] MOV AX,[SS:IOCALL_REQSTAT]; Get Status word
18268 ; AND AX,STBUI ; 200h ; Mask to busy bit
18269 ; 29/01/2024
18270 00002A09 80E402 and ah,STBUI>>8 ; 2
18271 00002A0C 8109 MOV CL,9
18272 00002A0E D3E8 SHR AX,CL ; Busy bit to bit 0
18273 ;
18274 00002A10 E95BDC ioctl_da_ok_j: ; 11/11/2022
18275 jmp SYS_RET_OK
18276 ; 29/01/2024
18277 ;-----
18278 ;
18279 ; IOCTL: AL = B
18280 ;
18281 ; ENTRY: DS = DOSDATA
18282 ;
18283 ;-----
18284 ;
18285 ; Set_Retry_Parameters:
18286 ; 09/09/2018
18287 00002A13 890E[1C00] MOV [RetryLoop],CX ; 0 retry loop count allowed
18288 00002A17 09D2 OR DX,DX ; zero retries not allowed
18289 00002A19 7485 JZ short ioctl_bad_fun
18290 ; 29/01/2024
18291 ;jnz short set_new_retry_cnt
18292 ;jmp ioctl_bad_fun
18293 ;set_new_retry_cnt:
18294 00002A1B 8916[1A00] MOV [RetryCount],DX ; Set new retry count
18295 doneok:
18296 ;jmp SYS_RET_OK ; Done
18297 ; 11/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
18298 ;jmp short ioctl_status_ret
18299 ; 16/12/2022
18300 ;jmp short ioctl_ok ; jmp SYS_RET_OK
18301 ; 29/01/2024
18302 00002A1F EBEF jmp short ioctl_da_ok_j
18303 ;-----
18304 ;
18305 ; IOCTL: AL = 9
18306 ;
18307 ; ENTRY: DS = DOSDATA
18308 ;
18309 ;-----
18310 ;
18311 ; 30/01/2024 - Retro DOS v5.0
18312 ;
18313 ; ioctl_drive_attr:
18314 ; MSDOS 3.3 (& MSDOS 6.0)
18315 mov al,bl
18316 00002A21 88D8 call GETTHISDRV
18317 00002A23 E86751 jc short ioctl_drv_err
18318 00002A26 7243 call Get_Driver_BL
18319 00002A28 E8BA00 ; MSDOS 6.0
18320 JC short ioctl_drv_err ; drive not valid
18321 00002A2B 723E ;
18322 ; 03/07/2024
18323 ; jnz = net drive (current dir is net)
18324 ; jz = local drive
18325 ;
18326 ; 03/07/2024
18327 ; 30/01/2024 - Retro DOS v5.0
18328 ; 30/01/2024 - PCDOS 7.1 IBMDOS.COM
18329 ;;;
18330 mov dx,942h ; 0942h -> Attribute word
18331 ; bit 11 - open/close/remmedia calls supported
18332 ; bit 8 - (new type driver)
18333 ; bit 6 - Generic IOCTL call supported
18334 ; bit 1 - driver supports 32-bit sector addressing
18335 00002A30 7504 jnz short ioctl_drive_attr2 ; NET device
18336 ; 30/01/2024
18337 ;

```

```

18338                                     ; NOTE: 'jnz' condition is correct for windows ME
18339                                     ; 'Get_Driver_BL' because it tests bit 0 of [drive_flags]
18340                                     ; but in PC DOS 7.1 'Get_Driver_BL', this flag
18341                                     ; (remote or removable? disk flag) is not tested
18342                                     ; the last test is net device test) ; (!)
18343                                     ;;;;
18344
18345                                     ;mov     dx,[es:di+4]
18346 00002A32 268B5504                 mov     dx,[es:di+SYSDEV.ATT]
18347                                     ; get device attribute word
18348
18349                                     ; 26/06/2024
18350 00002A36 88C3                     MOV     BL,AL ; Phys letter to BL (A=0)
18351
18352 ;hkn; SS override
18353 ; 30/01/2024
18354 ; (MSDOS 6.22 IO.SYS - DOSCODE:62B8h)
18355 ; (PCDOS 7.1 IBMDOS.COM - DOSCODE:69DBh)
18356 ;LES     DI,[SS:THISCDS] ; NOTE: PC DOS 7.1 has bug here,
18357 ; ds must be same with ss here...
18358 ; because there is 'les di, [ds:THISCDS]' in
18359 ; Get_Driver_BL
18360 ; and a second 'test byte ptr es:[di+44h],80h'
18361 ; is not necessary; also its result (jnz)
18362 ; overwrites DS. /// Erdogan Tan - 30/01/2024
18363
18364 ; 30/01/2024
18365 ; Retro DOS v5.0
18366 00002A38 C43E[A205]             les     di,[THISCDS]
18367
18368 ;test     word [es:di+43h],8000h
18369 ;TEST     word [ES:DI+curdir.flags],curdir_isnet
18370 ;test     byte [es:di+44h],80h
18371 00002A3C 26F6454480             TEST     byte [ES:DI+curdir.flags+1],(curdir_isnet>>8) ; (!)
18372 00002A41 7403                     JZ      short IOCTLShare
18373
18374 ;or        dx,1000h ; (MSDOS 3.3)
18375
18376 ; Net devices don't return a device attribute word.
18377 ; Bit 12 = 1, meaning net device, all others = 0.
18378
18379 00002A43 BA0010                 MOV     DX,1000h ; MSDOS 6.0
18380
18381 IOCTLShare:
18382 ; 30/01/2024
18383 ; ds = ss = DOSDATA segment
18384 ;push     ss
18385 ;pop      ds
18386
18387 00002A46 BE[BE03]               MOV     SI,OPENBUF
18388 00002A49 80C341                 ADD     BL,"A" ; 41h
18389 00002A4C 881C                     MOV     [SI],BL
18390 00002A4E C744013A00             MOV     WORD [SI+1],003AH ; ":",0
18391 00002A53 B80003                 MOV     AX,0300h
18392 00002A56 F8                     CLC
18393 ;INT      int_IBM
18394 00002A57 CD2A                 int     2Ah ; Microsoft Networks - CHECK DIRECT I/O
18395 ; DS:SI -> ASCIZ disk device name
18396 ; (may be full path or only drive
18397 ; specifier--must include the colon)
18398 ; Return: CF clear if absolute disk access allowed
18399 00002A59 7303                 JNC     short IOCTLLocal ; Not shared
18400 ;OR        DX,0200H ; Shared, bit 9
18401 ; 17/12/2022
18402 00002A5B 80CE02                 or      dh,02h
18403
18404 IOCTLLocal:
18405 ;test     word [es:di+43h],1000h
18406 ;TEST     word [ES:DI+curdir.flags],curdir_local
18407 ;test     byte [es:di+44h],10h
18408 ;TEST     byte [ES:DI+curdir.flags+1],(curdir_local>>8)
18409 ;JZ      short ioctl_set_DX
18410 ; 16/12/2022
18411 00002A63 7403                     jz      short _ioctl_set_DX
18412 ;OR        DX,8000h
18413 ; 17/12/2022
18414 ;or        dh,80h
18415 ;ioctl_set_DX:
18416 ;_ioctl_set_DX:
18417 00002A68 E972FE                 jmp     ioctl_set_dx
18418 ; 16/12/2022
18419 %if 0
18420 call     Get_User_Stack
18421 MOV     [SI+user_env.user_DX],DX
18422 ;;jmp     SYS_RET_OK
18423 ;; 25/06/2019
18424 ;jmp     SYS_RET_OK_c1c
18425 ; 11/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
18426 ioctl_gd_ok_j:
18427 jmp     short ioctl_da_ok_j
18428 %endif
18429
18430 ioctl_drv_err:
18431 00002A6B B00F                 mov     al,error_invalid_drive ; 0Fh
18432 ioctl_gd_err_j: ; 11/11/2022
18433 00002A6D E908DC                 jmp     SYS_RET_ERR
18434
18435 ;-----
18436 ;
18437 ; IOCTL: AL = A
18438 ;
18439 ; ENTRY: DS = DOSDATA
18440 ;
18441 ;-----
18442
18443 ioctl_handle_redir:
18444 00002A70 E8604C                 call    SFFromHandle ; ES:DI -> SFT
18445 00002A73 7303                 JNC     short ioctl_got_sft ; have valid handle
18446 00002A75 E929FE                 jmp     ioctl_bad_handle ; 10/08/2018
18447
18448 ioctl_got_sft:
18449 ;mov     dx,[es:di+5]
18450 00002A78 268B5505             MOV     DX,[ES:DI+SF_ENTRY.sf_flags] ; Get flags
18451 ;JMP     short ioctl_set_DX ; pass dx to user and return
18452 ; 16/12/2022
18453 00002A7C EBEA                 jmp     short _ioctl_set_DX
18454
18455 ; 16/12/2022
18456 ;ioctl_bad_funj:
18457 ;JMP     ioctl_bad_fun
18458
18459 ;-----
18460 ;
18461 ; IOCTL: AL= 4,5

```

```

18462 ;
18463 ; ENTRY: DS = DOSDATA
18464 ; SI = user's DS
18465 ;
18466 ;
18467 ; BUGBUG: Don't push anything on the stack between ioctl_get_dev: and
18468 ; the call to Check_If_Net because Check_If_Net gets our
18469 ; return address off the stack if the drive is invalid.
18470 ;
18471 ;-----
18472 ;
18473 ; 30/01/2024 - Retro DOS v5.0
18474 ;
18475 ;
18476 00002A7E E88F00 ioctl_get_dev:
18477 CALL Check_If_Net
18478 ;JNZ short ioctl_bad_funj ; There are no "net devices", and they
18479 ; ; certainly don't know how to do this
18480 ; ; call.
18481 ; 16/12/2022
18482 00002A81 7403 JZ short ioctl_do_string
18483 00002A83 E91AFF ioctl_bad_funj:
18484 JMP ioctl_bad_fun
18485 ;
18486 ;
18487 ;
18488 ;
18489 00002A86 26F6450540 ioctl_do_string:
18490 00002A8B 74F6 ;test word [es:di+4],4000h
18491 ;TEST word [ES:DI+SYSDEV.ATT],DEVIOTCL; See if device accepts control
18492 00002A8D C606[7E03]03 ;test byte [es:di+5],40h
18493 ;TEST byte [ES:DI+SYSDEV.ATT+1],(DEVIOTCL>>8)
18494 00002A92 A801 JZ short ioctl_bad_funj ; NO
18495 00002A94 7405 ; assume IOCTL read
18496 ; MOV byte [IOCALL_REQFUNC],DEVROIOTCL ; 3
18497 ; TEST AL,1 ; is it func. 4/5 or 2/3
18498 00002A96 C606[7E03]0C JZ short ioctl_control_call ; it is read. goto ioctl_control_call
18499 ; ; it is an IOCTL write
18500 MOV byte [IOCALL_REQFUNC],DEVWROIOTCL ; 12
18501 ;
18502 ;
18503 ;
18504 00002A9B B014 ioctl_control_call:
18505 ; 30/01/2024
18506 00002A9D 88DC ;MOV AL,DRDWRHL ; 22 ; MSDOS 6.22 MSDOS.SYS, windows ME IO.SYS
18507 00002A9F A3[7C03] ; 26/06/2024
18508 00002AA2 31C0 mov al,20 ; PCDOS 7.1 IBMDOS.COM
18509 00002AA4 A3[7F03] ioctl_setup_pkt:
18510 00002AA7 A2[8903] MOV AH,BL ; Unit number
18511 00002AAA 890E[8E03] MOV [IOCALL_REQLEN],AX
18512 00002AAE 8916[8A03] XOR AX,AX
18513 00002AB2 8936[8C03] MOV [IOCALL_REQSTAT],AX ; 0
18514 00002AB6 06 MOV [IOMED],AL
18515 00002AB7 1F MOV [IOSCNT],CX
18516 00002AB8 89FE MOV [IOXAD],DX
18517 00002ABA 16 MOV [IOXAD+2],SI
18518 00002ABB 07 PUSH ES
18519 ; POP DS
18520 00002ABC BB[7C03] MOV SI,DI ; DS:SI -> driver
18521 ; PUSH SS
18522 00002ABF E8D327 POP ES
18523 ;
18524 ;
18525 ;
18526 ;
18527 ;
18528 00002AC2 36F606[8003]80 ;hkn; SS override for IOCALL
18529 00002AC8 7507 ;test word [SS:IOCALL_REQSTAT],8000h
18530 ;TEST word [SS:IOCALL_REQSTAT],STERR ;Error?
18531 ;test byte [SS:IOCALL_REQSTAT+1],80h
18532 00002ACA 36A1[8E03] ;TEST byte [SS:IOCALL_REQSTAT+1],(STERR>>8)
18533 ;JNZ short ioctl_string_err
18534 00002ACE E99DDB ;hkn; SS override
18535 ; MOV AX,[SS:IOSCNT] ; Get actual bytes transferred
18536 ; 16/12/2022
18537 ; jmp SYS_RET_OK
18538 ; 11/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
18539 00002AD1 368B3E[7F03] ;jmp short ioctl_gd_ok_j
18540 ;
18541 00002AD6 81E7FF00 ioctl_string_err:
18542 00002ADA 89F8 MOV DI,[SS:IOCALL_REQSTAT];Get Error
18543 00002ADC E8B237 device_err:
18544 ; AND DI,STECODE ; 00FFh ; mask out irrelevant bits
18545 ; MOV AX,DI
18546 ; call SET_I24_EXTENDED_ERROR
18547 ;
18548 ;hkn; use SS override
18549 ;hkn; mov ax,[CS:EXTERR]
18550 00002ADF 36A1[2403] mov ax,[SS:EXTERR]
18551 ; jmp SYS_RET_ERR
18552 ; 11/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
18553 00002AE3 EB88 jmp short ioctl_gd_err_j
18554 ;
18555 ; 17/05/2019 - Retro DOS v4.0
18556 ;
18557 ;-----
18558 ; Proc name : Get_Driver_BL
18559 ;
18560 ; DS is DOSDATA
18561 ; BL is drive number (0=default)
18562 ; Returns pointer to device in ES:DI, unit number in BL if carry clear
18563 ; No regs modified
18564 ;-----
18565 ;
18566 ; 30/01/2024 - Retro DOS v5.0
18567 ;
18568 ;
18569 ;
18570 ;
18571 ;
18572 00002AE5 50 Get_Driver_BL:
18573 00002AE6 88D8 PUSH AX
18574 00002AE8 E8A250 MOV AL,BL ; Drive
18575 00002AEB 7221 call GETTHISDRV
18576 00002AED 30DB jc short ioctl_bad_drv
18577 00002AEF C606[2303]03 XOR BL,BL ; Unit zero on Net device
18578 00002AF4 C43E[A205] MOV byte [EXTERR_LOCUS],errLOC_Net ; 3
18579 ; LES DI,[THISCDS]
18580 ;test word [es:di+43h],8000h
18581 ;TEST word [ES:DI+curdir.flags],curdir_isnet
18582 ;test byte [es:di+44h],80h
18583 ;TEST byte [ES:DI+curdir.flags+1],(curdir_isnet>>8)
18584 00002AF8 26F6454480 ;les di,[es:di+45h]
18585 00002AFD 26C47D45 LES DI,[ES:DI+curdir.devptr] ; ES:DI -> Dpb or net dev
18586 00002B01 750B JNZ short got_dev_ptr ; Is net
18587 ; ;
18588 ; ;
18589 ; 30/01/2024 - PCDOS 7.1 IBMDOS.COM
18590 ; MOV byte [EXTERR_LOCUS],errLOC_Disk ; 2
18591 ; call set_exerr_locus_disk
18592 ; ;

```



```

18586             ;mov     bl,[es:di+1]
18587 00002B06 268A5D01      MOV     BL,[ES:DI+DPB.UNIT] ; Unit number
18588             ;les     di,[es:di+13h]
18589 00002B0A 26C47D13      LES     DI,[ES:DI+DPB.DRIVER_ADDR] ; Driver addr
18590 got_dev_ptr:
18591             ; 30/01/2024
18592             ; cf=0
18593             ; CLC
18594 ioctl_bad_drv:
18595 00002B0E 58             POP     AX
18596 00002B0F C3             retn
18597
18598 ;-----
18599 ; Proc Name : Check_If_Net:
18600 ;
18601 ;
18602 ; Checks if the device is over the net or not. Returns result in ZERO flag.
18603 ; If no device is found, the return address is popped off the stack, and a
18604 ; jump is made to ioctl_drv_err.
18605 ;
18606 ; On Entry:
18607 ; Registers same as those for Get_Driver_BL
18608 ;
18609 ; On Exit:
18610 ; ZERO flag - set if not a net device
18611 ;           - reset if net device
18612 ; ES:DI -> the device
18613 ;
18614 ;
18615 ; BUGBUG: This function assumes the following stack setup on entry
18616 ;
18617 ;     SP+2 -> Error return address
18618 ;     SP   -> Normal return address
18619 ;
18620 ;-----
18621
18622             ; 30/01/2024 - Retro DOS v5.0
18623             ; MSDOS 6.22 MSDOS.SYS - DOSCODE:639Ch
18624             ; PC DOS 7.1 IBMDOS.COM - DOSCODE:6A91h
18625             ; Windows ME IO.SYS - BIOSCODE:68E1h
18626
18627 Check_If_Net:
18628             ; MSDOS 3.3 (& MSDOS 6.0)
18629 00002B10 E8D2FF      CALL    Get_Driver_BL
18630 00002B13 7201      JC      short ioctl_drv_err_pop ; invalid drive letter
18631
18632 ; 30/01/2024 ('Get_Driver_BL' returns with
18633 ;           'curdir_isnet' condition/ZF, no need to a second test)
18634 %if 0
18635             ;;;
18636             ; (PCDOS 7.1 IBMDOS.COM, Windows ME IO.SYS)
18637             PUSH    ES
18638             PUSH    DI
18639             LES     DI,[THISCDS]
18640             ;test    word [es:di+43h],8000h
18641             ;TEST    word [ES:DI+curdir.flags],curdir_isnet
18642             ;test    byte [es:di+44h],80h
18643             TEST    byte [ES:DI+curdir.flags+1],(curdir_isnet>>8)
18644             POP     DI
18645             POP     ES
18646             ;;;
18647 %endif
18648 00002B15 C3             retn
18649
18650 ioctl_drv_err_pop:
18651 00002B16 58             pop     ax ; pop off return address
18652 00002B17 E951FF      jmp     ioctl_drv_err
18653
18654 ioctl_bad_funj3:
18655 00002B1A E983FE      jmp     ioctl_bad_fun
18656
18657 ioctl_string_errj:
18658 00002B1D EBB2      jmp     short ioctl_string_err ; 25/05/2019
18659
18660 ;-----
18661 ;
18662 ; IOCTL: AL = E, F
18663 ;
18664 ; ENTRY: DS = DOSDATA
18665 ;
18666 ;
18667 ; BUGBUG: Don't push anything on the stack between ioctl_drive_owner: and
18668 ; the call to Check_If_Net because Check_If_Net gets our
18669 ; return address off the stack if the drive is invalid.
18670 ;
18671 ;-----
18672
18673 ioctl_drive_owner:
18674             ; MSDOS 3.3 (& MSDOS 6.0)
18675 00002B1F E8EEFF      Call    Check_If_Net
18676 00002B22 75F6      JNZ     short ioctl_bad_funj3 ; There are no "net devices", and they
18677 ;                               ; certainly don't know how to do this
18678 ;                               ; call.
18679             ;TEST    word [ES:DI+SYSDEV.ATT],DEV320 ; See if device can handle this
18680             ; 09/09/2018
18681             ;test    byte [es:di+4],40h
18682 00002B24 26F6450440    TEST    byte [ES:DI+SYSDEV.ATT],DEV320 ; 0040h
18683 00002B29 74EF      JZ      short ioctl_bad_funj3 ; NO
18684             ;mov     byte [IOCALL_REQFUNC],23
18685 00002B2B C606[7E03]17    mov     byte [IOCALL_REQFUNC],DEVGETOWN ; default to get owner
18686 00002B30 3C0E      cmp     al,0Eh ; Get Owner ?
18687 00002B32 7405      jz      short GetOwner
18688
18689 00002B34 C606[7E03]18    SetOwner: MOV     byte [IOCALL_REQFUNC],DEVSETOWN ; 24
18690
18691 00002B39 B00D      GetOwner: MOV     AL,OWNHL ; 13
18692 00002B3B 88DC      MOV     AH,BL ; Unit number
18693 00002B3D A3[7C03]    MOV     [IOCALL_REQLEN],AX
18694 00002B40 31C0      XOR     AX,AX
18695 00002B42 A3[7F03]    MOV     [IOCALL_REQSTAT],AX
18696 00002B45 06             PUSH    ES
18697 00002B46 1F             POP     DS
18698 00002B47 89FE      MOV     SI,DI ; DS:SI -> driver
18699 00002B49 16             PUSH    SS
18700 00002B4A 07             POP     ES
18701 00002B4B BB[7C03]    MOV     BX,IOCALL ; ES:BX -> Call header
18702 00002B4E 1E             push    ds
18703 00002B4F 56             push    si
18704 00002B50 E84227      call    DEVIOCALL2
18705 00002B53 5E             pop     si
18706 00002B54 1F             pop     ds
18707 ;hkn; SS override
18708 ;TEST    word [SS:IOCALL_REQSTAT],STERR ;Error?
18709 ;test    byte [SS:IOCALL_REQSTAT+1],80h

```

```

18710 00002B55 36F606[8003]80      TEST    byte [SS:IOCALL_REQSTAT+1],(STERR>>8)
18711 00002B5B 75C0      jnz     short ioctl_string_errj
18712 00002B5D 36A0[7D03]      MOV     AL,[SS:IOCALL_REQUNIT]; Get owner returned by device
18713                                     ; owner returned is 1-based.
18714 00002B61 E90ADB      jmp     SYS_RET_OK
18715
18716 ;=====
18717 ; DELETE.ASM, MSDOS 6.0, 1991
18718 ;=====
18719 ; 07/08/2018 - Retro DOS v3.0
18720 ; 17/05/2019 - Retro DOS v4.0
18721
18722 ; TITLE  DOS_DELETE - Internal DELETE call for MS-DOS
18723 ; NAME   DOS_DELETE
18724
18725 ;
18726 ; Microsoft Confidential
18727 ; Copyright (C) Microsoft Corporation 1991
18728 ; All Rights Reserved.
18729 ;
18730
18731 ;** DELETE.ASM - Low level routine for deleting files
18732 ;-----
18733 ;           DOS_DELETE
18734 ;           REN_DEL_Check
18735 ;           FastOpen_Delete           ; DOS 3.3
18736 ;           FastOpen_Update          ; DOS 3.3
18737
18738 ; Revision history:
18739 ;
18740 ; A000 version 4.00   Jan. 1988
18741 ; A001 Fastopen Rename fix   April 1989
18742
18743 ;Installed = TRUE
18744
18745 ; i_need NoSetDir,BYTE
18746 ; i_need Creating,BYTE
18747 ; i_need DELALL,BYTE
18748 ; i_need THISDPB,DWORD
18749 ; i_need THISSFT,DWORD
18750 ; i_need THISCDS,DWORD
18751 ; i_need CURBUF,DWORD
18752 ; i_need ATTRIB,BYTE
18753 ; i_need SATTRIB,BYTE
18754 ; i_need WFP_START,WORD
18755 ; i_need REN_WFP,WORD           ;BN001
18756 ; i_need NAME1,BYTE           ;BN001
18757 ; i_need FoundDel,BYTE
18758 ; i_need AUXSTACK,BYTE
18759 ; i_need VOLCHNG_FLAG,BYTE
18760 ; i_need JShare,DWORD
18761 ; i_need FastOpenTable,BYTE     ; DOS 3.3
18762 ; i_need FastTable,BYTE        ; DOS 4.00
18763
18764 ; i_need Del_ExtCluster,WORD    ; DOS 4.00
18765
18766 ; i_need SAVE_BX,WORD           ; DOS 4.00
18767 ; i_need DMAADD,DWORD
18768 ; i_need RENAMEDMA,BYTE
18769
18770 ;-----
18771 ;
18772 ; Procedure Name : DOS_DELETE
18773 ;
18774 ; Inputs:
18775 ; [WFP_START] Points to WFP string ("d:/" must be first 3 chars, NUL
18776 ; terminated)
18777 ; [CURR_DIR_END] Points to end of Current dir part of string
18778 ; (= -1 if current dir not involved, else
18779 ; Points to first char after last "/" of current dir part)
18780 ; [THISCDS] Points to CDS being used
18781 ; (Low word = -1 if NUL CDS (Net direct request))
18782 ; [SATTRIB] Is attribute of search, determines what files can be found
18783 ; Function:
18784 ; Delete the specified file(s)
18785 ; Outputs:
18786 ; CARRY CLEAR
18787 ; OK
18788 ; CARRY SET
18789 ; AX is error code
18790 ; error_file_not_found
18791 ; Last element of path not found
18792 ; error_path_not_found
18793 ; Bad path (not in curr dir part if present)
18794 ; error_bad_curr_dir
18795 ; Bad path in current directory part of path
18796 ; error_access_denied
18797 ; Attempt to delete device or directory
18798 ; ***error_sharing_violation***
18799 ; Deny both access required, generates an INT 24.
18800 ; This error is NOT returned. The INT 24H is generated,
18801 ; and the file is ignored (not deleted). Delete will
18802 ; simply continue on looking for more files.
18803 ; Carry will NOT be set in this case.
18804 ; DS preserved, others destroyed
18805 ;-----
18806
18807
18808 FILEFOUND equ 01h
18809 FILEDELETED equ 10h
18810
18811 ; 12/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
18812 ; DOSCODE:63E9h (MSDOS 5.0, MSDOS.SYS)
18813
18814 ; 30/01/2024 - Retro DOS v5.0 (Modified MSDOS 5.0 IBMDOS.COM)
18815 ; DOSCODE:6B11h (PCDOS 7.1, IBMDOS.COM)
18816
18817 DOS_DELETE:
18818
18819 ;hkn; DOS_Delete is called from file.asm and fcbio.asm. DS has been set up
18820 ;hkn; appropriately at this point.
18821
18822 00002B64 E8C2EC      call    TestNet
18823 00002B67 7306      jnc     short LOCAL_DELETE
18824
18825 ;IF NOT Installed
18826 ; transfer NET_DELETE
18827 ;ELSE
18828 ;MOV     AX,(MULTINET SHL 8) | 19
18829 ;INT     2FH
18830 ;return
18831
18832 00002B69 B81311      mov     ax,1113h
18833 00002B6C CD2F      int     2Fh ; Multiplex - NETWORK REDIRECTOR - DELETE REMOTE FILE

```



```

18834                                     ; SS = DS = DOS CS, SDA first filename pointer ->
18835                                     ;                                     fully-qualified filename in DOS CS
18836                                     ; SDA CDS pointer -> current directory structure for drive with file
18837                                     ; Return: CF set on error
18838 00002B6E C3                          retn
18839 ;ENDIF
18840
18841 LOCAL_DELETE:
18842 00002B6F C606[6F05]00                MOV     byte [FOUNDDEL],0      ; No files found and no files deleted
18843 00002B74 E86BED                        call    ECritDisk
18844                                     ;mov     word [CREATING],0E500h
18845 00002B77 C706[7E05]00E5              MOV     WORD [CREATING],DIRFREE*256+0 ; Assume not del *.*
18846 00002B7D 8B36[B205]                  MOV     SI,[WFP_START]
18847 SKPNUL:
18848 00002B81 AC                           LODSB
18849 00002B82 08C0                          OR      AL,AL
18850 00002B84 75FB                          JNZ     short SKPNUL          ; go to end
18851 00002B86 83EE04                        SUB     SI,4                  ; Back over possible ".*"
18852 00002B89 813C2A2E                      CMP     WORD [SI],2E2Ah ; ".*"
18853 00002B8D 7506                          JNZ     short TEST_QUEST
18854 00002B8F 807C022A                      CMP     BYTE [SI+2],"*"
18855 00002B93 741F                          JZ      short CHECK_ATTS
18856 TEST_QUEST:
18857 00002B95 83EE09                        SUB     SI,9                  ; Back over possible "????????.???"
18858 00002B98 87FE                          XCHG    DI,SI
18859
18860 00002B9A 16                            push     ss
18861 ;pop     ds ; ! Retro DOS v3.0 BUG !
18862 00002B9B 07                            pop      es ; 17/05/2019
18863
18864 00002B9C B83F3F                        MOV     AX,"?" ; 3F3Fh
18865 00002B9F B90400                        MOV     CX,4                  ; four sets of "?"
18866 00002BA2 F3AF                        REPE    SCASW
18867 00002BA4 751C                          JNZ     short NOT_ALL
18868 00002BA6 87FE                          XCHG    DI,SI
18869 00002BA8 AD                           LODSW
18870 00002BA9 3D2E3F                        CMP     AX,3F2Eh ; ".?"
18871 00002BAC 7514                          JNZ     short NOT_ALL
18872 00002BAE AD                           LODSW
18873 00002BAF 3D3F3F                        CMP     AX,"?"
18874 00002BB2 750E                          JNZ     short NOT_ALL
18875 CHECK_ATTS:
18876 00002BB4 A0[6D05]                      MOV     AL,[SATTRIB]
18877 ;and     al,1Fh
18878 00002BB7 241F                          AND     AL,attr_hidden+attr_system+attr_directory+attr_volume_id+attr_read_only
18879 ; Look only at hidden bits
18880 ;cmp     al,1Fh
18881 00002BB9 3C1F                          CMP     AL,attr_hidden+attr_system+attr_directory+attr_volume_id+attr_read_only
18882 ; All must be set
18883 00002BBB 7505                          JNZ     short NOT_ALL
18884
18885 ; NOTE WARNING DANGER-----
18886 ; This DELALL stuff is not safe. It allows directories to be deleted.
18887 ; It should ONLY be used by FORMAT in the ROOT directory.
18888
18889 00002BBD C606[7F05]00                MOV     byte [DELALL],0      ; DEL *.* - flag deleting all
18890 NOT_ALL:
18891 00002BC2 C606[4C03]01                MOV     byte [NoSetDir],1
18892 00002BC7 E84F1F                        call    GetPathNoSet
18893 00002BCA 7312                          JNC     short Del_found
18894 00002BCC 750B                          JNZ     short _bad_path
18895 00002BCE 08C9                          OR      CL,CL
18896 00002BD0 7407                          JZ      short _bad_path
18897 No_file:
18898 00002BD2 B80200                        MOV     AX,error_file_not_found
18899 ErrorReturn:
18900 00002BD5 F9                            STC
18901 ;call    LCritDisk
18902 ;retn
18903 ; 18/12/2022
18904 00002BD6 E936ED                        jmp     LCritDisk
18905
18906 _bad_path:
18907 00002BD9 B80300                        MOV     AX,error_path_not_found
18908 00002BDC EBF7                          JMP     short ErrorReturn
18909
18910 Del_found:
18911 00002BDE 750C                          JNZ     short NOT_DIR        ; Check for dir specified
18912 00002BE0 803E[7F05]00                CMP     byte [DELALL],0      ; DelAll = 0 allows delete of dir.
18913 00002BE5 7405                          JZ      short NOT_DIR
18914 Del_access_err:
18915 00002BE7 B80500                        MOV     AX,error_access_denied
18916 00002BEA EBE9                          JMP     short ErrorReturn
18917
18918 NOT_DIR:
18919 00002BEC 08E4                          OR      AH,AH                ; Check if device name
18920 00002BEE 78F7                          JS      short Del_access_err ; Can't delete I/O devices
18921
18922 ; Main delete loop. CURBUF+2:BX points to a matching directory entry.
18923
18924 DELFILE:
18925 00002BF0 800E[6F05]01                OR      byte [FOUNDDEL],FILEFOUND ; file found, not deleted yet
18926
18927 ; If we are deleting the volume ID, then we set VOLUME_CHNG flag to make
18928 ; DOS issue a build BPB call the next time this drive is accessed.
18929
18930 00002BF5 1E                            PUSH     DS
18931 00002BF6 8A26[7F05]                  MOV     AH,[DELALL]
18932 00002BFA C53E[E205]                  LDS     DI,[CURBUF]
18933
18934 ;hkn; SS override
18935 00002BFE 36F606[6B05]01              TEST     byte [SS:ATTRIB],attr_read_only ; are we deleting RO files too?
18936 00002C04 7509                          JNZ     short DoDelete       ; yes
18937
18938 00002C06 F6470B01                      TEST     byte [Bx+dir_entry.dir_attr],attr_read_only
18939 00002C0A 7403                          JZ      short DoDelete       ; not read only
18940
18941 ; 30/01/2024 (PCDOS 7.1 IBMDOS.COM)
18942 skip_it:
18943 00002C0C 1F                            POP      DS
18944 00002C0D EB57                          JMP     SHORT DELNXT          ; Skip it (Note ES:BP not set)
18945
18946 DoDelete:
18947 00002C0F E8AF00                        call     REN_DEL_Check       ; Sets ES:BP = [THISDPB]
18948 ;JNC     short DEL_SHARE_OK
18949 ;POP     DS
18950 ;JMP     SHORT DELNXT          ; Skip it
18951 ; 30/01/2024
18952 00002C12 72F8                          JC      short skip_it
18953
18954 DEL_SHARE_OK:
18955 ; 17/05/2019 - Retro DOS v4.0
18956 ; MSDOS 6.0
18957 ;test     byte [di+5],40h

```

```

18958 00002C14 F6450540      TEST    byte [DI+BUFFINFO.buf_flags],buf_dirty
18959                        ;LB. if already dirty ;AN000;
18960 00002C18 7507          JNZ     short yesdirty ;LB. don't increment dirty count ;AN000;
18961 00002C1A E8AD3F        call    INC_DIRTY_COUNT ;LB.
18962                        ;or byte [di+5],40h ;AN000;
18963 00002C1D 804D0540      OR      byte [DI+BUFFINFO.buf_flags],buf_dirty
18964
18965 00002C21 8827          yesdirty: mov     [bx],ah
18966                        ;MOV [BX+dir_entry.dir_name],AH ; Put in E5H or 0
18967                        ;;;
18968                        ; 30/01/2024 - Retro DOS v5.0
18969                        ; (PCDOS 7.1 IBMDOS.COM)
18970 00002C23 31DB          xor     bx,bx ; 0
18971                        ;cmp [es:bp+0Fh],bx
18972 00002C25 26395E0F      cmp     [es:bp+DPB.FAT_SIZE],bx ; 0
18973 00002C29 7503          jnz     short yesdirty_fc_1 ; not FAT32
18974 00002C2B 8B5CFA      mov     bx,[si-6] ; high word of the first cluster (FAT32)
18975
18976 00002C2E 89DA          yesdirty_fc_1: mov     [ss:CLUSTNUM_HW],bx
18977                        mov     dx,bx ; * ; 30/01/2024 - Retro DOS v5.0
18978                        ;;;
18979
18980 00002C30 8B1C          MOV     BX,[SI] ; Get firclus pointer
18981 00002C32 1F          POP     DS
18982                        ;;;
18983 00002C33 8916[E80A]    mov     [CLUSTNUM_HW],dx ; * ; 30/01/2024 - Retro DOS v5.0
18984                        ;;;
18985 00002C37 800E[6F05]10  OR      byte [FOUNDEDEL],FILEDELETED ; 10h ; Deleted file
18986
18987                        ; 30/01/2024
18988                        ;CMP BX,2
18989                        ;JB short DELNXT ; File has invalid FIRCLUS (too small)
18990                        ;;;
18991                        ; 30/01/2024 - Retro DOS v5.0
18992                        ; (PCDOS 7.1 IBMDOS.COM)
18993                        ;cmp word [CLUSTNUM_HW],0
18994 00002C3C 21D2          and     dx,dx ; 30/01/2024 - Retro DOS v5.0
18995 00002C3E 7505          jnz     short yesdirty_fc_2
18996 00002C40 83FB02      cmp     bx,2
18997 00002C43 7221          jb     short DELNXT ; File has invalid FIRCLUS (too small)
18998
18999 00002C45 26837E0F00      yesdirty_fc_2: cmp     word [es:bp+0Fh],0
19000                        cmp     word [es:bp+DPB.FAT_SIZE],0
19001 00002C4A 750C          jnz     short yesdirty_fc_3 ; not FAT32
19002                        ;push bx
19003                        ;mov bx,[CLUSTNUM_HW]
19004                        ;;cmp bx,[es:bp+2Fh]
19005                        ;cmp bx,[es:bp+DPB.LAST_CLUSTER+2]
19006                        ;30/01/2024 - Retro DOS v5.0
19007                        ;mov dx,[CLUSTNUM_HW]
19008                        ; dx = [CLUSTNUM_HW] ; *
19009 00002C4C 263B562F      cmp     dx,[es:bp+DPB.LAST_CLUSTER+2]
19010                        ;pop bx
19011 00002C50 750A          jne     short yesdirty_fc_4
19012                        ;cmp bx,[es:bp+2Dh]
19013 00002C52 263B5E2D      cmp     bx,[es:bp+DPB.LAST_CLUSTER]
19014 00002C56 EB04          jmp     short yesdirty_fc_4
19015
19016 00002C58 263B5E0D      yesdirty_fc_3: ;;;
19017                        ;cmp bx,[es:bp+0Dh]
19018                        CMP     BX,[ES:BP+DPB.MAX_CLUSTER]
19019
19020 00002C5C 7708          yesdirty_fc_4: ; 30/01/2024
19021                        JA     short DELNXT ; File has invalid FIRCLUS (too big)
19022
19022 00002C5E E86F2F      call    RELEASE ; Free file data
19023 00002C61 7258          JC     short No_fileJ
19024
19025 ; DOS 3.3 FastOpen
19026
19027 00002C63 E8D900      CALL    FastOpen_Delete ; delete the dir info in fastopen
19028
19029 ; DOS 3.3 FastOpen
19030
19031 DELNXT:
19032 LES     BP,[THISDPB] ; Possible to get here without this set
19033 call    GETENTRY ; Registers need to be reset
19034 JC     short No_fileJ
19035 call    NEXTENT
19036 ;JNC short DELFILE
19037 ; 30/01/2024
19038 jc     short DELNXT2
19039 jmp     DELFILE
19040
19041 00002C77 C42E[8A05]    DELNXT2: LES     BP,[THISDPB] ; NEXTENT sets ES=DOSGROUP
19042
19043 ;;;
19044 ; 30/01/2024 - Retro DOS v5.0
19045 ; (PCDOS 7.1 IBMDOS.COM)
19046 ;
19047 00002C7B E8EE07      call    update_fat32_fsinfo
19048 ;;;
19049
19050 ; 12/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
19051 ;MOV AL,[ES:BP+DPB.DRIVE]
19052 ;;mov al,[es:bp+0]
19053 ; 15/12/2022
19054 00002C7E 268A4600      MOV     AL,[ES:BP]
19055 00002C82 E8133E      call    FLUSHBUF
19056 00002C85 7234          JC     short No_fileJ
19057
19058 ;
19059 ; Now we need to test FoundDel for our flags. The cases to consider are:
19060 ;
19061 ; not found not deleted file not found
19062 ; not found deleted *** impossible ***
19063 ; found not deleted access denied (read-only)
19064 ; found deleted no error
19065
19065 00002C87 F606[6F05]10      TEST    byte [FOUNDEDEL],FILEDELETED ; did we delete a file?
19066 00002C8C 7426          JZ     short DelError ; no, figure out what's wrong.
19067
19068 ; We set VOLCHNG_FLAG to indicate that we have changed the volume label
19069 ; and to force the DOS to issue a media check.
19070
19071 00002C8E F606[6B05]08      TEST    byte [ATTRIB],attr_volume_id ; 8
19072 00002C93 741C          jz     short No_Set_Flag
19073 00002C95 50          PUSH    AX
19074 00002C96 06          PUSH    ES
19075 00002C97 57          PUSH    DI
19076 00002C98 C43E[A205]    LES     DI,[THISCDS]
19077 00002C9C 268A25      MOV     AH,[ES:DI] ; Get drive
19078 00002C9F 80EC41      SUB     AH,'A' ; Convert to 0-based
19079 00002CA2 8826[A10A]    mov     [VOLCHNG_FLAG],AH
19080
19081 ; MSDOS 6.0

```

```

19082 00002CA6 30FF      XOR     BH,BH      ;>32mb delete volume id from boot record ;AN000;
19083 00002CA8 E8EB04    call    Set_Media_ID ;>32mb set volume id to boot record ;AN000;
19084
19085 00002CAB E8AD38    call    FATREAD_CDS    ; force media check
19086 00002CAE 5F        POP     DI
19087 00002CAF 07        POP     ES
19088 00002CB0 58        POP     AX
19089
19090 No_Set_Flag:
19091      ;call    LCritDisk      ; carry is clear
19092      ;retn
19093      ; 18/12/2022
19093 00002CB1 E95BEC    jmp     LCritDisk
19094
19095 00002CB4 F606[6F05]01 DelError:
19096 00002CB9 7503      TEST    byte [FOUNDDDEL],FILEFOUND ; not deleted. Did we find file?
19097      JNZ     short Del_access_errJ ; yes. Access denied
19097
19098 00002CBB E914FF    No_fileJ:
19099      JMP     No_file ; 10/08/2018      ; Nope
19099 Del_access_errJ:
19100 00002CBE E926FF    JMP     Del_access_err ; 10/08/2018
19101
19102      ; 08/08/2018 - Retro DOS v3.0
19103
19104      ;Break      <REN_DEL_Check - check for access for rename and delete>
19105      ;-----
19106      ; Procedure Name : REN_DEL_Check
19107      ;
19108      ; Inputs:
19109      ; [THISDPB] set
19110      ; [CURBUF+2]:BX points to entry
19111      ; [CURBUF+2]:SI points to firclus field of entry
19112      ; [WFP_Start] points to name
19113      ; Function:
19114      ; Check for Exclusive access on given file.
19115      ; Used by RENAME, SET_FILE_INFO, and DELETE.
19116      ; Outputs:
19117      ; ES:BP = [THISDPB]
19118      ; NOTE: The WFP string pointed to by [WFP_Start] will be Modified. The
19119      ; last element will be loaded from the directory entry. This is
19120      ; so the name given to the sharer doesn't have any meta chars in
19121      ; it.
19122      ; Carry set if sharing violation, INT 24H generated
19123      ; NOTE THAT AX IS NOT error_sharing_violation.
19124      ; This is because input AX is preserved.
19125      ; Caller must set the error if needed.
19126      ; Carry clear
19127      ; OK
19128      ; AX,DS,BX,SI,DI preserved
19129      ;-----
19130
19131      ; 31/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
19132
19133 REN_DEL_Check:
19134
19135 00002CC1 1E        PUSH    DS
19136 00002CC2 57        PUSH    DI
19137 00002CC3 50        PUSH    AX
19138 00002CC4 53        PUSH    BX
19139 00002CC5 56        PUSH    SI      ; Save CURBUF pointers
19140
19141 00002CC6 16        push    ss
19142 00002CC7 07        pop     es
19143
19144      ;hkn; context ES will assume ES to DOSDATA
19145      ;hkn; ASSUME      ES:DOSGROUP
19146
19147      ;hkn; SS override
19148 00002CC8 368B3E[B205] MOV     DI,[SS:WFP_START] ; ES:DI -> WFP
19149 00002CCD 89DE      MOV     SI,BX
19150
19151      ;hkn; SS override
19152 00002CCF 368E1E[E405] MOV     DS,[SS:CURBUF+2] ; DS:SI -> entry (FCB style name)
19153 00002CD4 89FB      MOV     BX,DI      ; Set backup limit for skipback
19154      ;ADD     BX,2      ; Skip over d: to point to leading '\'
19155      ; 31/01/2024
19156 00002CD6 43        inc     bx
19157 00002CD7 43        inc     bx
19158 00002CD8 E8EEEA    call    StrLen      ; CX is length of ES:DI including NUL
19159 00002CDB 49        DEC     CX      ; Don't include nul in count
19160 00002CDC 01CF      ADD     DI,CX      ; Point to NUL at end of string
19161 00002CDE E85E51    call    SkipBack    ; Back up one element
19162 00002CE1 47        INC     DI      ; Point to start of last element
19163
19164      ; 17/05/2019 - Retro DOS v4.0
19165      ;hkn; SS override
19166      ; MSDOS 6.0
19167 00002CE2 36893E[0106] MOV     [SS:SAVE_BX],DI ;IFS. save for DOS_RENAME ;AN000;
19168
19169 00002CE7 E8BDF9    call    PackName    ; Transfer name from entry to ASCIZ tail.
19170 00002CEA 5E        POP     SI      ; Get back entry pointers
19171 00002CEB 58        POP     BX
19172 00002CEC 53        PUSH    BX
19173 00002CED 56        PUSH    SI      ; Back on stack
19174
19175 00002CEE 16        push    ss
19176 00002CEF 1F        pop     ds
19177
19178      ;hkn; context DS will assume ES to DOSDATA
19179      ;hkn; ASSUME      DS:DOSGROUP
19180
19181      ; 07/07/2024
19182      ;push    es      ; (not necessary) ; 31/01/2024
19183      ;push    di
19184
19185      ; Close the file if possible by us.
19186      ;
19187      ;if installed
19188 00002CF0 FF1E[C400] call    far [JShare+(13*4)] ; 13 = ShCloseFile
19189      ;else
19190      ; Call    ShCloseFile
19191      ;endif
19192      ;;;
19193      ; 31/01/2024 - Retro DOS v5.0
19194      ; PCDOS 7.1 IBMDOS.COM
19195 00002CF4 803E[0303]FF cmp     byte [fShare],0FFh
19196 00002CF9 750A      jne     short rdc_1
19197 00002CFB 8C06[A005] mov     [THISFT+2],es
19198 00002CFF 893E[9E05] mov     [THISFT],di
19199 00002D03 E80A      jmp     short rdc_2
19200 rdc_1:
19201      ;;;
19202 00002D05 8C1E[A005] MOV     [THISFT+2],DS
19203
19204      ;hkn; AUXSTACK is in DOSDATA
19205 00002D09 C706[9E05][6507] MOV     word [THISFT],AUXSTACK-SF_ENTRY.size ; RENAMEDMA+(384-59)

```

```

19206                                     ; Scratch space
19207 rdc_2:                               ; 30/01/2024
19208                                     ; 07/07/2024
19209                                     ; pop di
19210                                     ; pop es
19211                                     ;
19212 00002D0F 30E4                       XOR AH,AH           ; Indicate file to DOOPEN (high bit off)
19213 00002D11 E8EF28                     call DOOPEN        ; Fill in SFT for share check
19214 00002D14 C43E[9E05]                 LES DI,[THISSFT]
19215                                     ;mov word [es:di+2],10h
19216 00002D18 26C745021000              MOV word [ES:DI+SF_ENTRY.sf_mode],SHARING_DENY_BOTH ; 10h
19217                                     ;MOV word [ES:DI+SF_ENTRY.sf_ref_count],1 ; Pretend open
19218                                     ;mov word [ES:DI],1
19219 00002D1E 26C7050100                  call ShareEnter
19220 00002D23 E86357                     jc short CheckDone
19221 00002D26 720D                       LES DI,[THISSFT]
19222 00002D28 C43E[9E05]                 ;MOV word [ES:DI+SF_ENTRY.sf_ref_count],0
19223                                     ;mov word [ES:DI],0 ; Pretend closed and free
19224 00002D2C 26C7050000
19225
19226 00002D31 E85057                     call ShareEnd      ; Tell sharer we're done with THISSFT
19227 00002D34 F8                         CLC
19228
19229 00002D35 C42E[8A05]                   CheckDone:
19230 00002D39 5E                         LES BP,[THISDPB]
19231 00002D3A 5B                         POP SI
19232 00002D3B 58                         POP BX
19233 00002D3C 5F                         POP AX
19234 00002D3D 1F                         POP DI
19235 00002D3E C3                         POP DS
19236                                     retn
19237
19238 ;Break <FastOpen_Delete - delete dir info in fastopen>
19239 -----
19240 ; Procedure Name : FastOpen_Delete
19241 ; Inputs:
19242 ; None
19243 ; Function:
19244 ; Call FastOpen to delete the dir info.
19245 ; Outputs:
19246 ; None
19247 -----
19248                                     ; 31/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
19249
19250 FastOpen_Delete:
19251 00002D3F 9C                           PUSHF              ; save flag
19252 00002D40 56                           PUSH SI            ; save registers
19253 00002D41 57                           push di           ; 31/01/2024 (PCDOS 7.1 IBMDOS.COM)
19254 00002D42 53                           PUSH BX
19255 00002D43 50                           PUSH AX
19256                                     ;mov si,[WFP_START] ; MSDOS 3.3
19257 ;hkn; SS override
19258 ; 17/05/2019 - Retro DOS v4.0
19259 ; MSDOS 6.0
19260 00002D44 368B36[B205]                 MOV SI,[ss:WFP_START] ; ds:si points to path name
19261
19262 00002D49 B003                         MOV AL,FONC_delete ; al = 3
19263
19264 ; 31/01/2024 (PCDOS 7.1 IBMDOS.COM)
19265 %if 0
19266 fastinvoke:
19267 ;hkn; FastTable is in DOSDATA
19268 MOV BX,FastTable+2
19269 CALL far [BX] ; call fastopen
19270 POP AX ; restore registers
19271 POP BX
19272 ;pop di ; 31/01/2024 (PCDOS 7.1 IBMDOS.COM)
19273 POP SI
19274 POPF ; restore flag
19275 retn
19276 %else
19277 00002D4B EB0F                         jmp short fastinvoke ; 31/01/2024
19278 %endif
19279
19280 ; 13/11/2022 Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
19281 ; DOSCODE:65A0h (MSDOS 5.0 MSDOS.SYS)
19282
19283 ; 31/01/2024 Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
19284 ; DOSCODE:65B4h (MSDOS 6.22 MSDOS.SYS)
19285 ; DOSCODE:6D07h (PCDOS 7.1 IBMDOS.COM)
19286
19287 ;Break <FastOpen_Rename - Rename directory> ; PTR 5622
19288 -----
19289 ; PROCEDURE Name : FastOpen_Rename
19290 ;
19291 ; Inputs:
19292 ; REN_WFP = Path Name
19293 ; NAME1 = New Name
19294 ; Function:
19295 ; Call FastOpen to rename the dir entry in the cache
19296 ; Outputs:
19297 ; None
19298 -----
19299
19300 FastOpen_Rename:
19301 ; 17/05/2019 - Retro DOS v4.0
19302 ; 08/08/2018 - Retro DOS v3.0
19303 ; MSDOS 6.0
19304 00002D4D 9C                           PUSHF              ;AN001 save flag
19305 00002D4E 56                           PUSH SI            ;AN001 save registers
19306 00002D4F 57                           PUSH DI            ;AN001
19307 00002D50 53                           PUSH BX            ;AN001
19308 00002D51 50                           PUSH AX            ;AN001
19309
19310 ;hkn; SS override
19311 00002D52 368B36[B405]                 MOV SI,[ss:REN_WFP] ;AN001 ;;AN001 ds:si-->Path name addr
19312
19313 ;hkn; NAME1 is in DOSDATA
19314 00002D57 BF[4B05]                     MOV DI,NAME1 ;;AN001 ds:di-->New name addr
19315 ;mov al,6
19316 00002D5A B006                         MOV AL,FONC_Rename ;;AN001 al = 6
19317
19318 fastinvoke: ; 31/01/2024 (PCDOS 7.1 IBMDOS.COM)
19319
19320 ;hkn; FastTable is in DOSDATA
19321 00002D5C BB[3E11]                     MOV BX,FastTable+2
19322 00002D5F FF1F                         CALL far [BX] ;;AN001 call fastopen
19323
19324 00002D61 58                           POP AX ; restore registers ;AN001
19325 00002D62 5B                           POP BX ;AN001
19326 00002D63 5F                           POP DI ;AN001
19327 00002D64 5E                           POP SI ;AN001
19328 00002D65 9D                           POPF ; restore flag ;AN001
19329 00002D66 C3                         retn ;AN001

```

```

19330
19331
19332 ;Break <FastOpen_Update - update dir info in fastopen>
19333 ;
19334 ; Procedure Name : FastOpen_Update
19335 ;
19336 ; Inputs:
19337 ; DL drive number (A=0,B=1,,,)
19338 ; CX first cluster #
19339 ; AH 0 updates dir entry
19340 ; 1 updates CLUSNUM , BP = new CLUSNUM
19341 ; ES:DI directory entry
19342 ; Function:
19343 ; Call FastOpen to update the dir info.
19344 ; Outputs:
19345 ; None
19346 ;
19347
19348 FastOpen_Update:
19349     PUSHF ; save flag
19350     PUSH SI
19351     push di ; 31/01/2024 (PCDOS 7.1 IBMDOS.COM)
19352     PUSH BX ; save regs
19353     PUSH AX
19354     MOV AL,FONC_update ; al = 4
19355     JMP short fastinvoke
19356
19357 ; 17/05/2019
19358
19359 ; MSDOS 6.0
19360 ;entry Fast_Dispatch ; future fastxxxx entry ;AN000;
19361 ;
19362 Fast_Dispatch:
19363 ;hkn; FastTable is in DOSDATA
19364     MOV SI,FastTable+2 ; index to the ;AN000;
19365 ;hkn; use SS override
19366     CALL far [SS:SI] ; RMFD call fastopen
19367     retn
19368
19369 ;=====
19370 ; RENAME.ASM, MSDOS 6.0, 1991
19371 ;=====
19372 ; 08/08/2018 - Retro DOS v3.0
19373 ; 17/05/2019 - Retro DOS v4.0
19374
19375 ; TITLE DOS_RENAME - Internal RENAME call for MS-DOS
19376 ; NAME DOS_RENAME
19377
19378 ;** Low level routine for renaming files
19379 ;-----
19380 ; DOS_RENAME
19381 ;
19382 ; Modification history:
19383 ;
19384 ; Created: ARR 30 March 1983
19385 ;-----
19386 ;
19387 ; Procedure Name : DOS_RENAME
19388 ;
19389 ; Inputs:
19390 ; [WFP_START] Points to SOURCE WFP string ("d:/" must be first 3
19391 ; chars, NUL terminated)
19392 ; [CURR_DIR_END] Points to end of Current dir part of string [SOURCE]
19393 ; (= -1 if current dir not involved, else
19394 ; points to first char after last "/" of current dir part)
19395 ; [REN_WFP] Points to DEST WFP string ("d:/" must be first 3
19396 ; chars, NUL terminated)
19397 ; [THISCDS] Points to CDS being used
19398 ; (Low word = -1 if NUL CDS (Net direct request))
19399 ; [ATTRIB] Is attribute of search, determines what files can be found
19400 ; Function:
19401 ; Rename the specified file(s)
19402 ; NOTE: This routine uses most of AUXSTACK as a temp buffer.
19403 ; Outputs:
19404 ; CARRY CLEAR
19405 ; OK
19406 ; CARRY SET
19407 ; AX is error code
19408 ; error_file_not_found
19409 ; No match for source, or dest path invalid
19410 ; error_not_same_device
19411 ; Source and dest are on different devices
19412 ; error_access_denied
19413 ; Directory specified (not simple rename),
19414 ; Device name given, Destination exists.
19415 ; NOTE: In third case some renames may have
19416 ; been done if metas.
19417 ; error_path_not_found
19418 ; Bad path (not in curr dir part if present)
19419 ; SOURCE ONLY
19420 ; error_bad_curr_dir
19421 ; Bad path in current directory part of path
19422 ; SOURCE ONLY
19423 ; error_sharing_violation
19424 ; Deny both access required, generates an INT 24.
19425 ; DS preserved, others destroyed
19426 ;-----
19427 ;
19428 ;
19429 ; 14/11/2022 Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
19430 ; 31/01/2024 Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
19431
19432 DOS_RENAME:
19433 ;hkn; DOS_RENAME is called from file.asm and fcbio.asm. DS has been set up
19434 ;hkn; at this point to DOSDATA.
19435
19436     call TestNet
19437     JNC short LOCAL_RENAME
19438
19439 ;IF NOT Installed
19440 ; transfer NET_RENAME
19441 ;ELSE
19442 ;MOV AX,(MULTINET SHL 8) OR 17
19443 ;INT 2FH
19444 ;return
19445
19446     mov ax, 1111h
19447     int 2Fh ; Multiplex - NETWORK REDIRECTOR - RENAME REMOTE FILE
19448 ; SS = DS = DOS CS,
19449 ; SDA first filename pointer = offset of fully-qualified old name
19450 ; SDA CDS pointer -> current directory
19451 ; Return: CF set on error
19452
19453     retn

```

```

19454 ;ENDIF
19455
19456 LOCAL_RENAME:
19457 MOV     byte [EXTERR_LOCUS],errLOC_Disk ; 2
19458 MOV     SI,[WFP_START]
19459 MOV     DI,[REN_WFP]
19460 MOV     AL,[SI]
19461 MOV     AH,[DI]
19462 OR      AX,2020H ; Lower case
19463 CMP     AL,AH
19464 JZ      short SAMEDRV
19465 MOV     AX,error_not_same_device ; 11h
19466 STC
19467 retn
19468
19469 SAMEDRV:
19470 PUSH    WORD [DMAADD+2]
19471 PUSH    WORD [DMAADD]
19472 MOV     [DMAADD+2],DS
19473
19474 ;hkn; RENAMEDMA is in DOSDATA
19475 MOV     WORD [DMAADD],RENAMEDMA
19476 MOV     byte [FOUND_DEV],0 ; Rename fails on DEVS, assume not a dev
19477 call    ECritDisk
19478 call    DOS_SEARCH_FIRST ; Sets [NoSetDir] to 1, [CURBUF+2]:BX
19479 ; points to entry
19480 JNC     short Check_Dev
19481 CMP     AX,error_no_more_files ; 12h
19482 JNZ     short GOTERR
19483 MOV     AX,error_file_not_found ; 2
19484 GOTERR:
19485 STC
19486 RENAME_POP:
19487 POP     WORD [DMAADD]
19488 POP     WORD [DMAADD+2]
19489 ;call    LCritDisk
19490 ;retn
19491 ; 16/12/2022
19492 jmp     LCritDisk
19493
19494 Check_Dev:
19495 ; 17/05/2019 - Retro DOS v4.0
19496 ;mov     ax,5
19497 MOV     AX,error_access_denied; Assume error
19498
19499 ; MSDOS 6.0
19500 PUSH    DS ;PTM. ;AN000;
19501 LDS     SI,[DMAADD] ;PTM. check if source a dir ;AN000;
19502 ;add     si,21
19503 ADD     SI,find_buf.attr ;PTM. ;AN000;
19504 ;test    byte [si+11],10h
19505 TEST    byte [SI+dir_entry.dir_attr],attr_directory ;PTM. ;AN000;
19506 JZ      short notdir ;PTM. ;AN000;
19507 MOV     SI,[REN_WFP] ;PTM. if yes, make sure path ;AN000;
19508 call    Check_PathLen2 ;PTM. length < 67 ;AN000;
19509 notdir:
19510 POP     DS ;PTM. ;AN000;
19511 JA      short GOTERR ;PTM. ;AN000;
19512
19513 ; MSDOS 3.3 & MSDOS 6.0
19514 CMP     byte [FOUND_DEV],0
19515 JNZ     short GOTERR
19516
19517 ; At this point a source has been found. There is search continuation info (a
19518 ; 1a DOS_SEARCH_NEXT) for the source at RENAMEDMA, together with the first
19519 ; directory entry found.
19520 ; [THISCDs], [THISDPB], and [THISDRV] are set and will remain correct
19521 ; throughout the RENAME since it is known at this point that the source and
19522 ; destination are both on the same device.
19523 ; [ATTRIB] is also set.
19524
19525 MOV     SI,BX
19526 ;add     si,26
19527 ADD     SI,dir_entry.dir_first
19528 call    REN_DEL_Check
19529 JNC     short REN_OK1
19530 MOV     AX,error_sharing_violation ; 20h
19531 JMP     short RENAME_POP
19532
19533 ;-----
19534 ; Check if the source is a file or directory. If file, delete the entry
19535 ; from the Fastopen cache. If directory, rename it later
19536 ;-----
19537
19538 REN_OK1:
19539 ; MSDOS 6.0
19540 ; 31/01/2024 (PCDOS 7.1 IBMDOS.COM)
19541 ;PUSH    SI
19542 LDS     SI,[DMAADD] ;BN00x; PTM. check if source a dir ;AN000;
19543 ;add     si,21
19544 ADD     SI,find_buf.attr ;;BN00XPTM.P5520 ;AN000;
19545 ;test    byte [si+11],10h
19546 TEST    byte [SI+dir_entry.dir_attr],attr_directory ;;BN00XPTM. ;AN000;
19547 ;JZ      short NOT_DIR1 ;;BN00XPTM. ;AN000;
19548 jnz     short SWAP_SOURCE ; 31/01/2024
19549 ;POP     SI ;BN00X
19550 ;JMP     SHORT SWAP_SOURCE ;BN00X
19551 ;NOT_DIR1: ;;BN00X it is a file, delete the entry
19552 ;POP     SI
19553
19554 ; MSDOS 3.3 (& MSDOS 6.0)
19555 call    FastOpen_Delete ; delete dir info in fastopen DOS 3.3
19556 SWAP_SOURCE:
19557 ; MSDOS 3.3
19558 ;MOV     SI,[REN_WFP]
19559 ;MOV     [WFP_START],SI
19560 ; MSDOS 6.0
19561 MOV     AX,[WFP_START] ; Swap source and destination
19562 MOV     SI,[REN_WFP] ; Swap source and destination
19563 MOV     [WFP_START],SI ; WFP_START = Destination path
19564 MOV     [REN_WFP],AX ; REN_WFP = Source path
19565 ; MSDOS 3.3 (& MSDOS 6.0)
19566 MOV     word [CURR_DIR_END],-1; No current dir on dest
19567 ;mov     word [CREATING],0E5FFh
19568 MOV     WORD [CREATING],DIRFREE*256+0FFh ; Creating, not DEL *.*
19569 ; A rename is like a CREATE_NEW as far
19570 ; as the destination is concerned.
19571 call    GetPathNoSet
19572
19573 ; If this GETPATH fails due to file not found, we know all renames will work
19574 ; since no files match the destination name. If it fails for any other
19575 ; reason, the rename fails on a path not found, or whatever (also fails if
19576 ; we find a device or directory). If the GETPATH succeeds, we aren't sure
19577 ; if the rename should fail because we haven't built an explicit name by

```



```

19578 ; substituting for the meta chars in it. In this case the destination file
19579 ; spec with metas is in [NAME1] and the explicit source name is at RENAMEDMA
19580 ; in the directory entry part.
19581
19582 00002E30 722A JC short NODEST
19583
19584 ; MSDOS 6.0
19585 ;JZ short BAD_ACC ; Dest string is a directory ;AC000;
19586 ; !! MSDOS 3.3 !!
19587 ;JZ short BAD_ACC ; !! ; Dest string is a directory
19588
19589 00002E32 08E4 OR AH,AH ; Device?
19590 00002E34 7933 JNS short SAVEDEST ; No, continue
19591
19592 00002E36 B80500 BAD_ACC: MOV AX,error_access_denied
19593 00002E39 F9 STC
19594
19595 00002E3A 9C RENAME_CLEAN: PUSHF ; Save carry state
19596 00002E3B 50 PUSH AX ; and error code (if carry set)
19597
19598 ;;;
19599 ; 31/01/2024 - Retro DOS v5.0
19600 00002E3C C42E[8A05] ; (PCDOS 7.1 IBMDOS.COM)
19601 00002E40 E82906 les bp,[THISDPB]
19602 call update_fat32_fsinfo
19603 00002E43 A0[7605] ;;;
19604 00002E46 E84F3C MOV AL,[THISDRV]
19605 00002E49 58 call FLUSHBUF
19606 00002E4A 803E[4A03]00 POP AX
19607 00002E4F 7504 CMP byte [FAILERR],0
19608 00002E51 9D JNZ short BAD_ERR ; User FAILED to I 24
19609 00002E52 E972FF POPF
19610 JMP RENAME_POP
19611
19612 00002E55 58 BAD_ERR: POP AX ; Saved flags
19613 ; 16/12/2022
19614 ; 14/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
19615
19616 00002E56 B80300 BAD_PATH: ; *
19617 00002E59 E96AFF MOV AX,error_path_not_found
19618 JMP GOTERR
19619
19620 00002E5C 75F8 NODEST: JNZ short BAD_PATH
19621 00002E5E 803E[4A03]00 CMP byte [FAILERR],0
19622 00002E63 75F1 JNZ short BAD_PATH ; Search for dest failed
19623 ; because user FAILED on I 24
19624
19625 00002E65 08C9 ; 14/11/2022
19626 OR CL,CL
19627 ;JNZ short SAVEDEST
19628 00002E67 74ED ; 17/05/2019
19629 JZ short BAD_PATH ; *
19630 ;BAD_PATH: ; *
19631 ; MOV AX,error_path_not_found
19632 ; ;STC
19633 ; ;JMP RENAME_POP
19634 ; ; 17/05/2019
19635 ; jmp GOTERR
19636
19637 ; 16/12/2022
19638 %if 0
19639 ; 14/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
19640 or cl,cl
19641 jnz short SAVEDEST
19642 ;jz short BAD_PATH ; *
19643 BAD_PATH: ; *
19644 ;mov ax,3
19645 mov ax,error_path_not_found
19646 stc
19647 jmp RENAME_POP
19648 %endif
19649
19650 00002E69 16 SAVEDEST: push ss
19651 00002E6A 07 pop es
19652
19653 ;hkn; NAME1 & NAME2 is in DOSDATA
19654 00002E6B BF[5705] MOV DI,NAME2
19655 00002E6E BE[4B05] MOV SI,NAME1
19656
19657 00002E71 B90B00 MOV CX,11
19658 00002E74 F3A4 REP MOVSB ; Save dest with metas at NAME2
19659
19660 ;;;
19661 ; 31/01/2024
19662 00002E76 A1[DC0A] ; (PCDOS 7.1 IBMDOS.COM)
19663 00002E79 A3[E60A] mov ax,[DIRSTART_HW]
19664 [DESTSTART_HW],ax
19665 00002E7C A1[C205] ;;;
19666 00002E7F A3[6405] MOV AX,[DIRSTART]
19667 [DESTSTART],AX
19668 BUILDDEST:
19669 ; 31/01/2024
19670 ;push ss
19671 ;pop es ; needed due to JMP BUILDDEST below
19672
19673 00002E82 BB[3506] ;hkn; RENAMEDMA, NAME1, NAME2 in DOSDATA
19674 00002E85 BF[4B05] MOV BX,RENAMEDMA+21 ; Source of replace chars
19675 00002E88 BE[5705] MOV DI,NAME1 ; Real dest name goes here
19676 MOV SI,NAME2 ; Raw dest
19677 00002E8B B90B00 MOV CX,11
19678
19679 ; 17/05/2019 - Retro DOS v4.0
19680
19681 ; MSDOS 6.0
19682 00002E8E E82601 CALL NEW_RENAME ;IFS. replace ? chars ;AN000;
19683
19684 ; MSDOS 3.3
19685
19686 ; 08/08/2018 - Retro DOS v3.0
19687 ; MSDOS 6.0
19688 ;-----
19689 ;Procedure: NEW_RENAME
19690 ;
19691 ;Input: DS:SI -> raw string with ?
19692 ; ES:DI -> destination string
19693 ; DS:BX -> source string
19694 ;Function: replace ? chars of raw string with chars in source string and
19695 ; put in destination string
19696 ;Output: ES:DI-> new string
19697 ;-----
19698 ;
19699 ;NEW_RENAME:
19700 ;NEWNAM:
19701 ; ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 341Ah

```

```

19702 ; LODSB
19703 ; CMP AL,"?"
19704 ; JNZ short NOCHG
19705 ; MOV AL,[BX] ; Get replace char
19706 ; NOCHG:
19707 ; STOSB
19708 ; INC BX ; Next replace char
19709 ; LOOP NEWNAM
19710 ; ; MSDOS 6.0
19711 ; ; retn
19712
19713 ; MSDOS 3.3 & MSDOS 6.0
19714 ; mov byte [ATTRIB],16h
19715 00002E91 C606[6B05]16 MOV byte [ATTRIB],attr_all; Stop duplicates with any attributes
19716 00002E96 C606[7E05]FF MOV byte [CREATING],0FFh
19717 00002E9B E8361F call DEVNAME ; Check if we built a device name
19718 00002E9E 7396 JNC short BAD_ACC
19719 ;;;
19720 ; 31/01/2024
19721 ; (PCDOS 7.1 IBMDOS.COM)
19722 00002EA0 8B1E[E60A] mov bx,[DESTSTART_HW]
19723 00002EA4 891E[EE0A] mov [ROOTCLUST_HW],bx
19724 ;;;
19725 00002EA8 8B1E[6405] MOV BX,[DESTSTART]
19726 00002EAC C42E[8A05] LES BP,[THISDPB]
19727 00002EB0 E8AC1A call SETDIRSRCH ; Reset search to start of dir
19728 00002EB3 7281 JC short BAD_ACC ; Screw up
19729 00002EB5 E88918 call FINDENTRY ; See if new name already exists
19730 ; JNC short BAD_ACC ; Error if found
19731 ; 31/01/2024
19732 00002EB8 7346 jnc short BAD_ACCJ
19733 00002EBA 803E[4A03]00 CMP byte [FAILERR],0
19734 00002EBF 753F JNZ short BAD_ACCJ ; Find failed because user FAILED to I 24
19735 00002EC1 A1[6405] MOV AX,[DESTSTART] ; DIRSTART of dest
19736 ;;;
19737 ; 31/01/2024
19738 00002EC4 8B16[E60A] mov dx,[DESTSTART_HW]
19739 00002EC8 C42E[8A05] les bp,[THISDPB]
19740 00002ECC 26837E0F00 cmp word [es:bp+DPB.FAT_SIZE],0
19741 ; cmp word [es:bp+0Fh],0
19742 00002ED1 7506 jnz short builddst_1 ; not FAT32
19743 00002ED3 3B16[3106] cmp dx,[RENAMEDMA+17] ; DIRSTART_HW of source
19744 00002ED7 7506 jne short builddst_2
19745 builddst_1:
19746 ;;;
19747 00002ED9 3B06[2F06] CMP AX,[RENAMEDMA+15] ; DIRSTART of source
19748 00002EDD 7451 JZ short SIMPLE_RENAME ; If =, just give new name
19749 builddst_2: ; 31/01/2024
19750 ; mov al,[RENAMEDMA+32]
19751 00002EDF A0[4006] MOV AL,[RENAMEDMA+21+dir_entry.dir_attr]
19752 00002EE2 A810 TEST AL,attr_directory ; 10h
19753 00002EE4 751A JNZ short BAD_ACCJ ; Can only do a simple rename on dirs,
19754 ; otherwise the . and .. entries get
19755 ; wiped.
19756 00002EE6 A2[6B05] MOV [ATTRIB],AL
19757 00002EE9 8C1E[A005] MOV [THISSFT+2],DS
19758
19759 ; hkn; AUXSTACK is in DOSDATA
19760 ; mov si,RENAMEDMA+145h
19761 00002EED BE[6507] MOV SI,AUXSTACK-SF_ENTRY.size ; RENAMEDMA+325
19762 00002EF0 8936[9E05] MOV [THISSFT],SI
19763 ; mov word [SI+2],2
19764 00002EF4 C744020200 MOV word [SI+SF_ENTRY.sf_mode],SHARING_COMPAT+open_for_both
19765 00002EF9 31C9 XOR CX,CX ; Set "device ID" for call into makenode
19766 00002EFB E88125 call RENAME_MAKE ; This is in mknode
19767 00002EFE 7303 JNC short GOT_DEST
19768 BAD_ACCJ:
19769 JMP BAD_ACC
19770
19771 GOT_DEST:
19772 push bx
19773 00002F03 53 LES DI,[THISSFT] ; RENAME_MAKE entered this into sharing
19774 00002F08 E87955 call ShareEnd ; we need to remove it.
19775 00002F0B 5B pop bx
19776
19777 ; A zero length entry with the correct new name has now been made at
19778 ; [CURBUF+2]:BX.
19779
19780 00002F0C C43E[E205] LES DI,[CURBUF]
19781
19782 ; 31/01/2024 - Retro DOS v5.0
19783 %if 0
19784 ; MSDOS 6.0
19785 ; test byte [es:di+5],40h
19786 TEST byte [ES:DI+BUFFINFO.buf_flags],buf_dirty
19787 ; LB. if already dirty ; AN000;
19788 JNZ short yesdirty1 ; LB. don't increment dirty count ; AN000;
19789 call INC_DIRTY_COUNT ; LB. ; AN000;
19790 ; or byte [es:di+5],40h
19791 OR byte [ES:DI+BUFFINFO.buf_flags],buf_dirty
19792 yesdirty1:
19793 %else
19794 ; 31/01/2024 (PCDOS 7.1 IBMDOS.COM)
19795 00002F10 E8AB3C call SET_BUF_DIRTY
19796 %endif
19797 00002F13 89DF MOV DI,BX
19798 ; add di,11
19799 00002F15 83C70B ADD DI,dir_entry.dir_attr ; Skip name
19800
19801 ; hkn; RENAMEDMA is in DOSDATA
19802 ; mov si,RENAMEDMA+32 ; 05/07/2024
19803 00002F18 BE[4006] MOV SI,RENAMEDMA+21+dir_entry.dir_attr
19804 ; mov cx,21
19805 00002F1B B91500 MOV CX,dir_entry.size-dir_entry.dir_attr
19806 00002F1E F3A4 REP MOVSB
19807 00002F20 E86E00 CALL GET_SOURCE
19808 00002F23 7269 JC short RENAME_OVER
19809 00002F25 89DF MOV DI,BX
19810 00002F27 8E06[E405] MOV ES,[CURBUF+2]
19811 00002F2B B0E5 MOV AL,DIRFREE ; 0E5h
19812 00002F2D AA STOSB ; "free" the source
19813 00002F2E EB13 JMP SHORT DIRTY_IT
19814
19815 SIMPLE_RENAME:
19816 00002F30 E85E00 CALL GET_SOURCE ; Get the source back
19817 00002F33 7259 JC short RENAME_OVER
19818 00002F35 89DF MOV DI,BX
19819 00002F37 8E06[E405] MOV ES,[CURBUF+2]
19820
19821 ; hkn; NAME1 is in DOSDATA
19822 00002F3B BE[4B05] MOV SI,NAME1 ; New Name
19823 00002F3E B90B00 MOV CX,11
19824 00002F41 F3A4 REP MOVSB
19825 DIRTY_IT:

```



```

19826 00002F43 8B3E[E205]      MOV     DI,[CURBUF]
19827
19828 ; 31/01/2024 - Retro DOS v5.0
19829 %if 0
19830 ; MSDOS 6.0
19831 TEST    byte [ES:DI+BUFFINFO.buf_flags],buf_dirty
19832 ;LB. if already dirty ;AN000;
19833 JNZ     short yesdirty2 ;LB. don't increment dirty count ;AN000;
19834 call    INC_DIRTY_COUNT ;LB. ;AN000;
19835
19836 OR      byte [ES:DI+BUFFINFO.buf_flags],buf_dirty
19837 %else
19838 ; 31/01/2024 (PCDOS 7.1 IBMDOS.COM)
19839 call    SET_BUF_DIRTY
19840 %endif
19841
19842 ;-----
19843 ; Check if the source is a directory of file. If directory rename it to the
19844 ; the new name in the Fastopen cache buffer. If file name it has been
19845 ; previously deleted.
19846 ;-----
19847
19848 ;yesdirty2: ; 31/01/2024
19849 ; MSDOS 6.0
19850 00002F4A 56      PUSH    SI
19851 00002F4B C536[2C03] LDS     SI,[DMAADD] ;;BN00XPTM. chek if source a dir ;AN000;
19852 ;add si,21
19853 00002F4F 83C615 ADD     SI,find_buf.attr ;;BN00XPTM.P5520 ;AN000;
19854 ;test byte [si+0Bh],10h
19855 00002F52 F6440B10 TEST    byte [SI+dir_entry.dir_attr],attr_directory ;;BN00XPTM. ;AN000;
19856 00002F56 7403    JZ      short NOT_DIR2 ;;BN00XPTM. ;AN000;
19857 00002F58 E8F2FD call    FastOpen_Rename ;;BN00X rename dir entry in fastopen
19858 ; 31/01/2024
19859 ;POP SI
19860 ;JMP SHORT NOT_DIRTY1
19861 NOT_DIR2: ;;BN00X it is a file, delete the entry
19862 00002F5B 5E      POP     SI
19863 NOT_DIRTY1: ;;BN00X
19864 NEXT_SOURCE:
19865 ;hkn; RENAMEDMA is in DOSDATA
19866 00002F5C BE[2106] MOV     SI,RENAMEDMA+1 ;Name
19867
19868 ; WARNING! Rename_Next leaves the disk critical section *ALWAYS*. We need
19869 ; to enter it before going to RENAME_Next.
19870
19871 00002F5F E880E9 call    ECritDisk
19872 00002F62 C606[7E05]00 MOV     byte [CREATING],0 ; Correct setting for search (we changed it
19873 ; to FF when we made the prev new file).
19874 00002F67 E8FF06 call    RENAME_NEXT
19875
19876 ; Note, now, that we have exited the previous ENTER and so are back to where
19877 ; we were before.
19878
19879 00002F6A 7222    JC      short RENAME_OVER
19880
19881 ;lea si,[bx+26]
19882 00002F6C 8D771A LEA     SI,[BX+dir_entry.dir_first]
19883 00002F6F E84FFD call    REN_DEL_Check
19884 00002F72 7306    JNC     short REN_OK2
19885 00002F74 B82000 MOV     AX,error_sharing_violation ; 20h
19886 jmp_to_rename_clean: ; 28/12/2022
19887 00002F77 E9C0FE JMP     RENAME_CLEAN ; 10/08/2018
19888
19889 ;-----
19890 ; Check if file or directory. If file, delete file from the Fastopen cache,
19891 ; if directory, rename directory name in the Fastopen cache.
19892 ;-----
19893
19894 REN_OK2:
19895 ; MSDOS 6.0
19896 ;mov al,[RENAMEDMA+32]
19897 00002F7A A0[4006] MOV     AL,[RENAMEDMA+21+dir_entry.dir_attr] ; PTR P5622
19898 ;test al,10h
19899 00002F7D A810    TEST    AL,attr_directory ;;BN00X directory
19900 00002F7F 7408    JZ      short Ren_Directory ;;BN00X no - file, delete it
19901
19902 ; MSDOS 3.3 & MSDOS 6.0
19903 00002F81 E8BBFD call    FastOpen_Delete ;;BN00X delete dir info in fastopen DOS 3.3
19904 jmp_to_builddest: ; 28/12/2022
19905 ; 31/01/2024
19906 00002F84 16      push    ss
19907 00002F85 07      pop     es
19908 00002F86 E9F9FE JMP     BUILDDDEST ;;BN00X
19909
19910 ; MSDOS 6.0
19911 Ren_Directory:
19912 00002F89 E8C1FD call    FastOpen_Rename ;;BN00X delete dir info in fastopen DOS 3.3
19913 ;JMP BUILDDDEST
19914 ; 28/12/2022
19915 00002F8C EBF6    jmp     short jmp_to_builddest
19916
19917 RENAME_OVER:
19918 00002F8E F8      CLC
19919 ;JMP RENAME_CLEAN ; 10/08/2018
19920 ; 28/12/2022
19921 00002F8F EBE6    jmp     short jmp_to_rename_clean
19922
19923 ;-----
19924 ; Procedure: GET_SOURCE
19925 ;
19926 ; Inputs:
19927 ; RENAMEDMA has source info
19928 ; Function:
19929 ; Re-find the source
19930 ; Output:
19931 ; [CURBUF] set
19932 ; [CURBUF+2]:BX points to entry
19933 ; Carry set if error (currently user FAILED to I 24)
19934 ; DS preserved, others destroyed
19935 ;-----
19936
19937 GET_SOURCE:
19938 ;;;
19939 ; 01/02/2024 - Retro DOS v5.0
19940 ; (PCDOS 7.1 IBMDOS.COM)
19941 00002F91 C42E[8A05] les     bp,[THISDPB]
19942 00002F95 31DB    xor     bx,bx ; 0
19943 ;cmp [es:bp+0Fh],bx
19944 00002F97 26395E0F cmp     [es:bp+DPB.FAT_SIZE],bx ; > 0 ?
19945 00002F9B 7504    jnz     short gs_cont ; yes, it is not FAT32
19946 00002F9D 8B1E[3106] mov     bx,[RENAMEDMA+17] ; DirStart+2
19947 gs_cont:
19948 00002FA1 891E[EE0A] mov     [ROOTCLUST_HW],bx ; hw of cluster number
19949 ;;;

```

```

19950 00002FA5 8B1E[2F06]      MOV     BX,[RENAMEDMA+15]      ; DirStart
19951                          ; 01/02/2024
19952                          ;LES     BP,[THISDPB]
19953 00002FA9 E8B319          call    SETDIRSRCH
19954 00002FAC 7214            JC      short gs_ret_label      ; retc
19955 00002FAE E8021E          call    STARTSRCH
19956 00002FB1 A1[2D06]        MOV     AX,[RENAMEDMA+13]      ; Lastent
19957                          ;call    GETENT
19958                          ; 18/12/2022
19959 00002FB4 E9EB18          jmp     GETENT
19960                          ;gs_ret_label:
19961                          ;retn
19962
19963                          ; MSDOS 6.0
19964
19965                          ;-----
19966                          ;Procedure: NEW_RENAME
19967                          ;
19968                          ;Input: DS:SI -> raw string with ?
19969                          ;      ES:DI -> destination string
19970                          ;      DS:BX -> source string
19971                          ;Function: replace ? chars of raw string with chars in source string and
19972                          ;      put in destination string
19973                          ;Output: ES:DI-> new string
19974                          ;-----
19975
19976                          NEW_RENAME:
19977                          ; 17/05/2019 - Retro DOS v4.0
19978                          NEWNAM:
19979                          ; DOSCODE:680Eh (MSDOS 6.21, MSDOS.SYS)
19980 00002FB7 AC                LODSB
19981 00002FB8 3C3F              CMP     AL,"?" ; 3Fh
19982 00002FBA 7502              JNZ     short NOCHG
19983                          MOV     AL,[BX]          ; Get replace char
19984 00002FBE AA                STOSB
19985 00002FBF 43                INC     BX          ; Next replace char
19986 00002FC0 E2F5              LOOP    NEWNAM
19987                          ; MSDOS 6.0
19988                          gs_ret_label:      ; 18/12/2022
19989 00002FC2 C3                retn
19990
19991                          ;=====
19992                          ; FINFO.ASM, MSDOS 6.0, 1991
19993                          ;=====
19994                          ; 08/08/2018 - Retro DOS v3.0
19995                          ; 17/05/2019 - Retro DOS v4.0
19996
19997                          ;** Low level routines for returning file information and setting file
19998                          ; attributes
19999                          ;
20000                          ; GET_FILE_INFO
20001                          ; SET_FILE_ATTRIBUTE
20002                          ;
20003                          ; Modification history:
20004                          ;
20005                          ; Created: ARR 30 March 1983
20006                          ;
20007                          ; M025: Return access_denied if attempting to set
20008                          ; attribute of root directory.
20009                          ;
20010
20011                          ;SUBTTL GET_FILE_INFO -- Get File Information
20012
20013                          ;-----
20014                          ; Procedure Name : GET_FILE_INFO
20015                          ;
20016                          ; Inputs:
20017                          ; [WFP_START] Points to WFP string ("d:/" must be first 3 chars, NUL
20018                          ; terminated)
20019                          ; [CURR_DIR_END] Points to end of Current dir part of string
20020                          ; (= -1 if current dir not involved, else
20021                          ; Points to first char after last "/" of current dir part)
20022                          ; [THISCDS] Points to CDS being used
20023                          ; (Low word = -1 if NUL CDS (Net direct request))
20024                          ; [SATTRIB] Is attribute of search, determines what files can be found
20025                          ; Function:
20026                          ; Get Information about a file
20027                          ; Returns:
20028                          ; CARRY CLEAR
20029                          ; AX = Attribute of file
20030                          ; CX = Time stamp of file
20031                          ; DX = Date stamp of file
20032                          ; BX:DI = Size of file (32 bit)
20033                          ; CARRY SET
20034                          ; AX is error code
20035                          ; error_file_not_found
20036                          ; Last element of path not found
20037                          ; error_path_not_found
20038                          ; Bad path (not in curr dir part if present)
20039                          ; error_bad_curr_dir
20040                          ; Bad path in current directory part of path
20041                          ; DS preserved, others destroyed
20042                          ;-----
20043
20044                          ; 14/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
20045
20046                          GET_FILE_INFO:
20047
20048                          ;hkn; get_file_info is called from file.asm and fcbio.asm. DS has been set
20049                          ;hkn; to DOSDATA at this point. So DOSassume is OK.
20050
20051 00002FC3 E863E8            call    TestNet
20052 00002FC6 7306            JNC     short LOCAL_INFO
20053
20054                          ;IF NOT Installed
20055                          ; transfer NET_GET_FILE_INFO
20056                          ;ELSE
20057                          ; MOV     AX,(MULTNET SHL 8) OR 15
20058                          ; INT     2FH
20059                          ; return
20060
20061 00002FC8 B80F11            mov     ax, 110Fh
20062 00002FCB CD2F            int     2Fh      ; Multiplex - NETWORK REDIRECTOR - GET REMOTE FILE'S ATTRIBUTES
20063                          ; SS = DOS CS, SDA first filename pointer -> fully-qualified name of file
20064                          ; SDA CDS pointer -> current directory
20065                          ; Return: CF set on error, AX = file attributes
20066 00002FCD C3                retn
20067                          ;ENDIF
20068
20069                          LOCAL_INFO:
20070 00002FCE E811E9            call    ECritDisk
20071 00002FD1 C606[4C03]01      MOV     byte [NoSetDir],1      ; if we find a dir, don't change to it
20072                          ; MSDOS 3.3
20073                          ;call    GETPATH

```

```

20074          ; MSDOS 6.0
20075 00002FD6 E8C900 call GET_FAST_PATH
20076          ; MSDOS 3.3 & MSDOS 6.0
20077 00002FD9 7312 JNC short info_check_dev
20078 NO_PATH:
20079 JNZ short bad_path1
20080 00002FDB 750B OR CL,CL
20081 00002FDD 08C9 JZ short bad_path1
20082 info_no_file:
20083 00002FE1 B80200 MOV AX,error_file_not_found
20084 BadRet:
20085 00002FE4 F9 STC
20086 JustRet:
20087 ;call LCritDisk
20088 ;retn
20089 ; 18/12/2022
20090 00002FE5 E927E9 jmp LCritDisk
20091
20092 bad_path1:
20093 00002FE8 B80300 MOV AX,error_path_not_found
20094 00002FEB EBF7 jmp short BadRet
20095
20096 info_check_dev:
20097 00002FED 08E4 OR AH,AH
20098 00002FEF 78F0 JS short info_no_file ; device
20099
20100          ; MSDOS 6.0
20101 ;SR;
20102 ; If root dir then CurBuf == -1. Check for this case and return subdir attr
20103 ;for a root dir
20104
20105 00002FF1 833E[E205]FF cmp word [CURBUF],-1 ;is it a root dir?
20106 00002FF6 7506 jne short not_root ;no, CurBuf ptr is valid
20107
20108 xor ah,ah
20109 00002FFA B010 mov al,attr_directory ; 10h
20110 ;clc
20111 ; 14/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
20112 ; (DOSCODE:683Eh)
20113 ; 16/12/2022
20114 ;clc
20115 00002FFC EBE7 jmp short JustRet
20116
20117 not_root:
20118 ; MSDOS 3.3 (& MSDOS 6.0)
20119 00002FFE 1E PUSH DS
20120 00002FFF 8E1E[E405] MOV DS,[CURBUF+2]
20121 00003003 89DE MOV SI,BX
20122 00003005 31DB XOR BX,BX ; Assume size=0 (dir)
20123 00003007 89DF MOV DI,BX
20124 ;mov cx,[si+16h]
20125 00003009 8B4C16 MOV CX,[SI+dir_entry.dir_time]
20126 ;mov dx,[si+18h]
20127 0000300C 8B5418 MOV DX,[SI+dir_entry.dir_date]
20128 0000300F 30E4 XOR AH,AH
20129 ;mov al,[si+0Bh]
20130 00003011 8A440B MOV AL,[SI+dir_entry.dir_attr]
20131 ;test al,10h
20132 00003014 A810 TEST AL,attr_directory
20133 00003016 7506 JNZ short NO_SIZE
20134 ;mov di,[si+1Ch]
20135 00003018 8B7C1C MOV DI,[SI+dir_entry.dir_size_l]
20136 ;mov bx,[si+1Eh]
20137 0000301B 8B5C1E MOV BX,[SI+dir_entry.dir_size_h]
20138 NO_SIZE:
20139 0000301E 1F POP DS
20140 ;CLC
20141 ; 14/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
20142 ; (DOSCODE:6864h)
20143 ; 16/12/2022
20144 ;clc
20145 0000301F EBC4 jmp short JustRet
20146
20147 ;Break <SET_FILE_ATTRIBUTE -- Set File Attribute>
20148 -----
20149 ; Procedure Name : SET_FILE_ATTRIBUTE
20150 ; Inputs:
20151 ; [WFP_START] Points to WFP string ("d:/" must be first 3 chars, NUL
20152 ; terminated)
20153 ; [CURR_DIR_END] Points to end of Current dir part of string
20154 ; (= -1 if current dir not involved, else
20155 ; Points to first char after last "/" of current dir part)
20156 ; [THISCDS] Points to CDS being used
20157 ; (Low word = -1 if NUL CDS (Net direct request))
20158 ; [SATTRIB] is attribute of search (determines what files may be found)
20159 ; AX is new attributes to give to file
20160 ; Function:
20161 ; Set File Attributes
20162 ; Returns:
20163 ; CARRY CLEAR
20164 ; No error
20165 ; CARRY SET
20166 ; AX is error code
20167 ; error_file_not_found
20168 ; Last element of path not found
20169 ; error_path_not_found
20170 ; Bad path (not in curr dir part if present)
20171 ; error_bad_curr_dir
20172 ; Bad path in current directory part of path
20173 ; error_access_denied
20174 ; Attempt to set an attribute which cannot be set
20175 ; (attr_directory, attr_volume_ID)
20176 ; error_sharing_violation
20177 ; Sharing mode of file did not allow the change
20178 ; (this request requires exclusive write/read access)
20179 ; (INT 24H generated)
20180 ; DS preserved, others destroyed
20181 -----
20182
20183 ; 01/02/2024 - Retro DOS v5.0
20184
20185 SET_FILE_ATTRIBUTE:
20186
20187 ;hkn; set_file_attr is called from file.asm. DS has been set
20188 ;hkn; to DOSDATA at this point. So DOSassume is OK.
20189
20190 00003021 A9D8FF TEST AX,~attr_changeable ; 0FFD8h
20191 00003024 7412 JZ short set_look
20192 _BAD_ACC:
20193 ;MOV byte [EXTERR_LOCUS],errLOC_Unk ; 1
20194 ;;;
20195 ; 01/02/2024 - Retro DOS v5.0
20196 ; (PCDOS 7.1 IBMDOS.COM)
20197 00003026 E8B4E2 call set_exerr_locus_unk

```

```

20198      ;;;
20199 00003029 C606[2703]07      MOV     byte [EXTERR_CLASS],errCLASS_Apperr ; 7
20200 0000302E C606[2603]04      MOV     byte [EXTERR_ACTION],errACT_Abort ; 4
20201 00003033 B80500            MOV     AX,error_access_denied ; 5
20202 00003036 F9                STC
20203 00003037 C3                retn
20204
20205      set_look:
20206      call     TestNet
20207 0000303B 7308                JNC     short LOCAL_SET
20208
20209      ;IF NOT Installed
20210      ; transfer NET_SEQ_SET_FILE_ATTRIBUTE
20211      ;ELSE
20212 0000303D 50                PUSH     AX
20213
20214      ;MOV     AX,(MultNET SHL 8) OR 14
20215      ;INT     2FH
20216
20217 0000303E B80E11            mov     ax, 110Eh
20218 00003041 CD2F            int     2Fh      ; Multiplex - NETWORK REDIRECTOR - SET REMOTE FILE'S ATTRIBUTES
20219                                ; SS = DOS CS, SDA first filename pointer -> fully-qualified name of file
20220                                ; SDA CDS pointer -> current directory
20221                                ; STACK: WORD new file attributes
20222                                ; Return: CF set on error
20223
20224 00003043 5B                POP      BX      ; clean stack
20225 00003044 C3                retn
20226      ;ENDIF
20227
20228      LOCAL_SET:
20229 00003045 E89AE8            call    ECritDisk
20230 00003048 50                PUSH     AX      ; Save new attributes
20231 00003049 C606[4C03]01      MOV     byte [NoSetDir],1 ; if we find a dir, don't change to it
20232 0000304E E8C21A            call    GETPATH   ; get path through fastopen if there ;AC000;
20233 00003051 7308                JNC     short set_check_device
20234 00003053 5B                POP      BX      ; Clean stack (don't zap AX)
20235 00003054 EB85                JMP      short NO_PATH
20236
20237      ; MSDOS 6.0
20238      cannot_set_root:
20239 00003056 B80500            mov     ax,error_access_denied ; M025:
20240                                ; M025: return error is attempting
20241                                ; M025: to set attr. of root
20242                                ; 01/02/2024 ; M025:
20243                                jmp     short OK_BYE
20244                                jmp     short BadRet
20245
20246      set_check_device:
20247      OR      AH,AH
20248      JNS     short set_check_share
20249      POP     AX
20250      call    LCritDisk
20251      JMP     short _BAD_ACC      ; device
20252
20253      set_check_share:
20254      POP     AX      ; Get new attributes
20255
20256      ; MSDOS 6.0
20257 00003066 833E[E205]FF      cmp     word [CURBUF], -1 ; M025: Q: is this the root dir
20258 0000306B 74E9                je      short cannot_set_root ; M025: Y: return error
20259
20260      ; MSDOS 3.3 & MSDOS 6.0
20261 0000306D E851FC            call    REN_DEL_Check
20262 00003070 7305                JNC     short set_do
20263 00003072 B82000            MOV     AX,error_sharing_violation ; 32
20264 00003075 EB28                jmp     short OK_BYE
20265
20266      set_do:
20267      ; MSDOS 3.3 & MSDOS 6.0
20268 00003077 C43E[E205]      LES     DI,[CURBUF]
20269 0000307B 2680670BD8      AND     BYTE [ES:BX+dir_entry.dir_attr],~attr_changeable ; 0D8h
20270 00003080 2608470B      OR      BYTE [ES:BX+dir_entry.dir_attr],AL
20271
20272      ; 01/02/2024 - Retro DOS v5.0
20273      %if 0
20274      ; MSDOS 6.0
20275      TEST     byte [ES:DI+BUFFINFO.buf_flags],buf_dirty
20276                                ;LB. if already dirty ;AN000;
20277      JNZ      short yesdirty3 ;LB. don't increment dirty count ;AN000;
20278      call     INC_DIRTY_COUNT ;LB. ;AN000;
20279      OR      byte [ES:DI+BUFFINFO.buf_flags],buf_dirty
20280      yesdirty3:
20281      %else
20282      ; 01/02/2024
20283      ; (PCDOS 7.1 IBMDOS.COM)
20284 00003084 E8373B            call    SET_BUF_DIRTY
20285      %endif
20286 00003087 A0[7605]          MOV     AL,[THISDRV]
20287      ;;; 10/1/86 F.C update fastopen cache
20288      PUSH     DX
20289      PUSH     DI
20290 0000308C B400            MOV     AH,0      ; dir entry update
20291 0000308E 88C2            MOV     DL,AL      ; drive number A=0,B=1,,
20292 00003090 89DF            MOV     DI,BX      ; ES:DI -> dir entry
20293 00003092 E8D2FC            call    FastOpen_Update
20294 00003095 5F                POP      DI
20295 00003096 5A                POP      DX
20296      ;;; 9/11/86 F.C update fastopen cache
20297 00003097 E8FE39            call    FLUSHBUF
20298 0000309A 7303                JNC     short OK_BYE
20299 0000309C B80200            MOV     AX,error_file_not_found
20300      OK_BYE:
20301      ;call    LCritDisk
20302      ;retn
20303      ; 16/12/2022
20304 0000309F E96DE8            jmp     LCritDisk
20305
20306      ;-----
20307
20308      ; 17/05/2019 - Retro DOS v4.0
20309
20310      ; MSDOS 6.0
20311      GET_FAST_PATH:
20312      ;hkn; use SS override for FastOpenFlg
20313 000030A2 36800E[4611]01      OR      byte [ss:FastOpenFlg],FastOpen_Set
20314                                ;FO. trigger fastopen ;AN000;
20315      call     GETPATH
20316      PUSHF
20317      AND     byte [ss:FastOpenFlg],Fast_yes ;FO. ;AN000;
20318                                ;FO. clear all fastopen flags ;AN000;
20319      POPF
20320      retn
20321                                ;FO. ;AN000;

```

```

20322 ;=====
20323 ; DUP.ASM, MSDOS 6.0, 1991
20324 ;=====
20325 ; 08/08/2018 - Retro DOS v3.0
20326 ; 17/05/2019 - Retro DOS v4.0
20327
20328 ;** Low level DUP routine for use by EXEC when creating a new process.
20329 ; Exports the DUP to the server machine and increments the SFT ref count
20330 ;
20331 ; DOS_DUP
20332 ;
20333 ; Modification history:
20334 ;
20335 ; Created: ARR 30 March 1983
20336
20337 ;BREAK <DOS_DUP -- DUP SFT across network>
20338 ;-----
20339 ; Procedure Name : DOS_DUP
20340 ;
20341 ; Inputs:
20342 ; [THISSFT] set to the SFT for the file being DUPed
20343 ; (a non net SFT is OK, in this case the ref
20344 ; count is simply incremented)
20345 ; Function:
20346 ; Signal to the devices that a logical open is occurring
20347 ; Returns:
20348 ; ES:DI point to SFT
20349 ; Carry clear
20350 ; SFT ref_count is incremented
20351 ; Registers modified: None.
20352 ; NOTE:
20353 ; This routine is called from $CREATE_PROCESS_DATA_BLOCK at DOSINIT
20354 ; time with SS NOT DOSGROUP. There will be no Network handles at
20355 ; that time.
20356 ;-----
20357
20358 DOS_DUP:
20359 ;LES DI,[CS:THISSFT] ; MSDOS 3.3
20360
20361 ; MSDOS 6.0
20362 000030B4 2E8E06[0700] mov es,[cs:DosDSeg]
20363 000030B9 26C43E[9E05] les di,[es:THISSFT]
20364
20365 ;Entry Dos_Dup_Direct
20366 DOS_Dup_Direct:
20367 000030BE E881E7 call IsSFTNet
20368 000030C1 7503 JNZ short DO_INC
20369 000030C3 E84E21 call DEV_OPEN_SFT
20370
20371 DO_INC:
20372 000030C6 26FF05 ;INC word [ES:DI+SF_ENTRY.sf_ref_count]
20373 ; inc word [ES:DI] ; Clears carry (if this ever wraps
20374 000030C9 C3 ; we're in big trouble anyway)
20375 ret
20376
20377 ;=====
20378 ; CREATE.ASM, MSDOS 6.0, 1991
20379 ;=====
20380 ; 08/08/2018 - Retro DOS v3.0
20381 ; 18/05/2019 - Retro DOS v4.0
20382
20383 ;TITLE DOS_CREATE/DOS_CREATE_NEW - Internal CREATE calls for MS-DOS
20384 ;NAME DOS_CREATE
20385 ;-----
20386 ;** Internal Create and Create new to create a local or NET file and SFT.
20387 ;
20388 ; DOS_CREATE
20389 ; DOS_CREATE_NEW
20390 ; SET_MKND_ERR
20391 ; SET_Media_ID
20392 ; SET_EXT_Mode
20393 ;
20394 ; Revision history:
20395 ;
20396 ; A000 version 4.00 Jan. 1988
20397 ; A001 D490 -- Change IOCTL subfunctions from 63h,43h to 66h, 46h
20398
20399 ;Installed = TRUE
20400
20401 ; i_need THISSFT,DWORD
20402 ; i_need THISCDS,DWORD
20403 ; I_need EXTERR,WORD
20404 ; I_Need ExtErr_locus,BYTE
20405 ; I_need JShare,DWORD
20406 ; I_need VOLCHNG_FLAG,BYTE
20407 ; I_need SATTRIB,BYTE
20408 ; I_need CALLVIDM,DWORD
20409 ; I_need EXTOPEN_ON,BYTE ;AN000; extended open
20410 ; I_need NAME1,BYTE ;AN000;
20411 ; I_need NO_NAME_ID,BYTE ;AN000;
20412 ; I_need Packet_Temp,WORD ;AN000;
20413 ; I_need DOS34_FLAG,WORD ;AN000;
20414 ; I_need SAVE_BX,WORD ;AN000;
20415
20416 ;***DOS_CREATE - Create a File
20417 ;-----
20418 ; DOS_Create is called to create the specified file, truncating
20419 ; the old one if it exists.
20420 ;
20421 ; ENTRY AX is Attribute to create
20422 ; (ds) = DOSDATA
20423 ; [WFP_START] Points to WFP string ("d:/" must be first 3 chars, NUL
20424 ; terminated)
20425 ; [CURR_DIR_END] Points to end of Current dir part of string
20426 ; (= -1 if current dir not involved, else
20427 ; Points to first char after last "/" of current dir part)
20428 ; [THISCDS] Points to CDS being used
20429 ; (Low word = -1 if NUL CDS (Net direct request))
20430 ; [THISSFT] Points to SFT to fill in if file created
20431 ; (sf_mode field set so that FCB may be detected)
20432 ; [SATTRIB] Is attribute of search, determines what files can be found
20433 ;
20434 ; EXIT sf_ref_count is NOT altered
20435 ; CARRY CLEAR
20436 ; THISSFT filled in.
20437 ; sf_mode = unchanged for FCB, sharing_compat + open_for_both
20438 ; CARRY SET
20439 ; AX is error code
20440 ; error_path_not_found
20441 ; Bad path (not in curr dir part if present)
20442 ; error_bad_curr_dir
20443 ; Bad path in current directory part of path
20444 ; error_access_denied
20445 ; Attempt to re-create read only file , or
20446 ; create a second volume id or create a dir

```

```

20446 ; error_sharing_violation
20447 ; The sharing mode was correct but not allowed
20448 ; generates an INT 24
20449 ; USES all but DS
20450 ;-----
20451 ;
20452 ; 14/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
20453 ; DOSCODE:6920h (MSDOS 5.0, MSDOS.SYS)
20454 ;
20455 ; 01/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
20456 ; DOSCODE:708Ah (PCDOS 7.1, IBMDOS.COM)
20457
20458 DOS_CREATE:
20459 ; 18/05/2019 - Retro DOS v4.0
20460 ; DOSCODE:6934h (MSDOS 6.21, MSDOS.SYS)
20461
20462 ;hkn; dispatched to from file.asm and fcbio.asm. DS set up to DOSDATA at
20463 ;hkn; this point.
20464
20465 000030CA 30E4 XOR AH,AH ; Truncate is OK
20466
20467 ; Enter here from Dos_Create_New
20468 ;
20469 ; (ah) = 0 iff truncate OK
20470
20471 Create_inter:
20472 000030CC A8C0 TEST AL, ~(attr_all+attr_ignore+attr_volume_id) ; 80h
20473 ; Mask out any meaningless bits
20474 000030CE 7511 JNZ short AttErr
20475 000030D0 A808 TEST AL, attr_volume_id
20476 000030D2 7407 JZ short NoReset
20477
20478 ; MSDOS 6.0
20479 ; 16/12/2022
20480 000030D4 800E[1106]80 OR byte [DOS34_FLAG], DBCS_VOLID ; 80h ; AN000; FOR dbcs valid
20481 ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
20482 ; or word [DOS34_FLAG], DBCS_VOLID ; 80h
20483
20484 000030D9 B008 MOV AL, attr_volume_id ; 8
20485
20486 000030DB 0C20 NoReset: OR AL, attr_archive ; File changed ; 20h
20487 000030DD A850 TEST AL, attr_directory+attr_device ; 50h
20488 000030DF 7408 JZ short ATT_OK
20489
20490 000030E1 B80500 AttErr: MOV AX, 5 ; Attribute problem
20491 ; MOV byte [EXTERR_LOCUS], errLOC_Unk ; 1
20492 ; 01/02/2024
20493 000030E4 E8F6E1 call set_exerr_locus_unk
20494 000030E7 EB62 JMP SHORT SET_MKND_ERR ; Gotta use MKDIR to make dirs, NEVER allow
20495 ; attr_device to be set.
20496
20497 000030E9 C43E[9E05] ATT_OK: LES DI, [THISSFT]
20498 000030ED 06 PUSH ES
20499 000030EE C436[A205] LES SI, [THISCDS]
20500 000030F2 83FEFF CMP SI, -1
20501 000030F5 751B JNE short TEST_RE_NET
20502
20503 ; No CDS, it must be redirected.
20504
20505 000030F7 07 POP ES
20506
20507 ; MSDOS 6.0
20508 ; Extended open hooks
20509 ; test byte [EXTOPEN_ON], 1
20510 000030F8 F606[F605]01 TEST byte [EXTOPEN_ON], EXT_OPEN_ON ; AN000; EO. from extended open
20511 000030FD 740D JZ short NOEXTOP ; AN000; EO. no, do normal
20512 ; AN000; EO.
20513 000030FF 50 IFS_extopen: PUSH AX
20514 ; MOV AX, (MULTNET SHL 8) OR 46 ; AN000; EO. pass create attr
20515 00003100 B82E11 mov ax, 112Eh ; AN000; EO. issue extended open verb
20516
20517 00003103 CD2F NOEXTOP2: ; 01/02/2024 (PCDOS 7.1 IBMDOS.COM)
20518 00003105 5B INT 2FH ; AN000; EO.
20519 00003106 C606[F605]00 POP BX ; AN000; EO. trash bx
20520 0000310B C3 MOV byte [EXTOPEN_ON], 0 ; AN000; EO.
20521 ; AN000; EO.
20522 NOEXTOP: retn
20523 ; Extended open hooks
20524
20525 ; IF NOT Installed
20526 ; transfer NET_SEQ_CREATE
20527 0000310C 50 ; ELSE
20528 PUSH AX
20529
20530 ; MOV AX, (MULTNET SHL 8) OR 24
20531 ; INT 2FH
20532 0000310D B81811 mov ax, 1118h
20533 ; 01/02/2024
20534 ; int 2Fh ; Multiplex - NETWORK REDIRECTOR - CREATE/TRUNCATE FILE
20535 ; ES:DI -> uninitialized SFT, SS = DOS CS
20536 ; SDA first filename pointer -> fully-qualified name of file
20537 ; STACK: WORD file creation mode???
20538
20539 ; POP BX ; BX is trashed anyway
20540 ; retn
20541 00003110 EBF1 jmp short NOEXTOP2 ; 01/02/2024
20542 ; ENDIF
20543
20544 ; We have a CDS. See if it's network
20545
20546 TEST_RE_NET:
20547 ; test word [es:si+43h], 8000h
20548 ; TEST word [ES:SI+curdir.flags], curdir_isnet
20549 ; 07/12/2022
20550 ; test byte [es:si+44h], 80h
20551 ; 17/12/2022
20552 00003112 26F6444480 test byte [ES:SI+curdir.flags+1], curdir_isnet >> 8
20553 00003117 07 POP ES
20554 00003118 7417 JZ short LOCAL_CREATE
20555
20556 ; MSDOS 6.0
20557 0000311A E8CA00 CALL Set_EXT_mode ; AN000; EO.
20558 0000311D 7205 JC SHORT dochk ; AN000; EO.
20559 ; or word [es:di+2], 2
20560 ; OR word [ES:DI+SF_ENTRY.sf_mode], SHARING_COMPAT+open_for_both ; IFS.
20561 ; 17/12/2022
20562 0000311F 26804D0202 or byte [ES:DI+SF_ENTRY.sf_mode], SHARING_COMPAT+open_for_both ; IFS.
20563
20564 ; Extended open hooks
20565 dochk:
20566 00003124 F606[F605]01 TEST byte [EXTOPEN_ON], EXT_OPEN_ON ; AN000; EO. from extended open
20567 00003129 75D4 JNZ short IFS_extopen ; AN000; EO. yes, issue extended open
20568 ; Extended open hooks
20569

```



```

20570 ;IF NOT Installed
20571 ; transfer NET_CREATE
20572 ;ELSE
20573 0000312B 50      PUSH    AX
20574
20575 ;MOV    AX,(MultNET SHL 8) OR 23
20576 ;INT    2FH
20577
20578 0000312C B81711  mov     ax,1117h
20579
20580 ; 01/02/2024
20581 ;int     2Fh ; Multiplex - NETWORK REDIRECTOR - CREATE/TRUNCATE REMOTE FILE
20582 ; ES:DI -> uninitialized SFT, SS = DOS CS
20583 ; SDA first filename pointer -> fully-qualified name of file to open
20584 ; SDA CDS pointer -> current directory
20585 ; Return: CF set on error
20586
20587 ;POP     BX ; BX is trashed anyway
20588 ;nomore:
20589 ;retn
20590 0000312F EBD2     jmp     short NOEXTOP2 ; 01/02/2024
20591 ;ENDIF
20592
20593 ;** It's a local create. We have a local CDS for it.
20594
20595 LOCAL_CREATE:
20596 ; MSDOS 6.0
20597 00003131 E8B300    CALL    Set_EXT_mode ;AN000;EO. set mode if from extended open
20598 00003134 7205     JC      short setdone ;AN000;EO.
20599
20600 ; MSDOS 3.3 & MSDOS 6.0
20601 ; 17/12/2022
20602 ;;or     word [es:di+2],2
20603 ;OR      word [ES:DI+SF_ENTRY.sf_mode],SHARING_COMPAT+open_for_both
20604 ;or     byte [es:di+2],2
20605 00003136 26804D0202 or     byte [ES:DI+SF_ENTRY.sf_mode],SHARING_COMPAT+open_for_both
20606
20607 0000313B E8A4E7    call    ECritDisk
20608 0000313E E81A23    call    MakeNode
20609 00003141 7317     JNC     short Create_ok
20610 00003143 C606[A10A]FF mov     byte [VOLCHNG_FLAG],-1; indicate no change in volume label
20611 00003148 E8C4E7    call    LCritDisk
20612
20613 ;entry SET_MKND_ERR
20614 SET_MKND_ERR:
20615
20616 ; Looks up MakeNode errors and converts them. AL is MakeNode
20617 ; error, SI is GETPATH bad spot return if path_not_found error.
20618
20619 ;hkn; CRTERRTAB is in TABLE seg (DOSCODE)
20620 0000314B BB[5231]  MOV     BX,CRTERRTAB
20621 ;XLAT ; MSDOS 3.3
20622 ; 18/05/2019 - Retro DOS v4.0
20623 0000314E 2E       CS
20624 0000314F D7       XLAT
20625 CreatBadRet:
20626 00003150 F9       STC
20627 00003151 C3       retn
20628
20629 ; 13/05/2019 - Retro DOS v4.0
20630 ; DOSCODE:69C4h (MSDOS 6.21, MSDOS.SYS)
20631 ; -----
20632
20633 ;** Internal Create and Create new to create a local or NET file and SFT.
20634
20635 ; 17/07/2018 - Retro DOS v3.0
20636 ; Offset 12B1h of IBMDOS.COM (MSDOS 3.3), 1987
20637
20638 ;CRTERRTAB: ; 19/07/2018 - MSDOS 3.3
20639 ; db 0,5,52h,50h,3,5,20h
20640
20641 ;CRTERRTAB: ; 18/05/2019 - MSDOS 6.0
20642 ; db 0,5,52h,50h,3,5,20h,2
20643
20644 ; 08/08/2018
20645
20646 CRTERRTAB: ;LABEL BYTE ; Lookup table for MakeNode returns
20647 DB 0 ; none
20648 DB error_access_denied ; MakeNode error 1
20649 DB error_cannot_make ; MakeNode error 2
20650 DB error_file_exists ; MakeNode error 3
20651 DB error_path_not_found ; MakeNode error 4
20652 DB error_access_denied ; MakeNode error 5
20653 DB error_sharing_violation ; MakeNode error 6
20654 ; MSDOS 6.0
20655 DB error_file_not_found ; MakeNode error 7
20656
20657 ; -----
20658
20659 ; we have just created a new file. This results in the truncation of old
20660 ; files. We must inform the sharer to slash all the open SFT's for this
20661 ; file to the current size.
20662
20663 ; If we created a volume id on the diskette, set the VOLCHNG_FLAG to logical
20664 ; drive number to force a Build BPB after Media Check.
20665
20666 ;;; FASTOPEN 8/29/86
20667 Create_ok:
20668 0000315A E8E2FB    call    FastOpen_Delete
20669 ;;; FASTOPEN 8/29/86
20670 0000315D A0[6D05]    mov     al,[ATTRIB]
20671 00003160 A808     test    al,attr_volume_id
20672 00003162 741C     jz      short NoVolLabel
20673 00003164 C43E[A205]    LES     DI,[THISCDS]
20674 ;mov     ah,[ES:DI+curdir.text]; get drive letter
20675 00003168 268A25    mov     ah,[ES:DI] ; 09/08/2018
20676 0000316B 80EC41    sub     ah,'A' ; 41h ; convert to drive number
20677 0000316E 8826[A10A] mov     [VOLCHNG_FLAG],ah ;Set flag to indicate volid change
20678
20679 ; 18/05/2019 - Retro DOS v4.0
20680
20681 ; MSDOS 6.0
20682 00003172 B701     MOV     BH,1 ;AN000;>32mb set volume id to boot record
20683 00003174 E81F00    CALL    Set_Media_ID ;AN000;>32mb
20684
20685 00003177 E868E7    call    ECritDisk
20686 0000317A E8DE33    call    FATREAD_CDS ; force a media check
20687 0000317D E88FE7    call    LCritDisk
20688
20689 NoVolLabel:
20690 00003180 B80200    MOV     ax,2
20691 00003183 C43E[9E05]    LES     DI,[THISSFT]
20692 ;if installed
20693 ;call JShare + 14 * 4

```

```

20694 00003187 FF1E[C800]      call    far [Jshare+(14*4)] ; 14 = ShSU
20695                          ;else
20696                          ; Call    ShSU
20697                          ;endif
20698 0000318B E881E7          call    LCritDisk
20699 0000318E E95601          jmp     SET_SFT_MODE
20700
20701                          ;-----
20702                          ; Procedure Name : Dos_Create_New
20703                          ;
20704                          ; Inputs:
20705                          ; [WFP_START] Points to WFP string ("d:/" must be first 3 chars, NUL
20706                          ; terminated)
20707                          ; [CURR_DIR_END] Points to end of Current dir part of string
20708                          ; (= -1 if current dir not involved, else
20709                          ; Points to first char after last "/" of current dir part)
20710                          ; [THISCDS] Points to CDS being used
20711                          ; (Low word = -1 if NUL CDS (Net direct request))
20712                          ; [THISSFT] Points to SFT to fill in if file created
20713                          ; (sf_mode field set so that FCB may be detected)
20714                          ; [SATTRIB] Is attribute of search, determines what files can be found
20715                          ; AX is Attribute to create
20716                          ; Function:
20717                          ; Try to create the specified file truncating an old one that exists
20718                          ; Outputs:
20719                          ; sf_ref_count is NOT altered
20720                          ; CARRY CLEAR
20721                          ; THISSFT filled in.
20722                          ; sf_mode = sharing_compat + open_for_both for Non-FCB SFT
20723                          ; CARRY SET
20724                          ; AX is error code
20725                          ; error_path_not_found
20726                          ; Bad path (not in curr dir part if present)
20727                          ; error_bad_curr_dir
20728                          ; Bad path in current directory part of path
20729                          ; error_access_denied
20730                          ; Create a second volume id or create a dir
20731                          ; error_file_exists
20732                          ; Already a file by this name
20733                          ; DS preserved, others destroyed
20734                          ;-----
20735
20736 DOS_Create_New:
20737 00003191 B401              MOV     AH,1          ; Truncate is NOT OK
20738 00003193 E936FF          JMP     Create_inter
20739
20740                          ; MSDOS 6.0
20741                          ;-----
20742                          ; Procedure Name : Set_Media_ID
20743                          ;
20744                          ; Inputs:
20745                          ; NAME1= Volume ID
20746                          ; BH= 0, delete volume id
20747                          ; 1, set new volume id
20748                          ; DS= DOSGROUP
20749                          ; Function:
20750                          ; Set Volume ID to DOS 4.00 Boot record.
20751                          ; Outputs:
20752                          ; CARRY CLEAR
20753                          ; volume id set
20754                          ; CARRY SET
20755                          ; AX is error code
20756                          ;-----
20757
20758                          ; 18/05/2019 - Retro DOS v4.0
20759                          ; 01/02/2024 - Retro DOS v5.0
20760
20761 00003196 50              Set_Media_ID:
20762 00003197 06              PUSH    AX          ;AN000; >32mb
20763 00003198 57              PUSH    ES          ;AN000; >32mb
20764                          PUSH    DI          ;AN000; >32mb
20765 00003199 FEC4              INC     AH          ;AN000; >32mb bl=drive #
20766 0000319B 88E3              MOV     BL,AH          ;AN000; >32mb bl=drive # (A=1,B=2,,)
20767 0000319D B00D              MOV     AL,0DH          ;AN000; >32mb generic IOCTL
20768                          ;MOV     CX,0866H          ;AN001; >32mb get media id
20769                          ; 01/02/2024
20770                          ; (PCDOS 7.1 IBMDOS.COM)
20771 0000319F B96648          mov     cx,4866h          ; ch = FAT32 disk drive (CATEGORY CODE)
20772                          ; cl = minor code,
20773                          ; get volume serial number (and name)
20774
20775 ;hkn; PACKET_TEMP is in DOSDATA
20776 000031A2 BA[A00D]        MOV     DX,Packet_Temp ;AN000; >32mb
20777
20778 Set_Media_ID_1:
20779 ; 01/02/2024
20780 000031A5 51              push    cx
20781
20782 000031A6 53              PUSH    BX          ;AN000; >32mb
20783 000031A7 52              PUSH    DX          ;AN000; >32mb
20784 000031A8 30FF          XOR     BH,BH          ;AN000; >32mb
20785
20786 ;invoke $IOCTL
20787 000031AA E8D7F6          call    _$IOCTL          ;AN000; >32mb
20788
20789 000031AD 5A              POP     DX          ;AN000; >32mb
20790 000031AE 5B              POP     BX          ;AN000; >32mb
20791
20792 ; 01/02/2024
20793 000031AF 59              pop     cx
20794 ;JC short geterr ;AN000; >32mb
20795 000031B0 730A          jnc     short Set_Media_ID_2
20796 000031B2 80FD48          cmp     ch,48h          ; is it FAT32 disk drive request?
20797 000031B5 F9              stc
20798 000031B6 7529          jne     short geterr          ; (ch=8 request failed!)
20799 000031B8 B508          mov     ch,8          ; set category code for (old) FAT disk drive
20800                          ; (except FAT32)
20801 000031BA EBE9          jmp     short Set_Media_ID_1 ; and try again
20802
20803 Set_Media_ID_2:
20804 000031BC 08FF          OR     BH,BH          ;AN000; >32mb delete volume id
20805 000031BE 7405          JZ     short NoName          ;AN000; >32mb yes
20806
20807 ;hkn; NAME1 is in DOSDATA
20808 000031C0 BE[4B05]        MOV     SI,NAME1          ;AN000; >32mb
20809
20810 000031C3 EB03          JMP     SHORT doset          ;AN000; >32mb yes
20811 NoName:
20812
20813 ;hkn; NO_NAME_ID is in DOSDATA
20814 000031C5 BE[C10D]        MOV     SI,NO_NAME_ID          ;AN000; >32mb
20815
20816 doset:
20817 000031C8 89D7          MOV     DI,DX          ;AN000; >32mb

```



```

20818      ;add    di,6
20819 000031CA 83C706      ADD    DI,MEDIA_ID_INFO.MEDIA_Label ;AN000;;>32mb
20820
20821      ;hkn; ES & DS must point to SS
20822      ;hkn; PUSH    CS      ;AN000;;>32mb move new volume id to packet
20823      PUSH    SS      ;AN000;;>32mb move new volume id to packet
20824
20825      POP     DS      ;AN000;;>32mb
20826
20827      ;hkn; PUSH    CS      ;AN000;;>32mb
20828      PUSH    SS      ;AN000;;>32mb
20829
20830      POP     ES      ;AN000;;>32mb
20831
20832      ; 01/02/2024
20833      push    cx
20834
20835      MOV     CX,11      ;AN000;;>32mb
20836      REP     MOVSB     ;AN000;;>32mb
20837
20838      ;MOV     CX,0846H   ;AN001;;>32mb
20839      ; 01/02/2024
20840      pop     cx
20841      mov     cl,46h     ; set volume serial number (and name)
20842      ;
20843      MOV     AL,0DH     ;AN000;;>32mb
20844      XOR     BH,BH     ;AN000;;>32mb
20845      ;invoke $IOCTL     ;AN000;;>32mb set volume id
20846      call    _$IOCTL
20847      geterr:
20848      ;hkn; PUSH    CS      ;AN000;;>32mb
20849      PUSH    SS      ;AN000;;>32mb
20850
20851      POP     DS      ;AN000;;>32mb ds= dosgroup
20852
20853      POP     DI      ;AN000;;>32mb
20854      POP     ES      ;AN000;;>32mb
20855      POP     AX      ;AN000;;>32mb
20856      retn     ;AN000;;>32mb
20857
20858      ; MSDOS 6.0
20859      ;-----
20860      ; Procedure Name : Set_EXT_mode
20861      ;
20862      ; Inputs:
20863      ; [EXTOPEN_ON]= flag for extended open
20864      ; SAVE_BX= mode specified in Extended Open
20865      ; Function:
20866      ; Set mode in ThisSFT
20867      ; Outputs:
20868      ; carry set,mode is set if from Extended Open
20869      ; carry clear, mode not set yet
20870      ;-----
20871
20872      ; 13/05/2019 - Retro DOS v4.0
20873
20874      Set_EXT_mode:
20875
20876      ;hkn; SS override
20877      TEST    byte [ss:EXTOPEN_ON],EXT_OPEN_ON ;AN000;EO. from extended open
20878      JZ      short NOTEX      ;AN000;EO. no, do normal
20879      PUSH    AX      ;AN000;EO.
20880
20881      ;hkn; SS override
20882      MOV     AX,[ss:SAVE_BX]      ;AN000;EO.
20883      ;or     [es:di+2],ax
20884      OR      [ES:DI+SF_ENTRY.sf_mode],AX ;AN000;EO.
20885      POP     AX      ;AN000;EO.
20886      STC      ;AN000;EO.
20887      NOTEX:
20888      retn     ;AN000;EO.
20889
20890      ;=====
20891      ; OPEN.ASM, MSDOS 6.0, 1991
20892      ;=====
20893      ; 08/08/2018 - Retro DOS v3.0
20894      ; 18/05/2019 - Retro DOS v4.0
20895
20896      ; TITLE  DOS_OPEN - Internal OPEN call for MS-DOS
20897      ; NAME   DOS_OPEN
20898
20899      ;** OPEN.ASM - File Open
20900      ;-----
20901      ; Low level routines for opening a file from a file spec.
20902      ; Also misc routines for sharing errors
20903      ;
20904      ; DOS_Open
20905      ; Check_Access_AX
20906      ; SHARE_ERROR
20907      ; SET_SFT_MODE
20908      ; Code_Page_Mismatched_Error      ; DOS 4.00
20909      ;
20910      ; Revision history:
20911      ;
20912      ; Created: ARR 30 March 1983
20913      ; A000 version 4.00 Jan. 1988
20914      ;
20915      ; M034 - The value in save_bx must be pushed on to the stack for
20916      ; remote extended opens and not save_cx.
20917      ;
20918      ; M035 - if open made from exec then we must set the appropriate bits
20919      ; on the stack before calling off to the redir.
20920      ; M042 - Bit 11 of DOS34_FLAG set indicates that the redir knows how
20921      ; to handle open from exec. In this case set the appropriate bit
20922      ; else do not.
20923      ;-----
20924
20925      ;Installed = TRUE
20926
20927      ; i_need NoSetDir,BYTE
20928      ; i_need THISSFT,DWORD
20929      ; i_need THISCDS,DWORD
20930      ; i_need CURBUF,DWORD
20931      ; i_need CurrentPDB,WORD
20932      ; i_need CURR_DIR_END,WORD
20933      ; I_need RetryCount,WORD
20934      ; I_need Open_Access,BYTE
20935      ; I_need fSharing,BYTE
20936      ; i_need JShare,DWORD
20937      ; I_need FastOpenFlg,byte
20938      ; I_need EXTOPEN_ON,BYTE      ;AN000;; DOS 4.00
20939      ; I_need ALLOWED,BYTE      ;AN000;; DOS 4.00
20940      ; I_need EXTERR,WORD      ;AN000;; DOS 4.00
20941      ; I_need EXTERR_LOCUS,BYTE      ;AN000;; DOS 4.00

```

```

20942 ; I_need EXTERR_ACTION,BYTE ;AN000;; DOS 4.00
20943 ; I_need EXTERR_CLASS,BYTE ;AN000;; DOS 4.00
20944 ; I_need CPSWFLAG,BYTE ;AN000;; DOS 4.00
20945 ; I_need EXITHOLD,DWORD ;AN000;; DOS 4.00
20946 ; I_need THISDPB,DWORD ;AN000;; DOS 4.00
20947 ; I_need SAVE_CX,WORD ;AN000;; DOS 4.00
20948 ; I_need SAVE_BX,WORD ;M034
20949 ;
20950 ; I_need DOS_FLAG,BYTE
20951 ; I_need DOS34_FLAG,WORD ;M042
20952 ;
20953 ;Break <DOS_Open - internal file access>
20954 ;-----
20955 ; Procedure Name : DOS_Open
20956 ;
20957 ; Inputs:
20958 ; [WFP_START] Points to WFP string ("d:/" must be first 3 chars, NUL
20959 ; terminated)
20960 ; [CURR_DIR_END] Points to end of Current dir part of string
20961 ; (= -1 if current dir not involved, else
20962 ; points to first char after last "/" of current dir part)
20963 ; [THISCDs] Points to CDS being used
20964 ; (Low word = -1 if NUL CDS (Net direct request))
20965 ; [THISSFT] Points to SFT to fill in if file found
20966 ; (sf_mode field set so that FCB may be detected)
20967 ; [SATTRIB] Is attribute of search, determines what files can be found
20968 ; AX is Access and Sharing mode
20969 ; High NIBBLE of AL (Sharing Mode)
20970 ; sharing_compat file is opened in compatibility mode
20971 ; sharing_deny_none file is opened Multi reader, Multi writer
20972 ; sharing_deny_read file is opened Only reader, Multi writer
20973 ; sharing_deny_write file is opened Multi reader, Only writer
20974 ; sharing_deny_both file is opened Only reader, Only writer
20975 ; Low NIBBLE of AL (Access Mode)
20976 ; open_for_read file is opened for reading
20977 ; open_for_write file is opened for writing
20978 ; open_for_both file is opened for both reading and writing.
20979 ;
20980 ; For FCB SFTs AL should = sharing_compat + open_for_both
20981 ; (not checked)
20982 ;
20983 ; Function:
20984 ; Try to open the specified file
20985 ;
20986 ; Outputs:
20987 ; sf_ref_count is NOT altered
20988 ; CARRY CLEAR
20989 ; THISSFT filled in.
20990 ; CARRY SET
20991 ; AX is error code
20992 ; error_file_not_found
20993 ; Last element of path not found
20994 ; error_path_not_found
20995 ; Bad path (not in curr dir part if present)
20996 ; error_bad_curr_dir
20997 ; Bad path in current directory part of path
20998 ; error_invalid_access
20999 ; Bad sharing mode or bad access mode or bad combination
21000 ; error_access_denied
21001 ; Attempt to open read only file for writting, or
21002 ; open a directory
21003 ; error_sharing_violation
21004 ; The sharing mode was correct but not allowed
21005 ; generates an INT 24 on compatibility mode SFTs
21006 ; DS preserved, others destroyed
21007 ;-----
21008 ; 18/05/2019 - Retro DOS v4.0
21009 ; DOSCODE:6A60h (MSDOS 6.21, MSDOS.SYS)
21010 ; 14/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
21011 ; DOSCODE:6A4Ch (MSDOS 5.0, MSDOS.SYS)
21012 ;
21013 ; 01/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
21014 ; DOSCODE:71BBh (PCDOS 7.1, IBMDOS.COM)
21015 ;
21016 DOS_OPEN:
21017 ; DS has been set up to DOSDATA in file.asm and fcbio2.asm.
21018 MOV byte [NoSetDir],0
21019 CALL Check_Access_AX
21020 JC short do_ret_label ; retc
21021 ;
21022 LES DI,[THISSFT]
21023 XOR AH,AH
21024 ;
21025 ; sleaze! move only access/sharing mode in. Leave sf_isFCB unchanged
21026 ;
21027 MOV [ES:DI+SF_ENTRY.sf_mode],AL ; For moment do this on FCBS too
21028 PUSH ES
21029 LES SI,[THISCDs]
21030 ; 18/08/2018
21031 CMP SI,-1 ; 0FFFFh
21032 JNZ short TEST_RE_NET1
21033 POP ES
21034 ;
21035 ; MSDOS 6.0
21036 ;Extended open hooks
21037 TEST byte [EXTOPEN_ON],EXT_OPEN_ON ;FT. from extended open ;AN000;
21038 JZ short _NOEXTOP ;FT. no, do normal ;AN000;
21039 ;_IFS_extopen: ;AN000;
21040 MOV AL,[SAVE_BX] ; M034 - save_bx has original bx
21041 ; with which call was made. This
21042 ; has the open access bits.
21043 ; M034 - FT. al= create attribute
21044 ;;MOV AL,[SAVE_CX]
21045 PUSH AX ;FT. pass create attr to IFS ;AN000;
21046 ;mov ax,112Eh
21047 ;MOV AX,(MULTNET SHL 8) OR 46 ;FT. issue extended open verb ;AN000;
21048 mov ax,(MULTNET*256)+46
21049 INT 2FH ;FT. ;AN000;
21050 POP BX ;FT. trash bx ;AN000;
21051 MOV byte [EXTOPEN_ON],0 ;FT. ;AN000;
21052 ;
21053 do_ret_label:
21054 retn ;FT. ;AN000;
21055 ;_NOEXTOP:
21056 ;Extended open hooks
21057 ;
21058 ;IF NOT Installed
21059 ;transfer NET_SEQ_OPEN
21060 ;ELSE
21061 ;
21062 do_net_int2f:
21063 test byte [DOS_FLAG],EXECOPEN ; Q: was this open call made from exec
21064 jz short not_exec_open ; N: just do net open
21065 ; Y: check to see if redir is aware

```

```

21066                                     ; of this
21067
21068                                     ; M042 - start
21069 ;test word [DOS34_FLAG],EXEC_AWARE_REDIR ; 800h
21070 00003238 F606[1206]08 test byte [DOS34_FLAG+1],(EXEC_AWARE_REDIR>>8)
21071                                     ; Q: does this redir know how to
21072                                     ; this
21073 0000323D 7402 jz short not_exec_open ; N: just do net open
21074                                     ; Y: set bit 3 of access byte and
21075                                     ; set sharing mode to DENY_WRITE
21076                                     ; M042 - end
21077
21078 ; NOTE: This specific mode has not been set for the code assembled
21079 ; under the "NOT Installed" conditional. Currently Installed is
21080 ; always one.
21081                                     ; M035 - set the bits on the stack
21082 ;mov al,23h
21083 0000323F B023 mov AL,SHARING_DENY_WRITE+EXEC_OPEN
21084
21085 not_exec_open:
21086 ; MSDOS 3.3 & MSDOS 6.0
21087 00003241 50 PUSH AX
21088
21089 ;MOV AX,(MULTINET SHL 8) OR 22
21090 ;INT 2FH
21091
21092 00003242 B81611 mov ax,1116h
21093 00003245 CD2F int 2Fh ; Multiplex - NETWORK REDIRECTOR - OPEN EXISTING REMOTE FILE
21094                                     ; ES:DI -> uninitialized SFT, SS = DOS CS
21095                                     ; SDA first filename pointer -> fully-qualified name of file to open
21096                                     ; STACK: WORD file open mode
21097                                     ; Return: CF set on error
21098
21099 00003247 5B POP BX ; clean stack
21100 ;do_ret_label: ; 09/08/2018
21101 00003248 C3 retn
21102 ;ENDIF
21103
21104 TEST_RE_NET1:
21105 ;TEST word [ES:SI+curdir.flags],curdir_isnet
21106 ; 17/12/2022
21107 00003249 26F6444480 test byte [ES:SI+curdir.flags+1],curdir_isnet>>8
21108 0000324E 07 POP ES
21109 ; 18/05/2019
21110 0000324F 7409 JZ short LOCAL_OPEN
21111
21112 ;Extended open hooks
21113 ; MSDOS 6.0
21114 00003251 F606[F605]01 TEST byte [EXTOPEN_ON],EXT_OPEN_ON ;FT. from extended open ;AN000;
21115 00003256 75C9 JNZ short _IFS_extopen ;FT. issue extended open ;AN000;
21116 ;Extended open hooks
21117
21118 ;IF NOT Installed
21119 ; transfer NET_OPEN
21120 ;ELSE
21121 00003258 EBD7 jmp short do_net_int2f
21122 ;ENDIF
21123
21124 LOCAL_OPEN:
21125 ; MSDOS 3.3 & MSDOS 6.0
21126 0000325A E885E6 call ECritDisk
21127
21128 ; DOS 3.3 FastOpen 6/16/86
21129
21130 ;or byte [FastOpenFlg],5
21131 0000325D 800E[4611]05 OR byte [FastOpenFlg],FastOpen_Set+Special_Fill_Set ; only open can
21132
21133 call GETPATH
21134
21135 ; DOS 3.3 FastOpen 6/16/86
21136
21137 JNC short open_found
21138 JNZ short bad_path2
21139 OR CL,CL
21140 JZ short bad_path2
21141
21142 0000326D B80200 OpenFNF: MOV AX,error_file_not_found ; 2
21143 OpenBadRet:
21144 ;hkn; FastOpenFlg is in DOSDATA use SS override
21145 ; 12/08/2018
21146 ;mov byte [cs:FastOpenFlg],0 ; IBMDOS.COM (MSDOS 3.3) offset 36CAh
21147 ; MSDOS 6.0
21148 00003270 368026[4611]80 AND BYTE [SS:FastOpenFlg],Fast_yes ;; DOS 3.3
21149 00003276 F9 STC
21150 ;call LCritDisk
21151 ; 16/12/2022
21152 00003277 E995E6 jmp LCritDisk
21153 ;JMP Clear_FastOpen ; 10/08/2018
21154 ;retn ; 08/09/2018
21155 ; 14/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
21156 ;jmp Clear_FastOpen
21157
21158 bad_path2:
21159 0000327A B80300 MOV AX,error_path_not_found ; 3
21160 0000327D EBF1 JMP short OpenBadRet
21161
21162 Open_Bad_Access:
21163 0000327F B80500 MOV AX,error_access_denied; 5
21164 00003282 EBEC JMP short OpenBadRet
21165
21166 Open_found:
21167 00003284 74F9 JZ short Open_Bad_Access ; test for directories
21168 00003286 08E4 OR AH,AH
21169 00003288 783E JS short open_ok ; Devices don't have attributes
21170 0000328A 8E06[E405] MOV ES,[CURBUF+2] ; get buffer location
21171 ;mov al,[es:bx+0Bh]
21172 0000328E 268A470B MOV AL,[ES:BX+dir_entry.dir_attr]
21173 00003292 A808 TEST AL,attr_volume_id ; can't open volume ids
21174 00003294 75E9 JNZ short Open_Bad_Access
21175 00003296 A801 TEST AL,attr_read_only ; check write on read only
21176 00003298 742E JZ short open_ok
21177
21178 ; The file is marked READ-ONLY. We verify that the open mode allows access to
21179 ; the read-only file. Unfortunately, with FCB's and net-FCB's we cannot
21180 ; determine at the OPEN time if such access is allowed. Thus, we defer such
21181 ; processing until the actual write operation:
21182 ;
21183 ; If FCB, then we change the mode to be read_only.
21184 ; If net_FCB, then we change the mode to be read_only.
21185 ; If not open for read then error.
21186
21187 0000329A 1E push ds
21188 0000329B 56 push si
21189 0000329C C536[9E05] LDS SI,[THISSFT]

```

```

21190             ;mov     cx,[si+2]
21191 000032A0 8B4C02 MOV     CX,[SI+SF_ENTRY.sf_mode]
21192             ; 17/12/2022
21193             ;test    ch,80h
21194 000032A3 F6C580 test    ch,sf_isFCB>>8
21195             ;TEST    CX,sf_isFCB ; 8000h ; is it FCB?
21196 000032A6 750A JNZ     short ResetAccess ; yes, reset the access
21197 000032A8 88CA MOV     DL,CL
21198 000032AA 80E2F0 AND     DL,SHARING_MASK ; 0F0h
21199 000032AD 80FA70 CMP     DL,SHARING_NET_FCB ; 70h ; is it net FCB?
21200 000032B0 7508 JNZ     short NormalOpen ; no
21201 ResetAccess:
21202             ; 14/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
21203             ;AND     CX,~access_mask ; 0FFF0h ; clear access
21204             ; 16/12/2022
21205             ;and     cl,0F0h ; 18/05/2019
21206             ;;;
21207             ; 01/02/2024 - Retro DOS v5.0
21208             ; (PCDOS 7.1 IBMDOS.COM)
21209             ;and     cx,0FFFCh
21210 000032B2 80E1FC and     cl,0FCh ; ~3 ; ~open_mode_mask ; clear access
21211             ;;;
21212             ; OR     CX,open_for_read ; 0 ; stick in open_for_read
21213 000032B5 894C02 MOV     [SI+SF_ENTRY.sf_mode],CX
21214 000032B8 EB0C JMP     SHORT FillSFT
21215
21216             ; The SFT is normal. See if the requested access is open_for_read
21217
21218 NormalOpen:
21219             ;AND     CL,access_mask ; 0Fh ; remove extras
21220             ;;;
21221             ; 01/02/2024 - Retro DOS v5.0
21222             ; (PCDOS 7.1 IBMDOS.COM)
21223 000032BA 80E103 and     cl,3 ; it was 'and cl,0Fh' in MSDOS 6.22
21224             ; (and cl,access_mask)
21225             ; this may be open_mode_mask
21226             ;;;
21227 000032BD 80F900 CMP     CL,open_for_read ; 0 ; is it open for read?
21228 000032C0 7404 JZ      short FillSFT ; yes
21229 000032C2 5E POP     SI
21230 000032C3 1F POP     DS
21231 000032C4 EBB9 JMP     short Open_Bad_Access
21232
21233             ; All done, restore registers and fill the SFT.
21234             ;
21235 FillSFT:
21236 000032C6 5E POP     SI
21237 000032C7 1F POP     DS
21238
21239 000032C8 E83823 open_ok:
21240             call     DOOPEN ; Fill in SFT
21241
21242 ;hkn; FastOpenFlg is in DOSDATA. use SS override
21243             ; 18/05/2019
21244 000032CB 368026[4611]80 and     byte [ss:FastOpenFlg],80h
21245             AND     BYTE [SS:FastOpenFlg],Fast_yes ; DOS 3.3
21246             ; 12/08/2018
21247             ;and     byte [FastOpenFlg],Fast_yes
21248
21249             ; MSDOS 6.0
21250 000032D1 E84300 CALL    DO_SHARE_CHECK
21251 000032D4 7303 JNC     short SHARE_OK
21252             ;call    LCritDisk
21253 000032D6 E936E6 jmp     LCritDisk
21254             ;JMP     short Clear_FastOpen
21255             ;retn ; 18/05/2019
21256             ; 14/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
21257             ;jmp     short Clear_FastOpen
21258
21259             ; MSDOS 3.3
21260 DO_SHARE_CHECK:
21261             ; MOV     CX,[RetryCount] ; Get # tries to do
21262             ; OpenShareRetry:
21263             ; push     CX ; Save number left to do
21264             ; call     SHARE_CHECK ; Final Check
21265             ; pop      CX ; CX = # left
21266             ; JNC     short SHARE_OK ; No problem with access
21267             ; call     Idle
21268             ; LOOP    OpenShareRetry ; One more retry used up
21269             ; OpenShareFail:
21270             ; LES     DI,[THISSFT]
21271             ; call     SHARE_ERROR
21272             ; JNC     short DO_SHARE_CHECK ; User wants more retry
21273
21274             ;12/08/2018
21275             ;mov     byte [ss:FastOpenFlg],0
21276             ;08/09/2018
21277             ;mov     byte [FastOpenFlg],0
21278             ;call     LCritDisk
21279             ;JMP     short Clear_FastOpen
21280             ;retn
21281
21282 SHARE_OK:
21283             ; MSDOS 3.3 & MSDOS 6.0
21284 000032D9 B80300 MOV     AX,3
21285 000032DC C43E[9E05] LES     DI,[THISSFT]
21286             ;if installed
21287             ;call     JShare + 14 * 4
21288 000032E0 FF1E[C800] call     far [JShare+(14*4)] ; 14 = ShSU
21289             ;else
21290             ; call     ShSU
21291             ;endif
21292 000032E4 E828E6 call     LCritDisk
21293
21294             ;FallThru Set_SFT_Mode
21295
21296             ;-----
21297             ; Procedure Name : SET_SFT_MODE
21298             ;
21299             ; Finish SFT initialization for new reference. Set the correct mode.
21300             ;
21301             ; Inputs:
21302             ; ThisSFT points to SFT
21303             ;
21304             ; Outputs:
21305             ; Carry clear
21306             ; Registers modified: AX.
21307             ;-----
21308
21309             ;hkn; called from create. DS already set up to DOSDATA.
21310
21311 SET_SFT_MODE:
21312 000032E7 C43E[9E05] LES     DI,[THISSFT]
21313 000032EB E8261F call     DEV_OPEN_SFT

```

```

21314             ;test word [es:di+2],8000h
21315             ; 17/12/2022
21316             ;test byte [es:di+3],80h
21317 000032EE 26F6450380 test byte [ES:DI+SF_ENTRY.sf_mode+1],sf_isFCB>>8
21318             ;TEST word [ES:DI+SF_ENTRY.sf_mode],sf_isFCB ; Clears carry
21319 000032F3 7407 JZ short Clear_FastOpen ; sf_mode correct (retz)
21320 000032F5 A1[3003] MOV AX,[CurrentPDB]
21321             ;mov [es:di+31h],ax
21322 000032F8 26894531 MOV [ES:DI+SF_ENTRY.sf_PID],AX ; For FCB sf_PID=PDB
21323
21324 Clear_FastOpen:
21325 000032FC C3 retn ;;;; DOS 3.3
21326
21327 ;-----
21328 ; Procedure Name : SHARE_ERROR
21329 ;
21330 ; Called on sharing violations. ES:DI points to SFT. AX has error code
21331 ; If SFT is FCB or compatibility mode gens INT 24 error.
21332 ; Returns carry set AX=error_sharing_violation if user says ignore (can't
21333 ; really ignore). Carry clear if user wants a retry. ES, DI, DS preserved
21334 ;-----
21335
21336 SHARE_ERROR:
21337 ; 17/12/2022
21338             ;test byte [es:di+3],80h
21339 000032FD 26F6450380 test byte [ES:DI+SF_ENTRY.sf_mode+1],sf_isFCB>>8 ; 80h
21340             ;TEST word [ES:DI+SF_ENTRY.sf_mode],sf_isFCB ; 8000h
21341 00003302 7509 JNZ short _HARD_ERR
21342 00003304 268A4D02 MOV CL,[ES:DI+SF_ENTRY.sf_mode]
21343 00003308 80E1F0 AND CL,SHARING_MASK ; 0F0h
21344             ;CMP CL,SHARING_COMPAT ; 0
21345             ;JNE short _NO_HARD_ERR
21346             ; 21/09/2023
21347 0000330B 7505 jnz short _NO_HARD_ERR
21348 _HARD_ERR:
21349 0000330D E83C51 call SHARE_VIOLATION
21350             ;retnc ; User wants retry
21351 00003310 73EA jnc short Clear_FastOpen
21352 _NO_HARD_ERR:
21353 00003312 B82000 MOV AX,error_sharing_violation ; 20h
21354 00003315 F9 STC
21355 00003316 C3 retn
21356
21357 ; MSDOS 6.0
21358 ;-----
21359 ; Procedure Name : DO_SHARE_CHECK
21360 ;
21361 ; Input: THISDPB, WFP_Start, THISSFT set
21362 ; Functions: check file sharing mode is valid
21363 ; Output: carry set, error
21364 ; carry clear, share ok
21365 ;-----
21366
21367 ; 18/05/2019 - Retro DOS v4.0
21368 DO_SHARE_CHECK:
21369 00003317 E8C8E5 call ECritDisk ; enter critical section
21370 OPN_RETRY:
21371 0000331A 8B0E[1A00] MOV CX,[RetryCount] ; Get # tries to do
21372 OpenShareRetry:
21373 0000331E 51 push CX ; Save number left to do
21374 0000331F E82551 call SHARE_CHECK ; Final Check
21375 00003322 59 pop CX ; CX = # left
21376 00003323 730E JNC short Share_Ok2 ; No problem with access
21377 00003325 E8BCE4 call Idle
21378 00003328 E2F4 LOOP OpenShareRetry ; One more retry used up
21379 OpenShareFail:
21380 0000332A C43E[9E05] LES DI,[THISSFT]
21381 0000332E E8CCFF call SHARE_ERROR
21382 00003331 73E7 JNC short OPN_RETRY ; User wants more retry
21383 Share_Ok2:
21384             ;call LCritDisk ; leave critical section
21385             ;retn
21386             ; 18/12/2022
21387 00003333 E9D9E5 jmp LCritDisk
21388
21389 ;-----
21390 ; Procedure Name : Check_Access
21391 ;
21392 ; Inputs:
21393 ; AX is mode
21394 ; High NIBBLE of AL (Sharing Mode)
21395 ; sharing_compat file is opened in compatibility mode
21396 ; sharing_deny_none file is opened Multi reader, Multi writer
21397 ; sharing_deny_read file is opened Only reader, Multi writer
21398 ; sharing_deny_write file is opened Multi reader, Only writer
21399 ; sharing_deny_both file is opened Only reader, Only writer
21400 ; Low NIBBLE of AL (Access Mode)
21401 ; open_for_read file is opened for reading
21402 ; open_for_write file is opened for writing
21403 ; open_for_both file is opened for both reading and writing.
21404 ; Function:
21405 ; Check this access mode for correctness
21406 ; Outputs:
21407 ; [open_access] = AL input
21408 ; Carry Clear
21409 ; Mode is correct
21410 ; AX unchanged
21411 ; Carry Set
21412 ; Mode is bad
21413 ; AX = error_invalid_access
21414 ; No other registers effected
21415 ;-----
21416
21417 ; 23/01/2024 - Retro DOS v5.0
21418 Check_Access_AX:
21419 00003336 A2[6E05] MOV [OPEN_ACCESS],AL
21420 00003339 53 PUSH BX
21421
21422 ; If sharing, then test for special sharing mode for FCBs
21423
21424 0000333A 88C3 MOV BL,AL
21425 0000333C 80E3F0 AND BL,SHARING_MASK ; 0F0h
21426
21427             ;CMP byte [FSHARING],-1
21428             ;JNZ short CheckShareMode ; not through server call, must be ok
21429             ; 23/01/2024
21430             ; PCDOS 7.1 IBMDOS.COM
21431 0000333F 803E[7205]00 cmp byte [FSHARING],0
21432 00003344 7405 jz short CheckShareMode ; not through server call, must be ok
21433
21434 00003346 80FB70 CMP BL,SHARING_NET_FCB ; 70h
21435 00003349 7405 JZ short CheckAccessMode ; yes, we have an FCB
21436 CheckShareMode:
21437 0000334B 80FB40 CMP BL,40h ; is this a good sharing mode?

```

```

21438 0000334E 770D      JA      short Make_Bad_Access
21439                      CheckAccessMode:
21440 00003350 88C3      MOV     BL,AL
21441                      ; 23/01/2024
21442 00003352 80E303    and     b1,3 ; PCDOS 7.1 IBMDOS.COM
21443                      ;AND    BL,access_mask ; 0Fh
21444 00003355 80FB02    CMP     BL,2
21445 00003358 7703      JA      short Make_Bad_Access
21446 0000335A 5B        POP     BX
21447 0000335B F8        CLC
21448 0000335C C3        retn
21449
21450                      Make_Bad_Access:
21451 0000335D B80C00    MOV     AX,error_invalid_access ; 0Ch
21452 00003360 5B        POP     BX
21453 00003361 F9        STC
21454 00003362 C3        retn
21455
21456                      ;=====
21457                      ; DINFO.ASM, MSDOS 6.0, 1991
21458                      ;=====
21459                      ; 08/08/2018 - Retro DOS v3.0
21460                      ; 18/05/2019 - Retro DOS v4.0
21461                      ; 02/02/2024 - Retro DOS v5.0
21462
21463                      ;** Low level routine for returning disk drive information from a local
21464                      ; or NET device
21465                      ;
21466                      ; DISK_INFO
21467                      ;
21468                      ; Modification history:
21469                      ;
21470                      ; Created: ARR 30 March 1983
21471                      ;
21472                      ; Break <DISK_INFO -- Get Disk Drive Information>
21473                      ;-----
21474                      ; Procedure Name : DISK_INFO
21475                      ;
21476                      ; Inputs:
21477                      ; [THISCD5] Points to the Macro List Structure of interest
21478                      ; (It MAY NOT be NUL, error not detected)
21479                      ; Function:
21480                      ; Get Interesting Drive Information
21481                      ; Returns:
21482                      ; DX = Number of free allocation units
21483                      ; BX = Total Number of allocation units on disk
21484                      ; CX = Sector size
21485                      ; AL = Sectors per allocation unit
21486                      ; AH = FAT ID BYTE
21487                      ; Carry set if error (currently user FAILED to I 24)
21488                      ; Segs except ES preserved, others destroyed
21489                      ;-----
21490
21491                      ;hkn; called from getset.asm and misc.asm. DS has already been set up to
21492                      ;hkn; DOSDATA.
21493
21494                      ; 02/02/2024 - Retro DOS v5.0
21495                      ; PCDOS 7.1 IBMDOS.COM - DOSCODE:732Bh
21496
21497                      DISK_INFO:
21498                      ; 08/08/2018 - Retro DOS v3.0
21499                      ; IBM DOS.COM (MSDOS 3.3, 1987) - Offset 37C5h
21500
21501 00003363 E8C3E4    call    TestNet
21502 00003366 7318      JNC     short LOCAL_DSK_INFO
21503
21504                      ;;;
21505                      ; 02/02/2024 (PCDOS 7.1 IBMDOS.COM)
21506 00003368 31F6      xor     si,si ; free cluster count hw = 0
21507 0000336A 31FF      xor     di,di ; number of clusters hw = 0
21508                      ;;;
21509
21510                      ;IF NOT Installed
21511                      ; transfer NET_DISK_INFO
21512                      ;ELSE
21513                      ;MOV     AX,(MULTNET SHL 8) OR 12
21514                      ;INT     2FH
21515                      ;return
21516
21517 0000336C B80C11    mov     ax,110Ch
21518 0000336F CD2F      int     2Fh ; Multiplex - NETWORK REDIRECTOR - GET DISK SPACE
21519                      ; ES:DI -> current directory
21520                      ; Return: AL = sectors per cluster, BX = total clusters
21521                      ; CX = bytes per sector, DX = number of available clusters
21522                      ;;;
21523                      ; 02/02/2024 (PCDOS 7.1 IBMDOS.COM)
21524 00003371 83FBFF    cmp     bx,0FFFFh
21525 00003374 7502      jne     short dsk_info_1
21526 00003376 89DF      mov     di,bx
21527                      dsk_info_1:
21528                      cmp     dx,0FFFFh
21529 0000337B 7502      jne     short disk_info_retn
21530 0000337D 89D6      mov     si,dx
21531                      disk_info_retn:
21532                      ;;;
21533 0000337F C3        retn
21534                      ;ENDIF
21535
21536                      LOCAL_DSK_INFO:
21537                      ;MOV     byte [EXTERR_LOCUS],errLOC_Disk
21538                      ;;;
21539                      ; 02/02/2024 (PCDOS 7.1 IBMDOS.COM)
21540 00003380 E863DF    call    set_exerr_locus_disk
21541                      ;;;
21542 00003383 E85CE5    call    ECritDisk
21543 00003386 E8D231    call    FATREAD_CDS ; perform media check.
21544                      ;JC     short CRIT_LEAVE
21545                      ;;; 02/02/2024
21546 00003389 720D      jc     short dsk_info_2
21547 0000338B 31C0      xor     ax,ax
21548 0000338D A3[E80A]    mov     [CLUSTNUM_HW],ax ; 0 ; clear high word of cluster number
21549                      ;;;
21550 00003390 B80200    MOV     BX,2
21551 00003393 E84B2F    call    UNPACK ; Get first FAT sector into CURBUF
21552                      ;JC     short CRIT_LEAVE
21553                      ;;;
21554                      ; 02/02/2024 - Retro DOS v5.0
21555 00003396 7303      jnc     short dsk_info_3
21556                      dsk_info_2:
21557                      ;jmp     CRIT_LEAVE
21558 00003398 E974E5    jmp     LCritDisk
21559                      dsk_info_3:
21560                      ;;;
21561 0000339B C536[E205]    LDS     SI,[CURBUF]

```



```

21562             ; 02/02/2024
21563             ;mov ah,[si+20]
21564             ;mov ah,[si+24] ; PCDOS 7.1 IBMDOS.COM
21565 0000339F 8A6418 MOV AH,[SI+BUFINSZ] ; get FAT ID BYTE
21566
21567 ;hkn; SS is DOSDATA
21568 000033A2 16 push ss
21569 000033A3 1F pop ds
21570
21571             ; 02/02/2024 - Retro DOS v5.0
21572 %if 0
21573             ;mov cx,[es:bp+0Dh]
21574 MOV CX,[ES:BP+DPB.MAX_CLUSTER]
21575
21576             ; Examine the current free count. If it indicates that we have an invalid
21577             ; count, do the expensive calculation.
21578
21579             ;mov dx,[es:bp+1Fh]
21580 MOV DX,[ES:BP+DPB.FREE_CNT] ; get free count
21581 CMP DX,-1 ; is it valid?
21582 JZ short DoScan
21583
21584             ; Check to see if it is in a reasonable range. If so, trust it and return.
21585             ; Otherwise, we need to blast out an internal error message and then recompute
21586             ; the count.
21587
21588 CMP DX,CX ; is it in a reasonable range?
21589 JB short GotVal ; yes, trust it.
21590 DoScan:
21591 XOR DX,DX
21592 DEC CX
21593 SCANFREE:
21594 call UNPACK
21595 JC short CRIT_LEAVE
21596 JNZ short NOTFREECLUS
21597 INC DX ; A free one
21598 NOTFREECLUS:
21599 INC BX ; Next cluster
21600 LOOP SCANFREE
21601 DEC BX ; BX was next cluster. Convert to
21602 ReturnVals:
21603 DEC BX ; count
21604 ;mov al,[es:bp+4]
21605 MOV AL,[ES:BP+DPB.CLUSTER_MASK]
21606 INC AL ; Sectors/cluster
21607 ;mov cx,[es:bp+2]
21608 MOV CX,[ES:BP+DPB.SECTOR_SIZE] ; Bytes/sector
21609 ;mov [es:bp+1Fh],dx
21610 MOV [ES:BP+DPB.FREE_CNT],DX
21611 CLC
21612 CRIT_LEAVE:
21613 ;call LCritDisk
21614 ;retn
21615 ; 17/12/2022
21616 jmp LCritDisk
21617
21618             ; We have correctly computed everything previously. Load up registers for
21619             ; return.
21620
21621 GotVal:
21622 MOV BX,CX ; get cluster count
21623 JMP short ReturnVals
21624
21625 %else
21626             ; 02/02/2024 (PCDOS 7.1 IBMDOS.COM)
21627 000033A4 31F6 xor si,si ; 0
21628 000033A6 89F7 mov di,si ; 0
21629             ;mov dx,[es:bp+1Fh]
21630 000033A8 268B561F mov dx,[es:bp+DPB.FREE_CNT] ; get free count
21631             ;cmp [es:bp+0Fh],si
21632 000033AC 2639760F cmp [es:bp+DPB.FAT_SIZE],si ; FAT32 (16 bit FAT size = 0) ?
21633 000033B0 7406 jz short dsk_info_4 ; yes
21634             ;mov cx,[es:bp+0Dh]
21635 000033B2 268B4E0D mov cx,[es:bp+DPB.MAX_CLUSTER]
21636 000033B6 EB18 jmp short dsk_info_5 ; zf=0, si=di=0
21637
21638 dsk_info_4:
21639             ;mov di,[es:bp+2Fh]
21640 000033B8 268B7E2F mov di,[es:bp+DPB.LAST_CLUSTER+2]
21641             ;mov cx,[es:bp+2Dh]
21642 000033BC 268B4E2D mov cx,[es:bp+DPB.LAST_CLUSTER]
21643             ;mov si,[es:bp+21h]
21644 000033C0 268B7621 mov si,[es:bp+DPB.FREE_CNT_HW] ; hw of free cluster count
21645 000033C4 39F2 cmp dx,si ; same (zero) ?
21646
21647             ;dsk_info_5:
21648 000033C6 7504 jnz short dsk_info_6 ; not same (not zero)
21649 000033C8 42 inc dx
21650             ;jz short dsk_info_8 ; 0FFFFh -> 0 (free count is invalid/initial)
21651             ; free count calculation is needed
21652             dec dx
21653 dsk_info_6:
21654 000033CC 39FE cmp si,di ; same hw ?
21655 000033CE 7502 jne short dsk_info_7 ; no
21656 dsk_info_5: ; 02/02/2024 - Retro DOS v5.0
21657 000033D0 39CA cmp dx,cx ; same lw ?
21658 dsk_info_7:
21659 000033D2 7268 jb short GotVal ; free cluster count < last cluster number
21660 000033D4 31D2 xor dx,dx ; 0
21661 dsk_info_8:
21662 000033D6 31F6 xor si,si ; 0
21663 000033D8 83E901 sub cx,1 ; last cluster number - 1 = number of clusters
21664 000033DB 19F7 sbb di,si
21665             ;or byte [es:bp+18h],1
21666             ;or byte [es:bp+DPB.FIRST_ACCESS],1 ; set first access bit 0
21667             ; (Update flag for FSINFO sector)
21668 SCANFREE:
21669 000033E2 56 push si
21670 000033E3 FF36[EC0A] push word [CCONTENT_HW]
21671 000033E7 57 push di
21672 000033E8 E8F62E call UNPACK
21673 000033EB 5F pop di
21674 000033EC 8F06[EC0A] pop word [CCONTENT_HW]
21675 000033F0 5E pop si
21676 000033F1 7246 jc short CRIT_LEAVE
21677 000033F3 7504 jnz short NOTFREECLUS
21678 000033F5 42 inc dx ; a free one
21679 000033F6 7501 jnz short NOTFREECLUS
21680 000033F8 46 inc si ; increase hw of free cluster count
21681
21682 NOTFREECLUS:
21683 000033F9 43 inc bx ; next cluster
21684 000033FA 7504 jnz short NOTFREECLUS2
21685 000033FC FF06[E80A] inc word [CLUSTNUM_HW] ; increase hw of (next) cluster number

```

```

21686 NOTFREECLUS2:
21687     sub     cx,1                ; decrease remain cluster count for calculation
21688     sbb     di,0
21689     jnz     short SCANFREE
21690     jcxz    NOTFREECLUS3       ; calculation completed
21691     jmp     short SCANFREE
21692
21693 NOTFREECLUS3:
21694     mov     di,[CLUSTNUM_HW]
21695     sub     bx,1
21696     sbb     di,0                ; di:bx = last cluster number
21697
21698 ReturnVals:
21699     xor     cx,cx
21700     sub     bx,1
21701     sbb     di,cx                ; di:bx = number of clusters
21702     ;mov     al,[es:bp+4]
21703     mov     al,[es:bp+DPB.CLUSTER_MASK] ; spc - 1
21704     inc     al                ; sectors per cluster
21705     ;mov     [es:bp+1Fh],dx
21706     mov     [es:bp+DPB.FREE_CNT],dx ; free cluster count,1w
21707     ;cmp     [es:bp+0Fh],cx ; 0
21708     cmp     [es:bp+DPB.FAT_SIZE],cx ; 0 ; FAT32 (16 bit FAT size = 0) ?
21709     jnz     short ReturnVals2    ; no
21710     ;mov     [es:bp+21h],si
21711     mov     [es:bp+DPB.FREE_CNT_HW],si ; hw of free cluster count
21712
21713     call    update_fat32_fsinfo
21714
21715 ReturnVals2:
21716     ;mov     cx,[es:bp+2]
21717     mov     cx,[es:bp+DPB.SECTOR_SIZE] ; bytes per sector
21718     cld
21719
21720 CRIT_LEAVE:
21721     ;call    LCritDisk
21722     ;retn
21723     jmp     LCritDisk
21724
21725 GotVal:
21726     mov     bx,cx
21727     jmp     short ReturnVals
21728
21729 %endif
21730
21731 ; ===== S U B R O U T I N E =====
21732
21733 ; 03/02/2024 - Retro DOS v5.0
21734
21735 modify_spc:
21736     push    ax                ; ax = sectors per cluster
21737     push    dx
21738     mul     cx                ; bytes per sector
21739     ;cmp     dx,0
21740     and     dx,dx
21741     jnz     short mspc_1
21742     cmp     ax,16384          ; 16 kilobytes (per cluster)
21743     ;***
21744     ; actual disk size limit
21745     ; without invalidating cluster counts is
21746     ; 2 GB (512K clusters * 8 sectors per cluster)
21747     ; (128K clusters * 32 sectors per cluster)
21748
21749 mspc_1:
21750     pop     dx
21751     pop     ax
21752     jbe     short mspc_3      ; bytes per cluster <= 16 KB
21753     ;***
21754     ; bytes per cluster > 16 KB
21755     xor     di,di
21756     mov     bx,0FFFEh        ; (invalidated)
21757     or      si,si            ; hw of free cluster count
21758     jz      short mspc_2     ; si = 0
21759     mov     si,di            ; si = 0
21760     mov     dx,bx            ; dx = bx = 0FFFEh (invalidated)
21761 mspc_2:
21762     retn
21763
21764 mspc_3:
21765     shl     ax,1              ; sectors per clust = sectors per clust * 2
21766     shr     di,1              ; (ax <= 32768) -modified spc limit-
21767     ; cluster count = cluster count / 2
21768     ; di:bx = modified value of total clusters
21769
21770 mspc_4:
21771     rcr     bx,1              ; free clusters = free clusters / 2
21772     shr     si,1              ; si:dx = modified value of free clusters
21773     rcr     dx,1
21774
21775 ; -----
21776     ; 03/02/2024
21777     ; PC DOS 7.1 IBMDOS.COM - DOSCODE:7431h
21778
21779 modify_cluster_count:
21780     or      di,di            ; hw of cluster count
21781     jnz     short modify_spc
21782     retn
21783
21784 ; ===== S U B R O U T I N E =====
21785
21786 ; write FSINFO sector onto disk
21787
21788 ; 03/02/2024 - Retro DOS v5.0
21789 ; -----
21790 ; FAT32 FSInfo Sector Structure
21791 ; -----
21792 ; ref: Microsoft FAT32 File System Specification (2000)
21793
21794 struc FSINFO ; Offset ;
21795     .LeadSig: resb 4 ; 0 ; Value 0x41615252. Lead Signature.
21796     .Reserved1: resb 480 ; 4 ; Reserved. Must be 0. Never be used.
21797     .StrucSig: resb 4 ; 484 ; Value 0x61417272. Fields Signature.
21798     .Free_Count: resb 4 ; 488 ; Last known free cluster count. (*)
21799     .Next_Free: resb 4 ; 492 ; Start clus for free clus srch. (**)
21800     .Reserved2: resb 12 ; 496 ; Reserved. Must be 0. Never be used.
21801     .TrailSig: resb 4 ; 508 ; Value 0xAA550000. Trail Signature.
21802     .size:
21803 endstruc
21804
21805 ; (*) If the value is 0xFFFFFFFF, then the free count is unknown
21806 ; and must be computed.
21807 ; (**) If the value is 0xFFFFFFFF, then free cluster search must be started
21808 ; from cluster 2.
21809 ; Lead Signature, Fields (Structure) Signature and Trail Signature
21810 ; are used to validate FSInfo sector.

```



```

21810
21811 ; -----
21812 ; 03/02/2024 - Retro DOS v5.0
21813 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:7436h
21814 ; (Windows ME IO.SYS - BIOSCODE:7227h)
21815
21816
21817 update_fat32_fsinfo:
21818     push    cx
21819     push    dx
21820     xor     cx,cx
21821     ;mov     dx,[es:bp+25h]
21822     mov     dx,[es:bp+DPB.FSINFO_SECTOR]
21823     ;cmp     [es:bp+0Fh],cx
21824     cmp     [es:bp+DPB.FAT_SIZE],cx ; 0
21825     ; (16bit FAT size field = 0 for FAT32 fs)
21826     jne     short u_fat32_inf_1
21827     cmp     dx,0FFFFh ; -1
21828     jne     short u_fat32_inf_2
21829
21830 u_fat32_inf_1:
21831     pop     dx
21832     pop     cx
21833     ;and     byte [es:bp+18h],0F4h
21834     and     byte [es:bp+DPB.FIRST_ACCESS],0F4h ; clear bit 0,1 and 3
21835     ; bit 0 - FSINFO update (dirty) bit
21836     ; bit 1 - BPB_RootClus update bit
21837     ; bit 3 - BPB_ExtFlags update bit
21838     retn
21839
21840 u_fat32_inf_2:
21841     push    ax
21842     push    bx
21843     push    di
21844     push    si
21845     push    ds
21846     cmp     byte [ss:BuffInHMA],0 ; is buffers in HMA?
21847     jz      short u_fat32_inf_3 ; no
21848     lds     di,[ss:LOMemBuff] ; read it into scratch buffer
21849     ;sub     di,24
21850     sub     di,BUFINSIZ ; space for buffer header
21851     ; (buffer header size = 24)
21852     ;cld
21853     jmp     short u_fat32_inf_4
21854
21855 u_fat32_inf_3:
21856     push    es
21857     push    bp
21858     call    GETCURHEAD ; ds:di = first buffer in queue
21859     push    dx
21860     call    BUFWRITE ; BufWrite writes a buffer to the disk,
21861     ; if it's dirty.
21862     pop     dx
21863     pop     bp
21864     pop     es
21865
21866 ;u_fat32_inf_4:
21867     jc      short u_fat32_inf_5
21868 u_fat32_inf_4:
21869     xor     cx,cx
21870     ;lea     bx,[di+24]
21871     lea     bx,[di+BUFINSIZ] ; buffer data address
21872     ;mov     byte [ss:ALLOWED],18h
21873     mov     byte [ss:ALLOWED],Allowed_FAIL+Allowed_RETRY
21874     mov     [ss:HIGH_SECTOR],cx ; 0
21875     inc     cx ; cx = sector count = 1
21876     ; es:bp = DPB
21877     push    bx ; ds:bx = buffer (data) address
21878     push    dx ; HIGH_SECTOR:dx = disk sector address
21879     call    DREAD ; read fs info sector
21880     pop     dx
21881     pop     bx
21882     jc      short u_fat32_inf_5
21883
21884     ;cmp     word [bx+FSINFO.LeadSig],5252h
21885     cmp     word [bx],5252h ; 'RR'
21886     jne     short u_fat32_inf_5
21887     ;cmp     word [bx+2],4161h ; 'aa' ; (NASM syntax)
21888     cmp     word [bx+FSINFO.LeadSig+2],4161h
21889     jne     short u_fat32_inf_5
21890
21891     ;cmp     word [bx+1E4h],7272h ; 'rr' at offset 484
21892     cmp     word [bx+FSINFO.StrucSig],7272h
21893     jne     short u_fat32_inf_5
21894
21895     ;cmp     word [bx+1E6h],6141h ; 'Aa' at offset 486
21896     cmp     word [bx+FSINFO.StrucSig+2],6141h
21897     jne     short u_fat32_inf_5
21898
21899     ;cmp     word [bx+1FEh],0AA55h ; boot signature at offset 510
21900     cmp     word [bx+FSINFO.TrailSig+2],0AA55h
21901     jne     short u_fat32_inf_5
21902
21903     ;mov     ax,[es:bp+1Fh]
21904     mov     ax,[es:bp+DPB.FREE_CNT]
21905     ;mov     [bx+1E8h],ax
21906     mov     [bx+FSINFO.Free_Count],ax ; at offset 488
21907     mov     ax,[es:bp+21h]
21908     mov     ax,[es:bp+DPB.FREE_CNT+2]
21909     ;mov     [bx+1EAh],ax
21910     mov     [bx+FSINFO.Free_Count+2],ax
21911
21912     ;mov     ax,[es:bp+39h]
21913     mov     ax,[es:bp+DPB.FAT32_NXTFREE]
21914     ;mov     [bx+1ECh],ax
21915     mov     [bx+FSINFO.Nxt_Free],ax ; at offset 492
21916     ;mov     ax,[es:bp+3Bh]
21917     mov     ax,[es:bp+DPB.FAT32_NXTFREE+2]
21918     ;mov     [bx+1EEh],ax
21919     mov     [bx+FSINFO.Nxt_Free+2],ax
21920
21921     xor     cx,cx
21922     mov     [ss:HIGH_SECTOR],cx ; 0
21923     inc     cx ; 1
21924     call    DWRITE
21925
21926 u_fat32_inf_5:
21927     pop     ds
21928     pop     si
21929     pop     di
21930     pop     bx
21931     pop     ax
21932     jmp     u_fat32_inf_1
21933
;=====

```

```

21934 ; ISEARCH.ASM, MSDOS 6.0, 1991
21935 ; =====
21936 ; 22/07/2018 - Retro DOS v3.0
21937
21938 ; TITLE DOS_SEARCH - Internal SEARCH calls for MS-DOS
21939 ; NAME DOS_SEARCH
21940
21941 ;** Low level routines for doing local and NET directory searches
21942 ;
21943 ; DOS_SEARCH_FIRST
21944 ; DOS_SEARCH_NEXT
21945 ; RENAME_NEXT
21946 ;
21947 ; Revision history:
21948 ;
21949 ; Created: ARR 30 March 1983
21950 ; A000 version 4.00 Jan. 1988
21951 ; A001 PTM 3564 -- search for fastopen
21952
21953 ;Installed = TRUE
21954
21955 ;-----
21956 ;
21957 ; Procedure Name : DOS_SEARCH_FIRST
21958 ;
21959 ; Inputs:
21960 ; [WFP_START] Points to WFP string ("d:/" must be first 3 chars, NUL
21961 ; terminated)
21962 ; [CURR_DIR_END] Points to end of Current dir part of string
21963 ; (= -1 if current dir not involved, else
21964 ; Points to first char after last "/" of current dir part)
21965 ; [THISCDS] Points to CDS being used
21966 ; (Low word = -1 if NUL CDS (Net direct request))
21967 ; [SATTRIB] Is attribute of search, determines what files can be found
21968 ; [DMAADD] Points to 53 byte buffer
21969 ; Function:
21970 ; Initiate a search for the given file spec
21971 ; Outputs:
21972 ; CARRY CLEAR
21973 ; The 53 bytes of DMAADD are filled in as follows:
21974 ;
21975 ; LOCAL
21976 ; Drive Byte (A=1, B=2, ...) High bit clear
21977 ; NEVER STORE DRIVE BYTE AFTER found_it
21978 ; 11 byte search name with Meta chars in it
21979 ; Search Attribute Byte, attribute of search
21980 ; WORD LastEnt value
21981 ; WORD DirStart
21982 ; 4 byte pad
21983 ; 32 bytes of the directory entry found
21984 ; NET
21985 ; 21 bytes First byte has high bit set
21986 ; 32 bytes of the directory entry found
21987 ;
21988 ; CARRY SET
21989 ; AX = error code
21990 ; error_no_more_files
21991 ; No match for this file
21992 ; error_path_not_found
21993 ; Bad path (not in curr dir part if present)
21994 ; error_bad_curr_dir
21995 ; Bad path in current directory part of path
21996 ; DS preserved, others destroyed
21997 ;-----
21998
21999 ; 24/01/2024
22000 %if 1
22001 ; 17/05/2019 - Retro DOS v4.0
22002 GET_FAST_SEARCH:
22003 ; 22/07/2018
22004 ; MSDOS 6.0
22005 ; 17/12/2022
22006 0000351E 36800E[1206]04 OR byte [ss:DOS34_FLAG+1],(SEARCH_FASTOPEN>>8) ; 04h
22007 ;OR word [ss:DOS34_FLAG],SEARCH_FASTOPEN ; 400h
22008 ;FO.trigger fastopen ;AN000;
22009 ;call DOS_SEARCH_FIRST
22010 ;retn
22011 ; 24/01/2024
22012 ; 17/12/2022
22013 ;jmp DOS_SEARCH_FIRST
22014 %endif
22015
22016 ; 14/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
22017 ; DOSCODE:6C22h (MSDOS 5.0, MSDOS.SYS)
22018
22019 ; 03/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
22020 ; DOSCODE:74E9h (PCDOS 7.1, IBMDOS.COM)
22021
22022 DOS_SEARCH_FIRST:
22023 ; IBMDOS.COM (MSDOS 3.3 kernel) - Offset 3826h
22024
22025 00003524 C43E[A205] LES DI,[THISCDS]
22026 00003528 83FFFF CMP DI,-1
22027 0000352B 7506 JNZ short TEST_RE_NET2
22028
22029 ;IF NOT Installed
22030 ; transfer NET_SEQ_SEARCH_FIRST
22031 ;ELSE
22032 ;mov ax,1119h
22033 0000352D B81911 MOV AX,(MultNET<<8)|25
22034 00003530 CD2F INT 2Fh
22035 00003532 C3 retn
22036 ;ENDIF
22037
22038 TEST_RE_NET2:
22039 ;test word [es:di+43h],8000h
22040 ; 17/12/2022
22041 ;test byte [es:di+44h],80h
22042 ; 28/12/2022
22043 00003533 26F6454480 test byte [ES:DI+curdir.flags+1],curdir_isnet>>8
22044 ;TEST word [ES:DI+curdir.flags],curdir_isnet
22045 00003538 7406 JZ short LOCAL_SEARCH_FIRST
22046
22047 ;IF NOT Installed
22048 ; transfer NET_SEARCH_FIRST
22049 ;ELSE
22050 ;mov ax,111Bh
22051 0000353A B81B11 MOV AX,(MultNET<<8)|27
22052 0000353D CD2F INT 2FH
22053 0000353F C3 retn
22054 ;ENDIF
22055 ; 18/05/2019 - Retro DOS v4.0
22056 LOCAL_SEARCH_FIRST:
22057 00003540 E89FE3 call ECritDisk

```

```

22058             ; MSDOS 6.0
22059             ;;test word [DOS34_FLAG],400h
22060             ; 17/12/2022
22061             ;test byte [DOS34_FLAG+1],04h
22062 00003543 F606[1206]04 test byte [DOS34_FLAG+1],(SEARCH_FASTOPEN>>8)
22063             ;TEST word [DOS34_FLAG],SEARCH_FASTOPEN ;AN000;
22064 00003548 7405 JZ short NOFN ;AN000;
22065             ;or byte [FastOpenFlg],1
22066 0000354A 800E[4611]01 OR byte [FastOpenFlg],FastOpen_Set ;AN000;
22067             NOFN: ;AN000;
22068 0000354F C606[4C03]01 MOV byte [NoSetDir],1 ; if we find a dir, don't change to it
22069
22070             ; 03/02/2024
22071             %if 0
22072             ; MSDOS 6.0
22073             CALL CHECK_QUESTION ;AN000;;FO. is '?' in path
22074             JNC short norm_GETPATH ;AN000;;FO. no
22075             %else
22076             ; 03/02/2024
22077 00003554 16 push ss
22078 00003555 1F pop ds ;AN000;;FO. ds:si -> final path
22079 00003556 8B36[B205] mov si,[WFP_START] ;AN000;;FO.
22080             getnext: ;AN000;
22081 0000355A AC lodsb ;AN000;;FO. get char
22082 0000355B 08C0 or al,al ;AN000;;FO. is it null
22083 0000355D 7409 jz short NO_Question ;AN000;;FO. yes
22084 0000355F 3C3F cmp al,'?' ;AN000;;FO. is '?'
22085 00003561 75F7 jne short getnext ;AN000;;FO. no
22086             %endif
22087             ;and byte [FastOpenFlg],80h
22088 00003563 8026[4611]80 AND byte [FastOpenFlg],Fast_yes ;AN000;;FO. reset fastopen
22089             NO_Question: ; 03/02/2024
22090             norm_GETPATH:
22091             call GETPATH
22092             ; BX = offset NAME1
22093             ;_getdone:
22094 0000356B 7318 JNC short find_check_dev
22095 0000356D 7511 JNZ short bad_path3
22096 0000356F 08C9 OR CL,CL
22097 00003571 740D JZ short bad_path3
22098             find_no_more:
22099             ;mov ax,12h
22100 00003573 B81200 MOV AX,error_no_more_files
22101             BadBye:
22102             ; MSDOS 6.0
22103 00003576 368026[4611]80 AND byte [SS:FastOpenFlg],Fast_yes ;AN000;;FO. reset fastopen
22104
22105             STC
22106             ;call LCritDisk
22107             ;retn
22108             ; 18/12/2022
22109 0000357D E98FE3 jmp LCritDisk
22110
22111             bad_path3:
22112             ;mov ax,3
22113 00003580 B80300 MOV AX,error_path_not_found
22114 00003583 EBF1 JMP short BadBye
22115
22116             find_check_dev:
22117             OR AH,AH
22118 00003587 790A JNS short found_entry
22119 00003589 C706[4803]FFFF MOV word [LASTENT],-1 ; Cause DOS_SEARCH_NEXT to fail
22120 0000358F FE06[7005] INC byte [FOUND_DEV] ; Tell DOS_RENAME we found a device
22121             found_entry:
22122
22123             ; We set the physical drive byte here Instead of after found_it; Doing
22124             ; a search-next may not have wfp_start set correctly
22125
22126 00003593 C43E[2C03] LES DI,[DMAADD]
22127 00003597 8B36[B205] MOV SI,[WFP_START] ; get pointer to beginning
22128 0000359B AC LODSB
22129 0000359C 2C40 SUB AL,'A'-1 ; logical drive
22130 0000359E AA STOSB ; High bit not set (local)
22131             found_it:
22132 0000359F C43E[2C03] LES DI,[DMAADD]
22133 000035A3 47 INC DI
22134
22135             ; MSDOS 6.0
22136 000035A4 1E PUSH DS ;FO.;AN001; save ds
22137
22138 000035A5 F606[4611]10 ;test byte [FastOpenFlg],10h
22139 000035AA 7408 TEST byte [FastOpenFlg],Set_For_Search ;FO.;AN001; from fastopen
22140 000035AC 89DE JZ short notfast ;FO.;AN001;
22141 000035AE 8E1E[E405] MOV SI,BX ;FO.;AN001;
22142 000035B2 EB03 MOV DS,[CURBUF+2] ;FO.;AN001;
22143             JMP SHORT movmov ;FO.;AN001;
22144
22145             notfast:
22146             MOV SI,NAME1 ; find_buf 2 = formatted name
22147             movmov:
22148             ; Special E5 code
22149             MOVSB
22150             CMP BYTE [ES:DI-1],5
22151             JNZ short NOTKANJB
22152             MOV BYTE [ES:DI-1],0E5H
22153             NOTKANJB:
22154             ;MOV CX,10
22155             ;REP MOVSB
22156             ; 03/02/2024
22157 000035C4 B90500 mov cx,5
22158 000035C7 F3A5 rep movsw
22159
22160             ; 08/09/2018
22161             POP DS ;FO.;AN001; restore ds
22162
22163             MOV AL,[ATTRIB]
22164             STOSB
22165             PUSH AX ; Save AH device info
22166             MOV AX,[LASTENT]
22167             STOSW
22168             MOV AX,[DIRSTART]
22169             STOSW
22170
22171             ; 03/02/2024 - Retro DOS v5.0
22172             ; PCDOS 7.1 IBMDOS.COM
22173             ;;;
22174             MOV AX,[DIRSTART_HW]
22175             STOSW
22176             add di,2
22177             ; 4 bytes of 21 byte cont structure left for NET stuff
22178             ;ADD DI,4
22179             ;;;
22180             POP AX ; Recover AH device info
22181             OR AH,AH

```

```

22182 000035E1 781B      JS      short DOSREL      ; Device entry is DOSGROUP relative
22183 000035E3 833E[E205]FF  CMP     WORD [CURBUF],-1
22184 000035E8 7510      JNZ     short OKSTORE
22185
22186      ; MSDOS 6.0
22187 000035EA F606[4611]10  TEST    byte [FastOpenFlg],Set_For_Search
22188      ; AN000;;F0. from fastopen and is good
22189 000035EF 7509      JNZ     short OKSTORE      ; AN000;;F0.
22190
22191      ; The user has specified the root directory itself, rather than some
22192      ; contents of it. We can't "find" that.
22193
22194 000035F1 26C745F8FFFF  MOV     WORD [ES:DI-8],-1      ; Cause DOS_SEARCH_NEXT to fail by
22195      ; stuffing a -1 at Lastent
22196 000035F7 E979FF      JMP     find_no_more
22197
22198 OKSTORE:
22199 000035FA 8E1E[E405]  MOV     DS,[CURBUF+2]
22200 DOSREL:
22201      ; BX = offset NAME1 (from GETPATH)
22202 000035FE 89DE      MOV     SI,BX      ; SI-> start of entry
22203
22204      ; NOTE: DOS_RENAME depends on BX not being altered after this point
22205
22206      ; ;mov cx,32
22207      ; MOV CX,dir_entry.size
22208      ; 03/02/2024
22209 00003600 B91000  mov     cx,dir_entry.size>>1
22210      ; ; ; 7/29/86
22211 00003603 89F8      MOV     AX,DI      ; save the 1st byte addr
22212      ; REP MOVSB
22213 00003605 F3A5      rep     movsw
22214      ;
22215 00003607 89C7      MOV     DI,AX      ; restore 1st byte addr
22216 00003609 26803D05  CMP     BYTE [ES:DI],05H      ; special char check
22217 0000360D 7504      JNZ     short NO05
22218 0000360F 26C605E5  MOV     BYTE [ES:DI],0E5H      ; convert it back to E5
22219 NO05:
22220
22221      ; ; ; ; 7/29/86
22222
22223      ; hkn; FastOpenFlg is in DOSDATA use SS
22224      ; 16/12/2022
22225      ; 14/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
22226      ; MSDOS 6.0
22227      ; AND byte [SS:FastOpenFlg],Fast_yes ; AN000;;F0. reset fastopen
22228      ; 18/05/2019 - Retro DOS v4.0
22229 00003613 16      push    ss
22230 00003614 1F      pop     ds
22231      ; 16/12/2022
22232 00003615 8026[4611]80  AND     byte [FastOpenFlg],Fast_yes ; 80h
22233
22234      ; hkn; SS is DOSDATA
22235      ; push ss
22236      ; pop ds
22237
22238      ; 27/06/2024
22239      ; cf=0
22240      ; CLC
22241
22242      ; call LCritDisk
22243      ; retn
22244      ; 16/12/2022
22245 0000361A E9F2E2  jmp     LCritDisk
22246
22247      ; BREAK <DOS_SEARCH_NEXT - scan for subsequent matches>
22248      ; -----
22249      ;
22250      ; Procedure Name : DOS_SEARCH_NEXT
22251      ;
22252      ; Inputs:
22253      ; [DMAADD] Points to 53 byte buffer returned by DOS_SEARCH_FIRST
22254      ; (only first 21 bytes must have valid information)
22255      ; Function:
22256      ; Look for subsequent matches
22257      ; Outputs:
22258      ; CARRY CLEAR
22259      ; The 53 bytes at DMAADD are updated for next call
22260      ; (see DOS_SEARCH_FIRST)
22261      ; CARRY SET
22262      ; AX = error code
22263      ; error_no_more_files
22264      ; No more files to find
22265      ; DS preserved, others destroyed
22266      ; -----
22267
22268      ; hkn; called from search.asm. DS already set up at this point.
22269
22270      ; 03/02/2024 - Retro DOS v5.0
22271      ; PC DOS 7.1 IBMDOS.COM - DOSCODE:75ECh
22272
22273 DOS_SEARCH_NEXT:
22274 0000361D C43E[2C03]  LES     DI,[DMAADD]      ; 24/01/2024 (Retro DOS v5-v4)
22275 00003621 268A05  MOV     AL,[ES:DI]
22276 00003624 A880      TEST    AL,80H      ; Test for NET
22277 00003626 7406      JZ      short LOCAL_SEARCH_NEXT
22278      ; IF NOT Installed
22279      ; transfer NET_SEARCH_NEXT
22280      ; ELSE
22281      ; mov ax,111Ch
22282 00003628 B81C11  MOV     AX,(MultNET<<8)|28
22283 0000362B CD2F      INT     2FH      ; Multiplex - NETWORK REDIRECTOR - FINDNEXT
22284      ; SS = DS = DOS CS, [DTA] = 21-byte findfirst search data
22285      ; Return: CF set on error, AX = DOS error code
22286      ; CF clear if successful
22287 0000362D C3      retn
22288      ; ENDIF
22289
22290 LOCAL_SEARCH_NEXT:
22291      ; AL is drive A=1
22292      ; mov byte [EXTERR_LOCUS],2
22293 0000362E C606[2303]02  MOV     byte [EXTERR_LOCUS],errLOC_Disk
22294 00003633 E8ACE2  call    ECritDisk
22295
22296      ; hkn; DummyCDS is in DOSDATA
22297 00003636 C706[A205][F304]  MOV     word [THISCDS],DUMMYCDS
22298      ; hkn; Segment address is DOSDATA - use ds
22299      ; hkn; MOV WORD [THISCDS+2],CS
22300 0000363C 8C1E[A405]  mov     [THISCDS+2],DS
22301
22302 00003640 0440      ADD     AL,'A'-1
22303 00003642 E89044  call    InitCDS
22304
22305      ; call GETTHISDRV      ; Set CDS pointer

```

```

22306
22307 00003645 7253          JC      short No_files          ; Bogus drive letter
22308 00003647 C43E[A205]    LES      DI,[THISCDS]          ; Get CDS pointer
22309                          ;les      bp,[es:di+45h]
22310 0000364B 26C46D45      LES      BP,[ES:DI+curdir.devptr] ; Get DPB pointer
22311 0000364F E813D0        call     GOTDPB              ; [THISDPB] = ES:BP
22312
22313                          ; 16/12/2022
22314 00003652 268A4600      mov      al,[ES:BP]
22315                          ; 14/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
22316                          ;mov     AL,[ES:BP+DPB.DRIVE] ; mov al,[ES:BP+0]
22317 00003656 A2[7605]      mov      [THISDRV],AL
22318                          ;mov     word [CREATING],0E500h
22319 00003659 C706[7E05]00E5 MOV      WORD [CREATING],(DIRFREE*256)+0
22320 0000365F C606[4C03]01  MOV      byte [NoSetDir],1      ; if we find a dir, don't change to it
22321 00003664 C536[2C03]    LDS      SI,[DMAADD]
22322 00003668 AC            LODSB
22323                          ; Drive Byte
22324
22325                          ;entry  RENAME_NEXT          ; Entry used by DOS_RENAME
22326 RENAME_NEXT:
22327 00003669 16            ;context ES
22328 0000366A 07            push     ss
22329                          pop      es
22330                          ; THIS BLOWS ES:BP POINTER TO DPB
22331 0000366B BF[4B05]      ;hkn; NAME1 is in DOSDATA
22332                          MOV      DI,NAME1
22333 0000366E B90B00        MOV      CX,11
22334 00003671 F3A4        REP      MOVSB
22335 00003673 AC            LODSB
22336                          ; Search name
22337                          ; Attribute
22338 00003674 36A2[6B05]    ;hkn; SS override
22339 00003678 AD            MOV      [SS:ATTRIB],AL
22340 00003679 09C0        LODSW
22341                          ; LastEnt
22342                          OR      AX,AX
22343 0000367B 781D        ; 03/02/2024
22344                          ;JNS     short cont_load
22345                          js      short No_files
22346                          ;No_files:
22347                          ;JMP     find_no_more
22348 0000367D 50            cont_load:
22349 0000367E AD            PUSH     AX
22350 0000367F 89C3        LODSW
22351                          MOV      BX,AX
22352                          ; Save LastEnt
22353                          ; DirStart
22354
22355 00003681 AD            ;;;
22356                          ; 03/02/2024 - Retro DOS v5.0
22357                          ; (PCDOS 7.1 IBMDOS.COM)
22358                          lodsw
22359                          ; DIRSTART_HW
22360
22361 00003682 16            ;hkn; SS is DOSDATA
22362 00003683 1F            ;context DS
22363 00003684 C42E[8A05]    push     ss
22364                          pop      ds
22365                          LES      BP,[THISDPB]
22366                          ; Recover ES:BP
22367
22368                          ;;;
22369 00003688 26837E0F00    ; 03/02/2024 - Retro DOS v5.0
22370 0000368D 7402        ; (PCDOS 7.1 IBMDOS.COM)
22371 0000368F 31C0        ;cmp     word [es:bp+0Fh],0
22372                          cmp     word [es:bp+DPB.FAT_SIZE],0
22373 00003691 A3[EE0A]      jz      short cont_load2 ; FAT32 fs
22374                          xor     ax,ax ; 0
22375                          cont_load2:
22376                          mov     [ROOTCLUST_HW],ax ; 0 or DIRSTART_HW
22377                          ;;;
22378 00003694 E8C812        ;invoke SetDirSrch
22379 00003697 7304        call     SETDIRSRCH
22380 00003699 58            JNC     short SEARCH_GOON
22381                          POP      AX
22382                          ; Clean stack
22383 0000369A E9D6FE        ;JMP     short No_files
22384                          ; 03/02/2024
22385                          No_files:
22386                          JMP     find_no_more
22387
22388 0000369D E81317        SEARCH_GOON:
22389 000036A0 58            call     STARTSRCH
22390 000036A1 E8FE11        POP      AX
22391 000036A4 72F4            ; Restore LastEnt
22392 000036A6 E8F210        call     GETENT
22393 000036A9 72EF            JC      short No_files
22394 000036AB 30E4            call     NEXTENT
22395 000036AD E9EFFE        JC      short No_files
22396                          XOR      AH,AH
22397                          ; If Search_Next, can't be a DEV
22398                          JMP     found_it ; 10/08/2018
22399
22400                          ; MSDOS 6.0
22401                          ;-----
22402                          ;
22403                          ; Procedure Name : CHECK_QUESTION
22404                          ;
22405                          ; Input: [WFP_START]= pointer to final path
22406                          ; Function: check '?' char
22407                          ; Output: carry clear, if no '?'
22408                          ; carry set, if '?' exists
22409                          ;-----
22410
22411 000036B0 00000000      ; 03/02/2024
22412                          %if 0
22413                          ; 07/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
22414                          CHECK_QUESTION:
22415                          ;hkn; wfp_start is in DOSDATA;hkn; MOV      WORD PTR ThisCDS+2,CS
22416                          ;hkn; PUSH      CS
22417                          ;AN000;;FO.
22418                          push     ss
22419                          POP      DS
22420                          ;AN000;;FO. ds:si -> final path
22421                          ; 16/12/2022
22422                          ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
22423                          MOV      SI,[WFP_START]
22424                          ;AN000;;FO.
22425                          ;mov     si,[ss:WFP_START]
22426                          getnext:
22427                          LODSB
22428                          ;AN000;;FO. get char
22429                          OR      AL,AL
22430                          ;AN000;;FO. is it null
22431                          JZ      short NO_Question
22432                          ;AN000;;FO. yes
22433                          CMP      AL,'?'
22434                          ;AN000;;FO. is '?'
22435                          JNZ      short getnext
22436                          ;AN000;;FO. no
22437                          STC
22438                          ;AN000;;FO.
22439                          NO_Question:
22440                          ;AN000;;FO.
22441                          retn
22442                          ;AN000;;FO.
22443                          %endif
22444
22445                          ;=====

```

```

22430 ; ABORT.ASM, MSDOS 6.0, 1991
22431 ;=====
22432 ; 23/07/2018 - Retro DOS v3.0
22433 ; 18/05/2019 - Retro DOS v4.0
22434
22435 ;**
22436 ;
22437 ; Internal Abort call closes all handles and FCBs associated with a process.
22438 ; If process has NET resources a close all is sent out over the net.
22439 ;
22440 ; DOS_ABORT
22441 ;
22442 ; Modification history:
22443 ;
22444 ; Created: ARR 30 March 1983
22445 ;
22446 ; M038 SR 10/16/90 Free SFT with the PSP of the process
22447 ; being terminated only if it is busy.
22448 ;
22449
22450 ;Break <DOS_ABORT -- CLOSE all files for process>
22451 ;-----
22452 ;
22453 ; Procedure Name : DOS_ABORT
22454 ;
22455 ; Inputs:
22456 ; [CurrentPDB] set to PID of process aborting
22457 ; Function:
22458 ; Close all files and free all SFTs for this PID
22459 ; Returns:
22460 ; None
22461 ; All destroyed except stack
22462 ;-----
22463
22464 ; 03/02/2024 - Retro DOS v5.0
22465 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:7685h
22466 ; (Win ME IO.SYS - BIOSCODE:74F4h)
22467
22468 DOS_ABORT:
22469 MOV ES,[SS:CurrentPDB] ; SS override
22470 MOV CX,[ES:PDB.JFN_Length] ; Number of JFNs
22471 reset_free_jfn:
22472 MOV BX,CX
22473 PUSH CX
22474 DEC BX ; get jfn (start with last one)
22475
22476 CALL _$CLOSE
22477 POP CX
22478 LOOP reset_free_jfn ; and do 'em all
22479
22480 ; Note: We do need to explicitly close FCBs. Reasons are as follows: If we
22481 ; are running in the no-sharing no-network environment, we are simulating the
22482 ; 2.0 world and thus if the user doesn't close the file, that is his problem
22483 ; BUT... the cache remains in a state with garbage that may be reused by the
22484 ; next process. We scan the set and blast the ref counts of the FCBs we own.
22485 ;
22486 ; If sharing is loaded, then the following call to close process will
22487 ; correctly close all FCBs. We will then need to walk the list AFTER here.
22488 ;
22489 ; Finally, the following call to NET_Abort will cause an EOP to be sent to all
22490 ; known network resources. These resources are then responsible for cleaning
22491 ; up after this process.
22492 ;
22493 ; sleazy, eh?
22494 ;
22495 ; context DS ; SS is DOSDATA
22496 push ss
22497 pop ds ; 09/09/2018
22498
22499 ; CallInstall Net_Abort, MultNET, 29
22500 mov ax,111dh
22501 int 2Fh ; Multiplex - NETWORK REDIRECTOR
22502 ; - CLOSE ALL REMOTE FILES FOR PROCESS
22503 ; DS???, SS = DOS CS
22504
22505 ;if installed
22506 call far [Jshare+(4*4)] ; 4 = MFTCloseP
22507 ;else
22508 call MFTCloseP
22509 ;endif
22510
22511 ; Scan the FCB cache for guys that belong to this process and zap their ref
22512 ; counts.
22513 les di,[ss:SFTFCB] ; SS override
22514 ; grab the pointer to the table
22515 ;;;
22516 ; 03/02/2024 - Retro DOS v5.0
22517 ; PCDOS 7.1 IBMDOS.COM
22518 SFTFCB_check:
22519 mov cx,es
22520 or cx,di
22521 jcxz FCBScanDone
22522 push di
22523 SFTFCB_OK:
22524 ;;;
22525 mov cx,[es:di+4]
22526 mov cx,[es:di+SFT.SFCount]
22527 jcxz FCBScanDone
22528 ; 27/06/2024
22529 ;;;
22530 jcxz SFTFCB_check2
22531 ;;;
22532 lea di,[di+6]
22533 LEA DI,[DI+SFT.SFTTable] ; point at table
22534 mov ax,[SS:PROC_ID] ; SS override
22535 FCBTest:
22536 cmp [es:di+31h],ax
22537 cmp [es:di+SF_ENTRY.sf_PID],ax ; is this one of ours
22538 jnz short FCBNext ; no, skip it
22539
22540 ; 03/02/2024 - Retro DOS v5.0
22541 %if 0
22542 mov word [es:di],0
22543 ;mov word [es:di+SF_ENTRY.sf_ref_count],0 ; yes, blast ref count
22544 %else
22545 ; 03/02/2024
22546 ; PCDOS 7.1 IBMDOS.COM
22547 call SFT_FREE
22548 %endif
22549
22550 FCBNext:
22551 add di,SF_ENTRY.size ; 59 (for MSDOS 6.0)
22552 loop FCBTest
22553

```



```

22554             ; 27/06/2024 (PCDOS 7.1 IBMDOS.COM)
22555             ;;
22556 SFTFCB_check2:
22557     pop      di
22558     les      di,[es:di]
22559     cmp      di,0FFFFh
22560     jne      short SFTFCB_check
22561             ;;
22562
22563 FCBScanDone:
22564
22565 ; Walk the SFT to eliminate all busy SFT's for this process.
22566
22567     XOR      BX,BX
22568 Scan:
22569     push     bx
22570     call     SFFromSFN
22571     pop      bx
22572     jnc      short Scan1
22573     ;retn
22574
22575     ; 18/12/2022
22576     ;jc      short NO_Question ; retn
22577     ; 03/02/2024
22578     jc      short RET2
22579
22580 ;M038
22581 ; Do what the comment above says, check for busy state
22582
22583 Scan1:
22584     ;cmp     word [es:di],0
22585     ;jz      short scan_next ; MSDOS 3.3
22586     ; MSDOS 6.0
22587     cmp     word [es:di],sf_busy ; -1
22588     ;cmp     word [es:di+SF_ENTRY.sf_ref_count],sf_busy
22589             ; Is Sft busy? ;M038
22590     jnz     short scan_next ; no
22591
22592 ; we have a SFT that is busy. See if it is for the current process
22593 ;
22594     mov     ax,[SS:PROC_ID] ; SS override
22595     ;cmp     [es:di+31h],ax
22596     cmp     [es:di+SF_ENTRY.sf_PID],ax
22597     jnz     short scan_next
22598     mov     ax,[SS:USER_ID] ; SS override
22599     ;sub     ax,[es:di+2Fh] ; PCDOS 7.1 IBMDOS.COM ; 03/02/2024
22600     ;cmp     [es:di+2Fh],ax
22601     cmp     [es:di+SF_ENTRY.sf_UID],ax
22602     jnz     short scan_next
22603
22604 ; This SFT is labelled as ours.
22605
22606 ; 03/02/2024 - Retro DOS v5.0
22607 %if 0
22608     mov     word [es:di],0
22609     ;mov     word [es:di+SF_ENTRY.sf_ref_count],0
22610 %else
22611     ; 03/02/2024
22612     ; PCDOS 7.1 IBMDOS.COM
22613     call    SFT_FREE
22614 %endif
22615
22616 scan_next:
22617     inc     bx
22618     jmp     short Scan
22619
22620 ;=====
22621 ; CLOSE.ASM, MSDOS 6.0, 1991
22622 ;=====
22623 ; 23/07/2018 - Retro DOS v3.0
22624 ; 18/05/2019 - Retro DOS v4.0
22625
22626 ;** Internal Close and Commit calls to close a local or NET SFT.
22627 ;
22628 ; DOS_CLOSE
22629 ; DOS_COMMIT
22630 ; FREE_SFT
22631 ; SetSFTTimes
22632 ;
22633 ; Revision history:
22634 ;
22635 ; AN000 version 4.00Jan. 1988
22636 ; A005 PTM 3718 --- lost clusters when fastopen installed
22637 ; A011 PTM 4766 --- C2 fastopen problem
22638
22639 ;Installed = TRUE
22640
22641 ;Break <DOS_CLOSE -- CLOSE FILE from SFT>
22642 ;-----
22643 ;
22644 ; Procedure Name : DOS_CLOSE
22645 ;
22646 ; Inputs:
22647 ; [THISSFT] set to the SFT for the file being used
22648 ; Function:
22649 ; Close the indicated file via the SFT
22650 ; Returns:
22651 ; sf_ref_count decremented otherwise
22652 ; ES:DI point to SFT
22653 ; Carry set if error
22654 ; AX has error code
22655 ; DS preserved, others destroyed
22656 ;-----
22657
22658 ;hkn; DOS_CLOSE called from fcbio.asm and handle.asm. DS already set up.
22659
22660 ; 18/05/2019 - Retro DOS v4.0
22661 ; DOSCODE:6E2Eh (MSDOS 6.21, MSDOS.SYS)
22662
22663 ; 14/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
22664 ; DOSCODE:6E1Ah (MSDOS 5.0, MSDOS.SYS)
22665
22666 ; 23/07/2018 - IBMDOS.COM (MSDOS 3.3), 1987 - Offset 39D0h
22667
22668 ; 03/02/2024 - Retro DOS v5.0
22669 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:76FEh
22670 ; (Win ME IO.SYS - BIOSCODE:7579h)
22671
22672 DOS_CLOSE:
22673     LES     DI,[THISSFT]
22674     ;mov     bx,[ES:DI+5]
22675     MOV     BX,[ES:DI+SF_ENTRY.sf_flags]
22676
22677 ; Network closes are handled entirely by the net code.

```

```

22678
22679 ;;test bx,8000h
22680 ;TEST BX,sf_isnet
22681 ; 17/12/2022
22682 ;test bh,80h
22683 00003730 F6C780 test bh,(sf_isnet>>8)
22684 00003733 7406 JZ short LocalClose
22685
22686 ;CallInstall Net_Close,MultNET,6
22687 00003735 B80611 mov ax,1106h
22688 00003738 CD2F int 2Fh ; Multiplex - NETWORK REDIRECTOR - CLOSE REMOTE FILE
22689 ; ES:DI -> SFT
22690 ; SFT DPB field -> DPB of drive containing file
22691 ; Return: CF set on error, AX = DOS error code
22692 ; CF clear if successful
22693 RET2: ; 03/02/2024
22694 0000373A C3 retn
22695
22696 ; All closes release the sharing information.
22697 ; No commit releases sharing information
22698 ;
22699 ; All closes decrement the ref count.
22700 ; No commit decrements the ref count.
22701
22702 LocalClose:
22703 0000373B E8A4E1 call ECritDisk
22704 0000373E E8C001 CALL SetsFTTimes
22705 00003741 E84801 CALL FREE_SFT ; dec ref count or mark as busy
22706
22707 ;hkn; SS is DOSDATA
22708 ;Context DS
22709 00003744 16 push ss
22710 00003745 1F pop ds
22711
22712 push ax
22713 push bx
22714 00003748 E8394D call ShareEnd
22715 0000374B 58 pop bx
22716 0000374C 58 pop ax
22717
22718 ; Commit enters here. AX from commit MUST be <> 1, BX is flags word
22719
22720 CloseEntry:
22721 0000374D 50 PUSH AX
22722
22723 ; File clean or device does not get stamped nor disk looked at.
22724
22725 ;test bx,0C0h
22726 ; 17/12/2022
22727 0000374E F6C3C0 test bl,devid_file_clean+devid_device
22728 ;TEST BX,devid_file_clean+devid_device
22729 00003751 7403 JZ short rdir
22730 ; 14/11/2022
22731 00003753 E9FD00 JMP FREE_SFT_OK ; either clean or device
22732 ;jnz short FREE_SFT_OK ; 24/07/2019
22733
22734 ; Retrieve the directory entry for the file
22735
22736 rdir:
22737 00003756 E84001 CALL DirFromSFT
22738 ;mov al,5
22739 00003759 B005 MOV AL,error_access_denied
22740
22741 ; 03/02/2024
22742 ;JNC short clook
22743 ;; 14/11/2022
22744 ;JMP CloseFinish ; pretend the close worked.
22745 ;;jc short CloseFinish ; 24/07/2019
22746
22747 ;;;
22748 ; 03/02/2024 - Retro DOS v5.0
22749 ; (PCDOS 7.1 IBMDOS.COM)
22750 0000375B 723D jc short jmp_to_CloseFinish ; pretend the close worked.
22751 rdir2:
22752 ;test word [si+2],4
22753 0000375D F6440204 test byte [si+SF_ENTRY.sf_mode],4 ; devid_device_null
22754 ; bit 2 - null device
22755 jnz short clook
22756
22757 push ds
22758 00003764 53 push bx
22759 ; 27/06/2024
22760 ;lds bx,[si+7]
22761 00003765 C55C07 lds bx,[si+SF_ENTRY.sf_devptr] ; pointer to DPB
22762
22763 00003768 8A1F mov bl,[bx] ; DPB.DRIVE
22764 0000376A 30FF xor bh,bh ; 0
22765 0000376C 36F687[0813]04 test byte [ss:bx+drive_flags],4
22766 ; bit 2 - last access date/time flag?
22767 ; or disk accessed flag !?
22768 00003772 5B pop bx
22769 00003773 7409 jz short skip_upd_laccdt ; no support for last access date&time
22770 00003775 1F pop ds
22771 00003776 1E push ds
22772 00003777 E8E6D3 call DATE16
22773 ;mov [es:di+12h],ax
22774 0000377A 26894512 mov [es:di+dir_entry.dir_lstaccdt],ax
22775 skip_upd_laccdt:
22776 0000377E 1F pop ds
22777 ;;;
22778
22779 clook:
22780
22781 ; ES:DI points to entry
22782 ; DS:SI points to SFT
22783 ; ES:BX points to buffer header
22784
22785 0000377F 57 push di
22786 00003780 56 push si
22787 ;lea si,[si+20h]
22788 00003781 8D7420 LEA SI,[SI+SF_ENTRY.sf_name]
22789
22790 ; ES:DI point to directory entry
22791 ; DS:SI point to unpacked name
22792
22793 00003784 E856E0 call XCHGP
22794
22795 ; ES:DI point to unpacked name
22796 ; DS:SI point to directory entry
22797
22798 00003787 E88F10 call MetaCompare
22799 0000378A E850E0 call XCHGP
22800 0000378D 5E pop si
22801 0000378E 5F pop di

```



```

22802 0000378F 740C      JZ      short CLOSE_GO      ; Name OK
22803
22804 00003791 89F7      Bye:    MOV     DI,SI
22805 00003793 1E          PUSH   DS
22806 00003794 07          POP    ES      ; ES:DI points to SFT
22807 00003795 16          PUSH   SS
22808 00003796 1F          POP    DS
22809 00003797 F9          STC
22810          ;mov    al,2
22811 00003798 B002        MOV     AL,error_file_not_found
22812
22813          jmp_to_CloseFinish: ; 03/02/2024
22814          ;JMP     CloseFinish ; 24/07/2019
22815          ; 03/02/2024
22816          ; (PCDOS 7.1 IBMDOS.COM)
22817 0000379A E9DE00      jmp     CloseFinish2
22818
22819          ; 18/05/2019 - Retro DOS v4.0
22820 CLOSE_GO:
22821          ; 03/02/2024
22822          ;mov    al,[si+4]
22823 0000379D 8A4404      mov     al,[si+SF_ENTRY.sf_attr]
22824
22825          ; MSDOS 6.0
22826          ;test   word [si+2],8000h
22827          ;TEST  word [SI+SF_ENTRY.sf_mode],sf_isFCB ; FCB ?
22828          ; 17/12/2022
22829          ;test   byte [si+3],80h
22830 000037A0 F6440380  test   byte [SI+SF_ENTRY.sf_mode+1],(sf_isFCB>>8) ; FCB ?
22831 000037A4 740A      JZ      short nofcb      ; no, set dir attr, sf_attr
22832          ; MSDOS 3.3 & MSDOS 6.0
22833          ;mov    ch,[es:di+0Bh]
22834 000037A6 268A6D0B  MOV     CH,[ES:DI+dir_entry.dir_attr]
22835
22836          ; 03/02/2024
22837          ;mov    al,[si+4]
22838          ;MOV     AL,[SI+SF_ENTRY.sf_attr]
22839
22840          ;hkn; SS override
22841 000037AA 36A2[6B05]  MOV     [SS:ATTRIB],AL
22842          ; MSDOS 3.3
22843          ;call  MatchAttributes
22844          ;JNZ   short Bye      ; attributes do not match
22845          ; 18/05/2019
22846 000037AE EB04      JMP     SHORT setattr      ;FT.
22847          nofcb:
22848          ; 03/02/2024
22849          ; MSDOS 6.0
22850          ;mov    al,[si+4]
22851          ;MOV     AL,[SI+SF_ENTRY.sf_attr] ;FT.      ;AN000;
22852
22853          MOV     [ES:DI+dir_entry.dir_attr],AL ;FT.      ;AN000;
22854          setattr:
22855          ; MSDOS 3.3 (& MSDOS 6.0)
22856          ;or     byte [es:di+0Bh],20h
22857 000037B4 26804D0B20  OR      BYTE [ES:DI+dir_entry.dir_attr],attr_archive ;Set archive
22858          ; MSDOS 6.0
22859          ;mov    ax,[es:di+1Ah]
22860 000037B9 268B451A  MOV     AX,[ES:DI+dir_entry.dir_first] ;AN011
22861          ;F.O. save old first cluster
22862          ;hkn; SS override
22863 000037BD 36A3[BD0E]  MOV     [SS:OLD_FIRSTCLUS],AX ;AN011;F.O. save old first cluster
22864
22865          ;;;
22866          ; 03/02/2024
22867          ; PCDOS 7.1 IBMDOS.COM
22868          ;mov    ax,[es:di+14h]
22869 000037C1 268B4514  mov     ax,[ES:DI+dir_entry.dir_fclus_hi] ; old first cluster, hw
22870          ; 27/06/2024
22871 000037C5 36A3[3A11]  MOV     [ss:OLD_FIRSTCLUS_HW],ax
22872          ;;;
22873          ;mov    ax,[si+0Bh]
22874          ;MOV     AX,[SI+SF_ENTRY.sf_firclus]
22875          ;;;
22876          ; 03/02/2024
22877          ; PCDOS 7.1 IBMDOS.COM
22878          ;mov    ax,[si+2Bh] ; mov ax,[si+SF_ENTRY.sf_chain]
22879 000037C9 8B442B      ; first cluster (32 bit) low word !
22880
22881          ;;;
22882          ;mov    [es:di+1Ah],ax
22883 000037CC 2689451A  MOV     [ES:DI+dir_entry.dir_first],AX      ;Set firclus pointer
22884          ;;; 03/02/2024
22885 000037D0 8B442D      mov     ax,[si+2Dh] ; mov ax,[si+SF_ENTRY.sf_chain+2]
22886          ; first cluster (32 bit) high word !
22887          ;mov    [es:di+14h],ax
22888 000037D3 26894514  mov     [es:di+dir_entry.dir_fclus_hi],ax
22889          ;;;
22890          ; 03/02/2024
22891          %if 0
22892          ;mov    ax,[si+11h]
22893          MOV     AX,[SI+SF_ENTRY.sf_size]
22894          ;mov    [es:di+1Ch],ax
22895          MOV     [ES:DI+dir_entry.dir_size_l],AX      ;Set size
22896          ;mov    ax,[si+13h]
22897          MOV     AX,[SI+SF_ENTRY.sf_size+2]
22898          ;mov    [es:di+1Eh],ax
22899          MOV     [ES:DI+dir_entry.dir_size_h],AX
22900          ;mov    ax,[si+0Fh]
22901          MOV     AX,[SI+SF_ENTRY.sf_date]
22902          ;mov    [es:di+18h],ax
22903          MOV     [ES:DI+dir_entry.dir_date],AX ;Set date
22904          ;mov    ax,[si+0Dh]
22905          MOV     AX,[SI+SF_ENTRY.sf_time]
22906          ;mov    [es:di+16h],ax
22907          MOV     [ES:DI+dir_entry.dir_time],AX ;Set time
22908          %else
22909          ; 03/02/2024 - Retro DOS v5.0
22910          push    si
22911 000037D7 56          add     si,0Dh
22912 000037D8 83C60D      lodsw    [si+SF_ENTRY.sf_time]
22913 000037DB AD          mov     [es:di+dir_entry.dir_time],ax ; Set time
22914 000037DC 26894516  lodsw    [si+SF_ENTRY.sf_date]
22915 000037E0 AD          mov     [es:di+dir_entry.dir_date],ax ; Set date
22916 000037E1 26894518  lodsw    [si+SF_ENTRY.sf_size]
22917 000037E5 AD          mov     [es:di+dir_entry.dir_size_l],ax      ; Set size
22918 000037E6 2689451C  lodsw    [si+SF_ENTRY.sf_size+2]
22919 000037EA AD          mov     [es:di+dir_entry.dir_size_h],ax
22920 000037EB 2689451E  pop     si
22921 000037EF 5E          %endif
22922
22923          ; MSDOS 6.0
22924          ;; File Tagging
22925

```

```

22926 000037F0 26F6470540      TEST    byte [ES:BX+BUFFINFO.buf_flags],buf_dirty
22927                                ;LB. if already dirty ;AN000;
22928 000037F5 7508             JNZ     short yesdirty4 ;LB. don't increment dirty count ;AN000;
22929                                ; 02/06/2019
22930 000037F7 E8D033           call    INC_DIRTY_COUNT ;LB. ;AN000;
22931                                ; MSDOS 3.3 (& MSDOS 6.0)
22932                                ;or    byte [es:bx+5],40h
22933 000037FA 26804F0540         OR      byte [ES:BX+BUFFINFO.buf_flags],buf_dirty ;Buffer dirty
22934 yesdirty4:
22935 000037FF 1E                push    ds
22936 00003800 56                push    si
22937
22938 ; 03/02/2024
22939 %if 0
22940                                ; MSDOS 6.0
22941                                ;mov    cx,[si+0Bh]
22942                                ; 07/12/2022
22943                                MOV     CX,[SI+SF_ENTRY.sf_firclus] ; do this for Fastopen
22944 %else
22945                                ; 03/02/2024
22946                                ; PCDOS 7.1
22947 00003801 8B4C2B           mov     cx,[si+2Bh] ; [es:di+SF_ENTRY.sf_chain]
22948                                ; first cluster (32 bit) low word !?
22949 %endif
22950
22951 ;hkn; SS override
22952 00003804 36A0[7605]        MOV     AL,[SS:THISDRV]
22953                                ; MSDOS 3.3
22954                                ;push    ss
22955                                ;pop     ds
22956                                ;MOV     AL,[THISDRV]
22957 ;;; 10/1/86 update fastopen cache
22958                                ; MSDOS 3.3 & MSDOS 6.0
22959 00003808 52                PUSH    DX
22960 00003809 B400             MOV     AH,0 ; dir entry update
22961 0000380B 88C2             MOV     DL,AL ; drive number A=0, B=1,,,
22962                                ;;;
22963                                ; 03/02/2024 - Retro DOS v5.0
22964                                ; PCDOS 7.1
22965 0000380D 53                push    bx ; *
22966 0000380E 8B5C2D           mov     bx,[si+2Dh] ; [es:di+SF_ENTRY.sf_chain+2]
22967                                ; first cluster (32 bit) high word !?
22968                                or      bx,bx
22969 00003813 7510             jnz     short do_update2
22970                                ;;;
22971                                ; MSDOS 6.0
22972 00003815 09C9             OR      CX,CX ;AN005; first cluster 0; may be truncated
22973 00003817 750C             JNZ     short do_update2 ;AN005; no, do update
22974 00003819 B403             MOV     AH,3 ;AN005; do a delete cache entry
22975                                ; 27/06/2024
22976                                ;;;
22977                                ;;mov    di,[si+1Bh]
22978                                ;MOV     DI,[SI+SF_ENTRY.sf_dirsec] ;AN005; cx:di = dir sector
22979                                ;;mov    cx,[si+1Dh]
22980                                ;MOV     CX,[SI+SF_ENTRY.sf_dirsec+2] ;AN005;
22981 0000381B C57C1B           lds     di,[si+SF_ENTRY.sf_dirsec]
22982 0000381E 8CD9             mov     cx,ds
22983                                ;;;
22984                                ;mov     dh,[si+1Fh]
22985 00003820 8A741F           MOV     DH,[SI+SF_ENTRY.sf_dirpos] ;AN005; dh = dir pos
22986 00003823 EB1A             JMP     SHORT do_update ;AN011;F.O.
22987
22988 do_update2: ;AN011;F.O.
22989                                ;;;
22990                                ; 03/02/2024
22991                                ; PCDOS 7.1
22992 00003825 363B1E[3A11]        cmp     bx,[ss:OLD_FIRSTCLUS_HW] ; same as old first cluster?
22993 0000382A 7507             jnz     short do_update3 ; no
22994                                ;;;
22995
22996 ;hkn; SS override fort OLD_FIRSTCLUS
22997                                ;
22998 0000382C 363B0E[BD0E]        CMP     CX,[SS:OLD_FIRSTCLUS] ;AN011;F.O. same as old first cluster?
22999 00003831 740C             JZ      short do_update ;AN011;F.O. yes
23000 do_update3: ; 03/02/2024
23001 00003833 B402             MOV     AH,2 ;AN011;F.O. delete the old entry
23002 00003835 368B0E[BD0E]        MOV     CX,[SS:OLD_FIRSTCLUS] ;AN011;F.O.
23003                                ;;;
23004                                ; 03/02/2024
23005                                ; PCDOS 7.1
23006 0000383A 368B1E[3A11]        mov     bx,[ss:OLD_FIRSTCLUS_HW]
23007                                ;;;
23008 do_update: ;AN005;
23009 ;hkn; SS is DOSDATA
23010 ;Context DS
23011 0000383F 16                push    ss
23012 00003840 1F                pop     ds
23013                                ;;;
23014                                ; 03/02/2024 - Retro DOS v5.0
23015 00003841 87DE             xchg    bx,si ; PCDOS 7.1 IBMDOS.COM
23016                                ;;;
23017                                ; MSDOS 3.3 & MSDOS 6.0
23018 00003843 E821F5           call    FastOpen_Update ; invoke fastopen
23019                                ;;;
23020                                ; 03/02/2024
23021 00003846 87DE             xchg    bx,si ; PCDOS 7.1 IBMDOS.COM
23022 00003848 5B                pop     bx ; *
23023                                ;;;
23024 00003849 5A                POP     DX
23025                                ;;;
23026 ;;; 10/1/86 update fastopen cache
23027 0000384A E84B32           call    FLUSHBUF ; flush all relevant buffers
23028 0000384D 5F                pop     di
23029 0000384E 07                pop     es
23030                                ;mov    al,5
23031 0000384F B005             MOV     AL,error_access_denied
23032 00003851 7215             JC      short CloseFinish
23033 FREE_SFT_OK:
23034                                ; 03/02/2024
23035                                ;CLC
23036                                ; signal no error.
23037                                ;;;
23038                                ; 03/02/2024 - Retro DOS v5.0
23039                                ; PCDOS 7.1 IBMDOS.COM
23040                                ;test    word [es:di+5],8080h
23041 00003853 26F745058080         test    word [es:di+SF_ENTRY.sf_flags],sf_isnet+devid_device
23042 00003859 7520             jnz     short CloseFinish2
23043 0000385B 06                push    es
23044 0000385C 55                push    bp
23045                                ;les    bp,[es:di+7]
23046 0000385D 26C46D07           les     bp,[es:di+SF_ENTRY.sf_devptr] ; DPB
23047 00003861 E808FC           call    update_fat32_fsinfo
23048 00003864 5D                pop     bp
23049 00003865 07                pop     es

```

```

23050 00003866 EB13      jmp      short CloseFinish2
23051
23052 CloseFinish:         ; 03/02/2024 - Retro DOS v5.0
23053 00003868 1E      push     ds
23054 00003869 53      push     bx
23055 0000386A 26C5D07 lds      bx,[es:di+7]      ; [es:di+SF_ENTRY.sf_devptr] ; DPB
23056 0000386E 8A1F      mov      bl,[bx]          ; DPB.DRIVE
23057 00003870 30FF      xor      bh,bh ; 0
23058 00003872 3680A7[0813]FB and      byte [ss:bx+drive_flags],0FBh ; clear bit 2
23059                                     ; bit 2 - last access date/time flag?
23060                                     ; or disk accessed (successful) flag !?
23061 00003878 5B      pop      bx
23062 00003879 1F      pop      ds
23063 0000387A F9      stc
23064                                     ;;;
23065
23066 CloseFinish2:         ; 03/02/2024 - Retro DOS v5.0
23067 ;Closefinish:
23068
23069 ; Indicate to the device that the SFT is being closed.
23070
23071 ;;;; 7/21/86
23072 0000387B 9C      PUSHF                     ; save flag from DirFromSFT
23073 0000387C E89D19 call     DEV_CLOSE_SFT
23074 0000387F 9D      POPF
23075 ;;;; 7/21/86
23076
23077 ; See if the ref count indicates that we have busied the SFT. If so, mark the
23078 ; SFT as being free. Note that we do NOT need to be in critSFT as we are ONLY
23079 ; going to be moving from busy to free.
23080
23081 00003880 59      POP      CX          ; get old ref count
23082 00003881 9C      PUSHF
23083                                     ; 03/02/2024
23084                                     ;DEC      CX          ; if cx != 1
23085                                     ;JNZ      short NoFree      ; then do NOT free SFT
23086 00003882 E203      loop     NoFree ; PCDOS 7.1 IBMDOS.COM
23087
23088 ; 03/02/2024 - Retro DOS v5.0
23089 %if 0
23090 mov      [es:di],cx
23091 ;MOV      [ES:DI+SF_ENTRY.sf_ref_Count],CX ; mov [es:di+0],cx
23092 %else
23093                                     ; 03/02/2024
23094                                     ; PCDOS 7.1 IBMDOS.COM
23095 00003884 E86412 call     SFT_FREE
23096 %endif
23097
23098 NoFree:
23099 00003887 E885E0 call     LCritDisk
23100 0000388A 9D      POPF
23101 0000388B C3      retn
23102
23103 ;-----
23104 ;
23105 ; Procedure Name : FREE_SFT
23106 ;
23107 ; ES:DI -> SFT. Decs sft_ref_count. If the count goes to 0, mark it as busy.
23108 ; Flags preserved. Return old ref count in AX
23109 ;
23110 ; Note that busy is indicated by the SFT ref count being -1.
23111 ;
23112 ;-----
23113
23114 FREE_SFT:
23115 0000388C 9C      PUSHF                     ; Save carry state
23116 0000388D 268B05 mov      ax,[es:di]
23117                                     ;MOV      AX,[ES:DI+SF_ENTRY.sf_ref_count]
23118 00003890 48      DEC      AX
23119 00003891 7501      JNZ      short SetCount
23120 00003893 48      DEC      AX
23121
23122 SetCount:
23123 00003894 268705 xchg     ax,[es:di]
23124 00003897 9D      ;XCHG     AX,[ES:DI+SF_ENTRY.sf_ref_count]
23125 00003898 C3      POPF
23126                                     retn
23127
23128 ; 18/05/2019 - Retro DOS v4.0
23129
23130 ;-----
23131 ;
23132 ; Procedure Name : DirFromSFT
23133 ;
23134 ; DirFromSFT - locate a directory entry given an SFT.
23135 ;
23136 ; Inputs: ES:DI point to SFT
23137 ;         DS = DOSDATA
23138 ; Outputs:
23139 ;         EXTERR_LOCUS = errLOC_Disk
23140 ;         CurBuf points to buffer
23141 ;         Carry Clear -> operation OK
23142 ;         ES:DI point to entry
23143 ;         ES:BX point to buffer
23144 ;         DS:SI point to SFT
23145 ;         Carry SET -> operation failed
23146 ;         registers trashified
23147 ; Registers modified: ALL
23148 ;-----
23149
23150 ; 04/02/2024 - Retro DOS v5.0
23151 ; PCDOS 7.1 IBMDOS.COM
23152
23153 DirFromSFT:
23154 ;mov      byte [EXTERR_LOCUS],2
23155 ;MOV      byte [EXTERR_LOCUS],errLOC_Disk
23156 00003899 E84ADA call     set_exerr_locus_disk
23157
23158 0000389C 06      push     es
23159 0000389D 57      push     di
23160                                     ; MSDOS 3.3
23161 ;mov      dx,[es:di+1Dh]
23162 ;MOV      dx,[ES:DI+SF_ENTRY.sf_dirsec]
23163 ; MSDOS 6.0
23164 ;mov      dx,[es:[di+1Dh]
23165 0000389E 268B551D MOV      DX,[ES:DI+SF_ENTRY.sf_dirsec+2] ;F.C. >32mb
23166 000038A2 8916[0706] MOV      [HIGH_SECTOR],DX ;F.C. >32mb
23167                                     ; 04/02/2024
23168 000038A6 52      push     dx
23169 ;mov      dx,[es:di+1Bh]
23170 000038A7 268B551B MOV      DX,[ES:DI+SF_ENTRY.sf_dirsec]
23171                                     ; 04/02/2024
23172                                     ; 19/05/2019
23173 ;PUSH      word [HIGH_SECTOR] ;F.C. >32mb

```

```

23174 ; MSDOS 3.3 & MSDOS 6.0
23175 000038AB 52 PUSH DX
23176 000038AC E89B2C call FATREAD_SFT ; ES:BP points to DPB, [THISDRV] set
23177 ; [THISDPB] set
23178 000038AF 5A POP DX
23179 000038B0 8F06[0706] POP word [HIGH_SECTOR] ; F.C. >32mb
23180 000038B4 721E JC short PopDone
23181 ; 22/09/2023
23182 ; XOR AL,AL ; * ; Pre read
23183 ; mov byte [ALLOWED],18h
23184 ; MOV byte [ALLOWED],Allowed_FAIL+Allowed_RETRY ; *
23185 ; call GETBUFFER
23186 ; 22/09/2023
23187 000038B6 E87030 call GETBUFFER ; * ; Pre read
23188 000038B9 7219 JC short PopDone
23189 000038BB 5E pop si
23190 000038BC 1F pop ds ; Get back SFT pointer
23191
23192 ; hkn; SS override
23193 000038BD 36C43E[E205] LES DI,[SS:CURBUF]
23194 ; or byte [es:di+5],4
23195 000038C2 26804D0504 OR byte [ES:DI+BUFFINFO.buf_flags],buf_isDIR
23196 000038C7 89FB MOV BX,DI ; ES:BX point to buffer header
23197 ; ; lea di,[di+16] ; MSDOS 3.3
23198 ; ; lea di,[di+20] ; MSDOS 6.0
23199 ; 04/02/2024
23200 ; lea di,[di+24] ; MSDOS 7.1
23201 000038C9 8D7D18 LEA DI,[DI+BUFINSZ] ; Point to buffer
23202 ; mov al,32
23203 000038CC B020 MOV AL,dir_entry.size
23204 ; mul byte [si+1Fh] ; MSDOS 6.0
23205 000038CE F6641F MUL byte [SI+SF_ENTRY.sf_dirpos]
23206 000038D1 01C7 ADD DI,AX ; Point at the entry
23207 000038D3 C3 retn ; carry is clear
23208
23209 000038D4 5F PopDone:
23210 000038D5 07 pop di
23211 pop es
23212 000038D6 C3 PopDone_retn:
23213 retn
23214
23215 ; -----
23216 ; ** DOS_Commit - Update Directory Entries
23217 ;
23218 ; ENTRY same as DOS_CLOSE (??? BUGBUG - update this jgl)
23219 ; (DS) = DOSGROUP
23220 ; EXIT Same as DOS_CLOSE except ref_count field is not altered
23221 ; USES all but DS
23222 ; -----
23223
23224 ; 14/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
23225 ; DOSCODE:6F72h (MSDOS 5.0, MSDOS.SYS)
23226
23227 ; 04/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
23228 ; DOSCODE:78B8h (PCDOS 7.1 IBMDOS.COM)
23229
23230 DOS_COMMIT:
23231 ; hkn; called from srvcall. DS already set up.
23232 000038D7 C43E[9E05] LES DI,[THISST]
23233 000038DB 268B5D05 mov bx,[es:di+5]
23234 MOV BX,[ES:DI+SF_ENTRY.sf_flags]
23235 ; test bx,0C0h
23236 ; 17/12/2022
23237 test bl,devid_file_clean+devid_device ; clears carry
23238 ; TEST BX,devid_file_clean+devid_device ; clears carry
23239 jnz short PopDone_retn
23240 000038E2 75F2 ; test bx,8000h
23241 ; 17/12/2022
23242 ; test bh,80h
23243 test bh,(sf_isnet>>8) ; 80h
23244 000038E4 F6C780 ; TEST BX,sf_isnet ; 8000h
23245 000038E7 7406 JZ short LOCAL_COMMIT
23246
23247 ; IF NOT Installed
23248 ; transfer NET_COMMIT
23249 ; ELSE
23250 ; mov ax,1107h
23251 MOV AX,(MultNET<<8)|7
23252 000038E9 B80711 int 2Fh ; Multiplex - NETWORK REDIRECTOR - COMMIT REMOTE FILE
23253 000038EC CD2F ; ES:DI -> SFT
23254 ; SFT DPB field -> DPB of drive containing file
23255 ; Return: CF set on error, AX = DOS error code
23256 ; CF clear if successful
23257 localcommit_retn: ; 18/12/2022
23258 retn
23259 000038EE C3 ; ENDIF
23260
23261 ; Perform local commit operation by doing a close but not releasing the SFT.
23262 ; There are three ways we can do this. One is to enter a critical section to
23263 ; protect a potential free. The second is to increment the ref count to mask
23264 ; the close decrementing.
23265 ;
23266 ; The proper way is to let the caller's of close decide if a decrement should
23267 ; be done. We do this by providing another entry into close after the
23268 ; decrement and after the share information release.
23269
23270 ; DOSCODE:6FA0h (MSDOS 6.21, MSDOS.SYS)
23271 ; DOSCODE:6F8Ch (MSDOS 5.0, MSDOS.SYS)
23272 ; 06/08/2025
23273 ; DOSCODE:78D0h (PCDOS 7.1, IBMDOS.COM)
23274
23275 LOCAL_COMMIT:
23276 ; 06/08/2025
23277 ; call ECritDisk
23278 ; MSDOS 6.0
23279 call ECritDisk ; PTM.
23280 000038EF E8F0DF call SetsFTTimes
23281 000038F2 E80C00 MOV AX,-1 ; 0FFFFh
23282 000038F5 B8FFFF call CloseEntry
23283 000038F8 E852FE ; MSDOS 6.0
23284 000038FB 9C PUSHF ; PTM. ; AN000;
23285 000038FC E81519 call DEV_OPEN_SFT ; PTM. increment device count ; AN000;
23286 ; 06/08/2025
23287 ; POPF ; PTM. ; AN000;
23288 ; call LCritDisk ; PTM. ; AN000;
23289 ; 18/12/2022
23290 ; jmp LCritDisk
23291 ; 06/08/2025
23292 jmp NoFree
23293 000038FF EB86 ; localcommit_retn:
23294 ; retn
23295
23296 ; Break <SetsFTTimes - signal a change in the times for an SFT>
23297

```

```

23298 ;-----
23299 ;
23300 ; Procedure Name : SetSFTTimes
23301 ;
23302 ; SetSFTTimes - Examine the flags for a SFT and set the time appropriately.
23303 ; Reflect these times in other SFT's for the same file.
23304 ;
23305 ; Inputs: ES:DI point to SFT
23306 ;         BX = sf_flags set appropriately
23307 ; Outputs: Set sft times to current time if File & dirty & !nodate
23308 ; Registers modified: All except ES:DI, BX, AX
23309 ;-----
23310 ;
23311 ;
23312 ; 04/02/2024 - Retro DOS v5.0
23313 ; PCDOS 7.1 IBMDOS.COM
23314 ;
23315 SetSFTTimes:
23316 ;
23317 ; 04/02/2024
23318 %if 0
23319 ; File clean or device does not get stamped nor disk looked at.
23320 ;
23321 ; test bx,0C0h
23322 ; 17/12/2022
23323 test b1,devid_file_clean+devid_device
23324 ; TEST BX,devid_file_clean+devid_device
23325 ; retnz ; clean or device => no timestamp
23326 jnz short localcommit_retn
23327 ;
23328 ; file and dirty. See if date is good
23329 ;
23330 ; test bx,4000h
23331 ; 17/12/2022
23332 ; test bh,40h
23333 test bh,(sf_close_nodate>>8)
23334 ; TEST BX,sf_close_nodate
23335 ; retnz ; nodate => no timestamp
23336 jnz short localcommit_retn
23337 %else
23338 ; 04/02/2024
23339 ; (PCDOS 7.1 IBMDOS.COM)
23340 ; test bx,40C0h
23341 test bx,sf_close_nodate+devid_file_clean+devid_device
23342 jnz short localcommit_retn
23343 %endif
23344 ;
23345 push ax
23346 push bx
23347 call DATE16 ; Date/Time to AX/DX
23348 ; mov [es:di+0Fh],ax
23349 MOV [ES:DI+SF_ENTRY.sf_date],AX
23350 ; mov [es:di+0Dh],dx
23351 MOV [ES:DI+SF_ENTRY.sf_time],DX
23352 XOR AX,AX
23353 ; if installed
23354 ; call JShare + 14 * 4
23355 call far [JShare+(14*4)] ; 14 = Shsu
23356 ; else
23357 ; call Shsu
23358 ; endif
23359 pop bx
23360 pop ax
23361 ret
23362 ;
23363 ;=====
23364 ; DIRCALL.ASM, MSDOS 6.0, 1991
23365 ;=====
23366 ; 23/07/2018 - Retro DOS v3.0
23367 ; 18/05/2019 - Retro DOS v4.0
23368 ;
23369 ; DOSCODE:6FDAh (MSDOS 6.21, MSDOS.SYS)
23370 ;
23371 ; TITLE DIRCALL - Directory manipulation internal calls
23372 ; NAME DIRCALL
23373 ;
23374 ; ** Low level directory manipulation routines for making removing and
23375 ; verifying local or NET directories
23376 ;
23377 ; DOS_MKDIR
23378 ; DOS_CHDIR
23379 ; DOS_RMDIR
23380 ;
23381 ; Modification history:
23382 ;
23383 ; Created: ARR 30 March 1983
23384 ;
23385 ; BREAK <DOS_MkDir - Make a directory entry>
23386 ;-----
23387 ;
23388 ; Procedure Name : DOS_MkDir
23389 ;
23390 ; Inputs:
23391 ; [WFP_START] Points to WFP string ("d:/" must be first 3 chars, NUL
23392 ; terminated)
23393 ; [CURR_DIR_END] Points to end of Current dir part of string
23394 ; (= -1 if current dir not involved, else
23395 ; Points to first char after last "/" of current dir part)
23396 ; [THISCDS] Points to CDS being used
23397 ; (Low word = -1 if NUL CDS (Net direct request))
23398 ; Function:
23399 ; Make a new directory
23400 ; Returns:
23401 ; Carry Clear
23402 ; No error
23403 ; Carry Set
23404 ; AX is error code
23405 ; error_path_not_found
23406 ; Bad path (not in curr dir part if present)
23407 ; error_bad_curr_dir
23408 ; Bad path in current directory part of path
23409 ; error_access_denied
23410 ; Already exists, device name
23411 ; DS preserved, Others destroyed
23412 ;-----
23413 ; hkn; called from path.asm. DS already set up.
23414 ;
23415 ; 17/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
23416 ; DOSCODE:6FC6h (MSDOS 5.0, MSDOS.SYS)
23417 ;
23418 ; 04/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
23419 ; DOSCODE:78FFh (PCDOS 7.1, IBMDOS.COM)
23420 ;
23421 ;

```

```

23422 DOS_MKDIR:
23423     call    TestNet
23424     JNC     short LOCAL_MKDIR
23425
23426 ;IF NOT Installed
23427 ; transfer NET_MKDIR
23428 ;ELSE
23429     ;mov     ax,1103h
23430     MOV     AX,(MULTINET<<8)|3
23431     int     2Fh ; Multiplex - NETWORK REDIRECTOR - MAKE REMOTE DIRECTORY
23432             ; SS = DOS CS
23433             ; SDA first filename pointer -> fully-qualified directory name
23434             ; SDA CDS pointer -> current directory
23435             ; Return: CF set on error, AX = DOS error code
23436             ; CF clear if successful
23437     retn
23438 ;ENDIF
23439
23440 NODEACCERRJ:
23441     ;mov     ax,5
23442     MOV     AX,error_access_denied
23443 _BadRet:
23444     STC
23445     ;call    LCritDisk
23446     ;retn
23447     ; 18/12/2022
23448     jmp     LCritDisk
23449
23450 PATHNFJ:
23451     call    LCritDisk
23452     jmp     SET_MKND_ERR ; Map the MakeNode error and return
23453
23454 LOCAL_MKDIR:
23455     call    ECritDisk
23456
23457 ; MakeNode requires an SFT to fiddle with. We Use a temp spot (RENTBUF)
23458
23459     MOV     [THISST+2],SS
23460
23461 ;hkn; DOSDATA
23462     MOV     WORD [THISST],RENTBUF
23463
23464 ; NOTE: Need WORD PTR because MASM takes type of
23465 ; TempSFT (byte) instead of type of sf_mft (word).
23466
23467     ;mov     word [RENTBUF+33h],0 ; MSDOS 6.0
23468     MOV     WORD [RENTBUF+SF_ENTRY.sf_MFT],0
23469             ; make sure SHARER won't complain.
23470
23471     ;mov     al,10h
23472     MOV     AL,attr_directory
23473     call    MakeNode
23474     JC      short PATHNFJ
23475     CMP     AX,3
23476     JZ      short NODEACCERRJ ; Can't make a device into a directory
23477     LES     BP,[THISDPB] ; Makenode zaps this
23478     LDS     DI,[CURBUF]
23479     SUB     SI,DI
23480     PUSH    SI ; Pointer to dir_first
23481
23482 ; 04/02/2024
23483 %if 0
23484     ; MSDOS 6.0
23485     ;push    word [DI+8]
23486     PUSH    WORD [DI+BUFFINFO.buf_sector+2] ; F.C. >32mb
23487     ; MSDOS 3.3 & MSDOS 6.0
23488     ;push    word [di+6]
23489     PUSH    WORD [DI+BUFFINFO.buf_sector] ; Sector of new node
23490 %else
23491     ; 04/02/2024
23492     ; (PCDOS 7.1 IBMDOS.COM)
23493     lds     ax,[di+BUFFINFO.buf_sector] ; Sector of new node
23494     push    ds
23495     push    ax
23496 %endif
23497
23498     push    ss
23499     pop     ds
23500
23501 ; 04/02/2024
23502 ; PUSH    word [DIRSTART] ; Parent for .. entry
23503 XOR     AX,AX
23504 ; MOV     [DIRSTART],AX ; Null directory
23505 ;;;
23506 ; Retro DOS v5.0
23507 ; PCDOS 7.1 IBMDOS.COM
23508 mov     dx,ax ; 0
23509 xchg     dx,[DIRSTART_HW] ; Null directory
23510 xchg     ax,[DIRSTART]
23511 ;
23512 ; cmp     word [es:bp+0Fh],0
23513 ; cmp     word [es:bp+DPB.FAT_SIZE],0
23514 ; jnz     short LOCAL_MKDIR_cont ; not FAT32
23515 ; cmp     [es:bp+35h],ax
23516 ; cmp     [es:bp+DPB.ROOT_CLUSTER],ax
23517 ; jne     short LOCAL_MKDIR_cont
23518 ; cmp     [es:bp+37h],dx
23519 ; cmp     [es:bp+DPB.ROOT_CLUSTER+2],dx
23520 ; jne     short LOCAL_MKDIR_cont
23521 ;
23522 ; xor     dx,dx
23523 ; xor     ax,ax
23524 ; dx:ax = 0 for root directory
23525 LOCAL_MKDIR_cont:
23526     push    dx
23527     ;;;
23528     push    ax
23529
23530     call    NEWDIR
23531     JC      short NODEEXISTSPOPDEL ; No room
23532     call    GETENT ; First entry
23533     JC      short NODEEXISTSPOPDEL ; Screw up
23534     LES     DI,[CURBUF]
23535
23536 ; 04/02/2024
23537 %if 0
23538     ; MSDOS 6.0
23539     TEST    byte [ES:DI+BUFFINFO.buf_flags],buf_dirty
23540             ;LB. if already dirty ;AN000;
23541     JNZ     short yesdirty5 ;LB. don't increment dirty count ;AN000;
23542     call    INC_DIRTY_COUNT ;LB. ;AN000;
23543
23544 ; MSDOS 3.3 & MSDOS 6.0
23545 ; or     byte [es:di+5],40h ; 07/12/2022
23546 OR      byte [ES:DI+BUFFINFO.buf_flags],buf_dirty

```



```

23546 %else
23547 ; 04/02/2024
23548 ; (PCDOS 7.1 IBMDOS.COM)
23549 00003999 E82232 call SET_BUF_DIRTY
23550 %endif
23551
23552 yesdirty5:
23553 ;;;add di,16 ; MSDOS 3.3
23554 ;;;add di,20 ; MSDOS 6.0
23555 ; 04/02/2024
23556 ;add di,24 ; PCDOS 7.1
23557 0000399C 83C718 ADD DI,BUFINSIZ ; Point at buffer
23558 0000399F B82E20 MOV AX,202EH ; ". "
23559 ;;;
23560 ; 04/02/2024 (PCDOS 7.1 IBMDOS.COM)
23561 000039A2 8B16[DC0A] mov dx,[DIRSTART_HW]
23562 000039A6 8916[F20A] mov [CLUSTERS_HW],dx
23563 ;;;
23564 000039AA 8B16[C205] MOV DX,[DIRSTART] ; Point at itself
23565 000039AE E8831A call SETDOTENT
23566 000039B1 B82E2E MOV AX,2E2EH ; ".."
23567 000039B4 5A POP DX ; Parent
23568 ;;;
23569 ; 04/02/2024 (PCDOS 7.1 IBMDOS.COM)
23570 000039B5 8F06[F20A] pop word [CLUSTERS_HW]
23571 ;;;
23572 000039B9 E8781A call SETDOTENT
23573 000039BC C42E[8A05] LES BP,[THISDPB]
23574 ; 22/09/2023
23575 ;;;mov byte [ALLOWED],18h
23576 ;MOV byte [ALLOWED],Allowed_FAIL+Allowed_RETRY ; *
23577 000039C0 5A POP DX ; Entry sector
23578 ; MSDOS 6.0
23579 000039C1 8F06[0706] POP word [HIGH_SECTOR] ;F.C. >32mb
23580
23581 ;XOR AL,AL ; * ; Pre read
23582 ;call GETBUFR
23583 ; 22/09/2023
23584 000039C5 E8612F call GETBUFFER ; * ; Pre read
23585 000039C8 7265 JC short NODEEXISTSP
23586 ;;;
23587 ; 04/02/2024 (PCDOS 7.1 IBMDOS.COM)
23588 000039CA A1[DC0A] mov ax,[DIRSTART_HW]
23589 ;;;
23590 000039CD 8B16[C205] MOV DX,[DIRSTART]
23591 000039D1 C53E[E205] LDS DI,[CURBUF]
23592 ;or byte [di+5],4
23593 000039D5 804D0504 OR byte [DI+BUFFINFO.buf_flags],buf_isDIR
23594 000039D9 5E POP SI ; dir_first pointer
23595 000039DA 01FE ADD SI,DI
23596 000039DC 8914 MOV [SI],DX ; dir_entry.dir_first
23597 ;;;
23598 ; 04/02/2024 (PCDOS 7.1 IBMDOS.COM)
23599 000039DE 8944FA mov [si-6],ax ; dir_entry.dir_fclus_hi
23600 ;;;
23601
23602 000039E1 31D2 XOR DX,DX
23603 000039E3 895402 MOV [SI+2],DX ; Zero size
23604 000039E6 895404 MOV [SI+4],DX
23605
23606 DIRUP:
23607 ; MSDOS 6.0
23608 TEST byte [DI+BUFFINFO.buf_flags],buf_dirty
23609 000039ED 7507 ;LB. if already dirty ;AN000;
23610 000039EF E8D831 JNZ short yesdirty6 ;LB. don't increment dirty count ;AN000;
23611 call INC_DIRTY_COUNT ;LB. ;AN000;
23612 ; MSDOS 3.3 & MSDOS 6.0
23613 ;or byte [di+5],40h
23614 000039F2 804D0540 OR byte [DI+BUFFINFO.buf_flags],buf_dirty ; Dirty buffer
23615
23616 000039F6 16 yesdirty6:
23617 000039F7 1F push ss
23618 pop ds
23619 ;;;
23620 000039F8 E871FA call update_fat32_fsinfo
23621 ;;;
23622 000039FB 268A4600 mov al,[es:bp]
23623 ;MOV AL,[ES:BP+DPB.DRIVE] ; mov al,[es:bp+0]
23624 000039FF E89630 call FLUSHBUF
23625 ;mov ax,5
23626 00003A02 B80500 MOV AX,error_access_denied
23627 ;call LCritDisk
23628 ;retn
23629 ; 18/12/2022
23630 00003A05 E907DF jmp LCritDisk
23631
23632 NODEEXISTSPODEL:
23633 ;;;
23634 ; 04/02/2024 (PCDOS 7.1 IBMDOS.COM)
23635 00003A08 5A pop dx ; Parent
23636 ;;;
23637 00003A09 5A POP DX ; Parent
23638 00003A0A 5A POP DX ; Entry sector
23639 ; MSDOS 6.0
23640 00003A0B 8F06[0706] POP word [HIGH_SECTOR] ; F.C. >32mb
23641 00003A0F C42E[8A05] LES BP,[THISDPB]
23642 ; 22/09/2023
23643 ;;;mov byte [ALLOWED],18h
23644 ;MOV byte [ALLOWED],Allowed_FAIL+Allowed_RETRY ; *
23645 ;XOR AL,AL ; * ; Pre read
23646 ;call GETBUFR
23647 ; 22/09/2023
23648 00003A13 E8132F call GETBUFFER ; * ; Pre read
23649 00003A16 7217 JC short NODEEXISTSP
23650 00003A18 C53E[E205] LDS DI,[CURBUF]
23651 ;or byte [di+5],4
23652 00003A1C 804D0504 OR byte [DI+BUFFINFO.buf_flags],buf_isDIR
23653 00003A20 5E POP SI ; dir_first pointer
23654 00003A21 01FE ADD SI,DI
23655 ;sub si,1Ah ; 26
23656 00003A23 83EE1A SUB SI,dir_entry.dir_first;Point back to start of dir entry
23657 00003A26 C604E5 MOV BYTE [SI],0E5H ; Free the entry
23658 00003A29 E8BDFF CALL DIRUP ; Error doesn't matter since erroring anyway
23659
23660 NODEEXISTSP:
23661 JMP NODEACCERRJ ; 10/08/2018
23662
23663 NODEEXISTSP:
23664 POP SI ; Clean stack
23665 JMP short NODEEXISTSP
23666
23667 ; 17/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
23668 ;BREAK <DOS_ChDir -- Verify a directory>
23669 ;-----

```

```

23670 ;
23671 ; Procedure Name : DOS_ChDir
23672 ;
23673 ; Inputs:
23674 ; [WFP_START] Points to WFP string ("d:/" must be first 3 chars, NUL
23675 ; terminated)
23676 ; [CURR_DIR_END] Points to end of Current dir part of string
23677 ; (= -1 if current dir not involved, else
23678 ; points to first char after last "/" of current dir part)
23679 ; [THISCDS] Points to CDS being used May not be NUL
23680 ; Function:
23681 ; validate the path for potential new current directory
23682 ; Returns:
23683 ; NOTE:
23684 ; [SATTRIB] is modified by this call
23685 ; Carry Clear
23686 ; CX is cluster number of the DIR, LOCAL CDS ONLY
23687 ; Caller must NOT set ID fields on a NET CDS.
23688 ; Carry Set
23689 ; AX is error code
23690 ; error_path_not_found
23691 ; Bad path
23692 ; error_access_denied
23693 ; device or file name
23694 ; DS preserved, Others destroyed
23695 ;-----
23696 ;hkn; called from path.asm and dir2.asm. DS already set up.
23697 ;
23698 ; 18/05/2019 - Retro DOS v4.0
23699 ; DOSCODE:70DAh (MSDOS 6.21, MSDOS.SYS)
23700 ;
23701 ; 17/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
23702 ; DOSCODE:70C6h (MSDOS 5.0, MSDOS.SYS)
23703 ;
23704 ; 04/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
23705 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:7A23h
23706 ;
23707 ; (Windows ME IO.SYS - BIOSCODE:7839h)
23708 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:70DAh)
23709 ;
23710 DOS_CHDIR:
23711 call TestNet
23712 JNC short LOCAL_CHDIR
23713
23714 ;IF NOT Installed
23715 ; transfer NET_CHDIR
23716 ;ELSE
23717 ;mov ax,1105h
23718 MOV AX,(MultNet<<8)|5
23719 int 2Fh ; Multiplex - NETWORK REDIRECTOR - CHDIR
23720 ; SS = DOS CS
23721 ; SDA first filename pointer -> fully-qualified directory name
23722 ; SDA CDS pointer -> current directory
23723 ; Return: CF set on error, AX = DOS error code
23724 ; CF clear if successful
23725 retn
23726 ;ENDIF
23727
23728 LOCAL_CHDIR:
23729 call ECritDisk
23730 ; MSDOS 6.0
23731 ;;test word [es:di+43h],2000h
23732 ;TEST word [ES:DI+curdir.flags],curdir_splce ;PTM.
23733 ; 17/12/2022
23734 ;test byte [es:di+44h],20h
23735 test byte [ES:DI+curdir.flags+1],(curdir_splce>>8) ;PTM.
23736 JZ short nojoin ;PTM.
23737 ;mov word [es:di+49h], 0FFFFh
23738 MOV word [ES:DI+curdir.ID],0FFFFh ;PTM.
23739 ;;;
23740 ; 04/02/2024 (PCDOS 7.1 IBMDOS.COM)
23741 ;mov word [es:di+4Bh], 0FFFFh
23742 mov word [es:di+curdir.ID+2],0FFFFh
23743 ;;;
23744 nojoin:
23745 ; MSDOS 3.3 & MSDOS 6.0
23746 MOV byte [NoSetDir],0 ; FALSE
23747 ;mov byte [SATTRIB],16h
23748 MOV byte [SATTRIB],attr_directory+attr_system+attr_hidden
23749 ; Dir calls can find these
23750 ; DOS 3.3 6/24/86 FastOpen
23751 OR byte [FastOpenFlg],FastOpen_Set ; set fastopen flag
23752 call GETPATH
23753 ;
23754 ; 04/02/2024
23755 ;PUSHF ;AN000;
23756 lahf
23757 AND byte [FastOpenFlg],Fast_yes ; clear it all ;AC000;
23758 ;POPF ;AN000;
23759 sahlf
23760
23761 ; DOS 3.3 6/24/86 FastOpen
23762 ; MSDOS 3.3
23763 ;mov byte [FastOpenFlg],0
23764 ;
23765 ;mov ax,3
23766 MOV AX,error_path_not_found
23767 JC short ChDirDone
23768 JNZ short NOTDIRPATH ; Path not a DIR
23769 MOV CX,[DIRSTART] ; Get cluster number
23770 ; 27/06/2024
23771 ;CLC
23772 ChDirDone:
23773 ;call LCritDisk
23774 ;retn
23775 ; 18/12/2022
23776 jmp LCritDisk
23777
23778 00003A77 E995DE
23779 ;BREAK <DOS_RmDir -- Remove a directory>
23780 ;-----
23781 ;
23782 ; Procedure Name : DOS_RmDir
23783 ;
23784 ; Inputs:
23785 ; [WFP_START] Points to WFP string ("d:/" must be first 3 chars, NUL
23786 ; terminated)
23787 ; [CURR_DIR_END] Points to end of Current dir part of string
23788 ; (= -1 if current dir not involved, else
23789 ; points to first char after last "/" of current dir part)
23790 ; [THISCDS] Points to CDS being used
23791 ; (Low word = -1 if NUL CDS (Net direct request))
23792 ; Function:
23793

```



```

23794 ; Remove a directory
23795 ; NOTE: Attempt to remove current directory must be detected by caller
23796 ; Returns:
23797 ; NOTE:
23798 ; [SATTRIB] is modified by this call
23799 ; Carry Clear
23800 ; No error
23801 ; Carry Set
23802 ; AX is error code
23803 ; error_path_not_found
23804 ; Bad path (not in curr dir part if present)
23805 ; error_bad_curr_dir
23806 ; Bad path in current directory part of path
23807 ; error_access_denied
23808 ; device or file name, root directory
23809 ; Bad directory ('.' '..' messed up)
23810 ; DS preserved, Others destroyed
23811 ;-----
23812 ;hkn; called from path.asm. DS already set up.
23813
23814 ; 18/05/2019 - Retro DOS v4.0
23815 ; DOSCODE:711Fh (MSDOS 6.21, MSDOS.SYS)
23816
23817 ; 17/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
23818 ; DOSCODE:710Bh (MSDOS 5.0, MSDOS.SYS)
23819
23820 ; 06/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
23821 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:7A6Bh
23822
23823 DOS_RMDIR:
23824 call TestNet
23825 JNC short LOCAL_RMDIR
23826
23827 ;IF NOT Installed
23828 ; transfer NET_RMDIR
23829 ;ELSE
23830 ;mov ax,1101h
23831 MOV AX,(MULTINET<<8)|1
23832 int 2Fh ; Multiplex - NETWORK REDIRECTOR - REMOVE REMOTE DIRECTORY
23833 ; SS = DOS CS
23834 ; SDA first filename pointer -> fully-qualified directory name
23835 ; SDA CDS pointer -> current directory
23836 ; Return: CF set on error, AX = DOS error code
23837 ; CF clear if successful
23838 retn
23839 ;ENDIF
23840
23841 LOCAL_RMDIR:
23842 call ECritDisk
23843 MOV byte [NoSetDir],0
23844 ;mov byte [SATTRIB],16h
23845 MOV byte [SATTRIB],attr_directory+attr_system+attr_hidden
23846 ; Dir calls can find these
23847 call GETPATH
23848 JC short NOPATH ; Path not found
23849 JNZ short NOTDIRPATH ; Path not a DIR
23850
23851 ; 06/04/2024
23852 %if 0
23853 MOV DI,[DIRSTART]
23854 OR DI,DI ; Root ?
23855 JNZ short rmdir_get_buf ; No
23856 JMP SHORT NOTDIRPATH
23857
23858 %else
23859 ; 06/02/2024 - Retro DOS v5.0
23860 ; PCDOS 7.1 IBMDOS.COM
23861 mov di,[DIRSTART]
23862 or di,di ; Root ?
23863 jnz short LOCAL_RMDIR_cont; No
23864
23865 ;cmp word [DIRSTART_HW],0
23866 [DIRSTART_HW],di ; 0
23867 jz short NOTDIRPATH ; root directory
23868
23869 LOCAL_RMDIR_cont:
23870 ;cmp word [es:bp+0Fh],0
23871 cmp word [es:bp+DPB.FAT_SIZE],0
23872 jnz short rmdir_get_buf ; not FAT32
23873 ;cmp di,[es:bp+35h]
23874 di,[es:bp+DPB.ROOT_CLUSTER]
23875 jne short rmdir_get_buf
23876 mov di,[DIRSTART_HW]
23877 ;cmp di,[es:bp+37h]
23878 cmp di,[es:bp+DPB.ROOT_CLUSTER+2]
23879 jne short rmdir_get_buf
23880 jmp short NOTDIRPATH
23881 %endif
23882
23883 NOPATH:
23884 ;mov ax,3
23885 MOV AX,error_path_not_found
23886 JMP _BadRet
23887
23888 NOTDIRPATHPOP:
23889 POP AX ; MSDOS 6.0 ;F.C. >32mb
23890 POP AX
23891 NOTDIRPATHPOP2:
23892 POP AX
23893 NOTDIRPATH:
23894 JMP NODEACCERRJ
23895
23896 rmdir_get_buf:
23897 LDS DI,[CURBUF]
23898 SUB BX,DI ; Compute true offset
23899 PUSH BX ; Save entry pointer
23900
23901 ; 06/04/2024
23902 %if 0
23903 ; MSDOS 6.0
23904 ;push word [di+8]
23905 PUSH WORD [DI+BUFFINFO.buf_sector+2] ;F.C. >32mb
23906
23907 ; MSDOS 3.3 (& MSDOS 6.0)
23908 ;push word [di+6]
23909 PUSH WORD [DI+BUFFINFO.buf_sector] ; save sector number
23910
23911 %else
23912 ; 06/02/2024 - Retro DOS v5.0
23913 ; PCDOS 7.1 IBMDOS.COM
23914 ;les di,[di+6]
23915 les di,[di+BUFFINFO.buf_sector]
23916 push es
23917 push di

```

```

23918 %endif
23919
23920 ;hkn; SS is DOSDATA
23921 ;context DS
23922 00003AD8 16 push ss
23923 00003AD9 1F pop ds
23924 ;context ES
23925 00003ADA 16 push ss
23926 00003ADB 07 pop es
23927
23928 ;hkn; NAME1 is in DOSDATA
23929 00003ADC BF[4B05] MOV DI,NAME1
23930 ;MOV AL,'?' ; 3Fh
23931 ;MOV CX,11
23932 ;REP STOSB
23933 ;XOR AL,AL
23934 ;STOSB ; Null terminate it
23935 ; 27/06/2024
23936 00003ADF B83F00 mov ax,3Fh
23937 00003AE2 B90A00 mov cx,10
23938 00003AE5 F3AA rep stosb ; al = "?"
23939 00003AE7 AB stosw ; ah = 0
23940 ;
23941 00003AE8 E8C812 call STARTSRCH ; Set search
23942 00003AEB E8B10D call GETENTRY ; Get start of directory
23943 00003AEE 72D6 JC short NOTDIRPATHPOP ; Screw up
23944 00003AF0 8E1E[E405] MOV DS,[CURBUF+2]
23945 00003AF4 89DE MOV SI,BX
23946 00003AF6 AD LODSW
23947 ;CMP AX,(' ' SHL 8) OR '.' ; First entry '..'?
23948 00003AF7 3D2E20 cmp ax,202Eh ; ". "
23949 00003AFA 75CA JNZ short NOTDIRPATHPOP ; Nope
23950 ;add si,30
23951 00003AFC 83C61E ADD SI,dir_entry.size-2 ; Next entry
23952 00003AFF AD LODSW
23953 ;CMP AX,('.', SHL 8) OR '.' ; Second entry '..'?
23954 ;cmp ax, '..'
23955 00003B00 3D2E2E cmp ax,2E2Eh
23956 00003B03 75C1 JNZ short NOTDIRPATHPOP ; Nope
23957
23958 ;hkn; SS is DOSDATA
23959 ;context DS
23960 00003B05 16 push ss
23961 00003B06 1F pop ds
23962 00003B07 C706[4803]0200 MOV word [LASTENT],2 ; Skip . and ..
23963 00003B0D E88F0D call GETENTRY ; Get next entry
23964 00003B10 72B4 JC short NOTDIRPATHPOP ; Screw up
23965 ;mov byte [ATTRIB],16h
23966 00003B12 C606[6B05]16 MOV byte [ATTRIB],attr_directory+attr_hidden+attr_system
23967 00003B17 E83B0C call SRCH ; Do a search
23968 00003B1A 73AA JNC short NOTDIRPATHPOP ; Found another entry!
23969 00003B1C 803E[4A03]00 CMP byte [FAILERR],0
23970 00003B21 75A3 JNZ short NOTDIRPATHPOP ; Failure of search due to I 24 FAIL
23971 00003B23 C42E[8A05] LES BP,[THISDPB]
23972 ;;;
23973 ; 06/02/2024 - Retro DOS v5.0
23974 ; PCDOS 7.1 IBMDOS.COM
23975 00003B27 8B1E[DC0A] mov bx,[DIRSTART_HW]
23976 00003B2B 891E[E80A] mov [CLUSTNUM_HW],bx
23977 ;;;
23978 00003B2F 8B1E[C205] MOV BX,[DIRSTART]
23979 00003B33 E89A20 call RELEASE ; Release data in sub dir
23980 00003B36 728E JC short NOTDIRPATHPOP ; Screw up
23981 ;;;
23982 ; 06/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
23983 00003B38 E831F9 call update_fat32_fsinfo
23984 ;;;
23985 00003B3B 5A POP DX ; Sector # of entry
23986 00003B3C 8F06[0706] POP word [HIGH_SECTOR] ; MSDOS 6.0 ; F.C. >32mb
23987 ; 22/09/2023
23988 ;mov byte [ALLOWED],18h
23989 ;MOV byte [ALLOWED],Allowed_FAIL+Allowed_RETRY ; *
23990 ;XOR AL,AL ; * ; Pre read
23991 ;call GETBUFR ; Get sector back
23992 00003B40 E8E62D call GETBUFR ; * ; Pre Read
23993 00003B43 7283 JC short NOTDIRPATHPOP2 ; Screw up
23994
23995 rmdir_fde: ; 06/02/2024
23996 00003B45 C53E[E205] LDS DI,[CURBUF]
23997 ;or byte [di+5],4
23998 00003B49 804D0504 OR byte [DI+BUFFINFO.buf_flags],buf_isDIR
23999 00003B4D 5B POP BX ; Pointer to start of entry
24000 00003B4E 01FB ADD BX,DI ; Corrected
24001 00003B50 C607E5 MOV BYTE [BX],0E5H ; Free the entry
24002
24003 ;DOS 3.3 FastOpen 6/16/86 F.C.
24004 00003B53 1E PUSH DS
24005
24006 ;hkn; SS is DOSDATA
24007 ;context DS
24008 00003B54 16 push ss
24009 00003B55 1F pop ds
24010
24011 ; MSDOS 6.0
24012 00003B56 E8E6F1 call FastOpen_Delete ; call fastopen to delete an entry
24013
24014 ; ; MSDOS 3.3
24015 ;_FastOpen_Delete:
24016 ; push ax
24017 ; mov si,[WFP_START]
24018 ; mov bx,FastTable
24019 ; ;mov al,3 ; FONC_delete
24020 ; mov al,FONC_delete
24021 ; call far [BX+2] ; FastTable+2
24022 ; pop ax
24023
24024 00003B59 1F POP DS
24025 ;DOS 3.3 FastOpen 6/16/86 F.C.
24026
24027 00003B5A E98CFE JMP DIRUP ; In MKDIR, dirty buffer and flush
24028
24029 ;=====
24030 ; DISK.ASM, MSDOS 6.0, 1991
24031 ;=====
24032 ; 23/07/2018 - Retro DOS v3.0
24033 ; 04/05/2019 - Retro DOS v4.0
24034 ; 06/02/2024 - Retro DOS v5.0
24035
24036 ; TITLE DISK - Disk utility routines
24037 ; NAME Disk
24038
24039 ;** Low level Read and write routines for local SFT I/O on files and devs
24040 ;
24041 ; SWAPCON

```

```

24042 ; SWAPBACK
24043 ; DOS_READ
24044 ; DOS_WRITE
24045 ; get_io_sft
24046 ; DirRead
24047 ; FIRSTCLUSTER
24048 ; SET_BUF_AS_DIR
24049 ; FATSecRd
24050 ; DREAD
24051 ; CHECK_WRITE_LOCK
24052 ; CHECK_READ_LOCK
24053 ;
24054 ; Revision history:
24055 ;
24056 ;         A000   version 4.00   Jan. 1988
24057 ;
24058 ;-----
24059 ; M065 : B#5276. On raw read/write of a block of characters if a critical
24060 ;         error happens, DOS retries the entire block assuming that
24061 ;         zero characters were transferred. Modified the code to take
24062 ;         into account the number of characters transfered before
24063 ;         retrying the operation.
24064 ;
24065 ;-----
24066 ;
24067 ;
24068 ;Installed = TRUE
24069 ;
24070 ;Break      <SwapCon, Swap Back - Old-style I/O to files>
24071 ;
24072 ;
24073 ; **** Drivers for file input from devices ****
24074 ;-----
24075 ; Indicate that there is no more I/O occurring through another SFT outside
24076 ; of handles 0 and 1
24077 ;
24078 ; Inputs: DS is DOSDATA
24079 ; Outputs: CONSWAP is set to false.
24080 ; Registers modified: none
24081 ;-----
24082 ;
24083 ; IBMDOS.COM (MSDOS 3.3) - Offset 3CF8h
24084 ;
24085 ; DOSCODE:71E3h (MSDOS 6.21, MSDOS.SYS)
24086 ; 04/05/2019 - Retro DOS v4.0
24087 ;
24088 ; DOSCODE:71CFh (MSDOS 5.0, MSDOS.SYS)
24089 ; 17/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
24090 ;
24091 ;
24092 00003B5D C606[5703]00 SWAPBACK:
24093 00003B62 C3          MOV     BYTE [CONSWAP],0      ; signal no conswaps
24094 ;
24095 ;-----
24096 ;
24097 ; Procedure Name : SWAPCON
24098 ;
24099 ; Copy ThisSFT to CONSFT for use by the 1-12 primitives.
24100 ;
24101 ; Inputs: ThisSFT as the sft of the desired file
24102 ;         DS is DOSDATA
24103 ; Outputs: CONSWAP is set. CONSFT = ThisSFT.
24104 ; Registers modified: none
24105 ;-----
24106 ;
24107 ; SWAPCON:
24108 ; MSDOS 3.3
24109 ; push es
24110 ; push di
24111 ; mov byte [CONSWAP],1
24112 ; les di,[THISFT]
24113 ; mov word [CONSFT],di
24114 ; mov word [CONSFT+2],es
24115 ; pop di
24116 ; pop es
24117 ; retn
24118 ;
24119 ; MSDOS 6.0
24120 00003B63 C606[5703]01 mov     byte [CONSWAP],1      ; ConSwap = TRUE
24121 00003B68 50          push    ax
24122 00003B69 A1[9E05]    mov     ax,[THISFT]
24123 00003B6C A3[E605]    mov     [CONSFT],ax
24124 00003B6F A1[A005]    mov     ax,[THISFT+2]
24125 00003B72 A3[E805]    mov     [CONSFT+2],ax
24126 00003B75 58          pop     ax
24127 00003B76 C3          retn
24128 ;
24129 ; DOSCODE:71FDh (MSDOS 6.21, MSDOS.SYS)
24130 ; 04/05/2019 - Retro DOS v4.0
24131 ;
24132 ;Break      <DOS_READ -- MAIN READ ROUTINE AND DEVICE IN ROUTINES>
24133 ;-----
24134 ;
24135 ; Inputs:
24136 ; ThisSFT set to the SFT for the file being used
24137 ; [DMAADD] contains transfer address
24138 ; CX = No. of bytes to read
24139 ; DS = DOSDATA
24140 ; Function:
24141 ; Perform read operation
24142 ; Outputs:
24143 ; Carry clear
24144 ; SFT Position and cluster pointers updated
24145 ; CX = No. of bytes read
24146 ; ES:DI point to SFT
24147 ; Carry set
24148 ; AX is error code
24149 ; CX = 0
24150 ; ES:DI point to SFT
24151 ; DS preserved, all other registers destroyed
24152 ;-----
24153 ;
24154 ;hkn; called from fcbio.asm, handle.asm and dev.asm. DS is be set up.
24155 ;
24156 ;
24157 ; DOSCODE:71E9h (MSDOS 5.0, MSDOS.SYS)
24158 ; 17/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
24159 ;
24160 ; 07/02/2024
24161 ; 06/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
24162 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:7B76h
24163 ;
24164 ; (Windows ME IO.SYS - BIOSCODE:7997h)
24165 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:71FDh)

```

```

24166
24167
24168 00003B77 C43E[9E05]
24169
24170
24171
24172
24173 00003B7B 268A4502
24174
24175
24176
24177
24178 00003B7F 2403
24179
24180
24181 00003B81 3C01
24182 00003B83 7503
24183 00003B85 E96406
24184
24185
24186 00003B88 E82405
24187 00003B8B E30B
24188 00003B8D E8B2DC
24189 00003B90 7408
24190
24191
24192
24193
24194
24195 00003B92 B80811
24196 00003B95 CD2F
24197
24198
24199
24200
24201 00003B97 C3
24202
24203
24204
24205
24206
24207
24208 00003B98 F8
24209 00003B99 C3
24210
24211
24212
24213
24214 00003B9A 26F6450580
24215 00003B9F 750E
24216
24217
24218 00003BA1 C606[2303]02
24219 00003BA6 E839DD
24220 00003BA9 E8F705
24221
24222
24223
24224
24225
24226 00003BAC E960DD
24227
24228
24229
24230
24231
24232
24233
24234 00003BAF C606[2303]04
24235
24236 00003BB4 268A5D05
24237 00003BB8 C43E[2C03]
24238
24239 00003BBC F6C340
24240 00003BBF 7407
24241
24242 00003BC1 F6C304
24243 00003BC4 7405
24244 00003BC6 30C0
24245
24246
24247
24248
24249 00003BC8 E93F01
24250
24251
24252
24253
24254
24255
24256 00003BCB F6C320
24257 00003BCE 7508
24258
24259 00003BD0 F6C301
24260 00003BD3 7458
24261 00003BD5 E96701
24262
24263
24264 00003BD8 06
24265 00003BD9 1F
24266
24267
24268
24269
24270
24271
24272
24273 00003BDA 36F606[5B0F]01
24274 00003BE0 7408
24275 00003BE2 F6C301
24276 00003BE5 7403
24277 00003BE7 E9A800
24278
24279
24280
24281
24282
24283
24284
24285
24286
24287
24288
24289

DOS_READ:
    LES     DI,[THISSFT]

; Verify that the sft has been opened in a mode that allows reading.

    ;mov     al,[es:di+2]
    MOV     AL,[ES:DI+SF_ENTRY.sf_mode]
    ;and     al,0Fh ; (MSDOS 5.0 & 6.22)
    AND     AL,access_mask
    ; 06/02/2024 - Retro DOS v5.0
    ; (PCDOS 7.1 & Win Me)
    and     al,3 ; open_mode_mask ?

    ;cmp     al,1
    CMP     AL,open_for_write
    JNE     short READ_NO_MODE ; Is read or both
    jmp     SET_ACC_ERR

READ_NO_MODE:
    call    SETUP
    JCXZ    NoIORet ; no bytes to read - fast return
    call    IsSFTNet
    JZ      short LOCAL_READ

; IF NOT Installed
; transfer NET_READ
; ELSE
    ;mov     ax,1108h
    MOV     AX,(MultNET<<8)|8
    int     2Fh ; Multiplex - NETWORK REDIRECTOR - READ FROM REMOTE FILE
    ; ES:DI -> SFT
    ; SFT DPB field -> DPB of drive containing file
    ; CX = number of bytes, SS = DOS CS, SDA DTA field -> user buffer
    ; Return: CF set on error, CX = bytes read
    retn
; ENDIF

; The user ended up requesting 0 bytes of input. We do nothing for this case
; except return immediately.

NoIORet:
    CLC
    retn

LOCAL_READ:
    ;test    word [es:di+5],80h
    ;TEST    word [ES:DI+SF_ENTRY.sf_flags],devid_device ; Check for named device I/O
    test    byte [ES:DI+SF_ENTRY.sf_flags],devid_device ; 02/06/2019
    JNZ     short READDEV

    ;mov     byte [EXTERR_LOCUS],2
    MOV     byte [EXTERR_LOCUS],errLOC_Disk
    call    ECritDisk
    call    DISKREAD

critexit:
    ;call    LCritDisk
    ;retn
    ; 16/12/2022
    jmp     LCritDisk

; We are reading from a device. Examine the status of the device to see if we
; can short-circuit the I/O. If the device is in the EOF state or if it is the
; null device, we can safely indicate no transfer.

READDEV:
    ;mov     byte [EXTERR_LOCUS],4
    MOV     byte [EXTERR_LOCUS],errLOC_SerDev
    ;mov     b1,[es:di+5]
    MOV     BL,[ES:DI+SF_ENTRY.sf_flags]
    LES     DI,[DMAADD]
    ;test    b1,40h
    test    BL,devid_device_EOF ; End of file?
    JZ      short ENDRDDEVJ3
    ;test    b1,4
    test    BL,devid_device_null ; NUL device?
    JZ      short TESTRAW ; NO
    XOR     AL,AL ; Indicate EOF by setting zero

ENDRDDEVJ3:
    ; 17/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility!)
    JMP     short ENDRDDEVJ2
    ; 16/12/2022
    jmp     ENDRDDEV ; 04/05/2019

; We need to hit the device. Figure out if we do a raw read or we do the
; bizarre std_con_string_input.

TESTRAW:
    ;test    b1,20h
    test    BL,devid_device_raw ; Raw mode?
    JNZ     short DVRDRAW ; Yes, let the device do all local editing
    ;test    b1,1
    test    BL,devid_device_con_in; Is it console device?
    JZ      short NOTRDCON
    JMP     READCON

DVRDRAW:
    PUSH    ES
    POP     DS ; xaddr to DS:DI

    ; 04/05/2019 - Retro DOS v4.0

    ; MSDOS 6.0

; SR;
; Check for win386 presence -- if present, do polled read of characters

    test    byte [ss:Iswin386],1 ; 19/05/2019
    jz      short ReadRawRetry ; not present
    test    b1,devid_device_con_in; is it console device?
    jz      short ReadRawRetry ; no, do normal read
    jmp     do_polling ; yes, do win386 polling loop

ReadRawRetry:
    ; 07/02/2024
    %if 0
        MOV     BX,DI ; DS:BX transfer addr
        ; 06/02/2024 ; *
        ; XOR     AX,AX ; Media Byte, unit = 0
        ; MOV     DX,AX ; Start at 0
        ; ; 06/02/2024
        ; cwd
        ; call    SETREAD
    %endif

```

```

24290         ; 06/02/2024 ; *
24291         call    SETREAD_X
24292 %else
24293 00003BEA E81B17        call    SETREAD_XJ
24294 %endif
24295
24296 00003BED 1E            PUSH    DS                ; Save Seg part of Xaddr
24297
24298 ;hkn; SS override
24299 00003BEE 36C536[9E05]  LDS     SI,[SS:THISSFT]
24300 00003BF3 E89C16        call    DEVIOCALL
24301 00003BF6 89FA          MOV     DX,DI                ; DS:DX is preserved by INT 24
24302 00003BF8 B486          MOV     AH,86H            ; Read error
24303
24304 ;hkn; SS override
24305 00003BFA 368B3E[5D03]  MOV     DI,[SS:DEVCALL_REQSTAT]
24306 ; MSDOS 3.3
24307 ;test di,8000h
24308 ;jz short CRDROK
24309 ; MSDOS 6.0
24310 00003BFF 09FF          or      di,di
24311 00003C01 7920          jns     short CRDROK        ; no errors
24312 ; MSDOS 3.3 (& MSDOS 6.0)
24313 00003C03 E81D24        call    CHARHARD
24314
24315 ; 06/02/2024 - Retro DOS v5.0
24316 %if 0
24317 MOV     DI,DX                ; DS:DI is Xaddr
24318 ; 04/05/2019
24319 ; MSDOS 6.0
24320 add     di,[ss:CALLSCNT]      ; update ptr and count to reflect the M065
24321 sub     cx,[ss:CALLSCNT]      ; number of chars xferred M065
24322 %else
24323 00003C06 368B3E[6C03]  mov     di,[ss:CALLSCNT]
24324 00003C0B 29F9          sub     cx,di                ; update transfer count
24325 00003C0D 01D7          add     di,dx                ; update pointer
24326 %endif
24327 ; MSDOS 3.3 (& MSDOS 6.0)
24328 00003C0F 08C0          OR      AL,AL
24329 00003C11 7410          JZ      short CRDROK        ; Ignore
24330 00003C13 3C03          CMP     AL,3
24331 00003C15 7403          JZ      short CRDFERR      ; fail.
24332 00003C17 1F          POP     DS                ; Recover saved seg part of Xaddr
24333 00003C18 EBD0          JMP     short ReadRawRetry  ; Retry
24334
24335 ; We have encountered a device-driver error. We have informed the user of it
24336 ; and he has said for us to fail the system call.
24337
24338 CRDFERR:
24339 00003C1A 5F          POP     DI                ; Clean stack
24340 DEVIOFERR:
24341
24342 ;hkn; SS override
24343 00003C1B 36C43E[9E05]  LES     DI,[SS:THISSFT]
24344 00003C20 E9C705        jmp     SET_ACC_ERR_DS
24345
24346 CRDROK:
24347 00003C23 5F          POP     DI                ; Chuck saved seg of Xaddr
24348 00003C24 89D7          MOV     DI,DX
24349
24350 ;hkn; SS override
24351 00003C26 36033E[6C03]  ADD     DI,[ss:CALLSCNT]      ; Amount transferred
24352 ;JMP SHORT ENDRDDEVJ3
24353 ; 16/12/2022
24354 00003C2B EB63          jmp     short ENDRDDEVJ2
24355
24356 ; We are going to do a cooked read on some character device. There is a
24357 ; problem here, what does the data look like? Is it a terminal device, line
24358 ; CR line CR line CR, or is it file data, line CR LF line CR LF? Does it have
24359 ; a ^Z at the end which is data, or is the ^Z not data? In any event we're
24360 ; going to do this: Read in pieces up to CR (CRs included in data) or ^Z (^Z
24361 ; included in data). this "simulates" the way con works in cooked mode
24362 ; reading one line at a time. With file data, however, the lines will look
24363 ; like, LF line CR. This is a little weird.
24364
24365 NOTRDCON:
24366 ;MOV     AX,ES
24367 ;MOV     DS,AX
24368 ; 07/02/2024
24369 00003C2D 06          push    es
24370 00003C2E 1F          pop     ds
24371
24372 ; 07/02/2024
24373 %if 0
24374 MOV     BX,DI
24375 ; 06/02/2024 ; *
24376 ;;XOR     DX,DX
24377 ;;MOV     AX,DX
24378 ;; 06/02/2024
24379 ;xor     ax,ax
24380 ;cwr
24381 PUSH     CX
24382 MOV     CX,1
24383 ;call    SETREAD
24384 ; 06/02/2024 ; *
24385 call    SETREAD_X
24386 POP     CX
24387 %else
24388 00003C2F 51          push    cx
24389 00003C30 B90100        mov     cx,1
24390 00003C33 E8D216        call    SETREAD_XJ
24391 00003C36 59          pop     cx
24392 %endif
24393
24394 ;hkn; SS override
24395 00003C37 36C536[9E05]  LDS     SI,[SS:THISSFT]
24396 ;lds     si,[si+7]
24397 00003C3C C57407        LDS     SI,[SI+SF_ENTRY.sf_devptr]
24398
24399 00003C3F E8C121        call    DSKSTATCHK
24400 00003C42 E85016        call    DEVIOCALL2
24401 00003C45 57          PUSH     DI                ; Save "count" done
24402 00003C46 B486          MOV     AH,86H
24403
24404 ;hkn; SS override
24405 00003C48 368B3E[5D03]  MOV     DI,[SS:DEVCALL_REQSTAT]
24406
24407 ; MSDOS 3.3
24408 ;test di,8000h
24409 ;jz short CRDOK
24410 ; MSDOS 6.0
24411 00003C4D 09FF          or      di,di
24412 00003C4F 7917          jns     short CRDOK
24413

```

```

24414 00003C51 E8CF23      call    CHARHARD
24415 00003C54 5F        POP     DI
24416
24417
24418 00003C55 36C706[6C03]0100      ;hkn; SS override
24419 00003C5C 3C01      MOV     word [SS:CALLSCNT],1
24420 00003C5E 74DF      CMP     AL,1
24421 00003C60 3C03      JZ      short DVRDLP          ; Retry
24422 00003C62 74B7      CMP     AL,3
24423 00003C64 30C0      JZ      short DEVIOFERR      ; FAIL
24424 00003C66 EB12      XOR     AL,AL                ; Ignore, Pick some random character
24425                                JMP     SHORT DVRDIGN
24426
24427 00003C68 5F        CRDOK:
24428                                POP     DI
24429
24430 00003C69 36833E[6C03]01      ;hkn; SS override
24431                                CMP     word [SS:CALLSCNT],1
24432                                ; 17/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility!)
24433                                JNZ     short ENDRDDEVJ2
24434                                ; 16/12/2022
24435                                ;jnz     short ENDRDDEV ; 24/07/2019
24436 00003C71 1E        PUSH    DS
24437
24438
24439 00003C72 368E1E[6A03]      ;hkn; SS override
24440 00003C77 8A05      MOV     DS,[SS:CALLXAD+2]
24441 00003C79 1F        MOV     AL,[DI]                ; Get the character we just read
24442                                POP     DS
24443
24444
24445 00003C7A 36FF06[6803]      ;hkn; SS override
24446 00003C7F 36C706[5D03]0000      INC     word [SS:CALLXAD]      ; Next character
24447 00003C86 47        MOV     word [SS:DEVCALL_REQSTAT],0
24448 00003C87 3C1A      INC     DI                    ; Next character
24449                                CMP     AL,1Ah                ; ^Z?
24450                                ; 17/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility!)
24451                                JZ      short ENDRDDEVJ2      ; Yes, done zero set (EOF)
24452                                ; 16/12/2022
24453                                ;jz     short ENDRDDEV ; 24/07/2019
24454 00003C8B 3C0D      CMP     AL,c_CR ; 0Dh        ; CR?
24455 00003C8D E0B0      LOOPNZ  DVRDLP                ; Loop if no, else done
24456 00003C8F 40        INC     AX                    ; Resets zero flag so NOT EOF, unless
24457                                ; AX=FFFF which is not likely
24458
24459
24460
24461 00003C90 EB78      ENDRDDEVJ2:
24462                                ; 16/12/2022
24463                                JMP     short ENDRDDEV          ; changed short to long for win386
24464                                ; 17/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
24465                                jmp     ENDRDDEV
24466
24467                                ; 04/05/2019
24468
24469                                ; MSDOS 6.0
24470
24471
24472
24473
24474
24475
24476
24477
24478
24479
24480
24481
24482
24483
24484
24485
24486 00003C92 E87316      ;SR;
24487                                ;Polling code for raw read on CON when WIN386 is present
24488                                ;
24489                                ;At this point -- ds:di is transfer address
24490                                ; cx is count
24491
24492
24493
24494
24495
24496
24497
24498
24499
24500
24501 00003CA4 26F6470480      do_polling:
24502                                ; 07/02/2024
24503                                %if 0
24504                                mov     bx,di                ;ds:bx is xfer address
24505                                ; 06/02/2024 ; *
24506                                xor     ax,ax
24507                                ;mov    dx,ax
24508                                ; 06/02/2024
24509                                ;cld
24510                                ;call   SETREAD                ;prepare device packet
24511                                ; 06/02/2024 ; *
24512                                call   SETREAD_X
24513                                %else
24514                                call   SETREAD_XJ
24515                                %endif
24516
24517
24518
24519
24520
24521
24522
24523
24524
24525
24526
24527
24528
24529 00003CC5 26C6470204      do_io:
24530 00003CCA 26C747120100      ;Change read to a NON-DESTRUCTIVE READ, NO WAIT
24531 00003CD0 1E        mov     byte [es:bx+2],DEVDRDND ; 5 ;Change command code
24532 00003CD1 36C536[9E05]      push    ds
24533 00003CD6 E8B915      lds     si,[ss:THISSFT]
24534                                call   DEVIOCALL
24535
24536
24537
24538
24539
24540
24541
24542
24543
24544
24545
24546
24547
24548
24549
24550
24551
24552
24553
24554
24555
24556
24557
24558
24559
24560
24561
24562
24563
24564
24565
24566
24567
24568
24569
24570
24571
24572
24573
24574
24575
24576
24577
24578
24579
24580
24581
24582
24583
24584
24585
24586
24587
24588
24589
24590
24591
24592
24593
24594
24595
24596
24597
24598
24599
24600
24601
24602
24603
24604
24605
24606
24607
24608
24609
24610
24611
24612
24613
24614
24615
24616
24617
24618
24619
24620
24621
24622
24623
24624
24625
24626
24627
24628
24629
24630
24631
24632
24633
24634
24635
24636
24637
24638
24639
24640
24641
24642
24643
24644
24645
24646
24647
24648
24649
24650
24651
24652
24653
24654
24655
24656
24657
24658
24659
24660
24661
24662
24663
24664
24665
24666
24667
24668
24669
24670
24671
24672
24673
24674
24675
24676
24677
24678
24679
24680
24681
24682
24683
24684
24685
24686
24687
24688
24689
24690
24691
24692
24693
24694
24695
24696
24697
24698
24699
24700
24701
24702
24703
24704
24705
24706
24707
24708
24709
24710
24711
24712
24713
24714
24715
24716
24717
24718
24719
24720
24721
24722
24723
24724
24725
24726
24727
24728
24729
24730
24731
24732
24733
24734
24735
24736
24737
24738
24739
24740
24741
24742
24743
24744
24745
24746
24747
24748
24749
24750
24751
24752
24753
24754
24755
24756
24757
24758
24759
24760
24761
24762
24763
24764
24765
24766
24767
24768
24769
24770
24771
24772
24773
24774
24775
24776
24777
24778
24779
24780
24781
24782
24783
24784
24785
24786
24787
24788
24789
24790
24791
24792
24793
24794
24795
24796
24797
24798
24799
24800
24801
24802
24803
24804
24805
24806
24807
24808
24809
24810
24811
24812
24813
24814
24815
24816
24817
24818
24819
24820
24821
24822
24823
24824
24825
24826
24827
24828
24829
24830
24831
24832
24833
24834
24835
24836
24837
24838
24839
24840
24841
24842
24843
24844
24845
24846
24847
24848
24849
24850
24851
24852
24853
24854
24855
24856
24857
24858
24859
24860
24861
24862
24863
24864
24865
24866
24867
24868
24869
24870
24871
24872
24873
24874
24875
24876
24877
24878
24879
24880
24881
24882
24883
24884
24885
24886
24887
24888
24889
24890
24891
24892
24893
24894
24895
24896
24897
24898
24899
24900
24901
24902
24903
24904
24905
24906
24907
24908
24909
24910
24911
24912
24913
24914
24915
24916
24917
24918
24919
24920
24921
24922
24923
24924
24925
24926
24927
24928
24929
24930
24931
24932
24933
24934
24935
24936
24937
24938
24939
24940
24941
24942
24943
24944
24945
24946
24947
24948
24949
24950
24951
24952
24953
24954
24955
24956
24957
24958
24959
24960
24961
24962
24963
24964
24965
24966
24967
24968
24969
24970
24971
24972
24973
24974
24975
24976
24977
24978
24979
24980
24981
24982
24983
24984
24985
24986
24987
24988
24989
24990
24991
24992
24993
24994
24995
24996
24997
24998
24999

```



```

24538 00003CDD 268B7F03      mov     di,[es:bx+SRHEAD.REQSTAT] ;get returned status
24539 00003CE1 F7C70080      test    di,STERR ; 8000h      ;was there an error during read?
24540                          ;jz      short next_char      ;no,read next character
24541                          ; 07/02/2024
24542 00003CE5 75C7          jnz     short invoke_charhard
24543
24544                          ; 07/02/2024
24545                          %if 0
24546                          ;invoke charhard      ;invoke int 24h handler
24547                          call    CHARHARD
24548                          mov     di,dx      ;restore di
24549                          or      al,al      ;
24550                          jz      short pop_done_read ;ignore by user,assume read is done
24551                          cmp     al,3
24552                          jz      short devrderr ;user issued a 'fail',indicate error
24553                          pop     ds
24554                          jmp     short do_io ;user issued a retry
24555                          %endif
24556
24557                          next_char:
24558                          pop     ds
24559 00003CE8 89D7          mov     di,dx
24560 00003CEA 49          dec     cx      ;decrement count
24561                          ;jcxz   done_read      ;all characters read in
24562                          ; 07/02/2024
24563 00003CEB 7418          jz      short done_read
24564 00003CED 26FF470E      inc     word [es:bx+14] ;update transfer address
24565 00003CF1 EBA2          jmp     short do_io ;read next character in
24566
24567                          devrderr:
24568 00003CF3 5F          pop     di      ;discard segment address
24569 00003CF4 36C43E[9E05]    les     di,[ss:THISSFT]
24570                          ;transfer SET_ACC_ERR_DS ;indicate error
24571 00003CF9 E9EE04          jmp     SET_ACC_ERR_DS
24572
24573                          no_char:
24574                          ;Since no character is available, we let win386 switch the VM out
24575
24576 00003CFC 50          push    ax
24577 00003CFD B484          mov     ah,84h ; Microsoft Networks - KEYBOARD BUSY LOOP
24578 00003CFF CD2A          int     2Ah ;indicate idle to WIN386
24579
24580                          ;When control returns from WIN386, we continue the raw read
24581
24582 00003D01 58          pop     ax
24583 00003D02 EB91          jmp     short do_io ; 27/06/2024
24584
24585                          pop_done_read:
24586                          pop     ds
24587                          done_read:
24588 00003D05 36033E[6C03]    add     di,[ss:CALLSCNT] ; 19/05/2019
24589
24590                          ; 16/12/2022
24591
24592                          ;jmp     ENDRDDEVJ3 ;jump back to normal DOS raw read exit
24593                          ;jmp     ENDRDDEV ; 04/05/2019
24594
24595                          ; 04/05/2019 - Retro DOS v4.0
24596                          ENDRDDEV:
24597 00003D0A 16          push    ss
24598 00003D0B 1F          pop     ds
24599 00003D0C EB1F          jmp     short endrddev1
24600
24601                          ; 17/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
24602                          ;jmp     ENDRDDEVJ3 ;jump back to normal DOS raw read exit
24603
24604                          TRANBUF:
24605 00003D0E AC          LODSB
24606 00003D0F AA          STOSB
24607 00003D10 3C0D          CMP     AL,c_CR ; 0Dh ; Check for carriage return
24608 00003D12 7503          JNZ     short NORMCH
24609 00003D14 C6040A      MOV     BYTE [SI],c_LF ; 0Ah
24610                          NORMCH:
24611 00003D17 3C0A          CMP     AL,c_LF ; 0Ah
24612 00003D19 E0F3          LOOPNZ TRANBUF
24613 00003D1B 7507          JNZ     short ENDRDCON
24614 00003D1D 31F6          XOR     SI,SI ; Cause a new buffer to be read
24615 00003D1F E80FDF      call    OUTT ; Transmit linefeed
24616 00003D22 0C01          OR      AL,1 ; Clear zero flag--not end of file
24617
24618                          ENDRDCON:
24619 00003D24 16          ;hkn; SS is DOSDATA
24620 00003D25 1F          push    ss
24621 00003D26 E834FE      pop     ds
24622 00003D29 8936[2200]    CALL    SWAPBACK
24623                          MOV     [CONTPOS],SI
24624
24625                          ; 16/12/2022
24626                          ;ENDRDDEV:
24627                          ;;hkn; SS is DOSDATA
24628                          ; push    ss
24629                          ; pop     ds
24630                          endrddev1: ; 04/05/2019
24631 00003D2D 893E[B805]    MOV     [NEXTADD],DI
24632 00003D31 7509          JNZ     short SETSFTC ; Zero set if Ctrl-Z found in input
24633 00003D33 C43E[9E05]    LES     DI,[THISSFT]
24634 00003D37 26806505BF    ;and byte [es:di+5],0BFh
24635                          AND     BYTE [ES:DI+SF_ENTRY.sf_flags],~devid_device_EOF
24636                          ; Mark as no more data available
24637                          SETSFTC:
24638                          ; 31/07/2019
24639                          ;call    SETSFT
24640 00003D3C E95A05          ;retn
24641                          jmp     SETSFT
24642
24643                          ; 16/12/2022
24644                          ; 17/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
24645                          ENDRDDEV:
24646                          ;hkn; SS is DOSDATA
24647                          push    ss
24648                          pop     ds
24649                          MOV     [NEXTADD],DI
24650                          JNZ     short SETSFTC ; Zero set if Ctrl-Z found in input
24651                          LES     DI,[THISSFT]
24652                          ;and byte [es:di+5],0BFh
24653                          AND     BYTE [ES:DI+SF_ENTRY.sf_flags],~devid_device_EOF
24654                          ; Mark as no more data available
24655                          SETSFTC:
24656                          ;call    SETSFT
24657                          ;retn
24658                          jmp     SETSFT
24659                          %endif
24660
24661                          READCON:

```

```

24662 00003D3F E821FE      CALL    SWAPCON
24663 00003D42 8B36[2200]    MOV     SI,[CONTPOS]
24664 00003D46 09F6      OR      SI,SI
24665 00003D48 75C4      JNZ     short TRANBUF
24666 00003D4A 803E[7B02]80    CMP     BYTE [CONBUF],128 ; 80h
24667 00003D4F 7406      JZ      short GETBUF
24668 00003D51 C706[7B02]80FF    MOV     WORD [CONBUF],0FF80H ; Set up 128-byte buffer with no template
24669
24670 00003D57 51      GETBUF:  PUSH    CX
24671 00003D58 06      PUSH    ES
24672 00003D59 57      PUSH    DI
24673
24674
24675 00003D5A BA[7B02]    ;hkn; CONBUF is in DOSDATA
24676      MOV     DX,CONBUF
24677 00003D5D E846DC    call    _$STD_CON_STRING_INPUT; Get input buffer
24678 00003D60 5F      POP     DI
24679 00003D61 07      POP     ES
24680 00003D62 59      POP     CX
24681
24682
24683 00003D63 BE[7D02]    ;hkn; CONBUF is in DOSDATA
24684      MOV     SI,CONBUF+2
24685 00003D66 803C1A    CMP     BYTE [SI],1AH ; Check for Ctrl-Z in first character
24686 00003D69 75A3      JNZ     short TRANBUF
24687 00003D6B B01A      MOV     AL,1AH
24688 00003D6D AA      STOSB
24689 00003D6E 4F      DEC     DI
24690 00003D6F B00A      MOV     AL,C_LF
24691 00003D71 E8BDDE    call    OUTT ; Send linefeed
24692 00003D74 31F6      XOR     SI,SI
24693 00003D76 EBAC      JMP     short ENDRDCON ; 04/05/2019
24694
24695 ; 24/07/2018 - Retro DOS v3.0
24696
24697 ;Break <DOS_WRITE -- MAIN WRITE ROUTINE AND DEVICE OUT ROUTINES>
24698 ;-----
24699 ;
24700 ; Procedure Name : DOS_WRITE
24701 ;
24702 ; Inputs:
24703 ; ThisSFT set to the SFT for the file being used
24704 ; [DMAADD] contains transfer address
24705 ; CX = No. of bytes to write
24706 ; Function:
24707 ; Perform write operation
24708 ; NOTE: If CX = 0 on input, file is truncated or grown
24709 ; to current sf_position
24710 ; Outputs:
24711 ; Carry clear
24712 ; SFT Position and cluster pointers updated
24713 ; CX = No. of bytes written
24714 ; ES:DI point to SFT
24715 ; Carry set
24716 ; AX is error code
24717 ; CX = 0
24718 ; ES:DI point to SFT
24719 ; DS preserved, all other registers destroyed
24720 ;-----
24721
24722 ; 04/05/2019 - Retro DOS v4.0
24723 ; DOSCODE:742Ch (MSDOS 6.21, MSDOS.SYS)
24724
24725 ; 17/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
24726 ; DOSCODE:7418h (MSDOS 5.0, MSDOS.SYS)
24727
24728 ; 08/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
24729 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:7D8Fh
24730
24731 DOS_WRITE:
24732 00003D78 C43E[9E05]    LES     DI,[THISFT]
24733      ;mov     al,[ES:DI+2]
24734 00003D7C 268A4502    MOV     AL,[ES:DI+SF_ENTRY.sf_mode]
24735      ;;and     al,0Fh
24736      ;AND     AL,access_mask
24737      ; 07/02/2024 - Retro DOS v5.0
24738      ; (PCDOS 7.1 & win Me)
24739 00003D80 2403      and     al,3 ; open_mode_mask ?
24740      ;cmp     al,0
24741 00003D82 3C00      CMP     AL,open_for_read
24742 00003D84 7503      JNE     short Check_FCB_RO ;Is write or both
24743
24744 00003D86 E96304    BadMode: jmp     SET_ACC_ERR
24745
24746 ; NOTE: The following check for writting to a Read Only File is performed
24747 ; ONLY on FCBs!!!!
24748 ; We ALLOW writes to Read Only files via handles to allow a CREATE
24749 ; of a read only file which can then be written to.
24750 ; This is OK because we are NOT ALLOWED to OPEN a RO file via handles
24751 ; for writting, or RE-CREATE an EXISTING RO file via handles. Thus,
24752 ; CREATING a NEW RO file, or RE-CREATING an existing file which
24753 ; is NOT RO to be RO, via handles are the only times we can write
24754 ; to a read-only file.
24755
24756 Check_FCB_RO:
24757      ;;test    word [es:di+2],8000h
24758      ;TEST     word [ES:DI+SF_ENTRY.sf_mode],sf_isFCB
24759      ;JZ       short WRITE_NO_MODE ; Not an FCB
24760
24761      ;test     byte [es:di+3],80h
24762 00003D89 26F6450380    TEST     byte [ES:DI+SF_ENTRY.sf_mode+1],(sf_isFCB>>8)
24763 00003D8E 7407      JZ       short WRITE_NO_MODE ; Not an FCB
24764
24765      ;test     byte [es:di+4],1
24766 00003D90 26F6450401    TEST     byte [ES:DI+SF_ENTRY.sf_attr],attr_read_only
24767 00003D95 75EF      JNZ     short BadMode ; Can't write to Read_Only files via FCB
24768
24769 00003D97 E81503    WRITE_NO_MODE: call    SETUP
24770 00003D9A E8A5DA    call    IsSFTNet
24771 00003D9D 7406      JZ       short LOCAL_WRITE
24772
24773 ;IF NOT Installed
24774 ; transfer NET_WRITE
24775 ;ELSE
24776      ;mov     ax,1109h
24777 00003D9F B80911    MOV     AX,(MultNet<<8)|9
24778 00003DA2 CD2F      int     2Fh ; Multiplex - NETWORK REDIRECTOR - WRITE TO REMOTE FILE
24779      ; ES:DI -> SFT
24780      ; SFT DPB field -> DPB of drive containing file
24781      ; CX = number of bytes, SS = DOS CS, SDA DTA field -> user buffer
24782      ; Return: CF set on error, CX = bytes written
24783 00003DA4 C3      retn
24784
24785 ;ENDIF

```



```

24786 LOCAL_WRITE:
24787 ;;test word [es:di+5],80h
24788 ;;TEST word [ES:DI+SF_ENTRY.sf_flags],devid_device
24789 ;;jnz short WRTDEV
24790
24791 ;;test byte [es:di+5],80h
24792 00003DA5 26F6450580 TEST byte [ES:DI+SF_ENTRY.sf_flags],devid_device ; Check for named device I/O
24793 00003DAA 756D jnz short WRTDEV
24794
24795 ;mov byte [EXTERR_LOCUS],2
24796 00003DAC C606[2303]02 MOV byte [EXTERR_LOCUS],errLOC_Disk
24797 00003DB1 E82EDB call ECritDisk
24798
24799 00003DB4 E8A805 call DISKWRITE
24800
24801 ; 04/05/2019 - Retro DOS v4.0
24802
24803 ; MSDOS 6.0
24804 ; Extended Open
24805 00003DB7 7210 JC short nocommit
24806
24807 00003DB9 C43E[9E05] LES DI,[THISSFT]
24808
24809 ;;test word [ES:DI+2],4000h
24810 ;;TEST word [ES:DI+SF_ENTRY.sf_mode],AUTO_COMMIT_WRITE
24811 ;;JZ short nocommit
24812
24813 ;test byte [ES:DI+3],40h
24814 00003DBD 26F6450340 TEST byte [ES:DI+SF_ENTRY.sf_mode+1],(AUTO_COMMIT_WRITE>>8)
24815 00003DC2 7405 JZ short nocommit
24816
24817 00003DC4 51 PUSH CX
24818 00003DC5 E80FFB call DOS_COMMIT
24819 00003DC8 59 POP CX
24820
24821 nocommit:
24822 ; Extended Open
24823 ;call LCritDisk
24824 ;retn
24825 ; 18/12/2022
24826 00003DC9 E943DB jmp LCritDisk
24827
24828 DVWRTRAW:
24829 XOR AX,AX ; Media Byte, unit = 0
24830 call SETWRITE
24831 PUSH DS ; Save seg of transfer
24832
24833 ;hkn; SS override
24834 LDS SI,[SS:THISSFT]
24835 call DEVIOCALL ; DS:SI -> DEVICE
24836
24837 MOV DX,DI ; Offset part of Xaddr saved in DX
24838 MOV AH,87H
24839
24840 ;hkn; SS override
24841 MOV DI,[SS:DEVCALL_REQSTAT]
24842
24843 ; MSDOS 3.3
24844 ;test di,8000h
24845 ;jz short CWRTRK
24846
24847 ; MSDOS 6.0
24848 or di,di
24849 jns short CWRTRK
24850
24851 ; MSDOS 3.3 (& MSDOS 6.0)
24852 call CHARHARD
24853
24854 ; 04/05/2019 - Retro DOS v4.0
24855
24856 ; 08/02/2024
24857 mov di,[ss:CALLSCNT]
24858 ; MSDOS 6.0
24859 ;sub cx,[ss:CALLSCNT] ; update ptr & count to reflect M065
24860 sub cx,di
24861 mov bx,dx ; number of chars xferred M065
24862 ;add bx,[ss:CALLSCNT] ; M065
24863 add bx,di ;
24864 mov di,bx ; M065
24865
24866 ; MSDOS 3.3
24867 ;MOV BX,DX ; Recall transfer addr M065
24868
24869 ; MSDOS 3.3 (& MSDOS 6.0)
24870 OR AL,AL
24871 JZ short CWRTRK ; Ignore
24872 CMP AL,3
24873 JZ short CWRFERR
24874 POP DS ; Recover saved seg of transfer
24875 JMP short DVWRTRAW ; Try again
24876
24877 CWRFERR:
24878 POP AX ; Chuck saved seg of transfer
24879 JMP CRDFERR ; Will pop one more stack element
24880
24881 CWRTRK:
24882 POP AX ; Chuck saved seg of transfer
24883 POP DS
24884 MOV AX,[CALLSCNT] ; Get actual number of bytes transferred
24885
24886 ENDWRDEV:
24887 LES DI,[THISSFT]
24888 MOV CX,AX
24889 ;call ADDREC
24890 ;retn
24891 ; 16/12/2022
24892 ; 10/06/2019
24893 jmp ADDREC
24894 ; 17/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
24895 ;call ADDREC
24896 ;retn
24897
24898 WRTNUL:
24899 MOV DX,CX ; Entire transfer done
24900 WRTCOOKJ:
24901 JMP WRTCOOKDONE
24902
24903 WRTDEV:
24904 ;mov byte [EXTERR_LOCUS],4
24905 MOV byte [EXTERR_LOCUS],errLOC_SerDev
24906 ;or byte [es:di+5],40h
24907 OR BYTE [ES:DI+SF_ENTRY.sf_flags],devid_device_EOF
24908 ; Reset EOF for input
24909 ;mov bl,[es:di+5]
24910 MOV BL,[ES:DI+SF_ENTRY.sf_flags]
24911 XOR AX,AX
24912 JCXZ ENDWRDEV ; problem of creating on a device.
24913 PUSH DS
24914 MOV AL,BL

```

```

24910 00003E2E C51E[2C03]      LDS     BX,[DMAADD]          ; Xaddr to DS:BX
24911 00003E32 89DF            MOV     DI,BX              ; Xaddr to DS:DI
24912                                ; 27/06/2024 (PCDOS 7.1 IBMDOS.COM)
24913                                ; 08/02/2024
24914 00003E34 99              cwd
24915                                ;XOR     DX,DX              ; Set starting point
24916                                ;test    al,20h
24917 00003E35 A820            test    AL,devid_device_raw ; Raw?
24918                                ;JZ      short TEST_DEV_CON
24919                                ;JMP     DVWRTRAW
24920                                ; 16/12/2022
24921 00003E37 7593            jnz     short DVWRTRAW
24922                                ; 17/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
24923                                ;JZ      short TEST_DEV_CON
24924                                ;JMP     short DVWRTRAW
24925
24926 TEST_DEV_CON:
24927                                ;test    al,2
24928 00003E39 A802            test    AL,devid_device_con_out ; Console output device?
24929 00003E3B 756E            jnz     short WRITECON
24930                                ;test    al,4
24931 00003E3D A804            test    AL,devid_device_null
24932 00003E3F 75D3            jnz     short WRTNUL
24933 00003E41 89D0            MOV     AX,DX
24934 00003E43 803F1A        CMP     BYTE [BX],1Ah          ; ^Z?
24935 00003E46 74CE            JZ      short WRTCOOKJ        ; Yes, transfer nothing
24936 00003E48 51            PUSH    CX
24937 00003E49 B90100        MOV     CX,1
24938 00003E4C E8FA14        call    SETWRITE
24939 00003E4F 59            POP     CX
24940
24941 ;hkn; SS override
24942 00003E50 36C536[9E05]    LDS     SI,[SS:THISSFT]
24943                                ;
24944                                ;SR; Removed x25 support from here
24945                                ;
24946                                ;lds     si,[si+7]
24947 00003E55 C57407        LDS     SI,[SI+SF_ENTRY.sf_devptr]
24948 DVWRTLP:
24949 00003E58 E8A81F        call    DSKSTATCHK
24950 00003E5B E83714        call    DEVIOCALL2
24951 00003E5E 57            PUSH    DI
24952 00003E5F B487            MOV     AH,87H
24953
24954 ;hkn; SS override
24955 00003E61 368B3E[5D03]    MOV     DI,[SS:DEVCALL_REQSTAT]
24956                                ;
24957                                ; MSDOS 3.3
24958                                ;test    di,8000h
24959                                ;jz      short CWROK
24960                                ;
24961                                ; MSDOS 6.0
24962 00003E66 09FF        or      di,di
24963 00003E68 7916        jns     short CWROK
24964                                ;
24965                                ; MSDOS 3.3 (& MSDOS 6.0)
24966 00003E6A E8B621        call    CHARHARD
24967 00003E6D 5F            POP     DI
24968
24969 ;hkn; SS override
24970 00003E6E 36C706[6C03]0100    MOV     word [SS:CALLSCNT],1
24971 00003E75 3C01        CMP     AL,1
24972 00003E77 74DF        JZ      short DVWRTLP ; Retry
24973 00003E79 08C0        OR      AL,AL
24974 00003E7B 740C        JZ      short DVWRTIGN ; Ignore
24975                                ; 10/08/2018
24976 00003E7D E99AFD        JMP     CRDFERR ; Fail, pops one stack element
24977 CWROK:
24978 00003E80 5F            POP     DI
24979
24980 ;hkn; SS override
24981 00003E81 36833E[6C03]00    CMP     word [SS:CALLSCNT],0
24982 00003E87 741C        JZ      short WRTCOOKDONE
24983 DVWRTIGN:
24984 00003E89 42            INC     DX
24985
24986 ;hkn; SS override for CALLXAD
24987 00003E8A 36FF06[6803]    INC     WORD [SS:CALLXAD]
24988 00003E8F 47            INC     DI
24989 00003E90 1E            PUSH    DS
24990 00003E91 368E1E[6A03]    MOV     DS,[SS:CALLXAD+2]
24991 00003E96 803D1A        CMP     BYTE [DI],1Ah ; ^Z?
24992 00003E99 1F            POP     DS
24993 00003E9A 7409        JZ      short WRTCOOKDONE
24994
24995 ;hkn; SS override
24996 00003E9C 36C706[5D03]0000    MOV     word [SS:DEVCALL_REQSTAT],0
24997 00003EA3 E2B3        LOOP   DVWRTLP
24998 WRTCOOKDONE:
24999 00003EA5 89D0        MOV     AX,DX
25000 00003EA7 1F            POP     DS
25001 00003EA8 E960FF        JMP     ENDWRDEV ; 10/08/2018
25002
25003 WRITECON:
25004 00003EAB 1E            PUSH    DS
25005
25006 ;hkn; SS is DOSDATA
25007 00003EAC 16            push    ss
25008 00003EAD 1F            pop     ds
25009 00003EAE E8B2FC        CALL    SWAPCON
25010 00003EB1 1F            POP     DS
25011 00003EB2 89DE        MOV     SI,BX
25012 00003EB4 51            PUSH    CX
25013 WRCONLP:
25014 00003EB5 AC            LODSB
25015 00003EB6 3C1A        CMP     AL,1Ah ; ^Z?
25016 00003EB8 7405        JZ      short CONEOF
25017 00003EBA E874DD        call    OUTT
25018 00003EBD E2F6        LOOP   WRCONLP
25019 CONEOF:
25020 00003EBF 58            POP     AX ; Count
25021 00003EC0 1F            POP     DS
25022 00003EC1 29C8        SUB     AX,CX ; Amount actually written
25023 00003EC3 E897FC        CALL    SWAPBACK
25024 00003EC6 E942FF        JMP     ENDWRDEV
25025
25026 ;-----
25027 ;
25028 ; Procedure Name : get_io_sft
25029 ;
25030 ; Convert JFN number in BX to sf_entry in DS:SI we get the normal SFT if
25031 ; CONSWAP is FALSE or if the handle desired is 2 or more. Otherwise, we
25032 ; retrieve the sft from ConsFT which is set by SwapCon.
25033 ;

```

```

25034 ;-----
25035 ;
25036 ; 04/05/2019 - Retro DOS v4.0
25037 ; DOSCODE:7583h (MSDOS 6.21, MSDOS.SYS)
25038 ; 17/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
25039 ; DOSCODE:756Fh (MSDOS 5.0, MSDOS.SYS)
25040
25041 GET_IO_SFT:
25042 ;test byte [SS:CONSWAP],0FFh
25043 00003EC9 36803E[5703]00 cmp byte [SS:CONSWAP],0 ;smr;SS Override
25044 00003ECF 7512 JNZ short GetRedir
25045
25046 GetNormal:
25047 00003ED1 16 push ss
25048 00003ED2 1F pop ds
25049 00003ED3 06 PUSH ES
25050 00003ED4 57 PUSH DI
25051 00003ED5 E8FB37 call SFFromHandle
25052 00003ED8 7206 JC short RET44P
25053 00003EDA 8CC6 MOV SI,ES
25054 00003EDC 8EDE MOV DS,SI
25055 00003EDE 89FE MOV SI,DI
25056 RET44P:
25057 00003EE0 5F POP DI
25058 00003EE1 07 POP ES
25059 00003EE2 C3 retn
25060 GetRedir:
25061 00003EE3 83FB01 CMP BX,1
25062 00003EE6 77E9 JA short GetNormal
25063 00003EE8 36C536[E605] LDS SI,[SS:CONSFT]
25064 00003EED F8 CLC
25065 get_io_sft_retn:
25066 retn
25067 ;Break <DIRREAD -- READ A DIRECTORY SECTOR>
25068 ;-----
25069 ;
25070 ; Procedure Name : DIRREAD
25071 ;
25072 ; Inputs:
25073 ; AX = Directory block number (relative to first block of directory)
25074 ; ES:BP = Base of drive parameters
25075 ; [DIRSEC] = First sector of first cluster of directory
25076 ; [CLUSNUM] = Next cluster
25077 ; [CLUSFAC] = Sectors/Cluster
25078 ; Function:
25079 ; Read the directory block into [CURBUF].
25080 ; Outputs:
25081 ; [NXTCLUSNUM] = Next cluster (after the one skipped to)
25082 ; [SECCLUSPOS] Set
25083 ; ES:BP unchanged
25084 ; [CURBUF] Points to Buffer with dir sector
25085 ; Carry set if error (user said FAIL to I 24)
25086 ; DS preserved, all other registers destroyed.
25087 ;-----
25088 ;hkn; called from dir.asm. DS already set up to DOSDATA.
25089
25090 ; 08/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
25091
25092 DIRREAD:
25093
25094 ; Note that ClusFac is the sectors per cluster. This is NOT necessarily
25095 ; the same as what is in the DPB! In the case of the root directory, we have
25096 ; ClusFac = # sectors in the root directory. The root directory is detected
25097 ; by DIRStart = 0.
25098
25099 00003EEF 31D2 XOR DX,DX
25100 ; 08/02/2024 (PCDOS 7.1)
25101 00003EF1 3916[DC0A] cmp [DIRSTART_HW],dx
25102 00003EF5 7509 jnz short SubDir
25103 ;CMP word [DIRSTART],0
25104 ; 21/09/2023
25105 00003EF7 3916[C205] cmp [DIRSTART],dx ; 0
25106 00003EF8 7503 jnz short SubDir
25107 00003EFD 92 XCHG AX,DX
25108 00003EFE EB0C JMP short DoRead
25109
25110 ; Convert the sector number in AX into cluster and sector-within-cluster pair
25111
25112 SubDir:
25113 MOV DL,AL
25114 00003F00 88C2 ;and dl,[es:bp+4]
25115 AND DL,[ES:BP+DPB.CLUSTER_MASK]
25116 00003F02 26225604 ; (DX) = sector-in-cluster
25117
25118 ;mov cl,[es:bp+5]
25119 MOV CL,[ES:BP+DPB.CLUSTER_SHIFT]
25120 00003F06 268A4E05 SHR AX,CL
25121 00003F0A D3E8 ; (DX) = position in cluster
25122 ; (AX) = number of clusters to skip
25123
25124 DoRead:
25125 MOV [SECCLUSPOS],DL
25126 MOV CX,AX
25127 MOV AH,DL
25128 00003F0C 8816[7305] ; (CX) = number of clusters to skip.
25129 00003F10 89C1 ; (AH) = remainder
25130 00003F12 88D4
25131
25132 ; 04/05/2019 - Retro DOS v4.0
25133
25134 ; 08/02/2024
25135 %if 0
25136 ; MSDOS 6.0
25137 ;MOV DX,[DIRSEC+2] ;>32mb
25138 ;MOV [HIGH_SECTOR],DX ;>32mb
25139 ;MOV DX,[DIRSEC]
25140 ;ADD DL,AH
25141 ;ADC DH,0
25142 ;ADC word [HIGH_SECTOR],0 ;>32mb
25143 ; 21/09/2023
25144 xor bx,bx ; 0
25145 mov dx,[DIRSEC]
25146 add dl,ah
25147 adc dh,b1 ; 0
25148 adc bx,[DIRSEC+2]
25149 mov [HIGH_SECTOR],bx
25150
25151 MOV BX,[CLUSNUM]
25152 MOV [NXTCLUSNUM],BX
25153 JCXZ FIRSTCLUSTER
25154 %else

```

```

25158 ; 08/02/2024 (PCDOS 7.1)
25159 mov bx,[CLUSNUM_HW]
25160 mov [NXTCLUSNUM_HW],bx
25161 mov [CLUSTNUM_HW],bx
25162 mov dx,[DIRSEC+2]
25163 mov [HIGH_SECTOR],dx
25164 mov dx,[DIRSEC]
25165 add dl,ah
25166 adc dh,0
25167 adc word [HIGH_SECTOR],0
25168 mov bx,[CLUSNUM]
25169 mov [NXTCLUSNUM],bx
25170 jcxz FIRSTCLUSTER
25171 %endif
25172
25173 SKPCLLP:
25174 call UNPACK
25175 jc short get_io_sft_retn
25176 ;;;
25177 ; 08/02/2024 (PCDOS 7.1)
25178 push word [CLUSTNUM_HW]
25179 pop word [CLUSTERS_HW]
25180 push word [CONTENT_HW]
25181 pop word [CLUSTNUM_HW]
25182 ;;;
25183 XCHG BX,DI
25184 call ISEOF ; test for eof based on fat size
25185 JAE short HAVESKIPPED
25186 LOOP SKPCLLP
25187 HAVESKIPPED:
25188 MOV [NXTCLUSNUM],BX
25189 ;;;
25190 ; 08/02/2024 (PCDOS 7.1)
25191 mov bx,[CLUSTNUM_HW]
25192 mov [NXTCLUSNUM_HW],bx
25193 ;
25194 mov bx,[CLUSTERS_HW]
25195 mov [CLUSTNUM_HW],bx
25196 ;;;
25197 MOV DX,DI
25198 MOV BL,AH
25199 call FIGREC
25200
25201 ;entry FIRSTCLUSTER
25202
25203 FIRSTCLUSTER:
25204 ; 22/09/2023
25205 ;mov byte [ALLOWED],18h
25206 ;MOV byte [ALLOWED],Allowed_RETRY+Allowed_FAIL ; *
25207 ;XOR AL,AL ; * ; Indicate pre-read
25208 ;call GETBUFFER
25209 call GETBUFFER ; * ; pre-read
25210 ;jc short get_io_sft_retn
25211 ; 08/02/2024
25212 jc short dirread_retn
25213
25214 ;entry SET_BUF_AS_DIR
25215
25216 SET_BUF_AS_DIR:
25217 ;
25218 ; Set the type of CURBUF to be a directory sector.
25219 ; Only flags are modified.
25220
25221 PUSH DS
25222 PUSH SI
25223 LDS SI,[CURBUF]
25224 ;or byte [si+5],4
25225 OR byte [SI+BUFFERINFO.buf_flags],buf_isDIR ; clears carry
25226 POP SI
25227 POP DS
25228 dirread_retn:
25229 retn
25230
25231 ;Break <FATSECRD -- READ A FAT SECTOR>
25232 ;-----
25233 ;
25234 ; Procedure Name : FATSECRD
25235 ; Inputs:
25236 ; Same as DREAD
25237 ; DS:BX = Transfer address
25238 ; CX = Number of sectors
25239 ; DX = Absolute record number
25240 ; ES:BP = Base of drive parameters
25241 ; Function:
25242 ; Calls BIOS to perform FAT read.
25243 ; Outputs:
25244 ; Same as DREAD
25245 ;-----
25246
25247 ; 04/05/2019 - Retro DOS v4.0
25248 ; 18/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
25249
25250 ; 09/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
25251
25252 FATSECRD:
25253 ;hkn; SS override
25254 ;mov byte [ss:ALLOWED],18h
25255 MOV byte [SS:ALLOWED],Allowed_RETRY+Allowed_FAIL
25256 MOV DI,CX
25257 ;mov cl,[es:bp+8]
25258 MOV CL,[ES:BP+DPB.FAT_COUNT]
25259 ; MSDOS 3.3
25260 ;mov al,[es:bp+0Fh]
25261 ;MOV AL,[ES:BP+DPB.FAT_SIZE]
25262 ;XOR AH,AH
25263 ; MSDOS 6.0
25264 ;mov ax,[es:bp+0Fh]
25265 MOV AX,[ES:BP+DPB.FAT_SIZE] ;>32mb
25266 XOR CH,CH
25267 ;;;
25268 ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
25269 or ax,ax
25270 jnz short FATSECRD_cont ; not FAT32
25271 ;test byte [es:bp+23h],80h
25272 test byte [es:bp+DPB.EXT_FLAGS],80h ; (FAT32 bs extd flags bit 7)
25273 jz short FATSECRD_cont
25274 ;mov cx,1
25275 mov cl,1 ; only one FAT is active
25276
25277 FATSECRD_cont:
25278 push word [ss:HIGH_SECTOR]
25279 ;;;
25280 PUSH DX
25281 NXTFAT:
25282 ; MSDOS 6.0

```

```

25282 ;hkn; SS override
25283 ; 09/02/2024 (PCDOS 7.1)
25284 ;MOV word [SS:HIGH_SECTOR],0 ;>32mb FAT sectors cannot exceed
25285 00003FB0 51 PUSH CX ;32mb
25286 00003FB1 50 PUSH AX
25287 ;;;
25288 ; 08/02/2024 (PCDOS 7.1 IBMDOS.COM)
25289 00003FB2 36FF36[0706] push word [ss:HIGH_SECTOR]
25290 00003FB7 52 push dx
25291 ;;;
25292 00003FB8 89F9 MOV CX,DI
25293 00003FBA E88600 call DSKREAD
25294 ;;;
25295 ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
25296 00003FBD 5A pop dx
25297 00003FBE 368F06[0706] pop word [ss:HIGH_SECTOR]
25298 ;;;
25299 00003FC3 58 POP AX
25300 00003FC4 59 POP CX
25301 00003FC5 7440 JZ short RET41P ; Carry clear
25302 ;;;
25303 ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
25304 00003FC7 09C0 or ax,ax
25305 00003FC9 7511 jnz short NXTFAT2 ; not FAT32
25306 ;add dx,[es:bp+31h]
25307 00003FCB 26035631 add dx,[es:bp+DPB.FAT32_SIZE]
25308 00003FCF 50 push ax
25309 ;mov ax,[es:bp+33h]
25310 00003FD0 268B4633 mov ax,[es:bp+DPB.FAT32_SIZE+2]
25311 00003FD4 361106[0706] adc [ss:HIGH_SECTOR],ax
25312 00003FD9 58 pop ax
25313 00003FDA EB08 jmp short NXTFAT3
25314 NXTFAT2:
25315 ;;;
25316 00003FDC 01C2 ADD DX,AX
25317 ;;;
25318 ; 09/02/2024 (PCDOS 7.1)
25319 00003FDE 368316[0706]00 adc word [ss:HIGH_SECTOR],0
25320 NXTFAT3:
25321 ;;;
25322 00003FE4 E2CA LOOP NXTFAT
25323 00003FE6 5A POP DX
25324 ;;;
25325 ; 09/02/2024 (PCDOS 7.1)
25326 00003FE7 368F06[0706] pop word [ss:HIGH_SECTOR]
25327 ;;;
25328 00003FEC 89F9 MOV CX,DI
25329 ;
25330 ; NOTE FALL THROUGH
25331 ;
25332 ;Break <DREAD -- DO A DISK READ>
25333 ;-----
25334 ;
25335 ; Procedure Name : DREAD
25336 ;
25337 ; Inputs:
25338 ; DS:BX = Transfer address
25339 ; CX = Number of sectors
25340 ; DX = Absolute record number (LOW)
25341 ; [HIGH_SECTOR] = Absolute record number (HIGH)
25342 ; ES:BP = Base of drive parameters
25343 ; [ALLOWED] must be set in case call to HARDERR needed
25344 ; Function:
25345 ; Calls BIOS to perform disk read. If BIOS reports
25346 ; errors, will call HARDERRRW for further action.
25347 ; Outputs:
25348 ; Carry set if error (currently user FAILED to INT 24)
25349 ; DS,ES:BP preserved. All other registers destroyed.
25350 ;-----
25351 ;
25352 ;entry DREAD
25353 DREAD:
25354 00003FEE E85200 call DSKREAD
25355 00003FF1 7497 jz short dirread_retn ; Carry clear
25356 ;hkn; SS override
25357 00003FF3 36C606[7505]00 MOV BYTE [SS:READOP],0 ; Read
25358 00003FF9 E89A00 call HARDERRRW
25359 00003FFC 3C01 CMP AL,1 ; Check for retry
25360 00003FFE 74EE JZ short DREAD
25361 ;
25362 fail_ignore: ; 09/02/2024
25363 00004000 3C03 CMP AL,3 ; Check for FAIL
25364 00004002 F8 CLC
25365 00004003 7501 JNZ short NO_CAR ; Ignore
25366 00004005 F9 STC
25367 NO_CAR:
25368 00004006 C3 retn
25369 ;
25370 ; 09/02/2024 - Retro DOS v5.0
25371 RET41P:
25372 00004007 5A POP DX
25373 ;;;
25374 ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
25375 00004008 368F06[0706] pop word [ss:HIGH_SECTOR]
25376 ;;;
25377 0000400D C3 retn
25378 ;
25379 ; 24/07/2018 - Retro DOS v3.0
25380 ;
25381 ;Break <CHECK_WRITE_LOCK>
25382 ;-----
25383 ;
25384 ; Procedure Name : CHECK_WRITE_LOCK
25385 ;
25386 ; Inputs:
25387 ; output of SETUP
25388 ; ES:DI -> SFT
25389 ; Function:
25390 ; check write lock
25391 ; Outputs:
25392 ; Carry set if error
25393 ; Carry clear if ok
25394 ;
25395 ;-----
25396 ;
25397 ; 04/05/2019 - Retro DOS v4.0
25398 ; 18/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
25399 ;
25400 CHECK_WRITE_LOCK:
25401 ; MSDOS 6.0
25402 ;test byte [es:di+4],8
25403 0000400E 26F6450408 TEST byte [ES:DI+SF_ENTRY.sf_attr],attr_volume_id ;volume id
25404 ;JZ short write_cont ;no
25405 ;;call SET_ACC_ERR_DS

```

```

25406             ;;retn
25407             ;;jnz SET_ACC_ERR_DS
25408             ; 19/08/2018
25409             ;jz short write_cont
25410             ;jmp SET_ACC_ERR_DS
25411             ; 18/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
25412 00004013 7403 JZ short write_cont
25413             ;call SET_ACC_ERR_DS
25414             ;retn
25415             ; 16/12/2022
25416 00004015 E9D201 jmp SET_ACC_ERR_DS
25417
25418 write_cont:
25419             PUSH CX                ;save reg
25420             OR CX,CX
25421             JNZ short Not_Truncate
25422             dec cx                ;(cx) = -1; check for lock on whole file
25423 Not_Truncate:
25424             MOV AL,80H            ;check write access
25425             call LOCK_CHECK        ;check lock
25426             POP CX               ;restore reg
25427             JNC short WRITE_OK    ;lock ok
25428             call WRITE_LOCK_VIOLATION ;issue I24
25429             JNC short write_cont   ;retry
25430 WRITE_OK:
25431             retn
25432
25433 ;Break <CHECK_READ_LOCK>
25434 ;-----
25435 ;
25436 ; Procedure Name : CHECK_READ_LOC
25437 ;
25438 ; Inputs:
25439 ; ES:DI -> SFT
25440 ; output of SETUP
25441 ; Function:
25442 ; check read lock
25443 ; Outputs:
25444 ; Carry set if error
25445 ; Carry clear if ok
25446 ;-----
25447
25448 CHECK_READ_LOCK:
25449 ; MSDOS 6.0
25450 ;test byte [es:di+4],8
25451 0000402C 26F6450408 TEST byte [ES:DI+SF_ENTRY.sf_attr],attr_volume_id ;volume id
25452             ;JZ short do_retry      ; no
25453             ;call SET_ACC_ERR
25454             ;retn
25455             ;jnz SET_ACC_ERR
25456             ; 16/12/2022
25457             ; 28/07/2019
25458 00004031 7403 JZ short do_retry
25459 00004033 E9B601 jmp SET_ACC_ERR
25460             ; 18/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
25461             ;JZ short do_retry
25462             ;call SET_ACC_ERR
25463             ;retn
25464 do_retry:
25465             xor al,al              ;check read access
25466             call LOCK_CHECK        ;check lock
25467             JNC short READLOCK_OK  ;lock ok
25468             call READ_LOCK_VIOLATION ;issue I24
25469             JNC short do_retry      ;retry
25470 READLOCK_OK:
25471             dw_ret_label:          ; 09/02/2024
25472 00004042 C3 retn
25473
25474 ;=====
25475 ; DISK2.ASM, MSDOS 6.0, 1991
25476 ;=====
25477 ; 24/07/2018 - Retro DOS v3.0
25478 ; 04/05/2019 - Retro DOS v4.0
25479
25480 ; TITLE DISK2 - Disk utility routines
25481 ; NAME Disk2
25482
25483 ;** Low level Read and write routines for local SFT I/O on files and devs
25484 ;
25485 ; DskRead
25486 ; DWRITE
25487 ; DSKWRITE
25488 ; HarderrRW
25489 ; SETUP
25490 ; BREAKDOWN
25491 ; READ_LOCK_VIOLATION
25492 ; WRITE_LOCK_VIOLATION
25493 ; DISKREAD
25494 ; SET_ACC_ERR_DS
25495 ; SET_ACC_ERR
25496 ; SETSFT
25497 ; SETCLUS
25498 ; AddRec
25499 ;
25500 ; Revision history:
25501 ;
25502 ; AN000 version 4.00 Jan. 1988
25503 ; M039 DB 10/17/90 - Disk read/write optimization
25504 ;
25505 ; 04/05/2019 - Retro DOS v4.0
25506 ; DOSCODE:7699h (MSDOS 6.21, MSDOS.SYS)
25507 ; 18/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
25508 ; DOSCODE:7685h (MSDOS 5.0, MSDOS.SYS)
25509
25510 ;Break <DSKREAD -- PHYSICAL DISK READ>
25511 ;-----
25512 ;
25513 ; Procedure Name : DSKREAD
25514 ;
25515 ; Inputs:
25516 ; DS:BX = Transfer addr
25517 ; CX = Number of sectors
25518 ; [HIGH_SECTOR] = Absolute record number (HIGH)
25519 ; DX = Absolute record number (LOW)
25520 ; ES:BP = Base of drive parameters
25521 ; Function:
25522 ; Call BIOS to perform disk read
25523 ; Outputs:
25524 ; DI = CX on entry
25525 ; CX = Number of sectors unsuccessfully transfered
25526 ; AX = Status word as returned by BIOS (error code in AL if error)
25527 ; Zero set if OK (from BIOS) (carry clear)
25528 ; Zero clear if error (carry clear)
25529 ; SI Destroyed, others preserved

```



```

25530 ;-----
25531
25532 DSKREAD:
25533 00004043 51      PUSH    CX
25534                ;mov    ah,[es:bp+17h] ; 04/05/2019
25535 00004044 268A6617 MOV     AH,[ES:BP+DPB.MEDIA]
25536                ;mov    al,[es:bp+1]
25537 00004048 268A4601 MOV     AL,[ES:BP+DPB.UNIT]
25538 0000404C 53      PUSH    BX
25539 0000404D 06      PUSH    ES
25540 0000404E E8C512  call    SETREAD
25541 00004051 EB22    JMP     short DODSKOP
25542
25543 ;Break    <DWRITE -- SEE ABOUT WRITING>
25544 ;-----
25545 ;
25546 ; Procedure Name : DWRITE
25547 ;
25548 ; Inputs:
25549 ; DS:BX = Transfer address
25550 ; CX = Number of sectors
25551 ; [HIGH_SECTOR] = Absolute record number (HIGH)
25552 ; DX = Absolute record number (LOW)
25553 ; ES:BP = Base of drive parameters
25554 ; [ALLOWED] must be set in case HARDERR called
25555 ; Function:
25556 ; Calls BIOS to perform disk write. If BIOS reports
25557 ; errors, will call HARDERRRW for further action.
25558 ; Output:
25559 ; Carry set if error (currently, user FAILED to I 24)
25560 ; BP preserved. All other registers destroyed.
25561 ;-----
25562 ;
25563 ;entry DWRITE
25564
25565 00004053 E81100  CALL    DSKWRITE
25566 00004056 74EA    jz     short dw_ret_label ; Carry clear (retz)
25567
25568 ;hkn; SS override
25569 00004058 36C606[7505]01 MOV     BYTE [SS:READOP],1 ; write
25570 0000405E E83500  call    HARDERRRW
25571 00004061 3C01    CMP     AL,1 ; Check for retry
25572 00004063 74EE    JZ     short DWRITE
25573
25574 ; 09/02/2024
25575 %if 0
25576     CMP     AL,3 ; Check for FAIL
25577     CLC
25578     JNZ     short NO_CAR2 ; Ignore
25579     STC
25580 NO_CAR2:
25581 dw_ret_label:
25582     retn
25583 %else
25584 ; 09/02/2024 - Retro DOS v5.0
25585 00004065 EB99    jmp     short fail_ignore
25586 %endif
25587
25588 ;Break    <DSKWRITE -- PHYSICAL DISK WRITE>
25589 ;-----
25590 ;
25591 ; Procedure Name : DSKWRITE
25592 ;
25593 ; Inputs:
25594 ; DS:BX = Transfer addr
25595 ; CX = Number of sectors
25596 ; DX = Absolute record number (LOW)
25597 ; [HIGH_SECTOR] = Absolute record number (HIGH)
25598 ; ES:BP = Base of drive parameters
25599 ; Function:
25600 ; Call BIOS to perform disk read
25601 ; Outputs:
25602 ; DI = CX on entry
25603 ; CX = Number of sectors unsuccessfully transfered
25604 ; AX = Status word as returned by BIOS (error code in AL if error)
25605 ; Zero set if OK (from BIOS) (carry clear)
25606 ; Zero clear if error (carry clear)
25607 ; SI Destroyed, others preserved
25608 ;-----
25609 ;
25610 ;entry DSKWRITE
25611
25612 DSKWRITE:
25613 00004067 51      PUSH    CX
25614                ;mov    ah,[es:bp+17h] ; 04/05/2019
25615 00004068 268A6617 MOV     AH,[ES:BP+DPB.MEDIA]
25616                ;mov    al,[es:bp+1]
25617 0000406C 268A4601 MOV     AL,[ES:BP+DPB.UNIT]
25618 00004070 53      PUSH    BX
25619 00004071 06      PUSH    ES
25620 00004072 E8D412  call    SETWRITE
25621
25622 00004075 8CD9    MOV     CX,DS ; Save DS
25623 00004077 1F      POP     DS ; DS:BP points to DPB
25624 00004078 1E      PUSH    DS
25625
25626 ;lds    si,[ds:bp+13h] ; 04/05/2019
25627 00004079 3EC57613 LDS     SI,[ds:BP+DPB.DRIVER_ADDR] ; 07/09/2018
25628 0000407D E81512  call    DEVIOCALL2
25629
25630 00004080 8ED9    MOV     DS,CX ; Restore DS
25631 00004082 07      POP     ES ; Restore ES
25632 00004083 5B      POP     BX
25633
25634 ;hkn; SS override
25635 00004084 368B0E[6C03] MOV     CX,[SS:CALLSCNT] ; Number of sectors transferred
25636 00004089 5F      POP     DI
25637 0000408A 29F9    SUB     CX,DI
25638 0000408C F7D9    NEG     CX ; Number of sectors not transferred
25639
25640 ;hkn; SS override
25641 0000408E 36A1[5D03] MOV     AX,[SS:DEVCALL_REQSTAT]
25642                ;test    ax,8000h
25643                ; 17/12/2022
25644                ;test    ah,80h
25645 00004092 F6C480  test    ah,(STERR>>8)
25646                ;test    AX,STERR
25647 00004095 C3      retn
25648
25649 ;Break    <HardErrRW - map extended errors and call harderr>
25650 ;-----
25651 ;
25652 ; Procedure Name : HardErrRW
25653 ;

```

```

25654 ; Inputs:
25655 ;   AX is error code from read or write
25656 ;   Other registers set as per HARDERR
25657 ; Function:
25658 ;   Checks the error code for special extended
25659 ;   errors and maps them if needed. Then invokes
25660 ;   Harderr
25661 ; Outputs:
25662 ;   Of HARDERR
25663 ; AX may be modified prior to call to HARDERR.
25664 ; No other registers altered.
25665 ;
25666 ;-----
25667 ;
25668 ; 18/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
25669 HARDERRRW:
25670 ;cmp     al,0Fh
25671 00004096 3C0F      CMP     AL,error_I24_wrong_disk
25672 00004098 7512      JNZ     short DO_ERR          ; Nothing to do
25673 ;
25674 ; MSDOS 3.3
25675 ;push    ds
25676 ;push    si
25677 ;lds     si,[ss:CALLVIDRW]
25678 ;mov     [ss:EXTERRPT+2],ds
25679 ;mov     [ss:EXTERRPT],si
25680 ;pop     si
25681 ;pop     ds
25682 ;
25683 ; MSDOS 6.0
25684 0000409A 50      push    ax
25685 0000409B 36A1[7003]   mov     ax,[ss:CALLVIDRW]          ; get ptr lo ;smr;SS override
25686 0000409F 36A3[2803]   mov     [ss:EXTERRPT],ax          ; set ext err ptr lo
25687 000040A3 36A1[7203]   mov     ax,[ss:CALLVIDRW+2]       ; get ptr hi from dev
25688 000040A7 36A3[2A03]   mov     [ss:EXTERRPT+2],ax        ; set ext err ptr hi
25689 000040AB 58      pop     ax
25690 DO_ERR:
25691 ;call    HARDERR
25692 ;retn
25693 ; 16/12/2022
25694 ; 10/06/2019
25695 000040AC E9A51F   jmp     HARDERR
25696 ; 18/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
25697 ;call    HARDERR
25698 ;retn
25699 ;
25700 ; 24/07/2018 - Retro DOS v3.0
25701 ;
25702 ;Break    <SETUP -- SETUP A DISK READ OR WRITE FROM USER>
25703 ;-----
25704 ;
25705 ; Procedure Name : SETUP
25706 ;
25707 ; Inputs:
25708 ;   ES:DI point to SFT (value also in THISSFT)
25709 ;   DMAAdd contains transfer address
25710 ;   CX = Byte count
25711 ;   DS = DOSDATA
25712 ;   WARNING Stack must be clean, two ret adrrs on stack, 1st of caller,
25713 ;   2nd of caller of caller.
25714 ;
25715 ; Outputs:
25716 ;   CX = byte count
25717 ;   [THISDPB] = Base of drive parameters if file
25718 ;   = Pointer to device header if device or NET
25719 ;   ES:DI Points to SFT
25720 ;   [NEXTADD] = Displacement of disk transfer within segment
25721 ;   [TRANS] = 0 (No transfers yet)
25722 ;   BytPos = Byte position in file
25723 ;
25724 ; The following fields are relevant to local files (not devices) only:
25725 ;   SecPos = Position of first sector (local files only)
25726 ;   [BYTSECPOS] = Byte position in first sector (local files only)
25727 ;   [CLUSNUM] = First cluster (local files only)
25728 ;   [SECCLUSPOS] = Sector within first cluster (local files only)
25729 ;   [THISDRV] = Physical unit number (local files only)
25730 ;
25731 ; RETURNS ONE LEVEL UP WITH:
25732 ;   CX = 0
25733 ;   CARRY = Clear
25734 ;   IF AN ERROR IS DETECTED
25735 ; All other registers destroyed
25736 ;-----
25737 ;
25738 ;hkn; called from disk.asm. DS has been set up to DOSDATA.
25739 ;
25740 ; DOSCODE:770Bh (MSDOS 6.21, MSDOS.SYS)
25741 ;
25742 ; 18/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
25743 ; DOSCODE:76F7h (MSDOS 5.0, MSDOS.SYS)
25744 ;
25745 ; 09/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
25746 ; DOSCODE:80D8h (PCDOS 7.1, IBMDOS.COM)
25747 ;
25748 SETUP:
25749 ; IBMDOS.COM (MSDOS 3.3) - Offset 411Bh
25750 ;
25751 ;lds     si,[es:di+7]
25752 000040AF 26C57507   LDS     SI,[ES:DI+SF_ENTRY.sf_devptr]
25753 ;
25754 ;hkn; SS override
25755 000040B3 368C1E[8C05]   MOV     [ss:THISDPB+2],ds
25756 ;
25757 ;hkn; SS is DOSDATA
25758 000040B8 16      push    ss
25759 000040B9 1F      pop     ds
25760 ;
25761 000040BA 8936[8A05]   MOV     [THISDPB],SI
25762 ;
25763 000040BE 8B1E[2C03]   MOV     BX,[DMAADD]
25764 000040C2 891E[B805]   MOV     [NEXTADD],BX          ;Set NEXTADD to start of xaddr
25765 000040C6 C606[7405]00   MOV     BYTE [TRANS],0        ;No transferes
25766 ;mov     ax,[es:di+15h]
25767 000040CB 268B4515   MOV     ax,[es:DI+SF_ENTRY.sf_position]
25768 ;mov     dx,[es:di+17h]
25769 000040CF 268B5517   MOV     dx,[es:DI+SF_ENTRY.sf_position+2]
25770 000040D3 8916[D005]   MOV     [BYTPOS+2],dx          ;Set it
25771 000040D7 A3[CE05]     MOV     [BYTPOS],AX
25772 ;test    word [es:di+5],8080h
25773 000040DA 26F745058080   TEST    word [ES:DI+SF_ENTRY.sf_flags],sf_isnet+devid_device
25774 000040E0 7555      JNZ     short NOSETSTUFF        ;Following not done on devs or NET
25775 000040E2 06      PUSH    ES
25776 000040E3 C42E[8A05]   LES     BP,[THISDPB]          ;Point at the DPB
25777 ;

```



```

25778 ; 18/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
25779 ;mov bl,[es:bp+0]
25780 ;MOV BL,[ES:BP+DPB.DRIVE]
25781 ; 05/12/2022
25782 000040E7 268A5E00 mov bl,[es:bp]
25783
25784 000040EB 881E[7605] MOV [THISDRV],BL ;Set THISDRV
25785 ;mov bx,[es:bp+2]
25786 000040EF 268B5E02 MOV BX,[ES:BP+DPB.SECTOR_SIZE]
25787
25788 ;; MSDOS 3.3
25789 ;cmp dx,bx
25790 ;jnb short EOFERR
25791 ;div bx
25792 ;mov [SECPOS],ax
25793 ;mov [BYTSECPOS],dx
25794 ;mov dx,ax
25795 ;and al,[es:bp+4]
25796 ;AND AL,[ES:BP+DPB.CLUSTER_MASK]
25797 ;mov [SECCLUSPOS],al
25798 ;mov ax,cx
25799 ;mov cl,[es:bp+5]
25800 ;MOV CL,[ES:BP+DPB.CLUSTER_SHIFT]
25801 ;shr dx,cl
25802 ;mov [CLUSNUM],dx
25803 ;pop es
25804 ;mov cx,ax
25805
25806 ; 04/05/2019 - Retro DOS v4.0
25807
25808 ; MSDOS 6.0
25809 ;M039: Optimized this section.
25810 000040F3 51 PUSH CX ;SHR32 and DIV32 use CX.
25811 000040F4 E81406 call DIV32 ;DX:AX/BX = CX:AX + DX (rem)
25812 000040F7 8916[CC05] MOV [BYTSECPOS],DX
25813 000040FB A3[C405] MOV [SECPOS],AX
25814 000040FE 890E[C605] MOV [SECPOS+2],CX
25815 00004102 89CA MOV DX,CX
25816
25817 00004104 89C3 MOV BX,AX
25818 ;and bl,[es:bp+4]
25819 00004106 26225E04 AND BL,[ES:BP+DPB.CLUSTER_MASK]
25820 0000410A 881E[7305] MOV [SECCLUSPOS],BL
25821
25822 0000410E E82106 call SHR32 ;(DX:AX SHR dpb_cluster_shift)
25823 00004111 59 POP CX ;CX = byte count.
25824
25825 ; 09/02/2024 (PCDOS7.1 IBMDOS.COM)
25826 ;JNZ short EOFERR ;cluster number above 64k
25827 ;;;
25828 ;cmp word [es:bp+0Fh],0
25829 00004112 26837E0F00 cmp word [es:bp+DPB.FAT_SIZE],0
25830 00004117 750C jnz short setup_1 ; not FAT32 fs
25831 ;cmp dx,[es:bp+2Fh]
25832 00004119 263B562F cmp dx,[es:bp+DPB.LAST_CLUSTER+2]
25833 0000411D 750E jne short setup_2
25834 ;cmp ax,[es:bp+2Dh]
25835 0000411F 263B462D cmp ax,[es:bp+DPB.LAST_CLUSTER]
25836 00004123 EB08 jmp short setup_2
25837 setup_1:
25838 00004125 09D2 ; FAT12 or FAT16 fs
25839 00004127 7523 or dx,dx
25840 ;cmp ax,[es:bp+0Dh]
25841 00004129 263B460D cmp ax,[es:bp+DPB.MAX_CLUSTER]
25842 setup_2:
25843 0000412D 771D ja short EOFERR
25844 0000412F 8916[DE0A] mov [CLUSNUM_HW],dx
25845 ;;;
25846 ;cmp ax,[es:bp+0Dh]
25847 ;CMP AX,[ES:BP+DPB.MAX_CLUSTER] ;>32mb if > disk size ;AN000;
25848 ;JA short EOFERR ;>32mb then EOF ;AN000;
25849
25850
25851 00004133 A3[BC05] MOV [CLUSNUM],AX
25852 00004136 07 POP ES ; ES:DI point to SFT
25853 ;M039
25854
25855 NOSETSTUFF:
25856 00004137 89C8 MOV AX,CX ; AX = Byte count.
25857 00004139 0306[2C03] ADD AX,[DMAADD] ; See if it will fit in one segment
25858 0000413D 730C JNC short setup_OK ; Must be less than 64K
25859 0000413F A1[2C03] MOV AX,[DMAADD]
25860 00004142 F7D8 NEG AX ; Amount of room left in segment (know
25861 ; less than 64K since max value of CX
25862 ; is FFFF).
25863 00004144 7501 JNZ short NoDec
25864 00004146 48 DEC AX
25865 NoDec:
25866 00004147 89C1 MOV CX,AX ; Can do this much
25867 00004149 E304 JCXZ NOROOM ; Silly user gave xaddr of FFFF in segment
25868 setup_OK:
25869 0000414B C3 retn
25870
25871 EOFERR:
25872 0000414C 07 POP ES ; ES:DI point to SFT
25873 0000414D 31C9 XOR CX,CX ; No bytes read
25874 ;;;;;;;;;; 7/18/86
25875 ; MSDOS 3.3
25876 ;MOV BYTE [DISK_FULL],1 ; set disk full flag
25877 ;;;;;;;;;;
25878 NOROOM:
25879 0000414F 5B POP BX ; Kill return address
25880 00004150 F8 CLC
25881 00004151 C3 retn ; RETURN TO CALLER OF CALLER
25882
25883 ;Break <BREAKDOWN -- CUT A USER READ OR WRITE INTO PIECES>
25884 ;-----
25885 ;
25886 ; Procedure Name : BREAKDOWN
25887 ;
25888 ; Inputs:
25889 ; CX = Length of disk transfer in bytes
25890 ; ES:BP = Base of drive parameters
25891 ; [BYTSECPOS] = Byte position within first sector
25892 ; DS = DOSDATA
25893 ; Outputs:
25894 ; [BYTCNT1] = Bytes to transfer in first sector
25895 ; [SECCNT] = No. of whole sectors to transfer
25896 ; [BYTCNT2] = Bytes to transfer in last sector
25897 ; AX, BX, DX destroyed. No other registers affected.
25898 ;-----
25899
25900 BREAKDOWN:
25901 00004152 A1[CC05] MOV AX,[BYTSECPOS]

```

```

25902 00004155 89CB      MOV     BX,CX
25903 00004157 09C0      OR      AX,AX
25904 00004159 740E      JZ      short SAVFIR ; Partial first sector?
25905                      ;sub     ax,[es:bp+2]
25906 0000415B 262B4602    SUB     AX,[ES:BP+DPB.SECTOR_SIZE]
25907 0000415F F7D8      NEG     AX ; Max number of bytes left in first sector
25908 00004161 29C3      SUB     BX,AX ; Subtract from total length
25909 00004163 7304      JAE     short SAVFIR
25910 00004165 01D8      ADD     AX,BX ; Don't use all of the rest of the sector
25911 00004167 31DB      XOR     BX,BX ; And no bytes are left
25912 SAVFIR:
25913 00004169 A3[D205]    MOV     [BYTCNT1],AX
25914 0000416C 89D8      MOV     AX,BX
25915 0000416E 31D2      XOR     DX,DX
25916                      ;div     word [ES:BP+2]
25917 00004170 26F77602    DIV     word [ES:BP+DPB.SECTOR_SIZE] ; How many whole sectors?
25918 00004174 A3[D605]    MOV     [SECCNT],AX
25919 00004177 8916[D405]    MOV     [BYTCNT2],DX ; Bytes remaining for last sector
25920                      ; MSDOS 3.3
25921                      ;OR      DX,[BYTCNT1] ; SMR ONESECTORFIX BUGBUG
25922                      ;retnz   ; NOT (BYTCNT1 = BYTCNT2 = 0)
25923                      ;CMP     AX,1
25924                      ;retnz
25925                      ;MOV     AX,[ES:BP+DPB.SECTOR_SIZE] ; Buffer EXACT one sector I/O
25926                      ;MOV     [BYTCNT2],AX
25927                      ;MOV     [SECCNT],DX ; DX = 0
25928 _RET45:
25929 0000417B C3      retn
25930
25931 ; DOSCODE:77BFh (MSDOS 6.21, MSDOS.SYS)
25932
25933 ;-----
25934 ;
25935 ; Procedure Name : READ_LOCK_VIOLATION
25936 ;
25937 ; ES:DI points to SFT. This entry used by NET_READ
25938 ; Carry set if to return error (CX=0,AX=error_sharing_violation).
25939 ; Else do retrys.
25940 ; ES:DI,DS,CX preserved
25941 ;
25942 ;-----
25943
25944 READ_LOCK_VIOLATION:
25945 0000417C C606[7505]00    MOV     byte [READOP],0
25946 ERR_ON_CHECK:
25947                      ;;test   word [es:di+2],8000h
25948                      ;TEST    word [ES:DI+SF_ENTRY.sf_mode],sf_isFCB
25949                      ;JNZ     short HARD_ERR
25950
25951                      ; 04/05/2019
25952                      ;test   byte [es:di+3],80h
25953 00004181 26F6450380    TEST    byte [ES:DI+SF_ENTRY.sf_mode+1],(sf_isFCB>>8)
25954 00004186 7508      JNZ     short HARD_ERR
25955
25956                      ;PUSH    CX
25957                      ;;mov    cl,[es:di+2]
25958                      ;MOV     CL,[ES:DI+SF_ENTRY.sf_mode]
25959                      ;;and    cl,0F0h
25960                      ;AND     CL,SHARING_MASK
25961                      ;;cmp    cl,0
25962                      ;CMP     CL,SHARING_COMPAT
25963                      ;POP     CX
25964                      ;JNE     short NO_HARD_ERR
25965                      ; 21/09/2023
25966 00004188 268A4502    mov     al,[ES:DI+SF_ENTRY.sf_mode]
25967 0000418C 24F0      and     al,SHARING_MASK
25968                      ;cmp     al,SHARING_COMPAT
25969                      ;jne     short NO_HARD_ERR
25970 0000418E 7505      jnz     short NO_HARD_ERR
25971 HARD_ERR:
25972 00004190 E86742    call    LOCK_VIOLATION
25973 00004193 73E6      jnc     short _RET45 ; User wants Retrys
25974 NO_HARD_ERR:
25975 00004195 31C9      XOR     CX,CX ;No bytes transferred
25976                      ;mov     ax,21h
25977 00004197 B82100    MOV     AX,error_lock_violation
25978 0000419A F9      STC
25979 RET3: ; 06/02/2024
25980 0000419B C3      retn
25981
25982 ;-----
25983 ;
25984 ; Procedure Name : WRITE_LOCK_VIOLATION
25985 ;
25986 ; Same as READ_LOCK_VIOLATION except for READOP.
25987 ; This entry used by NET_WRITE
25988 ;
25989 ;-----
25990
25991 WRITE_LOCK_VIOLATION:
25992 0000419C C606[7505]01    MOV     byte [READOP],1
25993 000041A1 EBDE      JMP     short ERR_ON_CHECK
25994
25995 ; 04/05/2019 - Retro DOS v4.0
25996
25997 ; DOSCODE:77ECh (MSDOS 6.21, MSDOS.SYS)
25998
25999 ;Break <DISKREAD -- PERFORM USER DISK READ>
26000
26001 ;-----
26002 ;
26003 ; Procedure Name : DISKREAD
26004 ;
26005 ; Inputs:
26006 ; Outputs of SETUP
26007 ; Function:
26008 ; Perform disk read
26009 ; Outputs:
26010 ; Carry clear
26011 ; CX = No. of bytes read
26012 ; ES:DI point to SFT
26013 ; SFT offset and cluster pointers updated
26014 ; Carry set
26015 ; CX = 0
26016 ; ES:DI point to SFT
26017 ; AX has error code
26018 ;-----
26019
26020 ;hkn; called from disk.asm. DS already set up.
26021
26022 ; 18/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
26023 ; DOSCODE:77D8h (MSDOS 5.0, MSDOS.SYS)
26024
26025 ; 09/02/2024
26026 ; 06/02/2024 - Retro DOS v5.0

```

```

26026 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:81D5h
26027
26028 DISKREAD:
26029 ;mov ax,[es:di+11h]
26030 000041A3 268B4511 MOV AX,[ES:DI+SF_ENTRY.sf_size]
26031 ;mov bx,[es:di+13h]
26032 000041A7 268B5D13 MOV BX,[ES:DI+SF_ENTRY.sf_size+2]
26033 000041AB 2B06[CE05] SUB AX,[BYTPOS]
26034 000041AF 1B1E[D005] SBB BX,[BYTPOS+2]
26035 000041B3 7226 JB short RDERR ;Read starts past EOF
26036 000041B5 750A JNZ short ENUF ;More than 64k to EOF
26037 000041B7 09C0 OR AX,AX
26038 000041B9 7420 JZ short RDERR ;Read starts at EOF
26039 000041BB 39C8 CMP AX,CX
26040 000041BD 7302 JAE short ENUF ;I/O fits
26041 000041BF 89C1 MOV CX,AX ;Limit read to up til EOF
26042
26043 ENUF:
26044 ; MSDOS 3.3
26045 ;test byte [es:di+4],8
26046 ;TEST byte [ES:DI+SF_ENTRY.sf_attr],attr_volume_id
26047 ;jnz short SET_ACC_ERR
26048 ;call LOCK_CHECK
26049 ;jnb short _READ_OK
26050 ;call READ_LOCK_VIOLATION
26051 ;jnb short ENUF
26052 ;retn
26053
26054 000041C1 E868FE ; MSDOS 6.0
26055 ;call CHECK_READ_LOCK ;IFS. check read lock ;AN000;
26056 ;JNC short _READ_OK ; There are no locks
26057 ;retn
26058 000041C4 72D5 ; 06/02/2024
26059 ;jc short RET3
26060
26061 _READ_OK:
26062 000041C6 C42E[8A05] LES BP,[THISDPB]
26063 000041CA E885FF CALL BREAKDOWN
26064
26065 ; 10/02/2024
26066 %if 0
26067 MOV CX,[CLUSNUM] ; *
26068 ;;;
26069 ; 06/02/2023 (PCDOS 7.1 IBMDOS.COM)
26070 mov dx,[CLUSNUM_HW] ; *
26071 ;;;
26072 ;call FNDCLUS
26073 ; MSDOS 6.0 ;M022 conditional removed here
26074 JC short SET_ACC_ERR_DS ; fix to take care of I24 fail
26075 ; migrated from 330a - HKN
26076 %else
26077 ; 10/02/2024 - Retro DOS v5.0
26078 000041CD E86715 call FNDCLUS_X ; *
26079 000041D0 721A jc short SET_ACC_ERR ; ds=ss
26080 %endif
26081 ;;;
26082 ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
26083 000041D2 833E[F80A]00 cmp word [CLSKIP_HW],0
26084 000041D7 7502 jnz short RDERR
26085 ;;;
26086 ;OR CX,CX
26087 ;JZ short SKIPERR
26088 ; 06/02/2024
26089 000041D9 E318 jcxz SKIPERR
26090
26091 RDERR:
26092 000041DB B40E MOV AH,0EH ;MS. read/data/fail ;AN000;
26093 000041DD E97202 jmp WRTERR22
26094
26095 ;RDLASTJ:
26096 ;JMP RDLAST ;M039
26097
26098 SETSFTJ2:
26099 000041E0 E9B600 JMP SETSFT
26100
26101 CANNOT_READ:
26102 ; MSDOS 3.3
26103 ;POP CX ;M039.
26104 ; MSDOS 3.3 & MSDOS 6.0
26105 000041E3 59 POP CX ;Clean stack.
26106 000041E4 5B POP BX
26107 ;;;
26108 ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
26109 ;pop word [CCONTENT_HW] ; PCDOS 7.1 BUG!?
26110 ; I think, this would be 'pop word [ss:CCONTENT_HW]'
26111 ; because ds<>ss while jumping here.
26112 ; Erdogan Tan - 09/02/2024
26113 000041E5 368F06[EC0A] pop word [ss:CCONTENT_HW] ; *
26114 ;;;
26115 ;entry SET_ACC_ERR_DS
26116 SET_ACC_ERR_DS:
26117 ;hkn; SS is DOSDATA
26118 ;Context DS
26119 push ss
26120 pop ds
26121 000041EA 16
26122 000041EB 1F
26123
26124 ;entry SET_ACC_ERR
26125 SET_ACC_ERR:
26126 000041EC 31C9 XOR CX,CX
26127 ;mov ax,5
26128 000041EE B80500 MOV AX,error_access_denied
26129 000041F1 F9 STC
26130 000041F2 C3 retn
26131
26132 SKIPERR:
26133 000041F3 8916[BA05] MOV [LASTPOS],DX
26134 000041F7 891E[BC05] MOV [CLUSNUM],BX
26135 ;;;
26136 ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
26137 000041FB 8B1E[E80A] mov bx,[CLUSTNUM_HW]
26138 000041FF 891E[DE0A] mov [CLUSNUM_HW],bx
26139 ;;;
26140 00004203 833E[D205]00 CMP word [BYTCNT1],0
26141 00004208 7405 JZ short RDMID
26142
26143 0000420A E82016 call BUFRD
26144 ;JC short SET_ACC_ERR_DS ; ds<>ss ; 10/02/2024
26145 ; 10/02/2024
26146 ; ds=ss
26147 0000420D 72DD jc short SET_ACC_ERR
26148
26149 RDMID:

```

```

26150 0000420F 833E[D605]00      CMP     word [SECCNT],0
26151                               ;JZ      RDLAST ; 10/08/2018
26152 00004214 7466               JZ       short RDLAST
26153
26154 00004216 E8A816             call    NEXTSEC
26155 00004219 72C5             JC       short SETSFTJ2
26156
26157 0000421B C606[7405]01      MOV     BYTE [TRANS],1          ; A transfer is taking place
26158 ONSEC:
26159 00004220 8A16[7305]        MOV     DL,[SECCLUSPOS]        ; (dx/DL = Extent start) ((dh = ?))
26160 00004224 8B0E[D605]        MOV     CX,[SECCNT]
26161                               ;;;
26162                               ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
26163 00004228 8B1E[DE0A]        mov     bx,[CLUSNUM_HW]
26164 0000422C 891E[E80A]        mov     [CLUSTNUM_HW],bx
26165                               ;;;
26166 00004230 8B1E[BC05]        MOV     BX,[CLUSNUM]
26167 RDLP:
26168 00004234 E8D116            call    OPTIMIZE
26169                               ;JC      short SET_ACC_ERR_DS ; ds<>ss ; 10/02/2024
26170                               ; 10/02/2024
26171                               ; ds=ss
26172 00004237 72B3             JC       short SET_ACC_ERR
26173                               ;;;
26174                               ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
26175 00004239 FF36[EC0A]        push    word [CONTENT_HW]      ; (Next physical cluster, hw)
26176                               ;;;
26177 0000423D 57                PUSH    DI                    ;DI = Next physical cluster.
26178 0000423E 50                PUSH    AX                    ;AX = # of sectors remaining.
26179 0000423F 53                PUSH    BX                    ;[DMAADD+2]:BX = Transfer address.
26180                               ;mov    byte [ALLOWED],38h
26181 00004240 C606[4B03]38      MOV     byte [ALLOWED],Allowed_RETRY+Allowed_FAIL+Allowed_IGNORE
26182 00004245 8E1E[2E03]        MOV     DS,[DMAADD+2]
26183
26184 00004249 52                PUSH    DX                    ;[HIGH_SECTOR]:DX = phys. sector #.
26185 0000424A 51                PUSH    CX                    ;CX = # of contiguous sectors to read.
26186
26187                               ; 04/05/2019 - Retro DOS v4.0
26188
26189                               ; 09/02/2024 - Retro DOS v5.0
26190                               ; PCDOS 7.1 IBMDOS.COM - DOSCODE:8284h
26191                               ; Windows ME IO.SYS - BIOSCODE:0B69Eh
26192                               ; MSDOS 6.22 IO.SYS - DOSCODE:787Bh
26193
26194                               ; NOTE: Secondary Buffer Cache is not used in PCDOS 7.1 IBMDOS.COM.
26195
26196                               ;call    nul_sub          ; PCDOS 7.1 (nul_sub: retn)
26197                               ;                               ; 'nul_sub:' at DOSCODE:AD57h
26198
26199                               ; MSDOS 6.0 (& Windows ME)
26200                               ;call    SET_RQ_SC_PARMS      ;LB. do this for SC ;AN000;
26201
26202                               ; MSDOS 3.3 (& MSDOS 6.0)
26203 0000424B E8A0FD            call    DREAD
26204
26205                               ; 10/02/2024
26206                               ; ds<>ss
26207
26208                               ; MSDOS 3.3
26209                               ;pop     bx
26210                               ;pop     dx
26211                               ;JC      short CANOT_READ
26212                               ;add     bx,dx ; (bx = Extent end)
26213                               ;mov     al,[es:bp] ; mov al,[es:bp+0]
26214                               ;;mov    al,[ES:BP+DPB.DRIVE]
26215                               ;call    SETVISIT
26216                               ; ->***
26217 ;M039
26218                               ; MSDOS 6.0
26219 0000424E 59                pop     cx
26220 0000424F 5A                pop     dx
26221 00004250 368F06[0C06]      pop     word [ss:TEMP_VAR]
26222 00004255 728C             JC      short CANOT_READ ; * 09/02/2024
26223
26224 00004257 368C1E[0E06]      mov     [ss:TEMP_VAR2],ds
26225
26226                               ;
26227                               ; CX = # of contiguous sectors read. (These constitute a block of
26228                               ; sectors, also termed an "Extent".)
26229                               ; [HIGH_SECTOR]:DX = physical sector # of first sector in extent.
26230                               ; [TEMP_VAR2]:[TEMP_VAR] = Transfer address (destination data address).
26231                               ; ES:BP -> Drive Parameter Block (DPB).
26232                               ;
26233                               ; The Buffer Queue must now be scanned: the contents of any dirty
26234                               ; buffers must be "read" into the transfer memory block, so that the
26235                               ; transfer memory reflects the most recent data.
26236 0000425C E87600            call    DskRdBufScan
26237
26238                               ;Context DS
26239 0000425F 16                push    ss
26240 00004260 1F                pop     ds
26241
26242 00004261 59                pop     cx
26243 00004262 5B                pop     bx
26244
26245                               ;
26246                               ; CX = # of sector remaining.
26247                               ; BX = Next physical cluster.
26248
26249                               ;;;
26250 00004263 8F06[E80A]        pop     word [CLUSTNUM_HW]      ; (Next physical cluster, hw)
26251                               ;;;
26252
26253 ;M039
26254
26255 ;;;;
26256                               ; 25/07/2018 - Retro DOS v3.0
26257                               ; ***->
26258                               ; MSDOS 3.3
26259                               ; IBMDOS.COM (1987) - Offset 42BDh
26260 ;bufq:
26261                               ; DX = Extent start.
26262                               ; BX = Extent end.
26263                               ; AL = Drive #.
26264                               ; DS:DI-> 1st buffer in queue.
26265
26266                               ;
26267                               ; or     byte [di+5],20h
26268                               ; or     byte [DI+BUFFINFO.buf_flags],buf_visit ; Bit 5 = reserved
26269                               ; ;cmp    al,[di+4]
26270                               ; cmp    al,[DI+BUFFINFO.buf_ID]
26271                               ; jnz    short bufq3
26272                               ; ;cmp    [di+6],dx
26273                               ; cmp    [DI+BUFFINFO.buf_sector],dx
26274                               ; jb     short bufq3 ; Jump if Extent start > buffer sector.

```

```

26274 ; cmp [di+6],bx
26275 ; cmp [DI+BUFFINFO.buf_sector],bx
26276 ; jnb short bufq3 ; Jump if Extent end >= buffer sector.
26277 ;
26278 ; Buffer sector is in the Extent (contiguous sectors to read)
26279 ;
26280 ; Buffer's sector is in Extent: if it is dirty, copy its contents to
26281 ; transfer memory; otherwise, just re-position it in the buffer queue
26282 ; as MRU (Most Recently Used).
26283 ;
26284 ; test byte [di+5],40h
26285 ; test byte [DI+BUFFINFO.buf_flags],buf_dirty ; Bit 6 = dirty flag
26286 ; jz short bufq2 ; clear buffer, check the next buff sec
26287 ; pop ax ; transfer address
26288 ; push ax
26289 ; push di
26290 ; push dx
26291 ; sub dx,[di+6]
26292 ; sub dx,[DI+BUFFINFO.buf_sector]
26293 ; neg dx
26294 ;
26295 ; DX = offset (in sectors) of buffer sector within Transfer memory
26296 ; block.
26297 ;
26298 ; mov si,di
26299 ; mov di,ax
26300 ; mov ax,dx
26301 ; mov cx,[es:bp+6]
26302 ; mov cx,[ES:BP+DPB.SECTOR_SIZE] ; CX = sector size (in bytes).
26303 ; mul cx
26304 ; add di,ax
26305 ;
26306 ; lea si,[si+16]
26307 ; lea si,[SI+BUFINSIZ] ;DS:SI -> buffer data.
26308 ; shr cx,1
26309 ; push es
26310 ; mov es,[SS:DMAADD+2]
26311 ;
26312 ; CX = sector size (in WORDs) ; CF=1 if odd # of bytes.
26313 ; DS:SI-> Buffer sector data.
26314 ; ES:DI-> Destination within Transfer memory block.
26315 ;
26316 ; rep movsw ;Copy buffer sector to Transfer memory
26317 ; adc cx,0 ;CX=1 if odd # of bytes, else CX=0.
26318 ; rep movsb ;Copy last byte.
26319 ; jnc short bufq1
26320 ; movsb
26321 ;bufq1:
26322 ; pop es
26323 ; pop dx
26324 ; pop di
26325 ; mov al,[es:bp] ; mov al,[es:bp+0]
26326 ; mov al,[ES:BP+DPB.DRIVE]
26327 ;bufq2:
26328 ; call SCANPLACE
26329 ;bufq3:
26330 ; call SKIPVISIT
26331 ; jnz short bufq
26332 ;
26333 ; push ss
26334 ; pop ds
26335 ; pop cx
26336 ; pop cx
26337 ; pop bx
26338 ;bufq4:
26339 ;;;;;;
26340
26341 00004267 E313 JCXZ RDLAST
26342
26343 00004269 E84C20 call ISEOF ; test for eof on fat size
26344 0000426C 732B JAE short SETSFT
26345
26346 0000426E B200 MOV DL,0
26347 ; INC word [LASTPOS] ; we'll be using next cluster
26348 ;;;
26349 ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
26350 ; add word [LASTPOS],1
26351 ; adc word [LASTPOS_HW],0
26352 ; 09/02/2024 - Retro DOS v5.0
26353 00004270 FF06[BA05] inc word [LASTPOS]
26354 00004274 75BE jnz short RDLP
26355 00004276 FF06[E20A] inc word [LASTPOS_HW]
26356 ;;;
26357 0000427A EBB8 JMP short RDLP ; 19/05/2019
26358
26359 RDLAST:
26360 0000427C A1[D405] MOV AX,[BYTCNT2]
26361 0000427F 09C0 OR AX,AX
26362 00004281 7416 JZ short SETSFT
26363 00004283 A3[D205] MOV [BYTCNT1],AX
26364
26365 00004286 E83816 call NEXTSEC
26366 00004289 720E JC short SETSFT
26367
26368 0000428B C706[CC05]0000 MOV word [BYTSECPOS],0
26369 00004291 E89915 call BUFRD
26370 ; 10/08/2018
26371 00004294 7303 JNC short SETSFT
26372 ; JMP SET_ACC_ERR_DS
26373 ; 10/02/2024
26374 ; ds=ss
26375 00004296 E953FF jmp SET_ACC_ERR
26376
26377 ;-----
26378 ;
26379 ; Procedure Name : SETSFT
26380 ; Inputs:
26381 ; [NEXTADD],[CLUSNUM],[LASTPOS] set to determine transfer size
26382 ; and set cluster fields
26383 ; Function:
26384 ; Update [THISSFT] based on the transfer
26385 ; Outputs:
26386 ; sf_position, sf_lstclus, and sf_cluspos updated
26387 ; ES:DI points to [THISSFT]
26388 ; CX No. of bytes transferred
26389 ; Carry clear
26390 ;
26391 ;-----
26392
26393 ;entry SETSFT
26394
26395 ; 26/07/2018 - Retro DOS v3.0
26396
26397 ; 09/02/2024 - Retro DOS v5.0

```

```

26398             ; (PCDOS 7.1 IBMDOS.COM)
26399
26400 SETSFT:
26401     LES     DI,[THISSFT]
26402
26403 ; Same as SETSFT except ES:DI already points to SFT
26404 ;entry SETCLUS
26405 SETCLUS:
26406     MOV     CX,[NEXTADD]
26407     SUB     CX,[DMAADD]           ; Number of bytes transfered
26408     ;;test  word [es:di+5],80h
26409     ;TEST   word [ES:DI+SF_ENTRY.sf_flags],devid_device
26410     ;JNZ    short ADDREC         ; don't set clusters if device
26411
26412     ; 04/05/2019 - Retro DOS v4.0
26413     ;test   byte [es:di+5],80h
26414     TEST    byte [ES:DI+SF_ENTRY.sf_flags],devid_device
26415     JNZ     short ADDREC         ; don't set clusters if device
26416
26417     ;;
26418     ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
26419     mov     ax,[CLUSNUM_HW]
26420     mov     [es:di+37h],ax
26421     ;mov     [es:di+SF_ENTRY.sf_1stclus+2],ax
26422     mov     ax,[LASTPOS_HW]
26423     mov     [es:di+0Bh],ax
26424     ;mov     [es:di+SF_ENTRY.sf_c luspos_hw],ax ; PCDOS 7.1 & Win ME
26425     ;;mov    [es:di+SF_ENTRY.sf_firclus],ax ; MSDOS 5.0-6.22
26426     ;;
26427
26428     MOV     AX,[CLUSNUM]
26429     ;;mov    [es:di+1Bh],ax ; MSDOS 3.3
26430     ;;mov    [es:di+35h],ax ; MSDOS 6.0 (& MSDOS 6.21)
26431     MOV     [ES:DI+SF_ENTRY.sf_1stclus],AX
26432     MOV     AX,[LASTPOS]
26433     ;mov     [es:di+19h],ax
26434     MOV     [ES:DI+SF_ENTRY.sf_c luspos],AX
26435
26436 ;-----
26437 ;
26438 ; Procedure : AddRec
26439 ; Inputs:
26440 ;     ES:DI points to SFT
26441 ;     CX is No. Bytes transferred
26442 ; Function:
26443 ;     Update the SFT offset based on the transfer
26444 ; Outputs:
26445 ;     sf_position updated to point to first byte after transfer
26446 ;     ES:DI points to SFT
26447 ;     CX No. of bytes transferred
26448 ;     Carry clear
26449 ;-----
26450
26451 ;entry AddRec
26452 ADDREC:
26453     JCXZ     RET28               ; If no records read, don't change position
26454     ;add     [es:di+15h],cx
26455     ADD     [ES:DI+SF_ENTRY.sf_position],CX ; Update current position
26456     ;adc     word [es:di+17h], 0
26457     ADC     WORD [ES:DI+SF_ENTRY.sf_position+2],0
26458 RET28:
26459     CLC
26460     retn
26461
26462 ; 25/07/2018
26463 ; MSDOS 6.0
26464 ;Break <DskRdBufScan -- Disk Read Buffer Scan>
26465 ;-----
26466 ;
26467 ; Procedure Name : DskRdBufScan
26468 ;
26469 ; Inputs:
26470 ;     CX = # of contiguous sectors read. (These constitute a block of
26471 ;     sectors, also termed an "Extent".)
26472 ;     [HIGH_SECTOR]:DX = physical sector # of first sector in extent.
26473 ;     [TEMP_VAR2]:[TEMP_VAR] = Transfer address (destination data address).
26474 ;     ES:BP -> Drive Parameter Block (DPB).
26475 ;
26476 ; Function:
26477 ;     The Buffer Queue is scanned: the contents of any dirty buffers are
26478 ;     "read" into the transfer memory block, so that the transfer memory
26479 ;     reflects the most recent data.
26480 ;
26481 ; Outputs:
26482 ;     Transfer memory updated as required.
26483 ;
26484 ; Uses:
26485 ;     DS,AX,BX,CX,SI,DI destroyed.
26486 ;     SS override for all global variables.
26487 ;
26488 ; Notes:
26489 ;     FIRST_BUFF_ADDR is set-up to contain the LAST buffer to check, rather
26490 ;     than the FIRST.
26491 ;-----
26492 ;M039: Created
26493
26494 ; 04/05/2019 - Retro DOS v4.0
26495 ; DOSCODE:78F0h (MSDOS 6.21, MSDOS.SYS)
26496
26497 ; 18/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
26498 ; DOSCODE:78DCh (MSDOS 5.0, MSDOS.SYS)
26499
26500 ; 09/02/2024 - Retro DOS v5.0
26501 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:8313h
26502
26503 ;procedure DskRdBufScan,NEAR
26504 ;
26505 ;ASSUME DS:NOTHING
26506
26507 DskRdBufScan:
26508     cmp     word [ss:DirtyBufferCount],0 ; Any dirty buffers?
26509     je      short bufx             ; -no, skip all work.
26510
26511     mov     bx,[ss:HIGH_SECTOR]
26512     mov     si,bx
26513     add     cx,dx
26514     adc     si,0
26515
26516     call    GETCURHEAD             ;DS:DI -> 1st buf in queue.
26517     ;mov     ax,[di+2]
26518     mov     ax,[di+8+BUFFINFO.buf_prev]
26519     mov     [ss:FIRST_BUFF_ADDR],ax
26520
26521 ; 18/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)

```



```

26522             ;mov  al,[es:bp+0]
26523             ;mov  al,[es:bp+DPB.DRIVE]
26524             ; 15/12/2022
26525 000042F3 268A4600 mov    al,[es:bp]
26526
26527             ; BX:DX = Extent start.
26528             ; SI:CX = Extent end + 1.
26529             ; AL = Drive #.
26530             ; DS:DI-> 1st buffer in queue.
26531 ;[FIRST_BUFF_ADDR] = Address offset of last buffer in queue.
26532
26533 bufq:
26534             ;cmp    al,[di+4]
26535 000042F7 3A4504     cmp    al,[di+BUFFINFO.buf_ID] ;Same drive?
26536 000042FA 7514     jne    short bufq1          ; -no, jump.
26537
26538             ; Cmp32  bx,dx,<WORD PTR [di.buf_sector+2]>,<WORD PTR [di.buf_sector]>
26539             ; ja     short bufq1          ;Jump if Extent start > buffer sector.
26540
26541             ;cmp    bx,[di+8]
26542 000042FC 3B5D08     cmp    bx,[di+BUFFINFO.buf_sector+2]
26543 000042FF 7503     jne    short bufq01
26544             ;cmp    dx,[di+6]
26545 00004301 3B5506     cmp    dx,[di+BUFFINFO.buf_sector]
26546 bufq01:
26547 00004304 770A     ja     short bufq1
26548
26549             ; Cmp32  si,cx,<WORD PTR [di.buf_sector+2]>,<WORD PTR [di.buf_sector]>
26550             ; ja     short bufq2          ;Jump if Extent end >= buffer sector.
26551
26552             ;cmp    si,[di+8]
26553 00004306 3B7508     cmp    si,[di+BUFFINFO.buf_sector+2]
26554 00004309 7503     jne    short bufq02
26555             ;cmp    cx,[di+6]
26556 0000430B 3B4D06     cmp    cx,[di+BUFFINFO.buf_sector]
26557 bufq02:
26558 0000430E 770A     ja     short bufq2
26559 bufq1:
26560 00004310 363B3E[7E11] cmp    di,[ss:FIRST_BUFF_ADDR] ;Scanned entire buffer queue?
26561 00004315 8B3D     mov    di,[di]
26562             ;mov    di,[di+BUFFINFO.buf_next] ; Set-up for next buffer.
26563 00004317 75DE     jne    short bufq          ; -no, do next buffer
26564 bufx:
26565 00004319 C3         retn                ;Exit.
26566
26567             ; Buffer's sector is in Extent: if it is dirty, copy its contents to
26568             ; transfer memory; otherwise, just re-position it in the buffer queue
26569             ; as MRU (Most Recently Used).
26570
26571 bufq2:
26572 0000431A 50         push    ax
26573             ;test   byte [di+5],40h
26574 0000431B F6450540 test   byte [di+BUFFINFO.buf_flags],buf_dirty ;Buffer dirty?
26575 0000431F 7430     jz     short bufq3          ; -no, jump.
26576
26577             ; SaveReg <cx,dx,si,di,es>
26578             push    cx
26579             push    dx
26580             push    si
26581             push    di
26582             push    es
26583
26584             mov     ax,dx
26585             ;sub    ax,[di+6]
26586 00004328 2B4506     sub    ax,[di+BUFFINFO.buf_sector]
26587 0000432B F7D8     neg    ax
26588
26589             ; AX = offset (in sectors) of buffer sector within Transfer memory
26590             ; block. (Note: the upper word of the sector # may be ignored
26591             ; since no more than 64k bytes will ever be read. This 64k limit
26592             ; is imposed by the input parameters of the disk read operation.)
26593
26594             ;lea     si,[di+20]
26595 0000432D 8D7518     lea     si,[di+BUFINSZ] ;DS:SI -> buffer data.
26596             ;mov     cx,[es:bp+2]
26597 00004330 268B4E02 mov     cx,[es:bp+DPB.SECTOR_SIZE] ;CX = sector size (in bytes).
26598 00004334 F7E1     mul     cx ;AX = offset (in bytes) of buf. sector
26599             ;mov     di,[ss:TEMP_VAR]
26600             ; 09/02/2024
26601 00004336 36C43E[0C06] les     di,[ss:TEMP_VAR]
26602 0000433B 01C7     add     di,ax
26603             ;mov     es,[ss:TEMP_VAR2]
26604 0000433D D1E9     shr     cx,1
26605
26606             ; CX = sector size (in WORDs) ; CF=1 if odd # of bytes.
26607             ; DS:SI-> Buffer sector data.
26608             ; ES:DI-> Destination within Transfer memory block.
26609
26610 ; 09/02/2024 - Retro DOS v5.0
26611 %if 0
26612 rep     movsw                ;Copy buffer sector to Transfer memory
26613             ; 04/05/2019
26614             ;adc     cx,0          ;CX=1 if odd # of bytes, else CX=0.
26615             ;rep     movsb          ;Copy last byte.
26616             ;jnc     short bufq03
26617             ;movsb
26618             ; 18/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
26619             ;adc     cx,0
26620             ;rep     movsb
26621             ; 22/09/2023
26622             ;jnc     short bufq03
26623             movsb
26624 %else
26625             ;;;
26626             ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
26627 0000433F 36803E[6A00]00 cmp     byte [ss:DDMOVE],0
26628             ;jz     short bufq03+1 ; rep movsw (skip 32bit prefix)
26629             ; 06/04/2024
26630 00004345 7403     jz     short bufq03
26631 00004347 D1E9     shr     cx,1
26632 bufq03:
26633             ;rep     movsd
26634             ; 06/04/2024
26635 00004349 66         db     66h ; 32bit prefix
26636 bufq03:
26637 0000434A F3A5     rep     movsw
26638             ;;;
26639 %endif
26640
26641 bufq03: ; 09/02/2024
26642             ;RestoreReg <es,di,si,dx,cx>
26643 0000434C 07         pop     es
26644 0000434D 5F         pop     di
26645 0000434E 5E         pop     si

```

```

26646 0000434F 5A      pop     dx
26647 00004350 59      pop     cx
26648
26649 ;          DS:DI -> current buffer.
26650 bufq3:
26651 00004351 89F8      mov     ax,di          ;DS:AX -> Current buffer.
26652 ;invoke SCANPLACE
26653 00004353 E87525      call    SCANPLACE
26654 00004356 363B06[7E11]  cmp     ax,[ss:FIRST_BUFF_ADDR] ;Last buffer?
26655 0000435B 58          pop     ax
26656 ;jne short bufq      ; -no, jump.
26657 ;jmp short bufx       ; -yes, exit.
26658 ; 12/06/2019
26659 ;retn
26660 ; 18/11/2022 (MSDOS 5.0 MSDOS.SYS compability)
26661 0000435C 7599      jne     short bufq
26662 ;jmp short bufx
26663 ; 09/02/2024
26664 0000435E C3          retn     ; Exit
26665
26666 ;EndProc DskRdBufScan
26667
26668 ;=====
26669 ; DISK3.ASM, MSDOS 6.0, 1991
26670 ;=====
26671 ; 04/05/2019 - Retro DOS v4.0
26672 ; 24/07/2018 - Retro DOS v3.0
26673
26674 ;Break <DISKWRITE -- PERFORM USER DISK WRITE>
26675 ;-----
26676 ;
26677 ; Procedure Name : DISKWRITE
26678 ;
26679 ; Inputs:
26680 ;     Outputs of SETUP
26681 ; Function:
26682 ;     Perform disk write
26683 ; Outputs:
26684 ;     Carry clear
26685 ;     CX = No. of bytes written
26686 ;     ES:DI point to SFT
26687 ;     SFT offset and cluster pointers updated
26688 ;     Carry set
26689 ;     CX = 0
26690 ;     ES:DI point to SFT
26691 ;     AX has error code
26692 ;-----
26693
26694 ; DOSCODE:797Ah (MSDOS 6.21, MSDOS.SYS)
26695
26696 ; 20/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
26697 ; DOSCODE:7966h (MSDOS 5.0, MSDOS.SYS)
26698
26699 ; 10/02/2024
26700 ; 09/02/2024 - Retro DOS v5.0
26701 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:83A3h
26702
26703 ; (Windows ME IO.SYS - BIOSCODE:0B7B2h)
26704 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:797Ah)
26705
26706 DISKWRITE:
26707 ; MSDOS 3.3
26708 ; IBMDOS.COM - Offset 436Dh
26709 ; ;test byte [es:di+4],8
26710 ; ;TEST byte [ES:DI+SF_ENTRY.sf_attr],attr_volume_id
26711 ; ;jz short write_cont
26712 ; ;jmp SET_ACC_ERR_DS
26713 ;write_cont:
26714 ; ;push cx
26715 ; ;or cx,cx
26716 ; ;jnz short Not_Truncate
26717 ; ;mov cx,-1
26718 ; ;dec cx
26719 ;Not_Truncate:
26720 ; ;call LOCK_CHECK
26721 ; ;pop cx
26722 ; ;jnb short _WRITE_OK
26723 ; ;call WRITE_LOCK_VIOLATION
26724 ; ;jnb short DISKWRITE
26725 ; ;retn
26726
26727 ; MSDOS 6.0
26728 0000435F E8ACFC      call    CHECK_WRITE_LOCK      ;IFS. check write lock;AN000;
26729 ; 19/08/2018
26730 00004362 7304      JNC     short _WRITE_OK      ;IFS. lock check ok ;AN000;
26731 00004364 C3          retn
26732
26733 WRTEOFJ:
26734 00004365 E95602      JMP     WRTEOF
26735
26736 _WRITE_OK:
26737 ; 27/07/2018
26738 ; IBMDOS.COM - Offset 438Eh
26739
26740 ; MSDOS 3.3 (& MSDOS 6.0)
26741 ; and word [es:di+5],0BFBFh
26742 00004368 26816505BFBF AND     word [ES:DI+SF_ENTRY.sf_flags],~(sf_close_nodate|devid_file_clean)
26743 ; Mark file as dirty, clear no date on close
26744 ; 10/02/2024
26745 %if 0
26746 ; 04/05/2019 - Retro DOS v4.0
26747
26748 ; MSDOS 6.0
26749 ;mov ax,[es:di+11h]
26750 MOV     AX,[ES:DI+SF_ENTRY.sf_size]          ;M039
26751 MOV     [TEMP_VAR],AX                      ;M039
26752 ;mov ax,[es:di+13h]
26753 MOV     AX,[ES:DI+SF_ENTRY.sf_size+2]       ;M039
26754 MOV     [TEMP_VAR2],AX                     ;M039
26755 %else
26756 ; 10/02/2024 (PCDOS 7.1 IBMDOS COM)
26757 ;les ax,[es:di+11h]
26758 0000436E 26C44511 les     ax,[es:di+SF_ENTRY.sf_size]
26759 00004372 8C06[0E06] mov     [TEMP_VAR2],es
26760 00004376 A3[0C06] mov     [TEMP_VAR],ax
26761 %endif
26762
26763 ; TEMP_VAR2:TEMP_VAR = Current file size (sf_size);M039
26764
26765 ; MSDOS 3.3 (& MSDOS 6.0)
26766 00004379 C42E[8A05] LES     BP,[THISDPB]
26767
26768 0000437D E8D2FD      call    BREAKDOWN
26769

```



```

26770 00004380 A1[CE05]      MOV     AX,[BYTPOS]
26771 00004383 8B16[D005]      MOV     DX,[BYTPOS+2]
26772 00004387 E3DC          JCXZ     WRTEOFJ          ;Make the file length = sf_position
26773 00004389 01C8          ADD      AX,CX
26774 0000438B 83D200        ADC      DX,0          ;DX:AX = last byte to write + 1.
26775
26776      ;;;
26777      ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
26778 0000438E 7303      jnc      short dskwrt_1
26779 ACC_ERRWJ2:
26780      ;;;jmp     SET_ACC_ERRW
26781      ;;;jmp     SET_ACC_ERR_DS ; ds<>ss ; 10/02/2024
26782      ; 10/02/2024
26783      ; ds=ss
26784 00004390 E959FE      jmp      SET_ACC_ERR
26785
26786 dskwrt_1:
26787      ;test     byte [es:di+3],10h
26788 00004393 26F6450310      test     byte [es:di+SF_ENTRY.sf_mode+1],10h
26789 00004398 750B          jnz      short dskwrt_3          ; > 2GB file size (up to 4GB) allowed
26790 0000439A 81FAFF7F      cmp      dx,7FFFh          ; check for 2GB file size limit
26791 0000439E 7503          jne      short dskwrt_2
26792 000043A0 83F8FF      cmp      ax,0FFFFh
26793 dskwrt_2:
26794 000043A3 77EB      ja      short ACC_ERRWJ2 ; error,
26795      ; file position/pointer overs 2GB limit!
26796 dskwrt_3:
26797      ;;;
26798
26799      ;mov      bx,[es:bp+2]
26800 000043A5 268B5E02      MOV      BX,[ES:BP+DPB.SECTOR_SIZE]
26801
26802      ; MSDOS 3.3
26803      ;cmp      dx,bx
26804      ;jnb      short WRTERR33
26805      ;div      bx
26806      ;mov      bx,ax
26807      ;OR      DX,DX
26808      ;JNZ      short CALCLUS
26809      ;dec      ax
26810 ;CALCLUS:
26811      ; MSDOS 3.3
26812      ;mov      cl,[es:bp+5]
26813      ;MOV      CL,[ES:BP+DPB.CLUSTER_SHIFT]
26814      ;shr      ax,cl
26815      ;push     ax
26816      ;push     dx
26817      ;push     es
26818      ;les      di,[THISSFT]
26819      ;mov      ax,[es:di+11h]
26820      ;mov      dx,[es:di+13h]
26821      ;mov      ax,[ES:DI+SF_ENTRY.sf_size]
26822      ;mov      dx,[ES:DI+SF_ENTRY.sf_size+2]
26823      ;pop      es
26824      ;DX:AX = current file size (in bytes).
26825      ;div      word [es:bp+2]
26826      ;div      word [ES:BP+DPB.SECTOR_SIZE]
26827      ;mov      cx,ax
26828      ;or      dx,dx
26829      ;jz      short NORND
26830      ;inc      ax
26831 ;NORND:
26832      ; MSDOS 6.0
26833 000043A9 E85F03      CALL     DIV32          ;DX:AX/BX = CX:AX + DX (rem.).
26834 000043AC 89C6          MOV      SI,AX
26835 000043AE 890E[0706]      MOV      [HIGH_SECTOR],CX
26836
26837      ; [HIGH_SECTOR]:SI = Last full sector to write.
26838
26839      OR      DX,DX
26840 000043B4 52          PUSH     DX          ;M039: Free DX for use by SHR32
26841 000043B5 89CA          MOV      DX,CX          ;M039
26842 000043B7 7506          JNZ      short CALCLUS
26843 000043B9 83E801      SUB      AX,1          ;AX must be zero base indexed ;AC000;
26844 000043BC 83DA00      SBB      DX,0          ;M039 ;F.C. >32mb ;AN000;
26845
26846 CALCLUS:
26847      ; MSDOS 6.0
26848 000043BF E87003      CALL     SHR32          ;F.C. >32mb ;AN000;
26849      ;POP      DX
26850      ;;;
26851      ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
26852 000043C2 59          pop      cx
26853 000043C3 87CA          xchg     cx,dx
26854 000043C5 51          push     cx ; **!
26855      ; cx:ax = Last cluster to write
26856      ;;;
26857
26858      ; AX = Last cluster to write.
26859      ; DX = # of bytes in last sector to write (the "tail").
26860      ; BX = [ES:BP+DPB.SECTOR_SIZE]
26861
26862 000043C6 50          PUSH     AX ; *!
26863 000043C7 52          PUSH     DX
26864 ;M039
26865 000043C8 8B16[0E06]      mov      dx,[TEMP_VAR2]
26866 000043CC A1[0C06]      mov      ax,[TEMP_VAR]          ;DX:AX = current file size (in bytes).
26867 000043CF E83903      call     DIV32          ;DX:AX/BX = CX:AX + DX (rem.)
26868 000043D2 890E[0E06]      mov      [TEMP_VAR2],cx
26869 000043D6 890E[CA05]      mov      [VALSEC+2],cx
26870 000043DA 89C1          mov      cx,ax
26871 000043DC 89F3          mov      bx,si
26872
26873      ; [HIGH_SECTOR]:BX = Last full sector to write.
26874      ; [VALSEC+2]:CX = Last full sector of current file.
26875      ; [TEMP_VAR2]:CX = Last full sector of current file.
26876      ; DX = # of bytes in last sector of current file.
26877 ;M039
26878 000043DE 09D2      OR      DX,DX
26879 000043E0 7407      JZ      short NORND
26880      ;ADD      AX,1          ;Round up if any remainder ;AC000;
26881      ;ADC      word [VALSEC+2],0
26882      ; 22/09/2023
26883 000043E2 40          inc      ax ; 0FFFFh -> 0
26884 000043E3 7504          jnz      short NORND
26885 000043E5 FF06[CA05]      inc      word [VALSEC+2]
26886
26887 NORND:
26888      ; MSDOS 3.3 & MSDOS 6.0
26889 000043E9 A3[C805]      MOV      [VALSEC],AX
26890
26891      ; [VALSEC] = Last sector of current file.
26892 000043EC 31C0      XOR      AX,AX
26893 000043EE A3[DE05]      MOV      [GROWCNT],AX

```

```

26894 000043F1 A3[E005]      MOV     [GROWCNT+2],AX
26895 000043F4 58          POP     AX ; # of bytes in last sector to write (the "tail").
26896
26897      ; MSDOS 6.0
26898 000043F5 8B3E[0706]      MOV     DI,[HIGH_SECTOR] ;F.C. >32mb ;AN000;
26899 000043F9 3B3E[0E06]      CMP     DI,[TEMP_VAR2] ;M039; F.C. >32mb ;AN000;
26900 000043FD 726C          JB      short NOGROW ;F.C. >32mb ;AN000;
26901 000043FF 7408          JZ      short lowsec ;F.C. >32mb ;AN000;
26902 00004401 29CB          SUB     BX,CX ;F.C. >32mb ;AN000;
26903 00004403 1B3E[0E06]      SBB     DI,[TEMP_VAR2] ;M039; F.C. >32mb di:bx no. of sectors ;AN000;
26904 00004407 EB08          JMP     short yesgrow ;F.C. >32mb ;AN000;
26905
lowsec:
26906      ;MOV     DI,0 ;F.C. >32mb
26907      ; 22/09/2023
26908 00004409 31FF          xor     di,di
26909      ; MSDOS 3.3 & MSDOS 6.0
26910 0000440B 29CB          SUB     BX,CX ; Number of full sectors
26911 0000440D 725C          JB      short NOGROW
26912 0000440F 744D          JZ      short TESTTAIL
26913
yesgrow:
26914      ; MSDOS 3.3 (& MSDOS 6.0)
26915 00004411 89D1          MOV     CX,DX
26916 00004413 93          XCHG     AX,BX
26917      ;mul     word [es:bp+2]
26918 00004414 26F76602      MUL     word [ES:BP+DPB.SECTOR_SIZE] ; Bytes of full sector growth
26919
26920      ; MSDOS 6.0
26921 00004418 8916[0706]      MOV     [HIGH_SECTOR],DX ;F.C. >32mb save dx ;AN000;
26922 0000441C A3[0E06]      MOV     [TEMP_VAR2],AX ;M039; F.C. >32mb save ax ;AN000;
26923 0000441F 89F8          MOV     AX,DI ;F.C. >32mb ;AN000;
26924      ;mul     word [es:bp+2]
26925 00004421 26F76602      MUL     word [ES:BP+DPB.SECTOR_SIZE] ;F.C. >32mb do higher word multiply ;AN000;
26926
26927 00004425 0306[0706]      ADD     AX,[HIGH_SECTOR] ;F.C. >32mb add lower value ;AN000;
26928      ;MOV     DX,AX ;F.C. >32mb DX:AX is the result of ;AN000;
26929      ; 09/02/2024
26930 00004429 92          xchg     ax,dx
26931 0000442A A1[0E06]      MOV     AX,[TEMP_VAR2] ;M039; F.C. >32mb a 32 bit multiply ;AN000;
26932
26933      ; MSDOS 3.3 (& MSDOS 6.0)
26934 0000442D 29C8          SUB     AX,CX ; Take off current "tail"
26935 0000442F 83DA00      SBB     DX,0 ; 32-bit extension
26936 00004432 01D8          ADD     AX,BX ; Add on new "tail"
26937 00004434 83D200      ADC     DX,0 ; ripple tim's head off
26938 00004437 EB2B          JMP     SHORT SETGRW
26939
HAVSTART:
26940      ;int 3
26941      ;;;
26942      ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
26943      mov     [CLSKIP_HW],si
26944 00004439 8936[F80A]      mov     [CLSKIP_HW],si
26945      ;;;
26946 0000443D 89C1          MOV     CX,AX
26947 0000443F E85A13      call    SKPCLP
26948      ;;;
26949      ; 09/02/2024
26950 00004442 A1[F80A]      mov     ax,[CLSKIP_HW]
26951 00004445 39C8          cmp     ax,cx
26952 00004447 7502          jne     short HAVSTART_cont
26953      ;;;
26954      ; 10/02/2024
26955 00004449 E311      JCXZ     DOWRTJ
26956      ; 16/12/2022
26957      ;jcxz     DOWRT
26958      ; 20/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
26959      ;jcxz     DOWRTJ
26960      ;;;
26961      ; 09/02/2024
26962      HAVSTART_cont:
26963      ;mov     [CCOUNT_HW],ax
26964      ;call    ALLOCATE
26965      ; 07/07/2024
26966 0000444B E88815      call    ALLOCATE_X
26967      ; 10/02/2024
26968 0000444E 730C          JNC     short DOWRTJ
26969      ; 16/12/2022
26970      ;jnc     short DOWRT
26971      ; 20/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
26972      ;jnc     short DOWRTJ
26973      ; 10/02/2024
26974      ;entry    WRTERR
26975
WRTERR:
26976      MOV     AH,0FH ;MS. write/data/fail/abort ;AN000;
26977 00004450 B40F
26978
26979      ;entry    WRTERR22
26980      WRTERR22:
26981 00004452 A0[7605]      MOV     AL,[THISDRV] ;MS. ;AN000;
26982
26983      ; 27/07/2018
26984      WRTERR33:
26985      ;MOV     CX,0 ;No bytes transferred
26986 00004455 31C9          XOR     CX,CX
26987
26988 00004457 C43E[9E05]      LES     DI,[THISSFT]
26989      ;CLC ; 19/05/2019
26990      ; 20/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
26991      ; 16/12/2022
26992      ;clc
26993 0000445B C3          retn
26994
26995      ; 10/02/2024
26996      ; 16/12/2022
26997      ; 20/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
26998      DOWRTJ:
26999 0000445C EB79          JMP     short DOWRT
27000
27001      TESTTAIL:
27002 0000445E 29D0          SUB     AX,DX
27003 00004460 7609          JBE     short NOGROW
27004 00004462 31D2          XOR     DX,DX
27005
27006      SETGRW:
27007 00004464 A3[DE05]      MOV     [GROWCNT],AX
27008 00004467 8916[E005]      MOV     [GROWCNT+2],DX
27009      NOGROW:
27010      ; 09/02/2024
27011      ;POP     AX ; *!
27012
27013      ; 10/02/2024
27014      %if 0
27015      MOV     CX,[CLUSNUM] ; *+ ; First cluster accessed
27016      ;;;
27017      ; 09/02/2024
27018      mov     dx,[CLUSNUM_HW] ; *+

```

```

27018      ;;;
27019      call    FNDCLUS
27020  %else
27021      ; 10/02/2024 - Retro DOS v5.0
27022 0000446B E8C912      call    FNDCLUS_X ; *+
27023  %endif
27024      ;;;
27025      ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
27026      pop     ax ; *!
27027 0000446F 5E          pop     si ; **! ; si:ax = Last cluster
27028      ;;;
27029      JC      short ACC_ERRWJ ; ds=ss
27030      MOV     [CLUSNUM],BX
27031 00004476 8916[BA05]  MOV     [LASTPOS],DX
27032
27033 0000447A 29D0          SUB     AX,DX ; Last cluster minus current cluster
27034      ; 09/02/2024
27035      JZ      short DOWRT ; If we have last clus, we must have first
27036      ;;;
27037      ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
27038      sbb     si,[LASTPOS_HW]
27039 00004480 8B16[E80A]  mov     dx,[CLUSTNUM_HW]
27040 00004484 8916[DE0A]  mov     [CLUSNUM_HW],dx
27041 00004488 7504          jnz     short NOGROW2
27042 0000448A 09C0          or      ax,ax
27043 0000448C 7449          jz      short DOWRT
27044
27045 0000448E 3B0E[F80A]  cmp     cx,[CLSKIP_HW]
27046 00004492 7502          jne     short dskwrt_4
27047      ;;;
27048 00004494 E3A3          JCXZ    HAVSTART ; See if no more data
27049      dskwrt_4: ; 09/02/2024
27050 00004496 51          PUSH    CX ; No. of clusters short of first
27051      ;;;
27052      ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
27053 00004497 FF36[F80A]  push    word [CLSKIP_HW]
27054 0000449B 8936[F00A]  mov     [CCOUNT_HW],si
27055      ;;;
27056 0000449F 89C1          MOV     CX,AX
27057 000044A1 E83515      call    ALLOCATE
27058      ;;;
27059      ; 09/02/2024
27060 000044A4 8F06[F80A]  pop     word [CLSKIP_HW]
27061      ;;;
27062 000044A8 59          POP     CX
27063 000044A9 72A5          JC      short WRTERR
27064 000044AB 8B16[BA05]  MOV     DX,[LASTPOS]
27065
27066      ; 09/02/2024
27067  %if 0
27068      INC     DX
27069      DEC     CX
27070      JZ      short NOSKIP
27071  %else
27072      ;;;
27073      ; 09/02/2024 Retro DOS v5.0
27074      ; (PCDOS 7.1 IBMDOS.COM)
27075      add     dx,1
27076      adc     word [LASTPOS_HW],0
27077 000044AF 42          inc     dx
27078 000044B0 7504          jnz     short dskwrt_10
27079 000044B2 FF06[E20A]  inc     word [LASTPOS_HW]
27080
27081 000044B6 83E901      sub     cx,1
27082 000044B9 831E[F80A]00  sbb     word [CLSKIP_HW],0
27083 000044BE 7504          jnz     short dskwrt_5
27084 000044C0 09C9          or      cx,cx
27085 000044C2 7405          jz      short NOSKIP
27086
27087      dskwrt_5:
27088      ;;;
27089 000044C4 E8D512      call    SKPCLP
27090 000044C7 7222          JC      short ACC_ERRWJ ; ds=ss ; 10/02/2024
27091
27092
27093 000044C9 891E[BC05]  MOV     [CLUSNUM],BX
27094      ;;;
27095      ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
27096 000044CD A1[E80A]      mov     ax,[CLUSTNUM_HW]
27097 000044D0 A3[DE0A]      mov     [CLUSNUM_HW],ax
27098      ;;;
27099 000044D3 8916[BA05]  MOV     [LASTPOS],DX
27100
27101 000044D7 833E[D205]00  DOWRT:  CMP     word [BYTCNT1],0
27102 000044DC 7410          JZ      short WRTMID
27103      ;;;
27104      ; 09/02/2024
27105 000044DE 8B1E[DE0A]  mov     bx,[CLUSNUM_HW]
27106 000044E2 891E[E80A]  mov     [CLUSTNUM_HW],bx
27107      ;;;
27108      ; 09/02/2024
27109      MOV     BX,[CLUSNUM] ; (not used in 'BUFWRT') ; 09/02/2024
27110 000044E6 E87D13      call    BUFWRT
27111      JC      short ACC_ERRWJ
27112      ; 09/02/2024
27113 000044E9 7303          jnc     short WRTMID
27114
27115      ; 09/02/2024
27116  ACC_ERRWJ:
27117      ; 10/08/2018
27118      ; JMP     SET_ACC_ERRW
27119      ; 16/12/2022
27120      ; jmp     SET_ACC_ERR_DS ; ds<>ss ; 10/02/2024
27121      ; 10/02/2024
27122      ; ds=ss
27123 000044EB E9FEFC      jmp     SET_ACC_ERR
27124      ; 20/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
27125      ; jmp     SET_ACC_ERRW
27126
27127
27128 000044EE A1[D605]      WRTMID: MOV     AX,[SECCNT]
27129 000044F1 09C0          OR      AX,AX
27130      ; 20/11/2022
27131      JZ      short WRTLAST ; 24/07/2019 ;M039
27132      ; 10/02/2024
27133 000044F3 7503          jnz     short WRTMID2
27134 000044F5 E98600      jmp     WRTLAST
27135
27136 000044F8 0106[C405]  WRTMID2: ADD     [SECPOS],AX
27137      ; 19/05/2019
27138      ; MSDOS 6.0
27139 000044FC 8316[C605]00  ADC     WORD [SECPOS+2],0 ; F.C. >32mb ; AN000;
27140 00004501 E8BD13      call    NEXTSEC
27141      ; 16/12/2022

```

```

27142 00004504 72E5          JC      short ACC_ERRWJ
27143                      ;JC      short SET_ACC_ERRW      ;M039
27144 00004506 C606[7405]01  MOV     BYTE [TRANS],1      ; A transfer is taking place
27145 0000450B 8A16[7305]    MOV     DL,[SECCCLUSPOS]      ; (dx/DL = Extent start) ((dh = ?))
27146                      ;;;
27147                      ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
27148 0000450F 8B1E[DE0A]    MOV     bx,[CLUSNUM_HW]
27149 00004513 891E[E80A]    MOV     [CLUSTNUM_HW],bx
27150                      ;;;
27151 00004517 8B1E[BC05]    MOV     BX,[CLUSNUM]
27152 0000451B 8B0E[D605]    MOV     CX,[SECCNT]
27153 WRTLPL:
27154 0000451F E8E613        call    OPTIMIZE
27155                      ; 09/02/2024
27156                      ;JC      short SET_ACC_ERRW
27157                      ; 16/12/2022
27158 00004522 72C7        JC      short ACC_ERRWJ      ; ds=ss ; 10/02/2024
27159
27160                      ;M039
27161                      ; DI = Next physical cluster.
27162                      ; AX = # sectors remaining.
27163                      ; [DMAADD+2]:BX = transfer address (source data address).
27164                      ; CX = # of contiguous sectors to write. (These constitute a block of
27165                      ; sectors, also termed an "Extent".)
27166                      ; [HIGH_SECTOR]:DX = physical sector # of first sector in extent.
27167                      ; ES:BP -> Drive Parameter Block (DPB).
27168                      ;
27169                      ; Purge the Buffer Queue and the Secondary Cache of any buffers which
27170                      ; are in Extent; they are being over-written.
27171
27172                      ;;;
27173                      ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
27174 00004524 FF36[EC0A]    push    word [CCCONTENT_HW]
27175                      ;;;
27176
27177 00004528 57            push    di      ; CCONTENT_HW:DI = Next physical cluster
27178 00004529 50            push    ax      ; AX = # sectors remaining
27179
27180                      ; MSDOS 3.3
27181                      ; IBMDOS.COM (1987) - Offset 4497h
27182                      ;push    dx
27183                      ;push    bx
27184                      ;mov     al,[es:bp]
27185                      ;;mov    AL,[ES:BP+DPB.DRIVE] ; mov al,[es:bp+0]
27186                      ;mov     bx,cx
27187                      ;add     bx,dx      ; (bx = Extent end)
27188
27189                      ; DX = Extent start.
27190                      ; BX = Extent end.
27191                      ; AL = Drive #.
27192
27193                      ;call    SETVISIT
27194
27195 wbufq1:
27196                      ;;or     byte [di+5],20h
27197                      ;or     byte [DI+BUFFINFO.buf_flags],buf_visit ; Bit 5 = reserved
27198                      ;;cmp    al,[di+4]
27199                      ;cmp    al,[DI+BUFFINFO.buf_ID]
27200                      ;jnz     short wbufq2      ; Jump if Extent start > buffer sector.
27201                      ;;cmp    [di+6],dx
27202                      ;cmp    [DI+BUFFINFO.buf_sector],dx
27203                      ;jb     short wbufq2
27204                      ;;cmp    [di+6],bx
27205                      ;cmp    [DI+BUFFINFO.buf_sector],bx
27206                      ;jnb     short wbufq2      ; Jump if Extent end >= buffer sector.
27207
27208                      ;; Buffer sector is in the Extent
27209
27210                      ;;mov    word [di+4],20FFh
27211                      ;mov    word [DI+BUFFINFO.buf_ID],20FFh
27212                      ;                      ; .buf_ID,      AL = FFh (Free buffer)
27213                      ;                      ; .buf_flags, AH = 0, reset/clear
27214                      ;call    SCANPLACE
27215 wbufq2:
27216                      ;call    SKIPVISIT
27217                      ;jnz     short wbufq1
27218                      ;pop     bx
27219                      ;pop     dx
27220
27221                      ; MSDOS 6.0
27222 0000452A E89101        call    DskWrtBufPurge      ;DS trashed.
27223
27224                      ;ASSUME DS:NOTHING
27225                      ;M039
27226                      ; MSDOS 3.3 & MSDOS 6.0
27227                      ;hkn: SS override for DMAADD and ALLOWED
27228 0000452D 368E1E[2E03]    MOV     DS,[SS:DMAADD+2]
27229                      ;mov     byte [ss:ALLOWED],38h
27230 00004532 36C606[4B03]38 MOV     byte [SS:ALLOWED],Allowed_RETRY+Allowed_FAIL+Allowed_IGNORE
27231
27232                      ; put logic from DWRITE in-line here so we can modify it
27233                      ; for DISK FULL conditions.
27234
27235                      ; 20/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
27236                      ; DOSCODE:7AD8h (MSDOS 5.0 MSDOS.SYS)
27237
27238                      ; 16/12/2022
27239                      ; MSDOS 3.3 (& MSDOS 5.0)
27240                      ;call    DWRITE
27241
27242 DWRITE_OKAY:
27243
27244                      ; 16/12/2022
27245                      ; MSDOS 5.0 (& MSDOS 3.3)
27246                      ;pop     cx
27247                      ;pop     bx
27248                      ;push    ss
27249                      ;pop     ds
27250                      ;jc      short SET_ACC_ERRW
27251                      ;jcxz    WRTLAST
27252                      ;mov     dl,0
27253                      ;inc     word [LASTPOS]
27254                      ;jmp     short WRTLPL
27255
27256                      ; 16/12/2022
27257                      ; 20/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
27258 DWRITE_LUP:
27259                      ; 23/07/2019 - Retro DOS v3.2
27260
27261                      ; MSDOS 6.0
27262 00004538 E82CFB        call    DSKWRITE
27263 0000453B 7417        jz      short DWRITE_OKAY ; ds<>ss
27264
27265                      ;; int      3

```

```

27266
27267 0000453D 3C27      cmp     al,error_handle_Disk_Full      ; compressed volume full?
27268 0000453F 742D      jz      short DWRITE_DISK_FULL
27269
27270      ; 16/12/2022
27271
27272      ;;hkn; SS override
27273 00004541 36C606[7505]01      MOV     BYTE [SS:READOP],1
27274 00004547 E84CFB      call    HARDERRRW
27275 0000454A 3C01      CMP     AL,1      ; Check for retry
27276 0000454C 74EA      JZ      short DWRITE_LUP
27277
27278      ; 16/12/2022
27279      ; 23/07/2019
27280      ; POP     CX ; *4*
27281      ; POP     BX ; *5*
27282
27283      ;
27284      ; push    ss
27285      ; pop     ds
27286
27287      ; 20/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
27288
27289      ; 16/12/2022
27290 0000454E 3C03      CMP     AL,3      ; Check for FAIL
27291 00004550 F8      CLC
27292 00004551 7501      JNZ     short DWRITE_OKAY ; Ignore
27293 00004553 F9      STC
27294
27295 DWRITE_OKAY:
27296      ; 16/12/2022
27297      ; 23/07/2019
27298      ; MSDOS 3.3 (& MSDOS 6.0)
27299 00004554 59      POP     CX ; *4*
27300 00004555 5B      POP     BX ; *5*
27301
27302      ; CX = # sectors remaining.
27303      ; BX = Next physical cluster.
27304
27305      ;;hkn; SS override
27306      ; Context DS
27307      ; 16/12/2022
27308      ; push    ss
27309      ; pop     ds
27310
27311      ; 09/02/2024 - Retro DOS v5.0
27312      ; 16/12/2022
27313      ; jc      short SET_ACC_ERRW
27314
27315      ; 16/12/2022
27316 00004556 16      push    ss
27317 00004557 1F      pop     ds
27318
27319      ;;
27320      ; 09/02/2024 - Retro DOS v5.0
27321      ; (PCDOS 7.1 IBMDOS.COM)
27322 00004558 8F06[E80A]      pop     word [CLUSTNUM_HW] ; CLUSTNUM_HW:BX = Next physical cluster
27323 0000455C 725D      jc      short SET_ACC_ERRW ; ds=ss
27324      ;;
27325
27326 0000455E E31E      JCXZ    WRTLAST
27327
27328      ; 10/02/2024
27329 00004560 B200      MOV     DL,0
27330      ; xor     dl,dl ; 23/07/2019
27331 00004562 FF06[BA05]      INC     word [LASTPOS]; We'll be using next cluster
27332      ;;
27333      ; 09/02/2024 - Retro DOS v5.0
27334      ; (PCDOS 7.1 IBMDOS.COM)
27335 00004566 75B7      jnz     short WRTLP
27336 00004568 FF06[E20A]      inc     word [LASTPOS_HW]
27337      ;;
27338 0000456C EBB1      JMP     short WRTLP
27339
27340      ; 23/07/2019 - Retro DOS v3.2
27341      ; 09/08/2018
27342      ; MSDOS 6.0
27343 DWRITE_DISK_FULL:
27344      ; Context DS      ; SQ 3-5-93 DS must be setup on return!
27345      ; 16/12/2022
27346 0000456E 16      push    ss
27347 0000456F 1F      pop     ds
27348 00004570 59      pop     cx      ; unjunk stack
27349 00004571 5B      pop     bx
27350      ;;
27351      ; 09/02/2024 (PCDOS 7.1 IBMDOS.COM)
27352 00004572 8F06[E80A]      pop     word [CLUSTNUM_HW]
27353      ;;
27354 00004576 C606[0B06]01      mov     byte [DISK_FULL],1
27355      ; stc
27356 0000457B E9D2FE      jmp     WRTERR ; 24/07/2019 ; go to disk full exit
27357
27358 WRTLAST:
27359 0000457E A1[D405]      MOV     AX,[BYTCNT2]
27360 00004581 09C0      OR      AX,AX
27361 00004583 7413      JZ      short FINWRT
27362 00004585 A3[D205]      MOV     [BYTCNT1],AX
27363 00004588 E83613      call    NEXTSEC
27364 0000458B 722E      JC      short SET_ACC_ERRW
27365 0000458D C706[CC05]0000      MOV     word [BYTSECPOS],0
27366 00004593 E8D012      call    BUFWRT
27367 00004596 7223      JC      short SET_ACC_ERRW
27368
27369 FINWRT:
27370 00004598 C43E[9E05]      LES     DI,[THISSFT]
27371 0000459C A1[DE05]      MOV     AX,[GROWCNT]
27372 0000459F 8B0E[E005]      MOV     CX,[GROWCNT+2]
27373 000045A3 09C0      OR      AX,AX
27374 000045A5 7502      JNZ     short UPDATE_size
27375 000045A7 E30F      JCXZ    SAMSIZ
27376
27377 UPDATE_size:
27378 000045A9 26014511      ; add     [es:di+11h],ax
27379      ADD     [ES:DI+SF_ENTRY.sf_size],AX
27380 000045AD 26114D13      ; adc     [es:di+13h],cx
27381      ADC     [ES:DI+SF_ENTRY.sf_size+2],CX
27382
27383      ; Make sure that all other SFT's see this growth also.
27384 000045B1 B80100      MOV     AX,1
27385      ; if installed
27386      ; call    JShare + 14 * 4
27387 000045B4 FF1E[C800]      call    far [JShare+(14*4)] ; 14 = shsu
27388      ; else
27389      ; call    shsu

```

```

27390 ;endif
27391
27392 SAMSIZ:
27393 jmp SETCLUS; ES:DI already points to SFT
27394
27395 ; 16/12/2022
27396 ; 20/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
27397 SET_ACC_ERRW:
27398 ; jmp SET_ACC_ERR_DS ; ds<>ss ; 10/02/2024
27399 ; 10/02/2024
27400 ; ds=ss
27401 jmp SET_ACC_ERR
27402
27403 WRTEOF:
27404 MOV CX,AX
27405 OR CX,DX
27406 ;JZ short KILLFIL
27407 ; 10/02/2024
27408 jnz short WRTEOF1
27409 jmp KILLFIL
27410
27411 ;;;
27412 ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
27413 WRTEOF1:
27414 cmp dx,7FFFh ; 2GB file size limit
27415 jne short WRTEOF2
27416 cmp ax,0FFFFh
27417 WRTEOF2:
27418 jna short WRTEOF3
27419 ;
27420 push es
27421 ; les di,ds:THISSFT
27422 ; 27/06/2024
27423 les di,[THISSFT]
27424 ; test byte [es:di+3],10h
27425 test byte [es:di+SF_ENTRY.sf_mode+1],10h
27426 pop es
27427 jz short SET_ACC_ERRW ; > 2GB file size not allowed
27428 WRTEOF3:
27429 ;;;
27430
27431 SUB AX,1
27432 SBB DX,0
27433
27434 ; MSDOS 3.3
27435 ; div word [es:bp+2]
27436 ; div word [ES:BP+DPB.SECTOR_SIZE]
27437 ; mov cl,[es:bp+5]
27438 ; mov cl,[ES:BP+DPB.CLUSTER_SHIFT]
27439 ; shr ax,cl
27440
27441 ; MSDOS 6.0
27442 PUSH BX
27443 mov bx,[es:bp+2]
27444 MOV BX,[ES:BP+DPB.SECTOR_SIZE] ; F.C. >32mb ; AN000;
27445 CALL DIV32 ; F.C. >32mb ; AN000;
27446 POP BX ; F.C. >32mb ; AN000;
27447 MOV DX,CX ; M039
27448 MOV [HIGH_SECTOR],CX ; M039: Probably extraneous, but not sure.
27449 CALL SHR32 ; F.C. >32mb ; AN000;
27450
27451 MOV CX,AX
27452 call FNDCLUS
27453 SET_ACC_ERRWJ2:
27454 JC short SET_ACC_ERRW
27455 ;;;
27456 ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
27457 mov ax,[CLSKIP_HW]
27458 cmp ax,cx
27459 jne short dskwrt_6
27460 ;;;
27461
27462 JCXZ RELFILE
27463
27464 ;;;
27465 ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
27466 dskwrt_6:
27467 ; mov [CCOUNT_HW],ax
27468 ;;;
27469 ;
27470 ; call ALLOCATE
27471 ; 07/07/2024
27472 call ALLOCATE_X
27473 ; JC short WRTERRJ ;;;;;;;;;; disk full
27474 ; 16/12/2022
27475 jnc short UPDATE
27476 JMP WRTERR
27477 UPDATE:
27478 LES DI,[THISSFT]
27479 MOV AX,[BYTPOS]
27480 ; mov [es:di+11h],ax
27481 MOV [ES:DI+SF_ENTRY.sf_size],AX
27482 MOV AX,[BYTPOS+2]
27483 ; mov [es:di+13h],ax
27484 MOV [ES:DI+SF_ENTRY.sf_size+2],AX
27485 ;
27486 ; Make sure that all other SFT's see this growth also.
27487 ;
27488 MOV AX,2
27489 ; if installed
27490 ; call JShare + 14 * 4
27491 call far [JShare+(14*4)] ; 14 = Shsu
27492 ; else
27493 ; call Shsu
27494 ; endif
27495 XOR CX,CX ; 0
27496 ; jmp ADDREC
27497 ; 08/02/2024
27498 retn
27499
27500 ; 16/12/2022
27501 ; WRTERRJ:
27502 ; JMP WRTERR
27503
27504 ;;;;;;;;;; 7/18/86
27505 ;;;;;;;;;;
27506
27507 RELFILE:
27508 ; MSDOS 6.0
27509 PUSH ES ; AN002; BL Reset Lstclus and cluspos to
27510 LES DI,[THISSFT] ; AN002; BL beginning of file if current
27511 ;;;
27512 ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
27513 push ax ; cluspos is past EOF.

```



```

27514 00004631 A1[E20A]      mov     ax,[LASTPOS_HW]
27515 00004634 263B450B      cmp     ax,[es:di+0Bh]      ; [ES:DI+SF_ENTRY.sf_firclus]
27516                                ;cmp
27517 00004638 58             pop     ax
27518 00004639 7504          jne     short dskwrt_7
27519                                ;;;
27520                                ;cmp
27521 0000463B 263B5519      CMP     DX,[ES:DI+SF_ENTRY.sf_cluspos]      ;AN002; BL cluspos is past EOF.
27522 dskwrt_7:                ; 10/02/2024
27523 0000463F 731C          JAE     short SKIPRESET      ;AN002; BL
27524                                ;;;
27525                                ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
27526 00004641 268B552B      mov     dx,[es:di+2Bh]      ; [ES:DI+SF_ENTRY.sf_chain]
27527                                ;mov
27528                                dx,[es:di+SF_ENTRY.sf_fccluster]      ; first cluster (32 bit) lw !?
27529                                ;;;
27530 00004645 26C745190000    MOV     word [es:di+19h],0
27531                                MOV     word [ES:DI+SF_ENTRY.sf_cluspos],0 ;AN002; BL
27532                                ; 10/02/2024
27533 %if 0
27534                                ;mov
27535                                dx,[es:di+0Bh]
27536                                MOV     DX,[ES:DI+SF_ENTRY.sf_firclus]      ;AN002; BL
27537 %else
27538                                ;;;
27539                                ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
27540                                mov     word [es:di+0Bh],0
27541                                ;mov
27542                                word [es:di+SF_ENTRY.sf_cluspos_hw],0
27543 %endif
27544                                ;mov
27545                                [es:di+35h],dx
27546                                MOV     [ES:DI+SF_ENTRY.sf_lstclus],DX      ;AN002; BL
27547                                ;;;
27548                                ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
27549                                mov     dx,[es:di+2Dh]      ; [ES:DI+SF_ENTRY.sf_chain]
27550                                ;mov
27551                                di,[es:di+SF_ENTRY.sf_fccluster+2] ; first clust (32 bit) hw !?
27552                                mov     [es:di+37h],dx
27553                                ;mov
27554                                [es:di+SF_ENTRY.sf_lstclus+2],dx
27555                                ;;;
27556 SKIPRESET:
27557 0000465D 07             POP     ES      ;AN002; BL
27558                                ;AN002; BL
27559                                ;
27560                                ; 10/02/2024
27561 %if 0
27562                                MOV     DX,0FFFFH
27563 %else
27564                                ;;;
27565                                ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
27566                                xor     dx,dx
27567                                dec     dx
27568                                mov     [CLUSDATA_HW],dx ; 0FFFFh
27569                                ;;;
27570 %endif
27571                                call    RELBLKS
27572                                ; 16/12/2022
27573                                ; 20/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
27574 dskwrt_8:                ; 10/02/2024
27575                                jnc     short UPDATE
27576 SET_ACC_ERRWJ:
27577                                ;JC     short SET_ACC_ERRWJ2
27578                                ;JMP     SHORT UPDATE
27579                                ; 16/12/2022
27580                                ;jmp     SET_ACC_ERR_DS ; ds<>ss
27581                                ; 10/02/2024
27582                                ; ds=ss
27583                                jmp     SET_ACC_ERR
27584                                ; 20/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
27585                                ;JC     short SET_ACC_ERRWJ2
27586                                ;JMP     SHORT UPDATE
27587 KILLFIL:
27588                                XOR     BX,BX
27589                                PUSH    ES
27590                                LES     DI,[THISSFT]
27591                                ;
27592                                ; 10/02/2024
27593 %if 0
27594                                ;mov
27595                                [es:di+19h],bx
27596                                MOV     [ES:DI+SF_ENTRY.sf_cluspos],BX
27597                                ;mov
27598                                [es:di+35h],bx ; 04/05/2019
27599                                MOV     [ES:DI+SF_ENTRY.sf_lstclus],BX
27600                                ;xchg
27601                                bx,[es:di+0Bh]
27602                                XCHG    BX,[ES:DI+SF_ENTRY.sf_firclus]
27603 %else
27604                                ;;;
27605                                ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
27606                                xchg    bx,[es:di+2Dh] ; [ES:DI+SF_ENTRY.sf_chain+2]
27607                                ;xchg
27608                                bx,[es:di+SF_ENTRY.sf_fccluster+2] ; first clust (32 bit) hw !?
27609                                mov     [CLUSTNUM_HW],bx
27610                                xor     bx,bx ; 0
27611                                mov     [es:di+0Bh],bx ; [ES:DI+SF_ENTRY.sf_firclus]
27612                                ;mov
27613                                [es:di+SF_ENTRY.sf_cluspos_hw],bx
27614                                mov     [es:di+37h],bx
27615                                ;mov
27616                                [es:di+SF_ENTRY.sf_lstclus+2],bx
27617                                ;mov
27618                                [es:di+19h],bx
27619                                mov     [es:di+SF_ENTRY.sf_cluspos],bx
27620                                ;mov
27621                                [es:di+35h],bx
27622                                mov     [es:di+SF_ENTRY.sf_lstclus],bx
27623                                xchg    bx,[es:di+2Bh] ; [ES:DI+SF_ENTRY.sf_chain]
27624                                ;xchg
27625                                bx,[es:di+SF_ENTRY.sf_fccluster]      ; first cluster (32 bit) lw !?
27626                                ;;;
27627 %endif
27628                                POP     ES
27629                                ;;;
27630                                ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
27631                                cmp     bx,[CLUSTNUM_HW]
27632                                jnz     short dskwrt_9
27633                                ;;;
27634                                OR     BX,BX
27635                                ;JZ     short UPDATEJ
27636                                ; 16/12/2022
27637                                ;jz     short UPDATE
27638                                ; 10/12/2024
27639                                jnz     short dskwrt_9
27640                                jmp     UPDATE
27641                                ;
27642                                ; 20/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
27643                                ;jz     short UPDATEJ
27644 dskwrt_9:                ; 10/02/2024
27645                                ; 10/23/86 FastOpen update
27646                                PUSH    ES      ; since first cluster # is 0
27647 000046A0 06

```

```

27638 000046A1 55      PUSH    BP                ; we must delete the old cache entry
27639 000046A2 50      PUSH    AX
27640 000046A3 51      PUSH    CX
27641 000046A4 52      PUSH    DX
27642 000046A5 C42E[8A05]  LES     BP,[THISDPB]        ; get current DPB
27643                      ; 15/12/2022
27644 000046A9 268A5600  mov     dl,[ES:BP] ; mov dl,[es:bp+0]
27645                      ; 20/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
27646                      ;MOV     DL,[ES:BP+DPB.DRIVE] ; get current drive
27647 000046AD 89D9      MOV     CX,BX                ; first cluster #
27648 000046AF B402      MOV     AH,2                ; delete cache entry by drive:firclus
27649 000046B1 E8B3E6      call    FastOpen_Update        ; call fastopen
27650 000046B4 5A      POP     DX
27651 000046B5 59      POP     CX
27652 000046B6 58      POP     AX
27653 000046B7 5D      POP     BP
27654 000046B8 07      POP     ES
27655                      ;; 10/23/86 FastOpen update
27656
27657 000046B9 E81415      call    RELEASE
27658                      ;JC      short SET_ACC_ERRWJ
27659                      ;UPDATEJ:
27660                      ; 20/11/2022
27661                      ;JMP     short UPDATE ; 10/08/2018
27662                      ; 10/02/2024
27663 000046BC EBAA      jmp     short dskwrt_8
27664
27665                      ;Break <DskWrtBufPurge -- Disk Write Buffer Purge>
27666                      ;-----
27667                      ;
27668                      ; Procedure Name : DskWrtBufPurge
27669                      ;
27670                      ; Inputs:
27671                      ;     CX = # of contiguous sectors to write. (These constitute a block of
27672                      ;     sectors, also termed an "Extent".)
27673                      ;     [HIGH_SECTOR]:DX = physical sector # of first sector in extent.
27674                      ;     ES:BP -> Drive Parameter Block (DPB).
27675                      ;
27676                      ; Function:
27677                      ;     Purge the Buffer Queue and the Secondary Cache of any buffers which
27678                      ;     are in Extent; they are being over-written.
27679                      ;
27680                      ; Outputs:
27681                      ;     (Same as Input.)
27682                      ;
27683                      ; Uses:
27684                      ;     All registers except DS,AX,SI,DI preserved.
27685                      ;     SS override for all global variables.
27686                      ;-----
27687                      ;M039: Created
27688
27689                      ;procedure DskWrtBufPurge,NEAR
27690                      ;
27691                      ;ASSUME DS:NOTHING
27692
27693                      ; 04/05/2019 - Retro DOS v4.0
27694                      ; DOSCODE:7C0Eh (MSDOS 6.21, MSDOS.SYS)
27695
27696                      ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
27697                      ; DOSCODE:7BD4h (MSDOS 5.0, MSDOS.SYS)
27698
27699                      ; 10/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
27700                      ; DOSCODE:8714h (PCDOS 7.1, IBMDOS.COM)
27701
27702                      ; NOTE: Secondary (Buffer) Cache is not used in PCDOS 7.1 IBMDOS.COM
27703
27704                      ; (win ME IO.SYS - BIOSCODE:B893h)
27705                      ; (MSDOS 6.22 IO.SYS - DOSCODE:7C0Eh)
27706
27707                      DskWrtBufPurge:
27708                      ;SaveReg <bx,cx>
27709                      push    bx
27710                      push    cx
27711
27712                      mov     bx,[ss:HIGH_SECTOR] ;BX:DX = Extent start (sector #).
27713                      mov     si,bx
27714                      add     cx,dx
27715                      adc     si,0                ;SI:DX = Extent end + 1.
27716
27717                      ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
27718                      ;;mov     al,[es:bp+0]
27719                      ;mov     al,[es:bp+DPB.DRIVE]
27720                      ; 15/12/2022
27721                      mov     al,[es:bp]
27722
27723                      ; 10/02/2024 - Retro DOS v5.0
27724                      ; PCDOS 7.1 IBMDOS.COM
27725                      %if 0
27726
27727                      ; BX:DX = Extent start.
27728                      ; SI:DX = Extent end + 1.
27729                      ; AL = Drive #
27730
27731                      cmp     word [ss:SC_CACHE_COUNT],0 ;Secondary cache in-use?
27732                      je      short nosc                ; -no, jump.
27733
27734                      ; If any of the sectors to be written are in the secondary cache (SC),
27735                      ; invalidate the entire SC. (This is an optimization; we really only
27736                      ; need to invalidate those sectors which intersect, but that's slower.)
27737
27738                      cmp     al,[ss:CurSC_DRIVE] ;same drive?
27739                      jne     short nosc                ; -no, jump.
27740
27741                      push    ax
27742                      mov     ax,[ss:CurSC_SECTOR]
27743                      mov     di,[ss:CurSC_SECTOR+2] ;DI:AX = SC start.
27744
27745                      ;Cmp32 si,cx,di,ax ;Extent end < SC start?
27746                      ;jbe     short sc5                ; -yes, jump.
27747
27748                      cmp     si,di
27749                      jne     short sc01
27750                      cmp     cx,ax
27751                      sc01:
27752                      jbe     short sc5
27753
27754                      add     ax,[ss:SC_CACHE_COUNT]
27755                      adc     di,0                ;DI:AX = SC end + 1.
27756
27757                      ;Cmp32 bx,dx,di,ax ;Extent start > SC end?
27758                      ;jae     short sc5                ; -yes, jump.
27759
27760                      cmp     bx,di
27761                      jne     short sc02
27762                      cmp     dx,ax

```



```

27762 sc02:
27763     jnb     short sc5
27764
27765     mov     word [ss:SC_STATUS],0 ;Extent intersects SC: invalidate SC.
27766 sc5:
27767     pop     ax
27768
27769 %endif
27770
27771 ;   Free any buffered sectors which are in Extent; they are being over-
27772 ;   written.
27773
27774 nosc:
27775     call    GETCURHEAD             ;DS:DI -> first buffer in queue.
27776
27777 _bufq:
27778     ;cmpo   al,[di+4]
27779 000046D3 3A4504     cmp     al,[di+BUFFINFO.buf_ID] ;Same drive?
27780 000046D6 7527     jne     short bufq5             ; -no, jump.
27781
27782 ;           cmp32   bx,dx,<WORD PTR [di.buf_sector+2]>,<WORD PTR [di.buf_sector]>
27783 ;           ja      short bufq5             ;Jump if Extent start > buffer sector.
27784
27785     ;cmp     bx,[di+8]
27786 000046D8 3B5D08     cmp     bx,[di+BUFFINFO.buf_sector+2]
27787 000046DB 7503     jne     short bufq04
27788     ;cmp     dx,[di+6]
27789 000046DD 3B5506     cmp     dx,[di+BUFFINFO.buf_sector]
27790 bufq04:
27791     ja      short bufq5
27792
27793 ;           cmp32   si,cx,<WORD PTR [di.buf_sector+2]>,<WORD PTR [di.buf_sector]>
27794 ;           jbe     short bufq5             ;Jump if Extent end < buffer sector.
27795
27796     ;cmp     si,[di+8]
27797 000046E2 3B7508     cmp     si,[di+BUFFINFO.buf_sector+2]
27798 000046E5 7503     jne     short bufq05
27799     ;cmp     cx,[di+6]
27800 000046E7 3B4D06     cmp     cx,[di+BUFFINFO.buf_sector]
27801 bufq05:
27802     jbe     short bufq5
27803
27804 ;   Buffer's sector is in Extent, so free it; it is being over-written.
27805
27806     ;test    byte [di+5],40h
27807 000046EC F6450540   test    byte [di+BUFFINFO.buf_flags],buf_dirty ;Buffer dirty?
27808 000046F0 7403     jz      short bufq4             ; -no, jump.
27809 000046F2 E8DB24     call    DEC_DIRTY_COUNT         ; -yes, decrement dirty count.
27810 bufq4:
27811     ;mov     word [di+4],20FFh
27812 000046F5 C74504FF20   mov     word [di+BUFFINFO.buf_ID],((buf_visit<<8)|0FFh)
27813
27814     call    SCANPLACE
27815 000046FD EB02     jmp     short bufq6
27816 bufq5:
27817     mov     di,[di]
27818     ;mov     di,[di+BUFFINFO.buf_next]
27819 bufq6:
27820     cmp     di,[ss:FIRST_BUFF_ADDR]       ;Scanned entire buffer queue?
27821 00004706 75CB     jne     short _bufq             ; --no, go do next buffer.
27822
27823     ;RestoreReg <cx,bx>
27824 00004708 59     pop     cx
27825 00004709 5B     pop     bx
27826 0000470A C3     retn
27827
27828 ;EndProc DskWrtBufPurge
27829
27830 ;Break <DIV32 -- PERFORM 32 BIT DIVIDE>
27831 ;-----
27832 ;
27833 ; Procedure Name : DIV32
27834 ;
27835 ; Inputs:
27836 ;   DX:AX = 32 bit dividend   BX= divisor
27837 ; Function:
27838 ;   Perform 32 bit division: DX:AX/BX = CX:AX + DX (rem.)
27839 ; Outputs:
27840 ;   CX:AX = quotient , DX= remainder
27841 ; Uses:
27842 ;   All registers except AX,CX,DX preserved.
27843 ;-----
27844 ;M039: DIV32 optimized for divisor of 512 (common sector size).
27845
27846 ; 04/05/2019 - Retro DOS v4.0
27847 ; DOSCODE:7C94h (MSDOS 6.21, MSDOS.SYS)
27848
27849 ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
27850 ; DOSCODE:7C5Ah (MSDOS 5.0, MSDOS.SYS)
27851
27852 DIV32:
27853 0000470B 81FB0002   cmp     bx,512
27854 0000470F 7515     jne     short div5
27855
27856     mov     cx,dx
27857 00004713 89C2     mov     dx,ax                 ; CX:AX = Dividend
27858 00004715 81E2FF01 and     dx,(512-1)             ; DX = Remainder
27859 00004719 88E0     mov     al,ah
27860 0000471B 88CC     mov     ah,cl
27861 0000471D 88E9     mov     cl,ch
27862 0000471F 30ED     xor     ch,ch
27863 00004721 D1E9     shr     cx,1
27864 00004723 D1D8     rcr     ax,1
27865 00004725 C3     retn
27866 div5:
27867 00004726 89C1     mov     cx,ax
27868 00004728 89D0     mov     ax,dx
27869 0000472A 31D2     xor     dx,dx
27870 0000472C F7F3     div     bx                     ; 0:AX/BX
27871 0000472E 91     xchg    cx,ax
27872 0000472F F7F3     div     bx                     ; DX:AX/BX
27873 00004731 C3     retn
27874
27875 ;Break <SHR32 -- PERFORM 32 BIT SHIFT RIGHT>
27876 ;-----
27877 ;
27878 ; Procedure Name : SHR32
27879 ;
27880 ; Inputs:
27881 ;   DX:AX = 32 bit sector number
27882 ; Function:
27883 ;   Perform 32 bit shift right
27884 ; Outputs:
27885 ;   AX = cluster number

```

```

27886 ; ZF = 1 if no error
27887 ; = 0 if error (cluster number > 64k)
27888 ; Uses:
27889 ; DX,CX
27890 -----
27891 ; M017 - SHR32 rewritten for better performance
27892 ; M039 - Additional optimization
27893
27894 ; 04/05/2019 - Retro DOS v4.0
27895 ; DOSCODE:7CBBh (MSDOS 6.21, MSDOS.SYS)
27896 ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
27897 ; DOSCODE:7C81h (MSDOS 5.0, MSDOS.SYS)
27898
27899 SHR32:
27900 ;mov c],[es:bp+5]
27901 00004732 268A4E05 mov c],[ES:BP+DPB.CLUSTER_SHIFT]
27902 00004736 30ED xor ch,ch ;ZF=1
27903 00004738 E306 jcxz norota
27904
27905 0000473A D1EA rotashft2:
27906 0000473C D1D8 shr dx,1 ;ZF reflects state of DX.
27907 0000473E E2FA rcr ax,1 ;ZF not affected.
27908 loop rotashft2
27909 00004740 C3 norota:
27910 retn
27911
27912 ;=====
27913 ; DIR.ASM, MSDOS 6.0, 1991
27914 ;=====
27915 ; 27/07/2018 - Retro DOS v3.0
27916 ; 19/05/2019 - Retro DOS v4.0
27917
27918 ; TITLE DIR - Directory and path cracking
27919 ; NAME Dir
27920
27921 ;Break <FINDENTRY -- LOOK FOR AN ENTRY>
27922 -----
27923 ; Procedure Name : FINDENTRY,SEARCH
27924 ;
27925 ; Inputs:
27926 ; [THISDPB] set
27927 ; [SECCLUSPOS] = 0
27928 ; [DIRSEC] = Starting directory sector number
27929 ; [CLUSNUM] = Next cluster of directory
27930 ; [CLUSFAC] = Sectors/Cluster
27931 ; [NAME1] = Name to look for
27932 ; Function:
27933 ; Find file name in disk directory.
27934 ; "?" matches any character.
27935 ; Outputs:
27936 ; Carry set if name not found
27937 ; ELSE
27938 ; Zero set if attributes match (always except when creating)
27939 ; AH = Device ID (bit 7 set if not disk)
27940 ; [THISDPB] = Base of drive parameters
27941 ; DS = DOSGROUP
27942 ; ES = DOSGROUP
27943 ; [CURBUF+2]:BX = Pointer into directory buffer
27944 ; [CURBUF+2]:SI = Pointer to First Cluster field in directory entry
27945 ; [CURBUF] has directory record with match
27946 ; [NAME1] has file name
27947 ; [LASTENT] is entry number of the entry
27948 ; All other registers destroyed.
27949 ;-----
27950
27951 ;hkn; called from rename.asm and dir2.asm. DS must be already set up at
27952 ;hkn; this point.
27953
27954 SEARCH:
27955 ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
27956 ; DOSCODE:7C90h (MSDOS 5.0, MSDOS.SYS)
27957
27958 ; 19/05/2019 - Retro DOS v4.0
27959 ; DOSCODE:7CCAh (MSDOS 6.21, MSDOS.SYS)
27960
27961 ; 27/07/2018 - Retro DOS v3.0
27962 ; IBMDOS.COM (MSDOS 3.3) - Offset 45B3h
27963 ; 15/03/2018 - Retro DOS v2.0
27964
27965 ; 24/01/2024
27966
27967 ; 13/02/2024 - Retro DOS v5.0
27968 ; DOSCODE:8797h (PCDOS 7.1, IBMDOS.COM)
27969
27970 ;entry FindEntry
27971 FINDENTRY:
27972 00004741 E86F06 call STARTSRCH
27973 00004744 A0[6B05] MOV AL,[ATTRIB]
27974 ;and al,9Eh
27975 00004747 24DE AND AL,~attr_ignore ; Ignore useless bits
27976 ;cmp al,8
27977 00004749 3C08 CMP AL,attr_volume_id ; Looking for vol ID only ?
27978 0000474B 7503 JNZ short NOTVOLSRCH ; No
27979 0000474D E83B02 CALL SETROOTSRCH ; Yes force search of root
27980 NOTVOLSRCH:
27981 00004750 E84C01 CALL GETENTRY
27982 ;JNC short SRCH
27983 ;JMP SETESRET
27984 ; 24/01/2024
27985 00004753 724F jc short SETESRET
27986
27987 ;entry Srch
27988 SRCH:
27989 ;;;
27990 ; 13/02/2024 (PCDOS 7.1 IBMDOS.COM)
27991 00004755 C706[A50A]0000 mov word [LNE_COUNT],0 ; reset long name entry count
27992 SRCH2:
27993 ;;;
27994
27995 0000475B 1E PUSH DS
27996 0000475C 8E1E[E405] MOV DS,[CURBUF+2]
27997
27998 ; (DS:BX) = directory entry address
27999
28000 00004760 8A27 mov ah,[BX]
28001 ;MOV AH,[BX+dir_entry.dir_name] ; mov ah,[bx+0]
28002 00004762 08E4 OR AH,AH ; End of directory?
28003 00004764 7461 JZ short FREE
28004
28005 ;;;
28006 ; 13/02/2024 (PCDOS 7.1 IBMDOS.COM)
28007 00004766 50 push ax
28008 00004767 8A470B mov al,[bx+0Bh] ; [BX+dir_entry.dir_attr]
28009 0000476A E8A103 call check_longname

```

```

28010 0000476D 58          pop     ax
28011 0000476E 7447        jz     short NEXTENT2
28012                      ;;;
28013
28014                      ;hkn; SS override
28015 00004770 363A26[7F05]  CMP     AH,[SS:DELALL]      ; Free entry?
28016 00004775 7450        JZ     short FREE
28017                      ;test byte [bx+0Bh],8
28018 00004777 F6470B08      TEST    byte [Bx+dir_entry.dir_attr],attr_volume_id
28019                      ; Volume ID file?
28020 0000477B 7405        JZ     short CHKFNAM      ; NO
28021
28022                      ;hkn; SS override
28023 0000477D 36FE06[7B05]  INC     BYTE [SS:VOLID]
28024                      CHKFNAM:
28025                      ; Context ES
28026 00004782 8CD6        MOV     SI,SS
28027 00004784 8EC6        MOV     ES,SI
28028 00004786 89DE        MOV     SI,BX
28029
28030                      ;hkn; NAME1 is in DOSDATA
28031 00004788 BF[4B05]      MOV     DI,NAME1
28032                      ;;;; 7/29/86
28033
28034                      ;hkn; SS override for NAME1
28035                      ;CMP BYTE [SS:NAME1],0E5H ; special char check
28036                      ;JNZ short NO_E5
28037                      ;MOV BYTE [SS:NAME1],05H
28038                      ; 22/09/2023
28039 0000478B 26803DE5      cmp     byte [es:di],0E5h
28040 0000478F 7504        jnz     short NO_E5
28041 00004791 26C60505      mov     byte [es:di],05h
28042                      NO_E5:
28043                      ;;;; 7/29/86
28044 00004795 E88100      CALL    MetaCompare
28045 00004798 7449        JZ     short FOUND
28046 0000479A 1F          POP     DS
28047
28048                      ;entry NEXTENT
28049                      NEXTENT:
28050                      LES     BP,[THISDPB]
28051 0000479F E88600      CALL    NEXTENTRY
28052 000047A2 73B1        JNC     short SRCH
28053                      ;JMP SHORT SETESRET
28054                      ; 24/01/2024
28055                      SETESRET:
28056 000047A4 16          PUSH    SS
28057 000047A5 07          POP     ES
28058
28059                      ;;;
28060                      ; 13/02/2024 (PCDOS 7.1 IBMDOS.COM)
28061 000047A6 FF36[DA05]      push    word [ENTLAST]
28062 000047AA 8F06[B50A]      pop     word [ENTLAST_PREV] ; previous ENTLAST
28063 000047AE 7306        jnc     short SETESRETN
28064 000047B0 C706[A50A]0000      mov     word [LNE_COUNT],0 ; reset long name entry count
28065                      SETESRETN:
28066                      ;;;
28067
28068 000047B6 C3          retn
28069
28070                      ;;;
28071                      ; 13/02/2024 (PCDOS 7.1 IBMDOS.COM)
28072                      NEXTENT2:
28073 000047B7 1F          pop     ds
28074 000047B8 FF06[A50A]      inc     word [LNE_COUNT] ; Long Name entry count
28075                      NEXTENT3:
28076 000047BC C42E[8A05]      les     bp,[THISDPB]
28077 000047C0 E86500      call    NEXTENTRY
28078 000047C3 7396        jnc     short SRCH2
28079 000047C5 EBDD        jmp     short SETESRET
28080                      ;;;
28081
28082                      FREE:
28083 000047C7 1F          POP     DS
28084 000047C8 8B0E[4803]      MOV     CX,[LASTENT]
28085 000047CC 3B0E[D805]      CMP     CX,[ENTFREE]
28086 000047D0 7304        JAE     short TSTALL
28087 000047D2 890E[D805]      MOV     [ENTFREE],CX
28088                      TSTALL:
28089 000047D6 3A26[7F05]      CMP     AH,[DELALL] ; At end of directory?
28090                      NEXTENTJ:
28091                      ;je short NEXTENT ; No - continue search
28092                      ;;;
28093                      ; 13/02/2024
28094 000047DA 74E0        je     short NEXTENT3 ; No - continue search
28095                      ;;;
28096 000047DC 890E[DA05]      MOV     [ENTLAST],CX
28097 000047E0 F9          STC
28098 000047E1 EBC1        JMP     SHORT SETESRET
28099
28100                      FOUND:
28101                      ; We have a file with a matching name. We must now consider the attributes:
28102                      ; ATTRIB Action
28103                      ; -----
28104                      ; Volume_ID Is Volume_ID in test?
28105                      ; Otherwise If no create then Is ATTRIB+extra superset of test?
28106                      ; If create then Is ATTRIB equal to test?
28107
28108 000047E3 8A2C        MOV     CH,[SI] ; Attributes of file
28109 000047E5 1F          POP     DS
28110 000047E6 8A26[6B05]      MOV     AH,[ATTRIB] ; Attributes of search
28111                      ;and ah,9Eh
28112 000047EA 80E4DE      AND     AH,~attr_ignore
28113                      ;lea si,[si+15]
28114 000047ED 8D740F      LEA     SI,[SI+dir_entry.dir_first-dir_entry.dir_attr]
28115                      ; point to first cluster field
28116
28117 000047F0 F6C508      ;test ch,8
28118 000047F3 7409        TEST    CH,attr_volume_id ; volume ID file?
28119                      JZ     short check_one_volume_id ; Nope check other attributes
28120 000047F5 F6C408      ;test ah,8
28121                      TEST    AH,attr_volume_id ; Can we find Volume ID?
28122                      ;JZ short NEXTENTJ ; Nope, (not even $FCB_CREATE)
28123                      ; 16/12/2022
28124 000047F8 74A1        jz     short NEXTENT ; 19/05/2019
28125                      ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
28126 000047FA 30E4        ;JZ short NEXTENTJ
28127 000047FC EB11        XOR     AH,AH ; Set zero flag for $FCB_CREATE
28128                      JMP     SHORT RETFF ; Found Volume ID
28129                      check_one_volume_id:
28130                      ;CMP ah,8
28131 000047FE 80FC08      CMP     AH,attr_volume_id ; Looking only for Volume ID?
28132                      ;JZ short NEXTENTJ ; Yes, continue search
28133 00004801 7498        ; 16/12/2022
28133                      je     short NEXTENT ; 19/05/2019

```

```

28134      ;JZ      short NEXTENTJ
28135 00004803 E8C105 CALL MatchAttributes
28136 00004806 7407 JZ SHORT RETFF
28137 00004808 F606[7E05]FF TEST BYTE [CREATING],-1 ; Pass back mismatch if creating
28138      ; 16/12/2022
28139      ;JZ      short NEXTENTJ ; Otherwise continue searching
28140 0000480D 748C JZ short NEXTENT ; 19/05/2019
28141      RETFF:
28142 0000480F C42E[8A05] LES BP,[THISDPB]
28143      ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
28144      ;MOV     AH,[ES:BP+DPB.DRIVE] ; mov ah,[es:bp+0]
28145      ; 15/12/2022
28146 00004813 268A6600 MOV AH,[ES:BP]
28147      ;SETESRET:
28148      ;PUSH    SS
28149      ;POP     ES
28150      ;retn
28151      ; 24/01/2024
28152 00004817 EB8B jmp short SETESRET
28153
28154      ;-----
28155      ;
28156      ; Procedure Name : MetaCompare
28157      ;
28158      ; Inputs:
28159      ; DS:SI -> 11 character FCB style name NO '?'
28160      ; Typically this is a directory entry. It MUST be in upper case
28161      ; ES:DI -> 11 character FCB style name with possible '?'
28162      ; Typically this is a FCB or SFT. It MUST be in upper case
28163      ; Function:
28164      ; Compare FCB style names allowing for ? match to any char
28165      ; Outputs:
28166      ; Zero if match else NZ
28167      ; Destroys CX,SI,DI all others preserved
28168      ;-----
28169
28170      ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
28171      ; DOSCODE:7D3Fh (MSDOS 5.0, MSDOS.SYS)
28172
28173      MetaCompare:
28174 00004819 B90B00 MOV CX,11
28175      WILDCRD:
28176 0000481C F3A6 REPE CMPSB
28177 0000481E 7407 JZ short MetaRet ; most of the time we will fail.
28178      CHECK_META:
28179 00004820 26807DFF3F CMP BYTE [ES:DI-1],"?"
28180 00004825 74F5 JZ short WILDCRD
28181      MetaRet:
28182 00004827 C3 retn ; Zero set, Match
28183
28184      ;Break <NEXTENTRY -- STEP THROUGH DIRECTORY>
28185      ;-----
28186      ;
28187      ; Procedure Name : NEXTENTRY
28188      ;
28189      ; Inputs:
28190      ; Same as outputs of GETENTRY, above
28191      ; Function:
28192      ; Update BX, and [LASTENT] for next directory entry.
28193      ; Carry set if no more.
28194      ;-----
28195
28196      NEXTENTRY:
28197      ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
28198      ; DOSCODE:7D4Eh (MSDOS 5.0, MSDOS.SYS)
28199
28200      ; 19/05/2019 - Retro DOS v4.0
28201      ; DOSCODE:7D88h (MSDOS 6.21, MSDOS.SYS)
28202
28203      ; 27/07/2018 - Retro DOS v3.0
28204      ; IBMDOS.COM (MSDOS 3.3) - Offset 4671h
28205      ; 15/03/2018 - Retro DOS v2.0
28206
28207      ; 14/02/2024 - Retro DOS v5.0
28208      ; DOSCODE:8884h (PCDOS 7.1, IBMDOS.COM)
28209
28210 00004828 A1[4803] MOV AX,[LASTENT]
28211 0000482B 3B06[DA05] CMP AX,[ENTLAST]
28212 0000482F 741C JZ short NONE
28213 00004831 40 INC AX
28214      ;ADD     BX,32
28215 00004832 8D5F20 LEA BX,[BX+32]
28216 00004835 39D3 CMP BX,DX
28217      ; 21/11/2022 - MSDOS 5.0 MSDOS.SYS (DOSCODE:7D5Dh)
28218      ;JB      short HAVIT ; MSDOS 6.0 src (dir.asm)
28219      ; 16/12/2022
28220 00004837 7540 jne short HAVIT ; MSDOS 6.21 (DOSCODE:7D97h)
28221
28222      ;;;
28223      ; 14/02/2024 (PCDOS 7.1 IBMDOS.COM)
28224 00004839 833E[C205]00 cmp word [DIRSTART],0
28225 0000483E 750F jnz short nextentry_cont
28226 00004840 833E[DC0A]00 cmp word [DIRSTART_HW],0
28227 00004845 7508 jnz short nextentry_cont
28228      ;cmp     ax,[es:bp+9]
28229 00004847 263B4609 cmp ax,[es:bp+DPB.ROOT_ENTRIES] ; Number of root dir entries
28230      ;jnb     short NONE
28231      ;jmp     short GETENT
28232 0000484B 7255 jb short GETENT
28233      NONE:
28234 0000484D F9 stc
28235 0000484E C3 retn
28236
28237      nextentry_cont:
28238      ;;;
28239
28240 0000484F 8A1E[7305] MOV BL,[SECCLUSPOS]
28241 00004853 FEC3 INC BL
28242 00004855 3A1E[7705] CMP BL,[CLUSFAC]
28243 00004859 7223 JB short SAMECLUS
28244      ;;;
28245      ; 14/02/2024 (PCDOS 7.1 IBMDOS.COM)
28246 0000485B 8B1E[E00A] mov bx,[NXTCLUSNUM_HW]
28247 0000485F 891E[E80A] mov [CLUSTNUM_HW],bx
28248      ;;;
28249 00004863 8B1E[DC05] MOV BX,[NXTCLUSNUM]
28250 00004867 E84E1A call IseOF
28251 0000486A 73E1 JAE short NONE
28252      ;;;
28253      ; 14/02/2024
28254 0000486C 833E[E80A]00 cmp word [CLUSTNUM_HW],0
28255 00004871 752F jnz short GETENT
28256      ;;;
28257      ; 23/07/2019

```

```

28258 00004873 83FB02      CMP     BX,2
28259                    ;JB     short NONE
28260                    ;JMP    short GETENT
28261                    ; 16/12/2022
28262 00004876 732A      jnb     short GETENT
28263                    ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
28264                    ;JB     short NONE
28265                    ;JMP    short GETENT
28266                    ; 14/02/2024 - Retro DOS v5.0
28267                    ;NONE:
28268                    ;STC
28269 00004878 C3      retn
28270
28271 00004879 A3[4803]    HAVIT:
28272 0000487C F8      MOV     [LASTENT],AX
28273                    CLC
28274 0000487D C3      nextentry_retn:
28275                    retn
28276
28277 0000487E 881E[7305]    SAMECLUS:
28278 00004882 A3[4803]    MOV     [SECCLUSPOS],BL
28279 00004885 1E      MOV     [LASTENT],AX
28280 00004886 C53E[E205]    PUSH    DS
28281                    LDS     DI,[CURBUF]
28282                    ; 19/05/2019
28283                    ; MSDOS 6.0
28284                    ;mov dx,[di+8]
28285                    ; 23/09/2023
28286                    ;MOV     DX,[DI+BUFFINFO.buf_sector+2] ;AN000; >32mb
28287                    ;hkn; SS override
28288                    ;MOV     [SS:HIGH_SECTOR],DX ;AN000; >32mb
28289                    ; 14/02/2024
28290                    %if 0
28291                    ; 23/09/2023
28292                    mov     si,[di+BUFFINFO.buf_sector+2]
28293
28294                    ;mov dx,[di+6]
28295                    MOV     DX,[DI+BUFFINFO.buf_sector] ;AN000; >32mb
28296
28297                    ;inc dx ; MSDOS 3.3
28298                    ; MSDOS 6.0
28299                    ;ADD     DX,1 ;AN000; >32mb
28300                    ;ADC     word [SS:HIGH_SECTOR],0 ;AN000; >32mb
28301                    ; 23/09/2023
28302                    inc     dx
28303                    jnz     short nextentry_fc
28304                    inc     si
28305                    ;inc word [SS:HIGH_SECTOR]
28306                    nextentry_fc:
28307                    ; 23/09/2023
28308                    mov     [SS:HIGH_SECTOR],si
28309                    ; MSDOS 3.3 & MSDOS 6.0
28310                    POP     DS
28311                    %else
28312                    ; 14/02/2024 - Retro DOS v5.0
28313 0000488A C55506      lds     dx,[di+BUFFINFO.buf_sector]
28314 0000488D 8CDE      mov     si,ds
28315 0000488F 1F      pop     ds
28316 00004890 42      inc     dx
28317 00004891 7501      jnz     short nextentry_fc
28318 00004893 46      inc     si
28319                    nextentry_fc:
28320 00004894 8936[0706]    mov     [HIGH_SECTOR],si
28321                    %endif
28322
28323 00004898 E8DEF6      call    FIRSTCLUSTER
28324 0000489B 31DB      XOR     BX,BX
28325 0000489D EB21      JMP     short SETENTRY
28326
28327                    ;-----
28328                    ;
28329                    ; Procedure Name : GETENTRY
28330                    ;
28331                    ; Inputs:
28332                    ; [LASTENT] has directory entry
28333                    ; ES:BP points to drive parameters
28334                    ; [DIRSEC],[CLUSNUM],[CLUSFAC],[ENTLAST] set for DIR involved
28335                    ; Function:
28336                    ; Locates directory entry in preparation for search
28337                    ; GETENT provides entry for passing desired entry in AX
28338                    ; Outputs:
28339                    ; [CURBUF+2]:BX = Pointer to next directory entry in CURBUF
28340                    ; [CURBUF+2]:DX = Pointer to first byte after end of CURBUF
28341                    ; [LASTENT] = New directory entry number
28342                    ; [NXTCLUSNUM],[SECCLUSPOS] set via DIRREAD
28343                    ; Carry set if error (currently user FAILED to I 24)
28344                    ;-----
28345
28346                    ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
28347                    GETENTRY:
28348                    ; 27/07/2018 - Retro DOS v3.0
28349 0000489F A1[4803]    MOV     AX,[LASTENT]
28350
28351                    ;entry GETENT
28352                    GETENT:
28353 000048A2 A3[4803]    MOV     [LASTENT],AX
28354
28355                    ;
28356                    ; Convert the entry number in AX into a byte offset from the beginning of the
28357                    ; directory.
28358                    ;
28359                    mov     cl,5 ; shift left by 5 = mult by 32
28360                    rol     ax,cl ; keep high order bits
28361                    mov     dx,ax
28362                    ; 19/05/2019 - Retro DOS v4.0
28363                    ;and     ax,0FFE0h
28364                    ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
28365                    ;and     ax,~(32-1) ; mask off high order bits
28366                    ; 16/12/2022
28367 000048AB 24E0      and     al,0E0h ; ~31
28368 000048AD 83E21F    and     dx,1Fh
28369                    ;and     dx,32-1 ; mask off low order bits
28370
28371                    ;
28372                    ; DX:AX contain the byte offset of the required directory entry from the
28373                    ; beginning of the directory. Convert this to a sector number. Round the
28374                    ; sector size down to a multiple of 32.
28375                    ;
28376                    mov     bx,[es:bp+2]
28377 000048B0 268B5E02    MOV     BX,[ES:BP+DPB.SECTOR_SIZE]
28378 000048B4 80E3E0      and     bl,0E0h
28379                    ;AND     BL,255-31 ; Must be multiple of 32
28380 000048B7 F7F3      DIV     BX
28381                    ; 14/02/2024
28382                    ;MOV     BX,DX ; Position within sector
28383                    ; NOTE: This BX value is not used in DIRREAD

```

```

28382                                     ; Erdogan Tan - 14/02/2024
28383                                     ; PUSH BX
28384 000048B9 52                         push dx
28385                                     ;
28386 000048BA E832F6                       call DIRREAD
28387 000048BD 5B                         POP BX
28388                                     ;retc
28389 000048BE 72BD                         jc short nextentry_retn
28390 SETENTRY:
28391 000048C0 8B16[E205]                   MOV DX,[CURBUF]
28392                                     ;;;add dx,16 ; MSDOS 3.3
28393                                     ;;;add dx,20 ; MSDOS 6.0
28394                                     ;;;add dx,24 ; PCDOS 7.1 ; 14/02/2024
28395 000048C4 83C218                       ADD DX,BUFINSIZ
28396 000048C7 01D3                       ADD BX,DX
28397                                     ;add dx,[es:bp+2]
28398 000048C9 26035602                     ADD DX,[ES:BP+DPB.SECTOR_SIZE] ; Always clears carry
28399                                     ; 29/12/2022
28400                                     ; MSDOS 6.21 MSDOS.SYS contains a 'CLC' here, at DOSCODE:7E15h
28401                                     ; 27/06/2024
28402                                     ; PCDOS 7.1 IBMDOS.COM contains 'CLC' here, at DOSCODE:8931h
28403 000048CD F8                         clc
28404 000048CE C3                         retn
28405
28406 -----
28407
28408 ; 23/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
28409 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:8933h
28410
28411 sft_fcb_table:
28412 000048CF 00<rep 14h>                 times 20 db 0
28413 sftfcb.cluster:
28414 000048E3 00000000                   dd 0
28415 sftfcb.direntry:
28416 000048E7 0000                       dw 0
28417 sftfcb_entry_size equ $ - sftfcb.cluster ; = 6
28418
28419 000048E9 00<rep 72h>                 times 114 db 0 ; 6*20 = 120 ; entry size = 6 bytes
28420
28421 SRCH_CLUSTER:
28422 0000495B 0000                       dw 0
28423 SRCH_CLUSTER_HW:
28424 0000495D 0000                       dw 0
28425
28426 -----
28427
28428 ;Break <SETDIRSRCH SETROOTSRCH -- Set Search environments>
28429 -----
28430 ;
28431 ; Procedure Name : SETDIRSRCH,SETROOTSRCH
28432 ;
28433 ; Inputs:
28434 ; BX cluster number of start of directory
28435 ; ES:BP Points to DPB
28436 ; DI next cluster number from fastopen extended info. DOS 3.3 only
28437 ; Function:
28438 ; Set up a directory search
28439 ; Outputs:
28440 ; [DIRSTART] = BX
28441 ; [CLUSFAC],[CLUSNUM],[SECCLUSPOS],[DIRSEC] set
28442 ; Carry set if error (currently user FAILED to I 24)
28443 ; destroys AX,DX,BX
28444 -----
28445
28446 ; 23/01/2024 - Retro DOS v5.0
28447 %if 0
28448 ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
28449 SETDIRSRCH:
28450 OR BX,BX
28451 JZ short SETROOTSRCH
28452 MOV [DIRSTART],BX
28453 ;mov al,[es:bp+4]
28454 MOV AL,[ES:BP+DPB.CLUSTER_MASK]
28455 INC AL
28456 MOV [CLUSFAC],AL
28457
28458 ; DOS 3.3 for FastOpen F.C. 6/12/86
28459 ;SAVE <SI>
28460 push si
28461 ;test byte [FastOpenFlg],2
28462 TEST byte [FastOpenFlg],Lookup_Success
28463 JNZ short UNP_OK
28464
28465 ; DOS 3.3 for FastOpen F.C. 6/12/86
28466 ;invoke UNPACK
28467 call UNPACK
28468 JNC short UNP_OK
28469 ;RESTORE <SI>
28470 pop si
28471 ;return
28472 retn
28473
28474 UNP_OK:
28475 MOV [CLUSNUM],DI
28476 MOV DX,BX
28477 XOR BL,BL
28478 MOV [SECCLUSPOS],BL
28479 ;invoke FIGREC
28480 call FIGREC
28481 ;RESTORE <SI>
28482 pop si
28483
28484 ; 19/05/2019 - Retro DOS v4.0
28485
28486 ; MSDOS 6.0
28487 ;PUSH DX ;AN000; >32mb
28488 ;MOV DX,[HIGH_SECTOR] ;AN000; >32mb
28489 ;MOV [DIRSEC+2],DX ;AN000; >32mb
28490 ;POP DX ;AN000; >32mb
28491
28492 ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
28493 ;push dx
28494 ;mov dx,[HIGH_SECTOR]
28495 ;mov [DIRSEC+2],dx
28496 ;pop dx
28497 ;MOV [DIRSEC],dx
28498 ; 16/12/2022
28499 mov ax,[HIGH_SECTOR]
28500 mov [DIRSEC+2],AX
28501 MOV [DIRSEC],DX
28502
28503 ; 16/12/2022
28504 ; cf=0 (at the return of FIGREC)
28505 CLC

```



```

28506         retn
28507
28508         ;entry SETROOTSRCH
28509 SETROOTSRCH:
28510         XOR     AX,AX
28511         MOV     [DIRSTART],AX
28512         ; 22/09/2023
28513         mov     [DIRSEC+2],ax ; 0
28514         MOV     [SECCLUSPOS],AL
28515         DEC     AX
28516         MOV     [CLUSNUM],AX
28517         ;mov     ax,[es:bp+0Bh]
28518         MOV     AX,[ES:BP+DPB.FIRST_SECTOR]
28519         ; 19/05/2019
28520         ;;mov     dx,[es:bp+10h] ; MSDOS 3.3
28521         ;mov     dx,[es:bp+11h] ; MSDOS 6.0
28522         MOV     DX,[ES:BP+DPB.DIR_SECTOR]
28523         SUB     AX,DX
28524         MOV     [CLUSFAC],AL
28525         MOV     [DIRSEC],DX                ;F.C. >32mb
28526         ; 22/09/2023
28527         ; MSDOS 6.0
28528         ;MOV     WORD [DIRSEC+2],0        ;F.C. >32mb
28529         CLC
28530         retn
28531 %else
28532         ;;;
28533         ; 23/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
28534         ; PCDOS 7.1 IBMDOS.COM - DOSCODE:89C3h
28535         ;;;
28536 SETDIRSRCH:
28537         mov     ax,[ROOTCLUST_HW]
28538         ;cmp     word [ROOTCLUST_HW],0
28539         and     ax,ax
28540         jnz     short SETDIRSRCH_FAT32
28541
28542         or      bx,bx
28543         ;jz     short SETROOTSRCH
28544         jz     short SETROOTSRCH2 ; ax = 0
28545 SETDIRSRCH_FAT32:
28546         ;mov     ax,[ROOTCLUST_HW]
28547         mov     [DIRSTART_HW],ax
28548         mov     [CLUSTNUM_HW],ax
28549
28550         mov     [DIRSTART],bx
28551         ;mov     al,[es:bp+4]
28552         mov     al,[es:bp+DPB.CLUSTER_MASK]
28553         ;inc     al
28554         inc     ax
28555         mov     [CLUSFAC],al
28556
28557         push    si
28558         ;test    byte [FastOpenFlg],2
28559         test    byte [FastOpenFlg],Lookup_Success
28560         jnz     short UNP_OK
28561
28562         call    UNPACK
28563         jnc     short UNP_OK
28564
28565         pop     si
28566         retn
28567
28568 SETROOTSRCH:
28569         xor     ax,ax ; 0
28570 SETROOTSRCH2:
28571         mov     [ROOTCLUST_HW],ax ;0
28572         ;cmp     word [es:bp+0Fh],0
28573         cmp     [es:bp+DPB.FAT_SIZE],ax ; 0
28574         jnz     short SETROOTSRCH_FAT ; not FAT32
28575 SETROOTSRCH_FAT32:
28576         ;;mov     bx,[es:bp+37h]
28577         ;mov     bx,[es:bp+DPB.ROOT_CLUSTER+2]
28578         ;mov     [ROOTCLUST_HW],bx
28579         ;;cmp     bx,[es:bp+2Fh]
28580         ;cmp     bx,[es:bp+DPB.LAST_CLUSTER+2]
28581         ; 27/06/2024 - Retro DOS v5.0
28582         mov     ax,[es:bp+DPB.ROOT_CLUSTER+2]
28583         mov     [ROOTCLUST_HW],ax
28584         cmp     ax,[es:bp+DPB.LAST_CLUSTER+2]
28585         ;mov     bx,[es:bp+35h]
28586         mov     bx,[es:bp+DPB.ROOT_CLUSTER]
28587         jne     short sdsrch_fat32_1
28588         ;cmp     bx,[es:bp+2Dh]
28589         cmp     bx,[es:bp+DPB.LAST_CLUSTER]
28590 sdsrch_fat32_1:
28591         ja     short sdsrch_fat32_3 ; **
28592         ; 27/06/2024
28593         and     ax,ax ; [ROOTCLUST_HW]
28594         ;cmp     word [ROOTCLUST_HW],0
28595         ;jnz     short sdsrch_fat32_2
28596         jnz     short SETDIRSRCH_FAT32
28597         cmp     bx,2
28598         ;jb     short sdsrch_fat32_3
28599 sdsrch_fat32_2:
28600         ;jmp     SETDIRSRCH_FAT32
28601         jnb     short SETDIRSRCH_FAT32
28602
28603 sdsrch_fat32_3:
28604         stc     ; **
28605         ;jmp     short setdirsrch_retn
28606         retn
28607
28608 UNP_OK:
28609         ; CCONTENT_HW:DI = Contents of FAT for given cluster
28610         ; (from UNPACK)
28611         mov     [CLUSNUM],di
28612         mov     dx,[CONTENT_HW]
28613         mov     [CLUSNUM_HW],dx
28614         mov     [cs:SRCH_CLUSTER],bx ; Directory start cluster number
28615         ; for searching/locating (directory entry)
28616         mov     dx,[CLUSTNUM_HW]
28617         mov     [cs:SRCH_CLUSTER_HW],dx
28618
28619         mov     dx,bx
28620         xor     bl,bl
28621         mov     [SECCLUSPOS],bl
28622
28623         ;CLUSTNUM_HW:DX = Physical cluster number (to FIGREC)
28624
28625         call    FIGREC
28626
28627         mov     si,[HIGH_SECTOR]
28628         mov     [DIRSEC+2],si
28629         pop     si

```

```

28630 SETROOTSRCH3:
28631 000049E6 8916[BE05]      mov     [DIRSEC],dx ; *
28632
28633      ;pop     si
28634 000049EA F8              clc     ; (this may not be needed,
28635                        ; FIGREC clears cf except an abnormal sector calc overflow)
28636 000049EB C3              retn
28637
28638 SETROOTSRCH_FAT:
28639      ;xor     ax,ax
28640 000049EC A3[C005]        mov     [DIRSEC+2],ax ; 0
28641 000049EF A3[C205]        mov     [DIRSTART],ax ; 0
28642 000049F2 A3[DC0A]        mov     [DIRSTART_HW],ax ; 0
28643 000049F5 2EA3[5D49]     mov     [cs:SRCH_CLUSTER_HW],ax ; 0
28644 000049F9 40              inc     ax ; search start dir cluster num = 1
28645                        ; (root directory)
28646 000049FA 2EA3[5B49]     mov     [cs:SRCH_CLUSTER],ax ; 1 ; FAT root directory (<2)
28647 000049FE 48              dec     ax ; 0
28648 000049FF A2[7305]        mov     [SECCCLUSPOS],al
28649 00004A02 48              dec     ax ; -1
28650 00004A03 A3[BC05]        mov     [CLUSNUM],ax ; 0FFFFh
28651 00004A06 A3[DE0A]        mov     [CLUSNUM_HW],ax ; 0FFFFh
28652
28653      ;mov     ax,[es:bp+0Bh]
28654 00004A09 268B460B        mov     ax,[es:bp+DPB.FIRST_SECTOR]
28655      ;mov     dx,[es:bp+11h]
28656 00004A0D 268B5611        mov     dx,[es:bp+DPB.DIR_SECTOR]
28657 00004A11 29D0              sub     ax,dx
28658 00004A13 A2[7705]        mov     [CLUSFAC],al
28659      ;mov     [DIRSEC],dx ; *
28660      ;clc
28661      ;setdirsrch_retn:
28662      ;retn
28663 00004A16 EBCE              jmp     short SETROOTSRCH3
28664      ;;;
28665 %endif
28666
28667 ;-----
28668
28669 ; 23/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
28670 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:8A90h
28671
28672 ; set SFT number and entry in the new internal table (only for FCB calls)
28673
28674 set_sftfcb_entry:
28675      push    ax
28676      push    cx
28677      push    dx
28678      push    bx
28679
28680      ;push    bp
28681      ;push    si
28682      ;push    di ; ES:DI = SFT entry
28683
28684 00004A1C E84B00          call    find_sft_entry_number
28685                        ; ax = SFT entry index number
28686                        ; of the last SFT entry
28687      xor     bx,bx
28688 00004A1F 31DB              mov     cx,20 ; find empty entry (slot) in the table
28689      ; Retro DOS v5.0
28690 00004A24 31D2              xor     dx,dx ; *
28691
28692 set_sftfcb_1:
28693      ;cmp     cs:(sftfcb_cluster+2)[bx],0
28694      ;cmp     word [cs:bx+sftfcb_cluster+2],0
28695      ;        ; directory (search) starting cluster, hw
28696 00004A26 2E3997[E548]     cmp     [cs:bx+sftfcb_cluster+2],dx ; 0
28697      ;jnz     short set_sftfcb_2
28698      ; 27/06/2024
28699      ;jnz     short set_sftfcb_3
28700
28701      ;cmp     cs:sftfcb_cluster[bx],
28702      ;cmp     word [cs:bx+sftfcb_cluster],0
28703      ;        ; directory (search) starting cluster, lw
28704 00004A2D 2E3997[E348]     cmp     [cs:bx+sftfcb_cluster],dx ; 0
28705
28706 set_sftfcb_2:
28707      ;jnz     short set_sftfcb_3
28708      ;        ; not empty (sftfcb table) entry
28709      ; Retro DOS v5.0
28710      ;push    cx
28711      mov     cx,[cs:SRCH_CLUSTER]
28712      ;        ; search start dir cluster number
28713 00004A39 2E898F[E348]     ;mov     cs:sftfcb_cluster[bx],cx
28714      mov     [cs:bx+sftfcb_cluster],cx
28715      ;        ; directory start cluster
28716      mov     cx,[cs:SRCH_CLUSTER_HW]
28717      ; PCDOS 7.1 BUG! (This would be '[cs:bx:sftfcb_cluster+2],cx')
28718      ; Erdogan Tan - 23/01/2024
28719      ;mov     cs:sftfcb_cluster[bx],cx
28720      ;mov     [cs:bx:sftfcb_cluster],cx ; PCDOS 7.1 IBMDOS.COM - DOSCODE:8ABFh
28721      ; Retro DOS v5.0
28722 00004A43 2E898F[E548]     mov     [cs:bx+sftfcb_cluster+2],cx
28723
28724      ;pop     cx
28725
28726      ;mov     dx,[ss:LASTENT] ; LAST (found) entry in the directory
28727      ;mov     cs:sftfcb_direntry[bx],dx
28728      ;mov     [cs:bx+sftfcb_direntry],dx
28729      ;        ; directory entry number
28730 00004A48 368B0E[4803]     mov     cx,[ss:LASTENT]
28731 00004A4D 2E898F[E748]     mov     [cs:bx+sftfcb_direntry],cx
28732
28733 00004A52 93              xchg     ax,bx
28734      ;xor     dx,dx ; *
28735      ; dx = 0
28736 00004A53 B90600          mov     cx,6 ; sftfcb table has 6 byte entries
28737 00004A56 F7F1              div     cx
28738 00004A58 93              xchg     ax,bx
28739 00004A59 2E8887[CF48]     mov     [cs:bx+sftfcb_table],al
28740      ;        ; put SFT entry number in SFT-FCB
28741      ;        ; table (offset is FCB index number 0 to 19)
28742 00004A5E EB05              jmp     short set_setfcb_4
28743
28744 set_sftfcb_3:
28745      ;add     bx,6
28746 00004A60 83C306          add     bx,sftfcb_entry_size ; 6
28747 00004A63 E2C1              loop    set_sftfcb_1
28748
28749 set_setfcb_4:
28750      ;pop     di
28751      ;pop     si
28752      ;pop     bp
28753 00004A65 5B              pop     bx

```



```

28754 00004A66 5A      pop     dx
28755 00004A67 59      pop     cx
28756 00004A68 58      pop     ax
28757 00004A69 C3      retn

28758
28759 ;-----
28760
28761 ; 24/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
28762 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:8AECh
28763
28764 ; 14/02/2024
28765 find_sft_entry_number:
28766 00004A6A 06      push    es      ; es:di = SFT entry
28767 00004A6B 31C9     xor     cx,cx
28768 00004A6D 8CC2     mov     dx,es
28769 00004A6F 2E8E06[0700]  mov     es,[cs:DosDSeg]
28770 00004A74 26C41E[2A00]  les     bx,[es:SFT_ADDR] ; address of the first SFT
28771
28772 00004A79 8CC0     f_sfte_1:
28773 00004A7B 39D0     mov     ax,es
28774 00004A7D 7512     cmp     ax,dx      ; same SFT segment ?
28775 00004A7F 89F8     jne     short f_sfte_2 ; no
28776 00004A81 29D8     mov     ax,di
28777       sub     ax,bx      ; ax = entry offset
28778       ;sub     ax,6      ; ax = offset from start of the SFT table
28779       sub     ax,SFT.SFTTable
28780       ;mov     bx,59
28781 00004A86 BB3B00     mov     bx,SF_ENTRY.size ; (SFT entry size)
28782 00004A88 F7F3     xor     dx,dx
28783 00004A8D 01C8     div     bx      ; ax = SFT entry index in the table
28784 00004A8F 07      add     ax,cx      ; ax = SFT index number (for requested SFT entry)
28785 00004A90 C3      pop     es
28786       retn
28787
28788 00004A91 26034F04  f_sfte_2:
28789 00004A95 26C41F     add     cx,[es:bx+4] ; SFT.SFCount ; number of entries in the table
28790 00004A98 83FBFF     les     bx,[es:bx] ; SFT.SFLink
28791 00004A9D F9      cmp     bx,0FFFFh ; -1 ; the last SFT
28792 00004A9E 07      jnz     short f_sfte_1
28793 00004A9F C3      stc
28794       pop     es
28795       retn
28796
28797 ;-----
28798
28799 ; 24/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
28800 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:8B22h
28801
28802 ; 14/02/2024
28803 int_2Fh_1230h: ; windows95 - FIND SFT ENTRY IN INTERNAL FILE TABLES
28804 00004AA0 E8C7FF     call    find_sft_entry_number
28805 00004AA3 7242     jc      short find_sfte_i_error
28806 00004AA5 06      push    es      ; ax = SFT entry index number (of the requested SFT entry)
28807 00004AA6 57      push    di
28808 00004AA7 0E      push    cs
28809 00004AA8 07      pop     es
28810 00004AA9 B91400     mov     cx,20      ; statically allocated with 20 entries,
28811       ; and used only for FCB calls
28812 00004AAC BF[CF48]  mov     di,sft_fcb_table ; new file system (internal) table
28813       ; only for fcb calls
28814
28815 scan_next_sftfcb:
28816 00004AAF F2AE     repne   scasb
28817 00004AB1 F9      stc
28818 00004AB2 7531     jnz     short sfte_i_notfound ; not found (cf=1)
28819 00004AB4 8D5DFF     lea     bx,[di-1] ; offset of the entry in the table
28820 00004AB7 81EB[CF48]  sub     bx,sft_fcb_table ; index into new file system table
28821 00004ABB 89DA     mov     dx,bx
28822 00004ABD 01DB     add     bx,bx      ; 2*bx
28823 00004ABF 01D3     add     bx,dx      ; 3*bx
28824 00004AC1 01DB     add     bx,bx      ; bx = 6*bx ; entry size = 6 bytes
28825 00004AC3 268BB7[E548]  mov     si,[es:bx+sftfcb.cluster+2] ; dir start cluster number, hw
28826 00004AC8 268B97[E748]  mov     dx,[es:bx+sftfcb.direntry] ; directory entry number
28827 00004ACD 51      push    cx
28828 00004ACE 268B8F[E348]  mov     cx,[es:bx+sftfcb.cluster] ; dir start cluster number, lw
28829 00004AD3 09C9     or      cx,cx
28830 00004AD5 750C     jnz     short sfte_i_found
28831 00004AD7 09F6     or      si,si
28832 00004AD9 7508     jnz     short sfte_i_found
28833 00004ADB 59      pop     cx
28834 00004ADC 09C9     or      cx,cx
28835 00004ADE 75CF     jnz     short scan_next_sftfcb
28836 00004AE0 F9      stc
28837 00004AE1 EB02     jmp     short sfte_i_notfound
28838
28839 sfte_i_found:
28840 00004AE3 58      pop     ax      ; clear stack
28841 00004AE4 F8      clc      ; found (cf=0)
28842
28843 sfte_i_notfound:
28844 00004AE5 5F      pop     di
28845 00004AE6 07      pop     es
28846
28847 find_sfte_i_error:
28848 00004AE7 B80000     mov     ax,0
28849 00004AEA C3      retn
28850
28851 ;-----
28852
28853 ; 24/01/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
28854 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:8B6Dh
28855
28856 ; 14/02/2024
28857 SFT_FREE:
28858 00004AEB 50      push    ax
28859 00004AEC 51      push    cx
28860 00004AED 52      push    dx
28861 00004AEE 53      push    bx
28862       ;push    bp      ; 14/02/2024 - Retro DOS v5.0
28863 00004AEF 56      push    si
28864       ;push    di
28865 00004AF0 26C7050000  mov     word [es:di],0 ; [ES:DI+SF_ENTRY.sf_ref_Count]
28866 00004AF5 E8A8FF     call    int_2Fh_1230h
28867 00004AF8 720E     jc      short sftf_1
28868 00004AFA 2EC787[E548]0000  mov     word [cs:bx+sftfcb.cluster+2],0 ; clear directory start cluster
28869 00004B01 2EC787[E348]0000  ; in the new (sftfcb) table
28870       mov     word [cs:bx+sftfcb.cluster],0 ; ((empty entry))
28871
28872 sftf_1:
28873       ;pop     di
28874       pop     si
28875       pop     bp
28876       pop     bx
28877       pop     dx
28878       pop     cx
28879       pop     ax
28880       retn
28881
28882 ; ===== S U B R O U T I N E =====
28883

```

```

28878 ; 13/02/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
28879 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:8B94h
28880
28881 check_longname:
28882 00004B0E 240F and al,0Fh
28883 00004B10 3C0F cmp al,0Fh
28884 00004B12 C3 retn
28885
28886 ;-----
28887
28888 ;=====
28889 ; DIR2.ASM, MSDOS 6.0, 1991
28890 ;=====
28891 ; 27/07/2018 - Retro DOS v3.0
28892 ; 19/05/2019 - Retro DOS v4.0
28893 ; 14/02/2024 - Retro DOS v5.0
28894
28895 ; TITLE DIR2 - Directory and path cracking
28896 ; NAME Dir2
28897
28898 ;Break <GETPATH -- PARSE A WFP>
28899 ;-----
28900
28901 ; Procedure Name : GETPATH
28902 ;
28903 ; Inputs:
28904 ; [WFP_START] Points to WFP string ("d:\" must be first 3 chars, NUL
28905 ; terminated; d:/ (note forward slash) indicates a real device).
28906 ; [CURR_DIR_END] Points to end of Current dir part of string
28907 ; (= -1 if current dir not involved, else
28908 ; Points to first char after last "/" of current dir part)
28909 ; [THISCD] Points to CDS being used
28910 ; [SATTRIB] Is attribute of search, determines what files can be found
28911 ; [NoSetDir] set
28912 ; [THISDPB] set to DPB if disk otherwise garbage.
28913 ; Function:
28914 ; Crack the path
28915 ; Outputs:
28916 ; Sets EXTERR_LOCUS = errLOC_Disk if disk file
28917 ; Sets EXTERR_LOCUS = errLOC_Unk if char device
28918 ; ID1 field of [THISCD] updated appropriately
28919 ; [ATTRIB] = [SATTRIB]
28920 ; ES:BP Points to DPB
28921 ; Carry set if bad path
28922 ; SI Points to path element causing failure
28923 ; Zero set
28924 ; [DIRSTART],[DIRSEC],[CLUSNUM], and [CLUSFAC] are set up to
28925 ; start a search on the last directory
28926 ; CL is zero if there is a bad name in the path
28927 ; CL is non-zero if the name was simply not found
28928 ; [ENTFREE] may have free spot in directory
28929 ; [NAME1] is the name.
28930 ; CL = 81H if '*'s or '?' in NAME1, 80H otherwise
28931 ; Zero reset
28932 ; File in middle of path or bad name in path or attribute mismatch
28933 ; or path too long or malformed path
28934 ; ELSE
28935 ; [CurBuf] = -1 if root directory
28936 ; [CURBUF] contains directory record with match
28937 ; [CURBUF+2]:BX Points into [CURBUF] to start of entry
28938 ; [CURBUF+2]:SI Points into [CURBUF] to dir_first field for entry
28939 ; AH = device ID
28940 ; bit 7 of AH set if device SI and BX
28941 ; will point DOSGROUP relative The firclus
28942 ; field of the device entry contains the device pointer
28943 ; [NAME1] Has name looked for
28944 ; If last element is a directory zero is set and:
28945 ; [DIRSTART],[SECCLUSPOS],[DIRSEC],[CLUSNUM], and [CLUSFAC]
28946 ; are set up to start a search on it.
28947 ; unless [NoSetDir] is non zero in which case the return is
28948 ; like that for a file (except for zero flag)
28949 ; If last element is a file zero is reset
28950 ; [DIRSEC],[CLUSNUM],[CLUSFAC],[NXTCLUSNUM],[SECCLUSPOS],
28951 ; [LASTENT],[ENTLAST] are set to continue search of last
28952 ; directory for further matches on NAME1 via the NEXTENT
28953 ; entry point in FindEntry (or GETENT entry in GETENTRY in
28954 ; which case [NXTCLUSNUM] and [SECCLUSPOS] need not be valid)
28955 ; DS preserved, Others destroyed
28956 ;-----
28957
28958 ;hkn; called from delete.asm, finfo.asm, mknnode.asm and rename.asm.
28959 ;hkn; DS already set up at this point.
28960
28961 ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
28962
28963 ; 15/02/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
28964 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:8B99h
28965
28966 GETPATH:
28967 ;mov word [CREATING],0E500h
28968 00004B13 C706[7E05]00E5 MOV WORD [CREATING],DIRFREE*256+0 ; Not Creating, not DEL *.*
28969
28970 ; Same as GetPath only CREATING and DELALL already set
28971
28972 ;entry GetPathNoSet
28973 GetPathNoSet:
28974 ;mov byte [EXTERR_LOCUS],2
28975 ;MOV byte [EXTERR_LOCUS],errLOC_Disk
28976 ; 15/02/2024 (PC DOS 7.1 IBMDOS.COM)
28977 00004B19 E8CAC7 call set_exerr_locus_disk
28978
28979 00004B1C C706[E205]FFFF ; MOV word [CURBUF],-1 ; initial setting
28980
28981 ; See if the input indicates a device that has already been detected. If so,
28982 ; go build the guy quickly. Otherwise, let findpath find the device.
28983
28984 00004B22 8B3E[B205] MOV DI,[WFP_START] ; point to the beginning of the name
28985 ;cmp word [DI+1],5C3Ah
28986 ;CMP WORD [DI+1],'\ ' << 8 + ':'
28987 00004B26 817D013A5C cmp word [DI+1],'\ '
28988 00004B2B 7435 JZ short CrackIt
28989
28990 ; Let ChkDev find it in the device list
28991
28992 00004B2D 83C703 ADD DI,3
28993 ; 18/08/2018
28994 ;MOV SI,DI ; let CHKDEV see the original name
28995 ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
28996 ; 16/12/2022
28997 ;mov si,di ; not required ! (it is written in CHKDEV proc already!)
28998 00004B30 E8B200 CALL CHKDEV
28999 00004B33 722B JC short InternalError
29000
29001 Build_devj:

```

```

29002 00004B35 A0[6D05]      MOV     AL,[SATTRIB]
29003 00004B38 A2[6B05]      MOV     [ATTRIB],AL
29004                          ; 15/02/2024
29005                          ;;mov byte [EXTERR_LOCUS],1
29006                          ;MOV     byte [EXTERR_LOCUS],errLOC_Unk ; In the particular case of
29007                          ;                          ; "finding" a char device
29008                          ;                          ; set LOCUS to Unknown. This makes
29009                          ;                          ; certain idiotic problems reported
29010                          ;                          ; by a certain 3 letter OEM go away.
29011                          ; 15/02/2024 (PCDOS 7.1 IBMDOS.COM)
29012 00004B3B E89FC7      call    set_exerr_locus_unk
29013
29014                          ; Take name in name1 and pack it back into where wfp_start points. This
29015                          ; guarantees wfp_start pointing to a canonical representation of a device.
29016                          ; We are allowed to do this as GetPath is *ALWAYS* called before entering a
29017                          ; wfp into the share set.
29018                          ;
29019                          ; We copy chars from name1 to wfp_start remembering the position of the last
29020                          ; non-space seen +1. This position is kept in DX.
29021
29022                          ;hkn; SS is DOSDATA
29023 00004B3E 16            push    ss
29024 00004B3F 07            pop     es
29025
29026                          ;hkn; NAME1 is in DOSDATA
29027 00004B40 BE[4B05]      mov     si,NAME1
29028 00004B43 8B3E[B205]   mov     di,[WFP_START]
29029 00004B47 89FA          mov     dx,di
29030 00004B49 B90800      mov     cx,8                ; 8 chars in device name
29031                          MoveLoop:
29032 00004B4C AC            lodsb
29033 00004B4D AA            stosb
29034 00004B4E 3C20          cmp     al," "
29035 00004B50 7402          jz      short NoSave
29036
29037 00004B52 89FA          mov     dx,di
29038                          NoSave:
29039 00004B54 E2F6          loop    MoveLoop
29040
29041                          ; DX is the position of the last seen non-space + 1. We terminate the name
29042                          ; at this point.
29043
29044 00004B56 89D7          mov     di,dx
29045                          ;mov     byte [di],0                ; end of string
29046                          ; 15/02/2024
29047 00004B58 880D          mov     [di],cl ; 0
29048 00004B5A E8D502      call    Build_device_ent    ; Clears carry sets zero
29049 00004B5D FEC0          inc     al                ; reset zero
29050 00004B5F C3            retn
29051
29052                          InternalError:
29053                          InternalError_loop:
29054 00004B60 EBFE          jmp     short InternalError_loop ; freeze
29055
29056                          ; Start off at the correct spot. Optimize if the current dir part is valid.
29057
29058                          CrackIt:
29059                          ; 15/02/2024
29060                          %if 0
29061                          MOV     SI,[CURR_DIR_END]        ; get current directory pointer
29062                          CMP     SI,-1                    ; valid?
29063                          JNZ     short LOOK_SING          ; Yes, use it.
29064                          LEA     SI,[DI+3]                ; skip D:\.
29065                          LOOK_SING:
29066                          %endif
29067                          ;mov     byte [ATTRIB],16h
29068 00004B62 C606[6B05]16  mov     byte [ATTRIB],attr_directory+attr_system+attr_hidden
29069                          ; Attributes to search through Dirs
29070 00004B67 C43E[A205]   les     DI,[THISCDS]
29071 00004B6B B8FFFF      mov     AX,-1 ; 0FFFFh
29072                          ;;;
29073                          ; 15/02/2024 (PCDOS 7.1 IBMDOS.COM)
29074                          ;mov     bx,[es:di+4Bh]
29075 00004B6E 268B5D4B     mov     bx,[es:di+curdir.ID+2]
29076 00004B72 891E[EE0A]   mov     [ROOTCLUST_HW],bx
29077                          ;;;
29078                          ;mov     bx,[es:di+73]
29079 00004B76 268B5D49     mov     BX,[ES:DI+curdir.ID]
29080 00004B7A 8B36[B605]   mov     SI,[CURR_DIR_END]
29081
29082                          ; AX = -1
29083                          ; BX = cluster number of current directory. This number is -1 if the media
29084                          ; has been uncertainly changed.
29085                          ; SI = offset in DOSGroup into path to end of current directory text. This
29086                          ; may be -1 if no current directory part has been used.
29087
29088 00004B7E 39C6          cmp     SI,AX                ; if Current directory is not part
29089 00004B80 7449          jz      short NO_CURR_D      ; then we must crack from root
29090                          ;;;
29091                          ; 15/02/2024 (PCDOS 7.1 IBMDOS.COM)
29092 00004B82 3906[EE0A]   cmp     [ROOTCLUST_HW],ax ; 0FFFFh
29093 00004B86 7504          jnz     short CrackIt2
29094                          ;;;
29095 00004B88 39C3          cmp     BX,AX                ; is the current directory cluster valid
29096
29097                          ; DOS 3.3 6/25/86
29098 00004B8A 743F          jz      short NO_CURR_D      ; no, crack from the root
29099
29100                          CrackIt2: ; 15/02/2024 - Retro DOS v5.0
29101
29102                          ;test byte [FastOpenFlg],1
29103 00004B8C F606[4611]01  test    byte [FastOpenFlg],FastOpen_Set ; for fastopen ?
29104 00004B91 7445          jz      short GOT_SEARCH_CLUSTER ; no
29105 00004B93 06            push    ES                ; save registers
29106 00004B94 57            push    DI
29107 00004B95 51            push    CX
29108 00004B96 FF74FF      push    word [SI-1]        ; save \ and 1st char of next element
29109 00004B99 56            push    SI
29110 00004B9A 53            push    BX
29111                          ;;; 15/02/2024
29112 00004B9B FF36[EE0A]   push    word [ROOTCLUST_HW]
29113                          ;;;
29114 00004B9F C644FF00      mov     BYTE [SI-1],0      ; call fastopen to look up cur dir info
29115 00004BA3 8B36[B205]   mov     SI,[WFP_START]
29116
29117                          ;hkn; FastopenTable, Dir_Info_Buff & FastOpen_Ext_Info are in DOSDATA
29118 00004BA7 BB[3C11]      mov     BX,FastOpenTable
29119 00004BAA BF[7C0D]      mov     DI,Dir_Info_Buff
29120 00004BAD B9[4711]      mov     CX,FastOpen_Ext_Info
29121                          ;mov     al,1
29122 00004BB0 B001      mov     AL,FONC_Look_up
29123 00004BB2 1E            push    DS
29124 00004BB3 07            pop     ES
29125                          ;call far [BX+2]

```

```

29126 00004BB4 FF5F02      CALL    far [BX+fastopen_entry.name_caching]
29127 00004BB7 7203        JC      short GO_Chk_end1      ;fastopen not installed, or wrong drive.
29128                                     ; Go to Got_Srch_cluster
29129                                     ; 29/12/2022
29130                                     ;CMP    BYTE [SI],0      ;fastopen has current dir info?
29131                                     ;JE      short GO_Chk_end      ;yes. Go to got_search_cluster
29132                                     ;stc
29133                                     ;jmp     short GO_Chk_end      ;Go to No_Curr_D
29134
29135 00004BB9 803C01        cmp     byte [si],1
29136 GO_Chk_end1:          ; 29/12/2022
29137 00004BBC F5            cmc
29138                                     ; [si] = 0 -> cf = 0
29139                                     ; [si] > 0 -> cf = 1
29140
29141 ;GO_Chk_end1:
29142 ; 29/12/2022
29143 ;clc
29144
29145 GO_Chk_end:            ; restore registers
29146 ;;; 15/02/2024
29147 00004BBD 8F06[EE0A]   pop     word [ROOTCLUST_HW]
29148 ;;;
29149 00004BC1 5B            POP     BX
29150 00004BC2 5E            POP     SI
29151 00004BC3 8F44FF        POP     word [SI-1]
29152 00004BC6 59            POP     CX
29153 00004BC7 5F            POP     DI
29154 00004BC8 07            POP     ES
29155 00004BC9 730D        JNC     short GOT_SEARCH_CLUSTER ; crack based on cur dir
29156
29157 ; DOS 3.3 6/25/86
29158 ;
29159 ; We must cract the path beginning at the root. Advance pointer to beginning
29160 ; of path and go crack from root.
29161
29162 NO_CURR_D:
29163 00004BCB 8B36[B205]   MOV     SI,[WFP_START]
29164                                     ;LEA     SI,[SI+3]      ; Skip "d:/"
29165                                     ; 15/02/2024
29166 00004BCF 83C603        add     si,3
29167 00004BD2 C42E[8A05]   LES     BP,[THISDPB]      ; Get ES:BP
29168 00004BD6 EB37        JMP     short ROOTPATH
29169
29170 ; We are able to crack from the current directory part. Go set up for search
29171 ; of specified cluster.
29172
29173 GOT_SEARCH_CLUSTER:
29174 00004BD8 C42E[8A05]   LES     BP,[THISDPB]      ; Get ES:BP
29175 00004BDC E880FD        call    SETDIRSRCH
29176                                     ;JC      short SETFERR
29177                                     ;JMP     short FINDPATH
29178                                     ; 16/12/2022
29179 00004BDF 733E        jnc     short FINDPATH ; 17/08/2018
29180                                     ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
29181                                     ;JC      short SETFERR
29182                                     ;JMP     short FINDPATH
29183
29184 00004BE1 30C9        SETFERR:
29185 00004BE3 F9            XOR     CL,CL      ; set zero
29186 00004BE4 C3            STC
29187                                     retn
29188
29189 ;-----
29190 ; Procedure Name : ChkDev
29191 ;
29192 ; Check to see if the name at DS:DI is a device. Returns carry set if not a
29193 ; device.
29194 ; Blasts CX,SI,DI,AX,BX
29195 ;-----
29196
29197 CHKDEV:
29198 00004BE5 89FE        MOV     SI,DI
29199                                     ;MOV     DI,SS
29200                                     ;MOV     ES,DI
29201                                     ; 27/06/2024
29202 00004BE7 16            push    ss
29203 00004BE8 07            pop     es
29204
29205 00004BE9 BF[4B05]     MOV     DI,NAME1
29206 00004BEC B90900     MOV     CX,9
29207
29208 00004BEF E8A411     TESTLOOP:
29209                                     call    GETLET
29210
29211 00004BF2 3C2E        CMP     AL,'.'
29212 00004BF4 740E        JZ      short TESTDEVICE
29213 00004BF6 E8F111     call    PATHCHRCMP
29214 00004BF8 7407        JZ      short NOTDEV
29215 00004BFB 08C0        OR      AL,AL
29216 00004BFD 7405        JZ      short TESTDEVICE
29217
29217 00004BFF AA            STOSB
29218 00004C00 E2ED        LOOP    TESTLOOP
29219
29220 00004C02 F9            NOTDEV:
29221 00004C03 C3            STC
29222                                     retn
29223
29224 TESTDEVICE:
29225                                     ;ADD     CX,2
29226                                     ; 24/09/2023
29227 00004C04 41            inc     cx
29228 00004C05 41            inc     cx
29229 00004C06 B020        MOV     AL,' '
29230 00004C08 F3AA        REP     STOSB
29231                                     ;MOV     AX,SS
29232                                     ;MOV     DS,AX
29233                                     ; 27/06/2024
29234 00004C0A 16            push    ss
29235 00004C0B 1F            pop     ds
29236                                     ;call    DEVNAME
29237                                     ;retn
29238 00004C0C E9C501     ; 18/12/2022
29239                                     jmp     DEVNAME
29240
29241 ;Break    <ROOTPATH, FINDPATH -- PARSE A PATH>
29242 ;-----
29243 ; Procedure Name : ROOTPATH,FINDPATH
29244 ;
29245 ; Inputs:
29246 ; Same as FINDPATH but,
29247 ; SI Points to asciz string of path which is assumed to start at
29248 ; the root (no leading '/').
29249 ; Function:

```

```

29250 ; Search from root for path
29251 ; Outputs:
29252 ; Same as FINDPATH but:
29253 ; If root directory specified, [CURBUF] and [NAME1] are NOT set, and
29254 ; [NoSetDir] is ignored.
29255 ;-----
29256 ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
29257 ; DOSCODE:7F47h (MSDOS 5.0, MSDOS.SYS)
29258
29259 ; 15/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
29260 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:8C9Dh
29261
29262 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:7F82h)
29263 ; (Windows ME IO.SYS - BIOSCODE:829Eh)
29264
29265
29266 ROOTPATH:
29267 00004C0F E879FD call SETROOTSRCH
29268 ; 24/09/2023
29269 00004C12 30E4 xor ah,ah
29270 ;CMP BYTE [SI],0
29271 00004C14 3824 cmp [si],ah ; 0
29272 00004C16 7507 jnz short FINDPATH
29273
29274 ;;;
29275 ; 15/02/2024
29276 ; Windows ME IO.SYS - BIOSCODE:82A6h
29277 ;mov word [CURBUF],0FFFFh
29278 ;;;
29279
29280 ; Root dir specified
29281 00004C18 A0[6D05] mov al,[ATTRIB]
29282 00004C1B A2[6B05] mov [ATTRIB],al
29283 ; 24/09/2023
29284 ;XOR AH,AH ; Sets "device ID" byte, sets zero
29285 ; (dir), clears carry.
29286 00004C1E C3 retn
29287
29288 ; Inputs:
29289 ; [ATTRIB] Set to get through directories
29290 ; [SATTRIB] Set to find last element
29291 ; ES:BP Points to DPB
29292 ; SI Points to asciz string of path (no leading '/').
29293 ; [SECCLUSPOS] = 0
29294 ; [DIRSEC] = Phys sec # of first sector of directory
29295 ; [CLUSNUM] = Cluster # of next cluster
29296 ; [CLUSFAC] = Sectors per cluster
29297 ; [NoSetDir] set
29298 ; [CURR_DIR_END] Points to end of Current dir part of string
29299 ; (= -1 if current dir not involved, else
29300 ; Points to first char after last "/" of current dir part)
29301 ; [THISCDS] Points to CDS being used
29302 ; [CREATING] and [DELALL] set
29303 ; Function:
29304 ; Parse path name
29305 ; Outputs:
29306 ; ID1 field of [THISCDS] updated appropriately
29307 ; [ATTRIB] = [SATTRIB]
29308 ; ES:BP Points to DPB
29309 ; [THISDPB] = ES:BP
29310 ; Carry set if bad path
29311 ; SI Points to path element causing failure
29312 ; Zero set
29313 ; [DIRSTART],[DIRSEC],[CLUSNUM], and [CLUSFAC] are set up to
29314 ; start a search on the last directory
29315 ; CL is zero if there is a bad name in the path
29316 ; CL is non-zero if the name was simply not found
29317 ; [ENTFREE] may have free spot in directory
29318 ; [NAME1] is the name.
29319 ; CL = 81H if '*'s or '?' in NAME1, 80H otherwise
29320 ; Zero reset
29321 ; File in middle of path or bad name in path
29322 ; or path too long or malformed path
29323 ;
29324 ; ELSE
29325 ; [CURBUF] contains directory record with match
29326 ; [CURBUF+2]:BX Points into [CURBUF] to start of entry
29327 ; [CURBUF+2]:SI Points to fcb_FIRCLUS field for entry
29328 ; [NAME1] Has name looked for
29329 ; AH = device ID
29330 ; bit 7 of AH set if device SI and BX
29331 ; will point DOSGROUP relative The firclus
29332 ; field of the device entry contains the device pointer
29333 ; If last element is a directory zero is set and:
29334 ; [DIRSTART],[SECCLUSPOS],[DIRSEC],[CLUSNUM], and [CLUSFAC]
29335 ; are set up to start a search on it,
29336 ; unless [NoSetDir] is non zero in which case the return is
29337 ; like that for a file (except for zero flag)
29338 ; If last element is a file zero is reset
29339 ; [DIRSEC],[CLUSNUM],[CLUSFAC],[NXTCLUSNUM],[SECCLUSPOS],
29340 ; [LASTENT],[ENTLAST] are set to continue search of last
29341 ; directory for further matches on NAME1 via the NEXTENT
29342 ; entry point in FindEntry (or GETENT entry in GETENTRY in
29343 ; which case [NXTCLUSNUM] and [SECCLUSPOS] need not be valid)
29344 ; Destroys all other registers
29345 ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
29346 ; DOSCODE:7F58h (MSDOS 5.0, MSDOS.SYS)
29347
29348 ; 15/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
29349 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:8CAEh
29350
29351 ;entry FINDPATH
29352 FINDPATH:
29353 00004C1F 06 push es ; Save ES:BP
29354 00004C20 56 push si
29355 00004C21 89F7 mov di,si
29356 00004C23 8B0E[C205] mov cx,[DIRSTART] ; Get start clus of dir being searched
29357 00004C27 833E[B605]FF cmp word [CURR_DIR_END],-1
29358 00004C2C 7415 jz short NOIDS ; No current dir part
29359 00004C2E 3B3E[B605] cmp di,[CURR_DIR_END]
29360 00004C32 750F jnz short NOIDS ; Not to current dir end yet
29361 00004C34 C43E[A205] les di,[THISCDS]
29362 ;;;
29363 ; 15/02/2024 (PCDOS 7.1 IBMDOS.COM)
29364 00004C38 A1[DC0A] mov ax,[DIRSTART_HW]
29365 ;mov [es:di+4Bh],ax
29366 00004C3B 2689454B mov [es:di+curdir.ID+2],ax
29367 ;;;
29368 ;mov [es:di+73],cx
29369 00004C3F 26894D49 mov [ES:DI+curdir.ID],CX ; Set current directory cluster
29370 NOIDS:
29371
29372 ; Parse the name off of DS:SI into NAME1. AL = 1 if there was a meta
29373 ; character in the string. CX,DI may be destroyed.

```

```

29374 ;
29375 ; invoke NAMETRANS
29376 ; MOV CL,AL
29377 ;
29378 ; The above is the slow method. The name has *already* been munged by
29379 ; TransPath so no special casing needs to be done. All we do is try to copy
29380 ; the name until ., \ or 0 is hit.
29381 ;
29382 ;MOV AX,SS
29383 ;MOV ES,AX
29384 ; 15/02/2024 (PCDOS 7.1 IBMDOS.COM)
29385 00004C43 16 push ss
29386 00004C44 07 pop es
29387 ;
29388 ;hkn; Name1 is in DOSDATA
29389 00004C45 BF[4B05] MOV DI,NAME1
29390 00004C48 B82020 MOV AX,' ' ; 2020h
29391 00004C4B AA STOSB
29392 00004C4C AB STOSW
29393 00004C4D AB STOSW
29394 00004C4E AB STOSW
29395 00004C4F AB STOSW
29396 00004C50 AB STOSW
29397 ;
29398 ;hkn; Name1 is in DOSDATA
29399 00004C51 BF[4B05] MOV DI,NAME1
29400 00004C54 30E4 XOR AH,AH ; bits for CL
29401 GetNam:
29402 ; 19/05/2019 - Retro DOS v4.0
29403 ;INC CL ; ?*! ; MSDOS 6.0 ; AN000; KK increment valid count
29404 ;
29405 ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
29406 ; 16/12/2022
29407 ;inc cl ; not required !
29408 ;
29409 00004C56 AC LODSB
29410 00004C57 3C2E CMP AL,'.' ; 2Eh
29411 00004C59 7412 JZ short _SetExt
29412 00004C5B 08C0 OR AL,AL
29413 00004C5D 7424 JZ short _GetDone
29414 00004C5F 3C5C CMP AL,'\' ; 5Ch
29415 00004C61 7420 JZ short _GetDone
29416 00004C63 3C3F CMP AL,'?' ; 3Fh
29417 00004C65 7503 JNZ short StoNam
29418 00004C67 80CC01 OR AH,1
29419 StoNam:
29420 00004C6A AA STOSB
29421 00004C6B EBE9 JMP short GetNam
29422 _SetExt:
29423 00004C6D BF[5305] MOV DI,NAME1+8
29424 GetExt:
29425 00004C70 AC LODSB
29426 00004C71 08C0 OR AL,AL
29427 00004C73 740E JZ short _GetDone
29428 00004C75 3C5C CMP AL,'\'
29429 00004C77 740A JZ short _GetDone
29430 00004C79 3C3F CMP AL,'?'
29431 00004C7B 7503 JNZ short StoExt
29432 00004C7D 80CC01 OR AH,1
29433 StoExt:
29434 00004C80 AA STOSB
29435 00004C81 EBED JMP short GetExt
29436 _GetDone:
29437 00004C83 4E DEC SI
29438 00004C84 88E1 MOV CL,AH ; 0 or 1 ; 29/12/2022
29439 00004C86 80C980 OR CL,80H
29440 00004C89 5F POP DI ; Start of this element
29441 00004C8A 07 POP ES ; Restore ES:BP
29442 00004C8B 39FE CMP SI,DI
29443 00004C8D 7503 JNZ short check_device
29444 00004C8F E9ED00 JMP _BADPATH ; NUL parse (two delims most likely)
29445 check_device:
29446 00004C92 56 PUSH SI ; Start of next element
29447 ;MOV AL,[SI]
29448 ; 15/02/2024
29449 00004C93 08C0 OR AL,AL
29450 ; 23/09/2023
29451 ;cmp byte [si],0
29452 00004C95 7508 JNZ short NOT_LAST
29453 ;
29454 ; for last element of the path switch to the correct search attributes
29455 ;
29456 00004C97 8A3E[6D05] MOV BH,[SATTRIB]
29457 00004C9B 883E[6B05] MOV [ATTRIB],BH
29458 ;
29459 NOT_LAST:
29460 ;
29461 ; check name1 to see if we have a device...
29462 ;
29463 00004C9F 06 PUSH ES ; Save ES:BP
29464 ;
29465 ;hkn; SS is DOSDATA
29466 ;context ES
29467 00004CA0 16 push ss
29468 00004CA1 07 pop es
29469 00004CA2 E82F01 call DEVNAME ; blast BX
29470 00004CA5 07 POP ES ; Restore ES:BP
29471 00004CA6 7208 JC short FindFile ; Not a device
29472 00004CA8 08C0 OR AL,AL ; Test next char again
29473 ;JZ short GO_BDEV
29474 ;JMP FILEINPATH ; Device name in middle of path
29475 ; 27/06/2024
29476 00004CAA 752D jnz short FILEINPATH_j
29477 ;
29478 GO_BDEV:
29479 00004CAC 5E POP SI ; Points to NUL at end of path
29480 00004CAD E985FE JMP Build_devJ
29481 ;
29482 FindFile:
29483 ;;;; 7/28/86
29484 00004CB0 803E[4B05]E5 CMP BYTE [NAME1],0E5H ; if 1st char = E5
29485 00004CB5 7505 JNZ short NOE5 ; no
29486 00004CB7 C606[4B05]05 MOV BYTE [NAME1],05H ; change it to 05
29487 NOE5:
29488 ;;;; 7/28/86
29489 00004CBC 57 PUSH DI ; Start of this element
29490 00004CBD 06 PUSH ES ; Save ES:BP
29491 00004CBE 51 PUSH CX ; CL return from NameTrans
29492 ;DOS 3.3 FastOpen 6/12/86 F.C.
29493 ;
29494 00004CBF E8B002 CALL LookupPath ; call fastopen to get dir entry
29495 00004CC2 7303 JNC short DIR_FOUND ; found dir entry
29496 ;
29497 ;DOS 3.3 FastOpen 6/12/86 F.C.

```



```

29498 00004CC4 E87AFA      call    FINDENTRY
29499
29500 00004CC7 59      DIR_FOUND:
29501 00004CC8 07      POP     CX
29502 00004CC9 5F      POP     ES
29503 00004CCA 7303     POP     DI
29504 00004CCC E9D500     JNC     short LOAD_BUF
29505                                JMP     BADPATHPOP
29506
29507 00004CCF C53E[E205]  LOAD_BUF:
29508                                LDS     DI,[CURBUF]
29509                                ;test  byte [bx+0Bh],10h
29510 00004CD3 F6470B10     TEST    BYTE [BX+dir_entry.dir_attr],attr_directory
29511 00004CD7 7503     JNZ     short GO_NEXT ; DOS 3.3
29512 00004CD9 E9A700     FILEINPATH_j:
29513                                JMP     FILEINPATH ; 27/06/2024 ; Error or end of path
29514
29515                                ; if we are not setting the directory, then check for end of string
29516
29517                                GO_NEXT:
29518                                ;hkn; SS override
29519 00004CDC 36803E[4C03]00     CMP     BYTE [SS:NoSetDir],0
29520 00004CE2 7423     JZ      short SetDir
29521 00004CE4 89FA     MOV     DX,DI ; Save pointer to entry
29522 00004CE6 8CD9     MOV     CX,DS
29523
29524                                ;hkn; SS is DOSDATA
29525                                ;context DS
29526 00004CE8 16     push    ss
29527 00004CE9 1F     pop     ds
29528 00004CEA 5F     POP     DI ; Start of next element
29529                                ; 19/05/2019 - Retro DOS v4.0
29530                                ; MSDOS 6.0
29531 00004CEB F606[4611]01     TEST    byte [FastOpenFlg],FastOpen_Set ;only DOSOPEN can take advantage of
29532 00004CF0 740B     JZ      short _nofast ; the FastOpen
29533 00004CF2 F606[4611]02     TEST    byte [FastOpenFlg],Lookup_Success ; Lookup just happened
29534 00004CF7 7404     JZ      short _nofast ; no
29535 00004CF9 8B3E[9C0D]     MOV     DI,[Next_Element_Start] ; no need to insert it again
29536 00004CFD 803D00     _nofast:
29537                                CMP     BYTE [DI],0
29538                                ;;JNZ  short NEXT_ONE ; DOS 3.3
29539                                ;;JMP  _SETRET ; retn ; Got it
29540                                ;retn ; 05/09/2018
29541                                ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
29542                                ;jmp  _SETRET
29543 00004D00 7431     ; 16/12/2022
29544                                jz      short _SETRET
29545
29546 00004D02 57     NEXT_ONE:
29547 00004D03 89D7     PUSH    DI ; Put start of next element back on stack
29548 00004D05 8ED9     MOV     DI,DX
29549                                MOV     DS,CX ; Get back pointer to entry
29550                                SetDir:
29551                                ;;
29552                                ; 15/02/2024 (PCDOS 7.1 IBMDOS.COM)
29553                                xor     dx,dx ; 0
29554 00004D09 2639560F     ;cmp  [es:bp+0Fh],dx
29555 00004D0D 7503     cmp     [es:bp+DPB.FAT_SIZE],dx ; 0
29556 00004D0F 8B54FA     jnz     short SetDir2 ; not FAT32
29557                                mov     dx,[si-6] ; dir_entry.dir_fclus_hi
29558 00004D12 368916[EE0A]     SetDir2:
29559                                mov     [ss:ROOTCLUST_HW],dx ; 0
29560 00004D17 8B14     ;;
29561                                MOV     DX,[SI] ; dir_entry.dir_first
29562
29563 00004D19 1E     ;DOS 3.3 FastOpen 6/12/86 F.C.
29564                                PUSH    DS ; save [curbuf+2]
29565 00004D1A 16     ;hkn; SS is DOSDATA
29566 00004D1B 1F     push    ss
29567                                pop     ds ; set DS Dosgroup
29568 00004D1C F606[4611]02     ;test  byte [FastOpenFlg],2
29569 00004D21 7411     TEST    byte [FastOpenFlg],Lookup_Success
29570 00004D23 89D3     JZ      short DO_NORMAL ; fastopen not in memory or path not
29571 00004D25 8B3E[BC05]     MOV     BX,DX ; not found
29572 00004D29 50     MOV     DI,[CLUSNUM] ; clusnum was set in LookupPath
29573 00004D2A E832FC     PUSH    AX ; save device id (AH)
29574 00004D2D 58     call    SETDIRSRCH
29575 00004D2E 83C402     POP     AX ; restore device id (AH)
29576 00004D31 EB36     ADD     SP,2 ; pop ds in stack
29577                                JMP     short FAST_OPEN_SKIP
29578
29579                                ; 16/12/2022
29580 00004D33 C3     _SETRET:
29581                                retn
29582
29583 00004D34 1F     DO_NORMAL:
29584                                POP     DS ; DS = [curbuf + 2]
29585                                ;DOS 3.3 FastOpen 6/12/86 F.C.
29586 00004D35 29FB     SUB     BX,DI ; Offset into sector of start of entry
29587 00004D37 29FE     SUB     SI,DI ; Offset into sector of dir_first
29588 00004D39 53     PUSH    BX
29589 00004D3A 50     PUSH    AX
29590 00004D3B 56     PUSH    SI
29591 00004D3C 51     PUSH    CX
29592
29593                                ; 15/02/2024
29594                                %if 0
29595                                ;push  word [di+6]
29596                                PUSH    WORD [DI+BUFFINFO.buf_sector] ;AN000;>32mb
29597                                ; 19/05/2019
29598                                ; MSDOS 6.0
29599                                ;push  word [di+8]
29600                                PUSH    WORD [DI+BUFFINFO.buf_sector+2] ;AN000;>32mb
29601                                %else
29602                                ; 15/02/2024 (PCDOS 7.1 IBMDOS.COM)
29603                                ;lds  bx,[di+6]
29604 00004D3D C55D06     lds     bx,[di+BUFFINFO.buf_sector]
29605 00004D40 53     push    bx
29606 00004D41 1E     push    ds
29607                                %endif
29608
29609 00004D42 89D3     MOV     BX,DX
29610
29611                                ;hkn; SS is DOSDATA
29612                                ;context DS
29613 00004D44 16     push    ss
29614 00004D45 1F     pop     ds
29615                                ;invoke SETDIRSRCH ; This uses UNPACK which might blow
29616 00004D46 E816FC     call    SETDIRSRCH ; the entry sector buffer
29617                                ; 19/05/2019
29618                                ; MSDOS 6.0
29619 00004D49 8F06[0706]     POP     word [HIGH_SECTOR]
29620 00004D4D 5A     POP     DX
29621 00004D4E 7203     JC      short SKIP_GETB

```

```

29622             ; 22/09/2023
29623             ;;mov byte [ALLOWED],18h
29624             ;MOV byte [ALLOWED],Allowed_RETRY+Allowed_FAIL ; *
29625             ;XOR AL,AL ; *
29626             ;;invoke GETBUFFER ; Get the entry buffer back
29627             ;call GETBUFFER
29628 00004D50 E8D61B call GETBUFFER ; * ; pre-read
29629             SKIP_GETB:
29630             POP CX
29631             POP SI
29632             POP AX
29633             POP BX
29634 00004D57 7305 JNC short SET_THE_BUF
29635 00004D59 5F POP DI ; Start of next element
29636 00004D5A 89FE MOV SI,DI ; Point with SI
29637 00004D5C EB21 JMP SHORT _BADPATH
29638
29639             SET_THE_BUF:
29640 00004D5E E81DF2 call SET_BUF_AS_DIR
29641 00004D61 8B3E[E205] MOV DI,[CURBUF]
29642 00004D65 01FE ADD SI,DI ; Get the offsets back
29643 00004D67 01FB ADD BX,DI
29644             ; DOS 3.3 FastOpen 6/12/86 F.C.
29645             FAST_OPEN_SKIP:
29646 00004D69 5F POP DI ; Start of next element
29647 00004D6A E8C202 CALL InsertPath ; insert dir entry info
29648             ; DOS 3.3 FastOpen 6/12/86 F.C.
29649 00004D6D 8A05 MOV AL,[DI]
29650 00004D6F 08C0 OR AL,AL
29651 00004D71 74C0 JZ short _SETRET ; At end
29652 00004D73 47 INC DI ; Skip over "/"
29653 00004D74 89FE MOV SI,DI ; Point with SI
29654 00004D76 E87110 call PATHCHRCMP
29655 00004D79 7503 JNZ short find_bad_name ; oops
29656 00004D7B E9A1FE JMP FINDPATH ; Next element
29657
29658             find_bad_name:
29659 00004D7E 4E DEC SI ; Undo above INC to get failure point
29660             _BADPATH:
29661 00004D7F 30C9 XOR CL,CL ; Set zero
29662 00004D81 EB28 JMP SHORT BADPRET
29663
29664             FILEINPATH:
29665 00004D83 5F POP DI ; Start of next element
29666
29667             ;hkn; SS is DOSDATA
29668             ;context DS ; Got to from one place with DS gone
29669 00004D84 16 push ss
29670 00004D85 1F pop ds
29671
29672             ; DOS 3.3 FastOpen
29673             ;test byte [FastOpenFlg],1
29674 00004D86 F606[4611]01 TEST byte [FastOpenFlg],FastOpen_Set ; do this here is we don't want to
29675 00004D8B 740B JZ short NO_FAST ; device info to fastopen
29676             ;test byte [FastOpenFlg],2
29677 00004D8D F606[4611]02 TEST byte [FastOpenFlg],Lookup_Success
29678 00004D92 7404 JZ short NO_FAST
29679 00004D94 8B3E[9C0D] MOV DI,[Next_Element_Start] ; This takes care of one time lookup
29680             ; success
29681             NO_FAST:
29682             ; DOS 3.3 FastOpen
29683 00004D98 8A05 MOV AL,[DI]
29684 00004D9A 08C0 OR AL,AL
29685             ;JZ short INCRET
29686             ;MOV SI,DI ; Path too long
29687             ;JMP SHORT BADPRET
29688             ; 27/06/2024
29689 00004D9C 750B jnz short BADPRET_X
29690
29691             INCRET:
29692             ; DOS 3.3 FasOpen 6/12/86 F.C.
29693             CALL InsertPath ; insert dir entry info
29694 00004D9E E88E02
29695
29696             ; DOS 3.3 FasOpen 6/12/86 F.C.
29697 00004DA1 FEC0 INC AL ; Reset zero
29698             ; 16/12/2022
29699             ;_SETRET:
29700 00004DA3 C3 retn
29701
29702             BADPATHPOP:
29703 00004DA4 5E POP SI ; Start of next element
29704 00004DA5 8A04 MOV AL,[SI]
29705             ; 27/06/2024
29706             ;MOV SI,DI ; Start of bad element
29707 00004DA7 08C0 OR AL,AL ; zero if bad element is last, non-zero if path too long
29708             BADPRET_X: ; 27/06/2024
29709 00004DA9 89FE mov si,di
29710             BADPRET:
29711 00004DAB A0[6D05] MOV AL,[SATTRIB]
29712 00004DAE A2[6B05] MOV [ATTRIB],AL ; Make sure return correct
29713 00004DB1 F9 STC
29714 00004DB2 C3 retn
29715
29716             ;Break <STARTSRCH -- INITIATE DIRECTORY SEARCH>
29717             ;-----
29718             ;
29719             ; Procedure Name : STARTSRCH
29720             ;
29721             ; Inputs:
29722             ; [THISDPB] Set
29723             ; Function:
29724             ; Set up a search for GETENTRY and NEXTENTRY
29725             ; Outputs:
29726             ; ES:BP = Drive parameters
29727             ; Sets up LASTENT, ENTFREE=ENTLAST=-1, VOLID=0
29728             ; Destroys ES,BP,AX
29729             ;-----
29730
29731             STARTSRCH:
29732 00004DB3 C42E[8A05] LES BP,[THISDPB]
29733 00004DB7 31C0 XOR AX,AX
29734 00004DB9 A3[4803] MOV [LASTENT],AX
29735 00004DBC A2[7B05] MOV [VOLID],AL ; No volume ID found
29736 00004DBF 48 DEC AX
29737 00004DC0 A3[D805] MOV [ENTFREE],AX
29738 00004DC3 A3[DA05] MOV [ENTLAST],AX
29739 00004DC6 C3 retn
29740
29741             ;BREAK <MatchAttributes - the final check for attribute matching>
29742             ;-----
29743             ; Procedure Name : MatchAttributes
29744             ;
29745             ; Input: [Attrib] = attribute to search for

```



```

29746 ; CH = found attribute
29747 ; Output: JZ <match>
29748 ; JNZ <nomatch>
29749 ; Registers modified: noneski
29750 ;-----
29751
29752 MatchAttributes:
29753     PUSH    AX
29754
29755 ;hkn; SS override
29756     MOV     AL,[ss:ATTRIB] ; AL <- SearchSet
29757     NOT      AL ; AL <- SearchSet'
29758     AND      AL,CH ; AL <- SearchSet' and FoundSet
29759     ;and     al,16h
29760     AND      AL,attr_all ; AL <- SearchSet' and FoundSet and Important
29761 ;
29762 ; the result is non-zero if an attribute is not in the search set
29763 ; and in the found set and in the important set. This means that we do not
29764 ; have a match. Do a JNZ <nomatch> or JZ <match>
29765 ;
29766     POP      AX
29767     retn
29768
29769 ; 19/05/2019 - Retro DOS v4.0
29770 ; DOSCODE:8148h (MSDOS 6.21, MSDOS.SYS)
29771
29772 ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
29773 ; DOSCODE:810Dh (MSDOS 5.0, MSDOS.SYS)
29774
29775 ; 16/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
29776 ; DOSCODE:8E74h (PCDOS 7.1, IBMDOS.COM)
29777
29778 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:8148h)
29779 ; (Windows ME IO.SYS - BIOSCODE:843Ch)
29780
29781 ;Break <DevName - Look for name of device>
29782 ;-----
29783 ;
29784 ; Procedure Name : DevName
29785 ;
29786 ; Inputs:
29787 ; DS,ES:DOSDATA
29788 ; Filename in NAME1
29789 ; ATTRIB set so that we can error out if looking for Volume IDs
29790 ; Function:
29791 ; Determine if file is in list of I/O drivers
29792 ; Outputs:
29793 ; Carry set if not a device
29794 ; ELSE
29795 ; Zero flag set
29796 ; BH = Bit 7,6 = 1, bit 5 = 0 (cooked mode)
29797 ; bits 0-4 set from low byte of attribute word
29798 ; DEVPT = DWORD pointer to Device header of device
29799 ; BX destroyed, others preserved
29800 ;-----
29801
29802 DEVNAME:
29803 ; 28/07/2018 - Retro DOS v3.0
29804 ; IBMDOS.COM (MSDOS 3.3) - Offset 49FBh
29805
29806     PUSH    SI
29807     PUSH    DI
29808     PUSH    CX
29809     PUSH    AX
29810
29811 ; E5 special code
29812     PUSH    WORD [NAME1]
29813     CMP     byte [NAME1],5
29814     JNZ     short NOKTR
29815     MOV     byte [NAME1],0E5h
29816
29817 NOKTR:
29818 ;test     byte [ATTRIB],8
29819 ;TEST     byte [ATTRIB],attr_volume_id ; If looking for VOL id don't find devs
29820     JNZ     short RET31
29821
29822 ;hkn; NULDEV is in DOSDATA
29823     MOV     SI,NULDEV
29824
29825 LOOKIO:
29826 ; 21/11/2022
29827 ;test     byte [SI+SYSDEV.ATT+1],80h
29828 ; 17/12/2022
29829 ;test     byte [si+5],80h
29830 ;test     byte [SI+SYSDEV.ATT+1],(DEVTP>>8)
29831 ;;test    word [si+4],8000h
29832 ;TEST     word [SI+SYSDEV.ATT],DEVTP
29833     JZ      short SKIPDEV ; Skip block devices (NET and LOCAL)
29834     MOV     AX,SI
29835     ;add     si,10
29836     ADD     SI,SYSDEV.NAME
29837
29838 ;hkn; NAME1 is in DOSDATA
29839     MOV     DI,NAME1
29840     MOV     CX,4 ; All devices are 8 letters
29841     REPE    CMPSW ; Check for name in list
29842     ;MOV     SI,AX
29843     ; 27/06/2024
29844     xchg    ax,si
29845     JZ      short IOCHK ; Found it?
29846
29847 SKIPDEV:
29848     LDS     SI,[SI] ; Get address of next device
29849     CMP     SI,-1 ; At end of list?
29850     JNZ     short LOOKIO
29851
29852 RET31:
29853     STC ; Not found
29854
29855 RETNV:
29856     MOV     CX,SS
29857     MOV     DS,CX
29858
29859     POP     WORD [NAME1]
29860     POP     AX
29861     POP     CX
29862     POP     DI
29863     POP     SI
29864     RETN
29865
29866 IOCHK:
29867 ;hkn; SS override for DEVPT
29868     MOV     [SS:DEVPT+2],DS ; Save pointer to device
29869     ;mov     bh,[si+4]
29870     MOV     BH,[SI+SYSDEV.ATT]
29871     OR      BH,0C0h
29872     and     bh,0DFh
29873     ;AND     BH,~(020h) ; Clears Carry

```

```

29870 00004E2B 368936[9A05]      MOV     [SS:DEVPT],SI
29871 00004E30 EBDE                JMP     short RETNV
29872
29873 ;BREAK <Build_device_ent - Make a Directory entry>
29874 ;-----
29875 ; Procedure Name : Build_device_ent
29876 ;
29877 ; Inputs:
29878 ; [NAME1] has name
29879 ; BH is attribute field (supplied by DEVNAME)
29880 ; [DEVPT] points to device header (supplied by DEVNAME)
29881 ; Function:
29882 ; Build a directory entry for a device at DEVFCB
29883 ; Outputs:
29884 ; BX points to DEVFCB
29885 ; SI points to dir_first field
29886 ; AH = input BH
29887 ; AL = 0
29888 ; dir_first = DEVPT
29889 ; Zero Set, Carry Clear
29890 ; DS,ES,BP preserved, others destroyed
29891 ;-----
29892
29893 Build_device_ent:
29894 00004E32 B82020      MOV     AX," " ; 2020h
29895
29896 ;hkn; DEVFCB is in DOSDATA
29897 00004E35 BF[5305]    MOV     DI,DEVFCB+8 ; Point to extent field
29898
29899 ; Fill dir_ext BUGBUG - use ERRNZs for this stuff!
29900
29901 00004E38 AB          STOSW
29902 00004E39 AA          STOSB ; Blank out extent field
29903
29904 00004E3A B040      ;mov    al,40h
29905                      MOV     AL,attr_device
29906
29907 ; Fill Dir_attr
29908 00004E3C AA          STOSB ; Set attribute field
29909 00004E3D 31C0      XOR     AX,AX
29910 00004E3F B90A00      MOV     CX,10
29911
29912 ; Fill dir_pad
29913
29914 00004E42 F3AB      REP     STOSW ; Fill rest with zeros
29915 00004E44 E819BD      call    DATE16
29916
29917 ;hkn; DEVFCB is in DOSDATA
29918 00004E47 BF[6105]    MOV     DI,DEVFCB+dir_entry.dir_time ; 09/08/2018
29919 00004E4A 92          XCHG     AX,DX
29920
29921 ; Fill dir_time
29922
29923 00004E4B AB          STOSW
29924 00004E4C 92          XCHG     AX,DX
29925
29926 ; Fill dir_date
29927
29928 00004E4D AB          STOSW
29929 00004E4E 89FE      MOV     SI,DI ; SI points to dir_first field
29930 00004E50 A1[9A05]    MOV     AX,[DEVPT]
29931
29932 ; Fill dir_first
29933
29934 00004E53 AB          STOSW ; Dir_first points to device
29935 00004E54 A1[9C05]    MOV     AX,[DEVPT+2]
29936
29937 ; Fill dir_size_1
29938
29939 00004E57 AB          STOSW
29940 00004E58 88FC      MOV     AH,BH ; Put device atts in AH
29941
29942 ;hkn; DEVFCB is in DOSDATA
29943 00004E5A B8[4B05]    MOV     BX,DEVFCB
29944 00004E5D 30C0      XOR     AL,AL ; Set zero, clear carry
29945 00004E5F C3          retn
29946
29947 ;Break <ValidateCDS - given a CDS, validate the media and the current directory>
29948 ;-----
29949 ;
29950 ; ValidateCDS - Get current CDS. Splice it. Call FatReadCDS to check
29951 ; media. If media has been changed, do DOS_Chdir to validate path.
29952 ; If invalid, reset original CDS to root.
29953 ;
29954 ; Inputs: ThisCDS points to CDS of interest
29955 ; SS:DI points to temp buffer
29956 ; Outputs: The current directory string is validated on the appropriate
29957 ; drive
29958 ; ThisDPB changed
29959 ; ES:DI point to CDS
29960 ; Carry set if error (currently user FAILED to I 24)
29961 ; Registers modified: all
29962 ;-----
29963
29964 ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
29965 ; DOSCODE:819Bh (MSDOS 5.0, MSDOS.SYS)
29966
29967 ; 16/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
29968 ; DOSCODE:8F01h (PCDOS 7.1, IBMDOS.COM)
29969
29970 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:81D6h)
29971 ; (Windows ME IO.SYS - BIOSCODE:84CBh)
29972
29973 validateCDS:
29974 ; 19/05/2019 - Retro DOS v4.0
29975 ; 28/07/2018 - Retro DOS v3.0
29976
29977 %define Temp [bp-2] ; word
29978 %define SaveCDS [bp-6] ; dword
29979 %define SaveCDSL [bp-6] ; word
29980 %define SaveCDSH [bp-4] ; word
29981
29982 ;Enter
29983 00004E60 55          push    bp
29984 00004E61 89E5      mov     bp,sp
29985 00004E63 83EC06      sub     sp,6
29986
29987 00004E66 897EFE      MOV     Temp,DI
29988
29989 ;hkn; SS override
29990 00004E69 36C536[A205]    LDS     SI,[SS:THISCDS]
29991 00004E6E 8976FA      MOV     SaveCDSL,SI
29992 00004E71 8C5EFC      MOV     SaveCDSH,DS
29993                      ;EnterCrit critDisk

```

```

29994 00004E74 E86BCA      call    ECritDisk
29995                      ; 21/11/2022
29996                      ;test byte [SI+curdir.flags+1],80h
29997                      ;test word [si+67],8000h
29998                      ; 17/12/2022
29999                      ;test byte [SI+68],80h
30000 00004E77 F6444480    test    byte [SI+curdir.flags+1],(curdir_isnet>>8)
30001                      ;TEST word [SI+curdir.flags],curdir_isnet ; Clears carry
30002 00004E7B 7403        JZ      short _DoSplice
30003 00004E7D E9A300      JMP     FatFail
30004                      _DoSplice:
30005 00004E80 30D2        XOR     DL,DL
30006 00004E82 368616[4C03] XCHG    DL,[SS:NoSetDir]
30007
30008                      ;hkn; SS is DOSDATA
30009                      ;Context ES
30010 00004E87 16          push    ss
30011 00004E88 07          pop     es
30012                      ;Invoke FStrcpy
30013 00004E89 E834C9      call    FStrCpy
30014 00004E8C 8B76FE      MOV     SI,Temp
30015
30016                      ;hkn; SS is DOSDATA
30017                      ;Context DS
30018 00004E8F 16          push    ss
30019 00004E90 1F          pop     ds
30020                      ;Invoke Splice
30021 00004E91 E84A30      call    Splice
30022
30023                      ;hkn; SS is DOSDATA
30024                      ;Context DS                      ; FatReadCDS (ThisCDS);
30025 00004E94 16          push    ss
30026 00004E95 1F          pop     ds
30027 00004E96 8816[4C03] MOV     [NoSetDir],DL
30028 00004E9A C43E[A205] LES     DI,[THISCDS]
30029                      ;SAVE <BP>
30030 00004E9E 55          push    bp
30031                      ;Invoke FATREAD_CDS
30032 00004E9F E8B916      call    FATREAD_CDS
30033                      ;RESTORE <BP>
30034 00004EA2 5D          pop     bp
30035 00004EA3 727E        JC      short FatFail
30036
30037 00004EA5 C536[A205]    LDS     SI,[THISCDS]                      ; if (ThisCDS->ID == -1) {
30038
30039                      ;;;
30040                      ; 16/02/2024 (PCDOS 7.1 IBMDOS.COM)
30041                      ;cmp word [si+48h],0FFFFh
30042 00004EA9 837C48FF      cmp     word [si+curdir.ID+2],-1
30043 00004EAD 7566          jnz     short RestoreCDS
30044                      ;;;
30045
30046                      ;cmp word [si+73],-1
30047 00004EAF 837C49FF      cmp     word [SI+curdir.ID],-1
30048 00004EB3 7560          JNZ     short RestoreCDS
30049
30050                      ;hkn; SS is DOSDATA
30051                      ;Context ES
30052 00004EB5 16          push    ss
30053 00004EB6 07          pop     es
30054
30055                      ;hkn; SS override
30056                      ;SAVE <wfp_start>                      ; t = wfp_start;
30057 00004EB7 36FF36[B205] push    word [SS:WFP_START]
30058                      ;cmp si,[bp-6]
30059 00004EBC 3B76FA        CMP     SI,SaveCDSL                      ; if not spliced
30060 00004EBF 750B          JNZ     short DoChdir
30061                      ;mov di,[bp-2]
30062 00004EC1 8B7EFE        MOV     DI,Temp
30063
30064                      ;hkn; SS override
30065 00004EC4 36893E[B205] MOV     [SS:WFP_START],DI                      ; wfp_start = d;
30066                      ;Invoke FStrCpy                      ; strcpy (d, ThisCDS->Text);
30067 00004EC9 E8F4C8      call    FStrCpy
30068                      DoChdir:
30069                      ;hkn; SS is DOSDATA
30070                      ;Context DS
30071 00004ECC 16          push    ss
30072 00004ECD 1F          pop     ds
30073                      ;SAVE <<WORD PTR SATtrib>,BP> ; c = DOSChDir ();
30074 00004ECE FF36[6D05]    push    word [SATTRIB]
30075 00004ED2 55          push    bp
30076                      ;Invoke DOS_ChDir
30077 00004ED3 E85CEB      call    DOS_CHDIR
30078                      ;RESTORE <BP,BX,wfp_start>                      ; wfp_start = t;
30079 00004ED6 5D          pop     bp
30080 00004ED7 5B          pop     bx
30081 00004ED8 8F06[B205]    pop     word [WFP_START]
30082 00004EDC 881E[6D05]    MOV     [SATTRIB],BL
30083                      ;;;
30084                      ; 16/02/2024 (PCDOS 7.1 IBMDOS.COM)
30085 00004EE0 8B3E[DC0A]    mov     di,[DIRSTART_HW]
30086                      ;;;
30087 00004EE4 C576FA        LDS     SI,SaveCDS
30088 00004EE7 7311          JNC     short SetCluster                      ; if (c == -1) {
30089
30090                      ;hkn; SS override for THISCDS
30091 00004EE9 368936[A205] MOV     [SS:THISCDS],SI                      ; ThisCDS = TmpCDS;
30092 00004EEE 368C1E[A405] MOV     [SS:THISCDS+2],DS
30093 00004EF3 31C9          XOR     CX,CX                      ; TmpCDS->text[3] = c = 0;
30094                      ;;;
30095                      ; 16/02/2024 (PCDOS 7.1 IBMDOS.COM)
30096 00004EF5 31FF          xor     di,di
30097                      ;;;
30098 00004EF7 884C03      MOV     [SI+3],CL                      ; }
30099                      SetCluster:
30100                      ; 16/02/2024
30101                      ;mov word [si+73],0FFFFh
30102                      ;MOV word [SI+curdir.ID],-1; TmpCDS->ID = -1;
30103                      ;
30104 00004EFA 36C536[A205]    LDS     SI,[SS:THISCDS]                      ; ThisCDS->ID = c;
30105                      ; 21/11/2022
30106                      ;test byte [si+curdir.flags+1],20h
30107                      ; 19/05/2019
30108                      ; MSDOS 6.0
30109                      ; 17/12/2022
30110                      ;test byte [si+68],20h
30111 00004EFF F6444420    test    byte [SI+curdir.flags+1],(curdir_splice>>8)
30112                      ;test word [si+67],2000h
30113                      ;TEST word [SI+curdir.flags],curdir_splice ;AN000;MS. for Join and Subst
30114 00004F03 7405        JZ      short _setdirclus                      ;AN000;MS.
30115 00004F05 B9FFFF      MOV     CX,-1 ; 0FFFFh                      ;AN000;MS.
30116                      ;;;
30117                      ; 16/02/2024 (PCDOS 7.1 IBMDOS.COM)

```

```

30118 00004F08 89CF      mov     di,cx
30119                ;;;
30120 _setdirclus:        ;;;
30121                ; 16/02/2024 (PCDOS 7.1 IBMDOS.COM)
30122                mov     [ss:DIRSTART_HW],di
30123 00004F0A 36893E[DC0A]  ;mov     [si+4Bh],di
30124                ;mov     [si+curdir.ID+2],di
30125 00004F0F 897C4B      ;;;
30126                ;mov     [si+73],cx
30127                MOV     [SI+curdir.ID],CX      ;      }
30128 00004F12 894C49      RestoreCDS:
30129                LES     DI,SaveCDS
30130 00004F15 C47EFA      MOV     [SS:THISCDS],DI
30131 00004F18 36893E[A205]  MOV     [SS:THISCDS+2],ES
30132 00004F1D 368C06[A405]  CLC
30133 00004F22 F8          FatFail:
30134                ;LeaveCrit critDisk
30135                call    LCritDisk
30136 00004F23 E8E9C9      ;les     di,[bp-6]
30137                LES     DI,SaveCDS
30138                ;Leave
30139 00004F26 C47EFA      mov     sp,bp
30140                pop     bp
30141 00004F29 89EC      retn
30142 00004F2B 5D
30143 00004F2C C3
30144
30145 ; 28/07/2018 - Retro DOS v3.0
30146 ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 43BDh
30147
30148 ;Break      <CheckThisDevice - Check for being a device>
30149
30150 -----
30151 ;
30152 ; CheckThisDevice - Examine the area at DS:SI to see if there is a valid
30153 ; device specified. We will return carry if there is a device present.
30154 ; The forms of devices we will recognize are:
30155 ;
30156 ; [path]device
30157 ;
30158 ; Note that the drive letter has *already* been removed. All other forms
30159 ; are not considered to be devices. If such a device is found we change
30160 ; the source pointer to point to the device component.
30161 ;
30162 ; Inputs: ES is DOSDATA
30163 ;         DS:SI contains name
30164 ; Outputs:      ES is DOSDATA
30165 ;             DS:SI point to name or device
30166 ;             Carry flag set if device was found
30167 ;             Carry flag reset otherwise
30168 ; Registers Modified: all except ES:DI, DS
30169 -----
30170
30171 CheckThisDevice:
30172     push    di
30173     push    si
30174     MOV     DI,SI
30175
30176 ; Check for presence of \dev\ (Dam multiplan!)
30177
30178     MOV     AL,[SI]
30179     call    PATHCHRCMP      ; is it a path char?
30180     JNZ     short ParseDev   ; no, go attempt to parse device
30181     INC     SI               ; simulate LODSB
30182
30183 ; We have the leading path separator. Look for DEV part.
30184
30185     LODSW
30186     OR      AX,2020h
30187     cmp     ax,"de"
30188     ;CMP    AX,"e"<< 8 + "d"
30189     JNZ     short NotDevice   ; not "de", assume not device
30190     LODSB
30191     OR      AL,20h
30192     CMP     AL,"v"           ; Not "v", assume not device
30193     JNZ     short NotDevice
30194     LODSB
30195     call    PATHCHRCMP      ; do we have the last path separator?
30196     JNZ     short NotDevice   ; no. go for it.
30197
30198 ; DS:SI now points to a potential drive. Preserve them as NameTrans advances
30199 ; SI and DevName may destroy DS.
30200
30201 ParseDev:
30202     push    ds
30203     push    si               ; preserve the source pointer
30204     call    NameTrans        ; advance DS:SI
30205     CMP     BYTE [SI],0      ; parse entire string?
30206     STC     ; simulate a Carry return from DevName
30207     JNZ     short SkipSearch ; no parse. simulate a file return.
30208
30209 ;hkn; SS is DOSDATA
30210     push    ss
30211     pop     ds
30212
30213 ; M026 - start - fix ported from ROMDOS2 for bug # 2849
30214 ;
30215 ; SR;
30216 ; We have to set Attrib before invoking DevName. Otherwise, the value from
30217 ; a previous DOS call is used and DevName thinks it is not a device if the
30218 ; old call set the volume attribute bit.
30219     mov     al,[SATTRIB]
30220     mov     [ATTRIB],al      ;set Attrib for DevName
30221
30222 ; M026 - end
30223
30224     call    DEVNAME
30225
30226 SkipSearch:
30227     pop     si
30228     pop     ds
30229
30230 ; SI points to the beginning of the potential device. If we have a device
30231 ; then we do not change SI. If we have a file, then we reset SI back to the
30232 ; original value. At this point Carry set indicates FILE.
30233
30234 CheckReturn:
30235     pop     di               ; get original SI
30236     JNC     short Check_Done ; if device then do not reset pointer
30237     MOV     SI,DI
30238
30239 Check_Done:
30240     pop     di
30241     CMC     ; invert carry. Carry => device
30242     retn

```

```

30242 NotDevice:
30243     STC
30244     JMP     short CheckReturn
30245
30246 ;BREAK <LookupPath - call fastopen to get dir entry info>
30247 -----
30248 ;
30249 ; Procedure Name : LookupPath
30250 ;
30251 ; Output DS:SI -> path name,
30252 ; ES:DI -> dir entry info buffer
30253 ; ES:CX -> extended dir info buffer
30254 ;
30255 ; carry flag clear : tables pointed by ES:DI and ES:CX are filled by
30256 ; FastOpen, DS:SI points to char just one after
30257 ; the last char of path name which is fully or
30258 ; partially found in FastOpen
30259 ; carry flag set : FastOpen not in memory or path name not found
30260 -----
30261 ;
30262 ; 21/02/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
30263 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:9015h
30264 ;
30265 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:82D9h)
30266
30267 LookupPath:
30268 ; PUSH AX
30269
30270 ;hkn; SS override
30271 ;test byte [ss:FastOpenFlg],1
30272 TEST byte [ss:FastOpenFlg],FastOpen_Set ; flg is set in DOSOPEN
30273 JNZ short FASTINST ; and this routine is
30274 NOLOOK:
30275 JMP NOLOOKUP ; executed once
30276
30277 ; 21/02/2024
30278 NOTFOUND:
30279 CMP AX,-1 ; not in memory ?
30280 JNZ short Partial_Success ; yes, in memory
30281 MOV byte [SS:FastOpenFlg],0 ; no more fastopen
30282 Partial_Success:
30283 ;and byte [SS:FastOpenFlg],0FBh
30284 AND byte [SS:FastOpenFlg],Special_Fill_Reset
30285 NOLOOKUP:
30286 ; POP AX
30287 STC
30288 RETN
30289
30290 FASTINST:
30291 ;hkn; SS override
30292 ;test byte [ss:FastOpenFlg],8
30293 TEST byte [ss:FastOpenFlg],No_Lookup ; no more lookup?
30294 JNZ short NOLOOK ; yes
30295 MOV BX,FastOpenTable ; get fastopen related tab
30296
30297 ;hkn; SS override
30298 MOV SI,[SS:WFP_START] ; si points to path name
30299 MOV DI,Dir_Info_Buff
30300 MOV CX,FastOpen_Ext_Info
30301 MOV AL,FONC_Look_up ; al = 1
30302 PUSH DS
30303 POP ES
30304
30305 ;hkn; SS override
30306 ;call far [bx+2]
30307 CALL far [BX+fastopen_entry.name_caching] ;call fastopen
30308 JC short NOTFOUND ; fastopen not in memory
30309 LEA BX,[SI-2]
30310
30311 ;hkn; SS override
30312 CMP BX,[SS:WFP_START] ; path found ?
30313 JZ short NOTFOUND ; no
30314
30315 ; 19/05/2019 - Retro DOS v4.0
30316 ; MSDOS 6.0
30317 CMP BYTE [SI],0 ; * ; 21/02/2024 ; fully or partially found
30318 JNZ short parfnd ;AN000;FO.
30319 PUSH CX ;AN000;FO.; partiallyfound
30320 ;AN000;FO.; is attribute matched ?
30321
30322 ;hkn; SS override for attrib/sattrib
30323 MOV CL,[ss:ATTRIB] ;AN000;FO.;
30324 MOV CH,[ss:SATTRIB] ;AN000;FO.; attrib=sattrib
30325 MOV [ss:ATTRIB],CH ;AN000;FO.;
30326 ;mov ch,[es:di+0Bh]
30327 MOV CH,[ES:DI+dir_entry.dir_attr] ;AN000;FO.;
30328 call MatchAttributes ;AN000;FO.;
30329 ;;; MOV [ss:ATTRIB],CL ;AN001;FO.; restore attrib
30330 POP CX ;AN000;FO.;
30331 JNZ short NOLOOKUP ;AN000;FO.; not matched
30332
30333 ; 21/02/2024 - Retro DOS v5.0
30334 ; PC DOS 7.1 IBMDOS.COM
30335 ;;;
30336 cmp byte [ss:NoSetDir],0
30337 jz short parfnd
30338 ;
30339 ;cmp byte [si],0 ; * ; byte [si] = 0
30340 ;jnz short parfnd
30341 ;
30342 ;test byte [es:di+0Bh],10h
30343 test byte [es:di+dir_entry.dir_attr],attr_directory
30344 jz short parfnd
30345 mov bx,cx
30346 jmp short parfnd2
30347 parfnd:
30348 mov bx,cx
30349 ;mov ax,[bx+0Bh]
30350 mov ax,[bx+FEI.dirstart]
30351 mov [ss:DIRSTART],ax
30352 ; 27/06/2024
30353 ;mov ax,[bx+7]
30354 mov ax,[bx+FEI.clusnum+2]
30355 mov [ss:CLUSNUM_HW],ax
30356 ;mov ax,[bx+5]
30357 mov ax,[bx+FEI.clusnum]
30358 mov [ss:CLUSNUM],ax
30359 parfnd2:
30360 mov [ss:NextElement_Start],si
30361 mov ax,[bx+9]
30362 mov ax,[bx+FEI.lastent]

```

```

30366 00005009 36A3[4803]      mov     [ss:LASTENT],ax
30367                      ;;;
30368
30369      ; 21/02/2024
30370      %if 0
30371      parfnd:
30372      ;hkn; SS override
30373      MOV     [SS:Next_Element_Start],SI      ; save si
30374      MOV     BX,CX
30375      ; MSDOS 6.0
30376      ;mov     ax,[bx+7]
30377      MOV     AX,[BX+FEI.lastent]              ;AN000;;FO. restore lastentry
30378      ;hkn; SS override for LASTENT, DIRSTART, CLUSNUM
30379      MOV     [SS:LASTENT],AX                  ;AN000;;FO.
30380      MOV     AX,[BX+FEI.dirstart]              ;AN001;;FO. restore dirstart
30381      MOV     [SS:DIRSTART],AX                  ;AN001;;FO.
30382      ; MSDOS 3.3 (& MSDOS 6.0)
30383      ;;mov     ax,[bx+3] ; MSDOS 3.3
30384      ;;mov     ax,[bx+5] ; MSDOS 6.0
30385      MOV     AX,[BX+FEI.clusnum]                ; restore next cluster num
30386      MOV     [SS:CLUSNUM],AX                    ;
30387      %endif
30388
30389 0000500D 06                PUSH     ES                      ; save ES
30390      ;hkn; SS override
30391      LES     BX,[SS:THISDPB]                    ; put drive id
30392      MOV     AH,[ES:BX] ; 15/08/2018
30393      ;MOV     AH,[ES:BX+DPB.DRIVE]                ; in AH for DOOPEN
30394      POP     ES                      ; pop ES
30395
30396      ;SR;
30397      ; we cannot have a root dir if we have come here. So, we zero out CurBuf to
30398      ; indicate it is not a root dir
30399      MOV     word [SS:CURBUF],0                ; indicate not root dir
30400      MOV     WORD [SS:CURBUF+2],ES              ; [curbuf+2].bx points to
30401      MOV     BX,DI                      ; start of entry
30402      ;lea     si,[di+1Ah]
30403      LEA     SI,[DI+dir_entry.dir_first]        ; [curbuf+2]:si points to
30404      ;                      ; dir_first field in the
30405      ;                      ; dir entry
30406
30407      ;hkn; SS override for FastOpenFlg
30408      ;or     byte [ss:FastOpenFlg],12h ; 29/12/2022
30409      OR      byte [SS:FastOpenFlg],Lookup_Success+Set_For_Search
30410      ; POP     AX
30411      RETN
30412
30413      ;BREAK <InsertPath - call fastopen to insert dir entry info>
30414      ;-----
30415      ; Procedure Name : InsertPath
30416      ; Input: FastOpen_Set flag set when from DOSOPEN otherwise 0
30417      ;       Lookup_Success flag set when got dir entry info from FASTOPEN
30418      ;       DS = DOSDATA
30419      ; Output: FastOpen_Ext_Info is set and path dir info is inserted
30420      ;-----
30421
30422      ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
30423      ; 21/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
30424
30425      InsertPath:
30426      PUSHF
30427 0000502F 9C                ;hkn; SS override for FastOpenFlag
30428      ;test    byte [SS:FastOpenFlg],1
30429      TEST     byte [SS:FastOpenFlg],FastOpen_Set ;only DOSOPEN can take advantage of
30430 00005030 36F606[4611]01    JZ      short GET_NEXT_ELEMENT      ; the FastOpen
30431 00005036 747C                ;test    byte [ss:FastOpenFlg],2
30432      TEST     byte [SS:FastOpenFlg],Lookup_Success ; Lookup just happened
30433 00005038 36F606[4611]02    JZ      short INSERT_DIR_INFO      ; no
30434 0000503E 740D                ;and     byte [ss:FastOpenFlg],0FDh
30435      AND      byte [SS:FastOpenFlg],Lookup_Reset ; we got dir info from fastopen so
30436 00005040 368026[4611]FD    MOV     DI,[SS:Next_Element_Start] ; no need to insert it again
30437 00005046 368B3E[9C0D]    JMP     short GET_NEXT2
30438 0000504B EB61
30439
30440      INSERT_DIR_INFO:                      ; save registers
30441      PUSH     DS
30442      PUSH     ES
30443      PUSH     BX
30444      PUSH     SI
30445      PUSH     DI
30446      PUSH     CX
30447      PUSH     AX
30448
30449      ;hkn; SS override
30450      LDS     DI,[SS:CURBUF]                    ; DS:DI -> buffer header
30451 00005059 BE[4711]          MOV     SI,FastOpen_Ext_Info
30452
30453      ; 21/02/2024
30454      %if 0
30455      ;mov     ax,[di+6]
30456      MOV     AX,[DI+BUFFINFO.buf_sector]        ; get directory sector
30457      ; MSDOS 6.0
30458      ;mov     [ss:si+1],ax
30459      MOV     [SS:SI+FEI.dirsec],AX              ;AN000; >32mb save dir sector
30460      ; 19/05/2019 - Retro DOS v4.0
30461      MOV     AX,[DI+BUFFINFO.buf_sector+2] ;AN000; >32mb
30462
30463      ;hkn; SS is DOSDATA
30464      push     ss
30465      pop      ds
30466      ; MSDOS 3.3
30467      ;;mov     [si+1],ax
30468      ;MOV     [SI+FEI.dirsec],AX
30469      ; MSDOS 6.0
30470      ;mov     [si+3],ax
30471      MOV     [SI+FEI.dirsec+2],AX              ;AN000;>32mb save high dir sector
30472      %else
30473      ;lds     ax,[di+6]
30474 0000505C C54506            lds     ax,[di+BUFFINFO.buf_sector] ; get directory sector
30475      ;mov     [ss:si+1],ax
30476      ; 27/06/2024
30477      ;mov     [ss:si+FEI.dirsec],ax
30478      ;
30479      ;mov     [ss:si+3],ax
30480      mov     [ss:si+FEI.dirsec+2],ds
30481      push     ss
30482      pop      ds
30483      ; 27/06/2024
30484      mov     [si+FEI.dirsec],ax
30485      %endif
30486
30487      ; 21/02/2024 (PCDOS 7.1 IBMDOS.COM)
30488      ;;;
30489      mov     ax,[CLUSNUM_HW]

```



```

30490      ;mov     [si+7],ax
30491 0000506B 894407      mov     [si+FEI.clusnum+2],ax
30492      ;;;
30493
30494      ; MSDOS 3.3 (& MSDOS 6.0)
30495      MOV     AX,[CLUSNUM]      ; save next cluster number
30496      ;mov     [si+5],ax ; MSDOS 6.0
30497      ;mov     [si+3],ax ; MSDOS 3.3
30498 00005071 894405      MOV     [SI+FEI.clusnum],AX
30499      ; MSDOS 6.0
30500 00005074 A1[4803]      MOV     AX,[LASTENT]      ;AN000;FO. save lastentry for search first
30501      ;mov     [si+7],ax
30502      ;mov     [si+9],ax ; PCDOS 7.1 ; 21/02/2024
30503 00005077 894409      MOV     [SI+FEI.lastent],AX ;AN000;FO.
30504 0000507A A1[C205]      MOV     AX,[DIRSTART]      ;AN001;FO. save for search first
30505      ;mov     [si+9],ax
30506      ;mov     [si+11],ax ; PCDOS 7.1 ; 21/02/2024
30507 0000507D 89440B      MOV     [SI+FEI.dirstart],AX ;AN001;FO.
30508
30509      ; MSDOS 3.3 (& MSDOS 6.0)
30510 00005080 89D8      MOV     AX,BX
30511      ;add     di,16 ; MSDOS 3.3
30512      ;add     di,20 ; MSDOS 6.0
30513      ;add     di,24 ; PCDOS 7.1 ; 21/02/2024
30514 00005082 83C718      ADD     DI,BUFINSIZ      ; DS:DI -> start of data in buffer
30515 00005085 29F8      SUB     AX,DI      ; AX=BX relative to start of sector
30516      ;mov     cl,32
30517 00005087 B120      MOV     CL,dir_entry.size
30518 00005089 F6F1      DIV     CL
30519      ;MOV     [SI+FEI.dirpos],AL ; save directory entry # in buffer
30520 0000508B 8804      mov     [si],al
30521
30522 0000508D 1E      PUSH    DS
30523 0000508E 07      POP     ES
30524
30525 0000508F 8E1E[E405]      MOV     DS,[CURBUF+2]
30526 00005093 89DF      MOV     DI,BX      ; DS:DI -> dir entry info
30527      ;cmp     word [di+1Ah],0
30528 00005095 837D1A00      CMP     word [DI+dir_entry.dir_first],0
30529      ; never insert info when file is empty
30530 00005099 740C      JZ      short SKIP_INSERT ; e.g. newly created file
30531
30532      ; 21/02/2024 - Erdogan Tan
30533      ; Note: PCDOS 7.1 IBMDOS.COM code doesn't check high word
30534      ; of the 1st cluster here
30535      ;;;
30536      ; 21/02/2024 - Retro DOS v5.0
30537      ;jnz     short dont_skip_insert
30538      ;cmp     word [di+14h],0
30539      ;cmp     word [di+dir_entry.dir_fclus_hi],0
30540      ;jz      short SKIP_INSERT
30541 ;dont_skip_insert: ; Retro DOS v5.0
30542      ;;;
30543
30544 0000509B 56      PUSH    SI      ; ES:BX -> extended info
30545 0000509C 5B      POP     BX
30546
30547      ;mov     al,2
30548 0000509D B002      MOV     AL,FONC_insert ; call fastopen insert operation
30549 0000509F BE[3C11]      MOV     SI,FastOpenTable
30550      ;call    far [es:si+2] ; call dword ptr es:[si+2] ; 29/12/2022
30551      ; 07/12/2022
30552 000050A2 26FF5C02      CALL    far [ES:SI+fastopen_entry.name_caching]
30553
30554 000050A6 F8      CLC
30555      SKIP_INSERT:
30556      POP     AX
30557 000050A8 59      POP     CX      ; restore registers
30558 000050A9 5F      POP     DI
30559 000050AA 5E      POP     SI
30560 000050AB 5B      POP     BX
30561 000050AC 07      POP     ES
30562 000050AD 1F      POP     DS
30563
30564      GET_NEXT2:
30565 000050AE 36800E[4611]08      ;or     [ss:FastOpenFlg],8
30566      OR      byte [SS:FastOpenFlg],No_Lookup
30567      ; we got dir info from fastopen so
30568 000050B4 9D      GET_NEXT_ELEMENT:
30569 000050B5 C3      POPF
30570      RETN
30571
30572      ;=====
30573      ; DEV.ASM (MSDOS 6.0, 1991)
30574      ;=====
30575      ; 17/07/2018 - Retro DOS v3.0
30576      ; 30/04/2019 - Retro DOS v4.0
30577      ; 21/02/2024 - Retro DOS v5.0
30578
30579      ;** Misc Routines to do 1-12 low level I/O and call devices
30580
30581      ; Offset 12B8h of IBMDOS.COM (MSDOS 3.3), 1987
30582
30583      ;DOSCODE:8401h (MSDOS 6.21 & 6.22, MSDOS.SYS) ; 21/02/2024
30584      ;BIOSCODE:8608h (Windows ME, IO.SYS) ; 21/02/2024
30585
30586      ;DOSCODE:9163h (PCDOS 7.1, IBMDOS.COM) ; 21/02/2024
30587
30588      ;Public DEV001S, DEV001E ; Pathgen labels
30589      ;DEV001s:
30590      ; length of packets
30591 000050B6 160E160D0D0E      LenTab: db DRDWRHL, DRDNDHL, DRDWRHL, DSTATHL, DFLSHL, DRDNDHL
30592      ; 21/02/2024
30593      ;LenTab: db 22,14,22,13,15,14 ; MSDOS 5.0-6.22 & Windows ME
30594      ;LenTab: db 22,14,22,13,13,14 ; PCDOS 7.1
30595
30596      ; Error Function
30597
30598      CmdTab:
30599      db 86h, DEVRD ; 0 input
30600      db 86h, DEVRDND ; 1 input status
30601      db 87h, DEVRT ; 2 output
30602      db 87h, DEVOST ; 3 output status
30603      db 86h, DEVIFL ; 4 input flush
30604      db 86h, DEVRDND ; 5 input status with system WAIT
30605
30606      ; Offset 12BEh of IBMDOS.COM (MSDOS 3.3), 1987
30607
30608      ;CmdTab:
30609      db 86h, 4
30610      db 86h, 5
30611      db 87h, 8
30612      db 87h, 10
30613      db 86h, 7
30614      db 86h, 5

```

```

30614
30615 ;DEV001E:
30616
30617 ; 30/04/2019 - Retro DOS v4.0
30618 ; DOSCODE:8413h (MSDOS 6.21, MSDOS.SYS)
30619
30620 ;Break <IOFUNC -- DO FUNCTION 1-12 I/O>
30621 -----
30622 ;
30623 ; Procedure Name : IOFUNC
30624 ;
30625 ; Inputs:
30626 ; DS:SI Points to SFT
30627 ; AH is function code
30628 ; = 0 Input
30629 ; = 1 Input Status
30630 ; = 2 Output
30631 ; = 3 Output Status
30632 ; = 4 Flush
30633 ; = 5 Input Status - System WAIT invoked for K09 if no char
30634 ; present.
30635 ; AL = character if output
30636 ; Function:
30637 ; Perform indicated I/O to device or file
30638 ; Outputs:
30639 ; AL is character if input
30640 ; If a status call
30641 ; zero set if not ready
30642 ; zero reset if ready (character in AL for input status)
30643 ; For regular files:
30644 ; Input Status
30645 ; Gets character but restores position
30646 ; Zero set on EOF
30647 ; Input
30648 ; Gets character advances position
30649 ; Returns ^Z on EOF
30650 ; Output Status
30651 ; Always ready
30652 ; AX altered, all other registers preserved
30653 -----
30654
30655 ; 21/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
30656 ; DOSCODE:83D8h (MSDOS 5.0, MSDOS.SYS)
30657
30658 ; 21/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
30659 ; DOSCODE:9175h (PCDOS 7.1, IBMDOS.COM)
30660
30661 ; DOSCODE:8413h (MSDOS 6.22, MSDOS.SYS)
30662 ; BIOSCODE:861Ah (Windows ME, IO.SYS)
30663
30664 IOFUNC:
30665 000050C8 368C16[8C03] MOV [SS:IOXAD+2],SS ; SS override for IOXAD, IOSCNT,
30666 ; DEVIobuf
30667 000050CD 36C706[8A03][BC03] MOV WORD [SS:IOXAD],DEVIobuf
30668 000050D4 36C706[8E03]0100 MOV WORD [SS:IOSCNT],1
30669 000050DB 36A3[BC03] MOV WORD [SS:DEVIobuf],AX
30670 ;test byte [si+6],80h
30671 ;TEST word [SI+SF_ENTRY.sf_flags],sf_isnet ; 8000h
30672 000050DF F6440680 test byte [SI+SF_ENTRY.sf_flags+1],(sf_isnet>>8)
30673 000050E3 7403 JZ short IOT022 ;AN000;
30674 000050E5 E9A300 JMP IOT0FILE ;AN000;
30675
30676 IOT022:
30677 ;test word [si+5],80h
30678 000050E8 F6440580 ;TEST word [SI+SF_ENTRY.sf_flags],devid_device
30679 000050EC 7503 test byte [SI+SF_ENTRY.sf_flags],devid_device
30680 000050EE E99A00 JNZ short IOT033 ;AN000;
30681 JMP IOT0FILE ;AN000;
30682 000050F1 06 IOT033:
30683 000050F2 E863B3 push es ; * (MSDOS 6.21)
30684 000050F5 8CDA call save_world
30685 000050F7 8CD3 MOV DX,DS
30686 000050F9 8EDB MOV BX,SS
30687 000050FB 8EC3 MOV DS,BX
30688 000050FD 31DB MOV ES,BX
30689 000050FF 80FC05 XOR BX,BX
30690 00005102 7502 cmp ah,5 ; system wait enabled?
30691 ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
30692 ; 16/12/2022
30693 ;or bh,04h
30694 ;or bx,0400H ; Set bit 10 in status word for driver
30695 ; It is up to device driver to carry out
30696 ; appropriate action.
30697 ; 04/07/2024
30698 00005104 B704 mov bh,4
30699 _no_sys_wait:
30700 00005106 891E[7F03] MOV [IOCALL_REQSTAT],BX
30701 0000510A 31DB XOR BX,BX
30702 0000510C 881E[8903] MOV [IOMED],BL
30703
30704 00005110 88E3 MOV BL,AH ; get function
30705 00005112 2E8AA7[B650] MOV AH,[cs:BX+LenTab]
30706 00005117 D1E3 SHL BX,1
30707 00005119 2E8B8F[BC50] MOV CX,[cs:BX+CmdTab]
30708 0000511E BB[7C03] MOV BX,IOCALL ; DOSDATA:037Ch
30709 00005121 8826[7C03] MOV [IOCALL_REQLEN],AH
30710 00005125 882E[7E03] MOV [IOCALL_REQFUNC],CH
30711
30712 00005129 8EDA MOV DS,DX
30713 0000512B E86401 CALL DEVIocall
30714 0000512E 368B3E[7F03] MOV DI,[SS:IOCALL_REQSTAT]; SS override
30715 00005133 21FF and di,di
30716 00005135 7833 js short DevErr
30717
30718 00005137 8CD0 OKDevIO:
30719 00005139 8ED8 MOV AX,SS
30720 MOV DS,AX
30721
30722 ;cmp ch,5
30723 0000513B 80FD05 CMP CH,DEVIRDND
30724 0000513E 7506 JNZ short DNODRD
30725 00005140 A0[8903] MOV AL,[IORCHR]
30726 00005143 A2[BC03] MOV [DEVIobuf],AL
30727
30728 00005146 8A26[8003] DNODRD:
30729 0000514A F6D4 MOV AH,[IOCALL_REQSTAT+1]
30730 NOT AH ; zero = busy, not zero = ready
30731 ;and ah,2
30732 AND AH,STBUI>>8
30733
30734 0000514F E8EFB2 QuickReturn:
30735 00005152 07 call restore_world ;AN000; 2/13/KK
30736 pop es ; * (MSDOS 6.21)
30737 ; SR;

```



```

30738 ; We return ax = -1 if the user failed on I24. This is the case if
30739 ; IoStatFail = -1 (set after return from the I24)
30740
30741 ; MSDOS 6.0
30742 00005153 9C pushf
30743 00005154 36A0[8300] mov al,[ss:IoStatFail] ;assume fail error
30744 00005158 98 cbw ;sign extend to word
30745
30746 ; 21/02/2024
30747 %if 0
30748 ; PCDOS 7.1 IBMDOS.COM
30749 ; cbw
30750 inc ax ; 21/02/2024 - Erdogan Tan
30751 ; this may be a BUG
30752 ; MSDOS 6.22 MSDOS.SYS & Win ME IO.SYS code is
30753 ; cbw
30754 ; cmp ax,-1
30755 ; jne short not_fail_ret
30756 ; inc byte [ss:IoStatFail]
30757 ; popf
30758 ; retn
30759 jnz short not_fail_ret
30760 dec ax
30761 jnz short not_fail_ret ; jns ?
30762 %else
30763 ;;cbw
30764 ;;cmp ax,-1 ; (MSDOS 6.22 & Windows ME)
30765 ;;jne short not_fail_ret
30766 ; 27/06/2024
30767 00005159 3CFF cmp al,0FFh ; -1
30768 0000515B 7507 jne short not_fail_ret
30769 %endif
30770
30771 0000515D 36FE06[8300] inc byte [ss:IoStatFail]
30772 00005162 9D popf
30773 00005163 C3 retn
30774
30775 not_fail_ret:
30776 00005164 36A1[BC03] mov ax,[ss:DEVIOBUF] ;ss override
30777 00005168 9D popf
30778 00005169 C3 retn
30779
30780 DevErr:
30781 0000516A 88CC MOV AH,CL
30782 0000516C E8B40E call CHARHARD
30783 0000516F 3C01 CMP AL,1
30784 00005171 7507 JNZ short NO_RETRY
30785 00005173 E8CBB2 call restore_world
30786 ; 12/05/2019
30787 00005176 07 pop es ; * (MSDOS 6.21)
30788 00005177 E94EFF JMP IOFUNC ; 10/08/2018
30789
30790 NO_RETRY:
30791 ; Know user must have wanted Ignore OR Fail. Make sure device shows ready
30792 ; ready so that DOS doesn't get caught in a status loop when user
30793 ; simply wants to ignore the error.
30794 ;
30795 ; SR; If fail wanted by user set ax to special value (ax = -1). This
30796 ; should be checked by the caller on return
30797
30798 ; SS override
30799 0000517A 368026[8003]FD and byte [SS:IOCALL_REQSTAT+1],0FDh
30800 ;AND BYTE [SS:IOCALL_REQSTAT+1],~(STBUI>>8)
30801
30802 ; SR;
30803 ; Check if user failed
30804
30805 ; MSDOS 6.0
30806 00005180 3C03 cmp al,3
30807 00005182 7505 jnz short not_fail
30808 00005184 36FE0E[8300] dec byte [ss:IoStatFail] ;set flag indicating fail on I24
30809 not_fail:
30810 00005189 EBAC JMP short OKDevIO
30811
30812 IOTOFIELD:
30813 0000518B 08E4 OR AH,AH
30814 0000518D 7421 JZ short IOIN
30815 0000518F FECC DEC AH
30816 00005191 7405 JZ short IOIST
30817 00005193 FECC DEC AH
30818 00005195 7411 JZ short IOUT
30819 IOUT_retn: ; 18/12/2022
30820 00005197 C3 retn ; NON ZERO FLAG FOR OUTPUT STATUS
30821
30822 IOIST:
30823 00005198 FF7415 ;push word [si+15h]
30824 ;push word [SI+SF_ENTRY.sf_position] ; Save position
30825 0000519B FF7417 ;push word [si+17h]
30826 0000519E E80F00 PUSH WORD [SI+SF_ENTRY.sf_position+2]
30827 CALL IOIN
30828 000051A1 8F4417 ;pop word [si+17h]
30829 POP WORD [SI+SF_ENTRY.sf_position+2] ; Restore position
30830 000051A4 8F4415 ;pop word [si+15h]
30831 000051A7 C3 POP WORD [SI+SF_ENTRY.sf_position]
30832 retn
30833 IOUT:
30834 000051A8 E82500 CALL SETXADDR
30835 000051AB E8CAEB call DOS_WRITE
30836 ;CALL RESTXADDR ; If you change this into a jmp don't
30837 000051AE EB4F ; 18/12/2022
30838 jmp RESTXADDR
30839 ;IOUT_retn:
30840 ;retn ; come crying to me when things don't
30841 ; work ARR
30842
30843 IOIN:
30844 000051B0 E81D00 CALL SETXADDR
30845 ; ; SS override for DOS34_FLAG
30846 ;OR word [SS:DOS34_FLAG],Disable_EOF_I24 ;AN000;
30847 ;or word [ss:DOS34_FLAG],40h
30848 ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
30849 ; 16/12/2022
30850 or byte [ss:DOS34_FLAG],40h
30851 CALL DOS_READ
30852 ;AND word [SS:DOS34_FLAG],NO_Disable_EOF_I24 ;AN000;
30853 ;and word [SS:DOS34_FLAG],0FFBFh
30854 ; 21/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
30855 ; 16/12/2022
30856 and byte [SS:DOS34_FLAG],0BFh ; 07/12/2022
30857 or CX,CX ; Check EOF
30858 CALL RESTXADDR
30859 ; SS override
30860 MOV AL,[ss:DEVIOBUF] ; Get byte from trans addr
30861 jnz short IOUT_retn
30862 MOV AL,1AH ; ^Z if no bytes
30863 retn

```

```

30862
30863
30864
30865 000051D0 368F06[6C03]
30866
30867 000051D5 06
30868
30869 000051D6 E87FB2
30870
30871
30872
30873
30874 000051D9 368C1E[A005]
30875
30876
30877
30878
30879
30880
30881
30882
30883
30884
30885
30886
30887
30888
30889
30890
30891
30892
30893
30894 000051DE 36C50E[2C03]
30895 000051E3 51
30896 000051E4 1E
30897 000051E5 36C50E[8A03]
30898 000051EA 368C1E[2E03]
30899 000051EF 16
30900 000051F0 1F
30901 000051F1 890E[2C03]
30902 000051F5 8936[9E05]
30903
30904 000051F9 8B0E[8E03]
30905 000051FD EB10
30906
30907
30908 000051FF 8F06[6C03]
30909 00005203 8F06[2E03]
30910 00005207 8F06[2C03]
30911
30912 0000520B E833B2
30913
30914 0000520E 07
30915
30916
30917 0000520F 36FF26[6C03]
30918
30919
30920
30921
30922
30923
30924
30925
30926
30927
30928
30929
30930
30931
30932
30933
30934
30935
30936
30937
30938
30939
30940 00005214 06
30941 00005215 E840B2
30942
30943 00005218 B00D
30944 0000521A EB06
30945
30946
30947
30948
30949
30950
30951
30952
30953
30954
30955
30956
30957
30958
30959 0000521C 06
30960 0000521D E838B2
30961
30962 00005220 B00E
30963
30964
30965
30966
30967
30968
30969
30970
30971
30972
30973
30974
30975
30976 00005222 26F6450680
30977 00005227 7564
30978 00005229 30E4
30979
30980 0000522B 26F6450580
30981
30982 00005230 26C47D07
30983 00005234 7511
30984
30985

SETXADDR:
    POP     WORD [SS:CALLSCNT]      ; SS override
                                           ; Return address
    push    es ; * (MSDOS 6.21)
    call    save_world
                                           ; SS override for DMAADD and THISSFT
    ; 24/09/2023
    ; PUSH   WORD [SS:DMAADD]        ; Save Disk trans addr
    ; PUSH   WORD [SS:DMAADD+2]
    MOV     [SS:THISSFT+2],DS
; 22/02/2024
%if 0
    push    ss
    pop     ds

    ; 24/09/2023
    push    word [DMAADD]
    push    word [DMAADD+2]

    MOV     [THISSFT],SI              ; Finish setting SFT pointer
    MOV     CX,[IOXAD+2]
    MOV     [DMAADD+2],CX
    MOV     CX,[IOXAD]
    MOV     [DMAADD],CX              ; Set byte trans addr
%else
    ; 22/02/2024
    ; PCDOS 7.1 IBMDOS.COM

    lds     cx,[ss:DMAADD]            ; Save Disk transfer address
    push    cx
    push    ds
    lds     cx,[ss:IOXAD]
    mov     [ss:DMAADD+2],ds         ; Set byte trans address
    push    ss
    pop     ds
    mov     [DMAADD],cx
    mov     [THISSFT],si
%endif
    MOV     CX,[IOSCNT]              ; ioscnt specifies length of buffer
    JMP     SHORT RESTRET            ; RETURN ADDRESS

RESTXADDR:
    POP     WORD [CALLSCNT]          ; Return address
    POP     WORD [DMAADD+2]          ; Restore Disk trans addr
    POP     WORD [DMAADD]

    call    restore_world

    pop     es ; * (MSDOS 6.21)
                                           ; SS override
RESTRET:
    JMP     WORD [SS:CALLSCNT]      ; Return address

; DOSCODE:8569h (MSDOS 6.21, MSDOS.SYS)
; 21/11/2022
; DOSCODE:852Eh (MSDOS 5.0, MSDOS.SYS)

; 22/02/2024 - Retro DOS v5.0
; DOSCODE:92CAh (PCDOS 7.1, IBMDOS.COM)

;Break <DEV_OPEN_SFT, DEV_CLOSE_SFT - OPEN or CLOSE A DEVICE>

;-----
; ** Dev_Open_SFT - Open the Device for an SFT
;
; Dev_Open_SFT issues an open call to the device associated with
; the SFT.
;
; ENTRY (ES:DI) = SFT
; EXIT  none
; USES  all
;-----

DEV_OPEN_SFT:
    push    es ; * (MSDOS 6.21)
    call    save_world
    ;mov    al,0Dh
    MOV     AL,DEVOPN
    JMP     SHORT DO_OPCLS

;-----
; Procedure Name : DEV_CLOSE_SFT
;
; Inputs:
; ES:DI Points to SFT
; Function:
; Issue a CLOSE call to the correct device
; Outputs:
; None
; ALL preserved
;-----

DEV_CLOSE_SFT:
    push    es ; * (MSDOS 6.21)
    call    save_world
    ;mov    al,0Eh
    MOV     AL,DEVCLS

    ; Main entry for device open and close. AL contains the function
    ; requested. Subtlety: if Sharing is NOT loaded then we do NOT issue
    ; open/close to block devices. This allows networks to function but
    ; does NOT hang up with bogus change-line code.

    ;entry DO_OPCLS
DO_OPCLS:
    ; Is the SFT for the net? If so, no action necessary.

    ; MSDOS 6.0
    ;test   word [es:di+5],8000h
    ;TEST   word [ES:DI+SF_ENTRY.sf_flags],sf_isnet
    test    byte [es:di+SF_ENTRY.sf_flags+1],(sf_isnet>>8)
    jnz     short OPCLS_DONE         ; NOP on net SFTs
    XOR     AH,AH                    ; Unit
    ;test   byte [es:di+5],80h
    TEST    byte [ES:DI+SF_ENTRY.sf_flags],devid_device
    ;les    di,[es:di+7]
    LES     DI,[ES:DI+SF_ENTRY.sf_devptr] ; Get DPB or device
    JNZ     short GOT_DEV_ADDR

    ; We are about to call device open/close on a block driver. If no

```

```

30986             ; sharing then just short circuit to done.
30987
30988             ; MSDOS 6.0
30989
30990 00005236 36803E[0303]01      CMP     byte [ss:fShare],1      ; SS override
30991 0000523C 764F                JBE     short OPCLS_DONE      ; AN010; /NC or no SHARE
30992                                     ; AN010; yes
30993
30994 ; 22/02/2024
30995 %if 0
30996             ; MSDOS 3.3 (& MSDOS 6.0)
30997             ;mov     ah,[es:di+1]
30998             MOV     AH,[ES:DI+DPB.UNIT]      ; (ah) = unit
30999             mov     cl,[es:di]
31000             ;MOV     CL,[ES:DI+DPB.DRIVE]    ; (cl) = drive
31001 %else
31002             ; 22/02/2024 - Retro DOS v5.0
31003             ;mov     cx,[es:di+DPB.DRIVE]
31004 00005241 88EC                mov     cx,[es:di]
31005             mov     ah,ch                    ; AH = unit
31006                                     ; CL = drive
31007 %endif
31008             ;;les     di,[es:di+12h] ; MSDOS 3.3
31009             ;les     di,[es:di+13h] ; MSDOS 6.0
31010             LES     DI,[ES:DI+DPB.DRIVER_ADDR] ; Get device
31011 GOT_DEV_ADDR:                ; ES:DI -> device
31012             ;test     word [es:di+4],800h
31013             ;TEST     word [ES:DI+SYSDEV.ATT],DEVOPCL
31014 00005247 26F6450508          test     byte [ES:DI+SYSDEV.ATT+1],(DEVOPCL>>8)
31015 0000524C 743F                JZ      short OPCLS_DONE      ; Device can't
31016 0000524E 06                  PUSH    ES
31017 0000524F 1F                  POP     DS
31018             MOV     SI,DI                    ; DS:SI -> device
31019
31020 OPCLS_RETRY:
31021             ;Context ES
31022 00005252 16                  push    ss
31023 00005253 07                  pop     es
31024 00005254 BF[5A03]          MOV     DI,DEVCALL          ; DEVCALL is in DOSDATA
31025
31026             MOV     BX,DI
31027 00005259 50                  PUSH    AX
31028             ;mov     al,13
31029 0000525A B00D                MOV     AL,DOPCLHL
31030 0000525C AA                  STOSB
31031 0000525D 58                  POP     AX                    ; Length
31032
31033 0000525E 86E0                XCHG    AH,AL
31034             ;STOSB
31035             ; 22/02/2024 (PCDOS 7.1 IBMDOS.COM)
31036 00005260 AB                  stosw
31037 00005261 86E0                XCHG    AH,AL                ; Unit, Command
31038             ;STOSB
31039             ; Command
31040 00005263 26C7050000          MOV     WORD [ES:DI],0      ; Status
31041 00005268 50                  PUSH    AX                    ; Save Unit,Command
31042             ;invoke DEVIOCALL2
31043 00005269 E82900              call     DEVIOCALL2
31044
31045             ;mov     di,[es:bx+3]
31046 0000526C 268B7F03            MOV     DI,[ES:BX+SRHEAD.REQSTAT]
31047             ;test     di,8000h
31048             ;jz      short OPCLS_DONEP
31049 00005270 21FF                and     di,di
31050 00005272 7918                jns     short OPCLS_DONEP    ; No error
31051             ; 21/11/2022
31052             ;test     word [si+4],8000h
31053             ;TEST     word [SI+SYSDEV.ATT],DEVTYP
31054             ;test     word [si+5],80h
31055 00005274 F6440580            test     byte [SI+SYSDEV.ATT+1],(DEVTYP>>8)
31056 00005278 7404                JZ      short BLKDEV
31057 0000527A B486                MOV     AH,86h              ; Read error in data, Char dev
31058 0000527C EB04                JMP     SHORT HRDERR
31059 BLKDEV:
31060             MOV     AL,CL                    ; Drive # in AL
31061 00005280 B406                MOV     AH,6                ; Read error in data, Blk dev
31062 HRDERR:
31063             ;invoke CHARHARD
31064 00005282 E89E0D              call     CHARHARD
31065 00005285 3C01                cmp     al,1
31066 00005287 7503                jne     short OPCLS_DONEP    ; IGNORE or FAIL
31067                                     ; Note that FAIL is essentially IGNORED
31068 00005289 58                  POP     AX                    ; Get back Unit, Command
31069 0000528A EBC6                JMP     short OPCLS_RETRY
31070
31071 OPCLS_DONEP:
31072             POP     AX                    ; Clean stack
31073 OPCLS_DONE:
31074 0000528D E8B1B1              call     restore_world
31075 00005290 07                  pop     es ; * (MSDOS 6.21)
31076             retn
31077
31078 ; 30/04/2019 - Retro DOS v4.0
31079 ; DOSCODE:85EAh (MSDOS 6.21, MSDOS.SYS)
31080
31081 ; 22/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
31082 ; DOSCODE:85AFh (MSDOS 5.0, MSDOS.SYS)
31083
31084 ; 22/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
31085 ; DOSCODE:9348h (PCDOS 7.1, IBMDOS.COM)
31086
31087 ;Break <DEVIOCALL, DEVIOCALL2 - CALL A DEVICE>
31088 ;-----
31089 ;** DevIoCall - Call Device
31090 ;
31091 ; ENTRY DS:SI Points to device SFT
31092 ; ES:BX Points to request data
31093 ; EXIT DS:SI -> Device driver
31094 ; USES DS:SI,AX
31095 ;-----
31096 ;** DevIoCall2 - Call Device
31097 ;
31098 ; ENTRY DS:SI Points to DPB
31099 ; ES:BX Points to request data
31100 ; EXIT DS:SI -> Device driver
31101 ; USES DS:SI,AX
31102 ;-----
31103
31104 DEVIOCALL:
31105             ;lds     si,[si+7]                ; SS override for CALLSSEC,
31106 00005292 C57407              LDS     SI,[SI+SF_ENTRY.sf_devptr] ; CALLNEWSC, HIGH_SECTOR & CALLDEVAD
31107
31108             ;entry DEVIOCALL2
31109 DEVIOCALL2:

```

```

31110 ;EnterCrit critDevice
31111 00005295 E888C6 call ECritDevice
31112
31113 ; MSDOS 6.0
31114 ;TEST word [SI+SYSDEV.ATT],DEVTYP ;AN000; >32mb block device ?
31115 ;test byte [si+5],80h
31116 00005298 F6440580 test byte [si+SYSDEV.ATT+1],(DEVTYP>>8)
31117 0000529C 7540 jnz short chardev2 ;AN000; >32mb no
31118
31119 ; 16/12/2022
31120 ; 22/11/2022
31121 0000529E 268A4702 mov al,[ES:BX+SRHEAD.REQFUNC] ; [es:bx+2]
31122 000052A2 3C04 cmp al,DEVVRD ; 4
31123 000052A4 7408 je short chkext
31124 000052A6 3C08 cmp al,DEVWRT ; 8
31125 000052A8 7404 je short chkext
31126 000052AA 3C09 cmp al,DEVWRTV ; 9
31127 000052AC 7530 jne short chardev2
31128
31129 ; 16/12/2022
31130 ; 22/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
31131 ;;cmp byte [es:bx+2],4
31132 ;CMP byte [ES:BX+SRHEAD.REQFUNC],DEVVRD ;AN000; >32mb read ?
31133 ;JZ short chkext ;AN000; >32mb yes
31134 ;;cmp byte [es:bx+2],8
31135 ;CMP byte [ES:BX+SRHEAD.REQFUNC],DEVWRT ;AN000; >32mb write ?
31136 ;JZ short chkext ;AN000; >32mb yes
31137 ;;cmp byte [es:bx+2],9
31138 ;CMP byte [ES:BX+SRHEAD.REQFUNC],DEVWRTV
31139 ; ;AN000; >32mb write/verify ?
31140 ;JNZ short chardev2 ;AN000; >32mb no
31141
31142 chkext:
31143
31144 ; 22/02/2024 - Retro DOS v5.0 (PCDOS 7.1)
31145 %if 0
31146 CALL RW_SC ;AN000;LB. use secondary cache if there
31147 JC short dev_exit ;AN000;LB. done
31148 %endif
31149
31150 ;test byte [si+4],2
31151 000052AE F6440402 TEST byte [SI+SYSDEV.ATT],EXTDRV ;AN000;>32mb extended driver?
31152 000052B2 741A JZ short chksector ;AN000;>32mb no
31153 000052B4 26800708 ADD BYTE [ES:BX],8 ;AN000;>32mb make length to 30
31154
31155 ;MOV AX,[SS:CALLSSEC] ;AN000;>32mb
31156 ;MOV word [SS:CALLSSEC],-1 ;AN000;>32mb old sector ==-1
31157 ; 22/02/2024
31158 000052B8 B8FFFF mov ax,-1 ; 0FFFFh
31159 000052BB 368706[6E03] xchg ax,[ss:CALLSSEC]
31160
31161 MOV [SS:CALLNEWSC],AX ;AN000;>32mb new sector =
31162 000052C4 36A1[0706] MOV AX,[SS:HIGH_SECTOR] ;AN000; >32mb low sector,high sector
31163 000052C8 36A3[7603] MOV [SS:CALLNEWSC+2],AX ;AN000; >32mb
31164 000052CC EB10 JMP short chardev2 ;AN000; >32mb
31165
31166 chksector: ;AN000; >32mb
31167 000052CE 36833E[0706]00 CMP word [SS:HIGH_SECTOR],0 ;AN000; >32mb if >32mb
31168 000052D4 7408 JZ short chardev2 ;AN000; >32mb then fake error
31169 ;mov word [es:bx+3],8107h
31170 000052D6 26C747030781 MOV word [ES:BX+SRHEAD.REQSTAT],STERR+STDON+error_I24_not_DOS_disk
31171 ;AN000; >32mb
31172 000052DC EB27 JMP SHORT dev_exit ;AN000; >32mb
31173
31174 chardev2: ;AN000;
31175 ; As above only DS:SI points to device header on entry, and DS:SI is
31176 ; preserved
31177
31178 ;;;
31179 ; 22/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
31180 000052DE 36FE06[B912] inc byte [ss:DEVIO_IN_PROGRESS] ; lock (deviocal1 in progress)
31181 ;;;
31182
31183 ;mov ax,[si+6]
31184 000052E3 8B4406 MOV AX,[SI+SYSDEV.STRAT]
31185 000052E6 36A3[7803] MOV [SS:CALLDEVAD],AX
31186 000052EA 368C1E[7A03] MOV [SS:CALLDEVAD+2],DS
31187 000052EF 36FF1E[7803] CALL far [SS:CALLDEVAD]
31188
31189 ;mov ax,[si+8]
31190 000052F4 8B4408 MOV AX,[SI+SYSDEV.INT]
31191 000052F7 36A3[7803] MOV [SS:CALLDEVAD],AX
31192 000052FB 36FF1E[7803] CALL far [SS:CALLDEVAD]
31193
31194 ;;;
31195 ; 22/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
31196 00005300 36FE0E[B912] dec byte [ss:DEVIO_IN_PROGRESS] ; unlock (deviocal1 completed)
31197 ;;;
31198
31199 ; 22/02/2024 - Retro DOS v5.0 (PCDOS 7.1)
31200 %if 0
31201 ; MSDOS 6.0
31202 CALL VIRREAD ;AN000;LB. move data from SC to buffer
31203 JC short chardev2 ;AN000;LB. bad sector or exceeds max sec
31204 %endif
31205
31206 dev_exit:
31207 ;LeaveCrit critDevice
31208 ;call LCritDevice
31209 ;ret
31210 ; 18/12/2022
31211 00005305 E929C6 jmp LCritDevice
31212
31213 ; DOSCODE:8669h (MSDOS 6.21, MSDOS.SYS)
31214 ; 22/11/2022
31215 ; DOSCODE:862Eh (MSDOS 5.0, MSDOS.SYS)
31216
31217 ;Break <SETREAD, SETWRITE -- SET UP HEADER BLOCK>
31218 ;-----
31219 ;
31220 ; Procedure Name : SETREAD, SETWRITE
31221 ;
31222 ; Inputs:
31223 ; DS:BX = Transfer Address
31224 ; CX = Record Count
31225 ; DX = Starting Record
31226 ; AH = Media Byte
31227 ; AL = Unit Code
31228 ; Function:
31229 ; Set up the device call header at DEVCALL
31230 ; Output:
31231 ; ES:BX Points to DEVCALL
31232 ; No other registers effected
31233 ;

```

```

31234 ;-----
31235
31236 SETREAD_XJ:
31237     ;;;
31238     ; 07/02/2024 - Retro DOS v5.0
31239     mov     bx,di
31240     jmp     short SETREAD_X
31241     ;;;
31242
31243 SETREAD_XT:
31244     ;;;
31245     ; 07/02/2024 - Retro DOS v5.0
31246     mov     bx,TIMEBUF
31247     push    bx
31248 SETREAD_XTC:
31249     mov     cx,6
31250     ;;;
31251 SETREAD_X:
31252     ;;;
31253     ; 06/02/2024 - Retro DOS v5.0
31254     xor     ax,ax
31255     ;mov     dx,ax ; 0
31256     cwd
31257     ;;;
31258
31259 ; -----
31260
31261 SETREAD:
31262     PUSH     DI
31263     PUSH     CX
31264     PUSH     AX
31265     MOV      CL,DEV RD ; mov cl,4
31266 SETCALLHEAD:
31267     MOV      AL,DRDWRHL ; mov al,16h
31268     PUSH     SS
31269     POP      ES
31270
31271     MOV      DI,DEVCALL ; DEVCALL is in DOSDATA
31272
31273     STOSB ; length
31274     POP      AX
31275     STOSB ; Unit
31276     PUSH     AX
31277     MOV      AL,CL
31278     STOSB ; Command code
31279     XOR      AX,AX
31280     STOSW ; Status
31281     ADD      DI,8 ; Skip link fields
31282     POP      AX
31283     XCHG     AH,AL
31284     STOSB ; Media byte
31285     XCHG     AL,AH
31286     PUSH     AX
31287     MOV      AX,BX
31288     STOSW
31289
31290     MOV      AX,DS
31291     STOSW ; Transfer addr
31292
31293     POP      CX ; Real AX
31294     POP      AX ; Real CX
31295     STOSW ; Count
31296
31297     XCHG     AX,DX ; AX=Real DX, DX=real CX, CX=real AX
31298     STOSW ; Start
31299     XCHG     AX,CX
31300     XCHG     DX,CX
31301     POP      DI
31302
31303     MOV      BX,DEVCALL ; DEVCALL is in DOSDATA
31304     retn
31305
31306 ;entry SETWRITE
31307 SETWRITE:
31308
31309 ; Inputs:
31310 ; DS:BX = Transfer Address
31311 ; CX = Record Count
31312 ; DX = Starting Record
31313 ; AH = Media Byte
31314 ; AL = Unit Code
31315 ; Function:
31316 ; Set up the device call header at DEVCALL
31317 ; Output:
31318 ; ES:BX Points to DEVCALL
31319 ; No other registers effected
31320
31321     PUSH     DI
31322     PUSH     CX
31323     PUSH     AX
31324     MOV      CL,DEVWRT ; mov cl,8
31325     ADD      CL,[SS:VERFLG] ; ss override
31326     JMP      SHORT SETCALLHEAD
31327
31328 ; 22/02/2024 - Retro DOS v5.0 (Modified PCDS 7.1 IBMDOS.COM)
31329 %if 0 ; SC
31330
31331 ; 30/04/2019 - Retro DOS v4.0
31332 ; DOSCODE:86A8h (MSDOS 6.21, MSDOS.SYS)
31333 ; 22/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
31334 ; DOSCODE:866Dh (MSDOS 5.0, MSDOS.SYS)
31335
31336 ;Break <RW_SC -- Read Write Secondary Cache>
31337 ;-----
31338
31339 ; Procedure Name : RW_SC
31340 ;
31341 ; Inputs:
31342 ; [SC_CACHE_COUNT]= secondary cache count
31343 ; [SC_STATUS]= SC validity status
31344 ; [SEQ_SECTOR]= last sector read
31345 ; Function:
31346 ; Read from or write through secondary cache
31347 ; Output:
31348 ; ES:BX Points to DEVCALL
31349 ; carry clear, I/O is not done
31350 ; [SC_FLAG]=1 if continuos sectors will be read
31351 ; carry set, I/O is done
31352 ;
31353 ;-----
31354
31355 RW_SC:
31356 ; ss override for all variables used.
31357

```

```

31358      CMP     word [ss:SC_CACHE_COUNT],0      ;AN000;LB. secondary cache exists?
31359      JZ       short scexit4                    ;AN000;LB. no, do nothing
31360      CMP     word [ss:CALLSCNT],1              ;AN000;LB. sector count = 1 (buffer I/O)
31361      JNZ     short scexit4                    ;AN000;LB. no, do nothing
31362      PUSH    CX                               ;AN000;LB.
31363      PUSH    DX                               ;AN000;LB. yes
31364      PUSH    DS                               ;AN000;LB. save registers
31365      PUSH    SI                               ;AN000;LB.
31366      PUSH    ES                               ;AN000;LB.
31367      PUSH    DI                               ;AN000;LB.
31368
31369      MOV     DX,[ss:CALLSSEC]                  ;AN000;LB. starting sector
31370      CMP     BYTE [ss:DEVCALL_REQFUNC],DEVRD ;AN000;LB. read ?
31371      JZ       short doread                    ;AN000;LB. yes
31372      CALL    INVALIDATE_SC                    ;AN000;LB. invalidate SC
31373      JMP     scexit2                           ;AN000;LB. back to normal
31374      scexit4:
31375      CLC                                       ;AN000;LB. I/O not done yet
31376      retn                                     ;AN000;LB.
31377      doread:
31378      CALL    SC2BUF                           ;AN000;LB. check if in SC
31379      JC       short readSC                    ;AN000;LB.
31380      MOV     word [ss:DEVCALL_REQSTAT],STDON ;AN000;LB. fake done and ok
31381      STC                                       ;AN000;LB. set carry
31382      JMP     short saveseq                   ;AN000;LB. save seq. sector #
31383      readSC:
31384      MOV     AX,[ss:HIGH_SECTOR]              ;AN000;LB. subtract sector num from
31385      MOV     CX,[ss:CALLSSEC]                ;AN000;LB. saved sequential sector
31386      SUB     CX,[ss:SEQ_SECTOR]              ;AN000;LB. number
31387      SBB     AX,[ss:SEQ_SECTOR+2]            ;AN000;LB.
31388      ; 24/09/2023
31389      ;CMP     AX,0                            ;AN000;LB. greater than 64K
31390      JNZ     short saveseq2                  ;AN000;LB. yes,save seq. sector #
31391      chklow:
31392      CMP     CX,1                            ;AN000;LB. <= 1
31393      JA       short saveseq2                  ;AN000;LB. no, not sequential
31394      MOV     word [ss:SC_STATUS],-1          ;AN000;LB. presume all SC valid
31395      MOV     AX,[ss:SC_CACHE_COUNT]          ;AN000;LB. yes, sequential
31396      MOV     [ss:CALLSCNT],AX               ;AN000;LB. read continuous sectors
31397      readsr:
31398      MOV     AX,[ss:CALLXAD+2]               ;AN000;LB. save buffer addr
31399      MOV     [ss:TEMP_VAR2],AX               ;AN000;LB. in temp vars
31400      MOV     AX,[ss:CALLXAD]                 ;AN000;LB.
31401      MOV     [ss:TEMP_VAR],AX               ;AN000;LB.
31402
31403      MOV     AX,[ss:SC_CACHE_PTR]             ;AN000;LB. use SC cache addr as
31404      MOV     [ss:CALLXAD],AX                 ;AN000;LB. transfer addr
31405      MOV     AX,[ss:SC_CACHE_PTR+2]          ;AN000;LB.
31406      MOV     [ss:CALLXAD+2],AX              ;AN000;LB.
31407      MOV     byte [ss:SC_FLAG],1            ;AN000;LB. flag it for later;
31408      MOV     AL,[ss:SC_DRIVE]               ;AN000;LB. current drive
31409      MOV     [ss:CurSC_DRIVE],AL            ;AN000;LB. set current drive
31410      MOV     AX,[ss:CALLSSEC]               ;AN000;LB. current sector
31411      MOV     [ss:CurSC_SECTOR],AX          ;AN000;LB. set current sector
31412      MOV     AX,[ss:HIGH_SECTOR]            ;AN000;LB.
31413      MOV     [ss:CurSC_SECTOR+2],AX         ;AN000;LB.
31414      saveseq2:
31415      CLC                                       ;AN000;LB. clear carry
31416      saveseq:
31417      MOV     AX,[ss:HIGH_SECTOR]             ;AN000;LB. save current sector #
31418      MOV     [ss:SEQ_SECTOR+2],AX           ;AN000;LB. for access mode ref.
31419      MOV     AX,[ss:CALLSSEC]               ;AN000;LB.
31420      MOV     [ss:SEQ_SECTOR],AX             ;AN000;LB.
31421      JMP     short scexit                    ;AN000;LB.
31422      scexit2:
31423      CLC                                       ;AN000;LB. clear carry
31424      scexit:
31425      POP     DI                               ;AN000;LB.
31426      POP     ES                               ;AN000;LB. restore registers
31427      POP     SI                               ;AN000;LB.
31428      POP     DS                               ;AN000;LB.
31429      POP     DX                               ;AN000;LB.
31430      POP     CX                               ;AN000;LB.
31431      retn                                     ;AN000;LB.
31432
31433      ;Break      <IN_SC -- check if in secondary cache>
31434      ;-----
31435      ;
31436      ; Procedure Name : IN_SC
31437      ;
31438      ; Inputs:  [SC_DRIVE]= requesting drive
31439      ;          [CURSC_DRIVE]= current SC drive
31440      ;          [CURSC_SECTOR]= starting scetor # of SC
31441      ;          [SC_CACHE_COUNT]= SC count
31442      ;          [HIGH_SECTOR]:DX= sector number
31443      ; Function:
31444      ; Check if the sector is in secondary cache
31445      ; Output:
31446      ; carry clear, in SC
31447      ; CX= the index in the secondary cache
31448      ; carry set, not in SC
31449      ;-----
31450      ;
31451      ; 22/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
31452      IN_SC:
31453      ; SS override for all variables used
31454      MOV     AL,[ss:SC_DRIVE]                 ;AN000;LB. current drive
31455      CMP     AL,[ss:CurSC_DRIVE]             ;AN000;LB. same as SC drive
31456      JNZ     short outrange2                  ;AN000;LB. no
31457      MOV     AX,[ss:HIGH_SECTOR]              ;AN000;LB. subtract sector num from
31458      MOV     CX,DX                            ;AN000;LB. secondary starting sector
31459      SUB     CX,[ss:CurSC_SECTOR]            ;AN000;LB. number
31460      SBB     AX,[ss:CurSC_SECTOR+2]          ;AN000;LB.
31461      ; 24/09/2023
31462      ;CMP     AX,0                            ;AN000;LB. greater than 64K
31463      JNZ     short outrange2                  ;AN000;LB. yes
31464      CMP     CX,[ss:SC_CACHE_COUNT]          ;AN000;LB. greater than SC count
31465      JAE     short outrange2                  ;AN000;LB. yes
31466      CLC                                       ;AN000;LB. clear carry
31467      ;JMP     short inexit                    ;AN000;LB. in SC
31468      ; 16/12/2022
31469      retn      ; 30/04/2019
31470      ; 22/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
31471      ;jmp     short inexit
31472
31473      outrange2:
31474      STC                                       ;AN000;LB. set carry
31475      inexit:
31476      retn                                     ;AN000;LB.
31477
31478      ;Break      <INVALIDATE_SC - invalide secondary cache>
31479      ;-----
31480      ;
31481

```



```

31482 ; Procedure Name : Invalidate_Sc
31483 ;
31484 ; Inputs: [SC_DRIVE]= requesting drive
31485 ;         [CURSC_DRIVE]= current SC drive
31486 ;         [CURSC_SECTOR]= starting scetor # of SC
31487 ;         [SC_CACHE_COUNT]= SC count
31488 ;         [SC_STATUS]= SC status word
31489 ;         [HIGH_SECTOR]:DX= sector number
31490 ;
31491 ; Function:
31492 ;         invalidate secondary cache if in there
31493 ; Output:
31494 ;         [SC_STATUS] is updated
31495 ;-----
31496
31497 INVALIDATE_SC:
31498 ; SS override for all variables used
31499
31500     CALL    IN_SC                ;AN000;;LB. in secondary cache
31501     JC      short outrange       ;AN000;;LB. no
31502     MOV     AX,1                 ;AN000;;LB. invalidate the sector
31503     SHL     AX,CL                ;AN000;;LB. in the secondary cache
31504     NOT     AX                   ;AN000;;LB.
31505     AND     [ss:SC_STATUS],AX    ;AN000;;LB. save the status
31506     outrange:                    ;AN000;;LB.
31507     retn                        ;AN000;;LB.
31508
31509 ; DOSCODE:87A5h (MSDOS 6.21, MSDOS.SYS)
31510 ; 22/11/2022
31511 ; DOSCODE:876Ah (MSDOS 5.0, MSDOS.SYS)
31512
31513 ;Break    <VIRREAD- virtually read data into buffer>
31514 ;-----
31515
31516 ; Procedure Name : SC_FLAG
31517 ;
31518 ; Inputs: SC_FLAG = 0, no sectors were read into SC
31519 ;         1, continuous sectors were read into SC
31520 ; Function:
31521 ;         Move data from SC to buffer
31522 ; Output:
31523 ;         carry clear, data is moved to buffer
31524 ;         carry set, bad sector or exceeds maximum sector
31525 ;         SC_FLAG =0
31526 ;         CALLSCNT=1
31527 ;         SC_STATUS= -1 if succeeded
31528 ;
31529 ;         0 if failed
31530 ;-----
31531
31532 VIRREAD:
31533 ; SS override for all variables used
31534
31535     CMP     byte [ss:SC_FLAG],0  ;AN000;;LB. from SC fill
31536     JZ      short sc2end         ;AN000;;LB. no
31537     MOV     AX,[ss:TEMP_VAR2]    ;AN000;;LB. restore buffer addr
31538     MOV     [ss:CALLXAD+2],AX    ;AN000;;LB.
31539     MOV     AX,[ss:TEMP_VAR]     ;AN000;;LB.
31540     MOV     [ss:CALLXAD],AX      ;AN000;;LB.
31541     MOV     byte [ss:SC_FLAG],0  ;AN000;;LB. reset sc_flag
31542     MOV     word [ss:CALLSCNT],1 ;AN000;;LB. one sector transferred
31543
31544     ;TEST word [ss:DEVCALL_REQSTAT],STERR ;AN000;;LB. error?
31545     test    byte [ss:DEVCALL_REQSTAT+1],(STERR>>8) ; 80h
31546     JNZ     short scerror        ;AN000;;LB. yes
31547     PUSH    DS                   ;AN000;;LB.
31548     PUSH    SI                   ;AN000;;LB.
31549     PUSH    ES                   ;AN000;;LB.
31550     PUSH    DI                   ;AN000;;LB.
31551     PUSH    DX                   ;AN000;;LB.
31552     PUSH    CX                   ;AN000;;LB.
31553     XOR     CX,CX                ;AN000;;LB. we want first sector in SC
31554     CALL    SC2BUF2              ;AN000;;LB. move data from SC to buf
31555     POP     CX                   ;AN000;;LB.
31556     POP     DX                   ;AN000;;LB.
31557     POP     DI                   ;AN000;;LB.
31558     POP     ES                   ;AN000;;LB.
31559     POP     SI                   ;AN000;;LB.
31560     POP     DS                   ;AN000;;LB.
31561     JMP     SHORT sc2end         ;AN000;;LB. return
31562
31563 scerror:
31564     MOV     word [ss:CALLSCNT],1 ;AN000;;LB. reset sector count to 1
31565     MOV     word [ss:SC_STATUS],0 ;AN000;;LB. invalidate all SC sectors
31566     MOV     byte [ss:CurSC_DRIVE],-1 ;AN000;;LB. invalidate drive
31567     STC
31568     retn                        ;AN000;;LB.
31569
31570 sc2end:
31571     CLC
31572     retn                        ;AN000;;LB. carry clear
31573
31574 ; 30/04/2019 - Retro DOS v4.0
31575 ; DOSCODE:87FDh (MSDOS 6.21, MSDOS.SYS)
31576 ; 22/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
31577 ; DOSCODE:87C2h (MSDOS 5.0, MSDOS.SYS)
31578
31579 ;Break    <SC2BUF- move data from SC to buffer>
31580 ;-----
31581
31582 ; Procedure Name : SC2BUF
31583 ;
31584 ; Inputs: [SC_STATUS] = SC validity status
31585 ;         [SC_SECTOR_SIZE] = request sector size
31586 ;         [SC_CACHE_PTR] = pointer to SC
31587 ; Function:
31588 ;         Move data from SC to buffer
31589 ; Output:
31590 ;         carry clear, in SC and data is moved
31591 ;         carry set, not in SC and data is not moved
31592 ;-----
31593
31594 SC2BUF:
31595 ; SS override for all variables used
31596
31597     CALL    IN_SC                ;AN000;;LB. in secondary cache
31598     JC      short noSC           ;AN000;;LB. no
31599     ; 24/09/2023
31600     JC      short scxit          ;AN000;;LB. check if valid sector
31601     MOV     AX,1                 ;AN000;;LB. in the secondary cache
31602     SHL     AX,CL                ;AN000;;LB.
31603     TEST    [ss:SC_STATUS],AX    ;AN000;;LB.
31604     JZ      short noSC           ;AN000;;LB. invalid
31605
31606 ;entry SC2BUF2
31607 SC2BUF2:
31608     MOV     AX,CX                ;AN000;;LB. times index with
31609     MUL     word [ss:SC_SECTOR_SIZE] ;AN000;;LB. sector size

```

```

31606         ; 24/09/2023
31607         mov     ax,[ss:SC_SECTOR_SIZE]
31608         xchg    ax,cx ; cx = [ss:SC_SECTOR_SIZE]
31609         mul     cx
31610         ADD     AX,[ss:SC_CACHE_PTR] ;AN000;LB. add SC starting addr
31611         ADC     DX,[ss:SC_CACHE_PTR+2] ;AN000;LB.
31612         MOV     DS,DX ;AN000;LB. DS:SI-> SC sector addr
31613         MOV     SI,AX ;AN000;LB.
31614         MOV     ES,[ss:CALLXAD+2] ;AN000;LB. ES:DI-> buffer addr
31615         MOV     DI,[ss:CALLXAD] ;AN000;LB.
31616         ; 24/09/2023
31617         ;MOV     CX,[ss:SC_SECTOR_SIZE] ;AN000;LB. count= sector size
31618         SHR     CX,1 ;AN000;LB. may use DWORD move for 386
31619         ;entry MOVWORDS
31620         MOVWORDS: ;AN000;
31621         CMP     byte [ss:DDMOVE],0 ;AN000;LB. 386 ?
31622         JZ      short nodd ;AN000;LB. no
31623         SHR     CX,1 ;AN000;LB. words/2
31624         DB      66H ;AN000;LB. use double word move
31625         nodd:
31626         REP     MOVSW ;AN000;LB. move to buffer
31627         CLC     ;AN000;LB. clear carry
31628         retn    ;AN000;LB. exit
31629         noSC: ;AN000;
31630         STC     ;AN000;LB. set carry
31631         sexit: ;AN000;
31632         retn    ;AN000;LB.
31633
31634         %endif ; SC
31635
31636         ;=====
31637         ; MKNODE.ASM, MSDOS 6.0, 1991
31638         ;=====
31639         ; 29/07/2018 - Retro DOS v3.0
31640         ; 19/05/2019 - Retro DOS v4.0
31641         ; 22/02/2024 - Retro DOS v5.0
31642
31643         ; TITLE MKNODE - Node maker
31644         ; NAME MKNODE
31645
31646         ** MKNODE.ASM
31647         -----
31648         Low level routines for making a new local file system node
31649         and filling in an SFT from a directory entry
31650
31651         ;
31652         ; BUILDDIR
31653         ; SETDOTENT
31654         ; MakeNode
31655         ; NEWENTRY
31656         ; FREEENT
31657         ; NEWDIR
31658         ; DOOPEN
31659         ; RENAME_MAKE
31660         ; CHECK_VIRT_OPEN
31661
31662         ; Revision history:
31663         ;
31664         ; AN000 version 4.0 Jan. 1988
31665         ; A004 PTM 3680 --- Make SFT NAME field offset same as 3.30
31666
31667         ;Break <BUILDDIR,NEWDIR -- ALLOCATE DIRECTORIES>
31668         -----
31669         ; Procedure Name : BUILDDIR,NEWDIR
31670         ;
31671         ; Inputs:
31672         ; ES:BP Points to DPB
31673         ; [THISSFT] Set if using NEWDIR entry point
31674         ; (used by ALLOCATE)
31675         ; [LASTENT] current last valid entry number in directory if no free
31676         ; entries
31677         ; [DIRSTART] Points to first cluster of dir (0 means root)
31678         ; Function:
31679         ; Grow directory if no free entries and not root
31680         ; Outputs:
31681         ; CARRY SET IF FAILURE
31682         ; ELSE
31683         ; AX entry number of new entry
31684         ; If a new dir [DIRSTART],[CLUSFAC],[CLUSNUM],[DIRSEC] set
31685         ; AX = first entry of new dir
31686         ; GETENT should be called to set [LASTENT]
31687         ;
31688         -----
31689
31690         ; 19/05/2019 - Retro DOS v4.0
31691         ; DOSCODE:8845h (MSDOS 6.21, MSDOS.SYS)
31692         ; 22/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
31693         ; DOSCODE:880Ah (MSDOS 6.21, MSDOS.SYS)
31694
31695         ; 24/09/2023 - Retro DOS v4.2 (Modified MSDOS 6.21 MSDOS.SYS)
31696         ; DOSCODE:8845h (MSDOS 6.22, MSDOS.SYS)
31697
31698         ; 23/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
31699         ; DOSCODE:940Ah (PCDOS 7.1, IBMDOS.COM)
31700
31701         ; (Windows ME IO.SYS - BIOSCODE:890Ch)
31702
31703         BUILDDIR:
31704         ; 29/07/2018 - Retro DOS v3.0
31705         ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 4E66h
31706
31707         MOV     AX,[ENTFREE]
31708         CMP     AX,-1 ; 0FFFFh
31709         ;JZ      short CHECK_IF_ROOT
31710         ;CLC
31711         ;retn
31712         ; 24/09/2023
31713         jne     short builddir_cmc_retn ; cf=1 (will be 0)
31714
31715         CHECK_IF_ROOT:
31716         ; 23/02/2024 (PCDOS 7.1 IBMDOS.COM)
31717         ;;;
31718         cmp     word [DIRSTART_HW],0
31719         jnz     short NEWDIR
31720         ;;;
31721         CMP     word [DIRSTART],0
31722         JNZ     short NEWDIR
31723         ;STC ; Can't grow root
31724         ; 24/09/2023
31725         ; [DIRSTART]=0, cf=0, zf=1 (cf will be 1 after cmc instruction)
31726         builddir_cmc_retn:
31727         ; 24/09/2023
31728         cmc     ; cf=1 <-> cf=0
31729         builddir_retn:

```



```

31730 0000536C C3                retn
31731
31732                ;entry   NEWDIR
31733 NEWDIR:
31734                ; 23/02/2024
31735                ;;;
31736 0000536D 8B1E[DC0A]          mov     bx,[DIRSTART_HW]
31737 00005371 891E[E80A]          mov     [CLUSTNUM_HW],bx
31738 00005375 09DB              or      bx,bx
31739                ;;;
31740 00005377 8B1E[C205]          MOV     BX,[DIRSTART]
31741                ;;;
31742 0000537B 7504              jnz     short NEWDIR2 ; 23/02/2024
31743                ;;;
31744 0000537D 09DB              OR      BX,BX
31745 0000537F 7405              JZ      short NULLDIR
31746 NEWDIR2:
31747 00005381 E8B708            call    GETEOF
31748 00005384 72E6              jc      short builddir_retn ; Screw up
31749 NULLDIR:
31750                ;MOV     CX,1
31751                ; 23/02/2024
31752                ;;;
31753 00005386 31C9              xor     cx,cx
31754 00005388 890E[F00A]          mov     [CCOUNT_HW],cx ; 0
31755 0000538C 41                inc     cx ; 1
31756                ;;;
31757 0000538D E84906            call    ALLOCATE
31758 00005390 72DA              jc      short builddir_retn
31759 00005392 8B16[C205]          MOV     DX,[DIRSTART]
31760                ; 23/02/2024
31761                ;;;
31762 00005396 3B16[DC0A]          cmp     dx,[DIRSTART_HW]
31763 0000539A 7519              jnz     short ADDINGDIR
31764                ;;;
31765 0000539C 09D2              OR      DX,DX
31766 0000539E 7515              JNZ     short ADDINGDIR
31767                ; 23/02/2024
31768                ;;;
31769 000053A0 FF36[E80A]          push    word [CLUSTNUM_HW]
31770 000053A4 8F06[EE0A]          pop     word [ROOTCLUST_HW]
31771                ;;;
31772 000053A8 E8B4F5            call    SETDIRSRCH
31773 000053AB 72BF              jc      short builddir_retn
31774 000053AD C706[4803]FFFF          MOV     word [LASTENT],-1
31775 000053B3 EB3F              JMP     SHORT GOTDIRREC
31776 ADDINGDIR:
31777 000053B5 53                PUSH     BX
31778                ; 23/02/2024
31779                ;;;
31780 000053B6 FF36[E80A]          push    word [CLUSTNUM_HW]
31781                ;
31782 000053BA 8B1E[DE0A]          mov     bx,[CLUSNUM_HW]
31783 000053BE 891E[E80A]          mov     [CLUSTNUM_HW],bx
31784                ;;;
31785 000053C2 8B1E[BC05]          MOV     BX,[CLUSNUM]
31786 000053C6 E8EF0E            call    ISEOF
31787                ;;;
31788 000053C9 8F06[E80A]          pop     word [CLUSTNUM_HW] ; 23/02/2024
31789                ;;;
31790 000053CD 5B                POP      BX
31791 000053CE 721D              JB      short NOTFIRSTGROW
31792                ;;; 10/17/86 update CLUSNUM in the fastopen cache
31793 000053D0 891E[BC05]          MOV     [CLUSNUM],BX
31794                ; 24/09/2023
31795                ;PUSH    CX ; (not necessary)
31796 000053D4 50                PUSH     AX
31797 000053D5 55                PUSH     BP
31798                ; 23/02/2024
31799                ;;;
31800 000053D6 A1[E80A]          mov     ax,[CLUSTNUM_HW]
31801 000053D9 A3[DE0A]          mov     [CLUSNUM_HW],ax
31802                ;;;
31803 000053DC B401              MOV     AH,1 ; CLUSNUM update
31804                ; 15/12/2022
31805 000053DE 268A5600          mov     dl,[ES:BP] ; 09/09/2018
31806                ; 22/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
31807                ;;mov    dl,[es:bp+0]
31808                ;MOV     DL,[ES:BP+DPB.DRIVE] ; drive #
31809 000053E2 8B0E[C205]          MOV     CX,[DIRSTART] ; first cluster #
31810 000053E6 89DD              MOV     BP,BX ; CLUSNUM
31811 000053E8 E87CD9            call    FastOpen_Update
31812 000053EB 5D                POP      BP
31813 000053EC 58                POP      AX
31814                ; 24/09/2023
31815                ;POP     CX
31816
31817                ;;; 10/17/86 update CLUSNUM in the fastopen cache
31818 NOTFIRSTGROW:
31819 000053ED 89DA              MOV     DX,BX
31820 000053EF 30DB              XOR     BL,BL
31821 000053F1 E8A305            call    FIGREC
31822 GOTDIRREC:
31823                ;mov     cl,[es:bp+4]
31824 000053F4 268A4E04          MOV     CL,[ES:BP+DPB.CLUSTER_MASK]
31825                ;INC     CL
31826                ; 27/06/2024
31827 000053F8 41                inc     cx
31828 000053F9 30ED              XOR     CH,CH
31829 ZERODIR:
31830 000053FB 51                PUSH     CX
31831                ; 22/09/2023
31832                ;;mov    byte [ALLOWED],18h
31833                ;MOV     byte [ALLOWED],Allowed_FAIL+Allowed_RETRY ; *
31834 000053FC B0FF              MOV     AL,0FFh
31835                ;call    GETBUFR
31836 000053FE E82A15            call    GETBUFRD ; *
31837 00005401 7302              JNC     short GET_SSIZE
31838 00005403 59                POP      CX
31839 00005404 C3                retn
31840
31841 GET_SSIZE:
31842                ;mov     cx,[es:bp+2]
31843 00005405 268B4E02          MOV     CX,[ES:BP+DPB.SECTOR_SIZE]
31844 00005409 06                PUSH     ES
31845 0000540A C43E[E205]          LES     DI,[CURBUF]
31846                ;or      byte [es:di+5],4
31847 0000540E 26804D0504          OR      byte [ES:DI+BUFFINFO.buf_flags],buf_isDIR
31848 00005413 57                PUSH     DI
31849                ;;;add    di,16 ; MSDOS 3.3
31850                ;;;add    di,20 ; MSDOS 6.0
31851                ;add     di,24 ; PCDOS 7.1 ; 23/02/2024
31852 00005414 83C718            ADD     DI,BUFINSIZ
31853 00005417 31C0              XOR     AX,AX

```

```

31854 00005419 D1E9      SHR     CX,1
31855 0000541B F3AB      REP     STOSW
31856 0000541D 7301      JNC     short EVENZ
31857 0000541F AA          STOSB
31858                      EVENZ:
31859 00005420 5F          POP     DI
31860
31861                      ; 23/02/2024
31862 %if 0
31863                      ; MSDOS 6.0
31864 TEST     byte [ES:DI+BUFFINFO.buf_flags],buf_dirty
31865                      ;LB. if already dirty ;AN000;
31866 JNZ     short yesdirty7 ;LB. don't increment dirty count ;AN000;
31867 call     INC_DIRTY_COUNT ;LB. ;AN000;
31868
31869                      ;or byte [es:di+5],40h
31870 OR       byte [ES:DI+BUFFINFO.buf_flags],buf_dirty
31871 %else
31872                      ; 23/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
31873 00005421 E89A17      call    SET_BUF_DIRTY
31874 %endif
31875
31876 yesdirty7:
31877 00005424 07          POP     ES
31878 00005425 59          POP     CX
31879
31880                      ; 19/05/2019 - Retro DOS v4.0
31881
31882                      ; MSDOS 3.3
31883                      ;INC     DX
31884
31885                      ; MSDOS 6.0
31886                      ; 24/09/2023
31887                      ;add     dx,1
31888                      ;;adc     word [HIGH_SECTOR],0
31889                      ;; 24/09/2023
31890                      ;; ax=0
31891                      ;adc     [HIGH_SECTOR],ax ; 0
31892                      ; 24/09/2023
31893 00005426 42          inc     dx
31894 00005427 7504      jnz     short loop_zerodir
31895 00005429 FF06[0706] inc     word [HIGH_SECTOR]
31896
31897 0000542D E2CC      loop_zerodir:
31898                      LOOP    ZERODIR
31899
31899 0000542F A1[4803]      MOV     AX,[LASTENT]
31900 00005432 40          INC     AX
31901                      ; 24/09/2023
31902                      ; cf=0
31903                      ;CLC
31904 00005433 C3          retn
31905
31906
31907
31908
31909
31910
31911
31912
31913
31914
31915
31916
31917
31918
31919
31920
31921
31922 00005434 AB          ;-----
31923 00005435 B90400      ; Procedure Name : SETDOTENT
31924 00005438 B82020      ;
31925 0000543B F3AB      ; set up a . or .. directory entry for a directory.
31926 0000543D AA          ;
31927                      ; Inputs: ES:DI point to the beginning of a directory entry.
31928                      ; AX contains "." or ".."
31929                      ; DX contains first cluster of entry
31930                      ;-----
31931
31932                      ; 23/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
31933
31934 SETDOTENT:
31935 ; Fill in name field
31936 STOSW
31937 MOV     CX,4
31938 MOV     AX," " ; 2020h
31939 REP     STOSW
31940 STOSB
31941
31942 ; Set up attribute
31943 ;mov     al, 10h
31944 MOV     AL,attr_directory
31945 STOSB
31946
31947 ; Initialize time and date of creation
31948 ;ADD     DI,10
31949 ; 23/04/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
31950 ;;;
31951 add     di,8
31952 mov     ax,[CLUSTERS_HW]
31953 stosw ; Set up first cluster field (hw)
31954 ;;;
31955 MOV     SI,[THISSFT]
31956 ;mov     ax,[si+0Dh]
31957 MOV     AX,[SI+SF_ENTRY.sf_time]
31958 STOSW
31959 ;mov     ax,[si+0Fh]
31960 MOV     AX,[SI+SF_ENTRY.sf_date]
31961 STOSW
31962
31963 ; Set up first cluster field (lw)
31964 MOV     AX,DX
31965 STOSW
31966
31967 ; 0 file size
31968 ;XOR     AX,AX
31969 xchg     ax,cx ; 23/02/2024
31970 STOSW
31971 STOSW
31972 retn
31973
31974 ;Break <MAKENODE -- CREATE A NEW NODE>
31975
31976
31977
31978
31979
31980
31981
31982
31983
31984
31985
31986
31987
31988
31989
31990
31991
31992
31993
31994
31995
31996
31997
31998
31999

```

```

;
; Procedure Name : MakeNode
;
; Inputs:
; AL - attribute to create
; AH = 0 if it is ok to truncate a file already by this name
; AH != 0 if truncation not allowed (preexisting file is an error)
;       (AH ignored on dirs and devices)
;
; NOTE: When making a DIR or volume ID, AH need not be set since
;       a name already existant is ALWAYS an error in these cases.
;
; [WFP_START] Points to WFP string ("d:/" must be first 3 chars, NUL
;       terminated)
; [CURR_DIR_END] Points to end of Current dir part of string
;       (= -1 if current dir not involved, else

```

```

31978 ; Points to first char after last "/" of current dir part)
31979 ; [THISCDs] Points to CDS being used
31980 ; [THISSFT] Points to an empty SFT. EXCEPT sf_mode filled in.
31981 ; Function:
31982 ; Make a new node
31983 ; Outputs:
31984 ; Sets EXTERR_LOCUS = errLOC_Disk or errLOC_Unk via GetPathNoset
31985 ; CARRY SET IF ERROR
31986 ; AX = 1 A node by this name exists and is a directory
31987 ; AX = 2 A new node could not be created
31988 ; AX = 3 A node by this name exists and is a disk file
31989 ; (AH was NZ on input)
31990 ; AX = 4 Bad Path
31991 ; SI return from GetPath maintained
31992 ; AX = 5 Attribute mismatch
31993 ; AX = 6 Sharing violation
31994 ; (INT 24 generated ALWAYS since create is always compat mode)
31995 ; AX = 7 file not found for Extended Open (not exists and fails)
31996 ; ELSE
31997 ; AX = 0 Disk Node
31998 ; AX = 3 Device Node (error in some cases)
31999 ; [DIRSTART],[DIRSEC],[CLUSFAC],[CLUSNUM] set to directory
32000 ; containing new node.
32001 ; [CURBUF+2]:BX Points to entry
32002 ; [CURBUF+2]:SI Points to entry.dir_first
32003 ; [THISSFT] is filled in
32004 ; sf_mode = unchanged.
32005 ; Attribute byte in entry is input AL
32006 ; DS preserved, others destroyed
32007 ;
32008 ;-----
32009 ;
32010 ; 19/05/2019 - Retro DOS v4.0
32011 ; DOSCODE:8925h (MSDOS 6.21, MSDOS.SYS)
32012 ;
32013 ; 22/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
32014 ; DOSCODE:88EAh (MSDOS 5.0, MSDOS.SYS)
32015 ;
32016 ; 23/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
32017 ; DOSCODE:951Ah (PCDOS 7.1, IBMDOS.COM)
32018 ;
32019 ; MakeNode:
32020 ; mov word [CREATING],0E5FFh
32021 0000545B C706[7E05]FFE5 MOV WORD [CREATING],DIRFREE*256 + 0FFh ; Creating, not DEL *.*
32022 00005461 50 PUSH AX ; Save AH value
32023 00005462 C606[4C03]00 MOV byte [NoSetDir],0
32024 00005467 A2[6D05] MOV [SATTRIB],AL
32025 0000546A E8ACF6 call GetPathNoSet
32026 0000546D 88CA MOV DL,CL ; Save CL info
32027 ; MOV CX,AX ; Device ID to CH
32028 ; 23/02/2024
32029 0000546F 91 xchg ax,cx
32030 00005470 58 POP AX ; Get back AH
32031 00005471 732D JNC short make_exists ; File existed
32032 00005473 7505 JNZ short make_err_4 ; Path bad
32033 00005475 80FA80 CMP DL,80h ; Check "CL" return from GETPATH
32034 00005478 7405 JZ short make_type ; Name simply not found, and no metas
32035 ; make_err_4:
32036 0000547A B004 MOV AL,4 ; case 1 bad path
32037 ; make_err_ret:
32038 ; XOR AH,AH
32039 ; 23/02/2024
32040 0000547C 98 cbw
32041 0000547D F9 STC
32042 ; make_retn: ; 22/11/2022
32043 0000547E C3 retn
32044 ;
32045 ; entry RENAME_MAKE ; Used by DOS_RENAME to "copy" a node
32046 RENAME_MAKE:
32047 make_type:
32048 ; Extended Open hooks
32049 ; MSDOS 6.0
32050 ; TESTB EXTOPEN_ON,EXT_OPEN_ON;FT. from extended open ;AN000;
32051 0000547F F606[F605]01 test byte [EXTOPEN_ON],EXT_OPEN_ON ; 1
32052 00005484 7411 JZ short make_type2 ;FT. no ;AN000;
32053 00005486 800E[F605]04 OR byte [EXTOPEN_ON],EXT_FILE_NOT_EXISTS ; 4
32054 ; ;FT. set for extended open ;AN000;
32055 ; TESTB EXTOPEN_FLAG,0F0H ;FT. not exists and fails ;AN000;
32056 0000548B F606[F405]F0 test byte [EXTOPEN_FLAG],0F0h
32057 00005490 7505 JNZ short make_type2 ;FT. no ;AN000;
32058 00005492 F9 STC ;FT. set carry ;AN000;
32059 00005493 B80700 MOV AX,7 ;FT. file not found ;AN000;
32060 ; 22/11/2022
32061 ; make_retn:
32062 ; return
32063 00005496 C3 retn ;FT. ;AN000;
32064 ;
32065 ; Extended Open hooks
32066 ;
32067 ; make_type2:
32068 00005497 C43E[9E05] LES DI,[THISSFT]
32069 0000549B 31C0 XOR AX,AX ; nothing exists Disk Node
32070 0000549D F9 STC ; Not found
32071 0000549E EB59 JMP short make_new
32072 ;
32073 ; The node exists. It may be either a device, directory or file:
32074 ; Zero set => directory
32075 ; High bit of CH on => device
32076 ; else => file
32077 ;
32078 ; make_exists:
32079 000054A0 7447 JZ short make_exists_dir
32080 000054A2 B003 MOV AL,3 ; file exists type 3 (error or device node)
32081 ; test byte [ATTRIB],18h
32082 000054A4 F606[6B05]18 TEST byte [ATTRIB],attr_volume_id+attr_directory
32083 000054A9 753A JNZ short make_err_ret_5
32084 ; ; Cannot already exist as Disk or Device Node
32085 ; ; if making DIR or Volume ID
32086 000054AB 08ED OR CH,CH
32087 000054AD 781A JS short make_share ; No further checks on attributes if device
32088 000054AF 08E4 OR AH,AH
32089 000054B1 75C9 JNZ short make_err_ret ; truncating NOT OK (AL = 3)
32090 000054B3 51 PUSH CX ; Save device ID
32091 000054B4 8E06[E405] MOV ES,[CURBUF+2]
32092 ; mov ch,[es:bx+0Bh]
32093 000054B8 268A6F0B MOV CH,[ES:BX+dir_entry.dir_attr] ; Get file attributes
32094 ; test ch,1
32095 000054BC F6C501 test CH,attr_read_only
32096 000054BF 7523 JNZ short make_err_ret_5P ; Cannot create on read only files
32097 000054C1 E803F9 call MatchAttributes
32098 000054C4 59 POP CX ; Devid back in CH
32099 000054C5 751E JNZ short make_err_ret_5 ; Attributes not ok
32100 000054C7 30C0 XOR AL,AL ; AL = 0, Disk Node
32101 ; make_share:

```

```

32102          ;XOR     AH,AH
32103          ; 23/02/2024
32104 000054C9 98      cbw
32105 000054CA 50      PUSH     AX          ; Save Disk or Device node
32106 000054CB 51      PUSH     CX          ; Save Device ID
32107 000054CC 88EC     MOV      AH,CH      ; Device ID to AH
32108 000054CE E83201    CALL     DOOPEN      ; Fill in SFT for share check
32109 000054D1 C43E[9E05] LES     DI,[THISSFT]
32110 000054D5 56      push     si
32111 000054D6 53      push     bx          ; Save CURBUF pointers
32112 000054D7 E8AF2F    call     ShareEnter
32113 000054DA 734E     jnc      short MakeEndShare
32114
32115          ; User failed request.
32116 000054DC 5B      pop      bx
32117 000054DD 5E      pop      si
32118 000054DE 59      pop      cx
32119 000054DF 58      pop      ax
32120
32121          Make_Share_ret:
32122 000054E0 B006     MOV      AL,6
32123 000054E2 EB98     JMP      short make_err_ret
32124
32125          make_err_ret_5P:
32126 000054E4 59      POP      CX          ; Get back device ID
32127          make_err_ret_5:
32128 000054E5 B005     MOV      AL,5          ; Attribute mismatch
32129          ; 22/11/2022
32130 000054E7 EB93     JMP      short make_err_ret
32131
32132          make_exists_dir:
32133 000054E9 B001     MOV      AL,1          ; exists as directory, always an error
32134          ; 22/11/2022
32135 000054EB EB8F     JMP      short make_err_ret
32136
32137          make_save:
32138 000054ED 50      PUSH     AX          ; Save whether Disk or File
32139 000054EE 89C8     MOV      AX,CX          ; Device ID to AH
32140 000054F0 E86800    CALL     NEWENTRY
32141 000054F3 58      POP      AX          ; 0 if Disk, 3 if File
32142 000054F4 73A0     jnc      short make_retn
32143 000054F6 B002     MOV      AL,2          ; create failed case 2
32144          make_save_retn:
32145 000054F8 C3      retn
32146
32147          make_new:
32148 000054F9 E8F1FF    call     make_save
32149 000054FC 72FA     jc      short make_save_retn ; case 2 fail
32150          ;test     byte [ATTRIB],10h
32151 000054FE F606[6B05]10    test     BYTE [ATTRIB],attr_directory
32152 00005503 75F3     jnz      short make_save_retn ; Don't "open" directories,
32153          ; so don't tell the sharer about them
32154 00005505 50      push     ax
32155 00005506 53      push     bx
32156 00005507 56      push     si
32157 00005508 E87E2F    call     ShareEnter
32158 0000550B 5E      pop      si
32159 0000550C 5B      pop      bx
32160 0000550D 58      pop      ax
32161 0000550E 73E8     jnc      short make_save_retn
32162
32163          ; We get here by having the user FAIL a share problem. Typically a failure of
32164          ; this nature is an out-of-space or an internal error. We clean up as best as
32165          ; possible: delete the newly created directory entry and return share_error.
32166
32167 00005510 50      PUSH     AX
32168 00005511 C43E[E205] LES     DI,[CURBUF]
32169          ;mov     byte [es:bx],0E5h
32170 00005515 26C607E5 MOV     BYTE [ES:BX],DIRFREE ; nuke newly created entry.
32171
32172          ; 23/02/2024
32173          %if 0
32174          ; MSDOS 6.0
32175          ;test     byte [es:di+5],40h
32176          TEST     byte [ES:DI+BUFFINFO.buf_flags],buf_dirty
32177          ;LB. if already dirty ;AN000;
32178          JNZ      short yesdirty8 ;LB. don't increment dirty count ;AN000;
32179          ; 22/11/2022
32180          call     INC_DIRTY_COUNT ;LB. ;AN000;
32181          ;or      byte [es:di+5],40h
32182          OR       byte [ES:DI+BUFFINFO.buf_flags],buf_dirty ; flag buffer as dirty
32183          yesdirty8:
32184          %else
32185          ; 23/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
32186 00005519 E8A216     call     SET_BUF_DIRTY
32187          %endif
32188 0000551C C42E[8A05] LES     BP,[THISDPB]
32189          ; 15/12/2022
32190 00005520 268A4600 mov     al,[ES:BP]
32191          ; 22/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
32192          ;mov     al,[es:bp+0]
32193          ;MOV     AL,[ES:BP+DPB.DRIVE] ; get drive for flush
32194 00005524 E87115     call     FLUSHBUF ; write out buffer.
32195 00005527 58      POP      AX
32196 00005528 EBB6     jmp      short Make_Share_ret
32197
32198          ; we have found an existing file. We have also entered it into the share set.
32199          ; At this point we need to call newentry to correctly address the problem of
32200          ; getting rid of old data (create an existing file) or creating a new
32201          ; directory entry (create a new file). Unfortunately, this operation may
32202          ; result in an INT 24 that the user doesn't return from, thus locking the file
32203          ; irretrievably into the share set. The correct solution is for us to LEAVE
32204          ; the share set now, do the operation and then reassert the share access.
32205          ;
32206          ; we are allowed to do this! There is no window! After all, we are in
32207          ; critDisk here and for someone else to get in, they must enter critDisk also.
32208
32209          ; 22/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
32210          ; DOSCODE:89C8h (MSDOS 5.0, MSDOS.SYS)
32211
32212          MakeEndShare:
32213 0000552A C43E[9E05] LES     DI,[THISSFT] ; grab SFT
32214 0000552E 31C0     XOR      AX,AX
32215 00005530 E8AFC3     call     ECritSFT
32216 00005533 268705     xchg     AX,[ES:DI]
32217          ;XCHG     AX,[ES:DI+SF_ENTRY.sf_ref_count]
32218 00005536 50      push     ax
32219 00005537 57      push     di
32220 00005538 06      push     es
32221 00005539 9C      PUSHF
32222 0000553A E8472F    call     ShareEnd ; remove sharing
32223 0000553D 9D      POPF
32224 0000553E 07      pop      es
32225 0000553F 5F      pop      di

```

```

32226 00005540 268F05      pop     word [ES:DI]
32227                      ;pop     word [ES:DI+SF_ENTRY.sf_ref_count]
32228 00005543 E8C9C3      call    LCritSFT
32229                      ; 22/11/2022
32230                      ; DOSCODE:89E4h (MSDOS 5.0, MSDOS.SYS)
32231 00005546 5B          pop     bx
32232 00005547 5E          pop     si
32233 00005548 59          pop     cx
32234 00005549 58          pop     ax
32235 0000554A E8A0FF      CALL    make_save
32236
32237                      ; If the user failed, we do not reenter into the sharing set.
32238
32239 0000554D 72A9          jc      short make_save_retn ; bye if error
32240 0000554F 50          push    ax
32241 00005550 53          push    bx
32242 00005551 56          push    si
32243 00005552 9C          PUSHF
32244 00005553 E8332F      call    ShareEnter
32245 00005556 9D          POPF
32246 00005557 5E          pop     si
32247 00005558 5B          pop     bx
32248 00005559 58          pop     ax
32249
32250                      ; If Share_check fails, then we have an internal ERROR!!!!
32251
32252 makeendshare_retn:
32253 0000555A C3          retn
32254
32255 -----
32256
32257 ; Procedure Name : NEWENTRY
32258
32259 ; Inputs:
32260 ; [THISFT] set
32261 ; [THISDPB] set
32262 ; [LASTENT] current last valid entry number in directory if no free
32263 ; entries
32264 ; [VOLID] set if a volume ID was found during search
32265 ; Attrib Contains attributes for new file
32266 ; [DIRSTART] Points to first cluster of dir (0 means root)
32267 ; CARRY FLAG INDICATES STATUS OF SEARCH FOR FILE
32268 ; NC means file existed (device)
32269 ; C means file did not exist
32270 ; AH = Device ID byte
32271 ; If FILE
32272 ; [CURBUF+2]:BX points to start of directory entry
32273 ; [CURBUF+2]:SI points to dir_first of directory entry
32274 ; If device
32275 ; DS:BX points to start of "fake" directory entry
32276 ; DS:SI points to dir_first of "fake" directory entry
32277 ; (has DWORD pointer to device header)
32278 ; Function:
32279 ; Make a new directory entry
32280 ; If an old one existed it is truncated first
32281 ; Outputs:
32282 ; Carry set if error
32283 ; Can't grow dir, atts didn't match, attempt to make 2nd
32284 ; vol ID, user FAILED to I 24
32285 ; else
32286 ; outputs of DOOPEN
32287 ; DS, BX, SI preserved (meaning on SI BX, not value), others destroyed
32288
32289 -----
32290
32291 ; 22/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
32292 ; DOSCODE:89F9h (MSDOS 5.0, MSDOS.SYS)
32293
32294 ; 23/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
32295 ; DOSCODE:961Ah (PCDOS 7.1, IBMDOS.COM)
32296
32297 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:8A34h)
32298 ; (Windows ME IO.SYS - BIOSCODE:8B7Dh)
32299
32300 NEWENTRY:
32301 0000555B C42E[8A05]    LES     BP,[THISDPB]
32302 0000555F 7315          JNC     short EXISTENT
32303 00005561 803E[4A03]00  CMP     byte [FAILERR],0
32304                      ;STC
32305                      ;jnz     short makeendshare_retn ; User FAILED, node might exist
32306                      ; 24/09/2023
32307 00005566 750C          jnz     short ERRRET3
32308 00005568 E8EAFD      CALL    BUILDDIR ; Try to build dir
32309 0000556B 72ED          jc      short makeendshare_retn ; Failed
32310 0000556D E832F3      call    GETENT ; Point at that free entry
32311 00005570 72E8          jc      short makeendshare_retn ; Failed
32312 00005572 EB0E          JMP     SHORT FREESPOT
32313
32314 ERRRET3:
32315 00005574 F9          STC
32316 newentry_retn:
32317 00005575 C3          retn
32318
32319 EXISTENT:
32320 00005576 08E4          OR      AH,AH ; Check if file is I/O device
32321 00005578 7903          JNS     short NOT_DEV1
32322 0000557A E98600      JMP     DOOPEN ; If so, proceed with open
32323
32324 NOT_DEV1:
32325 0000557D E84D01      call    FREEENT; Free cluster chain
32326 00005580 72F3          jc      short newentry_retn ; Failed
32327 FREESPOT:
32328 ;test     byte [ATTRIB],8
32329 00005582 F606[6B05]08  test     BYTE [ATTRIB],attr_volume_id
32330 00005587 7407          JZ      short NOTVOLID
32331 00005589 803E[7B05]00  CMP     BYTE [VOLID],0
32332 0000558E 75E4          JNZ     short ERRRET3 ; Can't create a second volume ID
32333 NOTVOLID:
32334 00005590 8E06[E405]    MOV     ES,[CURBUF+2]
32335 00005594 89DF          MOV     DI,BX
32336
32337 00005596 BE[4B05]    MOV     SI,NAME1
32338
32339 00005599 B90500      MOV     CX,5
32340 0000559C F3A5          REP     MOVSW
32341 0000559E A4          MOVSB ; Move name into dir entry
32342 0000559F A0[6B05]    MOV     AL,[ATTRIB]
32343 000055A2 AA          STOSB ; Attributes
32344
32345 ;; File Tagging for Create DOS 4.00
32346 000055A3 B105          MOV     CL,5 ;FT. assume normal FBUGBUG ;AN000;
32347                      ;; File Tagging for Create DOS 4.00
32348
32349 000055A5 31C0          XOR     AX,AX

```

```

32350 000055A7 F3AB      REP      STOSW      ; Zero pad
32351 000055A9 E8B4B5    call     DATE16
32352 000055AC 92      XCHG     AX,DX
32353 000055AD AB      STOSW      ; dir_time
32354 000055AE 92      XCHG     AX,DX
32355      ; 23/02/2024 (PCDOS 7.1 IBMDOS.COM)
32356      ;;;
32357 000055AF 268945FA  mov     [es:di-6],ax ; last access date
32358      ;;;
32359 000055B3 AB      STOSW      ; dir_date
32360 000055B4 31C0    XOR      AX,AX
32361 000055B6 57      PUSH     DI
32362      ; Correct SI input value
32363      ; (recomputed for new buffer)
32364 000055B8 AB      STOSW
32365 000055B9 AB      STOSW
32366      ; Zero dir_first and size
32367 000055BA 8B36[E205] updnxt:
32368      MOV      SI,[CURBUF]
32369      ; 19/05/2019 - Retro DOS v4.0
32370
32371      ; MSDOS 6.0
32372 000055BE 26F6440540 TEST     byte [ES:SI+BUFFINFO.buf_flags],buf_dirty
32373      ;LB. if already dirty ;AN000;
32374 000055C3 7508      JNZ      short yesdirty9 ;LB. don't increment dirty count ;AN000;
32375 000055C5 E80216     call     INC_DIRTY_COUNT ;LB. ;AN000;
32376
32377      ;or byte [es:si+5],40h
32378 000055C8 26804C0540 OR       byte [ES:SI+BUFFINFO.buf_flags],buf_dirty
32379 yesdirty9:
32380 000055CD C42E[8A05] LES      BP,[THISDPB]
32381      ; 15/12/2022
32382 000055D1 268A4600 MOV      AL,[ES:BP]
32383      ; 22/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
32384      ;;mov al,[es:bp+0]
32385      ;MOV AL,[ES:BP+DPB.DRIVE] ; Sets AH value again (in AL)
32386 000055D5 50      PUSH     AX
32387 000055D6 53      PUSH     BX
32388
32389      ; If we have a file, we need to increment the open ref. count so that
32390      ; we have some protection against invalid media changes if an Int 24
32391      ; error occurs.
32392      ; Do nothing for a device.
32393
32394 000055D7 06      push     es
32395 000055D8 57      push     di
32396 000055D9 C43E[9E05] LES      DI,[THISSFT]
32397      ;test word [es:di+5],80h
32398      ;TEST word [ES:DI+SF_ENTRY.sf_flags],devid_device
32399 000055DD 26F6450580 test     byte [ES:DI+SF_ENTRY.sf_flags],devid_device
32400 000055E2 7512      jnz      short GotADevice
32401 000055E4 1E      push     ds
32402 000055E5 53      push     bx
32403 000055E6 C51E[8A05] LDS      BX,[THISDPB]
32404      ;mov [es:di+7],bx
32405 000055EA 26895D07 MOV      [ES:DI+SF_ENTRY.sf_devptr],BX
32406 000055EE 8CDB      MOV      BX,DS
32407      ;mov [es:di+9],bx
32408 000055F0 26895D09 MOV      [ES:DI+SF_ENTRY.sf_devptr+2],BX
32409 000055F4 5B      pop      bx
32410 000055F5 1F      pop      ds ; need to use DS for segment later on
32411
32412      ; 23/02/2024 Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
32413      %if 0
32414      call     DEV_OPEN_SFT ; increment ref. count
32415      mov      byte [VIRTUAL_OPEN],1; set flag
32416      %endif
32417
32418      GotADevice:
32419      pop      di
32420      pop      es
32421
32422      call     FLUSHBUF
32423
32424      ; 23/02/2024 Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
32425      %if 0
32426      call     CHECK_VIRT_OPEN ; decrement ref. count;AN000;
32427      %endif
32428 000055FB 5B      POP      BX
32429 000055FC 58      POP      AX
32430 000055FD 5E      POP      SI
32431 000055FE 88C4      MOV      AH,AL ; Get SI input back
32432 00005600 7301      jnc      short DOOPEN ; Get I/O driver number back
32433 00005602 C3      retn      ; Failed
32434
32435      ;NOTE FALL THROUGH
32436
32437      ; DOSCODE:8AE4h (MSDOS 6.21, MSDOS.SYS)
32438
32439      ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
32440      ; DOSCODE:8AA9h (MSDOS 5.0, MSDOS.SYS)
32441
32442      ; DOOPEN
32443      -----
32444      ;
32445      ; Inputs:
32446      ; [THISDPB] points to DPB if file
32447      ; [THISSFT] points to SFT being used
32448      ; AH = Device ID byte
32449      ; If FILE
32450      ; [CURBUF+2]:BX points to start of directory entry
32451      ; [CURBUF+2]:SI points to dir_first of directory entry
32452      ; If device
32453      ; DS:BX points to start of "fake" directory entry
32454      ; DS:SI points to dir_first of "fake" directory entry
32455      ; (has DWORD pointer to device header)
32456      ; Function:
32457      ; Fill in SFT from dir entry
32458      ; Outputs:
32459      ; CARRY CLEAR
32460      ; sf_ref_count and sf_mode fields not altered
32461      ; sf_flags high byte = 0
32462      ; sf_flags low byte = AH except
32463      ; sf_flags Bit 6 set (not dirty or not EOF)
32464      ; sf_attr sf_date sf_time sf_name set from entry
32465      ; sf_position = 0
32466      ; If device
32467      ; sf_devptr = dword at dir_first (pointer to device header)
32468      ; sf_size = 0
32469      ; If file
32470      ; sf_firclus sf_size set from entry
32471      ; sf_devptr = [THISDPB]
32472      ; sf_cluspos = 0
32473      ; sf_lstclus = sf_firclus

```



```

32474 ; sf_dirsec sf_dirpos set
32475 ; DS,SI,BX preserved, others destroyed
32476 ;
32477 ;-----
32478 ;
32479 ; 23/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
32480 ; DOSCODE:96C3h (PCDOS 7.1, IBMDOS.COM)
32481 ;
32482 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:8AE4h)
32483 ; (Windows ME IO.SYS - BIOSCODE:8C4Ch)
32484 ;
32485 ;entry DOOPEN
32486 DOOPEN:
32487 ; Generate and store attribute
32488 ;
32489 00005603 88E6 MOV DH,AH ; AH to different place
32490 ; 23/02/2024 (PCDOS 7.1 IBMDOS.COM)
32491 ;;;
32492 00005605 30D2 xor dl,dl ; 0
32493 00005607 08E4 or ah,ah
32494 00005609 780D js short DEV_SFT0
32495 0000560B C43E[8A05] les di,[THISDPB]
32496 ;cmp word [es:di+0Fh],0
32497 0000560F 26837D0F00 cmp word [es:di+DPB.FAT_SIZE],0
32498 00005614 7402 jz short DEV_SFT0 ; FAT32
32499 00005616 FEC2 inc dl ; 1
32500 DEV_SFT0:
32501 ;;;
32502 00005618 C43E[9E05] LES DI,[THISSFT]
32503 ;add di,4
32504 0000561C 83C704 ADD DI,SF_ENTRY.sf_attr ; Skip ref_count and mode fields
32505 ; 24/09/2023
32506 0000561F 31C0 xor ax,ax
32507 ;XOR AL,AL ; Assume it's a device, devices have an
32508 ; attribute of 0 (for R/O testing etc).
32509 00005621 08F6 OR DH,DH ; See if our assumption good.
32510 00005623 7807 JS short DEV_SFT1 ; If device DS=DOSGROUP
32511 00005625 8E1E[E405] MOV DS,[CURBUF+2]
32512 ;mov al,[BX+0Bh]
32513 00005629 8A470B MOV AL,[BX+dir_entry.dir_attr]
32514 ; If file, get attrib from dir entry
32515 DEV_SFT1:
32516 0000562C AA STOSB ; sf_attr, ES:DI -> sf_flags
32517 ;
32518 ; Generate and store flags word
32519 ;
32520 ; 24/09/2023
32521 ;XOR AX,AX
32522 ; ah=0
32523 0000562D 88F0 MOV AL,DH
32524 ;or al,40h
32525 0000562F 0C40 OR AL,devide_file_clean
32526 00005631 AB STOSW ; sf_flags, ES:DI -> sf_devptr
32527 ;
32528 ; Generate and store device pointer
32529 ;
32530 00005632 1E PUSH DS
32531 ;lds ax,[bx+1Ah]
32532 00005633 C5471A LDS AX,[BX+dir_entry.dir_first] ; Assume device
32533 00005636 08F6 OR DH,DH
32534 00005638 7805 JS short DEV_SFT2
32535 ;
32536 ;hkn; SS override
32537 0000563A 36C506[8A05] LDS AX,[SS:THISDPB] ; was file
32538 DEV_SFT2:
32539 0000563F AB STOSW ; store offset
32540 00005640 8CD8 MOV AX,DS
32541 00005642 1F POP DS
32542 00005643 AB STOSW ; store segment
32543 ; ES:DI -> sf_firclus
32544 ;
32545 ; Generate pointer to, generate and store first cluster
32546 ; (irrelevant for devices)
32547 ;
32548 00005644 56 PUSH SI ; Save pointer to dir_first
32549 ;
32550 ; 23/02/2024
32551 %if 0
32552 ;
32553 MOVSW ; dir_first -> sf_firclus
32554 ; DS:SI -> dir_size_l, ES:DI -> sf_time
32555 ;
32556 ; Copy time/date of last modification
32557 ;
32558 ;sub si,6
32559 SUB SI,dir_entry.dir_size_l - dir_entry.dir_time
32560 %else
32561 ; 23/02/2024 - Retro DOS v5.0
32562 ; (PCDOS 7.1 IBMDOS.COM)
32563 ;;;
32564 00005645 83C720 add di,32 ; SF_ENTRY.sf_chain ; first cluster (32 bit) !?
32565 00005648 A5 movsw ; first cluster, lw
32566 ;add si,0FFF8h
32567 00005649 83C6F8 add si,-8
32568 0000564C AD lodsw ; 27/06/2024 ; first cluster, hw
32569 ;
32570 0000564D 08D2 or dl,dl
32571 0000564F 7402 jz short FILE_SFT0 ; FAT32
32572 00005651 31C0 xor ax,ax ; 0 ; clear hw of first cluster (invalid)
32573 FILE_SFT0:
32574 00005653 AB stosw
32575 ;add di,0FFDEh
32576 00005654 83C7DE add di,-34 ; SF_ENTRY.sf_time
32577 ;;;
32578 %endif
32579 ; DS:SI->dir_time
32580 00005657 A5 MOVSW ; dir_time -> sf_time
32581 ; DS:SI -> dir_date, ES:DI -> sf_date
32582 00005658 A5 MOVSW ; dir_date -> sf_date
32583 ; DS:SI -> dir_first, ES:DI -> sf_size
32584 ;
32585 ; Generate and store file size (0 for devices)
32586 ;
32587 00005659 AD LODSW ; skip dir_first, DS:SI -> dir_size_l
32588 0000565A AD LODSW ; dir_size_l in AX, DS:SI -> dir_size_h
32589 ;MOV CX,AX ; dir_size_l in CX
32590 ; 23/02/2024
32591 0000565B 91 xchg ax,cx
32592 0000565C AD LODSW ; dir_size_h (size AX:CX), DS:SI -> ???
32593 0000565D 08F6 OR DH,DH
32594 0000565F 7904 JNS short FILE_SFT1
32595 00005661 31C0 XOR AX,AX
32596 00005663 89C1 MOV CX,AX ; Devices are open ended
32597 FILE_SFT1:

```

```

32598 00005665 91      XCHG    AX,CX
32599 00005666 AB      STOSW                    ; Low word of sf_size
32600 00005667 91      XCHG    AX,CX
32601 00005668 AB      STOSW                    ; High word of sf_size
32602                                     ; ES:DI -> sf_position
32603 ; Initialize position to 0
32604
32605 00005669 31C0     XOR     AX,AX
32606 0000566B AB      STOSW
32607 0000566C AB      STOSW                    ; sf_position
32608                                     ; ES:DI -> sf_cluspos
32609
32610 ; Generate cluster optimizations for files
32611
32612 0000566D 08F6     OR      DH,DH
32613 0000566F 784E     JS      short DEV_SFT3
32614 00005671 AB      STOSW                    ; sf_cluspos ; 19h
32615
32616 ; 23/02/2024
32617 %if 0
32618     ;mov    ax,[bx+1Ah]
32619     MOV     AX,[BX+dir_entry.dir_first]
32620     ; 19/05/2019
32621     ; MSDOS 3.3
32622     ; STOSW                    ; sf_lstclus ; 1Bh
32623     ; MSDOS 6.0
32624     PUSH    DI                    ;AN004; save dirsec offset
32625     ;sub    di,1Bh
32626     SUB     DI,SF_ENTRY.sf_dirsec ;AN004; es:di -> SFT
32627     ;mov    [es:di+35h],ax
32628     MOV     [ES:DI+SF_ENTRY.sf_lstclus],AX ;AN004; save it
32629     POP     DI                    ;AN004; restore dirsec offset
32630 %else
32631     ; 23/02/2024 - Retro DOS v5.0
32632     ; (PCDOS 7.1 IBMDOS.COM)
32633     ;;;
32634     ;add    di,0FFF0h
32635     add     di,-16
32636     stosw                    ; sf_cluspos_h (sf_firclus in MSDOS 5.0-6.22)
32637     ;add    di,SF_ENTRY.sf_dirsec
32638     add     di,14                ; sf_dirsec ; 27
32639     mov     ax,[es:di+10h] ; [ES:DI+SF_ENTRY.sf_chain] ; 43
32640                                     ; first cluster (32 bit)
32641     mov     [es:di+1Ah],ax
32642     ;mov    [es:di+SF_ENTRY.sf_lstclus-SF_ENTRY.sf_dirsec],ax ; 53
32643     mov     ax,[es:di+12h] ; [ES:DI+SF_ENTRY.sf_chain+2] ; 45
32644     mov     [es:di+1Ch],ax
32645     ;mov    [es:di+SF_ENTRY.sf_lstclus+2-SF_ENTRY.sf_dirsec],ax ; 55
32646     ;;;
32647 %endif
32648
32649 ; DOS 3.3 FastOpen 6/13/86
32650
32651 00005689 1E      PUSH    DS
32652
32653 ;hkn; SS is DOSDATA
32654     push    ss
32655     pop     ds
32656     ;test   byte [FastOpenFlg],4
32657     TEST    byte [FastOpenFlg],Special_Fill_Set
32658     JZ      short Not_FastOpen
32659
32660 ;hkn; FastOpen_Ext_Info is in DOSDATA
32661     MOV     SI,FastOpen_Ext_Info
32662
32663     ;mov    ax,[si+1]
32664     MOV     AX,[SI+FEI.dirsec]
32665     STOSW                    ; sf_dirsec
32666     ; MSDOS 6.0
32667     ;mov    ax,[si+3]
32668     MOV     AX,[SI+FEI.dirsec+2]
32669     ;;; changed for >32mb
32670     STOSW                    ; sf_dirsec
32671     ; 19/08//2018
32672     mov     al,[SI]
32673     ;MOV     AL,[SI+FEI.dirpos] ; mov al,[SI+0]
32674     STOSB                    ; sf_dirpos
32675     POP     DS
32676     ;JMP     short Next_Name
32677     ; 24/09/2023
32678     jmp     short FILE_SFT2      ; cf=0 (after 'test' instruction)
32679
32680 ; DOS 3.3 FastOpen 6/13/86
32681
32682 Not_FastOpen:
32683     ;POP     DS                    ; normal path
32684
32685 ;hkn; SS override
32686     ;MOV     SI,[SS:CURBUF] ; DS:SI->buffer header
32687     ; 16/12/2022
32688     ; 28/07/2019
32689     mov     si,[CURBUF]
32690     pop     ds
32691     ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
32692     ;pop     ds
32693     ;mov     si,[ss:CURBUF]
32694
32695     ;mov     ax,[si+6]
32696     MOV     AX,[SI+BUFFINFO.buf_sector] ;F.C. >32mb ;AN000;
32697     STOSW                    ; sf_dirsec ;F.C. >32mb ;AN000;
32698     ; 19/05/2019
32699     ; MSDOS 6.0
32700     ;mov     ax,[si+8]
32701     MOV     AX,[SI+BUFFINFO.buf_sector+2] ;F.C. >32mb ;AN000;
32702     STOSW                    ; sf_dirsec ;F.C. >32mb ;AN000;
32703
32704     MOV     AX,BX
32705     ;;;add    si,16 ; MSDOS 3.3
32706     ;;;add    si,20 ; MSDOS 6.0
32707     ;add     si,24 ; PCDOS 7.1 ; 23/02/2024
32708     ADD     SI,BUFINSIZ ; DS:SI-> start of data in buffer
32709     SUB     AX,SI ; AX = BX relative to start of sector
32710     ;mov     cl,32
32711     MOV     CL,dir_entry.size
32712     DIV     CL
32713     STOSB                    ; sf_dirpos
32714 Next_Name:
32715     JMP     SHORT FILE_SFT2
32716
32717     ; 24/09/2023
32718     ; cf=0 (after 'or' instruction)
32719 DEV_SFT3:
32720     ;add     di,7
32721     ADD     DI,SF_ENTRY.sf_name-SF_ENTRY.sf_cluspos

```



```

32722 FILE_SFT2:
32723
32724 ; Copy in the object's name
32725
32726 MOV SI,BX ; DS:SI points to dir_name
32727 MOV CX,11
32728 REP MOVSB ; sf_name
32729 POP SI ; recover DS:SI -> dir_first
32730
32731 ;hkn; SS is DOSDATA
32732 push ss
32733 pop ds
32734 ; 24/09/2023
32735 ; cf=0
32736 ;CLC
32737 retn
32738
32739
32740
32741
32742
32743
32744
32745
32746
32747
32748
32749
32750
32751
32752
32753
32754
32755
32756
32757
32758
32759
32760
32761
32762
32763
32764
32765
32766
32767
32768
32769
32770
32771
32772
32773
32774
32775
32776
32777
32778
32779
32780
32781
32782
32783
32784
32785
32786
32787
32788
32789
32790
32791
32792
32793
32794
32795
32796
32797
32798
32799
32800
32801
32802
32803
32804
32805
32806
32807
32808
32809
32810
32811
32812
32813
32814
32815
32816
32817
32818
32819
32820
32821
32822
32823
32824
32825
32826
32827
32828
32829
32830
32831
32832
32833
32834
32835
32836
32837
32838
32839
32840
32841
32842
32843
32844
32845

```

```

;-----
; Procedure Name : FREEENT
;-----
; Inputs:
; ES:BP -> DPB
; [CURBUF] Set
; [CURBUF+2]:BX points to directory entry
; [CURBUF+2]:SI points to above dir_first
; Function:
; Free the cluster chain for the entry if present
; Outputs:
; Carry set if error (currently user FAILED to I 24)
; (NOTE dir_firclus and dir_size_l/h are wrong)
; DS BX SI ES BP preserved (BX,SI in meaning, not value) others destroyed
;-----
; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
; 24/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)

FREEENT:
PUSH DS
LDS DI,[CURBUF]
MOV CX,[SI] ; Get pointer to clusters
; 24/02/2023 (PCDOS 7.1 IBMDOS.COM)
;
; mov ax,[si-6] ; hw of first cluster (dir_first)
;
; 19/05/2019 - Retro DOS v4.0
; MSDOS 6.0
; mov dx,[di+8] ; 24/02/2024
MOV DX,[DI+BUFFINFO.buf_sector+2] ;F.C. >32mb ;AN000;
;hkn; SS override
MOV [SS:HIGH_SECTOR],DX ;F.C. >32mb ;AN000;
; mov dx,[di+6] ; 24/02/2024
MOV DX,[DI+BUFFINFO.buf_sector]
POP DS
; 24/02/2023 (PCDOS 7.1 IBMDOS.COM)
;
; cmp word [bp+0Fh],0
; cmp word [es:bp+DPB.FAT_SIZE],0
; jnz short freent2 ; not FAT32
; cmp ax,0
; or ax,ax
; jnz short freent1
;
; cmp cx,2
; jb short RET1 ; was 0 length file (or mucked Firclus if AX:CX=1)
freent1:
; cmp ax,[es:bp+2Fh]
; cmp ax,[es:bp+DPB.LAST_CLUSTER+2]
; jne short freent3
; cmp cx,[es:bp+2Dh]
; cmp cx,[es:bp+DPB.LAST_CLUSTER]
; jmp short freent3
freent2:
xor ax,ax
;
; CMP CX,2
; JB short RET1 ; was 0 length file (or mucked Firclus if CX=1)
;
; cmp cx,[es:bp+0Dh]
; CMP CX,[ES:BP+DPB.MAX_CLUSTER]
; 24/02/2024
; JA short RET1 ; Treat like zero length file (firclus mucked)
; ja short freent_retn ; 24/02/2024
SUB BX,DI
PUSH BX ; Save offset
PUSH word [HIGH_SECTOR] ;F.C. >32mb ;AN000;
PUSH DX ; Save sector number
MOV BX,CX
; 24/02/2023 (PCDOS 7.1 IBMDOS.COM)
;
; mov [CLUSTNUM_HW],ax
;
; call RELEASE ; Free any data allocated
POP DX
POP word [HIGH_SECTOR] ;F.C. >32mb ;AN000;
JNC short GET_BUF_BACK
POP BX
freent_retn:
retn ; Screw up

GET_BUF_BACK:
; 22/09/2023
; mov byte [ALLOWED],18h
; MOV byte [ALLOWED],Allowed_RETRY+Allowed_FAIL ; *
; XOR AL,AL ; *
; call GETBUFR ; Get sector back
call GETBUFFER ; * ; pre read
;
POP BX ; Get offset back
jc short freent_retn
call SET_BUF_AS_DIR
ADD BX,[CURBUF] ; Correct it for new buffer
;
; MOV SI,BX
; add si,1Ah
; ADD SI,dir_entry.dir_first; Get corrected SI
; 24/02/2024 (PCDOS 7.1 IBMDOS.COM)
; lea si,[bx+1Ah]
; lea si,[bx+dir_entry.dir_first]
RET1:
CLC
retn

```

```

32846 ; 23/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
32847 %if 0
32848 ;
32849 ;-----
32850 ;
32851 ; Procedure Name : CHECK_VIRT_OPEN
32852 ;
32853 ; CHECK_VIRT_OPEN checks to see if we had performed a "virtual open" (by
32854 ; examining the flag [VIRTUAL_OPEN] to see if it is 1). If we did, then
32855 ; it calls Dev_Close_SFT to decrement the ref. count. It also resets the
32856 ; flag [VIRTUAL_OPEN].
32857 ; No registers affected (including flags).
32858 ; On input, [THISSFT] points to current SFT.
32859 ;-----
32860 ;
32861 ;
32862 CHECK_VIRT_OPEN:
32863     PUSH    AX
32864     lahf    ; preserve flags
32865     CMP     byte [VIRTUAL_OPEN],0
32866     JZ      short ALL_CLOSED
32867     mov     byte [VIRTUAL_OPEN],0 ; reset flag
32868     push    es
32869     push    di
32870     LES     DI,[THISSFT]
32871     call    DEV_CLOSE_SFT
32872     pop     di
32873     pop     es
32874
32875 ALL_CLOSED:
32876     sahf    ; restore flags
32877     POP     AX
32878     retn
32879
32880 %endif
32881
32882 ;=====
32883 ; ROM.ASM, MSDOS 6.0, 1991
32884 ;=====
32885 ; 29/07/2018 - Retro DOS v3.0
32886 ; 20/05/2019 - Retro DOS v4.0
32887 ; 10/02/2024 - Retro DOS v5.0
32888
32889 ; TITLE ROM - Miscellaneous routines
32890 ; NAME ROM
32891
32892 ** Misc Low level routines for doing simple FCB computations, Cache
32893 reads and writes, I/O optimization, and FAT allocation/deallocation
32894
32895 ;
32896 ; SKPCLP
32897 ; FNDCLUS
32898 ; BUFSEC
32899 ; BUFRD
32900 ; BUFWRT
32901 ; NEXTSEC
32902 ; OPTIMIZE
32903 ; FIGREC
32904 ; ALLOCATE
32905 ; RESTFATBYT
32906 ; RELEASE
32907 ; RELBLKS
32908 ; GETEOF
32909
32910 ; Modification history:
32911 ;
32912 ; Created: ARR 30 March 1983
32913 ; M039: DB 10/25/90 - Disk read/write optimization.
32914
32915 ;Break <FNDCLUS -- Skip over allocation units>
32916 ;-----
32917 ;
32918 ; Procedure Name : FNDCLUS
32919 ;
32920 ; Inputs:
32921 ; CX = No. of clusters to skip
32922 ; ES:BP = Base of drive parameters
32923 ; [THISSFT] point to SFT
32924 ; Outputs:
32925 ; BX = Last cluster skipped to
32926 ; CX = No. of clusters remaining (0 unless EOF)
32927 ; DX = Position of last cluster
32928 ; Carry set if error (currently user FAILED to I 24)
32929 ; DI destroyed. No other registers affected.
32930 ;-----
32931 ;
32932 ;;;
32933 ; 10/02/2024 - Retro DOS v5.0
32934 ; (PCDOS 7.1 IBMDOS.COM)
32935
32936 FNDCLUS_X:
32937     mov     cx,[CLUSNUM]
32938     mov     dx,[CLUSNUM_HW]
32939     ;;;
32940
32941 ; 20/05/2019 - Retro DOS v4.0
32942 ; DOSCODE:8BF2h (MSDOS 6.21, MSDOS.SYS)
32943 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
32944 ; DOSCODE:8BB7h (MSDOS 5.0, MSDOS.SYS)
32945
32946 ; 25/02/2024
32947 ; 10/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
32948 ; DOSCODE:9801h (PCDOS 7.1 IBMDOS.COM)
32949
32950 FNDCLUS:
32951     PUSH    ES
32952     LES     DI,[THISSFT] ; setup addressability to SFT
32953
32954     ;;;
32955     ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
32956     mov     [CLSKIP_HW],dx
32957     ;mov     bx,[es:di+37h]
32958     mov     bx,[es:di+SF_ENTRY.sf_lstclus+2] ; 24/02/2024
32959     mov     [CLUSTNUM_HW],bx
32960     or      bx,bx ; *
32961     mov     dx,[es:di+0Bh] ; [ES:DI+SF_ENTRY.sf_fircclus] ; MSDOS 5.0-6.22
32962     ;mov     dx,[es:di+SF_ENTRY.sf_cluspos_hw] ; PCDOS 7.1
32963     mov     [LASTPOS_HW],dx
32964     ;;;
32965     ;mov     bx,[es:di+1Bh] ; MSDOS 3.3
32966     ;mov     bx,[es:di+35h] ; MSDOS 6.0
32967     MOV     BX,[ES:DI+SF_ENTRY.sf_lstclus]
32968     ;mov     dx,[es:di+19h]
32969     MOV     DX,[ES:DI+SF_ENTRY.sf_cluspos]

```

```

32970             ; 10/02/2024
32971             ;OR      BX,BX
32972             ;JZ      short NOCLUS
32973             ;;;
32974             ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
32975 00005762 7506     jnz      short fndclus_1 ; *
32976 00005764 09DB     or       bx,bx
32977 00005766 7502     jnz      short fndclus_1
32978 00005768 EB6A     jmp       NOCLUS
32979
32980 fndclus_1:
32981             ;;;
32982
32983             ; 10/02/2024
32984 %if 0
32985             SUB CX,DX
32986             JNB short FINDIT
32987             ADD     CX,DX
32988             XOR     DX,DX
32989             ;mov     bx,[es:di+0Bh]
32990             MOV     BX,[ES:DI+SF_ENTRY.sf_firclus]
32991 FINDIT:
32992             POP     ES
32993             JCXZ     RET9
32994 %else
32995             ;;;
32996             ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
32997 0000576A 29D1     sub      cx,dx
32998 0000576C 50       push     ax
32999 0000576D A1[E20A]  mov      ax,[LASTPOS_HW]
33000 00005770 1906[F80A] sbb      [CLSKIP_HW],ax
33001 00005774 58       pop      ax
33002 00005775 731C     jnb      short FINDIT
33003 00005777 8B1E[E20A] mov      bx,[LASTPOS_HW]
33004 0000577B 01D1     add      cx,dx
33005 0000577D 111E[F80A] adc      [CLSKIP_HW],bx
33006 00005781 31D2     xor      dx,dx
33007 00005783 8916[E20A] mov      [LASTPOS_HW],dx ; 0
33008 00005787 268B5D2D mov      bx,[es:di+2Dh] ; [ES:DI+SF_ENTRY.sf_chain+2] ; MSDOS 5.0-6.22
33009                                     ; [ES:DI+SF_ENTRY.sf_fcluster+2] ; PCDOS 7.1
33010 0000578B 891E[E80A] mov      [CLUSTNUM_HW],bx
33011 0000578F 268B5D2B mov      bx,[es:di+2Bh] ; [ES:DI+SF_ENTRY.sf_chain] ; MSDOS 5.0-6.22
33012                                     ; [ES:DI+SF_ENTRY.sf_fcluster] ; PCDOS 7.1
33013 FINDIT:
33014 00005793 07       pop      es
33015 00005794 3B0E[F80A] cmp      cx,[CLSKIP_HW]
33016 00005798 7502     jnz      short SKPCLP
33017 0000579A E337     jcxz     RET9      ; cf=0
33018             ;;;
33019 %endif
33020
33021             ;entry SKPCLP
33022 SKPCLP:
33023
33024             ; 10/02/2024
33025 %if 0
33026             call     UNPACK
33027             jc      short fndclus_retn      ; retc
33028
33029             ; 09/09/2018
33030
33031             ; MSDOS 3.3
33032             ;push     bx
33033             ;mov      bx,di
33034             ;call     ISEOF
33035             ;pop      bx
33036             ;jae      short RET9
33037
33038             ; 20/05/2019 - Retro DOS v4.0
33039
33040             ; MSDOS 6.0
33041             xchg      bx,di
33042             call     ISEOF
33043             xchg      bx,di
33044             jae      short RET9
33045
33046             XCHG      BX,DI
33047             INC       DX
33048
33049             LOOP      SKPCLP                ; RMFS
33050 %else
33051             ;;;
33052             ; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
33053
33054             ;push     word [LASTPOS_HW]
33055             ;push     dx
33056             ;push     word [CLSKIP_HW]
33057             ;push     cx
33058
33059 0000579C E8420B     call     UNPACK
33060
33061             ;pop      cx
33062             ;pop      word [CLSKIP_HW]
33063             ;pop      dx
33064             ;pop      word [LASTPOS_HW]
33065
33066 0000579F 7242     jc      short fndclus_retn
33067
33068 000057A1 FF36[E80A]  push     word [CLUSTNUM_HW]
33069 000057A5 53       push     bx
33070 000057A6 89FB     mov      bx,di
33071 000057A8 8B3E[EC0A]  mov      di,[CONTENT_HW]
33072 000057AC 893E[E80A]  mov      [CLUSTNUM_HW],di
33073 000057B0 E8050B     call     ISEOF
33074 000057B3 7206     jc      short SKPCLP2
33075 000057B5 5B       pop      bx
33076 000057B6 8F06[E80A]  pop      word [CLUSTNUM_HW]
33077             ;jmp      short RET9
33078             ; 25/02/2024
33079             retn
33080
33081 SKPCLP2:
33082             add      sp,4
33083             inc      dx
33084             jnz      short SKPCLP3
33085             inc      word [LASTPOS_HW]
33086
33087 SKPCLP3:
33088             sub      cx,1
33089             sbb      word [CLSKIP_HW],0
33090             jnz      short SKPCLP          ; loop
33091             or       cx,cx
33092             jnz      short SKPCLP          ; loop
33093             ;;;
33094 %endif

```

```

33094
33095
33096
33097
33098
33099 000057D3 C3
33100
33101
33102 000057D4 07
33103
33104
33105
33106
33107
33108
33109
33110
33111
33112
33113
33114
33115
33116
33117
33118
33119
33120 000057D5 4A
33121 000057D6 7904
33122 000057D8 FF0E[E20A]
33123
33124 000057DC 41
33125 000057DD 7504
33126 000057DF FF06[F80A]
33127
33128
33129
33130
33131
33132
33133 000057E3 C3
33134
33135
33136
33137
33138
33139
33140
33141
33142
33143
33144
33145
33146
33147
33148
33149
33150
33151
33152
33153
33154
33155
33156
33157
33158
33159
33160
33161
33162
33163
33164
33165
33166
33167
33168
33169
33170 000057E4 8B16[DE0A]
33171
33172 000057E8 8716[E80A]
33173 000057EC 52
33174
33175 000057ED 8B16[BC05]
33176 000057F1 8A1E[7305]
33177
33178 000057F5 C606[4B03] 38
33179 000057FA E89A01
33180 000057FD E83011
33181
33182
33183 00005800 8F06[E80A]
33184
33185 00005804 72DD
33186
33187 00005806 C606[7405] 01
33188 0000580B 8B36[B805]
33189 0000580F 89F7
33190 00005811 8B0E[D205]
33191 00005815 01CF
33192 00005817 893E[B805]
33193 0000581B C43E[E205]
33194
33195 0000581F 26804D0508
33196
33197
33198
33199 00005824 8D7D18
33200 00005827 033E[CC05]
33201 0000582B F8
33202 0000582C C3
33203
33204
33205
33206
33207
33208
33209
33210
33211
33212
33213
33214
33215
33216
33217

RET9:
;CLC
; 25/02/2024
; cf=0
retn

NOCLUS:
POP ES

; 10/02/2024
%if 0
INC CX
DEC DX
%else
;;;
; 10/02/2024 (PCDOS 7.1 IBMDOS.COM)
;push di
;xor di,di ; 0
;add cx,1
;adc [CLSKIP_HW],di ; CLSKIP_HW:BX = Last cluster skipped to
;sub dx,1
;sbb [LASTPOS_HW],di ; LASTPOS_HW:DX = Position of last cluster
;pop di
;;;
; 25/02/2024 - Retro DOS v5.0
dec dx
jns short noclus1
dec word [LASTPOS_HW]
noclus1:
inc cx
jnz short fndclus_retn
inc word [CLSKIP_HW]
%endif
; 25/02/2024
; cf=0
;CLC

fndclus_retn:
retn

;Break <BUFSEC -- BUFFER A SECTOR AND SET UP A TRANSFER>
;-----
;
; Procedure Name : BUFSEC
;
; Inputs:
; AH = priority of buffer
; AL = 0 if buffer must be read, 1 if no pre-read needed
; ES:BP = Base of drive parameters
; [CLUSNUM] = Physical cluster number
; [SECCLUSPOS] = Sector position of transfer within cluster
; [BYTCNT1] = Size of transfer
; Function:
; Insure specified sector is in buffer, flushing buffer before
; read if necessary.
; Outputs:
; ES:DI = Pointer to buffer
; SI = Pointer to transfer address
; CX = Number of bytes
; [NEXTADD] updated
; [TRANS] set to indicate a transfer will occur
; Carry set if error (user FAILED to I 24)
;-----
; 25/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
; PCDOS 7.1 IBMDOS.COM - DOSCODE:98C2h
;
; (MSDOS 5.0 MSDOS.SYS - DOSCODE:8BF1h) - Retro DOS v4.0 (& v4.1)
; (MSDOS 6.22 MSDOS.SYS - DOSCODE:8C2Ch) - Retro DOS v4.2
; (Windows ME IO.SYS - BIOSCODE:0BE33h)

BUFSEC:
; 25/02/2024
;
;push word [CLUSTNUM_HW]
mov dx,[CLUSNUM_HW]
;mov [CLUSTNUM_HW],dx
xchg dx,[CLUSTNUM_HW]
push dx ; [CLUSTNUM_HW]
;
MOV DX,[CLUSNUM]
MOV BL,[SECCLUSPOS]
;mov byte [ALLOWED],38h
MOV byte [ALLOWED],Allowed_FAIL+Allowed_RETRY+Allowed_IGNORE
CALL FIGREC
call GETBUFR
; 25/02/2024
;
pop word [CLUSTNUM_HW]
;
jc short fndclus_retn

MOV BYTE [TRANS],1 ; A transfer is taking place
MOV SI,[NEXTADD]
MOV DI,SI
MOV CX,[BYTCNT1]
ADD DI,CX
MOV [NEXTADD],DI
LES DI,[CURBUF]
;or byte [es:di+5],8
OR byte [ES:DI+BUFFINFO.buf_flags],buf_isDATA
;lea di,[di+16] ; MSDOS 3.3
;lea di,[di+20] ; MSDOS 6.0
;lea di,[di+24] ; PCDOS 7.1 ; 25/02/2024
LEA DI,[DI+BUFINSIZ] ; Point to buffer
ADD DI,[BYTSECPOS]
CLC ; (not necessary!?) ; 25/02/2024
retn

;Break <BUFRD, BUFWRT -- PERFORM BUFFERED READ AND WRITE>
;-----
;
; Procedure Name : BUFRD
;
; Do a partial sector read via one of the system buffers
; ES:BP Points to DPB
; Carry set if error (currently user FAILED to I 24)
;
; DS - set to DOSDATA
;-----

```

```

33218 ; 25/02/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
33219 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
33220 ; 20/05/2019 - Retro DOS v4.0
33221
33222 0000582D 06
33223 0000582E 31C0
33224 00005830 E8B1FF
33225 00005833 7303
33226
33227
33228 00005835 07
33229 00005836 EB2D
33230
33231
33232 00005838 8CC3
33233 0000583A 8E06[2E03]
33234 0000583E 8EDB
33235 00005840 87FE
33236 00005842 D1E9
33237
33238
33239
33240
33241
33242
33243
33244
33245
33246
33247
33248
33249 00005844 F3A5
33250
33251
33252
33253 00005846 7301
33254 00005848 A4
33255
33256
33257
33258
33259
33260 00005849 07
33261
33262 0000584A 36C53E[E205]
33263
33264
33265
33266 0000584F 8D5D18
33267 00005852 29DE
33268 00005854 E87B10
33269
33270 00005857 263B7602
33271 0000585B 7205
33272
33273
33274
33275
33276
33277 0000585D 36893E[6D00]
33278
33279
33280
33281 00005862 F8
33282
33283 00005863 16
33284 00005864 1F
33285
33286 00005865 C3
33287
33288
33289
33290
33291
33292
33293
33294
33295
33296
33297
33298
33299
33300
33301
33302
33303
33304
33305
33306
33307
33308
33309 00005866 FF06[C405]
33310 0000586A 7504
33311 0000586C FF06[C605]
33312
33313 00005870 A1[C605]
33314 00005873 3B06[CA05]
33315 00005877 B001
33316 00005879 770F
33317 0000587B 720B
33318 0000587D A1[C405]
33319
33320
33321
33322
33323
33324
33325
33326 00005880 3B06[C805]
33327 00005884 B001
33328 00005886 7702
33329
33330 00005888 30C0
33331
33332 0000588A 06
33333 0000588B E856FF
33334 0000588E 72A5
33335 00005890 8E1E[2E03]
33336 00005894 D1E9
33337
33338
33339
33340
33341

; 25/02/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
; 20/05/2019 - Retro DOS v4.0
BUF RD:
    PUSH        ES
    xor ax,ax
    CALL        BUF SEC
    JNC short   BUF_OK ; ds=ss

BUF_IO_FAIL:
    POP ES
    JMP         SHORT RBUF PLACED ; ds=ss

BUF_OK:
    MOV        BX,ES
    MOV        ES,[DMAADD+2]
    MOV        DS,BX
    XCHG       DI,SI
    SHR        CX,1
;M039
; MSDOS 3.3
; JNC short   EVEN RD
; MOVSB
; EVEN RD:
; REP        MOVSW

; CX = # of whole WORDs ; CF=1 if odd # of bytes.
; DS:SI-> Source within Buffer.
; ES:DI-> Destination within Transfer memory block.

; MSDOS 6.0
rep movsw
;adc cx,0
;rep movsb
; 16/12/2022
jnc short   EVEN RD ; **** 20/05/2019
movsb ; ****
; 25/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
;adc cx,0
;rep movsb
;M039
EVEN RD: ; ****
    POP ES
;hkn; SS override
    LDS        DI,[SS:CURBUF]
    ;lea bx,[di+16]
    ;lea bx,[di+20] ; MSDOS 6.0
    ;lea bx,[di+24] ; PC DOS 7.1; 25/02/2024
    LEA        BX,[DI+BUFINSIZ]
    SUB        SI,BX
    call       PLACEBUF
    ;cmp si,[es:bp+2]
    CMP        SI,[ES:BP+DPB.SECTOR_SIZE] ; Read Last byte?
    JB         short RBUF PLACEDC ; ds<=ss ; No, leave buf where it is
;M039
; MSDOS 3.3
; call       PLACEHEAD
; Make it prime candidate for chucking
; even though it is MRU.
; MSDOS 6.0
MOV         [ss:BufferQueue],DI ; Make it prime candidate for
;M039
; chucking even though it is MRU.

RBUF PLACEDC:
    CLC
;RBUF PLACED:
    push ss
    pop ds
RBUF PLACED: ; 25/02/2024 (ds=ss)
    retn

;-----
;
; Procedure : BUF WRT
;
; Do a partial sector write via one of the system buffers
; ES:BP Points to DPB
; Carry set if error (currently user FAILED to I 24)
;
; DS - set to DOS DATA
;-----
; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
; 20/05/2019 - Retro DOS v4.0
BUF WRT:
    ;MOV AX,[SECPOS]
    ; MSDOS 6.0
    ;ADD AX,1
    ; Set for next sector
    ;MOV [SECPOS],AX ; F.C. >32mb ; AN000;
    ;ADC word [SECPOS+2],0 ; F.C. >32mb ; AN000;
    ; 24/09/2023
    inc word [SECPOS]
    jnz short bufw_secpos
    inc word [SECPOS+2]
bufw_secpos:
    MOV AX,[SECPOS+2] ; F.C. >32mb ; AN000;
    CMP AX,[VALSEC+2] ; F.C. >32mb ; AN000;
    MOV AL,1 ; F.C. >32mb ; AN000;
    JA short NOREAD ; F.C. >32mb ; AN000;
    JB short _doread ; F.C. >32mb ; AN000;
    MOV AX,[SECPOS] ; F.C. >32mb ; AN000;
; MSDOS 3.3
; INC AX
; MOV [SECPOS],AX ; 09/09/2018
; 20/05/2019
; MSDOS 3.3 & MSDOS 6.0
CMP AX,[VALSEC] ; Has sector been written before?
MOV AL,1
JA short NO READ ; Skip preread if SECPOS>VALSEC
_doread:
    XOR AL,AL
NO READ:
    PUSH ES
    CALL BUF SEC
    JC short BUF_IO_FAIL
    MOV DS,[DMAADD+2]
    SHR CX,1
;M039
; MSDOS 3.3
; JNC short   EVEN WRT ; 09/09/2018
; MOVSB
; EVEN WRT:

```

```

33342      ;REP    MOVSW
33343
33344      ; CX = # of whole WORDs; CF=1 if odd # of bytes.
33345      ; DS:SI-> Source within Transfer memory block.
33346      ; ES:DI-> Destination within Buffer.
33347
33348      ; MSDOS 6.0
33349 00005896 F3A5      rep    movsw      ;Copy Transfer memory to Buffer.
33350      ;adc     cx,0      ;CX=1 if odd # of bytes, else CX=0.
33351      ;rep     movsb     ;Copy last byte.
33352      ; 16/12/2022
33353 00005898 7301      jnc     short EVENWRT ; **** 20/05/2019
33354 0000589A A4      movsb ; ****
33355      ; 25/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
33356      ;adc     cx,0
33357      ;rep     movsb
33358 ;M039
33359 EVENWRT: ; ****
33360 0000589B 07      POP     ES
33361
33362      ;hkn; ss override
33363 0000589C 36C51E[E205] LDS     BX,[SS:CURBUF]
33364
33365      ; MSDOS 6.0
33366 000058A1 F6470540      TEST    byte [BX+BUFFINFO.buf_flags],buf_dirty
33367      ;LB. if already dirty ;AN000;
33368 000058A5 7507      JNZ     short yesdirty10 ;LB. don't increment dirty count ;AN000;
33369 000058A7 E82013      call    INC_DIRTY_COUNT ;LB. ;AN000;
33370
33371      ;or     byte [bx+5],40h
33372 000058AA 804F0540      OR      byte [BX+BUFFINFO.buf_flags],buf_dirty
33373 yesdirty10:
33374      ;lea     si,[bx+16]
33375      ;lea     si,[bx+20] ; MSDOS 6.0
33376      ;lea     si,[bx+24] ; PCDOS 7.1; 25/02/2024
33377 000058AE 8D7718      LEA     SI,[BX+BUFINSIZ]
33378 000058B1 29F7      SUB     DI,SI ; Position in buffer
33379 ;M039
33380      ; MSDOS 3.3
33381      ;MOV     SI,DI
33382      ;MOV     DI,BX
33383      ;call    PLACEBUF
33384      ;cmp     si,[es:bp+2]
33385      ;CMP     SI,[ES:BP+DPB.SECTOR_SIZE] ; Written last byte?
33386      ;JB      short WBUFPLACED ; No, leave buf where it is
33387      ;call    PLACEHEAD ; Make it prime candidate for chucking
33388      ; even though it is MRU.
33389
33390 000058B3 16      ; 10/02/2024
33391 000058B4 1F      push    ss
33392      pop     ds
33393
33394      ; MSDOS 6.0
33395 000058B5 263B7E02      ;cmp     di,[es:bp+2]
33396 000058B9 7204      CMP     di,[ES:BP+DPB.SECTOR_SIZE] ; Written last byte?
33397      ;JB      short WBUFPLACED ; No, leave buf where it is
33398
33399      ; 10/02/2024
33400      ;MOV     [ss:BufferQueue],BX ; Make it prime candidate for
33401 000058BB 891E[6D00]      mov     [BufferQueue],bx ; chucking even though it is MRU.
33402 ;M039
33403
33404 WBUFPLACED:
33405 000058BF F8      CLC
33406      ; 10/02/2024
33407      ;push    ss
33408      ;pop     ds
33409 000058C0 C3      retn
33410
33411 ;Break <NEXTSEC -- Compute next sector to read or write>
33412 ;-----
33413 ;
33414 ; Procedure Name : NEXTSEC
33415 ;
33416 ; Compute the next sector to read or write
33417 ; ES:BP Points to DPB
33418 ;
33419 ;-----
33420
33421      ; 29/02/2024
33422      ; 26/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
33423
33424 NEXTSEC:
33425 000058C1 F606[7405]FF      test    byte [TRANS],0FFh ; -1
33426      ;JZ      short CLRET
33427      ; 29/02/2024
33428 000058C6 743D      JZ      short CLRET2
33429
33430 000058C8 A0[7305]      MOV     AL,[SECCLUSPOS]
33431 000058CB FEC0      INC     AL
33432      ;cmp     al,[es:bp+4]
33433 000058CD 263A4604      CMP     AL,[ES:BP+DPB.CLUSTER_MASK]
33434 000058D1 762E      JBE     short SAVPOS
33435
33436      ; 26/02/2024 (PCDOS 7.1 IBMDOS.COM)
33437      ;;;
33438 000058D3 8B1E[DE0A]      mov     bx,[CLUSNUM_HW]
33439 000058D7 891E[E80A]      mov     [CLUSTNUM_HW],bx
33440      ;;;
33441
33442 000058DB 8B1E[BC05]      MOV     BX,[CLUSNUM]
33443 000058DF E8D609      call    ISEOF
33444 000058E2 7322      JAE     short NONEXT
33445
33446 000058E4 E8FA09      call    UNPACK
33447      ;JC      short NONEXT
33448      ; 26/02/2024
33449 000058E7 721E      JC      short NONEXT2
33450 c lusgot:
33451      ; 26/02/2024 - Retro DOS v5.0
33452      ;;;
33453      ;push    di
33454      ;mov     di,[CCONTENT_HW]
33455      ;mov     [CLUSNUM_HW],di
33456      ;pop     di
33457 000058E9 FF36[EC0A]      push    word [CCONTENT_HW]
33458 000058ED 8F06[DE0A]      pop     word [CLUSNUM_HW]
33459      ;;;
33460 000058F1 893E[BC05]      MOV     [CLUSNUM],DI
33461 000058F5 FF06[BA05]      INC     word [LASTPOS]
33462      ; 26/02/2024 - Retro DOS v5.0
33463      ;;;
33464 000058F9 7504      JNZ     short clusgot1
33465 000058FB FF06[E20A]      inc     word [LASTPOS_HW]

```



```

33466 clusgot1:
33467     ;;;
33468     MOV     AL,0
33469 SAVPOS:
33470     MOV     [SECCLUSPOS],AL
33471 CLRET:
33472     CLC
33473 CLRET2:     ; 29/02/2024
33474     retn
33475 NONEXT:
33476     STC
33477 NONEXT2:    ; 26/02/2024
33478     retn
33479
33480 ;Break      <OPTIMIZE -- DO A USER DISK REQUEST WELL>
33481 ;-----
33482 ;
33483 ; Procedure Name : OPTIMIZE
33484 ;
33485 ; Inputs:
33486 ;     BX = Physical cluster
33487 ;     CX = No. of records
33488 ;     DL = sector within cluster
33489 ;     ES:BP = Base of drive parameters
33490 ;     [NEXTADD] = transfer address
33491 ; Outputs:
33492 ;     AX = No. of records remaining
33493 ;     BX = Transfer address
33494 ;     CX = No. of records to be transferred
33495 ;     DX = Physical sector address (LOW)
33496 ;     [HIGH_SECTOR] = Physical sector address (HIGH)
33497 ;     DI = Next cluster
33498 ;     [CLUSNUM] = Last cluster accessed
33499 ;     [NEXTADD] updated
33500 ;     Carry set if error (currently user FAILED to I 24)
33501 ;     ES:BP unchanged. Note that segment of transfer not set.
33502 ;
33503 ;-----
33504 ;
33505 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
33506 ; 27/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
33507
33508 OPTIMIZE:
33509     PUSH     DX
33510     ; 27/02/2024 (PCDOS 7.1 IBMDOS.COM)
33511     ;;;
33512     push     word [CLUSTNUM_HW]
33513     ;;;
33514     PUSH     BX
33515     ;mov     al,[es:bp+4]
33516     MOV     AL,[ES:BP+DPB.CLUSTER_MASK]
33517     INC     AL      ; Number of sectors per cluster
33518     MOV     AH,AL
33519     SUB     AL,DL      ; AL = Num of sectors left in first cluster
33520     MOV     DX,CX
33521     ;MOV     CX,0
33522     ; 25/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
33523     ; 16/12/2022
33524     xor     cx,cx      ; sub cx,cx
33525 OPTCLUS:
33526 ; AL has number of sectors available in current cluster
33527 ; AH has number of sectors available in next cluster
33528 ; BX has current physical cluster
33529 ; CX has number of sequential sectors found so far
33530 ; DX has number of sectors left to transfer
33531 ; ES:BP Points to DPB
33532 ; ES:SI has FAT pointer
33533
33534 do_norm3:
33535     call     UNPACK
33536     JC      short OP_ERR      ; ds=ss ; 27/02/2024
33537
33538 ;clusgot2:
33539     ADD     CL,AL
33540     ADC     CH,0
33541     CMP     CX,DX
33542     JAE     short BLKDON
33543     MOV     AL,AH
33544     INC     BX
33545     ; 27/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
33546     ;;;
33547     ; 28/06/2024
33548     push     di
33549     ;xor     di,di
33550     ;add     bx,1
33551     ;adc     [CLUSTNUM_HW],di
33552     ;
33553     inc     bx
33554     jnz     short clusgot2
33555     inc     word [CLUSTNUM_HW]
33556 clusgot2:
33557     mov     di,[CONTENT_HW]
33558     cmp     di,[CLUSTNUM_HW]
33559     ; 28/06/2024
33560     pop     di
33561     jne     short clusgot3
33562     cmp     di,bx
33563     je      short OPTCLUS
33564
33565 clusgot3:
33566     ;;;
33567     ;CMP     DI,BX
33568     ;JZ      short OPTCLUS
33569     ;DEC     BX
33570     ; 27/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
33571     ;;;
33572     ;sub     bx,1
33573     ;sbb     word [CLUSTNUM_HW],0
33574     ;
33575     dec     bx
33576     jns     short FINCLUS
33577     dec     word [CLUSTNUM_HW]
33578     ;;;
33579     ;push     bx
33580     ;add     [LASTPOS],bx
33581     ;mov     bx,[CLUSTNUM_HW]
33582     ;adc     [LASTPOS_HW],bx
33583     ;mov     [CLUSNUM_HW],bx
33584     ;pop     word [CLUSNUM]
33585     ;
33586     mov     [CLUSNUM],bx
33587     add     [LASTPOS],bx
33588
33589     0000594A 891E[BC05]
33590     0000594E 011E[BA05]

```

```

33590 00005952 A1[E80A]      mov     ax,[CLUSTNUM_HW]
33591 00005955 1106[E20A]      adc     [LASTPOS_HW],ax
33592 00005959 A3[DE0A]      mov     [CLUSNUM_HW],ax
33593                                     ;;;
33594 0000595C 29CA      SUB     DX,CX          ; Number of sectors still needed
33595 0000595E 52      PUSH    DX
33596 0000595F 89C8      MOV     AX,CX
33597                                     ;mul word[ES:BP+2]
33598 00005961 26F76602      MUL     word [ES:BP+DPB.SECTOR_SIZE]
33599                                     ; Number of sectors times sector size
33600 00005965 8B36[B805]      MOV     SI,[NEXTADD]
33601 00005969 01F0      ADD     AX,SI          ; Adjust by size of transfer
33602 0000596B A3[B805]      MOV     [NEXTADD],AX
33603 0000596E 58      POP     AX          ; Number of sectors still needed
33604 0000596F 5A      POP     DX          ; Starting cluster
33605                                     ; 27/02/2024 (PCDOS 7.1 IBMDOS.COM)
33606                                     ;SUB BX,DX          ; Number of new clusters accessed
33607                                     ;ADD [LASTPOS],BX
33608                                     ;;;
33609 00005970 5B      POP     bx          ; Starting cluster, hw
33610 00005971 891E[E80A]      mov     [CLUSTNUM_HW],bx
33611 00005975 2916[BA05]      sub     [LASTPOS],dx
33612 00005979 191E[E20A]      sbb     [LASTPOS_HW],bx
33613                                     ;;;
33614 0000597D 5B      POP     BX          ; BL = sector position within cluster
33615 0000597E E81600      call    FIGREC
33616 00005981 89F3      MOV     BX,SI
33617                                     ; 24/09/2023
33618                                     ; cf=0 (at the return of FIGREC)
33619                                     ;CLC
33620 00005983 C3      retn
33621 OP_ERR:
33622                                     ;ADD SP,4
33623                                     ; 27/02/2024 (PCDOS 7.1 IBMDOS.COM)
33624                                     ;;;
33625 00005984 83C406      add     sp,6
33626                                     ;;;
33627 00005987 F9      STC
33628 00005988 C3      retn
33629 BLKDON:
33630 00005989 29D1      SUB     CX,DX          ; Number of sectors in cluster we don't want
33631 0000598B 28CC      SUB     AH,CL          ; Number of sectors in cluster we accepted
33632 0000598D FECC      DEC     AH          ; Adjust to mean position within cluster
33633 0000598F 8826[7305]      MOV     [SECCLUSPOS],AH
33634 00005993 89D1      MOV     CX,DX          ; Anyway, make the total equal to the request
33635 00005995 EBB3      JMP     SHORT FINCLUS
33636
33637 ;Break <FIGREC -- Figure sector in allocation unit>
33638 -----
33639 ;
33640 ; Procedure Name : FIGREC
33641 ;
33642 ; Inputs:
33643 ; DX = Physical cluster number
33644 ; BL = Sector position within cluster
33645 ; ES:BP = Base of drive parameters
33646 ; Outputs:
33647 ; DX = physical sector number (LOW)
33648 ; [HIGH_SECTOR] Physical sector address (HIGH)
33649 ; No other registers affected.
33650 ;
33651 -----
33652
33653 ; 10/06/2019
33654 ; 20/05/2019 - Retro DOS v4.0
33655 ; DOSCODE:8D96h (MSDOS 6.21, MSDOS.SYS)
33656 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
33657 ; DOSCODE:8D5Bh (MSDOS 5.0, MSDOS.SYS)
33658
33659 ; 27/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
33660 ; DOSCODE:9A89h (PCDOS 7.1, IBMDOS.COM)
33661 FIGREC:
33662 00005997 51      PUSH    CX
33663                                     ; 27/02/2024
33664                                     ;;;
33665 00005998 50      push    ax
33666 00005999 31C9      xor     cx,cx
33667                                     ;;;
33668 ;mov cl,[es:bp+5]
33669 0000599B 268A4E05      MOV     CL,[ES:BP+DPB.CLUSTER_SHIFT]
33670                                     ;DEC DX
33671                                     ;DEC DX
33672                                     ; 27/02/2024
33673                                     ;;;
33674 ;mov ax,[ss:CLUSTNUM_HW]
33675 ; ds = ss ; Retro DOS v5.0
33676 0000599F A1[E80A]      mov     ax,[CLUSTNUM_HW]
33677 000059A2 83EA02      sub     dx,2
33678 000059A5 83D800      sbb     ax,0
33679 000059A8 E307      jcxz    noshift
33680                                     ;;;
33681
33682 ; 27/02/2024
33683 %if 0
33684 ; MSDOS 3.3
33685 ;SHL DX,CL
33686
33687 ;hkn; SS override HIGH_SECTOR
33688 ; MSDOS 6.0
33689 MOV     word [SS:HIGH_SECTOR],0          ; F.C. >32mb
33690                                     ; 24/09/2023
33691 xor     ch,ch          ; F.C. >32mb
33692 OR     CL,CL          ; F.C. >32mb
33693 JZ     short noshift   ; F.C. >32mb
33694                                     ; 27/02/2024
33695 ;XOR CH,CH          ; F.C. >32mb
33696 rotleft:          ; F.C. >32mb
33697 CLC          ; F.C. >32mb
33698 RCL     DX,1          ; F.C. >32mb
33699                                     ; 10/06/2019
33700 RCL     word [ss:HIGH_SECTOR],1          ; F.C. >32mb
33701 LOOP    rotleft          ; F.C. >32mb
33702 %else
33703 ; 27/02/2024 - Retro DOS v5.0
33704 ; (PCDOS 7.1 IBMDOS.COM)
33705 rotleft:
33706 000059AA F8      clc
33707 000059AB D1D2      rcl     dx,1
33708 000059AD D1D0      rcl     ax,1
33709 000059AF E2F9      loop   rotleft
33710 %endif
33711
33712 noshift:
33713 ; MSDOS 3.3 & MSDOS 6.0

```



```

33714 000059B1 08DA      OR      DL,BL
33715
33716      ; 27/02/2024 - Retro DOS v5.0
33717      ;;;
33718      ;cmp      word [es:bp+0Fh],0
33719 000059B3 26394E0F    cmp      word [es:bp+DPB.FAT_SIZE],cx ; 0
33720 000059B7 750A      jnz      short noshift1 ; not FAT32
33721      ;add      dx,[es:bp+29h]
33722 000059B9 26035629    add      dx,[es:bp+DPB.FCLUS_FSECTOR]
33723      ;adc      ax,[es:bp+2Bh]
33724 000059BD 2613462B    adc      ax,[es:bp+DPB.FCLUS_FSECTOR+2]
33725 000059C1 EB06      jmp      short noshift2
33726 noshift1:
33727      ;;;
33728
33729      ;add      dx,[es:bp+0Bh]
33730 000059C3 2603560B    ADD      DX,[ES:BP+DPB.FIRST_SECTOR]
33731      ; MSDOS 6.0
33732      ; 10/06/2019
33733      ;ADC      word [ss:HIGH_SECTOR],0          ;F.C. >32mb
33734      ; 24/09/2023
33735      ; cx=0
33736      ;ADC      word [ss:HIGH_SECTOR],cx ; 0
33737      ; 27/02/2024 - Retro DOS v5.0
33738      ;;;
33739 000059C7 11C8      adc      ax,cx ; adc ax,0
33740 noshift2:
33741      ;mov      [ss:HIGH_SECTOR],ax
33742 000059C9 A3[0706]    mov      [HIGH_SECTOR],ax
33743      ;
33744 000059CC 58      pop      ax
33745      ;;;
33746
33747      ; MSDOS 3.3 & MSDOS 6.0
33748 000059CD 59      POP      CX
33749 figrec_retn:
33750 000059CE C3      retn
33751
33752      ; 20/05/2019 - Retro DOS v4.0
33753      ; DOSCODE:8DC2h (MSDOS 6.21, MSDOS.SYS)
33754
33755      ; 30/07/2018 - Retro DOS v3.0
33756      ; IBMDOS.COM (MSDOS 3.3, 1987) - Offset
33757
33758      ;Break <ALLOCATE -- Assign disk space>
33759      -----
33760
33761      ; Procedure Name : ALLOCATE - Allocate Disk Space
33762      ;
33763      ; ALLOCATE is called to allocate disk clusters. The new clusters are
33764      ; FAT-chained onto the end of the existing file.
33765      ;
33766      ; The DPB contains the cluster # of the last free cluster allocated
33767      ; (dpb_next_free). We start at this cluster and scan towards higher
33768      ; numbered clusters, looking for the necessary free blocks.
33769      ;
33770      ; Once again, fancy terminology gets in the way of correct coding. When
33771      ; using next_free, start scanning AT THAT POINT and not the one following it.
33772      ; This fixes the boundary condition bug when only free = next_free = 2.
33773      ;
33774      ; If we get to the end of the disk without satisfaction:
33775      ;
33776      ; if (dpb_next_free == 2) then we've scanned the whole disk.
33777      ; return (insufficient_disk_space)
33778      ; ELSE
33779      ;     dpb_next_free = 2; start scan over from the beginning.
33780      ;
33781      ; Note that there is no multitasking interlock. There is no race when
33782      ; examining the entrys in an in-core FAT block since there will be no
33783      ; context switch. When UNPACK context switches while waiting for a FAT read
33784      ; we are done with any in-core FAT blocks, so again there is no race. The
33785      ; only special concern is that V2 and V3 MSDOS left the last allocated
33786      ; cluster as "00"; marking it EOF only when the entire alloc request was
33787      ; satisfied. We can't allow another activation to think this cluster is
33788      ; free, so we give it a special temporary mark to show that it is, indeed,
33789      ; allocated.
33790      ;
33791      ; Note that when we run out of space this algorithm will scan from
33792      ; dpb_next_free to the end, then scan from cluster 2 through the end,
33793      ; redundantly scanning the later part of the disk. This only happens when
33794      ; we run out of space, so sue me.
33795      ;
33796      ;-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
33797      ; C A V E A T P A T T E R S O N
33798      ;
33799      ; The use of FATBYT and RESTFATBYT is somewhat mysterious. Here is the
33800      ; explanation:
33801      ;
33802      ; In the NUL file case (sf_firclus currently 0) ALLOCATE is called with
33803      ; entry BX = 0. What needs to be done in this case is to stuff the cluster
33804      ; number of the first cluster allocated in sf_firclus when the ALLOCATE is
33805      ; complete. THIS VALUE IS SAVED TEMPORARILY IN CLUSTER 0, HENCE THE CURRENT
33806      ; VALUE IN CLUSTER 0 MUST BE SAVED AND RESTORED. This is a side effect of
33807      ; the fact that PACK and UNPACK don't treat requests for clusters 0 and 1 as
33808      ; errors. This "stuff" is done by the call to PACK which is right before
33809      ; the
33810      ;     LOOP findfre ; alloc more if needed
33811      ; instruction when the first cluster is allocated to the nul file. The
33812      ; value is recalled from cluster 0 and stored at sf_firclus at ads4:
33813      ;
33814      ; This method is obviously useless (because it is non-reentrant) for
33815      ; multitasking, and will have to be changed. Storing the required value on
33816      ; the stack is recommended. Setting sf_firclus at the PACK of cluster 0
33817      ; (instead of actually doing the PACK) is BAD because it doesn't handle
33818      ; problems with INT 24 well.
33819      ;
33820      ; C A V E A T P A T T E R S O N
33821      ;-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
33822      ;
33823      ; ENTRY BX = Last cluster of file (0 if null file)
33824      ; CX = No. of clusters to allocate
33825      ; ES:BP = Base of drive parameters
33826      ; [THISSFT] = Points to SFT
33827      ;
33828      ; EXIT 'C' set if insufficient space
33829      ; [FAILERR] can be tested to see the reason for failure
33830      ; CX = max. no. of clusters that could be added to file
33831      ; 'C' clear if space allocated
33832      ; BX = First cluster allocated
33833      ; FAT is fully updated
33834      ; sf_FIRCLUS field of SFT set if file was null
33835      ;
33836      ; USES ALL but SI, BP
33837

```

```

33838 ;callmagic proc near
33839 ;       push    ds                ;push segment of routine
33840 ;       push    Offset MagicPatch ;push offset for routine
33841 ;       retf      ;simulate jmp far
33842 ;       ;                       ;far return address is on
33843 ;       ;                       ;stack, so far return from
33844 ;       ;                       ;call will return this routine
33845 ;callmagic endp
33846
33847 ; 27/02/2024 - Retro DOS v5.0
33848 ;PCDOS 7.1 IBMDOS.COM - DOSCODE:9AC5h
33849
33850 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:8DC2h)
33851 ; (Windows ME IO.SYS - BIOSCODE:0C078h)
33852
33853 ; 25/09/2023
33854 %if 1 ; 27/02/2024
33855 callmagic:
33856     push    ds
33857     push    word [ss:OffsetMagicPatch]
33858     retf
33859 %endif
33860
33861 ; 10/02/2024 - Retro DOS v5.0
33862 %if 1
33863 ALLOCATE_X:
33864     mov     [CCOUNT_HW],ax
33865     jmp     short ALLOCATE
33866 %endif
33867
33868 ; 27/02/2024 - Retro DOS v5.0
33869 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:9ACFh
33870
33871 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:8DC9h)
33872 ; (Windows ME IO.SYS - BIOSCODE:0C07Fh)
33873
33874 ALLOCATE:
33875 ; 10/09/2018
33876 ;BEGIN MAGICDRV MODIFICATIONS
33877 ;
33878 ;7/5/92 scottq
33879 ;
33880 ;This is the disk compression patch location which allows
33881 ;the disk compression software to fail allocations if the
33882 ;FAT would allow allocation, but the free space for compressed
33883 ;data would not.
33884 ;
33885 ;;; call far ptr MAGICPATCH
33886 ;;; we cannot do a far call since we cannot have fix-ups[romdos,hidos],
33887 ;;; but we do know the segment and offset of the routine
33888 ;;; so simulate a far call to dosdata:magicpatch
33889 ;;; note dosassume above, so DS -> dosdata
33890
33891 ; MSDOS 6.0
33892 ;clc
33893 ;;; ;clear carry so we fall through
33894 ;push cs ;if no patch is present
33895 ;call callmagic ;push segment for far return
33896 ;jnc short Regular_Allocate_Path ;this is a near call
33897 ;jmp Disk_Full_Return
33898
33899 ; 27/02/2024 - Retro DOS v5.0
33900 ; DOSCODE:9ACFh (PCDOS 7.1, IBMDOS.COM)
33901
33902 ; 25/09/2023
33903 %if 1 ; 27/02/2024 - Retro DOS v5.0
33904     clc ; COUNT_HW:CX = Number of clusters to allocate
33905     push cs
33906     call callmagic
33907     jnc short Regular_Allocate_Path
33908     jmp Disk_Full_Return
33909 Regular_Allocate_Path:
33910 %endif
33911
33912 ; 20/05/2019 - Retro DOS v4.0
33913 ;END MAGICDRV MODIFICATIONS
33914
33915 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
33916 ; DOSCODE:8D87h (MSDOS 5.0, MSDOS.SYS)
33917
33918 ; 27/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
33919 ; DOSCODE:9AD6h (PCDOS 7.1, IBMDOS.COM)
33920
33921     PUSH    BX ; save (bx)
33922     XOR     BX,BX
33923
33924 ; 27/02/2024
33925 ;;;
33926     push    word [CLUSTNUM_HW]
33927     mov     [CLUSTNUM_HW],bx ; cluster 0
33928     ;;;
33929
33930     call    UNPACK
33931     MOV     [FATBYT],DI ; save correct cluster 0 value
33932
33933 ; 27/02/2024
33934 ;;; ; CCONTENT_HW:DI = content of FAT (for cluster 0)
33935     mov     bx,[CCONTENT_HW]
33936     mov     [FATBYT_HW],bx
33937     pop     word [CLUSTNUM_HW]
33938     ;;;
33939
33940     POP     BX
33941     jc     short figrec_retn ; abort if error [INTERR?]
33942
33943 ; 27/02/2024
33944 %if 0
33945     PUSH    CX
33946     PUSH    BX
33947
33948     MOV     DX,BX
33949     ;mov     bx,[es:bp+1Ch] ; MSDOS 3.3
33950     ;mov     bx,[es:bp+1Dh] ; MSDOS 6.0
33951     mov     bx,[ES:BP+DPB.NEXT_FREE]
33952     cmp     bx,2
33953     ja     short FINDFRE
33954
33955 ; couldn't find enough free space beyond dpb_next_free, or dpb_next_free is
33956 ; <2 or >dpb_max_clus. Reset it and restart the scan
33957
33958 ads1:
33959     ;mov     word [es:bp+1Ch],2 ; MSDOS 3.3
33960     ;mov     word [es:bp+1Dh],2 ; MSDOS 6.0
33961     mov     word [ES:BP+DPB.NEXT_FREE],2

```

```

33962             mov     bx,1                ; Counter next instruction so first
33963                                     ; cluster examined is 2
33964 %else
33965 ; 27/02/2024 - Retro DOS v5.0
33966 ; (PCDOS 7.1 IBMDOS.COM)
33967 ;;;
33968 00005A04 FF36[F00A] push     word [CCOUNT_HW]
33969 00005A08 51          push     cx
33970 00005A09 FF36[E80A] push     word [CLUSTNUM_HW]
33971 00005A0D 53          push     bx
33972
33973 00005A0E 89DA        mov     dx,bx
33974
33975 00005A10 31DB        xor     bx,bx ; 0
33976             ;cmp     [es:bp+0Fh],bx ; 0
33977 00005A12 26395E0F    cmp     [es:bp+DPB.FAT_SIZE],bx ; 0
33978 00005A16 871E[E80A] xchg     bx,[CLUSTNUM_HW]
33979 00005A1A 891E[EA0A] mov     [CLUSDATA_HW],bx
33980             ;mov     bx,[es:bp+1Dh]
33981 00005A1E 268B5E1D    mov     bx,[es:bp+DPB.NEXT_FREE]
33982 00005A22 7510        jnz     short ads1 ; not FAT32
33983
33984             ;mov     bx,[es:bp+3Bh]
33985 00005A24 268B5E3B    mov     bx,[es:bp+DPB.FAT32_NXTFREE+2]
33986 00005A28 891E[E80A] mov     [CLUSTNUM_HW],bx
33987             ;cmp     bx,0
33988 00005A2C 09DB        or     bx,bx
33989             ;mov     bx,[es:bp+39h]
33990 00005A2E 268B5E39    mov     bx,[es:bp+DPB.FAT32_NXTFREE]
33991 00005A32 7526        jnz     short FINDFRE
33992 ads1:
33993 00005A34 83FB02      cmp     bx,2
33994 00005A37 7721        ja     short FINDFRE
33995 ads2:
33996 00005A39 31DB        xor     bx,bx ; 0
33997             ;cmp     [es:bp+0Fh],bx
33998 00005A3B 26395E0F    cmp     [es:bp+DPB.FAT_SIZE],bx ; 0
33999 00005A3F 750A        jnz     short ads3 ; not FAT32
34000
34001             ;mov     word [es:bp+39h],2
34002 00005A41 26C746390200 mov     word [es:bp+DPB.FAT32_NXTFREE],2
34003             ;mov     [es:bp+3Bh],bx
34004 00005A47 26895E3B    mov     [es:bp+DPB.FAT32_NXTFREE+2],bx ; 0
34005 ads3:
34006 00005A4B 891E[E80A] mov     [CLUSTNUM_HW],bx ; 0
34007             ;mov     word [es:bp+1Dh],2
34008 00005A4F 26C7461D0200 mov     word [es:bp+DPB.NEXT_FREE],2
34009 00005A55 43          inc     bx ; 1
34010             ;or     [es:bp+18h],b1
34011 00005A56 26085E18    or     [es:bp+DPB.FIRST_ACCESS],b1 ; 1
34012             ;;;
34013 %endif
34014
34015 ; Scanning both forwards and backwards for a free cluster
34016 ;
34017 ; (BX) = forwards scan pointer
34018 ; (CX) = clusters remaining to be allocated
34019 ; (DX) = current last cluster in file
34020 ; (TOS) = last cluster of file
34021
34022 FINDFRE:
34023 %if 0
34024             INC     BX
34025             ;cmp     bx,[es:bp+0Dh]
34026             CMP     BX,[ES:BP+DPB.MAX_CLUSTER]
34027             ja     short ads7 ; at end of disk
34028             call    UNPACK ; check out this cluster
34029             jc     short ads4 ; FAT error [INTERR?]
34030             jnz     short FINDFRE ; not free, keep on truckin
34031
34032 ; Have found a free cluster. Chain it to the file
34033 ;
34034 ; (BX) = found free cluster #
34035 ; (DX) = current last cluster in file
34036
34037             ;mov     [es:bp+1Ch],bx
34038             ;mov     [es:bp+1Dh],bx ; MSDOS 6.0
34039             mov     [ES:BP+DPB.NEXT_FREE],bx ; next time start search here
34040             xchg     ax,dx ; save (dx) in ax
34041             mov     dx,1 ; mark this free guy as "1"
34042             call    PACK ; set special "temporary" mark
34043             jc     short ads4 ; FAT error [INTERR?]
34044             ;cmp     word [es:bp+1Eh],-1
34045             ;cmp     word [es:bp+1Fh],-1 ; MSDOS 6.0
34046             CMP     word [ES:BP+DPB.FREE_CNT],-1 ; Free count valid?
34047             JZ     short NO_ALLOC ; No
34048             ;dec     word [es:bp+1Eh]
34049             ;dec     word [es:bp+1Fh] ; MSDOS 6.0
34050             DEC     word [ES:BP+DPB.FREE_CNT] ; Reduce free count by 1
34051 NO_ALLOC:
34052             xchg     ax,dx ; (dx) = current last cluster in file
34053             XCHG     BX,DX
34054             MOV     AX,DX
34055             call    PACK ; link free cluster onto file
34056             ; CAVEAT.. On Nul file, first pass stuffs
34057             ; cluster 0 with FIRCLUS value.
34058             jc     short ads4 ; FAT error [INTERR?]
34059             xchg     BX,AX ; (BX) = last one we looked at
34060             mov     dx,bx ; (dx) = current end of file
34061             LOOP    FINDFRE ; alloc more if needed
34062 %else
34063 ; 27/02/2024 - Retro DOS v5.0
34064 ; (PCDOS 7.1 IBMDOS.COM)
34065 ;;;
34066             ;add     bx,1
34067             ;adc     word [CLUSTNUM_HW],0
34068 00005A5A 43          inc     bx
34069 00005A5B 7504        jnz     short FINDFRE2
34070 00005A5D FF06[E80A] inc     word [CLUSTNUM_HW]
34071 FINDFRE2:
34072             ;cmp     word [es:bp+0Fh],0
34073 00005A61 26837E0F00    cmp     word [es:bp+DPB.FAT_SIZE],0
34074 00005A66 7512        jnz     short ads4 ; not FAT32
34075             ; FAT32
34076 00005A68 53          push     bx
34077 00005A69 8B1E[E80A] mov     bx,[CLUSTNUM_HW]
34078             ;cmp     bx,[es:bp+2Fh]
34079 00005A6D 263B5E2F    cmp     bx,[es:bp+DPB.LAST_CLUSTER+2]
34080 00005A71 5B          pop     bx
34081 00005A72 750A        jne     short ads5
34082             ;cmp     bx,[es:bp+2Dh]
34083 00005A74 263B5E2D    cmp     bx,[es:bp+DPB.LAST_CLUSTER]
34084 00005A78 EB04        jmp     short ads5
34085 ads4:

```

```

34086      ;cmp     bx,[es:bp+0Dh]
34087 00005A7A 263B5E0D      cmp     bx,[es:bp+DPB.MAX_CLUSTER]
34088      ads5:
34089 00005A7E 7603      jbe     short ads6 ; jna short ads6
34090 00005A80 E9ED00      jmp     ads15
34091      ads6:
34092      ; 27/02/2024 - Retro DOS v5.0
34093      ;push    cx
34094      ;push    word [CCOUNT_HW]
34095      ;push    bx
34096      ;push    word [CLUSTNUM_HW]
34097      ;push    dx
34098      ;push    word [CLUSDATA_HW]
34099
34100 00005A83 E85B08      call    UNPACK
34101
34102      ; 27/02/2024 - Retro DOS v5.0
34103      ;pop     word [CLUSDATA_HW]
34104      ;pop     dx
34105      ;pop     word [CLUSTNUM_HW]
34106      ;pop     bx
34107      ;pop     word [CCOUNT_HW]
34108      ;pop     cx
34109
34110 00005A86 7303      jnc     short ads7
34111
34112 00005A88 E9A200      jmp     ads13
34113      ads7:
34114 00005A8B 75CD      jnz     short FINDFRE
34115
34116      ;mov     [es:bp+1Dh],bx
34117 00005A8D 26895E1D      mov     [es:bp+DPB.NEXT_FREE],bx
34118
34119      ;cmp     word [es:bp+0Fh],0
34120 00005A91 26837E0F00      cmp     word [es:bp+DPB.FAT_SIZE],0
34121 00005A96 750E      jnz     short ads8 ; not FAT32
34122      ; FAT32
34123 00005A98 53      push    bx
34124 00005A99 8B1E[E80A]      mov     bx,[CLUSTNUM_HW]
34125      ;mov     [es:bp+3Bh],bx
34126 00005A9D 26895E3B      mov     [es:bp+DPB.FAT32_NXTFREE+2],bx
34127 00005AA1 5B      pop     bx
34128      ;mov     [es:bp+39h],bx
34129 00005AA2 26895E39      mov     [es:bp+DPB.FAT32_NXTFREE],bx
34130      ads8:
34131      ;or      byte [es:bp+18h],1
34132 00005AA6 26804E1801      or      byte [es:bp+DPB.FIRST_ACCESS],1
34133
34134      ; 27/02/2024 - Retro DOS v5.0
34135      ;push    cx
34136      ;push    word [CCOUNT_HW]
34137
34138 00005AAB 53      push    bx
34139 00005AAC FF36[E80A]      push    word [CLUSTNUM_HW]
34140
34141 00005AB0 52      push    dx
34142 00005AB1 FF36[EA0A]      push    word [CLUSDATA_HW]
34143
34144 00005AB5 31D2      xor     dx,dx ; 0
34145 00005AB7 8916[EA0A]      mov     [CLUSDATA_HW],dx ; 0
34146 00005ABB 42      inc     dx ; 1 ; mark this free guy as "1"
34147 00005ABC E8D108      call    PACK ; set special "temporary" mark
34148
34149 00005ABF 8F06[E80A]      pop     word [CLUSTNUM_HW]
34150 00005AC3 5B      pop     bx
34151 00005AC4 8F06[EA0A]      pop     word [CLUSDATA_HW]
34152 00005AC8 5A      pop     dx
34153
34154      ; 27/02/2024 - Retro DOS v5.0
34155      ;pop     word [CCOUNT_HW]
34156      ;pop     cx
34157
34158 00005AC9 7262      jc      short ads13
34159
34160 00005ACB 50      push    ax
34161 00005ACC 31C0      xor     ax,ax ; 0
34162      ;cmp     [es:bp+0Fh],ax ; 0
34163 00005ACE 2639460F      cmp     [es:bp+DPB.FAT_SIZE],ax ; 0
34164 00005AD2 7517      jnz     short ads10 ; not FAT32
34165      ; FAT32
34166 00005AD4 48      dec     ax ; 0FFFFh ; -1
34167      ;cmp     [es:bp+21h],ax
34168 00005AD5 26394621      cmp     [es:bp+DPB.FREE_CNT+2],ax ; 0FFFFh
34169 00005AD9 7506      jnz     short ads9
34170      ;cmp     [es:bp+1Fh],ax
34171 00005ADB 2639461F      cmp     [es:bp+DPB.FREE_CNT],ax ; 0FFFFh
34172      ads9:
34173 00005ADF 7415      jz      short NO_ALLOC ; Free count not valid
34174      ads9:
34175      ; Reduce free count by 1
34176      ;add     [es:bp+1Fh],ax
34177 00005AE1 2601461F      add     [es:bp+DPB.FREE_CNT],ax ; -1
34178      ;adc     [es:bp+21h],ax
34179 00005AE5 26114621      adc     [es:bp+DPB.FREE_CNT+2],ax ; -1
34180 00005AE9 EB0B      jmp     short NO_ALLOC
34181      ads10:
34182 00005AEB 48      dec     ax ; 0FFFFh ; -1
34183      ;cmp     [es:bp+1Fh],ax
34184 00005AEC 2639461F      cmp     [es:bp+DPB.FREE_CNT],ax ; 0FFFFh
34185 00005AF0 7404      jz      short NO_ALLOC ; Free count not valid
34186      ;dec     word [es:bp+1Fh]
34187 00005AF2 26FF4E1F      dec     word [es:bp+DPB.FREE_CNT] ; Reduce free count by 1
34188      NO_ALLOC:
34189 00005AF6 58      pop     ax
34190
34191      ; 27/02/2024 - Retro DOS v5.0
34192      ;push    cx
34193      ;push    word [CCOUNT_HW]
34194
34195 00005AF7 52      push    dx
34196 00005AF8 FF36[EA0A]      push    word [CLUSDATA_HW]
34197
34198 00005AFC E89108      call    PACK
34199
34200 00005AFF 8F06[E80A]      pop     word [CLUSTNUM_HW]
34201 00005B03 5B      pop     bx
34202
34203      ; 27/02/2024 - Retro DOS v5.0
34204      ;pop     word [CCOUNT_HW]
34205      ;pop     cx
34206
34207 00005B04 7227      jc      short ads13
34208
34209 00005B06 8B16[E80A]      mov     dx,[CLUSTNUM_HW]

```

```

34210 00005B0A 8916[EA0A]      mov     [CLUSDATA_HW],dx
34211 00005B0E 89DA      mov     dx,bx
34212
34213 00005B10 83E901      sub     cx,1
34214 00005B13 831E[F00A]00  sbb     word [CCOUNT_HW],0
34215
34216 00005B18 3B0E[F00A]    cmp     cx,[CCOUNT_HW]
34217 00005B1C 7502      jne     short ads11
34218 00005B1E E303      jcxz    ads12
34219
34220 00005B20 E937FF  ads11:    jmp     FINDFRE
34221      ;;;
34222 %endif
34223
34224
34225 ; We've successfully extended the file. Clean up and exit
34226 ;
34227 ; (BX) = last cluster in file
34228
34229 ads12:      ; 27/02/2024
34230 00005B23 BAFFFF      MOV     DX,0FFFFH
34231      ;;;
34232 00005B26 8916[EA0A]    mov     [CLUSDATA_HW],dx
34233      ;;;
34234 00005B2A E86308      call    PACK          ; mark last cluster EOF
34235
34236 ; Note that FAT errors jump here to clean the stack and exit. This saves us
34237 ; 2 whole bytes. Hope its worth it...
34238 ;
34239 ; 'C' set if error
34240 ; calling (BX) and (CX) pushed on stack
34241
34242 ; 27/02/2024
34243 %if 0
34244
34245 ads4:
34246     POP     BX
34247     POP     CX          ; Don't need this stuff since we're successful
34248     jc      short figrec_retn
34249     call    UNPACK      ; Get first cluster allocated for return
34250     ; CAVEAT... In nul file case, UNPACKs cluster 0.
34251     jc      short figrec_retn
34252     call    RESTFATBYT  ; Restore correct cluster 0 value
34253     jc      short figrec_retn
34254     xchg     BX,DI      ; (DI) = last cluster in file upon our entry
34255     OR      DI,DI      ; clear 'C'
34256     jnz     short figrec_retn ; we were extending an existing file
34257 %else
34258 ; 27/02/2024 - Retro DOS v5.0
34259 ; (PCDOS 7.1 IBMDOS.COM)
34260 ;;;
34261 ads13:
34262 00005B2D 5B      pop     bx
34263 00005B2E 8F06[E80A]  pop     word [CLUSTNUM_HW]
34264 00005B32 59      pop     cx
34265 00005B33 8F06[F00A]  pop     word [CCOUNT_HW]
34266 00005B37 7301      jnc     short ads14
34267
34268 ads_ret:
34269     retn
34270 ads14:
34271 00005B3A E8A407      call    UNPACK      ; Get first cluster allocated for return
34272 00005B3D 72FA      jc      short ads_ret
34273 00005B3F E87200      call    RESTFATBYT  ; Restore correct cluster 0 value
34274 00005B42 72F5      jc      short ads_ret
34275 00005B44 50      push    ax
34276 00005B45 A1[EC0A]    mov     ax,[CCONTENT_HW]
34277 00005B48 8706[E80A]  xchg     ax,[CLUSTNUM_HW]
34278 00005B4C 87DF      xchg     bx,di      ; CLUSTNUM_HW:DI = last cluster in file upon our entry
34279 00005B4E 09F8      or      ax,di
34280 00005B51 75E6      pop     ax
34281      jnz     short ads_ret ; we were extending an existing file
34282      ;;;
34283 %endif
34284
34285 ; We were doing the first allocation for a new file. Update the SFT cluster
34286 ; info
34287 dofastk:
34288 ; 27/02/2024
34289 %if 0
34290 ; 20/05/2019
34291 ; MSDOS 6.0
34292 ; push dx ; * MSDOS 6.0
34293 ; mov dl,[es:bp+0]
34294 ; MOV DL,[ES:BP+DPB.DRIVE] ; get drive #
34295 ; mov dl,[es:bp]
34296
34297 ; 25/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
34298 ; DOSCODE:8DF9h (MSDOS 5.0, MSDOS.SYS)
34299
34300 ; 16/12/2022
34301 ; push dx ; *
34302 ; mov dl,[ES:BP+DPB.DRIVE]
34303 ; 15/12/2022
34304 ; mov dl,[es:bp]
34305
34306 ; MSDOS 3.3 & MSDOS 6.0
34307 PUSH     ES
34308 LES      DI,[THISSFT]
34309 ; mov [es:di+0Bh],bx
34310 MOV      [ES:DI+SF_ENTRY.sf_firclus],BX
34311 ; mov [es:di+1Bh],bx ; MSDOS 3.3
34312 ; mov [es:di+35h],bx ; MSDOS 6.0
34313 MOV      [ES:DI+SF_ENTRY.sf_1stclus],BX
34314 POP      ES
34315 ; retn
34316
34317 ; pop dx ; * MSDOS 6.0
34318
34319 ; 16/12/2022
34320 ; 25/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
34321 ; pop dx ; *
34322 %else
34323 ; 27/02/2024 - Retro DOS v5.0
34324 ; (PCDOS 7.1 IBMDOS.COM)
34325 ;;;
34326 00005B53 52      push    dx
34327 00005B54 06      push    es
34328 00005B55 C43E[9E05]  les     di,[THISSFT]
34329 00005B59 8B16[E80A]  mov     dx,[CLUSTNUM_HW]
34330 00005B5D 26895D2B  mov     [es:di+2Bh],bx ; [es:di+SF_ENTRYT.sf_chain]
34331 00005B61 2689552D  mov     [es:di+2Dh],dx ; [es:di+SF_ENTRYT.sf_chain+2]
34332      ; 32 bit first cluster, 1w
34333 00005B65 26895D35  mov     [es:di+35h],bx ; [es:di+SF_ENTRYT.sf_1stclus]

```

```

34334                                ; 32 bit last cluster, lw
34335 00005B69 26895537      mov     [es:di+37h],dx ; [es:di+SF_ENTRYT.sf_lstclus+2]
34336                                ; 32 bit last cluster, hw
34337 00005B6D 07           pop     es
34338 00005B6E 5A           pop     dx
34339                        ;;;
34340 %endif
34341
34342 00005B6F C3           retn
34343
34344 ;** we're at the end of the disk, and not satisfied. See if we've scanned ALL
34345 ; of the disk...
34346
34347 ; 27/02/2024
34348 %if 0
34349 ads7:
34350     ;cmp     word [es:bp+1Dh],2
34351     cmp     word [ES:BP+DPB.NEXT_FREE],2
34352     jnz     short ads1 ; start scan from front of disk
34353 %else
34354     ; 27/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
34355     ;;;
34356 ads15:
34357     ;cmp     word [es:bp+0Fh],0
34358     cmp     word [es:bp+DPB.FAT_SIZE],0
34359     jz      short ads16 ; FAT32
34360     ; FAT
34361     ;cmp     word [es:bp+1Dh],2
34362     cmp     word [es:bp+DPB.NEXT_FREE],2
34363     jmp     short ads17
34364 ads16:
34365     ;cmp     word [es:bp+3Bh],0
34366     cmp     word [es:bp+DPB.FAT32_NXTFREE+2],0
34367     ;jnz     short ads17
34368     jnz     short ads19 ; 27/02/2024
34369     ;cmp     word [es:bp+39h],2
34370     cmp     word [es:bp+DPB.FAT32_NXTFREE],2
34371 ads17:
34372     jz      short ads18
34373 ads19:
34374     jmp     ads2 ; start scan from front of disk
34375     ;;;
34376 %endif
34377
34378 ; Sorry, we've gone over the whole disk, with insufficient luck. Lets give
34379 ; the space back to the free list and tell the caller how much he could have
34380 ; had. We have to make sure we remove the "special mark" we put on the last
34381 ; cluster we were able to allocate, so it doesn't become orphaned.
34382 ;
34383 ; (CX) = clusters remaining to be allocated
34384 ; (TOS) = last cluster of file (before call to ALLOCATE)
34385 ; (TOS+1) = # of clusters wanted to allocate
34386
34387 ads18:                                ; 27/02/2024
34388 00005B8F 5B      POP     BX ; (BX) = last cluster of file
34389 00005B90 BAF0FF MOV     DX,0FFFFH
34390
34391 ; 27/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
34392 ;;;
34393 00005B93 8916[EA0A] mov     [CLUSDATA_HW],dx
34394 00005B97 8F06[E80A] pop     word [CLUSTNUM_HW]
34395 ;;;
34396
34397 00005B9B E83800 call    RELBLKS ; give back any clusters just allocated
34398 00005B9E 58      POP     AX ; No. of clusters requested
34399                                ; Don't "retc". We are setting Carry anyway,
34400                                ; Alloc failed, so proceed with return CX
34401                                ; setup.
34402 ;;;
34403 00005B9F 8F06[F20A] pop     word [CLUSTERS_HW]
34404 ;;;
34405
34406 00005BA3 29C8      SUB     AX,CX ; AX=No. of clusters allocated
34407
34408 ;;;
34409 00005BA5 831E[F20A]00 sbb     word [CLUSTERS_HW],0
34410 ;;;
34411
34412 00005BAA E80700 call    RESTFATBYT ; Don't "retc". We are setting Carry anyway,
34413                                ; Alloc failed.
34414 Disk_Full_Return: ;label added for magic patch 8-6-92 scottq
34415 ; MSDOS 6.0
34416 00005BAD C606[0B06]01 MOV     byte [DISK_FULL],1 ;MS. indicating disk full
34417 00005BB2 F9      STC
34418 00005BB3 C3      retn
34419
34420 ;-----
34421 ;
34422 ; Procedure Name : RESTFATBYT
34423 ;
34424 ; SEE ALLOCATE CAVEAT
34425 ; Carry set if error (currently user FAILED to I 24)
34426 ;-----
34427 ; 27/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
34428
34429 RESTFATBYT:
34430     PUSH     BX
34431     PUSH     DX
34432     PUSH     DI
34433
34434 00005BB7 31DB      XOR     BX,BX
34435
34436 ; 27/02/2024 - Retro DOS v5.0
34437 ; (PCDOS 7.1 IBMDOS.COM)
34438 ;;;
34439 ;push     word [CLUSTNUM_HW]
34440 ;push     word [CLUSDATA_HW]
34441 ;push     word [CCONTENT_HW] ; (*) (not necessary ?)
34442 00005BB9 891E[E80A] mov     [CLUSTNUM_HW],bx ; 0
34443 00005BBD 8B16[E40A] mov     dx,[FATBYT_HW]
34444 00005BC1 8916[EA0A] mov     [CLUSDATA_HW],dx
34445 ;;;
34446
34447 00005BC5 8B16[9605] MOV     DX,[FATBYT]
34448
34449 00005BC9 E8C407 call    PACK
34450
34451 ; 27/02/2024 - Retro DOS v5.0
34452 ; (PCDOS 7.1 IBMDOS.COM)
34453 ;;;
34454 ; 28/06/2024 ; (*)
34455 ;pop     word [CCONTENT_HW]
34456 ;pop     word [CLUSDATA_HW]
34457 ;pop     word [CLUSTNUM_HW]

```



```

34458             ;;
34459
34460 00005BCC 5F      POP     DI
34461 00005BCD 5A      POP     DX
34462 00005BCE 5B      POP     BX
34463
34464 ; 16/12/2022
34465 ; 25/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
34466 ;RELEASE_flush:
34467 00005BCF C3      retn
34468
34469 ;Break      <RELEASE -- DEASSIGN DISK SPACE>
34470 -----
34471 ;
34472 ; Procedure Name : RELEASE
34473 ;
34474 ; Inputs:
34475 ;     BX = Cluster in file
34476 ;     ES:BP = Base of drive parameters
34477 ; Function:
34478 ;     Frees cluster chain starting with [BX]
34479 ;     Carry set if error (currently user FAILED to I 24)
34480 ; AX,BX,DX,DI all destroyed. Other registers unchanged.
34481 ;
34482 ;-----
34483
34484 ; 28/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
34485 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:9D13h
34486
34487 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:8E86h)
34488 ; (Windows ME IO.SYS - BIOSCODE:0C27Fh)
34489
34490 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
34491 ; 20/05/2019 - Retro DOS v4.0
34492 RELEASE:
34493 00005BD0 31D2      XOR     DX,DX
34494
34495 ; 28/02/2024 - Retro DOS v5.0
34496 ; (PCDOS 7.1 IBMDOS.COM)
34497 ;;;
34498 00005BD2 8916[EA0A] mov     [CLUSDATA_HW],dx ; (dx = 0, release)
34499 ;;;
34500
34501 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:9D19h
34502 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:8E88h)
34503 ; (Windows ME IO.SYS - BIOSCODE:0C285h)
34504
34505 ;entry RELBLKS
34506 RELBLKS:
34507
34508 ; Enter here with DX=0FFFFH to put an end-of-file mark in the first cluster
34509 ; and free the rest in the chain.
34510
34511 00005BD6 E80807     call    UNPACK
34512 ;jc      short RELEASE_flush
34513 ;jz      short RELEASE_flush
34514 ; 28/12/2024 (PCDOS 7.1 IBMDOS.COM)
34515 00005BD9 7652      jbe     short RET12 ; jna short RET12
34516 ; CCONTENT_HW:DI= Content of FAT
34517 ; for given cluster (next cluster)
34518 00005BDB 89F8      MOV     AX,DI
34519 00005BDD 52      PUSH    DX
34520
34521 ;;; 28/12/2024 - Retro DOS v5.0
34522 ;push    ax
34523 ;;;
34524
34525 00005BDE E8AF07     call    PACK
34526
34527 ;;; 28/12/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
34528 ;pop     ax
34529 00005BE1 8B16[EC0A] mov     dx,[CCONTENT_HW]
34530 00005BE5 8916[E80A] mov     [CLUSTNUM_HW],dx ; next cluster (hw) to be released
34531 00005BE9 89C3      mov     bx,ax ; next cluster (lw) to be released
34532 ;;;
34533
34534 00005BEB 5A      POP     DX
34535 ;jc      short RELEASE_flush
34536 ; 28/12/2024 (PCDOS 7.1 IBMDOS.COM)
34537 00005BEC 723F      jc      short RET12
34538
34539 00005BEE 09D2      OR      DX,DX
34540 00005BF0 751F      JNZ     short NO_DEALLOC ; Was putting EOF mark
34541
34542 ; 28/02/2024
34543 %if 0
34544 ;;cmp    word [es:bp+1Eh],-1 ; MSDOS 3.3
34545 ;;cmp    word [es:bp+1Fh],-1 ; MSDOS 6.0
34546 ;cmp     word [ES:BP+DPB.FREE_CNT],-1 ; Free count valid?
34547 ;JZ      short NO_DEALLOC ; No
34548 ;INC     word [ES:BP+DPB.FREE_CNT] ; Increase free count by 1
34549 NO_DEALLOC:
34550 MOV     BX,AX
34551 dec     ax ; check for "1"
34552 jz      short RELEASE_flush ; is last cluster of incomplete chain
34553 %else
34554 ; 28/02/2024 - Retro DOS v5.0
34555 ; (PCDOS 7.1 IBMDOS.COM)
34556 ;;;
34557 00005BF2 3B16[EA0A] cmp     dx,[CLUSDATA_HW] ; > 0 ? (delete, release)
34558 00005BF6 7519      jnz     short NO_DEALLOC ; no, EOF (allocate, -1), (dx = 0)
34559 ;cmp     word [es:bp+1Fh],0FFFFh
34560 00005BF8 26837E1FFF cmp     word [es:bp+DPB.FREE_CNT],0FFFFh ; -1
34561 ; Free count valid?
34562 00005BFD 7412      jz      short NO_DEALLOC ; No (dx = 0)
34563 relblks_ifc:
34564 ;inc     word ptr es:[bp+1Fh]
34565 00005BFF 26FF461F inc     word [ES:BP+DPB.FREE_CNT] ; Increase free count by 1
34566 00005C03 7519      jnz     short NO_DEALLOC2
34567
34568 ;cmp     [es:bp+0Fh],dx
34569 00005C05 2639560F cmp     [es:bp+DPB.FAT_SIZE],dx
34570 00005C09 7513      jnz     short NO_DEALLOC2 ; not FAT32
34571
34572 ;inc     word [es:bp+21h]
34573 00005C0B 26FF4621 inc     word [es:bp+DPB.FREE_CNT+2]
34574 00005C0F EB0D      jmp     short NO_DEALLOC2
34575 NO_DEALLOC:
34576 ;cmp     [es:bp+0Fh],dx
34577 00005C11 2639560F cmp     [es:bp+DPB.FAT_SIZE],dx ; dx is -1 or 0
34578 ; (valid 16 bit fat size is another number)
34579 00005C15 7507      jnz     short NO_DEALLOC2 ; FAT (FAT16 or FAT12)
34580 ; dx = 0, FAT32
34581 ;cmp     word [es:bp+21h],0FFFFh

```

```

34582 00005C17 26837E21FF      cmp     word [es:bp+DPB.FREE_CNT+2],0FFFFh
34583 00005C1C 75E1          jnz     short relblks_ifc
34584                                NO_DEALLOC2:
34585 00005C1E 833E[E80A]00      cmp     word [CLUSTNUM_HW],0
34586 00005C23 7503          jnz     short NO_DEALLOC3
34587 00005C25 48              dec     ax                ; check for "1"
34588 00005C26 7405          jz      short RET12        ; is last cluster of incomplete chain
34589                                NO_DEALLOC3:
34590                                ;;;
34591                                %endif
34592
34593 00005C28 E88D06      call    ISEOF
34594 00005C2B 72A3          JB     short RELEASE ; Carry clear if JMP not taken
34595
34596                                ; 16/12/2022
34597                                ; 25/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
34598                                ; 28/02/2024 (PCDOS 7.1 IBMDOS.COM)
34599                                %if 0
34600                                RELEASE_flush:
34601                                ; MSDOS 6.0
34602                                mov     al,[es:bp]
34603                                ;MOV    AL,[ES:BP+DPB.DRIVE]
34604                                push    si                ; FLUSHBUF may trash these and we guarantee
34605                                push    cx                ; them to be preserved.
34606                                push    es
34607                                push    bp
34608                                call    FLUSHBUF          ; commit buffers for this drive
34609                                pop     bp
34610                                pop     es
34611                                pop     cx
34612                                pop     si
34613                                %endif                ; 28/02/2024
34614
34615                                RET12:
34616 00005C2D C3          retn
34617
34618                                ;Break    <GETEOF -- Find the end of a file>
34619                                ;-----
34620                                ;
34621                                ; Procedure Name : GETEOF
34622                                ;
34623                                ; Inputs:
34624                                ;     ES:BP Points to DPB
34625                                ;     BX = Cluster in a file
34626                                ;     DS = CS
34627                                ; Outputs:
34628                                ;     BX = Last cluster in the file
34629                                ;     Carry set if error (currently user FAILED to I 24)
34630                                ;     DI destroyed. No other registers affected.
34631                                ;
34632                                ;-----
34633
34634                                ; 28/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
34635                                %if 0
34636                                GETEOF:
34637                                call    UNPACK
34638                                jc      short RET12
34639                                push    BX
34640                                mov     BX,DI
34641                                call    ISEOF
34642                                pop     BX
34643                                jae     short RET12
34644                                mov     BX,DI
34645                                jmp     short GETEOF
34646                                %else
34647                                ; 28/02/2024 - Retro DOS v5.0
34648                                ;;;
34649                                GETEOF4:    ; **
34650 00005C2E 8F06[E80A]    pop     word [CLUSTNUM_HW] ; * ; 28/02/2024
34651                                GETEOF1:
34652                                mov     di,ax
34653                                mov     [CLUSTERS_HW],dx ; CLUSTERS_HW:DI = Cluster Count
34654                                pop     ax
34655                                pop     dx
34656                                retn
34657
34658                                ; PCDOS 7.1 IBMDOS.COM - DOSCODE:9D7Ch
34659                                GETEOF:
34660                                push    dx
34661                                push    ax
34662                                xor     dx,dx ; 0          ; cluster count = 0
34663                                xor     ax,ax ; 0
34664                                GETEOF2:
34665                                call    UNPACK
34666                                jc      short GETEOF1
34667                                ; CCONTENT_HW:DI = next cluster (cluster content)
34668                                inc     ax
34669                                jnz     short GETEOF3
34670                                inc     dx
34671                                GETEOF3:
34672                                push    word [CLUSTNUM_HW] ; * ; 28/02/2024
34673                                push    bx
34674                                ;push   word [CLUSTNUM_HW]
34675                                mov     bx,[CCONTENT_HW]
34676                                mov     [CLUSTNUM_HW],bx
34677                                mov     bx,di
34678                                call    ISEOF
34679                                ;pop     word [CLUSTNUM_HW] ; * ; 28/02/2024
34680                                pop     bx
34681                                ;jae     short GETEOF1 ; EOF
34682                                jae     short GETEOF4 ; ** ; 28/02/2024
34683                                ;mov     bx,[CCONTENT_HW] ; not EOF
34684                                ;mov     [CLUSTNUM_HW],bx
34685                                pop     bx ; * ; 28/02/2024
34686                                mov     bx,di
34687                                jmp     short GETEOF2 ; get next cluster
34688                                ;;;
34689                                %endif
34690
34691                                ;=====
34692                                ; FCB.ASM, MSDOS 6.0, 1991
34693                                ;=====
34694                                ; 30/07/2018 - Retro DOS v3.0
34695                                ; 20/05/2019 - Retro DOS v4.0
34696                                ; 29/02/2024 - Retro DOS v5.0
34697
34698                                ; TITLE   FCB - FCB parse calls for MSDOS
34699                                ; NAME     FCB
34700
34701                                ;** FCB.ASM - Low level routines for parsing names into FCBs and analyzing
34702                                ; filename characters
34703                                ;
34704                                ; MakeFcb
34705                                ; NameTrans

```



```

34706 ; PATHCHRCMP
34707 ; GetLet
34708 ; UCase
34709 ; GetLet3
34710 ; GetCharType
34711 ; TESTKANJ
34712 ; NORMSCAN
34713 ; DELIM
34714 ;
34715 ; Revision history:
34716 ;
34717 ; A000 version 4.00 Jan. 1988
34718 ;
34719 ; M048 - access FILE_UCASE_TAB using DS rather than SS.
34720
34721 TableLook EQU -1
34722
34723 SCANSEPARATOR EQU 1
34724 DRVBIT EQU 2
34725 NAMBIT EQU 4
34726 EXTBIT EQU 8
34727
34728 ;-----
34729 ;
34730 ; Procedure : MakeFcb
34731 ;
34732 ;-----
34733
34734 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
34735 ; DOSCODE:8E77h (MSDOS 5.0, MSDOS.SYS)
34736
34737 ; 29/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
34738 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:9DB0h
34739
34740 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:8ED3h)
34741 ; (Windows ME IO.SYS - BIOSCODE:8D92h)
34742
34743 MAKEFCB:
34744 ;hkn; SS override
34745 ;MOV BYTE [SS:SpaceFlag],0
34746 00005C64 30D2 XOR DL,DL ; Flag--not ambiguous file name
34747 ; 29/02/2024
34748 00005C66 368816[4E03] mov [ss:SpaceFlag],dl ; 0
34749 ;test al,2
34750 00005C6B A802 test AL,DRVBIT ; Use current drive field if default?
34751 00005C6D 7503 JNZ short DEFDRV
34752 ;MOV BYTE [ES:DI],0 ; No - use default drive
34753 ; 29/02/2024
34754 00005C6F 268815 mov [es:di],dl ; 0
34755 DEFDRV:
34756 00005C72 47 INC DI
34757 00005C73 B90800 MOV CX,8
34758 ;test al,4
34759 00005C76 A804 test AL,NAMBIT ; Use current name fields as default?
34760 00005C78 93 XCHG AX,BX ; Save bits in BX
34761 00005C79 B020 MOV AL," "
34762 00005C7B 7404 JZ short FILLB ; If not, go fill with blanks
34763 00005C7D 01CF ADD DI,CX
34764 00005C7F 31C9 XOR CX,CX ; Don't fill any
34765 FILLB:
34766 00005C81 F3AA REP STOSB
34767 00005C83 B103 MOV CL,3
34768 00005C85 F6C308 test BL,EXTBIT ; Use current extension as default
34769 00005C88 7404 JZ short FILLB2
34770 00005C8A 01CF ADD DI,CX
34771 00005C8C 31C9 XOR CX,CX
34772 FILLB2:
34773 00005C8E F3AA REP STOSB
34774 00005C90 91 XCHG AX,CX ; Put zero in AX
34775 00005C91 AB STOSW
34776 00005C92 AB STOSW ; Initialize two words after to zero
34777 00005C93 83EF10 SUB DI,16 ; Point back at start
34778 ;test bl,1
34779 00005C96 F6C301 test BL,SCANSEPARATOR; Scan off separators if not zero
34780 00005C99 7409 JZ short SKPSPC
34781 00005C9B E88800 CALL SCANB ; Peel off blanks and tabs
34782 00005C9E E81E01 CALL DELIM ; Is it a one-time-only delimiter?
34783 00005CA1 7504 JNZ short NOSCAN
34784 00005CA3 46 INC SI ; Skip over the delimiter
34785 SKPSPC:
34786 00005CA4 E87F00 CALL SCANB ; Always kill preceding blanks and tabs
34787 NOSCAN:
34788 00005CA7 E8EC00 CALL GETLET
34789 00005CAA 761E JBE short NODRV ; Quit if termination character
34790 00005CAC 803C3A CMP BYTE [SI],":" ; Check for potential drive specifier
34791 00005CAF 7519 JNZ short NODRV
34792 00005CB1 46 INC SI ; Skip over colon
34793 00005CB2 2C40 SUB AL,"@" ; Convert drive letter to drive number (A=1)
34794 00005CB4 760F JBE short BADDRV ; Drive letter out of range
34795
34796 00005CB6 50 PUSH AX
34797 00005CB7 E8B71E call GetVisDrv
34798 00005CBA 58 POP AX
34799 00005CBB 730A JNC short HAVDRV
34800
34801 ; 20/05/2019 - Retro DOS v4.0
34802 ; MSDOS 6.0
34803 ;hkn; SS override
34804 00005CBD 36803E[1006]1A CMP byte [SS:DrvErr],error_not_DOS_disk ; 1Ah
34805 ; if not FAT drive ;AN000;
34806 00005CC3 7402 JZ short HAVDRV ; assume ok ;AN000;
34807 BADDRV:
34808 00005CC5 B2FF MOV DL,-1
34809 HAVDRV:
34810 00005CC7 AA STOSB ; Put drive specifier in first byte
34811 00005CC8 46 INC SI
34812 00005CC9 4F DEC DI ; Counteract next two instructions
34813 NODRV:
34814 00005CCA 4E DEC SI ; Back up
34815 00005CCB 47 INC DI ; Skip drive byte
34816
34817 ;entry NORMSCAN
34818 NORMSCAN:
34819 00005CCC B90800 MOV CX,8
34820 00005CCF E82200 CALL GETWORD ; Get 8-letter file name
34821 00005CD2 803C2E CMP BYTE [SI],"."
34822 00005CD5 7510 JNZ short NODOT
34823 00005CD7 46 INC SI ; Skip over dot if present
34824
34825 ; 24/09/2023
34826 ;mov cx,3
34827 00005CD8 B103 mov cl,3 ; ch=0
34828
34829 ; MSDOS 6.0

```

```

34830 ;hkn; SS override
34831 ;TEST word [SS:DOS34_FLAG],DBCS_VOLID2 ; 100h ;AN000;
34832 ; 10/06/2019
34833 00005CDA 36F606[1206]01 test byte [SS:DOS34_FLAG+1],(DBCS_VOLID2>>8) ; 1
34834 00005CE0 7402 JZ short VOLOK ;AN000;
34835 00005CE2 A4 MOVSB ; 2nd byte of DBCS ;AN000;
34836 ; 24/09/2023
34837 ;MOV CX,2 ;AN000;
34838 00005CE3 49 dec cx ; cx=2
34839 ;JMP SHORT contvol ;AN000;
34840 VOLOK:
34841 ;MOV CX,3 ; Get 3-letter extension
34842 contvol:
34843 00005CE4 E81300 CALL MUSTGETWORD
34844 00005CE7 88D0 NODOT:
34845 MOV AL,DL
34846 ; MSDOS 6.0
34847 ;and word [ss:DOS34_FLAG],0FEFFh
34848 ; 18/12/2022
34849 00005CE9 368026[1206]FE and byte [ss:DOS34_FLAG+1],0FEh ; (~DBCS_VOLID2)>>8
34850 ;and word [ss:DOS34_FLAG],~DBCS_VOLID2 ; ### BUG FIX ###
34851
34852 retn
34853 00005CEF C3
34854
34855 NONAM:
34856 00005CF0 01CF ADD DI,CX
34857 00005CF2 4E DEC SI
34858 00005CF3 C3 retn
34859
34860 GETWORD:
34861 00005CF4 E89F00 CALL GETLET
34862 00005CF7 76F7 JBE short NONAM ; Exit if invalid character
34863 00005CF9 4E DEC SI
34864
34865 ; UGH!!! Horrible bug here that should be fixed at some point:
34866 ; If the name we are scanning is longer than CX, we keep on reading!
34867
34868 MUSTGETWORD:
34869 00005CFA E89900 CALL GETLET
34870
34871 ; If spaceFlag is set then we allow spaces in a pathname
34872
34873 ;IF NOT TABLELOOK
34874 ; JB short FILLNAM ; MSDOS 3.3
34875 ;ENDIF
34876 00005CFD 750C JNZ short MustCheckCX
34877
34878 ;hkn; SS override
34879 00005CFF 36F606[4E03]FF test BYTE [SS:SpaceFlag],0FFh
34880 00005D05 7419 JZ short FILLNAM
34881 00005D07 3C20 CMP AL," "
34882 00005D09 7515 JNZ short FILLNAM
34883
34884 MustCheckCX:
34885 00005D0B E3ED JCXZ MUSTGETWORD
34886 00005D0D 49 DEC CX
34887 00005D0E 3C2A CMP AL,"*" ; Check for ambiguous file specifier
34888 00005D10 7504 JNZ short NOSTAR
34889 00005D12 B03F MOV AL,"?"
34890 00005D14 F3AA REP STOSB
34891
34892 NOSTAR:
34893 00005D16 AA STOSB
34894 00005D17 3C3F CMP AL,"?"
34895 00005D19 75DF JNZ short MUSTGETWORD
34896 00005D1E EBDA OR DL,1 ; Flag ambiguous file name
34897 JMP short MUSTGETWORD
34898 FILLNAM:
34899 00005D20 B020 MOV AL," "
34900 00005D22 F3AA REP STOSB
34901 00005D24 4E DEC SI
34902 00005D25 C3 retn
34903
34904 SCANB:
34905 00005D26 AC LODSB
34906 00005D27 E89D00 CALL SPCHK
34907 00005D2C 4E JZ short SCANB
34908 DEC SI
34909 scanb_retn:
34910 retn
34911
34912 ;-----
34913 ; Procedure Name : NameTrans
34914 ;
34915 ; NameTrans is used by FindPath to scan off an element of a path. We must
34916 ; allow spaces in pathnames
34917 ;
34918 ; Inputs: DS:SI points to start of path element
34919 ; Outputs: Name1 has unpacked name, uppercased
34920 ; ES = DOSGroup
34921 ; DS:SI advanced after name
34922 ; Registers modified: DI,AX,BX,CX
34923 ;-----
34924
34925 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
34926 ; 20/05/2019 - Retro DOS v4.0
34927
34928 ; 29/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
34929 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:9E7Ch
34930
34931 NameTrans:
34932 ;hkn; SS override
34933 00005D2E 36C606[4E03]01 MOV BYTE [SS:SpaceFlag],1
34934 00005D34 16 push ss
34935 00005D35 07 pop es
34936
34937 ;hkn; NAME1 is in DOSDATA
34938 00005D36 BF[4B05] MOV DI,NAME1
34939 00005D39 57 PUSH DI
34940
34941 ; 29/02/2024
34942 %if 0
34943 MOV AX,' '; 2020h
34944 MOV CX,5
34945 STOSB
34946 REP STOSW ; Fill "FCB" at NAME1 with spaces
34947 XOR AL,AL ; Set stuff for NORMSCAN
34948 MOV DL,AL
34949 %else
34950 ; 29/02/2024
34951 ; (PCDOS 7.1 IBMDOS.COM)
34952 ;;;
34953

```

```

34954 00005D3A B020      mov     al, 20h ; ' '
34955 00005D3C B90B00    mov     cx, 11
34956 00005D3F F3AA      rep stosb      ; Fill "FCB" at NAME1 with spaces
34957 00005D41 91      xchg     ax, cx
34958 00005D42 99      cwd
34959      ;;;
34960      %endif
34961
34962 00005D43 AA      STOSB
34963 00005D44 5F      POP     DI
34964
34965 00005D45 E884FF    CALL    NORMSCAN
34966
34967      ;hkn; SS override for NAME1
34968 00005D48 36803E[4B05]E5    CMP     byte [SS:NAME1],0E5H
34969 00005D4E 75DD      jnz     short scanb_retn
34970 00005D50 36C606[4B05]05    MOV     byte [SS:NAME1],5 ; Magic name translation
34971 00005D56 C3      retn
34972
34973      ;Break      <GETLET, DELIM -- CHECK CHARACTERS AND CONVERT>
34974      ;=====
34975
34976      ; 20/05/2019 - Retro DOS v4.0
34977      ; DOSCODE:8FD2h (MSDOS 6.21, MSDOS.SYS)
34978
34979      ;If TableLook
34980
34981      ;hkn; Table SEGMENT
34982      ; PUBLIC CharType
34983      ;-----
34984
34985      ; Character type table for file name scanning
34986      ; Table provides a mapping of characters to validity bits.
34987      ; Four bits are provided for each character. Values 7Dh and above
34988      ; have all bits set, so that part of the table is chopped off, and
34989      ; the translation routine is responsible for screening these values.
34990      ; The bit values are defined in DOSSYM.INC
34991
34992      ;      ; ^A and NUL
34993      ;CharType:
34994      ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
34995      ;      ; ^C and ^B
34996      ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
34997      ;      ; ^E and ^D
34998      ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
34999      ;      ; ^G and ^F
35000      ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35001      ;      ; TAB and BS
35002      ; db LOW ((NOT FFCB+FCHK+FDELIM+FSPCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35003      ;      ; ^K and ^J
35004      ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35005      ;      ; ^M and ^L
35006      ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35007      ;      ; ^O and ^N
35008      ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35009      ;      ; ^Q and ^P
35010      ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35011      ;      ; ^S and ^R
35012      ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35013      ;      ; ^U and ^T
35014      ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35015      ;      ; ^W and ^V
35016      ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35017      ;      ; ^Y and ^X
35018      ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35019      ;      ; ESC and ^Z
35020      ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35021      ;      ; ^] and ^[      db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35022      ;      ; ^_ and ^^
35023      ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35024      ;      ; ! and SPACE
35025      ; db LOW (NOT FCHK+FDELIM+FSPCHK)
35026      ;      ; # and "
35027      ; db LOW (NOT FFCB+FCHK)
35028      ;      ; $ - )
35029      ; db 3 dup (0FFh)
35030      ;      ; + and *
35031      ; db LOW ((NOT FFCB+FCHK+FDELIM) SHL 4) OR 0Fh
35032      ;      ; - and '
35033      ; db NOT (FFCB+FCHK+FDELIM)
35034      ;      ; / and .
35035      ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FCHK) AND 0Fh
35036      ;      ; 0 - 9
35037      ; db 5 dup (0FFh)
35038      ;      ; ; and :
35039      ; db LOW ((NOT FFCB+FCHK+FDELIM) SHL 4) OR LOW (NOT FFCB+FCHK+FDELIM) AND 0Fh
35040      ;      ; = and <
35041      ; db LOW ((NOT FFCB+FCHK+FDELIM) SHL 4) OR LOW (NOT FFCB+FCHK+FDELIM) AND 0Fh
35042      ;      ; ? and >
35043      ; db NOT FFCB+FCHK+FDELIM
35044      ;      ; A - Z
35045      ; db 13 dup (0FFh)
35046      ;      ; \ and [
35047      ; db LOW ((NOT FFCB+FCHK) SHL 4) OR 0Fh
35048      ;      ; ^ and ]
35049      ; db LOW ((NOT FFCB+FCHK) SHL 4) OR LOW (NOT FFCB+FCHK) AND 0Fh
35050      ;      ; _ - {
35051      ; db 15 dup (0FFh)
35052      ;      ; } and |
35053      ; db NOT FFCB+FCHK+FDELIM
35054      ;
35055
35056      ;CharType_last equ ($ - CharType) * 2      ; This is the value of the last
35057      ;      ; character in the table
35058
35059      ;FCHK      equ 1      ; normal name char, no chks needed
35060      ;FDELIM    equ 2      ; is a delimiter
35061      ;FSPCHK    equ 4      ; set if character is not a space or equivalent
35062      ;FFCB      equ 8      ; is valid in an FCB
35063
35064      ; DOSCODE:8FD2h (MSDOS 6.21, MSDOS.SYS)
35065      ;-----
35066      ; DOSCODE:8F76h (MSDOS 5.0, MSDOS.SYS)
35067
35068      CharType: ; 63 bytes
35069      db 66h, 66h, 66h, 66h, 66h, 06h, 66h, 66h, 66h ; 0-7
35070      db 66h, 66h, 66h, 66h, 66h, 66h, 66h, 66h ; 8-15
35071      db 0F8h,0F6h,0FFh,0FFh,0FFh, 4Fh,0F4h, 6Eh ; 16-23
35072      db 0FFh,0FFh,0FFh,0FFh,0FFh, 44h, 44h,0F4h ; 24-31
35073      db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh ; 32-39
35074      db 0FFh,0FFh,0FFh,0FFh,0FFh, 6Fh, 66h,0FFh ; 40-47
35075      db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh ; 48-55
35076      db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0F4h ; 56-62
35077
35078      CharType_last equ ($ - CharType) * 2

```

```

35079 ; Offset 12CAh of IBMDOS.COM (MSDOS 3.3), 1987
35080 ;-----
35081 ; CharType:
35082 ; db 0F6h,0F6h,0F6h,0F6h,0F6h,0F6h,0F6h,0F6h
35083 ; db 0F6h,0F0h,0F6h,0F6h,0F6h,0F6h,0F6h,0F6h
35084 ; db 0F6h,0F6h,0F6h,0F6h,0F6h,0F6h,0F6h,0F6h
35085 ; db 0F6h,0F6h,0F6h,0F6h,0F6h,0F6h,0F6h,0F6h
35086 ; db 0F8h,0FFh,0F6h,0FFh,0FFh,0FFh,0FFh,0FFh
35087 ; db 0FFh,0FFh,0FFh,0F4h,0F4h,0FFh,0FEh,0F6h
35088 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35089 ; db 0FFh,0FFh,0F4h,0F4h,0F4h,0F4h,0F4h,0FFh
35090 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35091 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35092 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35093 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35094 ; db 0FFh,0FFh,0FFh,0F6h,0F6h,0F6h,0FFh,0FFh
35095 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35096 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35097 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35098 ; db 0FFh,0FFh,0FFh,0FFh,0F4h,0FFh,0FFh,0FFh
35099 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35100 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35101 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35102 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35103 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35104 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35105 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35106 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35107 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35108 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35109 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35110 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35111 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35112 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35113 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35114 ; db 0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh,0FFh
35115 ;
35116 ;hkn; Table ENDS
35117 ;
35118 ;ENDIF
35119 ;
35120 ; 20/05/2019 - Retro DOS v4.0
35121 ; DOSCODE:9011h (MSDOS 6.21, MSDOS.SYS)
35122 ;
35123 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
35124 ; DOSCODE:8FB5h (MSDOS 5.0, MSDOS.SYS)
35125 ;-----
35126 ;
35127 ; Procedure Names : GetLet, UCase, GetLet3
35128 ;
35129 ; These routines take a character, convert it to upper case, and check
35130 ; for delimiters. Three different entry points:
35131 ; GetLet - DS:[SI] = character to convert
35132 ; UCase - AL = character to convert
35133 ; GetLet3 - AL = character
35134 ; [BX] = translation table to use
35135 ;
35136 ; Exit (in all cases) : AL = upper case character
35137 ; CY set if char is control char other than TAB
35138 ; ZF set if char is a delimiter
35139 ;
35140 ; Uses : AX, flags
35141 ;
35142 ; NOTE: This routine exists in a fast table lookup version, and a slow
35143 ; inline version. Return with carry set is only possible in the inline
35144 ; version. The table lookup version is the one in use.
35145 ;-----
35146 ;
35147 ; This entry point has character at [SI]
35148 ;
35149 ; IBMDOS.COM (MSDOS 3.3, 1987) - Offset 5517h
35150 GETLET:
35151 LODSB
35152 00005D96 AC
35153 ;
35154 ; This entry point has character in AL
35155 ;
35156 ;entry UCase
35157 UCase:
35158 ; 09/08/2018
35159 ; IBMDOS.COM (MSDOS 3.3, 1987) - Offset 5518h
35160 _UCase:
35161 PUSH BX
35162 00005D98 BB[7E0B] MOV BX,FILE_UCASE_TAB+2
35163 ;
35164 ; Convert the character in AL to upper case
35165 ;
35166 gl_0:
35167 CMP AL,"a"
35168 JB short gl_2 ; Already upper case, go check type
35169 CMP AL,"z"
35170 JA short gl_1
35171 SUB AL,20H ; Convert to upper case
35172 ;
35173 ; Map European character to upper case
35174 ;
35175 gl_1:
35176 CMP AL,80H
35177 JB short gl_2 ; Not EuroChar, go check type
35178 SUB AL,80H ; translate to upper case with this index
35179 ;
35180 ; M048 - Start
35181 ; Lantastic call Ucase thru int 2f without setting SS to DOSDATA.
35182 ; So we shall set up DS and to access FILE_UCASE_TAB in BX and also
35183 ; preserve it.
35184 ;
35185 ; 09/08/2018 - Retro DOS v3.0
35186 ; MSDOS 3.3
35187 ; XLAT BYTE [CS:BX] ; ds as file_ucase_tab is in DOSDATA
35188 ; CS XLAT
35189 ;
35190 ; 20/05/2019 - Retro DOS v4.0
35191 ;
35192 ; MSDOS 6.0
35193 00005DAB 1E push ds
35194 ;getdseg <ds>
35195 00005DAC 2E8E1E[0700] mov ds,[cs:DosDSeg]
35196 00005DB1 D7 XLAT ; ds as file_ucase_tab is in DOSDATA
35197 00005DB2 1F pop ds
35198 ;
35199 ; M048 - End
35200 ;
35201 ; Now check the type
35202

```

```

35203 ;If TableLook
35204 gl_2:
35205 ; 20/05/2019 - Retro DOS v4.0
35206 00005DB3 50      PUSH    AX
35207
35208 ; MSDOS 3.3
35209 ;mov    bx,CharType
35210 ;; 09/08/2018
35211 ;;xlat  byte [cs:bx]
35212 ;cs     xlat
35213
35214 ; MSDOS 6.0
35215 00005DB4 E81800  CALL    GetCharType ; returns type flags in AL
35216
35217 ;test    al,1
35218 00005DB7 A801    TEST    AL,FCHK ; test for normal character
35219 00005DB9 58      POP     AX
35220
35221 00005DBA 5B      POP     BX
35222 00005DBB C3      RETN
35223
35224 ; This entry has character in AL and lookup table in BX
35225
35226 ; MSDOS 6.0
35227 ;entry  GetLet3
35228 GETLET3: ; 10/08/2018
35229 00005DBC 53      PUSH    BX
35230 00005DBD EBDC    JMP     short gl_0
35231 ;ELSE
35232 ;
35233 ;gl_2:
35234 ; POP     BX
35235 ; CMP     AL,"."
35236 ; retz
35237 ; CMP     AL,'"''
35238 ; retz
35239 ; CALL    PATHCHRCMP
35240 ; retz
35241 ; CMP     AL,"["
35242 ; retz
35243 ; CMP     AL,"]"
35244 ; retz
35245 ;ENDIF
35246
35247 ;-----
35248 ;
35249 ; DELIM - check if character is a delimiter
35250 ; Entry : AX = character to check
35251 ; Exit : ZF set if character is not a delimiter
35252 ; Uses : Flags
35253 ;-----
35254 ;
35255 ;entry  DELIM
35256 DELIM:
35257 ;IF TableLook
35258 ; 20/05/2019 - Retro DOS v4.0
35259 00005DBF 50      PUSH    AX
35260
35261 ; MSDOS 3.3
35262 ;push    bx
35263 ;mov     bx,CharType
35264 ;;09/08/2018
35265 ;;xlat  byte [cs:bx]
35266 ;cs     xlat
35267 ;pop     bx
35268
35269 ; MSDOS 6.0
35270 00005DC0 E80C00  CALL    GetCharType
35271
35272 ;test    al,2
35273 00005DC3 A802    TEST    AL,FDELIM
35274 00005DC5 58      POP     AX
35275 00005DC6 C3      RETN
35276
35277 ;ELSE
35278 ; CMP     AL,":"
35279 ; retz
35280 ;
35281 ; CMP     AL,"<"
35282 ; retz
35283 ; CMP     AL,"|"
35284 ; retz
35285 ; CMP     AL,">"
35286 ; retz
35287 ;
35288 ; CMP     AL,"+"
35289 ; retz
35290 ; CMP     AL,"="
35291 ; retz
35292 ; CMP     AL",";"
35293 ; retz
35294 ; CMP     AL"," "
35295 ; retz
35296 ;ENDIF
35297
35298 ;-----
35299 ;
35300 ; SPCHK - checks to see if a character is a space or equivalent
35301 ; Entry : AL = character to check
35302 ; Exit : ZF set if character is a space
35303 ; Uses : flags
35304 ;-----
35305 ;
35306 ;entry  SPCHK
35307 SPCHK:
35308 ;IF TableLook
35309 ; 20/05/2019 - Retro DOS v4.0
35310 00005DC7 50      PUSH    AX
35311
35312 ; MSDOS 3.3
35313 ;push    bx
35314 ;mov     bx,CharType
35315 ;; 09/08/2018
35316 ;;xlat  byte [cs:bx]
35317 ;cs     xlat
35318 ;pop     bx
35319
35320 ; MSDOS 6.0
35321 00005DC8 E80400  CALL    GetCharType
35322
35323 ;test    al,4
35324 00005DCB A804    TEST    AL,FSPCHK
35325 00005DCD 58      POP     AX

```

```

35327 00005DCE C3      RETN
35328                  ;ELSE
35329                  ;   CMP     AL,9           ; Filter out tabs too
35330                  ;   retz
35331                  ; ; WARNING! " " MUST be the last compare
35332                  ;   CMP     AL," "
35333                  ;   return
35334                  ;ENDIF
35335
35336                  ;-----
35337                  ;
35338                  ; GetCharType - return flag bits indicating character type
35339                  ; Bits are defined in DOSSYM.INC. Uses lookup table
35340                  ; defined above at label CharType.
35341                  ;
35342                  ; Entry : AL = character to return type flags for
35343                  ; Exit  : AL = type flags
35344                  ; Uses  : AL, flags
35345                  ;-----
35346
35347
35348                  ; 29/02/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
35349
35350                  ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
35351
35352                  ; 20/05/2019 - Retro DOS v4.0
35353                  ; MSDOS 6.0
35354
35355 GetCharType:
35356                  ;cmp     al,7Eh
35357 00005DCF 3C7E      cmp     al,CharType_last      ; beyond end of table?
35358 00005DD1 7314      jae     short gct_90      ; return standard value
35359
35360 00005DD3 53        push    bx
35361 00005DD4 BB[575D] mov     bx,CharType          ; load lookup table
35362 00005DD7 D0E8      shr     al,1              ; adjust for half-byte table entry size
35363                  ;xlat     cs:[bx]          ; get flags
35364 00005DD9 2ED7      cs     xlat
35365 00005DDB 5B        pop     bx
35366
35367                  ; carry clear from previous shift means we want the low nibble. Otherwise
35368                  ; we have to shift the flags down to the low nibble
35369
35370 00005DDC 7306      jnc     short gct_80          ; carry clear, no shift needed
35371
35372                  ; 29/02/2024
35373 %if 0
35374                  shr     al,1              ; we want high nibble, shift it down
35375                  shr     al,1
35376                  shr     al,1
35377                  shr     al,1
35378 %else
35379                  ; 29/02/2024 (PC DOS 7.1 IBMDOS.COM)
35380                  ;;;
35381 00005DDE 51        push    cx
35382 00005DDF B104      mov     cl,4              ; we want high nibble, shift it down
35383 00005DE1 D2E8      shr     al,cl
35384 00005DE3 59        pop     cx
35385                  ;;;
35386 %endif
35387
35388 gct_80:
35389 00005DE4 240F      and     al,0Fh          ; clear the unused nibble
35390 00005DE6 C3        retn
35391
35392 00005DE7 B00F      mov     al,0Fh          ; set all flags
35393 00005DE9 C3        retn
35394
35395                  ;-----
35396                  ;
35397                  ; Procedure : PATHCHRCMP
35398                  ;-----
35399
35400
35401 PATHCHRCMP:
35402 00005DEA 3C2F      cmp     AL,'/'
35403 00005DEC 7606      jbe     short PathRet
35404 00005DEE 3C5C      cmp     AL,'\'
35405 00005DF0 C3        retn
35406
35407 00005DF1 B05C      mov     AL,'\'
35408 00005DF3 C3        retn
35409
35410 00005DF4 74FB      jz     short GotFor
35411 00005DF6 C3        retn
35412
35413
35414                  ;=====
35415                  ; MSCRTL.C, MSDOS 6.0, 1991
35416                  ;=====
35417                  ; 30/07/2018 - Retro DOS v3.0
35418                  ; 29/04/2019 - Retro DOS v4.0
35419                  ; 29/02/2024 - Retro DOS v5.0
35420
35421                  ; 15/03/2018 - Retro DOS v2.0 (MSDOS 2.11, CTRL.C, 1983)
35422
35423                  ;** MSCRTL.C - ^C and error handler for MSDOS
35424
35425                  ; TITLE Control C detection, Hard error and EXIT routines
35426                  ; NAME IBMCTRLC
35427
35428                  ;** Low level routines for detecting special characters on CON input,
35429                  ; the ^C exit/int code, the Hard error INT 24 code, the
35430                  ; process termination code, and the INT 0 divide overflow handler.
35431
35432                  ; FATAL
35433                  ; FATAL1
35434                  ; reset_environment
35435                  ; DSKSTATCHK
35436                  ; SPOOLINT
35437                  ; STATCHK
35438                  ; CNTCHAND
35439                  ; DIVOV
35440                  ; CHARHARD
35441                  ; HardErr
35442
35443                  ; Revision history:
35444
35445                  ; AN000 version 4.0 Jan 1988
35446                  ; A002 PTM -- dir >lpt3.hangs
35447                  ; A003 PTM 3957- fake version for IBMCAHE.COM
35448
35449                  ; M011: NEC's 8086 clone chip uses Intel's undocumented bit number in
35450                  ; flags register. In order to return to user normally DOS used to
35451                  ; move F202 into flags, which sets bit number 1 in flags uncondit-

```



```

35451 ; ionally. Now it is modified to maintain the state of bit 1.
35452 ;
35453 ; M024: suppressed fail and ignore options if not in the middle of int
35454 ; 24 and if Ctrl P or ctrl printscrn is pressed in routine
35455 ; charhard.
35456 ;
35457 ; 29/04/2019 - Retro DOS v4.0
35458 ; MSDOS 6.0
35459 ; public LowInt23Addr
35460 LowInt23Addr: ; LABEL DWORD
35461 00005DF7 [F60F]0000 DW LowInt23, 0
35462 ;
35463 ; public LowInt24Addr
35464 LowInt24Addr: ; LABEL DWORD
35465 00005DFB [0A10]0000 DW LowInt24, 0
35466 ;
35467 ; public LowInt28Addr
35468 LowInt28Addr: ; LABEL DWORD
35469 00005DFF [1E10]0000 DW LowInt28, 0
35470 ;
35471 ;Break <Checks for ^C in CON I/O>
35472 ;
35473 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
35474 ; 05/05/2019 - Retro DOS v4.0
35475 ;
35476 ;-----
35477 ;
35478 ; Procedure Name : DSKSTATCHK
35479 ;
35480 ; Check for ^C if only one level in
35481 ;
35482 ;-----
35483 ;
35484 ; procedure DSKSTATCHK,NEAR ; Check for ^C if only one level in
35485 ;
35486 ; 29/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
35487 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:9F51h
35488 ;
35489 DSKSTATCHK:
35490 00005E03 36803E[2103]01 cmp byte [ss:INDOS], 1
35491 00005E09 7401 jz short dskstatchk1
35492 00005E0B C3 retn
35493 ;
35494 00005E0C 51 dskstatchk1: ; ...
35495 00005E0D 06 push cx
35496 00005E0E 53 push es
35497 00005E0F 1E push bx
35498 00005E10 56 push ds
35499 00005E11 8CD3 push si
35500 00005E13 8EC3 mov bx, ss
35501 00005E15 8EDB mov es, bx
35502 00005E17 C606[9403]05 mov ds, bx
35503 00005E1C C606[9203]0E mov byte [DSKSTCOM], 5 ; DEVRDND
35504 00005E21 C606[9503]00 mov byte [DSKSTCALL], 14 ; DRDNDHL
35505 00005E26 BB[9203] mov byte [DSKSTST], 0
35506 00005E29 C536[3200] mov bx, DSKSTCALL
35507 00005E2D E865F4 lds si, [BCON]
35508 00005E30 1E call DEVIOCALL2
35509 00005E31 16 push ds
35510 00005E32 1F push ss
35511 00005E33 F606[9603]02 pop ds
35512 00005E38 7409 test byte [DSKSTST+1], 2
35513 00005E3A 30C0 jz short _GotCh
35514 00005E3C 5E xor al, al
35515 ;
35516 00005E3C 5E RET36: ; ...
35517 00005E3D 5E pop si
35518 00005E3E 1F pop si
35519 00005E3F 58 pop ds
35520 00005E40 07 pop bx
35521 00005E41 59 pop es
35522 00005E42 C3 pop cx
35523 ;
35524 00005E43 A0[9F03] _GotCh:
35525 00005E46 3C03 mov al, [DSKCHRET]
35526 00005E48 75F2 cmp al, 3 ; "C"-"@"
35527 00005E4A C606[9403]04 jnz short RET36
35528 00005E4F C606[9203]16 mov byte [DSKSTCOM], 4 ; DEVRD
35529 00005E54 880E[9F03] mov byte [DSKSTCALL], 16h ; DRDWRHL
35530 00005E58 C706[9503]0000 mov [DSKCHRET], c1
35531 00005E5E C706[A403]0100 mov word [DSKSTST], 0
35532 00005E64 1F mov word [DSKSTCNT], 1
35533 00005E65 E82DF4 pop ds
35534 00005E68 5E call DEVIOCALL2 ; Eat the ^C
35535 00005E69 1F pop si
35536 00005E6A 58 pop ds
35537 00005E6B 07 pop bx
35538 00005E6C 59 pop es
35539 00005E6D E9CF00 pop cx
35540 jmp CNTCHAND
35541 ;
35542 ; 05/05/2019
35543 NOSTOP:
35544 00005E70 3C10 ; MSDOS 6.0
35545 00005E72 7509 CMP AL,"P"-"@"
35546 JNZ short check_next ; SS override
35547 00005E74 36803E[BC0D]00 CMP BYTE [SS:SCAN_FLAG],0 ; ALT_Q ?
35548 00005E7A 7405 JZ short INCHKJ ; no
35549 check_end: ; 24/09/2023
35550 00005E7C C3 retn
35551 check_next:
35552 ;IF NOT TOGLPRN
35553 ;CMP AL,"N"-"@"
35554 ;JZ short INCHKJ
35555 ;ENDIF
35556 ;
35557 00005E7D 3C03 CMP AL,"C"-"@"
35558 ; 24/09/2023
35559 ;JZ short INCHKJ
35560 ;check_end:
35561 ;retn
35562 00005E7F 75FB jnz short check_end
35563 ;
35564 ; 24/09/2023
35565 ; 08/09/2018
35566 INCHKJ: ; 10/08/2018
35567 00005E81 E9A500 jmp INCHK
35568 ;
35569 ; MSDOS 3.3
35570 ;CMP AL,"P"-"@" ; cmp al,16
35571 ;JZ short INCHKJ
35572 ;
35573 ; 15/04/2018
35574 ;;IF NOT TOGLPRN

```

```

35575             ;CMP     AL,"N"-"@"
35576             ;JZ SHORT INCHKJ
35577             ;;ENDIF
35578
35579             ;CMP     AL,"C"-"@" ; cmp al,3
35580             ;JZ short INCHKJ
35581             ;RETN
35582
35583             ; 08/09/2018
35584             ;INCHKJ:; 10/08/2018
35585             ; JMP     INCHK
35586
35587             ;-----
35588             ;
35589             ; Procedure Name : SpoolInt
35590             ;
35591             ; SpoolInt - signal processes that the DOS is truly idle. we are allowed to
35592             ; do this ONLY if we are working on a 1-12 system call AND if we are not in
35593             ; the middle of an INT 24.
35594             ;-----
35595             ;
35596
35597 SPOOLINT:
35598 00005E84 9C      PUSHF
35599             ; 15/03/2018
35600 00005E85 36803E[5803]00 CMP     BYTE [SS:IDLEINT],0 ; SS override
35601 00005E8B 7423    JZ      SHORT POPFRET
35602 00005E8D 36803E[2003]00 CMP     BYTE [SS:ERRORMODE],0
35603 00005E93 751B    JNZ      SHORT POPFRET ; No spool ints in error mode
35604
35605             ; 30/07/2018
35606
35607             ; Note that we are going to allow an external program to issue system
35608             ; calls at this time. we MUST preserve IdleInt across this.
35609
35610 00005E95 36FF36[5803] PUSH    WORD [SS:IDLEINT]
35611
35612             ; 05/05/2019 - Retro DOS v4.0
35613
35614             ; MSDOS 6.0
35615 00005E9A 36803E[660D]00 cmp     byte [SS:DosHashMA],0 ; Q: is dos running in HMA (M021)
35616 00005EA0 7504    jne     short do_low_int28 ; Y: the int must be done from low mem
35617 00005EA2 CD28    int     int_spooler ; int 28h; N: Execute user int 28 handler
35618 00005EA4 EB05    jmp     short spool_ret_addr
35619
35620 do_low_int28:
35621             ;call far [ss:LowInt28Addr]
35622 00005EA6 2EFF1E[FF5D] call    far [cs:LowInt28Addr] ; 05/05/2019
35623
35624 spool_ret_addr:
35625             ;INT     int_spooler ; INT 28h
35626
35627 00005EAB 368F06[5803] POP     WORD [SS:IDLEINT]
35628 POPFRET:
35629 00005EB0 9D      POPF
35630 _RET18:
35631 00005EB1 C3      RETN
35632
35633             ; 05/05/2019 - Retro DOS v4.0
35634             ; DOSCODE:9137h (MSDOS 6.21, MSDOS.SYS)
35635             ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
35636             ; DOSCODE:90DBh (MSDOS 5.0, MSDOS.SYS)
35637
35638             ;-----
35639             ;
35640             ; Procedure Name : STATCHK
35641             ;-----
35642             ;
35643
35644 STATCHK:
35645 00005EB2 E84EFF CALL     DSKSTATCHK ; Allows ^C to be detected under
35646             ; input redirection
35647 00005EB5 53      PUSH    BX
35648 00005EB6 31DB    XOR     BX,BX
35649 00005EB8 E80EE0 CALL     GET_IO_SFT
35650 00005EBB 5B      POP     BX
35651 00005EBC 72F3    JC      SHORT _RET18
35652
35653 00005EBE B401    MOV     AH,1
35654 00005EC0 E805F2 CALL     IOFUNC
35655 00005EC3 74BF    JZ      SHORT SPOOLINT
35656 00005EC5 3C13    CMP     AL,'S'-'@'
35657 00005EC7 75A7    JNZ     SHORT NOSTOP
35658
35659             ; 05/05/2019
35660             ; MSDOS 6.0 ; SS override
35661 00005EC9 36803E[BC0D]00 CMP     BYTE [SS:SCAN_FLAG],0 ; AN000; ALT_R ?
35662 00005ECF 75AB    JNZ     short check_end ; AN000; yes
35663
35664 00005ED1 30E4    XOR     AH,AH
35665 00005ED3 E8F2F1 CALL     IOFUNC ; Eat Cntrl-S
35666 00005ED6 EB4A    JMP     SHORT PAUSOSTRT
35667
35668 PRINTOFF:
35669 00005ED8 36F616[FE02] PRINTON:
35670             NOT     BYTE [SS:PFLAG] ; 14/03/2018
35671
35672             ; 30/07/2018 - Retro DOS v3.0
35673 00005EDD 53      PUSH    BX
35674 00005EDE B80400 MOV     BX,4
35675 00005EE1 E8E5DF call    GET_IO_SFT
35676 00005EE4 5B      POP     BX
35677 00005EE5 72CA    JC      short _RET18
35678 00005EE7 06      PUSH    ES
35679 00005EE8 57      PUSH    DI
35680 00005EE9 1E      PUSH    DS
35681 00005EEA 07      POP     ES
35682 00005EEB 89F7    MOV     DI,SI ; ES:DI -> SFT
35683             ;test word [es:di+5],800h
35684             ;TEST word [ES:DI+SF_ENTRY.sf_flags],sf_net_spool
35685             ; 05/05/2019
35686 00005EED 26F6450608 test    byte [ES:DI+SF_ENTRY.sf_flags+1],(sf_net_spool>>8)
35687 00005EF2 7418    JZ      short NORM_PR ; Not redirected, echo is OK
35688
35689             ;Callinstall NetSpoolEchoCheck,MultNet,38,<AX>,<AX>
35690             ; See if allowed
35691 00005EF4 50      push    ax
35692 00005EF5 B82611 mov     ax,1126h
35693 00005EF8 CD2F    int     2Fh ; Multiplex - NETWORK REDIRECTOR - ???
35694             ; Return: CF set on error, AX = error code
35695             ; STACK unchanged
35696
35697 00005EFA 58      pop     ax
35698
35697 00005EFB 730F    JNC     short NORM_PR ; Echo is OK
35698

```



```

35699                                     ; SS override
35700 00005EFD 36C606[FE02]00      MOV     BYTE [SS:PFLAG],0      ; If not allowed, disable echo
35701
35702                                     ; Call install NetSpoolClose,MultNet,36,<AX>,<AX> ; and close
35703
35704 00005F03 50                    push    ax
35705 00005F04 B82411                mov     ax,1124h
35706 00005F07 CD2F                  int     2Fh      ; Multiplex - NETWORK REDIRECTOR - ???
35707                                     ; ES:DI -> SFT, SS = DOS CS
35708 00005F09 58                    pop     ax
35709
35710 00005F0A EB10                  JMP     SHORT RETP6
35711 NORM_PR:
35712 00005F0C 36803E[FE02]00      CMP     BYTE [SS:PFLAG],0      ; SS override
35713 00005F12 7505                  JNZ     short PRNOPN
35714 00005F14 E805F3              call    DEV_CLOSE_SFT
35715 00005F17 EB03                  JMP     SHORT RETP6
35716 PRNOPN:
35717 00005F19 E8F8F2              call    DEV_OPEN_SFT
35718 RETP6:
35719 00005F1C 5F                    POP     DI
35720 00005F1D 07                    POP     ES
35721 STATCHK_RET:
35722 00005F1E C3                    RETN
35723 PAUSOLP:
35724 00005F1F E862FF              CALL     SPOOLINT
35725 PAUSOSTRT:
35726 00005F22 B401                  MOV     AH,1
35727 00005F24 E8A1F1              CALL     IOFUNC
35728 00005F27 74F6                  JZ      SHORT PAUSOLP
35729 INCHK:
35730 00005F29 53                    PUSH    BX
35731 00005F2A 31DB                  XOR     BX,BX
35732 00005F2C E89ADF              CALL     GET_IO_SFT
35733 00005F2F 5B                    POP     BX
35734 00005F30 72EC                  JC      SHORT STATCHK_RETN ; 30/07/2018
35735 00005F32 30E4                  XOR     AH,AH
35736 00005F34 E891F1              CALL     IOFUNC
35737                                     ; 30/07/2018
35738                                     ; MSDOS 3.3
35739                                     ; CMP     AL,'P'-'@' ;cmp al,16
35740                                     ; JNZ     SHORT NOPRINT
35741
35742                                     ; cmp     byte [SS:SCAN_FLAG],0
35743                                     ; JZ      SHORT PRINTON
35744                                     ; mov     byte [ss:SCAN_FLAG],0
35745
35746                                     ; 05/05/2019
35747                                     ; MSDOS 6.0
35748 00005F37 3C10                  CMP     AL,"P"-"@"
35749                                     ;;;; 7/14/86 ALT_Q key fix
35750 00005F39 749D                  JZ      short PRINTON      ; no! must be CTRL_P
35751 ;NOPRINT:
35752                                     ; IF      NOT TOGLPRN
35753                                     ; CMP     AL,"N"-"@"
35754                                     ; JZ      short PRINTOFF
35755                                     ; ENDF
35756 00005F3B 3C03                  CMP     AL,"C"-"@" ; cmp al,3
35757                                     ; retnz
35758 00005F3D 75DF                  jnz     short STATCHK_RETN
35759
35760                                     ; !! NOTE: FALL THROUGH !!
35761
35762 ;-----
35763 ;
35764 ; Procedure Name : CNTHAND ( CTRL_C HANDLER )
35765 ;
35766 ; "AC" and CR/LF is printed. Then the user registers are restored and the
35767 ; user CTRL-C handler is executed. At this point the top of the stack has 1)
35768 ; the interrupt return address should the user CTRL-C handler wish to allow
35769 ; processing to continue; 2) the original interrupt return address to the code
35770 ; that performed the function call in the first place. If the user CTRL-C
35771 ; handler wishes to continue, it must leave all registers unchanged and RET
35772 ; (not IRET) with carry CLEAR. If carry is SET then an terminate system call
35773 ; is simulated.
35774 ;
35775 ;-----
35776
35777                                     ; 29/02/2024 - Retro DOS v5.0
35778
35779 CNTCHAND:
35780                                     ; MSDOS 6.0
35781                                     ; SS override
35782                                     ; AN002; from RAWOUT
35783 ;TEST word [SS:DOS34_FLAG],CTRL_BREAK_FLAG
35784 ;JNZ short around_deadlock ; AN002;
35785
35786                                     ; 05/05/2019 - Retro DOS v4.0
35787                                     ; (MSDOS 6.21 MSDOS.SYS DOSCODE:91C4h, 29/12/2022)
35788 00005F3F 36F606[1206]02      TEST    byte [SS:DOS34_FLAG+1],(CTRL_BREAK_FLAG>>8) ; 2
35789 00005F45 7508                  JNZ     short around_deadlock ; AN002;
35790
35791 00005F47 B003                  MOV     AL,3
35792 00005F49 E879BD              CALL     BUFOUT
35793 00005F4C E80EBC              CALL     CRLF
35794 around_deadlock:
35795 00005F4F 16                    PUSH    SS
35796 00005F50 1F                    POP     DS
35797 00005F51 803E[5703]00        CMP     BYTE [CONSWAP],0
35798 00005F56 7403                  JZ      SHORT NOSWAP
35799 00005F58 E802DC              CALL     SWAPBACK
35800 NOSWAP:
35801 00005F5B FA                    CLI
35802 00005F5C 8E16[8605]          MOV     SS,[USER_SS]      ; Prepare to play with stack
35803 00005F60 8B26[8405]          MOV     SP,[USER_SP]      ; User stack now restored
35804 00005F64 E8DAA4              CALL     restore_world    ; User registers now restored
35805
35806                                     ; 30/07/2018 - Retro DOS v3.0
35807                                     ; MSDOS 3.3 (IBMDOS.COM - offset 56ACh)
35808                                     ; 14/03/2018 - Retro DOS v2.0
35809 ;MOV BYTE [CS:INDOS],0
35810 ;MOV BYTE [CS:ERRORMODE],0
35811 ;MOV [CS:ConC_Spsave],SP
35812 ;c1c ;30/07/2018
35813 ;INT int_ctrl_c ; 23h ; Execute user Ctrl-C handler
35814 ;;int 23h ; DOS - CONTROL "C" EXIT ADDRESS
35815 ; Return: return via RETF 2 with CF set
35816 ; DOS will abort program with errorlevel 0
35817 ; else
35818 ; interrupted DOS call continues
35819
35820                                     ; 05/05/2019 - Retro DOS v4.0
35821                                     ; MSDOS 6.0 (MSDOS 6.21, MSDOS.SYS,91ECh)
35822                                     ; CS was used to address these variables. we have to use DOSDATA

```

```

35823
35824 00005F67 07      pop     es ; * ; MSDOS 6.21 (MSDOS.SYS, DOSCODE:91ECh)
35825                      ; (pop es, after 'call restore_world')
35826 00005F68 1E      push    ds
35827                      ;getdseg <ds> ; ds -> dosdata
35828 00005F69 2E8E1E[0700] mov     ds,[cs:DosDSeg]
35829 00005F6E C606[2103]00 mov     byte [INDOS],0 ; Go to known state
35830                      ; 29/02/2024 (PCDOS 7.1 IBMDOS.COM)
35831                      ;;;
35832 00005F73 C606[B812]00 mov     byte [INDOS_FLAG],0 ; 2nd 'in dos' flag (what for?)
35833                      ;;;
35834 00005F78 C606[2003]00 mov     byte [ERRORMODE],0
35835 00005F7D 8926[3203] mov     [ConC_Spsave],SP ; save his SP
35836                      ; User SP has changed because of push. Adjust for it
35837 00005F81 8306[3203]02 add     word [ConC_Spsave],2
35838
35839 00005F86 803E[660D]00 cmp     byte [DosHashMA],0 ; Q: is dos running in HMA (M021)
35840 00005F8B 1F      pop     ds ; restore ds
35841 00005F8C 7505     jne     short do_low_int23 ; Y: the int must be done from low mem
35842 00005F8E F8      CLC
35843 00005F8F CD23     INT     int_ctrl_c ; int 23h ; N: Execute user Ctrl-C handler
35844 00005F91 EB06     jmp     short ctrlc_ret_addr
35845
35846                      ; 05/05/2019
35847 do_low_int23:
35848 00005F93 F8      cll
35849 00005F94 2EFF1E[F75D] call    far [cs:LowInt23Addr]
35850
35851                      ; 30/07/2018
35852
35853                      ; MSDOS 3.3 (IBMDOS.COM - Offset 56C0h)
35854
35855 ; The user has returned to us. The circumstances we allow are:
35856 ;
35857 ; IRET We retry the operation by redispaching the system call
35858 ; CLC/RETF POP the stack and retry
35859 ; ... Exit the current process with ^C exit
35860 ;
35861 ; User's may RETURN to us and leave interrupts on.
35862 ; Turn 'em off just to be sure
35863
35864 ctrlc_ret_addr: ; 05/05/2019
35865
35866 00005F99 FA      CLI
35867
35868                      ; MSDOS 3.3
35869 ;MOV     [CS:USER_IN_AX],ax ; save the AX
35870 ;PUSHF ; and the flags (maybe new call)
35871 ;POP     AX
35872
35873                      ; 05/05/2019
35874                      ; MSDOS 6.0
35875
35876                      ; we have to use DOSDATA for these variables. Previously CS was used
35877
35878 00005F9A 50      push    ax
35879 00005F9B 8CD8     mov     ax,ds
35880                      ;getdseg <ds> ; ds -> dosdata
35881 00005F9D 2E8E1E[0700] mov     ds,[cs:DosDSeg]
35882 00005FA2 A3[630D] mov     [TEMPSEG],ax
35883 00005FA5 58      pop     ax
35884 00005FA6 A3[3A03] MOV     [USER_IN_AX],ax ; save the AX
35885 00005FA9 9C      pushf ; and the flags (maybe new call)
35886 00005FAA 58      pop     ax
35887
35888 ; See if the input stack is identical to the output stack
35889
35890                      ; MSDOS 3.3
35891 ;CMP     SP,[CS:ConC_Spsave]
35892 ;JNZ     SHORT ctrlc_try_new ; current SP not the same as saved SP
35893
35894                      ; MSDOS 6.0
35895 CMP     SP,[ConC_Spsave]
35896 JNZ     SHORT ctrlc_try_new ; current SP not the same as saved SP
35897
35898 ; Repeat the operation by redispaching the system call.
35899
35900 ctrlc_repeat:
35901                      ; MSDOS 3.3
35902 ;MOV     AX,[CS:USER_IN_AX]
35903 ; 05/05/2019
35904 ; MSDOS 6.0
35905 mov     ax,[USER_IN_AX]
35906 mov     ds,[TEMPSEG] ; restore ds and original sp
35907                      ; MSDOS 3.3 & MSDOS 6.0
35908 ;transfer COMMAND
35909 COMMANDJ:
35910 00005FB8 E936A3   JMP     COMMAND
35911
35912 ; The current SP is NOT the same as the input SP. Presume that he
35913 ; RETF'd leaving some flags on the stack and examine the input
35914
35915 ctrlc_try_new:
35916 ; 29/02/2024
35917 ;ADD     SP,2 ; pop those flags
35918 ;
35919 ;;test ax,1
35920 ;TEST    AX,f_Carry ; did he return with carry?
35921 00005FBB A801     test    al,f_Carry ; test al,1
35922 ;
35923 ; 29/02/2024
35924 00005FBD 58      pop     ax ; (PCDOS 7.1 IBMDOS.COM)
35925 ;
35926 00005FBE 74F1     JZ      short ctrlc_repeat ; no carry set, just retry
35927
35928                      ; MSDOS 6.0
35929 00005FC0 8E1E[630D] mov     ds,[TEMPSEG] ; restore ds
35930
35931                      ; well... time to abort the user.
35932                      ; Signal a ^C exit and use the EXIT system call..
35933
35934 ctrlc_abort:
35935                      ; MSDOS 3.3
35936 ;;MOV     AX,(EXIT SHL 8) + 0
35937 ;;MOV     AX,(EXIT*256) + 0 ; 4C00h
35938 ;mov     byte [CS:DidCTRLC],0FFh ; 14/03/2018
35939 ;transfer COMMAND ; give up by faking $EXIT
35940 ;;JMP     SHORT COMMANDJ
35941 ;JMP     COMMAND
35942
35943                      ; 05/05/2019 - Retro DOS v4.0
35944                      ; MSDOS 6.0
35945 00005FC4 B8004C   MOV     AX,(EXIT<<8)+0 ; 4C00h
35946 00005FC7 1E      push    ds

```

```

35947 ;getdseg <ds> ; ds -> dosdata
35948 00005FC8 2E8E1E[0700] mov ds,[cs:DosDSeg]
35949 00005FCD C606[4D03]FF MOV byte [DidCTRLC],-1 ; 0FFh
35950 00005FD2 1F pop ds
35951 ;transfer COMMAND ; give up by faking $EXIT
35952 00005FD3 EBE3 JMP SHORT COMMANDJ
35953 ;JMP COMMAND
35954
35955 ;Break <DIVISION OVERFLOW INTERRUPT>
35956 ;-----
35957 ;
35958 ; Procedure Name : DIVOV
35959 ;
35960 ; Default handler for division overflow trap
35961 ;-----
35962 ;
35963 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
35964
35965 DIVOV:
35966 ; 05/05/2019 - Retro DOS v4.0
35967 ; 30/07/2018
35968 ; 07/07/2018 - Retro DOS v3.0
35969 00005FD5 BE[1B0A] mov si,DIVMES
35970 00005FD8 2E8B1E[2E0A] mov bx,[cs:DivMesLen]
35971 ;mov ax,cs
35972 ;mov ss,ax
35973 ; 05/05/2019
35974 ;getdseg <ss> ; we are in an ISR, flag is CLI
35975 00005FDD 2E8E16[0700] mov ss,[cs:DosDSeg]
35976 00005FE2 BC[A007] mov sp,AUXSTACK
35977 ;call RealDivov ; MSDOS 3.3
35978 00005FE5 E80200 call _OUTMES ; MSDOS 6.0
35979 00005FE8 EBDA jmp short ctrlc_abort ; Use Ctrl-C abort on divide overflow
35980
35981 ; 30/07/2018
35982 ;
35983 ; MSDOS 6.0
35984 ;-----
35985 ;
35986 ; Procedure Name : OutMes
35987 ;
35988 ;
35989 ; OutMes: perform message output
35990 ; Inputs: SS:SI points to message
35991 ; BX has message length
35992 ; Outputs: message to BCON
35993 ;
35994 ; Actually, cs:si points to the message now. The segment address is filled in
35995 ; at init. time ([diskchret+2]). This will be temporarily changed to DOSCODE.
35996 ; NB. This procedure is called only from DIVOV. -SR
35997 ;-----
35998 ;
35999 ;
36000 ; MSDOS 3.3
36001 ;-----
36002 ; RealDivov: perform actual divide overflow stuff.
36003 ; Inputs: none
36004 ; Outputs: message to BCON
36005 ;-----
36006 ;
36007 ; 05/05/2019 - Retro DOS v4.0
36008 ; DOSCODE:926Ch (MSDOS 6.21, MSDOS.SYS)
36009
36010 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
36011 ; DOSCODE:9210h (MSDOS 5.0, MSDOS.SYS)
36012 ;-----
36013 ;
36014 ; Procedure Name : OutMes
36015 ;
36016 ;
36017 ; OutMes: perform message output
36018 ; Inputs: SS:SI points to message
36019 ; BX has message length
36020 ; Outputs: message to BCON
36021 ;
36022 ; Actually, cs:si points to the message now. The segment address is filled in
36023 ; at init. time ([diskchret+2]). This will be temporarily changed to DOSCODE.
36024 ; NB. This procedure is called only from DIVOV. -SR
36025 ;-----
36026 ;
36027 ;
36028 ; 30/07/2018
36029 ; MSDOS 6.0
36030
36031 _OUTMES:
36032 ; MSDOS 3.3
36033 ;RealDivov:
36034 ; 07/07/2018 - Retro DOS v3.0
36035 ;Context ES
36036 00005FEA 16 push ss ; 05/05/2019
36037 00005FEB 07 ;PUSH CS ; 30/07/2018 ; get ES addressability
36038 POP ES
36039 ;Context DS
36040 00005FEC 16 push ss ; 05/05/2019
36041 00005FED 1F ;PUSH CS ; 30/07/2018 ; get DS addressability
36042 00005FEE C606[9403]08 POP DS
36043 00005FF3 C606[9203]16 MOV BYTE [DSKSTCOM],DEVWRT
36044 00005FF8 C706[9503]0000 MOV BYTE [DSKSTCALL],DRDWRHL
36045 ; BX = [DivMesLen] = 19
36046 00005FFE 891E[A403] MOV [DSKSTCNT],BX
36047 00006002 BB[9203] MOV BX,DSKSTCALL
36048 00006005 8936[A003] MOV [DSKCHRET+1],SI ; transfer address (need an EQU)
36049 ; 08/09/2018
36050 ;mov [DEVIobuf_PTR],si
36051 ; MSDOS 6.0
36052 ; CS is used for string, fill in
36053 ; segment address
36054 ;mov [DOSSEG_INIT],cs ; 29/02/2024
36055 00006009 8C0E[A203] MOV [DSKCHRET+3],CS
36056
36057 0000600D C536[3200] LDS SI,[BCON]
36058 00006011 E881F2 CALL DEVIocall2
36059
36060 ; 14/03/2018
36061 ; ;MOV WORD [CS:DSKCHRET+1],DEVIobuf
36062 ; 08/09/2018
36063 ;mov word [CS:DEVIobuf_PTR],DEVIobuf
36064 ; ;MOV WORD [CS:DSKSTCNT],1
36065
36066 ; 05/05/2019 - Retro DOS v4.0 (MSDOS 6.0, MSDOS 6.21)
36067
36068 ; ES still points to DOSDATA. ES is
36069 ; not destroyed by deviocall2. So use
36070 ; ES override.

```

```

36071
36072 00006014 26C706[A003][BC03]      MOV     WORD [ES:DSKCHRET+1],DEVIOPBUF
36073 0000601B 26C706[A403]0100      MOV     WORD [ES:DSKSTCNT],1
36074
36075 00006022 C3                      RETN
36076
36077 ;Break      <CHARHRD,HARDERR,ERROR -- HANDLE DISK ERRORS AND RETURN TO USER>
36078 ;-----
36079 ;
36080 ; Procedure Name : CHARHARD
36081 ;
36082 ;
36083 ; Character device error handler
36084 ; Same function as HARDERR
36085 ;-----
36086 ;
36087 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
36088 CHARHARD:
36089 ; 05/05/2019 - Retro DOS v4.0
36090 ; 30/07/2018
36091 ; 08/07/2018 - Retro DOS v3.0
36092
36093 ; MSDOS 6.0
36094
36095 ; M024 - start
36096 00006023 36803E[2003]00      cmp     byte [SS:ERRORMODE], 0; Q: are we in the middle of int 24
36097 ;jne      short @f           ; Y: allow fail
36098 00006029 750B      jne     short chard1
36099
36100 0000602B 80CC10      OR      AH,Allowed_RETRY ; 10h; assume ctrl p
36101
36102 0000602E 36F606[FE02]FF      test    byte [ss:PFLAG],-1 ; Q: has ctrl p been pressed
36103 00006034 7503      jnz     short ctrlp ; Y:
36104 ;@@:
36105 chard1:
36106 ; MSDOS 6.0 & MSDOS 3.3
36107 ; M024 - end
36108 ; Character device error handler
36109 ; Same function as HARDERR
36110
36111 ;or      ah,38h
36112 00006036 80CC38      or      ah,Allowed_IGNORE+Allowed_RETRY+Allowed_FAIL
36113 ctrlp: ; SS override for Allowed and EXITHOLD
36114 00006039 368826[4B03]      mov     [SS:ALLOWED],ah
36115
36116 ; 15/03/2018
36117 0000603E 368C06[8205]      MOV     [SS:EXITHOLD+2],ES
36118 00006043 36892E[8005]      MOV     [SS:EXITHOLD],BP
36119 00006048 56      PUSH    SI
36120 ;and     di,0FFh
36121 00006049 81E7FF00      AND     DI,STECODE
36122 0000604D 8CDD      MOV     BP,DS ;Device pointer is BP:SI
36123 0000604F E89D00      CALL    FATALC
36124 00006052 5E      POP     SI
36125 ;return
36126 00006053 C3      RETN
36127
36128 ;-----
36129 ;
36130 ; Procedure Name : HardErr
36131 ;
36132 ; Hard disk error handler. Entry conditions:
36133 ; DS:BX = Original disk transfer address
36134 ; DX = Original logical sector number
36135 ; CX = Number of sectors to go (first one gave the error)
36136 ; AX = Hardware error code
36137 ; DI = Original sector transfer count
36138 ; ES:BP = Base of drive parameters
36139 ; [READOP] = 0 for read, 1 for write
36140 ; Allowed Set with allowed responses to this error (other bits MUST BE 0)
36141 ; Output:
36142 ; [FAILERR] will be set if user responded FAIL
36143 ;-----
36144 ;
36145 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
36146 ; 29/02/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
36147 HARDERR:
36148 ; 05/05/2019 - Retro DOS v4.0
36149 ; 30/07/2018
36150 ; 08/07/2018 - Retro DOS v3.0
36151
36152 00006054 97      XCHG     AX,DI ; Error code in DI, count in AX
36153 ;and     di,0FFh
36154 00006055 81E7FF00      AND     DI,STECODE ; And off status bits
36155 ;CMP     DI,WRECODE ; Write Protect Error?
36156 ;cmp     di,0
36157 00006059 83FF00      cmp     DI,error_I24_write_protect ; Write Protect Error?
36158 0000605C 750A      JNZ     short NOSETWRPERR
36159 0000605E 50      PUSH    AX
36160 ; 25/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
36161 ;MOV     AL,[ES:BP+DPB.DRIVE]
36162 ;;MOV     AL,[ES:BP+0]
36163 ; 15/12/2022
36164 0000605F 268A4600      mov     al,[ES:BP]
36165 ; 15/03/2018
36166 00006063 36A2[2203]      MOV     [SS:WPERR],AL ; Flag drive with WP error
36167 00006067 58      POP     AX
36168 NOSETWRPERR:
36169 00006068 29C8      SUB     AX,CX ; Number of sectors successfully transferred
36170 0000606A 01C2      ADD     DX,AX ; First sector number to retry
36171 ; 29/02/2024 (PCDOS 7.1 IBMDOS.COM)
36172 ;;;
36173 0000606C 368316[0706]00      adc     word [ss:HIGH_SECTOR],0
36174 ;;;
36175 00006072 52      PUSH    DX
36176 ; 08/07/2018
36177 ;MUL     word [ES:BP+2] ; Number of bytes transferred
36178 00006073 26F76602      MUL     word [ES:BP+DPB.SECTOR_SIZE]
36179 00006077 5A      POP     DX
36180 00006078 01C3      ADD     BX,AX ; First address for retry
36181 ;XOR     AH,AH ; * ; Flag disk section in error
36182 ; 29/02/2024 (PCDOS 7.1 IBMDOS.COM)
36183 ;;;
36184 0000607A 31C0      xor     ax,ax ; * ; 29/02/2024 - Retro DOS v5.0
36185 ;cmp     word [ss:HIGH_SECTOR],0
36186 0000607C 363906[0706]      cmp     [ss:HIGH_SECTOR],ax ; 0 ; *
36187 00006081 7506      jnz     short TESTDIR
36188 ;;;
36189 ;CMP     DX,[ES:BP+6] ; In reserved area?
36190 00006083 263B5606      CMP     DX,[ES:BP+DPB.FIRST_FAT]
36191 00006087 7246      JB      SHORT ERRINT
36192
36193 TESTDIR: ; 29/02/2024
36194 00006089 FEC4      INC     AH ; Flag for FAT

```

```

36195
36196 ; 29/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
36197 ;;;
36198 ;cmp word [es:bp+0Fh],0
36199 0000608B 26837E0F00 cmp word [es:bp+DPB.FAT_SIZE],0
36200 00006090 7525 jnz short TESTDIR3 ; not FAT32
36201 00006092 52 push dx
36202 00006093 368B16[0706] mov dx,[ss:HIGH_SECTOR]
36203 ;cmp dx,[es:bp+2Bh]
36204 00006098 263B562B cmp dx,[es:bp+DPB.FCLUS_FSECTOR+2]
36205 0000609C 5A pop dx
36206 0000609D 7504 jnz short TESTDIR1 ; Err in FAT must force recomp of freespace
36207 ;cmp dx,[es:bp+29h]
36208 0000609F 263B5629 cmp dx,[es:bp+DPB.FCLUS_FSECTOR]
36209 TESTDIR1:
36210 000060A3 730E jnb short TESTDIR2
36211 ;mov word [es:bp+1Fh],0FFFFh ; -1
36212 000060A5 26C7461FFFFF mov word [es:bp+DPB.FREE_CNT],0FFFFh
36213 ;mov word [es:bp+21h],0FFFFh ; -1
36214 000060AB 26C74621FFFF mov word [es:bp+DPB.FREE_CNT+2],0FFFFh
36215 000060B1 EB1C jmp short ERRINT
36216 TESTDIR2:
36217 000060B3 FEC4 inc ah
36218 ;inc ah
36219 ;jmp short ERRINT
36220 ; 29/02/2024 - Retro DOS v5.0
36221 000060B5 EB16 jmp short ERRINT2
36222 TESTDIR3:
36223 ;;;
36224
36225 ;CMP DX,[ES:BP+10H] ; MSDOS 3.3
36226 ;cmp dx,[ES:BP+11h] ; MSDOS 6.0 - 05/05/2019
36227 000060B7 263B5611 CMP DX,[ES:BP+DPB.DIR_SECTOR] ; In FAT?
36228 ;JAE short TESTDIR ; No
36229 000060BB 7308 jae short TESTDIR4 ; 29/02/2024
36230
36231 ; Err in FAT must force recomp of freespace
36232 ;mov word [ES:BP+1Eh],-1 ; MSDOS 3.3
36233 ;mov word [ES:BP+1Fh],-1 ; MSDOS 6.0 - 05/05/2019
36234 000060BD 26C7461FFFFF MOV word [ES:BP+DPB.FREE_CNT],-1
36235
36236 JMP SHORT ERRINT
36237 ;TESTDIR:
36238 TESTDIR4: ; 29/02/2024
36239 000060C5 FEC4 INC AH
36240 ;CMP DX,[ES:BP+0BH] ; In directory?
36241 000060C7 263B560B CMP DX,[ES:BP+DPB.FIRST_SECTOR]
36242 000060CB 7202 JB SHORT ERRINT
36243 ERRINT2: ; 29/02/2024
36244 000060CD FEC4 INC AH ; Must be in data area
36245 ERRINT:
36246 000060CF D0E4 SHL AH,1 ; Make room for read/write bit
36247 000060D1 360A26[7505] OR AH,[SS:READOP] ; 15/03/2018
36248
36249 ; 15/08/2018
36250
36251 000060D6 360A26[4B03] OR AH,[SS:ALLOWED] ; SS override for allowed and EXITHOLD
36252 ; Set the allowed_ bits
36253 ;entry FATAL
36254 FATAL:
36255 ; 25/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
36256 ;MOV AL,[ES:BP+DPB.DRIVE]
36257 ;;MOV AL,[ES:BP+0] ; Get drive number
36258 ; 15/12/2022
36259 000060DB 268A4600 MOV AL,[ES:BP]
36260
36261 ;entry FATAL1
36262 FATAL1:
36263 ; 15/03/2018
36264 000060DF 368C06[8205] MOV [SS:EXITHOLD+2],ES
36265 000060E4 36892E[8005] MOV [SS:EXITHOLD],BP ; The only things we preserve
36266 ;LES SI,[ES:BP+12H] ; MSDOS 3.3
36267 ;LES SI,[ES:BP+13H] ; MSDOS 6.0 - 05/05/2019
36268 000060E9 26C47613 LES SI,[ES:BP+DPB.DRIVER_ADDR]
36269 000060ED 8CC5 MOV BP,ES ; BP:SI points to the device involved
36270
36271 ; DI has the INT-24-style extended error. We now map the error code
36272 ; for this into the normalized get extended error set by using the
36273 ; ErrMap24 table as a translate table. Note that we translate ONLY
36274 ; the device returned codes and leave all others beyond the look up
36275 ; table alone.
36276
36277 ; 08/07/2018 - Retro DOS v3.0
36278 FATALC:
36279 000060EF E89F01 call SET_I24_EXTENDED_ERROR
36280 ;cmp di,0Ch
36281 000060F2 83FF0C CMP DI,error_I24_gen_failure
36282 000060F5 7603 JBE short GOT_RIGHT_CODE ; Error codes above gen_failure get
36283 000060F7 BF0C00 MOV DI,error_I24_gen_failure; mapped to gen_failure. Real codes
36284 ; Only come via GetExtendedError
36285
36286 ;** -----
36287 ;
36288 ; Entry point used by REDIRECTOR on Network I 24 errors.
36289 ;
36290 ; ASSUME DS:NOTHING,ES:NOTHING,SS:DOSDATA
36291 ;
36292 ; ALL I 24 regs set up. ALL Extended error info SET. ALLOWED Set.
36293 ; EXITHOLD set for restore of ES:BP.
36294 ; -----
36295 ;entry NET_I24_ENTRY
36296 NET_I24_ENTRY:
36297 GOT_RIGHT_CODE:
36298 000060FA 36803E[2003]00 CMP BYTE [SS:ERRORMODE],0 ; No INT 24s if already INT 24
36299 00006100 7404 JZ SHORT NoSetFail
36300 00006102 B003 MOV AL,3
36301 00006104 EB74 JMP short FailRet
36302 NoSetFail:
36303 00006106 368926[8805] MOV [SS:CONTSTK],SP ; SS override
36304 0000610B 16 PUSH SS
36305 0000610C 07 POP ES
36306
36307 ; wango!!! We may need to free some user state info... In
36308 ; particular, we may have locked down a JFN for a user and he may
36309 ; NEVER return to us. Thus,we need to free it here and then
36310 ; reallocate it when we come back.
36311
36312 0000610D 36833E[AA05]FF CMP word [SS:SFN],-1 ; 0FFFFh
36313 00006113 740C JZ short _NoFree
36314 00006115 1E push ds
36315 00006116 56 push si
36316 00006117 36C536[AE05] LDS SI,[SS:PJFN]
36317 0000611C C604FF MOV BYTE [SI],0FFh
36318 0000611F 5E pop si

```

```

36319 00006120 1F          pop     ds
36320
36321 _NoFree:
36322 00006121 FA          CLI
36323
36324 00006122 36FE06[2003] INC     BYTE [SS:ERRORMODE] ; Prepare to play with stack
36325 00006127 36FE0E[2103] DEC     BYTE [SS:INDOS]      ; Flag INT 24 in progress
36326                                     ; INT 24 handler might not return
36327                                     ; 29/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
36328                                     ;;;
36329 0000612C 36FE0E[B812] dec     byte [ss:INDOS_FLAG] ; 2nd 'in dos' flag (what for?)
36330                                     ;;;
36331
36332                                     ; 05/05/2019 - Retro DOS v4.0 (MSDOS 6.0, MSDOS 6.21)
36333
36334                                     ;; Extended Open hooks
36335                                     ; AN000;IFS.I24 error disabled
36336                                     ;test byte [ss:EXTOPEN_ON],2
36337 00006131 36F606[F605]02 TEST    byte [ss:EXTOPEN_ON],EXT_OPEN_I24_OFF
36338 00006137 7404          JZ      short i24yes ; AN000;IFS.no
36339 faili24:                                     ; AN000;
36340 00006139 B003          MOV     AL,3 ; AN000;IFS.fake fail
36341 0000613B EB27          JMP     short passi24 ; AN000;IFS.exit
36342 i24yes:                                     ; AN000;
36343                                     ;; Extended Open hooks
36344
36345 0000613D 368E16[8605] MOV     SS,[SS:USER_SS]
36346 00006142 268B26[8405] MOV     SP,[ES:USER_SP] ; User stack pointer restored
36347
36348                                     ;;int 24h
36349                                     ;IN int_fatal_abort ; Fatal error interrupt vector,
36350                                     ; must preserve ES
36351                                     ; 05/05/2019
36352 00006147 26803E[660D]00 cmp     byte [es:DoshASHMA],0 ; Q: is dos running in HMA (M021)
36353 0000614D 7504          jne     short do_low_int24 ; Y: the int must be done from low mem
36354 0000614F CD24          INT     int_fatal_abort ; Fatal error interrupt vector,
36355                                     ; must preserve ES
36356 00006151 EB05          jmp     short criterr_ret_addr
36357
36358 do_low_int24:
36359                                     ; 05/05/2019
36360                                     ; MSDOS 6.0
36361 00006153 2EFF1E[FB5D] call    far [cs:LowInt24Addr]
36362 criterr_ret_addr:
36363 00006158 268926[8405] MOV     [ES:USER_SP],SP ; restore our stack
36364 0000615D 268C16[8605] MOV     [ES:USER_SS],SS
36365                                     ;MOV BP,ES
36366                                     ;MOV SS,BP
36367                                     ; 30/06/2024
36368 00006162 06          push    es
36369 00006163 17          pop     ss
36370 passi24:
36371 00006164 368B26[8805] MOV     SP,[SS:CONTSTK]
36372 00006169 36FE06[2103] INC     BYTE [SS:INDOS] ; Back in the DOS
36373
36374                                     ; 29/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
36375                                     ;;;
36376 0000616E 36FE06[B812] inc     byte [ss:INDOS_FLAG]
36377                                     ;;;
36378
36379 00006173 36C606[2003]00 MOV     BYTE [SS:ERRORMODE],0 ; Back from INT 24
36380 00006179 FB          STI
36381 FailRet:
36382 0000617A 36C42E[8005] LES     BP,[SS:EXITHOLD]
36383
36384                                     ; 08/07/2018
36385
36386                                     ; Triage the user's reply.
36387
36388 0000617F 3C01          CMP     AL,1
36389 00006181 723D          JB      short CheckIgnore ; 0 => ignore
36390 00006183 7445          JZ      short CheckRetry ; 1 => retry
36391 00006185 3C03          CMP     AL,3 ; 3 => fail
36392 00006187 7549          JNZ     short DoAbort ; 2, invalid => abort
36393
36394                                     ; The reply was fail. See if we are allowed to fail.
36395
36396                                     ; SS override for ALLOWED, EXTOPEN_ON,
36397                                     ; ALLOWED, FAILERR, WPERR, SFN, PJFN
36398
36399 00006189 36F606[4B03]08 ;test byte [ss:ALLOWED],8
36400 0000618F 7441          test    byte [ss:ALLOWED],Allowed_FAIL ; Can we?
36401 DoFail: jz      short DoAbort ; No, do abort
36402 00006191 B003          MOV     AL,3 ; just in case...
36403                                     ; AN000;EO. I24 error disabled
36404
36405                                     ; 05/05/2019
36406 00006193 36F606[F605]02 ;(MSDOS 6.0, MSCTRLC.ASM, 1991)
36407 00006199 7505          test    byte [ss:EXTOPEN_ON],EXT_OPEN_I24_OFF ; 2
36408 jnz     short Cleanup ; AN000;EO. no
36409 0000619B 36FE06[4A03] inc     byte [SS:FAILERR] ; Tell everybody
36410 Cleanup:
36411 000061A0 36C606[2203]FF MOV     byte [SS:WPERR],-1
36412 000061A6 36833E[AA05]FF CMP     word [SS:SFN],-1
36413                                     ; 25/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
36414 jnz     short Cleanup2
36415 ;retn
36416 ; 17/12/2022
36417 000061AC 7411          jz      short Cleanup_retn ; 08/07/2018 - Retro DOS v3.0
36418 Cleanup2:
36419 000061AE 1E          push    ds
36420 000061AF 56          push    si
36421 000061B0 50          push    ax
36422 000061B1 36A1[AA05] MOV     AX,[ss:SFN]
36423 000061B5 36C536[AE05] LDS     SI,[ss:PJFN]
36424 000061BA 8804          MOV     [SI],AL
36425 000061BC 58          pop     ax
36426 000061BD 5E          pop     si
36427 000061BE 1F          pop     ds
36428 Cleanup_retn:
36429 000061BF C3          retn
36430
36431                                     ; The reply was IGNORE. See if we are allowed to ignore.
36432
36433 CheckIgnore:
36434 ;test byte [ss:ALLOWED],20h
36435 000061C0 36F606[4B03]20 test    byte [ss:ALLOWED],Allowed_IGNORE ; Can we?
36436 CheckRI: ; 29/02/2024
36437 000061C6 74C9          jz      short DoFail ; No, do fail
36438 000061C8 EBD6          jmp     short Cleanup
36439
36440                                     ; The reply was RETRY. See if we are allowed to retry.
36441
36442 CheckRetry:

```



```

36443 ;test byte [ss:ALLOWED],10h
36444 000061CA 36F606[4B03]10 test byte [ss:ALLOWED],Allowed_RETRY ; Can we?
36445 ;jz short DoFail ; No, do fail
36446 ;JMP short Cleanup
36447 ; 29/02/2024 (PCDOS 7.1 IBMDOS.COM)
36448 000061D0 EBF4 jmp short CheckRI
36449
36450 ; The reply was ABORT.
36451 DoAbort:
36452 000061D2 16 push ss
36453 000061D3 1F pop ds
36454
36455 000061D4 803E[5703]00 CMP byte [CONSWAP],0
36456 000061D9 7403 JZ short NOSWAP2
36457 000061DB E87FD9 call SWAPBACK
36458 NOSWAP2:
36459 ; See if we are to truly abort. If we are in the process of aborting,
36460 ; turn this abort into a fail.
36461
36462 ;test [fAborting],0FFh
36463 ;jnz short DoFail
36464
36465 000061DE 803E[5903]00 cmp byte [fAborting],0
36466 000061E3 75AC JNZ short DoFail
36467
36468 ; Set return code
36469
36470 000061E5 C606[7C05]02 MOV BYTE [EXIT_TYPE],EXIT_HARD_ERROR ; 2
36471 000061EA 30C0 XOR AL,AL
36472
36473 ; we are truly aborting the process. Go restore information from
36474 ; the PDB as necessary.
36475
36476 000061EC E97510 jmp exit_inner
36477
36478 ;** -----
36479 ;
36480 ; reset_environment checks the DS value against the CurrentPDB. If they are
36481 ; different, then an old-style return is performed. If they are the same,
36482 ; then we release jfns and restore to parent. We still use the PDB at DS:0 as
36483 ; the source of the terminate addresses.
36484 ;
36485 ; Some subtlety: We are about to issue a bunch of calls that *may* generate
36486 ; INT 24s. We *cannot* allow the user to restart the abort process; we may
36487 ; end up aborting the wrong process or turn a terminate/stay/resident into a
36488 ; normal abort and leave interrupt handlers around. What we do is to set a
36489 ; flag that will indicate that if any abort code is seen, we just continue the
36490 ; operation. In essence, we dis-allow the abort response.
36491 ;
36492 ; output: none.
36493 ; -----
36494
36495 ;entry reset_environment
36496
36497 reset_environment:
36498 ; 30/07/2018 - Retro DOS v3.0
36499 ; IBMDOS.COM (MSDOS 3.3) - Offset 588Ah
36500
36501 ;***invoke Reset_Version ; AN007 ;MS. reset version number
36502
36503 000061EF 1E PUSH DS ; save PDB of process
36504
36505 ; There are no critical sections in force. Although we may enter
36506 ; here with critical sections locked down, they are no longer
36507 ; relevant. We may safely free all allocated resources.
36508
36509 000061F0 B482 MOV AH,82h
36510 ; Microsoft Networks - END DOS CRITICAL SECTIONS 0 THROUGH 7
36511 ;int 2Ah
36512 000061F2 CD2A INT int_IBM
36513
36514 ; SS override
36515 000061F4 36C606[5903]FF MOV byte [SS:fAborting],-1; signal abort in progress
36516
36517 ; DOS 4.00 doesn't need it
36518 ;CallInstall NetResetEnvironment, MultNET, 34
36519 ; Allow REDIR to clear some stuff
36520 ; On process exit.
36521 000061FA B82211 mov ax, 1122h
36522 000061FD CD2F int 2Fh ; Multiplex - NETWORK REDIRECTOR - PROCESS TERMINATION HOOK
36523 ; SS = DOS CS
36524 ;mov al,22h
36525 000061FF B022 MOV AL,int_terminate
36526 00006201 E865AD call _$GET_INTERRUPT_VECTOR; and who to go to
36527
36528 00006204 59 POP CX ; get ThisPDB
36529 00006205 06 push es
36530 00006206 53 push bx ; save return address
36531
36532 00006207 368B1E[3003] MOV BX,[SS:CurrentPDB] ; get currentPDB
36533 0000620C 8EDB MOV DS,BX
36534 0000620E A11600 MOV AX,[PDB.PARENT_PID] ; get parentPDB
36535
36536 ; AX = parentPDB, BX = CurrentPDB, CX = ThisPDB
36537 ; Only free handles if AX <> BX and BX = CX and [exit_code].upper
36538 ; is not Exit_keep_process
36539
36540 00006211 39D8 CMP AX,BX
36541 00006213 7418 JZ short reset_return ; parentPDB = CurrentPDB
36542 00006215 39CB CMP BX,CX
36543 00006217 7514 JNZ short reset_return ; CurrentPDB <> ThisPDB
36544 00006219 50 PUSH AX ; save parent
36545
36546 ; SS override
36547 ;cmp byte [SS:EXIT_TYPE],3
36548 0000621A 36803E[7C05]03 CMP BYTE [SS:EXIT_TYPE],EXIT_KEEP_PROCESS ; 15/08/2018
36549 00006220 7406 JZ short reset_to_parent ; keeping this process
36550
36551 ; We are truly removing a process. Free all allocation blocks
36552 ; belonging to this PDB
36553
36554 ;invoke arena_free_process
36555 00006222 E87E10 call arena_free_process
36556
36557 ; Kill off remainder of this process. Close file handles and signal
36558 ; to relevant network folks that this process is dead. Remember that
36559 ; CurrentPDB is STILL the current process!
36560
36561 ;invoke DOS_ABORT
36562 00006225 E888D4 call DOS_ABORT
36563
36564 reset_to_parent:
36565 ; SS override
36566 00006228 368F06[3003] POP word [SS:CurrentPDB] ; set up process as parent

```

```

36567
36568
36569
36570 0000622D 16
36571 0000622E 1F
36572
36573 0000622F B0FF
36574
36575
36576
36577
36578
36579 00006231 E8AEB6
36580
36581 00006234 E86108
36582
36583 00006237 E8D5B6
36584
36585
36586
36587
36588
36589
36590
36591
36592
36593 0000623A FA
36594 0000623B C606[2103]00
36595
36596
36597
36598 00006240 C606[B812]00
36599
36600
36601 00006245 C606[2203]FF
36602 0000624A C606[5903]00
36603 0000624F 8F06[8005]
36604 00006253 8F06[8205]
36605
36606
36607
36608 00006257 8E1E[3003]
36609 0000625B 8E1E3000
36610 0000625F 8B262E00
36611
36612 00006263 E8DBA1
36613
36614
36615 00006266 07
36616
36617
36618 00006267 50
36619 00006268 8CD8
36620
36621 0000626A 2E8E1E[0700]
36622 0000626F A3[630D]
36623 00006272 58
36624
36625
36626 00006273 A3[8405]
36627
36628 00006276 58
36629 00006277 58
36630 00006278 58
36631
36632
36633
36634
36635
36636
36637
36638 00006279 9F
36639 0000627A 86E0
36640 0000627C 2402
36641 0000627E B4F2
36642
36643
36644
36645
36646 00006280 50
36647
36648
36649
36650
36651
36652 00006281 FF36[8205]
36653 00006285 FF36[8005]
36654
36655
36656
36657
36658 00006289 A1[8405]
36659 0000628C 8E1E[630D]
36660
36661 00006290 CF
36662
36663
36664
36665
36666
36667
36668
36669
36670
36671
36672
36673
36674 00006291 50
36675
36676 00006292 B8[B80E]
36677 00006295 2D[AB0E]
36678
36679
36680
36681
36682
36683 00006298 1E
36684
36685 00006299 2E8E1E[0700]
36686
36687
36688
36689
36690 0000629E 39C7

reset_return:
;Context DS
push ss
pop ds
MOV AL,-1

; make sure that everything is clean In this case ignore any errors,
; we cannot "FAIL" the abort, the program being aborted is dead.

;EnterCrit critDisk
call ECritDisk
;invoke FLUSHBUF
call FLUSHBUF
;LeaveCrit critDisk
call LCritDisk

; 29/02/2024 - Retrto DOS v5.0
; PCDOS 7.1 IBMDOS.COM
%if 0
; Decrement open ref. count if we had done a virtual open earlier.
call CHECK_VIRT_OPEN
%endif

CLI
MOV BYTE [INDOS],0 ; Go to known state

; 29/02/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
;;;
mov byte [INDOS_FLAG],0
;;;

MOV BYTE [WPERR],-1 ; Forget about WP error
MOV byte [fAborting],0 ; let aborts occur
POP WORD [EXITHOLD]
POP WORD [EXITHOLD+2]

; Snake into multitasking... Get stack from CurrentPDB person
MOV DS,[CurrentPDB]
MOV SS,[PDB.USER_STACK+2]
MOV SP,[PDB.USER_STACK]

call restore_world

; 05/05/2019
pop es ; * ; MSDOS 6.21 (DOSCODE:94A8h, MSDOS.SYS)

; MSDOS 6.0
push ax
mov ax,ds ; set up ds, but save ds in TEMPSEG
; and not on stack.
; getdseg <ds> ; ds -> dosdata
mov ds,[cs:DosDSeg]
mov [TEMPSEG],ax
pop ax

; set up ds to DOSDATA
;MOV [CS:USER_SP],AX ; MSDOS 3.3
mov [USER_SP],ax

POP AX
POP AX
POP AX ; suck off CS:IP of interrupt...

; M011 : BEGIN

; MSDOS 3.3
; MOV AX,0F202h ; STI

; MSDOS 6.0
LAHF
XCHG AH,AL
AND AL,2
MOV AH,0F2h

; M011 : END

; MSDOS 3.3 (& MSDOS 6.0)
PUSH AX

;PUSH word [CS:EXITHOLD+2]
;PUSH word [CS:EXITHOLD]

; MSDOS 6.0
PUSH word [EXITHOLD+2]
PUSH word [EXITHOLD]

;MOV AX,[CS:USER_SP]

; MSDOS 6.0
MOV AX,[USER_SP]
mov ds,[TEMPSEG] ; restore ds

IRET ; Long return back to user terminate address

;-----
;
;
; Procedure Name : SET_I24_EXTENDED_ERROR
;
; This routine handles extended error codes.
; Input : DI = error code from device
; Output: All EXTERR fields are set
;-----

SET_I24_EXTENDED_ERROR:
PUSH AX
; ErrMap24End is in DOSDATA
MOV AX,ErrMap24End
SUB AX,ErrMap24
; Change to dosdata to access
; ErrMap24 and EXTERR -SR

; 05/05/2019 - Retro DOS v4.0

; MSDOS 6.0
push ds
;getdseg <ds> ; ds ->dosdata
mov ds,[cs:DosDSeg]

; AX is the index of the first unavailable error. Do not translate
; if greater or equal to AX.
CMP DI,AX

```



```

36691 000062A0 89F8      MOV     AX,DI
36692 000062A2 7306      JAE     short NoTrans
36693
36694      ;MOV     AL,[CS:DI+ErrMap24] ; MSDOS 3.3
36695 000062A4 8A85[AB0E] mov     al,[ErrMap24+di] ; MSDOS 6.0
36696 000062A8 30E4      XOR     AH,AH
36697 NoTrans:
36698      ;MOV     [CS:EXTERR],AX
36699 000062AA A3[2403]   mov     [EXTERR],AX
36700 000062AD 1F        pop     ds
36701      ;assume ds:nothing
36702 000062AE 58        POP     AX
36703
36704      ; Now Extended error is set correctly. Translate it to get correct
36705      ; error locus class and recommended action.
36706
36707 000062AF 56        PUSH    SI
36708      ; ERR_TABLE_24 is in DOSCODE
36709 000062B0 BE[5B0E]   MOV     SI,ERR_TABLE_24
36710 000062B3 E8FBA3   call    CAL_LK ; Set other extended error fields
36711 000062B6 5E        POP     SI
36712 000062B7 C3        retn
36713
36714      ;=====
36715      ; FAT.ASM, MSDOS 6.0, 1991
36716      ;=====
36717      ; 30/07/2018 - Retro DOS v3.0
36718      ; 20/05/2019 - Retro DOS v4.0
36719      ; 02/03/2024 - Retro DOS v5.0
36720
36721      ; TITLE FAT - FAT maintenance routines
36722      ; NAME FAT
36723
36724      ** FAT.ASM
36725      ;-----
36726      ; Low level local device routines for performing disk change sequence,
36727      ; setting cluster validity, and manipulating the FAT
36728      ;
36729      ; IsEof
36730      ; UNPACK
36731      ; PACK
36732      ; MAPCLUSTER
36733      ; FATREAD_SFT
36734      ; FATREAD_CDS
36735      ; FAT_operation
36736      ;
36737      ; Revision history:
36738      ;
36739      ; AN000 version Jan. 1988
36740      ; A001 PTM -- disk changed for look ahead buffers
36741      ;
36742      ; M014 - if a request for pack\unpack cluster 0 is made we write\read
36743      ; from CL0FATENTRY rather than disk.
36744      ;
36745      ; DOSCODE:94FAh (MSDOS 6.21, MSDOS.SYS)
36746
36747      ; 02/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
36748      ; PCDOS 7.1 IBMDOS.COM - DOSCODE:A40Ch
36749
36750      ;Break <IsEOF - check the quantity in BX for EOF>
36751      ;-----
36752      ;
36753      ; Procedure Name : IsEOF
36754      ;
36755      ; IsEOF - check the fat value in BX for eof.
36756      ;
36757      ; Inputs: ES:BP point to DPB
36758      ; BX has fat value
36759      ; Outputs: JAE eof
36760      ; Registers modified: none
36761      ;-----
36762
36763      IsEOF:
36764      ; 02/03/2024 - Retro DOS v5.0
36765      ; PCDOS 7.1 IBMDOS.COM
36766      ;;;
36767      call    IsFAT32 ; is it FAT32 drive ?
36768 000062B8 E89801   jnc     short IsEOF_FAT ; no, it has 12 or 16 bit FAT
36769 000062BB 730D
36770
36771      ; FAT32 fs
36772 000062BD 36813E[E80A]FF0F cmp     word [ss:CLUSTNUM_HW],0FFFh
36773 000062C4 7503     jnz     short IsEOF_other1 ; not EOF
36774 000062C6 83FBF8   cmp     bx,0FFF8h ; 32 bit compare
36775      IsEOF_other1:
36776 000062C9 C3        retn ; cf=0 -> EOF, cf=1 -> not EOF
36777
36778      IsEOF_FAT:
36779      ;;;
36780
36781      ;cmp     word [es:bp+0Dh],0FF6h
36782 000062CA 26817E0DF60F   CMP     word [ES:BP+DPB.MAX_CLUSTER],4096-10 ; is this 16 bit fat?
36783 000062D0 730B     JAE     short EOF16 ; yes, check for eof there
36784
36785      ;J.K. 8/27/86
36786      ;Modified to accept 0FF0h as an eof. This is to handle the diskfull case
36787      ;of any media that has "F0"(Other) as a MediaByte.
36788      ;Hopefully, this does not create any side effect for those who may use any value
36789      ;other than "FF8-FFF" as an EOF for their own file.
36790
36791 000062D2 81FBF00F   cmp     bx,0FF0h
36792 000062D6 7404     je     short IsEOF_other
36793
36794 000062D8 81FBF80F   CMP     BX,0FF8h ; do the 12 bit compare
36795      IsEOF_other:
36796 000062DC C3        retn
36797      EOF16:
36798 000062DD 83FBF8   CMP     BX,0FFF8h ; 16 bit compare
36799 000062E0 C3        retn ; cf=0 -> EOF, cf=1 -> not EOF
36800
36801      ; DOSCODE:9511h (MSDOS 6.21, MSDOS.SYS)
36802
36803      ;Break <UNPACK -- UNPACK FAT ENTRIES>
36804      ;-----
36805      ;
36806      ; Procedure Name : UNPACK
36807      ;
36808      ; Inputs:
36809      ; BX = Cluster number (may be full 16-bit quantity)
36810      ; ES:BP = Base of drive parameters
36811      ; Outputs:
36812      ; DI = Contents of FAT for given cluster (may be full 16-bit quantity)
36813      ; Zero set means DI=0 (free cluster)
36814      ; Carry set means error (currently user FAILED to I 24)

```

```

36815 ; SI Destroyed, No other registers affected. Fatal error if cluster too big.
36816 ;
36817 ; NOTE: if BX = 0 then DI = contents of CL0FATENTRY
36818 ;
36819 ;-----
36820 ;
36821 ; 02/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
36822 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:0A44Eh
36823 ;
36824 ; (MSDOS 6.22 MSDOS .SYS - DOSCODE:9511h)
36825 ; (Windows ME IO.SYS - BIOSCODE:0C368h)
36826 ;
36827 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
36828 ; DOSCODE:94B5h (MSDOS 5.0, MSDOS.SYS)
36829 ;
36830 ; 20/05/2019 - Retro DOS v4.0
36831 UNPACK:
36832 ; MSDOS 6.0 ; M014 - Start
36833 or bx,bx ; Q: are we unpacking cluster 0
36834 jnz short up_cont ; N: proceed with normal unpack
36835 ;
36836 ; 02/03/2024
36837 %if 0
36838 mov di,[CL0FATENTRY] ; Y: return value in CL0FATENTRY
36839 or di,di ; return z if di=0
36840 retn ; done
36841 %else
36842 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:A435h
36843 ;;;
36844 up_1:
36845 cmp [CLUSTNUM_HW],bx ; 0
36846 jne short up_cont
36847 mov di,[CL0FATENTRY_HW]
36848 or di,di
36849 mov [CCONTENT_HW],di
36850 mov di,[CL0FATENTRY]
36851 jnz short unpack_retn
36852 or di,di
36853 unpack_retn:
36854 retn ; [CCONTENT_HW]:DI = contents of CL0FATENTRY
36855 ;;;
36856 %endif
36857
36858 up_cont: ; M014 - End
36859 ; 02/03/2024 (PC DOS 7.1 IBMDOS.COM)
36860 ;;;
36861 ; cmp word [es:bp+0Fh],0
36862 cmp word [es:bp+DPB.FAT_SIZE],0
36863 jnz short up_fat ; not FAT32
36864 ; FAT32
36865 ; mov si,[es:bp+2Fh]
36866 mov si,[es:bp+DPB.LAST_CLUSTER+2]
36867 cmp [CLUSTNUM_HW],si
36868 jne short up_2
36869 ; cmp bx,[es:bp+2Dh]
36870 cmp bx,[es:bp+DPB.LAST_CLUSTER]
36871 jmp short up_2
36872 up_fat:
36873 ;;;
36874 ;
36875 ; MSDOS 3.3 & MSDOS 6.0
36876 ; cmp bx,[es:bp+0Dh]
36877 cmp bx,[es:bp+DPB.MAX_CLUSTER]
36878 up_2: ; 02/03/2024
36879 JA short HURTFAT
36880 CALL MAPCLUSTER
36881 jc short _DoContext
36882 ;
36883 ; 02/03/2024 (PC DOS 7.1 IBMDOS.COM)
36884 ;;;
36885 ; push word [di+2] ; offset CLUSSAVE+2
36886 pop word [ss:CCONTENT_HW] ; high word of cluster number
36887 ;;;
36888 ;
36889 MOV DI,[DI]
36890 JNZ short High12 ; MZ if high 12 bits, go get 'em
36891 ;
36892 ; 02/03/2024 (PC DOS 7.1 IBMDOS.COM)
36893 ;;;
36894 ; call IsFAT32
36895 jc short up_fat32 ; FAT32 volume (drive)
36896 mov word [ss:CCONTENT_HW],0
36897 ;;;
36898 ;
36899 MOV SI,[ES:BP+DPB.MAX_CLUSTER] ; MZ is this 16-bit fat?
36900 CMP SI,4096-10 ; 0FF6h
36901 JB short Unpack12 ; MZ No, go 'AND' off bits
36902 ;
36903 ; 02/03/2024
36904 %if 0
36905 OR DI,DI ; MZ set zero condition code, clears carry
36906 JMP SHORT _DoContext ; MZ go do context
36907 High12:
36908 SHR DI,1
36909 SHR DI,1
36910 SHR DI,1
36911 SHR DI,1
36912 %else
36913 ; 02/03/2024 (PC DOS 7.1 IBMDOS.COM)
36914 ;;;
36915 up_fat32:
36916 or di,di ; set zero condition code, clears carry
36917 ; mov si,ss
36918 ; mov ds,si
36919 push ss
36920 pop ds
36921 jnz short up_retn
36922 or [CCONTENT_HW],di ; [CCONTENT_HW]:DI = 0 -> zf = 1
36923 up_retn:
36924 retn
36925 ;
36926 High12:
36927 mov word [ss:CCONTENT_HW],0
36928 shr di,1
36929 shr di,1
36930 shr di,1
36931 shr di,1
36932 shr di,1
36933 ;;;
36934 %endif
36935
36936 Unpack12:
36937 AND DI,0FFFh ; Clears carry
36938 _DoContext:

```

```

36939 00006360 16      PUSH    SS
36940 00006361 1F      POP     DS
36941 00006362 C3      retn
36942
36943 HURTFAT:
36944
36945 ; 02/03/2024
36946 %if 0
36947 ;;mov word [es:bp+1Eh],0FFFFh
36948 ;;mov word [es:bp+1Fh],0FFFFh ; MSDOS 6.0
36949 MOV     word [ES:BP+DPB.FREE_CNT],-1 ; Err in FAT must force recomp of freespace
36950 %else
36951 ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
36952 ;;;
36953 00006363 E84C02     call    chk_set_first_access
36954 ;;;
36955 %endif
36956
36957 00006366 50      PUSH    AX
36958 00006367 B488     MOV     AH,Allowed_FAIL+80h ; 88h
36959
36960 ; 02/03/2024
36961 %if 0
36962
36963 ;hkn; SS override
36964 MOV     byte [SS:ALLOWED],Allowed_FAIL ; 8
36965 ;
36966 ; Signal Bad FAT to INT int_fatal_abort handler. We have an invalid cluster.
36967 ;
36968 MOV     DI,0FFFFh ; In case INT int_fatal_abort returns (it shouldn't)
36969 call    FATAL
36970 %else
36971 ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
36972 ;;;
36973 ;mov byte [ss:ALLOWED],8
36974 00006369 36C606[4B03]08 mov     byte [ss:ALLOWED],Allowed_FAIL
36975 ;
36976 ; 02/03/2024 - Retro DOS v5.0
36977 0000636F 31FF     xor     di,di
36978 00006371 4F      dec     di ; 0FFFFh
36979 00006372 57      push    di
36980 ; NOTE: DI would be 0FFFFh here (or 0FFFFh as in MSDOS 6.?)
36981 ; because, in SET_I24_EXTENDED_ERROR in FATAL,
36982 ; DI is compared with error code
36983 ;
36984 ; xor di,di
36985 ; dec di ; 0FFFFh
36986 ; push di
36987 ; call FATAL
36988 ; pop di
36989 ; mov word [ss:CCONTENT_HW], di
36990 ;
36991 ;
36992 ; Erdogan Tan - 02/03/2024 - 01/07/2024
36993 00006373 36FF36[E80A]     push    word [ss:CLUSTNUM_HW] ; (necessary!?)
36994 00006378 53      push    bx ; (necessary!?)
36995 00006379 E85FFD     call    FATAL
36996 0000637C 5B      pop     bx
36997 0000637D 368F06[E80A]     pop     word [ss:CLUSTNUM_HW]
36998 ;xor di,di
36999 ;dec di ; 0FFFFh
37000 ;mov word [ss:CCONTENT_HW],di
37001 ; 02/03/2024 - Retro DOS v5.0
37002 ;pop word [ss:CCONTENT_HW]
37003 ; 01/07/2024
37004 00006382 5F      pop     di
37005 00006383 36893E[EC0A]     mov     [ss:CCONTENT_HW], di
37006 ;;;
37007 %endif
37008 00006388 3C03     CMP     AL,3
37009 0000638A F8      CLC
37010 0000638B 7501     JNZ     short OKU_RET ; Try to ignore bad FAT
37011 0000638D F9      STC ; User said FAIL
37012
37013 0000638E 58      OKU_RET: POP     AX
37014 hurtfat_retn:
37015 0000638F C3      retn
37016
37017 ; DOSCODE:9565h (MSDOS 6.21, MSDOS.SYS)
37018
37019 ;Break <PACK -- PACK FAT ENTRIES>
37020 ;-----
37021 ;
37022 ; Procedure Name : PACK
37023 ;
37024 ; Inputs:
37025 ; BX = Cluster number
37026 ; DX = Data
37027 ; ES:BP = Pointer to drive DPB
37028 ; Outputs:
37029 ; The data is stored in the FAT at the given cluster.
37030 ; SI,DX,DI all destroyed
37031 ; Carry set means error (currently user FAILED to I 24)
37032 ; No other registers affected
37033 ;
37034 ; NOTE: if BX = 0 then data in DX is stored in CLOFATENTRY.
37035 ;
37036 ;-----
37037
37038 ; 02/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
37039
37040 ; INPUT:
37041 ; [CLUSDATA_HW]:DX = cluster data/content
37042 ; [CLUSTNUM_HW]:BX = cluster number (to be updated)
37043 ; ES:BP = pointer to DPB
37044
37045 ; 25/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
37046 ; 20/05/2019 - Retro DOS v4.0
37047 PACK:
37048 ; MSDOS 6.0 ; M014 - start
37049 00006390 09DB     or      bx,bx ; Q: are we packing cluster 0
37050 00006392 7513     jnz     short p_cont ; N: proceed with normal pack
37051
37052 ; 02/03/2024
37053 %if 0
37054 mov     [CLOFATENTRY],dx ; Y: place value in CLOFATENTRY
37055 retn ; done
37056 %else
37057 ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37058 ;;;
37059 00006394 391E[E80A]     cmp     [CLUSTNUM_HW],bx ; 0
37060 00006398 750D     jnz     short p_cont
37061
37062 ;push ax

```

```

37063             ;mov     ax,[CLUSDATA_HW]      ; Cluster Data/Content (High word)
37064             ;mov     [CLOFATENTRY_HW],ax
37065             ;pop     ax
37066 0000639A FF36[EA0A]      push    word [CLUSDATA_HW]
37067 0000639E 8F06[3B0F]      pop     word [CLOFATENTRY_HW]
37068
37069 000063A2 8916[8100]      mov     [CLOFATENTRY],dx
37070 000063A6 C3             retn
37071             ;;;
37072 %endif
37073
37074 p_cont:                                     ; M014 - end
37075             ; MSDOS 3.3 & MSDOS 6.0
37076 000063A7 E8BE00          CALL    MAPCLUSTER
37077 000063AA 72B4             JC      short _DoContext
37078 000063AC 8B35             MOV     SI,[DI]
37079 000063AE 740B             JZ      short ALIGNED          ; byte (not nibble) aligned
37080 000063B0 51             PUSH    CX                    ; move data to upper 12 bits
37081 000063B1 B104             MOV     CL,4
37082 000063B3 D3E2             SHL     DX,CL
37083 000063B5 59             POP     CX
37084 000063B6 83E60F          AND     SI,0FH                ; leave in original low 4 bits
37085 000063B9 EB2B             JMP     SHORT PACKIN
37086
37087 ALIGNED:
37088
37089             ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37090             ;;;
37091 000063BB E89500          call    IsFAT32
37092 000063BE 7313             jnb     short p_fat
37093
37094 000063C0 368B36[EA0A]      mov     si,[ss:CLUSDATA_HW]
37095 000063C5 337502          xor     si,[di+2]            ; hw of cluster number
37096                                     ; offset CLUSSAVE+2
37097 000063C8 81E6FF0F          and     si,0FFFh
37098 000063CC 317502          xor     [di+2],si
37099 000063CF 8915             mov     [di],dx
37100 000063D1 EB17             jmp     short PACKIN2
37101
37102 p_fat:
37103             ;;;
37104             ; cmp     word [es:bp+0Dh],0FF6h
37105 000063D3 26817E0DF60F      CMP     word [ES:BP+DPB.MAX_CLUSTER],4096-10 ; MZ 16 bit fats?
37106 000063D9 7309             JAE     short Pack16        ; MZ yes, go clobber original data
37107 000063DB 81E600F0          AND     SI,0F000h          ; MZ leave in upper 4 bits of original
37108             ; AND     DX,0FFFh          ; MZ store only 12 bits
37109             ; 01/07/2024
37110 000063DF 80E60F          and     dh,0Fh
37111 000063E2 EB02             JMP     SHORT PACKIN        ; MZ go store
37112
37113 Pack16:
37114             XOR     SI,SI          ; MZ no original data
37115 000063E6 09D6             OR      SI,DX
37116 000063E8 8935             MOV     [DI],SI
37117
37118 PACKIN2:      ; 02/03/2024
37119
37120 ;hkn; SS override
37121 000063EA 36C536[E205]      LDS     SI,[SS:CURBUF]
37122             ; MSDOS 6.0
37123 000063EF F6440540          TEST    byte [SI+BUFFINFO.buf_flags],buf_dirty
37124                                     ;LB. if already dirty ;AN000;
37125 000063F3 7507             JNZ     short yesdirty11    ;LB. don't increment dirty count ;AN000;
37126             ; 10/06/2019
37127 000063F5 E8D207          call    INC_DIRTY_COUNT    ;LB. ;AN000;
37128
37129             ;or     byte [si+5],40h
37130 000063F8 804C0540          OR      byte [SI+BUFFINFO.buf_flags],buf_dirty
37131                                     ;LB. ;AN000;
37132 yesdirty11: ;hkn; SS override
37133 000063FC 36803E[7805]00      CMP     BYTE [SS:CLUSSPLIT],0 ; 15/08/2018
37134 ;hkn; SS is DOSDATA
37135             push    ss
37136 00006403 1F             pop     ds
37137 00006404 7489             jz      short hurtfat_retn    ; Carry clear
37138 00006406 50             PUSH    AX
37139             ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37140             ;;;
37141 00006407 FF36[E80A]      push    word [CLUSTNUM_HW]
37142             ;;;
37143 0000640B 53             PUSH    BX
37144 0000640C 51             PUSH    CX
37145 0000640D A1[8E05]      MOV     AX,[CLUSSAVE]
37146 00006410 8E1E[E405]      MOV     DS,[CURBUF+2]
37147             ;;;add si,16 ; MSDOS 3.3
37148             ;;;add si,20 ; MSDOS 6.0
37149             ; 02/01/2024
37150             ;add     si,24 ; PCDOS 7.1
37151 00006414 83C618          ADD     SI,BUFINSIZ
37152 00006417 8824             MOV     [SI],AH
37153 ;hkn; SS is DOSDATA
37154             ;Context DS
37155 00006419 16             push    ss
37156 0000641A 1F             pop     ds
37157
37158 0000641B 50             PUSH    AX
37159
37160             ; MSDOS 6.0
37161 0000641C 8B16[9205]      MOV     DX,[CLUSSEC+2]        ;F.C. >32mb ;AN000;
37162 00006420 8916[0706]      MOV     [HIGH_SECTOR],DX    ;F.C. >32mb ;AN000;
37163
37164             ; MSDOS 3.3 & MSDOS 6.0
37165 00006424 8B16[9005]      MOV     DX,[CLUSSEC]
37166
37167             ;MOV     SI,1          ; *
37168             ;XOR     AL,AL          ; *
37169             ;call    GETBUFFRB      ; *
37170             ; 22/09/2023
37171 00006428 E8F704          call    GETBUFFRA ; *
37172
37173 0000642B 58             POP     AX
37174 0000642C 721B             JC      short POPP_RET
37175 0000642E C53E[E205]      LDS     DI,[CURBUF]
37176
37177             ; MSDOS 6.0
37178 00006432 F6450540          TEST    byte [DI+BUFFINFO.buf_flags],buf_dirty
37179                                     ;LB. if already dirty ;AN000;
37180 00006436 7507             JNZ     short yesdirty12    ;LB. don't increment dirty count ;AN000;
37181 00006438 E88F07          call    INC_DIRTY_COUNT    ;LB. ;AN000;
37182
37183             ;or     byte [di+5],40h
37184 0000643B 804D0540          OR      byte [DI+BUFFINFO.buf_flags],buf_dirty
37185 yesdirty12:
37186             ;;;add di,16

```

```

37187             ;;add di,20 ; MSDOS 6.0
37188             ;ADD DI,BUFINSIZ
37189             ;DEC DI
37190             ; 02/01/2024 - Retro DOS v5.0
37191             ;add di,23 ; PC DOS 7.1
37192 0000643F 83C717 add di,BUFINSIZ-1 ; 23 ; PC DOS 7.1 IBMDOS.COM
37193
37194             ;add di,[es:bp+2]
37195 00006442 26037E02 ADD DI,[ES:BP+DPB.SECTOR_SIZE]
37196 00006446 8805 MOV [DI],AL
37197 00006448 F8 CLC
37198 POPP_RET:
37199             ; 02/03/2024
37200 00006449 16 PUSH SS
37201 0000644A 1F POP DS
37202 0000644B 59 POP CX
37203 0000644C 5B POP BX
37204             ; 02/03/2024 (PC DOS 7.1 IBMDOS.COM)
37205             ;;;
37206             ;pop word [ss:CLUSTNUM_HW]
37207             ; 01/07/2024
37208             ; ds = ss
37209 0000644D 8F06[E80A] pop word [CLUSTNUM_HW]
37210             ;;;
37211 00006451 58 POP AX
37212 00006452 C3 retn
37213
37214             ; ===== S U B R O U T I N E =====
37215
37216             ; 02/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
37217             ; PC DOS 7.1 IBMDOS.COM - DOSCODE:0A5B9h
37218 IsFAT32:
37219             ;cmp word [es:bp+0Fh],1
37220 00006453 26837E0F01 cmp word [es:bp+DPB.FAT_SIZE],1
37221 00006458 730D jnb short isfat32eof_2 ; not FAT32
37222
37223             ;cmp word [es:bp+2Fh],0
37224 0000645A 26837E2F00 cmp word [es:bp+DPB.LAST_CLUSTER+2],0
37225 0000645F 7505 jnz short isfat32eof_1 ; FAT32
37226
37227             ;cmp word [es:bp+2Dh],0FFF6h
37228 00006461 26837E2DF6 cmp word [es:bp+DPB.LAST_CLUSTER],0FFF6h
37229 isfat32eof_1:
37230 00006466 F5 cmc ; cf=1 -> FAT32
37231 isfat32eof_2:
37232 00006467 C3 retn
37233
37234             ;-----
37235
37236             ; 31/07/2018 - Retro DOS v3.0
37237
37238 ;Break <MAPCLUSTER - BUFFER A FAT SECTOR>
37239
37240             ;-----
37241             ; Procedure Name : MAPCLUSTER
37242             ;
37243             ; Inputs:
37244             ; ES:BP Points to DPB
37245             ; BX Is cluster number
37246             ; Function:
37247             ; Get a pointer to the cluster
37248             ; Outputs:
37249             ; DS:DI Points to contents of FAT for given cluster
37250             ; DS:SI Points to start of buffer
37251             ; Zero Not set if cluster data is in high 12 bits of word
37252             ; Zero set if cluster data is in low 12 or 16 bits
37253             ; Carry set if failed.
37254             ; SI is destroyed.
37255             ;
37256             ;-----
37257
37258             ; 20/05/2019 - Retro DOS v4.0
37259             ; DOSCODE:9601h (MSDOS 6.21, MSDOS.SYS)
37260             ; 27/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
37261             ; DOSCODE:95A5h (MSDOS 5.0, MSDOS.SYS)
37262
37263             ; 02/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
37264             ; PC DOS 7.1 IBMDOS.COM - DOSCODE:0A5D7h
37265
37266 MAPCLUSTER:
37267             ; IBMDOS.COM (MSDOS 3.3, 1987) - Offset 5A15h
37268 00006468 C606[7805]00 MOV BYTE [CLUSPLIT],0
37269             ;SAVE <AX,BX,CX,DX>
37270 0000646D 50 push ax
37271 0000646E 53 push bx
37272 0000646F 51 push cx
37273 00006470 52 push dx
37274             ; 02/03/2024
37275             ;MOV AX,BX ; AX = BX
37276
37277             ; 02/03/2024 (PC DOS 7.1 IBMDOS.COM)
37278             ;;;
37279             ;push word [CLUSTNUM_HW]
37280 00006471 89D8 mov ax,bx
37281 00006473 8B16[E80A] mov dx,[CLUSTNUM_HW]
37282
37283 00006477 52 push dx ; 02/03/2024 - Retro DOS v5.0
37284
37285 00006478 31FF xor di,di
37286 0000647A E8D6FF call IsFAT32
37287 0000647D 730E jnc short mapcl1 ; not FAT32
37288             ; FAT32
37289 0000647F D1E0 shl ax,1
37290 00006481 D1D2 rcl dx,1
37291 00006483 D1D7 rcl di,1
37292 00006485 D1E0 shl ax,1
37293 00006487 D1D2 rcl dx,1
37294 00006489 D1D7 rcl di,1
37295             ; DI:DX:AX = 4*(DX:AX)
37296 0000648B EB14 jmp short Map16 ; 32 bit FAT
37297 mapcl1:
37298             ;;;
37299 0000648D 26817E0DF60F CMP word [ES:BP+DPB.MAX_CLUSTER],4096-10 ; MZ 16 bit fat?
37300             ; 02/03/2024 - Retro DOS v5.0
37301             ;JAE short Map16 ; MZ yes, do 16 bit algorithm
37302             ;SHR AX,1 ; AX = BX/2
37303
37304             ; 02/03/2024 (PC DOS 7.1 IBMDOS.COM)
37305             ;;;
37306 00006493 7304 jae short mapcl2 ; 16 bit FAT
37307             ; 12 bit FAT
37308 00006495 D1EA shr dx,1
37309 00006497 D1D8 rcr ax,1
37310 mapcl2:

```

```

37311 00006499 01D8      add     ax,bx
37312 0000649B 1316[E80A]    adc     dx,[CLUSTNUM_HW]
37313 0000649F 11FF      adc     di,di
37314      ;;;
37315 Map16:
37316
37317 ; 02/04/2024 - Retro DOS v5.0
37318 ; (PCDOS 7.1 IBMDOS.COM)
37319 %if 0
37320 ; MSDOS 6.0
37321 XOR     DI,DI ; *
37322 ; MZ skip prev => AX=2*BX
37323 ; >32mb fat ;AN000;
37324 ; MSDOS 3.3 (& MSDOS 6.0)
37325 ADD     AX,BX
37326 ADC     DI,DI ; * MSDOS 6.0
37327 %endif ; AX = 1.5*fat = byte offset in fat
37328 000064A1 268B4E02    MOV     CX,[ES:BP+DPB.SECTOR_SIZE]
37329
37330 ;IF FastDiv
37331 ;
37332 ; Gross hack: 99% of all disks have 512 bytes per sector. We test for this
37333 ; case and apply a really fast algorithm to get the desired results
37334 ;
37335 ; Divide method takes 157+4*4=173 (MOV and DIV)
37336 ; Fast method takes 39+20*4=119
37337 ;
37338 ; This saves a bunch.
37339
37340 000064A5 81F90002    CMP     CX,512
37341 000064A9 751C      jne     short _DoDiv
37342 ; 02/04/2024
37343 %if 0
37344 ;
37345 MOV     DX,AX
37346 AND     DX,512-1
37347 MOV     AL,AH
37348 ; MDOS 3.3
37349 ;shr     al,1
37350 ; MDOS 6.0
37351 shr     di,1
37352 rcr     al,1
37353 ; MDOS 3.3 (& MSDOS 6.0)
37354 xor     ah,ah
37355 %else
37356 ; 02/04/2024 - Retro DOS v5.0
37357 ; (PCDOS 7.1 IBMDOS.COM)
37358 ;;;
37359 000064AB 50      push    ax
37360 000064AC D1EF      shr     di,1
37361 000064AE D1DA      rcr     dx,1
37362 000064B0 D1D8      rcr     ax,1
37363 000064B2 88E0      mov     al,ah
37364 000064B4 88D4      mov     ah,dl
37365 000064B6 88F2      mov     dl,dh
37366 000064B8 30F6      xor     dh,dh
37367 000064BA D1EF      shr     di,1
37368 000064BC 10F6      adc     dh,dh
37369 000064BE 87D7      xchg    dx,di
37370 000064C0 5A      pop     dx
37371 000064C1 81E2FF01    and     dx,1FFh
37372 ;;;
37373 %endif
37374 000064C5 EB07      jmp     short DivDone
37375
37376 _DoDiv:
37377 ;ENDIF
37378
37379 ; 02/04/2024
37380 %if 0
37381 ; MSDOS 3.3
37382 ;xor     dx,dx
37383 ; MSDOS 6.0
37384 mov     dx,di
37385 ; MSDOS 3.3 (& MSDOS 6.0)
37386 DIV     CX
37387 %else
37388 ; 02/04/2024 - Retro DOS v5.0
37389 ; (PCDOS 7.1 IBMDOS.COM)
37390 ;;;
37391 000064C7 97      xchg    ax,di
37392 000064C8 92      xchg    ax,dx
37393 000064C9 F7F1      div     cx
37394 000064CB 97      xchg    ax,di
37395 000064CC F7F1      div     cx
37396 ;jmp     short DivDone
37397 ;;;
37398 %endif
37399
37400 ;IF FastDiv
37401 DivDone:
37402 ;ENDIF
37403 ;add     ax,[es:bp+6]
37404 000064CE 26034606    ADD     AX,[ES:BP+DPB.FIRST_FAT]
37405
37406 ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37407 ;;;
37408 000064D2 83D700    adc     di,0
37409 ;;;
37410
37411 000064D5 49      DEC     CX
37412 ; CX is sector size - 1
37413
37414 ;SAVE <AX,DX,CX>
37415
37416 ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37417 ;;;
37418 000064D6 57      push    di ; 4*
37419 ;;;
37420 000064D7 50      push    ax ; 3*
37421 000064D8 52      push    dx ; 2*
37422 000064D9 51      push    cx ; 1*
37423
37424 MOV     DX,AX
37425
37426 ; 02/03/2024 - Retro DOS v5.0
37427 000064DC 89FB      mov     [HIGH_SECTOR],di ; *!* ; PCDOS 7.1 IBMDOS.COM
37428 mov     bx,di
37429 ;
37430 ; MSDOS 6.0
37431 ; 22/09/2023
37432 ;MOV     word [HIGH_SECTOR],0 ; *!* ;F.C. >32mb low sector #
37433 ;
37434 ; MDOS 3.3 (& MSDOS 6.0)

```



```

37435 ;XOR AL,AL ; *
37436 ;MOV SI,1 ; *
37437 ;;invoke GETBUFFRB ; *
37438 ;call GETBUFFRB ; *
37439 ; 22/09/2023
37440 000064DE E83D04 call GETBUFFRC ; *! ; *!* ; 02/03/2024 - Retro DOS v5.0
37441
37442 ;RESTORE <CX,AX,DX> ; CX is sec siz-1, AX is offset in sec
37443 000064E1 59 pop cx ; 1*
37444 000064E2 58 pop ax ; 2*
37445 000064E3 5A pop dx ; 3*
37446 ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37447 ;;;
37448 000064E4 5B pop bx ; 4*
37449 ;;;
37450
37451 000064E5 725A JC short MAP_POP
37452
37453 000064E7 C536[E205] LDS SI,[CURBUF]
37454 ;;;lea di,[si+16]
37455 ;;;lea di,[si+20] ; MSDOS 6.0
37456 ;lea di,[si+24] ; PCDOS 7.1; 02/03/2024
37457 000064EB 8D7C18 LEA DI,[SI+BUFINSIZ]
37458 000064EE 01C7 ADD DI,AX
37459 000064F0 39C8 CMP AX,CX
37460 000064F2 7530 JNZ short MAPRET ; ds<>ss
37461 000064F4 8A05 MOV AL,[DI]
37462 ;Context DS ;hkn; SS is DOSDATA
37463 000064F6 16 push ss
37464 000064F7 1F pop ds
37465 000064F8 FE06[7805] INC BYTE [CLUSSPLIT]
37466 000064FC A2[8E05] MOV [CLUSSAVE],AL
37467 000064FF 8916[9005] MOV [CLUSSEC],DX
37468
37469 ; MSDOS 6.0
37470 ;MOV WORD [CLUSSEC+2],0 ;F.C. >32mb ;AN000;
37471 ;INC DX
37472 ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37473 ;;;
37474 00006503 31C0 xor ax,ax ; 0
37475 00006505 83C201 add dx,1
37476 00006508 891E[9205] mov word [CLUSSEC+2],bx
37477 0000650C 11C3 adc bx,ax
37478 ;mov [HIGH_SECTOR],bx ; *!*
37479 ;;;
37480
37481 ; 22/09/2023
37482 ;MOV word [HIGH_SECTOR],0 ; *! ;F.C. >32mb FAT sector <32mb ;AN000;
37483 ;
37484 ; MDOS 3.3 (& MSDOS 6.0)
37485 ;XOR AL,AL ; *
37486 ;MOV SI,1 ; *
37487 ;;invoke GETBUFFRB ; *
37488 ;call GETBUFFRB ; *
37489 ; 22/09/2023
37490 0000650E E80D04 call GETBUFFRC ; *! ; *!* ; 02/03/2024 - Retro DOS v5.0
37491 00006511 722E JC short MAP_POP
37492
37493 00006513 C536[E205] LDS SI,[CURBUF]
37494 00006517 8D7C18 LEA DI,[SI+BUFINSIZ]
37495 0000651A 8A05 MOV AL,[DI]
37496 ;Context DS ;hkn; SS is DOSDATA
37497 0000651C 16 push ss
37498 0000651D 1F pop ds
37499 0000651E A2[8F05] MOV [CLUSSAVE+1],AL
37500
37501 ;hkn; CLUSSAVE is in DOSDATA
37502 00006521 BF[8E05] MOV DI,CLUSSAVE
37503 MAPRET:
37504 ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37505 ;;;
37506 00006524 368F06[E80A] pop word [ss:CLUSTNUM_HW] ; (ds<>ss)
37507 ;;;
37508 ;RESTORE <DX,CX,BX>
37509 00006529 5A pop dx
37510 0000652A 59 pop cx
37511 0000652B 5B pop bx
37512
37513 0000652C 31C0 XOR AX,AX ; MZ allow shift to clear carry
37514
37515 ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37516 ;;;
37517 0000652E E822FF call IsFAT32
37518 00006531 720A jc short MapSet ; FAT32 fs
37519 ;;;
37520
37521 00006533 26817E0DF60F CMP word [ES:BP+DPB.MAX_CLUSTER],4096-10 ; MZ is this 16-bit fat?
37522 00006539 7302 JAE short MapSet ; MZ no, set flags
37523 0000653B 89D8 MOV AX,BX
37524 MapSet:
37525 0000653D A801 TEST AL,1 ; set zero flag if not on boundary
37526 ;RESTORE <AX>
37527 0000653F 58 pop ax
37528 00006540 C3 retn
37529
37530 MAP_POP:
37531 ; 02/03/2024 (PCDOS 7.1 IBMDOS.COM)
37532 ;;;
37533 ;pop word [ss:CLUSTNUM_HW]
37534 ; 02/03/2024 - Retro DOS v5.0
37535 00006541 8F06[E80A] pop word [CLUSTNUM_HW] ; (ds=ss)
37536 ;;;
37537 ;RESTORE <DX,CX,BX,AX>
37538 00006545 5A pop dx
37539 00006546 59 pop cx
37540 00006547 5B pop bx
37541 00006548 58 pop ax
37542 fatread_sft_retn: ; 17/12/2022
37543 00006549 C3 retn
37544
37545 ; 20/05/2019 - Retro DOS v4.0
37546 ; DOSCODE:96B3h (MSDOS 6.21, MSDOS.SYS)
37547 ; 27/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
37548 ; DOSCODE:9657h (MSDOS 5.0, MSDOS.SYS)
37549
37550 ; 03/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
37551 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0A6C6h
37552
37553 ;Break <FATREAD_SFT/FATREAD_CDS -- CHECK DRIVE GET FAT>
37554 ;-----
37555 ;
37556 ; Procedure Name : FATREAD_SFT
37557 ;
37558 ; Inputs:

```

```

37559 ; ES:DI points to an SFT for the drive of interest (local only,
37560 ; giving a NET SFT will produce system crashing results).
37561 ; DS DOSDATA
37562 ; Function:
37563 ; Can be used by an SFT routine (like CLOSE) to invalidate buffers
37564 ; if disk changed.
37565 ; In other respects, same as FATREAD_CDS.
37566 ; (note ES:DI destroyed!)
37567 ; Outputs:
37568 ; Carry set if error (currently user FAILED to I 24)
37569 ; NOTE: This routine may cause FATREAD_CDS to "miss" a disk change
37570 ; as far as invalidating curdir_ID is concerned.
37571 ; Since getting a true disk changed on this call is a screw up
37572 ; anyway, that's the way it goes.
37573 ;
37574 ;-----
37575
37576 FATREAD_SFT:
37577 0000654A 26C46D07 LES BP,[ES:DI+SF_ENTRY.sf_devptr]
37578
37579 fatread_gotdpb: ; 03/03/2024 (PCDOS 7.1 IBMDOS.COM)
37580 ; 27/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
37581 ; MOV AL,[ES:BP+DPB.DRIVE] ; [es:bp+0]
37582 ; 15/12/2022
37583 0000654E 268A4600 mov AL,[ES:BP]
37584 00006552 A27605 MOV [THISDRV],AL
37585 00006555 E80DA1 call GOTDPB ; Set THISDPB
37586 ; CALL FAT_GOT_DPB
37587 ; 17/12/2022
37588 00006558 E99C00 jmp FAT_GOT_DPB
37589 ;fatread_sft_retn:
37590 ;retn
37591
37592 ;-----
37593 ;
37594 ; Procedure Name : FATREAD_CDS
37595 ;
37596 ; Inputs:
37597 ; DS:DOSDATA
37598 ; ES:DI points to an CDS for the drive of interest (local only,
37599 ; giving a NET or NUL CDS will produce system crashing results).
37600 ; Function:
37601 ; If disk may have been changed, media is determined and buffers are
37602 ; flagged invalid. If not, no action is taken.
37603 ; Outputs:
37604 ; ES:BP = Drive parameter block
37605 ; THISDPB = ES:BP
37606 ; THISDRV set
37607 ; Carry set if error (currently user FAILED to I 24)
37608 ; DS preserved , all other registers destroyed
37609 ;
37610 ;-----
37611 ;
37612 ; 20/05/2019 - Retro DOS v4.0
37613 ; DOSCODE:96C5h (MSDOS 6.21, MSDOS.SYS)
37614 ; 27/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
37615 ; DOSCODE:9669h (MSDOS 5.0, MSDOS.SYS)
37616 ;
37617 ; 03/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
37618 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0A6D8h
37619 ; (Windows ME IO.SYS - BIOSCODE:0C644h)
37620
37621 FATREAD_CDS:
37622 0000655B 06 PUSH ES
37623 0000655C 57 PUSH DI
37624
37625 ;les bp,[es:di+45h]
37626 0000655D 26C46D45 LES BP,[ES:DI+curdir.devptr]
37627
37628 ; 03/03/2024
37629 %if 0
37630 ; 27/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
37631 ; MOV AL,[ES:BP+DPB.DRIVE] ; [es:bp+0]
37632 ; 15/12/2022
37633 mov AL,[ES:BP]
37634 MOV [THISDRV],AL
37635 call GOTDPB ;Set THISDPB
37636 CALL FAT_GOT_DPB
37637 %else
37638 ; 03/03/2024 (PCDOS 7.1 IBMDOS.COM)
37639 ;;;
37640 00006561 E8EAFB call fatread_gotdpb
37641 ;;;
37642 %endif
37643 00006564 5F POP DI ;Get back CDS pointer
37644 00006565 07 POP ES
37645 00006566 72E1 jc short fatread_sft_retn
37646 00006568 7542 JNZ short NO_CHANGE ;Media NOT changed
37647
37648 ; Media changed. We now need to find all CDS structures which use this
37649 ; DPB and invalidate their ID pointers.
37650
37651 MED_CHANGE:
37652 ;XOR AX,AX
37653 ;DEC AX ; AX = -1
37654 ; 03/03/2024
37655 0000656A B8FFFF mov ax,0FFFFh ; -1
37656
37657 0000656D 1E PUSH DS
37658 0000656E 8A0E4700 MOV CL,[CDSCOUNT]
37659 00006572 30ED XOR CH,CH ; CX is number of structures
37660 ;lds si,[es:di+45h]
37661 00006574 26C57545 LDS SI,[ES:DI+curdir.devptr] ; Find all CDS with this devptr
37662
37663 ;hkn; SS override
37664
37665 ; Find all CDSs with this DevPtr
37666 ;
37667 ; (ax) = -1
37668 ; (ds:si) = DevPtr
37669
37670 00006578 36C43E3C00 LES DI,[SS:CDSADDR] ; (es:di) = CDS pointer
37671 frcd20:
37672 ;;test word [es:di+43h],8000h
37673 ;TEST word [ES:DI+curdir.flags],curdir_isnet
37674 0000657D 26F6454480 TEST byte [ES:DI+curdir.flags+1],(curdir_isnet>>8)
37675 00006582 7522 JNZ short frcd25 ; Leave NET guys alone!!
37676
37677 ; MSDOS 3.3
37678 ;push es
37679 ;push di
37680 ;les di,[es:di+45h]
37681 ;;les di,[ES:DI+curdir.devptr]
37682 ;call POINTCOMP

```



```

37683             ;pop     di
37684             ;pop     es
37685             ;jnz     short frcd25
37686
37687             ; MSDOS 6.0
37688             cmp     si,[ES:DI+curdir.devptr]
37689             jne     short frcd25             ; no match
37690             mov     bx,ds
37691             cmp     bx,[ES:DI+curdir.devptr+2]
37692             jne     short frcd25             ; CDS not for this drive
37693
37694             ; MSDOS 3.3 (& MSDOS 6.0)
37695             ;test    [es:di+49h],ax ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0A70Fh
37696             test    [ES:DI+curdir.ID],AX
37697             ;JZ      short frcd25 ; = 0 ; If root (0), leave root
37698             ; !!! test es:[di+49h],ax
37699             ; !!! jz short frcd25
37700             ; PCDOS 7.1 IBMDOS.COM bug (!?) ; Erdogan Tan - 03/03/2024
37701             ; test dword ptr es:[di+49h],-1 ; win ME - BIOSCODE:0C694h
37702             ; jz short frcd25
37703             ;
37704             ; 03/03/2024 - Retro DOS v5.0
37705             jnz     short frcd24
37706
37707             ; 03/03/2024 (PCDOS 7.1 IBMDOS.COM)
37708             ;;;
37709             ;test    [es:di+4Bh],ax
37710             test    [es:di+curdir.ID+2],AX
37711             jz      short frcd25 ; = 0
37712             frcd24: ; 03/03/2024
37713             ;;;
37714             ;mov     [es:di+49h],ax
37715             MOV     [ES:DI+curdir.ID],AX ; else invalid
37716             ; 03/03/2024 (PCDOS 7.1 IBMDOS.COM)
37717             ;;;
37718             ;mov     [es:di+4Bh],ax
37719             mov     [es:di+curdir.ID+2],ax ; -1
37720             ;;;
37721             frcd25:
37722             ;add     di,81 ; MSDOS 3.3
37723             ;add     di,88 ; MSDOS 6.0
37724             ADD     DI,curdir.size ; Point to next CDS
37725             LOOP    frcd20
37726             POP     DS
37727
37728             NO_CHANGE:
37729             LES     BP,[THISDPB]
37730             CLC
37731             retn
37732
37733             ; ===== S U B R O U T I N E =====
37734             ; 05/01/2024 - Retro DOS 5.0
37735             ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0A7Fh
37736             ;;;
37737             chk_set_first_access:
37738             ;cmp     word [es:bp+0Fh],0
37739             cmp     word [es:bp+DPB.FAT_SIZE],0
37740             jnz     short chk_set_fa_1 ; FAT (FAT12 or FAT16)
37741             ; FAT32
37742             ;cmp     word [es:bp+21h],0FFFFh
37743             cmp     word [es:bp+DPB.FREE_CNT_HW],0FFFFh ; -1
37744             ;mov     word [es:bp+21h],0FFFFh ; High word of free cluster count
37745             mov     word [es:bp+DPB.FREE_CNT_HW],0FFFFh ; -1
37746             jne     short chk_set_fa_2
37747             chk_set_fa_1:
37748             ;cmp     word [es:bp+1Fh],0FFFFh
37749             cmp     word [es:bp+DPB.FREE_CNT],0FFFFh ; -1
37750             chk_set_fa_2:
37751             ;mov     word [es:bp+1Fh],0FFFFh ; Count of free clusters, -1 if unknown
37752             mov     word [es:bp+DPB.FREE_CNT],0FFFFh ; -1
37753             je      short chk_set_fa_3
37754             ;or     byte [es:bp+18h],1
37755             or     byte [es:bp+DPB.FIRST_ACCESS],1
37756             chk_set_fa_3:
37757             retn
37758             ;;;
37759             ;-----
37760
37761             ;Break <Fat_Operation - miscellaneous fat stuff>
37762             ;-----
37763             ;
37764             ; Procedure Name : FAT_operation
37765             ;-----
37766
37767             ; 27/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
37768
37769             ; 04/03/2024
37770             ; 03/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
37771
37772
37773             FAT_operation:
37774             ; 31/07/2018 - Retro DOS v3.0
37775             FATERR:
37776
37777             ; 03/03/2024
37778             %if 0
37779             ;mov     word [es:bp+1Eh],-1
37780             ;mov     word [es:bp+1Fh],-1 ; MSDOS 6.0
37781             MOV     word [ES:BP+DPB.FREE_CNT],-1
37782             ; Err in FAT must force recomp of freespace
37783             %else
37784             ; 03/03/2024 (PCDOS 7.1 IBMDOS.COM)
37785             ;;;
37786             call     chk_set_first_access
37787             ;;;
37788             %endif
37789             ;and     di,0FFh
37790             AND     DI,STECODE ; Put error code in DI
37791             ;mov     byte [ALLOWED],18h
37792             MOV     byte [ALLOWED],Allowed_FAIL+Allowed_RETRY
37793             ;mov     ah,1Ah
37794             MOV     AH,2+Allowed_FAIL+Allowed_RETRY ; while trying to read FAT
37795             MOV     AL,[THISDRV] ; Tell which drive
37796             call     FATAL1
37797             LES     BP,[THISDPB]
37798             CMP     AL,3
37799             JNZ     short FAT_GOT_DPB ; User said retry
37800
37801             FATERR_fail: ; 03/03/2024
37802             STC ; User said FAIL
37803             retn
37804
37805             FAT_GOT_DPB:
37806             ;Context DS ;hkn; SS is DOSDATA

```

```

37807 000065F7 16      push    ss
37808 000065F8 1F      pop     ds
37809
37810                ; 03/03/2024 (PCDOS 7.1 IBMDOS.COM)
37811                ;;
37812 000065F9 8CC0      mov     ax,es
37813 000065FB 09E8      or      ax,bp
37814 000065FD 74F6      jz      short FATERR_fail
37815                ;;
37816
37817                ;;mov    al,0Fh
37818                ;MOV     AL,DMEDHL
37819                ; 03/03/2024 (PCDOS 7.1 IBMDOS.COM)
37820                ;;
37821 000065FF B013      mov     al,19
37822                ;;
37823
37824                ;mov     ah,[es:bp+1]
37825 00006601 268A6601      MOV     AH,[ES:BP+DPB.UNIT]
37826 00006605 A3[5A03]      MOV     [DEVCALL_REQLEN],AX ; 09/09/2018
37827 00006608 C606[5C03]01  MOV     BYTE [DEVCALL_REQFUNC],DEVMDCH
37828 0000660D C706[5D03]0000  MOV     word [DEVCALL_REQSTAT],0
37829                ;;mov    al,[es:bp+16h]
37830                ;mov     al,[es:bp+17h] ; MSDOS 6.0
37831 00006613 268A4617      MOV     AL,[ES:BP+DPB.MEDIA]
37832 00006617 A2[6703]      MOV     [CALLMED],AL
37833 0000661A 06      PUSH    ES
37834 0000661B 1E      PUSH    DS
37835
37836                ;hkn; DEVCALL is in DOSDATA
37837 0000661C BB[5A03]      MOV     BX,DEVCALL
37838                ;;lds     si,[es:bp+12h]
37839                ;lds     si,[es:bp+13h] ; MSDOS 6.0
37840 0000661F 26C57613      LDS     SI,[ES:BP+DPB.DRIVER_ADDR] ; DS:SI Points to device header
37841 00006623 07      POP     ES ; ES:BX Points to call header
37842 00006624 E86EEC      call    DEVIOCALL2
37843                ;Context DS ;hkn; SS is DOSDATA
37844 00006627 16      push    ss
37845 00006628 1F      pop     ds
37846 00006629 07      POP     ES ; Restore ES:BP
37847 0000662A 8B3E[5D03]      MOV     DI,[DEVCALL_REQSTAT]
37848                ;test    di,8000h
37849                ;jnz     short FATERR
37850 0000662E 09FF      or      di,di
37851 00006630 78A7      js      short FATERR ; have error
37852
37853                ; 03/03/2024
37854                %if 0
37855                XOR     AH,AH
37856                ;xchg    ah,[es:bp+17h] ; MSDOS 3.3
37857                ;xchg    ah,[es:bp+18h] ; MSDOS 6.0
37858                XCHG    AH,[ES:BP+DPB.FIRST_ACCESS] ; Reset dpb_first_access
37859                %else
37860                ; 03/03/2024 (PCDOS 7.1 IBMDOS.COM)
37861                ;;
37862                ;mov     ah,[es:bp+18h]
37863 00006632 268A6618      mov     ah,[es:bp+DPB.FIRST_ACCESS]
37864 00006636 80E480      and     ah,80h ; isolate (FAT) first access bit
37865                ;and     byte [es:bp+18h],7Fh
37866 00006639 268066187F      and     byte [es:bp+DPB.FIRST_ACCESS],7Fh
37867                ; clear first access (FAT) bit 7
37868                ;;
37869                %endif
37870
37871 0000663E A0[7605]      MOV     AL,[THISDRV] ; Use physical unit number
37872                ; See if we had changed volume id by creating one on the diskette
37873 00006641 3806[A10A]      cmp     [VOLCHNG_FLAG],AL
37874 00006645 7508      jnz     short CHECK_BYT
37875 00006647 C606[A10A]FF      mov     byte [VOLCHNG_FLAG],-1 ; 0FFh
37876 0000664C E9B000      jmp     GOGETBPB ; Need to get device driver to read in
37877                ; new volume label.
37878
37879 0000664F 0A26[6803]      CHECK_BYT:
37880                OR      AH,[CALLRBYT]
37881                ;JNS     short CHECK_ZR ; ns = 0 or 1
37882                ;JMP     short NEWDSK
37883                ; 17/12/2022
37884 00006653 785E      js      short NEWDSK
37885                ; 27/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
37886                ;JNS     short CHECK_ZR ; ns = 0 or 1
37887                ;JMP     short NEWDSK
37888
37889 00006655 743B      CHECK_ZR:
37890                JZ      short CHKBUFFDIRT ; jump if I don't know
37891                ; 24/09/2023
37892                ; cf=0 (after 'or' instruction)
37893 00006657 C3      ;CLC
37894                retn ; If Media not changed (NZ)
37895
37896 00006658 06      DISK_CHNG_ERR:
37897 00006659 55      PUSH    ES
37898                PUSH    BP
37899                ;;les     bp,[es:bp+12h]
37900 0000665A 26C46E13      ;les     bp,[es:bp+13h] ; MSDOS 6.0
37901                LES     BP,[ES:BP+DPB.DRIVER_ADDR] ; Get device pointer
37902                ;;test    word [es:bp+4],800h
37903 0000665E 26F6460508      ;TEST    word [ES:BP+SYSDEV.ATT],DEVOPCL ; Did it set vol id?
37904 00006663 5D      test    byte [es:bp+SYSDEV.ATT+1],(DEVOPCL>>8)
37905 00006664 07      POP     BP
37906                POP     ES
37907                ;JZ      short FAIL_OPJ2 ; Nope, FAIL
37908                ; 03/03/2024
37909 00006665 7477      jz      short FAIL_OP
37910 00006667 1E      PUSH    DS ; Save buffer pointer for ignore
37911 00006668 57      PUSH    DI
37912 00006669 16      push    ss ;hkn; SS is DOSDATA
37913 0000666A 1F      pop     ds
37914 0000666B C606[4B03]18      ;mov     byte [ALLOWED],18h
37915 00006670 06      MOV     byte [ALLOWED],Allowed_FAIL+Allowed_RETRY
37916 00006671 C43E[6903]      PUSH    ES
37917 00006675 8C06[2A03]      LES     DI,[CALLVIDM] ; Get volume ID pointer
37918 00006679 07      MOV     [EXTERRPT+2],ES
37919 0000667A 893E[2803]      POP     ES
37920                MOV     [EXTERRPT],DI
37921 0000667E B80F00      ;mov     ax,0Fh
37922 00006681 C606[7505]01      MOV     AX,error_I24_wrong_disk
37923                MOV     byte [READOP],1 ; Write
37924                ;invoke    HARDERR
37925 00006686 E8CBF9      call    HARDERR
37926 00006689 5F      POP     DI ; Get back buffer for ignore
37927 0000668A 1F      POP     DS
37928 0000668B 3C03      CMP     AL,3
37929 0000668D 744F      FAIL_OPJ2:
37930 0000668F E965FF      JZ      short FAIL_OP
37931                JMP     FAT_GOT_DPB ; Retry

```

```

37931
37932
37933
37934
37935
37936
37937
37938
37939
37940
37941
37942
37943 00006692 833E[7100]00
37944
37945 00006697 741A
37946 00006699 E81D02
37947
37948
37949 0000669C 384504
37950 0000669F 7509
37951
37952 000066A1 F6450540
37953 000066A5 7403
37954
37955
37956
37957 000066A7 16
37958 000066A8 1F
37959
37960
37961
37962 000066A9 C3
37963
37964
37965
37966
37967
37968
37969
37970 000066AA 8B3D
37971
37972
37973
37974
37975
37976
37977
37978 000066AC 36393E[7E11]
37979 000066B1 75E9
37980
37981
37982
37983
37984
37985 000066B3 26C7461FFFFFF
37986
37987
37988
37989
37990 000066B9 26837E0F00
37991 000066BE 7506
37992
37993 000066C0 26C74621FFFF
37994
37995
37996
37997
37998
37999
38000 000066C6 E8F001
38001
38002
38003
38004
38005
38006 000066C9 384504
38007 000066CC 7513
38008
38009 000066CE F6450540
38010 000066D2 7584
38011
38012 000066D4 C74504FF20
38013 000066D9 E8EF01
38014
38015 000066DC EB05
38016
38017
38018
38019
38020
38021
38022 000066DE F9
38023
38024
38025
38026 000066DF EBC6
38027
38028
38029 000066E1 8B3D
38030
38031
38032
38033 000066E3 363B3E[7E11]
38034 000066E8 75DF
38035
38036 000066EA 36833E[7700]00
38037 000066F0 740D
38038 000066F2 363A06[7511]
38039 000066F7 7506
38040 000066F9 36C606[7511]FF
38041
38042
38043
38044
38045
38046
38047
38048
38049 000066FF 26C57E13
38050
38051
38052
38053 00006703 F6450520
38054 00006707 7533

CHKBUFFDIRT:
; 20/05/2019 - Retro DOS v4.0
; MSDOS 3.3
;lds di,[BUFFHEAD]
; MSDOS 6.0
;cmp word [ss:DirtyBufferCount],0 ; any dirty buffers ? ;hkn;
; 03/03/2024 - Retro DOS v5.0
; ds=ss
; ;
; ;
;cmp word [DirtyBufferCount],0 ; (win ME IO.SYS - BIOSCODE:0C7A7h)
; ;
;je short NEWDSK ; no, skip the check
;call GETCURHEAD ; get pointer to first buffer
nbuffer:
;cmp al,[di+4]
;cmp [di+BUFFINFO.buf_ID],al ; Unit OK ?
;jne short lfnxt ; no, go for next buffer
;test byte [di+5],40h
TEST byte [di+BUFFINFO.buf_flags],buf_dirty ; is the buffer dirty ?
;jz short lfnxt ; no, go for next buffer

FAIL_OP2: ; 03/03/2024
;Context DS
push ss
pop ds
; 24/09/2023
; cf=0 (after 'test' instruction)
;clc
;retn

lfnxt:
; 15/08/2018 - Retro DOS v3.0
; MSDOS 3.3
;lds di,[di]
; 20/05/2019 - Retro DOS v4.0
;mov di,[di]
; ;mov di,[di+BUFFINFO.buf_next] ; get next buffer
; MSDOS 3.3
;cmp di,-1
;jne short nbuffer
; MSDOS 6.0
;cmp [ss:FIRST_BUFF_ADDR],di ; is this where we started ?;hkn;
;jne short nbuffer ; no, check this guy also

; If no dirty buffers, assume Media changed
NEWDSK:
; ;mov word [es:bp+1Eh],0FFFFh ; MSDOS 3.3
; ;mov word [es:bp+1Fh],0FFFFh ; MSDOS 6.0
;mov word [ES:BP+DPB.FREE_CNT],-1 ; Media changed, must
; ;recompute
; 03/03/2024 (PCDOS 7.1 IBMDOS.COM)
; ;
; ;
;cmp word [es:bp+0Fh],0
;cmp word [es:bp+DPB.FAT_SIZE],0 ; = 0 for FAT32 fs
;jnz short newdsk2 ; not FAT32
;mov word [es:bp+21h],0FFFFh
;mov word [ES:BP+DPB.FREE_CNT_HW],0FFFFh ; -1
newdsk2:
; ;
; ;
; MSDOS 3.3
;call SETVISIT
; MSDOS 6.0
;call GETCURHEAD
nxbuffer:
; MSDOS 3.3
;or byte [di+5],20h
; MSDOS 3.3 & MSDOS 6.0
;cmp [di+4],al
;cmp [DI+BUFFINFO.buf_ID],al ; This drive ?
;jne short lfnxt2
;test byte [di+5],40h
TEST byte [DI+BUFFINFO.buf_flags],buf_dirty
;jnz short DISK_CHNG_ERR
;mov word [di+4],20FFh
;mov word [DI+BUFFINFO.buf_ID],(buf_visit*256)+0FFh ; free up
;call SCANPLACE
; MSDOS 6.0
;jmp short skpbuff

; 03/03/2024 - Retro DOS v5.0
FAIL_OP: ; This label & code is here
;Context DS ; for reachability
;push ss
;pop ds
;STC
; 03/03/2024
;retn
FAIL_OP2J: ; 03/03/2024
;jmp short FAIL_OP2 ; cf=1

lfnxt2:
;mov di,[di]
;mov di,[di+BUFFINFO.buf_next]
skpbuff:
; MSDOS 6.0
;cmp di,[ss:FIRST_BUFF_ADDR] ;hkn;
;jne short nxbuffer
; MSDOS 3.3
; ;
;CMP word [ss:SC_CACHE_COUNT],0 ;LB. look ahead buffers ? ;AN001;
;JZ short GOETBPB ;LB. no ;AN001;
;CMP AL,[ss:CurSC_DRIVE] ;LB. same as changed drive ;AN001;
;JNZ short GOETBPB ;LB. no ;AN001;
;MOV byte [ss:CurSC_DRIVE],-1 ;LB. invalidate look ahead buffers ;AN000;
;lfnxt2:
; MSDOS 3.3
;call SKIPVISIT
;jnz short nxbuffer
GOETBPB:
; MSDOS 3.3 & MSDOS 6.0
; ;lds di,[es:bp+12h]
; ;lds di,[es:bp+13h] ; MSDOS 6.0
;LDS DI,[ES:BP+DPB.DRIVER_ADDR]
; 20/05/2019
;test word [di+4],2000h
;TEST word [DI+SYSDEV.ATT],ISFATBYDEV
;TEST byte [DI+SYSDEV.ATT+1],(ISFATBYDEV>>8)
;JNZ short GETFREEBUF

```

```

38055 ;context DS ;hkn; SS is DOSDATA
38056 00006709 16 push ss
38057 0000670A 1F pop ds
38058
38059 ; 03/03/2024 (PCDOS 7.1 IBMDOS.COM)
38060 ;;;
38061 ;mov word [es:bp+2],200h
38062 0000670B 26C746020002 mov word [es:bp+DPB.SECTOR_SIZE],512 ; bytes per sector
38063 ;mov word [es:bp+6],1
38064 00006711 26C746060100 mov word [es:bp+DPB.FIRST_FAT],1 ; starting sector of FATS
38065 ;mov byte [es:bp+8],1
38066 00006717 26C6460801 mov byte [es:bp+DPB.FAT_COUNT],1 ; number of FATS
38067 ;mov word [es:bp+0Dh],3
38068 0000671C 26C7460D0300 mov word [es:bp+DPB.MAX_CLUSTER],3 ; cluster count + 1
38069 ;mov word [es:bp+0Fh],1
38070 00006722 26C7460F0100 mov word [es:bp+DPB.FAT_SIZE],1 ; FAT sectors (16 bit)
38071 00006728 C706[E80A]0000 mov word [CLUSTNUM_HW],0 ; high word of cluster number (for UNPACK)
38072 ;;;
38073
38074 0000672E BB0200 MOV BX,2
38075 00006731 E8ADFB CALL UNPACK ; Read the first FAT sector into CURBUF
38076 FAIL_OPJ:
38077 ;JC short FAIL_OP
38078 ; 03/03/2024
38079 00006734 72A9 JC short FAIL_OP2J ; cf=1
38080 00006736 C53E[E205] LDS DI,[CURBUF]
38081 0000673A EB22 JMP SHORT GOTGETBUF
38082
38083 GETFREEBUF:
38084 ; 03/03/2024 (PCDOS 7.1 IBMDOS.COM)
38085 ;;;
38086 0000673C 31D2 xor dx,dx ; 0 ; *
38087 0000673E 36C53E[7A00] lds di,[ss:LowMemBuff]
38088 ;sub di,24
38089 00006743 83EF18 sub di,BUFINSIZ
38090 00006746 363816[7900] cmp [ss:BuffInHMA],di ; 0
38091 0000674B 7511 jnz short GOTGETBUF ; buffer is in HMA
38092 ; use [LowMemBuff] to transfer at first
38093 ;;;
38094
38095 0000674D 06 PUSH ES ; Get a free buffer for BIOS to use
38096 0000674E 55 PUSH BP
38097 ; MSDOS 3.3
38098 ;LDS DI,[SS:BUFFHEAD] ; 15/08/2018
38099 ; 03/03/2024 ; *
38100 ; MSDOS 6.0
38101 ;XOR DX,DX ; * ;LB. fake to get 1st ;AN000;
38102 ;hkn; SS override
38103 0000674F 368916[0706] MOV [SS:HIGH_SECTOR],DX ; 0 ;LB. buffer addr ;AN000;
38104 00006754 E86201 call GETCURHEAD ;LB. ;AN000;
38105 ; MSDOS 3.3 & MSDOS 6.0
38106 00006757 E8C203 call BUFWRITE
38107 0000675A 5D POP BP
38108 0000675B 07 POP ES
38109 ;JC short FAIL_OPJ
38110 ;JC short FAIL_OP
38111 ; 03/03/2024 - Retro DOS v5.0
38112 0000675C 7281 JC short FAIL_OP2J ; cf=1
38113
38114 GOTGETBUF:
38115 ;;;add di,16
38116 ;add di,20 ; MSDOS 6.0
38117 ;add di,24 ; PC DOS 7.1 ; 03/03/2024
38118 0000675E 83C718 ADD DI,BUFINSIZ
38119
38120 ;hkn; SS override
38121 00006761 368C1E[6A03] MOV [SS:CALLXAD+2],DS
38122 ;Context DS ;hkn; SS is DOSDATA
38123 00006766 16 push ss
38124 00006767 1F pop ds
38125 00006768 893E[6803] MOV [CALLXAD],DI
38126 ;mov al,16h
38127 0000676C B016 MOV AL,DBPBHL ; 22
38128 ;mov ah,[es:bp+1]
38129 0000676E 268A6601 MOV AH,[ES:BP+DPB.UNIT]
38130 00006772 A3[5A03] MOV [DEVCALL_REQLEN],AX ; 09/09/2018
38131 00006775 C606[5C03]02 MOV BYTE [DEVCALL_REQFUNC],DEVBPB ; 2
38132 0000677A C706[5D03]0000 MOV word [DEVCALL_REQSTAT],0
38133 ;mov al,[es:bp+16h]
38134 ;mov al,[es:bp+17h]
38135 00006780 268A4617 MOV AL,[ES:BP+DPB.MEDIA]
38136 00006784 A2[6703] MOV [CALLMED],AL
38137 00006787 06 PUSH ES
38138 00006788 1E PUSH DS
38139
38140 ; 04/03/2024
38141 %if 0
38142 ;push word [es:bp+14h]
38143 ;push word [es:bp+15h] ; MSDOS 6.0
38144 PUSH WORD [ES:BP+DPB.DRIVER_ADDR+2]
38145 ;push word [es:bp+12h]
38146 ;push word [es:bp+13h] ; MSDOS 6.0
38147 PUSH WORD [ES:BP+DPB.DRIVER_ADDR]
38148
38149 ;hkn; DEVCALL is in DOSDATA
38150 MOV BX,DEVCALL
38151 POP SI
38152 POP DS ; DS:SI Points to device header
38153 %else
38154 ; 04/03/2024 - Retro DOS v5.0
38155 ; PC DOS 7.1 IBMDOS.COM
38156 00006789 BB[5A03] mov bx,DEVCALL
38157 ;lds si,[es:bp+13h]
38158 0000678C 26C57613 lds si,[es:bp+DPB.DRIVER_ADDR] ; DS:SI Points to device header
38159 %endif
38160
38161 00006790 07 POP ES ; ES:BX Points to call header
38162 ;invoke DEVIOCALL2
38163 00006791 E801EB call DEVIOCALL2
38164 00006794 07 POP ES ; Restore ES:BP
38165 ;Context DS
38166 00006795 16 push ss ;hkn; SS is DOSDATA
38167 00006796 1F pop ds
38168 00006797 8B3E[5D03] MOV DI,[DEVCALL_REQSTAT]
38169
38170 ; 04/03/2024
38171 %if 0
38172 ; MSDOS 3.3
38173 ;test di,8000h
38174 ;jnz short FATERRJ
38175 ; MSDOS 6.0
38176 or di,di
38177 js short FATERRJ ; have error
38178

```

```

38179 ; 04/03/2024
38180 ;;mov al,[es:bp+16h]
38181 ;;mov al,[es:bp+17h] ; MSDOS 6.0
38182 ;MOV AL,[ES:BP+DPB.MEDIA]
38183
38184 LDS SI,[CALLBPB]
38185 ;;mov word [es:bp+1ch],0
38186 ;mov word [es:bp+1dh],0 ; MSDOS 6.0
38187 MOV word [ES:BP+DPB.NEXT_FREE],0 ; recycle scanning pointer
38188
38189 ;invoke $SETDPB
38190 call _$SETDPB
38191
38192 ;hkn; SS override
38193 LDS DI,[SS:CALLXAD] ; Get back buffer pointer
38194
38195 ;mov al,[es:bp+8]
38196 MOV AL,[ES:BP+DPB.FAT_COUNT]
38197
38198 ; MSDOS 3.3
38199 ;;mov ah,[es:bp+0fh]
38200 ;MOV AH,[ES:BP+DPB.FAT_SIZE]
38201 ;;mov [di-8],ax
38202 ;MOV [DI+BUFFINFO.buf_wrtcnt-BUFINSIZ],AX
38203
38204 ; MSDOS 6.0
38205 ;mov [di-0Ah],al
38206 MOV [DI+BUFFINFO.buf_wrtcnt-BUFINSIZ],AL
38207 ;>32mb ;AN000;
38208 ;mov ax,[es:bp+0fh]
38209 MOV AX,[ES:BP+DPB.FAT_SIZE] ;>32mb
38210 ;mov [di-09h],ax ;AC000;
38211 MOV [DI+BUFFINFO.buf_wrtcntinc-BUFINSIZ],AX
38212 ;>32mb Correct buffer info ;AC000;
38213 ;Context DS ;hkn; SS is DOSDATA
38214 push ss
38215 pop ds
38216 XOR AL,AL ;Media changed (Z), Carry clear
38217 retn
38218
38219 FATERRJ:
38220 JMP FATERR
38221
38222 %else
38223 ; 04/03/2024 - Retro DOS v5.0
38224 ; PCDOS 7.1 IBMDOS.COM
38225 ;;;
38226 0000679B 09FF or di,di
38227 0000679D 790C jns short gotgetbuf2
38228 ; error
38229 0000679F C53E[6803] lds di,[CALLXAD] ; Buffer (data) address
38230 ;mov word [di-20],0FFh
38231 000067A3 C745ECFF00 mov word [di+BUFFINFO.buf_ID-BUFINSIZ],0FFh
38232 ; byte BUFFINFO.buf_ID = 0FFh ; FREE
38233 ; byte BUFFINFO.buf_flags = 0
38234 000067A8 E92EFE jmp FATERR
38235
38236 gotgetbuf2:
38237 000067AB C536[6C03] lds si,[CALLBPB] ; Address of the BPB (DEVCALL offset 18)
38238
38239 000067AF 31C9 xor cx,cx ; 0
38240 ;mov [es:bp+1dh],cx ; [ES:BP+DPB.NEXT_FREE] = 0
38241 ; 01/07/2024
38242 000067B1 26894E1D mov [es:bp+DPB.NEXT_FREE],cx ; 0
38243 ; recycle scanning pointer
38244
38245 000067B5 BA5241 mov dx,4152h ; 'RA' ; FAT32 extended BPB/DPB signature
38246
38247 ;cmp [si+0Bh],cx ; BPB.fatsecs ; 16 bit FAT size = 0 for FAT32 fs
38248 000067B8 394C0B cmp [si+A_BPB.SECTORSPERFAT],cx ; 0
38249 000067BB 7514 jnz short gotgetbuf3 ; not FAT32
38250
38251 ;mov [es:bp+39h],cx
38252 000067BD 26894E39 mov [es:bp+DPB.FAT32_NXTFREE],cx ; 0
38253 ;mov [es:bp+3Bh],cx
38254 000067C1 26894E3B mov [es:bp+DPB.FAT32_NXTFREE+2],cx ; 0
38255 000067C5 49 dec cx ; -1
38256 ;mov [es:bp+1fh],cx ; (DPB.FREE_CNT) set free count to -1 (unknown)
38257 000067C6 26894E1F mov [es:bp+DPB.FREE_CNT],cx ; 0FFFFh
38258 ;mov [es:bp+21h],cx
38259 000067CA 26894E21 mov [es:bp+DPB.FREE_CNT_HW],cx ; 0FFFFh
38260 ;
38261 000067CE B95845 mov cx,4558h ; 'XE' ; FAT32 extended BPB/DPB signature
38262 gotgetbuf3:
38263
38264 ;invoke $SETDPB
38265 000067D1 E891AC call _$SETDPB
38266
38267 ;hkn; SS override
38268 000067D4 36C53E[6803] LDS DI,[SS:CALLXAD] ; Get back buffer pointer
38269
38270 ;mov dx,[es:bp+25h]
38271 000067D9 268B5625 mov dx,[es:bp+DPB.FSINFO_SECTOR]
38272 000067DD 31C9 xor cx,cx ; 0
38273 ;cmp [es:bp+0fh],cx
38274 000067DF 26394E0F cmp [es:bp+DPB.FAT_SIZE],cx ; 16 bit FAT size field
38275 000067E3 7403 jz short gotgetbuf4 ; FAT32 fs
38276 000067E5 E99400 jmp gotgetbuf12 ; FAT fs
38277
38278 gotgetbuf4:
38279 000067E8 83FAFF cmp dx,0FFFFh ; invalid ?
38280 000067EB 7503 jnz short gotgetbuf5 ; no
38281 000067ED E98C00 jmp gotgetbuf12 ; skip reading FSINFO sector
38282
38283 gotgetbuf5:
38284 000067F0 36890E[0706] mov [ss:HIGH_SECTOR],cx ; 0
38285 000067F5 89FB mov bx,di
38286
38287 ;cmp byte [ss:BuffInHMA],0
38288 000067F7 36380E[7900] cmp [ss:BuffInHMA],cl ; 0
38289 000067FC 7405 jz short gotgetbuf6 ; buffer is in conventional (<=640KB) memory
38290 000067FE 36C51E[7A00] lds bx,[ss:LoMemBuff] ; use a buffer in conventional memory
38291 ;mov di,bx
38292 gotgetbuf6:
38293 00006803 36C606[4B03]18 mov byte [ss:ALLOWED],Allowed_FAIL+Allowed_RETRY ; 18h
38294 00006809 41 inc cx
38295 ;push di
38296 0000680A 53 push bx
38297 0000680B E8E0D7 call DREAD
38298 0000680E 5B pop bx
38299 ;pop di
38300 0000680F 89DF mov di,bx ; Retro DOS v5.0
38301 00006811 725F jc short gotgetbuf11 ; ds:di = (FSINFO sector) buffer
38302

```

```

38303             ; FSI_HeadSig = 41615252h
38304
38305 00006813 813D5252      cmp     word [di],5252h             ; 'RR' ; check if it is a valid FSINFO sector
38306             ; cmp     word [di+FSINFO.LeadSig],5252
38307 00006817 7559          jnz     short gotgetbuf11         ; not valid
38308             ; cmp     word [di+2],4161h             ; 'aA' ; (NASM syntax)
38309 00006819 817D026141     cmp     word [di+FSINFO.LeadSig+2],4161h
38310 0000681E 7552          jnz     short gotgetbuf11
38311             ; cmp     word [di+484],7272h           ; 'rr' ; FSI_StrucSig = 61417272h
38312 00006820 81BDE4017272  cmp     word [di+FSINFO.StrucSig],7272h
38313 00006826 754A          jnz     short gotgetbuf11
38314             ; cmp     word [di+486],6141h           ; 'Aa'
38315 00006828 81BDE6014161  cmp     word [di+FSINFO.StrucSig+2],6141h
38316 0000682E 7542          jnz     short gotgetbuf11         ; not valid
38317
38318             ; valid
38319 00006830 52             push    dx
38320 00006831 53             push    bx
38321 00006832 51             push    cx
38322
38323             ; mov     bx,[es:bp+2Fh]
38324 00006833 268B5E2F     mov     bx,[es:bp+DPB.LAST_CLUSTER+2]
38325             ; mov     cx,[es:bp+2Dh]
38326 00006837 268B4E2D     mov     cx,[es:bp+DPB.LAST_CLUSTER]
38327
38328             ; mov     ax,[di+488]                     ; FSI_FreeCount ; bx:cx = number of clusters + 1
38329 0000683B 8B85E801     mov     ax,[di+FSINFO.Free_Count]
38330             ; mov     dx,[di+490]                     ; FSI_FreeCount+2
38331 0000683F 8B95EA01     mov     dx,[di+FSINFO.Free_Count+2]
38332
38333 00006843 39DA          cmp     dx,bx                     ; is Free Count >= (Number of clusters + 1) ?
38334 00006845 7502          jne     short gotgetbuf7         ; if yes, it is invalid value (must be 0FFFFFFFh)
38335 00006847 39C8          cmp     ax,cx
38336 gotgetbuf7:
38337 00006849 7308          jnb     short gotgetbuf8         ; yes, invalid value (must be 0FFFFFFFh)
38338
38339             ; mov     [es:bp+1Fh],ax                 ; no,valid free count
38340 0000684B 2689461F     mov     [es:bp+DPB.FREE_CNT],ax ; save free count into [es:bp+DPB.FREE_CNT]
38341             ; mov     [es:bp+21h],dx
38342 0000684F 26895621     mov     [es:bp+DPB.FREE_CNT+2],dx
38343
38344 gotgetbuf8:
38345             ; mov     ax,[di+492]                     ; FSI_Nxt_Free
38346 00006853 8B85EC01     mov     ax,[di+FSINFO.Nxt_Free]
38347             ; mov     dx,[di+494]
38348 00006857 8B95EE01     mov     dx,[di+FSINFO.Nxt_Free+2]
38349
38350 0000685B 39DA          cmp     dx,bx                     ; is the next free clust num >= (num of clusters + 1) ?
38351 0000685D 7502          jne     short gotgetbuf9         ; invalid (if dx > bx)
38352 0000685F 39C8          cmp     ax,cx
38353 gotgetbuf9:
38354 00006861 730C          jnb     short gotgetbuf10        ; invalid
38355
38356             ; mov     [es:bp+39h],ax                 ; save next free (search) cluster number
38357 00006863 26894639     mov     [es:bp+DPB.FAT32_NXTFREE],ax ; into [es:bp+DPB.FAT32_NXTFREE]
38358             ; mov     [es:bp+3Bh],dx
38359 00006867 2689563B     mov     [es:bp+DPB.FAT32_NXTFREE+2],dx
38360             ; mov     [es:bp+1Dh],ax                 ; and into [es:bp+DPB.NEXT_FREE] ; low word
38361 0000686B 2689461D     mov     [es:bp+DPB.NEXT_FREE],ax
38362
38363 gotgetbuf10:
38364 0000686F 59             pop     cx
38365 00006870 5B             pop     bx
38366 00006871 5A             pop     dx
38367
38368 gotgetbuf11:
38369 00006872 36803E[7900]00  cmp     byte [ss:BuffInHMA],0 ; is buffer in HMA ?
38370             ; jnz     short gotgetbuf12             ; yes
38371 00006878 753A          jnz     short gotgetbuf16        ; 04/03/2024
38372
38373             ; mov     word [di-20],0FFh             ; invalidate buffer (set it as free buffer)
38374             ; 04/03/2024 - Retro DOS v5.0
38375 0000687A EB0F          jmp     short gotgetbuf17
38376
38377 gotgetbuf12:
38378 0000687C 36803E[7900]00  cmp     byte [ss:BuffInHMA],0
38379 00006882 7530          jnz     short gotgetbuf16
38380
38381             ; cmp     word [es:bp+6],1
38382 00006884 26837E0601     cmp     word [es:bp+DPB.FIRST_FAT],1 ; 1st FAT start sector
38383 00006889 7405          jz      short gotgetbuf13
38384
38385 gotgetbuf17:
38386             ; 04/03/2024
38387             ; mov     word [di-20],0FFh             ; invalidate buffer (set it as free buffer)
38388             ; di = buffer header + 4 (BUFINFO.buf_ID)
38389 0000688B C745ECFF00     mov     word [di+BUFINFO.buf_ID-BUFINSIZ],0FFh
38390
38391 gotgetbuf13:
38392             ; mov     al,[es:bp+8]
38393 00006890 268A4608     mov     al,[es:bp+DPB.FAT_COUNT]
38394             ; mov     [di-14],al                     ; buffer header address + 10
38395 00006894 8845F2     mov     [di+BUFINFO.buf_wrtcnt-BUFINSIZ],al
38396
38397             ; mov     ax,[es:bp+0Fh]
38398 00006897 268B460F     mov     ax,[es:bp+DPB.FAT_SIZE] ; 16 bit FAT size field
38399 0000689B 09C0          or      ax,ax
38400             ; jnz     short gotgetbuf14 ; FAT (FAT12 or FAT16) fs
38401
38402             ; FAT32 fs
38403             ; mov     ax,[es:bp+31h]                 ; FAT sectors (per one FAT)
38404 0000689F 268B4631     mov     ax,[es:bp+DPB.FAT32_SIZE]
38405             ; mov     [di-13],ax                     ; BUFINFO.buf_wrtcntinc ; # sectors between each write
38406 000068A3 8945F3     mov     [di+BUFINFO.buf_wrtcntinc-BUFINSIZ],ax
38407             ; mov     ax,[es:bp+33h]
38408 000068A6 268B4633     mov     ax,[es:bp+DPB.FAT32_SIZE+2]
38409             ; jmp     short gotgetbuf15
38410
38411 gotgetbuf14:
38412             ; mov     [di-13],ax                     ; BUFINFO.buf_wrtcntinc
38413 000068AC 8945F3     mov     [di+BUFINFO.buf_wrtcntinc-BUFINSIZ],ax
38414
38415             ; xor     ax,ax                         ; 0
38416 000068AF 29C0          sub     ax,ax ; 0
38417
38418 gotgetbuf15:
38419             ; mov     [di-15],ax                     ; BUFINFO.buf_wrtcntinc+2 ; hw of sectors per FAT
38420             ; PCDOS 7.1 BUG !? This would be 'mov [di-11],ax'
38421             ; Erdogan Tan - 03/03/2024
38422 000068B1 8945F5     mov     [di-11],ax
38423             ; mov     [di+BUFINFO.buf_wrtcntinc+2-BUFINSIZ],ax
38424
38425 gotgetbuf16:
38426             ; mov     ax,ss                         ; SS is DOSDATA
38427             ; mov     ds,ax

```



```

38427 000068B4 16      push    ss
38428 000068B5 1F      pop     ds
38429                ;xor     al,al      ; Media changed (Z),Carry clear
38430 000068B6 31C0     xor     ax,ax
38431 000068B8 C3      retn
38432
38433                ;;;
38434 %endif
38435
38436                ;=====
38437 ; STDBUF.ASM
38438                ;=====
38439 ; Retro DOS v2.0 - 12/03/2018
38440
38441 ;
38442 ; Standard buffer management for MSDOS
38443 ;
38444 ;
38445 ;.xlist
38446 ;.xcref
38447 ;INCLUDE STDSW.ASM
38448 ;.cref
38449 ;.list
38450
38451 ;TITLE      STDBUF - MSDOS buffer management
38452 ;NAME       STDBUF
38453
38454 ;INCLUDE BUF.ASM
38455
38456                ;=====
38457 ; BUF.ASM
38458                ;=====
38459 ; 31/07/2018 - Retro DOS v3.0
38460 ; Retro DOS v2.0 - 12/03/2018
38461
38462 ; buffer management for MSDOS
38463
38464 ;CODE      SEGMENT BYTE PUBLIC 'CODE'
38465 ;          ASSUME  SS:DOSGROUP,CS:DOSGROUP
38466
38467 ;SUBTTL SETVISIT,SKIPVISIT -- MANAGE BUFFER SCANS
38468
38469 ;SETVISIT:
38470 ; ; 31/07/2018 - Retro DOS v3.0
38471 ; ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 5CAfh
38472 ; Inputs:
38473 ;     None
38474 ; Function:
38475 ;     Set up a scan of I/O buffers
38476 ; Outputs:
38477 ;     All visit flags = 0
38478 ;     NOTE: This pre-scan is needed because a hard disk error
38479 ;           may cause a scan to stop in the middle leaving some
38480 ;           visit flags set, and some not set.
38481 ;     DS:DI Points to [BUFFHEAD]
38482 ; No other registers altered
38483
38484 ;     LDS     DI,[SS:BUFFHEAD] ; 15/03/2018
38485 ;     PUSH    AX
38486 ;     ;XOR     AX,AX      ;; MSDOS 2.11
38487 ;     ;mov     al,0DFh
38488 ;     mov     al,~buf_visit
38489 ;SETLOOP:
38490 ;     ;MOV     [DI+7],AL ;; MSDOS 2.11
38491 ;     ;and     [DI+5],al
38492 ;     AND     [DI+BUFFINFO.buf_flags],AL
38493 ;     LDS     DI,[DI]
38494 ;     CMP     DI,-1
38495 ;     JNZ     SHORT SETLOOP
38496 ;     POP     AX ; 09/09/2018
38497 ;     LDS     DI,[SS:BUFFHEAD] ; 15/03/2018
38498 ;SVISIT_RETN:
38499 ;     RETN
38500
38501 ;SKIPVISIT:
38502 ; ; 31/07/2018 - Retro DOS v3.0
38503 ; ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 5CC8h
38504 ;
38505 ; Inputs:
38506 ;     DS:DI Points to a buffer
38507 ; Function:
38508 ;     Skip visited buffers
38509 ; Outputs:
38510 ;     DS:DI Points to next unvisited buffer
38511 ;     Zero is set if skip to LAST buffer
38512 ; No other registers altered
38513
38514 ;     CMP     DI,-1
38515 ;     ;retz
38516 ;     JZ     SHORT SVISIT_RETN
38517
38518 ;     ;CMP     BYTE [DI+7],1 ;; MSDOS 2.11
38519 ;     ;;;retnz
38520 ;     ;JNZ     SHORT SVISIT_RETN
38521
38522 ;test     byte [di+5],20h
38523 ;TEST     byte [DI+BUFFINFO.buf_flags],buf_visit
38524 ;JNZ     short SKIPLoop
38525
38526 ;     push    ax
38527 ;     or      al,1
38528 ;     pop     ax
38529 ;     retn
38530
38531 ;SKIPLoop:
38532 ;     LDS     DI,[DI]
38533 ;     JMP     SHORT SKIPVISIT
38534
38535                ;=====
38536 ; BUF.ASM, MSDOS 6.0, 1991
38537                ;=====
38538 ; 31/07/2018 - Retro DOS v3.0
38539 ; 04/05/2019 - Retro DOS v4.0
38540 ; 06/03/2024 - Retro DOS v5.0
38541
38542 ; TITLE     BUF - MSDOS buffer management
38543 ; NAME      BUF
38544
38545 ;** BUF.ASM - Low level routines for buffer cache management
38546 ;
38547 ; GETCURHEAD
38548 ; ScanPlace
38549 ; PLACEBUF
38550 ; PLACEHEAD

```

```

38551 ; PointComp
38552 ; GETBUFFR
38553 ; GETBUFFRB
38554 ; FlushBuf
38555 ; BufWrite
38556 ; SET_RQ_SC_PARMS
38557 ;
38558 ; Revision history:
38559 ;
38560 ; AN000 version 4.00 Jan. 1988
38561 ; A004 PTM 3765 -- Disk reset failed
38562 ; M039 DB 10/17/90 - Disk write optimization
38563 ; I001 5.0 PTR 722211 - Preserve CY when in buffer in HMA
38564 ;
38565 ;Break <GETCURHEAD -- Get current buffer header>
38566 -----
38567 ; Procedure Name : GetCurHead
38568 ; Inputs:
38569 ; No Inputs
38570 ; Function:
38571 ; Returns the pointer to the first buffer in Queue
38572 ; and updates FIRST_BUFF_ADDR
38573 ; and invalidates LASTBUFFER (recency pointer)
38574 ; Outputs:
38575 ; DS:DI = pointer to the first buffer in Queue
38576 ; FIRST_BUFF_ADDR = offset ( DI ) of First buffer in Queue
38577 ; LASTBUFFER = -1
38578 ; No other registers altered
38579 -----
38580 ;
38581 ; 04/05/2019 - Retro DOS v4.0
38582 ; DOSCODE:98D2h (MSDOS 6.21, MSDOS.SYS)
38583 ; 27/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
38584 ; DOSCODE:9876h (MSDOS 5.0, MSDOS.SYS)
38585 ;
38586 ; 06/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
38587 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0AA4Bh
38588 ;
38589 GETCURHEAD:
38590 000068B9 36C53E[6D00] lds di,[ss:BufferQueue] ; Pointer to the first buffer
38591 000068BE 36C706[1E00]FFFF mov word [ss:LastBuffer],-1 ; invalidate last buffer
38592 000068C5 36893E[7E11] mov [ss:FIRST_BUFF_ADDR],di ; save first buffer address
38593 000068CA C3 retn
38594 ;
38595 ;Break <SCANPLACE, PLACEBUF -- PUT A BUFFER BACK IN THE POOL>
38596 -----
38597 ; Procedure Name : ScanPlace
38598 ; Inputs:
38599 ; Same as PLACEBUF
38600 ; Function:
38601 ; Save scan location and call PLACEBUF
38602 ; Outputs:
38603 ; DS:DI Points to saved scan location
38604 ; All registers, except DS:DI, preserved.
38605 -----
38606 ;M039: Rewritten to preserve registers.
38607 ;
38608 ;SCANPLACE:
38609 ; ; 31/07/2018 - Retro DOS v3.0
38610 ; ; IBMDOS.COM (MSDOS 3.3, 1987) - Offset 5DDCh
38611 ; push es
38612 ; les si,[di]
38613 ; ;les si,[DI+BUFFINFO.buf_link]
38614 ; call PLACEBUF
38615 ; push es
38616 ; pop ds
38617 ; mov di,si
38618 ; pop es
38619 ;scanplace_retn:
38620 ; retn
38621 ;
38622 ; MSDOS 6.0
38623 SCANPLACE:
38624 000068CB FF35 push word [di]
38625 ;push word [di+BUFFINFO.buf_next] ;Save scan location
38626 000068CD E80200 call PLACEBUF
38627 000068D0 5F pop di
38628 000068D1 C3 retn
38629 ;
38630 ;-----
38631 ; Procedure Name : PlaceBuf
38632 ; Input:
38633 ; DS:DI points to buffer (DS->BUFFINFO array, DI=offset in array)
38634 ; Function:
38635 ; Remove buffer from queue and re-insert it in proper place.
38636 ; NO registers altered
38637 -----
38638 ;
38639 ;procedure PLACEBUF,NEAR
38640 ;
38641 ; 27/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
38642 ; 20/05/2019 - Retro DOS v4.0
38643 PLACEBUF:
38644 ; 31/07/2018 - Retro DOS v3.0
38645 ;
38646 ; MSDOS 6.0
38647 000068D2 50 push AX ;Save only regs we modify ;AN000;
38648 000068D3 53 push BX ;AN000;
38649 ; 23/09/2023
38650 ;push SI ;AN000;
38651 ;
38652 000068D4 8B05 mov ax,[di]
38653 ;mov ax,[di+BUFFINFO.buf_next]
38654 000068D6 368B1E[6D00] mov bx,[ss:BufferQueue] ; bx = offset of head of list;smr;SS override
38655 ;
38656 000068DB 39D8 cmp ax,bx ;Buf = last? ;AN000;
38657 000068DD 7422 je short nret ;Yes, special case ;AN000;
38658 000068DF 39DF cmp di,bx ;Buf = first? ;AN000;
38659 000068E1 7506 jne short not_first ;Yes, special case ;AN000;
38660 000068E3 36A3[6D00] mov [ss:BufferQueue],ax ;smr;SS Override
38661 000068E7 EB18 jmp short nret ;Continue with repositioning;AN000;
38662 not_first:
38663 ; 23/09/2023
38664 000068E9 56 push si
38665 ;mov si,[di+2]
38666 000068EA 8B7502 mov SI,[DI+BUFFINFO.buf_prev] ;No, SI = prior Buf ;AN000;
38667 000068ED 8904 mov [si],ax
38668 ;mov [SI+BUFFINFO.buf_next],AX ; ax has di->buf_next ;AN000;
38669 000068EF 96 xchg si,ax
38670 ;mov [si+2],ax
38671 000068F0 894402 mov [SI+BUFFINFO.buf_prev],AX ; ;AN000;
38672 ;
38673 000068F3 8B7702 mov SI,[BX+BUFFINFO.buf_prev] ;SI-> last buffer ;AN000;
38674 000068F6 893C mov [si],di

```



```

38675 ;mov [SI+BUFFINFO.buf_next],DI ;Add Buf to end of list ;AN000;
38676 000068F8 897F02 mov [BX+BUFFINFO.buf_prev],DI ;AN000;
38677 000068FB 897502 mov [DI+BUFFINFO.buf_prev],SI ;Update link in Buf too ;AN000;
38678 000068FE 891D mov [di],bx
38679 ;mov [DI+BUFFINFO.buf_next],BX ;AN000;
38680 ; 23/09/2023
38681 00006900 5E pop si
38682 nret:
38683 ; 23/09/2023 ;AN000;
38684 ;pop SI ;AN000;
38685 00006901 5B pop BX ;AN000;
38686 00006902 58 pop AX ;AN000;
38687 ;AN000;
38688 ;cmp byte [di+4],0FFh
38689 00006903 807D04FF cmp byte [di+BUFFINFO.buf_ID],-1 ; Buffer FREE? ;AN000;
38690 00006907 7505 jne short pbx ; M039: -no, jump.
38691 00006909 36893E[6D00] mov [ss:BufferQueue],di ; M039: -yes, make it LRU.
38692 pbx:
38693 0000690E C3 retn ;AN000;
38694
38695 ; 31/07/2018 - Retro DOS v3.0
38696
38697 ; MSDOS 3.3
38698 ; IBMDOS.COM (MSDOS 3.3, 1987) - Offset 5DDCh
38699
38700 ;PLACEBUF:
38701 ; 15/03/2018 - Retro DOS v2.0 (MSDOS 2.11)
38702 ;
38703 ; CALL save_world
38704 ; LES CX,[DI]
38705 ; CMP CX,-1 ; Buf is LAST?
38706 ; JZ SHORT NRET ; Buffer already last
38707 ; MOV BP,ES ; Pointsave = Buf.nextbuf
38708 ; PUSH DS
38709 ; POP ES ; Buf is ES:DI
38710 ; 15/03/2018
38711 ; LDS SI,[SS:BUFFHEAD] ; Curbuf = HEAD
38712 ; CALL POINTCOMP ; Buf == HEAD?
38713 ; JNZ SHORT BUFLOOP
38714 ; MOV [SS:BUFFHEAD],CX
38715 ; MOV [SS:BUFFHEAD+2],BP ; HEAD = Pointsave
38716 ; JMP SHORT LOOKEND
38717 ;BUFLOOP:
38718 ; 31/07/2018
38719 ; mov ax,ds
38720 ; mov bx,si
38721 ; lds si,[SI+BUFFINFO.buf_link]
38722 ; LDS SI,[SI]
38723 ; CALL POINTCOMP
38724 ; jnz short BUFLOOP
38725 ;
38726 ; mov ds,ax
38727 ; mov si,bx
38728 ; mov [SI],cx
38729 ; mov [SI+BUFFINFO.buf_link],cx ; If Curbuf.nextbuf == buf
38730 ; mov [SI+2],bp
38731 ; mov [BX+BUFFINFO.buf_link+2],bp ; Curbuf.nextbuf = Pointsave
38732 ;LOOKEND:
38733 ; mov ax,ds
38734 ; mov bx,si
38735 ; LDS SI,[SI]
38736 ; CMP SI,-1
38737 ; jnz short LOOKEND
38738 ;GOTHEEND:
38739 ; mov ds,ax
38740 ; mov [BX],di
38741 ; MOV [BX+2],ES ; Curbuf.nextbuf = Buf
38742 ; MOV WORD [ES:DI],-1
38743 ; mov word [ES:DI+BUFFINFO.buf_link],-1
38744 ; MOV WORD [ES:DI+2],-1 ; Buf is LAST
38745 ; mov word [ES:DI+BUFFINFO.buf_link+2],-1
38746 ;NRET:
38747 ; CALL restore_world
38748 ;
38749 ; cmp byte [di+4],-1
38750 ; cmp byte [DI+BUFFINFO.buf_ID],-1 ; Free buffer ?
38751 ; jnz short scanplace_retn
38752 ; call PLACEHEAD
38753 ; retn
38754
38755 ;EndProc PLACEBUF
38756
38757 ;M039 - Removed PLACEHEAD.
38758 -----
38759 ; places buffer at head
38760 ; NOTE::::: ASSUMES THAT BUFFER IS CURRENTLY THE LAST
38761 ; ONE IN THE LIST!!!!!!
38762 ; BUGBUG ---- this routine can be removed because it has only
38763 ; BUGBUG ---- one instruction. This routine is called from
38764 ; BUGBUG ---- 3 places. ( Size = 3*3+6 = 15 bytes )
38765 ; BUGBUG ---- if coded in line = 3 * 5 = 15 bytes
38766 ; BUGBUG ---- But kept as it is for modularity
38767 -----
38768 ;procedure PLACEHEAD,NEAR
38769 ; mov word ptr [BufferQueue], di
38770 ; ret
38771 ;EndProc PLACEHEAD
38772 ;M039
38773
38774 -----
38775 ; Procedure Name : PLACEHEAD
38776 ;
38777 ; SAME AS PLACEBUF except places buffer at head
38778 -----
38779
38780 ; MSDOS 3.3 (Retro DOS v3.0)
38781 ; 05/09/2018
38782 ; MSDOS 2.11 (Retro DOS v2.0)
38783 ;PLACEHEAD:
38784 ; 31/07/2018 - Retro DOS v3.0
38785 ; IBMDOS.COM (MSDOS 3.3, 1987) - Offset 5D4Ah
38786 ;
38787 ; CALL save_world
38788 ; PUSH DS
38789 ; POP ES
38790 ; 15/03/2018 - Retro DOS v2.0 (MSDOS 2.11)
38791 ; LDS SI,[SS:BUFFHEAD]
38792 ; 31/07/2018 - Retro DOS v3.0 (MSDOS 3.3)
38793 ; CALL POINTCOMP
38794 ; JZ SHORT GOTHEEND2
38795 ; MOV [ES:DI],SI
38796 ; mov [ES:DI+BUFFINFO.buf_link],si
38797 ; MOV [ES:DI+2],DS
38798 ; mov [ES:DI+BUFFINFO.buf_link+2],ds

```

```

38799 ; MOV [SS:BUFFHEAD],DI
38800 ; MOV [SS:BUFFHEAD+2],ES
38801 ; LOOKEND2:
38802 ; mov ax,ds
38803 ; mov bx,si
38804 ; lds si,[SI+BUFFINFO.buf_link]
38805 ; LDS SI,[SI]
38806 ; CALL POINTCOMP
38807 ; JNZ SHORT LOOKEND2 ; 05/09/2018
38808 ; mov ds,ax
38809 ; mov word [bx],-1
38810 ; mov word [BX+BUFFINFO.buf_link],-1
38811 ; mov word [bx+2],-1
38812 ; mov word [BX+BUFFINFO.buf_link+2],-1
38813 ; GOTHEEND2:
38814 ; call restore_world
38815 ; placehead_retn:
38816 ; retn
38817
38818 ; 20/05/2019 - Retro DOS v4.0
38819 ; DOSCODE:9928h (MSDOS 6.21, MSDOS.SYS)
38820
38821 ; Break <POINTCOMP -- 20 BIT POINTER COMPARE>
38822 ; -----
38823 ;
38824 ; Procedure Name : PointComp
38825 ; Inputs:
38826 ; DS:SI & ES:DI
38827 ; Function:
38828 ; Checks for ((SI==DI) && (ES==DS))
38829 ; Assumes that pointers are normalized for the
38830 ; same segment
38831 ;
38832 ; Compare DS:SI to ES:DI (or DS:DI to ES:SI) for equality
38833 ; DO NOT USE FOR < or >
38834 ; No Registers altered
38835 ;
38836 ; -----
38837
38838 POINTCOMP:
38839 ; 31/07/2018 - Retro DOS v3.0
38840 ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 5D84h
38841 0000690F 39FE CMP SI,DI
38842 00006911 750A jnz short _ret_label ; return if nz
38843 ; jnz short placehead_retn
38844 00006913 51 PUSH CX
38845 00006914 52 PUSH DX
38846 00006915 8CD9 MOV CX,DS
38847 00006917 8CC2 MOV DX,ES
38848 00006919 39D1 CMP CX,DX
38849 0000691B 5A POP DX
38850 0000691C 59 POP CX
38851 _ret_label:
38852 0000691D C3 retn
38853
38854 ; 01/08/2018 - Retro DOS v3.0
38855 ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 5D93h
38856
38857 ; Break <GETBUFFR, GETBUFFRB -- GET A SECTOR INTO A BUFFER>
38858
38859 ; ** GetBuffr - Get a non-FAT Sector into a Buffer
38860 ; -----
38861 ; GetBuffr does normal ( non-FAT ) sector buffering
38862 ; It gets the specified local sector into one of the I/O buffers
38863 ; and shuffles the queue
38864 ;
38865 ; ENTRY (AL) = 0 means sector must be pre-read
38866 ; ELSE no pre-read
38867 ; (DX) = Desired physical sector number (LOW)
38868 ; HIGH_SECTOR = Desired physical sector number (HIGH)
38869 ; (ES:BP) = Pointer to drive parameters
38870 ; ALLOWED set in case of INT 24
38871 ; EXIT 'C' set if error (user FAIL response to INT24)
38872 ; 'C' clear if OK
38873 ; CURBUF Points to the Buffer for the sector
38874 ; the buffer type bits OF buf_flags = 0, caller must set it
38875 ; USES AX, BX, CX, SI, DI, Flags
38876 ; -----
38877
38878 ; ** GetBuffrb - Get a FAT Sector into a Buffer
38879 ; -----
38880 ; GetBuffrb reads a sector from the FAT file system's FAT table.
38881 ; It gets the specified sector into one of the I/O buffers
38882 ; and shuffles the queue. We need a special entry point so that
38883 ; we can read the alternate FAT sector if the first read fails, also
38884 ; so we can mark the buffer as a FAT sector.
38885 ;
38886 ; ENTRY (AL) = 0 means sector must be pre-read
38887 ; ELSE no pre-read
38888 ; (DX) = Desired physical sector number (LOW)
38889 ; (SI) != 0
38890 ; HIGH_SECTOR = Desired physical sector number (HIGH)
38891 ; (ES:BP) = Pointer to drive parameters
38892 ; ALLOWED set in case of INT 24
38893 ; EXIT 'C' set if error (user FAIL response to INT24)
38894 ; 'C' clear if OK
38895 ; CUR ddBUF Points to the Buffer for the sector
38896 ; the buffer type bits OF buf_flags = 0, caller must set it
38897 ; USES AX, BX, CX, SI, DI, Flags
38898 ; -----
38899
38900 ; 22/09/2023 - RetroDOS v4.2 MSDOS.SYS (optimization)
38901 GETBUFFRC:
38902 ; mov word [HIGH_SECTOR],0
38903 ; 02/03/2024 - Retro DOS v5.0
38904 0000691E 891E[0706] mov [HIGH_SECTOR],bx
38905 GETBUFFRA:
38906 00006922 30C0 xor al,al
38907 00006924 BE0100 mov si,1
38908 00006927 EB09 jmp short GETBUFFRB
38909
38910 ; 22/09/2023
38911 GETBUFFER:
38912 00006929 30C0 xor al,al
38913 GETBUFFRD:
38914 ; mov byte [ALLOWED],18h
38915 0000692B C606[4B03]18 mov byte [ALLOWED],Allowed_FAIL+Allowed_RETRY
38916
38917 ; 20/05/2019 - Retro DOS v4.0
38918 ; DOSCODE:9937h (MSDOS 6.21, MSDOS.SYS)
38919 ; 27/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
38920 ; DOSCODE:98DBh (MSDOS 5.0, MSDOS.SYS)
38921
38922 ; 07/07/2024

```

```

38923          ; 06/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
38924          ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0AAB0h
38925
38926          ; (windows Me IO.SYS - BIOSCODE:0CA3Dh)
38927          ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:9937h)
38928
38929 GETBUFFR:
38930 00006930 31F6      XOR     SI,SI
38931
38932          ; This entry point is called for FAT buffering with SI != 0
38933
38934 GETBUFFRB:
38935 00006932 A3[9405]   MOV     [PREREAD],AX          ; save pre-read flag
38936
38937          ; 06/03/2024 (PCDOS 7.1 IBMDOS.COM)
38938          ;;;
38939 00006935 09F6      OR      SI,SI
38940 00006937 7429      JZ      short getb1
38941
38942          ;cmp word [es:bp+0Fh],0
38943 00006939 26837E0F00 CMP     word [es:bp+DPB.FAT_SIZE],0
38944 0000693E 7522      JNZ     short getb1 ; not FAT32
38945
38946          ;mov ax,[es:bp+23h]
38947 00006940 268B4623 MOV     ax,[es:bp+DPB.EXT_FLAGS]
38948 00006944 A880      TEST    al,80h ; bit 7 -- 1 means only one FAT is active
38949 00006946 741A      JZ      short getb1
38950 00006948 83E00F AND     ax,0Fh ; Active FAT is the one referenced in bits 0-3
38951 0000694B 7415      JZ      short getb1
38952
38953 0000694D 52        PUSH    DX
38954 0000694E 89C1      MOV     CX,AX          ; Zero based number of active FAT.
38955          ; (Only valid if mirroring is disabled.)
38956          ;mul word [es:bp+33h]
38957 00006950 26F76633 MUL     word [es:bp+DPB.FAT32_SIZE+2]
38958 00006954 91        XCHG    AX,CX
38959          ;mul word [es:bp+31h]
38960 00006955 26F76631 MUL     word [es:bp+DPB.FAT32_SIZE]
38961 00006959 01D1      ADD     CX,DX
38962 0000695B 5A        POP     DX
38963 0000695C 01C2      ADD     DX,AX
38964 0000695E 110E[0706] ADC     [HIGH_SECTOR],CX
38965 getb1:
38966          ;;;
38967
38968          ; 15/12/2022
38969 00006962 268A4600 MOV     AL,[ES:BP]
38970          ; 27/11/2022 MSDOS 5.0 MSDOS.SYS compatibility)
38971          ;MOV AL,[ES:BP+DPB.DRIVE] ; mov al,[es:bp+0]
38972 00006966 C53E[1E00] LDS     DI,[LastBuffer] ; Get the recency pointer
38973
38974          ; 06/03/2024 (PCDOS 7.1 IBMDOS.COM)
38975          ;;;
38976 0000696A BBFFFF      MOV     BX,-1 ; 0FFFFh
38977          ;;;
38978
38979          ; MSDOS 6.0
38980 ;hkn; SS override
38981 0000696D 368B0E[0706] MOV     CX,[SS:HIGH_SECTOR] ; F.C. >32mb ;AN000;
38982
38983          ; See if this is the buffer that was most recently returned.
38984          ; A big performance win if it is.
38985
38986          ;CMP DI,-1 ; Recency pointer valid?
38987          ; 06/03/2024 - Retro DOS v5.0
38988 00006972 39DF      CMP     DI,BX ; -1
38989 00006974 7412      JE      short getb5 ; No
38990
38991          ;cmp dx,[di+6]
38992 00006976 3B5506 CMP     DX,[DI+BUFFINFO.buf_sector]
38993 00006979 750D      JNZ     short getb5 ; wrong sector
38994
38995          ; MSDOS 6.0
38996          ;cmp cx,[di+8]
38997 0000697B 3B4D08 CMP     CX,[DI+BUFFINFO.buf_sector+2] ; F.C. >32mb ;AN000;
38998 0000697E 7508      JNZ     short getb5 ; F.C. >32mb ;AN000;
38999
39000          ;cmp al,[di+4]
39001 00006980 3A4504 CMP     AL,[DI+BUFFINFO.buf_ID]
39002          ;JZ getb35 ; Just asked for same buffer
39003 00006983 7503      JNZ     short getb5
39004          ;jmp getb35
39005          ; 17/12/2022
39006          ; 28/07/2019
39007 00006985 E90301 JMP     getb35x
39008          ; 07/12/2022 MSDOS 5.0 MSDOS.SYS compatibility)
39009          ;jmp getb35
39010
39011          ; It's not the buffer most recently returned. See if it's in the
39012          ; cache.
39013          ;
39014          ; (cx:dx) = sector #
39015          ; (al) = drive #
39016          ; (si) = 0 iff non fat sector, != 0 if FAT sector read
39017          ; ??? list may be incomplete ???
39018
39019 getb5:
39020          ; MSDOS 3.3
39021          ;lds di,[SS:BUFFHEAD]
39022          ; MSDOS 6.0
39023 00006988 E82EFF      CALL    GETCURHEAD ; get Q Head
39024
39025 getb10:
39026 0000698B 3B5506 CMP     DX,[DI+BUFFINFO.buf_sector]
39027          CMP     short getb12 ; wrong sector lo
39028          ; 06/03/2024 (PCDOS 7.1 IBMDOS.COM)
39029          ;;;
39030 0000698E 750D      JNE     short getb11
39031          ;;;
39032
39033          ; MSDOS 6.0
39034          ;cmp cx,[di+8]
39035 00006990 3B4D08 CMP     CX,[DI+BUFFINFO.buf_sector+2] ; wrong sector hi
39036          ;jne short getb12
39037          ; 06/03/2024
39038          ;;;
39039 00006993 7508      JNE     short getb11
39040          ;;;
39041
39042          ;cmp al,[di+4]
39043 00006995 3A4504 CMP     AL,[DI+BUFFINFO.buf_ID]
39044          ;je short getb25 ; 05/09/2018 ; Found the requested sector
39045          ;jne short getb12
39046          ; 06/03/2024 (PCDOS 7.1 IBMDOS.COM)

```

```

39047             ;;;
39048 00006998 7503     jne     short getb11
39049             ;;;
39050 0000699A E9A400   jmp     getb25
39051
39052 getb11:
39053     ; 06/03/2024 (PCDOS 7.1 IBMDOS.COM)
39054     ;;;
39055     ;cmp     byte [di+4],0FFh
39056 0000699D 807D04FF  cmp     byte [di+BUFFINFO.buf_ID],0FFh    ; Free buffer ?
39057 000069A1 7502     jne     short getb12    ; no
39058 000069A3 89FB     mov     bx,di    ; save buffer (offset) addr
39059     ;;;
39060
39061 getb12:
39062     ; MSDOS 3.3
39063     ;mov     di,[DI]
39064     ;mov     di,[DI+BUFFINFO.buf_link]
39065     ;
39066     ; 15/08/2018
39067     ;lds     di,[di]
39068
39069     ;cmp     di,-1 ; 0FFFFh
39070     ;jne     short getb10
39071     ;lds     di,[SS:BUFFHEAD]
39072
39073     ; MSDOS 6.0
39074 000069A5 8B3D     mov     di,[di]
39075     ;mov     DI,[DI+BUFFINFO.BUF_NEXT]
39076 000069A7 363B3E[7E11]  cmp     DI,[SS:FIRST_BUFF_ADDR]    ; back at the front again?
39077 000069AC 75DD     jne     short getb10    ; no, continue looking
39078
39079     ; 06/03/2024 (PCDOS 7.1 IBMDOS.COM)
39080     ;;;
39081 000069AE 83FBFF    cmp     bx,0FFFFh    ; -1 ; invalid (not a free buffer address)
39082 000069B1 7404     je      short getb12x
39083 000069B3 89DF     mov     di,bx    ; restore free buffer (header offset) address
39084 000069B5 EB16     jmp     short getb13
39085
39086 getb12x:
39087     ;;;
39088     ; The requested sector is not available in the buffers. DS:DI now points
39089     ; to the first buffer in the Queue. Flush the first buffer & read in the
39090     ; new sector into it.
39091     ;
39092     ; BUGBUG - what goes on here? Isn't the first guy the most recently
39093     ; used guy? Shuld be for fast lookup. If he is, we shouldn't take
39094     ; him, we should take LRU. And the above lookup shouldn't be
39095     ; down a chain, but should be hashed.
39096     ;
39097     ; (DS:DI) = first buffer in the queue
39098     ; (CX:DX) = sector # we want
39099     ; (SI) = 0 iff non fat sector, != 0 if FAT sector read
39100
39101     ; MSDOS 3.3 & MSDOS 6.0
39102 ;hkn; SS override
39103     PUSH     CX    ; MSDOS 6.0
39104     push     si
39105     push     dx
39106     push     bp
39107     push     es
39108     CALL     BUFWRITE    ; write out the dirty buffer
39109     pop      es
39110     pop      bp
39111     pop      dx
39112     pop      si
39113     POP      word [SS:HIGH_SECTOR]    ; MSDOS 6.0
39114     ;jc      short getbx    ; if got hard error
39115 000069C8 7303     jnc     short getb13
39116 000069CA E9C800   jmp     getbx
39117
39118 getb13:
39119
39120     ; 06/03/2024 (PCDOS 7.1 IBMDOS.COM)
39121 %if 0
39122     ; MSDOS 6.0
39123     CALL     SET_RQ_SC_PARMS    ; set parms for secondary cache
39124 %endif
39125
39126     ; We're ready to read in the buffer, if need be. If the caller
39127     ; wanted to just *write* the buffer then we'll skip reading it in.
39128
39129 000069CD 30E4     XOR     AH,AH    ; initial flags
39130
39131 ;hkn; SS override
39132     ;test    byte [ss:PREREAD],0FFh
39133     ;jnz     short getb20
39134 000069CF 363826[9405]  cmp     [SS:PREREAD],ah ; 0    ; am to Read in the new sector?
39135 000069D4 7553     JNZ     short getb20    ; no, we're done
39136     ;;;lea     bx,[di+16] ; MSDOS 3.3
39137     ;lea     bx,[di+20] ; MSDOS 6.0
39138 000069D6 8D5D18    lea     bx,[di+24] ; PCDS 7.1; 06/03/2024
39139     LEA     BX,[DI+BUFINSIZ]    ; (ds:bx) = data address
39140     ;MOV     CX,1
39141     ; 22/09/2023
39142     sub     cx,cx ; 0
39143     push     si
39144     push     di
39145     push     dx
39146 000069DE 06     ; MSDOS 6.0
39147     push     es ; ***
39148
39149     ; Note: As far as I can tell, all disk reads into buffers go through
39150     ; this point. -mrw 10/88
39151
39152     ;cmp     byte [ss:BuffInHMA],0 ; is buffers in HMA?
39153     ; 22/09/2023
39154 000069DF 36380E[7900]  cmp     [ss:BuffInHMA],cl ; 0
39155 000069E4 7407     jz      short getb14
39156 000069E6 1E     push     ds ; **
39157 000069E7 53     push     bx ; *
39158 000069E8 36C51E[7A00]  lds     bx,[ss:LOMemBuff]    ; Then let's read it into scratch buff
39159
39160 getb14:
39161 ;M039: Eliminated redundant HMA code.
39162
39163     ; 22/09/2023
39164 000069ED 41     inc     cx ; cx = 1
39165
39166     ; MSDOS 3.3 (& MSDOS 6.0)
39167 000069EE 09F6     OR      SI,SI    ; FAT sector ?
39168 000069F0 7407     JZ      short getb15    ; no
39169
39170     call     FATSECRD
39171     ;mov     ah,2
39172     MOV     AH,buf_isFAT    ; Set buf_flags

```

```

39171
39172 000069F7 EB05          JMP     SHORT getb17          ; Buffer is marked free if read barfs
39173
39174 getb15:
39175 000069F9 E8F2D5        call    DREAD          ; Buffer is marked free if read barfs
39176 000069FC B400          MOV     AH,0            ; Set buf_flags to no type, DO NOT XOR!
39177 getb17:
39178
39179 ; 06/03/2024 - Retro DOS v5.0
39180 %if 0
39181 ; 17/12/2022
39182 ; 27/11/2022 MSDOS 5.0 MSDOS.SYS compatibility)
39183 ;;if 0
39184 ; MSDOS 6.0                                ;I001
39185 pushf                                        ;I001
39186 cmp     byte [SS:BuffInHMA],0 ; did we read into scratch buff ? ;I001
39187 jz      short not_in_hma          ; no ;I001
39188 ;mov     cx,[es:bp+2]
39189 mov     cx,[ES:BP+DPB.SECTOR_SIZE]
39190 shr     cx,1
39191 popf                                         ; Retrieve possible CY from DREAD ;I001
39192 mov     si,bx
39193 pop     di ; *
39194 pop     es ; **
39195 cld
39196 pushf                                        ; Preserve possible CY from DREAD ;I001
39197 rep     movsw                        ; move the contents of scratch buf;I001
39198 push    es
39199 pop     ds
39200 ;;endif
39201
39202 ;; 17/12/2022
39203 %if 0
39204 ; 27/11/2022 MSDOS 5.0 MSDOS.SYS compatibility)
39205 ; MSDOS 5.0
39206 pushf
39207 cmp     byte [SS:BuffInHMA],0 ; did we read into scratch buff ?
39208 jz      short not_in_hma          ; no
39209 popf
39210 mov     cx,[ES:BP+DPB.SECTOR_SIZE]
39211 shr     cx,1
39212 mov     si,bx
39213 pop     di ; *
39214 pop     es ; **
39215 cld
39216 rep     movsw
39217 push    es
39218 pop     ds
39219 jmp     short getb19 ; 27/11/2022
39220 %endif
39221
39222 not_in_hma:                                ;I001
39223 popf                                        ;I001
39224
39225 %else
39226 ; 06/03/2024
39227 ; PC DOS 7.1 IBMDOS.COM
39228 ;;
39229 mov     ch,0
39230 mov     cl,[ss:BuffInHMA] ; did we read into scratch buff ?
39231 jcxz    getb19 ; no
39232 ;mov     cx,[es:bp+2]
39233 mov     cx,[es:bp+DPB.SECTOR_SIZE]
39234 mov     si,bx
39235 pop     di
39236 pop     es
39237 pushf
39238 shr     cx,1
39239 cld
39240 cmp     byte [ss:DDMOVE],0
39241 ;jz      short getb18+1 ; rep movsw (skip 32bit prefix)
39242 ;jz      short getb18
39243 shr     cx,1
39244 ;getb18:
39245 ;rep     movsd ; move the contents of scratch buffer
39246 db      66h ; 32bit prefix
39247 getb18:
39248 rep     movsw
39249
39250 ;mov     dx,es
39251 ;mov     ds,dx
39252 push    es
39253 pop     ds
39254 popf                                         ; Retrieve possible CY from DREAD
39255 ;;
39256 %endif
39257
39258 getb19:
39259 pop     es ; ***
39260 pop     dx
39261 pop     di
39262 pop     si
39263 JC      short getbx
39264
39265 ; The buffer has the data setup in it (if we were to read)
39266 ; Setup the various buffer fields
39267 ;
39268 ; (ds:di) = buffer address
39269 ; (es:bp) = DPB address
39270 ; (HIGH_SECTOR:DX) = sector #
39271 ; (ah) = BUF_FLAGS value
39272 ; (si) = 0 if non fat sector, != 0 if FAT sector read
39273
39274 ;hkn; SS override
39275 getb20:
39276 00006A29 368B0E[0706]    MOV     CX,[SS:HIGH_SECTOR]
39277 ;mov     [di+8],cx
39278 00006A2E 894D08          MOV     [DI+BUFFINFO.buf_sector+2],CX
39279 ; MSDOS 3.3 (& MSDOS 6.0)
39280 ;mov     [di+6],dx
39281 00006A31 895506          MOV     [DI+BUFFINFO.buf_sector],DX
39282 ;;;mov    [di+0Ah],bp ; MSDOS 3.3
39283 ;;;mov    [di+0Dh],bp ; MSDOS 6.0
39284 ;mov     [di+0Fh],bp ; PC DOS 7.1 ; 06/03/2024
39285 00006A34 896D0F          MOV     [DI+BUFFINFO.buf_DPB],BP
39286 ;;;mov    [di+0Ch],es
39287 ;mov     [di+0Fh],es ; MSDOS 6.0
39288 ;mov     [di+11h],es ; PC DOS 7.1 ; 06/03/2024
39289 00006A37 8C4511          MOV     [DI+BUFFINFO.buf_DPB+2],ES
39290 ; 15/12/2022
39291 00006A3A 268A4600        mov     al,[es:bp]
39292 ;mov     al,[es:bp+0]
39293 ; 27/11/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
39294 ;MOV     AL,[ES:BP+DPB.DRIVE]

```

```

39295             ;mov     [di+4],ax
39296 00006A3E 894504      MOV     [DI+BUFFINFO.buf_ID],AX             ; Set ID and Flags
39297 getb25:
39298             ; MSDOS 3.3
39299             ;mov     ax,1
39300
39301             ; MSDOS 6.0
39302             ;mov     byte [di+0Ah],1
39303 00006A41 C6450A01     MOV     byte [DI+BUFFINFO.buf_wrtcnt],1             ; Default to not a FAT sector ;AC000;
39304 00006A45 31C0        XOR     AX,AX
39305
39306             ; 07/07/2024 (PCDOS 7.1)
39307             ;mov     [di+0Dh],ax ; 0
39308 00006A47 89450D     mov     [di+BUFFINFO.buf_wrtcntinc+2],ax ; 0
39309
39310             ; MSDOS 3.3 (& MSDOS 6.0)
39311 00006A4A 09F6        OR     SI,SI             ; FAT sector ?
39312 00006A4C 742C        JZ     short getb30             ; no
39313
39314             ; 06/03/2024 (PCDOS 7.1 IBMDOS.COM)
39315             ;;;
39316             ;;cmp     word [es:bp+0Fh],0
39317             ;;cmp     word [es:bp+DPB.FAT_SIZE],0
39318 00006A4E 2639460F     cmp     [es:bp+DPB.FAT_SIZE],ax ; 0
39319 00006A52 751B        jnz     short getb27             ; not FAT32
39320
39321             ; FAT32
39322             ;;test    word [es:bp+23h],80h
39323             ;;test    byte [es:bp+23h],80h
39324 00006A54 26F6462380   test    byte [es:bp+DPB.EXT_FLAGS],80h
39325 00006A59 7507        jnz     short getb26             ; bit 7 -- 1 means only one FAT is active
39326             ;mov     al,[es:bp+8]
39327 00006A5B 268A4608     mov     al,[es:bp+DPB.FAT_COUNT]
39328             ;mov     [di+0Ah],al
39329 00006A5F 88450A     mov     [di+BUFFINFO.buf_wrtcnt],al
39330 getb26:
39331             ;mov     ax,[es:bp+33h]
39332 00006A62 268B4633     mov     ax,[es:bp+DPB.FAT32_SIZE+2]
39333             ;mov     [di+0Dh],ax
39334 00006A66 89450D     mov     [di+BUFFINFO.buf_wrtcntinc+2],ax
39335             ;mov     ax,[es:bp+31h]
39336 00006A69 268B4631     mov     ax,[es:bp+DPB.FAT32_SIZE]
39337 00006A6D EB0B        jmp     short getb30
39338 getb27:
39339             ;;;
39340
39341             ;mov     al,[es:bp+8]
39342 00006A6F 268A4608     MOV     AL,[ES:BP+DPB.FAT_COUNT]             ; update number of copies of
39343
39344             ; MSDOS 6.0
39345 00006A73 88450A     MOV     [DI+BUFFINFO.buf_wrtcnt],AL             ; this sector present on disk
39346             ;mov     ax,[es:bp+0Fh]
39347 00006A76 268B460F     MOV     AX,[ES:BP+DPB.FAT_SIZE]             ; offset of identical FAT
39348             ; sectors
39349
39350             ; MSDOS 3.3
39351             ;;mov     ah,[es:bp+0Fh]
39352             ;;MOV     AH,[ES:BP+DPB.FAT_SIZE]
39353
39354             ; BUGBUG - dos 6 can clean this up by not setting wrtcntinc unless wrtcnt
39355             ; is set
39356
39357 getb30:
39358             ; MSDOS 6.0
39359 00006A7A 89450B     ;mov     [di+0Bh],ax
39360             MOV     [DI+BUFFINFO.buf_wrtcntinc],AX
39361
39362             ; MSDOS 3.3
39363             ;;mov     [di+8],ax ; 15/08/2018
39364             ;MOV     [DI+BUFFINFO.buf_wrtcnt],AX
39365 00006A7D E852FE     CALL     PLACEBUF
39366
39367 ;hkn; SS override for next 4
39368 getb35:
39369             ; 17/12/2022
39370             ; 07/12/2022 MSDOS 5.0 MSDOS.SYS compatibility)
39371             ; MSDOS 3.3 & MSDOS 5.0 & MSDOS 6.0
39372             ;MOV     [SS:CURBUF+2],DS
39373             ;MOV     [SS:LastBuffer+2],DS
39374             ;MOV     [SS:CURBUF],DI
39375             ;MOV     [SS:LastBuffer],DI
39376             ;CLC
39377
39378             ; 17/12/2022
39379             ; 07/12/2022
39380             ; Retro DOS v4.0
39381 00006A80 368C1E[2000]   mov     [ss:LastBuffer+2],ds
39382 00006A85 36893E[1E00]   mov     [ss:LastBuffer],di
39383 00006A8A F8             cld
39384
39385 getb35x: ; 28/07/2019
39386 00006A8B 368C1E[E405]   MOV     [ss:CURBUF+2],ds
39387 00006A90 36893E[E205]   MOV     [ss:CURBUF],di
39388
39389             ; Return with 'C' set appropriately
39390             ; (dx) = caller's original value
39391
39392 getbx:
39393             push     ss
39394             pop      ds
39395             ;retn
39396             ; 27/11/2022 MSDOS 5.0 MSDOS.SYS compatibility)
39397 getbuffrb_retn:
39398 00006A97 C3             ;flushbuf_retn: ; 17/12/2022
39399             retn
39400
39401 ;Break <FLUSHBUF -- WRITE OUT DIRTY BUFFERS>
39402 -----
39403 ; Input:
39404 ; DS = DOSGROUP
39405 ; AL = Physical unit number local buffers only
39406 ; = -1 for all units and all remote buffers
39407 ; Function:
39408 ; Write out all dirty buffers for unit, and flag them as clean
39409 ; Carry set if error (user FAILED to I 24)
39410 ; Flush operation completed.
39411 ; DS Preserved, all others destroyed (ES too)
39412 -----
39413
39414             ; 20/05/2019 - Retro DOS v4.0
39415             ; DOSCODE:9A35h (MSDOS 6.21, MSDOS.SYS)
39416
39417             ; 27/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
39418             ; DOSCODE:99DAh (MSDOS 5.0, MSDOS.SYS)

```



```

39419 ; 06/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
39420 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:0AC2Bh
39421
39422 FLUSHBUF:
39423 ; MSDOS 3.3
39424 ;;mov ah,-1 ; 01/08/2018 - Retro DOS v3.0
39425 ;lds di,[BUFFHEAD]
39426
39427 ; 06/03/2024 (PC DOS 7.1 IBMDOS.COM)
39428 ;;;
39429 cmp al,0FFh ; -1
39430 jne short flshbuf_2
39431 mov bx,26
39432
39433 flshbuf_1:
39434 ;and ss:drv_flags_1[bx],0F7h
39435 and byte [ss:bx+drive_flags-1],0F7h ; clear bit 3
39436 dec bx ; backward
39437 jnz short flshbuf_1
39438 flshbuf_2:
39439 ;;;
39440 ; MSDOS 6.0
39441 call GETCURHEAD
39442 ;TEST word [ss:DOS34_FLAG],FROM_DISK_RESET ; from disk reset ? ;hkn;
39443 TEST byte [ss:DOS34_FLAG],FROM_DISK_RESET ; 4
39444 jnz short scan_buf_queue
39445 cmp word [ss:DirtyBufferCount],0 ;hkn;
39446 je short end_scan
39447
39448 scan_buf_queue:
39449 call CHECKFLUSH
39450 ;push ax ; MSDOS 3.3
39451 ; MSDOS 6.0
39452 ;mov ah,[di+4]
39453 mov ah,[DI+BUFFINFO.buf_ID]
39454 cmp [SS:WPERR],ah ;hkn;
39455 je short free_the_buf
39456 ;TEST word [ss:DOS34_FLAG],FROM_DISK_RESET ; from disk reset ? ;hkn;
39457 TEST byte [ss:DOS34_FLAG],FROM_DISK_RESET ; 4
39458 jz short dont_free_the_buf
39459 ; MSDOS 3.3
39460 ;;mov al,[di+4]
39461 ;mov al,[DI+BUFFINFO.buf_ID]
39462 ;cmp [SS:WPERR],al ;hkn;
39463 ; 15/08/2018
39464 ;jne short dont_free_the_buf
39465 free_the_buf:
39466 ; MSDOS 6.0 (& MSDOS 3.3)
39467 mov word [DI+BUFFINFO.buf_ID],00FFh
39468 dont_free_the_buf:
39469 ;pop ax ; MSDOS 3.3
39470
39471 ; MSDOS 3.3
39472 ;mov di,[DI]
39473 ;;mov di,[DI+BUFFINFO.buf_link] ; .buf_next
39474 ;
39475 ; 15/08/2018
39476 ;lds di,[di]
39477 ;
39478 ;cmp di,-1 ; 0FFFFh
39479 ;jnz short scan_buf_queue
39480
39481 ; MSDOS 6.0
39482 mov di,[di]
39483 ;mov di,[DI+BUFFINFO.buf_next] ; .buf_link
39484 cmp di,[SS:FIRST_BUFF_ADDR] ;hkn;
39485 jne short scan_buf_queue
39486
39487 end_scan:
39488 push ss
39489 pop ds
39490 ; 01/08/2018 - Retro DOS v3.0
39491 ;cmp byte [FAILERR],0
39492 ;jne short bad_flush
39493 ;retn
39494 ;bad_flush:
39495 ;stc
39496 ;retn
39497
39498 ; 17/12/2022
39499 ; 27/11/2022 MSDOS 5.0 MSDOS.SYS compatibility)
39500 ; 01/08/2018 - Retro DOS v3.0
39501 cmp byte [FAILERR],1
39502 cmc
39503 flushbuf_retn:
39504 retn
39505
39506 ; 17/12/2022
39507 ; 27/11/2022 MSDOS 5.0 MSDOS.SYS compatibility)
39508 ;cmp byte [FAILERR],0
39509 ;jne short bad_flush
39510 ;retn
39511 ;bad_flush:
39512 ;stc
39513 ;retn
39514
39515 ;-----
39516 ;
39517 ; Procedure Name : CHECKFLUSH
39518 ;
39519 ; Inputs : AL - Drive number, -1 means do not check for drive
39520 ; DS:DI - pointer to buffer
39521 ;
39522 ; Function : Write out a buffer if it is dirty
39523 ;
39524 ; Carry set if problem (currently user FAILED to I 24)
39525 ;
39526 ;-----
39527
39528 ; 07/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
39529
39530 CHECKFLUSH:
39531 ; MSDOS 6.0
39532 mov ah,-1 ; 01/08/2018 Retro DOS v3.0
39533 ;cmp [di+4],ah
39534 cmp [DI+BUFFINFO.buf_ID],AH
39535 jz short flushbuf_retn ; skip free buffer, carry clear
39536 cmp AH,AL ;
39537 jz short DOBUFFER ; do this buffer
39538 ;cmp al,[di+4]
39539 cmp AL,[DI+BUFFINFO.buf_ID]
39540 clc
39541 jnz short flushbuf_retn ; Buffer not for this unit or SFT
39542

```

```

39543 ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
39544 ;;;
39545 xor     bx,bx
39546 mov     bl,al
39547 ;test    ss:drive_flags[bx],8
39548 test    byte [ss:bx+drive_flags],8 ; bit 3
39549 jnz     short flushbuf_retn
39550 ;;;
39551
39552 DOBUFFER:
39553 ;test    byte [di+5],40h
39554 TEST     byte [DI+BUFFINFO.buf_flags],buf_dirty
39555 jz       short flushbuf_retn ; Buffer not dirty, carry clear by TEST
39556 PUSH     AX
39557 ;push    word [di+4]
39558 PUSH     WORD [DI+BUFFINFO.buf_ID]
39559 CALL     BUFWRITE
39560 POP      AX
39561 JC      short LEAVE_BUF ; Leave buffer marked free (lost).
39562 ;and     ah,0BFh
39563 AND      AH,~buf_dirty ; Buffer is clean, clears carry
39564 ;mov     [di+4],ax
39565 MOV      [DI+BUFFINFO.buf_ID],AX
39566 LEAVE_BUF:
39567 POP      AX ; Search info
39568 checkflush_retn:
39569 retn
39570
39571 ;Break <BUFWRITE -- WRITE OUT A BUFFER IF DIRTY>
39572 -----
39573 ;
39574 ; BufWrite writes a buffer to the disk, if it's dirty.
39575 ;
39576 ; ENTRY DS:DI Points to the buffer
39577 ;
39578 ; EXIT Buffer marked free
39579 ; Carry set if error (currently user FAILED to I 24)
39580 ;
39581 ; USES All buf DS:DI
39582 ; HIGH_SECTOR
39583 -----
39584
39585 ; 20/05/2019 - Retro DOS v4.0
39586 ; DOSCODE:9AA0h (MSDOS 6.21, MSDOS.SYS)
39587
39588 ; 27/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
39589 ; DOSCODE:9A45h (MSDOS 5.0, MSDOS.SYS)
39590
39591 ; 07/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
39592 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0ACB1h
39593
39594 BUFWRITE:
39595 ; 10/09/2018
39596 ; 01/08/2018 - Retro DOS v3.0
39597 ; IBMDOS.COM (MSDOS 3.3, 1987) - Offset 5E94h
39598 MOV      AX,00FFh
39599 ;xchg    ax,[di+4]
39600 XCHG     AX,[DI+BUFFINFO.buf_ID] ; Free, in case write barfs
39601 CMP     AL,0FFh
39602 jz       short checkflush_retn ; Buffer is free, carry clear.
39603 ;test    ah,40h
39604 test    AH,buf_dirty
39605 jz       short checkflush_retn ; Buffer is clean, carry clear.
39606 ; MSDOS 6.0
39607 call    DEC_DIRTY_COUNT ; LB. decrement dirty count
39608
39609 ;hkn; SS override
39610 CMP     AL,[SS:WPERR]
39611 jz       short checkflush_retn ; If in WP error zap buffer
39612
39613 ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
39614 %if 0
39615 ;hkn; SS override
39616 ; MSDOS 6.0
39617 MOV     [SS:SC_DRIVE],AL ;LB. set it for invalidation ;AN000;
39618 %endif
39619 ; 07/03/2024
39620 ;;;les bp,[di+10] ; MSDOS 3.3
39621 ;;;les bp,[di+13] ; MSDOS 6.0
39622 ;les bp,[di+15] ; PCDOS 7.1 ; 07/03/2024
39623 ;LES BP,[DI+BUFFINFO.buf_DPB]
39624
39625 ;;;lea bx,[di+16]
39626 ;;;lea bx,[di+20] ; MSDOS 6.0
39627 ;lea bx,[di+24] ; PCDOS 7.1 ; 07/03/2024
39628 LEA     BX,[DI+BUFINSIZ] ; Point at buffer
39629
39630 ; 07/03/2024
39631 %if 0
39632 ;mov     dx,[di+6]
39633 MOV     DX,[DI+BUFFINFO.buf_sector] ;F.C. >32mb ;AN000;
39634
39635 ; MSDOS 6.0
39636 ;mov     cx,[di+8]
39637 MOV     CX,[DI+BUFFINFO.buf_sector+2] ;F.C. >32mb ;AN000;
39638
39639 ;hkn; SS override
39640 MOV     [SS:HIGH_SECTOR],CX ;F.C. >32mb ;AN000;
39641 %else
39642 ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
39643 ;;;
39644 ;les     dx,[di+6]
39645 les     dx,[di+BUFFINFO.buf_sector]
39646 mov     [ss:HIGH_SECTOR],es
39647
39648 ;;;les bp,[di+10] ; MSDOS 3.3
39649 ;;;les bp,[di+13] ; MSDOS 6.0
39650 ;les bp,[di+15] ; PCDOS 7.1 ; 07/03/2024
39651 les     bp,[di+BUFFINFO.buf_DPB]
39652 ;;;
39653 %endif
39654 ;mov     cl,[di+10] ; MSDOS 6.0 & PCDOS 7.1
39655 MOV     CL,[DI+BUFFINFO.buf_wrtcnt] ;>32mb ;AC000;
39656 ; MSDOS 3.3
39657 ;mov     cx,[di+8]
39658 mov     CX,[DI+BUFFINFO.buf_wrtcnt]
39659 MOV     AL,CH ; [DI+BUFFINFO.buf_wrtcntinc]
39660 XOR     CH,CH
39661 ;mov     ah,ch ; MSDOS 3.3
39662
39663 ;hkn; SS override for ALLOWED
39664 ;mov     byte [SS:ALLOWED],18h
39665 MOV     byte [SS:ALLOWED],Allowed_RETRY+Allowed_FAIL
39666 ;test    byte [di+5],8

```



```

39667      ; MSDOS 6.0 (& Retro DOS 3.0)
39668      ;test  ah,8
39669      test  AH,buf_isDATA
39670      JZ     short NO_IGNORE
39671      ;or     byte [SS:ALLOWED],20h
39672      OR     byte [SS:ALLOWED],Allowed_IGNORE
39673      NO_IGNORE:
39674      ;xor     ah,ah ; 10/09/2018 (MSDOS 3.3, Retro DOS v3.0)
39675      ; MSDOS 6.0
39676      ;mov     ax,[di+11]
39677      MOV     AX,[DI+BUFFINFO.buf_wrtcntinc] ;>32mb ;AC000;
39678
39679      ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
39680      ;;;
39681      ;mov     si,[di+13]
39682      mov     si,[di+BUFFINFO.buf_wrtcntinc+2]
39683      ;;;
39684
39685      PUSH     DI ; Save buffer pointer
39686      XOR     DI,DI ; Indicate failure
39687
39688      push     ds ; *
39689      push     bx ; **
39690      WRTAGAIN:
39691      ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
39692      ;;;
39693      push     si ; SI:AX = FAT size (in sectors)
39694      push     word [ss:HIGH_SECTOR]
39695      ;;;
39696      push     di ; ***
39697      push     cx ; ****
39698      push     ax ; *****
39699      ;MOV     CX,1
39700      ; 17/12/2022
39701      ; ch = 0
39702      mov     cl,1 ; 24/07/2019
39703      ; 27/11/2022 MSDOS 5.0 MSDOS.SYS compatibility)
39704      ;mov     cx,1
39705      push     bx ; *****
39706      push     dx ; *****
39707      push     ds ; *****
39708
39709      ; Note: As far as I can tell, all disk reads into buffers go through this point. -mrw 10/88
39710
39711      ; MSDOS 6.0
39712      ;cmp     byte [ss:BuffInHMA],0 ; 10/06/2019
39713      ; 22/09/2023
39714      cmp     [ss:BuffInHMA],ch ; 0
39715      jz     short NBUFFINHMA
39716      push     cx
39717      push     es
39718      mov     si,bx
39719      mov     cx,[es:bp+DPB.SECTOR_SIZE]
39720      shr     cx,1
39721      les     di,[ss:LoMemBuff] ; 10/06/2019
39722      mov     bx,di
39723      cld
39724      ;rep     movsw
39725      ; 07/03/2024 - Retro DOS v5.0
39726      ; (PCDOS 7.1 IBMDOS.COM)
39727      ;;;
39728      cmp     byte [ss:DDMOVE],0
39729      jz     short bufwr_movsw ; skip 32bit prefix
39730      shr     cx,1
39731      db     66h ; 32bit op prefix (rep movsd)
39732      bufwr_movsw:
39733      rep     movsw
39734      ;;;
39735      push     es
39736      pop     ds
39737      pop     es
39738      pop     cx
39739      NBUFFINHMA:
39740      call    DWRITE ; Write out the dirty buffer
39741      pop     ds ; *****
39742      pop     dx ; *****
39743      pop     bx ; *****
39744      pop     ax ; *****
39745      pop     cx ; ****
39746      pop     di ; ***
39747      ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
39748      ;;;
39749      pop     word [ss:HIGH_SECTOR]
39750      pop     si
39751      ;;;
39752      JC     short NOSET
39753      ; 05/07/2024 (PCDOS 7.1)
39754      ;mov     di,1
39755      INC     DI ; If at least ONE write succeeds,
39756      NOSET: ; the operation succeeds.
39757      ADD     DX,AX
39758      ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
39759      ;;;
39760      adc     [ss:HIGH_SECTOR],si
39761      ;;;
39762      LOOP    WRTAGAIN
39763      pop     bx ; **
39764      pop     ds ; *
39765      ;OR     DI,DI ; Clears carry
39766      ;JNZ    short BWROK ; At least one write worked
39767      ;STC ; DI never got INCed, all writes failed.
39768      ; 05/07/2024 (PCDOS 7.1)
39769      ;sub     di,1
39770      ; 22/09/2023
39771      cmp     di,1
39772      BWROK:
39773      POP     DI
39774      retn
39775
39776      ; 07/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
39777      %if 0
39778      ; NOTE: Secondary Buffer Cache is not used in PCDOS 7.1 IBMDOS.COM.
39779
39780      ;** Set_RQ_SC_Parms - Set Secondary Cache Parameters
39781      ;-----
39782      ; Set_RQ_SC_Parms sets the sector size and drive number value
39783      ; for the secondary cache. This updates SC_SECTOR_SIZE &
39784      ; SC_DRIVE even if SC is disabled to save the testing
39785      ; code and time
39786      ;
39787      ; ENTRY ES:BP = drive parameter block
39788      ;
39789      ; EXIT [SC_SECTOR_SIZE]= drive sector size
39790      ; [SC_DRIVE]= drive #

```

```

39791 ;
39792 ; USES Flags
39793 ;-----
39794 ;
39795 ; 27/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
39796 ; 04/05/2019 - Retro DOS v4.0
39797
39798 SET_RQ_SC_PARAMS:
39799 ;hkn; SS override for all variables used in this procedure.
39800 push ax
39801 ;mov ax,[es:bp+2]
39802 MOV ax,[ES:BP+DPB.SECTOR_SIZE] ; save sector size
39803 MOV [ss:SC_SECTOR_SIZE],ax
39804 ;;mov al,[es:bp+0]
39805 ; 27/11/2022 MSDOS 5.0 MSDOS.SYS compatibility)
39806 ;MOV al,[ES:BP+DPB.DRIVE] ; save drive #
39807 ; 15/12/2022
39808 mov al,[ES:BP]
39809 MOV [ss:SC_DRIVE],al
39810 pop ax
39811 srsp:
39812 retn ;LB. return
39813
39814 %endif
39815
39816 ; 07/03/2024 -Retro DOS v5.0
39817 ; (PCDOS 7.1 IBMDOS.COM - DOSCODE:0AD57h)
39818 %if 0
39819 null_sub:
39820 SET_RQ_SC_PARAMS:
39821 retn
39822 %endif
39823
39824 ; 01/02/2024 - Retro DOS v5.0
39825 ;-----
39826 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:AD58h
39827
39828 SET_BUF_DIRTY: ; ...
39829 ;test byte [es:di+5],40h
39830 test byte [es:di+BUFFINFO.buf_flags],buf_dirty
39831 jnz short yesdirty2
39832 ;or byte [es:di+5],40h
39833 or byte [es:di+BUFFINFO.buf_flags],buf_dirty
39834
39835 ;INC_DIRTY_COUNT:
39836 ; inc word [ss:DirtyBufferCount]
39837 ;yesdirty2:
39838 ; retn
39839
39840 ;Break <INC_DIRTY_COUNT-increment dirty count>
39841 ;-----
39842 ; Input:
39843 ; none
39844 ; Function:
39845 ; increment dirty buffers count
39846 ; Output:
39847 ; dirty buffers count is incremented
39848 ;
39849 ; All registers preserved
39850 ;-----
39851
39852 INC_DIRTY_COUNT:
39853 ;; BUGBUG ---- remove this routine
39854 ;; BUGBUG ---- only one instruction is needed (speed win, space loose)
39855 inc word [ss:DirtyBufferCount] ;hkn;
39856 yesdirty2: ; 01/02/2024
39857 retn
39858
39859 ;Break <DEC_DIRTY_COUNT-decrement dirty count>
39860 ;-----
39861 ; Input:
39862 ; none
39863 ; Function:
39864 ; decrement dirty buffers count
39865 ; Output:
39866 ; dirty buffers count is decremented
39867 ;
39868 ; All registers preserved
39869 ;-----
39870
39871 DEC_DIRTY_COUNT:
39872 cmp word [ss:DirtyBufferCount],0 ;hkn;
39873 jz short ddcx ; BUGBUG - shouldn't it be an
39874 dec word [ss:DirtyBufferCount]
39875 ; error condition to underflow here? ;hkn;
39876 ddcx:
39877 retn
39878
39879 ;=====
39880 ; MSPROC.ASM, MSDOS 6.0, 1992
39881 ;=====
39882 ; 02/08/2018 - Retro DOS v3.0
39883 ; 29/04/2019 - Retro DOS v4.0
39884 ; 07/03/2024 - Retro DOS v5.0
39885
39886 ; (15/04/2018 - Retro DOS v2.0, MSDOS 2.11 - PROC.ASM - 1983)
39887
39888 ; Pseudo EXEC system call for DOS
39889
39890 ; TITLE MSPROC - process maintenance
39891 ; NAME MSPROC
39892
39893 ; =====
39894 ; ** Process related system calls and low level routines for DOS 2.x.
39895 ; I/O specs are defined in DISPATCH.
39896 ;
39897 ; $WAIT
39898 ; $EXEC
39899 ; $Keep_process
39900 ; Stay_resident
39901 ; $EXIT
39902 ; $ABORT
39903 ; abort_inner
39904 ;
39905 ; Modification history:
39906 ;
39907 ; Created: ARR 30 March 1983
39908 ; AN000 version 4.0 jan. 1988
39909 ; A007 PTM 3957 - fake vesrion for IBMCACHE.COM
39910 ; A008 PTM 4070 - fake version for MS WINDOWS
39911 ;
39912 ; M000 added support for loading programs into UMBs 7/9/90
39913 ;
39914 ; M004 - MS PASCAL 3.2 support. Please see under tag M003 in

```

```

39915 ; dossym.inc. 7/30/90
39916 ;
39917 ; M005 - Support for EXE programs with out STACK segment and
39918 ; with resident size < 64K - 256 bytes. A 256 byte
39919 ; stack is provided at the end of the program. Note that
39920 ; only SP is changed.
39921 ; M020 - Fix for Rational bug for details see exepatch.asm
39922 ;
39923 ; M028 - 4b04 implementation
39924 ;
39925 ; M029 - Support for EXEs without stack rewritten. If EXE is
39926 ; in memory block >= 64K, sp = 0. If memory block
39927 ; obtained is <64K, point sp at the end of the memory
39928 ; block. For EXEs smaller than 64K, 256 bytes are still
39929 ; added for a stack segment which may be needed if it
39930 ; is loaded in low memory situations.
39931 ;
39932 ; M030 - Fixing bug in EXEPATCH & changing 4b04 to 4b05
39933 ;
39934 ; M040 - Bug #3052. The environment sizing code would flag a
39935 ; a bad environment if it reached 32767 bytes. Changed
39936 ; to allow 32768 bytes of environment.
39937 ;
39938 ; M047 - Release the allocated UMB when we failed to load a
39939 ; COM file high. Also ensure that if the biggest block
39940 ; into which we load the com file is less than 64K then
39941 ; we provide atleast 256 bytes of stack to the user.
39942 ;
39943 ; M050 - Made Lie table search CASE insensitive
39944 ;
39945 ; M060 - Removed special version table from the kernal and
39946 ; put it in a device drive which puts the address
39947 ; in the DOS DATA area location UU_IFS_DOS_CALL
39948 ; as a DWORD.
39949 ;
39950 ; M063 - Modified UMB support. If the HIGH_ONLY bit is set on
39951 ; entry do not try to load low if there is no space in
39952 ; UMBs.
39953 ;
39954 ; M068 - Support for copy protect apps. Call ChkCopyProt to
39955 ; set a20off_count. Set bit EXECA20BIT in DOS_FLAG. Also
39956 ; change return address to LeaveDos if AL=5.
39957 ;
39958 ; 20-Jul-1992 bens Added ifdef RESTRICTED_BUILD code that
39959 ; controls building a version of MSDOS.SYS that only
39960 ; runs programs from a fixed list (defined in the
39961 ; file RESTRICT.INC). Search for "RESTRICTED_BUILD"
39962 ; for details. This feature is used to build a
39963 ; "special" version of DOS that can be handed out to
39964 ; OEM/ISV customers as part of a "service" disk.
39965 ;
39966 ; =====
39967 ; SAVEXIT EQU 10
39968 ;
39969 ; BREAK <$WAIT - return previous process error code>
39970 ;
39971 ; $WAIT - Return previous process error code.
39972 ;
39973 ; Assembler usage:
39974 ;
39975 ; MOV AH, WaitProcess
39976 ; INT int_command
39977 ;
39978 ; ENTRY none
39979 ; EXIT (ax) = exit code
39980 ; USES all
39981 ; =====
39982 ;
39983 ; 20/05/2019 - Retro DOS v4.0
39984 ; DOSCODE:9B55h (MSDOS 6.21, MSDOS.SYS)
39985 ;
39986 ; 27/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
39987 ; DOSCODE:9A5Ah (MSDOS 5.0, MSDOS.SYS)
39988 ;
39989 ; 07/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
39990 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0AD78h
39991 ;
39992 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:9B55h)
39993 ; (Windows ME IO.SYS - BIOSCODE:944Ch)
39994 ;
39995 ;_$WAIT:
39996 ; 02/08/2018 - Retro DOS v3.0
39997 ; IBMDOS.COM (MSDOS 3.3, 1987) - Offset 5E1h
39998 ;
39999 00006BDE 31C0 xor AX,AX
40000 00006BE0 368706[3403] xchg AX,[ss:exit_code]
40001 00006BE5 E9869A jmp SYS_RET_OK
40002 ;
40003 ; =====
40004 ; BREAK <$exec - load/go a program>
40005 ; EXEC.ASM - EXEC System Call
40006 ;
40007 ;
40008 ; Assembler usage:
40009 ; lds DX, Name
40010 ; les BX, Blk
40011 ; mov AH, Exec
40012 ; mov AL, FUNC
40013 ; int INT_COMMAND
40014 ;
40015 ; AL Function
40016 ; --
40017 ; 0 Load and execute the program.
40018 ; 1 Load, create the program header but do not
40019 ; begin execution.
40020 ; 3 Load overlay. No header created.
40021 ;
40022 ; AL = 0 -> load/execute program
40023 ;
40024 ; +-----+
40025 ; | WORD segment address of |
40026 ; | environment. |
40027 ; +-----+
40028 ; | DWORD pointer to ASCIZ |
40029 ; | command line at 80h |
40030 ; +-----+
40031 ; | DWORD pointer to default |
40032 ; | FCB to be passed at 5Ch |
40033 ; +-----+
40034 ; | DWORD pointer to default |
40035 ; | FCB to be passed at 6Ch |
40036 ; +-----+
40037 ;
40038 ; AL = 1 -> load program

```

```

40039 ;
40040 ;
40041 ; +-----+
40042 ; | WORD segment address of |
40043 ; | environment.           |
40044 ; +-----+
40045 ; | DWORD pointer to ASCIZ |
40046 ; | command line at 80h   |
40047 ; +-----+
40048 ; | DWORD pointer to default |
40049 ; | FCB to be passed at 5Ch |
40050 ; +-----+
40051 ; | DWORD pointer to default |
40052 ; | FCB to be passed at 6Ch |
40053 ; +-----+
40054 ; | DWORD returned value of |
40055 ; | CS:IP                   |
40056 ; +-----+
40057 ; | DWORD returned value of |
40058 ; | SS:IP                   |
40059 ; +-----+
40060 ;
40061 ; AL = 3 -> load overlay
40062 ;
40063 ; +-----+
40064 ; | WORD segment address where |
40065 ; | file will be loaded.      |
40066 ; +-----+
40067 ; | WORD relocation factor to |
40068 ; | be applied to the image.  |
40069 ; +-----+
40070 ;
40071 ; Returns:
40072 ;     AX = error_invalid_function
40073 ;         = error_bad_format
40074 ;         = error_bad_environment
40075 ;         = error_not_enough_memory
40076 ;         = error_file_not_found
40077 ;
40078 ;
40079 ; Revision history:
40080 ;
40081 ;     A000   version 4.00   Jan. 1988
40082 ;
40083 ; =====
40084 ; Exec_Internal_Buffer      EQU      OPENBUF
40085 ; Exec_Internal_Buffer_Size EQU      (128+128+53+curdirLen)
40086 ;
40087 ; =====
40088 ;
40089 ; IF1      ; warning message on buffers
40090 ; %out     Please make sure that the following are contiguous and of the
40091 ; %out     following sizes:
40092 ; %out
40093 ; %out     OpenBuf      128
40094 ; %out     RenBuf      128
40095 ; %out     SearchBuf   53
40096 ; %out     DummyCDS    curdirLen
40097 ; ENDIF
40098 ;
40099 ; =====
40100 ;
40101 ; =====
40102 ;
40103 ; =====
40104 ;
40105 ; 20/05/2019 - Retro DOS v4.0
40106 ; DOSCODE:9B5Fh (MSDOS 6.21, MSDOS.SYS)
40107 ;
40108 ; 30/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
40109 ; DOSCODE:9B04h (MSDOS 5.0, MSDOS.SYS)
40110 ;
40111 ; 08/03/2024
40112 ; 07/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
40113 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:0AD82h
40114 ;
40115 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:9B5Fh)
40116 ; (Windows ME IO.SYS - BIOSCODE:946Fh)
40117 ;
40118 ; _$EXEC:
40119 ; 02/08/2018 - Retro DOS v3.0
40120 ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 5EF1h
40121 ;
40122 ; EXEC001S:
40123 ; LocalVar   Exec_Blk      ,DWORD
40124 ; LocalVar   Exec_Func     ,BYTE
40125 ; LocalVar   Exec_Load_High ,BYTE
40126 ; LocalVar   Exec_FH       ,WORD
40127 ; LocalVar   Exec_Rel_Fac  ,WORD
40128 ; LocalVar   Exec_Res_Len_Para ,WORD
40129 ; LocalVar   Exec_Environ  ,WORD
40130 ; LocalVar   Exec_Size     ,WORD
40131 ; LocalVar   Exec_Load_Block ,WORD
40132 ; LocalVar   Exec_DMA      ,WORD
40133 ; LocalVar   ExecNameLen   ,WORD
40134 ; LocalVar   ExecName      ,DWORD
40135 ;
40136 ; LocalVar   Exec_DMA_Save ,WORD
40137 ; LocalVar   Exec_NoStack  ,BYTE
40138 ;
40139 ; MSDOS 3.3 (& MSDOS 6.0)
40140 ; %define     Exec_Blk      dword [bp-4]
40141 ; %define     Exec_Blk     [bp-4] ; 09/08/2018
40142 ; %define     Exec_BlkL    word [bp-4]
40143 ; %define     Exec_BlkH    word [bp-2]
40144 ; %define     Exec_Func     byte [bp-5]
40145 ; %define     Exec_Load_High byte [bp-6]
40146 ; %define     Exec_FH      word [bp-8]
40147 ; %define     Exec_Rel_Fac word [bp-10]
40148 ; %define     Exec_Res_Len_Para word [bp-12]
40149 ; %define     Exec_Environ word [bp-14]
40150 ; %define     Exec_Size    word [bp-16]
40151 ; %define     Exec_Load_Block word [bp-18]
40152 ; %define     Exec_DMA      word [bp-20]
40153 ; %define     ExecNameLen   word [bp-22]
40154 ; %define     ExecName      dword [bp-26]
40155 ; %define     ExecName     [bp-26] ; 09/08/2018
40156 ; %define     ExecNameL    word [bp-26]
40157 ; %define     ExecNameH     word [bp-24]
40158 ; MSDOS 6.0
40159 ; %define     Exec_DMA_Save word [bp-28]
40160 ; %define     Exec_NoStack  byte [bp-29]
40161 ;
40162 ; =====

```

```

40163 ; validate function
40164 ; =====
40165 ;
40166 ; M068 - Start
40167 ;
40168 ; Reset the A20OFF_COUNT to 0. This is done as there is a
40169 ; possibility that the count may not be decremented all the way to
40170 ; 0. A typical case is if the program for which we intended to keep
40171 ; the A20 off for a sufficiently long time (A20OFF_COUNT int 21
40172 ; calls), exits pre-maturely due to error conditions.
40173 ;
40174 ; MSDOS 6.0
40175 00006BE8 36C606[8500]00 mov byte [ss:A20OFF_COUNT], 0
40176 ;
40177 ; If al=5 (ExecReady) we'll change the return address on the stack
40178 ; to be LeaveDos in msdisp.asm. This ensures that the EXECA200FF
40179 ; bit set in DOS_FLAG by ExecReady is not cleared in msdisp.asm
40180 ;
40181 00006BEE 3C05 cmp al,5 ; Q: is this ExecReady call
40182 ;jne short @f ; N: continue
40183 00006BF0 7505 jne short Exec_@f ; Y: change ret addr. to LeaveDos.
40184 ; Note CX is not input to ExecReady
40185 00006BF2 59 pop cx
40186 00006BF3 B9[F603] mov cx,LeaveDOS
40187 00006BF6 51 push cx
40188 ;@@:
40189 Exec_@f:
40190 ; M068 - End
40191 ;
40192 ;Enter
40193 ;
40194 00006BF7 55 push bp
40195 00006BF8 89E5 mov bp,sp
40196 ;sub sp,26 ; MSDOS 3.3
40197 ; 30/11/2022 (MSDOS 5.0, MSDOS.SYS compatibility)
40198 ;sub sp,29 ; MSDOS 6.0 & MSDOS 6.22 & PC DOS 7.1 ; 07/03/2024
40199 ; 17/12/2022
40200 ; 20/05/2019
40201 00006BFA 83EC1E sub sp,30 ; Retro DOS v4.0
40202 ;
40203 ; MSDOS 6.0
40204 00006BFD 3C05 cmp AL,5 ; only 0, 1, 3 or 5 are allowed ;M028
40205 ; M030
40206 00006BFF 7611 jna short Exec_Check_2
40207 ;
40208 ; MSDOS 3.3
40209 ;cmp AL,3
40210 ;jna short Exec_Check_2
40211 ;
40212 Exec_Bad_Fun:
40213 ;mov byte [ss:EXTERR_LOCUS],errLOC_unk ; 1
40214 ; Extended Error Locus;smr;SS Override
40215 ; 01/07/2024
40216 00006C01 E8D9A6 call set_exerr_locus_unk
40217 ;
40218 ;mov al,1
40219 00006C04 B001 mov al,error_invalid_function
40220 ;
40221 Exec_Ret_Err:
40222 ;Leave
40223 00006C06 89EC mov sp,bp
40224 00006C08 5D pop bp
40225 ;transfer SYS_RET_ERR
40226 00006C09 E96C9A jmp SYS_RET_ERR
40227 ;
40228 ; MSDOS 6.0
40229 ExecReadyJ:
40230 00006C0C E89C18 call ExecReady ; M028
40231 00006C0F E91204 jmp norm_ovl ; do a Leave & xfer sysret_OK ; M028
40232 ;
40233 Exec_Check_2:
40234 00006C12 3C02 cmp AL,2
40235 00006C14 74EB je short Exec_Bad_Fun
40236 ;
40237 ; MSDOS 6.0
40238 00006C16 3C04 cmp al,4 ; 2 & 4 are not allowed
40239 00006C18 74E7 je short Exec_Bad_Fun
40240 ;
40241 00006C1A 3C05 cmp al,5 ; M028 ; M030
40242 00006C1C 74EE je short ExecReadyJ ; M028
40243 ;
40244 ;mov [bp-4],bx
40245 00006C1E 895EFC mov Exec_BlkL,BX ; stash args
40246 ;mov [bp-2],es
40247 00006C21 8C46FE mov Exec_BlkH,ES
40248 ;mov [bp-5],al
40249 00006C24 8846FB mov Exec_Func,AL
40250 ;mov byte [bp-6],0
40251 00006C27 C646FA00 mov Exec_Load_High,0
40252 ;
40253 ;mov [bp-26],dx
40254 00006C2B 8956E6 mov ExecNameL,DX ; set up length of exec name
40255 ;mov [bp-24],ds
40256 00006C2E 8C5EE8 mov ExecNameH,DS
40257 00006C31 89D6 mov SI,DX ; move pointer to convenient place
40258 ;invoke DStrLen
40259 00006C33 E8A1AB call DStrLen
40260 ;mov [bp-22],cx
40261 00006C36 894EEA mov ExecNameLen,CX ; save length
40262 ;
40263 ; MSDOS 6.0
40264 00006C39 36A0[0203] mov al,[ss:AllocMethod] ; M063: save alloc method in
40265 00006C3D 36A2[8400] mov [ss:ALLOCMSAVE],al ; M063: AllocMsave
40266 ;
40267 ;xor AL,AL ; open for reading
40268 ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
40269 ;;;
40270 00006C41 B0A0 mov al,0A0h ; Access mode bits: (0 to 2)
40271 ; 000 read access
40272 ;
40273 ; Sharing mode bits: (4 to 6)
40274 ; 010 deny others write access
40275 ;
40276 ; bit 7 - private
40277 ;;;
40278 ;
40279 00006C43 55 push BP
40280 ;
40281 ; MSDOS 6.0
40282 ;or byte [ss:DOS_FLAG],1
40283 00006C44 36800E[8600]01 or byte [ss:DOS_FLAG],EXECOPEN ; this flag is set to indicate to
40284 ; the redir that this open call is
40285 ; due to an exec.
40286 ;

```

```

40287 ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
40288 ;;;
40289 00006C4A 1E push ds
40290 00006C4B 52 push dx
40291 ;;;
40292
40293 ;invoke $OPEN ; is the file there?
40294 00006C4C E87A13 call _$OPEN
40295
40296 ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
40297 ;;;
40298 00006C4F 5A pop dx
40299 00006C50 1F pop ds
40300 00006C51 7305 jnc short Exec_Open_Ok
40301
40302 00006C53 30C0 xor al,al ; open for reading
40303 00006C55 E87113 call _$OPEN
40304
40305 Exec_Open_Ok:
40306 ;;;
40307
40308 ; MSDOS 6.0
40309 00006C58 9C pushf
40310 ; 02/06/2019
40311 ;and byte [ss:DOS_FLAG],0FEh
40312 00006C59 368026[8600]FE and byte [ss:DOS_FLAG],~EXECOPEN ; reset flag
40313 00006C5F 9D popf
40314
40315 00006C60 5D pop BP
40316
40317 ; MSDOS 3.3 & MSDOS 6.0
40318 00006C61 72A3 jc short Exec_Ret_Err
40319
40320 ;mov [bp-8],ax
40321 00006C63 8946F8 mov Exec_FH,AX
40322 00006C66 89C3 mov BX,AX
40323 00006C68 30C0 xor AL,AL
40324 ;invoke $Ioctl
40325 00006C6A E817BC call _$IOCTL
40326 00006C6D 7207 jc short Exec_BombJ
40327
40328 ;test dl,80h
40329 00006C6F F6C280 test DL,devid_ISDEV
40330 00006C72 740A jz short Exec_Check_Environ
40331
40332 ;mov al,2
40333 00006C74 B002 mov AL,error_file_not_found
40334 Exec_BombJ:
40335 00006C76 E9C800 jmp Exec_Bomb
40336
40337 BadEnv:
40338 ;mov al,0Ah
40339 00006C79 B00A mov AL,error_bad_environment
40340 00006C7B E9C300 jmp Exec_Bomb
40341
40342 Exec_Check_Environ:
40343 ;mov word [bp-18],0
40344 00006C7E C746EE0000 mov Exec_Load_Block,0
40345 ;mov word [bp-14],0
40346 00006C83 C746F20000 mov Exec_Environ,0
40347 ; overlays... no environment
40348
40349 00006C88 F646FB02 ;test byte [bp-5],2
40350 00006C8C 7552 test Exec_Func,exec_func_overlay
40351 jnz short Exec_Read_Header
40352
40353 ;lds si,[bp-4]
40354 00006C8E C576FC lds SI,Exec_Blkl ; get block
40355 00006C91 8B04 mov ax,[SI]
40356 ;mov ax,[SI+EXEC1.ENVIRON] ; address of environ
40357 00006C93 09C0 ;mov ax,ax
40358 00006C95 750C jnz short Exec_Scan_Env
40359
40359 00006C97 368E1E[3003] mov DS,[SS:CurrentPDB] ;smr;SS Override
40360 ;mov ax,[44]
40361 00006C9C A12C00 mov AX,[PDB.ENVIRON]
40362
40363 ; MSDOS 6.0
40364 ;-----BUG 92 4/30/90-----
40365 ;
40366 ; Exec_environ is being correctly initialized after the environment has been
40367 ; allocated and copied form the parent's env. It must not be initialized here.
40368 ; Because if the call to $alloc below fails Exec_dealloc will deallocate the
40369 ; parent's environment.
40370 ; mov Exec_Environ,AX
40371 ;
40372 ;-----
40373
40374 ;mov [bp-14],ax
40375 ;mov Exec_Environ,ax
40376
40377 00006C9F 09C0 or AX,AX
40378 00006CA1 743D jz short Exec_Read_Header
40379
40380 Exec_Scan_Env:
40381 00006CA3 8EC0 mov ES,AX
40382 00006CA5 31FF xor DI,DI
40383 ;mov cx,7FFFh ; MSDOS 3.3
40384 00006CA7 B90080 mov CX,8000h ; MSDOS 6.0 ; at most 32k of environment ;M040
40385 00006CAA 30C0 xor AL,AL
40386
40387 Exec_Get_Environ_Len:
40388 00006CAC F2AE repnz scasb ; find that nul byte
40389 00006CAE 75C9 jnz short BadEnv
40390
40391 00006CB0 49 dec CX ; Dec CX for the next nul byte test
40392 00006CB1 78C6 js short BadEnv ; gone beyond the end of the environment
40393
40394 00006CB3 AE scasb ; is there another nul byte?
40395 00006CB4 75F6 jnz short Exec_Get_Environ_Len ; no, scan some more
40396
40397 00006CB6 57 push DI
40398 ;lea bx,[DI+11h]
40399 00006CB7 8D5D11 lea BX,[DI+0Fh+2]
40400 ;add bx,[bp-22]
40401 00006CBA 035EEA add BX,ExecNameLen ; BX <- length of environment
40402 ; remember argv[0] length
40403 ; round up and remember argc
40404 00006CBD B104 mov CL,4
40405 00006CBF D3EB shr BX,CL ; number of paragraphs needed
40406 00006CC1 06 push ES
40407 ;invoke $Alloc ; can we get the space?
40408 00006CC2 E84906 call _$ALLOC
40409 00006CC5 1F pop DS
40410 00006CC6 59 pop CX

```



```

40411
40412 ;jnc short Exec_Save_Environ
40413 ;jmp SHORT Exec_No_Mem ; nope... cry and sob
40414 ; 17/12/2022
40415 00006CC7 7272 jnc short Exec_No_Mem ; 02/06/2019
40416 ; 30/11/2022 (MSDOS 5.0, MSDOS.SYS compatibility)
40417 ;jnc short Exec_Save_Environ
40418 ;jmp SHORT Exec_No_Mem
40419
40420 Exec_Save_Environ:
40421 00006CC9 8EC0 mov ES,AX
40422 ;mov [bp-14],ax
40423 00006CCB 8946F2 mov Exec_Environ,AX ; save him for a rainy day
40424 00006CCE 31F6 xor SI,SI
40425 00006CD0 89F7 mov DI,SI
40426 00006CD2 F3A4 rep movsb ; copy the environment
40427 00006CD4 B80100 mov AX,1
40428 00006CD7 AB stosw
40429 ;lds si,[bp-26]
40430 00006CD8 C576E6 lds SI,ExecName
40431 ;mov cx,[bp-22]
40432 00006CDB 8B4EEA mov CX,ExecNameLen
40433 00006CDE F3A4 rep movsb
40434
40435 Exec_Read_Header:
40436 ; We read in the program header into the above data area and
40437 ; determine where in this memory the image will be located.
40438
40439 ;Context DS
40440 00006CE0 16 push ss
40441 00006CE1 1F pop ds
40442 ;mov cx,26
40443 00006CE2 B91A00 mov CX,exec_header_len ; header size
40444 00006CE5 BA[C70E] mov DX,exec_signature
40445 00006CE8 06 push ES
40446 00006CE9 1E push DS
40447 00006CEA E88204 call ExecRead
40448 00006CED 1F pop DS
40449 00006CEE 07 pop ES
40450 00006CEF 724E jnc short Exec_Bad_File
40451
40452 00006CF1 09C0 or AX,AX
40453 00006CF3 744A jz short Exec_Bad_File
40454 ;cmp ax,26
40455 00006CF5 83F81A cmp AX,exec_header_len ; did we read the right number?
40456 00006CF8 7519 jnz short Exec_Com_Filej ; yep... continue
40457
40458 00006CFA F706[D30E]FFFF test word [exec_max_BSS],-1 ; indicate load high?
40459 00006D00 7504 jnz short Exec_Check_Sig
40460
40461 ;mov byte [bp-6],0FFh
40462 00006D02 C646FAFF mov Exec_Load_High,-1
40463
40464 Exec_Check_Sig:
40465 00006D06 A1[C70E] mov AX,[exec_signature] ; rms;NSS
40466 ;cmp ax,5A4Dh ; 'MZ'
40467 00006D09 3D4D5A cmp AX,exe_valid_signature; zibo arises!
40468 00006D0C 7408 jz short Exec_Save_Start ; assume com file if no signature
40469
40470 ;cmp ax,4D5Ah ; 'ZM'
40471 00006D0E 3D5A4D cmp AX,exe_valid_old_signature ; zibo arises!
40472 00006D11 7403 jz short Exec_Save_Start ; assume com file if no signature
40473
40474 Exec_Com_Filej:
40475 00006D13 E9EC01 jmp Exec_Com_File
40476
40477 ; We have the program header... determine memory requirements
40478
40479 Exec_Save_Start:
40480 00006D16 A1[C80E] mov AX,[exec_pages] ; get 512-byte pages ;rms;NSS
40481 00006D19 B105 mov CL,5 ; convert to paragraphs
40482 00006D1B D3E0 shl AX,CL
40483 00006D1D 2B06[CF0E] sub AX,[exec_par_dir] ; AX = size in paragraphs ;rms;NSS
40484 ;mov [bp-12],ax
40485 00006D21 8946F4 mov Exec_Res_Len_Para,AX
40486
40487 ; Do we need to allocate memory?
40488 ; Yes if function is not load-overlay
40489
40490 ;test byte [bp-5],2
40491 00006D24 F646FB02 test Exec_Func,exec_func_overlay
40492 00006D28 7443 jz short Exec_Allocate ; allocation of space
40493
40494 ; get load address from block
40495
40496 ;les di,[bp-4]
40497 00006D2A C47EFC les DI,Exec_Blk
40498
40499 ; 07/03/2024
40500 %if 0
40501 mov ax,[es:di]
40502 ;mov AX,[ES:DI+EXEC3.load_addr]
40503 ;mov [bp-20],ax
40504 mov Exec_DMA,AX
40505
40506 ; 17/12/2022
40507 ;;mov ax,[es:di+2]
40508 ;mov AX,[ES:DI+EXEC3.reloc_fac]
40509 ;;mov [bp-10],ax
40510 ;mov Exec_Rel_Fac,AX
40511
40512 ; 17/12/2022
40513 ; 30/11/2022 (!most proper code!)
40514 ;mov dx,[es:di+2]
40515 mov dx,[ES:DI+EXEC3.reloc_fac]
40516 ;mov [bp-10],dx
40517 mov Exec_Rel_Fac,dx
40518
40519 %else
40520 ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
40521 ;;;
40522 00006D2D 06 push es
40523 00006D2E 26C405 les ax,[es:di]
40524 ;les ax,[ES:DI+EXEC3.load_addr]
40525 ;mov [bp-20],ax
40526 00006D31 8946EC mov Exec_DMA,ax
40527 ;mov [bp-10],es
40528 00006D34 8C46F6 mov Exec_Rel_Fac,es
40529 00006D37 07 pop es
40530 ;;;
40531 %endif
40532 ; ax = Exec_DMA
40533 jmp Exec_Find_Res
40534
40535 ; 17/12/2022

```

```

40535 ; 30/11/2022 (MSDOS 5.0, MSDOS.SYS compatibility)
40536 ; 27/09/2023
40537 %if 0
40538 ; 02/06/2019 - Retro DOS v4.0
40539 ;mov ax,[bp-20] ; **
40540 mov AX,Exec_DMA ; **
40541 ; 10/08/2018
40542 jmp Exec_Find_Res ; M000
40543 %endif
40544
40545 Exec_No_Mem:
40546 ;mov al,8
40547 00006D3B B008 mov AL,error_not_enough_memory
40548 00006D3D EB02 jmp short Exec_Bomb
40549
40550 Exec_Bad_File:
40551 ;mov al,0Bh
40552 00006D3F B00B mov AL,error_bad_format
40553
40554 Exec_Bomb:
40555 ;mov bx,[bp-8]
40556 00006D41 8B5EF8 mov BX,Exec_FH
40557 00006D44 E84104 call Exec_Dealloc
40558 ;LeaveCrit CritMem
40559 00006D47 E8C5AB call LCritMEM
40560 ;save <AX,BP>
40561 00006D4A 50 push ax
40562 00006D4B 55 push bp
40563 ;invoke $CLOSE
40564 00006D4C E81E0A call _$CLOSE
40565 ;restore <BP,AX>
40566 00006D4F 5D pop bp
40567 00006D50 58 pop ax
40568 00006D51 E9B2FE jmp Exec_Ret_Err
40569
40570 Exec_Chk_Mem:
40571 ; 24/09/2023
40572 ; ds = DOSDATA
40573 ; 17/12/2022
40574 ; 30/11/2022 (MSDOS 5.0, MSDOS.SYS compatibility)
40575 %if 0
40576 ; MSDOS 6.0 ; M063 - Start
40577 ;mov al,[ss:AllocMethod] ; save current alloc method in ax
40578 ; 10/06/2019
40579 00006D54 A0[0203] mov al,[AllocMethod]
40580 ;mov b1,[ss:ALLOCMSAVE]
40581 00006D57 8A1E[8400] mov b1,[ALLOCMSAVE]
40582 ;mov [ss:AllocMethod],b1 ; restore original allocmethod
40583 00006D5B 881E[0203] mov [AllocMethod],b1
40584
40585 00006D5F F6C340 test b1,HIGH_ONLY ; 40h ; Q: was the HIGH_ONLY bit already set
40586 00006D62 75D7 jnz short Exec_No_Mem ; Y: no space in UMBS. Quit
40587 ; ; N: continue
40588 ;
40589 00006D64 A840 test al,HIGH_ONLY ; Q: did we set the HIGH_ONLY bit
40590 00006D66 74D3 jz short Exec_No_Mem ; N: no memory
40591 ; 02/06/2019
40592 ;mov ax,[ss:SAVE_AX] ; Y: restore ax and
40593 00006D68 A1[8A00] mov ax,[SAVE_AX]
40594 ;jmp short Exec_Norm_Alloc ; Try again
40595 ; M063 - End
40596 00006D6B EB2B jmp short Exec_Norm_Alloc1
40597 %endif
40598
40599 ; 17/12/2022
40600 %if 0
40601 ; 30/11/2022 (MSDOS 5.0, MSDOS.SYS compatibility)
40602 ; MSDOS 6.0 ; M063 - Start
40603 mov al,[ss:AllocMethod] ; save current alloc method in ax
40604 mov b1,[ss:ALLOCMSAVE]
40605 mov [ss:AllocMethod],b1 ; restore original allocmethod
40606
40607 test b1,HIGH_ONLY ; 40h ; Q: was the HIGH_ONLY bit already set
40608 jnz short Exec_No_Mem ; Y: no space in UMBS. Quit
40609 ; ; N: continue
40610 ;
40611 test al,HIGH_ONLY ; Q: did we set the HIGH_ONLY bit
40612 jz short Exec_No_Mem ; N: no memory
40613
40614 mov ax,[ss:SAVE_AX] ; Y: restore ax and
40615 jmp short Exec_Norm_Alloc ; Try again
40616 ; M063 - End
40617 %endif
40618
40619 Exec_Allocate:
40620 ; 09/09/2018
40621
40622 ; M005 - START
40623 ; If there is no STACK segment for this exe file and if this
40624 ; not an overlay and the resident size is less than 64K -
40625 ; 256 bytes we shall add 256 bytes to the programs
40626 ; resident memory requirement and set Exec_SP to this value.
40627
40628 ; 17/12/2022
40629 00006D6D 29DB sub bx,bx ; 0
40630
40631 ; MSDOS 6.0
40632 ;mov byte [bp-29],0
40633 ;mov Exec_NoStack,0
40634 ; 17/12/2022
40635 00006D6F 885EE3 mov Exec_NoStack,b1 ; 0
40636 00006D72 391E[D50E] cmp [exec_SS],bx ; 0
40637 ;cmp word [exec_SS],0 ; Q: is there a stack seg
40638 00006D76 7511 jne short ea1 ; Y: continue normal processing
40639 00006D78 391E[D70E] cmp [exec_SP],bx ; 0
40640 ;cmp word [exec_SP],0 ; Q: is there a stack ptr
40641 00006D7C 750B jne short ea1 ; Y: continue normal processing
40642
40643 ;inc byte [bp-29]
40644 00006D7E FE46E3 inc Exec_NoStack
40645 00006D81 3DF00F cmp ax,1000h-10h ; 0FF0h ; Q: is this >= 64K-256 bytes
40646 00006D84 7303 jae short ea1 ; Y: don't set Exec_SP
40647
40648 00006D86 83C010 add ax,10h ; add 10h paras to mem requirement
40649 ea1:
40650 ; M005 - END
40651
40652 ; MSDOS 6.0 ; M000 - start
40653 ; 20/05/2019
40654 ; (ds = ss = DOSDATA)
40655 00006D89 F606[0203]80 test byte [AllocMethod],HIGH_FIRST ; 80h
40656 ; Q: is the alloc strat high_first
40657 00006D8E 7405 jz short Exec_Norm_Alloc ; N: normal allocate
40658 ; Y: set high_only bit

```



```

40659 00006D90 800E[0203]40      or      byte [AllocMethod],HIGH_ONLY ; 40h
40660                                     ; M000 - end
40661
40662 00006D95 A3[8A00]      Exec_Norm_Alloc:
40663                                     mov     [SAVE_AX],ax      ; M000: save ax for possible 2nd
40664 Exec_Norm_Alloc1:      ; 02/06/2019
40665                                     ; MSDOS 3.3
40666                                     ;push    ax      ; M000
40667
40668 00006D98 B8FFFF      mov     BX,0FFFFh      ; see how much room in arena
40669 00006D9B 1E      push    DS
40670                                     ;invoke $Alloc      ; should have carry set and BX has max
40671 00006D9C E86F05      call    _$ALLOC
40672 00006D9F 1F      pop     DS
40673
40674                                     ; MSDOS 6.0
40675 00006DA0 A1[8A00]      mov     AX,[SAVE_AX]      ; M000
40676                                     ; MSDOS 3.3
40677                                     ;pop     ax      ; M000
40678
40679 00006DA3 83C010      add     AX,10h      ; room for header
40680 00006DA6 83FB11      cmp     BX,11h      ; enough room for a header
40681                                     ; MSDOS 6.0
40682 00006DA9 72A9      jnb     short Exec_Chk_Mem      ; M000
40683                                     ; MSDOS 3.3
40684                                     ;jnb     short Exec_No_Mem
40685
40686 00006DAB 39D8      cmp     AX,BX      ; is there enough for bare image?
40687                                     ; MSDOS 6.0
40688 00006DAD 77A5      ja      short Exec_Chk_Mem      ; M000
40689                                     ; MSDOS 3.3
40690                                     ;ja      short Exec_No_Mem
40691
40692                                     ;test    byte [bp-6],0FFh
40693 00006DAF F646FAFF      test    Exec_Load_High,-1      ; if load high, use max
40694 00006DB3 7518      jnz     short Exec_BX_Max      ; use max
40695
40696                                     ; 09/09/2018
40697
40698 00006DB5 0306[D10E]      add     AX,[exec_min_BSS]      ; go for min allocation;rms;NSS
40699                                     ; MSDOS 6.0
40700 00006DB9 7299      jc      short Exec_Chk_Mem      ; M000
40701                                     ; MSDOS 3.3
40702                                     ;jc      short Exec_No_Mem
40703
40704 00006DBB 39D8      cmp     AX,BX      ; enough space?
40705                                     ; MSDOS 6.0
40706 00006DBD 7795      ja      short Exec_Chk_Mem      ; M000: nope...
40707                                     ; MSDOS 3.3
40708                                     ;ja      short Exec_No_Mem
40709
40710 00006DBF 2B06[D10E]      sub     AX,[exec_min_BSS]      ; rms;NSS
40711 00006DC3 0306[D30E]      add     AX,[exec_max_BSS]      ; go for the MAX
40712 00006DC7 7204      jc      short Exec_BX_Max
40713
40714 00006DC9 39D8      cmp     AX,BX
40715 00006DCB 7602      jbe     short Exec_Got_Block
40716
40717 Exec_BX_Max:
40718 00006DCD 89D8      mov     AX,BX
40719
40720 Exec_Got_Block:
40721                                     ; 03/08/2018 - Retro DOS v3.0
40722
40723 00006DCF 1E      push    DS
40724 00006DD0 89C3      mov     BX,AX
40725                                     ;mov     [bp-16],bx
40726 00006DD2 895EF0      mov     Exec_Size,BX
40727                                     ;invoke $Alloc      ; get the space
40728 00006DD5 E83605      call    _$ALLOC
40729 00006DD8 1F      pop     DS
40730                                     ; MSDOS 6.0
40731                                     ;jc      short Exec_Chk_Mem      ; M000
40732                                     ; MSDOS 3.3
40733                                     ;jc      short Exec_No_Mem
40734                                     ; 20/05/2019
40735 00006DD9 7303      jnc     short ea0
40736 00006ddb E976FF      jmp     Exec_Chk_Mem
40737
40738 ea0:
40739 00006DDE 8A0E[8400]      ; MSDOS 6.0
40740 00006DE2 880E[0203]      mov     cl,[ALLOCMSAVE]      ; M063:
40741                                     mov     [AllocMethod],cl      ; M063: restore allocmethod
40742
40743 ;M029; Begin changes
40744 ; This code does special handling for programs with no stack segment. If so,
40745 ;check if the current block is larger than 64K. If so, we do not modify
40746 ;Exec_SP. If smaller than 64K, we make Exec_SP = top of block. In either
40747 ;case Exec_SS is not changed.
40748
40749                                     ; MSDOS 6.0
40750 00006DE6 807EE300      ;cmp     byte [bp-29],0
40751                                     cmp     Exec_NoStack,0
40752                                     ;je      @f
40753                                     je      short ea2
40754 00006DEC 81FB0010      cmp     bx,1000h      ; Q: >= 64K memory block
40755                                     ;jae     @f      ; Y: Exec_SP = 0
40756 00006DF0 730C      jae     short ea2
40757
40758 ;Make Exec_SP point at the top of the memory block
40759
40760 00006DF2 B104      mov     cl,4
40761 00006DF4 D3E3      shl     bx,cl      ; get byte offset
40762 00006DF6 81EB0001      sub     bx,100h      ; take care of PSP
40763 00006DFA 891E[D70E]      mov     [exec_SP],bx      ; Exec_SP = top of block
40764
40765 ea2:
40766 ;@@:
40767 ;M029; end changes
40768
40769 00006DFE 8946EE      ;mov     [bp-18],ax
40770 00006E01 83C010      mov     Exec_Load_Block,AX
40771                                     add     AX,10h
40772 00006E04 F646FAFF      ;test    byte [bp-6],0FFh
40773 00006E08 7409      test    Exec_Load_High,-1
40774                                     jz      short Exec_Use_AX      ; use ax for load info
40775
40776 00006E0A 0346F0      ;add     ax,[bp-16]
40777                                     add     AX,Exec_Size      ; go to end
40778                                     ;sub     ax,[bp-12]
40779 00006E0D 2B46F4      sub     AX,Exec_Res_Len_Para      ; drop off header
40780 00006E10 83E810      sub     AX,10h      ; drop off pdb
40781
40782 Exec_Use_AX:
40783                                     ;mov     [bp-10],ax

```

```

40783 00006E13 8946F6      mov     Exec_Rel_Fac,AX      ; new segment
40784                      ;mov     [bp-20],ax
40785 00006E16 8946EC      mov     Exec_DMA,AX ; *+*      ; beginning of dma
40786
40787                      ; Determine the location in the file of the beginning of
40788                      ; the resident
40789
40790                      ; 17/12/2022
40791                      ; 30/11/2022 (MSDOS 5.0, MSDOS.SYS compatibility)
40792                      ;%if 0
40793
40794                      Exec_Find_Res:
40795                      ; MSDOS 6.0
40796                      ;mov     dx,[bp-20]
40797                      ;mov     DX,Exec_DMA ; *+*
40798                      ;mov     [bp-28],dx
40799                      ;mov     Exec_DMA_Save,DX
40800
40801                      ; 17/12/2022
40802                      ; AX = Exec_DMA
40803
40804                      ; 02/06/2019 - Retro DOS v4.0
40805                      ;mov     [bp-28],ax ; *+*
40806 00006E19 8946E4      mov     Exec_DMA_Save,AX ; *+*
40807
40808                      ;%endif
40809
40810                      ; 17/12/2022
40811                      %if 0
40812                      ; 30/11/2022 (MSDOS 5.0, MSDOS.SYS compatibility)
40813                      Exec_Find_Res:
40814                      ;mov     dx,[bp-20]
40815                      mov     DX,Exec_DMA ; *+*
40816                      ;mov     [bp-28],dx
40817                      mov     Exec_DMA_Save,DX
40818                      ;%endif
40819
40820                      ; MSDOS 3.3 (& MSDOS 6.0)
40821 00006E1C 8B16[CF0E]      mov     DX,[exec_par_dir]
40822 00006E20 52                      push    DX
40823 00006E21 B104      mov     CL,4
40824 00006E23 D3E2      shl     DX,CL      ; low word of location
40825 00006E25 58                      pop     AX
40826 00006E26 B10C      mov     CL,12
40827 00006E28 D3E8      shr     AX,CL      ; high word of location
40828 00006E2A 89C1      mov     CX,AX      ; CX <- high
40829
40830                      ; Read in the resident image (first, seek to it)
40831                      ;mov     bx,[bp-8]
40832 00006E2C 8B5EF8      mov     BX,Exec_FH
40833 00006E2F 1E                      push    DS
40834 00006E30 30C0      xor     AL,AL
40835                      ;invoke $Lseek      ; Seek to resident
40836 00006E32 E8A10A      call    _LSEEK
40837 00006E35 1F                      pop     DS
40838 00006E36 7303      jnc     short Exec_Big_Read
40839
40840 00006E38 E906FF      jmp     Exec_Bomb
40841
40842                      Exec_Big_Read:      ; Read resident into memory
40843                      ;mov     bx,[bp-12]
40844 00006E3B 8B5EF4      mov     BX,Exec_Res_Len_Para
40845 00006E3E 81FB0010      cmp     BX,1000h      ; Too many bytes to read?
40846 00006E42 7203      jb     short Exec_Read_OK
40847
40848 00006E44 BBE00F      mov     BX,0FE0h      ; Max in one chunk FE00 bytes
40849
40850                      Exec_Read_OK:
40851                      ;sub     [bp-12],bx
40852 00006E47 295EF4      sub     Exec_Res_Len_Para,BX ; We read (soon) this many
40853 00006E4A 53                      push    BX
40854 00006E4B B104      mov     CL,4
40855 00006E4D D3E3      shl     BX,CL      ; Get count in bytes from paras
40856 00006E4F 89D9      mov     CX,BX      ; Count in correct register
40857 00006E51 1E                      push    DS
40858                      ;mov     ds,[bp-20]
40859 00006E52 8E5EEC      mov     DS,Exec_DMA      ; Set up read buffer
40860
40861 00006E55 31D2      xor     DX,DX
40862 00006E57 51                      push    CX
40863 00006E58 E81403      call    ExecRead      ; Save our count
40864 00006E5B 59                      pop     CX      ; Get old count to verify
40865 00006E5C 1F                      pop     DS
40866 00006E5D 7248      jc     short Exec_Bad_FileJ
40867
40868 00006E5F 39C1      cmp     CX,AX      ; Did we read enough?
40869 00006E61 5B                      pop     BX      ; Get paragraph count back
40870 00006E62 7408      jz     short ExecCheckEnd ; and do reloc if no more to read
40871
40872                      ; The read did not match the request. If we are off by 512
40873                      ; bytes or more then the header lied and we have an error.
40874
40875 00006E64 29C1      sub     CX,AX
40876 00006E66 81F90002      cmp     CX,512
40877 00006E6A 733B      jae     short Exec_Bad_FileJ
40878
40879                      ; we've read in CX bytes... bump DTA location
40880
40881                      ExecCheckEnd:
40882                      ;add     [bp-20],bx
40883 00006E6C 015EEC      add     Exec_DMA,BX      ; Bump dma address
40884                      ;test    word [bp-12],0FFFFh
40885 00006E6F F746F4FFFF      test    Exec_Res_Len_Para,-1
40886 00006E74 75C5      jnz     short Exec_Big_Read
40887
40888                      ; The image has now been read in. We must perform relocation
40889                      ; to the current location.
40890
40891                      exec_do_reloc:
40892                      ;mov     cx,[bp-10]
40893 00006E76 8B4EF6      mov     CX,Exec_Rel_Fac
40894 00006E79 A1[D50E]      mov     AX,[exec_SS]      ; get initial SS ;rms;NSS
40895 00006E7C 01C8      add     AX,CX      ; and relocate him
40896 00006E7E A3[C10E]      mov     [exec_init_SS],AX ; rms;NSS
40897
40898 00006E81 A1[D70E]      mov     AX,[exec_SP]      ; initial SP ;rms;NSS
40899 00006E84 A3[BF0E]      mov     [exec_init_SP],AX ; rms;NSS
40900
40901 00006E87 C406[DB0E]      les     AX,[exec_IP]      ; rms;NSS
40902 00006E8B A3[C30E]      mov     [exec_init_IP],AX ; rms;NSS
40903 00006E8E 8CC0      mov     AX,ES      ; rms;NSS
40904 00006E90 01C8      add     AX,CX      ; relocated...
40905 00006E92 A3[C50E]      mov     [exec_init_CS],AX ; rms;NSS
40906

```

```

40907 00006E95 31C9          xor     CX,CX
40908 00006E97 8B16[DF0E]        mov     DX,[exec_rle_table] ; rms;NSS
40909                      ;mov     bx,[bp-8]
40910 00006E9B 8B5EF8          mov     BX,Exec_FH
40911 00006E9E 1E              push    DS
40912 00006E9F 31C0          xor     AX,AX
40913                      ;invoke $Lseek
40914 00006EA1 E8320A        call    _$LSEEK
40915 00006EA4 1F              pop     DS
40916 00006EA5 7303          jnc     short exec_get_entries
40917
40918 Exec_Bad_Filej:
40919 00006EA7 E995FE        jmp     Exec_Bad_File
40920
40921 exec_get_entries:
40922 00006EAA 8B16[CD0E]        mov     DX,[exec_rle_count] ; Number of entries left ;rms;NSS
40923
40924 exec_read_reloc:
40925 00006EAE 52          push    DX
40926                      ;mov     dx,OPENBUF
40927 00006EAF BA[BE03]        mov     DX,Exec_Internal_Buffer
40928                      ;;mov     cx,388 ; MSDOS 3.3 ; (390>>2)<<2
40929                      ;mov     cx,396 ; MSDOS 6.0 ; PCDOS 7.1 (08/03/2024)
40930 00006EB2 B98C01        mov     CX,((Exec_Internal_Buffer_Size)/4)*4 ; (397>>2)<<2
40931 00006EB5 1E              push    DS
40932 00006EB6 E8B602        call    ExecRead
40933 00006EB9 07              pop     ES
40934 00006EBA 5A              pop     DX
40935 00006EBB 72EA          jc      short Exec_Bad_Filej
40936
40937                      ;;mov     cx,97 ; MSDOS 3.3 ; (390>>2)
40938                      ;mov     cx,99 ; MSDOS 6.0 ; PCDOS 7.1 (08/03/2024)
40939 00006EBD B96300        mov     CX,(Exec_Internal_Buffer_Size)/4 ; (397>>2)
40940                      ; Pointer to byte location in header
40941                      ;mov     di,OPENBUF
40942 00006EC0 BF[BE03]        mov     DI,Exec_Internal_Buffer
40943                      ;mov     si,[bp-10]
40944 00006EC3 8B76F6        mov     SI,Exec_Rel_Fac ; Relocate a single address
40945
40946 exec_reloc_one:
40947 00006EC6 09D2          or      DX,DX ; Any more entries?
40948 00006EC8 7416          jz      short Exec_Set_PDBj
40949
40950 exec_get_addr:
40951 00006ECA 26C51D        lds     BX,[ES:DI] ; Get ra/sa of entry
40952 00006ECD 8CD8          mov     AX,DS ; Relocate address of item
40953
40954                      ; MSDOS 6.0
40955                      ;;; add     AX,SI ; MSDOS 3.3
40956                      ;add     ax,[bp-28]
40957 00006ECF 0346E4        add     AX,Exec_DMA_Save
40958
40959 00006ED2 8ED8          mov     DS,AX
40960 00006ED4 0137          add     [BX],SI
40961 00006ED6 83C704        add     DI,4
40962 00006ED9 4A              dec     DX
40963 00006EDA E2EA          loop   exec_reloc_one ; End of internal buffer?
40964
40965                      ; we've exhausted a single buffer's worth. Read in the next
40966                      ; piece of the relocation table.
40967
40968 00006EDC 06          push    ES
40969 00006EDD 1F              pop     DS
40970 00006EDE EBCE          jmp     short exec_read_reloc
40971
40972 Exec_Set_PDBj:
40973                      ; MSDOS 6.0
40974
40975                      ; we now determine if this is a buggy exe packed file and if
40976                      ; so we patch in the right code. Note that fixexepatch will
40977                      ; point to a ret if dos loads low. The load segment as
40978                      ; determined above will be in exec_dma_save
40979
40980 00006EE0 06          push    es
40981 00006EE1 50          push    ax ; M030
40982 00006EE2 51          push    cx ; M030
40983                      ;mov     es,[bp-28]
40984 00006EE3 8E46E4        mov     es,Exec_DMA_Save
40985 00006EE6 36A1[C50E]    mov     ax,[ss:exec_init_CS] ; M030
40986 00006EEA 368B0E[C30E]  mov     cx,[ss:exec_init_IP] ; M030
40987 00006EEF 36FF16[670D]  call    word [ss:FixExePatch]
40988
40989                      ; 30/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
40990                      ; (MSDOS 5.0 MSDOS.SYS does not contain 'Rational386Patch')
40991                      ;call    word [ss:Rational386PatchPtr]
40992
40993                      ; 08/03/2024 - Retro DOS v5.0
40994                      ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:9E6Fh)
40995                      ; (PCDOS 7.1 IBMDOS.COM - DOSCODE:0B096h)
40996 00006EF4 36FF16[3E13]  call    word [ss:Rational386PatchPtr]
40997
40998                      ; 08/03/2024
40999                      ; (Windows ME IO.SYS - BIOSCODE:976Fh)
41000                      ;call    Rational386Patch
41001
41002 00006EF9 59          pop     cx ; M030
41003 00006EFA 58          pop     ax ; M030
41004 00006EFB 07          pop     es
41005
41006 00006EFC E9DD00        jmp     Exec_Set_PDB
41007
41008 Exec_No_Memj:
41009 00006EFF E939FE        jmp     Exec_No_Mem
41010
41011                      ; we have a .COM file. First, determine if we are merely
41012                      ; loading an overlay.
41013
41014 Exec_Com_File:
41015                      ;test     byte [bp-5],2
41016 00006F02 F646FB02    test     Exec_Func,exec_func_overlay
41017 00006F06 742D          jz      short Exec_Alloc_Com_File
41018                      ;lds     si,[bp-4]
41019 00006F08 C576FC        lds     SI,Exec_Blk ; get arg block
41020 00006F0B AD          lodsw ; get load address
41021                      ;mov     [bp-20],ax
41022 00006F0C 8946EC        mov     Exec_DMA,AX
41023 00006F0F B8FFFF        mov     AX,0FFFFh
41024 00006F12 EB63          jmp     short Exec_Read_Block ; read it all!
41025
41026 Exec_Chk_Com_Mem:
41027                      ; MSDOS 6.0
41028 00006F14 36A0[0203]    mov     al,[ss:AllocMethod] ; M063 - start
41029 00006F18 368A1E[8400]  mov     bl,[ss:ALLOCMSAVE] ; save current alloc method in ax
41030 00006F1D 36881E[0203]  mov     [ss:AllocMethod],bl ; restore original allocmethod

```

```

41031 00006F22 F6C340      test    b1,HIGH_ONLY ; 40h      ; Q: was the HIGH_ONLY bit already set
41032 00006F25 75D8        jnz     short Exec_No_Memj      ; Y: no space in UMBS. Quit
41033                                     ; N: continue
41034
41035 00006F27 A840        test    al,HIGH_ONLY      ; Q: did we set the HIGH_ONLY bit
41036 00006F29 74D4        jz      short Exec_No_Memj      ; N: no memory
41037
41038      ;mov     ax,[bp-18]
41039      mov     ax,Exec_Load_Block ; M047: ax = block we just allocated
41040 00006F2E 31DB        xor     bx,bx              ; M047: bx => free arena
41041 00006F30 E87102      call    ChangeOwner        ; M047: free this block
41042
41043 00006F33 EB0E        jmp     short Exec_Norm_Com_Alloc
41044                                     ; M063 - End
41045
41046      ; we must allocate the max possible size block (ick!)
41047      ; and set up CS=DS=ES=SS=PDB pointer, IP=100, SP=max
41048      ; size of block.
41049
41050 Exec_Alloc_Com_File:
41051      ; MSDOS 6.0
41052 00006F35 36F606[0203]80 test    byte [ss:AllocMethod],HIGH_FIRST ; 80h
41053                                     ; Q: is the alloc strat high_first
41054 00006F3B 7406        jz      short Exec_Norm_Com_Alloc ; N: normal allocate
41055                                     ; Y: set high_only bit
41056 00006F3D 36800E[0203]40 or      byte [ss:AllocMethod],HIGH_ONLY ; 40h
41057                                     ; M000 - end
41058 Exec_Norm_Com_Alloc:
41059      ; MSDOS 3.3 (& MSDOS 6.0)
41060 00006F43 BBFFFF      mov     BX,0FFFFh
41061      ;invoke $Alloc
41062 00006F46 E8C503      call    _$ALLOC
41063 00006F49 09DB      or      BX,BX
41064      ; MSDOS 6.0
41065 00006F4B 74C7        jz      short Exec_Chk_Com_Mem; M000
41066      ; MSDOS 3.3
41067      ;jz      short Exec_No_Memj
41068
41069      ;mov     [bp-16],bx
41070 00006F4D 895EF0      mov     Exec_Size,BX
41071 00006F50 53          push    BX
41072      ;invoke $ALLOC
41073 00006F51 E8BA03      call    _$ALLOC
41074 00006F54 5B          pop     BX
41075      ;mov     [bp-18],ax
41076 00006F55 8946EE      mov     Exec_Load_Block,AX
41077
41078 00006F58 83C010      add     AX,10h
41079      ;mov     [bp-20],ax
41080 00006F5B 8946EC      mov     Exec_DMA,AX
41081
41082 00006F5E 31C0        xor     AX,AX
41083 00006F60 81FB0010      cmp     BX,1000h
41084 00006F64 730E        jae     short Exec_Read_Com
41085                                     ; presume 64k read...
41086                                     ; 64k or more in block?
41087                                     ; yes, read only 64k
41088
41086 00006F66 89D8        mov     AX,BX
41087 00006F68 B104        mov     CL,4
41088 00006F6A D3E0        shl     AX,CL
41089      ; 17/12/2022
41090      ; 30/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
41091      ; (MSDOS 5.0, MSDOS.SYS compatibility)
41092      ; MSDOS 5.0
41093      ;cmp     AX,100h
41094      ; 02/06/2019 - Retro DOS v4.0
41095      ; MSDOS 6.0
41096      ; 17/12/2022
41097 00006F6C 3D0002      cmp     AX,200h
41098 00006F6F 76A3        jbe     short Exec_Chk_Com_Mem; M000: jump if not
41099      ;jbe     short Exec_No_Memj ; M000: jump if not
41100      ; Retro DOS v3.0 modification (on MSDOS 6.0 code) -03/08/2018-
41101      ;jbe     short Exec_Chk_Com_Mem; M000: jump if not
41102      ;jbe     short Exec_No_Memj ; M000: jump if not
41103
41104      ; M047: size of the block is < 64K
41105 00006F71 2D0001      sub     ax,100h
41106      ; M047: reserve 256 bytes for stack
41107
41108 Exec_Read_Com:
41109      ; MSDOS 3.3 (& MSDOS 6.0)
41109 00006F74 2D0001      sub     AX,100h
41110      ; remember size of psp
41110 Exec_Read_Block:
41111      ; save number to read
41112      ;mov     bx,[bp-8]
41113 00006F78 8B5EF8      mov     BX,Exec_FH
41114 00006F7B 31C9        xor     CX,CX
41115 00006F7D 31C0        xor     AX,AX
41116      ;mov     DX,CX
41117      ; 08/03/2024
41118 00006F7F 99          cwd
41119      ;invoke $Lseek
41120 00006F80 E85309      call    _$LSEEK
41121 00006F83 59          pop     CX
41122      ; number to read
41123 00006F84 8E5EEC      mov     ds,[bp-20]
41124 00006F87 31D2      mov     DS,Exec_DMA
41125 00006F89 51          xor     DX,DX
41126 00006F8A E8E201      push    CX
41127 00006F8D 5E          call    ExecRead
41128 00006F8E 7303        pop     SI
41129 00006F90 E9ACFD      jnc     short OkRead
41130      jmp     Exec_Bad_File
41131      ; 10/09/2018
41132 OkRead:
41133 00006F93 39F0        cmp     AX,SI
41134      ; did we read them all?
41135      ; MSDOS 6.0
41136      ;jz      short Exec_Chk_Com_Mem; M00: exactly the wrong number...no
41137      ; MSDOS 3.3
41138      ;jz      short Exec_No_Memj ; M00: exactly the wrong number...
41139 00006F95 7503        jne     short OkRead2
41140 00006F97 E97AFF      jmp     Exec_Chk_Com_Mem
41141
41142 OkRead2:
41143      ; MSDOS 6.0
41143 00006F9A 368A1E[8400]      mov     b1,[ss:ALLOCMSAVE]
41144 00006F9F 36881E[0203]      mov     [ss:AllocMethod],b1
41145      ; M063
41146      ; M063: restore alloc method
41147      ; MSDOS 3.3 (& MSDOS 6.0)
41148      ;test    byte [bp-5],2
41148 00006FA4 F646FB02      test    Exec_Func,exec_func_overlay
41149 00006FA8 7532        jnz     short Exec_Set_PDB
41150      ; no starto, chumo!
41151
41152      ;mov     ax,[bp-20]
41152 00006FAA 8B46EC      mov     AX,Exec_DMA
41153 00006FAD 83E810      sub     AX,10h
41154 00006FB0 36A3[C50E]      mov     [SS:exec_init_CS],AX

```

```

41155 00006FB4 36C706[C30E]0001      mov     word [SS:exec_init_IP],100h ; initial IP is 100h
41156
41157                                ; SI is AT MOST FF00h. Add FE to account for PSP - word
41158                                ; of 0 on stack.
41159
41160 00006FBB 81C6FE00      add     SI,0FEh                ; make room for stack
41161
41162                                ; MSDOS 6.0
41163 00006FBF 83FEFE      cmp     si,0FFFEh            ; M047: Q: was there >= 64K available
41164 00006FC2 7404      je      short Exec_St_Ok    ; M047: Y: stack is fine
41165 00006FC4 81C60001    add     si,100h             ; M047: N: add the xtra 100h for stack
41166
41167 Exec_St_Ok:
41168                                ; MSDOS 3.3 (& MSDOS 6.0)
41169 00006FC8 368936[BFOE]    mov     [SS:exec_init_SP],SI ; max value for read is also SP!;smr;SS Override
41170 00006FCD 36A3[C10E]    mov     [SS:exec_init_SS],AX ;smr;SS Override
41171 00006FD1 8ED8      mov     DS,AX
41172 00006FD3 C7040000    mov     WORD [SI],0         ; 0 for return
41173
41174                                ; MSDOS 6.0
41175
41176                                ; M068
41177                                ;
41178                                ; We now determine if this is a Copy Protected App. If so the
41179                                ; A200FF_COUNT is set to 6. Note that ChkCopyProt will point to
41180                                ; a ret if DOS is loaded low. Also DS contains the load segment.
41181
41182 00006FD7 36FF16[6100]    call    word [ss:ChkCopyProt]
41183
41184 Exec_Set_PDB:
41185                                ; MSDOS 3.3 (& MSDOS 6.0)
41186                                ;mov     bx,[bp-8]
41187 00006FDC 8B5EF8      mov     BX,Exec_FH          ; we are finished with the file.
41188 00006FDF E8A601      call    Exec_Dealloc
41189 00006FE2 55      push    BP
41190                                ;invoke $Close
41191 00006FE3 E88707      call    _$CLOSE            ; release the jfn
41192 00006FE6 5D      pop     BP
41193 00006FE7 E89001      call    Exec_Alloc
41194                                ;test    byte [bp-5],2
41195 00006FEA F646FB02    test    Exec_Func,exec_func_overlay
41196 00006FEE 743A      jz      short Exec_Build_Header
41197
41198                                ; MSDOS 6.0
41199 00006FF0 E8BF01      call    Scan_Execname
41200 00006FF3 E8D301      call    Scan_Special_Entries
41201
41202 ;SR;
41203 ;The current lie strategy uses the PSP to store the lie version. However,
41204 ;device drivers are loaded as overlays and have no PSP. To handle them, we
41205 ;use the Sysinit flag provided by the BIOS as part of a structure pointed at
41206 ;by BiosDataPtr. If this flag is set, the overlay call has been issued from
41207 ;Sysinit and therefore must be a device driver load. We then get the lie
41208 ;version for this driver and put it into the Sysinit PSP. When the driver
41209 ;issues the version check, it gets the lie version until the next overlay
41210 ;call is issued.
41211 00006FF6 36803E[2213]00    cmp     byte [ss:DriverLoad],0;was Sysinit processing done?
41212 00006FFC 7426      je      short norm_ovl     ;yes, no special handling
41213 00006FFE 56      push    si
41214 00006FFF 06      push    es
41215 00007000 36C436[2313]    les     si,[ss:BiosDataPtr] ;get ptr to BIOS data block
41216
41217                                ; (es:si points to 'SysinitPresent' address/flag in retrodos4.s)
41218 00007005 26803C00    cmp     byte [es:si],0     ;in Sysinit?
41219 00007009 7411      je      short sysinit_done ;no, Sysinit is finished
41220
41221 0000700B 368E06[3003]    mov     es,[ss:CurrentPDB] ;es = current PSP (Sysinit PSP)
41222 00007010 36FF36[BB0E]    push    word [ss:SPECIAL_VERSION]
41223                                ;pop     word [es:40h] ; 08/03/2024
41224 00007015 268F064000    pop     word [es:PDB.Version] ;store lie version in Sysinit PSP
41225                                ;;; PDB.VERSION
41226 0000701A EB06      jmp     short setver_done
41227 sysinit_done:
41228 0000701C 36C606[2213]00    mov     byte [ss:DriverLoad],0;Sysinit done,special handling off
41229 setver_done:
41230                                pop     es
41231 00007023 5E      pop     si
41232 norm_ovl:
41233                                ;leave
41234 00007024 89EC      mov     sp,bp
41235 00007026 5D      pop     bp
41236
41237                                ;transfer SYS_RET_OK
41238 00007027 E94496      jmp     SYS_RET_OK         ; overlay load -> done
41239
41240 Exec_Build_Header:
41241                                ;mov     dx,[bp-18]
41242 0000702A 8B56EE      mov     DX,Exec_Load_Block
41243                                ; assign the space to the process
41244                                ;mov     si,1
41245 0000702D BE0100    mov     SI,ARENA.OWNER     ; pointer to owner field
41246                                ;mov     ax,[bp-14]
41247 00007030 8B46F2      mov     AX,Exec_Environ    ; get environ pointer
41248 00007033 09C0      or      AX,AX
41249 00007035 7405      jz      short No_Owner     ; no environment
41250
41251 00007037 48      dec     AX                  ; point to header
41252 00007038 8ED8      mov     DS,AX
41253 0000703A 8914      mov     [SI],DX            ; assign ownership
41254 No_Owner:
41255                                ;mov     ax,[bp-18]
41256 0000703C 89D0      mov     AX,Exec_Load_Block ; get load block pointer
41257                                ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
41258                                ; 17/12/2022
41259 0000703C 89D0      mov     ax,dx ; 06/06/2019
41260                                ;mov     ax,Exec_Load_Block ; get load block pointer
41261
41262 0000703E 48      dec     AX                  ; point to header
41263 0000703F 8ED8      mov     DS,AX
41264 00007041 8914      mov     [SI],DX            ; assign ownership
41265
41266                                ; MSDOS 6.0
41267 00007043 1E      push    DS                 ;AN000;MS. make ES=DS
41268 00007044 07      pop     ES                 ;AN000;MS.
41269                                ;mov     di,8
41270 00007045 BF0800    mov     DI,ARENA.NAME      ;AN000;MS. ES:DI points to destination
41271 00007048 E86701      call    Scan_Execname      ;AN007;MS. parse execname
41272                                ; ds:si->name, cx=name length
41273 0000704B 51      push    CX                 ;AN007;MS. save for fake version
41274 0000704C 56      push    SI                 ;AN007;MS. save for fake version
41275
41276 MoveName:
41277 0000704D AC      lodsb
41278 0000704E 3C2E      cmp     AL,'.'

```

```

41279 00007050 7408      jz      short Mem_Done      ;AN000;;MS. no, move to header
41280                                ;AN000;
41281 00007052 AA        stosb                                ;AN000;;MS. move char
41282                                ; MSKK bug fix - limit length copied
41283 00007053 83FF10     cmp     di,16 ; ARENAHEADERSIZE      ; end of memory arena block?
41284 00007056 7302      jae     short Mem_Done      ; jump if so
41285                                ;
41286 00007058 E2F3      ;loop MoveName      ;AN000;;MS. continue
41287 Mem_Done:          ;AN000;
41288 0000705A 30C0      xor     AL,AL      ;AN000;;MS. make ASCIIIZ
41289                                ;
41290 0000705C 83FF10     cmp     di,16
41291 0000705F 7301      cmp     DI,ARENAHEADERSIZE ; 16 ;AN000;MS. if not all filled
41292                                ;AN000;MS.
41293 00007061 AA        stosb                                ;AN000;MS.
41294
41295 Fill8:              ;AN000;
41296 00007062 5E        pop     SI      ;AN007;MS. ds:si -> file name
41297 00007063 59        pop     CX      ;AN007;MS.
41298
41299 00007064 E86201     call    Scan_Special_Entries ;AN007;MS.
41300
41301 ; MSDOS 3.3 (& MSDOS 6.0)
41302 00007067 52        push    DX
41303                                ;mov si,[bp-16]
41304 00007068 8B76F0     mov     SI,Exec_Size
41305 0000706B 01D6     add     SI,DX
41306                                ;Invoke $Dup_PDB      ; ES is now PDB
41307 0000706D E8CEA5     call    _$DUP_PDB
41308 00007070 5A        pop     DX
41309
41310                                ;push word [bp-14]
41311 00007071 FF76F2     push    Exec_Environ
41312                                ;pop word [ES:2Ch]
41313 00007074 268F062C00 pop     word [ES:PDB.ENVIRON]
41314
41315 ; MSDOS 6.0      ; *** Added for DOS 5.00
41316                                ; version number in PSP
41317 00007079 36FF36[B0E] push    word [ss:SPECIAL_VERSION] ; Set the DOS version number to
41318 0000707E 268F064000 pop     word [ES:PDB.Version] ; to be used for this application
41319                                ; PDB.VERSION
41320
41321 ; MSDOS 3.3 (& MSDOS 6.0) ; set up proper command line stuff
41322 ;lds si,[bp-4]
41323 00007083 C576FC     lds     SI,Exec_Blk      ; get the block
41324 00007086 1E        push    DS      ; save its location
41325 00007087 56        push    SI
41326                                ;lds si,[si+6]
41327 00007088 C57406     lds     SI,[SI+EXEC0.5C_FCB] ; get the 5c fcb
41328
41329 ; DS points to user space 5C FCB
41330
41331 0000708B B90C00     mov     CX,12      ; copy drive, name and ext
41332 0000708E 51        push    CX
41333 0000708F BF5C00     mov     DI,5Ch
41334 00007092 8A1C     mov     BL,[SI]
41335 00007094 F3A4     rep     movsb
41336
41337 ; DI = 5Ch + 12 = 5Ch + 0Ch = 68h
41338
41339 ;xor AX,AX      ; zero extent, etc for CPM
41340 00007096 91        xchg    ax,cx      ; 08/03/2024
41341 00007097 AB        stosw
41342 00007098 AB        stosw
41343
41344 ; DI = 5Ch + 12 + 4 = 5Ch + 10h = 6Ch
41345
41346 00007099 59        pop     CX
41347 0000709A 5E        pop     SI      ; get block
41348 0000709B 1F        pop     DS
41349 0000709C 1E        push    DS      ; save (again)
41350 0000709D 56        push    SI
41351                                ;lds si,[si+0Ah]
41352 0000709E C5740A     lds     SI,[SI+EXEC0.6C_FCB] ; get 6C FCB
41353
41354 ; DS points to user space 6C FCB
41355
41356 000070A1 8A3C     mov     BH,[SI]      ; do same as above
41357 000070A3 F3A4     rep     movsb
41358 000070A5 AB        stosw
41359 000070A6 AB        stosw
41360 000070A7 5E        pop     SI      ; get block (last time)
41361 000070A8 1F        pop     DS
41362                                ;ld si,[si+2]
41363 000070A9 C57402     lds     SI,[SI+EXEC0.COM_LINE]; command line
41364
41365 ; DS points to user space 80 command line
41366
41367 000070AC 80C980     or      CL,80h
41368 000070AF 89CF     mov     DI,CX
41369 000070B1 F3A4     rep     movsb      ; Wham!
41370
41371 ; Process BX into default AX (validity of drive specs on args).
41372 ; We no longer care about DS:SI.
41373
41374 000070B3 FEC9     dec     CL      ; get 0FFh in CL
41375 000070B5 88F8     mov     AL,BH
41376 000070B7 30FF     xor     BH,BH
41377                                ;invoke GetVisDrv
41378 000070B9 E8B50A     call    GetVisDrv
41379 000070BC 7302      jnc     short Exec_BL
41380
41381 000070BE 88CF     mov     BH,CL
41382
41383 Exec_BL:
41384 000070C0 88D8     mov     AL,BL
41385 000070C2 30DB     xor     BL,BL
41386                                ;invoke GetVisDrv
41387 000070C4 E8AA0A     call    GetVisDrv
41388 000070C7 7302      jnc     short Exec_Set_Return
41389
41390 000070C9 88CB     mov     BL,CL
41391
41392 Exec_Set_Return:
41393                                ;invoke Get_User_Stack      ; get his return address
41394 000070CB E8A993     call    Get_User_Stack
41395
41396 ; 08/03/2024
41397 %if 0
41398                                ;push word [si+14h]
41399                                ;push word [SI+user_env.user_CS] ; suck out the CS and IP
41400                                ;push word [si+12h]
41401                                ;push word [SI+user_env.user_IP]
41402                                ;push word [si+14h]

```



```

41403      push    word [SI+user_env.user_CS]    ; suck out the CS and IP
41404      ;push   word [si+12h]
41405      push    word [SI+user_env.user_IP]
41406      ;pop     word [ES:0Ah]
41407      pop     word [ES:PDB.EXIT]
41408      ;pop     word [ES:0Ch]
41409      pop     word [ES:PDB.EXIT+2]
41410      %else
41411      ; 07/03/2024 (PCDOS 7.1 IBMDOS.COM)
41412      ;;;
41413      ;lds     ax,[si+12h]
41414      lds     ax,[SI+user_env.user_IP] ; suck out the CS and IP
41415      000070CE C54412      push    ds
41416      000070D1 1E          push    ax
41417      000070D2 50          ;mov     [es:0Ah],ax
41418      000070D3 26A30A00    mov     [ES:PDB.EXIT],ax
41419      ;mov     [es:0Ch],ds
41420      000070D7 268C1E0C00  mov     [ES:PDB.EXIT+2],ds
41421      ;;;
41422      %endif
41423
41424      xor     AX,AX
41425      mov     DS,AX
41426      ; save them where we can get them
41427      ; later when the child exits.
41428
41429      ;pop     word [88h]
41430      pop     word [addr_int_terminate] ; 22h*4
41431      000070E4 8F068A00    pop     word [90h]
41432      pop     word [addr_int_terminate+2] ; (22h*4)+2
41433      000070E8 36C706[2C03]8000 mov     WORD [SS:DMAADD],80h ; SS Override
41434      000070EF 368E1E[3003] mov     DS,[SS:CurrentPDB] ; SS Override
41435      000070F4 368C1E[2E03] mov     [SS:DMAADD+2],DS ; SS Override
41436
41437      ;test    byte [bp-5],1
41438      000070F9 F646FB01    test    Exec_Func,exec_func_no_execute
41439      000070FD 7427          jz      short exec_go
41440
41441      lds     SI,[SS:exec_init_SP] ; get stack SS Override
41442      ;les     di,[bp-4]
41443      les     DI,Exec_Blk ; and block for return
41444      ;mov     [es:di+10h],ds
41445      00007107 268C5D10    mov     [ES:DI+EXEC1.SS],DS ; return SS
41446
41447      0000710B 4E          dec     SI ; 'push' default AX
41448      0000710C 4E          dec     SI
41449      0000710D 891C          mov     [SI],BX ; save default AX reg
41450      ;mov     [es:di+0Eh], si
41451      0000710F 2689750E    mov     [ES:DI+EXEC1.SP],SI ; return 'SP'
41452
41453      lds     AX,[SS:exec_init_IP] ; SS Override
41454      ;mov     [es:di+14h],ds
41455      00007118 268C5D14    mov     [ES:DI+EXEC1.CS],DS ; initial entry stuff
41456      ;mov     [es:di+12h],ax
41457      0000711C 26894512    mov     [ES:DI+EXEC1.IP],AX
41458
41459      ;leave
41460      00007120 89EC          mov     sp,bp
41461      00007122 5D          pop     bp
41462
41463      ;transfer SYS_RET_OK
41464      00007123 E94895      jmp     SYS_RET_OK
41465
41466      exec_go:
41467      lds     SI,[SS:exec_init_IP] ; get entry point SS Override
41468      les     DI,[SS:exec_init_SP] ; new stack SS Override
41469      mov     AX,ES
41470
41471      ; MSDOS 6.0
41472      00007132 36803E[660D]00 cmp     byte [SS:DosHashMA],0 ; Q: is dos in HMA (M021)
41473      00007138 741A          je      short Xfer_To_User ; N: transfer control to user
41474
41475      0000713A 1E          push    ds ; Y: control must go to low mem stub
41476
41477      0000713B 2E8E1E[0700] mov     ds,[cs:DosDSeg] ; where we disable a20 and xfer
41478      ; control to user
41479      00007140 800E[8600]04 or      byte [DOS_FLAG],EXECA200FF ; M068:
41480      ; M004: Set bit to signal int 21
41481      ; ah = 25 & ah= 49. See dossym.inc
41482      ; under TAG M003 & M009 for
41483      ; explanation
41484      00007145 8916[6300] mov     [A200FF_PSP],dx ; M068: set the PSP for which A20 is
41485      ; M068: going to be turned OFF.
41486
41487      00007149 8CD8          mov     ax,ds ; ax = segment of low mem stub
41488      0000714B 1F          pop     ds
41489
41490      0000714C 50          push    ax ; ret far into the low mem stub
41491      0000714D B8[2410]      mov     ax,disa20_xfer
41492      00007150 50          push    ax
41493      00007151 8CC0          mov     AX,ES ; restore ax
41494      00007153 CB          retf
41495
41496      Xfer_To_User:
41497      ; DS:SI points to entry point
41498      ; AX:DI points to initial stack
41499      ; DX has PDB pointer
41500      ; BX has initial AX value
41501
41502      00007154 FA          cli
41503      ; 15/08/2018
41504      00007155 36C606[2103]00 mov     BYTE [SS:INDOS],0 ; SS Override
41505      ; 01/07/2024
41506      0000715B 36C606[B812]00 mov     byte [ss:INDOS_FLAG],0
41507
41508      00007161 8ED0          mov     SS,AX ; set up user's stack
41509      00007163 89FC          mov     SP,DI ; and SP
41510      00007165 FB          sti
41511
41512      00007166 1E          push    DS ; fake long call to entry
41513      00007167 56          push    SI
41514      00007168 8EC2          mov     ES,DX ; set up proper seg registers
41515      0000716A 8EDA          mov     DS,DX
41516      0000716C 89D8          mov     AX,BX ; set up proper AX
41517
41518      0000716E CB          retf
41519
41520      ; 04/08/2018 - Retro DOS v3.0
41521
41522      ;-----
41523      ;
41524      ;-----
41525
41526      ExecRead:

```

```

41527 0000716F E81600      CALL    Exec_Dealloc
41528                      ;mov    bx,[bp-8]
41529 00007172 8B5EF8      MOV     bx,Exec_FH
41530
41531 00007175 55          PUSH    BP
41532 00007176 E8FB06      call    _$READ
41533 00007179 5D          POP     BP
41534
41535                      ;CALL    Exec_Alloc
41536                      ;retn
41537                      ; 18/12/2022
41538                      ;jmp     short Exec_Alloc
41539
41540                      ; 18/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
41541
41542                      ;
41543                      ;
41544                      ;
41545                      ;
41546                      Exec_Alloc:
41547 0000717A 53          push    BX
41548                      ;mov    BX,[CS:CurrentPDB] ; MSDOS 3.3
41549                      ; 20/05/2019 - Retro DOS v4.0
41550                      ; MSDOS 6.0
41551 0000717B 368B1E[3003] mov     bx,[SS:CurrentPDB] ; SS Override
41552 00007180 E81000      call    ChangeOwners
41553 00007183 E889A7      call    LCritMEM
41554 00007186 5B          pop     BX
41555 00007187 C3          retn
41556
41557                      ;
41558                      ;
41559                      ;
41560                      ;
41561                      Exec_Dealloc:
41562 00007188 53          push    BX
41563                      ;mov    bx,0
41564 00007189 29DB      sub     BX,BX ; (bx) = ARENA_OWNER_SYSTEM
41565 0000718B E854A7      call    ECritMEM
41566 0000718E E80200      call    ChangeOwners
41567 00007191 5B          pop     BX
41568 00007192 C3          retn
41569
41570                      ; 18/12/2022
41571                      %if 0
41572                      ;
41573                      ;
41574                      ;
41575                      ;
41576                      Exec_Alloc:
41577                      push    BX
41578                      ;mov    BX,[CS:CurrentPDB] ; MSDOS 3.3
41579                      ; 20/05/2019 - Retro DOS v4.0
41580                      ; MSDOS 6.0
41581                      mov     bx,[SS:CurrentPDB] ; SS Override
41582                      call    ChangeOwners
41583                      call    LCritMEM
41584                      pop     BX
41585                      retn
41586
41587                      %endif
41588
41589                      ;
41590                      ;
41591                      ;
41592                      ;
41593                      ChangeOwners:
41594 00007193 9C          pushf
41595 00007194 50          push    AX
41596                      ;mov    ax,[bp-14]
41597 00007195 8B46F2      mov     AX,Exec_Environ
41598 00007198 E80900      call    ChangeOwner
41599                      ;mov    ax,[bp-18]
41600 0000719B 8B46EE      mov     AX,Exec_Load_Block
41601 0000719E E80300      call    ChangeOwner
41602 000071A1 58          pop     AX
41603 000071A2 9D          popf
41604                      chgown_retn:
41605 000071A3 C3          retn
41606
41607                      ;
41608                      ;
41609                      ;
41610                      ;
41611                      ChangeOwner:
41612 000071A4 09C0      or      AX,AX ; is area allocated?
41613 000071A6 74FB      jz      short chgown_retn ; no, do nothing
41614 000071A8 48          dec     AX
41615 000071A9 1E          push    DS
41616 000071AA 8ED8      mov     DS,AX
41617 000071AC 891E0100 mov     [ARENA.OWNER],BX
41618 000071B0 1F          pop     DS
41619 000071B1 C3          retn
41620
41621                      ;
41622                      ;
41623                      ;
41624                      ;
41625                      ; 20/05/2019 - Retro DOS v4.0
41626
41627                      ; MSDOS 6.0
41628                      Scan_Execname:
41629                      ;lds    si,[bp-26] ; 08/03/2024
41630 000071B2 C576E6      lds     SI,ExecName ; DS:SI points to name
41631                      Scan_Execname1:
41632                      ; M028
41633 000071B5 89F1      Save_Begin:
41634                      mov     CX,SI ; CX= starting addr
41635                      Scan0:
41636                      lodsb ; get char
41637 000071B8 3C3A      cmp     AL,':' ; is ':', may be A:name
41638 000071BA 74F9      jz      short Save_Begin ; yes, save si
41639 000071BC 3C5C      cmp     AL,'\' ; is '\', may be A:\name
41640 000071BE 74F5      jz      short Save_Begin ; yes, save si
41641 000071C0 3C00      cmp     AL,0 ; is end of name
41642 000071C2 75F3      jnz     short Scan0 ; no, continue scanning
41643 000071C4 29CE      sub     SI,CX ; get name's length
41644 000071C6 87F1      xchg    SI,CX ; cx= length, si= starting addr
41645
41646 000071C8 C3          retn
41647
41648                      ;
41649                      ;
41650                      ;

```



```

41651
41652 ; 20/05/2019 - Retro DOS v4.0
41653
41654 ; 01/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
41655 ; DOSCODE:A0EDh (MSDOS 5.0, MSDOS.SYS)
41656
41657 ; 09/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
41658 ; DOSCODE:B370h (PCDOS 7.1, IBMDOS.COM)
41659
41660 ; MSDOS 6.0
41661
41662 Scan_Special_Entries:
41663
41664 000071C9 49      dec     CX          ; cx= name length
41665 ;M060      mov     DI,[Special_Entries] ; es:di -> addr of special entries
41666 ;          ;reset to current version
41667 ;mov      word [ss:SPECIAL_VERSION],1406h
41668 ;          ; (MSDOS 6.21, MSDOS.SYS, DOSCODE:A14Eh)
41669 ;mov      word [ss:SPECIAL_VERSION],5
41670 ;          ; (MSDOS 5.0, MSDOS.SYS, DOSCODE:A0EEh)
41671
41672 ;          ; 5 for Retro DOS 4.0 (01/12/2022, MSDOS 5.0)
41673 000071CA 36C706[BB0E]070A mov     word [ss:SPECIAL_VERSION],(MINOR_VERSION<<8)+MAJOR_VERSION
41674 ;          ; 0005h for Retro DOS v4.1 (MSDOS 5.0)
41675 ;          ; 24/09/2023
41676 ;          ; 1606h for Retro DOS v4.2 (MSDOS 6.22)
41677
41678 ;          ; 09/03/2024
41679 ;          ; 0A07h for Retro DOS v5.0 (PCDOS 7.1)
41680 ;          ; (0008h for Windows ME)
41681
41682 ;***call   Reset_Version
41683
41684 ;M060      push    SS
41685 ;M060      pop     ES
41686
41687 000071D1 36C43E[5D00]      les     DI,[SS:UU_IFS_DOS_CALL] ;M060; ES:DI --> Table in SETVER.SYS
41688 000071D6 8CC0      mov     AX,ES ;M060; First do a NULL ptr check to
41689 000071D8 09F8      or      AX,DI ;M060; be sure the table exists
41690 000071DA 7427      jz      short End_List ;M060; if ZR then no table
41691
41692 GetEntries:
41693 000071DC 268A05      mov     AL,[ES:DI] ; end of list
41694 000071DF 08C0      or      AL,AL
41695 000071E1 7420      jz      short End_List ; yes
41696
41697 000071E3 36893E[0E06] mov     [ss:TEMP_VAR2],DI ; save di
41698 000071E8 38C8      cmp     AL,CL ; same length ?
41699 000071EA 751B      jnz     short SkipOne ; no
41700
41701 000071EC 47      inc     DI ; es:di -> special name
41702 000071ED 51      push    CX ; save length and name addr
41703 000071EE 56      push    SI
41704
41705 ; M050 - BEGIN
41706
41707 000071EF 50      push    ax ; save len
41708 sse_next_char:
41709 000071F0 AC      lodsb
41710 000071F1 E8A3EB      call    UCase
41711 000071F4 AE      scasb
41712 000071F5 750D      jne     short Not_Matched
41713 000071F7 E2F7      loop    sse_next_char
41714
41715 ; repz     cmpsb ; same name ?
41716 ; jnz      short Not_Matched ; no
41717
41718 ; 09/03/2024 - PCDOS 7.1 IBMDOS.COM
41719 ;pop      ax ; take len off the stack
41720
41721 ; M050 - END
41722
41723 000071F9 268B05      mov     AX,[ES:DI] ; get special version
41724 000071FC 36A3[BB0E]  mov     [ss:SPECIAL_VERSION],AX ; save it
41725
41726 ;***mov    AL,[ES:DI+2] ; get fake count
41727 ;***mov    [ss:FAKE_COUNT],AL ; save it
41728
41729 ; 09/03/2024 - PCDOS 7.1 IBMDOS.COM
41730 00007200 58      pop     ax ; take len off the stack
41731
41732 00007201 5E      pop     SI
41733 00007202 59      pop     CX
41734 ; 18/12/2022
41735 ;jmp      SHORT End_List
41736
41737 ; 18/12/2022
41738 End_List:
41739 00007203 C3      retn
41740
41741 Not_Matched:
41742 00007204 58      pop     ax ; get len from stack ; M050
41743 00007205 5E      pop     SI ; restore si,cx
41744 00007206 59      pop     CX
41745
41746 SkipOne:
41747 00007207 368B3E[0E06] mov     DI,[ss:TEMP_VAR2] ; restore old di use SS override
41748 0000720C 30E4      xor     AH,AH ; position to next entry
41749 0000720E 01C7      add     DI,AX
41750
41751 00007210 83C703      add     DI,3 ; DI -> next entry length
41752 ;***add    DI,4 ; DI -> next entry length
41753
41754 00007213 EBC7      jmp     short GetEntries
41755
41756 ; 18/12/2022
41757 ;End_List:
41758 ;retn
41759
41760 ; 04/08/2018 - Retro DOS v3.0
41761 ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 633Dh
41762
41763 ;-----
41764 ;SUBTTL Terminate and stay resident handler
41765 ;
41766 ; Input:  DX is an offset from CurrentPDB at which to
41767 ;         truncate the current block.
41768 ;
41769 ; output: The current block is truncated (expanded) to be [DX+15]/16
41770 ;         paragraphs long. An exit is simulated via resetting CurrentPDB
41771 ;         and restoring the vectors.
41772 ;
41773 ;-----
41774

```

```

41775 ; 20/05/2019 - Retro DOS v4.0
41776 ; DOSCODE:A19Bh (MSDOS 6.21, MSDOS.SYS)
41777
41778 ; 01/12/2022 - Retro DOS v4.0 (Modified MSDOS5.0 MSDOS.SYS)
41779 ; DOSCODE:A13Bh (MSDOS 5.0, MSDOS.SYS)
41780
41781 ; 09/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
41782 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0B3BCh
41783
41784 _$KEEP_PROCESS:
41785 00007215 50      push    AX          ; keep exit code around
41786 ;mov     byte [SS:EXIT_TYPE],3
41787 00007216 36C606[7C05]03 mov     BYTE [SS:EXIT_TYPE],EXIT_KEEP_PROCESS
41788 0000721C 368E06[3003]   mov     ES,[SS:CurrentPDB]
41789 00007221 83FA06     cmp     DX,6h          ; keep enough space around for system
41790 00007224 7303      jae     short Keep_Shrink ; info
41791
41792 00007226 BA0600     mov     DX,6h
41793
41794 Keep_Shrink:
41795 00007229 89D3     mov     BX,DX
41796 0000722B 53      push    BX
41797 0000722C 06      push    ES
41798 0000722D E82F02     call   _$SETBLOCK      ; ignore return codes.
41799 00007230 1F      pop     DS
41800 00007231 5B      pop     BX
41801 00007232 7207     jc      short Keep_Done ; failed on modification
41802
41803 00007234 8CD8     mov     AX,DS
41804 00007236 01D8     add     AX,BX
41805 ;mov     [2],ax
41806 00007238 A30200     mov     [PDB.BLOCK_LEN],AX ;PBUGBUG
41807
41808 Keep_Done:
41809 0000723B 58      pop     AX
41810 0000723C EB26     jmp     SHORT exit_inner ; and let abort take care of the rest
41811
41812 ;-----
41813 ;
41814 ;-----
41815
41816 STAY_RESIDENT:
41817 ;mov     ax,3100h
41818 0000723E B80031     mov     AX,(KEEP_PROCESS<<8)+0 ; Lower part is return code;PBUGBUG
41819 00007241 83C20F     add     DX,15
41820 00007244 D1DA     rcr     DX,1
41821 00007246 B103     mov     CL,3
41822 00007248 D3EA     shr     DX,CL
41823
41824 0000724A E9A490     jmp     COMMAND
41825
41826 ;-----
41827 ;SUBTTL $EXIT - return to parent process
41828 ; Assembler usage:
41829 ;     MOV     AL, code
41830 ;     MOV     AH, Exit
41831 ;     INT     int_command
41832 ; Error return:
41833 ;     None.
41834 ;-----
41835
41836 ; 20/05/2019 - Retro DOS v4.0
41837 ; DOSCODE:A1D3h (MSDOS 6.21, MSDOS.SYS)
41838
41839 ; 01/12/2022 - Retro DOS v4.0 (Modified MSDOS5.0 MSDOS.SYS)
41840 ; DOSCODE:A173h (MSDOS 5.0, MSDOS.SYS)
41841
41842 ; 09/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
41843 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0B3F4h
41844
41845 _$EXIT:
41846 ; 04/08/2018 - Retro DOS v3.0
41847 ; IBMDOSDOS.COM (MSDOS 3.3, 1987) - offset 6375h
41848 0000724D 30E4     xor     AH,AH
41849 0000724F 368626[4D03] xchg    AH,[SS:DidCTRLC]
41850 00007254 08E4     or      AH,AH
41851 ;mov     BYTE [SS:EXIT_TYPE],0
41852 00007256 36C606[7C05]00 mov     BYTE [SS:EXIT_TYPE],EXIT_TERMINATE
41853 0000725C 7406     jz      short exit_inner
41854 ;mov     BYTE [SS:EXIT_TYPE],1
41855 0000725E 36C606[7C05]01 mov     BYTE [SS:EXIT_TYPE],EXIT_CTRL_C
41856
41857 ;entry   Exit_inner
41858 exit_inner:
41859 00007264 E81092     call   Get_User_Stack ;PBUGBUG
41860
41861 00007267 36FF36[3003] push    word [ss:CurrentPDB]
41862 ;pop     word [si+14h]
41863 0000726C 8F4414     pop     word [SI+user_env.user_CS] ;PBUGBUG
41864 0000726F EB08     jmp     short abort_inner
41865
41866 ;BREAK <$ABORT -- Terminate a process>
41867 ;-----
41868 ; Inputs:
41869 ; user_CS:00 must point to valid program header block
41870 ; Function:
41871 ; Restore terminate and Cntrl-C addresses, flush buffers and transfer
41872 ; to the terminate address
41873 ; Returns:
41874 ; TO THE TERMINATE ADDRESS
41875 ;-----
41876
41877 _$ABORT:
41878 00007271 30C0     xor     AL,AL
41879 ;mov     byte [SS:EXIT_TYPE],0
41880 ;mov     byte [SS:EXIT_TYPE],AL ; = 0
41881 00007273 36C606[7C05]00 mov     byte [SS:EXIT_TYPE],EXIT_ABORT
41882
41883 ; abort_inner must have AL set as the exit code! The exit type
41884 ; is retrieved from exit_type. Also, the PDB at user_CS needs
41885 ; to be correct as the one that is terminating.
41886
41887 abort_inner:
41888 00007279 368A26[7C05] mov     AH,[SS:EXIT_TYPE]
41889 0000727E 36A3[3403]   mov     [SS:exit_code],AX
41890 00007282 E8F291     call   Get_User_Stack
41891
41892 ;mov     ds,[si+14h]
41893 00007285 8E5C14     mov     DS,[SI+user_env.user_CS] ; set up old interrupts ;PBUGBUG
41894 00007288 31C0     xor     AX,AX
41895 0000728A 8EC0     mov     ES,AX
41896 ;mov     si,10
41897 0000728C BE0A00     mov     SI,SAVEEXIT
41898 ;mov     di,88h

```

```

41899 0000728F BF8800      mov     DI,addr_int_terminate
41900 00007292 A5          movsw
41901 00007293 A5          movsw
41902 00007294 A5          movsw
41903 00007295 A5          movsw
41904 00007296 A5          movsw
41905 00007297 A5          movsw
41906 00007298 E954EF      jmp     reset_environment
41907
41908
41909
41910
41911
41912
41913
41914
41915
41916
41917
41918
41919
41920
41921
41922
41923
41924
41925
41926 0000729B C3          retn
41927
41928
41929
41930
41931
41932
41933
41934
41935
41936
41937
41938
41939
41940
41941
41942
41943
41944
41945
41946
41947
41948
41949
41950
41951
41952
41953
41954
41955
41956
41957
41958
41959
41960
41961
41962
41963
41964
41965
41966
41967
41968
41969
41970
41971
41972
41973
41974
41975
41976
41977
41978
41979
41980
41981
41982
41983
41984
41985
41986
41987
41988
41989
41990
41991
41992
41993
41994
41995
41996
41997
41998
41999
42000
42001
42002
42003
42004
42005
42006
42007
42008
42009
42010
42011
42012
42013
42014
42015
42016
42017
42018
42019
42020
42021
42022

;-----
; fixexepatch will point to this is DOS loads low.
;-----
; MSDOS 6.0

; 29/04/2019 - Retro DOS v4.0
; DOSCODE:A221h (MSDOS 6.21, MSDOS.SYS)

; 01/12/2022 - Retro DOS v4.0 (Modified MSDOS5.0 MSDOS.SYS)
; DOSCODE:A1C1h (MSDOS 5.0, MSDOS.SYS)

; 09/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
; PCDOS 7.1 IBMDOS.COM - DOSCODE:0B442h

RetExePatch: ; proc near

    retn

;=====
; ALLOC.ASM, MSDOS 6.0, 1991
;=====
; 04/08/2018 - Retro DOS v3.0
; 14/05/2019 - Retro DOS v4.0
; 10/03/2024 - Retro DOS v5.0

; TITLE ALLOC.ASM - memory arena manager NAME Alloc

; **
; Microsoft Confidential
; Copyright (C) Microsoft Corporation 1991
; All Rights Reserved.
;
; Memory related system calls and low level routines for MSDOS 2.X.
; I/O specs are defined in DISPATCH.
;
; $ALLOC
; $SETBLOCK
; $DEALLOC
; $AllocOper
; arena_free_process
; arena_next
; check_signature
; Coalesce
;
; Modification history:
;
; Created: ARR 30 March 1983
;
; Revision: M000 - added support for allocating UMBS. 7/9/90
;           M003 - added support for link/unlink UMBS from
;                 DOS arena chain. 7/18/90
;           M009 - Added error returns invalid function and
;                 arena trashed in set link state call.
;           M010 - Release UMB arenas allocated to current PDB
;                 if UMB_HEAD is initialized.
;
;           M016 - MACE utilities mkeyrate.com version 1.0
;                 support. Please see under M009 in
;                 ..\inc\dossym.inc. 8/31/90.
;
;           M061 - In GetLastArena, if linking in UMBS check to make
;                 sure that umb_head arena is valid and also make
;                 sure that the previous arena is pointing to
;                 umb_head.
;
;           M064 - allow HIGH_ONLY bit to be set by a call to
;                 set allloc strategy.
;                 use STRAT_MASK to mask out bits 6 & 7 of
;                 bx in AllocSetStrat.
;
;           M068 - use a count value (A20OFF_COUNT) rather than
;                 a bit to indicate to dos dispatcher to turn
;                 a20 off before iret. See M016.
;
; BREAK <memory allocation utility routines>

; 15/04/2018 - Retro DOS v2.0
;-----
; xenix memory calls for MSDOS
;
; CAUTION: The following routines rely on the fact that arena_signature and
; arena_owner_system are all equal to zero and are contained in DI.
; INCLUDE DOSSEG.ASM

; CODE SEGMENT BYTE PUBLIC 'CODE'
; ASSUME SS:DOSGROUP,CS:DOSGROUP

; .xlist
; .xcref
; INCLUDE DOSSYM.ASM
; INCLUDE DEVSYM.ASM
; .cref
; .list

; TITLE ALLOC.ASM - memory arena manager
; NAME Alloc

; SUBTTL memory allocation utility routines
; PAGE
;
; arena data
;
; i_need arena_head,WORD ; seg address of start of arena
; i_need CurrentPDB,WORD ; current process data block addr
; i_need FirstArena,WORD ; first free block found
; i_need BestArena,WORD ; best free block found
; i_need LastArena,WORD ; last free block found
; i_need AllocMethod,BYTE ; how to alloc first(best)last
;-----

```

```

42023
42024 ; 10/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
42025 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:0B443h
42026 ;;;
42027 test_umb_flag:
42028 0000729C 36F606[8900]01 test byte [ss:UMBFLAG],LINKSTATE ; 1 ; Q: are umb's linked
42029 000072A2 C3 retn ; ZF=1 -> N: scan from arena_head
42030 ; ZF=0 -> Y: start_arena = umb_head
42031 ;;;
42032
42033 ;** Arena_Free_Process
42034 ;-----
42035 ; Free all arena blocks allocated to a process
42036 ;
42037 ; ENTRY (bx) = PID of process
42038 ; EXIT none
42039 ; USES ?????? BUGBUG
42040 ;-----
42041
42042 ; 01/12/2022 - Retro DOS v4.0 (Modified MSDOS5.0 MSDOS.SYS)
42043 ; DOSCODE:A1C2h (MSDOS 5.0, MSDOS.SYS)
42044
42045 ; 10/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
42046 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:0B44Ah
42047
42048 arena_free_process:
42049 ; 14/05/2019 - Retro DOS v4.0
42050 ; 04/08/2018 - Retro DOS v3.0
42051 000072A3 36A1[2400] MOV AX,[ss:arena_head]
42052 arena_free_process_start:
42053 000072A7 BF0000 MOV DI,ARENA.SIGNATURE ; 0
42054 ;MOV AX,[ss:arena_head] ; 15/04/2018
42055 000072AA E82F00 CALL check_signature ; ES <- AX, check for valid block
42056
42057 arena_free_process_loop:
42058 ;retc
42059 000072AD 7225 JC SHORT AFP_RETN ; Retro DOS v2.0 - 05/03/2018
42060 000072AF 06 PUSH ES
42061 000072B0 1F POP DS
42062 ;cmp [1],bx
42063 000072B1 391E0100 CMP [ARENA.OWNER],BX ; is block owned by pid?
42064 000072B5 7504 JNZ SHORT arena_free_next ; no, skip to next
42065 ;mov [1],di
42066 000072B7 893E0100 MOV [ARENA.OWNER],DI ; yes... free him
42067
42068 arena_free_next:
42069 ;cmp byte [di],5Ah ; 'z'
42070 000072BB 803D5A CMP BYTE [DI],arena_signature_end
42071 ; ; end of road, Jack?
42072 ;retz ; never come back no more
42073 ;JZ SHORT AFP_RETN ; MSDOS 3.3 (& MSDOS 2.11)
42074 ; 14/05/2019
42075 ; MSDOS 6.0
42076 000072BE 7405 JZ short arena_chk_umbs
42077
42078 000072C0 E81200 CALL arena_next ; next item in ES/AX carry set if trash
42079 000072C3 EBE8 JMP SHORT arena_free_process_loop
42080
42081 ; MSDOS 6.0
42082 arena_chk_umbs: ; M010 - Start
42083 ; 20/05/2019
42084 000072C5 36A1[8C00] mov ax,[ss:UMB_HEAD] ; ax = umb_head
42085 000072C9 83F8FF cmp ax,0FFFFh ; Q: is umb_head initialized
42086 000072CC 741D JE short ret_label ; N: we're done
42087
42088 000072CE 8CDF mov di,ds ; di = last arena
42089 000072D0 39C7 cmp di,ax ; Q: is last arena above umb_head
42090 ;jae short ret_label ; Y: we've scanned umbs also. done.
42091 ;jmp short arena_free_process_start
42092 ; M010 - End
42093 ; 10/03/2024 (PC DOS 7.1 IBMDOS.COM)
42094 000072D2 72D3 jnb short arena_free_process_start
42095
42096 ; 10/03/2024
42097 AFP_RETN:
42098 000072D4 C3 RETN
42099
42100 ; BREAK <Arena Helper Routines>
42101
42102 ;** Arena_Next - Find Next item in Arena
42103 ;-----
42104 ; ENTRY DS - pointer to block head
42105 ; (di) = 0
42106 ; EXIT AX,ES - pointers to next head
42107 ; 'C' set iff arena damaged
42108 ;-----
42109
42110 arena_next:
42111 000072D5 8CD8 MOV AX,DS ; AX <- current block
42112 000072D7 03060300 ADD AX,[ARENA.SIZE] ; AX <- AX + current block length
42113 000072DB 40 INC AX ; remember that header!
42114
42115 ; fall into check_signature and return
42116 ;
42117 ; CALL check_signature ; ES <- AX, carry set if error
42118 ; RETN
42119
42120 ;** Check_Signature - Check Memory Block Signature
42121 ;-----
42122 ; ENTRY (AX) = address of block header
42123 ; (di) = 0
42124 ; EXIT ES = AX
42125 ; 'C' clear if signature good
42126 ; 'C' set if signature bad
42127 ; USES ES, Flags
42128 ;-----
42129
42130 check_signature:
42131
42132 000072DC 8EC0 MOV ES,AX ; ES <- AX
42133 ;cmp byte [es:di],4Dh ; 'M'
42134 000072DE 26803D4D CMP BYTE [ES:DI],arena_signature_normal
42135 ; ; IF next signature = not_end THEN
42136 000072E2 7407 JZ SHORT check_signature_ok ; GOTO ok
42137 ;cmp byte [es:di],5Ah ; 'z'
42138 000072E4 26803D5A CMP BYTE [ES:DI],arena_signature_end
42139 ; ; IF next signature = end then
42140 000072E8 7401 JZ SHORT check_signature_ok ; GOTO ok
42141 000072EA F9 STC ; set error
42142
42143 ret_label: ; MSDOS 6.0
42144 ;AFP_RETN: ; 10/03/2024
42145 ; Retro DOS v2.0 - 05/03/2018
42146 check_signature_ok:
COALESCE_RETN:

```

```

42147 000072EB C3      RETN
42148
42149      ;** Coalesce - Combine free blocks ahead with current block
42150      ;-----
42151      ; Coalesce adds the block following the argument to the argument block,
42152      ; iff it's free. Coalesce is usually used to join free blocks, but
42153      ; some callers (such as $setblock) use it to join a free block to it's
42154      ; preceeding allocated block.
42155      ;
42156      ; ENTRY (ds) = pointer to the head of a free block
42157      ; (di) = 0
42158      ; EXIT 'C' clear if OK
42159      ; (ds) unchanged, this block updated
42160      ; (ax) = address of next block, IFF not at end
42161      ; 'C' set if arena trashed
42162      ; USES (cx)
42163      ;-----
42164
42165      Coalesce:
42166      ;cmp byte [di],5Ah ; 'Z'
42167 000072EC 803D5A      CMP BYTE [DI],arena_signature_end
42168      ; IF current signature = END THEN
42169      ; GOTO ok
42170      ;retz
42171 000072EF 74FA      jz short COALESCE_RETn
42172 000072F1 E8E1FF      CALL arena_next ; ES, AX <- next block, Carry set if error
42173 000072F4 72F5      jc short COALESCE_RETn ; IF no error THEN GOTO check
42174
42175      coalesce_check:
42176      ;cmp [es:1],di
42177 000072F6 26393E0100      CMP [ES:ARENA.OWNER],DI
42178      ;retnz
42179 000072FB 75EE      jnz SHORT COALESCE_RETn ; IF next block isnt free THEN return
42180      ;mov cx,[es:3]
42181 000072FD 268B0E0300      MOV CX,[ES:ARENA.SIZE] ; CX <- next block size
42182 00007302 41          INC CX ; CX <- CX + 1 (for header size)
42183      ;ADD [3],CX
42184 00007303 010E0300      ADD [ARENA.SIZE],CX ; current size <- current size + CX
42185 00007307 268A0D      MOV CL,[ES:DI] ; move up signature
42186 0000730A 880D      MOV [DI],CL
42187 0000730C EBDE      JMP SHORT Coalesce ; try again
42188
42189      ; 04/08/2018 - Retro DOS v3.0
42190      ; IBMDOS.COM (MSDOS 3.3, 1987) - Offset 641Fh
42191
42192      ; BREAK <$Alloc - allocate space in memory>
42193
42194      ; MSDOS 6.0
42195      ;-----
42196      ;** $Alloc - Allocate Memory Space
42197      ;-----
42198      ; $Alloc services the INT21 that allocates memory space to a program.
42199      ; Alloc returns a pointer to a free block of memory that
42200      ; has the requested size in paragraphs.
42201      ;
42202      ; If the allocation strategy is HIGH_FIRST or HIGH_ONLY memory is
42203      ; scanned from umb_head if not from arena_head. If the strategy is
42204      ; HIGH_FIRST the scan is continued from arena_head if a block of
42205      ; appropriate size is not found in the UMBS. If the strategy is
42206      ; HIGH_FIRST+HIGH_ONLY only the UMBS are scanned for memory.
42207      ;
42208      ; In either case if bit 0 of UmbFlag is not initialized then the scan
42209      ; starts from arena_head.
42210      ;
42211      ; Assembler usage:
42212      ; MOV BX,size
42213      ; MOV AH,Alloc
42214      ; INT 21h
42215      ;
42216      ; BUGBUG - a lot can be done to improve performance. We can set marks
42217      ; so that we start searching the arena at it's first non-trivial free
42218      ; block, we can peephole the code, etc. (We can move some subr calls
42219      ; inline, etc.) I assume that this is called rarely and that the arena
42220      ; doesn't have too many memory objects in it beyond the first free one.
42221      ; verify that this is true; if so, this can stay as is
42222      ;
42223      ; ENTRY (bx) = requested size, in bytes
42224      ; (DS) = (ES) = DOSGROUP
42225      ; EXIT 'C' clear if memory allocated
42226      ; (ax:0) = address of requested memory
42227      ; 'C' set if request failed
42228      ; (AX) = error_not_enough_memory
42229      ; (bx) = max size we could have allocated
42230      ; (ax) = error_arena_trashed
42231      ; USES All
42232      ;-----
42233
42234      ; MSDOS 2.11 (& MSDOS 3.3)
42235      ;-----
42236      ;SUBTTL $Alloc - allocate space in memory
42237      ;-----
42238      ; Assembler usage:
42239      ; MOV BX,size
42240      ; MOV AH,Alloc
42241      ; INT 21h
42242      ; AX:0 is pointer to allocated memory
42243      ; BX is max size if not enough memory
42244      ;
42245      ; Description:
42246      ; Alloc returns a pointer to a free block of
42247      ; memory that has the requested size in paragraphs.
42248      ;
42249      ; Error return:
42250      ; AX = error_not_enough_memory
42251      ; = error_arena_trashed
42252      ;-----
42253
42254      ; DOSCODE:A28Eh (MSDOS 6.21, MSDOS.SYS)
42255
42256      ; 01/12/2022 - Retro DOS v4.0 (Modified MSDOS5.0 MSDOS.SYS)
42257      ; DOSCODE:A22Eh (MSDOS 5.0, MSDOS.SYS)
42258
42259      ; 10/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
42260      ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0B4B5h
42261
42262      _$ALLOC:
42263      ; 25/05/2019 (Procedure has been checked and confirmed)
42264      ; 14/05/2019 - Retro DOS v4.0
42265      ; 04/08/2018 - Retro DOS v3.0
42266      ;EnterCritMem
42267 0000730E E8D1A5      call ECritMEM ; MSDOS 3.3 & MSDOS 6.0
42268
42269      ; 17/12/2022
42270      ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)

```

```

42271 ;%if 0
42272 ; 14/05/2019
42273 00007311 16 push ss
42274 00007312 1F pop ds
42275
42276 ; MSDOS 6.0
42277 ;mov ax,[ss:arena_head]
42278 ;mov [ss:START_ARENA],ax ; assume LOW_FIRST
42279
42280 00007313 A1[2400] mov ax,[arena_head]
42281 00007316 A3[8E00] mov [START_ARENA],ax
42282
42283 ;test byte [ss:AllocMethod],HIGH_FIRST+HIGH_ONLY
42284 00007319 F606[0203]C0 test byte [AllocMethod],HIGH_FIRST+HIGH_ONLY
42285 ; Q: should we start scanning from
42286 ; UMB's
42287 0000731E 740B jz short norm_alloc ; N: scan from arena_head
42288
42289 ; 10/03/2024
42290 %if 0
42291 ;cmp word [ss:UMB_HEAD],-1 ; Q: Has umb_head been initialized
42292 ;cmp word [UMB_HEAD],-1
42293 ;je short norm_alloc ; N: scan from arena_head
42294
42295 ;test byte [ss:UMBFLAG],LINKSTATE ; Q: are umb's linked
42296 test byte [UMBFLAG],LINKSTATE ; 1
42297 jz short norm_alloc ; N: scan from arena_head
42298 %else
42299 ; 10/03/2024 (PCDOS 7.1 IBMDOS.COM)
42300 ;;;
42301 00007320 E879FF call test_umb_flag
42302 00007323 7406 jz short norm_alloc
42303 ;;;
42304 %endif
42305
42306 ;mov ax,[ss:UMB_HEAD]
42307 ;mov [ss:START_ARENA],ax ; start_arena = umb_head
42308 00007325 A1[8C00] mov ax,[UMB_HEAD]
42309 00007328 A3[8E00] mov [START_ARENA],ax
42310 ; M000 - end
42311 norm_alloc:
42312 0000732B 31C0 XOR AX,AX
42313 0000732D 89C7 MOV DI,AX
42314 ; 15/03/2018
42315 ;MOV [SS:FirstArena],AX ; init the options
42316 ;MOV [SS:BestArena],AX
42317 ;MOV [SS:LastArena],AX
42318 ; 14/05/2019
42319 0000732F A3[4003] MOV [FirstArena],AX ; init the options
42320 00007332 A3[4203] MOV [BestArena],AX
42321 00007335 A3[4403] MOV [LastArena],AX
42322 00007338 50 PUSH AX ; alloc_max <- 0
42323 ; 04/08/2018
42324 start_scan:
42325 ;MOV AX,[ss:arena_head] ; AX <- beginning of arena
42326 ;MOV AX,[arena_head]
42327
42328 ; 14/05/2019
42329 ; MSDOS 6.0
42330 ;mov ax,[ss:START_ARENA] ; M000: AX <- beginning of arena
42331 00007339 A1[8E00] mov ax,[START_ARENA]
42332
42333 ; 27/09/2023 (BugFix) (*)
42334 ; ( jump from 'alloc_chk' (ds<>ss, ax = [ss:START_ARENA]))
42335 start_scan_x:
42336
42337 0000733C E89DFF CALL check_signature ; ES <- AX, carry set if error
42338 0000733F 7233 JC SHORT alloc_err ; IF error THEN GOTO err
42339
42340 ;%endif
42341
42342 ; 17/12/2022
42343 %if 0
42344 ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42345
42346 ; MSDOS 6.0
42347 mov ax,[ss:arena_head]
42348 mov [ss:START_ARENA],ax ; assume LOW_FIRST
42349
42350 test byte [ss:AllocMethod],HIGH_FIRST+HIGH_ONLY
42351 ; Q: should we start scanning from
42352 ; UMB's
42353 jz short norm_alloc ; N: scan from arena_head
42354
42355 ;cmp word [ss:UMB_HEAD],-1 ; Q: Has umb_head been initialized
42356 ;je short norm_alloc ; N: scan from arena_head
42357
42358 test byte [ss:UMBFLAG],LINKSTATE ; Q: are umb's linked
42359 jz short norm_alloc ; N: scan from arena_head
42360
42361 mov ax,[ss:UMB_HEAD]
42362 mov [ss:START_ARENA],ax ; start_arena = umb_head
42363 ; M000 - end
42364 norm_alloc:
42365 XOR AX,AX
42366 MOV DI,AX
42367 ; 15/03/2018
42368 MOV [SS:FirstArena],AX ; init the options
42369 MOV [SS:BestArena],AX
42370 MOV [SS:LastArena],AX
42371 PUSH AX ; alloc_max <- 0
42372 ; 04/08/2018
42373 start_scan:
42374 ;MOV AX,[ss:arena_head] ; AX <- beginning of arena
42375 ; 14/05/2019
42376 ; MSDOS 6.0
42377 mov ax,[ss:START_ARENA] ; M000: AX <- beginning of arena
42378 CALL check_signature ; ES <- AX, carry set if error
42379 JC SHORT alloc_err ; IF error THEN GOTO err
42380 %endif
42381
42382 alloc_scan:
42383 00007341 06 PUSH ES
42384 00007342 1F POP DS ; DS <- ES
42385 00007343 393E0100 CMP [ARENA.OWNER],DI ; 0
42386 00007347 7466 JZ SHORT alloc_free ; IF current block is free THEN examine
42387
42388 alloc_next:
42389
42390 ; 10/03/2024
42391 %if 0
42392 ; MSDOS 6.0 ; M000 - start
42393 test byte [ss:UMBFLAG],LINKSTATE ; Q: are umb's linked
42394 jz short norm_strat ; N: see if we reached last arena

```



```

42395 %else
42396 ; 10/03/2024 (PCDOS 7.1 IBMDOS.COM)
42397 ;;;
42398 00007349 E850FF call test_umb_flag
42399 0000734C 741C jz short norm_strat
42400 ;;;
42401 %endif
42402 0000734E 36F606[0203]80 test byte [ss:AllocMethod],HIGH_FIRST
42403 ; Q: is alloc strategy high_first
42404 00007354 7414 jz short norm_strat ; N: see if we reached last arena
42405 00007356 36A1[8E00] mov ax,[ss:START_ARENA]
42406 0000735A 363B06[2400] cmp ax,[ss:arena_head] ; Q: did we start scan from
42407 ; arena_head
42408 0000735F 7509 jne short norm_strat ; N: see if we reached last arena
42409 00007361 8CD8 mov ax,ds ; ax = current block
42410 00007363 363B06[8C00] cmp ax,[ss:UMB_HEAD] ; Q: check against umb_head
42411 00007368 EB03 jmp short alloc_chk_end
42412
42413 norm_strat:
42414 ;cmp byte [di],5Ah ; 'Z'
42415 0000736A 803D5A cmp BYTE [DI],arena_signature_end
42416 ; IF current block is last THEN
42417
42418 0000736D 740E jz short alloc_end ; GOTO end
42419 0000736F E863FF call arena_next ; AX, ES <- next block, Carry set if error
42420 00007372 73CD jnc short alloc_scan ; IF no error THEN GOTO scan
42421
42422 alloc_err:
42423 00007374 58 pop ax
42424
42425 alloc_trashed:
42426 ;LeaveCrit critMem
42427 00007375 E897A5 call LCritMEM ; MSDOS 3.3 & MSDOS 6.0
42428
42429 alloc_trashed2: ; 10/03/2024 - Retro DOS v5.0 (PCDOS 7.1) -jmp from GetLastArena-
42430 ;error error_arena_trashed
42431 ;mov al,7
42432 00007378 B007 mov AL,error_arena_trashed
42433
42434 0000737A E9FB92 jmp SYS_RET_ERR
42435
42436 alloc_end:
42437 ; 18/05/2019
42438 0000737D 36833E[4003]00 cmp word [ss:FirstArena],0
42439 00007383 7403 jz short alloc_chk
42440 00007385 E98400 jmp alloc_do_split
42441
42442 alloc_chk:
42443 ; MSDOS 6.0
42444 00007388 36A1[2400] mov ax,[ss:arena_head]
42445 0000738C 363B06[8E00] cmp ax,[ss:START_ARENA] ; Q: started scanning from arena_head
42446 00007391 740E je short alloc_fail ; Y: not enough memory
42447 ; N:
42448 ; Q: is the alloc strat HIGH_ONLY
42449 00007393 36F606[0203]40 test byte [ss:AllocMethod],HIGH_ONLY
42450 00007399 7506 jnz short alloc_fail ; Y: return size of largest UMB
42451
42452 0000739B 36A3[8E00] mov [ss:START_ARENA],ax ; N: start scanning from arena_head
42453 ; 27/09/2023 (*)
42454 0000739F EB9B jmp short start_scan_x ; (*) ; (BugFix)
42455 ;jmp short start_scan
42456 ; M000 - end
42457
42458 alloc_fail:
42459 ;invoke Get_User_Stack
42460 000073A1 E8D390 call Get_User_Stack
42461 000073A4 5B pop bx
42462 ;MOV [SI].user_BX,BX
42463 000073A5 895C02 mov [SI+user_env.user_BX],bx
42464 ;LeaveCrit critMem
42465 000073A8 E864A5 call LCritMEM ; MSDOS 3.3 & MSDOS 6.0
42466 ;error error_not_enough_memory
42467 ;mov al,8
42468 000073AB B008 mov AL,error_not_enough_memory
42469 ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42470 000073AD EBCB jmp short alloc_errj
42471 ;JMP SYS_RET_ERR
42472
42473 alloc_free:
42474 000073AF E83AFF call Coalesce ; add following free block to current
42475 000073B2 72C0 jc short alloc_err ; IF error THEN GOTO err
42476 000073B4 8B0E0300 mov cx,[ARENA.SIZE]
42477 000073B8 5A pop dx ; check for max found size
42478 000073B9 39D1 cmp cx,dx
42479 000073BB 7602 jna short alloc_test
42480 000073BD 89CA mov dx,cx
42481
42482 alloc_test:
42483 000073BF 52 push dx
42484 000073C0 39CB cmp bx,cx ; IF BX > size of current block THEN
42485 000073C2 7785 ja short alloc_next ; GOTO next
42486
42487 ; 15/03/2018
42488 000073C4 36833E[4003]00 cmp word [ss:FirstArena],0
42489 000073CA 7505 jnz short alloc_best
42490 000073CC 368C1E[4003] mov [ss:FirstArena],ds ; save first one found
42491
42492 000073D1 36833E[4203]00 cmp word [ss:BestArena],0
42493 000073D7 740E jz short alloc_make_best ; initial best
42494 000073D9 06 push es
42495 000073DA 368E06[4203] mov es,[ss:BestArena]
42496 000073DF 26390E0300 cmp [es:ARENA.SIZE],cx ; is size of best larger than found?
42497 000073E4 07 pop es
42498 000073E5 7605 jbe short alloc_last
42499
42500 000073E7 368C1E[4203] mov [ss:BestArena],ds ; assign best
42501
42502 000073EC 368C1E[4403] mov [ss:LastArena],ds ; assign last
42503 000073F1 E955FF jmp alloc_next
42504
42505 ;
42506 ; split the block high
42507
42508 000073F4 368E1E[4403] mov ds,[ss:LastArena]
42509 000073F9 8B0E0300 mov cx,[ARENA.SIZE]
42510 000073FD 29D9 sub cx,bx
42511 000073FF 8CDA mov dx,ds
42512 00007401 7449 je short alloc_set_owner ; sizes are equal, no split
42513 00007403 01CA add dx,cx ; point to next block
42514 00007405 8EC2 mov es,dx ; no decrement!
42515 00007407 49 dec cx
42516 00007408 87D9 xchg bx,cx ; bx has size of lower block
42517 0000740A EB2B jmp short alloc_set_sizes ; cx has upper (requested) size
42518 ;

```

```

42519 ; we have scanned memory and have found all appropriate blocks
42520 ; check for the type of allocation desired; first and best are identical
42521 ; last must be split high
42522 ;
42523 alloc_do_split:
42524 ;
42525 ; 17/12/2022
42526 ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42527 ;%if 0
42528 ; 14/05/2019
42529 ; MSDOS 6.0 ; M000 - start
42530 ;xor cx,cx
42531 0000740C 368A0E[0203] mov cl,[ss:AllocMethod]
42532 ;and cx,STRAT_MASK ; 0FF3Fh; mask off bit 7
42533 00007411 80E13F and cl,3Fh
42534 ;cmp cx,BEST_FIT ; 1 ; Q; is the alloc strategy best_fit
42535 00007414 80F901 cmp cl,BEST_FIT
42536 00007417 77DB ja short alloc_do_split_high
42537 ;%endif
42538 ;
42539 ; 17/12/2022
42540 ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42541 ; MSDOS 6.0 & MSDOS 5.0
42542 ;xor cx,cx
42543 ;mov cl,[ss:AllocMethod]
42544 ;and cx,STRAT_MASK ; 0FF3Fh; mask off bit 7
42545 ;cmp cx,BEST_FIT ; 1 ; Q; is the alloc strategy best_fit
42546 ;ja short alloc_do_split_high
42547 ;
42548 ; 15/03/2018
42549 ;;CMP BYTE [SS:AllocMethod], 1
42550 ; 04/08/2018
42551 ;CMP BYTE [SS:AllocMethod],BEST_FIT
42552 ;JA SHORT alloc_do_split_high
42553 ;
42554 00007419 368E1E[4003] MOV DS,[SS:FirstArena]
42555 0000741E 7205 JB SHORT alloc_get_size
42556 00007420 368E1E[4203] MOV DS,[SS:BestArena]
42557 ;
42558 alloc_get_size:
42559 00007425 8B0E0300 MOV CX,[ARENA.SIZE]
42560 00007429 29D9 SUB CX,BX ; get room left over
42561 0000742B 8CD8 MOV AX,DS
42562 0000742D 89C2 MOV DX,AX ; save for owner setting
42563 0000742F 741B JE SHORT alloc_set_owner ; IF BX = size THEN (don't split)
42564 00007431 01D8 ADD AX,BX
42565 00007433 40 INC AX ; remember the header
42566 00007434 8EC0 MOV ES,AX ; ES <- DS + BX (new header location)
42567 00007436 49 DEC CX ; CX <- size of split block
42568 ;
42569 00007437 891E0300 alloc_set_sizes: MOV [ARENA.SIZE],BX ; current size <- BX
42570 0000743B 26890E0300 MOV [ES:ARENA.SIZE],CX ; split size <- CX
42571 ;mov b1,4Dh ; 'M'
42572 00007440 834D MOV BL,arena_signature_normal
42573 00007442 861D XCHG BL,[DI] ; current signature <- 4D
42574 00007444 26881D MOV [ES:DI],BL ; new block sig <- old block sig
42575 00007447 26893E0100 MOV [ES:ARENA.OWNER],DI
42576 ;
42577 alloc_set_owner:
42578 0000744C 8EDA MOV DS,DX
42579 0000744E 36A1[3003] MOV AX,[SS:CurrentPDB] ; 15/03/2018
42580 00007452 A30100 MOV [ARENA.OWNER],AX
42581 00007455 8CD8 MOV AX,DS
42582 00007457 40 INC AX
42583 00007458 5B POP BX
42584 ;LeaveCrit critMem
42585 00007459 E8B3A4 call LCritMEM ; MSDOS 3.3 & MSDOS 6.0
42586 ;
42587 ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42588 alloc_ok:
42589 ;transfer SYS_RET_OK
42590 0000745C E90F92 JMP SYS_RET_OK
42591 ;
42592 ; BREAK $SETBLOCK - change size of an allocated block (if possible)
42593 ;
42594 ; MSDOS 6.0
42595 -----
42596 ** $SETBLOCK - Change size of an Allocated Block
42597 ;
42598 ; Setblock changes the size of an allocated block. First, we coalesce
42599 ; any following free space onto this block; then we try to trim the
42600 ; block down to the size requested.
42601 ;
42602 ; Note that if the guy wants to grow the block but that growth fails,
42603 ; we still go ahead and coalesce any trailing free blocks onto it.
42604 ; Thus the maximum-size-possible value that we return has already
42605 ; been allocated! This is a bug, dare we fix it? BUGBUG
42606 ;
42607 ; NOTE - $SETBLOCK is in bed with $ALLOC and jumps into $ALLOC to
42608 ; finish it's work. For this reason we build the allocsf
42609 ; structure on the frame, to make us compatible with $ALLOCS
42610 ; code.
42611 ;
42612 ; ENTRY (es) = segment of old block
42613 ; (bx) = newsize
42614 ; (ah) = SETBLOCK
42615 ;
42616 ; EXIT 'C' clear if OK
42617 ; 'C' set if error
42618 ; (ax) = error_invalid_block
42619 ; = error_arena_trashed
42620 ; = error_not_enough_memory
42621 ; = error_invalid_function
42622 ; (bx) = maximum size possible, iff (ax) = error_not_enough_memory
42623 ; USES ???? BUGBUG
42624 ;
42625 -----
42626 ; MSDOS 2.11 (& MSDOS 3.3)
42627 -----
42628 SUBTTL $SETBLOCK - change size of an allocated block (if possible)
42629 ;
42630 ; Assembler usage:
42631 ; MOV ES,block
42632 ; MOV BX,newsize
42633 ; MOV AH,setblock
42634 ; INT 21h
42635 ; if setblock fails for growing, BX will have the maximum
42636 ; size possible
42637 ; Error return:
42638 ; AX = error_invalid_block
42639 ; = error_arena_trashed
42640 ; = error_not_enough_memory
42641 ; = error_invalid_function
42642 ;

```



```

42643
42644
42645
42646
42647 0000745F E880A4
42648
42649 00007462 BF0000
42650 00007465 8CC0
42651 00007467 48
42652 00007468 E871FE
42653 0000746B 7303
42654
42655
42656 0000746D E905FF
42657
42658
42659 00007470 8ED8
42660 00007472 E877FE
42661 00007475 72F6
42662 00007477 8B0E0300
42663 0000747B 51
42664 0000747C 39CB
42665 0000747E 76A5
42666 00007480 E91EFF
42667
42668
42669
42670
42671
42672
42673
42674
42675
42676
42677
42678
42679
42680
42681
42682
42683
42684
42685
42686
42687
42688
42689
42690
42691
42692
42693
42694
42695
42696
42697
42698
42699
42700
42701 00007483 E85CA4
42702
42703
42704 00007486 36F606[8600]04
42705
42706
42707 0000748C 740D
42708 0000748E 36803E[8500]00
42709 00007494 7505
42710
42711
42712 00007496 36FE06[8500]
42713
42714 0000749B BF0000
42715 0000749E 8CC0
42716 000074A0 48
42717 000074A1 E838FE
42718 000074A4 720A
42719 000074A6 26893E0100
42720
42721 000074AB E861A4
42722
42723
42724
42725 000074AE EBAC
42726
42727
42728
42729
42730 000074B0 E85CA4
42731
42732
42733 000074B3 B009
42734
42735
42736
42737 000074B5 E9C091
42738
42739
42740
42741
42742
42743
42744
42745
42746
42747
42748
42749
42750
42751
42752
42753
42754
42755
42756
42757
42758
42759
42760
42761
42762
42763
42764
42765
42766

; $SETBLOCK:
; 04/08/2018 - Retro DOS v3.0
; EnterCrit critMem
call ECritMEM ; MSDOS 3.3 & MSDOS 6.0

MOV DI,ARENA.SIGNATURE
MOV AX,ES
DEC AX
CALL check_signature
JNC SHORT setblock_grab

setblock_bad:
JMP alloc_trashed

setblock_grab:
MOV DS,AX
CALL Coalesce
JC SHORT setblock_bad
MOV CX,[ARENA.SIZE]
PUSH CX
CMP BX,CX
JBE SHORT alloc_get_size
JMP alloc_fail

; BREAK $DEALLOC - free previously allocated piece of memory

; MSDOS 6.0
;-----
** $DEALLOC - Free Heap Memory
;
; ENTRY (es) = address of item
;
; EXIT 'C' clear of OK
; 'C' set if error
; (AX) = error_invalid_block
; USES ??? BUGBUG
;
; MSDOS 2.11 (& MSDOS 3.3)
;-----
SUBTTL $DEALLOC - free previously allocated piece of memory
;
; Assembler usage:
; MOV ES,block
; MOV AH,dealloc
; INT 21h
;
; Error return:
; AX = error_invalid_block
; = error_arena_trashed
;-----

; 01/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
; 10/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
_$DEALLOC:
; 14/05/2019 - Retro DOS v4.0
; 04/08/2018 - Retro DOS v3.0
; EnterCrit critMem
call ECritMEM ; MSDOS 3.3 & MSDOS 6.0

; MSDOS 6.0 ; M016, M068 - Start
test byte [ss:DOS_FLAG],EXECA200FF ; Q: was the previous call an int 21
; ; exec call
jz short deallocate ; N: continue
cmp byte [ss:A200FF_COUNT], 0 ; Q: is count 0
jne short deallocate ; N: continue
;mov byte [ss:A200FF_COUNT], 1 ; Y: set count to 1
; 25/09/2023
inc byte [ss:A200FF_COUNT]
dealloc: ; M016, M068 - End
MOV DI,ARENA.SIGNATURE ; = 0
MOV AX,ES
DEC AX
CALL check_signature
JC SHORT dealloc_err
MOV [ES:ARENA.OWNER],DI
;LeaveCrit critMem
call LCritMEM ; MSDOS 3.3 & MSDOS 6.0
; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
;transfer SYS_RET_OK
dealloc_ok:
jmp short alloc_ok
;JMP SYS_RET_OK

dealloc_err:
;LeaveCrit critMem
call LCritMEM ; MSDOS 3.3 & MSDOS 6.0
;error error_invalid_block
;mov al,9
MOV AL,error_invalid_block
; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
dealloc_errj:
AllocOperErrj: ; 17/12/2022
JMP SYS_RET_ERR

; BREAK $AllocOper - get/set allocation mechanism

; MSDOS 6.0
;-----
** $AllocOper - Get/Set Allocation Mechanism
;
; Assembler usage:
; MOV AH,AllocOper
; MOV BX,method
; MOV AL,func
; INT 21h
;
; ENTRY
; (a) = 0
; Get allocation Strategy in (ax)
;
; (a) = 1, (bx) = method = zw0000xy
; Set allocation strategy.
; w = 1 => HIGH_ONLY
; z = 1 => HIGH_FIRST
; xy = 00 => FIRST_FIT
; = 01 => BEST_FIT
; = 10 => LAST_FIT
;
; (a) = 2
; Get UMB link state in (a)
;
; (a) = 3

```

```

42767 ; Set UMB link state
42768 ; (bx) = 0 => Unlink UMBs
42769 ; (bx) = 1 => Link UMBs
42770 ;
42771 ;
42772 ; EXIT 'C' clear if OK
42773 ;
42774 ; if (al) = 0
42775 ; (ax) = existing method
42776 ; if (al) = 1
42777 ; Sets allocation strategy
42778 ; if (al) = 2
42779 ; (al) = 0 => UMBs not linked
42780 ; (al) = 1 => UMBs linked in
42781 ; if (al) = 3
42782 ; Links/unlinks the UMBs into DOS chain
42783 ;
42784 ; 'C' set if error
42785 ; AX = error_invalid_function
42786 ;
42787 ; Rev. M000 - added support for HIGH_FIRST in (al) = 1. 7/9/90
42788 ; Rev. M003 - added functions (al) = 2 and (al) = 3. 7/18/90
42789 ; Rev. M009 - (al) = 3 will return 'invalid function' in ax if
42790 ; umbhead has'nt been initialized by sysinit and 'trashed
42791 ; arena' if an arena sig is damaged.
42792 ;-----
42793 ; MSDOS 2.11 (& MSDOS 3.3)
42794 ;-----
42795 ; SUBTTL $AllocOper - get/set allocation mechanism
42796 ;
42797 ; Assembler usage:
42798 ; MOV AH,AllocOper
42799 ; MOV BX,method
42800 ; MOV AL,func
42801 ; INT 21h
42802 ;
42803 ;
42804 ; Error return:
42805 ; AX = error_invalid_function
42806 ;-----
42807 ;
42808 ; 01/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
42809 ; 10/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
42810 ;_ $ALLOPER:
42811 ; 14/05/2019 - Retro DOS v4.0
42812 ; MSDOS 6.0
42813 000074B8 08C0 or al,al ; 0
42814 000074BA 7411 jz short AllocGetStrat
42815 ; 17/12/2022
42816 ; cmp al,1
42817 ; jz short AllocSetStrat
42818 ;
42819 ; 01/12/2022
42820 ; cmp al,2
42821 ; jb short AllocSetStrat
42822 ; ja short AllocSetLink
42823 ; jmp short AllocGetLink
42824 ; AllocGetLink:
42825 ; MSDOS 6.0
42826 ; mov al,[ss:UMBFLAG] ; return link state in al
42827 ; and al,LINKSTATE
42828 ; transfer SYS_RET_OK
42829 ; jmp SYS_RET_OK
42830 ;
42831 000074BC 3C02 cmp al,2
42832 ; 17/12/2022
42833 000074BE 7216 jb short AllocSetStrat ; al = 1
42834 000074C0 7425 je short AllocGetLink
42835 ;
42836 ; cmp al,2
42837 ; jz short AllocGetLink
42838 000074C2 3C03 cmp al,3
42839 000074C4 7429 jz short AllocSetLink
42840 ;
42841 ; 15/04/2018
42842 ; CMP AL,1
42843 ; JB SHORT AllocOperGet
42844 ; JZ SHORT AllocOperSet
42845 ;
42846 ; AllocOperError:
42847 ;
42848 ; 10/03/2024
42849 %if 0
42850 ; 04/08/2018 - Retro DOS v3.0
42851 ; MSDOS 3.3 (& MSDOS 6.0) ; Extended Error Locus
42852 ; mov byte [ss:EXTERR_LOCUS],5
42853 ; MOV byte [SS:EXTERR_LOCUS],errLOC_Mem
42854 ; error error_invalid_function
42855 %else
42856 ; 10/03/2024 - Retro DOS v5.0
42857 ; (PCDOS 7.1 IBMDOS.COM)
42858 ; ;
42859 000074C6 E8279E call set_exerr_locus_mem
42860 ; ;
42861 %endif
42862 ; mov al,1
42863 000074C9 B001 MOV AL,error_invalid_function
42864 ; 17/12/2022
42865 ; AllocOperErrj:
42866 ; JMP SYS_RET_ERR
42867 ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42868 ; jmp short dealloc_errj
42869 ; 17/12/2022
42870 000074CB EBE8 jmp short AllocOperErrj
42871 ;
42872 ; 10/03/2024 - Retro DOS v5.0
42873 ; (PCDOS 7.1 IBMDOS.COM)
42874 %if 0
42875 ; AllocArenaError:
42876 ; MSDOS 6.0
42877 ; MOV byte [ss:EXTERR_LOCUS],errLOC_Mem
42878 ; ; M009: Extended Error Locus
42879 ; error error_arena_trashed ; M009:
42880 ; mov al,7
42881 ; MOV AL,error_arena_trashed
42882 ; JMP SYS_RET_ERR
42883 ; jmp short AllocOperErrj ; 17/12/2022
42884 %endif
42885 ;
42886 ; AllocGetStrat:
42887 ; MSDOS 6.0
42888 ; AllocOperGet:
42889 000074CD 36A0[0203] MOV AL,[SS:AllocMethod]
42890 000074D1 30E4 XOR AH,AH

```

```

42891         ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42892         ;transfer SYS_RET_OK
42893 AllocOperOk:
42894         ; 17/12/2022
42895         ;jmp short dealloc_ok
42896 000074D3 E99891 JMP SYS_RET_OK
42897
42898 AllocSetStrat:
42899         ; 14/05/2019
42900         ; MSDOS 6.0
42901 000074D6 53 push bx ; M000 - start
42902         ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42903         ;and bx,STRAT_MASK ; 0FF3Fh; M064: mask off bit 6 & 7
42904         ; 17/12/2022
42905 000074D7 80E33F and bl,3Fh
42906 000074DA 83FB02 cmp bx,2 ; BX must be 0-2
42907         ;cmp bl,2
42908 000074DD 5B pop bx ; M000 - end
42909 000074DE 77E6 ja short AllocOperError
42910
42911 AllocOperSet:
42912 000074E0 36881E[0203] MOV [SS:AllocMethod],BL
42913         ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42914         ;transfer SYS_RET_OK
42915 AllocOperOkj:
42916 000074E5 EBEC jmp short AllocOperOk
42917         ;JMP SYS_RET_OK
42918
42919 AllocGetLink:
42920         ; MSDOS 6.0
42921 000074E7 36A0[8900] mov al,[ss:UMBFLAG] ; return link state in al
42922         ;and al,1
42923 000074EB 2401 and al,LINKSTATE
42924         ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42925         ;transfer SYS_RET_OK
42926 AllocOperOkj2:
42927         ; 17/12/2022
42928 000074ED EBE4 jmp short AllocOperOk
42929         ;jmp short AllocOperOkj
42930         ;;JMP SYS_RET_OK
42931
42932 AllocSetLink:
42933         ; MSDOS 6.0
42934 000074EF 368B0E[8C00] mov cx,[ss:UMB_HEAD] ; cx = umb_head
42935         ;
42936         ; 10/03/2024
42937 %if 0
42938         cmp cx,0FFFFh ; Q: has umb_head been initialized
42939         je short AllocOperError ; N: error
42940         ; Y: continue
42941         ; M009 - end
42942
42943         cmp bx,1
42944         jb short UnlinkUmbs ; 0
42945         jz short LinkUmbs ; 1
42946 %else
42947         ; 10/03/2024 - Retro DOS v5.0
42948         ; (PCDOS 7.1 IBMDOS.COM)
42949         ;;
42950         inc cx ; Q: has umb_head been initialized
42951         jz short AllocOperError ; -1 ; N: error
42952         dec cx
42953         dec bx
42954         jz short LinkUmbs ; 1
42955         inc bx
42956         ;jz short UnlinkUmbs ; 0
42957         ;;
42958 %endif
42959         ;jmp short AllocOperError
42960         ; 10/03/2024
42961         jnz short AllocOperError ; > 1
42962
42963 UnlinkUmbs:
42964         ; 10/03/2024
42965 %if 0
42966         ;test byte [ss:UMBFLAG],1
42967         test byte [ss:UMBFLAG],LINKSTATE ; Q: umbs unlinked?
42968         jz short unlinked ; Y: return
42969
42970         call GetLastArena ; get arena before umb_head in DS
42971         jc short AllocArenaError ; M009: arena trashed
42972 %else
42973         ; 10/03/2024 - Retro DOS v5.0
42974         ; (PCDOS 7.1 IBMDOS.COM)
42975         ;;
42976 000074FE E89BFD call test_umb_flag ; test byte [ss:UMBFLAG],LINKSTATE
42977         ; Q: umbs unlinked?
42978         jz short unlinked ; Y: return
42979 00007501 740E call GetLastArena ; get arena before umb_head in DS
42980         ;;
42981 %endif
42982         ; make it last
42983 00007506 C60600005A mov byte [0],arena_signature_end
42984
42985         ;and byte [ss:UMBFLAG],0FEh
42986 0000750B 368026[8900]FE and byte [ss:UMBFLAG],~LINKSTATE ; indicate unlink'd state in umbflag
42987
42988 unlinked:
42989         ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
42990         ;transfer SYS_RET_OK
42991         ; 17/12/2022
42992 00007511 EBC0 jmp short AllocOperOk
42993         ;jmp short AllocOperOkj2
42994         ;;JMP SYS_RET_OK
42995
42996 LinkUmbs:
42997         ; 10/03/2024
42998 %if 0
42999         test byte [ss:UMBFLAG],LINKSTATE ; Q: umbs linked?
43000         jnz short linked ; Y: return
43001
43002         call GetLastArena ; get arena before umb_head
43003         jc short AllocArenaError ; M009: arena trashed
43004
43005         ; make it normal. M061: ds points to
43006         ; arena before umb_head
43007 %else
43008         ; 10/03/2024 - Retro DOS v5.0
43009         ; (PCDOS 7.1 IBMDOS.COM)
43010         ;;
43011 00007513 E886FD call test_umb_flag ; Q: umbs linked?
43012 00007516 750E jnz short linked ; Y: return
43013 00007518 E80D00 call GetLastArena ; get arena before umb_head
43014         ;;

```

```

43015 %endif
43016 0000751B C60600004D mov byte [0],arena_signature_normal
43017
43018 00007520 36800E[8900]01 or byte [ss:UMBFLAG],LINKSTATE ; indicate link'd state in umbflag
43019 linked:
43020 ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
43021 ;transfer SYS_RET_OK
43022 ; 17/12/2022
43023 00007526 EBAB jmp short AllocOperOk
43024 ;jmp short unlinked
43025 ;;JMP SYS_RET_OK
43026
43027 ; MSDOS 6.0
43028 -----
43029 ; Procedure Name : GetLastArena - M003
43030
43031 ; Inputs : cx = umb_head
43032 ;
43033 ;
43034 ; Outputs : If UMBs are linked
43035 ; ES = umb_head
43036 ; DS = arena before umb_head
43037 ; else
43038 ; DS = last arena
43039 ; ES = next arena. will be umb_head if NC.
43040 ;
43041 ; CY if error
43042 ;
43043 ; Uses : DS, ES, DI, BX
43044 -----
43045
43046 ; 14/05/2019 - Retro DOS v4.0
43047 ; DOSCODE:A4D6h (MSDOS 6.21, MSDOS.SYS)
43048
43049 ; 01/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
43050 ; DOSCODE:A476h (MSDOS 5.0, MSDOS.SYS)
43051
43052 ; 10/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
43053 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0B6D9h
43054
43055 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0A4D6h)
43056 ; (Windows ME IO.SYS - BIOSCODE:9F25h)
43057
43058 GetLastArena:
43059 00007528 50 push ax ; save ax
43060
43061 00007529 36A1[2400] mov ax,[ss:arena_head]
43062 0000752D 8EC0 mov es,ax ; es = arena_head
43063 0000752F 31FF xor di,di
43064
43065 00007531 26803D5A cmp byte [es:di],arena_signature_end
43066 ; Q: is this the last arena
43067 00007535 7416 je short GLA_done ; Y: return last arena in ES
43068
43069 GLA_next:
43070 00007537 8ED8 mov ds,ax
43071 00007539 E899FD call arena_next ; ax, es -> next arena
43072 ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
43073 ;jc short GLA_err
43074 ; 17/12/2022
43075 0000753C 7222 jc short GLA_err2
43076
43077 ; 10/03/2024
43078 %if 0
43079 test byte [ss:UMBFLAG],LINKSTATE ; Q: are UMBs linked
43080 jnz short GLA_chkumb ; Y: terminating condition is
43081 ; umb_head
43082 ; N: terminating condition is 05Ah
43083
43084 %else
43085 ; 10/03/2024 (PCDOS 7.1 IBMDOS.COM)
43086 ;;;
43087 call test_umb_flag
43088 jnz short GLA_chkumb
43089 ;;;
43090 %endif
43091 cmp byte [es:di],arena_signature_end
43092 ; Q: is this the last arena
43093 jmp short GLA_@f
43094 GLA_chkumb:
43095 cmp ax,cx ; Q: is this umb_head
43096 0000754B 75EA jne short GLA_next ; N: get next arena
43097
43098 GLA_done:
43099
43100 ; 10/03/2024
43101 %if 0
43102 test byte [ss:UMBFLAG],LINKSTATE ; M061 - Start
43103 jnz short GLA_ret ; Q: are UMBs linked
43104 ; Y: we're done
43105 ; N: let us confirm that the next
43106 ; arena is umb_head
43107
43108 %else
43109 ; 10/03/2024 (PCDOS 7.1 IBMDOS.COM)
43110 ;;;
43111 call test_umb_flag
43112 jnz short GLA_ret ; cf=0
43113 ;;;
43114 %endif
43115 mov ds,ax
43116 call arena_next ; ax, es -> next arena
43117 ;jc short GLA_err
43118 jc short GLA_err2
43119 cmp ax,cx ; Q: is this umb_head
43120 jne short GLA_err ; N: error
43121 ; M061 - End
43122
43123 GLA_ret:
43124 ; 17/12/2022
43125 ;clc
43126 ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
43127 ;clc
43128 pop ax ; M061
43129 retn ; M061
43130
43131 GLA_err:
43132 stc ; M061
43133 GLA_err2:
43134 pop ax
43135
43136 ; 10/03/2024 - Retro DOS v5.0
43137 ; (PCDOS 7.1 IBMDOS.COM)
43138 ;;;
43139 pop ax ; return address to _$ALLOCOPER (the caller)
43140 call set_exerr_locus_mem

```

```

43139 00007565 E910FE      jmp     alloc_trashed2 ; (error return from _$ALLOCOPER)
43140                      ;;;
43141
43142                      ; 10/03/2024
43143                      ;retn
43144
43145                      ;=====
43146                      ; SRVCALL.ASM, MSDOS 6.0, 1991
43147                      ;=====
43148                      ; 04/08/2018 - Retro DOS v3.0
43149
43150                      ; TITLE SRVCALL - Server DOS call
43151                      ; NAME SRVCALL
43152
43153                      ;** SRVCALL.ASM - Server DOS call functions
43154                      ;
43155                      ;
43156                      ; $ServerCall
43157                      ;
43158                      ; Modification history:
43159                      ;
43160                      ; Created: ARR 08 August 1983
43161
43162                      ;AsmVars <Installed>
43163
43164                      ;include dpl.asm
43165
43166                      ;Installed = TRUE
43167
43168                      ; 29/04/2019 - Retro DOS v4.0 (MSDOS 6.0, MSDOS 6.21)
43169                      ; -----
43170                      ; 01/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
43171
43172                      ;BREAK <ServerCall -- Server DOS call>
43173
43174                      ; DOSCODE:A517h (MSDOS 6.21, MSDOS.SYS)
43175                      ; DOSCODE:A4B7h (MSDOS 5.0, MSDOS.SYS)
43176
43177                      ; 10/03/2024
43178                      ; DOSCODE:B71Ah (PCDOS 7.1, IBMDOS.COM) ; Retro DOS v5.0
43179                      ; DOSCODE:A517h (MSDOS 6.22, MSDOS.SYS) ; Retro DOS v4.2
43180                      ; BIOSCODE:9F66h (Windows ME, IO.SYS)
43181
43182                      ;hkn; TABLE SEGMENT
43183                      ;Public SRVC001S,SRVC001E
43184                      ;SRVC001S label byte
43185
43186                      SRVC001S:
43187
43188 00007568 [6C75]      SERVERTAB: dw     SERVER_DISP
43189 0000756A [BB75]      SERVERLEAVE: dw     SERVERRETURN
43190 0000756C 0B          SERVER_DISP: db     (SERVER_DISP_END-SERVER_DISP-1)/2 ; = 11
43191 0000756D [2176]          dw     SRV_CALL      ; 0
43192 0000756F [BC75]          dw     COMMIT_ALL   ; 1
43193 00007571 [F275]          dw     CLOSE_NAME    ; 2
43194 00007573 [FB75]          dw     CLOSE_UID     ; 3
43195 00007575 [0276]          dw     CLOSE_UID_PID ; 4
43196 00007577 [0976]          dw     GET_LIST     ; 5
43197 00007579 [6276]          dw     GET_DOS_DATA  ; 6
43198 0000757B [8676]          dw     SPOOL_OPER   ; 7
43199 0000757D [8676]          dw     SPOOL_OPER   ; 8
43200 0000757F [8676]          dw     SPOOL_OPER   ; 9
43201 00007581 [9276]          dw     _$SetExtendedError ; 10
43202
43203                      SERVER_DISP_END: ; LABEL BYTE
43204
43205                      ;SRVC001E label byte
43206
43207                      SRVC001E:
43208
43209                      ;hkn; TABLE ENDS
43210
43211                      ; -----
43212                      ;
43213                      ; Procedure Name : $ServerCall
43214                      ;
43215                      ; Inputs:
43216                      ; DS:DX -> DPL (except calls 7,8,9)
43217                      ; Function:
43218                      ; AL=0 Server DOS call
43219                      ; AL=1 Commit All files
43220                      ; AL=2 Close file by name (SHARING LOADED ONLY) DS:DX in DPL -> name
43221                      ; AL=3 Close all files for DPL_UID
43222                      ; AL=4 Close all files for DPL_UID/PID_PID
43223                      ; AL=5 Get open file list entry
43224                      ; IN: BX File Index
43225                      ; CX User Index
43226                      ; OUT:ES:DI -> Name
43227                      ; BX = UID
43228                      ; CX = # locked blocks held by this UID
43229                      ; AL=6 Get DOS data area
43230                      ; OUT: DS:SI -> Start
43231                      ; CX size in bytes of swap if indos
43232                      ; DX size in bytes of swap always
43233                      ; AL=7 Get truncate flag
43234                      ; AL=8 Set truncate flag
43235                      ; AL=9 Close all spool files
43236                      ; AL=10 SetExtendedError
43237                      ; -----
43238
43239                      ; 10/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
43240                      ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0B735h
43241
43242                      _$ServerCall:
43243                      ; 13/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
43244                      ; DOSCODE:A4D2h (MSDOS 5.0 MSDOS.SYS)
43245                      ; 10/06/2019
43246                      ; 29/04/2019 - Retro DOS v4.0
43247                      ; DOSCODE:A532h (MSDOS 6.21 MSDOS.SYS)
43248
43249                      ; 05/08/2018 - Retro DOS v3.0
43250                      ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 657Bh
43251
43252 00007583 3C07      CMP     AL,7
43253 00007585 7204      JB      short SET_STUFF
43254 00007587 3C09      CMP     AL,9
43255 00007589 761A      JBE     short NO_SET_ID      ; No DPL on calls 7,8,9
43256
43257 0000758B 89D6      SET_STUFF: MOV     SI,DX      ; Point to DPL with DS:SI
43258                      ;mov     bx,[si+12h]
43259 0000758D 8B5C12     MOV     BX,[SI+DPL.UID]
43260
43261                      ; MSDOS 6.0
43262                      ;SR;

```

```

43263 ; WIN386 updates the USER_ID itself. If WIN386 is present we skip the updating
43264 ; of USER_ID
43265
43266 00007590 36F606[5B0F]01 test byte [SS:Iswin386],1
43267 00007596 7505 jnz short skip_win386
43268
43269 ;hkn; SS override for user_id and proc_id
43270 ; 15/08/2018
43271 00007598 36891E[3E03] MOV [SS:USER_ID],BX ; Set UID
43272
43273 skip_win386:
43274 0000759D 8B5C14 MOV BX,[SI+DPL.PID]
43275 000075A0 36891E[3C03] MOV [SS:PROC_ID],BX ; Set process ID
43276
43277 NO_SET_ID:
43278 000075A5 2EFF36[6A75] ; 10/06/2019 - Retro DOS v4.0
43279 000075AA 2EFF36[6875] PUSH word [cs:SERVERLEAVE] ; push return address
43280 000075AF 50 PUSH word [cs:SERVERTAB] ; push table address
43281 000075B0 E84BA2 PUSH AX
43282 call TableDispatch
43283
43284 ;hkn; SS override
43285 ;;mov byte [SS:EXETERR_LOCUS],1
43286 ;MOV byte [SS:EXTERR_LOCUS],errLOC_Unk ; Extended Error Locus
43287 000075B3 E8279D ; 01/07/2024
43288 call set_exerr_locus_unk
43289
43290 ;error error_invalid_function
43291 000075B6 B001 ;mov al,1
43292 MOV AL,error_invalid_function
43293 000075B8 E9BD90 servercall_error:
43294 JMP SYS_RET_ERR
43295
43296 000075BB C3 SERVERRETURN:
43297 retn
43298
43299 ; Commit - iterate through the open file list and make sure that the
43300 ; directory entries are correctly updated.
43301
43302 ; 01/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
43303 000075BC 31DB COMMIT_ALL:
43304 000075BE 16 XOR BX,BX ; for (i=0; ThisSFT=getSFT(i); i++)
43305 000075BF 1F push ss
43306 000075C0 E81FA3 pop ds
43307 call ECritSFT ; Gonna scan SFT cache, lock it down
43308 000075C3 53 CommitLoop:
43309 000075C4 E82201 push bx
43310 000075C7 7222 call SFFFromSFN
43311 000075C9 26833D00 JC short CommitDone
43312 cmp word [es:di],0
43313 ;CMP word [ES:DI+SF_ENTRY.sf_Ref_Count],0
43314 000075CD 7418 ; if (ThisSFT->refcount != 0)
43315 JZ short CommitNext
43316 000075CF 26833DFF ;cmp word [es:di],0FFFFh ; -1
43317 cmp word [ES:DI],sf_busy
43318 ;CMP word [ES:DI+SF_ENTRY.sf_Ref_Count],sf_busy
43319 000075D3 7412 ; BUSY SFTs have god knows what
43320 JZ short CommitNext ; in them.
43321 000075D5 26F6450680 ; 17/12/2022
43322 test byte [ES:DI+SF_ENTRY.sf_flags+1],(sf_isnet>>8) ; 80h
43323 000075DA 750B ;TEST word [ES:DI+SF_ENTRY.sf_flags],sf_isnet ; 8000h
43324 JNZ short CommitNext ; Skip Network SFTs so the SERVER
43325 000075DC 893E[9E05] ; doesn't deadlock
43326 000075E0 8C06[A005] MOV [THISST],DI
43327 000075E4 E8F0C2 MOV [THISST+2],ES
43328 call DOS_COMMIT ; DOSCommit ();
43329 000075E7 5B CommitNext:
43330 000075E8 43 pop bx
43331 000075E9 EBD8 INC BX
43332 JMP short CommitLoop
43333 000075EB E821A3 CommitDone:
43334 000075EE 5B call LCritSFT
43335 pop bx
43336 ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
43337 000075EF E97C90 Commit_Ok:
43338 jmp SYS_RET_OK
43339
43340 CLOSE_NAME:
43341
43342 ;if installed
43343
43344 ;hkn; SS override
43345 000075F2 36FF1E[A400] ;call far [ss:MFTclon]
43346 call far [SS:JShare+(5*4)] ; 5 = MFTclon
43347 ;else
43348 ; Call MFTclon
43349 ;endif
43350
43351 CheckReturns:
43352 ; 10/03/2024
43353 %if 0
43354 JC short func_err
43355 ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
43356 ;transfer SYS_RET_OK
43357 Commit_Okj:
43358 jmp short Commit_Ok
43359 ;jmp SYS_RET_OK
43360 %else
43361 000075F7 73F6 jnc short Commit_Ok
43362 %endif
43363
43364 func_err:
43365 ;transfer SYS_RET_ERR
43366 ;jmp SYS_RET_ERR
43367 000075F9 EBB0 jmp short servercall_error
43368
43369 CLOSE_UID:
43370
43371 ;if installed
43372 ;hkn; SS override
43373 ;call far [ss:MFTclu]
43374 000075FB 36FF1E[9C00] call far [SS:JShare+(3*4)] ; 3 = MFTclu
43375 ;else
43376 ; Call MFTclu
43377 ;endif
43378 00007600 EBF5 JMP short CheckReturns
43379
43380 CLOSE_UID_PID:
43381
43382 ;if installed
43383 ;hkn; SS override
43384 ;call far [ss:MFTcloseP]
43385 00007602 36FF1E[A000] call far [SS:JShare+(4*4)] ; 4 = MFTcloseP
43386 ;else

```



```

43387 ; Call MFTCloseP
43388 ;endif
43389 00007607 EBEE JMP short CheckReturns
43390
43391 GET_LIST:
43392
43393 ;if installed
43394 ;hkn; SS override
43395 ;call far [ss:MFT_get]
43396 00007609 36FF1E[B400] Call far [SS:JShare+(9*4)] ; 9 = MFT_get
43397 ;else
43398 ; Call MFT_get
43399 ;endif
43400 0000760E 72E9 JC short func_err
43401 00007610 E8648E call Get_User_Stack
43402 ;mov [si+2],bx
43403 00007613 895C02 MOV [SI+user_env.user_BX],BX
43404 ;mov [si+10],di
43405 00007616 897C0A MOV [SI+user_env.user_DI],DI
43406 ;mov [si+16],es
43407 00007619 8C4410 MOV [SI+user_env.user_ES],ES
43408 SetCXOK:
43409 ;mov [si+4],cx
43410 0000761C 894C04 MOV [SI+user_env.user_CX],CX
43411 ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
43412 ;transfer SYS_RET_OK
43413 Commit_Ok2:
43414 ; 17/12/2022
43415 0000761F EBCE jmp short Commit_Ok
43416 ;jmp short Commit_Ok2
43417 ;;jmp SYS_RET_OK
43418
43419 SRV_CALL:
43420 00007621 58 POP AX ; get rid of call to $srvcall
43421 00007622 1E push ds
43422 00007623 56 push si
43423 00007624 E8508E call Get_User_Stack
43424 00007627 5F pop di
43425 00007628 07 pop es
43426
43427 ; DS:SI point to stack
43428 ; ES:DI point to DPL
43429
43430 00007629 E8B1A1 call XCHGP
43431
43432 ; DS:SI point to DPL
43433 ; ES:DI point to stack
43434 ;
43435 ;
43436 ; we now copy the registers from DPL to save stack
43437 0000762C 56 push si
43438 0000762D B90600 MOV CX,6
43439 00007630 F3A5 REP MOVSW ; Put in AX,BX,CX,DX,SI,DI
43440 00007632 47 INC DI
43441 00007633 47 INC DI ; Skip user_BP
43442 00007634 A5 MOVSW ; DS
43443 00007635 A5 MOVSW ; ES
43444 00007636 5E pop si ; DS:SI -> DPL
43445 00007637 8B04 mov ax,[SI]
43446 ;MOV AX,[SI+DPL.AX]
43447 ;mov bx,[si+2]
43448 00007639 8B5C02 MOV BX,[SI+DPL.BX]
43449 ;mov cx,[si+4]
43450 0000763C 8B4C04 MOV CX,[SI+DPL.CX]
43451 ;mov dx,[si+6]
43452 0000763F 8B5406 MOV DX,[SI+DPL.DX]
43453 ;mov di,[si+10]
43454 00007642 8B7C0A MOV DI,[SI+DPL.DI]
43455 ;mov es,[si+14]
43456 00007645 8E440E MOV ES,[SI+DPL.ES]
43457 ;push word [si+8]
43458 00007648 FF7408 PUSH word [SI+DPL.SI]
43459 ;mov ds,[si+12]
43460 0000764B 8E5C0C MOV DS,[SI+DPL.DS]
43461 0000764E 5E POP SI
43462
43463 ;hkn; SS override for next 3
43464 0000764F 368C1E[EC05] MOV [SS:SAVEDS],DS
43465 00007654 36891E[EA05] MOV [SS:SAVEBX],BX
43466 00007659 36C606[7205]FF MOV byte [SS:FSHARING],-1 ; set no redirect flag
43467 0000765F E9138D jmp REDISP
43468
43469 GET_DOS_DATA:
43470 00007662 16 push ss
43471 00007663 07 pop es
43472 00007664 8F[2003] MOV DI,SWAP_START
43473 00007667 B9[FA0A] MOV CX,SWAP_END
43474 0000766A BA[3A03] MOV DX,SWAP_ALWAYS
43475 0000766D 29F9 SUB CX,DI
43476 0000766F 29FA SUB DX,DI
43477 00007671 D1E9 SHR CX,1 ; div by 2, remainder in carry
43478 00007673 83D100 ADC CX,0 ; div by 2 + round up
43479 00007676 D1E1 SHL CX,1 ; round up to 2 boundary.
43480 00007678 E8FC8D call Get_User_Stack
43481 ;mov [si+14],es
43482 0000767B 8C440E MOV [SI+user_env.user_DS],ES
43483 ;mov [si+8],di
43484 0000767E 897C08 MOV [SI+user_env.user_SI],DI
43485 ;mov [si+6],dx
43486 00007681 895406 MOV [SI+user_env.user_DX],DX
43487 00007684 EB96 JMP short SetCXOK
43488
43489 SPOOL_OPER:
43490 ;CallInstall NETSPOOLoper,MultNET,37,AX,BX
43491
43492 00007686 50 push ax
43493 00007687 B82511 mov ax,1125h
43494 0000768A CD2F int 2Fh ; Multiplex - NETWORK REDIRECTOR - REDIRECTED PRINTER MODE
43495 ; STACK: WORD subfunction
43496 ; Return: CF set on error, AX = error code
43497 ; STACK unchanged
43498 0000768C 5B pop bx
43499 ; 17/12/2022
43500 ;JC short func_err2
43501 0000768D 7390 Jnc short Commit_Ok2
43502 ; 01/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
43503 ;;jmp SYS_RET_OK
43504 ;jmp short Commit_Ok2
43505
43506 func_err2:
43507 0000768F E9E68F jmp SYS_RET_ERR
43508
43509 ;Break <$SetExtendedError - set extended error for later retrieval>
43510 ;-----

```

```

43511 ;
43512 ; Procedure Name : $SetExtendedError
43513 ;
43514 ; $SetExtendedError takes extended error information and loads it up for the
43515 ; next extended error call. This is used by interrupt-level processors to
43516 ; mask their actions.
43517 ;
43518 ; Inputs: DS:SI points to DPL which contains all registers
43519 ; Outputs: none
43520 ;
43521 ;-----
43522 _$SetExtendedError:
43523 ;hkn; SS override for all variables used
43524
43525 mov ax,[si]
43526 ;MOV AX,[SI+DPL.AX]
43527 00007692 8B04 MOV [SS:EXTERR],AX
43528 00007694 36A3[2403] ;mov ax,[si+10]
43529 MOV AX,[SI+DPL.DI]
43530 00007698 8B440A MOV [SS:EXTERRPT],AX
43531 0000769B 36A3[2803] ;mov ax,[si+14]
43532 0000769F 8B440E MOV AX,[SI+DPL.ES]
43533 000076A2 36A3[2A03] MOV [SS:EXTERRPT+2],AX
43534 000076A6 8B4402 ;mov ax,[si+2]
43535 000076A9 36A3[2603] MOV AX,[SI+DPL.BX]
43536 000076AD 8B4404 MOV [SS:EXTERR_ACTION],AX
43537 000076B0 368826[2303] ;mov ax,[si+4]
43538 000076B5 C3 MOV AX,[SI+DPL.CX]
43539 retn MOV [SS:EXTERR_LOCUS],AH
43540
43541 ;=====
43542 ; UTIL.ASM, MSDOS 6.0, 1991
43543 ;=====
43544 ; 05/08/2018 - Retro DOS v3.0
43545 ; 05/05/2019 - Retro DOS v4.0
43546 ; 11/03/2024 - Retro DOS v5.0
43547
43548 ;** Handle related utilities for MSDOS 2.X.
43549 ;-----
43550 pJFNFromHandle written
43551 SFFromHandle written
43552 SFFromSFN written
43553 JFNFree written
43554 SFNFree written
43555
43556 Modification history:
43557
43558 Created: MZ 1 April 1983
43559 ;-----
43560 ; BREAK <pJFNFromHandle - return pointer to JFN table entry>
43561
43562 ;** pJFNFromHandle - Translate Handle to Pointer to JFN
43563 ;-----
43564 pJFNFromHandle takes a file handle and turns that into a pointer to
43565 the JFN entry (i.e., to a byte holding the internal file handle #)
43566
43567 NOTE:
43568 This routine is called from $CREATE_PROCESS_DATA_BLOCK which is called
43569 at DOSINIT time with SS NOT DOSGROUP
43570
43571 ENTRY (bx) = handle
43572 EXIT 'C' clear if ok
43573 (es:di) = address of JFN value
43574 'C' set if error
43575 (ax) = error code
43576 USES AX, DI, ES, Flags
43577 ;-----
43578 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
43579 ; 11/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
43580
43581 pJFNFromHandle:
43582 ; 05/05/2019 - Retro DOS v4.0
43583 ;getdseg <es> ; es -> dosdata
43584 mov es,[cs:DosDSeg]
43585
43586 ;MOV ES,[cs:CurrentPDB] ; get user process data block
43587 mov es,[es:CurrentPDB]
43588
43589 ;cmp bx,[ES:32h]
43590 CMP BX,[ES:PDB.JFN_Length]; is handle greater than allocated
43591 JB short pjfn10 ; no, get offset
43592 ReturnCarry_inv_hnd1: ; 05/08/2018 - Retro DOS v3.0
43593 ;mov al,6
43594 MOV AL,error_invalid_handle ; appropriate error
43595 ReturnCarry:
43596 STC ; signal error
43597 retn ; go back
43598
43599 pjfn10:
43600 ;les di,[es:34h]
43601 LES DI,[ES:PDB.JFN_Pointer] ; get pointer to beginning of table
43602 ADD DI,BX ; add in offset, clear 'C'
43603 ;clc
43604 pJFNFromHandle_error:
43605 retn
43606
43607 ;BREAK <SFFromHandle - return pointer (or error) to SF entry from handle>
43608 ;-----
43609 ;
43610 ; Procedure Name : SFFromHandle
43611 ;
43612 ; SFFromHandle - Given a handle, get JFN and then index into SF table
43613 ;
43614 ; Input: BX has handle
43615 ; Output: Carry Set
43616 ; AX has error code
43617 ; Carry Reset
43618 ; ES:DI has pointer to SF entry
43619 ; Registers modified: If error, AX,ES, else ES:DI
43620 ; NOTE:
43621 ; This routine is called from $CREATE_PROCESS_DATA_BLOCK which is called
43622 ; at DOSINIT time with SS NOT DOSGROUP
43623 ;-----
43624 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
43625 ; 11/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
43626
43627 SFFromHandle:
43628 CALL pJFNFromHandle ; get jfn pointer
43629
43630
43631
43632
43633
43634 000076D3 E8E0FF

```



```

43635 ;retc ; return if error
43636 000076D6 72FA jc short pJFNFromHandle_error
43637 000076D8 26803DFF CMP BYTE [ES:DI],-1 ; unused handle
43638 ;JNZ short GetSF ; nope, suck out SF
43639 ;;mov al,6
43640 ;MOV AL,error_invalid_handle ; appropriate error
43641 ;jmp short ReturnCarry ; signal it
43642 ; 17/12/2022
43643 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
43644 000076DC 74E9 jz short ReturnCarry_inv_hndl ; Retro DOS v3.0 modification
43645 ;JNZ short GetSF ; nope, suck out SF
43646 ;;mov al,6
43647 ;MOV AL,error_invalid_handle ; appropriate error
43648 ;jmp short ReturnCarry ; signal it
43649 GetSF:
43650 000076DE 53 push bx ; save handle
43651 000076DF 268A1D MOV BL,[ES:DI] ; get SFN
43652 000076E2 30FF XOR BH,BH ; ignore upper half
43653 000076E4 E80200 CALL SFFFromSFN ; get real sf spot
43654 000076E7 5B pop bx ; restore
43655 000076E8 C3 retn ; say goodbye
43656
43657 ;BREAK <SFFFromSFN - index into SF table for SFN>
43658
43659 ;** SFFFromSFN - Get an SF Table entry from an SFN
43660 ;-----
43661 ; SFFFromSfn uses an SFN to index an entry into the SF table. This
43662 ; is more than just a simple index instruction because the SF table
43663 ; can be made up of multiple pieces chained together. We follow the
43664 ; chain to the right piece and then do the index operation.
43665 ;
43666 ; NOTE:
43667 ; This routine is called from SFFFromHandle which is called
43668 ; at DOSINIT time with SS NOT DOSGROUP
43669 ;
43670 ; ENTRY BX has SF index
43671 ; EXIT 'C' clear if OK
43672 ; ES:DI points to SF entry
43673 ; 'C' set if index too large
43674 ; USES BX, DI, ES
43675 ;-----
43676
43677 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
43678 ; 11/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
43679
43680 SFFFromSFN:
43681 ; 05/05/2019 - Retro DOS v4.0
43682 ;getdseg <es> ; es -> dosdata
43683 000076E9 2E8E06[0700] mov es,[cs:DosDSeg]
43684
43685 ;LES DI,[CS:SFT_ADDR] ; (es:di) = start of SFT table
43686 000076EE 26C43E[2A00] les di,[es:SFT_ADDR]
43687
43688 sfsfn5:
43689 000076F3 263B5D04 ;cmp bx,[es:di+4]
43690 000076F7 720E CMP BX,[ES:DI+SFT.SFCount]; is handle in this table?
43691 ;JB short sfsfn7 ; yes, go grab it
43692 000076F9 262B5D04 ;sub bx,[es:di+4]
43693 000076FD 26C43D SUB BX,[ES:DI+SFT.SFCount]
43694 ;les di,[es:di] ; 14/08/2018
43695 00007700 83FFFF ;LES DI,[ES:DI+SFT.SFLink] ; get next table segment
43696 00007703 75EE CMP DI,-1 ; end of tables?
43697 00007705 F9 JNZ short sfsfn5 ; no, try again
43698 00007706 C3 STC
43699 ;retn ; return with error, not found
43700 00007707 50
43701 ;push ax
43702 ;;mov ax,53 ; MSDOS 3.3
43703 ;mov ax,59 ; MSDOS 5.0 to PCDOS 7.1 (to win ME) ; 11/03/2024
43704 ;MOV AX,SF_ENTRY.size ; put it in a nice place
43705
43706 ; 17/12/2022
43707 00007708 B03B mov al,SF_ENTRY.size ; 28/05/2019
43708 ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
43709 ;mov ax,SF_ENTRY.size ; 59
43710 0000770A F6E3 MUL BL ; (ax) = offset into this SF block
43711 0000770C 01C7 ADD DI,AX ; add base of SF block
43712 0000770E 58 pop ax
43713 ;add di,6
43714 0000770F 83C706 ADD DI,SFT.SFTable ; offset into structure, 'C' cleared
43715 00007712 C3 retn ; return with 'C' clear
43716
43717 ; BREAK <JFNFree - return a jfn pointer if one is free>
43718
43719 ;** JFNFree - Find a Free JFN Slot
43720 ;-----
43721 ; JFNFree scansthrough the JFN table and returns a pointer to a free slot
43722 ;
43723 ; ENTRY (ss) = DOSDATA
43724 ; EXIT 'C' clear if OK
43725 ; (bx) = new handle
43726 ; (es:di) = pointer to JFN slot
43727 ; 'C' set if error
43728 ; (al) = error code
43729 ; USES bx, di, es, flags
43730 ;-----
43731
43732 JFNFree:
43733 00007713 31DB XOR BX,BX ; (bx) = initial JFN to try
43734
43735 jfnf1:
43736 00007715 E89EFF CALL pJFNFromHandle ; get the appropriate handle
43737 0000771A 26803DFF JC short jfnf5 ; no more handles
43738 0000771E 7405 CMP BYTE [ES:DI],-1 ; free?
43739 00007720 43 JE short jfnfx ; yes, carry is clear
43740 00007721 EBF2 INC BX ; no, next handle
43741 ;JMP short jfnf1 ; and try again
43742
43743 ; Error. 'C' set
43744 jfnf5:
43745 00007723 B004 ;mov al,4
43746 ;MOV AL,error_too_many_open_files
43747 00007725 C3 jfnfx:
43748 ;retn ; bye
43749
43750 ; BREAK <SFNFree - Allocate a free SFN>
43751
43752 ;** SFNFree - Allocate a Free SFN/SFT
43753 ;-----
43754 ; SFNFree scans through the sf table looking for a free entry
43755 ; If it finds one it partially allocates it by setting SFT_REF_COUNT = -1
43756 ;
43757 ; The problem is that we want to mark the SFT busy so that other threads
43758 ; can't allocate the SFT before we're finished marking it up. However,
43759 ; we can't just mark it busy because we may get blown out of our open

```

```

43759 ; by INT24 and leave the thing orphaned. To solve this we mark it
43760 ; "allocation in progress" by setting SFT_REF_COUNT = -1. If we see
43761 ; an SFT with this value we look to see if it belongs to this user
43762 ; and process. If it does belong to us then it must be an orphan
43763 ; and we reclaim it.
43764 ;
43765 ; BUGBUG - improve the performance. I guess it's smaller to call SFFromSFN
43766 ; over and over, but we could at least set a high water mark...
43767 ; cause an N^2 loop calling slow SFFromSFN is real slow, too slow
43768 ; even though this is not a frequently called routine - jgl
43769 ;
43770 ; ENTRY (ss) = DOSDATA
43771 ; EXIT 'C' clear if no error
43772 ; (bx) = SFN
43773 ; (es:di) = pointer to SFT
43774 ; es:[di].SFT_REF_COUNT = -1
43775 ; 'C' set if error
43776 ; (al) = error code
43777 ; USES bx, di, es, Flags
43778 ;-----
43779 ;
43780 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
43781 ; DOSCODE:A682h (MSDOS 5.0 MSDOS.SYS)
43782 ;
43783 ; 11/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
43784 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0B8DEh
43785 ;
43786 SFNFree:
43787 ; 12/08/2018
43788 ; 05/08/2018 - Retro DOS v3.0
43789 ;
43790 ; MSDOS 6.0
43791 00007726 50 push ax
43792 00007727 31DB xor bx,bx ; (bx) = SFN to consider
43793 ;
43794 00007729 53 sfnf5: push bx
43795 0000772A E8BCFF call SFFromSFN ; get the potential handle
43796 0000772D 5B pop bx
43797 0000772E 723A jc short sfnf95 ; no more free SFNs
43798 00007730 26833D00 cmp word [ES:DI],0
43799 ;cmp word [ES:DI+SF_ENTRY.sf_ref_count],0 ; free?
43800 00007734 741D je short sfnf20 ; yep, got one
43801 ;
43802 ;cmp word [es:di],0FFFFh ; -1
43803 00007736 26833DFF cmp word [ES:DI],sf_busy
43804 ;cmp word [ES:DI+SF_ENTRY.sf_ref_count],sf_busy
43805 0000773A 7403 je short sfnf10 ; special busy mark
43806 ;
43807 0000773C 43 sfnf7: inc bx ; try the next one
43808 0000773D EBEA jmp short sfnf5
43809 ;
43810 ; The SFT has the special "busy" mark; if it belongs to us then
43811 ; it was abandoned during a earlier call and we can use it.
43812 ;
43813 ; (bx) = SFN
43814 ; (es:di) = pointer to SFT
43815 ; (TOS) = caller's (ax)
43816 ;
43817 ;
43818 0000773F 36A1[3E03] sfnf10: mov ax,[SS:USER_ID]
43819 ;cmp [es:di+2Fh],ax
43820 00007743 2639452F ;cmp [ES:DI+SF_ENTRY.sf_UID],ax
43821 00007747 75F3 jnz short sfnf7 ; not ours
43822 00007749 36A1[3C03] mov ax,[SS:PROC_ID]
43823 ;cmp [es:di+31h],ax
43824 0000774D 26394531 ;cmp [ES:DI+SF_ENTRY.sf_PID],ax
43825 00007751 75E9 jnz short sfnf7 ; can't use this one, try the next
43826 ;
43827 ; We have an SFT to allocate
43828 ;
43829 ; (bx) = SFN
43830 ; (es:di) = pointer to SFT
43831 ; (TOS) = caller's (ax)
43832 ;
43833 ;
43834 ;
43835 ;
43836 ;
43837 00007753 26C705FFFF sfnf20: ; cf = 0 ;; Retro DOS v3.0
43838 ;mov word [es:di],0FFFFh
43839 mov word [ES:DI],sf_busy
43840 ;mov word [ES:DI+SF_ENTRY.sf_ref_count],sf_busy ; make sure that this is allocated
43841 00007758 36A1[3E03] mov ax,[SS:USER_ID]
43842 ;mov [es:di+2Fh],ax
43843 0000775C 2689452F ;mov [ES:DI+SF_ENTRY.sf_UID],ax
43844 00007760 36A1[3C03] mov ax,[SS:PROC_ID]
43845 ;mov [es:di+31h],ax
43846 00007764 26894531 ;mov [ES:DI+SF_ENTRY.sf_PID],ax
43847 sfnf21: ;; Retro DOS v3.0
43848 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
43849 ;pop ax
43850 ;;clc
43851 ;retn ; return with no error
43852 ; 17/12/2022
43853 00007768 58 pop ax
43854 ;clc
43855 00007769 C3 retn
43856 ;
43857 ;** Error - no more free SFNs
43858 ;
43859 ; 'C' set
43860 ; (TOS) = saved ax
43861 ;
43862 ;
43863 0000776A 58 sfnf95: pop ax
43864 ;
43865 ; 11/03/2024
43866 %if 0
43867 ;mov al,4
43868 mov al,error_too_many_open_files
43869 retn ; return with 'C' and error
43870 %else
43871 ; 11/03/2024 (PCDOS 7.1 IBMDOS.COM)
43872 ;;;
43873 0000776B EBB6 jmp short jfnf5
43874 ;;;
43875 %endif
43876 ;=====
43877 ; HANDLE.ASM, MSDOS 6.0, 1991
43878 ;=====
43879 ;
43880 ; 13/07/2018 - Retro DOS v3.0
43881 ; 20/05/2019 - Retro DOS v4.0
43882 ; 11/03/2024 - Retro DOS v5.0

```

```

43883
43884 ; DOSCODE:A72Bh (MSDOS 6.21, MSDOS.SYS)
43885 ;
43886 ; BREAK <$Close - return a handle to the system>
43887 ;-----
43888 ;
43889 ;** $Close - Close a file Handle
43890 ;
43891 ; BUGBUG - close gets called a LOT with invalid handles - sizzle that
43892 ; path
43893 ;
43894 ; Assembler usage:
43895 ;     MOV     BX, handle
43896 ;     MOV     AH, Close
43897 ;     INT     int_command
43898 ;
43899 ; ENTRY (bx) = handle
43900 ; EXIT  <normal INT21 return convention>
43901 ; USES   all
43902 ;-----
43903 ;
43904 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
43905 ; DOSCODE:A6CBh (MSDOS 5.0 MSDOS.SYS)
43906 ;
43907 ; 11/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
43908 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0B926h
43909 ;
43910 ; (Windows ME IO.SYS - BIOSCODE:0A145h)
43911 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0A72Bh)
43912 ;
43913 ;
43914 ;_SCLOSE:
43915 ; Grab the SFT pointer from the JFN.
43916 ;
43917 0000776D E8F602    call    CheckOwner      ; get system file entry
43918 00007770 722B     jc      short CloseError    ; error return
43919 00007772 16       push    ss
43920 00007773 1F       pop     ds
43921 00007774 893E[9E05] mov     [THISST],DI      ; For DOS_CLOSE
43922 00007778 8C06[A005] mov     [THISST+2],ES    ; save offset of pointer
43923 ;
43924 ; DS:SI point to JFN table entry.
43925 ; ES:DI point to SFT
43926 ;
43927 ; We now examine the user's JFN entry; If the file was a 70-mode file (network
43928 ; FCB, we examine the ref count on the SFT; if it was 1, we free the JFN.
43929 ; If the file was not a net FCB, we free the JFN too.
43930 ;
43931 ;CMP     word [ES:DI+SF_ENTRY.sf_ref_count],1
43932 0000777C 26833D01 cmp     word [ES:DI],1    ; will the SFT become free?
43933 00007780 740A     jz      short FreeJFN        ; yes, free JFN anyway.
43934 ;mov     al,[ES:DI+2]
43935 00007782 268A4502 mov     AL,[ES:DI+SF_ENTRY.sf_mode]
43936 ;and     al,0F0h
43937 00007786 24F0     AND     AL,SHARING_MASK
43938 ;cmp     al,70h
43939 00007788 3C70     CMP     AL,SHARING_NET_FCB
43940 0000778A 7407     JZ      short PostFree      ; 70-mode and big ref count => free it
43941 ;
43942 ; The JFN must be freed. Get the pointer to it and replace the contents with
43943 ; -1.
43944 ;
43945 ;FreeJFN:
43946 0000778C E827FF    call    pJFNFromHandle    ; d = pJFN (handle);
43947 0000778F 26C605FF mov     BYTE [ES:DI],0FFh ; release the JFN
43948 ;PostFree:
43949 ;
43950 ; ThisSFT is correctly set, we have DS = DOSDATA. Looks OK for a DOS_CLOSE!
43951 ;
43952 00007793 E892BF    call    DOS_CLOSE
43953 ;
43954 ; DOS_Close may return an error. If we see such an error, we report it but
43955 ; the JFN stays closed because DOS_Close always frees the SFT!
43956 ;
43957 00007796 7205     JC      short CloseError
43958 ;mov     ah,3Eh
43959 00007798 B43E     MOV     AH,CLOSE        ; MZ Bogus multiplan fix
43960 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
43961 ;CloseOk:
43962 0000779A E9D18E    jmp     SYS_RET_OK
43963 ;CloseError:
43964 ;CommitError: ; 11/03/2024
43965 0000779D E9D88E    jmp     SYS_RET_ERR
43966 ;
43967 ; BREAK <$Commit - commit the file>
43968 ;-----
43969 ;
43970 ;** $Commit - Commit a File
43971 ;
43972 ; $Commit "commits" a file to disk - all of it's buffers are
43973 ; flushed out. BUGBUG - I'm pretty sure that $Commit doesn't update
43974 ; the directory entry, etc., so this commit is pretty useless. check
43975 ; and fix this!! jgl
43976 ;
43977 ; Assembler usage:
43978 ;     MOV     BX, handle
43979 ;     MOV     AH, Commit
43980 ;     INT     int_command
43981 ;
43982 ; ENTRY (bx) = handle
43983 ; EXIT  none
43984 ; USES   all
43985 ;-----
43986 ;_SCOMMIT:
43987 ; Grab the SFT pointer from the JFN.
43988 ;
43989 ;
43990 000077A0 E8C302    call    CheckOwner      ; get system file entry
43991 ;JC      short CommitError    ; error return
43992 ; 11/03/2024
43993 000077A3 72F8     jc      short CommitError
43994 ;
43995 000077A5 16       push    ss
43996 000077A6 1F       pop     ds
43997 000077A7 893E[9E05] mov     [THISST],DI      ; For DOS_COMMIT
43998 000077AB 8C06[A005] mov     [THISST+2],ES    ; save offset of pointer
43999 ;
44000 ; ThisSFT is correctly set, we have DS = DOSDATA. Looks OK for a DOS_COMMIT
44001 ;
44002 ; ES:DI point to SFT
44003 ;
44004 000077AF E825C1    call    DOS_COMMIT
44005 000077B2 72E9     JC      short CommitError
44006 ; 07/12/2022

```

```

44007      ;jc      short CloseError
44008      ;mov      ah,68h
44009 000077B4 B468      MOV      AH,COMMIT
44010      ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44011      ;jmp      SYS_RET_OK
44012      CommitOk:
44013 000077B6 EBE2      jmp      short CloseOk
44014
44015      ; 11/03/2024
44016      ;CommitError:
44017      ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44018      ; jmp      SYS_RET_ERR
44019      ; jmp      short CloseError
44020
44021      ; BREAK <$ExtHandle - extend handle count>
44022
44023      ;** $ExtHandle - Extend Handle Count
44024      ;-----
44025      ; Assembler usage:
44026      ; MOV      BX, Number of Opens Allowed (MAX=65534;66535 is
44027      ; MOV      AX, 6700H reserved to mark SFT
44028      ; INT      int_command busy )
44029
44030      ; ENTRY (bx) = new number of handles
44031      ; EXIT 'C' clear if OK
44032      ; 'C' set iff err
44033      ; (ax) = error code
44034      ; AX = error_not_enough_memory
44035      ; error_too_many_open_files
44036      ; USES      all
44037      ;-----
44038
44039      ; 11/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
44040
44041      _$ExtHandle:
44042 000077B8 31ED      XOR      BP,BP ; 0: enlarge 1: shrink 2:psp
44043      ;cmp      bx,20
44044 000077BA 83FB14      CMP      BX,FILPERPROC
44045 000077BD 7303      JAE      short exth2 ; Don't set less than FilPerProc no
44046 000077BF BB1400      MOV      BX,FILPERPROC
44047      exth2:
44048 000077C2 368E06[3003] MOV      ES,[ss:CurrentPDB] ; get user process data block;smr;SS Override
44049      ;mov      cx,[ES:32h]
44050 000077C7 268B0E3200 MOV      CX,[ES:PDB.JFN_Length]; get number of handle allowed
44051 000077CC 39CB      CMP      BX,CX ; the requested == current
44052      ;JE      short ok_done ; yes and exit
44053      ; 11/03/2024
44054 000077CE 74CA      JE      short CloseOk
44055 000077D0 771E      JA      short larger ; go allocate new table
44056
44057      ; We're going to shrink the # of handles available
44058
44059      ;MOV      BP,1 ; shrink
44060      ; 11/03/2024
44061 000077D2 45      inc      bp
44062
44063      ;mov      ds,[ES:36h]
44064 000077D3 268E1E3600 MOV      DS,[ES:PDB.JFN_Pointer+2] ;
44065 000077D8 89DE      MOV      SI,BX ;
44066 000077DA 29D9      SUB      CX,BX ; get difference
44067
44068      ; BUGBUG - code a SCASB here, should be a bit smaller
44069      chck_handles:
44070 000077DC 803CFF      CMP      BYTE [SI],-1 ; scan through handles to ensure close
44071 000077DF 7539      JNZ      short too_many_files ; status
44072 000077E1 46      INC      SI
44073 000077E2 E2F8      LOOP     chck_handles
44074
44075 000077E4 83FB14      CMP      BX,FILPERPROC ; = 20
44076 000077E7 7707      JA      short larger ; no
44077
44078      ;MOV      BP,2 ; psp
44079      ; 11/03/2024
44080 000077E9 45      inc      bp
44081      ;mov      di,24
44082 000077EA BF1800      MOV      DI,PDB.JFN_TABLE ; es:di -> jfn table in psp
44083 000077ED 53      PUSH     BX
44084 000077EE EB1B      JMP      short movhandl
44085
44086      larger:
44087      ;CMP      BX,-1 ; 65535 is not allowed
44088      ;JZ      short invalid_func ; 10/08/2018
44089      ; 11/03/2024 (PCDOS 7.1 IBMDOS.COM)
44090      ;;;
44091 000077F0 43      inc      bx
44092 000077F1 747D      jz      short invalid_func
44093 000077F3 4B      dec      bx
44094      ;;;
44095 000077F4 F8      CLC
44096 000077F5 53      PUSH     BX ; save requested number
44097 000077F6 83C30F      ADD      BX,0FH ; adjust to paragraph boundary
44098 000077F9 B104      MOV      CL,4
44099      ;ror      bx,cl ; MSDOS 3.3
44100 000077FB D3DB      RCR      BX,CL ; DOS 4.00 fix ;AC000;
44101      ;AND      BX,1FFFH ; clear most 3 bits
44102      ; 01/07/2024
44103 000077FD 80E71F      and      bh,1Fh
44104
44105 00007800 55      PUSH     BP
44106 00007801 E80AFB      call     _$ALLOC ; allocate memory
44107 00007804 5D      POP      BP
44108 00007805 7264      JC      short no_memory ; not enough memory
44109
44110 00007807 8EC0      MOV      ES,AX ; es:di points to new table memory
44111 00007809 31FF      XOR      DI,DI
44112      movhandl:
44113 0000780B 368E1E[3003] MOV      DS,[ss:CurrentPDB] ; get user PDB address;smr;SS Override
44114
44115 00007810 F7C50300 test      BP,3 ; enlarge ?
44116 00007814 7409      JZ      short enlarge ; yes
44117 00007816 59      POP      CX ; cx = the amount you shrink
44118 00007817 51      PUSH     CX
44119 00007818 EB09      JMP      short copy_hand
44120
44121      ; Done. 'C' clear
44122
44123      ; 17/12/2022
44124      ;ok_done:
44125      ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44126      ; jmp      short CommitOk
44127      ; 17/12/2022
44128      ; jmp      SYS_RET_OK
44129
44130      too_many_files:

```

```

44131             ;mov     al,4
44132 0000781A B004     MOV     AL,error_too_many_open_files
44133             ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44134             ;jmp     SYS_RET_ERR
44135 CommitErrorj:
44136             ;jmp     short CommitError
44137             ; 17/12/2022
44138 0000781C E9598E     jmp     SYS_RET_ERR
44139
44140             ; 11/03/2024
44141             ; 17/12/2022
44142             ;ok_done:
44143             ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44144             ;jmp     short CommitOk
44145             ; 17/12/2022
44146             ; jmp     SYS_RET_OK
44147
44148 enlarge:
44149             ;mov     cx,[32h]
44150 0000781F 8B0E3200     MOV     CX,[PDB.JFN_Length] ; get number of old handles
44151 copy_hand:
44152             MOV     DX,CX
44153             ;lds     si,[34h]
44154 00007825 C5363400     LDS     SI,[PDB.JFN_Pointer] ; get old table pointer
44155 00007829 F3A4         REP     MOVSB ; copy information to new table
44156 0000782B 59          POP     CX ; get new number of handles
44157 0000782C 51          PUSH    CX ; save it again
44158 0000782D 29D1         SUB     CX,DX ; get the difference
44159 0000782F B0FF         MOV     AL,-1 ; set availability to handles
44160 00007831 F3AA         REP     STOSB
44161 00007833 368E1E[3003] MOV     DS,[ss:CurrentPDB] ; get user process data block;smr;SS Override
44162             ;cmp     word [34h],0
44163 00007838 833E340000     CMP     WORD [PDB.JFN_Pointer],0 ; check if original table pointer
44164 0000783D 750D         JNZ     short update_info ; yes, go update PDB entries
44165 0000783F 55          PUSH    BP
44166 00007840 1E          PUSH    DS ; save old table segment
44167 00007841 06          PUSH    ES ; save new table segment
44168 00007842 8E063600     MOV     ES,[PDB.JFN_Pointer+2]; get old table segment
44169 00007846 E83AFC         call    _$DEALLOC ; deallocate old table memory
44170 00007849 07          POP     ES ; restore new table segment
44171 0000784A 1F          POP     DS ; restore old table segment
44172 0000784B 5D          POP     BP
44173
44174 update_info:
44175 0000784C F7C50200     test    BP,2 ; psp?
44176 00007850 7408         JZ      short non_psp ; no
44177             ;mov     word [34h],18h ; 24
44178 00007852 C70634001800     MOV     WORD [PDB.JFN_Pointer],PDB.JFN_TABLE ; restore
44179 00007858 EB06         JMP     short final
44180 non_psp:
44181             ;mov     word [34h],0
44182 0000785A C70634000000     MOV     WORD [PDB.JFN_Pointer],0 ; new table pointer offset always 0
44183 final:
44184             ;mov     [36h],es
44185 00007860 8C063600     MOV     [PDB.JFN_Pointer+2],ES; update table pointer segment
44186             ;pop     word [32h]
44187 00007864 8F063200     POP     word [PDB.JFN_Length] ; restore new number of handles
44188             ; 11/03/2024
44189 ok_done:
44190             ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44191 00007868 E9038E     jmp     SYS_RET_OK
44192 ;ok_done-j:
44193             ; jmp     short ok_done
44194
44195 no_memory:
44196 0000786B 5B          POP     BX ; clean stack
44197             ;mov     al,8
44198 0000786C B008         MOV     AL,error_not_enough_memory
44199             ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44200             ;jmp     SYS_RET_ERR
44201 CommitErrorj2:
44202 0000786E EBAC         jmp     short CommitErrorj
44203
44204 invalid_func:
44205             ;mov     al,1
44206 00007870 B001         MOV     AL,error_invalid_function
44207             ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44208             ;jmp     SYS_RET_ERR
44209 CommitErrorj3:
44210             ;jmp     short CommitErrorj2
44211             ; 17/12/2022
44212 00007872 EBA8         jmp     short CommitErrorj
44213
44214             ; 20/05/2019 - Retro DOS v4.0
44215             ; DOSCODE:A83Ah (MSDOS 6.21, MSDOS.SYS)
44216
44217             ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
44218             ; DOSCODE:A7DAh (MSDOS 5.0 MSDOS.SYS)
44219
44220             ; BREAK <$READ - Read from a file handle>
44221             ;-----
44222             ;
44223             ;** $Read - Read from a File Handle
44224             ;
44225             ; Assembler usage:
44226             ;
44227             ; LDS     DX, buf
44228             ; MOV     CX, count
44229             ; MOV     BX, handle
44230             ; MOV     AH, Read
44231             ; INT     int_command
44232             ; AX has number of bytes read
44233             ;
44234             ; ENTRY (bx) = file handle
44235             ; (cx) = byte count
44236             ; (ds:dx) = buffer address
44237             ; EXIT Through system call return so that to user:
44238             ; 'C' clear if OK
44239             ; (ax) = bytes read
44240             ; 'C' set if error
44241             ; (ax) = error code
44242             ;-----
44243
44244             ; 12/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
44245             ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0BA2Eh
44246
44247             ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0A83Ah)
44248             ; (Windows ME IO.SYS - BIOSCODE:0A256h)
44249
44250 _$READ:
44251             MOV     SI,DOS_READ
44252 00007874 BE[773B]     ReadDo:
44253             call    pJFNFromHandle
44254 00007877 E83CFE

```

```

44255 0000787A 7208          JC      short ReadError
44256
44257 0000787C 268A05        MOV     AL,[ES:DI]
44258 0000787F E8E401        call    CheckOwner      ; get the handle
44259 00007882 7303          JNC     short ReadSetup  ; no errors do the operation
44260
44261          ; Have an error. 'C' set
44262
44263 ReadError:
44264          ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44265          ;;jmp SYS_RET_ERR      ; go to error traps
44266          ;;jmp short CommitErrorj3
44267          ; 17/12/2022
44268 00007884 E9F18D        jmp     SYS_RET_ERR
44269
44270 ReadSetup:
44271 00007887 36893E[9E05]      MOV     [ss:THISSFT],DI      ; save offset of pointer;smr;SS Override
44272 0000788C 368C06[A005]      MOV     [ss:THISSFT+2],ES   ; save segment value ;smr;SS Override
44273          ; 20/05/2019 - Retro DOS v4.0
44274          ; MSDOS 6.0
44275          ;; Extended Open
44276          ;test byte [es:di+3],20h
44277 00007891 26F6450320      test    byte [ES:DI+SF_ENTRY.sf_mode+1],(INT_24_ERROR>>8)
44278          ;AN000;;EO. need i24
44279 00007896 7406          JZ      short needi24      ;AN000;;EO. yes
44280 00007898 36800E[F605]02      OR      byte [ss:EXTOPEN_ON],EXT_OPEN_I24_OFF ; 2
44281          ;AN000;;EO. set it off;smr;SS Override
44282          needi24:
44283          ;AN000;
44284          ; 12/03/2024
44285          %if 0
44286
44287          ;; Extended Open
44288          push word [SS:DMAADD]
44289          push word [SS:DMAADD+2] ;smr;SS Override
44290
44291          ;;;; BAD SPOT FOR 286!!! SEGMENT ARITHMETIC!!!
44292
44293          ; 26/07/2019
44294
44295          ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44296          ;
44297          ; (It is not necessary to call 'Align_Buffer' proc here/below because
44298          ; there is not another caller; it is better to put the code in this proc
44299          ; here instead of calling it as a subroutine; but I have modified code
44300          ; here for MSDOS 5.0 MSDOS.SYS address compatibility)
44301
44302          ; MSDOS 6.0
44303          CALL Align_Buffer      ;AN000;MS. align user's buffer
44304
44305          ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44306          ; MSDOS 3.3
44307          ;MOV BX,DX      ; copy offset
44308          ;push cx      ; don't stomp on count
44309          ;MOV CL,4      ; bits to shift bytes->para
44310          ;SHR BX,CL      ; get number of paragraphs
44311          ;pop cx      ; get count back
44312          ;MOV AX,DS      ; get original segment
44313          ;ADD AX,BX      ; get new segment
44314          ;MOV DS,AX      ; in seg register
44315          ;AND DX,0Fh      ; normalize offset
44316          ;MOV [ss:DMAADD],DX ; use user DX as offset ;smr;SS Override
44317          ;MOV [ss:DMAADD+2],DS ; use user DS as segment for DMA
44318          ;smr;SS Override
44319          %else
44320          ; 12/03/2024 (PCDOS 7.1 IBMDOS.COM)
44321          ;;;;
44322 0000789E 8CD8          mov     ax,ds      ; original segment
44323 000078A0 36C51E[2C03]      lds     bx,[ss:DMAADD]
44324 000078A5 53          push    bx
44325 000078A6 1E          push    ds
44326 000078A7 89D3          mov     bx,dx
44327 000078A9 D1EB          shr     bx,1
44328 000078AB D1EB          shr     bx,1
44329 000078AD D1EB          shr     bx,1
44330 000078AF D1EB          shr     bx,1
44331 000078B1 01D8          add     ax,bx      ; new segment
44332 000078B3 83E20F          and     dx,0Fh      ; normalize offset
44333          ;mov [ss:DMAADD],dx ; use user DX as offset
44334          ; 23/03/2024
44335 000078B6 36A3[2E03]      mov     [ss:DMAADD+2],ax ; use user DS as segment for DMA
44336          ;;;;
44337
44338          %endif
44339
44340          ;;;; END BAD SPOT FOR 286!!! SEGMENT ARITHMETIC!!!
44341
44342 000078BA 16          push    ss      ; go for DOS addressability
44343 000078BB 1F          pop     ds
44344
44345          ; 12/03/2024 - Retro DOS v5.0
44346          ;;;;
44347 000078BC 8916[2C03]      mov     [DMAADD],dx
44348          ;;;;
44349
44350 000078C0 FFD6          CALL    SI ; DOS_READ      ; indirect call to operation
44351
44352 000078C2 8F06[2E03]      pop     word [DMAADD+2]
44353 000078C6 8F06[2C03]      pop     word [DMAADD]
44354          ;JNC short READ_OK      ;AN002;
44355          ;JMP short ReadError      ;AN002; if error, say bye bye
44356          ; 17/12/2022
44357 000078CA 72B8          JC      short ReadError
44358          ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44359          ;jnc short READ_OK      ;AN002;
44360          ;jmp short ReadError
44361
44362 READ_OK:
44363 000078CC 89C8          MOV     AX,CX      ; get correct return in correct reg
44364 Read_okj:
44365          ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44366          ;;jmp SYS_RET_OK      ; successful return
44367          ;jmp short ok_done_j
44368          ; 17/12/2022
44369 000078CE E99D8D        jmp     SYS_RET_OK
44370
44371          ; 13/07/2018 - Retro DOS v3.0
44372
44373          ;-----
44374
44375          ; Input: DS:DX points to user's buffer addr
44376          ; Function: rearrange segment and offset for READ/WRITE buffer
44377          ; Output: [DMAADD] set
44378

```



```

44379 ; 12/03/2024
44380 %if 0
44381 ; 20/05/2019 - Retro DOS v4.0
44382 ; 26/07/2019
44383 ; ; MSDOS 6.0
44384 ; Align_Buffer:
44385 ; MOV BX,DX ; copy offset
44386 ; push CX ; don't stomp on count
44387 ; MOV CL,4 ; bits to shift bytes->para
44388 ; SHR BX,CL ; get number of paragraphs
44389 ; pop CX ; get count back
44390 ; MOV AX,DS ; get original segment
44391 ; ADD AX,BX ; get new segment
44392 ; MOV DS,AX ; in seg register
44393 ; AND DX,0Fh ; normalize offset
44394 ; MOV [ss:DMAADD],DX ; use user DX as offset ;smr;SS Override
44395 ; MOV [ss:DMAADD+2],DS ; use user DS as segment for DMA
44396 ; ;smr;SS Override
44397 ; retn
44398 ;
44399 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44400 Align_Buffer:
44401 ; MOV BX,DX ; copy offset
44402 ; push CX ; don't stomp on count
44403 ; MOV CL,4 ; bits to shift bytes->para
44404 ; SHR BX,CL ; get number of paragraphs
44405 ; pop CX ; get count back
44406 ; MOV AX,DS ; get original segment
44407 ; ADD AX,BX ; get new segment
44408 ; MOV DS,AX ; in seg register
44409 ; AND DX,0Fh ; normalize offset
44410 ; MOV [ss:DMAADD],DX ; use user DX as offset ;smr;SS Override
44411 ; MOV [ss:DMAADD+2],DS ; use user DS as segment for DMA
44412 ; ;smr;SS Override
44413 ; retn
44414 ;
44415 ;endif
44416 ; 20/05/2019 - Retro DOS v4.0
44417 ; DOSCODE:A8A0h (MSDOS 6.21, MSDOS.SYS)
44418 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
44419 ; DOSCODE:A840h (MSDOS 5.0 MSDOS.SYS)
44420 ; 12/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
44421 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0BA8Ch)
44422 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0A8A0h)
44423 ; (Windows ME IO.SYS - BIOSCODE:0A2B9h)
44424 ;BREAK <$WRITE - write to a file handle>
44425 ;-----
44426 ; Assembler usage:
44427 ; LDS DX, buf
44428 ; MOV CX, count
44429 ; MOV BX, handle
44430 ; MOV AH, write
44431 ; INT int_command
44432 ; AX has number of bytes written
44433 ; Errors:
44434 ; AX = write_invalid_handle
44435 ; = write_access_denied
44436 ; Returns in register AX
44437 ;-----
44438 ;_WRITE:
44439 ; MOV SI,DOS_WRITE
44440 ; JMP short ReadDo
44441 ;BREAK <$LSEEK - move r/w pointer>
44442 ;-----
44443 ; Assembler usage:
44444 ; MOV DX, offsetlow
44445 ; MOV CX, offsethigh
44446 ; MOV BX, handle
44447 ; MOV AL, method
44448 ; MOV AH, LSeek
44449 ; INT int_command
44450 ; DX:AX has the new location of the pointer
44451 ; Error returns:
44452 ; AX = error_invalid_handle
44453 ; = error_invalid_function
44454 ; Returns in registers DX:AX
44455 ;-----
44456 ; 21/05/2019 - Retro DOS v4.0
44457 ; DOSCODE:A8A5h (MSDOS 6.21, MSDOS.SYS)
44458 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
44459 ; DOSCODE:A845h (MSDOS 5.0 MSDOS.SYS)
44460 ; 12/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
44461 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0BA91h
44462 ;_LSEEK:
44463 ; call CheckOwner ; get system file entry
44464 ; 17/12/2022
44465 ;LSeekError:
44466 ;JNC short CHKOWN_OK ;AN002;
44467 ;JMP short ReadError ;AN002; error return
44468 ; 17/12/2022
44469 ; 02/06/2019
44470 ;Jc short ReadError
44471 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44472 ;JNC short CHKOWN_OK ;AN002;
44473 ;JMP short ReadError ;AN002; error return
44474 ;CHKOWN_OK:
44475 ; ;AN002;
44476 ; CMP AL,2 ; is the seek value correct?
44477 ; JBE short LSeekDisp ; yes, go dispatch
44478 ; 12/03/2024
44479 %if 0
44480 ;mov byte [ss:EXTERR_LOCUS],1
44481 ;MOV byte [ss:EXTERR_LOCUS],errLOC_Unk ; Extended Error Locus
44482
44483
44484
44485
44486
44487
44488
44489
44490
44491
44492
44493
44494
44495
44496
44497
44498
44499
44500
44501
44502

```

```

44503                                     ;smr;SS Override
44504
44505 %else
44506 ; 12/03/2024 (PCDOS 7.1 IBMDOS.COM)
44507 ;;;
44507 000078DF E8FB99      call     set_exerr_locus_unk    ; Extended Error Locus
44508 ;;;
44509 %endif
44510 ;mov     al,1
44511 000078E2 B001      mov     al,error_invalid_function ; invalid method
44512 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44513 LSeekError2:
44514 000078E4 EB9E      jmp      short ReadError
44515
44516 LSeekDisp:
44517 000078E6 3C01      cmp     al,1                ; best way to dispatch; check middle
44518 000078E8 720A      jb      short LSeekStore   ; just store CX:DX
44519 000078EA 771B      ja      short LSeekEOF     ; seek from end of file
44520 ;add     dx,[es:di+21]
44521 000078EC 26035515   add     dx,[ES:DI+SF_ENTRY.sf_position]
44522 ;adc     cx,[es:di+23]
44523 000078F0 26134D17   adc     cx,[ES:DI+SF_ENTRY.sf_position+2]
44524
44525 LSeekStore:
44526 000078F4 89C8      mov     ax,cx              ; AX:DX
44527 000078F6 92        xchg    ax,dx              ; DX:AX is the correct value
44528
44529 LSeekSetpos:
44529 000078F7 26894515   mov     [es:di+21],ax
44530 ;mov     [ES:DI+SF_ENTRY.sf_position],ax
44531 ;mov     [es:di+23],dx
44531 000078FB 26895517   mov     [ES:DI+SF_ENTRY.sf_position+2],dx
44532 000078FF E8758B      call    Get_User_Stack
44533 ;mov     [si+6],dx
44534 00007902 895406      mov     [SI+user_env.user_DX],dx ; return DX:AX
44535 ;jmp     SYS_RET_OK     ; successful return
44536 ; 25/06/2019
44537 ;jmp     SYS_RET_OK_c1c
44538 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44539 ;jmp     SYS_RET_OK_c1c
44540
44541 LSeekOK:
44541 00007905 EBC7      jmp      short Read_Ok_j
44542
44543 LSeekEOF:
44544 ;;test   word [es:di+5],8000h
44545 ;test   word [ES:DI+SF_ENTRY.sf_flags],sf_isnet
44546 ; 21/05/2019 - Retro DOS v4.0
44547 00007907 26F6450680 test    byte [ES:DI+SF_ENTRY.sf_flags+1],(sf_isnet>>8)
44548 0000790C 750A      jnz     short Check_LSeek_Mode; Is Net
44549
44550 LOCAL_LSeek:
44551 0000790E 26035511   add     dx,[es:di+17]
44552 ;add     dx,[ES:DI+SF_ENTRY.sf_size]
44553 ;adc     cx,[es:di+19]
44553 00007912 26134D13   adc     cx,[ES:DI+SF_ENTRY.sf_size+2]
44554 00007916 EBDC      jmp      short LSeekStore   ; go and set the position
44555
44556 Check_LSeek_Mode:
44557 ;;test   word [es:di+2],8000h
44558 ;test   word [ES:DI+SF_ENTRY.sf_mode],sf_isFCB
44559 ; 21/05/2019
44560 00007918 26F6450380 test    byte [ES:DI+SF_ENTRY.sf_mode+1],(sf_isFCB>>8)
44561 0000791D 75EF      jnz     short LOCAL_LSeek   ; FCB treated like local file
44562 ;mov     ax,[es:di+2]
44563 0000791F 268B4502   mov     ax,[ES:DI+SF_ENTRY.sf_mode]
44564 ;and     ax,0F0h
44565 00007923 25F000      and     ax,SHARING_MASK
44566 ;cmp     ax,40h
44567 00007926 83F840      cmp     ax,SHARING_DENY_NONE
44568 00007929 7405      jz      short NET_LSEEK     ; LSEEK exported in this mode
44569 ;cmp     ax,30h
44570 0000792B 83F830      cmp     ax,SHARING_DENY_READ
44571 0000792E 75DE      jnz     short LOCAL_LSeek   ; Treated like local Lseek
44572
44573 NET_LSEEK:
44574 ; jmp     short LOCAL_LSeek
44575 ; REMOVE ABOVE INSTRUCTION TO ENABLE DCR 142
44576 ;call     Install_Net_Lseek,MultNET,33
44577 ;jnc     short LSeekSetPos
44578
44578 00007930 B82111      mov     ax,1121h
44579 00007933 CD2F      int     2Fh                ; Multiplex - NETWORK REDIRECTOR - SEEK FROM END OF REMOTE FILE
44580 ; CX:DX = offset (in bytes) from end
44581 ; ES:DI -> SFT, SFT DPB field -> DPB of drive with file
44582 ; SS = DOS CS
44583 ; Return: CF set on error
44584 ; CF clear if successful, DX:AX = new file position
44585 00007935 73C0      jnb     short LSeekSetpos
44586 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44587 ;jmp     SYS_RET_ERR
44588
44589 ;LSeekError3:
44590 ; 17/12/2022
44591 LSeekError:
44591 ;jmp     short LSeekError2
44592
44593 00007937 E93E8D      dupErr: ; 17/12/2022
44594 ;jmp     SYS_RET_ERR
44595
44596 ; 21/05/2019 - Retro DOS v4.0
44597 ; DOSCODE:A95Bh (MSDOS 6.21, MSDOS.SYS)
44598
44599 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
44600 ; DOSCODE:A8FBh (MSDOS 5.0 MSDOS.SYS)
44601
44602 ;BREAK <$DUP - duplicate a jfn>
44603 ;-----
44604 ;
44605 ; Assembler usage:
44606 ;     MOV     BX, fh
44607 ;     MOV     AH, Dup
44608 ;     INT     int_command
44609 ;     AX has the returned handle
44610 ;
44611 ; Errors:
44612 ;     AX = dup_invalid_handle
44613 ;     = dup_too_many_open_files
44614 ;-----
44615 ; 12/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
44616 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:0BBEBh
44617
44618 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0A95Bh)
44619 ; (Windows ME IO.SYS - BIOSCODE:0A3EFh)
44620
44621 _$DUP:
44622 0000793A 89D8      mov     ax,bx              ; save away old handle in AX
44623 0000793C E8D4FD      call    JFNFree           ; free handle? into ES:DI, new in BX
44624
44625 DupErrorCheck:
44626 00007941 06        jc      short DupErr      ; nope, bye
44627 ;push    es

```



```

44627 00007942 57      push    di                ; save away SFT
44628 00007943 5E      pop     si                ; into convenient place DS:SI
44629 00007944 1F      pop     ds
44630 00007945 93      XCHG    AX,BX                ; get back old handle
44631 00007946 E81D01    call   CheckOwner            ; get sft in ES:DI
44632 00007949 72EC    JC      short DupErr        ; errors go home
44633 0000794B E870B7    call   DOS_Dup_Direct
44634 0000794E E865FD    call   pJFNFromHandle        ; get pointer
44635 00007951 268A1D    MOV     BL,[ES:DI]           ; get SFT number
44636 00007954 881C     MOV     [SI],BL              ; stuff in new SFT
44637                                     ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44638                                     ; jmp     SYS_RET_OK          ; and go home
44639 00007956 EB5A     jmp     short ok_ret
44640
44641                                     ; 17/12/2022
44642 ;DupErr:
44643                                     ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44644                                     ; jmp     SYS_RET_ERR
44645                                     ; jmp     short ft_error
44646
44647 ;BREAK <$DUP2 - force a dup on a particular jfn>
44648 -----
44649 ;
44650 ; Assembler usage:
44651 ;     MOV     BX, fh
44652 ;     MOV     CX, newfh
44653 ;     MOV     AH, Dup2
44654 ;     INT     int_command
44655 ; Error returns:
44656 ;     AX = error_invalid_handle
44657 -----
44658
44659
44660 _$DUP2:
44661     push    bx
44662     push    cx                ; save source
44663     MOV     BX,CX            ; get one to close
44664     call   _SCLOSE           ; close destination handle
44665     pop     bx
44666     pop     ax                ; old in AX, new in BX
44667     call   pJFNFromHandle    ; get pointer
44668     JMP     short DupErrorCheck ; check error and do dup
44669
44670 ;BREAK <FileTimes - modify write times on a handle>
44671 -----
44672 ;
44673 ; Assembler usage:
44674 ;     MOV     AH, FileTimes (57H)
44675 ;     MOV     AL, func
44676 ;     MOV     BX, handle
44677 ; ; if AL = 1 then then next two are mandatory
44678 ;     MOV     CX, time
44679 ;     MOV     DX, date
44680 ;     INT     21h
44681 ; ; if AL = 0 then CX/DX has the last write time/date
44682 ; ; for the handle.
44683 ;
44684 ; AL=02      get extended attributes
44685 ; BX=handle
44686 ; CX=size of buffer (0, return max size )
44687 ; DS:SI query list (si=-1, selects all EA)
44688 ; ES:DI buffer to hold EA list
44689 ;
44690 ; AL=03      get EA name list
44691 ; BX=handle
44692 ; CX=size of buffer (0, return max size )
44693 ; ES:DI buffer to hold name list
44694 ;
44695 ; AL=04      set extended attributes
44696 ; BX=handle
44697 ; ES:DI buffer of EA list
44698 ;
44699 ;
44700 ; Error returns:
44701 ;     AX = error_invalid_function
44702 ;     = error_invalid_handle
44703 -----
44704
44705
44706 ; 21/05/2019 - Retro DOS v4.0
44707 ; DOSCODE:A90Dh (MSDOS 6.21, MSDOS.SYS)
44708
44709 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
44710 ; DOSCODE:A8ADh (MSDOS 5.0 MSDOS.SYS)
44711
44712 ; 12/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
44713 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0BAF5h
44714
44715 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0A90Dh)
44716 ; (Windows ME IO.SYS - BIOSCODE:0A327h)
44717
44718 _$FILE_TIMES:
44719
44720 ; 12/03/2024
44721 %if 0
44722 ; 13/07/2018 - Retro DOS v3.0
44723
44724 ; MSDOS 3.3
44725 ; cmp     al,2                ; correct subfunction ?
44726 ; jnb     short ft1
44727
44728 ; ; mov     byte [ss:EXTERR_LOCUS], 1
44729 ; mov     byte [ss:EXTERR_LOCUS],errLOC_Unk ; Extended Error Locus
44730 ; ; SS Overr
44731 ; ; mov     al,1
44732 ; mov     al,error_invalid_function ; give bad return
44733 ; jmp     SYS_RET_ERR
44734
44735 ; MSDOS 6.0
44736 ; cmp     al,2                ; correct subfunction ?
44737 ; jae     short inval_func
44738 ;ft1:
44739     call   CheckOwner            ; get sft
44740     ; 17/12/2022
44741     JC      short LSeekError    ; bad handle
44742
44743     or     al,al                ; get time/date ?
44744     jnz     short ft_set_time
44745
44746 ;----- here we get the time & date from the sft for the user
44747
44748     cli                                ; is this cli/sti reqd ? BUGBUG
44749     mov     cx,[es:di+13]
44750     mov     cx,[es:di+SF_ENTRY.sf_time] ; get the time

```

```

44751         ;mov     dx,[es:di+15]
44752         mov     dx,[es:di+SF_ENTRY.sf_date] ; & date
44753         sti
44754         call    Get_User_Stack
44755         ;mov     [si+4],cx
44756         mov     [si+user_env.user_CX],cx
44757         ;mov     [si+6],dx
44758         mov     [si+user_env.user_DX],dx
44759         jmp     short ok_ret
44760
44761         ;----- here we set the time in sft
44762
44763         ft_set_time:
44764         call    ECritSFT
44765         ;mov     [es:di+13],cx
44766         mov     [es:di+SF_ENTRY.sf_time],cx ; drop in new time
44767         ;mov     [es:di+15],dx
44768         mov     [es:di+SF_ENTRY.sf_date],dx ; and date
44769
44770         xor     ax, ax
44771         call    far [ss:JShare+(14*4)] ; 14 = shSU ; SS Override
44772
44773         ;----- set the flags in SFT entry
44774         ;and     word [es:di+5],0FFBFh
44775         ; 18/12/2022
44776         ;and     byte [es:di+5],0BFh
44777         and     byte [es:di+SF_ENTRY.sf_flags],~devid_file_clean
44778         and     word [es:di+SF_ENTRY.sf_flags],~devid_file_clean
44779                                     ; mark file as dirty
44780         ;or      word [es:di+5],4000h
44781         ; 17/12/2022
44782         ;or      byte [es:di+6],40h
44783         or      byte [es:di+SF_ENTRY.sf_flags+1],(sf_close_nodate>>8)
44784         ;or      word [es:di+SF_ENTRY.sf_flags],sf_close_nodate
44785                                     ; ask close not to
44786                                     ; bother about date
44787                                     ; and time
44788         call    LCritSFT
44789
44790         ok_ret:
44791         ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44792         ; 17/12/2022
44793         jmp     SYS_RET_OK
44794         ;jmp     short LSeekOk
44795
44796         inval_func:
44797         ;mov     byte [ss:EXTERR_LOCUS],1
44798         mov     byte [ss:EXTERR_LOCUS],errLOC_Unk ; Extended Error Locus
44799                                     ;SS Overr
44800
44801         ;mov     al,1
44802         mov     al,error_invalid_function ; give bad return
44803         ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
44804         ft_error:
44805         ;;jmp     SYS_RET_ERR
44806         ;jmp     short LSeekError3
44807         ; 17/12/2022
44808         jmp     short LSeekError
44809
44810         %else
44811         ; 12/03/2024 - Retro DOS v5.0
44812         ; PCDOS 7.1 IBMDOS.COM
44813         ;;      ; ((Ref: Ralf Brown's Interruption List))
44814         cmp     al,7 ; SET CREATION DATE AND TIME
44815         ; 6 = GET CREATION DATE AND TIME
44816         ; 5 = SET LAST ACCESS DATE AND TIME
44817         ;jnb     short LSeekError1
44818         ; 12/03/2024
44819         jnb     short inval_func
44820         cmp     al,4 ; GET LAST ACCESS DATE AND TIME
44821         jnb     short ft1
44822         cmp     al,2 ; 0 = GET FILE'S LAST-WRITTEN DATE AND TIME
44823         ; 1 = SET FILE'S LAST-WRITTEN DATE AND TIME
44824         ;jnb     short LSeekError1 ; 2,3 -> invalid
44825         ; 12/03/2024
44826         jnb     short inval_func
44827         ft1:
44828         call    CheckOwner ; get sft
44829         jc      short LSeekError ; bad handle
44830
44831         cmp     al,1
44832         ja      short ft2 ; PCDOS 7.1 (MSDOS 7) sub function
44833
44834         or      al,al ; get time/date ?
44835         jnz     short ft_set_time ; no,set
44836
44837         ;lds     cx,[es:di+0Dh]
44838         lds     cx,[es:di+SF_ENTRY.sf_time]
44839         ;mov     dx,ds ; get the time
44840         call    Get_User_Stack ; & date ; [es:di+SF_ENTRY.sf_date]
44841         ;mov     [si+4],cx
44842         mov     [si+user_env.user_CX],cx
44843         ;mov     [si+6],dx
44844         mov     [si+user_env.user_DX],dx
44845
44846         jmp     short ft_ok
44847
44848         ft_set_time:
44849         call    ECritSFT
44850
44851         ;mov     [es:di+0Dh],cx
44852         mov     [es:di+SF_ENTRY.sf_time],cx ; drop in new time
44853         ;mov     [es:di+0Fh],dx
44854         mov     [es:di+SF_ENTRY.sf_date],dx ; and date
44855
44856         xor     ax,ax ; 0
44857         ;call    ss:ShSU
44858         call    far [ss:JShare+(14*4)] ; 14 = shSU
44859
44860         ;;and     word [es:di+5],0FFBFh
44861         ;and     word [es:di+SF_ENTRY.sf_flags],~devid_file_clean
44862         and     byte [es:di+SF_ENTRY.sf_flags],~devid_file_clean ; 0BFh
44863
44864         ;;or      word [es:di+5],4000h
44865         ;or      word [es:di+SF_ENTRY.sf_flags],sf_close_nodate
44866         or      byte [es:di+SF_ENTRY.sf_flags+1],(sf_close_nodate>>8) ; 40h
44867
44868         call    LCritSFT
44869         ft_ok:
44870         ; 12/03/2024
44871         ok_ret:
44872         jmp     SYS_RET_OK
44873
44874         ft2:
44875         ;test     byte [es:di+5],80h

```

```

44875 000079B5 26F6450580      test    byte [es:di+SF_ENTRY.sf_flags],devid_device
44876 000079BA 7505          jnz     short ft3             ; device
44877
44878 000079BC E8839E          call    IsSFTNet
44879
44880 000079BF 7415          ;ft3:  jz     short ft5             ; local file
44881                                ; 12/03/2024
44882 000079C1 A801          ft3:    test    al,1             ; 0 = GET, 1 = SET
44883 000079C3 750F          jnz     short ft_okj         ; SET
44884
44885                                ;mov    dx,[es:di+0Fh]
44886 000079C5 268B550F      mov     dx,[es:di+SF_ENTRY.sf_date]
44887
44888
44889
44890 000079C9 29C9          ft7:    ; 12/03/2024
44891                                sub     cx,cx ; 0 ; Retro DOS v5.0
44892                                ; (Windows ME IO.SYS BIOSCODE:A348h)
44893
44894
44895
44896 000079CB E8A98A          ft4:    call    Get_User_Stack
44897                                ;mov     [si+4],cx
44898 000079CE 894C04      ;mov     [si+user_env.user_CX],cx ; time
44899                                ;mov     [si+6],dx
44900 000079D1 895406      ;mov     [si+user_env.user_DX],dx ; date
44901
44902
44903 000079D4 EBDC          ft_okj: jmp     short ft_ok
44904
44905
44906
44907
44908
44909
44910
44911
44912
44913
44914 000079D6 16          ft5:    push    ss             ; Retrieve the directory entry for the file
44915 000079D7 1F          pop     ds
44916 000079D8 50          push    ax
44917 000079D9 52          push    dx
44918 000079DA 51          push    cx             ; ES:DI point to SFT
44919 000079DB E8BBBE      call    DirFromSFT        ; locate a directory entry given an SFT
44920
44921
44922
44923
44924 000079DE 59          pop     cx
44925 000079DF 5A          pop     dx
44926 000079E0 730B      jnc     short ft6         ; ES:DI point to entry
44927                                ; (DS:SI point to SFT)
44928                                ; (ES:BX point to buffer header)
44929
44930
44931
44932
44933
44934 000079E2 59          pop     cx
44935
44936
44937
44938
44939
44940
44941
44942
44943
44944
44945
44946
44947
44948
44949
44950
44951
44952
44953
44954
44955
44956
44957
44958
44959
44960
44961
44962
44963
44964
44965
44966
44967
44968
44969
44970
44971
44972
44973
44974
44975
44976
44977
44978
44979
44980
44981
44982
44983
44984
44985
44986
44987
44988
44989
44990
44991
44992
44993
44994
44995
44996
44997
44998
000079E3 EB05
000079E5 E8F598
000079E8 B001
000079EA E98B8C
000079ED 58
000079EE A801
000079F0 7512
000079F2 268B5512
000079F6 3C06
000079F8 75CF
000079FA 268B4D0E
000079FE 268B5510
00007A02 EBC7
00007A04 F6C21F
00007A07 7504
00007A09 B00D
00007A0B EBDD
00007A0D F7C2E001
00007A11 74F6
00007A13 52
00007A14 81E2E001
00007A18 81FA8001
00007A1C 5A
00007A1D 77EA
00007A1F 3C05
00007A21 7506
00007A23 26895512
00007A27 EB20
00007A29 89C8
00007A2B 251FF8
00007A2E 80FCB8
00007A31 77D6
00007A33 3C1D
00007A35 77D2
00007A37 89C8
00007A39 25E007
00007A3C 3D6007
00007A3F 77C8
00007A41 26895510
00007A45 26894D0E
00007A49 26F6470540
00007A4E 7508

      test    byte [es:di+SF_ENTRY.sf_flags],devid_device
      jnz     short ft3             ; device

      call    IsSFTNet

;ft3:  jz     short ft5             ; local file
      ; 12/03/2024
      ft3:    test    al,1             ; 0 = GET, 1 = SET
      jnz     short ft_okj         ; SET

      ;mov    dx,[es:di+0Fh]
      mov     dx,[es:di+SF_ENTRY.sf_date]

      ;;;

ft7:    ; 12/03/2024
      sub     cx,cx ; 0 ; Retro DOS v5.0
      ; (Windows ME IO.SYS BIOSCODE:A348h)

      ;;;

ft4:    call    Get_User_Stack
      ;mov     [si+4],cx
      ;mov     [si+user_env.user_CX],cx ; time
      ;mov     [si+6],dx
      ;mov     [si+user_env.user_DX],dx ; date
ft_okj: jmp     short ft_ok

ft5:    push    ss             ; Retrieve the directory entry for the file
      pop     ds
      push    ax
      push    dx
      push    cx             ; ES:DI point to SFT
      call    DirFromSFT        ; locate a directory entry given an SFT
      pop     cx
      pop     dx
      jnc     short ft6         ; ES:DI point to entry
      ; (DS:SI point to SFT)
      ; (ES:BX point to buffer header)

      pop     cx
;ft_error:
      ;jmp     short LSeekError2
      ; 12/03/2024
      ;jmp     short LSeekError
      jmp     short ft_error

      ;;;
      ; 12/03/2024 - Retro DOS v5.0
      inval_func:
      call    set_exerr_locus_unk ; Extended Error Locus

      ;mov     al,1
      mov     al,error_invalid_function ; give bad return
ft_error:
      ;jmp     short LSeekError
      ; 12/03/2024
      jmp     SYS_RET_ERR
      ;;;

ft6:    pop     ax
      test    al,1             ; SET ?
      jnz     short ft8         ; yes

      ; 12/03/2024 (ft7:, cx=0)
      xor     cx,cx ; 0 ; (always) last access time = 0

      ;mov     dx,[es:di+12h]
      mov     dx,[es:di+dir_entry.dir_lstaccdte]

      cmp     al,6             ; GET CREATION DATE AND TIME ?
      jne     short ft7         ; no,GET LAST ACCESS DATE AND TIME

      ;mov     cx,[es:di+0Eh]
      mov     cx,[es:di+dir_entry.dir_crttime]
      ;mov     dx,[es:di+10h]
      mov     dx,[es:di+dir_entry.dir_crtdate]
;ft7:    jmp     short ft4

ft8:    ; check date (set) values
      test    dl,1Fh           ; DAY (1 to 31) > 0 ?
      jnz     short ft9         ; yes,valid

ft_err_invd:
      ;mov     al,0Dh
      mov     al,error_invalid_data
ft_errj: jmp     short ft_error

ft9:    test    dx,1E0h         ; MONTH (1 to 12) > 0 ?
      jz     short ft_err_invd ; no,invalid
      push    dx
      and     dx,1E0h         ; isolate MONTH
      cmp     dx,180h         ; > 12 ?
      pop     dx
      ja     short ft_err_invd ; yes,invalid
      cmp     al,5             ; SET LAST ACCESS DATE AND TIME ?
      jnz     short ft10        ; no,SET CREATION DATE AND TIME

      ;mov     [es:di+12h],dx
      mov     [es:di+dir_entry.dir_lstaccdte],dx
      jmp     short ft11

ft10:   ; check time (set) values
      mov     ax,cx
      and     ax,0F81Fh        ; isolate seconds/2 and hour
      cmp     ah,0B8h         ; HOUR > 23 ?
      ja     short ft_err_invd ; yes,invalid
      cmp     al,1Dh         ; SECONDS/2 > 29 (count of 2 seconds)
      ja     short ft_err_invd ; yes,invalid
      mov     ax,cx
      and     ax,7E0h         ; isolate MINUTE
      cmp     ax,760h         ; MINUTE > 59 ?
      ja     short ft_err_invd ; yes,invalid

      ;mov     [es:di+10h],dx
      mov     [es:di+dir_entry.dir_crtdate],dx
      ;mov     [es:di+0Eh],cx
      mov     [es:di+dir_entry.dir_crttime],cx
ft11:   ;test    byte [es:bx+5],40h
      test    byte [es:bx+BUFFINFO.buf_flags],buf_dirty
      jnz     short ft12

```

```

44999
45000 00007A50 E877F1      call    INC_DIRTY_COUNT
45001
45002      ;or    byte [es:bx+5],40h
45003 00007A53 26804F0540  or     byte [es:bx+BUFFINFO.buf_flags],buf_dirty
45004
45005 00007A58 06          ft12:   push    es
45006 00007A59 1F          pop     ds
45007 00007A5A 89DF      mov     di,bx      ; DS:DI - pointer to buffer
45008 00007A5C B0FF      mov     al,0FFh    ; Drive number
45009                                ; -1 means do not check for drive
45010 00007A5E E886F0      call    CHECKFLUSH
45011 00007A61 7287      jc      short ft_error
45012      ;ok_ret:  ; 12/03/2024
45013 00007A63 E9088C      jmp     SYS_RET_OK
45014      ;;;
45015 %endif
45016
45017 ;Break      <CheckOwner - verify ownership of handles from server>
45018 -----
45019 ; CheckOwner - Due to the ability of the server to close file handles for a
45020 ; process without the process knowing it (delete/rename of open files, for
45021 ; example), it is possible for the redirector to issue a call to a handle
45022 ; that it does not rightfully own. We check here to make sure that the
45023 ; issuing process is the owner of the SFT. At the same time, we do a
45024 ; SFFromHandle to really make sure that the SFT is good.
45025 ;
45026 ; ENTRY    BX has the handle
45027 ;          User_ID is the current user
45028 ; EXIT     Carry Clear => ES:DI points to SFT
45029 ;          Carry Set => AX has error code
45030 ; USES     none
45031 -----
45032
45033 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
45034 ; 21/05/2019 - Retro DOS v4.0
45035 CheckOwner:
45036 ; 13/07/2018 - Retro DOS v3.0
45037
45038 00007A66 E86AFC      call    SFFromHandle
45039 00007A69 721B      jc      short co_ret_label    ; retc
45040
45041 00007A6B 50          push    ax
45042
45043      ; MSDOS 6.0
45044
45045 ;SR; WIN386 patch - Do not check for USER_ID for using handles since these
45046 ;SR; are shared across multiple VMs in win386.
45047
45048 00007A6C 36F606[5B0F]01 test    byte [ss:Iswin386],1 ; 02/06/2019
45049 00007A72 7404      jz      short no_win386      ;win386 is not present
45050 00007A74 31C0      xor     ax,ax              ;set the zero flag
45051 00007A76 EB08      jmp     short _skip_win386
45052
45053 no_win386:
45054 00007A78 36A1[3E03]  mov     ax,[SS:USER_ID]      ;smr;SS override
45055      ;cmp    ax,[es:di+47]
45056 00007A7C 263B452F  cmp     ax,[es:di+SF_ENTRY.sf_UID]
45057
45058 _skip_win386:
45059 00007A80 58          pop     ax
45060
45061      ; 17/12/2022
45062 00007A81 7403      jz      short co_ret_label
45063      ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
45064      ;jnz    short CheckOwner_err
45065      ;retn
45066
45067 CheckOwner_err:
45068      ;mov    al,6
45069 00007A83 B006      mov     al,error_invalid_handle
45070 00007A85 F9          stc
45071
45072 co_ret_label:
45073 00007A86 C3          retn
45074
45075 ;=====
45076 ; MACRO.ASM, MSDOS 6.0, 1991
45077 ;=====
45078 ; Retro DOS v3.0 - 11/07/2018
45079 ; 21/05/2019 - Retro DOS v4.0
45080 ; 13/03/2024 - Retro DOS v5.0
45081
45082 ; TITLE    MACRO - Pathname and macro related internal routines
45083 ; NAME     MACRO
45084
45085 ; Microsoft Confidential
45086 ; Copyright (C) Microsoft Corporation 1991
45087 ; All Rights Reserved.
45088
45089 ;** MACRO.ASM
45090 ;
45091 ; $AssignOper
45092 ; FIND_DPB
45093 ; InitCDS
45094 ; $UserOper
45095 ; GetVisDrv
45096 ; GetThisDrv
45097 ; GetCDSFromDrv
45098 ;
45099 ; Revision history:
45100 ;
45101 ; Created: MZ 4 April 1983
45102 ; MZ 18 April 1983    Make TransFCB handle extended FCBS
45103 ; AR 2 June 1983      Define/Delete macro for NET redir.
45104 ; MZ 3 Nov 83         Fix InitCDS to reset length to 2
45105 ; MZ 4 Nov 83         Fix NetAssign to use STRLEN only
45106 ; MZ 18 Nov 83        Rewrite string processing for subtree
45107 ;                      aliasing.
45108 ;
45109 ; MSDOS performs several types of name translation. First, we maintain for
45110 ; each valid drive letter the text of the current directory on that drive.
45111 ; For invalid drive letters, there is no current directory so we pretend to
45112 ; be at the root. A current directory is either the raw local directory
45113 ; (consisting of drive:\path) or a local network directory (consisting of
45114 ; \machine\path. There is a limit on the point to which a .. is allowed.
45115 ;
45116 ; Given a path, MSDOS will transform this into a real from-the-root path
45117 ; without . or .. entries. Any component that is > 8.3 is truncated to
45118 ; this and all * are expanded into '?'s.
45119 ;
45120 ; The second part of name translation involves subtree aliasing. A list of
45121 ; subtree pairs is maintained by the external utility SUBST. The results of
45122 ; the previous 'canonicalization' are then examined to see if any of the

```

```

45123 ; subtree pairs is a prefix of the user path. If so, then this prefix is
45124 ; replaced with the other subtree in the pair.
45125 ;
45126 ; A third part involves mapping this "real" path into a "physical" path. A
45127 ; list of drive/subtree pairs are maintained by the external utility JOIN.
45128 ; The output of the previous translation is examined to see if any of the
45129 ; subtrees in this list are a prefix of the string. If so, then the prefix
45130 ; is replaced by the appropriate drive letter. In this manner, we can
45131 ; 'mount' one device under another.
45132 ;
45133 ; The final form of name translation involves the mapping of a user's
45134 ; logical drive number into the internal physical drive. This is
45135 ; accomplished by converting the drive number into letter:CON, performing
45136 ; the above translation and then converting the character back into a drive
45137 ; number.
45138 ;
45139 ; There are two main entry points: TransPath and TransFCB. TransPath will
45140 ; take a path and form the real text of the pathname with all . and ..
45141 ; removed. TransFCB will translate an FCB into a path and then invoke
45142 ; TransPath.
45143 ;
45144 ; A000 version 4.00 Jan. 1988
45145 ;
45146 ;Installed = TRUE
45147 ;
45148 ; I_need ThisCDS,DWORD ; pointer to CDS used
45149 ; I_need CDSAddr,DWORD ; pointer to CDS table
45150 ; I_need CDSCount,BYTE ; number of CDS entries
45151 ; I_need CurDrv,BYTE ; current macro assignment (old
45152 ; ; current drive)
45153 ; I_need NUMIO,BYTE ; Number of physical drives
45154 ; I_need fSharing,BYTE ; TRUE => no redirection allowed
45155 ; I_need DummyCDS,80h ; buffer for dummy cds
45156 ; I_need DIFFNAM,BYTE ; flag for MyName being set
45157 ; I_need MYNAME,16 ; machine name
45158 ; I_need MYNUM,WORD ; machine number
45159 ; I_need DPBHEAD,DWORD ; beginning of DPB chain
45160 ; I_need EXTERR_LOCUS,BYTE ; Extended Error Locus
45161 ; I_need DrvErr,BYTE ; drive error
45162 ;
45163 ;BREAK <$AssignOper -- Set up a Macro>
45164 ;-----
45165 ; Inputs:
45166 ; AL = 00 get assign mode (ReturnMode)
45167 ; AL = 01 set assign mode (SetMode)
45168 ; AL = 02 get attach list entry (GetAsgList)
45169 ; AL = 03 Define Macro (attch start)
45170 ; BL = Macro type
45171 ; = 0 alias
45172 ; = 1 file/device
45173 ; = 2 drive
45174 ; = 3 Char device -> network
45175 ; = 4 File device -> network
45176 ; DS:SI -> ASCIZ source name
45177 ; ES:DI -> ASCIZ destination name
45178 ; AL = 04 Cancel Macro
45179 ; DS:SI -> ASCIZ source name
45180 ; AL = 05 Modified get attach list entry
45181 ; AL = 06 Get ifsfunc item
45182 ; AL = 07 set in_use of a drive's CDS
45183 ; DL = drive number, 0=default 0=A,,
45184 ; AL = 08 reset in_use of a drive's CDS
45185 ; DL = drive number, 0=A, 1=B,,
45186 ; Function:
45187 ; Do macro stuff
45188 ; Returns:
45189 ; Std Xenix style error return
45190 ;-----
45191 ;
45192 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
45193 ; 21/05/2019 - Retro DOS v4.0
45194 ;
45195 ;_AssignOper:
45196 ; MSDOS 6.0
45197 00007A87 3C07 CMP AL,7 ; set in_use ? ;AN000;
45198 00007A89 7525 JNZ short chk08 ; no ;AN000;
45199 ; srinuse: ;AN000;
45200 00007A8B 50 PUSH AX ; save al ;AN000;
45201 00007A8C 88D0 MOV AL,DL ; AL= drive id ;AN000;
45202 00007A8E E84B01 CALL GetCDSFromDrv ; ds:si -> cds ;AN000;
45203 00007A91 58 POP AX ; ;AN000;
45204 00007A92 7216 JC short baddrv ; bad drive ;AN000;
45205 ; cmp word [si+45h],0
45206 00007A94 837C4500 CMP WORD [SI+curdir.devptr],0 ; dpb ptr =0 ? ;AN000;
45207 00007A98 7410 JZ short baddrv ; no ;AN000;
45208 00007A9A 3C07 CMP AL,7 ; set ? ;AN000;
45209 00007A9C 7506 JNZ short resetdrv ; no ;AN000;
45210 ; or word [si+43h],4000h
45211 ; 17/12/2022
45212 ; or byte [si+44h],40h
45213 00007A9E 804C4440 or byte [SI+curdir.flags+1],(curdir_inuse>>8)
45214 ; OR word [SI+curdir.flags],curdir_inuse ; set in_use ;AN000;
45215 00007AA2 EB19 JMP SHORT okdone ; ;AN000;
45216 ; resetdrv:
45217 ; and word [si+43h],0BFFFh ;AN000;
45218 ; 18/12/2022
45219 00007AA4 806444BF and byte [SI+curdir.flags+1],0BFh ; (~curdir_inuse)>>8
45220 ; AND word [SI+curdir.flags],~curdir_inuse ; reset in_use ;AN000;
45221 00007AA8 EB13 JMP SHORT okdone ; ;AN000;
45222 ; 17/12/2022
45223 ; baddrv: ;AN000;
45224 00007AAA B80F00 MOV AX,error_invalid_drive ; error ;AN000;
45225 ;
45226 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
45227 ; JMP SHORT ASS_ERR ; ;AN000;
45228 ; 17/12/2022
45229 ; 21/05/2019
45230 ; ASS_ERR:
45231 jmp SYS_RET_ERR
45232 00007AAD E9C88B
45233 ;
45234 ; chk08: ;AN000;
45235 00007AB0 3C08 CMP AL,8 ; reset inuse ? ;AN000;
45236 00007AB2 74D7 JZ short srinuse ; yes ;AN000;
45237 ;
45238 ; IF NOT INSTALLED
45239 ; transfer NET_ASSOPER
45240 ; ELSE
45241 ; MSDOS 3.3 (& MSDOS 6.0)
45242 00007AB4 50 PUSH AX
45243 ; mov ax,111Eh
45244 ; MOV AX,(MULTNET SHL 8) OR 30
45245 00007AB5 B81E11 mov ax,(MULTNET*256)+30
45246 00007AB8 CD2F int 2Fh ; Multiplex - NETWORK REDIRECTOR - DO REDIRECTION

```

```

45247             ; SS = DOS CS
45248             ; STACK: WORD function to execute
45249             ; Return: CF set on error, AX = error code
45250             ; STACK unchanged
45251 00007ABA 5B      POP     BX          ; Don't zap error code in AX
45252 00007ABB 72F0    JC       short ASS_ERR
45253 okdone:
45254 00007ABD E9AE8B   jmp     SYS_RET_OK
45255
45256             ; 17/12/2022
45257             ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
45258 ;ASS_ERR:
45259             ;jmp     SYS_RET_ERR
45260
45261             ;ENDIF
45262
45263 ;Break <FIND_DPB - Find a DPB from a drive number>
45264 -----
45265 ;** FIND_DPB - Find a DPB from a Drive #
45266 ;
45267 ; ENTRY AL has drive number A = 0
45268 ; EXIT 'C' set
45269 ;       No DPB for this drive number
45270 ;       'C' clear
45271 ;       DS:SI points to DPB for drive
45272 ; USES SI, DS, Flags
45273 -----
45274
45275             ; 21/05/2019 - Retro DOS v4.0
45276 FIND_DPB:
45277 00007AC0 36C536[2600] LDS     SI,[SS:DPBHEAD]          ;smr;SS Override
45278 fdpb5:
45279 CMP     SI,-1
45280 JZ      short fdpb10
45281 cmp     al,[si]
45282 ;CMP    AL,[SI+DPB.DRIVE]
45283 00007ACC 7406    jz      short ret_label15      ; Carry clear (retz)
45284 ;lds    si,[si+18h] ; MSDOS 3.3
45285 ;lds    si,[si+19h] ; MSDOS 6.0
45286 00007ACE C57419 LDS     SI,[SI+DPB.NEXT_DPB]
45287 00007AD1 EBF2    JMP     short fdpb5
45288 fdpb10:
45289 00007AD3 F9      STC
45290 ret_label15:
45291 00007AD4 C3      retn
45292
45293 ; Break <InitCDS - set up an empty CDS>
45294 -----
45295 ;** InitCDS - Setup an Empty CDS
45296 ;
45297 ; ENTRY ThisCDS points to CDS
45298 ; AL has uppercase drive letter
45299 ; EXIT ThisCDS is now empty
45300 ;       (ES:DI) = CDS
45301 ;       'C' set if no DPB associated with drive
45302 ; USES AH,ES,DI, Flags
45303 -----
45304
45305             ; 21/05/2019 - Retro DOS v4.0
45306             ; DOSCODE:A9FDh (MSDOS 6.21, MSDOS.SYS)
45307
45308             ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
45309             ; DOSCODE:A99Dh (MSDOS 5.0, MSDOS.SYS)
45310
45311             ; 13/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
45312             ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0BC91h
45313
45314 InitCDS:
45315             ; 19/08/2018
45316             ; 05/08/2018 - Retro DOS v3.0
45317             ; MSDOS 6.0
45318 00007AD5 50      push    ax          ; save (AL) for caller
45319 00007AD6 36C43E[A205] LES     DI,[SS:THISCDS]          ; (es:di) = CDS address
45320 ;mov    word [es:di+67],0
45321 00007ADB 26C745430000 MOV    word [ES:DI+curdir.flags],0 ; "free" CDS
45322 00007AE1 2C40    SUB     AL,"A"-1          ; A = 1
45323 00007AE3 363806[4600] CMP     [SS:NUMIO],AL          ;smr;SS Override
45324 00007AE8 7236    JC       short icdsx          ; Drive does not map a physical drive
45325 00007AEA 48      dec     ax          ; (AL) = 0 if A, 1 if B, etc.
45326 00007AEB 50      PUSH    AX          ; save drive number for later
45327 00007AEC 0441    add     al,"A"
45328 00007AEE B43A    MOV     AH,':'
45329 00007AF0 268905 mov     [ES:DI],ax
45330 ;MOV    [ES:DI+curdir.text],AX          ; set "x:"
45331 ;mov    ax,""
45332 ;mov    [es:di+2],ax
45333 ;MOV    word [ES:DI+curdir.text+2],""          ; NUL terminate
45334 00007AF3 26C745025C00 mov    word [ES:DI+curdir.text+2],005ch ; 19/08/2018
45335 ;or     word [es:di+67],4000h
45336 ;or     byte [es:di+68],40h
45337 00007AF9 26804D4440 OR      byte [ES:DI+curdir.flags+1],(curdir_inuse>>8)
45338 00007AFE 29C0    sub     ax,ax
45339 ;MOV    [es:di+73],ax ; 0
45340 00007B00 26894549 MOV     [ES:DI+curdir.ID],ax
45341 ;mov    [es:di+75],ax ; 0
45342 00007B04 2689454B MOV     [ES:DI+curdir.ID+2],ax
45343 00007B08 B002    mov     al,2
45344 ;mov    [es:di+79],ax ; 2
45345 00007B0A 2689454F MOV     [ES:DI+curdir.end],ax
45346 00007B0E 58      POP     AX          ; (al) = drive number
45347 00007B0F 1E      push    ds
45348 00007B10 56      push    si
45349 00007B11 E8ACFF   call    FIND_DPB
45350 00007B14 7208    JC       short icds5          ; 0000PPPPSSSS!!!!
45351 ;mov    [es:di+69],si
45352 00007B16 26897545 MOV     [ES:DI+curdir.devptr],SI
45353 ;mov    [es:di+71],ds
45354 00007B1A 268C5D47 MOV     [ES:DI+curdir.devptr+2],DS
45355 icds5:
45356 00007B1E 5E      pop     si
45357 00007B1F 1F      pop     ds
45358 icdsx:
45359 00007B20 58      pop     ax
45360 RET45:
45361 00007B21 C3      retn
45362
45363 ;Break <$UserOper - get/set current user ID (for net)>
45364 -----
45365 ; $UserOper - retrieve or initiate a user id string. MSDOS will only
45366 ; maintain this string and do no verifications.
45367 ;
45368 ; Inputs: AL has function type (0-get 1-set 2-printer-set 3-printer-get
45369 ;         4-printer-set-flags,5-printer-get-flags)
45370 ;         DS:DX is user string pointer (calls 1,2)

```



```

45371 ; ES:DI is user buffer (call 3)
45372 ; BX is assign index (calls 2,3,4,5)
45373 ; CX is user number (call 1)
45374 ; DX is flag word (call 4)
45375 ; Outputs: If AL = 0 then the current user string is written to DS:DX
45376 ; and user CX is set to the user number
45377 ; If AL = 3 then CX bytes have been put at input ES:DI
45378 ; If AL = 5 then DX is flag word
45379 ;-----
45380 ;
45381 ; 13/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
45382 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
45383 ; 21/05/2019 - Retro DOS v4.0
45384 _$UserOper:
45385 ; 05/08/2018 - Retro DOS v3.0
45386 ; MSDOS 6.0 (& MSDOS 3.3)
45387 ; PUSH AX
45388 ; SUB AL,1 ; quick dispatch on 0,1
45389 ; POP AX
45390 ; 01/07/2024
45391 cmp al,1
45392 JB short UserGet ; return to user the string
45393 JZ short UserSet ; set the current user
45394 CMP AL,5 ; test for 2,3,4 or 5
45395 JBE short UserPrint ; yep
45396 ;
45397 ; 13/03/2024
45398 %if 0
45399 ; mov byte [ss:EXTERR_LOCUS],1
45400 MOV byte [ss:EXTERR_LOCUS],errLOC_Unk ;smr;SS Override
45401 ; Extended Error Locus
45402 %else
45403 ; 13/03/2024 (PCDOS 7.1 IBMDOS.COM)
45404 ;;;
45405 call set_exerr_locus_unk ; Extended Error Locus
45406 ;;;
45407 %endif
45408 ;error error_invalid_function; not 0,1,2,3
45409 ;mov al,1
45410 MOV AL,error_invalid_function
45411 useroper_error:
45412 ; 17/12/2022
45413 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
45414 JMP SYS_RET_ERR
45415 ; jmp short ASS_ERR
45416 ;
45417 UserGet:
45418 ; Transfer MYNAME to DS:DX
45419 ; Set Return CX to MYNUM
45420 PUSH DS ; switch registers
45421 POP ES
45422 MOV DI,DX ; destination
45423 MOV CX,[ss:MYNUM] ; Get number ;smr;SS Override
45424 call Get_User_Stack
45425 ;mov [si+4],cx
45426 MOV [SI+user_env.user_CX],CX ; Set number return
45427 push ss ; point to DOSDATA
45428 pop ds
45429 MOV SI,MYNAME ; point source to user string
45430 UserMove:
45431 MOV CX,15
45432 REP MOVSB ; blam.
45433 XOR AX,AX ; 16th byte is 0
45434 STOSB
45435 UserBye:
45436 jmp SYS_RET_OK ; no errors here
45437 ;
45438 UserSet:
45439 ; Transfer DS:DX to MYNAME
45440 ; CX to MYNUM
45441 MOV [ss:MYNUM],CX ;smr;SS Override
45442 MOV SI,DX ; user space has source
45443 push ss
45444 pop es
45445 MOV DI,MYNAME ; point dest to user string
45446 INC byte [ss:DIFFNAM] ; signal change ;smr;SS Override
45447 JMP short UserMove
45448 ;
45449 UserPrint:
45450 ; IF NOT Installed
45451 ; transfer PRINTER_GETSET_STRING
45452 ; ELSE
45453 PUSH AX
45454 ;mov ax,111Fh
45455 ;MOV AX,(MultNET SHL 8) OR 31
45456 mov ax,(MultNET<<8)|31
45457 int 2Fh ; Multiplex - NETWORK REDIRECTOR - PRINTER SETUP
45458 ; STACK: WORD function
45459 ; Return: CF set on error, AX = error code
45460 ; STACK unchanged
45461 POP DX ; Clean stack
45462 ;JNC short OKPA
45463 jnc short UserBye ; 21/05/2019
45464 ; 17/12/2022
45465 jmp short useroper_error
45466 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
45467 ; jnb short OKPA
45468 ; jmp short useroper_error
45469 ;
45470 ; 17/12/2022
45471 ; OKPA:
45472 ; jmp short UserBye
45473 ;
45474 ; ENDIF
45475 ;
45476 ;Break <GetVisDrv - return visible drive>
45477 ;-----
45478 ; GetVisDrv - correctly map non-spliced inuse drives
45479 ;
45480 ;
45481 ; Inputs: AL has drive identifier (0=default)
45482 ; Outputs: Carry Set - invalid drive/macro
45483 ; Carry Clear - AL has physical drive (0=A)
45484 ; ThisCDS points to CDS
45485 ; Registers modified: AL
45486 ;-----
45487 ;
45488 ; 21/05/2019 - Retro DOS v4.0
45489 ; DOSCODE:AA9Fh (MSDOS 6.21, MSDOS.SYS)
45490 ;
45491 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
45492 ; DOSCODE:AA3Fh (MSDOS 5.0, MSDOS.SYS)
45493 ;
45494 GetVisDrv:

```

```

45495 ; 05/08/2018 - Retro DOS v3.0
45496 ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 6839h
45497 CALL GETTHISDRV ; get inuse drive
45498 jc short RET45
45499 push ds
45500 push si
45501 LDS SI,[SS:THISCDS] ;smr;SS override
45502 ;test word [si+67],2000h
45503 ; 17/12/2022
45504 ;test byte [si+68],20h
45505 test byte [SI+curdir.flags+1],(curdir_splice>>8)
45506 ;TEST word [SI+curdir.flags],curdir_splice
45507 pop si
45508 pop ds
45509 jz short RET45 ; if not spliced, return OK
45510 ; MSDOS 6.0
45511 ;mov byte [ss:DrvErr],0Fh
45512 MOV byte [SS:DrvErr],error_invalid_drive ;IFS. ;AN000;smr;SS override
45513 STC ; signal error
45514 retn
45515
45516 ;Break <Getthisdrv - map a drive designator (0=def, 1=A...)>
45517 -----
45518 ; GetThisDrv - look through a set of macros and return the current drive and
45519 ; macro pointer
45520 ;
45521 ; Inputs: AL has drive identifier (1=A, 0=default)
45522 ; Outputs:
45523 ; Carry Set - invalid drive/macro
45524 ; Carry Clear - AL has physical drive (0=A)
45525 ; ThisCDS points to macro
45526 ; Registers modified: AL
45527 -----
45528
45529 ; 21/05/2019 - Retro DOS v4.0
45530 ; DOSCODE:AABCh (MSDOS 6.21, MSDOS.SYS)
45531
45532 ; 02/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
45533 ; DOSCODE:AA5Ch (MSDOS 5.0, MSDOS.SYS)
45534
45535 ; 13/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
45536 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0BD48h
45537
45538 GETTHISDRV:
45539 ; 05/08/2018
45540 ; 12/07/2018 - Retro DOS v3.0
45541 ; IBMDOS.COM (MSDOS 3.3, 1987) - offset 6850h
45542 ; MSDOS 3.3 (& MSDOS 6.0)
45543 OR AL,AL ; are we using default drive?
45544 JNZ SHORT GTD10 ; no, go get the CDS pointers
45545 MOV AL,[SS:CURDRV] ; get the current drive
45546 ;INC ax ; Counteract next instruction
45547 ; 04/09/2018
45548 ;inc al
45549 ; 07/12/2022
45550 inc ax
45551 GTD10:
45552 ;DEC AX
45553 ; 02/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
45554 dec ax ; 0 = A
45555 ;dec al
45556 PUSH DS ; save world
45557 PUSH SI
45558
45559 ; 13/03/2024
45560 %if 0
45561 ;mov byte [ss:EXTERR_LOCUS],2
45562 MOV BYTE [SS:EXTERR_LOCUS],errLOC_Disk ;smr;SS override
45563 TEST BYTE [SS:FSHARING],-1 ; Logical or Physical?;smr;SS override
45564 JZ SHORT GTD20 ; Logical
45565 %else
45566 ; 13/03/2024 (PCDOS 7.1 IBMDOS.COM)
45567 ;;;
45568 call set_exerr_locus_disk
45569 cmp byte [ss:FSHARING],0 ; Logical or Physical?
45570 jz short GTD20 ; Logical
45571 ;;;
45572 %endif
45573 PUSH AX
45574 PUSH ES
45575 PUSH DI
45576 MOV WORD [SS:THISCDS],DUMMYCDS ;smr;SS override
45577 ;mov [SS:THISCDS+2],CS ; MSDOS 3.3
45578 MOV [SS:THISCDS+2],SS ; MSDOS 6.0 ;thisCDS = &DummyCDS;smr;
45579 ADD AL,'A'
45580 CALL InitCDS ; InitCDS(c);
45581 ;test word [es:di+67],4000h
45582 ; 17/12/2022
45583 ;test byte [es:di+68],40h
45584 test byte [ES:DI+curdir.flags+1],(curdir_inuse>>8)
45585 ;TEST WORD [ES:DI+curdir.flags],curdir_inuse ; clears carry
45586 POP DI
45587 POP ES
45588 POP AX
45589 JZ SHORT GTD30 ; Not a physical drive.
45590 JMP SHORT GTDX ; carry clear
45591 GTD20:
45592 CALL GetCDSFromDrv
45593 JC SHORT GTD30 ; Unassigned CDS -> return error already set
45594 ;test word [si+43h],4000h
45595 ; 17/12/2022
45596 ;test byte [si+44h],40h
45597 test byte [SI+curdir.flags+1],(curdir_inuse>>8)
45598 ;TEST WORD [SI+curdir.flags],curdir_inuse ; clears Carry
45599 JNZ SHORT GTDX ; carry clear
45600 GTD30:
45601 ; 21/05/2019
45602 ; MSDOS 6.0
45603 MOV AL,error_invalid_drive; invalid FAT drive
45604 MOV BYTE [ss:DrvErr],AL ; save this for IOCTL
45605
45606 ; 13/03/2024
45607 %if 0
45608 ; MSDOS 3.3 (& MSDOS 6.0)
45609 MOV BYTE [ss:EXTERR_LOCUS],errLOC_Unk
45610 %else
45611 ; 13/03/2024 (PCDOS 7.1 IBMDOS.COM)
45612 ;;;
45613 call set_exerr_locus_unk
45614 ;;;
45615 %endif
45616 STC
45617 GTDX:
45618 POP SI ; restore world

```



```

45619 00007BDA 1F      POP     DS
45620 00007BDB C3      RETN
45621
45622 ;Break <GetCDSFromDrv - convert a drive number to a CDS pointer>
45623 -----
45624 ; GetCDSFromDrv - given a physical drive number, convert it to a CDS
45625 ; pointer, returning an error if the drive number is greater than the
45626 ; number of CDS's
45627 ;
45628 ; Inputs: AL is physical unit # A=0...
45629 ; Outputs: Carry Set if Bad Drive
45630 ;          Carry Clear
45631 ;          DS:SI -> CDS
45632 ;          [THISCDS] = DS:SI
45633 ; Registers modified: DS,SI
45634 -----
45635
45636 ; 21/05/2019 - Retro DOS v4.0
45637 GetCDSFromDrv:
45638 00007BDC 363A06[4700] CMP     AL,[SS:CDSCOUNT] ; is this a valid designator;smr;SS Override
45639 ;JB     SHORT GetCDS ; cf=1 ; yes, go get the macro
45640 ;STC    ; signal error
45641 ;RETN   ; bye
45642 ; 23/09/2023
45643 00007BE1 F5      cmc     ; cf=1 <-> cf=0
45644 00007BE2 7217    jc      short GetCDS_retn
45645
45646 GetCDS:
45647 ; 23/09/2023
45648 00007BE4 50      ;PUSH  BX
45649 00007BE5 36C536[3C00] PUSH  AX
45650 LDS     SI,[SS:CDSADDR] ; get pointer to table;smr;SS Override
45651 ;mov    bl,81 ; MSDOS 3.3
45652 ;mov    bl,88 ; MSDOS 6.0
45653 ; 23/09/2023
45654 ;MOV    BL,curdir.size ; size in convenient spot
45655 00007BEA B458    ;MUL    BL ; get net offset
45656 00007BEC F6E4    mov     ah,curdir.size
45657 00007BEE 01C6    mul     ah
45658 00007BF0 368936[A205] ADD     SI,AX ; * ; convert to true pointer
45659 00007BF5 368C1E[A405] MOV     [SS:THISCDS],SI ; store convenient offset;smr;SS Override
45660 00007BFA 58      MOV     [SS:THISCDS+2],DS ; store convenient segment;smr;SS Override
45661 POP     AX
45662 ; 23/09/2023
45663 ;POP    BX
45664 ; (cf must be 0 here) ; *
45665 ;CLC    ; no error
45666 00007BFB C3      GetCDS_retn:
45667 RETN ; bye!
45668
45669 ;=====
45670 ; MACRO2.ASM, MSDOS 6.0, 1991
45671 ;=====
45672 ; Retro DOS v3.0 - 12/07/2018
45673 ; 22/05/2019 - Retro DOS v4.0
45674 ; 13/03/2024 - Retro DOS v5.0
45675
45676 ;BREAK <TransFCB - convert an FCB into a path, doing substitution>
45677 -----
45678 ; TransFCB - Copy an FCB from DS:DX into a reserved area doing all of the
45679 ; gritty substitution.
45680 ;
45681 ; Inputs: DS:DX - pointer to FCB
45682 ;          ES:DI - point to destination
45683 ; Outputs: Carry Set - invalid path in final map
45684 ;          Carry Clear - FCB has been mapped into ES:DI
45685 ;          Sattrib is set from possibly extended FCB
45686 ;          ExtFCB set if extended FCB found
45687 ; Registers modified: most
45688 -----
45689
45690 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
45691 TransFCB:
45692 ; 22/05/2019 - Retro DOS v4.0
45693 ; 12/07/2018 - Retro DOS v3.0
45694 ;LocalVar FCBtmp,16
45695 ;ENTER
45696 00007BFC 55      push    bp
45697 00007BFD 89E5    mov     bp,sp
45698 00007BFF 83EC10 sub     sp,15 ; MSDOS 3.3
45699 00007C02 16      sub     sp,16 ; MSDOS 6.0
45700 00007C03 07      push    ss
45701 00007C04 06      pop     es
45702 00007C05 57      push    es
45703 push     di
45704 ;lea     di,[bp-15] ; MSDOS 3.3
45705 00007C06 8D7EF0 ;LEA     DI,FCBtmp
45706 00007C09 36C606[6C05]00 lea     di,[bp-16] ; point to FCB temp area
45707 00007C0F 36C606[6D05]00 mov     byte [SS:EXTFCB],0 ; no extended FCB found;smr;SS Override
45708 00007C15 E832A6 mov     byte [SS:SATTRIB],0 ; default search attributes;smr;SS Override
45709 ; call    GetExtended ; get FCB, extended or not
45710 00007C18 740D    jz      short GetDrive ; not an extended FCB, get drive
45711 00007C1A 8A44FF mov     AL,[SI-1] ; get attributes
45712 00007C1D 36A2[6D05] mov     [SS:SATTRIB],AL ; store search attributes;smr;SS Override
45713 00007C21 36C606[6C05]FF mov     byte [SS:EXTFCB],-1 ; signal extended FCB ;smr;SS Override
45714
45715 00007C27 AC      GetDrive:
45716 00007C28 E862FF lodsb ; get drive byte
45717 00007C2B 722A    call    GETTHISDRV
45718 00007C2D E86F03 jc      short BadPack
45719 call     TextFromDrive ; convert 0-based drive to text
45720
45721 ; Scan the source to see if there are any illegal chars
45722
45723 ;mov     bx,CharType ; load lookup table
45724 00007C30 B90B00 mov     cx,11
45725 00007C33 56      push    si ; back over name, ext
45726 00007C34 AC      FCBScan:
45727 lodsb ; get a byte
45728
45729 ; 09/08/2018
45730 ;xlat    byte [es:bx]
45731 ;es      xlat
45732
45733 ; 22/05/2019 - Retro DOS v4.0
45734 00007C35 E897E1 call    GetCharType ; get flags
45735
45736 00007C38 A808    test    al,8
45737 00007C3A 741B    jz      short BadPack
45738
45739 00007C3C E2F6    NextCh:
45740 00007C3E 5E      loop   FCBScan
45741 00007C3F 89FB    pop     si
45742 00007C41 E863AA mov     bx,di
45743 call     PackName ; crunch the path

```

```

45743 00007C44 5F      pop     di                ; get original destination
45744 00007C45 07      pop     es
45745 00007C46 16      push    ss                ; get DS addressability
45746 00007C47 1F      pop     ds
45747      ;lea     si,[bp-15] ; MSDOS 3.3
45748      ;LEA     SI,FCBTmp      ; point at new pathname
45749 00007C48 8D76F0      lea     si,[bp-16]
45750 00007C4B 803F00      cmp     byte [bx],0
45751 00007C4E 7407      jz      short BadPack
45752 00007C50 55      push    bp
45753 00007C51 E80E00      call    TransPathSet      ; convert the path
45754 00007C54 5D      pop     bp
45755 00007C55 7303      jnc     short FCBRet      ; bye with transPath error code
45756      BadPack:
45757 00007C57 F9      STC
45758      ;mov     al,3
45759 00007C58 B003      MOV     AL,error_path_not_found
45760      FCBRet:
45761      ;LEAVE
45762 00007C5A 89EC      mov     sp,bp
45763 00007C5C 5D      pop     bp
45764      TransPath_retn:
45765 00007C5D C3      retn
45766
45767 ; 12/07/2018 - Retro DOS v3.0
45768
45769 ;BREAK <TransPath - copy a path, do string sub and put in current dir>
45770 -----
45771 ;
45772 ; TransPath - copy a path from DS:SI to ES:DI, performing component string
45773 ; substitution, insertion of current directory and fixing . and ..
45774 ; entries. Perform splicing. Allow input string to match splice
45775 ; exactly.
45776 ;
45777 ; TransPathSet - Same as above except No splicing is performed if input path
45778 ; matches splice.
45779 ;
45780 ; TransPathNoSet - No splicing/local using is performed at all.
45781 ;
45782 ; The following anomalous behaviour is required:
45783 ;
45784 ; Drive letters on devices are ignored. (set up DummyCDS)
45785 ; Paths on devices are ignored. (truncate to 0-length)
45786 ; Raw net I/O sets ThisCDS => NULL.
45787 ; fSharing => dummyCDS and no subst/splice. Only canonicalize.
45788 ;
45789 ; Other behaviour:
45790 ;
45791 ; ThisCDS set up.
45792 ; FatRead done on local CDS.
45793 ; ValidateCDS done on local CDS.
45794 ;
45795 ; Brief flowchart:
45796 ;
45797 ; if fSharing then
45798 ;     set up DummyCDS (ThisCDS)
45799 ;     canonicalize (sets cMeta)
45800 ;     splice
45801 ;     fatRead
45802 ;     return
45803 ; if \\ or d:\ lead then
45804 ;     set up null CDS (ThisCDS)
45805 ;     canonicalize (sets cMeta)
45806 ;     return
45807 ; if device then
45808 ;     set up dummyCDS (ThisCDS)
45809 ;     canonicalize (sets cMeta)
45810 ;     return
45811 ; if file then
45812 ;     getCDS (sets (ThisCDS) from name)
45813 ;     validateCDS (may reset current dir)
45814 ;     Copy current dir
45815 ;     canonicalize (set cMeta)
45816 ;     splice
45817 ;     generate correct CDS (ThisCDS)
45818 ;     if local then
45819 ;         fatread
45820 ;     return
45821 ;
45822 ; Inputs: DS:SI - point to ASCIZ string path
45823 ; DI - point to buffer in DOSDATA
45824 ; Outputs: Carry Set - invalid path specification: too many .., bad
45825 ; syntax, etc. or user FAILED to I 24.
45826 ; WFP_Start - points to beginning of buffer
45827 ; Curr_Dir_End - points to end of current dir in path
45828 ; DS - DOSDATA
45829 ; Registers modified: most
45830 -----
45831 ;
45832 ; 22/05/2019
45833 ; 13/05/2019 - Retro DOS v4.0
45834 ; DOSCODE:AB99h (MSDOS 6.21, MSDOS.SYS)
45835 ;
45836 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
45837 ; DOSCODE:AB39h (MSDOS 5.0, MSDOS.SYS)
45838 ;
45839 ; 13/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
45840 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:0BE1Dh
45841
45842      TransPath:
45843      XOR     AL,AL
45844 00007C5E 30C0      JMP     SHORT SetSplice
45845 00007C60 EB02
45846      TransPathSet:
45847 00007C62 B0FF      MOV     AL,-1
45848      SetSplice:
45849 00007C64 36A2[4C03]      MOV     [SS:NoSetDir],AL      ; NoSetDir = !fExact; ;smr;SS Override
45850 00007C68 B0FF      MOV     AL,-1
45851      TransPathNoSet:
45852 00007C6A 36A2[7105]      MOV     [SS:FSPLICE],AL      ; fSplice = TRUE; ;smr;SS Override
45853 00007C6E 36C606[7A05]FF      MOV     byte [SS:CMETA],-1      ;smr;SS Override
45854 00007C74 36893E[B205]      MOV     [SS:WFP_START],DI      ;smr;SS Override
45855 00007C79 36C706[B605]FFFF      MOV     word [SS:Curr_Dir_End],-1 ; crack from start;smr;SS Override
45856 00007C80 16      push    ss
45857 00007C81 07      pop     es
45858      ;lea     bp,[di+134]
45859 00007C82 8DAD8600      LEA     BP,[DI+TEMPLEN]      ; end of buffer
45860 ;
45861 ; if this is through the server dos call, fsharing is set. We set up a
45862 ; dummy cds and let the operation go.
45863 ;
45864 ; ;TEST byte [SS:FSHARING],-1 ; if no sharing ;smr;SS Override
45865 ; ;JZ short CheckUNC ; skip to UNC check
45866 ; 13/03/2024 (PCDOS 7.1 IBMDOS.COM)

```

```

45867      ;;;
45868 00007C86 36803E[7205]00      cmp     byte [ss:FSHARING],0
45869 00007C8C 7435                jz      short CheckUNC
45870      ;;;
45871      ;
45872      ; ES:DI point to buffer
45873      ;
45874 00007C8E E8F802      CALL     DriveFromText      ; get drive and advance DS:SI
45875 00007C91 E8F9FE      call    GETTHISDRV        ; Set ThisCDS and convert to 0-based
45876 00007C94 722A      jc      short NoPath
45877 00007C96 E80603      CALL     TextFromDrive      ; drop in new
45878 00007C99 8D5D01      LEA     BX,[DI+1]          ; backup limit
45879 00007C9C E83401      CALL     Canonicalize       ; copy and canonicalize
45880 00007C9F 72BC      jc      short TransPath_retn ; errors
45881      ;
45882      ; Perform spllices for net guys.
45883      ;
45884 00007CA1 16      push    ss
45885 00007CA2 1F      pop     ds
45886 00007CA3 8B36[B205]      MOV     SI,[WFP_START]      ; point to name
45887 00007CA7 F606[7105]FF      TEST    byte [FSPLICE],-1
45888 00007CAC 7403      JZ      short NoServerSplice
45889 00007CAE E82D02      CALL     Splice
45890      NoServerSplice:
45891 00007CB1 16      push    ss
45892 00007CB2 1F      pop     ds      ; for FATREAD
45893 00007CB3 C43E[A205]      LES     DI,[THISCDS]        ; for fatread
45894 00007CB7 E8289C      call    ECritDisk
45895 00007CBA E89EE8      call    FATREAD_CDS
45896 00007CBD E84F9C      call    LCritDisk
45897      NoPath:
45898      ;mov     al,3
45899 00007CC0 B003      MOV     AL,error_path_not_found ; Set up for possible bad path error
45900 00007CC2 C3      retn      ; any errors are in Carry flag
45901      ;
45902      ; Let the network decide if the name is for a spooled device. It will map
45903      ; the name if so.
45904      ;
45905      ;
45906 00007CC3 36C706[A205]FFFF      CheckUNC:
45907      MOV     WORD [SS:THISCDS],-1 ; NULL thisCDS ;smr;SS Override
45908 00007CCA 882311      ;call Install NetSpoolCheck,MultNET,35
45909 00007CCD CD2F      mov     ax,1123h
45910      int     2Fh      ; Multiplex - NETWORK REDIRECTOR - QUALIFY REMOTE FILENAME
45911      ; DS:SI -> ASCII filename to canonicalize
45912      ; ES:DI -> 128-byte buffer for qualified name
45913 00007CCF 7329      ; Return: CF set if not resolved
45914      JNC     short UNCDone
45915      ;
45916      ; At this point the name is either a UNC-style name (prefixed with two leading
45917      ; \\s) or is a local file/device. Remember that if a net-spooled device was
45918      ; input, then the name has been changed to the remote spooler by the above net
45919      ; call. Also, there may be a drive in front of the \\.
45920      ;
45921 00007CD1 E8B502      NO_CHECK:
45922 00007CD4 50      CALL     DriveFromText      ; eat drive letter
45923 00007CD5 8B04      PUSH     AX      ; save it
45924 00007CD7 E810E1      MOV     AX,[SI]      ; get first two bytes of path
45925 00007CDA 86E0      call    PATHCHRCMP        ; convert to normal form
45926 00007CDC E80BE1      XCHG     AH,AL      ; swap for second byte
45927 00007CDF 751F      call    PATHCHRCMP        ; convert to normal form
45928 00007CE1 38C4      JNZ     short CheckDevice   ; not a path char
45929 00007CE3 751B      CMP     AH,AL      ; are they same?
45930      JNZ     short CheckDevice ; nope
45931      ;
45932      ; We have a UNC request. We must copy the string up to the beginning of the
45933      ; local machine root path
45934 00007CE5 58      POP     AX
45935 00007CE6 A5      MOVSW      ; get the lead \\.
45936      UNCCpy:
45937 00007CE7 AC      LODSB      ; get a byte
45938 00007CE8 E8ACE0      call    UCase      ;AN000; convert the char
45939 00007CEB 08C0      OR     AL,AL
45940 00007CED 740E      JZ      short UNCTerm      ; end of string. All done.
45941 00007CEF E8F8E0      call    PATHCHRCMP        ; is it a path char?
45942 00007CF2 89FB      MOV     BX,DI      ; backup position
45943 00007CF4 AA      STOSB
45944 00007CF5 75F0      JNZ     short UNCCpy      ; no, go copy
45945 00007CF7 E8D900      CALL     Canonicalize      ; wham (and set cMeta)
45946      UNCDone:
45947 00007CFA 16      push    ss
45948 00007CFB 1F      pop     ds
45949 00007CFC C3      retn      ; return error code
45950      UNCTerm:
45951 00007CFD AA      STOSB      ;AN000;
45952 00007CFE EBFA      JMP     short UNCDone      ;AN000;
45953      ;
45954      CheckDevice:
45955      ;
45956      ; Check DS:SI for device. First eat any path stuff
45957      ;
45958 00007D00 58      POP     AX      ; retrieve drive info
45959 00007D01 803C00      CMP     BYTE [SI],0      ; check for null file
45960 00007D04 7504      JNZ     short CheckPath
45961      ;mov     al,2
45962 00007D06 B002      MOV     AL,error_file_not_found ; bad file error
45963 00007D08 F9      STC      ; signal error on null input
45964 00007D09 C3      RETN      ; bye!
45965      CheckPath:
45966 00007D0A 50      push    ax
45967 00007D0B 55      push    bp      ; save drive number
45968      ;
45969      ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
45970      %if 0
45971      ; MSDOS 6.0
45972      ;;;BUGBUG BUG 10-26-1992 scottq
45973      ;;;This is a hack for the CDRom extensions (2.1) who scan looking
45974      ;;;for the following POP BP == 5Dh (restore <bp,ax>).
45975      ;;;The problem is that a direct call to CheckThisDevice can (and did)
45976      ;;;end up having a 5D in the opcode's displacement field. The
45977      ;;;scanning code would choke on this thinking it was a POP BP instruction.
45978      ;;;
45979      ;;;what we do here is do a call to a function that is less than 5Dh
45980      ;;;bytes away (and assert its not exactly 5D away) that jmps (transfers)
45981      ;;;to the correct function. This cannot accidently insert a 5Dh.
45982      ;;;
45983      ;;;More info:
45984      ;;; This particular scan is begun at the UNCDone label for 32 bytes
45985      ;;;looking for pop BP, so you cannot put a 5D between here and there.
45986      ;;;
45987      call    no5Dshere
45988      start5Dhack:
45989      ;following is replaced with 5Dhack code--Invoke CheckThisDevice
45990      backfrom5Dhack:

```

```

45991
45992
45993
45994
45995
45996
45997
45998
45999
46000
46001
46002
46003
46004
46005 00007D0C E81ED2
46006
46007 00007D0F 5D
46008 00007D10 58
46009 00007D11 731C
46010
46011
46012
46013
46014
46015 00007D13 36C606[7205]FF
46016 00007D19 E871FE
46017 00007D1C 36C606[7205]00
46018
46019
46020
46021
46022
46023
46024
46025 00007D22 E87A02
46026 00007D25 B02F
46027 00007D27 AA
46028 00007D28 E8869A
46029
46030 00007D2B F8
46031 00007D2C 16
46032 00007D2D 1F
46033
46034 00007D2E C3
46035
46036
46037
46038
46039
46040
46041
46042
46043
46044
46045
46046
46047
46048
46049
46050
46051
46052
46053
46054
46055
46056
46057 00007D2F E83FFE
46058
46059 00007D32 B003
46060 00007D34 72F8
46061
46062
46063
46064
46065
46066
46067 00007D36 1E
46068 00007D37 56
46069 00007D38 06
46070 00007D39 57
46071 00007D3A E823D1
46072
46073 00007D3D 5F
46074 00007D3E 07
46075 00007D3F 5E
46076 00007D40 1F
46077
46078 00007D41 B003
46079
46080 00007D43 72E9
46081
46082
46083
46084
46085 00007D45 1E
46086 00007D46 56
46087 00007D47 36C536[A205]
46088 00007D4C 89FB
46089
46090 00007D4E 035C4F
46091
46092 00007D51 8DAD8600
46093
46094 00007D55 E8689A
46095 00007D58 4F
46096
46097
46098
46099 00007D59 B05C
46100 00007D5B 263845FF
46101 00007D5F 7401
46102 00007D61 AA
46103
46104
46105
46106
46107 00007D62 4F
46108 00007D63 5E
46109 00007D64 1F
46110
46111
46112
46113
46115 00007D65 E8CE00

%endif
; 13/03/2024
; PCDOS 7.1 IBMDOS.COM - DOSCODE:BECBh
; (MSDOS 6.22 MSDOS.SYS - DOSCODE:AC47h)
; ((Windows ME IO.SYS - BIOSCODE:A6C2h))
%if 0
call no5Dshere
%else
; 13/03/2024 - Retro DOS v5.0
; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
; Note: 'call no5Dshere' is not required for MSDOS 5.0 MSDOS.SYS
call CheckThisDevice ; E8h,6Fh,0D6h
%endif
pop bp
pop ax ; get drive letter back
JNC short DoFile ; yes we have a file.

; We have a device. AX has drive letter. At this point we may fake a CDS ala
; sharing DOS call. We know by getting here that we are NOT in a sharing DOS
; call.
MOV byte [SS:FSHARING],-1 ; simulate sharing dos call;smr;SS Override
call GETTHISDRV ; set ThisCDS and init DUMMYCDS
MOV byte [SS:FSHARING],0 ; ;smr;SS Override

; Now that we have noted that we have a device, we put it into a form that
; getpath can understand. Normally getpath requires d:\ to begin the input
; string. We relax this to state that if the d:\ is present then the path
; may be a file. If D:/ (note the forward slash) is present then we have
; a device.
CALL TextFromDrive
MOV AL,'/' ; path sep.
STOSB
call StrCpy ; move remainder of string
CLC ; everything OK.
push ss
pop ds ; remainder of OK stuff
DoFile_retn:
retn

; 13/03/2024 - Retro DOS v5.0
; PCDOS 7.1 IBMDOS.COM - DOSCODE:BEEHh
; (MSDOS 6.22 MSDOS.SYS - DOSCODE:AC6Ah)
; ((Windows ME IO.SYS - BIOSCODE:A6F2h))
%if 1
; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
%if 0
no5Dshere:
; 10/08/2018
jmp CheckThisDevice ; snoop for device
%endif

; .erre (no5Dshere - start5Dhack - 5D)

; We have a file. Get the raw CDS.
DoFile:
; MSDOS 3.3 (& MSDOS 6.0)
call GetVisDrv ; get proper CDS
;mov al,3
MOV AL,error_path_not_found ; Set up for possible bad file error
jc short DoFile_retn ; CARRY set -> bogus drive/spliced

; ThisCDS has correct CDS. DS:SI advanced to point to beginning of path/file.
; Make sure that CDS has valid directory; ValidateCDS requires a temp buffer
; Use the one that we are going to use (ES:DI).
;SAVE <DS,SI,ES,DI> ; save all string pointers.
push ds
push si
push es
push di
call ValidateCDS ; poke CDS and make everything OK
;RESTORE <DI,ES,SI,DS> ; get back pointers
pop di
pop es
pop si
pop ds
;mov al,3
MOV AL,error_path_not_found ; Set up for possible bad path error
;retc ; someone failed an operation
jc short DoFile_retn

; ThisCDS points to correct CDS. It contains the correct text of the
; current directory. Copy it in.
push ds
push si
LDS SI,[SS:THISCDS] ; point to CDS ;smr;SS Override
MOV BX,DI ; point to destination
;add bx,[si+79] ; MSDOS 6.0
ADD BX,[SI+curdir.end] ; point to backup limit
;lea bp,[di+134]
LEA BP,[DI+TEMPLEN] ; regenerate end of buffer
;AN000;
call FStrCpy ; copy string. ES:DI point to end
DEC DI ; point to NUL byte

; Make sure that there is a path char at end.
MOV AL,'\'
CMP [ES:DI-1],AL
JZ short GetOrig
STOSB

; Now get original string.
GetOrig:
DEC DI ; point to path char
pop si
pop ds

; BX points to the end of the root part of the CDS (at where a path char
; should be). Now, we decide whether we use this root or extend it with the
; current directory. See if the input string begins with a leading
CALL PathSep ; is DS:SI a path sep?

```

```

46116 00007D68 7511      JNZ     short PathAssure      ; no, DI is correct. Assure a path char
46117 00007D6A 08C0      OR      AL,AL                ; end of string?
46118 00007D6C 7410      JZ      short DoCanon        ; yes, skip.
46119
;
; The string does begin with a \. Reset the beginning of the canonicalization
; to this root. Make sure that there is a path char there and advance the
; source string over all leading \s.
46120
;
46121
46122
46123
46124 00007D6E 89DF      MOV     DI,BX                ; back up to root point.
46125
SkipPath:
46126 00007D70 AC          LODSB
46127 00007D71 E876E0     call    PATHCHRCMP
46128 00007D74 74FA      JZ      short SkipPath
46129 00007D76 4E          DEC     SI
46130 00007D77 08C0      OR      AL,AL
46131 00007D79 7403      JZ      short DoCanon
46132
; DS:SI start at some file name. ES:DI points at some path char. Drop one in
; for yucks.
46133
46134
46135
46136
PathAssure:
46137 00007D7B B05C      MOV     AL,'\' ; 5Ch
46138 00007D7D AA          STOSB
46139
; ES:DI point to the correct spot for canonicalization to begin.
46140
; BP is the max extent to advance DI
46141
; BX is the backup limit for ..
46142
46143
46144
DoCanon:
46145 00007D7E E85200     call    Canonicalize          ; wham.
46146                                ;retc          ; badly formatted path.
46147 00007D81 72AB      jc      short DoFile_retn
46148
; The string has been moved to ES:DI. Reset world to DOS context, pointers
; to wfp_start and do string substitution. BP is still the max position in
; buffer.
46149
46150
46151
46152
46153 00007D83 16          push    ss
46154 00007D84 1F          pop     ds
46155 00007D85 8B3E[B205]   MOV     DI,[WFP_START]        ; DS:SI point to string
46156 00007D89 C536[A205]   LDS     SI,[THISCDS]          ; point to CDS
46157 00007D8D E81702     CALL    PathPref              ; is there a prefix?
46158 00007D90 7514      JNZ     short DoSplice        ; no, do splice
46159
; we have a match. Check to see if we ended in a path char.
46160
46161
46162 00007D92 8A44FF     MOV     AL,[SI-1]             ; last char to match
46163 00007D95 E852E0     call    PATHCHRCMP            ; did we end on a path char? (root)
46164 00007D98 740C      JZ      short DoSplice        ; yes, no current dir here.
46165                                ; 2/13/KK
46166
Pathline:
46166 00007D9A 26803D00     cmp     BYTE [ES:DI],0        ; end at NUL?
46167 00007D9E 7406      JZ      short DoSplice
46168 00007DA0 47          INC     DI                    ; point to after current path char
46169 00007DA1 36893E[B605]   MOV     [SS:CURR_DIR_END],DI ; point to correct spot ;smr;SS override
46170
; Splice the result.
46171
46172
46173
DoSplice:
46174 00007DA6 16          push    ss
46175 00007DA7 1F          pop     ds
46176 00007DA8 8B36[B205]   MOV     SI,[WFP_START]        ; back to DOSDATA
46177 00007DAC 31C9      XOR     CX,CX                 ; point to beginning of string
46178 00007DAE F606[7105]FF     TEST    byte [FSPLICE],-1
46179 00007DB3 7403      JZ      short SkipSplice
46180 00007DB5 E82601     CALL    Splice                ; replaces in place.
46181
SkipSplice:
46182
; The final thing is to assure ourselves that a FATREAD is done on the local
; device.
46183
46184
46185
46186 00007DB8 16          push    ss
46187 00007DB9 1F          pop     ds
46188 00007DBA C43E[A205]   LES     DI,[THISCDS]          ; point to correct drive
46189                                ;test word [es:di+67],8000h
46190                                ; 17/12/2022
46191                                ;test byte [es:di+68],80h
46192 00007DBE 26F6454480     test    byte [ES:DI+curdir.flags+1],curdir_isnet>>8 ; 04/12/2022
46193                                ;TEST word [ES:DI+curdir.flags],curdir_isnet ; 8000h
46194 00007DC3 750D      JNZ     short Done            ; net, no fatread necessary (retnz)
46195 00007DC5 E30B      JCXZ    Done
46196 00007DC7 E8189B     call    ECritDisk
46197 00007DCA E88EE7     call    FATREAD_CDS
46198 00007DCD E83F9B     call    LCritDisk
46199                                ;mov al, 3
46200 00007DD0 B003      MOV     AL,error_path_not_found ; Set up for possible bad path error
46201
Done:
46202 00007DD2 C3          retn                        ; any errors in carry flag.
46203
; 13/07/2018
46204
;BREAK <Canonicalize - copy a path and remove . and .. entries>
46205
;-----
46206
; Canonicalize - copy path removing . and .. entries.
46207
;
; Inputs:      DS:SI - point to ASCIZ string path
;              ES:DI - point to buffer
;              BX - backup limit (offset from ES) points to slash
;              BP - end of buffer
; Outputs:     Carry Set - invalid path specification: too many .., bad
;              syntax, etc.
;              Carry Clear -
;              DS:DI - advanced to end of string
;              ES:DI - advanced to end of canonicalized form after nul
; Registers modified: AX CX DX (in addition to those above)
;-----
46208
;
46209
46210
46211
46212
46213
46214
46215
46216
46217
46218
46219
46220
46221
46222
46223
46224
46225
46226
46227
46228
46229
46230 00007DD3 AC          LODSB
46231 00007DD4 E813E0     call    PATHCHRCMP            ; while (PathChr (*s))
46232 00007DD7 7507      JNZ     short CanonDec
46233 00007DD9 39EF      CMP     DI,BP                ; if (d > dlim)
46234 00007ddb 7319      JAE     short CanonBad        ; goto error;
46235 00007DDD AA          STOSB
46236 00007DDE EBF3      JMP     short Canonicalize     ; *d++ = *s++;
46237
CanonDec:
46238 00007DE0 4E          DEC     SI
46239

```

```

46240 ; Main canonicalization loop. We come here with DS:SI pointing to a textual
46241 ; component (no leading path separators) and ES:DI being the destination
46242 ; buffer.
46243
46244 CanonLoop:
46245
46246 ; If we are at the end of the source string, then we need to check to see that
46247 ; a potential drive specifier is correctly terminated with a path sep char.
46248 ; Otherwise, do nothing
46249
46250 XOR     AX,AX
46251 CMP     [SI],AL
46252 JNZ     short DoComponent
46253 CMP     BYTE [ES:DI-1],':'
46254 JNZ     short DoTerminate
46255 MOV     AL,'\'
46256 STOSB
46257 MOV     AL,AH
46258 DoTerminate:
46259 STOSB
46260 CLC
46261 retn
46262
46263 CanonBad:
46264 CALL    ScanPathChar
46265 ;mov    al,3
46266 MOV     AL,error_path_not_found
46267 JZ      short PathEnc
46268 ;mov    al,2
46269 MOV     AL,error_file_not_found
46270 PathEnc:
46271 STC
46272 CanonBad_retn:
46273 retn
46274
46275 ; We have a textual component that we must copy. We uppercase it and truncate
46276 ; it to 8.3
46277
46278 DoComponent:
46279 CALL    CopyComponent
46280 JC      short CanonBad_retn
46281
46282 ; We special case the . and .. cases. These will be backed up.
46283
46284 ;CMP    WORD PTR ES:[DI], '.' + (0 SHL 8)
46285 CMP     WORD [ES:DI],002Eh
46286 JZ      short Skip1
46287 ;CMP    WORD PTR ES:[DI], '..'
46288 CMP     WORD [ES:DI],2E2Eh
46289 JNZ     short CanonNormal
46290 DEC     DI
46291 Skip1:
46292 CALL    SkipBack
46293 ;mov    al,3
46294 ; 07/07/2024 (*)
46295 MOV     AL,error_path_not_found
46296 JC      short CanonBad_retn
46297 JMP     short CanonPath
46298
46299 ; We have a normal path. Advance destination pointer over it.
46300
46301 CanonNormal:
46302 ADD     DI,CX
46303
46304 ; We have successfully copied a component. We are now pointing at a path
46305 ; sep char or are pointing at a nul or are pointing at something else.
46306 ; If we point at something else, then we have an error.
46307
46308 CanonPath:
46309 CALL    PathSep
46310 JNZ     short CanonBad
46311
46312 ; Copy the first path char we see.
46313
46314 LODSB
46315 CALL    PATHCHRCMP
46316 JNZ     short CanonDec
46317 CMP     DI,BP
46318 JAE     short CanonBad
46319 STOSB
46320
46321 ; Skip all remaining path chars
46322
46323 CanonPathLoop:
46324 LODSB
46325 CALL    PATHCHRCMP
46326 JZ      short CanonPathLoop
46327 DEC     SI
46328 JMP     short CanonLoop
46329
46330 ;BREAK <PathSep - determine if char is a path separator>
46331 ;-----
46332 ; PathSep - look at DS:SI and see if char is / \ or NUL
46333 ; Inputs: DS:SI - point to a char
46334 ; Outputs: AL has char from DS:SI (/ => \)
46335 ; Zero set if AL is / \ or NUL
46336 ; Zero reset otherwise
46337 ; Registers modified: AL
46338 ;-----
46339
46340 PathSep:
46341 MOV     AL,[SI]
46342 PathSepGotch:
46343 OR      AL,AL
46344 JZ      short CanonBad_retn
46345 CALL    PATHCHRCMP
46346 ;retn
46347 ; 18/12/2022
46348 JMP     PATHCHRCMP
46349
46350 ;BREAK <SkipBack - move backwards to a path separator>
46351 ;-----
46352 ; SkipBack - look at ES:DI and backup until it points to a / ; Inputs: ES:DI - point to a char
46353 ; BX has current directory back up limit (point to a / \)
46354 ; Outputs: ES:DI backed up to point to a path char
46355 ; AL has char from output ES:DI (path sep if carry clear)
46356 ; Carry set if illegal backup
46357 ; Carry Clear if ok
46358 ; Registers modified: DI,AL
46359 ;-----
46360
46361 SkipBack:
46362 CMP     DI,BX
46363 ; while (TRUE) {

```



```

46365 00007E41 720A      JB      short SkipBad      ;      if (d < dlim)
46366 00007E43 4F        DEC      DI                ;      goto err;
46367 00007E44 268A05    MOV     AL,[ES:DI]         ;      if (pathchr (*--d))
46368 00007E47 E8A0DF    call    PATHCHRCMP        ;      break;
46369 00007E4A 75F3      JNZ     short SkipBack    ;      }
46370                ;CLC                ;      return (0);
46371                ; 01/07/2024
46372                ; cf=0
46373 00007E4C C3        retn                 ;
46374                ;err:
46375                ;mov     al,3
46376 00007E4D B003    MOV     AL,error_path_not_found ; bad path error
46377                ;STC                ;      return (-1);
46378                ; 01/07/2024
46379                ; cf=1
46380 00007E4F C3        retn                 ;
46381
46382 ;Break <CopyComponent - copy out a file path component>
46383 ;-----
46384 ;      CopyComponent - copy a file component from a path string (DS:SI) into ES:DI
46385 ;
46386 ;      Inputs:      DS:SI - source path
46387 ;                  ES:DI - destination
46388 ;                  ES:BP - end of buffer
46389 ;      Outputs:    Carry Set - too long
46390 ;                  Carry Clear - DS:SI moved past component
46391 ;                  CX has length of destination
46392 ;      Registers modified: AX,CX,DX
46393 ;-----
46394
46395 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
46396
46397 ; 13/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
46398
46399 CopyComponent:
46400
46401 %define CopyBP      [BP]      ; word
46402 %define CopyD       [BP+2]    ; dword
46403 %define CopyDoff    [BP+2]    ; word
46404 %define CopyS       [BP+6]    ; dword
46405 %define CopySoff    [BP+6]    ; word
46406 %define CopyTemp    [BP+10]   ; byte
46407
46408 00007E50 83EC0E    SUB     SP,14              ; room for temp buffer
46409 00007E53 1E        push    ds
46410 00007E54 56        push    si
46411 00007E55 06        push    es
46412 00007E56 57        push    di
46413 00007E57 55        push    bp
46414 00007E58 89E5     MOV     BP,SP
46415 00007E5A B42E     MOV     AH,'.'
46416 00007E5C AC        LODSB
46417 00007E5D AA        STOSB
46418 00007E5E 38E0     CMP     AL,AH
46419 00007E60 7518     JNZ     short NormalComp    ;      if ((*d++=*s++) == '.') {
46420 00007E62 E8D1FF    CALL    PathSep            ;      if (!pathsep(*s))
46421 00007E65 740B     JZ      short NullTerm
46422
46423 00007E67 AC        TryTwoDot:
46424 00007E68 AA        LODSB
46425 00007E69 38E0     CMP     AL,AH
46426 00007E6B 7557     JNZ     short CopyBad
46427 00007E6D E8C6FF    CALL    PathSep
46428 00007E70 7552     JNZ     short CopyBad
46429
46430 00007E72 30C0     NullTerm:
46431 00007E74 AA        XOR     AL,AL
46432 00007E75 897606    MOV     CopySoff,SI
46433 00007E78 EB47     JMP     SHORT _GoodRet
46434
46435 00007E7A 8B7606    NormalComp:
46436 00007E7D E8AEDE    MOV     SI,CopySoff ; [bp+6]
46437 00007E80 3B7606    call    NameTrans
46438 00007E83 743F     CMP     SI,CopySoff
46439 00007E85 36F606[7205]FF JZ      short CopyBad
46440 00007E8B 7510     TEST   byte [SS:FSHARING],-1 ;      if (!fsharing) {;smr;SS override
46441 00007E8D 80E201    JNZ     short DoPack
46442 00007E90 360016[7A05] AND     DL,1
46443 00007E95 7F2D     ADD     [ss:CMETA],DL
46444 00007E97 7504     JG      short CopyBad
46445 00007E99 08D2     JNZ     short DoPack
46446 00007E9B 742F     OR      DL,DL
46447 00007E9D 897606    JZ      short CopyBadPath
46448 00007E9D 897606    DoPack:
46449 00007EA0 16        MOV     CopySoff,SI ; [bp+6]
46450 00007EA1 1F        push    ss
46451 00007EA2 BE[4B05] pop     ds
46452 00007EA5 8D7E0A    MOV     SI,NAME1
46453 00007EA8 57        LEA     DI,CopyTemp ; [bp+10]
46454 00007EA9 E8FBA7    push    di
46455 00007EAC 5F        call    PackName
46456 00007EAD E81999    ;      PackName (Name1, temp);
46457 00007EB0 49        pop     di
46458 00007EB1 034E02    call    StrLen
46459 00007EB1 034E02    ;      if (strlen(temp)+d > bp)
46460                DEC     CX
46461                ADD     CX,CopyDoff ; [bp+2]
46462                ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
46463 00007EB4 3B4E00    ; cmp     CX,[bp+0]
46464 00007EB7 730B     ; 15/12/2022
46465 00007EB9 89FE     ; cmp     CX,[bp]
46466 00007EBB C47E02    CMP     CX,CopyBP ; [bp+0]
46467 00007EBE E8FF98    JAE     short CopyBad
46468                ;      return (-1);
46469 00007EC1 F8        MOV     SI,DI
46470 00007EC2 EB0B    LES     DI,CopyD ; [bp+2]
46471                call    FStrCpy
46472                ;      }
46473 00007EC4 F9        _GoodRet:
46474 00007EC5 E8F800    CLC
46475 00007EC8 B002     JMP     SHORT CopyEnd
46476 00007ECA 7503     ;      return 0;
46477
46478 00007EC4 F9        CopyBad:
46479 00007EC5 E8F800    STC
46480 00007EC8 B002     CALL    ScanPathChar
46481 00007ECA 7503     ; check for path chars in rest of string
46482                ;mov     al,2
46483 00007EC8 B002     MOV     AL,error_file_not_found ; Set up for bad file error
46484 00007ECA 7503     JNZ     short CopyEnd
46485
46486 00007EC4 F9        CopyBadPath:
46487 00007EC5 E8F800    STC
46488 00007EC8 B003    ;mov     al,3
46489 00007ECA 7503     MOV     AL,error_path_not_found ; Set bad path error
46490
46491 00007EC4 F9        CopyEnd:
46492 00007EC5 E8F800    pop     bp
46493 00007EC8 B002     pop     di
46494 00007ECA 7503     pop     es
46495 00007EC8 B002     pop     si
46496 00007ECA 7503     pop     ds
46497 00007EC8 B002     LAHF
46498 00007EC8 B002     ADD     SP,14
46499 00007EC8 B002     ; reclaim temp buffer

```

```

46489 00007ED8 E8EE98      call    StrLen
46490 00007EDB 49          DEC     CX
46491 00007EDC 9E          SAHF
46492 00007EDD C3          retn
46493
46494 ; 14/05/2019 - Retro DOS v4.0
46495 ; DOSCODE:AE22h (MSDOS 6.21, MSDOS.SYS)
46496
46497 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
46498 ; DOSCODE:ADBFh (MSDOS 5.0, MSDOS.SYS)
46499
46500 ;Break <Splice - pseudo mount by string substitution>
46501 -----
46502 ; Splice - take a string and substitute a prefix if one exists. Change
46503 ; ThisCDS to point to physical drive CDS.
46504 ; Inputs: DS:SI point to string
46505 ; NoSetDir = TRUE => exact matches with splice fail
46506 ; Outputs: DS:SI points to thisCDS
46507 ; ES:DI points to DPB
46508 ; String at DS:SI may be reduced in length by removing prefix
46509 ; and substituting drive letter.
46510 ; CX = 0 If no splice done
46511 ; CX <> 0 otherwise
46512 ; ThisCDS points to proper CDS if spliced, otherwise it is
46513 ; left alone
46514 ; ThisDPB points to proper DPB
46515 ; Registers modified: DS:SI, ES:DI, BX,AX,CX
46516 -----
46517
46518 ; 13/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
46519 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0C0A6h
46520
46521 Splice:
46522 00007EDE 36F606[5A00]FF TEST    byte [SS:SPLICES],-1 ;smr;SS Override
46523 00007EE4 7469      JZ      short AllDone
46524 00007EE6 36FF36[A205] push    word [SS:THISCDS]
46525 00007EEB 36FF36[A405] push    word [SS:THISCDS+2] ; TmpCDS = ThisCDS;smr;SS Override
46526 00007EF0 1E          push    ds
46527 00007EF1 56          push    si
46528 00007EF2 5F          pop     di
46529 00007EF3 07          pop     es
46530 00007EF4 31C0      XOR     AX,AX ; for (i=1; s = GetCDSFromDrv (i); i++)
46531
46532 00007EF6 E8E3FC      call    GetCDSFromDrv
46533 00007EF9 72A4      JC      short SpliceDone
46534 00007EFB FEC0      INC     AL
46535 ; 17/12/2022
46536 ; test byte [si+68],20h
46537 00007EFD F6444420 test    byte [si+curdir.flags+1],curdir_splice>>8 ; 04/12/2022
46538 ; ;test word [si+67],2000h
46539 ; ;TEST word [SI+curdir.flags],curdir_splice
46540 00007F01 74F3      JZ      short SpliceScan ; if ( Spliced (i) ) {
46541 00007F03 57          push    di
46542 00007F04 E8A000      call    PathPref ; if (!PathPref (s, d))
46543 00007F07 7403      JZ      short SpliceFound ;
46544
46545 00007F09 5F          pop     di
46546 00007F0A EBEA      JMP     short SpliceScan ; continue;
46547
46548 00007F0C 26803D00 SpliceFound: CMP     BYTE [ES:DI],0 ; if (*s || NoSetDir) {
46549 00007F10 7508      JNZ     short SpliceDo
46550 00007F12 36F606[4C03]FF TEST    byte [ss:NoSetDir],-1 ;smr;SS Override
46551 00007F18 75EF      JNZ     short SpliceSkip
46552
46553 00007F1A 89FE      MOV     SI,DI ; p = src + strlen (p);
46554 00007F1C 06          push    es
46555 00007F1D 1F          pop     ds
46556 00007F1E 5F          pop     di
46557 00007F1F E87F00      call    TextFromDrive1 ; src = TextFromDrive1(src,i);
46558 00007F22 36A1[B605] MOV     AX,[SS:Curr_Dir_End] ;smr;SS Override
46559 00007F26 09C0      OR      AX,AX
46560 00007F28 7808      JS      short NoPoke
46561 00007F2A 01F8      ADD     AX,DI ; curdirend += src-p;
46562 00007F2C 29F0      SUB     AX,SI
46563 00007F2E 36A3[B605] MOV     [SS:Curr_Dir_End],AX ;smr;SS Override
46564
46565 00007F32 803C00      CMP     BYTE [SI],0 ; if (*p)
46566 00007F35 7503      JNZ     short SpliceCopy ; *src++ = '\\';
46567 00007F37 B05C      MOV     AL,"\"
46568 00007F39 AA          STOSB
46569
46570 00007F3A E88398      SpliceCopy: ; strcpy (src, p);
46571 00007F3D 83C404      call    FStrCpy
46572 00007F40 80C901      ADD     SP,4 ; throw away saved stuff
46573 00007F43 EB0C      OR      CL,1 ; signal splice done.
46574 ; JMP     SHORT DoSet ; return;
46575 00007F45 368F06[A405] SpliceDone: pop    word [SS:THISCDS+2] ; ThisCDS = TmpCDS;
46576 00007F4A 368F06[A205] pop    word [SS:THISCDS] ;smr;SS Override
46577
46578 00007F4F 31C9      AllDone: XOR     CX,CX
46579
46580 00007F51 36C536[A205] DoSet: LDS     SI,[SS:THISCDS] ; ThisDPB = ThisCDS->devptr;smr;SS Override
46581 ; les di,[si+69]
46582 00007F56 C47C45      LES     DI,[SI+curdir.devptr]
46583 00007F59 36893E[8A05] MOV     [SS:THISDPB],DI ;smr;SS Override
46584 00007F5E 368C06[8C05] MOV     [SS:THISDPB+2],ES ;smr;SS Override
46585
46586 00007F63 C3          Splice_retn: retn
46587
46588 ; 15/05/2019 - Retro DOS v4.0
46589 ; DOSCODE:AEA9h (MSDOS 6.21, MSDOS.SYS)
46590
46591 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
46592 ; DOSCODE:AE46h (MSDOS 5.0, MSDOS.SYS)
46593
46594 ; 13/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
46595 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0C12Dh
46596
46597 ;Break <$NameTrans - partially process a name>
46598 -----
46599 ; $NameTrans - allow users to see what names get mapped to. This call
46600 ; performs only string substitution and canonicalization, not splicing. Due
46601 ; to Transpath playing games with devices, we need to insure that the output
46602 ; has drive letter and : in it.
46603 ;
46604 ; Inputs: DS:SI - source string for translation
46605 ; ES:DI - pointer to buffer
46606 ;
46607 ; Outputs:
46608 ; Carry Clear
46609 ; Buffer at ES:DI is filled in with data
46610 ; ES:DI point byte after nul byte at end of dest string in buffer
46611 ; Carry Set
46612 ; AX = error_path_not_found
46613 ; Registers modified: all

```



```

46613 ;-----
46614
46615 _$NameTrans:
46616 00007F64 1E      push    ds
46617 00007F65 56      push    si
46618 00007F66 06      push    es
46619 00007F67 57      push    di
46620 00007F68 51      push    cx ; MSDOS 6.0
46621
46622 ; MSDOS 6.0
46623 ; M027 - Start
46624 ;
46625 ; Sattrib must be set up with default values here. Otherwise, the value from
46626 ; a previous DOS call is used for attrib and DevName thinks it is not a
46627 ; device if the old call set the volume attribute bit. Note that devname in
46628 ; dir2.asm gets ultimately called by Transpath. See also M026. Also save
46629 ; and restore CX.
46630
46631 ;mov    ch,16h
46632 00007F69 B516    mov     ch,attr_hidden+attr_system+attr_directory
46633 00007F6B E8EE02  call    SetAttrib
46634
46635 ; M027 - End
46636
46637 ; MSDOS 3.3 (& MSDOS 6.0)
46638 00007F6E BF[BE03] MOV     DI,OPENBUF
46639 00007F71 E8EAF0 CALL    TransPath ; to translation (everything)
46640 00007F74 59      pop     cx ; MSDOS 6.0
46641 00007F75 5F      pop     di
46642 00007F76 07      pop     es
46643 00007F77 5E      pop     si
46644 00007F78 1F      pop     ds
46645 00007F79 7303    JNC     short TransOK
46646 00007F7B E9FA86 jmp     SYS_RET_ERR
46647
46648 00007F7E BE[BE03] TransOK: MOV     SI,OPENBUF
46649 00007F81 16      push    ss
46650 00007F82 1F      pop     ds
46651
46652 00007F83 E83A98 ;GotText: call    FStrCpy
46653 00007F86 E9E586 jmp     SYS_RET_OK
46654
46655 ;Break <DriveFromText - return drive number from a text string>
46656 ;-----
46657 ; DriveFromText - examine DS:SI and remove a drive letter, advancing the
46658 ; pointer.
46659 ;
46660 ; Inputs:    DS:SI point to a text string
46661 ; Outputs:   AL has drive number
46662 ;           DS:SI advanced
46663 ; Registers modified: AX,SI.
46664 ;-----
46665
46666 DriveFromText:
46667 00007F89 30C0    XOR     AL,AL ; drive = 0;
46668 ;CMP     BYTE [SI],0 ; if (*s &&
46669 ; 23/09/2023
46670 00007F8B 3804    cmp     [si],al ; 0
46671 00007F8D 74D4    jz      short Splice_retn
46672 00007F8F 807C013A    cmp     BYTE [SI+1],':' ; s[1] == ':' {
46673 00007F93 75CE    jnz     short Splice_retn ;
46674 00007F95 AD      LODSW ; drive = (*s | 020) - 'a'+1;
46675 00007F96 0C20    OR      AL,20h ; (convert to lowercase)
46676 ;sub     al,60h
46677 00007F98 2C60    SUB     AL,'a'-1 ; s += 2;
46678 00007F9A 75C7    jnz     short Splice_retn ;
46679 00007F9C B0FF    MOV     AL,-1 ; nuke AL...
46680 ; 23/09/2023
46681 ;dec     al ; -1
46682 00007F9E C3      retn ; }
46683
46684 ;Break <TextFromDrive - convert a drive number to a text string>
46685 ;-----
46686 ; TextFromDrive - turn AL into a drive letter: and put it at es:di with
46687 ; trailing :. TextFromDrive1 takes a 1-based number.
46688 ;
46689 ; Inputs:    AL has 0-based drive number
46690 ; Outputs:   ES:DI advanced
46691 ; Registers modified: AX
46692 ;-----
46693
46694 TextFromDrive:
46695 00007F9F FEC0    INC     AL
46696 TextFromDrive1:
46697 ;add     al,40h
46698 00007FA1 0440    ADD     AL,'A'-1 ; *d++ = drive-1+'A';
46699 00007FA3 B43A    MOV     AH,":" ; 3Ah ; strcat (d, ":");
46700 00007FA5 AB      STOSW
46701
46702 00007FA6 C3      PathPref_retn: retn
46703
46704 ;Break <PathPref - see if one path is a prefix of another>
46705 ;-----
46706 ; PathPref - compare DS:SI with ES:DI to see if one is the prefix of the
46707 ; other. Remember that only at a pathchar break are we allowed to have a
46708 ; prefix: A:\ and A:\FOO
46709 ;
46710 ; Inputs:    DS:SI potential prefix
46711 ;           ES:DI string
46712 ; Outputs:   Zero set => prefix found
46713 ;           DI/SI advanced past matching part
46714 ;           Zero reset => no prefix, DS/SI garbage
46715 ; Registers modified: CX
46716 ;-----
46717
46718 PathPref:
46719 00007FA7 E82D98 call    DStrLen ; get length
46720 00007FAA 49      DEC     CX ; do not include nul byte
46721 00007FAB F3A6    REPZ    CMPSB ; compare
46722 00007FAD 75F7    jnz     short PathPref_retn ; if NZ then return NZ
46723 00007FAF 50      push    ax ; save char register
46724 00007FB0 8A44FF    MOV     AL,[SI-1] ; get last byte to match
46725 00007FB3 E834DE call    PATHCHRCMP ; is it a path char (Root!)
46726 00007FB6 7406    JZ      short Prefix ; yes, match root (I hope)
46727 ; 2/13/KK
46728 00007FB8 268A05 NotSep: MOV     AL,[ES:DI] ; get next char to match
46729 00007FBB E87AFE call    PathSepGotCh ; was it a pathchar?
46730
46731 Prefix:
46732 00007FBE 58      pop     ax ; get back original
46733 00007FBF C3      retn
46734
46735 ;Break <ScanPathChar - see if there is a path character in a string>
46736 ;-----
46737 ; ScanPathChar - search through the string (pointed to by DS:SI) for

```

```

46737 ; a path separator.
46738 ;
46739 ; Input: DS:SI target string (null terminated)
46740 ; Output: Zero set => path separator encountered in string
46741 ; Zero clear => null encountered
46742 ; Registers modified: SI
46743 ;-----
46744
46745 ScanPathChar:
46746 00007FC0 AC LODSB ; fetch a character
46747 00007FC1 E874FE call PathSepGotCh
46748 00007FC4 75FA JNZ short ScanPathChar ; not \, / or NUL => go back for more
46749 ; call PATHCHRCMP ; path separator?
46750 ; retn
46751 ; 18/12/2022
46752 00007FC6 E921DE jmp PATHCHRCMP
46753
46754 ;=====
46755 ; FILE.ASM, MSDOS 6.0, 1991
46756 ;=====
46757 ; 14/07/2018 - Retro DOS v3.0
46758
46759 ; 13/05/2019 - Retro DOS v4.0
46760 ; DOSCODE:AF10h (MSDOS 6.21, MSDOS.SYS)
46761
46762 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
46763 ; DOSCODE:AEADh (MSDOS 5.0, MSDOS.SYS)
46764
46765 ; 14/03/2024 - Retro DOS v5.0
46766
46767 ; MSDOS 2.11
46768 ;BREAK <$Open - open a file handle>
46769 ;-----
46770 ; Assembler usage:
46771 ; LDS DX, Name
46772 ; MOV AH, Open
46773 ; MOV AL, access
46774 ; INT int_command
46775
46776 ; ACCESS Function
46777 ;-----
46778 ; open_for_read file is opened for reading
46779 ; open_for_write file is opened for writing
46780 ; open_for_both file is opened for both reading and writing.
46781
46782 ; Error returns:
46783 ; AX = error_invalid_access
46784 ; = error_file_not_found
46785 ; = error_access_denied
46786 ; = error_too_many_open_files
46787 ;-----
46788
46789 ; MSDOS 6.0
46790 ; BREAK <$Open - open a file from a path string>
46791 ;-----
46792
46793 ** $Open - Open a File
46794
46795 ; given a path name in DS:DX and an open mode in AL, $Open opens the
46796 ; file and returns a handle
46797
46798 ; ENTRY (DS:DX) = pointer to asciz name
46799 ; (AL) = open mode
46800 ; EXIT 'C' clear if OK
46801 ; (ax) = file handle
46802 ; 'C' set if error
46803 ; (ax) = error code
46804 ; USES all
46805
46806 ;-----
46807
46808 ; 13/05/2019 - Retro DOS v4.0
46809 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
46810
46811 ; 14/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
46812 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0C194h
46813 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0AF10h)
46814 ; (Windows ME IO.SYS - BIOSCODE:0A9A7h)
46815
46816 _$Open:
46817 00007FC9 30E4 xor ah,ah ; MSDOS 6.0
46818
46819 _$Open2:
46820 00007FCB B516 ;mov ch,16h
46821 00007FCD E88C02 mov ch,attr_hidden+attr_system+attr_directory
46822 00007FDD B9FB31 call SetAttrib
46823 mov cx,DOS_OPEN
46824
46825 ;xor ah,ah ; MSDOS 3.3
46826 00007FDD 50 push ax
46827
46828 ;* General file open/create code. The $CREATE call and the various
46829 ; $OPEN calls all come here.
46830
46831 ; We'll share a lot of the standard stuff of allocating SFTs, cracking
46832 ; path names, etc., and then dispatch to our individual handlers.
46833 ; WARNING - this info and list is just a guess, not definitive - jgl
46834
46835 ; (TOS) = create mode
46836 ; (CX) = address of routine to call to do actual function
46837 ; (DS:DX) = ASCIZ name
46838 ; SAttrib = Attribute mask
46839
46840 ; Get a free SFT and mark it "being allocated"
46841
46842 AccessFile:
46843 call ECritSFT
46844 00007FD7 E84CF7 call SFNFree ; get a free sfn
46845 00007FDA E83299 call LCritSFT
46846 ;jc short OpenFail ; oops, no free sft's
46847 ; 14/03/2024
46848 00007FDD 7248 jc short OpenFail
46849
46850 00007FDF 36891EAA05 MOV [SS:SFN],BX ; save the SFN for later;smr;SS Override
46851 00007FE4 36893E9E05 MOV [SS:THISSFT],DI ; save the SF offset ;smr;SS Override
46852 00007FE9 368C06A005 MOV [SS:THISSFT+2],ES ; save the SF segment ;smr;SS Override
46853
46854 ; Find a free area in the user's JFN table.
46855
46856 00007FEE E822F7 call JFNFree ; get a free jfn
46857 ;jnc short SaveJFN
46858 ; 14/03/2024
46859 00007FF1 7234 jc short OpenFail
46860 ;

```

```

46861 ;OpenFailJ:
46862 ;JMP OpenFail ; there were free JFNs... try SFN
46863
46864 SaveJFN:
46865 mov [ss:PJFN],DI ; save the jfn offset ;smr;SS Override
46866 mov [ss:PJFN+2],ES ; save the jfn segment;smr;SS Override
46867 mov [ss:JFN],BX ; save the jfn itself ;smr;SS Override
46868
46869 ; We have been given an JFN. We lock it down to prevent other tasks from
46870 ; reusing the same JFN.
46871
46872 mov BX,[ss:SFN] ;smr;SS Override
46873 mov [ES:DI],BL ; assign the JFN
46874 mov SI,DX ; get name in appropriate place
46875 mov DI,OPENBUF ; appropriate buffer
46876 push CX ; save routine to call
46877 call TransPath ; convert the path
46878 pop bx ; (bx) = routine to call
46879
46880 LDS SI,[ss:THISSFT] ;smr;SS Override
46881 ;JC short OpenCleanJ ; no error, go and open file
46882 ; 14/03/2024
46883 JC short OpenClean
46884
46885 CMP byte [ss:CMETA],-1 ;smr;SS Override
46886 JZ short SetSearch
46887 ;mov al,2
46888 MOV AL,error_file_not_found ; no meta chars allowed
46889
46890 OpenCleanJ:
46891 JMP short OpenClean
46892 ; 14/03/2024 (PCDOS 7.1 IBMDOS.COM)
46893 ;;;
46894
46895 OpenFail:
46896 STI
46897 pop cx ; Clean stack
46898 ;
46899 jmp short OpenCritLeave
46900 ;;;
46901
46902 SetSearch:
46903 pop ax ; Mode (Open), Attributes (Create)
46904
46905 ; We need to get the new inheritance bits.
46906
46907 xor cx,cx
46908 ; MSDOS 6.0
46909 ;mov [si+2],cx ; 0
46910 MOV [SI+SF_ENTRY.sf_mode],cx ; initialize mode field to 0
46911 ;mov [si+51],cx ; 0
46912 MOV [SI+SF_ENTRY.sf_MFT],cx ; clean out sharing info
46913 ;
46914 CMP BX,DOS_OPEN
46915 JNZ short _DoOper
46916 ;test al,80h
46917 test AL,SHARING_NO_INHERIT ; look for no inher
46918 JZ short _DoOper ; 10/08/2018
46919 AND AL,7Fh ; mask off inherit bit
46920 ;mov cx,1000h
46921 MOV CX,sf_no_inherit
46922 _DoOper:
46923 ;; MSDOS 3.3
46924 ;;mov word [si+2], 0
46925 ;;mov word [si+33h], 0
46926 ;MOV word [SI+SF_ENTRY.sf_mode],0
46927 ;MOV word [SI+SF_ENTRY.sf_MFT],0
46928
46929 ; MSDOS 6.0
46930 ;** Check if this is an extended open. If so you must set the
46931 ; modes in sf_mode. Call Set_EXT_mode to do all this. See
46932 ; Set_EXT_mode in creat.asm
46933
46934 ; MSDOS 6.0
46935 ;SAVE <di, es> ;M022 conditional removed here
46936 push di
46937 push es
46938 push ds
46939 pop es
46940 push si
46941 pop di ; (es:di) = SFT address
46942 call Set_EXT_mode
46943 ;RESTORE <es, di>
46944 pop es
46945 pop di
46946
46947 ;Context DS
46948 push ss
46949 pop ds
46950
46951 push cx
46952 CALL BX ; blam!
46953 pop cx
46954 LDS SI,[THISSFT]
46955 JC short OpenE2 ;AN000;FT. chek extended open hooks first
46956 ;JC short OpenE ; MSDOS 3.3
46957
46958 ; The SFT was successfully opened. Remove busy mark.
46959
46960 OpenOK:
46961 ;MOV word [SI+SF_ENTRY.sf_ref_count],1
46962 mov word [SI],1
46963 ;or [SI+5],cx
46964 OR [SI+SF_ENTRY.sf_flags],CX ; set no inherit bit if necessary
46965
46966 ; If the open mode is 70, we scan the system for other SFT's with the same
46967 ; contents. If we find one, then we can 'collapse' thissft onto the already
46968 ; opened one. Otherwise we use this new one. We compare uid/pid/mode/mft
46969 ;
46970 ; Since this is only relevant on sharer systems, we stick this code into the
46971 ; sharer.
46972 ; 01/07/2024
46973 ;;;
46974 push ds
46975 pop es
46976 mov di,si
46977 call set_sftfcb_entry ; set SFT-FCB entry
46978 ; in the internal (SFT_FCB) table
46979 ; (used for FCB calls only!)
46980 ;;;
46981
46982 MOV AX,[ss:JFN] ;smr;SS Override
46983 Call far [ss:JShare+(12*4)]; 12 = ShCol ;smr;SS Override
46984

```

```

46985 00008071 36C706[AA05]FFFF      MOV     word [ss:SFN],-1      ; clear out sfn pointer      ;smr;SS Override
46986                                     ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
46987 OpenOkj:
46988 00008078 E9F385                  jmp     SYS_RET_OK          ; bye with no errors
46989
46990                                     ; Extended open hooks check
46991                                     ;
46992                                     ; AL has error code. Stack has argument to dos_open/dos_create.
46993
46994 OpenClean:
46995 0000807B 5B                        pop     bx                  ; clean off stack
46996 OpenE:
46997                                     ;MOV     word [SI+SF_ENTRY.sf_ref_count],0 ; release SFT
46998 0000807C C7040000                  mov     word [SI],0
46999 00008080 36C536[AE05]              LDS     SI,[ss:PJFN]        ;smr;SS Override
47000 00008085 C604FF                    MOV     BYTE [SI],0FFh     ; free the SFN...
47001
47002                                     ; 14/03/2024
47003                                     ;JMP     SHORT OpenCritLeave
47004                                     ;
47005 OpenFail:
47006                                     ;STI
47007                                     ;pop     cx                  ; Clean stack
47008
47009 OpenCritLeave:
47010 00008088 36C706[AA05]FFFF      MOV     word [SS:SFN],-1    ; remove mark.
47011
47012                                     ; MSDOS 6.0
47013 ; File Tagging DOS 4.00
47014 0000808F 36833E[2403]25          CMP     word [SS:EXTERR],error_Code_Page_Mismatched
47015                                     ;AN000;;FT. code page mismatch
47016 00008095 7538                      JNZ     short NORERR        ;AN000;;FT. no
47017 00008097 E9E685                  jmp     From_GetSet         ;AN000;;FT. yes
47018
47019                                     ; 14/03/2024
47020                                     ; MSDOS 6.0
47021 Extended open hooks check
47022 OpenE2:
47023 0000809A 83F857                    CMP     AX,error_invalid_parameter ;AN000;;EO. IFS extended open ?
47024 0000809D 75DD                      JNZ     short OpenE         ;AN000;;EO. no.
47025 0000809F EBE7                      JMP     short OpenCritLeave  ;AN000;;EO. keep handle
47026
47027                                     ; 15/03/2024
47028 %if 0
47029 NORERR:
47030                                     ; File Tagging DOS 4.00
47031                                     ;
47032 jmp     SYS_RET_ERR            ; no free, return error
47033 %endif
47034
47035                                     ; MSDOS 2.11
47036 BREAK <$CREAT - create a new file and open him for input>
47037 -----
47038 ; Assembler usage:
47039 ;     LDS     DX, name
47040 ;     MOV     AH, Creat
47041 ;     MOV     CX, access
47042 ;     INT     21h
47043 ; AX now has the handle
47044 ;
47045 ; Error returns:
47046 ;     AX = error_access_denied
47047 ;     = error_path_not_found
47048 ;     = error_too_many_open_files
47049 -----
47050
47051                                     ; MSDOS 6.0
47052 BREAK <$Creat - create a brand-new file>
47053 -----
47054
47055 ** $Creat - Create a File
47056
47057 $Creat creates the directory entry specified in DS:DX and gives it the
47058 initial attributes contained in CX
47059
47060 ENTRY (DS:DX) = ASCIZ path name
47061 (CX) = initial attributes
47062 'C' set if error
47063 (ax) = error code
47064 'c' clear if OK
47065 (ax) = file handle
47066 USES all
47067 -----
47068
47069
47070 _$CREAT:
47071 000080A1 51                        push    cx                  ; Save attributes on stack
47072 000080A2 B9[CA30]                  mov     CX,DOS_CREATE      ; routine to call
47073 AccessSet:
47074                                     ;mov     byte [ss:SATTRIB],6
47075 000080A5 36C606[6D05]06          mov     byte [ss:SATTRIB],attr_hidden+attr_system ;smr;SS Override
47076                                     ; 10/08/2018
47077 000080AB E926FF                    JMP     AccessFile         ; use good ol' open
47078
47079
47080                                     ; MSDOS 6.0 (MSDOS 3.3)
47081 BREAK <$CHMOD - change file attributes>
47082 -----
47083
47084 ** $CHMOD - Change File Attributes
47085
47086 ; Assembler usage:
47087 ;     LDS     DX, name
47088 ;     MOV     CX, attributes
47089 ;     MOV     AL,func (0=get, 1=set)
47090 ;     INT     21h
47091 ;
47092 ; Error returns:
47093 ;     AX = error_path_not_found
47094 ;     AX = error_access_denied
47095 -----
47096
47097                                     ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
47098
47099                                     ; 14/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
47100                                     ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0C27Ch
47101                                     ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0AFF4h)
47102                                     ; (Windows ME IO.SYS - BIOSCODE:0AAB2h)
47103
47104 _$CHMOD:
47105                                     ; 14/03/2024 (PCDOS 7.1 IBMDOS.COM)
47106                                     ;;;
47107 000080AE 3CFF                      cmp     al,0FFh           ; MS-DOS 7.20 (win98) - EXTENDED-LENGTH FILENAME OPERATIONS
47108                                     ; AX = 43FFh

```

```

47109                                     ; BP = 5053h ('PS')
47110                                     ; CL = function
47111                                     ; 39h "mkdir" create directory
47112                                     ; DS:DX -> ASCIZ pathname
47113                                     ; 56h rename file
47114                                     ; DS:DX -> ASCIZ filename of existing file (no wildcards)
47115                                     ; ES:DI -> ASCIZ new filename (no wildcards)
47116                                     ;
47117                                     ; ref: Ralf Brown's Interrupt List
47118 000080B0 7520                jnz     short std_chmod
47119 000080B2 81FD5350          cmp     bp,5053h
47120 000080B6 7515                jnz     short chmod_x_2
47121 000080B8 88CC                mov     ah,cl
47122                                     ;mov     word [ss:PATHNAMELEN],128
47123 000080BA 36C606[5411]80          mov     byte [ss:PATHNAMELEN],128 ; Retro DOS v5.0
47124 000080C0 80F939          cmp     cl,39h
47125 000080C3 7503                jne     short chmod_x_1
47126 000080C5 E95AA7          jmp     mkdir_x
47127 chmod_x_1:
47128 000080C8 80F956          cmp     cl,56h
47129 000080CB 7472                je      short rename_x
47130 chmod_x_2:
47131 000080CD B001                mov     al,1
47132                                     ;jmp     short NORERR
47133                                     ; 15/03/2024
47134 NORERR:
47135 000080CF E9A685          jmp     SYS_RET_ERR
47136
47137 std_chmod:
47138     ;;;
47139
47140     ; 05/08/2018 - Retro DOS v3.0
47141     ; IBMDOS.COM (MSDOS 3.3, 1987) - Offset 6FCCh
47142 000080D2 BF[BE03]          MOV     DI,OPENBUF ; appropriate buffer
47143 000080D5 50                push    ax
47144 000080D6 51                push    cx
47145 000080D7 89D6                MOV     SI,DX
47146 000080D9 E886FB          call    TransPathSet ; get correct path
47147 000080DC 59                pop     cx
47148 000080DD 58                pop     ax
47149 000080DE 7255                JC      short ChModErr ; and get function and attrs back
47150 000080E0 16                push    ss ; errors get mapped to path not found
47151 000080E1 1F                pop     ds ; set up for later possible calls
47152 000080E2 803E[7A05]FF          CMP     byte [CMETA],-1
47153 000080E7 754C                JNZ     short ChModErr
47154                                     ;mov     byte [SATTRIB],16h
47155 000080E9 C606[6D05]16          MOV     byte [SATTRIB],attr_hidden+attr_system+attr_directory
47156 000080EE 2C01                SUB     AL,1 ; fast way to discriminate
47157 000080F0 7209                JB      short ChModGet ; 0 -> go get value
47158 000080F2 7415                JZ      short ChModSet ; 1 -> go set value
47159
47160     ; 14/04/2024
47161 %if 0
47162     ;mov     byte [EXTERR_LOCUS],1
47163     MOV     byte [EXTERR_LOCUS],errLOC_Unk ; Extended Error Locus
47164 %else
47165     ; 14/03/2024 (PCDOS 7.1 IBMDOS.COM)
47166     ;;;
47167 000080F4 E8E691          call    set_exerr_locus_unk ; Extended Error Locus
47168     ;;;
47169 %endif
47170     ;mov     al,1
47171 000080F7 B001                mov     al,error_invalid_function ; bad value
47172     ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47173 chmod_errj:
47174     ;jmp     SYS_RET_ERR
47175     ;jmp     short ChModE
47176 000080F9 EBD4                jmp     short NORERR ; 06/12/2022
47177 ChModGet:
47178     call    GET_FILE_INFO ; suck out the o1' info
47179 000080FE 7237                JC      short ChModE ; error codes are already set for ret
47180 00008100 E87483          call    get_user_stack ; point to user saved variables
47181     ;mov     [SI+4],ax
47182 00008103 894404          MOV     [SI+user_env.user_CX],AX ; return the attributes
47183     ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47184 OpenOkj2:
47185     ; 17/12/2022
47186     ;jmp     SYS_RET_OK ; say sayonara
47187     ;jmp     short OpenOkj
47188     ; 25/06/2019
47189 00008106 E96885          jmp     SYS_RET_OK_c1c
47190
47191 ChModSet:
47192     MOV     AX,CX ; get attrs in position
47193 0000810B E813AF          call    SET_FILE_ATTRIBUTE ; go set
47194 0000810E 7227                JC      short ChModE ; errors are set
47195     ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47196     ;jmp     SYS_RET_OK
47197 OpenOkj3:
47198     ;jmp     short OpenOkj2
47199     ; 17/12/2022
47200 00008110 E95B85          jmp     SYS_RET_OK
47201
47202     ; 17/12/2022
47203 %if 0
47204 ChModErr:
47205 NotFound: ; 17/12/2022
47206     ;mov     al,3
47207     mov     al,error_path_not_found
47208 ChModE:
47209     ; 17/12/2022
47210     ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47211     ;jmp     SYS_RET_ERR
47212     ;jmp     short chmod_errj
47213     ; 17/12/2022
47214     jmp     short NORERR
47215 %endif
47216
47217     ; 22/05/2019 - Retro DOS v4.0
47218     ; DOSCODE:B039h (MSDOS 6.21, MSDOS.SYS)
47219
47220     ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
47221     ; DOSCODE:AFD6h (MSDOS 5.0, MSDOS.SYS)
47222
47223     ; BREAK <$UNLINK - delete a file entry>
47224     ;-----
47225     ;
47226     ;** $UNLINK - Delete a File
47227     ;
47228     ;
47229     ; Assembler usage:
47230     ; LDS     DX, name
47231     ; IF VIA SERVER DOS CALL
47232     ; MOV     CX,SEARCH_ATTRIB

```

```

47233      ;      MOV      AH, Unlink
47234      ;      INT      21h
47235      ;
47236      ;      ENTRY    (ds:dx) = path name
47237      ;      (cx) = search_attribute, if via server_dos
47238      ;      EXIT     'C' clear if no error
47239      ;      'C' set if error
47240      ;      (ax) = error code
47241      ;      = error_file_not_found
47242      ;      = error_access_denied
47243      ;
47244      ;-----
47245      ;
47246      ; 15/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
47247      ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0C2E4h
47248      ;
47249      _$UNLINK:
47250      push    cx                ; Save possible CX input parm
47251      MOV     SI,DX              ; Point at input string
47252      MOV     DI,OPENBUF        ; temp spot for path
47253      call    TransPathSet      ; go get normalized path
47254      pop     cx
47255      JC      short ChModErr     ; badly formed path
47256      CMP     byte [ss:CMETA],-1 ; meta chars? ;smr;SS override
47257      JNZ     short NotFound
47258      push    ss
47259      pop     ds
47260      ;mov     ch,6
47261      mov     ch,attr_hidden+attr_system ; unlink appropriate files
47262      call    SetAttrib
47263      call    DOS_DELETE        ; remove that file
47264      ;JC      short UnlinKE     ; error is there
47265      ; 17/12/2022
47266      JC      short NORERR
47267      ;
47268      ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47269      Unlinkok:
47270      ;jmp     SYS_RET_OK        ; okay doksy
47271      jmp     short OpenOkj3
47272      ;
47273      ; 17/12/2022
47274      ChModErr: ; 17/12/2022
47275      NotFound:
47276      ;mov     al,3
47277      MOV     AL,error_path_not_found
47278      ChModE: ; 17/12/2022
47279      UnlinKE:
47280      ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47281      ;jmp     SYS_RET_ERR      ; bye
47282      ;jmp     short ChModE
47283      ; 17/12/2022
47284      jmp     short NORERR
47285      ;
47286      ;BREAK <$RENAME - move directory entries around>
47287      ;-----
47288      ;
47289      ;      Assembler usage:
47290      ;      LDS      DX, source
47291      ;      LES      DI, dest
47292      ;      IF VIA SERVER DOS CALL
47293      ;      MOV      CX,SEARCH_ATTRIB
47294      ;      MOV      AH, Rename
47295      ;      INT      21h
47296      ;
47297      ;      Error returns:
47298      ;      AX = error_file_not_found
47299      ;      = error_not_same_device
47300      ;      = error_access_denied
47301      ;
47302      ;-----
47303      ;
47304      ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
47305      ; 15/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
47306      ;
47307      _$RENAME:
47308      ; 15/03/2024 (PCDOS 7.1 IBMDOS.COM)
47309      ;;;
47310      ;mov     word [ss:PATHNAMELEN],67
47311      ; 15/03/2024 - Retro DOS v5.0
47312      mov     byte [ss:PATHNAMELEN],67 ; DIRSTRLEN = 67
47313      rename_x:
47314      ;;;
47315      ;
47316      ; MSDOS 3.3 (& MSDOS 6.0)
47317      push    cx
47318      push    ds
47319      push    dx                ; save source and possible CX arg
47320      PUSH    ES
47321      POP     DS                ; move dest to source
47322      MOV     SI,DI             ; save for offsets
47323      MOV     DI,RENBUF
47324      call    TransPathSet      ; munge the paths
47325      PUSH    word [ss:WFP_START] ; get pointer ;smr;SS override
47326      POP     word [ss:REN_WFP] ; stash it ;smr;SS override
47327      pop     si
47328      pop     ds
47329      pop     cx                ; get back source and possible CX arg
47330      epjc2:
47331      JC      short ChModErr     ; get old error
47332      CMP     byte [ss:CMETA],-1 ; ;smr;SS override
47333      JNZ     short NotFound
47334      push    cx
47335      MOV     DI,OPENBUF        ; save possible CX arg
47336      call    TransPathSet      ; appropriate buffer
47337      pop     cx                ; wham
47338      ;JC      short epjc2
47339      ; 15/03/2024
47340      JC      short ChModErr
47341      ;
47342      push    ss
47343      pop     ds
47344      CMP     byte [CMETA],-1
47345      JB      short NotFound
47346      ;
47347      ; MSDOS 6.0
47348      ;PUSH    WORD [THISCDs]      ;AN000;;MS.save thiscds
47349      ;PUSH    WORD [THISCDs+2]    ;AN000;;MS.
47350      ; 15/03/2024 (PCDOS 7.1 IBMDOS.COM)
47351      ;;;
47352      les     di,[THISCDs]
47353      push    di
47354      push    es
47355      ;;;
47356

```



```

47357 0000817C BF[BE03]      MOV     DI,OPENBUF          ;AN000;;MS.
47358 0000817F 16          PUSH    SS              ;AN000;;MS.
47359 00008180 07          POP     ES               ;AN000;;MS.es:di-> source
47360 00008181 30C0        XOR     AL,AL              ;AN000;;MS.scan all CDS
47361                                ;AN000;;
47362 00008183 E856FA        call   GetCDSFromDrv        ;AN000;;MS.
47363 00008186 720F        JC     short dorn          ;AN000;;MS. end of CDS
47364 00008188 E80496        call   StrCmp              ;AN000;;MS. current dir ?
47365 0000818B 7404        JZ     short rnerr         ;AN000;;MS. yes
47366 0000818D FEC0        INC     AL                ;AN000;;MS. next
47367 0000818F EBF2        JMP     short rnloop       ;AN000;;MS.
47368                                ;AN000;;
47369                                ;ADD     SP,4              ;AN000;;MS. pop thiscds
47370                                ; 15/03/2024 (PCDOS 7.1 IBMDOS.COM)
47371 00008191 58          pop     ax
47372 00008192 58          pop     ax
47373
47374                                ;error error_current_directory ;AN000;;MS.
47375 00008193 B010        mov     al,error_current_directory
47376                                ;jmp     SYS_RET_ERR
47377                                ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47378 00008195 EBA0        jmp     short UnlinkE
47379
47380 dorn:
47381
47382 ; 15/03/2024
47383 %if 0                                ;AN000;;
47384     POP     WORD [SS:THISCDS+2]      ;AN000;;MS.;PBUGBUG;SS REQD??
47385     POP     WORD [SS:THISCDS]        ;AN000;;MS.;PBUGBUG;SS REQD??
47386 %endif
47387     push    ss
47388     pop     ds
47389
47390 ; 15/03/2024
47391 %if 1
47392     pop     word [THISCDS+2]
47393     pop     word [THISCDS]
47394 %endif
47395 ; MSDOS 3.3 (& MSDOS 6.0)
47396 ;mov     ch,16h
47397     mov     ch,attr_directory+attr_hidden+attr_system
47398                                ; rename appropriate files
47399     call   SetAttrib
47400     call   DOS_RENAME              ; do the deed
47401     JC     short UnlinkE          ; errors
47402
47403 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47404 %if 0
47405     jmp     SYS_RET_OK
47406     jmp     short Unlinkok
47407
47408 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
47409
47410 ; 14/07/2018 - Retro DOS v3.0
47411 ; MSDOS 3.3 (& MSDOS 6.0)
47412
47413 ;Break <$CreateNewFile - Create a new directory entry>
47414 -----
47415 ; CreateNew - Create a new directory entry. Return a file handle if there
47416 ; was no previous directory entry, and fail if a directory entry with
47417 ; the same name existed previously.
47418 ;
47419 ; Inputs: DS:DX point to an ASCIZ file name
47420 ; CX contains default file attributes
47421 ; Outputs: Carry Clear:
47422 ;           AX has file handle opened for read/write
47423 ;           Carry Set:
47424 ;           AX has error code
47425 ; Registers modified: All
47426 -----
47427 _$CreateNewFile:
47428     push    cx                  ; Save attributes on stack
47429     MOV     CX,DOS_Create_New    ; routine to call
47430     JMP     AccessSet            ; use good ol' open
47431
47432 ** BinToAscii - convert a number to a string.
47433 -----
47434 ; BinToAscii converts a 16 bit number into a 4 ascii characters.
47435 ; This routine is used to generate temp file names so we don't spend
47436 ; the time and code needed for a true hex number, we just use
47437 ; A thorough 0.
47438 ;
47439 ; ENTRY (ax) = value
47440 ; (es:di) = destination
47441 ; (es:di) updated by 4
47442 ; USES cx, di, flags
47443 -----
47444 ; MSDOS 3.3
47445 ;BinToAscii:
47446 ; mov     cx,4
47447 ;bta5:
47448 ; push    cx
47449 ; mov     cl,4
47450 ; rol     ax,cl
47451 ; push    ax
47452 ; and     al,0Fh
47453 ; add     al,'0'
47454 ; cmp     al,'9'
47455 ; jbe     short bta6
47456 ; add     al,7
47457 ;bta6:
47458 ; stosb
47459 ; pop     ax
47460 ; pop     cx
47461 ; loop    bta5
47462 ; retn
47463
47464 ; 15/03/2024
47465 ; MSDOS 5.0-6.22 & windows ME
47466 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0B0D9h)
47467 ; (Windows ME IO.SYS - BIOSCODE:0ABA4h)
47468 %if 0
47469
47470 ; MSDOS 6.0
47471 BinToAscii:
47472     mov     cx,404h              ; (ch) = digit counter, (cl) = shift cnt
47473 ;bta5:
47474     ROL     AX,CL                ; move leftmost nibble into rightmost
47475     push    ax                  ; preserve remainder of digits
47476     AND     AL,0Fh              ; grab low nibble
47477     ADD     AL,'A'              ; turn into ascii
47478     STOSB                        ; drop in the character
47479     pop     ax                  ; (ax) = shifted number
47480     dec     ch

```

```

47481      jnz     short bta5          ; process 4 digits
47482      retn
47483  %else
47484      ; 15/03/2024
47485      ; PC DOS 7.1 IBMDOS.COM - DOSCODE:0C385h
47486
47487  BinToAscii:
47488      push    ax                  ; convert a number to a string ; ax = value
47489      xchg    ah,al
47490      ;db      0D4h,10h
47491      aam     10h                  ; AH = AL / 16 and AL = remainder
47492      add     ax,4141h             ; 'AA'
47493      stosw
47494      pop     ax
47495      ;db      0D4h,10h
47496      aam     10h
47497      add     ax,4141h             ; add ax,'AA'
47498      stosw
47499      retn
47500  %endif
47501
47502  ;Break      <$CreateTempFile - create a unique name>
47503  ;-----
47504  ; $CreateTemp - given a directory, create a unique name in that directory.
47505  ; Method used is to get the current time, convert to a name and attempt
47506  ; a create new. Repeat until create new succeeds.
47507  ;
47508  ; Inputs: DS:DX point to a null terminated directory name.
47509  ;         CX contains default attributes
47510  ; Outputs: Unique name is appended to DS:DX directory.
47511  ;          AX has handle
47512  ; Registers modified: all
47513  ;-----
47514
47515      ; 10/07/2024
47516      ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
47517      ; 15/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
47518
47519  _$CreateTempFile:
47520      ;Enter
47521      push    bp
47522      mov     bp,sp
47523
47524      ;LocalVar  EndPtr,DWORD
47525      ;LocalVar  FilPtr,DWORD
47526      ;LocalVar  Attr,WORD
47527
47528      sub     sp,10
47529
47530      ;test     cx,0FFD8h
47531      test    CX,~attr_changeable
47532      jz      short OKatts          ; Ok if no non-changeable bits set
47533
47534  ; We need this "hook" here to detect these cases (like user sets one both of
47535  ; vol_id and dir bits) because of the structure of the or $CreateNewFile loop
47536  ; below. The code loops on error_access_denied, but if one of the non
47537  ; changeable attributes is specified, the loop COULD be infinite or WILL be
47538  ; infinite because CreateNewFile will fail with access_denied always. Thus we
47539  ; need to detect these cases before getting to the loop.
47540
47541      ;mov     ax, 5
47542      MOV     AX,error_access_denied
47543      JMP     SHORT SETTMPERR
47544
47545  OKatts:
47546      ;MOV     attr,CX              ; save attribute
47547      mov     [bp-10],cx
47548      ;MOV     FilPtrL,dx           ; pointer to file
47549      mov     [bp-8],dx
47550      ;MOV     FilPtrH,ds
47551      mov     [bp-6],ds
47552      ;MOV     EndPtrH,ds           ; seg pointer to end of dir
47553      mov     [bp-2],ds
47554      PUSH    DS
47555      POP     ES                    ; destination for nul search
47556      MOV     DI,dx
47557      MOV     CX,DI
47558      NEG     CX                    ; number of bytes remaining in segment
47559      ; MSDOS 6.0
47560      OR      CX,CX                 ; AN000;MS. cx=0 ? ds:dx on segment boundary
47561      JNZ     short okok            ; AN000;MS. no
47562      ;MOV     CX,-1                ; AN000;MS.
47563      ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47564      ; 17/12/2022
47565      dec     cx ; mov cx,-1
47566      ;mov     cx,-1 ; 0FFFh
47567
47568  okok:
47569      XOR     AX,AX                 ; AN000;
47570      REPNZ   SCASB                 ; AN000;
47571      ; AN000;
47572      DEC     DI                    ; point back to the nul
47573      MOV     AL,[ES:DI-1]          ; Get char before the NUL
47574      call    PATHCHRCMP           ; Is it a path separator?
47575      jz      short SETENDPTR       ; Yes
47576
47577  STOREPTH:
47578      MOV     AL,'\'
47579      STOSB                          ; Add a path separator (and INC DI)
47580
47581  SETENDPTR:
47582      ; 10/07/2024 - Retro DOS v5.0
47583      ;MOV     EndPtrL,DI           ; pointer to the tail
47584      ; 09/07/2024 (Retro DOS v4 BugFix - Erdogan Tan - Istanbul)
47585      ; (Note: I find this Retro DOS v4 Kernel bug while searching the reason
47586      ; of the AutoCAD R12 running/startup problem. Now it is solved here.)
47587      ;mov     [bp-4],di ; (Retro DOS v4 !Bug!)
47588      mov     [bp-4],di ; !Fix!
47589
47590  CreateLoop:
47591      push    ss                    ; let ReadTime see variables
47592      pop     ds
47593      push    bp
47594      call    READTIME              ; go get time
47595      pop     bp
47596
47597      ;
47598      ; Time is in CX:DX. Go drop it into the string.
47599      ;
47600      ;les     di,EndPtr            ; point to the string
47601      les     di,[bp-4]
47602      mov     ax,cx
47603      call    BinToAscii            ; store upper word
47604      mov     ax,dx
47605      call    BinToAscii            ; store lower word
47606      xor     al,al
47607      STOSB                          ; nul terminate
47608      LDS     DX,FilPtr             ; get name
47609      lds     dx,[bp-8]

```



```

47605             ;MOV     CX,Attr           ; get attr
47606 0000821D 8B4EF6      mov     cx,[bp-10]
47607 00008220 55          push    bp
47608 00008221 E889FF      CALL    _$CreateNewFile           ; try to create a new file
47609 00008224 5D          pop     bp
47610 00008225 732A      JNC     short Createdone      ; failed, go try again
47611
47612             ; The operation failed and the error has been mapped in AX. Grab the extended
47613             ; error and figure out what to do.
47614
47615             ;; MSDOS 3.3                ; M049 - start
47616             ;; mov     ax,[ss:EXTERR]          ;smr;SS Override
47617             ;; cmp     al,error_file_exists
47618             ;; jz      short CreateLoop        ; file existed => try with new name
47619             ;; cmp     al,error_access_denied
47620             ;; jz      short CreateLoop        ; access denied (attr mismatch)
47621
47622             ; MSDOS 6.0
47623             ;cmp     al,50h
47624 00008227 3C50      CMP     AL,error_file_exists ; Q: did file already exist
47625 00008229 74D8      JZ      short CreateLoop        ; Y: try again
47626             ;cmp     al,5
47627 0000822B 3C05      CMP     AL,error_access_denied; Q: was it access denied
47628 0000822D 7521      JNZ     short SETTMPERR          ; N: Error out
47629
47630             ; Y: Check to see if we got this due
47631             ; to the network drive. Note that
47632             ; the redir will set the exterr
47633             ; to error_cannot_make if this is
47634             ; so.
47635             ; 15/03/2024
47636             %if 0
47637             CMP     byte [SS:EXTERR],error_net_access_denied ; M069
47638             ; See if it's REALLY an att mismatch
47639             je      short SETTMPERR          ; no, network error, stop
47640             ;M070
47641             ; If the user failed on an I24, we do not want to try again
47642             ;
47643             cmp     byte [SS:EXTERR],error_FAIL_I24 ;User failed on I24? ;M070
47644             ;je      short SETTMPERR          ;yes, do not try again ;M070
47645             ;jmp     short CreateLoop        ;attr mismatch, try again ;M070
47646             ; 17/12/2022
47647             jne     short CreateLoop ; 10/06/2019
47648             ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47649             ;jz      short SETTMPERR
47650             ;jmp     short CreateLoop
47651             %else
47652             ; 15/03/2024 (PCDOS 7.1 IBMDOS.COM)
47653             ;;
47654 0000822F 36A0[2403]   mov     al,[ss:EXTERR]
47655 00008233 3C41      cmp     al,41h          ; error_net_access_denied
47656 00008235 7419      je      short SETTMPERR
47657 00008237 3C53      cmp     al,53h          ; error_FAIL_I24
47658 00008239 7415      je      short SETTMPERR
47659 0000823B 3C50      cmp     al,50h          ; error_file_exists
47660 0000823D 74C4      je      short CreateLoop
47661 0000823F 36C43E[A205] les     di,[ss:THISCDS]
47662 00008244 83FFFF      cmp     di,0FFFFh ; -1
47663 00008247 74BA      je      short CreateLoop ; No CDS
47664             ;;test word [es:di+43h],8000h
47665             ;test word [es:di+curdir.flags],curdir_isnet
47666 00008249 26F6454480 test    byte [es:di+curdir.flags+1],(curdir_isnet>>8)
47667 0000824E 75B3      jnz     short CreateLoop
47668             ;;
47669             %endif
47670
47671             ;; MOV     AL,error_access_denied; Return this "extended" error
47672             ; M049 - end
47673             SETTMPERR:
47674             STC
47675             Createdone:
47676             ;Leave
47677             mov     sp,bp
47678             pop     bp
47679             JC      short CreateFail
47680             jmp     SYS_RET_OK              ; success!
47681             CreateFail:
47682             jmp     SYS_RET_ERR
47683
47684             ; SetAttrib will set the search attribute (SAttrib) either to the normal
47685             ; (CH) or to the value in CL if the current system call is through
47686             ; serverdoscall.
47687             ;
47688             ; Inputs: fSharing == FALSE => set sattrib to CH
47689             ; fSharing == TRUE => set sattrib to CL
47690             ; Outputs: none
47691             ; Registers changed: CX
47692
47693             SetAttrib:
47694             ;test byte [ss:FSHARING],-1          ;smr;SS Override
47695             ;jnz short Set
47696             ; 15/03/2024
47697 0000825C 36803E[7205]00 cmp     byte [ss:FSHARING],0
47698 00008262 7502      jnz     short Set
47699
47700             mov     cl,ch
47701             Set:
47702             mov     byte [ss:SATTRIB],cl          ;smr;SS Override
47703             retn
47704
47705             ;-----
47706             ; 16/03/2024 - Retro DOS v5.0
47707             ext_invalid:
47708             ;mov     al,1
47709             mov     al,error_invalid_function
47710             eo_err:
47711             ;jmp     SYS_RET_ERR
47712             jmp     short CreateFail
47713
47714             ; 14/07/2018 - Retro DOS v3.0
47715             ; MSDOS 6.0
47716
47717             ; 29/04/2019 - Retro DOS v4.0
47718
47719             ;Break <Extended_Open- Extended open the file>
47720             ;-----
47721             ; Input: AL= 0 reserved AH=6CH
47722             ; BX= mode
47723             ; CL= create attribute CH=search attribute (from server)
47724             ; DX= flag
47725             ; DS:SI = file name
47726             ; ES:DI = parm list
47727             ; DD SET EA list (-1) null
47728             ; DW n parameters

```

```

47729 ; DB type (TTTTTLL)
47730 ; DW IOMODE
47731 ; Function: Extended Open
47732 ; Output: carry clear
47733 ; AX= handle
47734 ; CX=1 file opened
47735 ; 2 file created/opened
47736 ; 3 file replaced/opened
47737 ; carry set: AX has error code
47738 ;-----
47739 ;
47740 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
47741 ; 16/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
47742
47743 _$Extended_Open: ;AN000;
47744 ;ASSUME CS:DOSCODE,SS:DOSDATA ;AN000;
47745 00008270 368916[F405] MOV [SS:EXTOPEN_FLAG],DX ;AN000;EO. save ext. open flag;smr;SS Override
47746 00008275 36C706[F705]0000 MOV word [SS:EXTOPEN_IO_MODE],0 ;AN000;EO. initialize IO mode;smr;SS Override
47747 ; 17/12/2022
47748 0000827C F6C6FE test dh,0FEh ; 04/12/2022
47749 ;;test dx,0FE00h
47750 ;TEST DX,RESERVED_BITS_MASK ;AN000;EO. reserved bits 0 ?
47751 0000827F 75EB JNZ short ext_inval2 ;AN000;EO. no
47752 00008281 88D4 MOV AH,DL ;AN000;EO. make sure flag is right
47753 00008283 80FA00 CMP DL,0 ;AN000;EO. all fail ?
47754 00008286 74E4 JZ short ext_inval2 ;AN000;EO. yes, error
47755 ;and dl,0Fh
47756 00008288 80E20F AND DL,EXISTS_MASK ;AN000;EO. get exists action byte
47757 0000828B 80FA02 CMP DL,2 ;AN000;EO. > 2
47758 0000828E 77DC JA short ext_inval2 ;AN000;EO. yes, error
47759 ;and ah,0F0h
47760 00008290 80E4F0 AND AH,NOT_EXISTS_MASK ;AN000;EO. get no exists action byte
47761 00008293 80FC10 CMP AH,10H ;AN000;EO. > 10
47762 00008296 77D4 JA short ext_inval2 ;AN000;EO. yes, error
47763
47764 00008298 368C06[FB05] MOV [SS:SAVE_ES],ES ;AN000;EO. save API parms;smr;SS Override
47765 0000829D 36893E[F905] MOV [SS:SAVE_DI],DI ;AN000;EO.;smr;SS Override
47766 000082A2 36FF36[F405] PUSH word [SS:EXTOPEN_FLAG] ;AN000;EO.;smr;SS Override
47767 000082A7 368F06[FD05] POP word [SS:SAVE_DX] ;AN000;EO.;smr;SS Override
47768 000082AC 36890E[FF05] MOV [SS:SAVE_CX],CX ;AN000;EO.;smr;SS Override
47769 000082B1 36891E[0106] MOV [SS:SAVE_BX],BX ;AN000;EO.;smr;SS Override
47770 000082B6 368C1E[0506] MOV [SS:SAVE_DS],DS ;AN000;EO.;smr;SS Override
47771 000082BB 368936[0306] MOV [SS:SAVE_SI],SI ;AN000;EO.;smr;SS Override
47772 000082C0 89F2 MOV DX,SI ;AN000;EO. ds:dx points to file name
47773 000082C2 89D8 MOV AX,BX ;AN000;EO. ax= mode
47774 ; 16/03/2024
47775 %if 0
47776 JMP SHORT goopen2 ;AN000;;EO. do normal
47777 ext_inval2: ;AN000;;EO.
47778 ;mov al,1
47779 mov al,error_invalid_function ;AN000;EO.. invalid function
47780 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47781 eo_err:
47782 jmp SYS_RET_ERR
47783 jmp short CreateFail
47784 %endif
47785 ; 16/03/2024
47786 %if 0
47787 ext_inval_parm: ;AN000;EO..
47788 POP CX ;AN000;EO.. pop up satck
47789 POP SI ;AN000;EO..
47790 ;error error_invalid_data ;AN000;EO.. invalid parms
47791 ;mov al,13
47792 mov al,error_invalid_data
47793 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47794 ;;jmp SYS_RET_ERR
47795 jmp short eo_err
47796 ; 17/12/2022
47797 jmp short CreateFail
47798 %endif
47799
47800 ; 17/12/2022
47801 ;error_return: ;AN000;EO.
47802 ; retn ;AN000;EO.. return with error
47803
47804 goopen2: ;AN000;
47805 ; 17/12/2022
47806 ;test bh,20h
47807 test bh,INT_24_ERROR>>8 ; 04/12/2022
47808 000082C4 F6C720 ;;test bx,2000h
47809 ;TEST BX,INT_24_ERROR ;AN000;EO.. disable INT 24 error ?
47810 JZ short goopen ;AN000;EO.. no
47811 000082C7 7406 ;or byte [SS:EXTOPEN_ON],2
47812 000082C9 36800E[F605]02 OR byte [SS:EXTOPEN_ON],EXT_OPEN_I24_OFF ;AN000;EO.. set bit to disable;smr;SS Override
47813 goopen: ;AN000;
47814 ;or byte [SS:EXTOPEN_ON],1
47815 000082CF 36800E[F605]01 OR byte [SS:EXTOPEN_ON],EXT_OPEN_ON ;AN000;EO.. set Extended Open active;smr;SS Override
47816 ;AND word [SS:EXTOPEN_FLAG],0FFh ;AN000;EO.create new ?;smr;SS Override
47817 ; 18/12/2022
47818 mov byte [SS:EXTOPEN_FLAG+1],0 ; AND word [SS:EXTOPEN_FLAG],0FFh
47819 000082D5 36C606[F505]00 cmp word [SS:EXTOPEN_FLAG],10h
47820 000082DB 36833E[F405]10 cmp word [SS:EXTOPEN_FLAG],EXT_EXISTS_FAIL+EXT_NEXISTS_CREATE ;AN000;FT.;smr;SS Override
47821 000082E1 7516 JNZ short chknext ;AN000;;EO. no
47822 000082E3 E8C7FE call _$CreateNewFile ;AN000;;EO. yes
47823 000082E6 723F JC short error_return ;AN000;;EO. error
47824
47825 000082E8 36803E[F605]00 CMP byte [SS:EXTOPEN_ON],0 ;AN000;;EO. IFS does it;smr;SS Override
47826 000082EE 7438 JZ short ok_return2 ;AN000;;EO. yes
47827 ;mov word [SS:EXTOPEN_FLAG],2
47828 000082F0 36C706[F405]0200 MOV word [SS:EXTOPEN_FLAG],ACTION_CREATED_OPENED ;AN000;EO. created/opened;smr;SS Override
47829 000082F7 EB7F JMP short setXAttr ; 16/03/2024 ;AN000;;EO. set XAS
47830
47831 ; 17/12/2022
47832 ;ok_return2:
47833 ; jmp SYS_RET_OK ;AN000;;EO.
47834
47835 chknext:
47836 ; 17/12/2022
47837 test byte [SS:EXTOPEN_FLAG],EXT_EXISTS_OPEN ; 1
47838 000082F9 36F606[F405]01 ;;test word [SS:EXTOPEN_FLAG],1
47839 ;TEST word [SS:EXTOPEN_FLAG],EXT_EXISTS_OPEN ;AN000;;EO. exists open;smr;SS Override
47840 JNZ short exist_open ;AN000;;EO. yes
47841 000082FF 752A call _$CREAT ;AN000;;EO. must be replace open
47842 00008301 E89DFD JC short error_return ;AN000;;EO. return with error
47843 00008304 7221 CMP byte [SS:EXTOPEN_ON],0 ;AN000;;EO. IFS does it;smr;SS Override
47844 00008306 36803E[F605]00 JZ short ok_return2 ;AN000;;EO. yes
47845 0000830C 741A MOV word [SS:EXTOPEN_FLAG],ACTION_CREATED_OPENED ;AN000;EO. presume create/open;smr;SS Override
47846 0000830E 36C706[F405]0200 TEST byte [SS:EXTOPEN_ON],EXT_FILE_NOT_EXISTS ;AN000;;EO. file not exists ?;smr;SS Override
47847 00008315 36F606[F605]04 JNZ short setXAttr ;AN000;;EO. no
47848 0000831B 755B MOV word [SS:EXTOPEN_FLAG],ACTION_REPLACED_OPENED ;AN000;;EO. replaced/opened;smr;SS Override
47849 0000831D 36C706[F405]0300 JMP SHORT setXAttr ;AN000;;EO. set XAS
47850 00008324 EB52 error_return2:
47851 ; Set Carry again to flag error ;AN001;
47852 00008326 F9 STC

```

```

47853 error_return: ; 17/12/2022
47854 00008327 C3      retn                ;AN000;;EO. return with error
47855
47856 ; 17/12/2022
47857 ok_return:
47858 ok_return2:
47859 00008328 E94383   jmp     SYS_RET_OK
47860
47861 exist_open:
47862 ;test byte [SS:FSHARING],-1 ;AN000;;EO. server doscall?;smr;SS Override
47863 ;jz short noserver ;AN000;;EO. no
47864 ; 16/03/2024
47865 ;;;
47866 0000832B 36803E[F205]00 cmp byte [ss:FSHARING],0 ; server doscall?
47867 00008331 7402      jz short noserver ; no
47868 ;;;
47869 00008333 88E9      MOV CL,CH ;AN000;;EO. cl=search attribute
47870
47871 00008335 E893FC     call _$open2 ;AN000;;EO. do open
47872 00008338 732F     JNC short ext_ok ;AN000;;EO.
47873 0000833A 36803E[F605]00 CMP byte [SS:EXTOPEN_ON],0 ;AN000;;EO. error and IFS call;smr;SS Override
47874 00008340 74E4     JZ short error_return2 ;AN000;;EO. return with error
47875
47876 local_extopen:
47877 00008342 83F802     cmp ax,2
47878 00008345 75DF     CMP AX,error_file_not_found ;AN000;;EO. file not found error
47879 ;test word [SS:EXTOPEN_FLAG],10h ;AN000;;EO. no,
47880 ; 17/12/2022
47881 00008347 36F606[F405]10 test byte [SS:EXTOPEN_FLAG],EXT_NEXISTS_CREATE ; 10h
47882 ;TEST word [SS:EXTOPEN_FLAG],EXT_NEXISTS_CREATE ;AN000;;EO. want to fail;smr;SS Override
47883 ;JNZ short do_creat ;AN000;;EO. yes
47884 ;JMP short extexit ;AN000;;EO. yes
47885 ; 17/12/2022
47886 0000834D 7446     jz short extexit ; 10/06/2019
47887 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
47888 ;jnz short do_creat
47889 ;jmp short extexit
47890
47891 0000834F 368B0E[FF05] MOV CX,[SS:SAVE_CX] ;AN000;;EO. get ds:dx for file name;smr;SS Override
47892 00008354 36C536[0306] LDS SI,[SS:SAVE_SI] ;AN000;;EO. cx = attribute;smr;SS Override
47893 00008359 89F2     MOV DX,SI ;AN000;;EO.
47894 0000835B E843FD     call _$CREATE ;AN000;;EO. do create
47895 0000835E 7235     JC short extexit ;AN000;;EO. error
47896 ;mov word [SS:EXTOPEN_FLAG],2
47897 00008360 36C706[F405]0200 MOV word [SS:EXTOPEN_FLAG],ACTION_CREATED_OPENED
47898 ;AN000;;EO. is created/opened;smr;SS Override
47899 00008367 EB0F     JMP SHORT setXAttr ;AN000;;EO. set XAS
47900
47901 ext_ok:
47902 00008369 36803E[F605]00 CMP byte [SS:EXTOPEN_ON],0 ;AN000;;EO. IFS call ?;smr;SS Override
47903 0000836F 74B7     JZ short ok_return ;AN000;;EO. yes
47904 ;mov word [SS:EXTOPEN_FLAG],1
47905 00008371 36C706[F405]0100 MOV word [SS:EXTOPEN_FLAG],ACTION_OPENED ;AN000;;EO. opened;smr;SS Override
47906
47907 setXAttr:
47908 ; 29/04/2019
47909 00008378 50      push ax
47910 00008379 E8FB80     call Get_User_Stack ;AN000;;EO.
47911 0000837C 36A1[F405] MOV AX,[SS:EXTOPEN_FLAG] ;AN000;;EO.;smr;SS Override
47912 ;mov [si+4],ax
47913 00008380 894404 MOV [SI+user_env.user_CX],AX ;AN000;;EO. set action code for cx
47914 00008383 58      pop ax ;AN000;;EO.
47915 00008384 8904     mov [si],ax
47916 ;MOV [SI+user_env.user_AX],AX ;AN000;;EO. set handle for ax
47917 ; 17/12/2022
47918 00008386 EBA0     jmp short ok_return
47919 ;ok_return:
47920 ;jmp SYS_RET_OK ;AN000;;EO.
47921
47922 ; 16/03/2024
47923 %if 0
47924 extexit2: ;AN000; ERROR RECOVERY
47925 POP BX ;AN000;EO. close the handle
47926 PUSH AX ;AN000;EO. save error code from set XA
47927 ;cmp word [SS:EXTOPEN_FLAG],2
47928 CMP word [SS:EXTOPEN_FLAG],ACTION_CREATED_OPENED ;AN000;EO. from create;smr;SS Override
47929 ;AN000;EO.
47930 JNZ short justopen ;AN000;EO. cx = attribute;smr;SS Override
47931 LDS SI,[SS:SAVE_SI]
47932 LDS DX,[SI] ;AN000;EO.
47933 call _$UNLINK ;AN000;EO. delete the file
47934 JMP SHORT reserror ;AN000;EO.
47935
47936 justopen: ;AN000;
47937 call _$CLOSE ;AN000;EO. pretend never happend
47938 reserror: ;AN000;
47939 POP AX ;AN000;EO. restore error code from set XA
47940
47941 JMP SHORT extexit ;AN000;EO.
47942
47943 ext_file_unfound: ;AN000;
47944 ;mov ax,2
47945 MOV AX,error_file_not_found ;AN000;EO.
47946 JMP SHORT extexit ;AN000;EO.
47947 ext_inval:
47948 ;mov ax,1
47949 MOV AX,error_invalid_function ;AN000;EO.
47950
47951 lockoperr: ; 17/12/2022
47952 extexit:
47953 jmp SYS_RET_ERR ;AN000;EO.
47954
47955 %endif
47956
47957 ;=====
47958 ; LOCK.ASM, MSDOS 6.0, 1991
47959 ;=====
47960 ; 14/07/2018 - Retro DOS v3.0
47961 ; 22/05/2019 - Retro DOS v4.0
47962
47963 ;BREAK <$LockOper - Lock Calls>
47964 ;-----
47965 ;
47966 ; Assembler usage:
47967 ; MOV BX, Handle (DOS 3.3)
47968 ; MOV CX, OffsetHigh
47969 ; MOV DX, OffsetLow
47970 ; MOV SI, LengthHigh
47971 ; MOV DI, LengthLow
47972 ; MOV AH, LockOper
47973 ; MOV AL, Request
47974 ; INT 21h
47975 ;
47976 ; Error returns:
47977 ; AX = error_invalid_handle

```

```

47977 ; = error_invalid_function
47978 ; = error_lock_violation
47979 ;
47980 ; Assembler usage:
47981 ; MOV AX, 5C?? (DOS 4.00)
47982 ;
47983 ; 0? lock all
47984 ; 8? lock write
47985 ; ?2 lock multiple
47986 ; ?3 unlock multiple
47987 ; ?4 lock/read
47988 ; ?5 write/unlock
47989 ; ?6 add (lseek EOF/lock/write/unlock)
47990 ;
47991 ; MOV BX, Handle
47992 ; MOV CX, count or size
47993 ; LDS DX, buffer
47994 ; INT 21h
47995 ;
47996 ; Error returns:
47997 ; AX = error_invalid_handle
47998 ; = error_invalid_function
47999 ; = error_lock_violation
48000 ; -----
48001 ;
48002 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
48003 ;
48004 ; 17/03/2024
48005 ; 16/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
48006 ;
48007 ; _$LockOper:
48008 00008388 3C01 CMP AL,1
48009 0000838A 770C JA short lock_bad_func
48010 ;
48011 0000838C 57 PUSH DI ; Save LengthLow
48012 0000838D E843F3 call SFFromHandle ; ES:DI -> SFT
48013 00008390 731B JNC short lock_do ; have valid handle
48014 00008392 5F POP DI ; Clean stack
48015 ;mov al,6
48016 00008393 B006 mov al,error_invalid_handle
48017 ;
48018 ; 16/03/2024
48019 ; extexit:
48020 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
48021 ; lockoperr:
48022 00008395 E9E082 jmp SYS_RET_ERR
48023 ; 17/12/2022
48024 ; jmp short lockoperr ; jmp SYS_RET_ERR
48025 ;
48026 ; lock_bad_func:
48027 ;
48028 ; 16/03/2024
48029 ; %if 0
48030 ; mov byte [ss:EXTERR_LOCUS],1
48031 MOV byte [SS:EXTERR_LOCUS],errLOC_Unk ; Extended Error Locus;smr;SS Override
48032 ;
48033 ; %else
48034 ; 16/03/2024 (PCDOS 7.1 IBMDOS.COM)
48035 00008398 E8428F call set_exerr_locus_unk ; Extended Error Locus
48036 ;
48037 ; %endif
48038 ;mov al,1
48039 0000839B B001 mov al,error_invalid_function
48040 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
48041 ; lockoperrj:
48042 jmp SYS_RET_ERR
48043 0000839D EBF6 jmp short lockoperr
48044 ;
48045 ; 22/05/2019 - Retro DOS v4.0
48046 ;
48047 ; MSDOS 6.0
48048 ; Align_buffer call has been deleted, since it corrupts the DTA (6/5/88) P5013
48049 ; Dead code deleted, MD, 23 Mar 90
48050 ;
48051 ; lock_do:
48052 ; MSDOS 3.3
48053 ; or al,al
48054 ; pop ax
48055 ; jz short DOS_Lock
48056 ; DOS_Unlock:
48057 ; test word [es:di+5],8000h
48058 ; test word [ES:DI+SF_ENTRY.sf_flags],sf_isnet
48059 ; JZ short LOCAL_UNLOCK
48060 ; push ax
48061 ; mov ax,110Bh
48062 ; int 2Fh ; Multiplex - NETWORK REDIRECTOR - UNLOCK REGION OF FILE
48063 ; ; BX = file handle, CX:DX = starting offset, SI = high word of size
48064 ; ; STACK: WORD low word of size, ES:DI -> SFT for file
48065 ; ; SFT DPB field -> DPB of drive containing file
48066 ; ; Return: CF set error
48067 ; pop bx
48068 ; jmp short ValChk
48069 ;
48070 ; LOCAL_UNLOCK:
48071 ; Call far [ss:JShare+(7*4)] ; 7 = clr_block ;smr;SS Override
48072 ; ValChk:
48073 ; JNC short Lock_OK
48074 ; lockerror:
48075 ; jmp SYS_RET_ERR
48076 ; Lock_OK:
48077 ; MOV AX,[SS:Temp_VAR] ;AN000;MS. AX= number of bytes ;smr;SS Override
48078 ; jmp SYS_RET_OK
48079 ; DOS_Lock:
48080 ; test word [es:di+5],8000h
48081 ; test word [ES:DI+SF_ENTRY.sf_flags],sf_isnet
48082 ; JZ short LOCAL_LOCK
48083 ; CallInstall NET_XLock,MultNET,10
48084 ; mov ax,110Ah
48085 ; int 2Fh ; Multiplex - NETWORK REDIRECTOR - LOCK REGION OF FILE
48086 ; ; BX = file handle, CX:DX = starting offset, SI = high word of size
48087 ; ; STACK: WORD low word of size, ES:DI -> SFT
48088 ; ; SFT DPB field -> DPB of drive containing file, SS = DOS CS
48089 ; ; Return: CF set error
48090 ; JMP short ValChk
48091 ;
48092 ; LOCAL_LOCK:
48093 ; Call far [ss:JShare+(6*4)] ; 6 = Set_Block ;smr;SS Override
48094 ; JMP short ValChk
48095 ;
48096 ; 17/12/2022
48097 ; LOCAL_UNLOCK:
48098 ; MSDOS 3.3
48099 ; Call far [ss:JShare+(7*4)] ; 7 = clr_block ;smr;SS Override
48100 ; MSDOS 6.0

```

```

48101 0000839F FF1E[AC00]      call far [Jshare+(7*4)]      ; 7 = clr_block ;smr;SS override
48102                          valchk:
48103 000083A3 7302            jnc short Lock_OK
48104                          lockerror:
48105                          ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
48106                          ; jmp SYS_RET_ERR
48107                          ; jmp short lockoperrj
48108                          ; 17/12/2022
48109 000083A5 EBEE            jmp short lockoperr      ; jmp SYS_RET_ERR
48110                          Lock_OK:
48111                          ; MOV AX,[SS:TEMP_VAR] ;AN000;;MS. AX= number of bytes ;smr;SS override
48112                          ; 10/06/2019
48113 000083A7 A1[0C06]        mov ax,[TEMP_VAR]
48114 000083AA E9C182          jmp SYS_RET_OK
48115                          ; 22/05/2019
48116                          lock_do:
48117                          ; MSDOS 6.0
48118                          MOV BX,AX                      ; save AX
48119 000083AD 89C3            MOV BP,Lock_Buffer          ; get DOS LOCK buffer
48120 000083AF BD[A903]        ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
48121                          ; mov [bp+0],dx
48122                          ; MOV [BP+LockBuf.Lock_position],DX ; set low offset
48123                          ; 15/12/2022
48124                          mov [bp],dx
48125 000083B2 895600          ; mov [bp+2],cx
48126                          ; MOV [BP+LockBuf.Lock_position+2],CX; set high offset
48127 000083B5 894E02          MOV [BP+LockBuf.Lock_position+2],CX; set high offset
48128                          ; 16/03/2024
48129                          ; POP CX                      ; get low length
48130                          ; mov [bp+4],cx
48131                          ; MOV [BP+LockBuf.Lock_length],CX ; set low length
48132                          ; 10/06/2019
48133 000083B8 8F4604          pop word [bp+LockBuf.Lock_length]
48134                          ; mov [bp+6],si
48135                          MOV [BP+LockBuf.Lock_length+2],SI ; set high length
48136 000083BB 897606          MOV CX,1                      ; one range
48137 000083BE B90100
48138                          ;
48139                          ; PUSH CS                      ;
48140                          ; POP DS                      ; DS:DX points to
48141                          ;
48142 000083C1 16              push ss
48143 000083C2 1F              pop ds
48144                          ;
48145 000083C3 89EA            MOV DX,BP                      ; Lock_Buffer
48146                          ; test al,1
48147 000083C5 A801            TEST AL,UNLOCK_ALL          ; function 1
48148                          ; JNZ short DOS_Unlock        ; yes
48149                          ; JMP short DOS_Lock           ; function 0
48150                          ; 17/12/2022
48151                          ; 10/06/2019
48152 000083C7 740E            jz short DOS_Lock
48153                          ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
48154                          ; JNZ short DOS_Unlock
48155                          ; JMP short DOS_Lock
48156                          ;
48157                          DOS_Unlock:
48158                          ; test word [es:di+5],8000h
48159                          ; test word [ES:DI+SF_ENTRY.sf_flags],sf_isnet
48160 000083C9 26F6450680      test byte [ES:DI+SF_ENTRY.sf_flags+1],(sf_isnet>>8)
48161 000083CE 74CF            JZ short LOCAL_UNLOCK
48162                          ;
48163                          ; 17/03/2024
48164                          ; lock_unlock: ; 22/05/2019
48165                          ;
48166                          ; CallInstall Net_Xlock,MultNET,10
48167                          ;
48168                          ; MSDOS 3.3
48169                          ; mov ax,110Bh
48170                          ; int 2Fh ; Multiplex - NETWORK REDIRECTOR - UNLOCK REGION OF FILE
48171                          ; BX = file handle, CX:DX = starting offset, SI = high word of size
48172                          ; STACK: WORD low word of size, ES:DI -> SFT for file
48173                          ; SFT DPB field -> DPB of drive containing file
48174                          ; Return: CF set error
48175                          ;
48176                          ; 17/03/2024 - Retro DOS v5.0
48177                          lock_unlock:
48178                          ;
48179                          ; MSDOS 6.0
48180 000083D0 B80A11          mov ax,110Ah
48181 000083D3 CD2F            int 2Fh ; Multiplex - NETWORK REDIRECTOR - LOCK REGION OF FILE
48182                          ; BX = file handle, CX:DX = starting offset, SI = high word of size
48183                          ; STACK: WORD low word of size, ES:DI -> SFT
48184                          ; SFT DPB field -> DPB of drive containing file, SS = DOS CS
48185                          ; Return: CF set error
48186                          ;
48187 000083D5 EBCC            JMP SHORT valchk
48188                          ;
48189                          ; 17/12/2022
48190                          %if 0
48191                          LOCAL_UNLOCK:
48192                          ; MSDOS 3.3
48193                          ; call far [ss:Jshare+(7*4)] ; 7 = clr_block ;smr;SS override
48194                          ; MSDOS 6.0
48195                          ; call far [Jshare+(7*4)] ; 7 = clr_block ;smr;SS override
48196                          valchk:
48197                          jnc short Lock_OK
48198                          lockerror:
48199                          ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
48200                          ; jmp SYS_RET_ERR
48201                          ; jmp short lockoperrj
48202                          Lock_OK:
48203                          ; MOV AX,[SS:TEMP_VAR] ;AN000;;MS. AX= number of bytes ;smr;SS override
48204                          ; 10/06/2019
48205                          mov ax,[TEMP_VAR]
48206                          jmp SYS_RET_OK
48207                          %endif
48208                          ;
48209                          DOS_Lock:
48210                          ; test word [es:di+5],8000h
48211                          ; test word [ES:DI+SF_ENTRY.sf_flags],sf_isnet
48212 000083D7 26F6450680      test byte [ES:DI+SF_ENTRY.sf_flags+1],(sf_isnet>>8)
48213                          ; JZ short LOCAL_LOCK
48214                          ; 17/03/2024
48215 000083DC 75F2            jnz short lock_unlock
48216                          ;
48217                          ; 17/03/2024
48218                          %if 0
48219                          ; CallInstall NET_XLock,MultNET,10
48220                          ;
48221                          mov ax,110Ah
48222                          int 2Fh ; Multiplex - NETWORK REDIRECTOR - LOCK REGION OF FILE
48223                          ; BX = file handle, CX:DX = starting offset, SI = high word of size
48224                          ; STACK: WORD low word of size, ES:DI -> SFT

```

```

48225                                     ; SFT DPB field -> DPB of drive containing file, SS = DOS CS
48226                                     ; Return: CF set error
48227
48228     JMP     short valchk
48229 %endif
48230
48231 LOCAL_LOCK:
48232     ; MSDOS 3.3
48233     ; call far [ss:JShare+(6*4)] ; 6 = Set_Block ;smr;SS Override
48234     ; MSDOS 6.0
48235 000083DE FF1E[A800]    call far [JShare+(6*4)] ; 6 = Set_Block ;smr;SS Override
48236
48237 000083E2 EBBF          JMP     short valchk
48238
48239 ; 14/07/2018 - Retro DOS v3.0
48240 ; LOCK_CHECK
48241 ; MSDOS 6.0 (& MSDOS 3.3)
48242
48243 ;-----
48244 ; Inputs:
48245 ;   Outputs of SETUP
48246 ;   [USER_ID] Set
48247 ;   [PROC_ID] Set
48248 ; Function:
48249 ;   Check for lock violations on local I/O
48250 ;   Retries are attempted with sleeps in between
48251 ; Outputs:
48252 ;   Carry clear
48253 ;   Operation is OK
48254 ;   Carry set
48255 ;   A lock violation detected
48256 ; Outputs of SETUP preserved
48257 ;-----
48258
48259 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
48260 ; 22/05/2019 - Retro DOS v4.0
48261
48262 000083E4 8B1E[1A00]    LOCK_CHECK:
48263     MOV     BX,[RetryCount] ; Number retries
48264 000083E8 53           LockRetry:
48265 000083E9 50           push    bx ; save regs
48266                                     push    ax ; MSDOS 6.0
48267
48268     ; MSDOS 3.3
48269     ; call far [ss:JShare+(8*4)] ; 8 = chk_block
48270 000083EA FF1E[B000]    ; MSDOS 6.0
48271                                     call far [JShare+(8*4)] ; 8 = chk_block
48272
48273 000083EE 58           pop     ax ; MSDOS 6.0
48274 000083F0 7307         pop     bx ; restore regs
48275     jnc     short lc_ret_label ; There are no locks (retnc)
48276
48277 000083F2 E8EF93       LockN:
48278 000083F5 4B           call    idle ; wait a while
48279 000083F6 75F0         DEC     BX ; remember a retry
48280 000083F8 F9           JNZ     short LockRetry ; more retries left...
48281 000083F9 C3           STC
48282     lc_ret_label:
48283     retn
48284
48285 ; 14/07/2018 - Retro DOS v3.0
48286 ; LOCK_VIOLATION
48287 ; MSDOS 6.0 (& MSDOS 3.3)
48288
48289 ;-----
48290 ; Inputs:
48291 ;   [THISDPB] set
48292 ;   [READOP] indicates whether error on read or write
48293 ; Function:
48294 ;   Handle Lock violation on compatibility (FCB) mode SFTs
48295 ; Outputs:
48296 ;   Carry set if user says FAIL, causes error_lock_violation
48297 ;   Carry clear if user wants a retry
48298 ;
48299 ; DS, ES, DI, CX preserved, others destroyed
48300 ;-----
48301
48302 ; 19/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
48303
48304 000083FA 1E           LOCK_VIOLATION:
48305 000083FB 06           PUSH    DS
48306 000083FC 57           PUSH    ES
48307 000083FD 51           PUSH    DI
48308 000083FE B82100       PUSH    CX
48309     ;mov     ax,21h
48310     MOV     AX,error_lock_violation
48311     ;mov     byte [ALLOWED],18h
48312     MOV     MOV     byte [ALLOWED],Allowed_FAIL+Allowed_RETRY
48313 00008400 BF0100       LES     BP,[THISDPB]
48314 0000840D 89F9         MOV     DI,1 ; Fake some registers
48315     MOV     CX,DI
48316
48317 ; 19/03/2024 (PCDOS 7.1 IBMDOS.COM)
48318 ;;;
48319 0000840F 31D2         xor     dx,dx ; 0
48320 00008415 750E         ;cmp     [es:bp+0Fh],dx
48321     cmp     [es:bp+DPB.FAT_SIZE],dx ; 0 ?
48322     jnz     short lockv_1 ; not FAT32
48323     ; FAT32
48324     ;mov     dx,[es:bp+2Bh]
48325     mov     dx,[es:bp+DPB.FCLUS_FSECTOR+2]
48326     mov     [HIGH_SECTOR],dx
48327     ;mov     dx,[es:bp+29h]
48328     mov     dx,[es:bp+DPB.FCLUS_FSECTOR]
48329     jmp     short lockv_2
48330
48331 lockv_1:
48332     mov     [HIGH_SECTOR],dx ; 0
48333     ;;;
48334     ;mov     dx,[es:bp+11]
48335     MOV     DX,[ES:BP+DPB.FIRST_SECTOR]
48336
48337 lockv_2:
48338     ; 19/03/2024
48339     call    HARDERR
48340     POP     CX
48341
48342 sharev_3:
48343     ; 01/07/2024
48344     POP     DI
48345     POP     ES
48346     POP     DS
48347     CMP     AL,1
48348     jz      short lc_ret_label ; 1 = retry, carry clear
48349     STC
48350     retn
48351
48352 ; 14/07/2018 - Retro DOS v3.0
48353 ;-----

```



```

48349
48350 ; do a retz to return error
48351
48352 ; 22/05/2019 - Retro DOS v4.0
48353 checkShare:
48354 ; MSDOS 3.3
48355 ; cmp byte [cs:fShare],0
48356 ; ret
48357
48358 ; MSDOS 6.0
48359 0000843A 1E push ds ;smr;
48360 ; getdseg <ds> ; ds -> dosdata
48361 0000843B 2E8E1E[0700] mov ds,[cs:DosDSeg]
48362 00008440 803E[0303]00 cmp byte [fShare],0
48363 00008445 1F pop ds ;smr;
48364 00008446 C3 ret
48365
48366 ;=====
48367 ; SHARE.ASM, MSDOS 6.0, 1991
48368 ;=====
48369 ; 14/07/2018 - Retro DOS v3.0
48370 ; 22/05/2019 - Retro DOS v4.0
48371 ; 19/03/2024 - Retro DOS v5.0
48372
48373 ; SHARE_CHECK
48374 ;-----
48375 ; Inputs:
48376 ; [THISSFT] Points to filled in local file/device SFT for new
48377 ; instance of file sf_mode ALWAYS has mode (even on FCB SFTs)
48378 ; [WFP_START] has full path of name
48379 ; [USER_ID] Set
48380 ; [PROC_ID] Set
48381 ; Function:
48382 ; Check for sharing violations on local file/device access
48383 ; Outputs:
48384 ; Carry clear
48385 ; Sharing approved
48386 ; Carry set
48387 ; A sharing violation detected
48388 ; AX is error code
48389 ; USES ALL but DS
48390 ;-----
48391
48392 ; 22/05/2019 - Retro DOS v4.0
48393 SHARE_CHECK:
48394 ; 26/07/2019
48395 00008447 FF1E[9400] call far [JShare+(1*4)] ; 1 = MFT_Enter
48396 shchk_retn:
48397 0000844B C3 ret
48398
48399 ; SHARE_VIOLATION
48400 ;-----
48401 ; Inputs:
48402 ; [THISDPB] Set
48403 ; AX has error code
48404 ; Function:
48405 ; Handle sharing errors
48406 ; Outputs:
48407 ; Carry set if user says FAIL, causes error_sharing_violation
48408 ; Carry clear if user wants a retry
48409 ;
48410 ; DS, ES, DI preserved, others destroyed
48411 ;-----
48412
48413 ; 19/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
48414
48415 SHARE_VIOLATION:
48416 0000844C 1E PUSH DS
48417 0000844D 06 PUSH ES
48418 0000844E 57 PUSH DI
48419
48420 0000844F 31D2 xor dx,dx ; Retro DOS v5.0
48421
48422 ;
48423 00008451 8816[7505] ; MOV byte [READOP],0 ; All share errors are reading
48424 ; mov [READOP],dx ; 0
48425 ;
48426 00008455 C606[4B03]18 ; mov byte [ALLOWED],18h
48427 0000845A C42E[8A05] MOV byte [ALLOWED],Allowed_FAIL+Allowed_RETRY
48428 0000845E BF0100 LES BP,[THISDPB]
48429 00008461 89F9 MOV DI,1 ; Fake some registers
48430 ; MOV CX,DI
48431
48432 ; 19/03/2024
48433 ; mov dx,[es:bp+17]
48434 ; MOV DX,[ES:BP+DPB.DIR_SECTOR]
48435
48436 ; 19/03/2024 - Retro DOS v5.0
48437 ; PCDOS 7.1 IBMDOS.COM
48438 ;;;
48439 00008463 2639560F ; cmp word [es:bp+0Fh],0
48440 00008467 740A cmp word [es:bp+DPB.FAT_SIZE],dx ; 0
48441 ; jz short sharev_1 ; FAT32
48442 ; FAT16 or FAT12
48443 00008469 8916[0706] ; mov word [HIGH_SECTOR],0
48444 ; mov [HIGH_SECTOR],dx ; 0
48445 0000846D 268B5611 ; mov dx,[es:bp+11h]
48446 00008471 E80C mov dx,[es:bp+DPB.DIR_SECTOR]
48447 ; jmp short sharev_2
48448 sharev_1:
48449 00008473 268B562B ; mov dx,[es:bp+2Bh]
48450 00008477 8916[0706] mov dx,[es:bp+DPB.FCLUS_FSECTOR+2]
48451 ; mov [HIGH_SECTOR],dx
48452 0000847B 268B5629 ; mov dx,[es:bp+29h]
48453 ; mov dx,[es:bp+DPB.FCLUS_FSECTOR]
48454 sharev_2:
48455 0000847F E8D2DB ; call HARDERR
48456 ; 01/07/2024
48457 %if 0
48458 POP DI
48459 POP ES
48460 POP DS
48461 CMP AL,1
48462 jz short shchk_retn ; 1 = retry, carry clear
48463 STC
48464 ret
48465 %else
48466 00008482 EBAD jmp short sharev_3
48467 %endif
48468
48469 ;-----
48470 ; ShareEnd - terminate sharing info on a particular SFT/UID/PID. This does
48471 ; NOT perform a close, it merely asserts that the sharing information
48472 ; for the SFT/UID/PID may be safely released.

```

```

48473 ;
48474 ; Inputs: ES:DI points to an SFT
48475 ; Outputs: None
48476 ; Registers modified: all except DS,ES,DI
48477 ;-----
48478
48479 ShareEnd:
48480 ; 26/07/2019
48481 00008484 FF1E[9800] call far [Jshare+(2*4)] ; 2 = MFTClose
48482 00008488 C3 retn
48483
48484 ;Break <ShareEnter - attempt to enter a node into the sharing set>
48485 ;-----
48486 ; ShareEnter - perform a retried entry of a node into the sharing set. If
48487 ; the max number of retries is exceeded, we notify the user via int 24.
48488 ;
48489 ; Inputs: ThisSFT points to the SFT
48490 ; WFP_Start points to the WFP
48491 ; Outputs: Carry clear => successful entry
48492 ; Carry set => failed system call
48493 ; Registers modified: all
48494 ;-----
48495
48496 ShareEnter:
48497 00008489 51 push cx
48498
48499 0000848A 8B0E[1A00] retry: mov cx,[RetryCount]
48500
48501 0000848E C43E[9E05] attempt: les di,[THISFT] ; grab sft
48502 00008492 31C0 XOR AX,AX
48503 ;mov [es:di+51],ax
48504 00008494 26894533 MOV [ES:DI+SF_ENTRY.sf_MFT],AX ; indicate free SFT
48505 00008498 51 push cx
48506 00008499 E8ABFF call SHARE_CHECK ; attempt to enter into the sharing set
48507 0000849C 59 pop cx
48508 0000849D 730A jnc short done ; success, let the user see this
48509 0000849F E84293 call Idle ; wait a while
48510 000084A2 E2EA loop attempt ; go back for another attempt
48511 000084A4 E8A5FF call SHARE_VIOLATION ; signal the problem to the user
48512 000084A7 73E1 jnc short retry ; user said to retry, go do it
48513
48514 000084A9 59 done: pop cx
48515 000084AA C3 retn
48516
48517 ;=====
48518 ; EXEPATCH.ASM (MSDOS 6.0, 1991)
48519 ;=====
48520 ; 29/04/2019 - Retro DOS 4.0
48521 ; 20/03/2024 - Retro DOS 5.0
48522
48523 ;** EXEPATCH.ASM
48524 ;-----
48525 ; Contains the foll:
48526 ;
48527 ; - code to find and overlay buggy unpack code
48528 ; - new code to be overlayed on buggy unpack code
48529 ; - old code sequence to identify buggy unpack code
48530 ;
48531 ; Revision history:
48532 ;
48533 ; Created: 5/14/90
48534 ;-----
48535
48536 ;-----
48537 ;
48538 ; M020 : Fix for rational bug - for details see routine header
48539 ; M028 : 4b04 implementation
48540 ; M030 : Fixing bug in EXEPACKPATCH (EXEC_CS is an un-relocated value)
48541 ; M032 : set turnoff bit only if DOS in HMA.
48542 ; M033 : if IP < 2 then not exepacked.
48543 ; M046 : support for a 4th version of exepacked files.
48544 ; M068 : support for copy protected apps.
48545 ; M071 : use A200FF_COUNT of 10.
48546 ;
48547 ;-----
48548
48549 PATCH1_COM_OFFSET EQU 06CH
48550 PATCH1_OFFSET EQU 028H
48551 PATCH1_CHKSUM EQU 0EF4EH
48552 CHKSUM1_LEN EQU 11CH/2 ; 142
48553
48554 PATCH2_COM_OFFSET EQU 076H
48555 PATCH2_OFFSET EQU 032H
48556
48557 ; The strings that start at offset 076h have two possible
48558 ; check sums that are defined as PATCH2_CHKSUM PATCH2A_CHKSUM
48559
48560 PATCH2_CHKSUM EQU 78B2H
48561 CHKSUM2_LEN EQU 119H/2
48562 PATCH2A_CHKSUM EQU 1C47H ; M046
48563 CHKSUM2A_LEN EQU 103H/2 ; M046
48564
48565 PATCH3_COM_OFFSET EQU 074H
48566 PATCH3_OFFSET EQU 032H
48567 PATCH3_CHKSUM EQU 4EDEH
48568 CHKSUM3_LEN EQU 117H/2
48569
48570 ;** Data structure passed for ExecReady call
48571 ;
48572 ;
48573 ; struc ERStruc
48574 ; .ER_Reserved: resw 1 ; reserved, should be zero
48575 ; .ER_Flags: resw 1
48576 ; .ER_ProgName: resd 1 ; ptr to ASCIIZ str of prog name
48577 ; .ER_PSP: resw 1 ; PSP of the program
48578 ; .ER_StartAddr: resd 1 ; Start CS:IP of the program
48579 ; .ER_ProgSize: resd 1 ; Program size including PSP
48580 ; .size:
48581 ; endstruc
48582
48583 ;DOSCODE SEGMENT
48584 ; 22/05/2019 - Retro DOS v4.0
48585 ; DOSCODE:B3DDh (MSDOS 6.21, MSDOS.SYS)
48586
48587 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
48588 ; DOSCODE:B37Ah (MSDOS 5.0, MSDOS.SYS)
48589
48590 ; 20/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
48591 ; DOSCODE:C697h (PCDOS 7.1, IBMDOS.COM)
48592
48593 ; M028 - BEGIN
48594
48595 ;-----
48596 ;

```



```

48597 ; Procedure Name : ExecReady
48598 ;
48599 ; Input : DS:DX -> ERStruc (see exe.inc)
48600 ;
48601 ;-----
48602
48603 ExecReady:
48604 000084AB 89D6 mov si, dx ; move the pointer into a friendly one
48605 ; ;test word [si+2],1
48606 ; 17/12/2022
48607 000084AD F6440201 test byte [si+ERStruc.ER_Flags], ER_EXE ; 1
48608 ;test word [si+ERStruc.ER_Flags], ER_EXE ; COM or EXE ?
48609 000084B1 7418 jz short er_setver ; only setver for .COM files
48610
48611 ;mov ax, [si+8]
48612 000084B3 8B4408 mov ax, [si+ERStruc.ER_PSP]
48613 000084B6 83C010 add ax, 10h
48614 000084B9 8EC0 mov es, ax
48615
48616 ;mov cx, [si+10]
48617 000084BB 8B4C0A mov cx, [si+ERStruc.ER_StartAddr] ; M030
48618 ;mov ax, [si+12] ; 11/04/2024
48619 000084BE 8B440C mov ax, [si+ERStruc.ER_StartAddr+2] ; M030
48620
48621 ;call [ss:FixExePatch]
48622 000084C1 36FF16[670D] call word [ss:FixExePatch] ; 28/12/2022
48623
48624 ; 20/03/2024
48625 ; 04/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
48626 000084C6 36FF16[3E13] call word [ss:Rational386PatchPtr]
48627
48628 er_setver:
48629 ; ;test word [si+2],2 ; Q: is this an overlay
48630 ; 17/12/2022
48631 000084CB F6440202 test byte [si+ERStruc.ER_Flags], ER_OVERLAY ; 2
48632 ;test word [si+ERStruc.ER_Flags], ER_OVERLAY
48633 000084CF 7518 jnz short er_chkddoshi ; Y: set A20OFF_COUNT if DOS high
48634 ; N: set up lie version first
48635 000084D1 1E push ds
48636 000084D2 56 push si
48637 ;lds si, [si+4]
48638 000084D3 C57404 lds si, [si+ERStruc.ER_ProgName]
48639 000084D6 E8DCEC call Scan_Execname1
48640 000084D9 E8EDEC call Scan_Special_Entries
48641 000084DC 5E pop si
48642 000084DD 1F pop ds
48643 ;mov es, [si+8]
48644 000084DE 8E4408 mov es, [si+ERStruc.ER_PSP]
48645 000084E1 36A1[BB0E] mov ax, [ss:SPECIAL_VERSION]
48646 ;mov [es:40h], ax ; 11/04/2024
48647 000084E5 26A34000 mov [es:PDB.Version], ax
48648
48649 er_chkddoshi:
48650 000084E9 36803E[660D]00 cmp byte [ss:DosHashMA], 0 ; M032: Q: is dos in HMA (M021)
48651 000084EF 741F je short er_done ; M032: N: done
48652
48653 ;mov ax, [si+8] ; M068 - Start
48654 mov ax, [si+ERStruc.ER_PSP]; ax = PSP
48655 000084F1 8B4408
48656
48657 ;or byte [ss:DOS_FLAG], 4
48658 000084F4 36800E[8600]04 or byte [ss:DOS_FLAG], EXECA200FF ; Set bit to signal int 21
48659 ; ah = 25 & ah= 49. See dossym.inc
48660 ; under TAG M003 & M009 for
48661 ; explanation
48662 ; ;test word [si+2],1
48663 ; 17/12/2022
48664 000084FA F6440201 test byte [si+ERStruc.ER_Flags], ER_EXE ; 1
48665 ;test word [si+ERStruc.ER_Flags], ER_EXE ; Q: COM file
48666 000084FE 7507 jnz short er_setA20 ; N: inc a20off_count, set
48667 ; a20off_psp and ret
48668 00008500 1E push ds
48669 00008501 8ED8 mov ds, ax ; DS = load segment of com file.
48670 00008503 E8AC05 call IsCopyProt ; check if copy protected
48671 00008506 1F pop ds
48672
48673 er_setA20:
48674 ; We need to inc the A20OFF_COUNT here. Note that if the count
48675 ; is non-zero at this point it indicates that the A20 is to be
48676 ; turned off for that many int 21 calls made by the app. In
48677 ; addition the A20 has to be turned off when we exit from this
48678 ; call. Hence the inc.
48679
48680 00008507 36FE06[8500] inc byte [ss:A20OFF_COUNT]
48681 0000850C 36A3[6300] mov [ss:A20OFF_PSP], ax ; set the PSP for which A20 is to be
48682 ; turned OFF.
48683 er_done: ; M068 - End
48684 00008510 31C0 xor ax, ax
48685 00008512 C3 retn
48686
48687 ; M028 - END
48688
48689 ; 20/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
48690 ; %if 1
48691 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
48692 ; %if 0
48693
48694 ;-----
48695 ;
48696 ; procedure : Rational386Patch
48697 ;
48698 ; Older versions of the Rational DOS Extender have several bugs which trash
48699 ; 386 registers (usually just the high word of 32 bit registers) during
48700 ; interrupt processing. Lotus 123 3.1+ is a popular application that uses a
48701 ; version of the Rational extender with the 32 bit register trashing bugs.
48702 ;
48703 ; This routine applies patches to the Rational DOS Extender to work around
48704 ; most of the register trashing bugs.
48705 ;
48706 ; Note that there are additional register trashing bugs not fixed by these
48707 ; patches. In particular, the high word of ESP and the FS and GS registers
48708 ; may be modified on interrupts.
48709 ;
48710 ; There are two different Rational DOS Extender patches in this module.
48711 ; Rational386Patch is to correct 386 register trashing bugs on 386 or later
48712 ; processors. This patch code is executed when MS-DOS is running on a 386
48713 ; or later processor, regardless of whether MS-DOS is running in the HMA
48714 ; or not.
48715 ;
48716 ; The other Rational patch routine (RationalPatch, below) fixes a register
48717 ; trashing problem on 286 processors, and is only executed if MS-DOS is
48718 ; running in the HMA.
48719 ;
48720 ; This patch detection and replacement is based on an example supplied by

```

```

48721 ; Ben Williams at Rational.
48722 ;
48723 ;-----
48724 ;
48725 ; 22/05/2019 - Retro DOS v4.0
48726 ; DOSCODE:B448h (MSDOS 6.21, MSDOS.SYS)
48727 ;-----
48728 ;
48729 ;
48730 ; INPUT : ES = segment where program got loaded
48731 ;
48732 ;-----
48733 ;
48734 rpFind1:
48735     db      0FAh, 0E4h, 21h, 60h, 33h, 0C0h, 0E6h, 43h, 8Bh, 16h
48736
48737 rpFind1Len equ    $ - rpFind1
48738
48739 ;     cli
48740 ;     in     al, 21h
48741 ;     pusha
48742 ;     xor     ax, ax
48743 ;     out     43h, al
48744 ;     mov     dx, ...
48745
48746 rpFind1a:
48747     db      0B0h, 0Eh, 0E6h, 37h, 33h, 0C0h, 0E6h, 0F2h
48748
48749 rpFind1aLen equ    $ - rpFind1a
48750
48751 ;     mov     al, 0Eh
48752 ;     out     37h, al
48753 ;     xor     ax, ax
48754 ;     out     0F2h, al
48755
48756 ; bug # 1 -- loss of high EAX on 386+ if not VCPI or DPMI
48757
48758 rpFind2:
48759     db      0Fh, 20h, 0C0h
48760
48761 rpFind2Len equ    $ - rpFind2
48762
48763 ;     mov     eax, cr0           ;may be preceeded by PUSH CX (51h)
48764
48765 rpFind3:
48766     db      0Fh, 22h, 0C0h, 0EAh
48767
48768 rpFind3Len equ    $ - rpFind3
48769
48770 ;     mov     cr0, eax           ;may be preceeded by POP CX (59h)
48771 ;     jmp     far ptr xxx        ;change far ptr to go to replace3
48772 ;     mov     ss, bx             ;8E D3 ... and come back at or after this
48773
48774 ; note, there is no rpRep11 string
48775
48776 rpRep12:
48777     db      66h, 50h, 51h, 0Fh, 20h, 0C0h
48778
48779 rpRep12Len equ    $ - rpRep12
48780
48781 ;     push     eax
48782 ;     push     cx
48783 ;     mov     eax, cr0
48784
48785 rpRep13:
48786     db      8Eh, 0D3h, 59h, 66h, 58h
48787
48788 rpRep13Len equ    $ - rpRep13
48789
48790 ;     mov     ss, bx
48791 ;     pop     cx
48792 ;     pop     eax
48793
48794 ; bug # 2 -- loss of high EAX and ESI on 386+ only if VCPI
48795
48796 rpFind4:
48797     db      93h, 58h, 8Bh, 0Cch
48798
48799 rpFind4Len equ    $ - rpFind4
48800
48801 ;     xchg     bx, ax
48802 ;     pop     ax
48803 ;     mov     cx, sp
48804
48805 rpFind5:
48806     db      0B8h, 0Ch, 0DEh, 0CDh, 67h, 8Bh, 0E1h, 0FFh, 0E3h
48807
48808 rpFind5Len equ    $ - rpFind5
48809
48810 ;     mov     ax, DE0Ch
48811 ;     int     67h
48812 ;     mov     sp, cx
48813 ;     jmp     bx
48814
48815 rpRep14:
48816     db      93h, 58h, 8Bh, 0Cch
48817     db      02Eh, 066h, 0A3h
48818
48819 rpRep14o1Len equ    $ - rpRep14
48820
48821     db      00h, 00h
48822     db      02Eh, 066h, 89h, 36h
48823
48824 rpRep14o2Len equ    $ - rpRep14
48825
48826     db      00h, 00h
48827
48828 rpRep14Len equ    $ - rpRep14
48829
48830 ;     xchg     bx, ax
48831 ;     pop     ax
48832 ;     mov     cx, sp
48833 ;     mov     dword ptr cs:[xxxx], eax
48834 ;     mov     dword ptr cs:[xxxx], esi
48835
48836 rpRep15:
48837     db      8Bh, 0E1h
48838     db      2Eh, 66h, 0A1h
48839
48840 rpRep15o1Len equ    $ - rpRep15
48841
48842     db      00h, 00h
48843     db      2Eh, 66h, 8Bh, 36h

```

```

48844
48845
48846
48847 0000855E 0000
48848 00008560 FFE3
48849
48850
48851
48852
48853
48854
48855
48856
48857
48858
48859
48860 00008562 FA5251
48861
48862
48863
48864
48865
48866
48867
48868
48869 00008565 B80CDE6626FF1E
48870
48871
48872
48873
48874
48875
48876
48877 0000856C 595A5B
48878
48879
48880
48881
48882
48883
48884
48885
48886 0000856F FA6650665366516652
48887
48888
48889
48890
48891
48892
48893
48894
48895
48896
48897 00008578 665A6659665B66585B
48898
48899
48900
48901
48902
48903
48904
48905
48906
48907
48908
48909
48910 00008581 60061EB800008ED8
48911
48912
48913
48914
48915
48916
48917
48918
48919
48920
48921 00008589 1F0761
48922
48923
48924
48925
48926
48927
48928
48929
48930 0000858C 6660061E
48931
48932
48933
48934
48935
48936
48937
48938
48939 00008590 1F076661C3
48940
48941
48942
48943
48944
48945
48946
48947
48948
48949
48950
48951 00000000 ????
48952 00000002 ????
48953 00000004 ????
48954 00000006 ????
48955 00000008 ????
48956
48957
48958
48959
48960
48961
48962 00008595 [2585]
48963 00008597 0300
48964 00008599 [2885]
48965 0000859B 0400
48966 0000859D 2000
48967

rpRep15o2Len equ $ - rpRep15
    db    00h, 00h
    db    0FFh, 0E3h

rpRep15Len equ    $ - rpRep15
;    mov    sp, cx
;    mov    eax, dword ptr cs:[xxxx]
;    mov    esi, dword ptr cs:[xxxx]
;    jmp    bx

; bug # 3 -- loss of high EAX, EBX, ECX, EDX on 386+ only if VCPI

rpFind6:
    db    0FAh, 52h, 51h

rpFind6Len equ    $ - rpFind6
;    cli
;    push    dx
;    push    cx

rpFind7a:
    db    0B8h, 0Ch, 0DEh, 66h, 26h, 0FFh, 1Eh

rpFind7aLen equ    $ - rpFind7a
;    mov    ax, 0DE0Ch
;    call    fword ptr es:[xxxx]

rpFind7b:
    db    59h, 5Ah, 5Bh

rpFind7bLen equ    $ - rpFind7b
;    pop     cx
;    pop     dx
;    pop     bx

rpRep16:
    db    0FAh, 66h, 50h, 66h, 53h, 66h, 51h, 66h, 52h

rpRep16Len equ    $ - rpRep16
;    cli
;    push    eax
;    push    ebx
;    push    ecx
;    push    edx

rpRep17:
    db    66h, 5Ah, 66h, 59h, 66h, 5Bh, 66h, 58h, 5Bh

rpRep17Len equ    $ - rpRep17
;    pop     edx
;    pop     ecx
;    pop     ebx
;    pop     eax
;    pop     bx

; bug # 4 -- loss of high EAX and EBX on 386+ only if VCPI

rpFind8:
    db    60h, 06h, 1Eh, 0B8h, 00h, 00h, 8Eh, 0D8h

rpFind8Len equ    $ - rpFind8
;    pusha
;    push    es
;    push    ds
;    mov     ax, dgroup    ;jump back to here from replace8
;    mov     ds, ax

rpFind9 :
    db    1Fh, 07h, 61h

rpFind9Len equ    $ - rpFind9
;    pop     ds
;    pop     es
;    popa

rpRep18:
    db    66h, 60h, 06h, 1Eh

rpRep18Len equ    $ - rpRep18
;    pushad
;    push    es
;    push    ds

rpRep19:
    db    1Fh, 07h, 66h, 61h, 0C3h

rpRep19Len equ    $ - rpRep19
;    pop     ds
;    pop     es
;    popad
;    retn                ;no need to jmp back to main-line

;-----
struc SearchPair
    .sp_off1: resw 1 ; offset of 1st search string
    .sp_len1: resw 1 ; length of 1st search string
    .sp_off2: resw 1 ; 2nd string
    .sp_len2: resw 1 ; 2nd string
    .sp_diff: resw 1 ; max difference between offsets
    .size:
endstruc

;rpBug1Strs SearchPair    <offset rpFind2, rpFind2Len, offset rpFind3, rpFind3Len, 20h>

rpBug1Strs:
    dw    rpFind2
    dw    rpFind2Len ; 3
    dw    rpFind3
    dw    rpFind3Len ; 4
    dw    20h

```

```

48968 ;rpBug2Strs SearchPair    <offset rpFind4, rpFind4Len, offset rpFind5, rpFind5Len, 80h>
48969
48970 rpBug2Strs:
48971     dw    rpFind4
48972     dw    rpFind4Len ; 4
48973     dw    rpFind5
48974     dw    rpFind5Len ; 9
48975     dw    80h
48976
48977 ;rpBug3Strs SearchPair    <offset rpFind6, rpFind6Len, offset rpFind7a, rpFind7aLen, 80h>
48978
48979 rpBug3Strs:
48980     dw    rpFind6
48981     dw    rpFind6Len ; 3
48982     dw    rpFind7a
48983     dw    rpFind7aLen ; 7
48984     dw    80h
48985
48986 ;rpBug4Strs SearchPair    <offset rpFind8, 4, offset rpFind9, rpFind9Len, 80h>
48987
48988 rpBug4Strs:
48989     dw    rpFind8
48990     dw    rpRep18Len ; 4 ; 20/03/2024
48991     dw    rpFind9
48992     dw    rpFind9Len ; 3
48993     dw    80h
48994
48995 ;-----
48996
48997 struct StackVars
48998     .sv_wVersion:    resw 1        ; Rational extender version #
48999     .sv_cbCodeSeg:   resw 1        ; code seg size to scan
49000     .sv_pPatch:      resw 1        ; offset of next avail patch byte
49001     .size:
49002 endstruc
49003
49004 ;-----
49005
49006 ; 22/05/2019 - Retro DOS v4.0
49007
49008 Rational386Patch:
49009     ; Do a few quick checks to see if this looks like a Rational
49010     ; Extended application. Hopefully this will quickly weed out
49011     ; most non Rational apps.
49012
49013     cmp    word [es:0],395        ; version number goes here - versions
49014     jae    short rp3QuickOut      ; 3.95+ don't need patching
49015
49016     cmp    word [es:0Ch],20h      ; always has this value here
49017     jne    short rp3QuickOut
49018
49019     push    ax ; *
49020
49021     mov     ax,18h                ; extender has 18h at
49022     cmp     [es:24],ax           ; offsets 24, 28, & 36
49023     jne     short rp3Q0_ax
49024     cmp     [es:28],ax
49025     jne     short rp3Q0_ax
49026     cmp     [es:36],ax
49027     je      short rp3Maybe
49028
49029 rp3Q0_ax:
49030     pop     ax
49031 rp3QuickOut:
49032     retn
49033
49034 ; It might be the rational extender, do more extensive checking
49035
49036 rp3Maybe:
49037     cld
49038
49039 ; 20/03/2024
49040 %if 0
49041     push    bx                    ; note ax pushed above
49042     push    cx
49043     push    dx
49044     push    si
49045     push    di
49046     push    es
49047     push    ds                    ; we use all of them
49048     push    bp
49049 %else
49050     ; 20/03/2024 (PCDOS 7.1 IBMDOS.COM)
49051     ;;;
49052     pusha
49053     push    es
49054     push    ds
49055     ;;;
49056 %endif
49057     sub     sp,StackVars.size    ; 6; make space for stack variables
49058     mov     bp,sp
49059
49060     push    cs
49061     pop     ds
49062
49063     mov     ax,[es:0]            ; save version #
49064     ;mov     [bp+StackVars.sv_wVersion],ax
49065     mov     [bp],ax
49066     ; check that binary version # matches
49067     ; ascii string
49068     call    VerifyVersion
49069     jne     short rp3Exit_j
49070
49071 ; Looks like this is it, find where to put the patch code. The
49072 ; patch will be located on top of Rational code specific to 80286
49073 ; processors, so these patches MUST NOT be applied if running on
49074 ; an 80286 system.
49075
49076 ; Rational says the code to patch will never be beyond offset 46xxh
49077
49078     mov     cx,4500h             ; force search len to 4700h (searches
49079     ;mov     [bp+2],cx
49080     mov     [bp+StackVars.sv_cbCodeSeg],cx    ; start at offset 200h)
49081
49082     mov     es,[es:20h]          ; es=code segment
49083
49084     mov     si,rpFind1           ; string to find
49085     mov     dx,rpFind1Len ; 10   ; length to match
49086     call    ScanCodeSeq          ; look for code seq
49087     jz      short rpGotPatch
49088
49089 ; According to Rational, some very old versions of the extender may not
49090 ; have the find1 code sequence. If the find1 code wasn't found above,
49091 ; try an alternative patch area which is on top of NEC 98xx switching code.
49092
49093     mov     si,rpFind1a

```

```

49092 00008619 BA0800      mov     dx,rpFind1aLen ;8
49093 0000861C E86001      call    ScanCodeSeq
49094 0000861F 7403          jz      short rpGotPatch
49095
49096
49097 00008621 E9FE00      jmp     rp3Exit
49098
49099      ; Found the location to write patch code! DI = offset in code seg.
49100
49101
49102
49103 00008624 897EFC      ;mov     [bp+4],di
49104      mov     [bp-StackVars.sv_pPatch],di ; save patch pointer
49105
49106      ;-----
49107      ; Bug # 1 -- loss of high EAX on 386+ if not VCPI or DPMI
49108
49109      ;cmp     word [bp+0],381
49110      ;cmp     word [bp+StackVars.sv_wVersion],381 ; only need bug 1 if version
49111 00008627 817E007D01      cmp     word [bp],381
49112 0000862C 7348          jae     short rpBug2 ; < 3.81
49113
49114 0000862E BB[9585]      mov     bx,rpBug1Strs ; locate find2 & find3 code
49115 00008631 E8F600      call    FindBadCode
49116 00008634 7240          jc      short rpBug2
49117
49118      ; si = rpFind2 offset, di = rpFind3 offset
49119
49120 00008636 57          push    di
49121 00008637 89F7      mov     di,si ; rpFind2 offset
49122 00008639 BA0300      mov     dx,rpFind2Len ; 3
49123 0000863C 26807DFF51      cmp     byte [es:di-1],51h ; find2 preceeded by push cx?
49124 00008641 7502          jne     short rp_no_cx
49125
49126 00008643 4F          dec     di ; yes, gobble up push cx too
49127 00008644 42          inc     dx
49128
49129 00008645 BE[2C85]      rp_no_cx: mov     si,rpRep12 ; patch out find2 sequence
49130 00008648 B90600      mov     cx,rpRep12Len ; 6
49131 0000864B E80501      call    GenPatch
49132
49133 0000864E 5F          pop     di ; rpFind3 offset
49134 0000864F 26807DFF59      cmp     byte [es:di-1],59h ; find3 preceeded by pop cx?
49135 00008654 7505          jne     short rp_no_cx2
49136
49137 00008656 26C645FF90      mov     byte [es:di-1],90h ; yes, no-op it
49138
49139      rp_no_cx2: ;mov     ax,[bp+4]
49140 0000865B 8B4604      mov     ax,[bp+StackVars.sv_pPatch] ; change offset of far jmp
49141      ;mov     [es:di+4],ax
49142 0000865E 26894504      mov     [es:di+rpFind3Len],ax ; to go to patch code
49143
49144 00008662 57          push    di ; save find3 offset
49145 00008663 BE[3285]      mov     si,rpRep13 ; copy rep13 to patch area
49146 00008666 B90500      mov     cx,rpRep13Len ; 5
49147 00008669 E8FB00      call    CopyPatch
49148
49149 0000866C 5B          pop     bx ; find3 offset
49150 0000866D 83C308      add     bx,rpFind3Len+4 ; 8 ; skip over find3 and far jmp
49151 00008670 E80001      call    GenJump ; jmp back from patch area
49152      ;mov     [bp+4],di
49153 00008673 897E04      mov     [bp+StackVars.sv_pPatch], di ; to main-line, update patch
49154      ; area pointer
49155
49156      ;-----
49157      ; Bug # 2 -- loss of high regs on 386+ under VCPI only
49158
49159
49160 00008676 BB[9F85]      rpBug2: mov     bx,rpBug2Strs ; locate find4 & find5 code
49161 00008679 E8AE00      call    FindBadCode
49162 0000867C 7242          jc      short rpBug3
49163
49164      ; si = rpFind4 offset, di = rpFind5 offset
49165
49166      ;push    word [bp+4]
49167 0000867E FF7604      push    word [bp+StackVars.sv_pPatch] ; save current patch pointer
49168      ; (where rep14 goes)
49169 00008681 57          push    di ; save find5 offset
49170
49171 00008682 89F7      mov     di,si
49172 00008684 BA0400      mov     dx,rpFind4Len ; 4
49173 00008687 BE[4485]      mov     si,rpRep14
49174 0000868A B90F00      mov     cx,rpRep14Len ; 15
49175 0000868D E8C300      call    GenPatch ; patch out find4 code
49176
49177 00008690 5F          pop     di ; find5 offset
49178 00008691 83C705      add     di,5 ; keep 5 bytes of find5 code
49179
49180 00008694 8B5E04      ;mov     bx,[bp+4]
49181 00008697 53          mov     bx,[bp+StackVars.sv_pPatch] ; jump to patch area
49182 00008698 E8D800      push    bx ; save rep15 location
49183      call    GenJump
49184
49185 0000869B BE[5385]      mov     si,rpRep15 ; copy rep15 code to patch
49186 0000869E B90F00      mov     cx,rpRep15Len ; 15 ; area -- it has a jmp bx
49187 000086A1 E8C300      call    CopyPatch ; so no need to jmp back to
49188      ; main-line code
49189
49190      ; patches have been made, now update the patch code to store/load dwords just
49191      ; after the code in the patch area
49192
49193 000086A4 5F          pop     di ; rep15 location
49194 000086A5 5E          pop     si ; rep14 location
49195
49196 000086A6 8B4604      ;mov     ax,[bp+4]
49197      mov     ax,[bp+StackVars.sv_pPatch] ; (where dwords go)
49198
49199 000086A9 26894407      ;mov     [es:si+7],ax
49200      mov     [es:si+rpRep14o1Len],ax ; offset for EAX
49201      ;mov     [es:di+5],ax
49202 000086AD 26894505      mov     [es:di+rpRep15o1Len],ax
49203 000086B1 83C004      add     ax,4
49204      ;mov     [es:si+0Dh],ax
49205 000086B4 2689440D      mov     [es:si+rpRep14o2Len],ax ; offset for ESI
49206      ;mov     [es:di+0Bh],ax
49207 000086B8 2689450B      mov     [es:di+rpRep15o2Len],ax
49208
49209 000086BC 83460408      ;add     word [bp+4],8
49210      add     word [bp+StackVars.sv_pPatch],8 ; reserve space for 2 dwords in
49211      ; patch area
49212
49213      ;-----
49214      ; Bug # 3 -- loss of high regs on 386+ under VCPI only
49215
49216
49217      rpBug3:

```

```

49216 000086C0 BB[A985]      mov     bx, rpBug3Strs      ; locate find6 & find7a code
49217 000086C3 E86400      call    FindBadCode
49218 000086C6 722C      jc      short rpBug4
49219
49220      ; add     di, 9
49221 000086C8 83C709      add     di, rpFind7aLen + 2      ; skip over offset in find7a
49222 000086CB 56      push    si      ; code and locate find7b
49223 000086CC BE[6C85]      mov     si, rpFind7b      ; sequence
49224 000086CF BA0300      mov     dx, rpFind7bLen ; 3
49225 000086D2 E8AD00      call    ScanCodeSeq_di
49226 000086D5 5E      pop     si
49227 000086D6 751C      jnz     short rpBug4
49228
49229 000086D8 57      push    di      ; save find7b code offset
49230
49231 000086D9 89F7      mov     di, si
49232 000086DB BA0300      mov     dx, rpFind6Len ; 3
49233 000086DE BE[6F85]      mov     si, rpRepl6
49234 000086E1 B90900      mov     cx, rpRepl6Len ; 9
49235 000086E4 E86C00      call    GenPatch      ; patch out find6 code
49236
49237 000086E7 5F      pop     di
49238 000086E8 BA0300      mov     dx, rpFind7bLen ; 3
49239 000086EB BE[7885]      mov     si, rpRepl7
49240 000086EE B90900      mov     cx, rpRepl7Len ; 9
49241 000086F1 E85F00      call    GenPatch      ; patch out find7b code
49242
49243      ; -----
49244      ; Bug # 4 -- loss of high regs on 386+ under VCPI only
49245
49246 rpBug4:
49247      ; cmp     word [bp+0], 360
49248      ; cmp     word [bp+StackVars.sv_wVersion], 360 ; only applies if
49249 000086F4 817E006801      cmp     word [bp], 360
49250 000086F9 7627      jbe     short rp3Exit      ; version > 3.60 and < 3.95
49251
49252 000086FB BB[B385]      mov     bx, rpBug4Strs      ; locate find8 & find9 code
49253 000086FE E82900      call    FindBadCode
49254 00008701 721F      jc      short rp3Exit
49255
49256 00008703 57      push    di      ; save find9 code offset
49257
49258 00008704 89F7      mov     di, si
49259 00008706 BA0300      mov     dx, 3
49260 00008709 BE[8C85]      mov     si, rpRepl8
49261 0000870C B90400      mov     cx, rpRepl8Len ; 4
49262 0000870F E84100      call    GenPatch      ; patch out find8 code
49263
49264 00008712 5F      pop     di      ; find9 offset
49265      ; mov     bx, [bp+4]
49266 00008713 8B5E04      mov     bx, [bp+StackVars.sv_pPatch] ; patch find9 to jmp to
49267 00008716 E85A00      call    GenJump      ; patch area
49268
49269 00008719 BE[9085]      mov     si, rpRepl9
49270 0000871C B90500      mov     cx, rpRepl9Len ; 5
49271 0000871F E84500      call    CopyPatch      ; copy replacement code to
49272      ; patch area--it does a RET
49273      ; so no jmp back to main-line
49274 00008722 83C406      rp3Exit: add     sp, StackVars.size
49275
49276      ; 20/03/2024
49277 %if 0
49278      pop     bp
49279      pop     ds
49280      pop     es
49281      pop     di
49282      pop     si
49283      pop     dx
49284      pop     cx
49285      pop     bx
49286 %else
49287      ; 20/03/2024 (PCDOS 7.1 IBMDOS.COM)
49288      ;;;
49289 00008725 1F      pop     ds
49290 00008726 07      pop     es
49291 00008727 61      popa
49292      ;;;
49293 %endif
49294 00008728 58      pop     ax ; *
49295 00008729 C3      retn
49296
49297      ; -----
49298      ; FindBadCode
49299      ;
49300      ; Searches Rational code segment looking for a pair of find strings (all
49301      ; patches have at least two find strings).
49302      ;
49303      ; Entry:
49304      ; ES = code segment to search
49305      ; DS:BX = search pair structure for this search
49306      ; [bp].sv_cbCodeSeg = length of code seg to search
49307      ;
49308      ; Exit:
49309      ; CY flag clear if both strings found, and
49310      ; SI = offset in ES of 1st string
49311      ; DI = offset in ES of 2nd string
49312      ; CY set if either string not found, or strings too far apart
49313      ;
49314      ; Used:
49315      ; CX
49316      ;
49317      ; -----
49318
49319      ; struct SearchPair
49320      ; .sp_off1: resw 1 ; offset of 1st search string
49321      ; .sp_len1: resw 1 ; length of 1st search string
49322      ; .sp_off2: resw 1 ; 2nd string
49323      ; .sp_len2: resw 1 ; 2nd string
49324      ; .sp_diff: resw 1 ; max difference between offsets
49325      ; .size:
49326      ; endstruc
49327
49328 FindBadCode:
49329      ; mov     cx, [bp+2]
49330      ; mov     cx, [bp+StackVars.sv_cbCodeSeg]      ; search length
49331 0000872A 8B4E02      mov     si, [bx] ; mov si, [bx+0]
49332      ; mov     si, [bx+Searchpair.sp_off1] ; ds:si -> search string
49333 0000872D 8B37      ; mov     dx, [bx+2]
49334      ; mov     dx, [bx+Searchpair.sp_len1] ; dx = search len
49335      ; call    ScanCodeSeq
49336      ; jnz     short fbc_error      ; done if 1st not found
49337 0000872F 8B5702
49338 00008732 E84A00
49339 00008735 751A

```



```

49340
49341 00008737 57      push    di                ; save 1st string offset
49342
49343      ;mov     si,[bx+4]
49344 00008738 8B7704    mov     si,[bx+SearchPair.sp_off2]
49345      ;mov     dx,[bx+6]
49346 0000873B 8B5706    mov     dx,[bx+SearchPair.sp_len2]
49347 0000873E E84100    call    ScanCodeSeq_di        ; don't change flags after this!
49348
49349 00008741 5E        pop     si                ; restore 1st string offset
49350 00008742 750D    jnz     short fbc_error
49351
49352 00008744 89F8    mov     ax,di                ; sanity check that
49353 00008746 29F0    sub     ax,si                ; si < di && di - si <= allowed diff
49354 00008748 7207    jc      short fbc_error
49355      ;cmp     ax,[bx+8]
49356 0000874A 3B4708    cmp     ax,[bx+SearchPair.sp_diff]
49357 0000874D 7702    ja      short fbc_error
49358
49359 0000874F F8        cld
49360 00008750 C3        retn
49361
49362 fbc_error:
49363 00008751 F9        stc
49364 00008752 C3        retn
49365
49366 ;-----
49367 ;
49368 ; GenPatch
49369 ;
49370 ; Generate a patch sequence. 1) insert a jump at the buggy code location
49371 ; (jumps to the patch code area), 2) copy the selected patch code to the
49372 ; patch area, 3) insert a jump from the patch area back to the main-line
49373 ; code.
49374 ;
49375 ; Entry:
49376 ;   ES:DI = start of buggy code to be patched
49377 ;   DX     = length of buggy code to be patched
49378 ;   DS:SI = replacement patch code
49379 ;   CX     = length of replacement patch code
49380 ;   [bp].sv_pPatch = offset in ES of where to copy patch code
49381 ;
49382 ; Exit:
49383 ;   DI, [bp].sv_pPatch = byte after generated patch code
49384 ;
49385 ; Used:
49386 ;   AX, BX, SI, Flags
49387 ;
49388 ;-----
49389
49390 GenPatch:
49391 00008753 57      push    di                ;save offset of buggy code
49392
49393      ;mov     bx,[bp+4]
49394 00008754 8B5E04    mov     bx,[bp+StackVars.sv_pPatch]
49395      ;jump from buggy code to patch area
49396 00008757 E81900    call    GenJump
49397
49398 0000875A E80A00    call    CopyPatch            ;copy replacement code to patch area
49399
49400 0000875D 5B        pop     bx                ;offset of buggy code + buggy code
49401 0000875E 01D3    add     bx,dx                ; length = return from patch offset
49402
49403 00008760 E81000    call    GenJump            ;jump from patch area back to main-
49404      ;mov     [bp+4],di
49405 00008763 897E04    mov     [bp+StackVars.sv_pPatch],di
49406      ; line code, update patch pointer
49407 00008766 C3        retn
49408
49409 ;-----
49410 ;
49411 ; CopyPatch
49412 ;
49413 ; Copies patch code to patch location.
49414 ;
49415 ; Entry:
49416 ;   DS:SI = patch code to be copied
49417 ;   ES     = segment of code to patch
49418 ;   CX     = length of code to copy
49419 ;   [bp].sv_pPatch = offset in ES of where to copy patch code
49420 ;
49421 ; Exit:
49422 ;   DI, [bp].sv_pPatch = byte after copied patch code
49423 ;
49424 ; Used:
49425 ;   SI, Flags
49426 ;
49427 ;-----
49428
49429 CopyPatch:
49430 00008767 51      push    cx
49431      ;mov     di,[bp+4]
49432 00008768 8B7E04    mov     di,[bp+StackVars.sv_pPatch] ;patch pointer is the dest offset
49433 0000876B FC        cld
49434 0000876C F3A4    rep movsb
49435 0000876E 59        pop     cx
49436      ;mov     [bp+4],di
49437 0000876F 897E04    mov     [bp+StackVars.sv_pPatch],di ;update net pointer location
49438 00008772 C3        retn
49439
49440 ;-----
49441 ;
49442 ; GenJump
49443 ;
49444 ; Generates a rel16 JMP instruction at location 'from' to location 'to'.
49445 ;
49446 ; Entry:
49447 ;   ES:DI = from location (where to put jmp instruction)
49448 ;   BX     = to location (where to jump to)
49449 ;
49450 ; Exit:
49451 ;   DI = byte after generated jump
49452 ;
49453 ; Used:
49454 ;   AX
49455 ;
49456 ;-----
49457
49458 GenJump:
49459 00008773 B0E9    mov     al,0E9h            ; jmp rel16 opcode
49460 00008775 AA    stosb
49461
49462 00008776 89D8    mov     ax,bx            ; calc offset to 'to' location
49463 00008778 29F8    sub     ax,di

```

```

49464 0000877A 83E802      sub     ax,2
49465
49466 0000877D AB          stosw           ; output offset
49467
49468 0000877E C3          retn
49469
49470
49471
49472
49473
49474
49475
49476
49477
49478
49479
49480
49481 0000877F BF0002      mov     di,200h
49482 ScanCodeSeq_di:
49483 00008782 51          push    cx
49484 00008783 29D1      sub     cx,dx
49485 00008785 41          inc     cx
49486
49487 00008786 56          scsagain:
49488 00008787 57          push    si
49489 00008788 51          push    di
49490 00008789 89D1      push    cx
49491 0000878B F3A6      mov     cx,dx
49492 0000878D 59          rep     cmpsb
49493 0000878E 5F          pop     cx
49494 0000878F 5E          pop     di
49495 00008790 7403      pop     si
49496 00008792 47          je      short scsfound
49497 00008793 E2F1      inc     di
49498
49499 00008795 59          loop   scsagain
49500
49501 00008796 C3          scsfound:
49502
49503
49504
49505
49506
49507
49508
49509
49510
49511
49512
49513
49514
49515
49516 00008797 268B362A00      mov     si,[es:2Ah]
49517
49518 0000879C B30A      mov     bl,10
49519 0000879E 83C603      add     si,3
49520
49521 000087A1 E80E00      call    VVDigit
49522 000087A4 75F0      jne     short vvexit
49523 000087A6 E80900      call    VVDigit
49524 000087A9 75EB      jne     short vvexit
49525 000087AB 26803C2E      cmp     byte [es:si], '.'
49526 000087AF 75E5      jne     short vvexit
49527 000087B1 4E          dec     si
49528
49529
49530
49531
49532
49533
49534 000087B2 F6F3      ;call VVDigit
49535 000087B4 80C430      ; 18/12/2022
49536 000087B7 4E          ;jmp short VVDigit
49537 000087B8 26386401      ;vvexit:
49538 000087BC B400      ;retn
49539 000087BE C3          VVDigit:
49540
49541
49542
49543
49544
49545
49546
49547
49548
49549
49550
49551
49552
49553
49554
49555
49556
49557 000087BF 06          div     bl
49558 000087C0 8CD8      add     ah,'0'
49559
49560
49561
49562 000087C2 2BC2      dec     si
49563
49564
49565
49566 000087C4 8ED8      cmp     [es:si+1],ah
49567 000087C6 8EC0      mov     ah,0
49568 000087C8 BF0F00      retn
49569 000087CB 57
49570 000087CC B91000
49571 000087CF B0FF
49572 000087D1 F3AE
49573 000087D3 47
49574 000087D4 8BF7
49575 000087D6 5F
49576 000087D7 58
49577
49578
49579
49580 000087D8 2BC2
49581
49582
49583
49584 000087DA 8EC0
49585
49586 000087DC B90402
49587

```



```

49588 000087DF 8BC6      db 8Bh,0C6h      ;mov ax,si
49589 000087E1 F7D0      db 0F7h,0D0h      ;not ax
49590 000087E3 D3E8      db 0D3h,0E8h      ;shr ax,cl
49591 000087E5 7413      db 74h,13h        ;jz short SI_ok
49592 000087E7 8CDA      db 8Ch,0DAh        ;mov dx,ds
49593 000087E9 83CEF0     db 83h,0CEh,0F0h   ;or si,0FFF0H
49594 000087EC 2BD0      db 2Bh,0D0h        ;sub dx,ax
49595 000087EE 7308      db 73h,08h         ;jnc short SItoDS
49596 000087F0 F7DA      db 0F7h,0DAh        ;neg dx
49597 000087F2 D3E2      db 0D3h,0E2h        ;shl dx,cl
49598 000087F4 2BF2      db 2Bh,0F2h        ;sub si,dx
49599 000087F6 33D2      db 33h,0D2h        ;xor dx,dx
49600                                     ;SItoDS:
49601 000087F8 8EDA      db 8Eh,0DAh        ;mov ds,dx
49602                                     ;SI_ok:
49603 000087FA 87F7      db 87h,0F7h        ;xchg si,di
49604 000087FC 1E        db 1Eh             ;push ds
49605 000087FD 06        db 06h             ;push es
49606 000087FE 1F        db 1Fh             ;pop ds
49607 000087FF 07        db 07h             ;pop es
49608 00008800 FECF      db 0FEh,0CDh        ;dec ch
49609 00008802 75DB      db 75h,0DBh        ;jnz short norm_agr
49610 00008804 AC        db 0ACh             ;lodsb
49611 00008805 92        db 92h             ;xchg dx,ax
49612 00008806 4E        db 4Eh             ;dec si
49613 00008807 AD        db 0ADh             ;lodsw
49614 00008808 8BC8      db 8Bh,0C8h        ;mov cx,ax
49615 0000880A 46        db 46h             ;inc si
49616 0000880B 8AC2      db 8Ah,0C2h        ;mov al,dl
49617 0000880D 24FE      db 24h,0FEh        ;and al,0FEH
49618 0000880F 3CB0      db 3Ch,0B0h        ;cmp al,RPTREC
49619 00008811 7505      db 75h,05h         ;jne short TryEnum
49620 00008813 AC        db 0ACh             ;lodsb
49621 00008814 F3AA      db 0F3h,0AAh       ;rep stosb
49622
49623 ; db 0EBh,07h,90h     ;jmp short TryNext
49624 00008816 EB06      db 0EBh,06h       ;jmp short TryNext
49625
49626                                     ;TryEnum:
49627 00008818 3CB2      db 3Ch,0B2h        ;cmp al,ENMREC
49628 0000881A 756C      db 75h,6Ch         ;jne short CorruptExe
49629 0000881C F3A4      db 0F3h,0A4h       ;rep movsb
49630                                     ;TryNext:
49631
49632 0000881E 92        db 92h             ;xchg dx,ax
49633 ; db 8Ah,0C2h         ;mov al,dl
49634
49635 0000881F A801      db 0A8h,01h        ;test al,1
49636 00008821 74B9      db 74h,0B9h        ;jz short NextRec
49637 00008823 9090      db 90h,90h         ;nop,nop
49638
49639 last_stop equ $-second
49640 size_str1 equ $-str1
49641
49642 ; The following is the code that we need to look for in the exe
49643 ; file.
49644
49645 scan_patch1: ; label byte
49646
49647 00008825 8CC3      db 8Ch,0C3h        ;mov bx,es
49648 00008827 8CD8      db 8Ch,0D8h        ;mov ax,ds
49649 00008829 2BC2      db 2Bh,0C2h        ;sub ax,dx
49650 0000882B 8ED8      db 8Eh,0D8h        ;mov ds,ax
49651 0000882D 8EC0      db 8Eh,0C0h        ;mov es,ax
49652 0000882F BF0F00    db 0BFh,0Fh,00h     ;mov di,000FH
49653 00008832 B91000    db 0B9h,10h,00h     ;mov cx,0010H
49654 00008835 B0FF      db 0B0h,0FFh       ;mov al,0FFH
49655 00008837 F3AE      db 0F3h,0AEh       ;repz scasb
49656 00008839 47        db 47h             ;inc di
49657 0000883A 8BF7      db 8Bh,0F7h        ;mov si,di
49658 0000883C 8BC3      db 8Bh,0C3h        ;mov ax,bx
49659 0000883E 2BC2      db 2Bh,0C2h        ;sub ax,dx
49660 00008840 8EC0      db 8Eh,0C0h        ;mov es,ax
49661 00008842 BF0F00    db 0BFh,0Fh,00h     ;mov di,000FH
49662                                     ;NextRec:
49663 00008845 B104      db 0B1h,04h        ;mov cl,4
49664 00008847 8BC6      db 8Bh,0C6h        ;mov ax,si
49665 00008849 F7D0      db 0F7h,0D0h        ;not ax
49666 0000884B D3E8      db 0D3h,0E8h        ;shr ax,cl
49667 0000884D 7409      db 74h,09h         ;jz short SI_ok
49668 0000884F 8CDA      db 8Ch,0DAh        ;mov dx,ds
49669 00008851 2BD0      db 2Bh,0D0h        ;sub dx,ax
49670 00008853 8EDA      db 8Eh,0DAh        ;mov ds,dx
49671 00008855 83CEF0     db 83h,0CEh,0F0h   ;or si,0FFF0H
49672                                     ;SI_ok:
49673 00008858 8BC7      db 8Bh,0C7h        ;mov ax,di
49674 0000885A F7D0      db 0F7h,0D0h        ;not ax
49675 0000885C D3E8      db 0D3h,0E8h        ;shr ax,cl
49676 0000885E 7409      db 74h,09h         ;jz short DI_ok
49677 00008860 8CC2      db 8Ch,0C2h        ;mov dx,es
49678 00008862 2BD0      db 2Bh,0D0h        ;sub dx,ax
49679 00008864 8EC2      db 8Eh,0C2h        ;mov es,dx
49680 00008866 83CF0F     db 83h,0CFh,0F0h   ;or di,0FFF0H
49681                                     ;DI_ok:
49682
49683 size_scan_patch1 equ $-scan_patch1
49684
49685 scan_patch2: ; label byte
49686
49687 00008869 8CC3      db 8Ch,0C3h        ;mov bx,es
49688 0000886B 8CD8      db 8Ch,0D8h        ;mov ax,ds
49689 0000886D 48        db 48h             ;dec ax
49690 0000886E 8ED8      db 8Eh,0D8h        ;mov ds,ax
49691 00008870 8EC0      db 8Eh,0C0h        ;mov es,ax
49692 00008872 BF0F00    db 0BFh,0Fh,00h     ;mov di,000FH
49693 00008875 B91000    db 0B9h,10h,00h     ;mov cx,0010H
49694 00008878 B0FF      db 0B0h,0FFh       ;mov al,0FFH
49695 0000887A F3AE      db 0F3h,0AEh       ;repz scasb
49696 0000887C 47        db 47h             ;inc di
49697 0000887D 8BF7      db 8Bh,0F7h        ;mov si,di
49698 0000887F 8BC3      db 8Bh,0C3h        ;mov ax,bx
49699 00008881 48        db 48h             ;dec ax
49700 00008882 8EC0      db 8Eh,0C0h        ;mov es,ax
49701 00008884 BF0F00    db 0BFh,0Fh,00h     ;mov di,000FH
49702                                     ;NextRec:
49703 00008887 B104      db 0B1h,04h        ;mov cl,4
49704 00008889 8BC6      db 8Bh,0C6h        ;mov ax,si
49705 0000888B F7D0      db 0F7h,0D0h        ;not ax
49706 0000888D D3E8      db 0D3h,0E8h        ;shr ax,cl
49707 0000888F 740A      db 74h,0Ah         ;jz short SI_ok
49708 00008891 8CDA      db 8Ch,0DAh        ;mov dx,ds
49709 00008893 2BD0      db 2Bh,0D0h        ;sub dx,ax
49710 00008895 8EDA      db 8Eh,0DAh        ;mov ds,dx
49711 00008897 81CEF0FF     db 81h,0CEh,0F0h,0FFh

```

```

49712                                     ;or      si,0FFF0H
49713                                     ;SI_ok:
49714 db 8Bh,0C7h                         ;mov     ax,di
49715 db 0F7h,0D0h                         ;not     ax
49716 db 0D3h,0E8h                         ;shr     ax,cl
49717 db 74h,0Ah                          ;jz      short DI_ok
49718 db 8Ch,0C2h                         ;mov     dx,es
49719 db 2Bh,0D0h                         ;sub     dx,ax
49720 db 8Eh,0C2h                         ;mov     es,dx
49721 db 81h,0CFh,0F0h,0FFh               ;or      di,0FFF0H
49722                                     ;DI_ok:
49723
49724
49725 size_scan_patch2 equ $-scan_patch2
49726
49727 scan_patch3: ; label byte
49728
49729 db 8Ch,0C3h                         ;mov     bx,es
49730 db 8Ch,0D8h                         ;mov     ax,ds
49731 db 48h                              ;dec     ax
49732 db 8Eh,0D8h                         ;mov     ds,ax
49733 db 8Eh,0C0h                         ;mov     es,ax
49734 db 0BFh,0Fh,00h                     ;mov     di,000FH
49735 db 0B9h,10h,00h                     ;mov     cx,0010H
49736 db 0B0h,0FFh                       ;mov     al,0FFH
49737 db 0F3h,0AEh                       ;repz    scasb
49738 db 47h                              ;inc     di
49739 db 8Bh,0F7h                         ;mov     si,di
49740 db 8Bh,0C3h                         ;mov     ax,bx
49741 db 48h                              ;dec     ax
49742 db 8Eh,0C0h                         ;mov     es,ax
49743 db 0BFh,0Fh,00h                     ;mov     di,000FH
49744                                     ;NextRec:
49745 db 0B1h,04h                         ;mov     cl,4
49746 db 8Bh,0C6h                         ;mov     ax,si
49747 db 0F7h,0D0h                         ;not     ax
49748 db 0D3h,0E8h                         ;shr     ax,cl
49749 db 74h,09h                          ;jz      short SI_ok
49750 db 8Ch,0DAh                         ;mov     dx,ds
49751 db 2Bh,0D0h                         ;sub     dx,ax
49752 db 8Eh,0DAh                         ;mov     ds,dx
49753 db 83h,0CEh,0F0h                   ;or      si,0FFF0H
49754                                     ;SI_ok:
49755 db 8Bh,0C7h                         ;mov     ax,di
49756 db 0F7h,0D0h                         ;not     ax
49757 db 0D3h,0E8h                         ;shr     ax,cl
49758 db 74h,09h                          ;jz      short DI_ok
49759 db 8Ch,0C2h                         ;mov     dx,es
49760 db 2Bh,0D0h                         ;sub     dx,ax
49761 db 8Eh,0C2h                         ;mov     es,dx
49762 db 83h,0CFh,0F0h                   ;or      di,0FFF0H
49763                                     ;DI_ok:
49764
49765 size_scan_patch3 equ $-scan_patch3
49766
49767 scan_com: ; label byte
49768
49769 db 0ACh                              ;lods b
49770 db 8Ah,0D0h                         ;mov     dl,al
49771 db 4Eh                              ;dec     si
49772 db 0ADh                              ;lodsw
49773 db 8Bh,0C8h                         ;mov     cx,ax
49774 db 46h                              ;inc     si
49775 db 8Ah,0C2h                         ;mov     al,dl
49776 db 24h,0FEh                         ;and     al,0FEH
49777 db 3Ch,0B0h                         ;cmp     al,RPTREC
49778 db 75h,06h                         ;jne     short TryEnum
49779 db 0ACh                              ;lods b
49780 db 0F3h,0AAh                         ;rep stosb
49781 db 0EBh,07h,90h                     ;jmp     short TryNext
49782                                     ;TryEnum:
49783 db 3Ch,0B2h                         ;cmp     al,ENMREC
49784 db 75h,6Bh                         ;jne     short CorruptExe
49785 db 0F3h,0A4h                         ;rep movsb
49786                                     ;TryNext:
49787 db 8Ah,0C2h                         ;mov     al,dl
49788 db 0A8h,01h                         ;test    al,1
49789 ; db 74h,0BAh                       ;jz      short NextRec
49790
49791 size_scan_com equ $-scan_com
49792
49793 ;-----
49794
49795 ; 23/05/2019 - Retro DOS v4.0
49796 ; DOSCODE:B852h (MSDOS 6.21, MSDOS.SYS)
49797
49798 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
49799 ; DOSCODE:B530h (MSDOS 5.0, MSDOS.SYS)
49800
49801 ; 21/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
49802 ; DOSCODE:CB02h (PCDOS 7.1 IBMDOS.COM)
49803
49804 ExePatch:
49805 ; 21/03/2024 - Retro DOS v5.0
49806 ; ;28/12/2022 - Retro DOS v4.1
49807 call ExePackPatch
49808 call word [ss:RationalPatchPtr]
49809 ret
49810 ; 28/12/2022
49811 ; jmp short ExePackPatch
49812
49813 ;-----
49814 ;
49815 ; Procedure Name : ExePackPatch
49816 ;
49817 ; Inputs : DS -> DOSDATA
49818 ; ES:0 -> read in image
49819 ; ax:cx = start cs:ip of program
49820 ;
49821 ; Output :
49822 ;
49823 ; 1. If ES <= 0ffff
49824 ; 2. if exepack signature ('RB') found
49825 ; 3. if common code to patch compares (for 3 diff. versions)
49826 ; 4. if rest of the code & checksum compares
49827 ; 5. overlay buggy code with code in
49828 ; doscode:str1.
49829 ; 6. endif
49830 ; 7. endif
49831 ; 8. endif
49832 ; 9. endif
49833 ;
49834 ; Uses : NONE
49835 ;

```

```

49836 ;-----
49837 ;
49838 ; 21/03/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
49839 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
49840 ; 23/05/2019 - Retro DOS v4.0
49841 ExePackPatch:
49842 00008918 53      push    bx
49843 00008919 8CC3    mov     bx,es      ; bx has load segment
49844 0000891B 81FBFF0F cmp     bx,0FFFFh   ; Q: is the load segment > 64K
49845 0000891F 7602    jbe     short ep_cont ; N:
49846 00008921 5B      pop     bx      ; Y: no need to patch
49847 00008922 C3      retn
49848 ep_cont:
49849 00008923 1E      push    ds
49850 00008924 06      push    es
49851 00008925 50      push    ax
49852 00008926 51      push    cx
49853 00008927 56      push    si
49854 00008928 57      push    di
49855 ;
49856 ; M033 - start
49857 ; exepacked programs have an IP of 12h (>=2)
49858 ;
49859 00008929 83E902 sub     cx,2        ; Q: is IP >=2
49860 0000892C 7303    jnb     short epp_1   ; N: exit
49861 0000892E E9B500 jmp     ep_notpacked
49862 ;
49863 ; ax:cx now points to location of
49864 ; 'RB' if this is an exepacked file.
49865 ; M033 - end
49866 00008931 89CF    mov     di,cx
49867 00008933 8EC0    mov     es,ax
49868 00008935 36893E[8700] mov     [ss:UNPACK_OFFSET],di ; save pointer to 'RB' in
49869 ; unpack_offset
49870 ;
49871 0000893A 26813D5242 cmp     word [es:di],'RB' ; 4252h
49872 ;ljne ep_notpacked
49873 0000893F 7403    je      short epp_2
49874 00008941 E9A200 jmp     ep_notpacked
49875 epp_2:
49876 00008944 0E      push    cs
49877 00008945 1F      pop     ds      ; set ds to cs
49878 ;
49879 ;add di,6Ch
49880 00008946 83C76C add     di,PATCH1_COM_OFFSET ; es:di -> points to place in packed
49881 ; file where we hope to find
49882 ; scan string.
49883 ;
49884 00008949 E8A200 call    chk_common_str      ; check for match
49885 ;
49886 0000894C 7521    jnz     short ep_chkpatch2 ; Q: does the patch match
49887 ; N: check at patch2_offset
49888 ; Y: check for rest of patch string
49889 0000894E BE[2588] mov     si,scan_patch1
49890 ;
49891 00008951 368B3E[8700] mov     di,[ss:UNPACK_OFFSET] ; ds:si -> scan string
49892 ; restore di to point to 'RB'
49893 ;
49894 ;add di,28h
49895 00008956 83C728 add     di,07/12/2022
49896 ; di,PATCH1_OFFSET
49897 ; es:di -> points to place in packed
49898 ; file where we hope to find
49899 ; scan string.
49900 ;
49901 00008959 B144    mov     cx,68
49902 ;mov cx,size_scan_patch1
49903 ; 21/03/2024
49904 0000895B BB8E00 mov     cl,size_scan_patch1 ; 68
49905 ;
49906 0000895E B84EEF mov     bx,142
49907 00008961 E89E00 mov     bx,CHKSUM1_LEN
49908 00008964 7207    mov     ax,0EF4Eh
49909 ;mov ax,PATCH1_CHKSUM
49910 ; call chk_patchsum
49911 ; check if patch and chk sum compare
49912 ; Q: did we pass the test
49913 ; N: exit
49914 ; Y: overlay code with new
49915 00008966 BE[BF87] mov     si,str1
49916 ;
49917 ;mov cx,102
49918 ;mov cx,size_str1
49919 ; 21/03/2024
49920 00008969 B166    mov     cl,size_str1 ; 102
49921 ;
49922 0000896B F3A4    rep     movsb
49923 ep_done1:
49924 jmp     short ep_done ; 21/03/2024
49925 ep_chkpatch2:
49926 ;mov di,76h
49927 0000896F BF7600 mov     di,PATCH2_COM_OFFSET ; es:di -> possible location of patch
49928 ; in another version of unpack
49929 ; call chk_common_str
49930 ; check for match
49931 ;
49932 00008972 E87900 jnz     short ep_chkpatch3 ; Q: does the patch match
49933 ; N: check for patch3_offset
49934 ; Y: check for rest of patch string
49935 ;
49936 00008975 753D    mov     si,scan_patch2
49937 ;
49938 ; ds:si -> scan string
49939 ;
49940 ;mov di,32h
49941 0000897A BF3200 mov     di,PATCH2_OFFSET ; es:di -> points to place in packed
49942 ; file where we hope to find
49943 ; scan string.
49944 ;
49945 ;mov cx,68
49946 ;mov cx,size_scan_patch2
49947 ; 21/03/2024
49948 0000897D B144    mov     cl,size_scan_patch2 ; 68
49949 ;mov bx,140
49950 0000897F BB8C00 mov     bx,CHKSUM2_LEN
49951 ;mov ax,78B2h
49952 00008982 B8B278 mov     ax,PATCH2_CHKSUM
49953 00008985 E87A00 call    chk_patchsum
49954 ; check if patch and chk sum compare
49955 ;
49956 ; M046 - Start
49957 ; Q: did we pass the test
49958 ; Y: overlay code with new
49959 ; N: try with a different checksum
49960 00008988 7310    jnc     short ep_patchcode2
49961 ;
49962 ; ds:si -> scan string
49963 ;
49964 ;mov cx,68
49965 ;mov cx,size_scan_patch2
49966 ; 21/03/2024
49967 0000898A BE[6988] mov     si,scan_patch2
49968 ;
49969 ;mov bx,129
49970 0000898D B144    mov     cl,size_scan_patch2 ; 68
49971 ;mov bx,129
49972 0000898F BB8100 mov     bx,CHKSUM2A_LEN
49973 ;mov ax,1C47h

```

```

49960 00008992 B8471C      mov     ax,PATCH2A_CHKSUM
49961 00008995 E86A00      call    chk_patchsum      ; check if patch and chk sum compare
49962                                ; Q: did we pass the test
49963 00008998 724C      jc      short ep_notpacked ; N: try with a different checksum
49964                                ; Y: overlay code with new
49965
49966                                ; M046 - End
49967 0000899A BE[BF87]      ep_patchcode2:
49968                                mov     si,str1
49969                                ;;mov   cx,3
49970                                ;mov    cx,first_stop
49971                                ; 21/03/2024
49972 0000899D B103      mov     cl,first_stop ; 3
49973 0000899F F3A4      rep     movsb
49974 000089A1 B89048      mov     ax,4890h          ; ax = opcodes for dec ax, nop
49975                                stosw
49976                                ;add    si,2
49977 000089A5 46      ; 21/03/2024
49978 000089A6 46      inc     si
49979                                inc     si
49980                                ;;mov   cx,20
49981                                ;mov    cx,second_stop
49982                                ; 21/03/2024
49983 000089A7 B114      mov     cl,second_stop ; 20
49984 000089A9 F3A4      rep     movsb
49985                                stosw          ; put in dec ax and nop
49986                                ;add    si,2
49987 000089AC 46      ; 21/03/2024
49988 000089AD 46      inc     si
49989                                inc     si
49990                                ;;mov   cx,75
49991                                ;mov    cx,last_stop
49992                                ; 21/03/2024
49993 000089AE B14B      mov     cl,last_stop ; 75
49994 000089B0 F3A4      rep     movsb
49995 000089B2 EB32      jmp     short ep_done
49996
49997                                ep_chkpatch3:
49998 000089B4 BF7400      ;mov    di,74h
49999                                mov     di,PATCH3_COM_OFFSET ; es:di -> possible location of patch
50000                                ; in another version of unpack
50001                                call    chk_common_str      ; check for match
50002 000089BA 752A      jnz     short ep_notpacked ; Q: does the patch match
50003                                ; N: exit
50004                                ; Y: check for rest of patch string
50005 000089BC BE[AD88]      mov     si,scan_patch3
50006                                ; ds:si -> scan string
50007                                ;mov    di,32h
50008 000089BF BF3200      mov     di,PATCH3_OFFSET ; es:di -> points to place in packed
50009                                ; file where we hope to find
50010                                ; scan string.
50011                                ;;mov   cx,66
50012                                ;mov    cx,size_scan_patch3
50013                                ; 21/03/2024
50014 000089C2 B142      mov     cl,size_scan_patch3 ; 66
50015                                ;mov    bx,139
50016 000089C4 BB8B00      mov     bx,CHKSUM3_LEN
50017                                ;mov    ax,4EDEh
50018 000089C7 B8DE4E      mov     ax,PATCH3_CHKSUM
50019 000089CA E83500      call    chk_patchsum      ; check if patch and chk sum compare
50020 000089CD 7217      jc      short ep_notpacked ; Q: did we pass the test
50021                                ; N: exit
50022                                ; Y: overlay code with new
50023 000089CF BE[BF87]      mov     si,str1
50024                                ;;mov   cx,3
50025                                ;mov    cx,first_stop
50026                                ; 21/03/2024
50027 000089D2 B103      mov     cl,first_stop ; 3
50028 000089D4 F3A4      rep     movsb
50029 000089D6 B048      mov     al,48h          ; al = opcode for dec ax
50030 000089D8 AA      stosb
50031                                ;add    si,2
50032                                ; 21/03/2024
50033 000089D9 46      inc     si
50034 000089DA 46      inc     si
50035                                ;;mov   cx,20
50036                                ;mov    cx,second_stop
50037                                ; 21/03/2024
50038 000089DB B114      mov     cl,second_stop ; 20
50039 000089DD F3A4      rep     movsb
50040 000089DF AA      stosb          ; put in dec ax
50041                                ;add    si,2
50042                                ; 21/03/2024
50043 000089E0 46      inc     si
50044 000089E1 46      inc     si
50045                                ;;mov   cx,75
50046                                ;mov    cx,last_stop
50047                                ; 21/03/2024
50048 000089E2 B14B      mov     cl,last_stop ; 75
50049 000089E4 F3A4      rep     movsb
50050
50051                                ep_notpacked:
50052                                ;stc
50053                                ep_done:
50054 000089E6 5F      pop     di
50055 000089E7 5E      pop     si
50056 000089E8 59      pop     cx
50057 000089E9 58      pop     ax
50058 000089EA 07      pop     es
50059 000089EB 1F      pop     ds
50060 000089EC 5B      pop     bx
50061 000089ED C3      retn
50062
50063                                ;-----
50064                                ;
50065                                ; Procedure Name : chk_common_str
50066                                ;
50067                                ; Input          : DS = DOSCODE
50068                                ;              ; ES:DI points to string in packed file
50069                                ;
50070                                ; Output         : Z if match else NZ
50071                                ;-----
50072
50073                                ; 23/05/2019 - Retro DOS v4.0
50074                                chk_common_str:
50075                                mov     si,scan_com
50076 000089EE BE[EF88]      ; ds:si -> scan string
50077                                ;mov    cx,32
50078                                ;mov    cx,size_scan_com
50079 000089F1 B92000      repe    cmpsb
50080                                ; M046 - start
50081 000089F4 F3A6      ; a fourth possible version of these exepacked programs have a
50082
50083

```

```

50084      ; 056h instead of 06Bh. See scan_com above
50085      ;
50086      db 75h, 6Bh      ;jne CorruptExe
50087      ;
50088      ; If the mismatch at this point is due to a 56h instead of 6Bh
50089      ; we shall try to match the rest of the string
50090      ;
50091
50092      jz     short ccs_done
50093      cmp     byte [es:di-1],56h
50094      jnz     short ccs_done
50095
50096      repe     cmpsb
50097      ccs_done:      ; M046 - end
50098      retn
50099
50100      ;-----
50101      ;
50102      ; Procedure Name : chk_patchsum
50103      ;
50104      ; Input      : DS:SI -> string we're looking for
50105      ;             : ES:DI -> offset in packed file
50106      ;             : CX   = scan length
50107      ;             : BX   = length of check sum
50108      ;             : AX   = value of check sum
50109      ;
50110      ; Output     : if patch & check sum compare
50111      ;             : NC
50112      ;             : else
50113      ;             : CY
50114      ;
50115      ; Uses       : AX, BX, CX, SI
50116      ;-----
50117      ;
50118      ; 23/05/2019 - Retro DOS v4.0
50119      chk_patchsum:
50120      push     di
50121      00008A02 57
50122
50123      repe     cmpsb
50124
50125      jnz     short cp_fail      ; Q: does the patch match
50126      ; N: exit
50127      ; Y:
50128
50129      ; we do a check sum starting from the location of the
50130      ; exepack signature 'RB' up to 11c/2 bytes, the end of the
50131      ; unpacking code.
50132
50133      00008A07 368B3E[8700]      mov     di,[ss:UNPACK_OFFSET] ; di -> start of unpack code
50134      00008A0C 89D9      mov     cx,bx      ; cx = length of check sum
50135
50136      mov     bx,ax      ; save check sum passed to us in bx
50137      xor     ax,ax
50138      ep_chksum:
50139      add     ax,[es:di]
50140      ;add     di,2
50141      ; 01/07/2024 (PCDOS 7.1 IBMDOS.COM)
50142      inc     di
50143      inc     di
50144      loop    ep_chksum
50145
50146      pop     di      ; restore di
50147
50148      cmp     ax,bx      ; Q: does the check sum match
50149      ;jne     short cp_fail      ; N: exit
50150      ; Y:
50151      ; 25/09/2023
50152      ;clc
50153      ;retn
50154      00008A1C 74E3      je     short ccs_done ; cf=0
50155
50156      cp_fail:
50157      stc
50158      retn
50159
50160      ; 21/03/2024 - Retro DOS v5.0
50161      ;%if 1
50162      ; 28/12/2022 - Retro DOS v4.1
50163      ;%if 0
50164      ;-----
50165
50166      ; M020 : BEGIN
50167      ;
50168      ;-----
50169      ;
50170      ; procedure : RationalPatch
50171      ;
50172      ; A routine (in Ration DOS extender) which is invoked at hardware interrupts
50173      ; clobbers CX register on 286 machines. (123 release 3 uses Rational DOS
50174      ; extender). This routine identifies Buggy Rational EXEs and fixes the bug.
50175      ;
50176      ; THE BUG is in the following code sequence:
50177      ;
50178      ; 8b 0e 10 00      mov     cx, ds:[10h]      ; delay count
50179      ; 90              even      ; word align
50180      ; e2 fe          loop     $      ; wait      CLOBBERS CX
50181      ; e8 xx xx      call    set_A20      ; enable A20
50182      ;
50183      ; This patch routine replaces the mov & the loop with a far call into a
50184      ; routine in DOS data segment which is in low memory (because A20 line
50185      ; is off). The routine (RatBugCode) in DOS data saves & restores CX around
50186      ; a mov & loop.
50187      ;
50188      ; Identification of Buggy Rational EXE
50189      ; =====
50190      ;
50191      ; (ALL OFFSETS ARE IN THE PROGRAM SECTION - EXCLUDING THE EXE HEADER)
50192      ;
50193      ;
50194      ; OFFSET      Contains
50195      ; -----
50196      ; 0000h      100 times version number in binary
50197      ;             bug exists in version 3.48 thru 3.83 (both inclusive)
50198      ;
50199      ; 000ah      the WORDS : 0000h, 0020h, 0000h, 0040h, 0001h
50200      ;
50201      ; 002ah      offset where version number is stored in ASCII
50202      ;             e.g. '3.48A'
50203      ;
50204      ; 0030h      offset of copyright string. Copyright strings either
50205      ;             start with "DOS/16M Copyright...." or
50206      ;             "Copyright....". The string contains
50207      ;             "Rational Systems, Inc."

```

```

50208 ;
50209 ; 0020h word : Paragraph offset of the buggy code segment
50210 ; from the program image
50211 ; 0016h word : size of buggy code segment
50212 ;
50213 ; Buggy code is definite to start after offset 200h in its segment
50214 ;
50215 ;-----
50216 ;
50217 ; 23/05/2019 - Retro DOS v4.0
50218 ; DOSCODE:B976h (MSDOS 6.21, MSDOS.SYS)
50219 ;
50220 ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
50221 ; DOSCODE:B654h (MSDOS 5.0, MSDOS.SYS)
50222 ;
50223 RScanPattern1:
50224 00008A20 000020000000400001- db 0, 0, 20h, 0, 0, 0, 40h, 0, 1, 0
50225 00008A29 00
50226 ;
50227 RLen1 equ $ - RScanPattern1
50228 ;
50229 00008A2A 8B0E100090E2FEE8 RScanPattern2:
50230 db 8Bh, 0Eh, 10h, 00h, 90h, 0E2h, 0FEh, 0E8h
50231 ;
50232 RLen2 equ $ - RScanPattern2
50233 ;
50234 00008A32 8B0E1000E2FEE8 RScanPattern3:
50235 db 8Bh, 0Eh, 10h, 00h, 0E2h, 0FEh, 0E8h
50236 ;
50237 RLen3 equ $ - RScanPattern2
50238 ; DOSCODE:B98Fh (MSDOS 6.21, MSDOS.SYS)
50239 ; DOSCODE:B66Dh (MSDOS 5.0, MSDOS.SYS)
50240 ;
50241 ;-----
50242 ;
50243 ; INPUT : ES = segment where program got loaded
50244 ;
50245 ;-----
50246 ;
50247 ; 21/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
50248 ; DOSCODE:CC2Fh (PCDOS 7.1 IBMDOS.COM)
50249 ;
50250 RationalPatch:
50251 00008A39 FC cld
50252 ;
50253 ; 21/03/2024
50254 %if 0
50255 push ax
50256 push bx
50257 push cx
50258 push dx
50259 push si
50260 push di
50261 %else
50262 ; 21/03/2024 (PCDOS 7.1 IBMDOS.COM)
50263 ;;;
50264 00008A3A 60 pusha
50265 ;;;
50266 %endif
50267 00008A3B 06 push es
50268 00008A3C 1E push ds ; we use all of them
50269 00008A3D BF0A00 mov di,0Ah ; look for pat1 at offset 0Ah
50270 00008A40 0E push cs
50271 00008A41 1F pop ds
50272 ;
50273 00008A42 BE[208A] mov si,RScanPattern1
50274 ;mov
50275 00008A45 B90A00 mov cx,10
50276 00008A48 F3A6 rep cmpsb ; do we have the pattern ?
50277 00008A4A 754A jne short rpexit
50278 00008A4C 26A10000 mov ax,[es:0]
50279 00008A50 3D5C01 cmp ax,348 ; is it a buggy version ?
50280 00008A53 7241 jb short rpexit
50281 00008A55 3D7F01 cmp ax,383 ; is it a buggy version ?
50282 00008A58 773C ja short rpexit
50283 ;
50284 00008A5A E83AFD call VerifyVersion
50285 00008A5D 7537 jne short rpexit
50286 ;
50287 00008A5F 268B0E1600 mov cx,[es:16h] ; Length of buggy code seg
50288 00008A64 81E90002 sub cx,200h ; Length we search (we start
50289 ; at offset 200h)
50290 00008A68 268E062000 mov es,[es:20h] ; es=buggy code segment
50291 00008A6D BE[2A8A] mov si,RScanPattern2
50292 ;mov
50293 00008A70 BA0800 mov dx,8
50294 00008A73 E809FD call ScanCodeSeq ; look for code seq with nop
50295 00008A76 740B jz short rpfound
50296 ;
50297 00008A78 BE[328A] mov si,RScanPattern3
50298 ;mov
50299 00008A7B BA0F00 mov dx,15
50300 00008A7E E8FEFC call ScanCodeSeq ; look for code seq w/o nop
50301 00008A81 7513 jnz short rpexit
50302 ;
50303 rpfound:
50304 ;
50305 ; we set up a far call into DOS data
50306 ; dx has the length of the code seq we were searching for
50307 ;
50308 00008A83 B09A mov al,9Ah ; far call opcode
50309 00008A85 AA stosb
50310 00008A86 B8[2611] mov ax,RatBugCode
50311 00008A89 AB stosw
50312 00008A8A 8CD0 mov ax,ss
50313 00008A8C AB stosw
50314 00008A8D 89D1 mov cx,dx
50315 00008A8F 83E906 sub cx,6 ; filler (with NOPs)
50316 00008A92 B090 mov al,90h
50317 00008A94 F3AA rep stosb
50318 ;
50319 00008A96 1F rpexit:
50320 00008A97 07 pop ds
50321 ;
50322 ; 21/03/2024
50323 %if 0
50324 pop di
50325 pop si
50326 pop dx
50327 pop cx
50328 pop bx
50329 pop ax
50330 %else

```



```

50331             ; 21/03/2024 (PCDOS 7.1 IBMDOS.COM)
50332             ;;;
50333 00008A98 61      popa
50334             ;;;
50335             %endif
50336 00008A99 C3      retn
50337
50338             ; M020 END
50339
50340             ;-----
50341             ; 21/03/2024
50342             ;%endif             ; 28/12/2022
50343
50344             ;-----
50345             ;
50346             ; M068
50347             ;
50348             ; Procedure Name : IsCopyProt
50349             ;
50350             ; Inputs           : DS:100 -> start of com file just read in
50351             ;
50352             ; Outputs          : sets the A20OFF_COUNT variable to 10 if
50353             ;                  the program loaded in DS:100 uses a MICROSOFT
50354             ;                  copy protect scheme that relies on the A20 line
50355             ;                  being turned off for it's scheme to work.
50356             ;
50357             ;                  Note: The int 21 function dispatcher will turn
50358             ;                  a20 off, if the A20OFF_COUNT is non-zero
50359             ;                  and dec the A20OFF_COUNT before iretting
50360             ;                  to the user.
50361             ;
50362             ; Uses              : ES, DI, SI, CX
50363             ;
50364             ;-----
50365
50366             ; 23/05/2019 - Retro DOS v4.0
50367
50368 CPStartOffset    EQU    0175h
50369 CPID1offset EQU    011Bh
50370 CPID2offset EQU    0173h
50371 CPID3offset EQU    0146h
50372 CPID4offset EQU    0124h
50373 ID1            EQU    05343h
50374 ID2            EQU    05044h
50375 ID3            EQU    0F413h
50376 ID4            EQU    08000h
50377
50378             ; DOSCODE:B9FAh (MSDOS 6.21, MSDOS.SYS)
50379
50380             ; 04/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
50381             ; DOSCODE:B71Ch (MSDOS 5.0, MSDOS.SYS)
50382
50383             ; 21/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
50384             ; DOSCODE:CC90h (PCDOS 7.1 IBMDOS.COM)
50385
50386 CPScanPattern:
50387 00008A9A 89264801    db      89h,26h,48h,01h          ; mov [148],sp
50388 00008A9E 8C0E4C01    db      8Ch,0Eh,4Ch,01h          ; mov [14C],cs
50389 00008AA2 C7064A010001 db      0C7h,06h,4Ah,01h,00h,01h ; mov [14A],100h
50390 00008AA8 8C0E1301    db      8Ch,0Eh,13h,01h          ; mov [113],cs
50391 00008AAC B82001      db      0B8h,20h,01h          ; mov ax,120h
50392 00008AAF BE0001      db      0BEh,00h,01h          ; mov si,100h
50393
50394 CPSPlen        EQU $ - CPScanPattern
50395
50396             ; DOSCODE:BA12h (MSDOS 6.21, MSDOS.SYS)
50397             ; DOSCODE:B734h (MSDOS 5.0, MSDOS.SYS)
50398             ; 21/03/2024
50399             ; DOSCODE:CCA8h (PCDOS 7.1 IBMDOS.COM)
50400
50401 IsCopyProt:
50402 00008AB2 813E1B014353 cmp     word [CPID1offset],ID1
50403 00008AB8 752D      jne     short CP_done
50404
50405 00008ABA 813E73014450 cmp     word [CPID2offset],ID2
50406 00008AC0 7525      jne     short CP_done
50407
50408 00008AC2 813E460113F4 cmp     word [CPID3offset],ID3
50409 00008AC8 751D      jne     short CP_done
50410
50411 00008ACA 813E24010080 cmp     word [CPID4offset],ID4
50412 00008AD0 7515      jne     short CP_done
50413
50414 00008AD2 0E          push    cs
50415 00008AD3 07          pop     es
50416 00008AD4 BF[9A8A]   mov     di,CPScanPattern ; es:di -> Pattern to find
50417
50418 00008AD7 BE7501      mov     si,CPStartOffset ; ds:si -> possible location
50419                                ; of pattern
50420
50421 00008ADA B91800      mov     cx,CPSPlen ; 24 ; cx = length of pattern
50422 00008ADD F3A6      repe    cmpsb
50423 00008ADF 7506      jnz     short CP_done
50424
50425 00008AE1 36C606[8500]0A mov     byte [ss:A20OFF_COUNT],0Ah ; M071
50426 CP_done:
50427 00008AE7 C3      retn
50428
50429 ;DOSCODE ENDS
50430
50431             ;END
50432
50433             ;-----
50434
50435             ;align 2 ; 05/09/2018 (Error!)
50436
50437             ; 07/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
50438             ;align 16 ; 08/09/2018 (OK.)
50439             align 2
50440
50441             ; 06/08/2018 - Retro DOS v3.0
50442             ;=====
50443             ; MSINIT.ASM
50444             ;=====
50445             ; 22/04/2019 - Retro DOS v4.0 (MSINIT.ASM, MSDOS 6.0, 1991)
50446             ;
50447             ; MAIN ENTRY FOR DOS INITIALIZATION
50448             ;
50449             ; 15/07/2018 - Retro DOS v3.0
50450             ; (MSDOS 3.3, IBMDOS.COM, 1987)
50451
50452             ; temp iret instruction
50453
50454             ; 05/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)

```

```

50455 ; DOSCODE:B76Ah (MSDOS 5.0, MSDOS.SYS)
50456
50457 initiret: ; MSDOS 6.0
50458 SYSBUF:
50459 ;IRETT: ; 06/05/2019
50460 00008AE8 CF      ired
50461
50462 ; 22/04/2019 - Retro DOS v4.0
50463
50464 ; pointer to the BIOS data segment that will be available just to the
50465 ; initialization code
50466
50467 00008AE9 7000      InitBioDataSeg: dw 70h ; KERNEL_SEGMENT = 0070h
50468
50469 ; 05/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
50470 ; DOSCODE:B76Dh (MSDOS 5.0, MSDOS.SYS)
50471
50472 ; 22/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
50473 ; DOSCODE:CCE1h (PCDOS 7.1 IBMDOS.COM)
50474
50475 ; Convert AX from a number of bytes to a number of paragraphs (round up).
50476
50477 ParaRound:
50478 00008AEB 83C0F      add ax, 15
50479 00008AEE D1D8      rcr ax, 1
50480 00008AF0 D1E8      shr ax, 1
50481 00008AF2 D1E8      shr ax, 1
50482 00008AF4 D1E8      shr ax, 1
50483 00008AF6 C3        retn
50484
50485 ; -----
50486
50487 ; 22/03/2024 - Retro DOS v5.0
50488 %if 1
50489 ; 28/12/2022 - Retro DOS v4.1
50490 %if 0
50491
50492 ; -----
50493 ;
50494 ; Procedure Name : whatCPUType
50495 ;
50496 ; Inputs      : none
50497 ;
50498 ; Outputs     : AL = 0 if CPU < 286
50499 ;               = 1 if CPU == 286
50500 ;               = 2 if CPU >= 386
50501 ;
50502 ; Regs. Mod.   : AX
50503 ;
50504 ; -----
50505
50506 whatCPUType:
50507 ; 25/04/2019 - Retro DOS v4.0
50508 ;get_cpu_type ; done with a MACRO which can't be generated > once
50509
50510 ;CPUType.INC (MSDOS 6.0, 1991)
50511
50512 ; Note: this must be a macro, and not a subroutine in the BIOS since
50513 ; it is called from both CODE and SYSINITSEG.
50514 ;
50515 ;-----GET_CPU_TYPE-----May, 88 by M.Williamson
50516 ; Returns: AX = 0 if 8086 or 8088
50517 ;             = 1 if 80286
50518 ;             = 2 if 80386
50519 ;
50520 ; 04/11/2022
50521 ; MSDOS 5.0 MSDOS.SYS - DOSCODE:0BB03h
50522
50523 ; 22/03/2024
50524 ; PCDOS 7.1 IBMDOS.COM - DOSCODE:0CCEDh
50525
50526 Get_CPU_Type:      ;macro
50527 00008AF7 9C        pushf
50528 00008AF8 53        push bx
50529 00008AF9 31DB      xor bx,bx
50530
50531 00008AFB 31C0      xor ax,ax
50532 00008AFD 50        push ax
50533 00008AFE 9D        popf
50534 00008AFF 9C        pushf
50535 00008B00 58        pop ax
50536 00008B01 2500F0    and ax,0F000h
50537 00008B04 3D00F0    cmp ax,0F000h
50538 00008B07 740E      je short cpu_8086
50539
50540 00008B09 B800F0    mov ax,0F000h
50541 00008B0C 50        push ax
50542 00008B0D 9D        popf
50543 00008B0E 9C        pushf
50544 00008B0F 58        pop ax
50545 00008B10 2500F0    and ax,0F000h
50546 00008B13 7401      jz short cpu_286
50547
50548 00008B15 43        inc bx
50549
50550 00008B16 43        inc bx
50551
50552 00008B17 89D8      mov ax,bx
50553 00008B19 5B        pop bx
50554 00008B1A 9D        popf
50555
50556 ;endm
50557
50558 ; 04/11/2022 (MSDOS 5.0 MSDOS.SYS)
50559 00008B1B C3        retn ; 19/09/2023
50560
50561 ; -----
50562 %endif ; 28/12/2022
50563
50564 ;=====
50565
50566 ; MAIN ENTRY FOR DOS INITIALIZATION
50567
50568 ; 05/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
50569 ; DOSCODE:B779h (MSDOS 5.0 MSDOS.SYS)
50570
50571 ; 23/03/2024
50572 ; 22/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
50573 ; DOSCODE:CCEDh (PCDOS 7.1 IBMDOS.COM)
50574
50575 ; DOSCODE:BA57h (MSDOS 6.22 MSDOS.SYS)
50576 ; BIOSCODE:CF81h (Windows ME IO.SYS)
50577
50578 ; 30/05/2019

```



```

50579 ; 22/04/2019 - Retro DOS v4.0
50580 ; 07/07/2018 - Retro DOS v3.0
50581 ; Retro DOS v2.0 - 03/03/2018
50582 ; 03/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
50583 ; MSDOS 5.0 - MSDOS.SYS, offset 79A9h
50584 DOSINIT:
50585 ; MSDOS 6.21 - MSDOS.SYS, offset 7C77h
50586 ;
50587 ; Far call from SYSINIT
50588 ; DX = Memory size in paragraphs
50589 ; DS:SI = [DEVICE_LIST] (SYSINIT.S)
50590 ; (Retro DOS v2.0, 16/03/2018)
50591 ;
50592 ; ES:DI = ptr to BIOS communication block (sysinit3.s)
50593 ; (Retro DOS v4.0, 20/04/2019)
50594 ;
50595 00008B1C FA CLI
50596 00008B1D FC CLD
50597 ;
50598 ; 03/11/2022
50599 ;push dx ; 30/05/2019 ; save parameters from BIOS
50600 ;
50601 ; 17/12/2022
50602 ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
50603 ;push dx ; =* ; save parameters from BIOS
50604 ;
50605 00008B1E 56 push si
50606 00008B1F 1E push ds
50607 00008B20 57 push di ;save di (ptr to BiosComBlock)
50608 ;
50609 00008B21 8CC3 mov bx,es ;bx:di = ptr to BiosComBlock
50610 ;
50611 ; First, move the DOS data segment to its final location in low memory
50612 ;
50613 ;;;mov ax,0BF69h ; MSDOS 6.21 MSDOS.SYS, file offset 7C7Fh
50614 ;;;mov ax,0BC77h ; MSDOS 5.0 MSDOS.SYS, file offset 79B1h
50615 ; 22/03/2024
50616 ;mov ax,0D20Fh ; PCDOS 7.1 IBMDOS.COM, file offset 92FFh
50617 00008B23 B8[DF8F] mov ax,MEMSTRT ; get offset of end of init code
50618 ;
50619 ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
50620 00008B26 83C0F add ax,15 ; round to nearest paragraph
50621 ;and ax,~15 ; 0FFF0h ; boundary
50622 ; 12/04/2024
50623 00008B29 24F0 and al,0F0h
50624 ;
50625 00008B2B 89C6 mov si,ax ; si = offset of DOSDATA in current
50626 ; ; code segment
50627 ; 05/12/2022
50628 ; 30/04/2019 - Retro DOS v4.0
50629 ;xor si,si
50630 ;
50631 ;mov ax,cs
50632 ;mov ds,ax ; ds = current code segment
50633 ; ; DS:SI now points to dosdata
50634 ; 22/03/2024
50635 00008B2D 0E push cs
50636 00008B2E 1F pop ds
50637 ;
50638 ;mov es,[cs:0BA49h] ; MSDOS 6.21 IO.SYS, offset 7C8Eh
50639 ;mov es,[cs:InitBioDataSeg] ; First access to DosDataSg in
50640 ; ; BData segment. Cannot use
50641 ; ; getdseg macro here!!!
50642 ;
50643 00008B2F 8E06[E98A] mov es,[InitBioDataSeg]
50644 ; 07/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
50645 ;mov es,[cs:InitBioDataSeg] ; ds = cs !
50646 ;
50647 ;mov es,[es:3]
50648 00008B33 268E060300 mov es,[es:DosDataSg] ; Get free location in low memory
50649 ;
50650 00008B38 31FF xor di,di ; ES:DI now points to RAM data
50651 ;
50652 ;mov cx,4970 ; offset 0BA78h in MSDOS 6.21 MSDOS.SYS)
50653 ;mov cx,4976 ; 25/05/2019
50654 ; 22/03/2024
50655 ;mov cx,4934 ; PCDOS 7.1 IBMDOS.COM
50656 ; 05/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
50657 ;mov cx,4962
50658 ;mov cx,MSDAT001E ; get end of dosdata = size of dosdata
50659 00008B3A B94613 mov cx,DOSDATASIZE ; = 4962 for MSDOS 5.0 MSDOS.SYS
50660 ; ; = 4934 for PCDOS 7.1 IBMDOS.COM ; 01/07/2024
50661 00008B3D F3A4 rep movsb ; move data to final location
50662 ;
50663 00008B3F 5F pop di ; restore ptr to BiosComBlock
50664 00008B40 1F pop ds ; restore parms from BIOS
50665 00008B41 5E pop si
50666 ; 17/12/2022
50667 ;pop dx ; 30/05/2019
50668 ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
50669 ;pop dx ; =*
50670 ;
50671 00008B42 06 push es
50672 00008B43 1E push ds
50673 00008B44 07 pop es ; es:si -> device chain
50674 00008B45 1F pop ds ; ds points to dosdata
50675 ;
50676 ;SR;
50677 ;We get a ptr to the BIOS exchange data block. This has been setup right
50678 ;now so that the EXEC call knows when SysInit is present to do the special
50679 ;lie table handling for device drivers. This can be expanded later on to
50680 ;establish a communication block from the BIOS to the DOS.
50681 ;
50682 ;mov [1040h],di ; Offset 0BA87h in MSDOS 6.21 MSDOS.SYS)
50683 ;mov [1042h],bx
50684 00008B46 893E[2313] mov [BiosDataPtr],di
50685 00008B4A 891E[2513] mov [BiosDataPtr+2],bx ; save ptr to BiosComBlock
50686 ;
50687 00008B4E 2E8C1E[0700] mov [cs:DosDSeg],ds ; set pointer to dosdata in code seg
50688 ;
50689 ; Set the segment of LowInt23/24/28Addr in msctrlc.asm to dosdata
50690 ;
50691 00008B53 2E8C1E[F95D] mov [cs:LowInt23Addr+2],ds; set pointers in code seg
50692 00008B58 2E8C1E[FD5D] mov [cs:LowInt24Addr+2],ds
50693 00008B5D 2E8C1E[015E] mov [cs:LowInt28Addr+2],ds
50694 ;
50695 ;mov [346h],dx ; MSDOS 6.21 DOSDATA addresses
50696 ;mov [584h],sp
50697 ;mov [586h],ss
50698 00008B62 8916[4603] mov [ENDMEM],dx ; =*
50699 00008B66 8926[8405] mov [USER_SP],sp
50700 00008B6A 8C16[8605] mov [USER_SS],ss
50701 ;
50702 ;mov ax,ds ; set up ss:sp to dosdata:diskstack

```

```

50703             ;mov     ss,ax
50704             ; 01/07/2024
50705 00008B6E 1E    push    ds
50706 00008B6F 17    pop     ss
50707
50708             ;mov     sp,920h          ; MSDOS 6.21 DOSDATA address
50709             ;mov     sp,offset dosdata:dskstack
50710 00008B70 BC[2009] mov     sp,DSKSTACK      ; 920h ; PC DOS 7.1 IBMDOS.COM ; 22/03/2024
50711
50712             ;M023
50713             ; Init patch ptrs to default values
50714
50715             ; 22/03/2024
50716             %if 0
50717             ;mov     word [1212h],RetExePatch
50718             ;mov     word [1214h],RetExePatch
50719             ;mov     word [61h],RetExePatch
50720             mov     word [FixExePatch],RetExePatch      ; M023
50721             ; 28/12/2022 - Retro DOS v4.1
50722             ;mov     word [RationalPatchPtr],RetExePatch ; M023
50723             mov     word [ChkCopyProt],RetExePatch      ; M068
50724             %else
50725             ; 22/03/2024 (PCDOS 7.1 IBMDOS.COM)
50726             ;;;
50727             mov     ax,RetExePatch
50728             mov     [FixExePatch],ax
50729             mov     [RationalPatchPtr],ax ; 25/03/2024
50730             mov     [ChkCopyProt],ax
50731             ;;;
50732             %endif
50733
50734             ; 22/03/2024
50735             %if 1
50736             ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
50737             ;%if 0      ; 19/09/2023
50738
50739             ; Setup to call 386 Rational DOS Extender patch routine if running on
50740             ; a 386 or later. Unlike other patches, this is not dependent on MS-DOS
50741             ; running in the HMA.
50742
50743             call     whatCPUtype      ; get cpu type (0 < 286,1==286,2 >= 386)
50744             cmp     al,2              ; 386 or later?
50745             mov     ax,Rational386Patch
50746             jae     short di_set_patch
50747             mov     ax,RetExePatch ; < 386, don't need this patch
50748             di_set_patch:
50749             mov     [Rational386PatchPtr],ax ; patch routine or RET instr.
50750
50751             %endif
50752             ; Set up the variable temp_dosloc to point to the dos code segment
50753
50754             mov     ax,cs              ; ax = current segment of DOS code
50755
50756             ; ax now holds segment of DOS code
50757             mov     [TEMP_DOSLOC],ax  ; store temp location of DOS
50758
50759             mov     word [NULDEV+2],es ; nuldev -> points to device chain
50760             mov     word [NULDEV],si
50761             ;SR;
50762             ; There are some locations in the win386 instance data structures
50763             ; which need to be set up with the DOS data segment. First, initialize
50764             ; the segment part of the instance table pointer in the SIS.
50765
50766             ;mov     [0FF2h],ds ; [win386_Info+14+2]
50767             ;mov     [0EF1h],ds ; PC DOS 7.1 IBMDOS.COM ; 22/03/2024
50768             mov     [win386_Info+win386_SIS.Instance_Data_Ptr+2],ds
50769
50770             ; Now initialize the segment part of the pointer to the data in each
50771             ; instance table entry.
50772
50773             push     si                ; preserve pointer to device chain
50774             ; 18/12/2022
50775             ; cx = 0
50776             mov     cx,7
50777             ;mov     cx,7              ; There are 7 entries in the instance table
50778             ; M019
50779             ;mov     si,0FF6h ; offset (dosdata:Instance_Table+2)
50780             ;mov     si,0EF9h ; PC DOS 7.1 IBMDOS.COM ; 22/03/2024
50781             mov     si,Instance_Table+2 ; point si to segment field
50782             Instance_init_loop:
50783             mov     [si],ds           ; set offset in instance entry
50784             add     si,6
50785             add     si,size_of_win386_IIS ; move on to next entry
50786             loop    Instance_init_loop
50787
50788             ;Initialize the WIN386 2.xx instance table with the DOS data segment value
50789
50790             ; 18/12/2022
50791             mov     cx,5
50792             ;mov     cx,5              ; There are five entries in the instance table
50793
50794             ;mov     si,(offset dosdata:OldInstanceJunk) + 6
50795             ;mov     si,11EDh          ; point si to segment field
50796             ;mov     si,1102h ; PC DOS 7.1 IBMDOS.COM ; 22/03/2024
50797             mov     si,OldInstanceJunk+6
50798             OldInstance_init_loop:
50799             mov     [si],ds           ; set offset in instance entry
50800             add     si,6              ; move on to next entry
50801             loop    OldInstance_init_loop
50802             pop     si                ; restore pointer to device chain
50803
50804             ; End of WIN386 2.xx compatibility bullshit
50805
50806             ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
50807             %if 0
50808             ; 30/04/2019
50809             push     es
50810             pop      ds
50811             ; ds:si points to console device
50812
50813             ; 24/04/2019 - Retro DOS v4.0
50814
50815             ; 15/07/2018
50816             ; MSDOS 3.3 (IBMDOS.COM, 1987)
50817             ; (Set INT 2Ah handler address to an 'IRET')
50818
50819             ; need crit vector init'd to use devio call
50820             push     ds                ; preserve segment of device chain
50821             push     es ; 30/04/2019
50822
50823             %endif
50824             ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
50825             push     es
50826             ; 17/12/2022

```

```

50827             ;pop    ds
50828             ;push   ds
50829
50830 00008BBB 31C0      xor    ax,ax
50831 00008BBD 8ED8      mov    ds,ax          ; point DS to int vector table
50832 00008BBF B8[E88A]  mov    ax,initiret
50833             ;mov    [0A8h],ax ; [2Ah*4]
50834 00008BC2 A3A800    mov    [addr_int_ibm],ax
50835 00008BC5 8CC8      mov    ax,cs
50836             ;mov    [0AAh],ax ; [(2Ah*4)+2]
50837 00008BC7 A3AA00    mov    [addr_int_ibm+2],ax
50838 00008BCA 1F         pop     ds          ; restore segment of device chain
50839
50840 00008BCB E84702     call   CHARINIT      ; initialize console driver
50841 00008BCE 56         push   si          ; save pointer to header
50842
50843             push   ss          ; move pointer to dos data...
50844 00008BD0 07         pop     es          ; ...into ES
50845
50846             ;initialize sft for file 0 (CON)
50847
50848             ; 07/07/2018 - Retro DOS v3.0
50849             ; 24/04/2019 - Retro DOS v4.0
50850             ;mov    di,SFTABL+6      ; SFT0_SFTable ; 22/03/2024
50851 00008BD1 BF[D200]  MOV     DI,SFTABL+SFT.SFTable ; Point to sft 0
50852 00008BD4 B80300    MOV     AX,3
50853 00008BD7 AB         STOSW      ; Refcount
50854             ;DEC     AL
50855             ; 22/03/2024 - Retro DOS v5.0
50856 00008BD8 48         dec     ax
50857 00008BD9 AB         STOSW      ; Access rd/wr, compatibility
50858 00008BDA 30C0      XOR     AL,AL
50859 00008BDC AA         STOSB      ; attribute
50860             ;mov    al,0C3h
50861 00008BDD B0C3      mov     al,devid_device_EOF|devid_device|ISCIN|ISCOUT
50862 00008BDF AB         STOSW      ; flags
50863 00008BE0 89F0      mov     ax,si
50864 00008BE2 AB         stosw      ; device pointer in devptr
50865 00008BE3 8CD8      mov     ax,ds
50866 00008BE5 AB         stosw
50867 00008BE6 31C0      xor     ax,ax ; 0
50868
50869             ; 22/03/2024
50870             %if 0
50871             stosw      ; firclus
50872             stosw      ; time
50873             stosw      ; date
50874             dec     ax ; -1
50875             stosw      ; size
50876             stosw
50877             inc     ax ; 0
50878             stosw      ; position
50879             stosw
50880             %else
50881             ; 22/03/2024 (PCDOS 7.1 IBMDOS.COM)
50882             ;;
50883 00008BE8 83C720     add     di,32      ; SFTABL+SFT.SFTable + 43
50884 00008BEB AB         stosw      ; SFT0_SFTable + 43 ; .sf_fclus32
50885 00008BEC AB         stosw      ; SF_ENTRY.sf_fclus32+2
50886 00008BED 83C7DE     add     di,-34      ; 0FFDEh ; SFTABL+SFT.SFTable + 13
50887 00008BF0 AB         stosw      ; SFT0_SFTable + 13 ; .sf_time
50888 00008BF1 AB         stosw      ; SF_ENTRY.sf_date
50889 00008BF2 48         dec     ax
50890 00008BF3 AB         stosw      ; SFT0_SFTable + 17 ; .sf_size
50891 00008BF4 AB         stosw      ; SF_ENTRY.sf_size + 2
50892 00008BF5 40         inc     ax
50893 00008BF6 AB         stosw      ; SFT0_SFTable + 21 ; .sf_position
50894 00008BF7 AB         stosw      ; SF_ENTRY.sf_position + 2
50895             ;;
50896             %endif
50897
50898             ;add     di,7      ; SFT0_SFTable + 32 ; 22/03/2024
50899 00008BF8 83C707     add     di,SF_ENTRY.sf_name-SF_ENTRY.sf_cluspos
50900             ; point at name
50901             ;add     si,10
50902 00008BFB 83C60A     add     si,SYSDEV.NAME ; sdevname
50903             ; point to name
50904 00008BFE B90400     mov     cx,4
50905 00008C01 F3A5      rep     movsw      ; name
50906 00008C03 B103      mov     cl,3
50907 00008C05 B020      mov     al," "
50908 00008C07 F3AA      rep     stosb      ; extension
50909
50910 00008C09 5E         pop     si          ; get back pointer to header
50911
50912             ; mark device as CON I/O
50913
50914             ; 15/07/2018
50915 00008C0A 804C0403    OR     BYTE [SI+4],ISCIN|ISCOUT ; or byte [si+4],3
50916             ; 12/03/2018
50917             ;mov    [ss:32h],si
50918 00008C0E 368936[3200] MOV     [SS:BCON],SI
50919             ;mov    [ss:34h],ds
50920 00008C13 368C1E[3400] MOV     [SS:BCON+2],DS
50921
50922             ; initialize each device until the clock device is found
50923
50924 CHAR_INIT_LOOP:
50925 00008C18 C534      LDS     SI,[SI]          ; AUX device
50926 00008C1A E8F801     call   CHARINIT      ;15/07/2018
50927             ;test   byte [SI+4],8
50928             TEST    BYTE [SI+SYSDEV.ATT],ISCLOCK
50929 00008C1D F6440408    JZ     SHORT CHAR_INIT_LOOP
50930 00008C21 74F5      ; 12/03/2018
50931             ;mov    [ss:2Eh],si
50932             MOV     [SS:BCLOCK],SI
50933 00008C23 368936[2E00] ;mov    [ss:30h],ds
50934             MOV     [SS:BCLOCK+2],DS
50935 00008C28 368C1E[3000] ;MOV     BP,MEMSTRT ; Retro DOS 3.0 ; ES:BP points to DPB
50936             ;MOV
50937
50938             ;mov    bp,4970      ; bp = pointer to free mem
50939             ;mov    bp,4976      ; 25/05/2019 - Retro DOS v4.0
50940             ; 05/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0, MSDOS.SYS)
50941             ;mov    bp,4962 ; (MSDOS 5.0 MSDOS.SYS)
50942             ; 22/03/2024 - Retro DOS v5.0
50943             ;mov    bp,4934 ; (PCDOS 7.1 IBMDOS.COM)
50944 00008C2D BD4613     mov     bp,MSDAT001E      ; es:bp points to dpb area
50945
50946 00008C30 36892E[2600] mov     [ss:DPBHEAD],bp      ; set offset of pointer to DPB's
50947 00008C35 368C06[2800] mov     [ss:DPBHEAD+2],es    ; set segment of pointer to DPB's
50948
50949 PERDRV:
50950             ;lds     si,[SI+SYSDEV.NEXT] ; 15/07/2018

```

```

50951 00008C3A C534          LDS SI,[SI]          ; Next device
50952 00008C3C 83FEFF        CMP SI,-1      ; 0FFFFh
50953          ;JZ          SHORT CONTINIT
50954          ; 23/03/2024
50955          ;;;
50956 00008C3F 7503          jnz          short PERDRV2
50957 00008C41 E99C00        jmp          CONTINIT
50958 PERDRV2:
50959          ;;;
50960
50961 00008C44 E8CE01          call         CHARINIT
50962
50963          ; Retro DOS v2.0 - 16/03/2018 (NOTE for 'CHARINIT' return):
50964          ; [CALLUNIT] = Number of drives for (Disk) Block Dev Driver ([DRVMAX])
50965          ; (.when the command is 'DSK$INIT', as in 'CHARINIT')
50966          ; [CALLBPB] = [DEVCALL.COUNT] = Address of the BPB (DEVCALL offset 18)
50967          ; (REF: MSDOS 3.3 MSBIO2.ASM, MSDATA.INC, MSDISK.ASM, MSBIO1.ASM)
50968          ; (. 'DSK$IN' in MSBIO1.ASM)
50969          ; DEVCALL.MEDIA = CALLUNIT (DEVCALL offset 13)
50970
50971          ; 15/07/2018
50972          ;test word [SI+4],8000h          ; DEVTYP
50973          ; 17/12/2022
50974          ;test byte [SI+5],80h
50975 00008C47 F6440580        test        byte [SI+SYSDEV.ATT+1],(DEVTP>>8) ; 80h
50976          ;TEST word [SI+SYSDEV.ATT],DEVTP ; 8000h
50977 00008C4B 75ED          jnz          SHORT PERDRV          ; Skip any other character devs
50978
50979 00008C4D 368A0E[6703]      MOV CL,[SS:CALLUNIT] ; 12/03/2018
50980 00008C52 30ED          XOR          CH,CH
50981          ; 07/07/2018
50982          ;MOV [SI+10],CL          ; Number of units in name field
50983 00008C54 884C0A          mov         [si+SYSDEV.NAME],cl          ; sdevname
50984 00008C57 368A16[4600]    MOV         DL,[SS:NUMIO] ; 15/03/2018
50985 00008C5C 30F6          XOR          DH,DH
50986 00008C5E 36000E[4600]    ADD         [SS:NUMIO],CL ; 12/03/2018
50987 00008C63 1E          PUSH        DS
50988 00008C64 56          PUSH        SI
50989 00008C65 36C51E[6C03]    LDS         BX,[SS:CALLBPB]          ; 12/03/2018
50990
50991 PERUNIT:
50992 00008C6A 8B37          MOV         SI,[BX]          ; DS:SI Points to BPB
50993 00008C6C 43          INC         BX
50994 00008C6D 43          INC         BX          ; On to next BPB
50995          ; 15/12/2022
50996          ; 07/07/2018
50997          ;mov [ES:BP+DPB.DRIVE],DL
50998 00008C6E 26885600        MOV         [ES:BP],DL
50999          ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
51000          ;mov [ES:BP+0],DL
51001          ;mov [ES:BP+DPB.DRIVE],DL
51002
51003          ;MOV [ES:BP+1],DH
51004 00008C72 26887601        MOV         [ES:BP+DPB.UNIT],DH
51005 00008C76 53          PUSH        BX
51006 00008C77 51          PUSH        CX
51007 00008C78 52          PUSH        DX
51008
51009          ; 23/03/2024 (PCDOS 7.1 IBMDOS.COM)
51010          ;;;
51011 00008C79 BA5241          mov         dx,4152h ; 'RA'          ; signature ('AR') for FAT32 extended BPB/DPB
51012 00008C7C 31C9          xor         cx,cx ; 0
51013          ;mov [es:bp+29],cx
51014 00008C7E 26894E1D        mov         [es:bp+DPB.NEXT_FREE],cx
51015 00008C82 394C0B          cmp         [si+A_BPB.SECTORSPERFAT],cx ; 16 bit FAT size = 0 ?
51016          ;cmp [si+11],cx          ; BPB_FATSz16
51017 00008C85 7514          jnz         short PERUNIT2 ; FAT (FAT12 or FAT16) -old DPB-
51018          ;mov [es:bp+57],cx          ; FAT32 -new- DPB
51019 00008C87 26894E39        mov         [es:bp+DPB.FAT32_NXTFREE],cx
51020          ;mov [es:bp+59],cx
51021 00008C8B 26894E3B        mov         [es:bp+DPB.FAT32_NXTFREE+2],cx
51022 00008C8F 49          dec         cx ; 0FFFFh ; -1
51023          ;mov [es:bp+31],cx
51024 00008C90 26894E1F        mov         [es:bp+DPB.FREE_CNT],cx
51025          ;mov [es:bp+33],cx
51026 00008C94 26894E21        mov         [es:bp+DPB.FREE_CNT_HW],cx
51027 00008C98 B95845          mov         cx,4558h ; 'XE'          ; signature ('EX') for FAT32 extended BPB/DPB
51028 PERUNIT2:
51029          ;;;
51030
51031          ;invoke $SETDPB
51032 00008C9B E8C787          CALL        _$SETDPB          ; build DPB!
51033
51034          ; 07/07/2018
51035          ;MOV AX,[ES:BP+2]
51036 00008C9E 268B4602        mov         ax,[ES:BP+DPB.SECTOR_SIZE]
51037          ; 12/03/2018
51038 00008CA2 363B06[3600]    CMP         AX,[SS:MAXSEC]          ; Q:is this the largest sector so far
51039 00008CA7 7604          JBE         SHORT NOTMAX          ; N:
51040 00008CA9 36A3[3600]        MOV         [SS:MAXSEC],AX          ; Y: save it in maxsec
51041 NOTMAX:
51042          ; set the next dpb field in the currently built bpb
51043          ; and mark as never accessed
51044
51045          ; 24/04/2019
51046 00008CAD 89E8          mov         ax,bp          ; get pointer to DPB
51047          ;add ax,33
51048          ;add ax,61 ; next DPB (PCDOS 7.1 DPB size = 61) ; 23/03/2024
51049 00008CAF 83C03D        add         ax,DPBSIZ          ; advance pointer to next DPB
51050          ; set seg & offset of next DPB
51051          ;mov [es:bp+25],ax
51052 00008CB2 26894619        mov         [es:bp+DPB.NEXT_DPB],ax
51053          ;mov [es:bp+27],es
51054 00008CB6 268C461B        mov         [es:bp+DPB.NEXT_DPB+2],es
51055          ; mark as never accessed
51056          ;mov byte [es:bp+24],0FFh
51057 00008CBA 26C64618FF        mov         byte [es:bp+DPB.FIRST_ACCESS],-1
51058
51059 00008CBF 5A          POP         DX
51060 00008CC0 59          POP         CX
51061 00008CC1 5B          POP         BX
51062 00008CC2 8CD8          MOV         AX,DS          ; save segment of bpb array
51063 00008CC4 5E          POP         SI
51064 00008CC5 1F          POP         DS
51065          ; ds:si -> device header
51066          ; store it in the corresponding dpb
51067          ; 07/07/2018
51068          ;MOV [ES:BP+19],SI ; 24/04/2019
51069 00008CC6 26897613        mov         [ES:BP+DPB.DRIVER_ADDR],si
51070          ;MOV [ES:BP+21],DS ; 24/04/2019
51071 00008CCA 268C5E15        mov         [ES:BP+DPB.DRIVER_ADDR+2],ds
51072
51073 00008CCE 1E          PUSH        DS          ; save pointer to device header
51074 00008CCF 56          PUSH        SI

```

```

51075 00008CD0 FEC6      INC     DH                ; inc unit #
51076 00008CD2 FEC2      INC     DL                ; inc drive #
51077 00008CD4 8ED8      MOV     DS,AX              ; restore segment of BPB array
51078                      ; ;add bp,33 ; 24/04/2019
51079                      ; ;add bp,61 ; 23/03/2024 (PCDOS 7.1)
51080 00008CD6 83C53D    ADD     BP,DPBSIZ          ; advance pointer to next dpb
51081 00008CD9 E28F      LOOP    PERUNIT          ; process all units in each driver
51082
51083 00008CDB 5E          POP     SI                ; restore pointer to device header
51084 00008CDC 1F          POP     DS
51085 00008CDD E95AFF      JMP     PERDRV          ; process all drivers in chain
51086
51087 CONTINIT:
51088                      ; 24/04/2019
51089                      ; ;sub bp,33 ; set link in last DPB to -1
51090                      ; ;sub bp,61 ; 23/03/2024 (PCDOS 7.1)
51091 00008CE0 83ED3D      sub     bp,DPBSIZ          ; back up to last dpb
51092                      ; ;set last link offset & segment
51093                      ; 23/03/2024 - Retro DOS v5.0
51094                      ; if 0
51095                      ; mov word [bp+25],0FFFFh
51096                      ; mov word [bp+DPB.NEXT_DPB],-1
51097                      ; mov word [bp+27],0FFFFh
51098                      ; mov word [bp+DPB.NEXT_DPB+2],-1
51099                      ; else
51100                      ; 23/03/2024 (PCDOS 7.1 IBMDOS.COM)
51101                      ; ;
51102 00008CE3 B8FFFF      mov     ax,0FFFFh ; -1
51103                      ; mov word [bp+25],ax
51104 00008CE6 894619      mov     word [bp+DPB.NEXT_DPB],ax ; -1
51105                      ; mov word [bp+27],ax
51106 00008CE9 89461B      mov     word [bp+DPB.NEXT_DPB+2],ax ; -1
51107                      ; ;
51108                      ; endif
51109                      ; ;add bp,33
51110                      ; ;add bp,61 ; 23/03/2024 (PCDOS 7.1)
51111 00008CEC 83C53D      add     BP,DPBSIZ          ; advance to free memory again
51112                      ; ;the DPB chain is done.
51113 00008CEF 16          push    ss
51114 00008CF0 1F          pop     ds
51115
51116 00008CF1 89E8          mov     ax,bp
51117 00008CF3 E8F5FD      call   ParaRound          ; round up to segment
51118
51119 00008CF6 8CDA          mov     dx,ds              ; dx = dosdata segment
51120 00008CF8 01C2          add     dx,ax              ; dx = ds+ax first free segment
51121
51122 00008CFA BB0F00      mov     bx,0Fh
51123
51124                      ; 24/05/2019
51125                      ; mov cx,[ENDMEM]
51126                      ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
51127                      ; 17/12/2022
51128                      ; mov cx,[ENDMEM]
51129                      ; ;set seg inpacketto dosdata
51130 00008CFD 8C1E[A203]    mov     [DSKCHRET+3],ds ; mov [DOSSEG_INIT],ds
51131
51132                      ; Patch in the segments of the interrupt vectors with current code segment.
51133                      ; Also patch in the segment of the pointers in the dosdata area.
51134                      ; ;
51135                      ; Note: Formerly, temp_dosloc was initialized to -1 until after these
51136                      ; calls were done. The procedure patch_misc_segments is called multiple
51137                      ; times, and relies on temp_dosloc being initialized to -1 as a flag
51138                      ; for the first invocation. Thus, we must set it to -1 for this call.
51139
51140 00008D01 52          push    dx                ; preserve first free segment
51141
51142 00008D02 A1[A30A]      mov     ax,[TEMP_DOSLOC]   ; ax = segment to patch in
51143 00008D05 8EC0          mov     es,ax             ; es = segment of DOS
51144 00008D07 C706[A30A]FFFF mov     word [TEMP_DOSLOC],-1 ; -1 means first call to patch_misc_segments
51145
51146 00008D0D E8BC01      call   patch_vec_segments ; uses AX as doscode segment
51147 00008D10 E8F101      call   patch_misc_segments ; patch in segments for sharer and
51148                      ; other tables with seg in ES.
51149                      ; 17/12/2022
51150                      ; cx = 0
51151 00008D13 8C06[A30A]    mov     [TEMP_DOSLOC],es   ; put back segment of dos code
51152
51153 00008D17 5A          pop     dx                ; restore first free segment
51154
51155                      ; We shall now proceed to set the offsets of the interrupt vectors handled
51156                      ; by DOS to their appropriate values in DOSCODE. In case the DOS loads in
51157                      ; HIMEM the offsets also will be patched to their appropriate values in the
51158                      ; low_mem_stub by seg_reinit.
51159
51160                      ; xor ax,ax ; 0
51161                      ; mov ds,ax
51162                      ; mov es,ax
51163                      ; 17/12/2022
51164                      ; cx = 0
51165                      ; xor cx,cx ; 0
51166 00008D18 8ED9          mov     ds,cx
51167 00008D1A 8EC1          mov     es,cx
51168
51169                      ; set the segment of int 24 vector that was
51170                      ; left out by patch_vec_segments above.
51171
51172                      ; 17/12/2022
51173                      ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
51174                      ; if 0
51175                      ; 24/05/2019
51176                      ; ;mov di,90h
51177                      ; ;mov di,4*int_fatal_abort
51178                      ; ;mov di,addr_int_fatal_abort
51179 00008D1C BF9200      mov     di,addr_int_fatal_abort+2 ; 24/05/2019
51180
51181 00008D1F 36A1[A30A]    mov     ax,[ss:TEMP_DOSLOC]
51182                      ; ;mov [di+2],ax ; int 24h segment
51183 00008D23 8905          mov     [di],ax ; 24/05/2019
51184
51185                      ; ;mov di,82h
51186                      ; ;mov di,INTBASE+2
51187
51188                      ; endif
51189                      ; 17/12/2022
51190                      ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
51191                      ; ;mov di,90h
51192                      ; ;mov di,4*int_fatal_abort
51193                      ; ;mov di,addr_int_fatal_abort
51194                      ; ;mov ax,[ss:TEMP_DOSLOC]
51195                      ; ;mov [di+2],ax ; int 24h segment
51196                      ; ;mov di,82h
51197                      ; ;mov di,INTBASE+2
51198

```

```

51199 ; set default divide trap offset
51200
51201 ;mov word ptr ds:[0],offset doscode:divov
51202 00008D25 C7060000[D55F] mov word [0],DIVOV
51203
51204 ; set vectors 20-28 and 2a-3f to point to iret.
51205
51206 ;mov di,80h
51207 00008D2B BF8000 mov di,INTBASE
51208 ;mov ax,offset doscode:irett
51209 00008D2E B8[CB02] mov ax,IRETT
51210
51211 ; 17/12/2022
51212 ; cx = 0
51213 00008D31 B109 mov cl,9
51214 ;mov cx,9 ; set 9 offsets (skip 2 between each)
51215 ; sets offsets for ints 20h-28h
51216
51217 00008D33 AB iset1:
51218 stosw
51219 ;add di,2
51220 00008D34 47 ; 20/09/2023
51221 00008D35 47 inc di
51222 00008D36 E2FB inc di
51223 loop iset1
51224 00008D38 83C704 add di,4 ; skip vector 29h
51225
51226 ; mov cx,6 ; set 6 offsets (skip 2 between each)
51227 ; sets offsets for ints 2Ah-2Fh
51228
51229 ;iset2:
51230 ; stosw
51231 ; add di,2
51232 ; loop iset2
51233 ; 30h & 31h is the CPM call entry point whose segment address is set up by
51234 ; patch_vec_segments above. So skip it.
51235
51236 ; add di,8 ; skip vector 30h & 31h
51237
51238 ;;;
51239 ; 06/05/2019 - Retro DOS v4.0
51240 ;mov cx,5 ; set offsets for int 2Ah-2Eh
51241 ; 17/12/2022
51242 00008D3B B105 mov cl,5 ; 28/06/2019
51243 ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
51244 ;mov cx,6
51245
51246 00008D3D AB iset2:
51247 stosw
51248 ;add di,2
51249 00008D3E 47 ; 20/09/2023
51250 00008D3F 47 inc di
51251 00008D40 E2FB inc di
51252 loop iset2
51253 ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
51254 ; 17/12/2022
51255 00008D42 83C70C add di,12 ; skip vectors 2Fh, 30h & 31h
51256 ;add di,8
51257 ;;;
51258
51259 ; 17/12/2022
51260 00008D45 B10E mov cl,14
51261 ;mov cx,14 ; set 14 offsets (skip 2 between each)
51262 ; sets offsets for ints 32h-3Fh
51263
51264 00008D47 AB iset3:
51265 stosw
51266 ;add di,2
51267 00008D48 47 ; 20/09/2023
51268 00008D49 47 inc di
51269 00008D4A E2FB inc di
51270 loop iset3
51271
51272 ;if installed
51273 ; set the offset of int2f handler
51274 00008D4C C706BC00[3A07] ;mov word [0BCh],INT2F
51275 ;mov word [02Fh*4],INT2F
51276 00008D52 36A1[A30A] ; set segment to doscode as we have to do int 2f to check for XMS
51277 ;mov ax,[ss:TEMP_DOSLOC] ; get segment of doscode
51278 00008D56 A3BE00 ;mov [0BEh],ax
51279 ;mov [(02Fh*4)+2],ax
51280 ;endif
51281 ; set up entry point call at vectors 30-31h. Note the segment of the
51282 ; long jump will be patched in by seg_reinit
51283
51284 00008D59 C606C000EA ;mov byte [C0h],0EAh
51285 ;mov byte [ENTRYPOINT],mi_long_jump
51286 00008D5E C706C100[CC02] ;mov byte [C1h],CALL_ENTRY
51287 ;mov word [ENTRYPOINT+1],CALL_ENTRY
51288 00008D64 C7068000[C502] mov word [addr_int_abort],QUIT ; INT 20h
51289 00008D6A C7068400[F102] mov word [addr_int_command],COMMAND ; INT 21h
51290 00008D70 C70688000001 mov word [addr_int_terminate],100h ; INT 22h
51291 00008D76 89168A00 mov word [addr_int_terminate+2],dx
51292 00008D7A C7069400[2D05] mov word [addr_int_disk_read],ABSDRD ; INT 25h
51293 00008D80 C7069800[DF05] mov word [addr_int_disk_write],ABSDWRT ; INT 26h
51294 00008D86 C7069C00[3E72] mov word [addr_int_keep_process],STAY_RESIDENT ; INT 27h
51295
51296 00008D8C 16 push ss
51297 00008D8D 1F pop ds
51298
51299 ; 24/05/2019
51300 ;push ss
51301 ;pop es
51302 ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
51303 ; 17/12/2022
51304 ;push ss
51305 ;pop es
51306
51307 00008D8E 52 push dx ; remember address of arena
51308
51309 00008D8F 42 inc dx ; leave room for arena header
51310 ;mov [330h],dx
51311 00008D90 8916[3003] mov [CurrentPDB],dx ; set current pdb
51312
51313 00008D94 31FF xor di,di ; point es:di at end of memory
51314 00008D96 8EC2 mov es,dx ; ...where psp will be
51315 00008D98 31C0 xor ax,ax
51316 ;mov cx,80h ; psp is 128 words
51317 ; 17/12/2022
51318 00008D9A B180 mov cl,128 ; 28/06/2019
51319 ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
51320 ;mov cx,128
51321
51322 00008D9C F3AB rep stosw ; zero out psp area

```



```

51323 00008D9E A1[4603]          mov     ax,[ENDMEM]
51324
51325          ; 17/12/2022
51326          ; cx = 0
51327 00008DA1 E82189          call    SETMEM          ; build psp at dx; ax is memory size
51328
51329          ; ds, es now point to PSP
51330
51331 00008DA4 16          push    ss
51332 00008DA5 1F          pop     ds
51333
51334          ;mov     di,24
51335 00008DA6 BF1800          mov     di,PDB.JFN_TABLE      ; es:di -> pdb_jfn_table in psp
51336 00008DA9 31C0          xor     ax,ax
51337 00008DAB AB          stosw
51338 00008DAC AA          stosb          ; 0,1 and 2 are con device
51339 00008DAD B0FF          mov     al,0FFh
51340          ;mov     cx,FILPERPROC-3 ; 17
51341          ; 17/12/2022
51342          ; cx = 4
51343 00008DAF B111          mov     cl,FILPERPROC-3 ; 17
51344 00008DB1 F3AA          rep     stosb          ; rest are unused
51345
51346 00008DB3 16          push    ss
51347 00008DB4 07          pop     es
51348          ; must be set to print messages
51349 00008DB5 8C1E[2C00]          mov     [SFT_ADDR+2],ds
51350
51351          ; after this point the char device functions for con will work for
51352          ; printing messages
51353
51354          ; 24/04/2019 - Retro DOS v4.0
51355
51356          ; 12/05/2019
51357
51358          ;write_version_msg:
51359          ;
51360          ;if     (not ibm)
51361          ;mov     si,offset doscode:header
51362          ;mov     si,HEADER
51363          ;outmes:
51364          ;lods     cs:byte ptr [si]
51365          ;cs
51366          ;lodsb
51367          ;cmp     al,"$"
51368          ;je     short outdone
51369          ;call    OUTT
51370          ;jmp     short outmes
51371          ;outdone:
51372          ;push     ss          ; out stomps on segments
51373          ;pop      ds
51374          ;push     ss
51375          ;pop      es
51376          ;endif
51377
51378          ; at this point es is dosdata
51379
51380          ; Fill in the segment addresses of sysinitvar and country_cdpq
51381          ; in sysinittable (ms_data.asm)
51382
51383          ;mov     si,0D28h
51384 00008DB9 BE[580D]          mov     si,SysInitTable
51385
51386          ; 17/12/2022
51387          ; ds = es = ss
51388
51389          ; 23/03/2024
51390          ;;;
51391          ; 17/12/2022
51392          ;; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
51393
51394          ;mov     [es:si+6],es
51395          ;mov     [es:si+SYSI_EXT.Country_Tab+2],es
51396          ;mov     [es:si+2],es
51397          ;mov     [es:si+SYSI_EXT.SysInitVars+2],es
51398
51399 00008DBC 8C4406          mov     [si+SYSI_EXT.Country_Tab+2],es
51400 00008DBF 8C4402          mov     [si+SYSI_EXT.SysInitVars+2],es
51401
51402          ; buffhead -> dosdata:hashinitvar
51403
51404          ;mov     [es:BUFFHEAD+2],es      ; BUGBUG - unused, remove this
51405 00008DC2 8C06[3A00]          mov     [BUFFHEAD+2],es
51406          ;mov     si,offset dosdata:hashinitvar ; and all other references
51407          ;mov     si,6Dh
51408 00008DC6 BE[6D00]          mov     si,HASHINITVAR
51409          ;mov     [es:BUFFHEAD],si
51410 00008DC9 8936[3800]          mov     [BUFFHEAD],si
51411
51412 00008DCD 5A          pop     dx          ; restore address of arena
51413
51414          ;mov     [032Ch+2],dx
51415 00008DCE 8916[2E03]          mov     [DMAADD+2],dx
51416
51417          ;mov     [es:arena_head],dx
51418 00008DD2 8916[2400]          mov     [arena_head],dx
51419          ;;;
51420
51421 00008DD6 8EDA          mov     ds,dx
51422
51423          ;mov     byte [0],'Z'
51424 00008DD8 C60600005A          mov     byte [ARENA.SIGNATURE],arena_signature_end
51425          ;mov     word [1],0
51426 00008DDD C70601000000          mov     word [ARENA.OWNER],arena_owner_system
51427
51428 00008DE3 36A1[4603]          mov     ax,[ss:ENDMEM]
51429 00008DE7 29D0          sub     ax,dx
51430 00008DE9 48          dec     ax
51431          ;mov     [3],ax ; 23/03/2024
51432 00008DEA A30300          mov     [ARENA.SIZE],ax
51433
51434          ; point to sft 0
51435
51436          ;mov     di,offset dosdata:sftabl + sftable
51437          ;mov     di,SFTABL+6
51438 00008DED BF[D200]          mov     di,SFTABL+SFT.SFTable
51439 00008DF0 B80300          mov     ax,3
51440 00008DF3 AB          stosw          ; adjust refcount
51441
51442          ; es:di is shared data area i.e., es:di -> dosdata:sysinttable
51443
51444          ;mov     di,offset dosdata:sysinittable
51445          ;mov     di,0D28h
51446          ;mov     di,0D58h          ; 23/03/2024 (PCDOS 7.1)

```

```

51447 00008DF4 BF[580D]      mov     di,SysInitTable
51448
51449 00008DF7 42             inc     dx             ; advance dx from arena to psp
51450 00008DF8 8EDA          mov     ds,dx           ; point ds to psp
51451
51452
51453
51454 00008DFA BA[688E]      mov     dx,seg_reinit
51455 00008DFD B9[BF87]      mov     cx,exepatch_start
51456 00008E00 81E9[0000]    sub     cx,_$STARTCODE      ; cx = (doscode - exepatch) - dosinit
51457
51458 00008E04 B8[E88A]      mov     ax,SYSBUF
51459 00008E07 2D[0000]      sub     ax,_$STARTCODE      ; ax = size of doscode - dosinit
51460
51461 00008E0A 368B26[8405]    mov     sp,[ss:USER_SP]      ; use ss override for next 2
51462 00008E0F 368E16[8605]    mov     ss,[ss:USER_SS]
51463
51464 00008E14 CB          retf
51465
51466
51467
51468
51469
51470
51471
51472
51473
51474
51475
51476
51477 00008E15 36C606[5A03]19    CHARINIT:
51478
51479 00008E1B 36C606[5B03]00    ; 11/04/2024
51480
51481 00008E21 36C606[5C03]00    ; 23/03/2024 - Retro DOS v5.0
51482
51483 00008E27 36C706[5D03]0000    ; 24/04/2019 - Retro DOS v4.0
51484 00008E2E 06             ; 07/07/2018 - Retro DOS v3.0
51485 00008E2F 53             ;mov     byte [ss:035Ah],26 ; 1Ah ; MSDOS 6.22
51486 00008E30 50             MOV BYTE [SS:DEVCALL_REQLEN],DINITHL ; 25 ; PCDOS 7.1
51487 00008E31 BB[5A03]      ;mov     byte [ss:035Bh],0
51488
51489 00008E34 16             MOV BYTE [SS:DEVCALL_REQUNIT],0
51490 00008E35 07             ;mov     byte [ss:035Ch],0
51491 00008E36 E85CC4        MOV BYTE [SS:DEVCALL_REQFUNC],DEVINIT
51492 00008E39 58             ;mov     word [ss:035BD],0
51493 00008E3A 5B             MOV WORD [SS:DEVCALL_REQSTAT],0
51494 00008E3B 07             PUSH     ES
51495 00008E3C C3             PUSH     BX
51496
51497
51498
51499
51500
51501
51502
51503
51504
51505
51506
51507
51508
51509
51510
51511
51512
51513 00008E3D 50             PUSH     SS ; 30/04/2019
51514
51515 00008E3E B80043        POP     ES
51516 00008E41 CD2F          CALL     DEVIOCALL2
51517
51518
51519
51520 00008E43 3C80          POP     AX
51521 00008E45 751D          POP     BX
51522
51523
51524
51525
51526 00008E47 53             POP     ES
51527 00008E48 52             RETN
51528 00008E49 1E             ; 25/04/2019 - Retro DOS v4.0
51529 00008E4A 06             ;
51530
51531 00008E4B B81043        ;
51532 00008E4E CD2F          ; check_XMM: routine to check presence of XMM driver
51533
51534
51535
51536 00008E50 2E8E1E[0700]    ;
51537
51538 00008E55 891E[7B10]      ; Exit: Sets up the XMM entry point in XMMcontrol in DOSDATA
51539 00008E59 8C06[7D10]      ;
51540
51541 00008E5D F8             ; USED: none
51542 00008E5E 07             ;
51543 00008E5F 1F             ;
51544 00008E60 5A             ;
51545 00008E61 5B             ;
51546 00008E62 58             ;
51547 00008E63 C3             ;
51548
51549
51550
51551
51552 00008E64 F9             check_XMM: ; proc near
51553 00008E65 58             ;
51554 00008E66 C3             ; determine whether or not an XMM driver is installed
51555
51556
51557
51558
51559
51560
51561
51562
51563
51564
51565
51566
51567
51568
51569
51570

```



```

51571 ;           else if AX = 1
51572 ;           patch segment of vectors with segment in ES
51573 ;
51574 ; NOTE           : This routine can be called at most twice!
51575 ;
51576 ; Regs Mod.   : es, ax, di, cx, bx
51577 ;-----
51578
51579 00008E67 00      num_entry: db      0           ; keeps track of the # of times this routine
51580 ;                                     ; has been called. (0 or 1)
51581
51582 ; 04/11/2022 - Retro DOS v4.0 (ref: MSDOS 5.0)
51583 ; MSDOS 5.0 MSDOS.SYS - DOSCODE:0BAB7h
51584 ; 25/05/2019 - Retro DOS v4.0 (ref: MSDOS 6.21)
51585 ; MSDOS 6.21 MSDOS.SYS - DOSCODE:0BDA5h
51586
51587 ; 23/03/2024 - Retro DOS v5.0 (ref: PC DOS 7.1)
51588 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:0D07Eh
51589
51590 seg_reinit: proc far
51591 00008E68 1E      push     ds
51592
51593 00008E69 2E8E1E[0700] mov     ds,[cs:DosDSeg]
51594
51595 00008E6E E89300   call     patch_misc_segments ; patch in segments for sharer and
51596 ;                                     ; other tables with seg in ES.
51597 ; 17/12/2022
51598 ; cx = 0
51599 00008E71 39C8     cmp      ax,cx ; 0
51600 ; cmp     ax,0
51601 00008E73 754A     jne      short patch_vec_seg ; patch vectors with segment in es
51602 ; 23/03/2024
51603 ; or      ax, ax
51604 ; jnz     short patch_vec_seg
51605
51606 ; 23/03/2024
51607 ; cmp     [cs:num_entry],al ; 0
51608 ; 17/12/2022
51609 00008E75 2E380E[678E] cmp     [cs:num_entry],cl ; 0
51610 ; cmp     byte [cs:num_entry],0 ; Q: is it the first call to this
51611 00008E7A 7508     jne      short second_entry ; N: just patch the stub with
51612 ;                                     ; segment in ES
51613 ;                                     ; Y: patch the vectors with stub
51614 00008E7C 8CD8     mov      ax,ds
51615 00008E7E E84B00   call     patch_vec_segments ; patch the segment of vectors
51616 00008E81 E8CA00   call     patch_offset       ; patch the offsets of vectors
51617 ;                                     ; with those in the stub.
51618 ; 17/12/2022
51619 ; cx = 0
51620
51621 00008E84 8CC0     second_entry: mov     ax,es ; patch the stub with segment in es
51622
51623 ; mov     di,OFFSET DOSDATA:DOSINTTABLE
51624 ; ; mov    di,1062h ; (same table addr for MSDOS 5.0 and MSDOS 6.21)
51625 ; mov     di,0F7Ah ; 23/03/2024 ; PC DOS 7.1
51626 00008E86 BF[7A0F] mov     di,DOSINTTABLE
51627
51628 ; 17/12/2022
51629 ; cx = 0
51630 ; ; mov    cx,9
51631 ; mov     cl,9
51632 ; 23/03/2024 - Retro DOS v5.0 (PC DOS 7.1 IBMDOS.COM)
51633 ; ; ;
51634 ; mov     cx,8
51635 00008E89 B108     mov     cl,8
51636 ; ; ;
51637 00008E8B 1E      push     ds
51638 00008E8C 07      pop      es ; es:di -> DOSINTTABLE
51639
51640 dosinttabloop:
51641 ; add     di,2
51642 ; ; 19/06/2023
51643 00008E8D 47      inc     di
51644 00008E8E 47      inc     di
51645 00008E8F AB      stosw
51646 00008E90 E2FB     loop    dosinttabloop
51647
51648 ; For ROMDOS, this routine will only be called when the DOS wants to
51649 ; use the HMA, so we don't want to check CS
51650
51651 ; ifndef ROMDOS
51652 00008E92 3D00F0   cmp     ax,0F000h ; Q: is the DOS running in the HMA
51653 00008E95 722D     jb      short sr_done ; N: done
51654 ; endif
51655 00008E97 E8A3FF   call     check_XMM ; Y: set up the XMS entry point
51656 00008E9A 7228     jc      short sr_done ; failed to set up XMS do not do
51657 ;                                     ; A20 toggling in the stub.
51658 ; 17/12/2022
51659 ; cx = 0
51660 00008E9C E82A01   call     patch_in_nops ; enable the stub to check A20 state
51661 ; M021-
51662 ; ; mov    byte [1211h],1
51663 ; ; mov    byte [0D66h],1 ; 23/03/2024 ; PC DOS 7.1
51664 00008E9F C606[660D]01 mov     byte [DoshHashMA],1 ; set flag telling DOS control of HMA
51665
51666 ;                                     ; set pointer to the routine that
51667 ;                                     ; patches buggy exepacked code.
51668 ; mov     [FixExePatch],offset DOSCODE:ExePatch
51669 00008EA4 C706[670D]0F89 mov     word [FixExePatch],ExePatch
51670 ;                                     ; M068: set pointer to the routine
51671 ;                                     ; M068: that detects copy protected
51672 ;                                     ; M068: apps
51673 ; mov     [ChkCopyProt],offset DOSCODE:IsCopyProt
51674 00008EAA C706[6100]B28A mov     word [ChkCopyProt],IsCopyProt
51675
51676 ; 23/03/2024 - PC DOS 7.1 IBMDOS.COM - DOSCODE:0D0C7h
51677 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0BDF1h)
51678 00008EB0 E844FC   call     whatCPUtype
51679 00008EB3 3C01     cmp     al,1
51680 00008EB5 750D     jne     short sr_done ; we need Rational Patch only on 286 systems
51681 00008EB7 C706[690D]398A mov     word [RationalPatchPtr],RationalPatch
51682
51683 ; 19/09/2023
51684 00008EBD EB05     jmp     short sr_done
51685
51686 ; 23/03/2024
51687 patch_vec_seg: ; patch vectors with segment in es
51688 mov     ax,es
51689 00008EC1 E80800   call     patch_vec_segments ; patch in DOSCODE for the segments
51690 ;                                     ; NOTE we don't have to patch the
51691 ;                                     ; offsets as they have been already
51692 ;                                     ; set to the doscode offsets at
51693 ;                                     ; DOSINIT.
51694 sr_done:

```

```

51695 00008EC4 2EC606[678E]01      mov     byte [cs:num_entry],1
51696 00008ECA 1F                  pop     ds
51697 00008ECB CB                  retf    ; ! far return !
51698
51699
51700
51701
51702
51703
51704
51705
51706
51707
51708
51709
51710
51711
51712
51713
51714
51715
51716 00008ECC 06                  push    es
51717
51718 00008ECD 31C9                  xor     cx,cx ; 0
51719 00008ECF 8EC1                  mov     es,cx
51720
51721
51722 00008ED1 BF8200                  ;mov    di,82h
51723                                     mov     di,INTBASE+2          ; di -> segment of int 20h vector
51724 00008ED4 26A30200                  mov     [es:2],ax          ; segment of default divide trap handler
51725                                     ; set vectors 20h & 21h
51726                                     ; 04/11/2022
51727                                     ;mov     cx,2
51728                                     ; 17/12/2022
51729                                     ;mov     cl,2
51730
51731
51732 00008ED8 AB                  ps_set1:
51733                                     stosw   ; int 20h segment
51734                                     ;add     di,2
51735                                     ; 17/12/2022
51736 00008ED9 47                  inc     di
51737 00008EDA 47                  inc     di
51738                                     ;loop    ps_set1
51739
51740 00008EDB AB                  ; 17/12/2022
51741                                     stosw   ; int 21h segment
51742                                     ;inc     di
51743                                     ;inc     di
51744
51745 00008EDC 83C706                  ;add     di,4
51746                                     add     di,6 ; *          ; skip int 22h vector
51747 00008EDF AB                  stosw
51748 00008EE0 83C706                  add     di,6          ; set int 23h
51749                                     ; skip int 24h
51750                                     ; set vectors 25h-28h and 2Ah-3Fh
51751                                     ; 04/11/2022
51752                                     ;mov     cx,4
51753                                     ; 17/12/2022
51754 00008EE3 B104                  mov     cl,4          ; set 4 segments
51755
51756 00008EE5 AB                  ps_set2:
51757                                     stosw
51758                                     ;add     di,2
51759                                     ; 17/12/2022
51760 00008EE6 47                  inc     di
51761 00008EE7 47                  inc     di
51762 00008EE8 E2FB                  loop    ps_set2
51763 00008EEA 83C704                  add     di,4
51764                                     ; skip int 29h vector (fast con) as it may
51765                                     ; already be set.
51766                                     ; 04/11/2022
51767                                     ;mov     cx,6
51768 00008EED B106                  ; 17/12/2022
51769                                     mov     cl,6          ; set segs for ints 2Ah-2Fh
51770 00008EEF AB                  ps_set3:
51771                                     stosw
51772                                     ;add     di,2
51773 00008EF0 47                  inc     di
51774 00008EF1 47                  inc     di
51775 00008EF2 E2FB                  loop    ps_set3
51776
51777                                     ; 30h & 31h is the CPM call entry point whose segment address is set up by
51778                                     ; below. So skip it.
51779
51780 00008EF4 83C708                  add     di,8          ; skip vector 30h & 31h
51781
51782                                     ; 04/11/2022
51783                                     ;mov     cx,14
51784                                     ; 17/12/2022
51785 00008EF7 B10E                  mov     cl,14          ; sets segs for ints 32h-3Fh
51786
51787 00008EF9 AB                  ps_set4:
51788                                     stosw
51789                                     ;add     di,2
51790 00008EFA 47                  inc     di
51791 00008EFB 47                  inc     di
51792 00008EFC E2FB                  loop    ps_set4
51793
51794                                     ; set offset of int2f
51795
51796                                     ;if installed
51797                                     ; mov     word ptr es:[02fh * 4],offset doscode:int2f
51798                                     ;endif
51799                                     ;mov     [es:0C3h],ax
51800 00008EFE 26A3C300                  mov     [es:ENTRYPOINT+3],ax
51801                                     ; 17/12/2022
51802                                     ; cx = 0
51803 00008F02 07                  pop     es
51804 00008F03 C3                  retn
51805
51806
51807
51808
51809
51810
51811
51812
51813
51814
51815
51816
51817
51818

```

```

;
;
; Procedure Name : patch_misc_segments
;
; Inputs      : es = segment to patch in
;              ds = dosdata
;
; outputs     : patches in the sharer and other tables in the dos
;              with right dos code segment in es
;
; Regs Mod   : DI,SI,CX
;
;
;

```

```

51819
51820
51821
51822 00008F04 53
51823 00008F05 06
51824 00008F06 50
51825
51826 00008F07 8CC0
51827
51828 00008F09 1E
51829 00008F0A 07
51830
51831
51832
51833
51834
51835 00008F0B BF[9000]
51836
51837
51838 00008F0E 8B1E[A30A]
51839
51840
51841 00008F12 B9F00
51842
51843
51844
51845 00008F15 47
51846 00008F16 47
51847 00008F17 83FBFF
51848 00008F1A 7405
51849
51850 00008F1C 263B1D
51851 00008F1F 7501
51852
51853 00008F21 AB
51854
51855 00008F22 E2F1
51856
51857
51858
51859
51860
51861
51862
51863
51864 00008F24 BE[2A12]
51865
51866
51867
51868
51869
51870
51871
51872
51873 00008F27 8C5C4F
51874 00008F2A 8C5C54
51875 00008F2D 8C5C59
51876 00008F30 8C5C5E
51877 00008F33 8C9C8000
51878 00008F37 8C5C63
51879
51880
51881
51882
51883
51884
51885 00008F3A BE[3C11]
51886
51887
51888
51889 00008F3D 83FBFF
51890
51891 00008F40 7405
51892
51893
51894
51895
51896 00008F42 3B5C04
51897
51898
51899 00008F45 7503
51900
51901
51902
51903 00008F47 894404
51904
51905
51906
51907 00008F4A 58
51908 00008F4B 07
51909 00008F4C 5B
51910
51911 00008F4D C3
51912
51913
51914
51915
51916
51917
51918
51919
51920
51921
51922
51923
51924
51925
51926
51927 00008F4E 06
51928
51929 00008F4F 31C0
51930 00008F51 8EC0
51931
51932
51933
51934
51935 00008F53 26C7060000[9E0F]
51936
51937
51938 00008F5A BF8000
51939
51940
51941 00008F5D B8[6E10]
51942

```

```

patch_misc_segments:
    push    bx
    push    es
    push    ax

    mov     ax,es                ; ax -> DOS segment

    push    ds
    pop     es                  ; es -> DOSDATA

; initialize the jump table for the sharer...
    ;mov     di,offset dosdata:jshare
    ;mov     di,90h
    mov     di,jshare
    ;mov     bx,[0AAAh]
    ;mov     bx,[0AA3h] ; 23/03/2024 ; PC DOS 7.1
    mov     bx,[TEMP_DOSLOC]    ; bx = location to which the share
                                ; table was patched during the first
                                ; call to this routine

    mov     cx,15
jumptableloop:
    add     di,2                ; skip offset
    ; 17/12/2022
    inc     di
    inc     di
    cmp     bx,-1 ; 0FFFFh      ; Q: is this called for the 1st time
    je      short share_patch   ; Y: patch in sharer table
                                ; N:
                                ; Q: has share been installed
    cmp     bx,[es:di]          ; Y: don't patch in sharer table
    jne     short no_share_patch

share_patch:
    stosw                        ; drop in segment
no_share_patch:
    loop    jumtableloop

                                ; BUGBUG patching the country info
                                ; with dosdata can be done inline
                                ; in dosinit.
                                ; for dos 3.3 country info
                                ; table address

    ;mov     si,offset dosdata:country_cdpq
    ;mov     si,122Ah
    mov     si,COUNTRY_CDPG

                                ; initialize double word
                                ; pointers with dosdata in ds

    ;mov     [si+4Fh],ds
    ;mov     [si+54h],ds
    ;mov     [si+59h],ds
    ;mov     [si+5Eh],ds
    ;mov     [si+80h],ds
    ;mov     [si+63h],ds
    mov     [si+DOS_CCDPG.ccUcase_ptr+2],ds
    mov     [si+DOS_CCDPG.ccFileUcase_ptr+2],ds
    mov     [si+DOS_CCDPG.ccFileChar_ptr+2],ds
    mov     [si+DOS_CCDPG.ccCollate_ptr+2],ds
    mov     [si+DOS_CCDPG.ccMono_ptr+2],ds
    mov     [si+DOS_CCDPG.ccBCS_ptr+2],ds

                                ; fastopen routines are in doscode
                                ; so patch with doscode seg in ax

    ;mov     si,offset dosdata:fastopentable
    ;mov     si,0D30h
    mov     si,FastOpenTable

    ; 17/12/2022
    ; bx = [TEMP_DOSLOC]
    cmp     bx,-1
    ;cmp     word [TEMP_DOSLOC],-1 ; Q: first time
    je      short fast_patch    ; Y: patch segment
    ;mov     cx,[TEMP_DOSLOC]

                                ; Q: has fastopen patched in it's
                                ; segment

    ; 17/12/2022
    cmp     bx,[si+fastopen_entry.name_caching+2]
    ;cmp     cx,[si+4]
    ;cmp     cx,[si+fastopen_entry.name_caching+2]
    jne     short no_fast_patch ; Y: don't patch in doscode seg

fast_patch:
    ;mov     [si+4],ax
    mov     [si+fastopen_entry.name_caching+2],ax
no_fast_patch:
    ; 17/12/2022
    ; cx = 0
    pop     ax
    pop     es
    pop     bx

    retn

;-----
;
; Procedure Name : patch_offset
;
; Inputs      : NONE
;
; Outputs     : Patches in the offsets in the low_mem_stub for vectors
;               0,20-28,3a-3f, and 30,31
;
;
;
; Regs. Mod  : AX,DI,CX
;-----

patch_offset:
    push    es                ; preserve es

    xor     ax,ax
    mov     es,ax

                                ; set default divide trap address
    ;mov     word ptr es:[0],offset dosdata:ldivov
    ;mov     word [es:0],108Ah
    ;mov     word [es:0],0F9Eh ; 23/03/2024 ; PC DOS 7.1
    mov     word [es:0],ldivov

    ;mov     di,80h
    mov     di,INTBASE        ; di-> offset of int 20 handler
    ;mov     ax,offset dosdata:lirett
    ;mov     ax,10DAh
    mov     ax,lirett

                                ; set vectors 20h & 21h to point to iret.

```

```

51943             ; 17/12/2022
51944             ; cx = 0
51945
51946             ;mov    cx,2             ; set 2 offsets (skip 2 between each)
51947 po_iset1:
51948 00008F60 AB    stosw    ; int 20h offset
51949             ;add    di,2 ; *
51950             ;loop   po_iset1
51951             ; 17/12/2022
51952 00008F61 47    inc     di
51953 00008F62 47    inc     di
51954 00008F63 AB    stosw    ; int 21h offset
51955
51956             ;add    di,4             ; skip vector 22h
51957             ; 17/12/2022
51958 00008F64 83C706 add     di,6 ; *
51959
51960 00008F67 AB    stosw    ; set offset of 23h
51961             ;add    di,6             ; skip 24h
51962             ; 19/09/2023
51963 00008F68 83C712 add     di,18 ; skip 23h segment and int 24-25-26-27h
51964
51965             ; set vectors 25-28 and 2a-3f to iret.
51966             ; 04/11/2022
51967             ;mov    cx,4             ; set 4 offsets (skip 2 between each)
51968             ; 19/09/2023
51969             ; 17/12/2022
51970             ;mov    cl,4             ; sets offsets for ints 25h-28h
51971 po_iset2:
51972 00008F6B AB    stosw    ; set offset for int 28h ; 19/09/2023
51973             ;add    di,2
51974             ; 19/09/2023
51975             ; 17/12/2022
51976             ;inc     di
51977             ;inc     di
51978             ; 19/09/2023
51979             ;loop   po_iset2
51980
51981             ;add    di,4             ; skip vector 29h
51982             ; 19/09/2023
51983 00008F6C 83C706 add     di,6 ; skip int 28h segment and int 29h ; 19/09/2023
51984
51985             ; 04/11/2022
51986             ;mov    cx,6             ; set 6 offsets (skip 2 between each)
51987             ; 17/12/2022
51988             ;mov    cl,6             ; sets offsets for ints 2Ah-2Fh
51989 00008F6F B105    mov     cl,5             ; sets offsets for ints 2Ah-2Eh
51990 po_iset3:
51991 00008F71 AB    stosw
51992             ;add    di,2
51993             ; 17/12/2022
51994 00008F72 47    inc     di
51995 00008F73 47    inc     di
51996 00008F74 E2FB    loop    po_iset3
51997
51998             ; 30h & 31h is the CPM call entry point whose offset address is set up by
51999             ; below. So skip it.
52000
52001             ;add    di,8             ; skip vector 30h & 31h
52002             ; 17/12/2022
52003 00008F76 83C70C add     di,12 ; skip vector 2Fh, 30h & 31h
52004
52005             ; 04/11/2022
52006             ;mov    cx,14            ; set 14 offsets (skip 2 between each)
52007             ; 17/12/2022            ; sets offsets for ints 32h-3Fh
52008             ;mov    cl,14 ; 26/06/2019
52009 00008F79 B10E    mov     cl,14 ; 26/06/2019
52010 po_iset4:
52011 00008F7B AB    stosw
52012             ;add    di,2
52013             ; 17/12/2022
52014 00008F7C 47    inc     di
52015 00008F7D 47    inc     di
52016 00008F7E E2FB    loop    po_iset4
52017
52018             ;if installed
52019             ;mov     word ptr es:[02fh * 4],offset dosdata:lint2f
52020             ;mov     word [es:0BCh],10C6h ; (MSDOS 5.0 & 6.21)
52021             ;mov     word [es:0BCh],0FDAh ; PCDOS 7.1 ; 23/03/2024
52022 00008F80 26C706BC00[DA0F] mov     word [es:(2Fh*4)],lint2f
52023             ;endif
52024
52025             ; set up entry point call at vectors 30-31h
52026             ;mov     byte [es:0C0h],0EAh
52027 00008F87 26C606C000EA mov     byte [es:ENTRYPOINT],mi_long_jump
52028             ;mov     word [es:0C1h],10D0h ;; 0FE4h ; 23/03/2024 (PCDOS 7.1)
52029
52030 00008F8D 26C706C100[E40F] mov     word [es:ENTRYPOINT+1],lcall_entry
52031
52032             ; 19/09/2023
52033             ;mov     word [es:80h],1094h ;; 0FA8h ; 23/03/2024
52034 00008F94 26C7068000[A80F] mov     word [es:addr_int_abort],lquit ; int 20h
52035             ;mov     word [es:84h],109Eh ;; 0FB2h
52036 00008F9B 26C7068400[B20F] mov     word [es:addr_int_command],lcommand ; int 21h
52037             ;mov     word [es:94h],10A8h ;; 0FBCh
52038 00008FA2 26C7069400[BC0F] mov     word [es:addr_int_disk_read],labsdrd ; int 25h
52039             ;mov     word [es:98h],10B2h ;; 0FC6h
52040 00008FA9 26C7069800[C60F] mov     word [es:addr_int_disk_write],labsdwrw ; int 26h
52041             ;mov     word [es:9Ch],10BCh ;; 0FD0h
52042 00008FB0 26C7069C00[D00F] mov     word [es:addr_int_keep_process],lstay_resident ; int 27h
52043
52044             ; 17/12/2022
52045             ; CX = 0
52046 00008FB7 07      pop     es             ; restore es
52047 00008FB8 C3      retn
52048
52049 -----
52050             ;
52051             ; Procedure Name :      patch_in_nops
52052             ;
52053             ; Entry          :      ES -> DOSDATA
52054             ;
52055             ; Regs Mod       :      cx, di
52056             ;
52057             ; Description:
52058             ; This routine patches in 2 nops at the offsets specified in
52059             ; patch_table. This basically enables the low mem stub to start
52060             ; making XMS calls.
52061             ;
52062             ; -----
52063
52064             ; 04/11/2022
52065             ; MSDOS 5.0 MSDOS.SYS - DOSCODE:0BC50h
52066

```

```

52067             ; 23/03/2024
52068             ; MSDOS 6.22 MSDOS.SYS - DOSCODE:0BF42h
52069             ; 23/03/2024 - Retro DOS v5.0
52070             ; PC DOS 7.1 IBMDOS.COM - DOSCODE:0D1E9h
52071
52072 patch_table:      ; label byte
52073             ;dw offset dosdata:i0patch
52074             ;dw offset dosdata:i20patch
52075             ;dw offset dosdata:i21patch
52076             ;dw offset dosdata:i25patch
52077             ;dw offset dosdata:i26patch
52078             ;dw offset dosdata:i27patch
52079             ;dw offset dosdata:i2fpatch
52080             ;dw offset dosdata:cpmpatch
52081 00008FB9 [9E0F] dw i0patch
52082 00008FBB [A80F] dw i20patch
52083 00008FBD [B20F] dw i21patch
52084 00008FBF [BC0F] dw i25patch
52085 00008FC1 [C60F] dw i26patch
52086 00008FC3 [D00F] dw i27patch
52087 00008FC5 [DA0F] dw i2fpatch
52088 00008FC7 [E40F] dw cpmpatch
52089
52090 patch_table_size equ ($-patch_table)/2
52091
52092 patch_in_nops:
52093     push ax
52094     push si
52095     mov si,patch_table
52096     mov ax,9090h ; nop, nop
52097     ; 17/12/2022
52098     ; cx = 0
52099     ;mov cx,8
52100     ;mov cx,patch_table_size ; 8
52101     ;mov cl,patch_table_size ; 8
52102 pin_loop:
52103     mov di,[cs:si]
52104     stosw
52105     ;add si,2
52106     ; 17/12/2022
52107     inc si
52108     inc si
52109     loop pin_loop
52110     pop si
52111     pop ax
52112     retn
52113
52114 ; 05/12/2022 (MSDOS 5.0 MSDOS.SYS compatibility)
52115 ; -----
52116 ; MSDOS 5.0 - MSDOS.SYS offset BC77h, file offset 7EA7h
52117 ; -----
52118 ; 23/03/2024
52119 ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0BF69h)
52120 ; PC DOS 7.1 IBMDOS.COM - DOSCODE:0D20Fh
52121
52122 ; 05/12/2022 - temporary ; (paragraph alinment)
52123 DOSCODE_END:
52124
52125     ; 23/03/2024
52126     ;;times 9 db 0 ; db 9 dup(0)
52127     ;; 18/12/2022
52128     ;dw 0 ; times 2 db 0
52129
52130     ; 23/03/2024
52131     ;times 7 db 0 ; MSDOS 6.22 MSDOS.SYS
52132
52133     ; 23/03/2024 - Retro DOS v5.0
52134     ;db 0 ; PC DOS 7.1 IBMDOS.COM
52135     ; 01/07/2024
52136     ;times 12 db 0
52137
52138     ; 01/07/2024 - Retro DOS v5.0
52139     ; PC DOS 7.1 IBMDOS.COM - DOSCODE:0D0Fh
52140 00008FDE 00 db 0
52141
52142 ;align 16
52143     ; DOSCODE:BC80h (MSDOS 5.0 MSDOS.SYS file offset 7EB0h)
52144     ; MSDOS.SYS file offset: 32432 (start of DOSDATA)
52145
52146     ; 23/03/2024
52147     ; PC DOS 7.1 IBMDOS.COM - DOSCODE:0D210h
52148     ; (MSDOS 6.22 MSDOS.SYS - DOSCODE:0BF70h)
52149
52150 ; -----
52151
52152 ;memstrt label word
52153 ; -----
52154 ; MSDOS 6.21 - MSDOS.SYS offset BF69h, file offset 8189h
52155 ; -----
52156
52157 MEMSTRT: ; 25/04/2019 - Retro DOS v4.0
52158
52159 ; if not ROMDOS, then we close the dos code segment, otherwise we close
52160 ; the dos initialization segment
52161
52162 ;ifndef ROMDOS
52163
52164 ;doscode ends
52165
52166 ;else
52167
52168 ;;dosinitseg ends
52169
52170 ;endif ; ROMDOS
52171
52172 ;=====
52173
52174 ; DPUBLIC <ParaRound, cXMM_no_driver, cXMMexit, char_init_loop, charinit>
52175 ; DPUBLIC <check_XMM, continit, dosinttabloop, endlst>
52176 ; DPUBLIC <initiret, iset1, iset2, jumptabloop, nxtentry>
52177 ; DPUBLIC <notmax, patch_offset, perdrv>
52178 ; DPUBLIC <perunit, po_iset1, po_iset2, po_iset3>
52179 ; DPUBLIC <ps_set1, ps_set2, ps_set3, seg_reinit>
52180 ; DPUBLIC <sr_done, version_fake_table, xxx>
52181
52182 ;; burası doscode sonu
52183
52184 ;=====
52185 ; DOSDATA
52186 ;=====
52187 ; 29/04/2019 - Retro DOS 4.0
52188 ; 24/03/2024 - Retro DOS 5.0
52189
52190 ;[BITS 16]

```

```

52191 ;[ORG 0]
52192
52193 ; 25/04/2019 - Retro DOS v4.0
52194
52195 ;=====
52196 ; DOSDATA - MSDOS 6.21 - MSDOS.SYS Offset 0BF70h, file offset 8190h
52197 ;=====
52198
52199 ;align 16
52200 ; DOSDATA (MSDOS.SYS kernel DATA) segment starts here...
52201 ; (4970 bytes for MSDOS 6.21)
52202 ; (4976 bytes for Retro DOS v4.0, 25/05/2019 modification.)
52203
52204 ;=====
52205 ; MCONST.ASM (MSDOS 6.0, 1991)
52206 ;=====
52207 ; 03/11/2022 - Retro DOS 4.0 (Modified MSDOS 5.0 MSDOS.SYS)
52208 ; 25/04/2019 - Retro DOS 4.0 (MSDOS 6.21)
52209 ; 16/07/2018 - Retro DOS 3.0
52210
52211 ;Break <Initialized data and data used at DOS initialization>
52212 ;-----
52213
52214 ; We need to identify the parts of the data area that are relevant to tasks
52215 ; and those that are relevant to the system as a whole. Under 3.0, the system
52216 ; data will be gathered with the system code. The process data under 2.x will
52217 ; be available for swapping and under 3.0 it will be allocated per-process.
52218 ;
52219 ; The data that is system data will be identified by [SYSTEM] in the comments
52220 ; describing that data item.
52221
52222 ;DOSDATA SEGMENT
52223
52224 ; 04/11/2022
52225 ;[ORG 0]
52226
52227 ; -----
52228 ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
52229 ; -----
52230 ; DOSDATA segment start offset from beginning of MSDOS.SYS file: 32432 (7EB0h)
52231 ; (3DD0h+7EB0h = 0BC80h) - for MSDOS 5.0 kernel file -
52232 ; -----
52233
52234 ; 04/11/2022
52235
52236 ;DOSDATA:0000h
52237
52238 align 16
52239 00008FDF 90
52240
52241 ; -----
52242 ; 06/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
52243 ; -----
52244
52245 segment .data vstart=0 ; 06/12/2022
52246
52247 ; =====
52248
52249 ; 06/12/2022
52250 ;DOSDATASTART equ $
52251 DOSDATASTART:
52252
52253 ;hkn; add 4 bytes to get correct offsets since jmp has been removed in START
52254
52255 ;; 03/11/2022
52256 ;jmp DOSINIT ; MSDOS 5.0 - MSDOS.SYS (DOSDATA:0000h)
52257
52258 ; 04/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
52259 ;db 4 dup (?)
52260 00000000 00<rep 4h> times 4 db 0
52261
52262 ; 29/04/2019 - Retro DOS v4.0 modification
52263 ;dw _$STARTCODE ; DOSCODE offset and/or size of DOSDATA
52264 ;dw 0
52265
52266 ;EVEN
52267
52268 ;align 2
52269
52270 ; WANGO!!! The following word is used by SHARE and REDIR to determin data
52271 ; area compatability. This location must be incremented EACH TIME the data
52272 ; area here gets mucked with.
52273 ;
52274 ; Also, do NOT change this position relative to DOSDATA:0.
52275
52276 MSCT001S: ; LABEL BYTE
52277
52278 DataVersion:
52279 00000004 0100 dw 1 ;AC000; [SYSTEM] version number for DOS DATA
52280
52281 ;hkn; add 8 bytes to get correct offsets since BugTyp, BugLev and "BUG " has
52282 ;hkn; been removed to DOSCODE above
52283
52284 ;M044
52285 ; First part of save area for saving last para of window memory
52286
52287 winoldPatch1: ; db 8 dup (?) ;M044
52288 00000006 00<rep 8h> times 8 db 0
52289
52290 ; MSDOS 6.21 DOSDATA:000Eh
52291 MYNUM: ; Offset 000Eh
52292 0000000E 0000 dw 0 ; [SYSTEM] A number that goes with MYNAME
52293 FCBLRU: ; [SYSTEM] LRU count for FCB cache
52294 00000010 0000 dw 0
52295 OpenLRU:
52296 00000012 0000 dw 0 ; [SYSTEM] LRU count for FCB cache opens
52297 OEM_HANDLER:
52298 00000014 FFFFFFFF dd -1 ; [SYSTEM] Pointer to OEM handler code
52299
52300 ; BUGBUG - who uses LeaveAddr? what if we want to rework the
52301 ;; way that we leave DOS???? - jgl
52302
52303 LeaveAddr:
52304 00000018 [F603] dw LeaveDOS ; <<OFFSET DOSCODE:LeaveDOS>> ; [SYSTEM]
52305 RetryCount:
52306 0000001A 0300 dw 3 ; [SYSTEM] Share retries
52307 RetryLoop:
52308 0000001C 0100 dw 1 ; [SYSTEM] Share retries
52309 LastBuffer:
52310 0000001E FFFFFFFF dd -1 ; [SYSTEM] Buffer queue recency pointer
52311 CONTPOS:
52312 00000022 0000 dw 0 ; [SYSTEM] location in buffer of next read
52313 arena_head:
52314 00000024 0000 dw 0 ; [SYSTEM] Segment # of first arena in memory

```



```

52315 ; 16/07/2018
52316 ; *****
52317 ; NOTE: INT 21H AH=52H ! (http://stanislavs.org/helppc/int_21-52.html)
52318 ; *****
52319 ; INT 21,52 - Get Pointer to DOS "INVARs" (Undocumented)
52320 ;
52321 ;
52322 ; AH = 52h
52323 ;
52324 ; on return:
52325 ; ES:BX = pointer to DOS "invars", a table of pointers used by DOS.
52326 ; Known "invars" fields follow (varies with DOS version):
52327 ;
52328 ; Offset Size Description
52329 ;
52330 ; -12 word sharing retry count (DOS 3.1-3.3)
52331 ; -10 word sharing retry delay (DOS 3.1-3.3)
52332 ; -8 dword pointer to current disk buffer (DOS 3.x)
52333 ; -4 word pointer in DOS code segment of unread CON input;
52334 ; 0 indicates no unread input (DOS 3.x)
52335 ; -2 word segment of first Memory Control Block (MCB)
52336 ; 00 dword pointer to first DRIVE PARAMETER TABLE (A:) in chain
52337 ; 04 dword pointer to DOS System File Table (SFT)
52338 ; 08 dword pointer to $CLOCK device driver
52339 ; 0C dword pointer to CON device driver
52340 ; 10 byte number of logical drives in system
52341 ; 11 word maximum bytes/block of any block device
52342 ; 13 dword pointer to DOS cache buffer header
52343 ; 17 18bytes NUL device header, first 4 bytes of device header
52344 ; point to the next device in device chain
52345 ;
52346 ; *****
52347 ;
52348 ; The following block of data is used by SYSINIT.
52349 ; Do not change the order or size of this block
52350 ;
52351 ;SYSINITVAR:
52352 ;-----
52353 ;SYSINITVARS:
52354 ;DPBHEAD:
52355 ; dd 0 ; [SYSTEM] Pointer to head of DPB-FAT list
52356 ;SFT_ADDR:
52357 ; dd SFTABL ; [SYSTEM] Pointer to first SFT table
52358 ;BCLOCK:
52359 ; dd 0 ; [SYSTEM] The CLOCK device
52360 ;BCON:
52361 ; dd 0 ; [SYSTEM] Console device entry points
52362 ;MAXSEC:
52363 ; dw 128 ; [SYSTEM] Maximum allowed sector size
52364 ;BUFFHEAD:
52365 ; dd 0 ; [SYSTEM] Pointer to head of buffer queue
52366 ;CDSADDR:
52367 ; dd 0 ; [SYSTEM] Pointer to curdir structure table
52368 ;SFTFCB:
52369 ; dd 0 ; [SYSTEM] pointer to FCB cache table
52370 ;KEEPCOUNT:
52371 ; dw 0 ; [SYSTEM] count of FCB opens to keep
52372 ;NUMIO:
52373 ; db 0 ; [SYSTEM] Number of disk tables
52374 ;CDSCOUNT:
52375 ; db 0 ; [SYSTEM] Number of CDS structures in above
52376 ;
52377 ; A fake header for the NUL device
52378 ;NULDEV:
52379 ; dd 0 ; [SYSTEM] Link to rest of device list
52380 ; dw 8004h
52381 ; dw DEVTYP|ISNULL ; [SYSTEM] Null device attributes
52382 ; dw SNULDEV ; [SYSTEM] Strategy entry point
52383 ; dw INULDEV ; [SYSTEM] Interrupt entry point
52384 ; db "NUL " ; [SYSTEM] Name of null device
52385 ;SPLICES:
52386 ; db 0 ; [SYSTEM] TRUE => splices being done
52387 ;
52388 ;Special_Entries:
52389 ; dw 0 ; [SYSTEM] address of special entries ;AN000;
52390 ;UU_IFS_DOS_CALL:
52391 ; dd 0 ; [SYSTEM] entry for IFS DOS service ;AN000;
52392 ;
52393 ; UU_IFS_HEADER:
52394 ; dd 0 ; [SYSTEM] IFS header chain ;AN000;
52395 ;
52396 ;ChkCopyProt:
52397 ; dw 0 ; M068
52398 ;A20OFF_PSP:
52399 ; dw 0 ; M068
52400 ;BUFFERS_PARM1:
52401 ; dw 0 ; [SYSTEM] value of BUFFERS= ,m ;AN000;
52402 ;BUFFERS_PARM2:
52403 ; dw 0 ; [SYSTEM] value of BUFFERS= ,n ;AN000;
52404 ;BOOTDRIVE:
52405 ; db 0 ; [SYSTEM] the boot drive ;AN000;
52406 ;DDMOVE:
52407 ; db 0 ; [SYSTEM] 1 if we need DWORD move ;AN000;
52408 ;EXT_MEM_SIZE:
52409 ; dw 0 ; [SYSTEM] extended memory size ;AN000;
52410 ;
52411 ;HASHINITVAR: ; LABEL WORD ; AN000;
52412 ;
52413 ; Replaced by next two declarations
52414 ;
52415 ;UU_BUF_HASH_PTR:
52416 ; dd 0 ; [SYSTEM] buffer Hash table addr
52417 ;UU_BUF_HASH_COUNT:
52418 ; dw 1 ; [SYSTEM] number of Hash entries
52419 ;
52420 ;BufferQueue:
52421 ; dd 0 ; [SYSTEM] Head of the buffer Queue
52422 ;DirtyBufferCount:
52423 ; dw 0 ; [SYSTEM] Count of Dirty buffers in the Que
52424 ; BUGBUG ---- change to byte
52425 ;SC_CACHE_PTR:
52426 ; dd 0 ; [SYSTEM] secondary cache pointer
52427 ;SC_CACHE_COUNT:
52428 ; dw 0 ; [SYSTEM] secondary cache count
52429 ;BuffInHMA:
52430 ; db 0 ; Flag to indicate that buffs are in HMA
52431 ;LoMemBuff:
52432 ; dd 0 ; Ptr to intermediate buffer
52433 ; in Low mem when buffs are in HMA
52434 ;
52435 ; All variables which have UU_ as prefix can be reused for other
52436 ; purposes and can be renamed. All these variables were used for
52437 ; EMS support of Buffer Manager. Now they are useless for Buffer
52438 ; manager ---- MOHANS

```

```

52439 ;
52440 ; I_am UU_BUF_EMS_FIRST_PAGE,3,<0,0,0>
52441 UU_BUF_EMS_FIRST_PAGE:
52442 0000007E 000000 db 0,0,0 ; holds the first page above 640K
52443
52444 ; I_am UU_BUF_EMS_NPA640,WORD,<0> ; holds the number of pages
52445 ; UU_BUF_EMS_NPA640: ; above 640K
52446 ; dw 0
52447
52448 CL0FATENTRY:
52449 00000081 FFFF dw -1 ; M014: Holds the data that
52450 ; is used in pack/unpack rts.
52451 ; in fat.asm if cluster 0 is specified.
52452 ; SR;
52453
52454 00000083 00 IoStatFail:
52455 db 0 ; IoStatFail has been added to
52456 ; record a fail on an I24
52457 ; issued from IOFUNC on a status call.
52458
52459 ; *** I_am UU_BUF_EMS_MODE,BYTE,<-1> ; EMS mode ; AN000;
52460 ; *** I_am UU_BUF_EMS_HANDLE,BYTE ; buffer EMS handle ; AN000;
52461 ; *** I_am UU_BUF_EMS_PAGE_FRAME,WORD,<-1>; EMS page frame # ; AN000;
52462 ; *** I_am UU_BUF_EMS_SEG_CNT,WORD,<1> ; EMS seg count ; AN000;
52463 ; *** I_am UU_BUF_EMS_PFRAME,WORD ; EMS page frame seg address ; AN000;
52464 ; *** I_am UU_BUF_EMS_RESERV,WORD,<0> ; reserved ; AN000;
52465 ;
52466 ; *** I_am UU_BUF_EMS_MAP_BUFF,1,<0> ; this is not used to save the
52467 ; state of the buffers page.
52468 ; This one byte is retained to
52469 ; keep the size of this data
52470 ; block the same.;
52471
52472 00000084 00 ALLOCMSAVE:
52473 db 0 ; M063: temp var. used to
52474 ; M063: save alloc method in
52475 ; M063: msproc.asm
52476
52477 00000085 00 A20OFF_COUNT:
52478 db 0 ; M068: indicates the # of
52479 ; M068: int 21 calls for
52480 ; M068: which A20 is off
52481
52482 00000086 00 DOS_FLAG:
52483 db 0 ; see DOSSYM.INC for Bit
52484 ; definitions
52485
52486 00000087 0000 UNPACK_OFFSET:
52487 dw 0 ; saves pointer to the start
52488 ; of unpack code in exepatch.
52489 ; asm.
52490
52491 00000088 00 UMBFLAG:
52492 db 0 ; M003: bit 0 indicates the
52493 ; M003: link state of the UMBs
52494 ; M003: whether linked or not
52495 ; M003: to the DOS arena chain
52496
52497 0000008A 0000 SAVE_AX:
52498 dw 0 ; M000: temp variable to store ax
52499 ; M000: in msproc.asm
52500
52501 0000008C FFFF UMB_HEAD:
52502 dw -1 ; M000: this is initialized to
52503 ; M000: the first umb arena by
52504 ; M000: BIOS sysinit.
52505
52506 0000008E 0100 START_ARENA:
52507 dw 1 ; M000: this is the first arena
52508 ; M000: from which DOS will
52509 ; M000: start its scan for alloc.
52510
52511 ; End of SYSINITVar block
52512 -----
52513 ; 25/04/2019 - Retro DOS v4.0
52514
52515 ; 16/07/2018
52516 ; MSDOS 3.3 (& MDOS 6.0)
52517
52518 ;
52519 ; Sharer jump table
52520 ;
52521
52522 ;PUBLIC JShare
52523 ;EVEN
52524
52525 ;JShare LABEL DWORD
52526 ; DW OFFSET DOSCODE:BadCall, 0
52527 ; DW OFFSET DOSCODE:OKCall, 0 ; 1 MFT_enter
52528 ; DW OFFSET DOSCODE:OKCall, 0 ; 2 MFTClose
52529 ; DW OFFSET DOSCODE:BadCall, 0 ; 3 MFTclu
52530 ; DW OFFSET DOSCODE:BadCall, 0 ; 4 MFTcloseP
52531 ; DW OFFSET DOSCODE:BadCall, 0 ; 5 MFTclon
52532 ; DW OFFSET DOSCODE:BadCall, 0 ; 6 set_block
52533 ; DW OFFSET DOSCODE:BadCall, 0 ; 7 clr_block
52534 ; DW OFFSET DOSCODE:OKCall, 0 ; 8 chk_block
52535 ; DW OFFSET DOSCODE:BadCall, 0 ; 9 MFT_get
52536 ; DW OFFSET DOSCODE:BadCall, 0 ; 10 ShSave
52537 ; DW OFFSET DOSCODE:BadCall, 0 ; 11 Shchk
52538 ; DW OFFSET DOSCODE:OKCall, 0 ; 12 Shcol
52539 ; DW OFFSET DOSCODE:BadCall, 0 ; 13 ShCloseFile
52540 ; DW OFFSET DOSCODE:BadCall, 0 ; 14 ShSU
52541
52542 align 2
52543
52544 JShare:
52545 DW BadCall,0
52546 MFT_enter: DW OKCall, 0 ; 1 MFT_enter
52547 MFTClose: DW OKCall, 0 ; 2 MFTClose
52548 MFTclu: DW BadCall,0 ; 3 MFTclu
52549 MFTcloseP: DW BadCall,0 ; 4 MFTcloseP
52550 MFTclon: DW BadCall,0 ; 5 MFTclon
52551 set_block: DW BadCall,0 ; 6 set_block
52552 clr_block: DW BadCall,0 ; 7 clr_block
52553 chk_block: DW OKCall, 0 ; 8 chk_block
52554 MFT_get: DW BadCall,0 ; 9 MFT_get
52555 ShSave: DW BadCall,0 ; 10 ShSave
52556 Shchk: DW BadCall,0 ; 11 Shchk
52557 Shcol: DW OKCall, 0 ; 12 Shcol
52558 ShCloseFile: DW BadCall,0 ; 13 ShCloseFile
52559 ShSU: DW BadCall,0 ; 14 ShSU
52560
52561 ;=====
52562 ; CONST2.ASM (MSDOS 6.0, 1991)
52563 ;=====
52564 ; 25/04/2019 - Retro DOS 4.0
52565 ; 16/07/2018 - Retro DOS 3.0
52566
52567 ;Break <Initialized data and data used at DOS initialization>
52568 ;-----

```



```

52563 ; We need to identify the parts of the data area that are relevant to tasks
52564 ; and those that are relevant to the system as a whole. Under 3.0, the system
52565 ; data will be gathered with the system code. The process data under 2.x will
52566 ; be available for swapping and under 3.0 it will be allocated per-process.
52567 ;
52568 ; The data that is system data will be identified by [SYSTEM] in the comments
52569 ; describing that data item.
52570
52571 ;DOSDATA SEGMENT WORD PUBLIC 'DATA'
52572
52573 ;
52574 ; Table of routines for assignable devices
52575 ;
52576 ; MSDOS allows assignment if the following standard devices:
52577 ; stdin (usually CON input)
52578 ; stdout (usually CON output)
52579 ; auxin (usually AUX input)
52580 ; auxout (usually AUX output)
52581 ; stdlpt (usually PRN output)
52582 ;
52583 ; SPECIAL NOTE:
52584 ; Status of a file is a strange idea. We choose to handle it in this manner:
52585 ; If we're not at end-of-file, then we always say that we have a character.
52586 ; Otherwise, we return ^Z as the character and set the ZERO flag. In this
52587 ; manner we can support program written under the old DOS (they use ^Z as EOF
52588 ; on devices) and programs written under the new DOS (they use the ZERO flag
52589 ; as EOF).
52590
52591 ; Default SFTs for boot up
52592
52593 ;PUBLIC SFTABL
52594
52595 SFTABL: ; LABEL DWORD ; [SYSTEM] file table
52596 000000CC FFFF DW -1 ; [SYSTEM] link to next table
52597 000000CE FFFF DW -1 ; [SYSTEM] link seg to next table
52598 ;dw 5 ; 24/03/2024
52599 000000D0 0500 DW sf_default_number ; [SYSTEM] Number of entries in table
52600 ;times 295 db 0 ; MSDOS 6.0 ; PCDOS 7.1 ; 24/03/2024
52601 000000D2 00<rep 127h> times (sf_default_number*sf_entry_size) db 0
52602
52603 ; the next two variables relate to the position of the logical stdout/stdin
52604 ; cursor. They are only meaningful when stdin/stdout are assigned to the
52605 ; console.
52606 ; DOSDATA:01F9h (MSDOS 6.21) ; PCDOS 7.1 ; 24/03/2024
52607 000001F9 00 CARPOS: db 0 ; [SYSTEM] cursor position in stdin
52608 000001FA 00 STARTPOS: db 0 ; [SYSTEM] position of cursor at beginning
52609 ; of buffered input call
52610 000001FB 00<rep 80h> INBUF: times 128 db 0 ; [SYSTEM] general device input buffer
52611 0000027B 00<rep 83h> CONBUF: times 131 db 0 ; [SYSTEM] The rest of INBUF and console buffer
52612 ; DOSDATA:02FEh (MSDOS 6.21) ; PCDOS 7.1
52613 000002FE 00 PFLAG: db 0 ; [SYSTEM] printer echoing flag
52614 000002FF 00 VERFLG: db 0 ; [SYSTEM] Initialize with verify off
52615 00000300 03 CHARCO: db 00000011b ; 3 ; [SYSTEM] Allows statchks every 4 chars...
52616 ; switch_character:
52617 00000301 2F chSwitch: db '/' ; UNUSED - obsolete datum, can be reused
52618 00000302 00 AllocMethod: db 0 ; [SYSTEM] how to alloc first(best)last
52619 00000303 00 fShare: db 0 ; [SYSTEM] TRUE => sharing installed
52620 00000304 01 DIFFNAM: db 1 ; [SYSTEM] Indicates when MYNAME has changed
52621 00000305 20<rep 10h> MYNAME: times 16 db 20h ; [SYSTEM] My network name
52622
52623 ; The following table is a list of addresses that the sharer patches to be
52624 ; PUSH AX to enable the critical sections
52625
52626 ; DOSDATA:0315h (MSDOS 6.21) ; PCDOS 7.1 ; 24/03/2024
52627
52628 ;PUBLIC CritPatch
52629
52630 CritPatch: ; LABEL WORD
52631
52632 ;IRP sect,<critDisk,critDevice>
52633
52634 ;IF (NOT REDIRECTOR) AND (NOT SHAREF)
52635 ;
52636 ;SR; Change code patch address to a variable in data segment
52637 ;
52638 ; dw OFFSET DOSDATA: redir_patch
52639 ; dw OFFSET DOSDATA: redir_patch
52640 ;
52641 ;;hkn Short_Addr E&sect
52642 ;;hkn Short_Addr L&sect
52643 ;
52644 ;ELSE
52645 ; DW 0
52646 ; DW 0
52647 ;ENDIF
52648 ;ENDM
52649 ; DW 0
52650
52651 ; 25/07/2019 - Retro DOS v4.0 (MSDOS 6.21)
52652
52653 00000315 [650D] dw redir_patch
52654 00000317 [650D] dw redir_patch
52655 00000319 [650D] dw redir_patch
52656 0000031B [650D] dw redir_patch
52657
52658 0000031D 0000 dw 0
52659
52660 ; WARNING!!! PRINT and PSPRINT *REQUIRE* ErrorMode to precede INDOS.
52661 ; Also, IBM server 1.0 requires this also.
52662
52663 ;db 90h ; 24/03/2024
52664
52665 ;EVEN ; Force swap area to start on word boundry
52666
52667 0000031F 90 align 2
52668 ;PUBLIC SWAP_START
52669
52670 ; 10/03/2024 - Retro DOS v5.0
52671 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0320h
52672 SWAP_START: ; LABEL BYTE
52673 00000320 00 ERRORMODE: db 0 ; Flag for INT 24 processing
52674 00000321 00 INDOS: db 0 ; DOS status for interrupt processing
52675 00000322 FF WPERR: db -1 ; write protect error flag
52676 00000323 00 EXTERR_LOCUS: db 0 ; Extended Error Locus
52677 00000324 0000 EXTERR: dw 0 ; Extended Error code
52678
52679 ;WARNING Following two bytes Accessed as word in $GetExtendedError
52680 00000326 00 EXTERR_ACTION: db 0 ; Extended Error Action
52681 00000327 00 EXTERR_CLASS: db 0 ; Extended Error Class
52682 ; end warning
52683
52684 00000328 00000000 EXTERRPT: dd 0 ; Extended Error pointer
52685
52686 0000032C 80000000 DMAADD: dd 80h ; User's disk transfer address (disp/seg)

```

```

52687 00000330 0000 CurrentPDB: dw 0 ; Current process identifier
52688 00000332 0000 ConC_Spsave: dw 0 ; saved SP before ^C
52689 00000334 0000 exit_code: dw 0 ; exit code of last proc.
52690 00000336 00 CURDRV: db 0 ; Default drive (init A)
52691 00000337 00 CNTCFLAG: db 0 ; ^C check in dispatch disabled
52692 ; ; F.C. 2/17/86
52693 00000338 00 CPSWFLAG: db 0 ; Code Page Switching Flag DOS 4.00
52694 00000339 00 CPSWSAVE: db 0 ; copy of above in case of ABORT
52695 ;align 2
52696 ; 10/03/2024 - Retro DOS v5.0
52697 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:033Ah
52698 SWAP_ALWAYS: ; 05/08/2018
52699 0000033A 0000 USER_IN_AX: dw 0 ; User INPUT AX value (used for
52700 ; extended error type stuff.
52701 ; NOTE: does not have Correct value on
52702 ; 1-12, OEM, Get/Set CurrentPDB,
52703 ; GetExtendedError system calls)
52704 0000033C 0000 PROC_ID: dw 0 ; PID for sharing (0 = local)
52705 0000033E 0000 USER_ID: dw 0 ; Machine for sharing (0 = local)
52706 00000340 0000 FirstArena: dw 0 ; first free block found
52707 00000342 0000 BestArena: dw 0 ; best free block found
52708 00000344 0000 LastArena: dw 0 ; last free block found
52709 00000346 0000 ENDMEM: dw 0 ; End of memory used in DOSINIT
52710 00000348 0000 LASTENT: dw 0 ; Last entry for directory search
52711 0000034A 00 FAILERR: db 0 ; NZ if user did FAIL on I 24
52712 0000034B 00 ALLOWED: db 0 ; Allowed I 24 answers (see allowed_)
52713 0000034C 00 NoSetDir: db 0 ; true -> do not set directory
52714 0000034D 00 DidCTRLC: db 0 ; true -> we did a ^C exit
52715 0000034E 00 SpaceFlag: db 0 ; true -> embedded spaces are allowed in FC
52716
52717 ; Warning! The following items are accessed as a WORD in TIME.ASM
52718 ;EVEN
52719 0000034F 90 align 2
52720 ; DOSDATA:0350h (MSDOS 6.21)
52721 00000350 00 DAY: db 0 ; Day of month
52722 00000351 00 MONTH: db 0 ; Month of year
52723 00000352 0000 YEAR: dw 0 ; Year (with century)
52724 00000354 FFFF DAYCNT: dw -1 ; Day count from beginning of year
52725 00000356 00 WEEKDAY: db 0 ; Day of week
52726 ; end warning
52727
52728 00000357 00 CONSWAP: db 0 ; TRUE => console was swapped during device read
52729 00000358 01 IDLEINT: db 1 ; TRUE => idle int is allowed
52730 00000359 00 fAborting: db 0 ; TRUE => abort in progress
52731
52732 ; Combination of all device call parameters
52733 ;PUBLIC DEVCALL ;
52734 ;DEVCALL SRHEAD <> ; basic header for disk packet
52735 DEVCALL: ; 08/08/2018
52736 0000035A 00 DEVCALL_REQLEN: db 0 ; Length in bytes of request block
52737 0000035B 00 DEVCALL_REQUNIT: db 0 ; Device unit number
52738 0000035C 00 DEVCALL_REQFUNC: db 0 ; Type of request
52739 0000035D 0000 DEVCALL_REQSTAT: dw 0 ; Status word
52740 0000035F 00<rep 8h> times 8 db 0 ; Reserved for queue links
52741
52742 ;PUBLIC CALLUNIT
52743 CALLUNIT: ; LABEL BYTE ; unit number for disk
52744 CALLFLSH: ; LABEL WORD ;
52745 00000367 00 CALLMED: db 0 ; media byte
52746 CALLBR: ; LABEL DWORD ;
52747 ;PUBLIC CALLXAD
52748 CALLXAD: ; LABEL DWORD ;
52749 00000368 00 CALLRBYT: db 0 ;
52750 ;PUBLIC CALLVIDM
52751 CALLVIDM: ; LABEL DWORD ;
52752 00000369 00<rep 3h> times 3 db 0 ;
52753 ;PUBLIC CallBPB
52754 CALLBPB: ; LABEL DWORD ;
52755 CALLSCNT: ;
52756 0000036C 0000 dw 0 ;
52757 ;PUBLIC CALLSSEC
52758 CALLSSEC: ; LABEL WORD ;
52759 0000036E 0000 dw 0 ;
52760 00000370 00000000 CALLVIDRW: dd 0 ;
52761 ;MSDOS 6.0
52762 00000374 00000000 CALLNEWSC: dd 0 ; starting sector for >32mb
52763 00000378 00000000 CALLDEVAD: dd 0 ; stash for device entry point
52764
52765 ; Same as above for I/O calls ;
52766 ;
52767 ;PUBLIC IOCall
52768 ;IOCALL SRHEAD <> ;
52769 IOCALL: ; 07/08/2018
52770 0000037C 00 IOCALL_REQLEN: db 0 ; Length in bytes of request block
52771 0000037D 00 IOCALL_REQUNIT: db 0 ; Device unit number
52772 0000037E 00 IOCALL_REQFUNC: db 0 ; Type of request
52773 0000037F 0000 IOCALL_REQSTAT: dw 0 ; Status word
52774 00000381 00<rep 8h> times 8 db 0 ; Reserved for queue links
52775 IOFLSH: ; LABEL WORD ;
52776 ;PUBLIC IORCHR
52777 IORCHR: ; LABEL BYTE ;
52778 00000389 00 IOMED: db 0 ;
52779 0000038A 00000000 IOXAD: dd 0 ;
52780 0000038E 0000 IOSCNT: dw 0 ;
52781 00000390 0000 IOSSEC: dw 0 ;
52782
52783 ; Call struct for DSKSTATCHK ;
52784 00000392 0E DSKSTCALL: db DRDNDHL ; = 14
52785 00000393 00 db 0 ;
52786 00000394 05 DSKSTCOM: db DEVRDND ; = 5
52787 00000395 0000 DSKSTST: dw 0 ;
52788 00000397 00<rep 8h> times 8 db 0 ;
52789 0000039F 00 DSKCHRET: db 0 ;
52790
52791 ;hkn; short_addr has been changed to provide offset in DOSCODE.
52792 ;hkn; deviobuf is in DATA seg (DOSDATA)
52793 ;hkn short_addr DEVIOPTR ;
52794
52795 000003A0 [BC03] DEVIOPTR_PTR dw DEVIOPTR
52796 000003A2 0000 DOSSEG_INIT dw 0 ; DOS segment set at Init
52797 000003A4 0100 DSKSTCNT: dw 1 ;
52798 000003A6 0000 dw 0 ;
52799
52800 000003A8 00 CreatePDB: db 0 ; flag for creating a process
52801
52802 ;MSDOS 6.0
52803 Lock_Buffer: ; LABEL DWORD ;MS. DOS Lock Buffer for Ext Lock
52804 000003A9 00000000 dd 0 ;MS. position
52805 000003AD 00000000 dd 0 ;MS. length
52806
52807 ;hkn; the foll. was moved from dosmes.asm.
52808
52809 ; 29/01/2024 - Retro DOS v5.0 (PCDOS v7.1 IBMDOS.COM)
52810 ; DOSDATA:03B1h

```

```

52811 000003B1 90      IOCTL_drvnum:      db 90h ; nop
52812
52813      ;EVEN
52814 align 2              ; needed to maintain offsets
52815
52816      ; DOSDATA:03B2h (MSDOS 6.21)
52817
52818 000003B2 0000      USERNUM:      dw 0      ; 24 bit user number
52819 000003B4 00      db 0
52820
52821      ;IF IBM
52822      ;IF IBMCOPYRIGHT
52823 000003B5 00      ; 24/03/2024 (PCDOS v7.1 IBMDOS.COM)
52824      OEMNUM:      db 0      ; 8 bit OEM number
52825      ;ELSE
52826      ;OEMNUM:      db 0FFh      ; 8 bit OEM number
52827      ;ENDIF
52828      ;ELSE
52829      ;OEMNUM:      db 0FFh      ; (MSDOS 6.22) ; 24/03/2024
52830      ;ENDIF
52831
52832      ;=====
52833      ; MS_DATA.ASM (MSDOS 6.0, 1991)
52834      ;=====
52835      ; 25/04/2019 - Retro DOS 4.0
52836
52837      ; Retro DOS v4.0 NOTE: (by Erdogan Tan, 25/04/2019)
52838      ;-----
52839      ; This data section which was named as uninitialized data
52840      ; (as overlayed by initialization code) but follows
52841      ; initialized data section from DOSDATA:03B6h address
52842      ; (in otherwords, the method is different than MSDOS 3.3,
52843      ; and there is not overlaying..)
52844      ; *****
52845      ; Reference: MSDOS 6.21 kernel DOSDATA section (4970 bytes)
52846      ; follows DOSCODE section in the kernel file (MSDOS.SYS)
52847      ; (it is located at offset 0BF70h, file offset 0BF70h-3DE0h)
52848      ; but starts from offset 0 (ORG 0) and ends at offset 1370h.
52849      ; TIMEBUF is at offset 03B6h.
52850      ; *****
52851
52852      ;Break <Uninitialized data overlayed by initialization code>
52853      ;-----
52854      ;DOSDATA SEGMENT WORD PUBLIC 'DATA'
52855      ; Init code overlaps with data area below
52856
52857      ; ORG 0
52858
52859      MSDAT001s: ; label byte
52860
52861      ; DOSDATA:03B6h ; MSDOS 6.21 (MSDOS.SYS, file offset 0BF70h-3DE0h+3B6h)
52862 000003B6 0000<rep 3h> TIMEBUF: ; times 6 db 0
52863 000003BC 0000      times 3 dw 0      ; Time read from clock device
52864      DEVI0BUF: dw 0      ; Buffer for I/O under file assignment
52865
52866      ; The following areas are used as temp buffer in EXEC system call
52867
52868      ; DOSDATA:03BEh
52869 000003BE 00<rep 80h> OPENBUF: ;times 64 dw 0
52870      times 128 db 0      ; buffer for name operations
52871 0000043E 00<rep 80h> RENBUF:
52872      times 128 db 0      ; buffer for rename destination
52873
52874      ; Buffer for search calls
52875 000004BE 00<rep 35h> SEARCHBUF:
52876      times 53 db 0      ; internal search buffer
52877 000004F3 00<rep 58h> DUMMYCDS: ;times 88 db 0
52878      times curdirLen db 0
52879
52880      ; End of contiguous buffer
52881
52882      ; Temporary directory entry for use by many routines. Device directory
52883      ; entries (bogus) are built here.
52884
52885      ; DOSDATA:054Bh
52886
52887      DEVFCB: ; LABEL BYTE ; Uses NAME1, NAME2, combined
52888
52889      ; WARNING.. do not alter position of NAME1 relative to DEVFCB
52890      ; without first examining BUILD_DEVICE_ENT. Look carefully at DOS_RENAME
52891      ; as well as it is the only guy who uses NAME2 and DESTSTART.
52892
52893 0000054B 00<rep Ch> NAME1: times 12 db 0 ; File name buffer
52894
52895 00000557 00<rep Dh> NAME2: times 13 db 0 ;
52896      DESTSTART:
52897      dw 0 ;
52898      ;DB ((SIZE DIR_ENTRY) - ($ - DEVFCB)) DUP (?)
52899      ;times 5 db 0
52900 00000566 00<rep 5h> times ((dir_entry.size)-($-DEVFCB)) db 0
52901
52902      ; End Temporary directory entry.
52903
52904 0000056B 00      ATTRIB: db 0 ; storage for file attributes
52905      EXTFCB: db 0 ; TRUE => extended FCB in use
52906 0000056C 00
52907      SATTRIB: db 0 ; Storage for search attributes
52908 0000056D 00
52909      OPEN_ACCESS: db 0 ; access of open system call
52910 0000056E 00
52911      FOUNDDEL: db 0 ; true => file was deleted
52912 0000056F 00
52913      FOUND_DEV: db 0 ; true => search found a device
52914 00000570 00
52915      FSPLICE: db 0 ; true => do a splice in transpath
52916 00000571 00
52917      FSHARING: db 0 ; TRUE => no redirection
52918 00000572 00
52919      SECCLUSPOS: db 0 ; Position of first sector within cluster
52920 00000573 00
52921 00000574 00      TRANS: db 0 ;
52922 00000575 00      READOP: db 0 ;
52923
52924 00000576 00      THISDRV: db 0 ;
52925
52926 00000577 00      CLUSFAC: db 0 ;
52927
52928 00000578 00      CLUSSPLIT: db 0 ;
52929
52930 00000579 00      INSMODE: db 0 ; true => insert mode in buffered read
52931 0000057A 00      CMETA: db 0 ; count of meta'ed components found
52932 0000057B 00      VOLID: db 0 ;
52933
52934 0000057C 00      EXIT_TYPE: db 0 ; type of exit...

```

```

52935             ; 24/03/2024
52936 0000057D 00 db 0 ; PCDOS 7.1 - DOSDATA:057Dh
52937             ;db 90h ; MSDOS 6.22 - DOSDATA:057Dh
52938
52939             ;EVEN
52940
52941 align 2
52942
52943             ; DOSDATA:057Eh
52944
52945             ; WARNING - the following two items are accessed as a word
52946
52947 CREATING:
52948 db 0 ; true => creating a file
52949 0000057F 00 DELALL: db 0 ; = 0 iff BUGBUG
52950             ; = DIRFREE iff BUGBUG
52951
52952 00000580 00000000 EXITHOLD: dd 0 ; Temp location for proc terminate
52953
52954 00000584 0000 USER_SP: dw 0 ; User SP for system call
52955
52956 00000586 0000 USER_SS: dw 0 ; User SS for system call
52957
52958 00000588 0000 CONTSTK: dw 0 ;
52959
52960 0000058A 00000000 THISDPB: dd 0 ;
52961
52962 0000058E 0000 CLUSSAVE: dw 0 ;
52963
52964 00000590 00000000 CLUSSEC: dd 0 ;>32mb AC0000
52965
52966 00000594 0000 PREREAD: dw 0 ; 0 means preread; 1 means optional
52967 00000596 0000 FATBYT: dw 0 ; Used by ALLOCATE
52968
52969 00000598 0000 FATBYTE: dw 0 ; Used by $SLEAZEFUNC
52970
52971 0000059A 00000000 ; DOSDATA:059Ah
52972 DEVPT: dd 0 ;
52973 0000059E 00000000 THISSFT: dd 0 ; Address of user SFT
52974
52975 000005A2 00000000 THISCDS: dd 0 ; Address of current CDS
52976
52977 000005A6 00000000 THISFCB: dd 0 ; Address of user FCB
52978 000005AA FFFF SFN: dw -1 ; SystemFileNumber found for accessfile
52979 000005AC 0000 JFN: dw 0 ; JobFileNumber found for accessfile
52980 000005AE 00000000 PJFN: dd 0 ; PointerJobFileNumber found for accessfile
52981
52982 000005B2 0000 WFP_START: dw 0 ;
52983
52984 000005B4 0000 REN_WFP: dw 0 ;
52985
52986 000005B6 0000 CURR_DIR_END: dw 0 ;
52987
52988 000005B8 0000 NEXTADD: dw 0 ;
52989
52990 000005BA 0000 LASTPOS: dw 0 ;
52991
52992 000005BC 0000 CLUSNUM: dw 0 ;
52993 000005BE 00000000 DIRSEC: dd 0 ;>32mb AC0000
52994
52995 000005C2 0000 DIRSTART: dw 0 ;
52996 000005C4 00000000 SECPPOS: dd 0 ;>32mb Position of first sector accessed
52997 000005C8 00000000 VALSEC: dd 0 ;>32mb Number of valid (previously written)
52998             ; sectors
52999
53000 000005CC 0000 BYTSECPPOS: dw 0 ; Position of first byte within sector
53001 BYTPOS: ;times 4 db 0 ; Byte position in file of access
53002 000005CE 0000<rep 2h> times 2 dw 0
53003
53004 000005D2 0000 BYTCNT1: dw 0 ; No. of bytes in first sector
53005
53006 000005D4 0000 BYTCNT2: dw 0 ; No. of bytes in last sector
53007 000005D6 0000 SECCNT: dw 0 ; No. of whole sectors
53008
53009 ; DOSDATA:05D8h
53010 000005D8 0000 ENTFREE: dw 0 ;
53011
53012 000005DA 0000 ENTLAST: dw 0 ;
53013
53014 000005DC 0000 NXTCLUSNUM: dw 0 ;
53015
53016 000005DE 00000000 GROWCNT: dd 0 ;
53017 000005E2 00000000 CURBUF: dd 0 ;
53018 000005E6 00000000 CONSFT: dd 0 ; SFT of console swapped guy.
53019 000005EA 0000 SAVEBX: dw 0 ;
53020 000005EC 0000 SAVEDS: dw 0 ;
53021
53022 000005EE 0000 RESTORE_TMP: dw 0 ; return address for restore world
53023 000005F0 0000 NSS: dw 0
53024 000005F2 0000 NSP: dw 0
53025 ; DOSDATA:05F4h
53026
53027 000005F4 0000 EXTOPEN_FLAG: dw 0 ;FT. extended open input flag ;AN000;
53028
53029 000005F6 00 EXTOPEN_ON: db 0 ;FT. extended open conditional flag ;AN000;
53030
53031 000005F7 0000 EXTOPEN_IO_MODE: dw 0 ;FT. extended open io mode ;AN000;
53032
53033 000005F9 0000 SAVE_DI: dw 0 ;FT. extended open saved DI ;AN000;
53034
53035 000005FB 0000 SAVE_ES: dw 0 ;FT. extended open saved ES ;AN000;
53036
53037 000005FD 0000 SAVE_DX: dw 0 ;FT. extended open saved DX ;AN000;
53038
53039 000005FF 0000 SAVE_CX: dw 0 ;FT. extended open saved CX ;AN000;
53040
53041 00000601 0000 SAVE_BX: dw 0 ;FT. extended open saved BX ;AN000;
53042
53043 00000603 0000 SAVE_SI: dw 0 ;FT. extended open saved SI ;AN000;
53044
53045 00000605 0000 SAVE_DS: dw 0 ;FT. extended open saved DS ;AN000;
53046
53047 ; DOSDATA:0607h
53048
53049 ; HIGH_SECTOR is a hack to allow passing 32-bit sector numbers where
53050 ; we used to just pass 16 bits in a register. Now High_SECTOR holds
53051 ; the high 16, the low 16 are still in the register.
53052
53053 HIGH_SECTOR:
53054 00000607 0000 dw 0 ;>32mb higher sector # ;AN000;
53055
53056 ; 25/09/2023
53057 OffsetMagicPatch:
53058 00000609 [4013] dw MagicPatch ;scottq 8/6/92

```

```

53059             ; 06/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
53060             ;dw      0
53061
53062             ;see dos\mpatch.asm
53063 0000060B 00      db      0          ;>32mb indicating disk full when 1 ;AN000;
53064 TEMP_VAR:      dw      0          ; temporary variable for everyone ;AN000;
53065 0000060C 0000
53066 TEMP_VAR2:      dw      0          ; temporary variable 2 for everyone ;AN000;
53067 0000060E 0000      DrvErr:  db      0          ; used to save drive error ;AN000;
53068 00000610 00
53069 DOS34_FLAG:      dw      0          ; common flag for DOS 3.4 ;AN000;
53070 00000611 0000
53071 NO_FILTER_PATH:  dd      0          ; pointer to original path ;AN000;
53072 00000613 00000000
53073 NO_FILTER_DPATH: dd      0          ; pointer to original path of destination ;AN000;
53074 00000617 00000000
53075 ; M008
53076 AbsRdWr_SS:      dw      0          ; INT 25/26 user stack segment
53077 0000061B 0000
53078 AbsRdWr_SP:      dw      0          ; INT 25/26 user stack offset
53079 0000061D 0000
53080
53081             ; I_am UU_Callback_flag,BYTE,<0> ; Unused
53082 ; M008
53083             ; 24/03/2024
53084             ; MSDOS 5.0 MSDOS.SYS - DOSDATA:061Fh
53085             ; MSDOS 6.22 MSDOS.SYS - DOSDATA:061Fh
53086             ; 01/01/2024
53087             ; PCDOS 7.1 IBMDOS.COM - DOSDATA:061Fh
53088 0000061F 00      db      0
53089
53090             ; make those pushes fast!!!
53091             ;EVEN
53092
53093 align 2
53094
53095 StackSize equ 180h ; 384 ; gross but effective
53096
53097 ;StackSize equ 300h ; 768 ; This is a "trial" change IBM hasn't
53098 ; ; made up their minds about
53099
53100             ; WARNING!!!! DskStack may grow into AUXSTACK due to interrupt service.
53101             ; This is NO problem as long as AUXSTACK comes immediately before DSKSTACK
53102
53103 RENAMEDMA: ; LABEL BYTE ; See DOS_RENAME
53104
53105 00000620 00<rep 180h>      times      StackSize db 0 ; db 384 dup(0) ; PCDOS 7.1
53106 AUXSTACK: ; LABEL BYTE
53107
53108 000007A0 00<rep 180h>      times      StackSize db 0 ; db 384 dup(0) ; PCDOS 7.1
53109 DSKSTACK: ; LABEL BYTE
53110             ; 24/03/2024
53111             ; 01/01/2024 - Retro DOS v5.0 (PCDOS 7.1 IBMDOS.COM)
53112             ; (PCDOS 7.1 IBMDOS.COM - DOSDATA:0920h)
53113 00000920 402349424D3A31322E-      db '@#IBM:12.01.2003.build_1.32#@ IBMDOS.COM(USA)',0
53113 00000929 30312E323030332E62-
53113 00000932 75696C645F312E3332-
53113 0000093B 23402049424D444F53-
53113 00000944 2E434F4D2855534129-
53113 0000094D 00
53114 0000094E 00<rep 152h>      times StackSize-($-DSKSTACK) db 0 ; db 338 dup(0) ; PCDOS 7.1
53115 ;times StackSize db 0
53116 IOSTACK: ; LABEL BYTE
53117
53118 ; DOSDATA:0AA0h
53119
53120 ; patch space for Boca folks.
53121 ; Say what????!!! This does NOT go into the swappable area!
53122 ; NOTE: we include the decl of ibmpatch in ms-dos even though it is not needed.
53123 ; This allows the REDIRector to work on either IBM or MS-DOS.
53124
53125 IBMPATCH: ; label byte
53126 PRINTER_FLAG:
53127 00000AA0 00      db      0          ; [SYSTEM] status of PRINT utility
53128 VOLCHNG_FLAG:
53129 ;db      0          ; [SYSTEM] true if volume label created
53130             ; 24/03/2024 (PCDOS 7.1 IBMDOS.COM)
53131 00000AA1 FF      db      0FFh ; -1
53132 VIRTUAL_OPEN:
53133 00000AA2 00      db      0          ; [SYSTEM] non-zero if we opened a virtual file
53134
53135 ; 24/03/2024 - Retro DOS v5.0
53136 ; PCDOS 7.1 IBMDOS.COM
53137 %if 0
53138
53139 ; Following 4 variables moved to MSDATA.asm from Mstable.asm (P4986)
53140
53141             ; DOSDATA:0AA3h ; MSDOS 6.22 ; MSDOS 5.0
53142 FSeek_drive:
53143      db      0          ;AN000; fastseek drive #
53144 FSeek_firclus:
53145      dw      0          ;AN000; fastseek first cluster #
53146 FSeek_logclus:
53147      dw      0          ;AN000; fastseek logical cluster #
53148 FSeek_logsave:
53149      dw      0          ;AN000; fastseek returned log clus #
53150
53151 %endif
53152
53153 ; DOSDATA:0AAAh ; MSDOS 6.22 ; MSDOS 5.0
53154 ; DOSDATA:0AA3h ; PCDOS 7.1 ; 24/03/2024
53155
53156 TEMP_DOSLOC:
53157 00000AA3 FFFF      dw      -1          ;stores the temporary location of dos
53158 ;at SYSINIT time.
53159
53160 ; 10/03/2024
53161 ;SWAP_END: ; LABEL BYTE
53162
53163 ; THE FOLLOWING BYTE MUST BE HERE, IMMEDIATELY FOLLOWING SWAP_END. IT CANNOT
53164 ; BE USED. If the size of the swap data area is ODD, it will be rounded up
53165 ; to include this byte.
53166             ; 24/03/2024
53167             ; 05/01/2024 (PCDOS 7.1 IBMDOS.COM)
53168             ;db      0          ; DOSDATA:0AACh (MSDOS 5.0-6.22 MSDOS.SYS)
53169
53170 ; DOSDATA:0AADh
53171
53172 ;hkn;      DB      (512+80+32-(SWAP_END-ibmpatch)) DUP (?)
53173
53174 ; -----
53175 ; 05/01/2024 - Retro DOS v5.0
53176 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0AA5h
53177

```

```

53178             ; 24/01/2024
53179 LNE_COUNT:
53180 00000AA5 0000          dw      0          ; long name entry count (for file)
53181
53182 00000AA7 00<rep Eh>      times 14 db 0
53183
53184 ENTLAST_PREV:
53185 00000AB5 0000          dw      0          ; previous ENTLAST (for long name search !?)
53186
53187 00000AB7 00<rep 24h>     times 36 db 0
53188
53189 absdrw_extd:
53190 00000ADB 00            db      0
53191 DIRSTART_HW:
53192 00000ADC 0000          dw      0
53193 CLUSNUM_HW:
53194 00000ADE 0000          dw      0
53195 NXTCLUSNUM_HW:
53196 00000AE0 0000          dw      0
53197 LASTPOS_HW:
53198 00000AE2 0000          dw      0
53199 FATBYT_HW:
53200 00000AE4 0000          dw      0
53201 DESTSTART_HW:
53202 00000AE6 0000          dw      0
53203 CLUSTNUM_HW:
53204 00000AE8 0000          dw      0
53205 CLUSDATA_HW:             ; cluster data (0 = release, -1 = allocate) ; 30/01/2024
53206 00000AEA 0000          dw      0
53207 CCONTENT_HW:           ; UNPACK output
53208 00000AEC 0000          dw      0
53209 ROOTCLUST_HW:
53210 00000AEE 0000          dw      0
53211 CCOUNT_HW:             ; 09/02/2024 ; for ALLOCATE
53212 00000AF0 0000          dw      0
53213 CLUSTERS_HW:
53214 00000AF2 0000          dw      0
53215 00000AF4 0000          dw      0
53216 00000AF6 0000          dw      0
53217 CLSKIP_HW:
53218 00000AF8 0000          dw      0
53219
53220             ; 10/03/2024 - Retro DOS v5.0
53221             ; (PCDOS 7.1 IBMDOS.COM)
53222 SWAP_END:  ; LABEL  BYTE
53223
53224 ;DOSDATA      ENDS
53225
53226 ;=====
53227 ; DOSTAB.ASM (MSDOS 6.0, 1991)
53228 ;=====
53229 ; 27/04/2019 - Retro DOS 4.0
53230 ; 16/07/2018 - Retro DOS 3.0
53231
53232 ;DOSDATA Segment
53233
53234 ; DOSDATA:0AADh (MSDOS 6.21, MSDOS.SYS)
53235
53236 ; DOSDATA:0AFAh (PCDOS 7.1, IBMDOS.COM) ; 05/01/2024
53237
53238 ;
53239 ; upper case table
53240 ; -----
53241 UCASE_TAB:  ; label  byte
53242             dw      128
53243             db      128,154,069,065,142,065,143,128
53244             db      069,069,069,073,073,073,142,143
53245             db      144,146,146,079,153,079,085,085
53246             db      089,153,154,155,156,157,158,159
53247             db      065,073,079,085,165,165,166,167
53248             db      168,169,170,171,172,173,174,175
53249             db      176,177,178,179,180,181,182,183
53250             db      184,185,186,187,188,189,190,191
53251             db      192,193,194,195,196,197,198,199
53252             db      200,201,202,203,204,205,206,207
53253             db      208,209,210,211,212,213,214,215
53254             db      216,217,218,219,220,221,222,223
53255             db      224,225,226,227,228,229,230,231
53256             db      232,233,234,235,236,237,238,239
53257             db      240,241,242,243,244,245,246,247
53258             db      248,249,250,251,252,253,254,255
53259 ;
53260 ; file upper case table
53261 ; -----
53262 FILE_UCASE_TAB:  ; label  byte
53263             dw      128
53264             db      128,154,069,065,142,065,143,128
53265             db      069,069,069,073,073,073,142,143
53266             db      144,146,146,079,153,079,085,085
53267             db      089,153,154,155,156,157,158,159
53268             db      065,073,079,085,165,165,166,167
53269             db      168,169,170,171,172,173,174,175
53270             db      176,177,178,179,180,181,182,183
53271             db      184,185,186,187,188,189,190,191
53272             db      192,193,194,195,196,197,198,199
53273             db      200,201,202,203,204,205,206,207
53274             db      208,209,210,211,212,213,214,215
53275             db      216,217,218,219,220,221,222,223
53276             db      224,225,226,227,228,229,230,231
53277             db      232,233,234,235,236,237,238,239
53278             db      240,241,242,243,244,245,246,247
53279             db      248,249,250,251,252,253,254,255
53280
53281 ; 24/03/2024 - Retro DOS v5.0
53282 ; PCDOS 7.1 IBMDOS.COM
53283 %if 0
53284             ; MSDOS 5.0-6.22 MSDOS.SYS - DOSDATA:0BB1h
53285 ;
53286 ; file char list
53287 ; -----
53288 FILE_CHAR_TAB:  ; label  byte
53289             dw      22          ; length
53290             db      1,0,255      ; include all
53291             db      0,0,20h      ; exclude 0 - 20h
53292             db      2,14,'.'"/\[:|<+=;',' ; exclude 14 special
53293             ;db      24 dup (?)      ; reserved
53294             times 24 db 0
53295 %endif
53296
53297             ; MSDOS 5.0-6.22 MSDOS.SYS - DOSDATA:0BE1h
53298
53299             ; 24/03/2024
53300             ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0BFEh
53301 ;

```



```

53302 ; collate table
53303 -----
53304 COLLATE_TAB:      ;label   byte
53305 00000BFE 0001          dw      256
53306 00000C00 0001020304050607 db      0,1,2,3,4,5,6,7
53307 00000C08 08090A0B0C0D0E0F db      8,9,10,11,12,13,14,15
53308 00000C10 1011121314151617 db      16,17,18,19,20,21,22,23
53309 00000C18 18191A1B1C1D1E1F db      24,25,26,27,28,29,30,31
53310 00000C20 2021222324252627 db      " ", "!", "#", "$", "%", "&", "'",
53311 00000C28 28292A2B2C2D2E2F db      "( ", ") ", "* ", " ", " ", " ", "/",
53312 00000C30 3031323334353637 db      "0", "1", "2", "3", "4", "5", "6", "7",
53313 00000C38 38393A3B3C3D3E3F db      "8", "9", ":", ";", "<", "=", ">", "?",
53314 00000C40 4041424344454647 db      "@", "A", "B", "C", "D", "E", "F", "G",
53315 00000C48 48494A4B4C4D4E4F db      "H", "I", "J", "K", "L", "M", "N", "O",
53316 00000C50 5051525354555657 db      "P", "Q", "R", "S", "T", "U", "V", "W",
53317 00000C58 58595A5B5C5D5E5F db      "X", "Y", "Z", "[", "\\", "]", "^", "_",
53318 00000C60 6041424344454647 db      "`", "A", "B", "C", "D", "E", "F", "G",
53319 00000C68 48494A4B4C4D4E4F db      "H", "I", "J", "K", "L", "M", "N", "O",
53320 00000C70 5051525354555657 db      "P", "Q", "R", "S", "T", "U", "V", "W",
53321 00000C78 58595A5B5C5D5E5F db      "X", "Y", "Z", "[", " ", " ]", " ^",
53322 00000C80 4355454141414143 db      "C", "U", "E", "A", "A", "A", "C",
53323 00000C88 4545454949494141 db      "E", "E", "E", "I", "I", "I", "A", "A",
53324 00000C90 4541414F4F4F5555 db      "E", "A", "A", "O", "O", "O", "U", "U",
53325 00000C98 594F552424242424 db      "Y", "O", "U", "S", "S", "S", "S",
53326 00000CA0 41494F554E4EA6A7 db      "A", "I", "O", "U", "N", "N", 166, 167
53327 00000CA8 3FA9AAABAC212222 db      "?", 169, 170, 171, 172, " ",
53328 00000CB0 B0B1B2B3B4B5B6B7 db      176, 177, 178, 179, 180, 181, 182, 183
53329 00000CB8 B8B9BABBBBCBDBEBF db      184, 185, 186, 187, 188, 189, 190, 191
53330 00000CC0 C0C1C2C3C4C5C6C7 db      192, 193, 194, 195, 196, 197, 198, 199
53331 00000CC8 C8C9CACBCCDCCECF db      200, 201, 202, 203, 204, 205, 206, 207
53332 00000CD0 D0D1D2D3D4D5D6D7 db      208, 209, 210, 211, 212, 213, 214, 215
53333 00000CD8 D8D9DADBDCDDDEDF db      216, 217, 218, 219, 220, 221, 222, 223
53334 00000CE0 E053          db      224, "S"
53335 00000CE2 E2E3E4E5E6E7 db      226, 227, 228, 229, 230, 231
53336 00000CE8 E8E9EAEBECEDEEEF db      232, 233, 234, 235, 236, 237, 238, 239
53337 00000CF0 F0F1F2F3F4F5F6F7 db      240, 241, 242, 243, 244, 245, 246, 247
53338 00000CF8 F8F9FABFBFCFDFEFF db      248, 249, 250, 251, 252, 253, 254, 255
53339
53340 ; -----<MSKK01>-----
53341
53342 ; MSDOS 5.0-6.22 MSDOS.SYS - DOSDATA:0CE3h
53343 ; 24/03/2024
53344 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0D00h
53345
53346 ; 29/04/2019
53347
53348 ; dbcs is not supported in DOS 3.3
53349 ;         DBCS_TAB             CC_DBCS <>
53350 ;
53351 ; DBCS for DOS 4.00                                2/12/KK
53352
53353 DBCS_TAB:      ; label byte                ;AN000; 2/12/KK
53354 -----<MSKK01>-----
53355 #ifdef DBCS
53356 #if JAPAN
53357     dw      6                      ; <MSKK01>
53358     db      081h,09fh            ; <MSKK01>
53359     db      0e0h,0fch            ; <MSKK01>
53360     db      0,0                  ; <MSKK01>
53361
53362     db      0,0,0,0,0,0,0,0,0    ; <MSKK01>
53363 #endif
53364 #if TAIWAN
53365     dw      4                      ; <TAIWAN>
53366     db      081h,0FEh            ; <TAIWAN>
53367     db      0,0                  ; <TAIWAN>
53368
53369     db      0,0,0,0,0,0,0,0,0,0
53370 #endif
53371 #if KOREA
53372     dw      4                      ; Key1
53373     db      0A1h,0FEh            ; <KOREA>
53374     db      0,0                  ; <KOREA>
53375
53376     db      0,0,0,0,0,0,0,0,0,0
53377 #endif
53378 #else
53379     dw      0                      ;AN000; 2/12/KK max number
53380     db      16 dup(0)              ;AN000; 2/12/KK
53381     times 16 db 0
53382
53383     dw      6                      ; 2/12/KK
53384     db      081h,09Fh            ; 2/12/KK
53385     db      0E0h,0FCh            ; 2/12/KK
53386     db      0,0                  ; 2/12/KK
53387
53388 #endif
53389 ; -----<MSKK01>-----
53390
53391 ; 24/03/2024 - Retro DOS v5.0
53392 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0D12h
53393 %if 1
53394 IBMDOSVERSION:
53395     ;db 7
53396     ;db 10                        ; MSVERSION
53397 00000D12 07          db MAJOR_VERSION
53398 00000D13 0A          db MINOR_VERSION
53399
53400 00000D14 C8A6        YRTAB:      db 200,166           ; Leap Year
53401 00000D16 C8A5C8A5C8A5 db      200,165,200,165,200,165
53402
53403 00000D1C 1F          MONTAB:      db 31
53404 february:
53405 00000D1D 1C1F1E1F1E1F1F1E1F- db      28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31
53406 00000D26 1E1F
53407 %endif
53408
53409 ; 24/03/2024 - Retro DOS v5.0
53410 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0D28h
53411 %if 1
53412 ;
53413 ; file char list
53414 -----
53415 FILE_CHAR_TAB:      ; label byte                ; length
53416 00000D28 1600          dw      22                    ; include all
53417 00000D2A 0100FF        db      1,0,255                ; exclude 0 - 20h
53418 00000D2D 000020        db      0,0,20h               ; exclude 14 special
53419 00000D30 020E2E222F5C5B5D3A- db      2,14,"."/\[:|<=>;,'
53420 00000D39 7C3C3E2B3D3B2C ; reserved
53421 ;db 24 dup(?)
53422 times 24 db 0
53423 %endif
53424
53425 ; 24/03/2024 - Retro DOS v5.0

```

```

53424 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0D58h
53425 %if 1
53426 SysInitTable:
53427 00000D58 [2600] dw DPBHEAD ; SYSINITVARS
53428 00000D5A 0000 dw 0
53429 00000D5C [2A12] dw COUNTRY_CDPG
53430 00000D5E 0000 dw 0
53431 00000D60 000000 db 0,0,0
53432 TEMPSEG:
53433 00000D63 0000 dw 0
53434 redir_patch:
53435 00000D65 00 db 0
53436 DosHashMA:
53437 00000D66 00 db 0
53438 FixExePatch:
53439 00000D67 0000 dw 0
53440 RationalPatchPtr:
53441 00000D69 0000 dw 0
53442 %endif
53443
53444 ; MSDOS 5.0-6.22 MSDOS.SYS - DOSDATA:0CF5h
53445 ; 24/03/2024
53446 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0D6Bh
53447
53448 ; -----
53449 ;
53450 ; CASE MAPPER ROUTINE FOR 80H-FFH character range, DOS 3.3
53451 ; ENTRY: AL = Character to map
53452 ; EXIT: AL = The converted character
53453 ; Alters no registers except AL and flags.
53454 ; The routine should do nothing to chars below 80H.
53455 ; -----
53456 ; Example:
53457
53458 MAP_CASE:
53459 ; Procedure MAP_CASE, FAR
53460
53461 00000D6B 3C80 cmp al,80h ;Map no chars below 80H ever
53462 ;JAE short Map1
53463 ;RETF
53464 ; 24/03/2024 (PCDOS 7.1 IBMDOS.COM)
53465 ;;;
53466 00000D6D 720C jb short L_RET ;Map no chars below 80H ever
53467 ;;;
53468 Map1:
53469 00000D6F 2C80 sub al,80h ;Turn into index value
53470 00000D71 1E push ds
53471 00000D72 53 push bx
53472 00000D73 BB[FC0A] mov bx,UCASE_TAB+2
53473
53474 00000D76 0E push cs ;Move to DS
53475 00000D77 1F pop ds
53476 00000D78 D7 xlat ;Get upper case character
53477 00000D79 5B pop bx
53478 00000D7A 1F pop ds
53479
53480 00000D7B CB L_RET:
53481 RETF
53482
53483 ;EndProc MAP_CASE
53484
53485 ; -----
53486 ; 24/03/2024 - Retro DOS v5.0
53487 ; PCDOS 7.1 IBMDOS.COM
53488 %if 0
53489
53490 ; The variables for ECS version are moved here for the same data alignments
53491 ; as IBM-DOS and MS-DOS.
53492
53493 InterChar:
53494 db 0 ; Interim character flag ( 1= interim) ;AN000;
53495 ;----- NOTE: NEXT TWO BYTES SOMETIMES USED AS A WORD !! -----
53496 DUMMY: ; LABEL WORD
53497 InterCon:
53498 db 0 ; Console in Interim mode ( 1= interim) ;AN000;
53499 SaveCurFlg:
53500 db 0 ; Print, do not advance cursor flag ;AN000;
53501
53502 ; -----
53503
53504 ; 17/01/2024 - Retro DOS v5.0
53505 ;TEMPSEG: dw 0 ;hkn; used to store ds.
53506 ;redir_patch:
53507 ; db 0
53508
53509 ; DOSDATA:0D0Dh
53510
53511 Mark1: ; label byte
53512
53513 ;IF2
53514 ; IF ((OFFSET MARK1) GT (OFFSET MSVERSION) )
53515 ; %OUT !DATA CORRUPTION!MARK1 OFFSET TOO BIG. RE-ORGANIZE DATA.
53516 ; ENDF
53517 ;ENDIF
53518
53519 times 5 db 0
53520
53521 ;#####
53522 ;
53523 ; ** HACK FOR DOS 4.0 REDIR **
53524 ;
53525 ; The redir requires the following:
53526 ;
53527 ; MSVERS offset D12H
53528 ; YRTAB offset D14H
53529 ; MONTAB offset D1CH
53530 ;
53531 ; WARNING! WARNING!
53532 ;
53533 ; MARK1 SHOULD NOT BE >= 0D12H. IF SOME VARIABLE IS TO BE ADDED ABOVE DO SO
53534 ; WITHOUT VIOLATING THIS AND UPDATE THE FOLL. LINE
53535 ;
53536 ; CURRENTLY MARK1 = 0D0DH
53537 ;
53538 ;#####
53539
53540 ;ORG 0D12h
53541
53542 ; DOSDATA:0D12h (MSDOS 6.21, MSDOS.SYS)
53543
53544 ;db 6
53545 ;db 20
53546
53547 ; offset 0C78h in IBMDOS.COM (MSDOS 3.3, 1987)

```



```

53548 MSVERSION:                ; MS-DOS version in hex for $GET_VERSION
53549 MSMAJORV: DB      MAJOR_VERSION ; DOS_MAJOR_VERSION
53550 MSMINORV: DB      MINOR_VERSION ; DOS_MINOR_VERSION
53551
53552 ; YRTAB & MONTAB moved from TABLE segment in ms_table.asm
53553 ;
53554 ; I_am YRTAB,8,<200,166,200,165,200,165,200,165>
53555 ; I_am MONTAB,12,<31,28,31,30,31,30,31,31,30,31,30,31>
53556
53557 ; Days in year
53558
53559 YRTAB:
53560 DB      200,166                ; Leap year
53561 DB      200,165
53562 DB      200,165
53563 DB      200,165
53564
53565 ; Days of each month
53566
53567 MONTAB:
53568 DB      31                    ; January
53569 february:
53570 DB      28                    ; February--reset each
53571                                ; time year changes
53572 DB      31                    ; March
53573 DB      30                    ; April
53574 DB      31                    ; May
53575 DB      30                    ; June
53576 DB      31                    ; July
53577 DB      31                    ; August
53578 DB      30                    ; September
53579 DB      31                    ; October
53580 DB      30                    ; November
53581 DB      31                    ; December
53582
53583 ;-----THE FOLL. BLOCK MOVED FROM TABLE SEG IN MS_TABLE.ASM-----
53584
53585 ; SYS init extended table, DOS 3.3 F.C. 5/29/86
53586
53587 SysInitTable:
53588 ;dw      SYSINITVAR
53589 dw      SYSINITVARS ; pointer to sysinit var
53590 dw      0 ; segment
53591 dw      COUNTRY_CDPG ; pointer to country tabl
53592 dw      0 ; segment of pointer
53593
53594 ;;;
53595 ; 17/01/2024 - Retro DOS v5.0
53596 ; PC DOS 7.1 IBMDOS.COM - DOSDATA:0D60h
53597 ;
53598 times 3 db 0
53599 TEMPSEG:
53600 dw      0
53601 redir_patch:
53602 db      0
53603
53604 ;%endif
53605
53606 ; 17/01/2024
53607 ;%if 1
53608
53609 ; M021-
53610 ;
53611 ; DosHashMA - This flag is set by seg_reinit when the DOS actually
53612 ; takes control of the HMA. When running, this word is a reliable
53613 ; indicator that the DOS is actually using HMA. You can't just use
53614 ; CS, because ROMDOS uses HMA with CS < F000.
53615
53616 DosHashMA:
53617 db      0
53618 FixExePatch:
53619 dw      0 ; M012
53620
53621 ;%endif
53622
53623 UnknownPatch:
53624 dw      0
53625 ;;;
53626
53627 ; 17/01/2024
53628 ;%if 0
53629
53630 ; DOS 3.3 F.C. 6/12/86
53631 ; FASTOPEN communications area DOS 3.3 F.C. 5/29/86
53632
53633 FastTable: ; a better name
53634 FastOpenTable:
53635 dw      2 ; number of entries
53636 dw      FastRet ; pointer to ret instr.
53637 dw      0 ; and will be modified by
53638 dw      FastRet ; FASTxxx when loaded in
53639 dw      0
53640
53641 ; DOS 3.3 F.C. 6/12/86
53642
53643 FastFlg: ; flags
53644 FastOpenFlg:
53645 db      0 ; don't change the foll: order
53646
53647 ;%endif
53648
53649 ; FastOpen_Ext_Info is used as a temporary storage for saving dirpos,dirsec
53650 ; and clusnum which are filled by DOS 3.nc when calling FastOpen Insert
53651 ; or filled by FastOpen when calling FastOpen Lookup
53652
53653 FastOpen_Ext_Info: ; label byte ;dirpos
53654 ;db      SIZE FASTOPEN_EXTENDED_INFO dup(0)
53655 ;times 11 db 0
53656 times FEI.size db 0
53657
53658 %endif ; 24/03/2024
53659
53660 ; Dir_Info_Buff is a dir entry buffer which is filled by FastOpen
53661 ; when calling FastOpen Lookup
53662
53663 Dir_Info_Buff: ; label byte
53664 ;db      SIZE dir_entry dup (0)
53665 ;times 32 db 0
53666 00000D7C 00<rep 20h>
53667 times dir_entry.size db 0
53668
53669 Next_Element_Start:
53670 dw      0 ; save next element start offset
53671
53672 ; 24/03/2024

```

```

53672 %if 0
53673 ; MSDOS 6.22 MSDOS.SYS - DOSDATA:0D68h
53674
53675 Del_ExtCluster:
53676 dw 0 ; for dos_delete
53677 %endif
53678
53679 ; The following is a stack and its pointer for interrupt 2F which is used
53680 ; by NLSFUNC. There is no significant use of this stack, we are just trying
53681 ; not to destroy the INT 21 stack saved for the user.
53682
53683 ; MSDOS 6.22 MSDOS.SYS - DOSDATA:0D6Ah
53684 ; 24/03/2024
53685 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0D9Eh
53686
53687 USER_SP_2F: ; LABEL WORD
53688 dw FAKE_STACK_2F
53689
53690 Packet_Temp: ; label word ; temporary packet used by readtime
53691 DOS_TEMP: ; label word ; temporary word
53692
53693 FAKE_STACK_2F:
53694 ; dw 14 dup (0) ; 12 register temporary storage
53695 times 14 dw 0
53696
53697 ; 24/03/2024 - Retro DOS v5.0
53698 ; PCDOS 7.1 IBMDOS.COM
53699 %if 0
53700 Hash_Temp: ; label word ; temporary word
53701 ; dw 4 dup (0) ; temporary hash table during config.sys
53702 times 4 dw 0
53703 %endif
53704
53705 SCAN_FLAG:
53706 db 0 ; flag to indicate key ALT_Q
53707
53708 DATE_FLAG:
53709 dw 0 ; flag to update the date
53710
53711 ; 24/03/2024
53712 %if 0
53713 ; MSDOS 5.0-6.22 MSDOS.SYS - DOSDATA:0D93h
53714
53715 FETCHI_TAG: ; label word ; OBSOLETE - no longer used
53716 dw 0 ; formerly part of IBM's piracy protection
53717
53718 MSG_EXTERROR: ; label DWORD ; for system message addr
53719 dd 0 ; for extended error
53720 dd 0 ; for parser
53721 dd 0 ; for critical error
53722 dd 0 ; for IFS
53723 dd 0 ; for code reduction
53724
53725 SEQ_SECTOR: ; label DWORD ; last sector read
53726 dd -1
53727 SC_SECTOR_SIZE:
53728 dw 0 ; sector size for SC
53729 SC_DRIVE:
53730 db 0 ; drive # for secondary cache
53731
53732 ; 17/01/2024
53733 CurSC_DRIVE:
53734 db -1 ; current SC drive
53735
53736 CurSC_SECTOR:
53737 dd 0 ; current SC starting sector
53738 SC_STATUS:
53739 dw 0 ; SC status word
53740 SC_FLAG:
53741 db 0 ; SC flag
53742
53743 %endif
53744 ; 24/03/2024
53745 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0DBFh
53746 ; (MSDOS 5.0-6.22 MSDOS.SYS - DOSDATA:0DB8h)
53747 AbsDskErr:
53748 dw 0 ; Storage for Abs dsk read/write err
53749
53750 NO_NAME_ID: ; label byte
53751 db 'NO NAME' ; null media id
53752
53753 ;hkn; moved from TABLE segment in kstrin.asm
53754
53755 KISTR001S: ; label byte ; 2/17/KK
53756 LOOKSIZ: db 0 ; 0 if byte, NZ if word 2/17/KK
53757 KISTR001E: ; label byte ; 2/17/KK
53758
53759 ; the nul device driver used to be part of the code. However, since the
53760 ; header is in the data, and the entry points are only given as an offset,
53761 ; the strategy and interrupt entry points must also be in the data now.
53762
53763 ; MSDOS 5.0-6.22 MSDOS.SYS - DOSDATA:0DC6h
53764 ; 24/03/2024
53765 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0DCDh
53766
53767 SNULDEV:
53768 ;procedure snuldev,far
53769 ;or word [es:bx+3],100h
53770 ; 17/12/2022
53771 ;or byte [es:bx+4],01h
53772 ; 05/01/2024 - Retro DOS v4.2 (*)
53773 ; (Original MSDOS and RetroDOS DATA address compatibility) (*)
53774 ;or byte [es:bx+SRHEAD.REQSTAT+1],(STDON>>8)
53775 ; 24/03/2024 (PCDOS 7.1 IBMDOS.COM)
53776 or word [es:bx+SRHEAD.REQSTAT],STDON ; set done bit
53777 ; 05/01/2024 - Retro DOS v5.0
53778 ;or byte [es:bx+SRHEAD.REQSTAT+1],(STDON>>8)
53779 INULDEV:
53780 retf ; must not be a return!
53781 ;endproc snuldev
53782
53783 ;M044
53784 ; Second part of save area for saving last para of windows memory
53785
53786 ; 17/01/2024
53787 ;winoldPatch2:
53788 ; db 8 dup (?) ; M044
53789 ; times 8 db 0
53790
53791 ; 24/03/2024 (PCDOS 7.1 IBMDOS.COM)
53792 db 0
53793
53794 UmbSave2:
53795 db 5 dup (?) ; M062

```

```

53795 00000DD5 00<rep 5h>          times 5 db 0
53796                                UmbSaveFlag:
53797 00000DDA 00                    db 0 ; M062
53798
53799                                ; DOSDATA:0DDbH
53800
53801 Mark2: ; label byte
53802
53803 ;IF2
53804 ; IF ((OFFSET MARK2) GT (OFFSET ERR_TABLE_21) )
53805 ; %OUT !DATA CORRUPTION!MARK2 OFFSET TOO BIG. RE-ORGANIZE DATA.
53806 ; ENDIF
53807 ;ENDIF
53808
53809 ;#####
53810 ;
53811 ; ** HACK FOR DOS 4.0 REDIR **
53812 ;
53813 ; The redir requires the following:
53814 ;
53815 ; ERR_TABLE_21 offset DDBH
53816 ; ERR_TABLE_24 offset E5BH
53817 ; ErrMap24 offset EABH
53818 ;
53819 ; WARNING! WARNING!
53820 ;
53821 ; MARK2 SHOULD NOT BE >= 0DDbH. IF SOME VARIABLE IS TO BE ADDED ABOVE DO SO
53822 ; WITHOUT VIOLATING THIS AND UPDATE THE FOLL. LINE
53823 ;
53824 ; CURRENTLY MARK2 = 0DD0H
53825 ;
53826 ;#####
53827 ;
53828 ;ORG 0DDbH
53829
53830 ; DOSDATA:0DDbH (MSDOS 6.21, MSDOS.SYS)
53831 ; 24/03/2024
53832 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0DDbH
53833
53834 ; -----
53835 ;
53836 ; The following table defines CLASS ACTION and LOCUS info for the INT 21H
53837 ; errors. Each entry is 4 bytes long:
53838 ;
53839 ; Err#,Class,Action,Locus
53840 ;
53841 ; A value of 0FFh indicates a call specific value (ie. should already
53842 ; be set). AN ERROR CODE NOT IN THE TABLE FALLS THROUGH TO THE CATCH ALL AT
53843 ; THE END, IT IS ASSUMES THAT CLASS, ACTION, LOCUS IS ALREADY SET.
53844 ;
53845 ; -----
53846
53847 ;ErrTab Macro err,class,action,locus
53848 ;ifidn <locus>,<0FFh>
53849 ; DB error_&err,errCLASS_&class,errACT_&action,0FFh
53850 ;ELSE
53851 ; DB error_&err,errCLASS_&class,errACT_&action,errLOC_&locus
53852 ;ENDIF
53853 ;ENDM
53854
53855 ERR_TABLE_21: ; LABEL BYTE
53856 DB error_invalid_function, errCLASS_Apperr, errACT_Abort, 0FFh
53857 DB error_file_not_found, errCLASS_NotFnd, errACT_User, errLOC_Disk
53858 DB error_path_not_found, errCLASS_NotFnd, errACT_User, errLOC_Disk
53859 DB error_too_many_open_files, errCLASS_OutRes, errACT_Abort, errLOC_Unk
53860 DB error_access_denied, errCLASS_Auth, errACT_User, 0FFh
53861 DB error_invalid_handle, errCLASS_Apperr, errACT_Abort, errLOC_Unk
53862 DB error_arena_trashed, errCLASS_Apperr, errACT_Panic, errLOC_Mem
53863 DB error_not_enough_memory, errCLASS_OutRes, errACT_Abort, errLOC_Mem
53864 DB error_invalid_block, errCLASS_Apperr, errACT_Abort, errLOC_Mem
53865 DB error_bad_environment, errCLASS_Apperr, errACT_Abort, errLOC_Mem
53866 DB error_bad_format, errCLASS_BadFmt, errACT_User, errLOC_Unk
53867 DB error_invalid_access, errCLASS_Apperr, errACT_Abort, errLOC_Unk
53868 DB error_invalid_data, errCLASS_BadFmt, errACT_Abort, errLOC_Unk
53869 DB error_invalid_drive, errCLASS_NotFnd, errACT_User, errLOC_Disk
53870 DB error_current_directory, errCLASS_Auth, errACT_User, errLOC_Disk
53871 DB error_not_same_device, errCLASS_Unk, errACT_User, errLOC_Disk
53872 DB error_no_more_files, errCLASS_NotFnd, errACT_User, errLOC_Disk
53873 DB error_file_exists, errCLASS_Already, errACT_User, errLOC_Disk
53874 DB error_sharing_violation, errCLASS_Locked, errACT_DlyRet, errLOC_Disk
53875 DB error_lock_violation, errCLASS_Locked, errACT_DlyRet, errLOC_Disk
53876 DB error_out_of_structures, errCLASS_OutRes, errACT_Abort, 0FFh
53877 DB error_invalid_password, errCLASS_Auth, errACT_User, errLOC_Unk
53878 DB error_cannot_make, errCLASS_OutRes, errACT_Abort, errLOC_Disk
53879 DB error_not_supported, errCLASS_BadFmt, errACT_User, errLOC_Net
53880 DB error_already_assigned, errCLASS_Already, errACT_User, errLOC_Net
53881 DB error_invalid_parameter, errCLASS_BadFmt, errACT_User, errLOC_Unk
53882 DB error_FAIL_I24, errCLASS_Unk, errACT_Abort, errLOC_Unk
53883 DB error_sharing_buffer_exceeded, errCLASS_OutRes, errACT_Abort, errLOC_Mem
53884 ; MSDOS 6.0
53885 DB error_handle_EOF, errCLASS_OutRes, errACT_Abort, errLOC_Unk ;AN000;
53886 DB error_handle_disk_Full, errCLASS_OutRes, errACT_Abort, errLOC_Unk ;AN000;
53887 DB error_sys_comp_not_loaded, errCLASS_Unk, errACT_Abort, errLOC_Disk ;AN001;
53888 DB 0FFh, 0FFh, 0FFh, 0FFh
53889
53890 ; MSDOS 3.3 (IBMDOS.COM, 1987) - offset 0D2Ah
53891 ;ERR_TABLE_21: db 1,7,4,0FFh
53892 ; db 2,8,3,2
53893 ; db 3,8,3,2
53894 ; db 4,1,4,1
53895 ; db 5,3,3,0FFh
53896 ; db 6,7,4,1
53897 ; db 7,7,5,5
53898 ; db 8,1,4,5
53899 ; db 9,7,4,5
53900 ; db 0Ah,7,4,5
53901 ; db 0Bh,9,3,1
53902 ; db 0Ch,7,4,1
53903 ; db 0Dh,9,4,1
53904 ; db 0Fh,8,3,2
53905 ; db 10h,3,3,2
53906 ; db 11h,0Bh,3,2
53907 ; db 12h,8,3,2
53908 ; db 50h,0Ch,3,2
53909 ; db 20h,0Ah,2,2
53910 ; db 21h,0Ah,2,2
53911 ; db 54h,1,4,0FFh
53912 ; db 56h,3,3,1
53913 ; db 52h,1,4,2
53914 ; db 32h,9,3,3
53915 ; db 55h,0Ch,3,3
53916 ; db 57h,9,3,1
53917 ; db 53h,0Dh,4,1
53918 ; db 24h,1,4,5

```

```

53919 ; MSDOS 6.0 (MSDOS 6.21)
53920 ;         db 26h,1,4,1
53921 ;         db 27h,1,4,1
53922 ;         db 5Ah,0Dh,4,2
53923 ; MSDOS 6.0 & MSDOS 3.3
53924 ;         db 0FFh,0FFh,0FFh,0FFh
53925
53926 ; DOSDATA:0E5Bh (MSDOS 6.21, MSDOS.SYS)
53927
53928 ; -----
53929 ;
53930 ; The following table defines CLASS ACTION and LOCUS info for the INT 24H
53931 ; errors. Each entry is 4 bytes long:
53932 ;
53933 ;         Err#,Class,Action,Locus
53934 ;
53935 ; A Locus value of 0FFh indicates a call specific value (ie. should already
53936 ; be set). AN ERROR CODE NOT IN THE TABLE FALLS THROUGH TO THE CATCH ALL AT
53937 ; THE END.
53938 ;
53939 ; -----
53940
53941 ERR_TABLE_24: ; LABEL    BYTE
53942 DB error_write_protect,    errCLASS_Media,    errACT_IntRet,    errLOC_Disk
53943 DB error_bad_unit,         errCLASS_Intrn,    errACT_Panic,     errLOC_Unk
53944 DB error_not_ready,       errCLASS_HrdFail, errACT_IntRet,    0FFh
53945 DB error_bad_command,     errCLASS_Intrn,    errACT_Panic,     errLOC_Unk
53946 DB error_CRC,             errCLASS_Media,    errACT_Abort,     errLOC_Disk
53947 DB error_bad_length,     errCLASS_Intrn,    errACT_Panic,     errLOC_Unk
53948 DB error_seek,           errCLASS_HrdFail, errACT_Retry,     errLOC_Disk
53949 DB error_not_DOS_disk,    errCLASS_Media,    errACT_IntRet,    errLOC_Disk
53950 DB error_sector_not_found, errCLASS_Media,    errACT_Abort,     errLOC_Disk
53951 DB error_out_of_paper,    errCLASS_TempSit, errACT_IntRet,    errLOC_SerDev
53952 DB error_write_fault,     errCLASS_HrdFail, errACT_Abort,     0FFh
53953 DB error_read_fault,      errCLASS_HrdFail, errACT_Abort,     0FFh
53954 DB error_gen_failure,     errCLASS_Unk,      errACT_Abort,     0FFh
53955 DB error_sharing_violation, errCLASS_Locked,   errACT_DlyRet,    errLOC_Disk
53956 DB error_lock_violation,  errCLASS_Locked,   errACT_DlyRet,    errLOC_Disk
53957 DB error_wrong_disk,      errCLASS_Media,    errACT_IntRet,    errLOC_Disk
53958 DB error_not_supported,   errCLASS_BadFmt,   errACT_User,      errLOC_Net
53959 DB error_FCB_unavailable, errCLASS_Apperr,   errACT_Abort,     errLOC_Unk
53960 DB error_sharing_buffer_exceeded, errCLASS_OutRes, errACT_Abort,     errLOC_Mem
53961 DB 0FFh,                  errCLASS_Unk,      errACT_Panic,     0FFh
53962
53963 ; MSDOS 3.3 (IBMDOS.COM, 1987) - Offset 0D9Eh
53964 ; ERR_TABLE_24:         db 13h,0Bh,7,2
53965 ;         db 14h,4,5,1
53966 ;         db 15h,5,7,0FFh
53967 ;         db 16h,4,5,1
53968 ;         db 17h,0Bh,4,2
53969 ;         db 18h,4,5,1
53970 ;         db 19h,5,1,2
53971 ;         db 1Ah,0Bh,7,2
53972 ;         db 1Bh,0Bh,4,2
53973 ;         db 1Ch,2,7,4
53974 ;         db 1Dh,5,4,0FFh
53975 ;         db 1Eh,5,4,0FFh
53976 ;         db 1Fh,0Dh,4,0FFh
53977 ;         db 20h,0Ah,2,2
53978 ;         db 21h,0Ah,2,2
53979 ;         db 22h,0Bh,7,2
53980 ;         db 32h,9,3,3
53981 ;         db 23h,7,4,1
53982 ;         db 24h,1,4,5
53983 ;         db 0FFh,0Dh,5,0FFh
53984
53985 ; DOSDATA:0EABh (MSDOS 6.21, MSDOS.SYS)
53986
53987 ; -----
53988 ;
53989 ; We need to map old int 24 errors and device driver errors into the new set
53990 ; of errors. The following table is indexed by the new errors
53991 ;
53992 ; -----
53993
53994 ;Public ErrMap24
53995 ErrMap24: ; Label    BYTE
53996 DB error_write_protect    ; 0
53997 DB error_bad_unit         ; 1
53998 DB error_not_ready        ; 2
53999 DB error_bad_command      ; 3
54000 DB error_CRC              ; 4
54001 DB error_bad_length      ; 5
54002 DB error_seek            ; 6
54003 DB error_not_DOS_disk    ; 7
54004 DB error_sector_not_found ; 8
54005 DB error_out_of_paper    ; 9
54006 DB error_write_fault     ; A
54007 DB error_read_fault      ; B
54008 DB error_gen_failure      ; C
54009 DB error_gen_failure      ; D RESERVED
54010 DB error_gen_failure      ; E RESERVED
54011 DB error_wrong_disk       ; F
54012
54013 ;ErrMap24: db 13h, 14h, 15h, 16h, 17h, 18h, 19h, 1Ah
54014 ;         db 1Bh, 1Ch, 1Dh, 1Eh, 1Fh, 1Fh, 1Fh, 22h
54015
54016 ErrMap24End: ; LABEL    BYTE
54017
54018 ; DOSDATA:0EBBh (MSDOS 6.21, MSDOS.SYS)
54019
54020 ; 09/03/2024 - Retro DOS v5.0
54021 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0EBBh (SPECIAL_VERSION)
54022
54023 ; MSDOS 5.0 MSDOS.SYS - DOSDATA:0EBBh (FIRST_BUFF_ADDR)
54024 ; MSDOS 6.22 MSDOS.SYS - DOSDATA:0EBBh (FIRST_BUFF_ADDR)
54025 ; Windows ME IO.SYS - DOSDATA:0EBBh (SPECIAL_VERSION)
54026
54027 ; -----
54028 ;
54029 ; 27/04/2019 - Retro DOS v4.0
54030
54031 ; 17/01/2024
54032 ; FIRST_BUFF_ADDR:
54033 ;         dw 0 ; first buffer address
54034
54035 SPECIAL_VERSION:
54036         dw 0 ; AN006; used by INT 2F 47H
54037
54038 ; 25/03/2024 - Retro DOS v5.0
54039 ; PCDOS 7.1 IBMDOS.COM
54040 %if 0
54041 FAKE_COUNT:
54042         times 255 db 0 ; AN008; fake version count

```

```

54043 %endif
54044
54045 OLD_FIRSTCLUS:
54046     dw      0 ;AN011; save old first cluster for fastopen
54047
54048 ; -----
54049
54050 ;smr; moved from TABLE segment in exec.asm
54051
54052 exec_init_SP: dw 0
54053 exec_init_SS: dw 0
54054 exec_init_IP: dw 0
54055 exec_init_CS: dw 0
54056
54057 exec_signature:
54058     dw      0 ; must contain 4D5A (yay zibo!)
54059 exec_len_mod_512:
54060     dw      0 ; low 9 bits of length
54061
54062 exec_pages:
54063     dw      0 ; number of 512b pages in file
54064 exec_rle_count:
54065     dw      0 ; count of reloc entries
54066 exec_par_dir:
54067     dw      0 ; number of paragraphs before image
54068 exec_min_BSS:
54069     dw      0 ; minimum number of para of BSS
54070 exec_max_BSS:
54071     dw      0 ; max number of para of BSS
54072 exec_SS:
54073     dw      0 ; stack of image
54074 exec_SP:
54075     dw      0 ; SP of image
54076 exec_chksum:
54077     dw      0 ; checksum of file (ignored)
54078 exec_IP:
54079     dw      0 ; IP of entry
54080 exec_CS:
54081     dw      0 ; CS of entry
54082 exec_rle_table:
54083     dw      0 ; byte offset of reloc table
54084
54085 exec_header_len equ $-exec_signature ;PBUGBUG
54086
54087 ;smr; eom
54088 ; -----
54089
54090 ;SR;
54091 ; WIN386 instance table for DOS
54092
54093 win386_Info:
54094     ;db      3,0
54095     ; 17/01/2024 - PC DOS 7.1 IBMDOS.COM - DOSDATA:0EE1h
54096     db      4,0 ; WIN386_SIS version
54097     dd      0 ; .Next_Dev_Ptr
54098 win386_Inf_Virt_Dev_Ptr:
54099     dd      0 ; .Virt_Dev_File_Ptr
54100     dd      0 ; .Reference_Data
54101
54102 Instance_Data_Ptr:
54103     dw      Instance_Table, 0
54104     ; 17/01/2024 - PC DOS 7.1 IBMDOS.COM
54105     ;
54106     dw      Unknown_Table, 0 ; (what is this and what for ?)
54107     ; (win386_IIS.size)
54108     ;
54109 Instance_Table:
54110     dw      CONTPOS,0,2
54111     dw      BCON,0,4
54112     dw      CARPOS,0,106h
54113     dw      CHARCO,0,1
54114     dw      exec_init_SP,0,34 ; M074
54115     dw      UMBFLAG,0,1 ; M019
54116     dw      UMB_HEAD,0,2 ; M019
54117     ;
54118     ; 17/01/2024 - PC DOS 7.1 IBMDOS.COM
54119     dw      DOS_FLAG,0C9h,1
54120     dw      INDOS_FLAG,0C9h,1 ; (what for a 2nd INDOS flag, windows?)
54121     dw      DEVIO_IN_PROGRESS,0C9h,1
54122     ; "devio call in progress" status flag ptr
54123     ;
54124     dw      0, 0
54125     ;
54126     ;
54127     ; 17/01/2024 - PC DOS 7.1 IBMDOS.COM
54128     dw      0FFFFh
54129     dw      0FFFFh
54130 CLOFATENTRY_HW:
54131     dw      0FFFFh
54132 Unknown_Table:
54133     dw      0
54134     dw      0C9h
54135     dw      SFTABL ; DOSDATA:00CCh
54136     dw      CARPOS
54137     dw      0C9h
54138     dw      UNKNOWN1 ; ? (points to DOSDATA:114Dh)
54139     dw      0
54140     dw      0
54141     ;
54142     ;
54143 ; M001; SR;
54144 ; M001; On DOSMGR call ( cx == 0 ), we need to return a table of offsets of
54145 ; M001; some DOS variables. Note that the only really important variable in
54146 ; M001; this is User_Id. The other variables are needed only to patch stuff
54147 ; M001; which does not need to be done in DOS 5.0.
54148
54149 ; 29/12/2022
54150 ; (MSDOS 6.21 MSDOS.SYS - DOSDATA:1022h)
54151 ; 25/03/2024
54152 ; PC DOS 7.1 IBMDOS.COM - DOSDATA:0F4Dh
54153
54154 win386_DOSVars:
54155     db      5 ;Major version 5 ; M001
54156     db      0 ;Minor version 0 ; M001
54157     dw      SAVEDS ; M001
54158     dw      SAVEBX ; M001
54159     dw      INDOS ; M001
54160     dw      USER_ID ; M001
54161     dw      CritPatch ; M001
54162     dw      UMB_HEAD ; M012
54163
54164 ;SR;
54165 ; Flag to indicate whether WIN386 is running or not
54166

```

```

54167      Iswin386:
54168 00000F5B 00      db      0
54169
54170      ; 09/03/2024 (PCDOS 7.1 IBMDOS.COM)
54171
54172 00000F5C 00      db      0
54173
54174      ; 09/03/2024 (PCDOS 7.1 IBMDOS.COM)
54175 %if 0
54176
54177 ;M018
54178 ; This variable contains the path to the vxD device needed for win386
54179
54180 ; 09/01/2024
54181 ;VxDpath: db      'c:\wina20.386',0      ;M018
54182
54183 ;End WIN386 support
54184
54185 ; -----
54186
54187 ;SR;
54188 ; These variables have been added for the special lie support for device
54189 ;drivers.
54190 ;
54191
54192 DriverLoad:
54193 db      1      ;initialized to do special handling
54194 BiosDataPtr:
54195 dd      0
54196
54197 %endif
54198
54199 ; 25/03/2024
54200 %if 1
54201 ; 29/12/2022 - Retro DOS v4.1
54202 %if 0
54203
54204 ; 27/04/2019 - Retro DOS v4.0
54205 ; 04/11/2022
54206 ; DOSDATA:1044h (MSDOS 6.21 & MSDOS 5.0, MSDOS.SYS)
54207
54208 ; -----
54209 ; Patch for Sidekick
54210 ;
54211 ; A documented method for finding the offset of the Errormode flag in the
54212 ; dos swappable data area if for the app to scan in the dos segment (data)
54213 ; for the following sequence of instructions.
54214 ;
54215 ; Ref: Part C, Article 11, pg 356 of MSDOS Encyclopedia
54216 ;
54217 ; The Offset of Errormode flag is 0320h
54218 ;
54219 ; -----
54220
54221 00000F5D 36F6062003FF      db      036h, 0F6h, 06h, 020h, 03h, 0FFh ; test ss:[errormode], -1
54222 00000F63 750C      db      075h, 0Ch      ; jnz NearLabel
54223 00000F65 36FF365803      db      036h, 0FFh, 036h, 058h, 03h      ; push ss:[NearWord]
54224 00000F6A CD28      db      0CDh, 028h      ; int 28h
54225
54226 ; -----
54227 ; Patch for PortOfEntry - M036
54228 ;
54229 ; PortOfEntry by Sector Technology uses an un documented way of determining
54230 ; the offset of Errormode flag. The following patch is to support them in
54231 ; DOS 5.0. The corresponding code is actually in msdisp.asm
54232 ;
54233 ; -----
54234
54235 00000F6C 803E200300      db      080h, 03Eh, 020h, 03h, 00h      ; cmp [errormode], 0
54236 00000F71 7537      db      075h, 037h      ; jnz NearLabel
54237 00000F73 BCA00A      db      0BCh, 0A0h, 0Ah      ; mov sp, dosdata:iostack
54238
54239 %endif ; 29/12/2022
54240
54241 ; DOSDATA:105Dh (MSDOS 6.21, MSDOS.SYS)
54242 ; 25/03/2024
54243 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0F76h
54244
54245 ; -----
54246
54247 ;*** New FCB Implementation
54248 ; This variable is used as a cache in the new FCB implementation to remember
54249 ; the address of a local SFT that can be recycled for a regenerate operation
54250
54251 00000F76 00000000      LocalSFT: dd      0      ; 0 to indicate invalid pointer
54252
54253 ;DOSDATA ENDS
54254
54255 ;=====
54256 ; LMSTUB.ASM (MSDOS 6.0, 1991)
54257 ;=====
54258 ; 27/04/2019 - Retro DOS 4.0
54259 ; 25/03/2024 - Retro DOS 5.0
54260
54261 ;DOSDATA SEGMENT WORD PUBLIC 'DATA'
54262
54263 ; -----
54264 ; Low Memory Stub for DOS when DOS runs in HMA
54265 ; -----
54266
54267 ;db      90h
54268
54269 ;EVEN
54270 align 2
54271
54272 ; DOSDATA:1062h (MSDOS 5.0-6.22, MSDOS.SYS)
54273 ; 25/03/2024
54274 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0F7Ah
54275
54276 DOSINTTABLE:      ; LABEL DWORD
54277
54278 ;DW      OFFSET DOSCODE:DIVOV      , 0
54279 ;DW      OFFSET DOSCODE:QUIT      , 0
54280 ;DW      OFFSET DOSCODE:COMMAND      , 0
54281 ;DW      OFFSET DOSCODE:ABSDRD      , 0
54282 ;DW      OFFSET DOSCODE:ABSDWRT      , 0
54283 ;DW      OFFSET DOSCODE:Stay_resident      , 0
54284 ;DW      OFFSET DOSCODE:INT2F      , 0
54285 ;DW      OFFSET DOSCODE:CALL_ENTRY      , 0
54286 ;DW      OFFSET DOSCODE:IRETT      , 0
54287
54288 00000F7A [D55F]0000      dw      DIVOV      , 0 ; DOSINTTABLE+0
54289 00000F7E [C502]0000      dw      QUIT      , 0 ; DOSINTTABLE+4
54290 00000F82 [F102]0000      dw      COMMAND      , 0 ; DOSINTTABLE+8

```



```

54291 00000F86 [2D05]0000      dw      ABSDRD      , 0 ; DOSINTTABLE+12
54292 00000F8A [DF05]0000      dw      ABSDWRT     , 0 ; DOSINTTABLE+16
54293 00000F8E [3E72]0000      dw      STAY_RESIDENT , 0 ; DOSINTTABLE+20
54294 00000F92 [3A07]0000      dw      INT2F       , 0 ; DOSINTTABLE+24
54295 00000F96 [CC02]0000      dw      CALL_ENTRY    , 0 ; DOSINTTABLE+28
54296
54297      ; 25/03/2024 - Retro DOS v5.0
54298      ; PCDOS 7.1 IBMDOS.COM
54299      %if 0
54300      dw      IRETT      , 0 ; DOSINTTABLE+32
54301      %endif
54302
54303 00000F9A 0000      SS_Save: dw 0      ; save user's stack segment
54304 00000F9C 0000      SP_Save: dw 0      ; save user's stack offset
54305
54306      ;
54307      ;
54308      ; LOW MEM STUB:
54309      ;
54310      ; The low mem stub contains the entry points into DOS for all interrupts
54311      ; handled by DOS. This stub is installed if the user specifies that the
54312      ; DOS load in HIMEM. Each entry point does this.
54313      ;
54314      ; 1. if jmp to 8 has been patched out
54315      ; 2. if A20 OFF
54316      ; 3. Enable A20
54317      ; 4. else
54318      ; 5. just go to dos entry
54319      ; 6. endif
54320      ; 7. else
54321      ; 8. just go to dos entry
54322      ; 9. endif
54323      ;
54324      ;
54325      ;
54326      ; 27/04/2019 - Retro DOS v4.0
54327      ;
54328      ; DOSDATA:108Ah (MSDOS 6.21, MSDOS.SYS)
54329      ;
54330      ; 25/03/2024 - Retro DOS v5.0
54331      ; PCDOS 7.1 IBMDOS.COM - DOSDATA:0F9Eh
54332      ;
54333      ;
54334      ;
54335      ; DIVIDE BY 0 handler
54336      ;
54337      ;
54338      ;
54339      ;
54340      ;ldivov:
54341      ; The following jump, skipping the XMS calls will be patched to
54342      ; NOPS by SEG_REINIT if DOS successfully loads high. This jump is
54343      ; needed because the stub is installed even before the XMS driver
54344      ; is loaded if the user specifies dos=high in the config.sys
54345      ;i0patch:
54346      jmp      short divov_cont
54347
54348      call      EnsureA20ON      ; we must turn on A20 if OFF
54349      divov_cont:
54350      jmp      far [cs:DOSINTTABLE] ; jmp to DOS
54351      ;
54352      ;
54353      ; INT 20h Handler
54354      ;
54355      ; Here we do not have to set up the stack to return here as the abort call
54356      ; will return to the address after the int 21 ah=4b call. This would be the
54357      ; common exit point if A20 had been OFF (for TOGGLE DOS) and the A20 line
54358      ; will be restored then.
54359      ;
54360      ;
54361      ;
54362      ;quit:
54363      ; The following jump, skipping the XMS calls will be patched to
54364      ; NOPS by SEG_REINIT if DOS successfully loads high. This jump is
54365      ; needed because the stub is installed even before the XMS driver
54366      ; is loaded if the user specifies dos=high in the config.sys
54367      ;i20patch:
54368      jmp      short quit_cont
54369
54370      call      EnsureA20ON      ; we must turn on A20 if OFF
54371      quit_cont:
54372      jmp      far [cs:DOSINTTABLE+4]; jump to DOS
54373      ;
54374      ;
54375      ;
54376      ; INT 21h Handler
54377      ;
54378      ;
54379      ;
54380      ;lcommand:
54381      ; The following jump, skipping the XMS calls will be patched to
54382      ; NOPS by SEG_REINIT if DOS successfully loads high. This jump is
54383      ; needed because the stub is installed even before the XMS driver
54384      ; is loaded if the user specifies dos=high in the config.sys
54385      ;i21patch:
54386      jmp      short command_cont
54387
54388      call      EnsureA20ON      ; we must turn on A20 if OFF
54389      command_cont:
54390      jmp      far [cs:DOSINTTABLE+8]; jmp to DOS
54391      ;
54392      ;
54393      ;
54394      ; INT 25h
54395      ;
54396      ;
54397      ;
54398      ;labsdrd:
54399      ; The following jump, skipping the XMS calls will be patched to
54400      ; NOPS by SEG_REINIT if DOS successfully loads high. This jump is
54401      ; needed because the stub is installed even before the XMS driver
54402      ; is loaded if the user specifies dos=high in the config.sys
54403      ;i25patch:
54404      jmp      short absdrd_cont
54405
54406      call      EnsureA20ON      ; we must turn on A20 if OFF
54407      absdrd_cont:
54408      jmp      far [cs:DOSINTTABLE+12] ; jmp to DOS
54409      ;
54410      ;
54411      ;
54412      ; INT 26h
54413      ;
54414      ;

```

```

54415
54416
54417
54418
54419
54420
54421
54422 00000FC6 EB03
54423
54424 00000FC8 E8BA00
54425
54426 00000FCB 2EFF2E[8A0F]
54427
54428
54429
54430
54431
54432
54433
54434
54435
54436
54437
54438
54439
54440 00000FD0 EB03
54441
54442 00000FD2 E8B000
54443
54444 00000FD5 2EFF2E[8E0F]
54445
54446
54447
54448
54449
54450
54451
54452
54453
54454
54455
54456
54457
54458 00000FDA EB03
54459
54460 00000FDC E8A600
54461
54462 00000FDF 2EFF2E[920F]
54463
54464
54465
54466
54467
54468
54469
54470
54471
54472
54473
54474
54475
54476 00000FE4 EB03
54477
54478 00000FE6 E89C00
54479
54480 00000FE9 2EFF2E[960F]
54481
54482
54483
54484
54485
54486
54487
54488
54489
54490
54491
54492
54493
54494
54495
54496
54497
54498
54499
54500
54501
54502
54503
54504
54505
54506
54507
54508
54509
54510
54511
54512
54513
54514
54515
54516 00000FEE 00000000
54517 00000FF2 00000000
54518
54519
54520
54521
54522
54523
54524
54525
54526
54527
54528
54529 00000FF6 2E8F06[EE0F]
54530 00000FFB 2E8F06[F00F]
54531
54532 00001000 CD23
54533
54534
54535
54536 00001002 E88000
54537
54538 00001005 2EFF2E[EE0F]

; The following jump, skipping the XMS calls will be patched to
; NOPS by SEG_REINIT if DOS successfully loads high. This jump is
; needed because the stub is installed even before the XMS driver
; is loaded if the user specifies dos=high in the config.sys
labswrt:
    jmp     short absdwrt_cont

    call    EnsureA20ON          ; we must turn on A20 if OFF
absdwrt_cont:
    jmp     far [cs:DOSINTTABLE+16] ; jmp to DOS

;-----
;
; INT 27h
;
;-----

; The following jump, skipping the XMS calls will be patched to
; NOPS by SEG_REINIT if DOS successfully loads high. This jump is
; needed because the stub is installed even before the XMS driver
; is loaded if the user specifies dos=high in the config.sys
istay_resident:
    jmp     short sr_cont

    call    EnsureA20ON          ; we must turn on A20 if OFF
sr_cont:
    jmp     far [cs:DOSINTTABLE+20] ; jmp to DOS

;-----
;
; INT 2Fh
;
;-----

; The following jump, skipping the XMS calls will be patched to
; NOPS by SEG_REINIT if DOS successfully loads high. This jump is
; needed because the stub is installed even before the XMS driver
; is loaded if the user specifies dos=high in the config.sys
int2f:
    jmp     short int2f_cont

    call    EnsureA20ON          ; we must turn on A20 if OFF
int2f_cont:
    jmp     far [cs:DOSINTTABLE+24] ; jmp to DOS

;-----
;
; CPM entry
;
;-----

; The following jump, skipping the XMS calls will be patched to
; NOPS by SEG_REINIT if DOS successfully loads high. This jump is
; needed because the stub is installed even before the XMS driver
; is loaded if the user specifies dos=high in the config.sys
call_entry:
    jmp     short callentry_cont

    call    EnsureA20ON          ; we must turn on A20 if OFF
callentry_cont:
    jmp     far [cs:DOSINTTABLE+28] ; jmp to DOS

;-----
;
; 25/03/2024 - Retro DOS v5.0
; PCDOS 7.1 IBMDOS.COM
%if 0

lirett:
    iret

%endif

;-----
;
; LowIntXX:
;
; Interrupts from DOS that pass control to a user program must be done from
; low memory, as the user program may change the state of the A20 line or
; they may require that the A20 line be OFF. The following piece of code is
; far call'd from the following places in DOS:
;
; 1. msctrlc.asm where dos issues an int 23h (ctrlc)
; 2. msctrlc.asm where dos issues an int 24h (critical error)
; 3. msctrlc.asm where dos issues an int 28h (idle int)
;
; The int 23 and int 24 handlers may decide to do a far return instead of an
; IRET and leave the flags on the stack. Therefore we save the return address
; before doing the ints and then do a far jump back into DOS.
;
;-----

; 25/03/2024 - Retro DOS v5.0
; PCDOS 7.1 IBMDOS.COM - DOSDATA:0FEEh
; (MSDOS 5.0-6.22 MSDOS.SYS - DOSDATA:10DBh)

DosRetAddr23:    dd    0
DosRetAddr24:    dd    0

; 25/03/2024 - Retro DOS v5.0
; PCDOS 7.1 IBMDOS.COM
%if 0
DosRetAddr28:    dd    0
%endif

; Execute int 23h from low memory
LowInt23:
; save the return address that is on
; the stack
    pop     word [cs:DosRetAddr23]
    pop     word [cs:DosRetAddr23+2]

    int     23h ; ctrl C
; turn on A20 it has been turned OFF
; by int 28/23/24 handler.

    call    EnsureA20ON          ; M011: we must turn on A20 if OFF

    jmp     far [cs:DosRetAddr23] ; jump back to DOS

```



```

54539
54540
54541 ; Execute int 24h from low memory
54542 LowInt24:
54543 ; save the return address that is on
54544 ; the stack
54545 0000100A 2E8F06[F20F] pop word [cs:DosRetAddr24]
54546 0000100F 2E8F06[F40F] pop word [cs:DosRetAddr24+2]
54547
54548 00001014 CD24 int 24h ; crit error
54549 ; turn on A20 it has been turned OFF
54550 ; by int 28/23/24 handler.
54551
54552 00001016 E86C00 call EnsureA20ON ; M011: we must turn on A20 if OFF
54553
54554 00001019 2EFF2E[F20F] jmp far [cs:DosRetAddr24] ; jump back to DOS
54555
54556 ; Execute int 28h from low memory
54557 LowInt28:
54558 int 28h ; idle int
54559 ; turn on A20 it has been turned OFF
54560 ; by int 28/23/24 handler.
54561
54562 00001020 E86200 call EnsureA20ON ; M011: we must turn on A20 if OFF
54563
54564 00001023 CB retf
54565
54566 ; DOSDATA:1115h (MSDOS 6.21, MSDOS.SYS)
54567
54568 ;-----
54569 ;
54570 ; int 21 ah=4b (exec) call will jump to the following label before xferring
54571 ; control to the exec'd program. We turn off A20 inorder to allow programs
54572 ; that have been packed by the faulty exepack utility to unpack correctly.
54573 ; This is so because exepac'd programs rely on address wrap.
54574 ;
54575 ;-----
54576
54577 ; 25/03/2024 - Retro DOS v5.0
54578 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:1024h
54579
54580 disa20_xfer:
54581 00001024 E84800 call XMMDisableA20 ; disable A20
54582
54583 ; Look at msproc.asm at label exec_go for understanding the following:
54584
54585 ; DS:SI points to entry point
54586 ; AX:DI points to initial stack
54587 ; DX has PDB pointer
54588 ; BX has initial AX value
54589
54590 00001027 FA cli
54591 00001028 2EC606[2103]00 mov byte [cs:INDOS],0 ; SS Override
54592
54593 ; 25/03/2024 (PCDOS 7.1 IBMDOS.COM)
54594 ;;;
54595 0000102E 36C606[B812]00 mov byte [ss:INDOS_FLAG],0
54596
54597 ;;;
54598 00001034 8ED0 mov SS,AX ; set up user's stack
54599 00001036 89FC mov SP,DI ; and SP
54600 00001038 FB sti
54601
54602 00001039 1E push DS ; fake long call to entry
54603 0000103A 56 push SI
54604 0000103B 8EC2 mov ES,DX ; set up proper seg registers
54605 0000103D 8EDA mov DS,DX
54606 0000103F 89D8 mov AX,BX ; set up proper AX
54607 00001041 CB retf
54608
54609 ;-----
54610 ;
54611 ; M003:
54612 ;
54613 ; If an int 21 ah=25 call is made immediately after an exec call, DOS will
54614 ; come here, turn A20 OFF restore user stack and registers before returning
54615 ; to user. This is done in dos\msdisp.asm. This has been done to support
54616 ; programs compiled with MS PASCAL 3.2. See under TAG M003 in DOSSYM.INC for
54617 ; more info.
54618
54619 ; Also at this point DS is DOSDATA. So we can assume DS DOSDATA. Note that
54620 ; SS is also DOS stack. It is important that we do the XMS call on DOS's
54621 ; stack to avoid additional stack overhead for the user.
54622 ;
54623 ;-----
54624
54625 disa20_iret:
54626 00001042 E82A00 call XMMDisableA20
54627 00001045 FE0E[2103] dec byte [INDOS]
54628
54629 ; 25/03/2024 (PCDOS 7.1 IBMDOS.COM)
54630 ;;;
54631 00001049 FE0E[B812] dec byte [INDOS_FLAG]
54632
54633 ;;;
54634 0000104D 8E16[8605] mov SS,[USER_SS] ; restore user stack
54635 00001051 8B26[8405] mov SP,[USER_SP]
54636 00001055 89E5 mov BP,SP
54637 ;mov [BP+user_env.user_AX],AL
54638 00001057 884600 mov [bp],al
54639
54640 ; 25/03/2024
54641 %if 0
54642 mov AX,[NSP]
54643 [USER_SP],AX
54644 mov AX,[NSS]
54645 [USER_SS],AX
54646 %else
54647 ; 25/03/2024 (PCDOS 7.1 IBMDOS.COM)
54648 ;;;
54649 ;;;
54650 ;les ax, dword ptr ds:NSS
54651 0000105A C406[F005] les ax,[NSS]
54652 0000105E 8C06[8405] mov [USER_SP],es
54653 00001062 A3[8605] mov [USER_SS],ax
54654 ;;;
54655 %endif
54656 00001065 58 pop AX ; restore user regs
54657 00001066 5B pop BX
54658 00001067 59 pop CX
54659 00001068 5A pop DX
54660 00001069 5E pop SI
54661 0000106A 5F pop DI
54662 0000106B 5D pop BP

```

```

54663 0000106C 1F      pop     DS
54664 0000106D 07      pop     ES
54665      liret:      ; 25/03/2024 (PCDOS 7.1 IBMDOS.COM)
54666 0000106E CF      ired
54667
54668      ;*****
54669      ;***XMMDisableA20 - switch 20th address line
54670      ;
54671      ; This routine is used to disable the 20th address line in
54672      ; the system using XMM calls.
54673      ;
54674      ; ENTRY none ;ds = _DATA
54675      ; EXIT A20 line disabled
54676      ; USES NOTHING
54677      ;*****
54678
54679      XMMDisableA20:
54680      push     bx
54681 0000106F 53      push     ax
54682 00001070 50      ;mov     ah,XMM_LOCAL_DISABLE_A20
54683      mov     ah,6
54684 00001071 B406      call    far [cs:XMMcontrol]
54685 00001073 2EFF1E[7B10]  pop     ax
54686 00001078 58      pop     bx
54687 00001079 5B      retn
54688 0000107A C3
54689
54690      ; The entry point in the BIOS XMS driver is defined here.
54691
54692      XMMcontrol:
54693 0000107B 00000000      dd     0
54694
54695      ;-----
54696      ; 24/03/2024 - Retro DOS v5.0
54697      ; PCDOS 7.1 IBMDOS.COM - DOSDATA:107Fh
54698      ;;;
54699 0000107F 00000000      dd     0
54700      SC_STATUS:
54701 00001083 0000      dw     0
54702      ;;;
54703
54704      ;-----
54705      ;***EnsureA20ON - Ensures that A20 is ON
54706      ;
54707      ; This routine is used to query the A20 state in
54708      ; the system using XMM calls.
54709      ;
54710      ; ENTRY: none
54711      ;
54712      ; EXIT : A20 will be ON
54713      ;
54714      ; USES : NONE
54715      ;-----
54716
54717      ; 19/09/2023
54718      ; LowMemory: ; label dword ; Set equal to 0000:0080
54719      ; dw 00080h
54720      ; dw 00000h
54721      ;
54722      ; HighMemory: ; label dword ; Set equal to FFFF:0090
54723      ; dw 00090h
54724      ; dw 0FFFFh
54725      ;
54726      ; 25/03/2024 - Retro DOS v5.0
54727      ; PCDOS 7.1 IBMDOS.COM - DOSDATA:1085h
54728      ; (MSDOS 5.0-6.22 MSDOS.SYS - DOSDATA:116Fh)
54729
54730      EnsureA20ON:
54731      pushf
54732      push     ds
54733 00001085 9C      push     es
54734 00001086 1E      push     cx
54735 00001087 06      push     si
54736 00001088 51      push     di
54737 00001089 56
54738 0000108A 57
54739
54740      ; 19/09/2023
54741      ; lds si,[cs:LowMemory] ; Compare the 4 words at 0000:0080
54742      ; les di,[cs:HighMemory] ; with the 4 at FFFF:0090
54743      ;
54744      ; 25/03/2024
54745      ; PCDOS 7.1 IBMDOS.COM
54746      ;;;
54747 0000108B 31F6      xor     si,si
54748 0000108D 8EDE      mov     ds,si
54749 0000108F 4E      dec     si
54750 00001090 BF9000      mov     di,90h ; 0FFFFh:0090h ; HighMemory
54751 00001093 8EC6      mov     es,si
54752 00001095 BE8000      mov     si,80h ; 0000h:0080h ; LowMemory
54753      ;;;
54754
54755 00001098 B90400      mov     cx,4
54756 0000109B FC      cld
54757 0000109C F3A7      repe    cmpsw
54758
54759 0000109E 7407      jz     short EA20_OFF
54760      EA20_RET:
54761 000010A0 5F      pop     di
54762 000010A1 5E      pop     si
54763 000010A2 59      pop     cx
54764 000010A3 07      pop     es
54765 000010A4 1F      pop     ds
54766 000010A5 9D      popf
54767 000010A6 C3      retn
54768
54769      EA20_OFF:
54770      ; We are going to do the XMS call on the DOS's AuxStack.
54771      ; NOTE: ints are disabled at this point.
54772
54773 000010A7 53      push     bx
54774 000010A8 50      push     ax
54775
54776      ; 25/03/2024
54777      %if 0
54778      mov     ax,ss ; save user's stack pointer
54779      mov     [cs:SS_Save],ax
54780      mov     [cs:SP_Save],sp
54781      mov     ax,cs
54782      mov     ss,ax
54783      %else
54784      ; 25/03/2024 (PCDOS 7.1 IBMDOS.COM)
54785      ;;;
54786 000010A9 8CC8      mov     ax,cs

```

```

54787 000010AB 2E8C16[9A0F]      mov     [cs:SS_Save],ss
54788 000010B0 2E8926[9C0F]      mov     [cs:SP_Save],sp
54789 000010B5 8ED0                mov     ss,ax
54790                                ;;
54791                                %endif
54792 000010B7 BC[A007]          mov     sp,AUXSTACK
54793                                ; ss:sp -> DOSDATA:AuxStack
54794                                ;mov     ah,XMM_LOCAL_ENABLE_A20
54795 000010BA B405          mov     ah,5
54796 000010BC 2EFF1E[7B10]      call    far [cs:XMMcontrol]
54797 000010C1 09C0          or     ax,ax
54798 000010C3 740E          jz     short XMMerror          ; AX = 0 fatal error
54799                                ;
54800                                ;mov     ax,[cs:SS_Save]          ; restore user stack
54801                                ;mov     ss,ax
54802                                ; 25/03/2024 (PCDOS 7.1 IBMDOS.COM)
54803                                ;;
54804 000010C5 2E8E16[9A0F]      mov     ss,[cs:SS_Save]
54805                                ;;
54806 000010CA 2E8B26[9C0F]      mov     sp,[cs:SP_Save]
54807                                ;
54808 000010CF 58            pop     ax
54809 000010D0 5B            pop     bx
54810                                ;
54811 000010D1 EB CD          jmp     short EA20_RET
54812                                ;
54813 XMMerror:                                ; M006 - Start
54814 000010D3 B40F          mov     ah,0Fh          ; get video mode
54815 000010D5 CD10          int     10h
54816 000010D7 3C07          cmp     al,7          ; Q: are we an MDA
54817 000010D9 7405          je     short XMMcont    ; Y: do not change mode
54818                                ;xor     ah,ah ; 0          ; set video mode
54819                                ;mov     al,02h          ; 80 x 25 text
54820                                ; 25/03/2024 (PCDOS 7.1 IBMDOS.COM)
54821                                ;;
54822 000010DB B80200        mov     ax,2
54823                                ;;
54824 000010DE CD10          int     10h
54825 XMMcont:                                ;
54826 000010E0 B80005        mov     ax,05h          ; set display page
54827                                ;xor     al,al          ; page 0
54828                                ; 25/03/2024 (PCDOS 7.1 IBMDOS.COM)
54829                                ;;
54830 000010E3 B80005        mov     ax,500h
54831                                ;;
54832 000010E5 BE[1312]      mov     si,XMMERRMSG
54833 000010E7 0E            push    cs
54834 000010E9 1F            pop     ds
54835 000010EA FC            cld          ; clear direction flag
54836 XMMprnt:                                ;
54837 000010EB AC            lodsb
54838 000010ED 3C24          cmp     al,'$'          ; indicates end of XMMERRMSG
54839 000010EF 7409          jz     short XMMstall    ; function 0Eh
54840 000010F0 B40E          mov     ah,0Eh
54841 000010F2 BB0700        mov     bx,7
54842 000010F4 CD10          int     10h
54843 000010F6 EB F2          jmp     short XMMprnt
54844                                ;
54845 XMMstall:                                ;
54846 000010F8 FB            sti          ; allow the user to warm boot
54847 000010FA EB FD          jmp     short XMMstall    ; M006 - End
54848                                ;-----
54849                                ; 27/04/2019 - Retro DOS v4.0
54850                                ;
54851                                ; retrodos4.s ; offset 0Ch in BIOS segment (0070h)
54852                                ;ALTAH     equ 0Ch
54853                                ;
54854                                ;This has been put in for WIN386 2.XX support. The format of the instance
54855                                ;table was different for this. Segments will be patched in at init time.
54856                                ;
54857                                ; 25/03/2024
54858                                ; PCDOS 7.1 IBMDOS.COM - DOSDATA:10FCh
54859                                ; (MSDOS 5.0-6.22 MSDOS.SYS - DOSDATA:11E7h)
54860                                ;
54861 OldInstanceJunk:
54862 000010FC 7000          dw     70h          ;segment of BIOS
54863 000010FE 0000          dw     0           ;indicate stacks in SYSINIT area
54864 00001100 0600          dw     6           ;6 instance items
54865                                ;
54866                                ;dw     0,offset dosdata:contpos, 2
54867                                ;dw     0,offset dosdata:bcon, 4
54868                                ;dw     0,offset dosdata:carpos,106h
54869                                ;dw     0,offset dosdata:charco, 1
54870                                ;dw     0,offset dosdata:exec_init_sp, 34          ;M032
54871                                ;dw     070h,offset BData:altah, 1          ; altah byte in bios
54872                                ;
54873 00001102 0000[2200]0200      dw     0,CONTPOS,2
54874 00001104 0000[3200]0400      dw     0,BCON,4
54875 00001106 0000[F901]0601      dw     0,CARPOS,106h
54876 00001108 0000[0003]0100      dw     0,CHARCO,1
54877 0000110A 0000[BF0E]2200      dw     0,exec_init_SP,34
54878 0000110C 70000C000100      dw     70h,ALTAH,1          ; altah byte in bios
54879                                ;
54880                                ;-----
54881                                ; 24/03/2024
54882                                ; 17/01/2024
54883                                %if 0
54884                                ;
54885                                ; M021-
54886                                ;
54887                                ; DosHashMA - This flag is set by seg_reinit when the DOS actually
54888                                ; takes control of the HMA. When running, this word is a reliable
54889                                ; indicator that the DOS is actually using HMA. You can't just use
54890                                ; CS, because ROMDOS uses HMA with CS < F000.
54891                                ;
54892 DosHashMA:
54893 db     0
54894 FixExePatch:
54895 dw     0          ; M012
54896                                ;
54897                                ; 21/03/2024 - Retro DOS v5.0
54898                                ; 28/12/2022 - Retro DOS v4.1
54899 RationalPatchPtr:
54900 dw     0          ; M012
54901                                ;
54902                                %endif
54903                                ; End M021
54904

```

```

54911 ;-----
54912 ; 21/03/2024 - Retro DOS v5.0
54913 %if 1
54914 ; 28/12/2022 - Retro DOS v4.1
54915 ;%if 0
54916
54917 ; M020 Begin
54918
54919 RatBugCode: ; proc far
54920     push    cx
54921     mov     cx,[10h]
54922 rbc_loop:
54923     ;loop    $
54924     ;loop    rbc_loop
54925     pop     cx
54926     retf
54927
54928 ; M020 End
54929
54930 %endif
54931 ;-----
54932
54933 UmbSave1:
54934     ;db      11 dup (?)      ; M023
54935     times   11 db 0
54936
54937 ;-----
54938 ; 16/01/2024 - RetroDOS v5.0
54939 ; PCDOS 7.1 IBMDOS.COM - DOSDATA: 113Ah
54940
54941 OLD_FIRSTCLUS_HW:
54942     dw      0
54943
54944 ; 17/01/2024
54945 %if 1
54946
54947 ; DOS 3.3 F.C. 6/12/86
54948 ; FASTOPEN communications area DOS 3.3 F.C. 5/29/86
54949
54950 FastTable: ; a better name
54951 FastOpenTable:
54952     dw      2 ; number of entries
54953     dw      FastRet ; pointer to ret instr.
54954     dw      0 ; and will be modified by
54955     dw      FastRet ; FASTxxx when loaded in
54956     dw      0
54957
54958 ; DOS 3.3 F.C. 6/12/86
54959
54960 FastFlg: ; flags
54961 FastOpenFlg:
54962     db      0 ; don't change the foll: order
54963
54964 %endif
54965
54966 ; 24/03/2024 - Retrto DOS v5.0
54967 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:1147h
54968 %if 1
54969
54970 FastOpen_Ext_Info:
54971     ;db      6 dup(0)
54972     times   6 db 0
54973
54974 UNKNOWN1:
54975     dw      0
54976     ;db      5 dup(0)
54977     times   5 db 0
54978
54979 PATHNAMELEN:
54980     dw      67
54981
54982     ;db      31 dup(0)
54983     times   31 db 0
54984
54985 %endif
54986
54987 ; 17/01/2024
54988 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:1175h
54989 CurSC_DRIVE:
54990     db      -1 ; 0FFh ; current SC drive
54991
54992 ;M044
54993 ; Second part of save area for saving last para of windows memory
54994
54995 ; 17/01/2024
54996 WinoldPatch2:
54997     ;db      8 dup (?) ; M044
54998     times   8 db 0
54999
55000 FIRST_BUFF_ADDR:
55001     dw      0
55002
55003 ;-----
55004 ; 17/01/2024 - RetroDOS v5.0
55005 ; PCDOS 7.1 IBMDOS.COM
55006 ; DOSDATA:1180h
55007
55008 %if 1
55009
55010 ;=====
55011 ; WPATCH.INC (MSDOS 6.0, 1991) ;;; windows 3.1 patches ;;;
55012 ;=====
55013 ; 27/04/2019 - Retro DOS 4.0
55014
55015 ; DOSDATA:12CFh (MSDOS 6.21, MSDOS.SYS)
55016
55017 ; Retro DOS v5.0
55018 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:1180h
55019
55020 ; 05/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
55021 ; DOSDATA:12CFh (MSDOS 5.0, MSDOS.SYS)
55022
55023 ; first and second DOS patches
55024 ; Non-console device read/write (system calls 3Fh and 40h)
55025 ;
55026 ; Code in disk.asm, 2 locations, one for read, one for write
55027 ;
55028 ; DVRDLP:
55029 ; DVWRTLTP:
55030
55031 ;
55032 ; 036h lds si,SS:[????] ; ThisSFT
55033 ; lds si,si+7 ; sf_devptr
55034 ; 0E8h call ???? <- "simulate" int28 event ; DSKSTATCHK
55035
55036 DOSP1_ID: db 036h,0C5h,036h

```

```

55035 00001183 3605C57407E8      DOSP1_THISSFT:  db      036h,005h,0C5h,074h,007h,0E8h
55036                                DOSP1_ID_LEN      equ      $-DOSP1_ID
55037
55038 00001189 9090                db      90h, 90h
55039
55040 0000118B 36C536              DOSP12_ID:  db      036h,0C5h,036h
55041 0000118E 3605C57407E8      DOSP12_THISSFT: db      036h,005h,0C5h,074h,007h,0E8h
55042                                DOSP12_ID_LEN      equ      $-DOSP1_ID
55043
55044                                ; DOSDATA:12E3h
55045
55046                                ; Third/Fourth DOS patch - System call 3Fh (Read) from console
55047                                ;
55048                                ; Code in disk.asm, 1 location
55049                                ; GETBUF:
55050                                ;
55051                                ; 051h      push    cx      <- begin special int28 mode
55052                                ;      push    es
55053                                ;      push    di
55054                                ;      mov     dx,???? ; offset dosgroup:CONBUF
55055                                ;      call    ???? ; $STD_CON_STRING_INPUT
55056                                ;      pop     di
55057                                ;      pop     es
55058                                ; 059h      pop     cx      <- end special int28 mode
55059
55060 00001194 510657BA      DOSP3_ID:  db      051h,006h,057h,0BAh
55061 00001198 2902E8      DOSP3_CONBUF: db      029h,002h,0E8h
55062                                DOSP3_ID_LEN      equ      $-DOSP3_ID
55063 0000119B 9AE35F07      DOSP3_ID_LEN      db      09Ah,0E3h,05Fh,007h ; ???? , pop di, pop es
55064 0000119F 59          DOSP4_ID:  db      059h ; pop cx
55065                                DOSP4_ID_OFF      equ      (DOSP4_ID - DOSP3_ID)
55066
55067                                ; DOSDATA:12EFh
55068
55069                                ; Fifth DOS patch - System call 40h (write) to console
55070                                ;
55071                                ; Code in disk.asm, 1 location
55072                                ;
55073                                ;      push    cx
55074                                ; WRCONLP: lodsb
55075                                ;      cmp     al,1Ah
55076                                ;      jz      ????
55077                                ;      call    ???? <- "simulate" int28 event
55078                                ;      loop    WRCONLP
55079                                ; CONEOF:  pop     ax
55080
55081 000011A0 51          DOSP5_ID:  db      051h ; push cx
55082 000011A1 AC3C1A7405      db      0ACh,03Ch,01Ah,074h,005h
55083 000011A6 E8          db      0E8h ; call
55084                                DOSP5_ID_LEN      equ      $-DOSP5_ID
55085
55086                                ; DOSDATA:12F6h
55087
55088                                ; Seventh DOS patch - System call entry, patch USER_ID with VMid for share
55089                                ;
55090                                ; Code in disp.asm, 1 location
55091                                ;
55092                                ;
55093                                ;      mov     [SavedS],ds
55094                                ;      mov     [SaveBX],bx
55095                                ;      mov     bx,cs
55096                                ;      mov     ds,bx
55097                                ;      inc     [indos]
55098                                ;      xor     ax,ax
55099                                ;      mov     [USER_ID],AX      <- Patch to set USER_ID to VMID
55100
55101 000011A7 2E8C1E      DOSP7_ID:  db      02Eh,08Ch,01Eh
55102 000011AA 7E05          DOSP7_SAVEDS: db      07Eh,05h ; mov [SavedS],ds
55103 000011AC 2E891E      db      02Eh,089h,01Eh
55104 000011AF 7C05          DOSP7_SAVEBX: db      07Ch,05h ; mov [SaveBX],bx
55105 000011B1 8CCB      db      08Ch,0CBh ; mov bx,cs
55106 000011B3 8EDB      db      08Eh,0DBh ; mov ds,bx
55107 000011B5 FE06      db      0FEh,006h
55108 000011B7 CF02      DOSP7_INDOS: db      0CFh,002h ; inc [indos]
55109 000011B9 33C0      db      033h,0C0h ; xor ax,ax
55110                                DOSP7_ID_LEN      equ      $-DOSP7_ID
55111
55112                                ; DOSDATA:130Ah
55113
55114                                ; Eighth DOS patch - OWNER check in handle calls. For share, need to NOP test
55115                                ;
55116                                ; Code in handle.asm, 1 location in routine CheckOwner
55117                                ;
55118                                ;
55119                                ;
55120                                ;      push    ax
55121                                ;      mov     ax,ss:[USER_ID] <- patch to XOR AX,AX to set zero
55122                                ;      cmp     ax,es:[di.sf_UID] <- NOP
55123                                ;      pop     ax
55124                                ;      jz      ????
55125
55126 000011BB 50          DOSP8_ID:  db      050h ; push ax
55127 000011BC 36A1      db      036h,0A1h
55128 000011BE EA02      DOSP8_USER_ID: db      0EAh,002h ; mov ax,ss:[USER_ID]
55129 000011C0 263B45      db      026h,03Bh,045h ; cmp ax,es:[di+2F]
55130                                DOSP8_ID_LEN      equ      $-DOSP8_ID
55131 000011C3 2F58      db      02Fh,058h ; pop ax
55132
55133                                ; DOSDATA:1314h
55134
55135                                ; 10th, 11th, 12th DOS patch - System call 3Fh (Read) in raw mode
55136                                ;
55137                                ; Take RAW read to STDIN SFT and turn it into a polling loop doing
55138                                ; a yeild when a character is not ready to be read.
55139                                ;
55140                                ; Code in disk.asm, 3 locations
55141                                ;
55142                                ; DVRDRAW:
55143                                ;      PUSH    ES
55144                                ;      POP     DS
55145                                ; ReadRawRetry: <- Patch 10
55146                                ;      MOV     BX,DI
55147                                ;      XOR     AX,AX <- Reenter #2
55148                                ;      MOV     DX,AX
55149                                ;      call    SETREAD
55150                                ;      PUSH    DS <- Reenter #1
55151                                ;      LDS     SI,[THISSFT]
55152                                ;      call    DEVIOCALL
55153                                ;      MOV     DX,DI
55154                                ;      MOV     AH,86h
55155                                ;      MOV     DI,[DEVCALL.REQSTAT]
55156                                ;      TEST    DI,STERR
55157                                ;      JZ      CRDROK
55158                                ;      call    CHARHARD

```

```

55159 ; MOV DI,DX
55160 ; OR AL,AL
55161 ; JZ CRDROK
55162 ; CMP AL,3
55163 ; JZ CRDFERR
55164 ; POP DS
55165 ; JMP ReadRawRetry
55166 ;
55167 ; CRDFERR:
55168 ; POP DI <- Patch 11
55169 ; DEVIOFERR:
55170 ; LES DI,[THISSFT]
55171 ; jmp SET_ACC_ERR_DS
55172 ;
55173 ; CRDROK:
55174 ; POP DI <- Patch 12
55175 ; MOV DI,DX
55176 ; ADD DI,[CALLSCNT]
55177 ; JMP SHORT ENDRDDEVJ3
55178 ;
55179 000011C5 061F DOSP10_ID: db 006H,01FH
55180 DOSP10_LOC_OFFSET equ $-DOSP10_ID
55181 000011C7 8BDF DOSP10_LOC: db 08BH,0DFH
55182 DOSP10_REENT2_OFFSET equ $-DOSP10_LOC
55183 000011C9 33C08BD0E8 033H,0C0H,08BH,0D0H,0E8H
55184 DOSP10_ID_LEN equ $-DOSP10_ID
55185 000011CE DF0E 0DFH,00EH
55186 DOSP10_REENT1_OFFSET equ $-DOSP10_LOC
55187 000011D0 1E36C5363605E8AF0E 01EH,036H,0C5H,036H,036H,005H,0E8H,0AFH,00EH
55188 000011D9 8BD7B486368B3E 08BH,0D7H,0B4H,086H,036H,08BH,03EH
55189 DOSP10_PACKVAL_OFFSET equ $-DOSP10_ID
55190 000011E0 0903 009H,003H
55191 000011E2 F7C700807419E84717 0F7H,0C7H,000H,080H,074H,019H,0E8H,047H,017H
55192 000011EB 8BFA0AC074103C0374- 08BH,0FAH,00AH,0C0H,074H,010H,03CH,003H,074H,003H
55192 000011F4 03
55193 000011F5 1FEBCF
55194 DOSP11_LOC_OFFSET db 01FH,0EBH,0CFH
55195 000011F8 5F equ $-DOSP10_ID
55196 DOSP11_REENT_OFFSET db 05FH
55197 000011F9 36C43E3605E9A104 equ $-DOSP10_LOC
55198 DOSP12_LOC_OFFSET db 036H,0C4H,03EH,036H,005H,0E9H,0A1H,004H
55199
55200 00001201 5F8BFA DOSP12_LOC_OFFSET equ $-DOSP10_ID
55201 ; DOSDATA:1353h db 05FH,08BH,0FAH
55202 ;
55203 ; 13th DOS patch - Actually a SYSINIT patch. Patches the stack fault code
55204 ; which prints the fatal stack fault error on DOS >= 3.20.
55205 ;
55206 ; Sets focus to current VM so user can see fatal message.
55207 ;
55208 ;
55209 ; 10: lodsb <- Setfocus here
55210 ; cmp al, '$'
55211 ; je 11
55212 ; mov bl, 7
55213 ; mov ah, 0Eh
55214 ; int 10h
55215 ; jmp 10
55216 ; 11: jmp $
55217
55218 00001204 AC DOSP13_ID: db 0ACh
55219 00001205 3C24 db 03Ch,024h
55220 00001207 7408 db 074h,008h
55221 00001209 B307 db 0B3h,007h
55222 0000120B B40E db 0B4h,00Eh
55223 0000120D CD10 db 0CDh,010h
55224 0000120F EBF3 db 0EBh,0F3h
55225 00001211 EBFE db 0EBh,0FEh
55226 DOSP13_ID_LEN equ $-DOSP13_ID
55227
55228 ; 05/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
55229 ; DOSDATA:1362h (MSDOS 5.0 MSDOS.SYS)
55230
55231 ; 05/01/2024
55232 %endif ; 05/11/2022
55233
55234 ;-----
55235 ;
55236 ; DOSDATA:122Ah
55237
55238 Mark3: ; label byte
55239
55240 ;IF2
55241 ; IF ((OFFSET MARK3) GT (OFFSET COUNTRY_CDPG) )
55242 ; %OUT !DATA CORRUPTION!MARK3 OFFSET TOO BIG. RE-ORGANIZE DATA.
55243 ; ENDIF
55244 ;ENDIF
55245
55246 ;-----
55247
55248 ; 27/04/2019 - Retro DOS v4.0
55249 ; 05/01/2024 - Retro DOS v5.0
55250
55251 ;include msdos.c12 ; XMMERRMSG
55252
55253 ; DOSDATA:12B8h (MSDOS 6.22, MSDOS.SYS) ; 17/01/2024
55254 ; DOSDATA:1213h (PCDOS 7.1, IBMDOS.COM) ; 05/01/2024
55255
55256 XMMERRMSG:
55257 00001213 0D0A db 0Dh,0Ah
55258 00001215 413230204861726477- db 'A20 Hardware Error',0Dh,0Ah,'$'
55258 0000121E 617265204572726F72-
55258 00001227 0D0A24
55259
55260 ;-----
55261 ; 17/01/2024 - Note: COUNTRY_CDPG must be at DOSDATA:122Ah (to 12B8h) addr
55262 ; It is fixed at 122Ah in PCDOS 7.1 IBMDOS.COM and MSDOS 5.0-6.22 MSDOS.SYS
55263 ;-----
55264
55265 ;#####
55266 ;
55267 ; ** HACK FOR DOS 4.0 REDIR **
55268 ;
55269 ; The dos 4.X redir requires that country_cdpG is at offset 0122ah. Any new
55270 ; data variable that is to be added to DOSDATA must go in between Mark3
55271 ; COUNTRY_CDPG if it can.
55272 ;
55273 ; MARK3 SHOULD NOT BE > 122AH
55274 ;
55275 ; As of 9/6/90, this area is FULL!
55276 ;
55277 ;#####
55278
55279 ;ORG 0122Ah

```



```

55280
55281 ; DOSDATA:122Ah (MSDOS 6.21, MSDOS.SYS)
55282
55283 ; 09/01/2024 - Retro DOS v5.0
55284 ; PC DOS 7.1 IBMDOS.COM - DOSDATA:122Ah
55285
55286 ; 17/01/2024
55287 ; MSDOS 5.0 MSDOS.SYS - DOSDATA:122Ah
55288 ; MSDOS 6.22 MSDOS.SYS - DOSDATA:122Ah
55289 ; 09/03/2024
55290 ; windows ME IO.SYS - DOSDATA:122Ah
55291
55292 ; The following table is used for DOS 3.3
55293 ; DOS country and code page information is defined here for DOS 3.3.
55294 ; The initial value for ccdosCountry is 1 (USA).
55295 ; The initial value for ccdosCodepage is 850.
55296
55297 ; country and code page information
55298 ; -----
55299 COUNTRY_CDPG: ; label byte
55300 0000122A 0000000000000000 db 0,0,0,0,0,0,0,0 ; reserved words
55301 00001232 5C434F554E5452592E- db '\COUNTRY.SYS',0 ; path name of country.sys
55301 0000123B 53595300
55302 ; db 51 dup (?)
55303 0000123F 00<rep 33h> times 51 db 0
55304 ; -----<MSKK01>-----
55305 ; ifdef DBCS
55306 ; ifdef JAPAN
55307 ; dw 932 ; system code page id (JAPAN)
55308 ; endif
55309 ; ifdef TAIWAN
55310 ; dw 938 ; system code page id (TAIWAN)
55311 ; endif
55312 ; ifdef KOREA
55313 ; dw 934 ; system code page id (KOREA IBM)
55314 ; endif
55315 ; else
55316 00001272 B501 dw 437 ; system code page id
55317 ; endif
55318 ; -----<MSKK01>-----
55319 00001274 0600 dw 6 ; number of entries
55320 COUNTRY_CDPG_76: ; COUNTRY_CDPG + 76
55321 00001276 02 db SetUcase ; 2 ; Ucase type
55322 00001277 [FA0A] dw UCASE_TAB ; pointer to upper case table
55323 00001279 0000 dw 0 ; segment of poiter
55324 0000127B 04 db SetUcaseFile ; 4 ; Ucase file char type
55325 0000127C [7C0B] dw FILE_UCASE_TAB ; pointer to file upper case table
55326 0000127E 0000 dw 0 ; segment of poiter
55327 00001280 05 db SetFileList ; 5 ; valid file chars type
55328 00001281 [280D] dw FILE_CHAR_TAB ; pointer to valid file char tab
55329 00001283 0000 dw 0 ; segment of poiter
55330 00001285 06 db SetCollate ; 6 ; collate type
55331 00001286 [FE0B] dw COLLATE_TAB ; pointer to collate table
55332 00001288 0000 dw 0 ; segment of poiter
55333 0000128A 07 db SetDBCS ; 7 ; AN000; DBCS Ev 2/12/KK
55334 0000128B [000D] dw DBCS_TAB ; AN000; pointer to DBCS Ev table 2/12/KK
55335 0000128D 0000 dw 0 ; AN000; segment of pointer 2/12/KK
55336 0000128F 01 db SetCountryInfo ; 1 ; country info type
55337 00001290 2600 dw NEW_COUNTRY_SIZE ; extended country info size
55338 ; -----<MSKK01>-----
55339 ; ifdef DBCS
55340 ; .....
55341 ; else
55342 ; 09/01/2024 - Retro DOS v5.0
55343 ; PC DOS 7.1 IBMDOS.COM - DOSDATA:1292h
55344 _COUNTRY_ID:
55345 00001292 0100 dw 1 ; USA country id
55346 00001294 B501 dw 437 ; USA system code page id
55347 COUNTRY_CDPG_108: ; COUNTRY_CDPG + 108
55348 00001296 0000 dw 0 ; date format
55349 00001298 2400000000 db '$',0,0,0,0,0 ; currency symbol
55350 0000129D 2C00 db ',','0 ; thousand separator
55351 0000129F 2E00 db ',','0 ; decimal separator
55352 000012A1 2D00 db '-',0 ; date separator
55353 000012A3 3A00 db ':','0 ; time separator
55354 000012A5 00 db 0 ; currency format flag
55355 000012A6 02 db 2 ; # of digits in currency
55356 000012A7 00 db 0 ; time format
55357 000012A8 [6B0D] dw MAP_CASE ; mono case routine entry point
55358 000012AA 0000 dw 0 ; segment of entry point
55359 000012AC 2C00 db ',','0 ; data list separator
55360 000012AE 00000000000000000000- dw 0,0,0,0,0 ; reserved
55360 000012B7 00
55361 ; endif
55362 ; -----<MSKK01>-----
55363 ; -----
55364 ; 01/01/2024 - Retro DOS v5.0 (Modified PC DOS 7.1 IBMDOS.COM)
55365
55366 ; PC DOS 7.1 IBMDOS.COM - DOSDATA:12B8h
55367
55368 INDOS_FLAG:
55369 000012B8 00 db 0 ; duplicated INDOS flag, what for ?
55370 ; (PC DOS 7.1 kernel CODE always updates it together
55371 ; with 'INDOS' flag !?)
55372
55373 ; 09/01/2024
55374 DEVIO_IN_PROGRESS:
55375 000012B9 00 db 0
55376
55377 ; 09/01/2024
55378 ; PC DOS 7.1 IBMDOS.COM - DOSDATA:12BAh
55379 ; -----
55380 ; INTERNATIONALIZATION INFORMATION
55381 ; for Get/Set Extended Country Info functions
55382
55383 000012BA 454E5500 _ENU: db 'ENU',0
55384 000012BE 55534100 _USA: db 'USA',0
55385 000012C2 5553 _US: db 'US'
55386 000012C4 0100 dw 1
55387 000012C6 0200 dw 2
55388 000012C8 0000 dw 0
55389 000012CA 414D00 _AM: db 'AM',0
55390 000012CD 504D00 _PM: db 'PM',0
55391 _MMDDYY:
55392 000012D0 4D2F642F7979202020- db 'M/d/yy dddd,MMMdd,yyyy '
55392 000012D9 2020646464642C4D4D-
55392 000012E2 4D4D64642C79797979-
55392 000012EB 202020202020202020
55393 000012F4 00 db 0
55394 000012F5 00 db 0
55395 000012F6 0000 dw 0
55396
55397 ; 09/01/2024
55398 ; PC DOS 7.1 IBMDOS.COM - DOSDATA:12F8h

```

```

55399
55400
55401 000012F8 633A5C77696E613230-
55401 00001301 2E33383600
55402 00001306 0000
55403
55404
55405
55406
55407
55408 00001308 00<rep 1Ah>
55409
55410
55411 00001322 01
55412
55413 00001323 0000<rep 2h>
55414 00001327 00<rep 5h>
55415 0000132C 0400
55416 0000132E [B812]
55417 00001330 [0813]
55418 00001332 [3613]
55419 00001334 [3A13]
55420
55421 00001336 59
55422 00001337 4E
55423
55424 00001338 79
55425
55426 00001339 6E
55427
55428
55429 0000133A 00000000
55430
55431
55432
55433
55434
55435
55436
55437
55438
55439
55440
55441
55442
55443
55444
55445
55446
55447
55448
55449
55450
55451
55452
55453
55454
55455
55456
55457
55458
55459
55460
55461
55462
55463
55464
55465
55466
55467
55468
55469
55470
55471
55472
55473
55474
55475
55476
55477
55478
55479
55480
55481
55482
55483
55484
55485
55486
55487
55488
55489
55490
55491
55492
55493
55494
55495
55496
55497
55498
55499
55500
55501
55502
55503
55504
55505
55506
55507
55508
55509
55510
55511
55512
55513
55514
55515
55516
55517
55518
55519
55520
55521

vxDpath:
    db 'c:\wina20.386',0
    dw 0
    ; 09/01/2024
    ; PCDS 7.1 IBMDOS.COM - DOSDATA:1308h

drive_flags:
    times 26 db 0

DriverLoad:
    db 1
BiosDataPtr:
    ; 08/03/2024
    times 2 dw 0
    times 5 db 0
    dw 4
    dw INDOS_FLAG
    dw drive_flags
    dw NLS_YES ; "YN"
    dw unknown_zero_dd
NLS_YES:
    db 'Y'
NLS_NO:
    db 'N'
NLS_yes2:
    db 'y'
NLS_no2:
    db 'n'

unknown_zero_dd:
    dd 0

; -----
; 27/04/2019 - Retro DOS v4.0
;include msdos.c12 ; XMMERRMSG
; DOSDATA:12B8h (MSDOS 6.21, MSDOS.SYS)
;XMMERRMSG:
;    db 0dh,0Ah
;    db 'A20 Hardware Error',0dh,0Ah,'$'
; DOSDATA ends
; 05/11/2022
; -----
; End of MSDOS 5.0 MSDOS.SYS /// Retro DOS v4.0 (2022) - 05/11/2022
; -----
; 28/12/2022 - Retro DOS v4.1
; (windows 3.1 and Rational Extender patches are removed/disabled)
; (windows 3.1 does not use the patches below if DOS version is MSDOS 5.0)
; -----
%if 0
; -----
; 05/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
; =====
; WPATCH.INC (MSDOS 6.0, 1991) ;;; windows 3.1 patches ;;;
; =====
; 27/04/2019 - Retro DOS 4.0

;DOSDATA Segment
; DOSDATA:12CFh (MSDOS 6.21, MSDOS.SYS)
; Retro DOS v5.0
; PCDS 7.1 IBMDOS.COM - DOSDATA:1180h
; 05/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
; DOSDATA:12CFh (MSDOS 5.0, MSDOS.SYS)
; first and second DOS patches
; Non-console device read/write (system calls 3Fh and 40h)
;
; Code in disk.asm, 2 locations, one for read, one for write
; DVRDLP:
; DVWRTLP:
;
;
;
; 036h lds si,SS:[????] ; ThisSFT
; lds si,si+7 ; sf_devptr
; 0E8h call ???? <- "simulate" int28 event ; DSKSTATCHK

DOSP1_ID: db 036h,0C5h,036h
DOSP1_THISSFT: db 036h,005h,0C5h,074h,007h,0E8h
DOSP1_ID_LEN equ $-DOSP1_ID
db 90h, 90h

DOSP12_ID: db 036h,0C5h,036h
DOSP12_THISSFT: db 036h,005h,0C5h,074h,007h,0E8h
DOSP12_ID_LEN equ $-DOSP1_ID

; DOSDATA:12E3h
; Third/Fourth DOS patch - System call 3Fh (Read) from console
;
; Code in disk.asm, 1 location
; GETBUF:
;
; 051h push cx <- begin special int28 mode
; push es
; push di
; mov dx,???? ; offset dosgroup:CONBUF
; call ???? ; $STD_CON_STRING_INPUT
; pop di
; pop es
; 059h pop cx <- end special int28 mode

DOSP3_ID: db 051h,006h,057h,0BAh
DOSP3_CONBUF: db 029h,002h,0E8h
DOSP3_ID_LEN equ $-DOSP3_ID
db 09Ah,0E3h,05Fh,007h ; ???? , pop di, pop es
DOSP4_ID: db 059h ; pop cx
DOSP4_ID_OFF equ (DOSP4_ID - DOSP3_ID)

; DOSDATA:12EFh
; Fifth DOS patch - System call 40h (write) to console

```



```

55522 ;
55523 ; Code in disk.asm, 1 location
55524 ;
55525 ;     push    cx
55526 ;     WRCONLP: lodsb
55527 ;     cmp     al,1Ah
55528 ;     jz      ????
55529 ;     call    ???? <- "simulate" int28 event
55530 ;     loop    WRCONLP
55531 ;     CONEOF: pop     ax
55532 ;
55533 DOSP5_ID:  db      051h          ; push cx
55534           db      0ACh,03Ch,01Ah,074h,005h
55535           db      0E8h          ; call
55536 DOSP5_ID_LEN equ     $-DOSP5_ID
55537 ;
55538 ; DOSDATA:12F6h
55539 ;
55540 ; Seventh DOS patch - System call entry, patch USER_ID with VMid for share
55541 ;
55542 ; Code in disp.asm, 1 location
55543 ;
55544 ;
55545 ;     mov     [SaveDS],ds
55546 ;     mov     [SaveBX],bx
55547 ;     mov     bx,cs
55548 ;     mov     ds,bx
55549 ;     inc     [indos]
55550 ;     xor     ax,ax
55551 ;     mov     [USER_ID],AX      <- Patch to set USER_ID to VMID
55552 ;
55553 DOSP7_ID:  db      02Eh,08Ch,01Eh
55554 DOSP7_SAVEDS: db      07Eh,05h          ; mov [SaveDS],ds
55555           db      02Eh,089h,01Eh
55556 DOSP7_SAVEBX: db      07Ch,05h          ; mov [SaveBX],bx
55557           db      08Ch,0CBh          ; mov bx,cs
55558           db      08Eh,0DBh          ; mov ds,bx
55559           db      0FEh,006h
55560 DOSP7_INDOS: db      0CFh,002h          ; inc [indos]
55561           db      033h,0C0h          ; xor ax,ax
55562 DOSP7_ID_LEN equ     $-DOSP7_ID
55563 ;
55564 ; DOSDATA:130Ah
55565 ;
55566 ; Eighth DOS patch - OWNER check in handle calls. For share, need to NOP test
55567 ;
55568 ; Code in handle.asm, 1 location in routine CheckOwner
55569 ;
55570 ;
55571 ;
55572 ;     push    ax
55573 ;     mov     ax,ss:[USER_ID]      <- patch to XOR AX,AX to set zero
55574 ;     cmp     ax,es:[di.sf_UID]    <- NOP
55575 ;     pop     ax
55576 ;     jz      ????
55577 ;
55578 DOSP8_ID:  db      050h          ; push ax
55579           db      036h,0A1h
55580 DOSP8_USER_ID: db      0EAh,002h          ; mov ax,ss:[USER_ID]
55581           db      026h,03Bh,045h          ; cmp ax,es:[di+2F]
55582 DOSP8_ID_LEN equ     $-DOSP8_ID
55583           db      02Fh,058h          ; pop ax
55584 ;
55585 ; DOSDATA:1314h
55586 ;
55587 ; 10th, 11th, 12th DOS patch - System call 3Fh (Read) in raw mode
55588 ;
55589 ;     Take RAW read to STDIN SFT and turn it into a polling loop doing
55590 ;     a yeild when a character is not ready to be read.
55591 ;
55592 ; Code in disk.asm, 3 locations
55593 ;
55594 ;     DVRDRAW:
55595 ;     PUSH     ES
55596 ;     POP      DS
55597 ;     ReadRawRetry:      <- Patch 10
55598 ;     MOV      BX,DI
55599 ;     XOR      AX,AX      <- Reenter #2
55600 ;     MOV      DX,AX
55601 ;     call     SETREAD
55602 ;     PUSH     DS      <- Reenter #1
55603 ;     LDS      SI,[THISSFT]
55604 ;     call     DEVIOCALL
55605 ;     MOV      DX,DI
55606 ;     MOV      AH,86h
55607 ;     MOV      DI,[DEVCALL.REQSTAT]
55608 ;     TEST     DI,STERR
55609 ;     JZ       CRDROK
55610 ;     call     CHARHARD
55611 ;     MOV      DI,DX
55612 ;     OR       AL,AL
55613 ;     JZ       CRDROK
55614 ;     CMP      AL,3
55615 ;     JZ       CRDFERR
55616 ;     POP      DS
55617 ;     JMP      ReadRawRetry
55618 ;
55619 ;     CRDFERR:
55620 ;     POP      DI      <- Patch 11
55621 ;     DEVIOFERR:
55622 ;     LES      DI,[THISSFT]
55623 ;     jmp      SET_ACC_ERR_DS
55624 ;
55625 ;     CRDROK:
55626 ;     POP      DI      <- Patch 12
55627 ;     MOV      DI,DX
55628 ;     ADD      DI,[CALLSCNT]
55629 ;     JMP      SHORT ENDRDDEVJ3
55630 ;
55631 DOSP10_ID:  db      006h,01Fh
55632 DOSP10_LOC_OFFSET equ     $-DOSP10_ID
55633 DOSP10_LOC:  db      08Bh,0DFh
55634 DOSP10_REENT2_OFFSET equ     $-DOSP10_LOC
55635           db      033h,0C0h,08Bh,0D0h,0E8h
55636 DOSP10_ID_LEN equ     $-DOSP10_ID
55637           db      0DFh,00Eh
55638 DOSP10_REENT1_OFFSET equ     $-DOSP10_LOC
55639           db      01Eh,036h,0C5h,036h,036h,005h,0E8h,0AFh,00Eh
55640           db      08Bh,0D7h,0B4h,086h,036h,08Bh,03Eh
55641 DOSP10_PACKVAL_OFFSET equ     $-DOSP10_ID
55642           db      009h,003h
55643           db      0F7h,0C7h,000h,080h,074h,019h,0E8h,047h,017h
55644           db      08Bh,0FAh,00Ah,0C0h,074h,010h,03Ch,003h,074h,003h
55645           db      01Fh,0EBh,0CFh

```

```

55646 DOSP11_LOC_OFFSET equ    $-DOSP10_ID
55647 db    05FH
55648 DOSP11_REENT_OFFSET equ    $-DOSP10_LOC
55649 db    036H,0C4H,03EH,036H,005H,0E9H,0A1H,004H
55650
55651 DOSP12_LOC_OFFSET equ    $-DOSP10_ID
55652 db    05FH,08BH,0FAH
55653 ; DOSDATA:1353h
55654
55655 ; 13th DOS patch - Actually a SYSINIT patch. Patches the stack fault code
55656 ; which prints the fatal stack fault error on DOS >= 3.20.
55657 ;
55658 ; Sets focus to current VM so user can see fatal message.
55659 ;
55660 ;
55661 ; 10: lodsb          <- Setfocus here
55662 ; cmp al, '$'
55663 ; je l1
55664 ; mov bl, 7
55665 ; mov ah, 0Eh
55666 ; int 10h
55667 ; jmp l0
55668 ; l1: jmp $
55669
55670 DOSP13_ID: db    0ACh          ; 10: lodsb
55671 db    03Ch,024h          ; cmp al, '$'
55672 db    074h,008h          ; je l1
55673 db    0B3h,007h          ; mov bl, 7
55674 db    0B4h,00Eh          ; mov ah, 0Eh
55675 db    0CDh,010h          ; int 10h
55676 db    0EBh,0F3h          ; jmp l0
55677 db    0EBh,0FEh          ; l1: jmp $
55678 DOSP13_ID_LEN equ    $-DOSP13_ID
55679
55680 ; 05/11/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
55681 ; DOSDATA:1362h (MSDOS 5.0 MSDOS.SYS)
55682
55683 ; 06/12/2022
55684 ;DOSDATASIZE equ    $ - DOSDATASTART ; 4962 bytes (1362h)
55685
55686 ; DOSDATA ends
55687
55688 ; 05/01/2024
55689 %endif ; 05/11/2022
55690
55691 ;=====
55692 ; MPATCH.ASM (MSDOS 6.0, 1993)
55693 ;=====
55694 ; 27/04/2019 - Retro DOS 4.0
55695
55696 ;mpatch.asm -- holds data patch location for callouts
55697 ; -- allocate cluster in rom.asm
55698 ;
55699 ; This area is pointed to by OffsetMagicPatch[609h] in fixed DOS data.
55700 ; Currently, this location is used only by magicdrv.sys's patch to
55701 ; cluster allocation, however it can be expanded to be used by other
55702 ; patches. This is important since we have an easy-access pointer to
55703 ; this location in OffsetMagicPatch. Magicdrv.sys is guaranteed to
55704 ; only patch out a far call/retf, so any space after that could be
55705 ; used as a patch by using OffsetMagicPatch+6. See rom.asm on how
55706 ; to call out here.
55707 ;
55708 ; Currently, we allocate only the minimum space required for the 6
55709 ; byte magicdrv patch, so if you change the dos data, you may want
55710 ; to reserve space here if your new data will be position dependent
55711 ; and would prohibit growing of this table.
55712 ;
55713 ;history - created 8-7-92 by scottq
55714 ; - added Rational386PatchPtr 2-1-93 by jimmat
55715 ;
55716 ;Exported Functions
55717 ;=====
55718 ;MagicPatch - callout patched by magidrv.sys for cluster allocations
55719
55720 ; DosData Segment
55721
55722 ; DOSDATA:1362h (MSDOS 6.21, MSDOS.SYS)
55723
55724 ; -----
55725
55726 ; Rational386PatchPtr points to either a RET instruction (80286 or less) or
55727 ; a routine to fix buggy versions of the Rational DOS Extender (80386 or
55728 ; greater). Added to this file because it needed to be somewhere and is
55729 ; 'patch' related.
55730
55731 Rational386PatchPtr:
55732 0000133E 0000 dw    0 ; points to patch routine or RET instr.
55733 ; -----
55734
55735 ; 25/03/2024
55736 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:1340h
55737 ; (Windows ME IO.SYS - DOSDATA:133Eh)
55738
55739 MagicPatch:
55740 ;MagicPatch proc far
55741 00001340 CB retf ;default is to just return to allocate
55742 00001341 90 nop ;however, this code will be patched
55743 00001342 90 nop ;by magicdrv.sys to
55744 00001343 90 nop ; call far ?:?
55745 00001344 90 nop ; retf or perhaps just jmp far
55746 00001345 90 nop ;retf/nop take one byte, so we need six instructions
55747 ;for 6 byte patch
55748 ;MagicPatch endp
55749
55750 ; -----
55751
55752 ;DosData Ends
55753
55754 ; MSDOS 5.0-6.22 MSDOS.SYS - DOSDATA:136Ah
55755 ;
55756 ; 25/03/2024 - Retro DOS v5.0
55757 ; PCDOS 7.1 IBMDOS.COM - DOSDATA:1346h
55758
55759 ;; windows ME IO.SYS
55760 ;; db 12 dup(0)
55761
55762 ; (Windows ME IO.SYS - DOSDATA:1344h)
55763
55764 ;-----
55765
55766 ;DOSDATALAST SEGMENT
55767
55768 ; 29/04/2019 - Retro DOS v4.0
55769

```

```

55770 ;-----
55771 ; 25/05/2019 - Retro DOS v4.0 Modification (paragraph alignment)
55772
55773 ;db 0,1,12,64,19,0 ; ! Magic numbers !
55774
55775 ;align 16
55776
55777 ; !!! DOSDATA:1370h ; Retro DOS v4.0 only!
55778
55779 ;-----
55780
55781 ; 05/01/2024
55782 ;%endif ; 05/11/2022
55783
55784 ; 05/12/2022
55785 ;MSDAT001E: ; label byte
55786
55787 ; 22/03/2024 - Retro DOS v5.0 (Modified PCDOS 7.1 IBMDOS.COM)
55788 ; 05/12/2022 - Retro DOS v4.0 (Modified MSDOS 5.0 MSDOS.SYS)
55789 DOSDATAEND equ $
55790 DOSDATASIZE equ DOSDATAEND - DOSDATASTART ; = 4962 for MSDOS 5.0 MSDOS.SYS
55791 ; = 4970 for MSDOS 6.22 MSDOS.SYS
55792 MSDAT001E equ DOSDATAEND - DOSDATASTART ; = 4934 for PCDOS 7.1 IBMDOS.COM
55793 ; (= 4944 for Windows ME IO.SYS)
55794
55795 ;DOSDATALAST ENDS
55796
55797 ; Retro DOS v4.0 by Erdogan Tan (Redevelopment of MSDOS 5.0 KERNEL via NASM)
55798 ; DECEMBER 2022, ISTANBUL - TURKIYE.
55799 ;=====
55800 ; END
55801 ;=====
55802 ; Retro DOS v4.0 by Erdogan Tan (Redevelopment of MSDOS 6.21 KERNEL via NASM)
55803 ;=====
55804 ; MAY 2019, ISTANBUL - TURKIYE.
55805 ;=====
55806 ; 25/03/2024 - RETRO DOS v5.0 KERNEL code is READY! (Start: 01/01/2024)

```